



Teradata Factory Xpress

Course # 56749

Version 16.20.4



Student Manual

Teradata Global Learning

Notes

teradata.

Course Overview

teradata.

Teradata Factory Xpress

Teradata Concepts
Physical Design and Implementation

Teradata Proprietary and Confidential

Notes

Table of Contents

Course Materials	0-6
Options for Displaying PDF Files.....	0-8
Example of Left Page – Right Page Display.....	0-10
View and search a PDF	0-10
PDF Comment and Markup Tools	0-12
Example of Highlighter and Sticky Note Tools	0-14
Example of Add Text Tool	0-16
Open a PDF File at Last Page Viewed.....	0-18
Course Description.....	0-20
Who Should Attend.....	0-22
Prerequisites	0-22
Class Format	0-24
Classroom Rules	0-24
Outline of the Class.....	0-26
Teradata Certification Tests (14.0)	0-28
Teradata Certification Tests	0-30

Course Materials

The Teradata Factory Xpress course materials that are provided on a USB flash drive are listed on the facing page. These materials are provided at the beginning of the class.

The Teradata Factory Xpress Student Manual and the Lab Workbook have been created as PDF files which can be viewed using Adobe® Reader®.

These PDF files were created using Adobe Acrobat® and commenting has been enabled for both files. This allows you to use Adobe® Reader® Comment and Markup tools to place your own notes and comments within the files.

Teradata Factory Xpress course materials include:

- Paper copy of TF Xpress Lab Workbook
- Electronic copy (PDF files) of Student Manual and Lab Workbook

Contents of the flash drive include:

- Teradata Factory Xpress Class Files
 - Class Files (these PDF files allow use of Comment and Markup tools)
 - TF Xpress v1620.4 Lab Workbook.pdf
 - TF Xpress v1620.4 Student Manual.pdf
 - Miscellaneous Documents and Software
 - Java JRE for Windows and JDK for MAC OS
 - Teradata Tools and Utilities (TTU)
 - TTU Packages
 - TeradataToolsAndUtilities_windows_16.xxxx.zip
 - DatabaseManagement_windows_16.xxxx.zip
 - Teradata Studio for Windows
 - Teradata Studio for MAC OS
 - TD Express Lab Setup (numbered in order of installation)

Options for Displaying PDF Files

Adobe® Reader® is a tool that you can use to open and view the Teradata Factory Xpress PDF course files. You can also use the Adobe Reader to make comments or notes and save your PDF file.

Since the Teradata Factory Xpress course materials have been created in a book format (left page - right page), you may want to set options in Adobe Reader to view the materials in a book format.

- **The left page contains additional information about the right or slide page.**
- **The right page is copy of the PPT slide that is used during the presentation.**

To view the Teradata Factory Xpress Student Manual in a book format using Adobe Reader, use the View Menu > Page Display and set the following options.

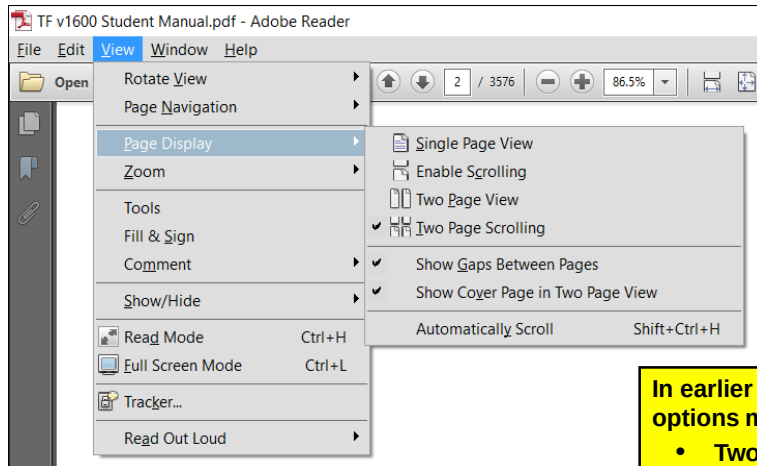
- Two-Up Continuous
- Show Gaps Between Pages (normally checked or set by default)
- Show Cover Page During Two-up

Options for Displaying PDF Files

The TF Xpress course materials are created in a left page – right page format.

- Left page – contains additional information about the slide page
- Right page – copy of the PPT slide that is used during the presentation

To display PDF files in a book type (left page – right page) format, Adobe Reader options need to be set.



In earlier versions of Adobe Reader, the options may be named:

- Two-Up Continuous
- Show Gaps Between Pages
- Show Cover Page During Two-Up

Example of Left Page – Right Page Display

The facing page illustrates an example of displaying the Teradata Factory Xpress Student Manual in a left page – right page format.

View and search a PDF

In the Adobe Reader toolbar, use the Zoom tools and the Magnification menu to enlarge or reduce the page. Use the options on the View menu to change the page display. There are various options in the Tools menu to provide you with more ways to adjust the page for better viewing (Tools > Select & Zoom).

After setting the Page Display options, an example of the left page – right page format is displayed below.

TF v1600 Student Manual.pdf - Adobe Reader

File Edit View Window Help

Open 96 / 3576 59%

Teradata and MPP Systems

Teradata is the software that makes a MPP system appear to be a single system to users and administrators.

What is a Node?

- The "compute or processing node" is the basic building block of the hardware for a Teradata system. The processing node contains the primary processor logic (CPUs), memory, and I/O functionality. Physically, a node is a computer chassis that has its own baseboard with CPUs, memory, I/O cards, power supplies, fans, etc. and has internal disks for its operating system (e.g., Linux) and Teradata software. The physical nodes are independent of each other, but are interconnected with Teradata software and the BYNET.
- Teradata is the database application that executes on one or more processing nodes and makes a multi-node system appear as a single database system to users. Each node is effectively an SMP (Symmetrical Multi-Processing) node. Depending on the configuration, a varying number of AMPs and/or PEs can execute on a node.
- A single node system is often referred to as an SMP system.
- A multi-node system (collection of SMP nodes) is referred to as a MPP system.

The BYNET (Banyan Network) is the software and hardware interconnect that provides high performance networking capabilities to Teradata MPP (Massively Parallel Processing) systems.

Using communication switching techniques, the BYNET allows for point-to-point, multi-cast, and broadcast communications among the nodes, thus supporting a monumental increase in throughput in very large databases. This technology allows Teradata users to grow massively parallel databases without fear of a communications bottleneck for any database operations.

Teradata and MPP Systems

Teradata is the software that makes a MPP system appear to be a single system to users and administrators.

The major components of the Teradata Database are implemented as virtual processors (vprocs).

- Parsing Engine (PE)
- Access Module Processor (AMP)

The Communication Layer or Message Passing Layer (MPL) consists of PDE and BYNET SW/HW and connects multiple nodes together.

This PDF file has been created to allow the use of comment and markup tools.

PDF Comment and Markup Tools

The Teradata Factory Xpress course materials have "commenting" enabled. Therefore, you can make comments in these files using the commenting and markup tools. Of the many commenting and markup tools that are available, you may find it easier to use the following tools (highlighted on the facing page).

- Add Sticky Note
- Highlight Text Tool
- Add Text

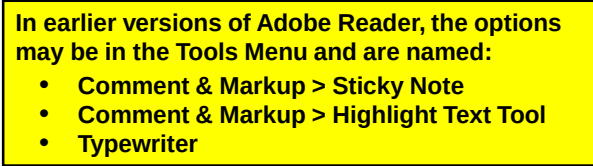
Comments can include both notes and drawings (if you have the time during class). You can enter a text message using the Sticky Note tool. You can use a drawing tool to add a line, circle, or other shape and then type a note in the associated pop-up note.

You can enable the Comment & Markup Toolbar or you can simply select the tools using the pull-down menus

After you add a note or comment, it stays selected until you click elsewhere on the page. A selected comment is highlighted by a blue halo to help you find the markup on the page.

- **Sticky Note**
- **Highlight Text Tool**
- **Add Text Comment**

Highlight Text Tool



Example of Highlighter and Sticky Note Tools

The facing page illustrates an example of using the Highlighter and Sticky Note tools.

Select a commenting or markup tool.

Choose Tools > Comment & Markup > *Highlighter or Sticky Note (or another tool)*

Note: After you make an initial comment, the tool changes back to the Select tool so that you can move, resize, or edit your comment. (The Pencil, Highlight Text, and Line tools stay selected.)

To keep a commenting tool selected so you can add multiple comments without reselecting the tool, do the following:

- Select the tool you want to use (but don't use it yet).
- Choose View > Toolbars > Properties Bar.
- Select Keep Tool Selected.

You can change the font of a text in a sticky note. Open the sticky note, choose View > Toolbars > Properties Bar, select the text in a note, and then change the font size in the Properties Bar.

Example of Highlighter and Sticky Note Tools

The left page illustrates the Highlighter tool and the right page illustrates the Sticky Note tool.

The screenshot shows the Adobe Reader interface with the document 'TF v1600 Student Manual.pdf'. The left page, titled 'Teradata and MPP Systems', contains text about Teradata's architecture. A yellow highlighter tool has been used to highlight the following text:

The "compute or processing node" is the basic building block of the hardware for a Teradata system. The processing node contains the primary processor logic (CPU), memory, and I/O functionality. Physically, a node is a computer chassis that has its own broadcast with CPU, memory, I/O cards, power supplies, fans, etc. and has internal disks for its operating system (e.g., Linux) and Teradata software. The physical nodes are independent of each other, but are interconnected with Teradata software and the BYNET.

The right page shows a diagram of the Teradata architecture, including the BYNET 0 and BYNET 1 components, and a grid of nodes (Node 0 to Node 3) with their respective components (PE, AMP, PDE, Link). A yellow sticky note is placed over the diagram, containing the following text:

An Intelliflex node is a 1U chassis and contains 2 eight-core CPUs.

Up to 13 Intelliflex nodes can be configured in a clique - 12 TPA nodes and 1 HSN node.

The right sidebar of the Adobe Reader shows the 'Annotations' panel with a list of comments. The first comment is from 'lc130182' on 4/4/2017 12:11:41 PM, and the second is from 'lc130182' on 4/4/2017 12:12:56 PM, which is the sticky note shown on the right page.

After including highlights and comments in your PDF file, a recommendation is to use the Save AS option and save the updated version with a new name – maybe include your initials.

Example of Add Text Tool

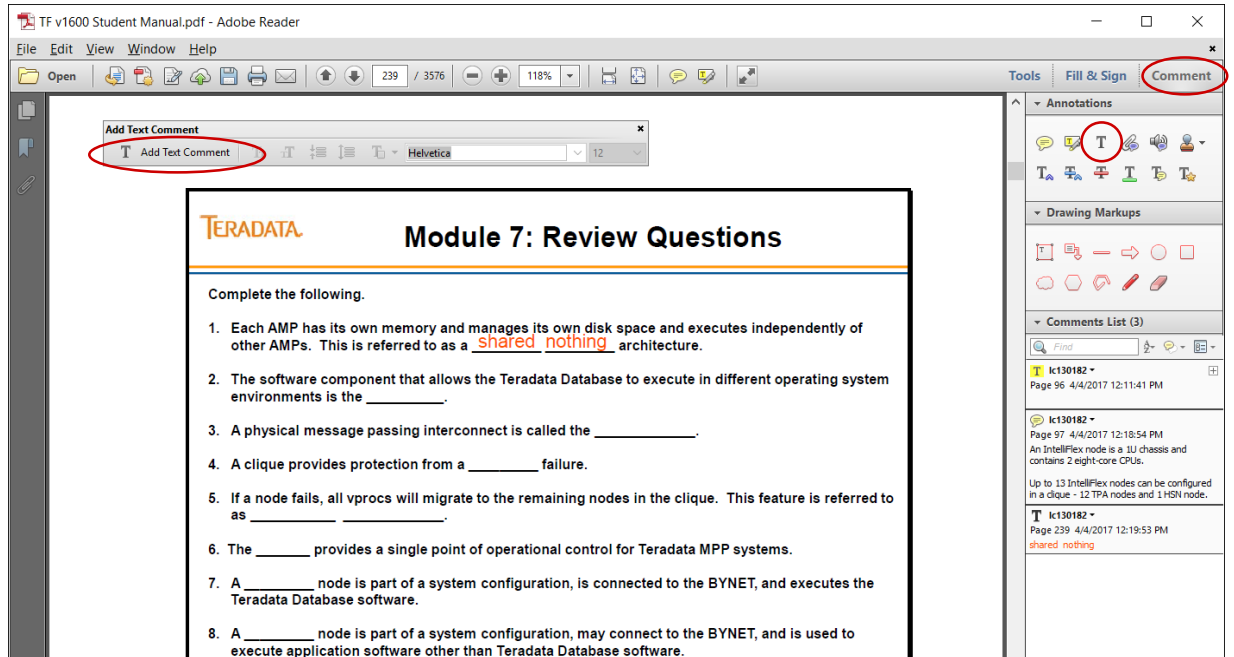
The facing page illustrates an example of using the Add Text tool.

The Add Text Toolbar may be useful when completing review questions as shown on the facing page. You already have the answer to one of hundreds of questions in this course.

After making notes and comments, save your changes. You may want to save your changes to a different PDF file name in order to preserve the original PDF file.

Example of Add Text Tool

The Add Text Comment tool can be used to add text at any location in the PDF file.



The Add Text Comment allows you to changes fonts, colors, etc.

Open a PDF File at Last Page Viewed

If you are reading a 1700+ page PDF manual and you get to page 500, you may want to start on page 500 the next time you open the PDF file. The default is to return you to the beginning of the PDF file. There are multiple options that can be used to open your PDF file at the last page you viewed.

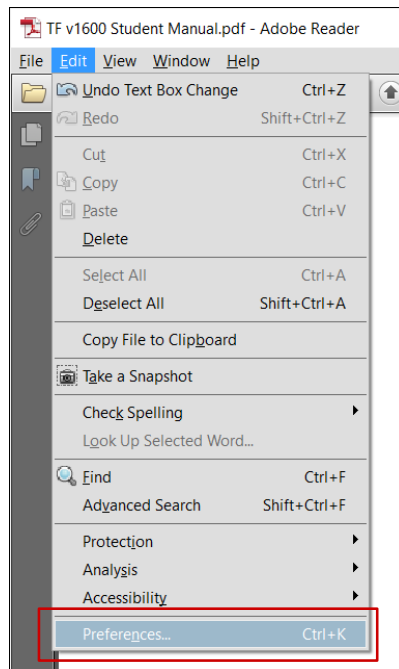
One option is to set a bookmark and select the bookmark when you reopen the file.

Another option is set your preferences so you open every PDF document at the last page that was viewed before closing it.

The next time you reopen your PDF file, you will be able to continue reading from exactly where you were when you closed the PDF document.

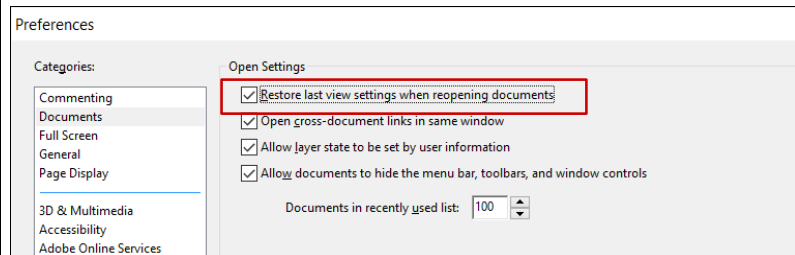
This setting applies to all of the PDF files you have opened recently.

Open PDF file at Last Page Viewed



You may want to set your preferences so you can open your PDF documents at the last page that was viewed before closing the document.

Edit > Preferences > Documents



Course Description

This course provides information on the following major topics:

- Teradata Concepts
- System Architectures (e.g., IntelliBase™ and IntelliFlex™ Systems)
- Teradata Physical Database Design
- Teradata SQL ANSI Differences

Description

The primary focus of this five day course is to teach you about the design and implementation of the Teradata Database.

The major topics in this course include:

- Teradata Database features and functions
- The parallelism of the Teradata Database
- How Teradata is implemented on MPP systems (e.g., IntelliFlex™ systems)
- How to perform physical database design for Teradata Database
- Teradata SQL ANSI Differences
- Introduction to export/import capabilities using Teradata Parallel Transporter (TPT) operators

Who Should Attend

This class is a learning event for relational database experienced individuals who need to learn the Teradata Database. This course is designed for Teradata practitioners who need to get hands-on practice with the Teradata Database in a learning environment.

- Professional Services Consultants
- Channel Partners

Prerequisites

An understanding of relational databases, SQL, and the logical data model is **necessary** before attending this course.

Experience with large systems, relational databases and SQL, and an understanding of the Linux operating system is **useful, but not required** before attending this course.

There are Web Based Training classes that provide information about Teradata concepts and SQL.

- Overview of Teradata
- Teradata SQL

Who Should Attend and Prerequisites

Who Should Attend

This course is designed for ...

- Teradata Professional Services Consultants
- Channel Partners

Prerequisites

Required:

- An understanding of the logical data model, relational, SQL, and data processing concepts.

Useful, but not required:

- Experience with relational databases and SQL
- Experience with large systems used with Teradata

Class Format

This five-day class will be conducted as a series of lectures with classroom discussions, review questions, and workshops.

Classroom Rules

The classroom rules are listed on the facing page.

Class Format and Rules

Class Format

This five day class consists of ...

- Instructor presentations
- Class discussions
- Review Questions
- Workshop exercises

Classroom Rules

The classroom rules are ...

- Turn off your cellular phones.
- Turn off the volume on your laptops.
- During lecture, only use your laptop to follow the class materials.
- Come to class on time in the morning and after breaks.
- Enjoy the week.

Outline of the Class

An outline of the class is described on the following page. Major topic examples are listed for each week.

Teradata Concepts

Teradata features and functions

Parallelism and Teradata

MPP System Architectures

Characteristics of MPP systems (e.g., IntelliFlex™, etc.) – typical configurations

Disk Array subsystems and how Teradata utilizes disk arrays

Teradata Physical Database Design

Primary and secondary index selection; partitioned, NoPI, and columnar tables

How the Teradata database works

Collecting Statistics and Explains

SQL ANSI syntax & features; Teradata and ANSI transaction modes

Temporary tables, System Calendar, and Teradata System Limits

Introduction to Application Utilities

Teradata Parallel Transporter (TPT) operators

Teradata Certification Tests (14.0)

The facing page lists the various Teradata certification tests. Depending upon the tests that are completed, you can earn various Teradata Certified designations such as Teradata Certified Professional.

The Teradata 14 Certification tests require knowledge plus experience with Teradata. This manual will help you prepare for these Teradata 14 tests, but many of the test questions are scenario-based and Teradata experience is needed to answer these types of questions.

Teradata 14.0 Certification Tests

- ✓ 141 – Teradata 14 Basics
- 142 – Teradata 14 SQL
- ✓ 143 – Teradata 14 Physical Design and Implementation
- 144 – Teradata 14 Database Administration
- 145 – Teradata 14 Solutions Development
- 146 – Teradata 14 Enterprise Architecture

- 147 – Teradata 14 Comprehensive Mastery

By passing all seven Teradata 14 certification tests, you become a Teradata 14 Certified Master.

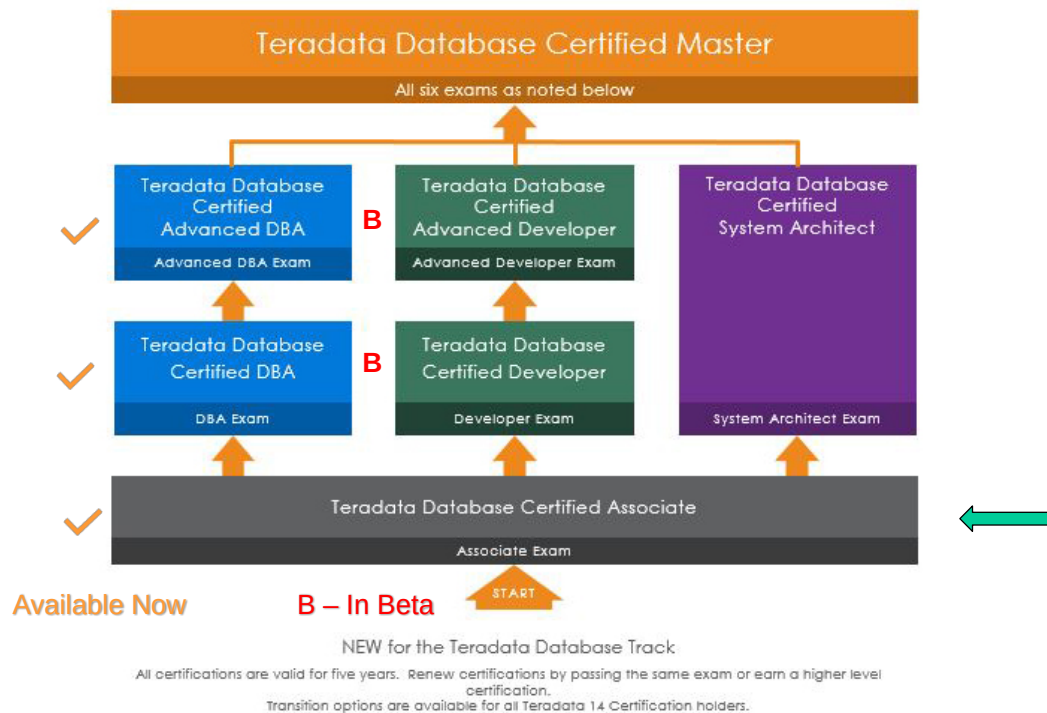
- ✓ This course (along with Teradata experience) will help you prepare for these tests.

Teradata Certification Tests

The facing page identifies the various Teradata certification tests starting in 2017 and continuing into 2018. Depending upon the tests that are completed, you can earn various Teradata Certified designations.

The Teradata Certification tests require knowledge plus experience with Teradata release 16 and higher. This manual will help you prepare for these Teradata tests, but many of the test questions are scenario-based and Teradata experience is needed to answer these types of questions.

Teradata Certification Tests



Notes

Module 1



Teradata Overview

After completing this module, you will be able to:

- **Describe the purpose of the Teradata product**
- **Understand the history of the Teradata Corporation**
- **List major architectural features of the product**
- **Where does Teradata fit?**

Teradata Proprietary and Confidential

Notes

Table of Contents

What is Teradata?	1-4
How large is a Trillion and a Quadrillion?.....	1-4
Teradata – A Brief History.....	1-6
Teradata – The beginning.....	1-8
What is a Data Warehouse?	1-10
Data Marts	1-10
Independent Data Marts	1-10
Logical Data Marts.....	1-10
Dependent Data Marts.....	1-10
What is a Relational Database?.....	1-12
Primary Key	1-12
Answering Questions with a Relational Database	1-14
Foreign Key.....	1-14
Teradata's Competitive Advantages	1-16
Teradata Vantage	1-18
What is the Teradata NewSQL Engine?	1-20
Teradata Everywhere™	1-22
Module 1: Review Questions	1-24

What is Teradata?

Teradata is a Relational Database Management System (RDBMS) for the world's largest commercial databases. It is possible to have databases with petabytes of data in size. This characteristic makes Teradata an obvious choice for large data warehousing applications. With its parallelism and scalability, Teradata allows you to start small with a single node and grow large with many nodes through linear expandability.

Teradata is comparable to a large database server, with multiple client application making inquiries against it concurrently.

Teradata 14.0 was released in February, 2012.
Teradata 14.10 was released in July, 2013.
Teradata 15.0 was released in April, 2014.
Teradata 15.10 was released in June, 2015.
Teradata 16.00 was released in March, 2017.
Teradata 16.10 was released in August, 2017.
Teradata 16.20 was released on February 23, 2018.
Teradata 16.20 Update 1, Teradata Vantage October, 2018.

The acronym SUSE comes from the German name "Software und System Entwicklung" which means Software and Systems Development.

The ability to manage terabytes and petabytes of data is accomplished using the concept of parallelism, wherein many individual processors perform smaller tasks concurrently to accomplish an operation against a huge repository of data.

Acronyms: SLES – SUSE Linux Enterprise Server
SUSE – Software und System Entwicklung (German name which means Software and Systems Development)

How large is a Trillion and a Quadrillion?

The **Teradata database** was the first commercial database system to support a trillion bytes of data. It is hard to imagine the size of a trillion. To put it in perspective, the life span of the average person is 2.5 gigaseconds (or said differently 2,500,000,000 seconds). A trillion seconds is 31,688 years!

Teradata has customers with multiple petabytes of data. One petabyte is one quadrillion bytes of data. A petabyte is effectively 1000 terabytes.

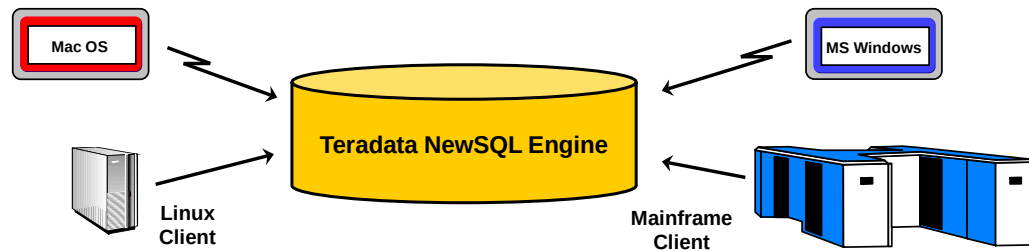
1 Kilobyte (KB)	= 1024 bytes
1 Megabyte (MB)	= 1024^2 >= 1,000,000 bytes
1 Gigabyte (GB)	= 1024^3 >= 1,000,000,000 bytes
1 Terabyte (TB)	= 1024^4 >= 1,000,000,000,000 bytes
1 Petabyte (PB)	= 1024^5 >= 1,000,000,000,000,000 bytes
1 Exabyte	= 1024^6 >= 1,000,000,000,000,000,000 bytes
1 Zetabyte	= 1024^7 >= 1,000,000,000,000,000,000,000 bytes
1 Yottabyte	= 1024^8 >= 1,000,000,000,000,000,000,000,000 bytes

What is Teradata?

The **Teradata NewSQL Engine (database)** is a Relational Database Management System.

Designed to run the world's largest commercial databases.

- Preferred solution for enterprise data warehousing
- Acts as a "database server" to client applications throughout the enterprise
- Uses parallelism to manage terabytes or petabytes of data
 - A terabyte is a trillion bytes of data – 10^{12} .
 - A petabyte is a quadrillion bytes of data – 10^{15} , effectively 1000 terabytes.
- Capable of supporting many concurrent users from various client platforms (over TCP/IP or IBM channel connections).
- The latest Teradata release is 16.20 and executes as a SUSE Linux (SLES 11) application.



Teradata – A Brief History

The **Teradata** Corporation was founded in 1979 in Los Angeles, California. The corporate goal was the creation of a “database computer” which could handle billions of rows of data, up to and beyond a terabyte of data storage. It took five years of development before a product was shipped to a first customer in 1984. In 1982, the YNET technology was patented as the enabling technology for the parallelism that was at the heart of the architecture. The YNET was the interconnect which allowed hundreds of individual processors to share the same bandwidth.

In 1987, Teradata went public with its first stock offering. In 1988, Teradata partnered with the NCR Corporation to build the next generation of database computers (e.g., 3700). At this time, Dr. Phillip Neches (one of the initial Teradata founders) became the Chief Scientist of NCR and started a movement within NCR to build systems supporting the Teradata database. Before either company could market its next generation product, NCR was purchased by AT&T Corporation at the end of 1991. AT&T purchased Teradata and folded Teradata into the NCR structure in January of 1992. The new division was named AT&T GIS (Global Information Solutions).

In 1996, AT&T spun off three separate companies, one of which was NCR which then returned to its old name. Teradata was a division of NCR from 1997 until 2001. In 1997, Teradata (as part of NCR) had become the world leader in scalable data warehouse solutions.

In 2007, NCR and Teradata separated as two corporations. Teradata has developed into the industry leader in data warehousing.

Since 2007, the Teradata Corporation has acquired numerous software companies and technologies such as Claraview, Aster, eCircle, RainStor, and Big Data Partnership. These acquisitions have increased Teradata's analytical and consulting capabilities.

Teradata – A Brief History

- 1979 – Teradata Corp founded in Los Angeles, California
 - Development begins on a massively parallel computer and YNET technology
- 1984 – Teradata markets the first database computer DBC/1012
 - First system purchased by Wells Fargo Bank of California
- 1989 – Teradata and NCR partner on next generation of DBC.
- 1992 – NCR Corporation is acquired by AT&T and Teradata is merged into NCR within AT&T and is named AT&T GIS (Global Information Solutions).
- 1996 – [AT&T spins off NCR Corporation with Teradata; Teradata Version 2 is released.](#)
- 1997 – Teradata becomes the industry leader in data warehousing.
- 2000 – The first 100+ Terabyte system is put into production.
- 2007 – [NCR and Teradata become two separate corporations. Teradata 12.0 is released.](#)
- 2012 – [Teradata 14.0 is released on February 14, 2012.](#)
 - More than 34 customers with 1 PB or larger systems
- 2013 – Teradata 14.10 is released (July, 2013) as well as 2700/6700 systems.
- 2014 – Teradata 15.0 is released (April, 2014) as well as 2750/6750 systems.
- 2015 – Teradata 15.10 is released (June, 2015) as well as 1800/2800/6800 systems.
- 2016 – [Teradata 15.10 is released on IntelliFlex systems \(BYNET connected storage\) and as an AWS \(Amazon Web Services\) cloud implementation.](#)
- 2017 – Teradata 16.00 (March, 2017) and Teradata 16.10 (August, 2017) are released.
- 2018 – Teradata 16.20 (Feb.) followed by update 1 (Oct.) [Teradata Vantage](#) are released.

Teradata – The beginning

The beginning of **Teradata in the 1980's**. The model was to use large numbers of small computers to solve big data problems. Of course, big data definition in the early days was a gigabyte instead of multiple petabytes. However, it was still a big problem for people at the time to be able to put enough compute power to work to solve the problems of the day on the data that people were trying to understand.

Teradata was incorporated in the Los Angeles area in 1979, the product of a three-year collaboration between Caltech scientists and Citibank's advanced technology group to develop a massive database management system based on parallel processing.



B&W image of Teradata's executive staff standing beside one of their Data Base Computers. "Teradata Executive Staff -- (standing left to right) Dr. Philip M. Neches, Vice President and Chief Scientist; Dr. Jack E. Shemer, President and Chairman of the Board; Catrroll Reed, Vice President of Research and Development; William P. Worth, Vice President of Manufacturing; Walter E. Muir, Vice President of Marketing; Jerold J. Modes, Vice President and Chief Financial Officer.

Teradata – The beginning

Teradata back in the beginning...1980's

One of the First Shared Nothing Technologies

- Microprocessor – 8086
- Personal computer – Commodore 64, Apple I, TRS80
- Disk drive – 200 MB weighs 30 lbs and required a washing machine sized cabinet
- Enterprise computing – Blue or Bunch mainframe
- Client/Server computing – didn't exist
- Internet and e-mail – only if you were a university researcher or DoD



Teradata was founded to solve enterprise Terabyte problems utilizing microprocessors and commodity disks.

What is a Data Warehouse?

A **data warehouse** is a central, enterprise-wide database that contains information extracted from the operational data stores. Data warehouses have become more common in corporations where enterprise-wide detail data may be used in on-line analytical processing to make strategic and tactical business decisions. Warehouses often carry many years worth of detail data so that historical trends may be analyzed using the full power of the data.

Many data warehouses get their data directly from operational systems so that the datum is timely and accurate. While data warehouses may begin somewhat small in scope and purpose, they often grow quite large as their utility becomes more fully exploited by the enterprise.

Data Warehousing is a process, not a product. It is a technique to properly assemble and manage data from various sources to answer business questions not previously possible or known.

Data Marts

A **data mart** is a special purpose subset of enterprise data used by a particular department, function or application. Data marts may have both summary and detail data, however, usually the data has been pre-aggregated or transformed in some way to better handle the particular type of requests of a specific user community.

Independent Data Marts

Independent data marts are created directly from operational systems, just as is a data warehouse. In the data mart, the datum is usually transformed as part of the load process. Data might be aggregated, dimensionalized or summarized historically, as the requirements of the data mart dictate.

Logical Data Marts

Logical data marts are not separate physical structures but rather are an existing part of the data warehouse. Because in theory the data warehouse contains the detail data of the entire enterprise, a logical view of the warehouse might provide the specific information for a given user community, much as a physical data mart would. Without the proper technology, a logical data mart can be a slow and frustrating experience for end users. With the proper technology, it removes the need for massive data loading and transforming, making a single data store available for all user needs.

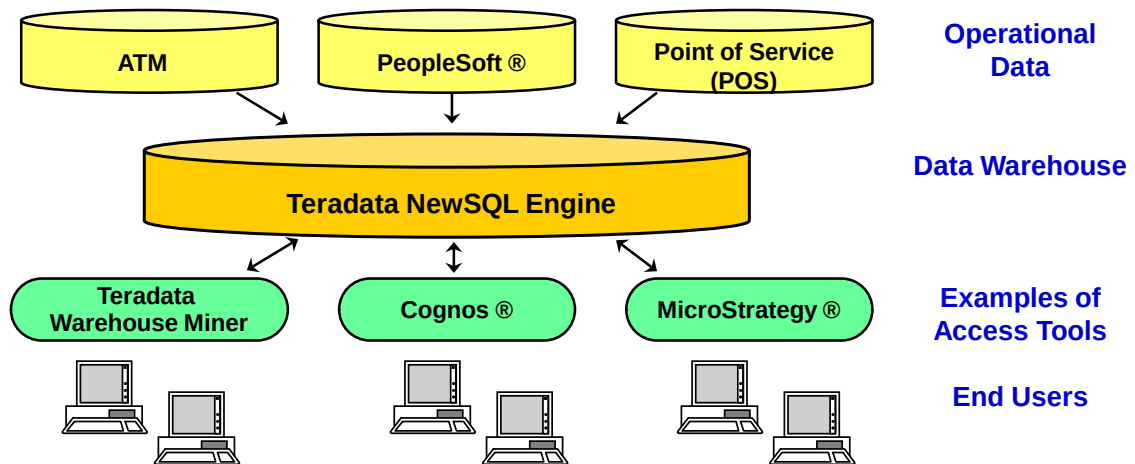
Dependent Data Marts

Dependent data marts are created from the detail data in the data warehouse. While having many of the advantages of the logical data mart, this approach still requires the movement and transformation of data but may provide a better vehicle for performance-critical user queries.

What is a Data Warehouse?

A Data Warehouse is a central, **enterprise-wide database** that contains information extracted from Operational Data Stores (ODS).

- Based on enterprise-wide model
- Can begin small but may grow large rapidly
- Populated by extraction/loading data from operational systems
- Responds to end-user "what if" queries
- Can store detailed as well as summary data



What is a Relational Database?

A **database** is a collection of permanently stored data that is used by an application or enterprise. A database contains **logically related data**. Basically, that means that the database was created with a purpose in mind. A database supports **shared access** by many users. A database also is **protected** to control access and **managed** to retain its value and integrity.

The key to understanding relational databases is the concept of the **table** made up of **rows** and **columns**.

A **column** always contains like data. In the example on the following page, the column named LAST NAME contains last name, and never anything else. The position of the column in the table is arbitrary.

A **row** is one instance of all the columns of a table. In our example, all of the information about a single employee is in one row. The sequence of the rows in a table is arbitrary.

Specifically, in a Relational Database, tables are defined as a named collection of one or more named columns by zero or more rows of related information.

Notice that each row of the table is about a person. There are no rows with data on two people, nor are there rows with information on anything other than people. This may seem obvious, but the concept underlying it is very important.

Each row represents an occurrence of an **entity** defined by the table. An **entity** is defined as a person, place or thing about which the table contains information. In this case the entity is the employee.

Primary Key

Tables, made up of rows and columns, represent **entities** or relationships. Entities are the people, places, things, or events that the Entity Tables Model. Each table holds only one kind of row, and each row is uniquely identified within a table by a **Primary Key (PK)**.

A Primary Key is **required**. A Primary Key **can be more than one column**. A Primary Key **uniquely identifies** each row in a table. **No duplicate values are allowed**. **Only one** Primary Key is allowed per table. The Primary Key for the EMPLOYEE table is the Employee number. No two employees can have the same number.

Because it is used to identify, the Primary Key **cannot be NULL**. There must be something in that field to uniquely identify each occurrence. Primary Key values **cannot be changed**. Historical information as well as relationships with other entities may be lost if a PK value is changed or re-used.

What is a Relational Database?

- A Relational Database consists of a set of logically related tables.
- A table is a two dimensional representation of data consisting of rows and columns.
- Each row is in the table uniquely identified by a **Primary Key (PK)** – 1 or more columns.
 - A PK cannot have duplicate values and cannot be NULL; only one per table.
 - A PK are considered “non-changing” values.
- A table may optionally have 1 or more **Foreign Keys (FK)**.
 - A FK can be 1 or more columns, can have duplicate values, and allows NULLs
 - Each FK value must exist somewhere as a PK value

Column
↓

Employee Table	EMPLOYEE NUMBER	MANAGER EMPLOYEE NUMBER	DEPT NUMBER	JOB CODE	LAST NAME	FIRST NAME	HIRE DATE	BIRTH DATE	SALARY AMOUNT
	PK	FK	FK	FK					
	1006	1019	301	312101	Stein	John	861015	631015	3945000
	1008	1019	301	312102	Kanieski	Carol	870201	680517	3925000
	1007	1005	?	432101	Villegas	Arnando	870102	470131	5970000
Row →	1003	0801	401	411100	Trader	James	860731	570619	4785000

This Employee table has 9 columns and 4 rows of sample data – one row per employee.

There is no prescribed order for the rows of the table.

There is only one row “format” for the entire table.

Missing data values are represented by “NULLS”.

Answering Questions with a Relational Database

A relational database is a collection of **relational tables** stored in a single installation of a **relational database management system (RDBMS)**. The words “management system” indicate that not only is this a relational database but also there is underlying software to provide additional functions that the industry expects. This includes transaction integrity, security, journaling, and other features that are expected of databases in general. The Teradata database is a Relational Database Management System.

Relational databases do not use access paths to locate data, rather data connections are made by **data values**. In other words, data connections are made by matching values in one column with the values in a corresponding column in another table. This connection is referred to as a **JOIN** in relational terminology.

The diagram on the facing page show how the values in one table may be matched to values in another. Both tables have a column named “Department Number”. That connection allows the database to answer questions like, “What is the name of the department in which an employee works?”

One reason relational databases are so powerful is that, unlike other databases, they are based on a mathematical model developed by Dr. Edgar Codd and implement a query language solidly founded in **set theory**.

To summarize, a relational database is a **collection of tables**. The data contained in the tables can be associated using data values, specifically, columns with **matching data values**.

Foreign Key

Relational Databases permit associations by data value across more than one table. **Foreign Keys (FKs)** model the relationships between entities.

On the facing page you will see that the employee table has 3 FK columns, one of which models the relationship between employees and their departments. A second one models the relationship between employees and their job codes.

A third FK column is used to model the relationship between employees and each other. This is called a “recursive” relationship.

Rules of Foreign Keys include:

- Duplicate values are allowed in a FK column.
- Missing values are allowed in a FK column.
- Values may be changed in a FK column.
- Each FK value must exist as a Primary Key.

Note that Dept_Number is the **Primary Key** for the DEPARTMENT table.

Employee (partial listing)

EMPLOYEE NUMBER	MANAGER EMPLOYEE NUMBER	DEPT NUMBER	JOB CODE	LAST NAME	FIRST NAME	HIRE DATE	BIRTH DATE	SALARY AMOUNT
PK	FK	FK	FK					
1006	1019	301	312101	Stein	John	861015	631015	3945000
1008	1019	301	312102	Kanieski	Carol	870201	680517	3925000
1005	0801	403	431100	Ryan	Loretta	861015	650910	4120000
1004	1003	401	412101	Johnson	Darlene	861015	560423	4630000
1007	1005	403	432101	Villegas	Arnando	870102	470131	5970000
1003	0801	401	411100	Trader	James	860731	570619	4785000

Department

DEPT NUMBER	DEPARTMENT NAME	BUDGET AMOUNT	MANAGER EMPLOYEE NUMBER
PK			FK
501	marketing sales	80050000	1017
301	research and development	46560000	1019
403	education	93200000	1005
402	software support	30800000	1011
401	customer support	98230000	1003

Questions:

1. Name the department in which James Trader works.
2. Who manages the Education Department?
3. Identify by name an employee who works for James Trader.

Teradata's Competitive Advantages

As technology has improved, a number of aspects of the decision support environment have changed (improved). DSS systems are expected to:

- Store and efficiently process detailed data (reduces the need for summarized data).
- Process ad hoc queries in a timely fashion.
- Contain current (up-to-date) data.

Teradata meets these requirements. The facing page lists a number of the key competitive advantages that Teradata provides. This course will look at these features in detail and explain why these are competitive advantages.

Teradata provides a central, enterprise-wide database that contains information extracted from operational data stores. It provides for a single version of the business (or truth). Characteristics include:

- Based on enterprise-wide model – this type of model provides the ability to look/work across functional processes.
- Customers can begin small (right size), but may grow large rapidly
- Populated by extraction/loading of data from operational systems
- Allows end-users to submit “what if” queries

Examples of applications that Teradata enables include:

- Customer Relationship Management (CRM)
- Campaign Management
- Yield Management
- Supply Chain Management

Some of the reasons that Teradata is the leader in data warehousing include:

- Scalable – supports a small (500 GB) to a massive (Petabytes) database.
- Provides a query optimizer with approximately 30+ years of experience in large-table query planning.
- Does not require complex indexing schemes, complex data partitioning or time-consuming reorganizations (re-orgs).
- Supports ad hoc querying against the detail data in the warehouse, not just summary data in the data mart.
- Designed and built with parallelism from day one (not a parallel retrofit).

Teradata's Competitive Advantages

This course will describe various features (and competitive advantages) of the Teradata database through discussions, examples, and lab exercises.

- **Unlimited, Proven Scalability** – amount of data and number of users; allows for an enterprise wide model of the data.
- **Unlimited Parallelism** – parallel access, sorts, and aggregations.
- **Mature Optimizer** – handles complex queries, up to 128 joins per query, ad-hoc processing.
- **Models the Business** – normalized data (usually in 3NF), robust view processing, & provides star schema capabilities.
- Provides a “single version of the business”.
- **Low TCO (Total Cost of Ownership)** – ease of setup, maintenance, & administration; no re-orgs, lowest disk to data ratio, and robust expansion utility (RECONFIG).
- **High Availability** – no single point of failure.
- **Parallel Load and Unload utility** – robust, parallel, and scalable import and export capabilities via Teradata Parallel Transporter (TPT).

Teradata Vantage

The illustration on the facing page illustrates our first release of the analytics platform (named Teradata Vantage) where Teradata provides access to the most popular analytic tools from Teradata's SQL based tools used by power users to Jupyter, one of the favorites of data scientists and of course R studio, the most popular R integrated development environments. We also provide support for the most preferred languages for analytics: SQL, Python (limited support) and R.

All these tools and languages access the engines through a common interface called the x-engine orchestration layer... that routes the analytic task to the right engine. This means you can mix and match analytics from any supported engine within a single workflow. This is the multi-genre capability that you mentioned earlier.

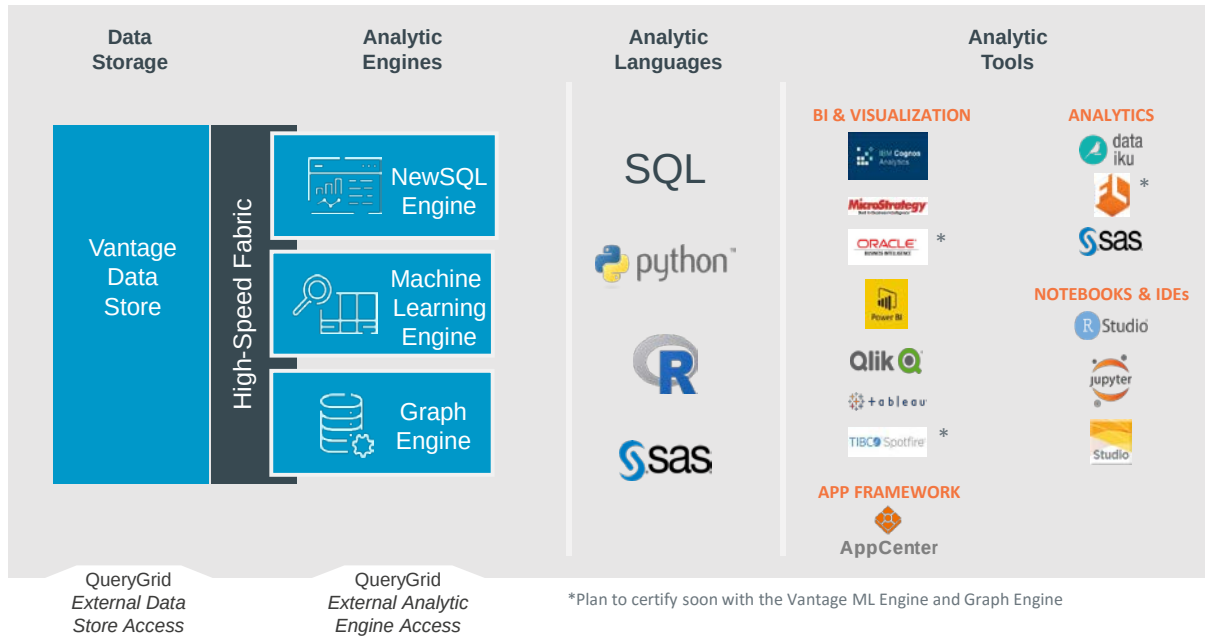
Our analytic engines will process the analytic tasks either in the SQL, Machine Learning or Graph engine that can all access the same data within the persistent data storage.

Teradata's Vantage leverages docker containers to scale out the **Machine Learning and graph engines**. These containers are managed by Kubernetes that enables scalability and flexibility to add new containers quickly to meet business needs.

Docker Containers: Packages software into standardized units for development, shipment and deployment:

- A container image is a lightweight, stand-alone, executable package of a piece of software that includes everything needed to run it: code, runtime, system tools, system libraries, settings. Available for both Linux and Windows based apps, containerized software will always run the same, regardless of the environment. Containers isolate software from its surroundings, for example differences between development and staging environments and help reduce conflicts between teams running different software on the same infrastructure. Teradata uses Docker containers to package analytic engines, UDA software and languages into separate executable engines.
- Kubernetes is an open-source system for automating deployment, **scaling**, and management of containerized applications. It groups containers that make up an application into logical units for easy management and discovery. Kubernetes manages containerized workloads and services, that facilitates both declarative configuration and automation. Kubernetes builds upon 15 years of experience of running production workloads at Google, combined with best-of-breed ideas and practices from the community.

Teradata Vantage



What is the Teradata NewSQL Engine?

The facing page describes the linkage of the NewSQL engine to the Teradata database.

The NewSQL engine is based on the analytic processing power of the Teradata database. There are over 1000 functions available on the NewSQL engine from Teradata developed products such as warehouse miner, geospatial, matrix and time series functions, and our partner product such as Fuzzy Logix and SAS. It also includes our 4D Analytics – Geospatial, Temporal, and Time Series.

As of New Feature Update 1, we have these 10 analytic functions natively available in the Database.

A few notes:

The first two, nPath and Sessionize are available starting from Teradata 15.10, but updates have been done.

Time Series Aggregation function became available as of the initial release of Teradata database 16.20.

All the other functions are new to Teradata database, though some of you familiar with Teradata Aster may be familiar with them.

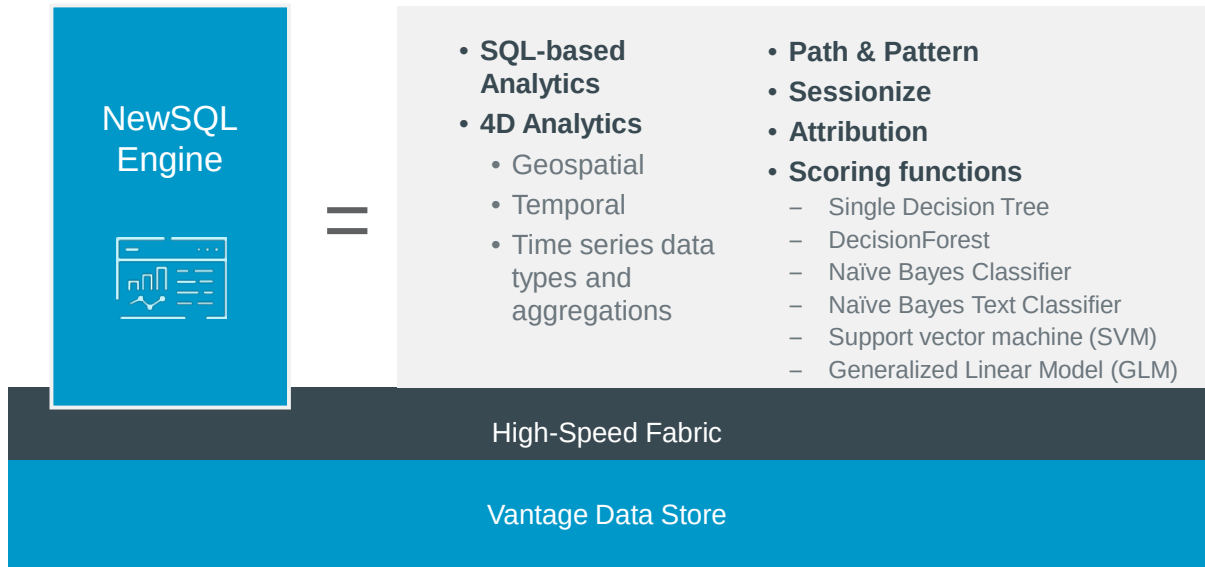
This makes up the NewSQL engine's analytic capabilities. When we combine this with the persistent data storage, together this is what makes up the Teradata database with all the benefits of a production ready business critical environment.

Notes:

SVM – Support Vector Machine

GLM – Generalized Linear Models

What is the Teradata NewSQL Engine?



Teradata Everywhere™

What you will learn about how Teradata works applies to all the various deployment options because it is the **same software**.

Teradata Everywhere messaging;

The Teradata analytics platform delivers a modern analytic environment is flexible to address the needs of your business today and the future as your business changes by enables analytic users throughout the organization to use their preferred analytic tools and engines across data sources, at scale.

Deploy anywhere enables analytic processing across flexible deployment options, including the Teradata Cloud and public clouds, as well as on-premises on Teradata hardware or commodity hardware.

Teradata IntelliCloud™ is our as-a-service offering in which we manage performance, security, availability, and operations of customer data and analytics infrastructure so they can focus on data and analytics.

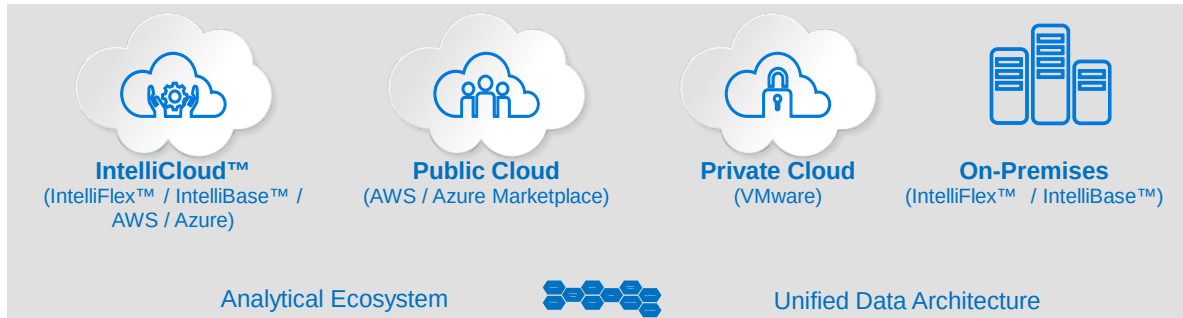
IntelliCloud™ consists of software, services, infrastructure, and support that Teradata cloud experts set up and manage for the customer.

Buy Any Way — Empowers companies to purchase software in more accommodating ways based on specific use cases through simplified pricing bundles, subscription-based licenses, and as-a-Service options.

Move Anytime — Future-proofs buying decisions by taking advantage of our software license portability that provides flexibility to run analytics across deployment options.

The flexibility of Teradata Everywhere de-risks major decisions, allowing you to move forward today with confidence knowing your financial, architectural, and application development investments are protected.

What you will learn in this course applies “everywhere”...



Same software available on every deployment option.

Module 1: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 1: Review Questions

1. Which feature allows Teradata to process enormous volumes of data quickly? ____
 - a. High availability software and hardware components
 - b. High performance servers from Intel
 - c. Proven Scalability
 - d. Parallelism

2. The Teradata NewSQL Engine is primarily a ____ .
 - a. Client
 - b. Server

3. Which choice represents a quadrillion bytes or a Petabyte (PB) of data? ____
 - a. 10^9
 - b. 10^{12}
 - c. 10^{15}
 - d. 10^{18}

4. In a relational table, the set of columns that uniquely identifies a row is the ____ .

Notes



Teradata Basics

After completing this module, you will be able to:

- List and describe the major components of the Teradata architecture.
- Describe how the components interact to manage incoming and outgoing data.
- List 5 types of Teradata database objects.

Teradata Proprietary and Confidential

Notes

Table of Contents

Major Components of Teradata	2-4
Teradata Storage Architecture.....	2-6
Teradata Retrieval Architecture	2-8
Multiple Tables on Multiple AMPs	2-10
Here's how it works:.....	2-10
Teradata Objects.....	2-12
Tables	2-12
Views	2-12
Macros.....	2-12
Triggers	2-12
Stored Procedures.....	2-12
The Data Dictionary Directory (DD/D)	2-14
Structured Query Language (SQL)	2-16
Data Definition Language (DDL)	2-16
Data Manipulation Language (DML)	2-16
QueryGrid (Teradata 15.0).....	2-16
Data Control Language (DCL).....	2-16
User Assistance	2-16
CREATE TABLE – Example of DDL	2-18
Views	2-20
Single-table View	2-20
Multi-Table Views	2-22
Macros.....	2-24
Features of Macros	2-24
Benefits of Macros	2-24
HELP Commands	2-26
SHOW Command	2-28
EXPLAIN Facility	2-30
Summary	2-32
Module 2: Review Questions	2-34

Major Components of Teradata

Up until now we have discussed relational databases in terms of how the user perceives them – as a collection of tables that relate to one another. Now it's time to describe the **components** of the system.

The major software components are the **Parsing Engine (PE)** and the **Access Module Processor (AMP)**.

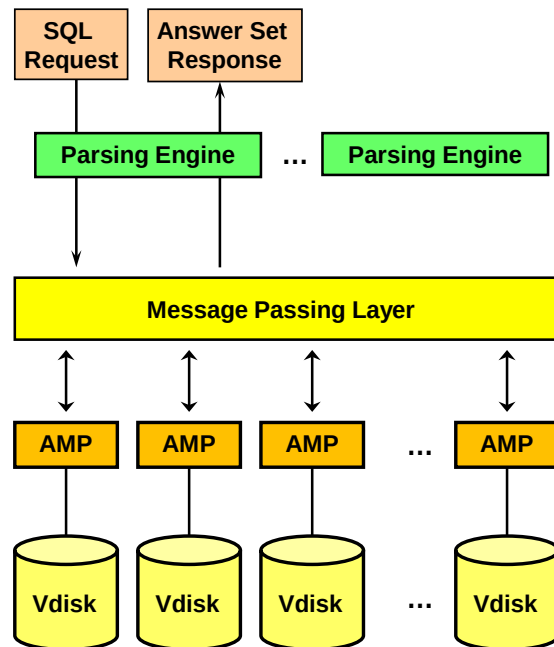
The **Parsing Engine** is a component that interprets SQL requests, receives input records and passes data. To do that it sends the messages through the Message Passing Layer to the AMPs.

The Message Passing Layer (MPL) handles the internal communication of the Teradata NewSQL Engine. The **MPL** is a combination of hardware and software (BYNET and PDE as we will see later). All communication between PEs and AMPs is done via the Message Passing Layer.

The **Access Module Processor (AMP)** is responsible for managing a portion of the database. An AMP will control some portion of each table on the system. AMPs do all of the physical work associated with generating an answer set including, sorting, aggregating, formatting and converting.

A **Virtual Disk** is disk space associated with an AMP. Tables/data rows are stored in this space. A virtual disk is usually assigned to two or more disk drives in a disk array. This concept will be discussed in detail later in the course.

Major Components of Teradata



Parsing Engines (PE)

- Manage sessions for users
- Parse, optimize, and send your request to the AMPs as execution steps
- Returns answer set response back to client

Message Passing Layer (MPL)

- Allows PEs and AMPs to communicate with each other

Access Module Processors (AMP)

- Owns and manages its storage
- Performs the steps sent by the PEs

Virtual Disks (Vdisk)

- Space owned by the AMP and is used to hold user data (rows within tables).
- Maps to physical space in a disk array.

AMPs store and retrieve rows to and from disk.

Teradata Storage Architecture

On the facing page you will see a simplified view of how the physical components of a Teradata NewSQL Engine work to insert a row of data.

The PEs and AMPs are actually implemented as virtual processors (vprocs) in the system. A vproc is effectively a group of processes that represents a Teradata software component.

The **Parsing Engine** interprets the SQL command and converts the data record from the host into an AMP message.

- The Parsing Engine is a component that interprets SQL requests, receives input records and passes data. To do that it sends the messages through the Message Passing Layer to the AMPs.

The **Message Passing Layer** distributes the row to the appropriate Access Module Processor (AMP).

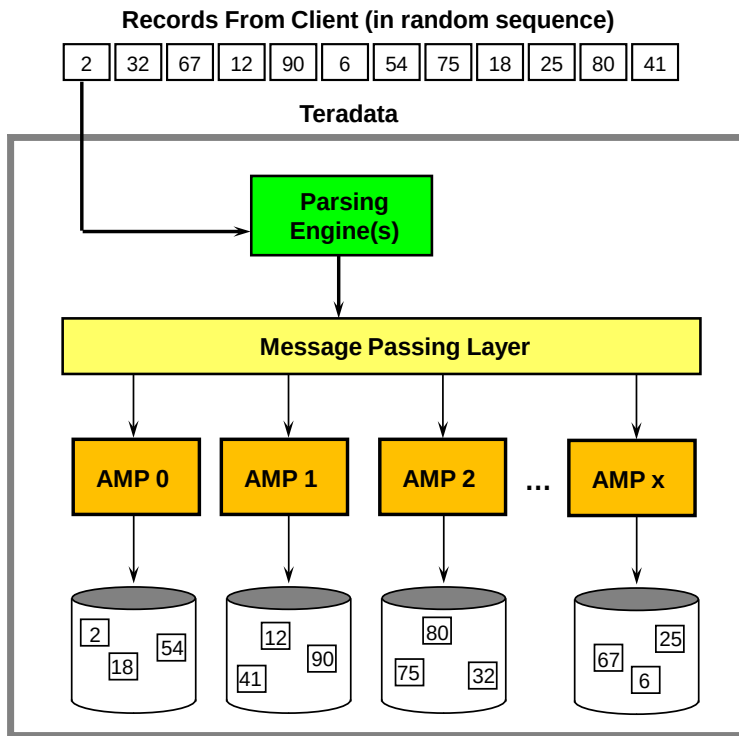
- The Message Passing Layer is implemented as hardware and/or software, depending on the platform used. It determines which vprocs should receive a message.

The **AMP** formats the row and writes it to its associated disks (Vdisks) which are assigned to physical disks in a disk array. The physical disk holds the row for subsequent access.

The **Host** or **Client system** supplies the records. These records are the raw data from which the database will be constructed.

Think of the **AMP (Access Module Processor)** as a independent computer designed for and dedicated to managing a portion of the entire database. It performs all the database management functions – such as sorting, aggregating, and formatting the data. It receives data from the PE, formats the rows, and distributes the rows to the disk storage units it controls. It also retrieves the rows requested by the parsing engine.

Teradata Storage Architecture



The [Parsing Engine](#) dispatches request to insert a row.

The [Message Passing Layer](#) ensures that a row gets to the appropriate AMP (Access Module Processor).

The [AMP](#) stores the row on its associated (logical) disk.

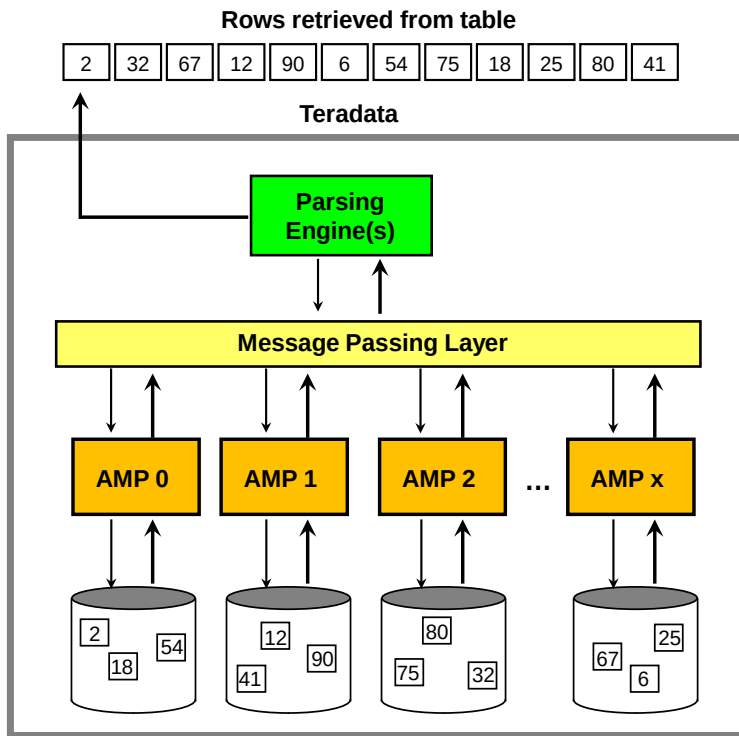
An AMP manages a logical or [virtual disk](#) which is mapped to multiple physical disks in a disk array.

Teradata Retrieval Architecture

Retrieving data from the Teradata NewSQL Engine simply reverses the process of the storage model. A request is made for data and is passed on to a Parsing Engine (PE). The PE optimizes the request for efficient processing and creates tasks for the AMPs to perform, which will result in the request being satisfied. These tasks are then dispatched to the AMPs via the Message Passing Layer. Often times all AMPs must participate in creating the answer set, such as in returning all rows of a table. Other times, only one or a few AMPs need participate, depending on the nature of the request. The PE will ensure that only the AMPs that are needed will be assigned tasks on behalf of this request.

Once the AMPs have been given their assignments, they will retrieve the desired rows from their respective disks. If sorting, aggregating or formatting of any kind is needed, the AMPs will also take care of that. The rows are then returned to the requesting PE via the Message Passing Layer. The PE takes the returned answer set and returns it to the requesting client application.

Teradata Retrieval Architecture



The [Parsing Engine](#) dispatches a request to retrieve one or more rows.

The [Message Passing Layer](#) ensures that the appropriate AMP(s) are activated.

The [AMP\(s\)](#) locate and retrieve desired row(s) in parallel access.

[Message Passing Layer](#) returns the retrieved rows to PE.

The [PE](#) returns row(s) to requesting client application.

Multiple Tables on Multiple AMPs

Logically, you might think that the Teradata NewSQL Engine would assign each table to a specific AMP, and that the AMP would put that table on a single disk. However, as you see on the facing page, that's not what will happen. The system takes the rows that compose a table and divides those rows up among all available AMPs.

Here's how it works:

Tables are distributed across all AMPs. This distribution of rows should be even across all AMPs. This way, a request to get the rows of a given table will result in the workload being evenly distributed across the AMPs.

- Each table has some rows distributed to **each AMP**.
- Each AMP controls one **logical storage unit** which may consist of several physical disks
- Each AMP **places, maintains, and manages** the rows on its own disks.
- Large configurations may have **hundreds of AMPs**.
- **Full table scans**, operations that require looking at all the rows of a table, access all AMPs in parallel. That parallelism is what makes possible the accessing of enormous amounts of data.

Consider the following three tables: EMPLOYEE, DEPARTMENT, and JOB.

The Teradata NewSQL Engine takes the rows from each of the tables and divides them up among all the AMPs. The AMPs divide the rows up among their disks. Notice that **each** AMP gets part of **each** table. Dividing up the tables this way means that all the AMPs and their associated disks will be activated in a full table scan, thus speeding up requests against these tables.

In our example, if you assume four AMPs, each AMP would get approximately **25% of each table**. If, however, AMP #1 were to get 90% of the rows from the EMPLOYEE table that would be called "lumpy or skewed" data distribution. Skewed data distribution would slow the system down because any request that required scanning all the rows of EMPLOYEE would have three AMPs sitting idle while AMP #1 finished its work. It is better to divide all the tables up evenly among all the available AMPs. You will see how this distribution is controlled in a later chapter.

Multiple Tables on Multiple AMPs

EMPLOYEE Table

DEPARTMENT Table

JOB Table

Parsing Engine

Message Passing Layer

AMP 0

AMP 1

AMP 2

AMP x

EMPLOYEE Rows
DEPARTMENT Rows
JOB Rows

EMPLOYEE Rows
DEPARTMENT Rows
JOB Rows

EMPLOYEE Rows
DEPARTMENT Rows
JOB Rows

EMPLOYEE Rows
DEPARTMENT Rows
JOB Rows

Row from each table will usually be stored on each AMP.

Each AMP may have rows from all tables.

Ideally, each AMP will hold roughly the same amount of data.

Teradata Objects

A “**database**” or “**user**” in Teradata NewSQL Engine systems is a collection of objects such as **tables, views, macros, triggers, stored procedures, user-defined functions, or indexes (join and hash)**. Database objects are created and accessed using standard **Structured Query Language** or SQL. Starting with Teradata 14.10, extended object names feature, allows object names to be up to 128 characters where prior it was a 30-byte limit.

All database object definitions are stored in a system database called **the Data Dictionary/Directory** (DD/D).

Databases provide a **logical grouping for information**. They are also the foundation for space allocation and access control. A description of some of the objects follows.

Tables

A table is the logical structure of data in a relational database. It is a **two-dimensional structure made up of columns and rows**. A user defines a table by giving it a table name that refers to the type of data that will be stored in the table. A **column** represents attributes of the table. Column names are given to each column of the table. All the information in a column is the same type, for example, date of birth. Each occurrence of an entity is stored in the table as a **row**. Entities are the people, things, or events that the table is about. Thus a row would represent a particular person, thing, or event.

Views

A view is a **pre-defined subset of one of more tables or other views**. It does not exist as a real table, but serves as a reference to existing tables or views. One way to think of a view is as a virtual table. Views have definitions in the data dictionary, but do not contain any physical rows. The database administrator can use views to control access to the underlying tables. Views can be used to hide columns from users, to insulate applications from database changes, and to simplify or standardize access techniques.

Macros

A macro is a predefined, stored set of one or more SQL commands and optionally, report formatting commands. Macros are used to simplify the execution of frequently used SQL commands.

Triggers

A trigger is a set of SQL statements usually associated with a column or a table and when that column changes, the trigger is fired – effectively executing the SQL statements.

Stored Procedures

A stored procedure is a program that is stored within Teradata and executes within Teradata. A stored procedure uses permanent disk space. A stored procedure is a pre-defined set of statements invoked through a single SQL CALL statement. Stored procedures may contain both Teradata SQL statements and procedural statements (in Teradata, referred to as Stored Procedure Language, or SPL).

Teradata Objects

Examples of objects within a Teradata database or user include:

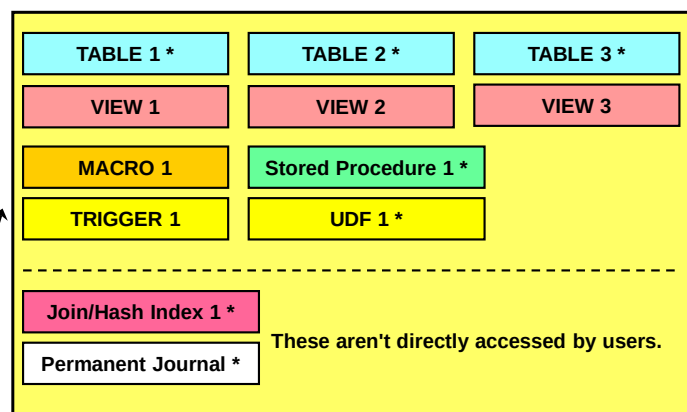
- Tables** – rows and columns of data
- Views** – predefined subsets of existing tables
- Macros** – predefined, stored SQL statements
- Triggers** – SQL statements associated with a table
- Stored Procedures** – program stored within Teradata
- User-Defined Function** – function (C program) to provide additional SQL functionality
- Join and Hash Indexes** – separate index structures stored as objects within a database
- Permanent Journals** – table used to store before and/or after images for recovery

DATABASE or USER can have a mix of various objects.

* - require Permanent Space

These objects are created, maintained, and deleted using SQL.

Object definitions are stored in the DD/D.



The Data Dictionary Directory (DD/D)

The Data Dictionary/Directory is an integrated set of system tables which store database object definitions and accumulate information about **users, databases, resource usage, data demographics, and security rules**. It records specifications about **tables, views, and macros**. It also contains information about **ownership, space allocation, accounting, and access rights (privileges)** for these objects.

Data Dictionary/Directory information is updated automatically during the processing of Teradata SQL **data definition (DDL) statements**. It is used by the Parser to obtain information needed to process all Teradata SQL statements.

Users may access the DD/D through Teradata-supplied views, if permitted by the system administrator.

The Data Dictionary Directory (DD/D)

The DD/D ...

- is an integrated set of system tables
- contains definitions of and information about all objects in the system
- is entirely maintained by the Teradata NewSQL Engine
- is “data about the data” or “metadata”
- is distributed across all AMPs like all tables
- may be queried by administrators or support staff
- is normally accessed via Teradata supplied views

Examples of DD/D views:

- | | |
|----------------|---|
| DBC.TablesV | - information about objects (e.g., tables) in a database/user |
| DBC.UsersV | - information about all users |
| DBC.AllRightsV | - information about access rights |
| DBC.AllSpaceV | - information about space utilization |

Structured Query Language (SQL)

Structured Query Language (SQL) is the language of relational databases. It is sometimes referred to as a "Fourth Generation Language (4GL)" to differentiate it from "Third Generation Languages" such as FORTRAN and COBOL, though it is quite different from other 4GL's. It acts as an intermediary between the user and the database.

SQL is different in some very important ways from other computer languages. Its statements resemble English-like structures. It provides powerful, set-oriented database manipulation including structural modification, data retrieval, modification, and security functions.

SQL is a non-procedural language. Because of its set orientation it does not require IF, GOTO, DO, FOR NEXT or PERFORM statements.

We'll describe three important subsets of SQL – the Data Definition Language, the Data Manipulation Language, and the Data Control Language.

Data Definition Language (DDL)

The DDL allows a user to define the **database objects and the relationships that exist among them**. Examples of DDL uses are creating or modifying tables and views.

Data Manipulation Language (DML)

The DML consists of the **statements that manipulate, change or retrieve the data rows** of the database. If the DDL defines the database, the DML lets the user change the information contained in the database. The DML is the most commonly used subset of SQL. It is used to select, update, delete, and insert rows.

QueryGrid (Teradata 15.0)

Teradata QueryGrid, allows Teradata to orchestrate a query across multiple platforms within the enterprise. It provides access to data on other platforms from within a query and enables the customer to run the right work on the right platform.

Data Control Language (DCL)

The Data Control Language is used to **restrict or permit a user's access** in various ways. It can selectively limit a user's ability to retrieve, add, or modify data. It is used to grant and revoke access privileges on tables and views. An example is granting update privileges on a table, or read privileges on a view to specified users.

User Assistance

These commands allow you to list the objects in a database, or the characteristics of a table, see how a query will execute, or show you the details of your system. They vary widely from vendor to vendor.

Structured Query Language (SQL)

SQL is a query language for Relational Database Systems and is used to access Teradata.

- A fourth-generation language
- A set-oriented language
- A non-procedural language (e.g., doesn't have IF, DO, FOR NEXT, etc.)

SQL consists of:

Data Definition Language (DDL)

- Defines database structures (tables, users, views, macros, triggers, etc.)

CREATE DROP ALTER

Data Manipulation Language (DML)

- Manipulates rows and data values

SELECT INSERT UPDATE DELETE

Data Control Language (DCL)

- Grants and revokes access rights

GRANT REVOKE

Teradata SQL also includes Teradata Extensions to SQL

HELP SHOW EXPLAIN CREATE MACRO

CREATE TABLE – Example of DDL

To create and store the table structure definition in the DD/D, you can execute the CREATE TABLE DDL statement as shown on the facing page.

An example of the output from a SHOW TABLE command follows:

SHOW TABLE Employee;

```
CREATE SET TABLE HR.Employee, Fallback,  
    NO BEFORE JOURNAL,  
    NO AFTER JOURNAL,  
    CHECKSUM = DEFAULT,  
    DEFAULT MERGEBLOCKRATIO,  
    MAP = TD_MAP1  
(  
    EmpNumber INTEGER NOT NULL,  
    EmpMgrNumber INTEGER NOT NULL,  
    DeptNumber INTEGER COMPRESS,  
    JobCode INTEGER COMPRESS ,  
    LastName CHAR(20) NOT CASESPECIFIC NOT NULL,  
    FirstName VARCHAR(20) NOT CASESPECIFIC,  
    HireDate DATE FORMAT 'YYYY-MM-DD'  
    BirthDate DATE FORMAT 'YYYY-MM-DD',  
    SalaryAmount DECIMAL(10,2) COMPRESS 0  
)  
  
UNIQUE PRIMARY INDEX (EmpNumber)  
INDEX (DeptNumber);
```

You can create secondary indexes after a table has been created by executing the CREATE INDEX command. An example of creating an index for the **JobCode** column is shown on the facing page.

Examples of the DROP INDEX and DROP TABLE commands are also shown on the facing page.

CREATE TABLE – Example of DDL

```
CREATE TABLE Employee
( EmpNumber      INTEGER      NOT NULL
, EmpMgrNumber   INTEGER      COMPRESS
, DeptNumber     INTEGER      COMPRESS
, JobCode        INTEGER      COMPRESS
, LastName       CHAR(20)     NOT NULL
, FirstName      VARCHAR(20)
, HireDate       DATE         FORMAT 'YYYY-MM-DD'
, BirthDate      DATE         FORMAT 'YYYY-MM-DD'
, SalaryAmount   DECIMAL (10,2) COMPRESS 0
)
UNIQUE PRIMARY INDEX (EmpNumber)
INDEX (DeptNumber);
```

Other DDL Examples

```
CREATE INDEX (JobCode) ON Employee;
DROP INDEX (JobCode) ON Employee;
DROP TABLE Employee;
```

Views

A **view** is a pre-defined subset or filter of one or more tables. Views are used to control access to the underlying tables and simplify access to data. Authorized users may use views to read data specified in the view and/or to update data specified in the view.

Views are used to simplify query requests, to limit access to data, and to allow different users to look at the same data from different perspectives.

A view is a window that accesses selected portions of a database. Views can show parts of one table (single-table view), more than one table (multi-table view), or a combination of tables and other views. To the user, views look just like tables.

Views are **an alternate way of organizing and presenting information**. A view, like a table, has rows and columns. However, the rows and columns of a view are not stored directly but are derived from the rows and columns of tables whenever the view is referenced. A view looks like a table, but has no data of its own, and therefore takes up no storage space except for its definition. One way to think of a view is as if it was a window through which you can look at selected portions of a table or tables.

Single-table View

A **single-table view** takes specified columns and/or rows from a table and makes them available in a fashion that looks like a table. An example might be an employee table from which you select only certain columns for employees in a particular department number, for example, department 403, and present them in a view.

Example of a CREATE VIEW statement:

```
CREATE VIEW Emp403_v AS
SELECT      EmpNumber
            ,DeptNumber
            ,LastName
            ,FistName
            ,HireDate
FROM        Employee
WHERE       DeptNumber = 403;
```

It is also possible to execute SHOW VIEW viewname;

Views

Views are pre-defined filters of existing tables consisting of specified columns and/or rows from the table(s).

A single table view:

- is a window into an underlying table
- allows users to read and update a subset of the underlying table
- has no data of its own

Employee (Table)

Emp Number	EmpMgr Number	Dept Number	Job Code	Last Name	First Name	Hire Date	Birth Date	Salary Amount
PK	FK	FK	FK			NN	NN	
1006	1019	301	312101	Stein	John	861015	631015	3945000
1008	1019	301	312102	Kanieski	Carol	870201	680517	3925000
1005	0801	403	431100	Ryan	Loretta	861015	650910	4120000
1004	1003	401	412101	Johnson	Darlene	861015	560423	4630000
1007	1005	403	432101	Villegas	Arnando	870102	470131	5970000
1003	0801	401	411100	Trader	James	860731	570619	4785000

Emp403_v (View)

EmpNumber	DeptNumber	LastName	FirstName	HireDate
1005	403	Villegas	Arnando	870102
801	403	Ryan	Loretta	861015

Multi-Table Views

A **multi-table view** combines data from more than one table into one pre-defined view. These views are also called “**join views**” because more than one table is involved.

An example might be a view that shows employees and the name of their department, information that comes from two different tables.

Note: **Multi-table Views are read only.** The user cannot update the data via the view.

One might wish to create a view containing the last name and department name for all employees.

A **Join** operation joins rows of multiple tables and creates rows in work space or spool. These are rows that contain data from more than one table but are not maintained anywhere in permanent storage. These rows in spool are created dynamically as part of a join operation. Rows are matched up based on Primary and Foreign Key relationships.

Example of SQL to create a join view:

```
CREATE VIEW EmpDept_v AS
  SELECT   LastName
          ,DeptName
  FROM     Employee E INNER JOIN Department D
  ON       E.DeptNumber = D.DeptNumber;
```

An example of reading via this view is:

```
SELECT   LastName
          ,DeptName
FROM     EmpDept_v;
```

This example utilizes an alias name of **E** for the **Employee** table and **D** for the **Department** table.

Multi-Table Views

A multi-table view allows users to access data from multiple tables as if it were in a single table. Multi-table views (i.e., join views) are used for reading only, not updating.

Employee (Table)

Emp Number	EmpMgr Number	Dept Number	Job Code	Last Name	First Name
PK	FK	FK	FK		
1006	1019	301	312101	Stein	John
1008	1019	301	312102	Kanieski	Carol
1005	0801	403	431100	Ryan	Loretta
1004	1003	401	412101	Johnson	Darlene
1007	1005	403	432101	Villegas	Arnando
1003	0801	401	411100	Trader	James

Department (Table)

Dept Number	Dept Name	Budget Amount	EmpMgr Number
PK			FK
501	Marketing Sales	80050000	1017
301	Research & Development	46560000	1019
302	Product Planning	22600000	1016
403	Education	93200000	1005
402	Software Support	30800000	1011
401	Customer Support	98230000	1003

Joined Together

Example of SQL to create a join view:

```
CREATE VIEW EmpDept_v AS
SELECT      LastName
            ,DeptName
FROM        Employee E
INNER JOIN  Department D
ON          E.DeptNumber = D.DeptNumber;
```

EmpDept_v (View)

LastName	DeptName
Stein	Research & Development
Kanieski	Research & Development
Ryan	Education
Johnson	Customer Support
Villegas	Education
Trader	Customer Support

Macros

The **Macro** facility allows you to define a sequence of Teradata SQL statements (and optionally Teradata report formatting statements) so that they execute as a single transaction. Macros reduce the number of keystrokes needed to perform a complex task. This saves you time, reduces the chance of errors, reduces the communication volume to Teradata, and allows efficiencies internal to Teradata. Macros are a Teradata SQL extension.

Features of Macros

- Macros are SQL statements stored in the data dictionary.
- They can be modified and executed at will.
- They are re-optimized at execution time.
- They can be executed by interactive or batch applications.
- They are executed by one **EXECUTE** command.
- They can accept user-provided parameter values.

Benefits of Macros

- Macros simplify and control access to the system.
- They enhance system security.
- They provide an easy way of installing referential integrity.
- They reduce the amount of source code transmitted from the client application.
- They are stored in the Teradata data dictionary and are available to all connected hosts.

To create a macro:

```
CREATE MACRO Customer_List AS  
(SELECT CustName FROM Customer);;
```

To execute a macro:

```
EXEC Customer_List;
```

To replace a macro:

```
REPLACE MACRO Customer_List AS  
(SELECT CustNumber, CustName FROM Customer);;
```

To drop a macro:

```
DROP MACRO Customer_List;
```

A **MACRO** is a predefined set of SQL statements which is logically stored in a database.

Macros may be created for frequently occurring queries or sets of operations.

Macros have many features and benefits:

- Simplify end-user access
- Control which operations may be performed by users
- May accept user-provided parameter values
- Are stored in the Teradata Database, thus available to all clients
- Reduces query size, thus reduces LAN/channel traffic
- Are optimized at execution time
- May contain multiple SQL statements

To create a macro:

```
CREATE MACRO Customer_List AS (SELECT CustName FROM Customer);
```

To execute a macro:

```
EXEC Customer_List;
```

To replace a macro:

```
REPLACE MACRO Customer_List AS  
(SELECT CustNumber, CustName FROM Customer);
```

HELP Commands

HELP commands (a Teradata SQL extension) are available to display information on database objects:

- Databases and Users
- Tables
- Views
- Macros
- Triggers
- Join Indexes
- Hash Indexes
- Stored Procedures
- User-Defined Functions

The facing page contains an example of a HELP DATABASE command. This command lists the tables, views, macros, triggers, etc. in the specified database.

The **Kind** (TableKind) column codes represent the following:

A	Aggregate function
B	Combined aggregate and ordered analytical function
C	Table operator parser contract function
D	JAR
E	External stored procedure
F	Standard function
G	Trigger
H	Instance or constructor method
I	Join index
J	Journal
L	User-defined table operator
M	Macro
N	Hash index
O	Table with no primary index and no partitioning
P	Stored procedure
Q	Queue table
R	Table function
S	Ordered analytical function
T	Table with a primary index, partitioning, or both
U	User-defined data type
V	View
X	Authorization
Y	GLOP set

The more frequently used types are in **bold**.

HELP Commands

Databases and Users

```
HELP DATABASE Customer_Service;
HELP USER Dave_Jones;
```

Tables, Views, Macros, etc.

```
HELP TABLE Employee;
HELP VIEW Emp_v;
HELP MACRO Payroll_3;
HELP COLUMN Employee.*;
HELP COLUMN Employee.LastName;

HELP INDEX Employee;
HELP TRIGGER Raise_Trigger;
HELP STATISTICS Employee;
HELP CONSTRAINT Employee.over_21;
HELP JOIN INDEX CustOrder_JI;
HELP SESSION;
```

This is not an inclusive list of HELP commands.

Example:

```
HELP DATABASE Customer_Service;
```

*** Help information returned. 15 rows.

*** Total elapsed time was 1 second.

Table/View/Macro name	Kind	Comment
Contact	T	?
Customer	T	?
Cust_Comp_Orders	V	?
CustOrder_JI	I	?
Department	T	?
:	:	:
Orders	T	?
Orders_Temp	O	?
Orders_HI	N	?
Raise_Trigger	G	?
Set_Ansidate_on	M	?

SHOW Command

HELP commands display information about database objects (users/databases, tables, views, macros, triggers, and stored procedures) and session characteristics.

SHOW commands (another Teradata extension) display the data definition (DDL) associated with database objects (tables, views, macros, triggers, or stored procedures).

BTEQ includes a SHOW command, in addition to and separate from the SQL SHOW command. The BTEQ SHOW provides information on the formatting and display settings for the current BTEQ session, if applicable.

SHOW Command

SHOW commands display how an object was created. Examples include:

<u>Command</u>		<u>Returns statement</u>
SHOW TABLE	table_name;	CREATE TABLE ...
SHOW VIEW	view_name;	CREATE VIEW ...
SHOW MACRO	macro_name;	CREATE MACRO ...
SHOW TRIGGER	trigger_name;	CREATE TRIGGER ...
SHOW PROCEDURE	procedure_name;	CREATE PROCEDURE ...

SHOW TABLE Employee;

```
CREATE SET TABLE HR.Employee, Fallback,
NO BEFORE JOURNAL,
NO AFTER JOURNAL,
CHECKSUM = DEFAULT,
DEFAULT MERGEBLOCKRATIO,
MAP = TD_MAP1
(
EmpNumber INTEGER NOT NULL,
EmpMgrNumber INTEGER COMPRESS,
DeptNumber INTEGER COMPRESS,
JobCode INTEGER COMPRESS,
LastName CHAR(20) CHARACTER SET LATIN NOT CASESPECIFIC,
FirstName VARCHAR(20) CHARACTER SET LATIN NOT CASESPECIFIC,
HireDate DATE NOT NULL,
BirthDate DATE NOT NULL,
SalaryAmount DECIMAL(10,2) COMPRESS 0)
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( DeptNumber );
```

EXPLAIN Facility

The **EXPLAIN** facility (a very useful and robust Teradata extension) allows you to preview how Teradata will execute a query you have requested. It returns a summary of the steps the Teradata NewSQL Engine would perform to execute the request. **EXPLAIN** also discloses the strategy and access method to be used, how many rows will be involved, and its “cost” in minutes and seconds. You can use **EXPLAIN** to evaluate a query performance and to develop an alternative processing strategy that may be more efficient. **EXPLAIN** works on any SQL request. The request is fully parsed and optimized, but it is not run. Instead, the complete plan is returned to the user in readable English statements.

EXPLAIN also provides information about locking, sorting, row selection criteria, join strategy and conditions, access method, and parallel step processing.

There are a lot of reasons for using **EXPLAIN**. The main ones we’ve already pointed out – it lets you know how the system will do the job, what kind of results you will get back, and the relative cost of the query. **EXPLAIN** is also useful for performance tuning, debugging, pre-validation of requests, and for technical training.

The following is an example of an **EXPLAIN** on a very simple query doing a FTS (Full Table Scan).

EXPLAIN SELECT * FROM Employee WHERE DeptNumber = 1018;

Explanation (full)

-
- 1) First, we lock a distinct HR."pseudo table" for read on a RowHash to prevent global deadlock for HR.Employee.
 - 2) Next, we lock HR.Employee for read.
 - 3) We do an all-AMPs RETRIEVE step from HR.Employee by way of index # 4 "HR.Employee.DeptNumber = 1018" with no residual conditions into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 9 rows (657 bytes). The estimated time for this step is 0.02 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.02 seconds.

EXPLAIN Facility

The **EXPLAIN** modifier in front of any SQL statement generates an English translation of the Parser's plan.

The request is fully parsed and optimized, but not actually executed.

EXPLAIN returns:

- Text showing how a statement will be processed (a plan)
- An estimate of how many rows will be involved
- A relative cost of the request (in units of time)

This information is useful for:

- predicting row counts
- predicting performance
- testing queries before production
- analyzing various approaches to a problem

EXPLAIN SELECT * FROM Employee WHERE DeptNumber = 1018;

- :
- 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from HR.Employee by way of index # 4 "HR.Employee.DeptNumber = 1018" with no residual conditions into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 9 rows (657 bytes). The estimated time for this step is 0.02 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.02 seconds.

Summary

The Teradata NewSQL Engine is a high-performance database system that permits the processing of enormous quantities of detail data, quantities which are beyond the capability of conventional systems.

The system is **specifically designed for large relational databases**. From the beginning, Teradata was created to do one thing: manage enormous amounts of data.

Over **one thousand terabytes of on-line storage capacity** is currently available making it an ideal solution for enterprise data warehouses or even smaller data marts.

Uniform data distribution across multiple processors facilitates **parallel processing**. The system is designed in such a way that the component parts divides the work up into approximately equal pieces. This keeps all the parts busy all the time; this enables the system to accommodate a larger number of users and/or more data.

Open architecture adapts readily to new technology. As higher-performance industry standard computer chips and disk drives are made available, they are easily incorporated into the architecture.

As the configuration grows, **performance increase is linear**.

Structured Query Language (SQL) is the industry standard for communicating with relational databases.

The Teradata NewSQL Engine currently runs as a **database server** application under Linux.

The major components of the Teradata NewSQL Engine are:

Parsing Engines (PE)

- Manage sessions for users
- Parse, optimize, and send your request to the AMPs as execution steps
- Returns answer set response back to client

Message Passing Layer (MPL)

- Allows PEs and AMPs to communicate with each other

Access Module Processors (AMP)

- Owns and manages its storage
- Performs the steps sent by the PEs

Virtual Disks (Vdisk)

- Space owned by the AMP and is used to hold user data (rows within tables).
- Maps to physical space in a disk array.

Module 2: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 2: Review Questions

1. What language is used to access a Teradata table?
2. What are five Teradata database objects?
3. What are four major components of the Teradata architecture?
4. What are views?
5. What are macros?

Notes



Teradata Architecture

After completing this module, you will be able to:

- Describe the purpose of the PE and the AMP.
- Describe the overall Teradata NewSQL Engine parallel architecture.
- Describe the relationship of the Teradata NewSQL Engine to its client side applications.

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata and MPP Systems.....	3-4
Teradata Functional Overview	3-6
Channel-Attached Client Software Overview.....	3-8
Network-Attached Client Software Overview	3-10
The Parsing Engine	3-12
Message Passing Layer	3-14
The Access Module Processor (AMP).....	3-16
Teradata Parallelism.....	3-18
Linear Growth and Expandability.....	3-20
Module 3: Review Questions	3-22

Teradata and MPP Systems

Teradata is the software that makes a MPP system appear to be a single system to users and administrators.

What is a Node?

- The “compute or processing node” is the basic building block of the hardware for a Teradata system. The processing node contains the primary processor logic (CPUs), memory, and I/O functionality. Physically, a node is a computer chassis that has its own baseboard with CPUs, memory, I/O cards, power supplies, fans, etc. and has internal disks for its operating system (e.g., Linux) and Teradata software. The physical nodes are independent of each other, but are interconnected with Teradata software and the BYNET.
- Teradata is the database application that executes on one or more processing nodes and makes a multi-node system appear as a single database system to users. Each node is effectively an SMP (Symmetrical Multi-Processing) node. Depending on the configuration, a varying number of AMPs and/or PEs can execute on a node.
- A single node system is often referred to as an SMP system.
- A multi-node system (collection of SMP nodes) is referred to a MPP system.

The BYNET (BanYan NETwork) is the software and hardware interconnect that provides high performance networking capabilities to Teradata MPP (Massively Parallel Processing) systems.

Using communication switching techniques, the **BYNET** allows for **point-to-point**, **multicast**, and **broadcast** communications among the nodes, thus supporting a monumental increase in throughput in very large databases. This technology allows Teradata users to grow massively parallel databases without fear of a communications bottleneck for any database operations.

Although the BYNET software also supports the **multicast** protocol, Teradata software uses the **point-to-point** protocol whenever possible. When an all-AMP operation is needed, Teradata software uses the **broadcast** protocol to broadcast the request to the AMPs.

The BYNET is linearly scalable for point-to-point communications. For each new node added to the system, an additional 56 Gb (with BYNET Version 5 - Infiniband) of bandwidth is added, thus providing scalability as the system grows. Scalability comes from the fact that multiple point-to-point circuits can be established concurrently. With the addition of another node, more circuits can be established concurrently.

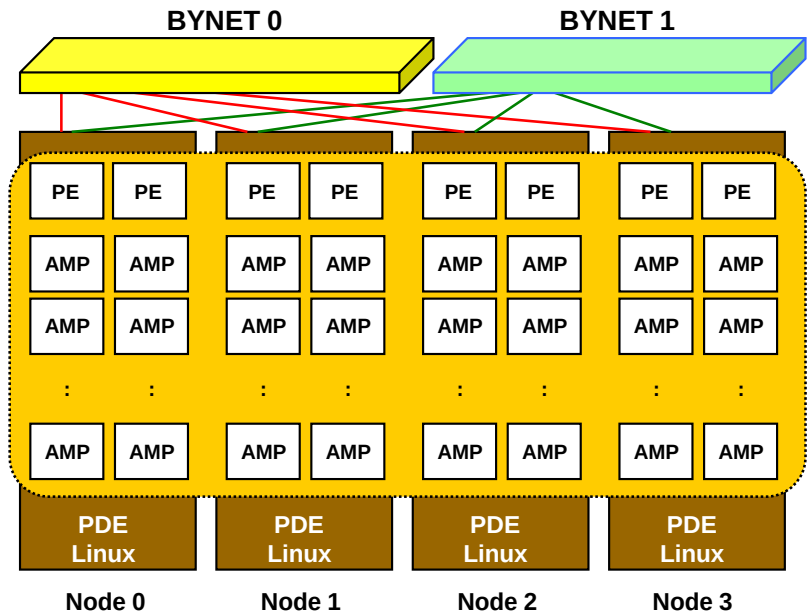
Teradata and MPP Systems

Teradata is the software that makes a MPP system appear to be a single system to users and administrators.

The major components of the Teradata NewSQL Engine are implemented as virtual processors (vproc).

- Parsing Engine (PE)
- Access Module Processor (AMP)

The Communication Layer or Message Passing Layer (MPL) consists of PDE and BYNET SW/HW and connects multiple nodes together.



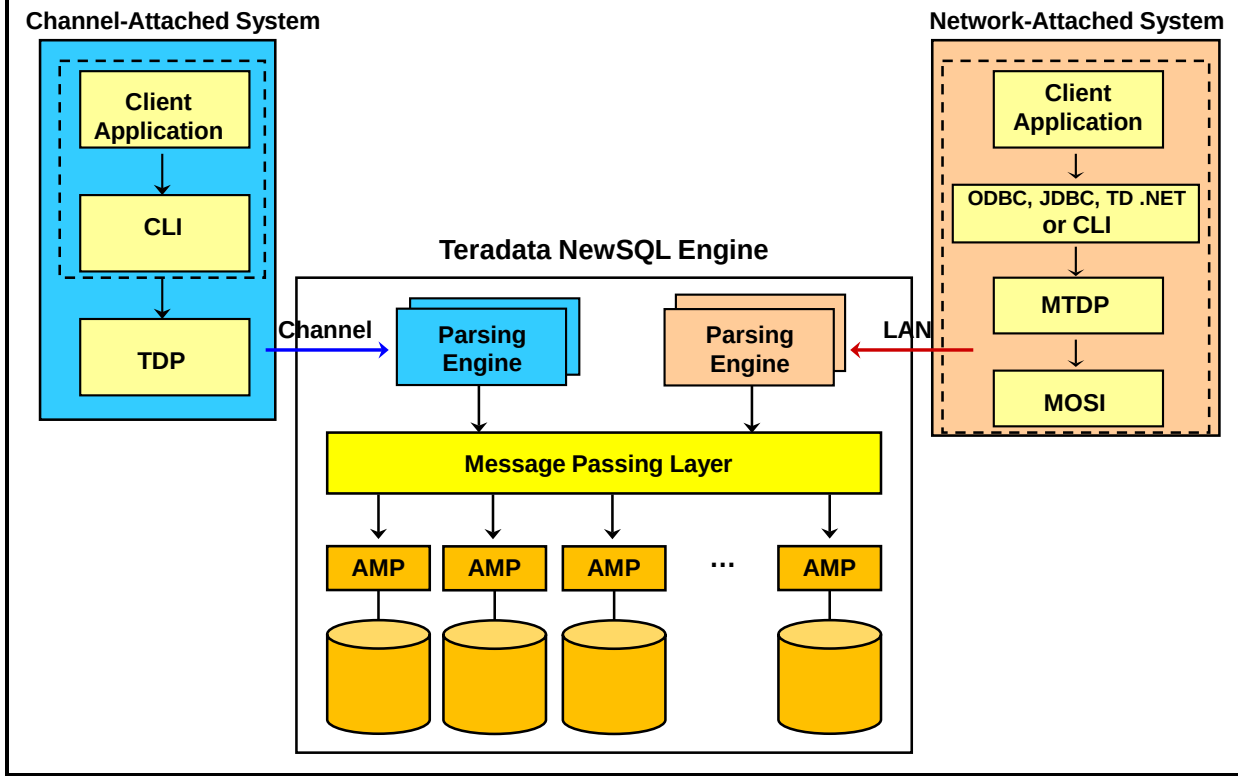
Teradata Functional Overview

The client may be a mainframe system (e.g., IBM) in which case it is **channel-attached** to the Teradata database. Also, a client may be a PC or UNIX-based system that is **LAN or network-attached**.

The client application submits an SQL request to the Teradata database, receives the response, and submits the response to the user.

The **Call Level Interface (CLI)** is a library of routines that resides on the client side. Client application programs use these routines to perform operations such as logging on and off, submitting SQL queries and receiving responses which contain the answer set. These routines are 98% the same in a network-attached environment as they are in a channel-attached.

Teradata Functional Overview



Channel-Attached Client Software Overview

In **channel-attached systems**, there are three major software components, which play important roles in getting the requests to and from the Teradata database.

The **client application** is either written by a programmer or is one of Teradata's provided utility programs. Many client applications are written as "front ends" for SQL submission, but they also are written for file maintenance and report generation. Any client-supported language may be used provided it can interface to the Call Level Interface (CLI).

For example, a user could write a COBOL application with "embedded SQL". The application developer would have to use the Teradata COBOL Preprocessor and COBOL compiler programs to generate an object module and link this object module with the CLI. The CLI application interface provides maximum control over Teradata connectivity and access.

The **Call Level Interface (CLI)** is the lowest level interface to the Teradata database. It consists of system calls which create sessions, allocate request and response buffers, create and de-block "parcels" of information, and fetch response information to the requesting client.

The **Teradata Director Program (TDP)** is a Teradata-supplied program that must run on any client system that will be channel-attached to the Teradata database. The TDP manages the session traffic between the Call-Level Interface and the Database. Its functions include session initiation and termination, logging, verification, recovery, and restart, as well as physical input to and output from the PEs, (including session balancing) and the maintenance of queues. The TDP may also handle system security.

The **Host Channel Adapter** is a mainframe hardware component that allows the mainframe to connect to a FICON, ESCON, or Bus/Tag (older and no longer supported) channel.

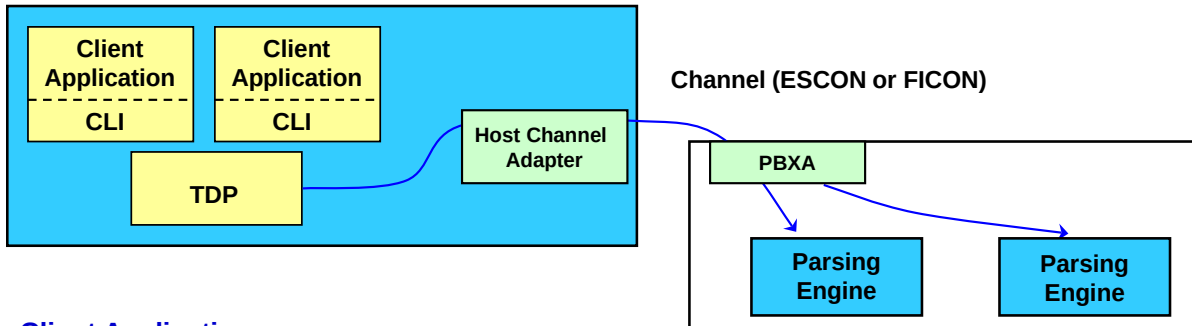
The PBXA (PCI Bus eXtended Adapter) is a PCI adapter card that allows a Teradata server to connect to a FICON channel.

The PBSA (PCI Bus ESCON Adapter) is a PCI adapter card that allows a Teradata server to connect to an ESCON channel.

The PBCA (PCI Bus Channel Adapter) is an older PCI adapter card that allows a Teradata server to connect to a Bus/Tag channel. Bus/Tag connections are no longer supported.

Channel-Attached Client Software Overview

Channel-Attached System



Client Application

- Your own application(s)
- Teradata utilities (BTEQ, etc.)

CLI (Call-Level Interface) Service Routines

- Request and Response Control
- Parcel creation and blocking/unblocking
- Buffer allocation and initialization

TDP (Teradata Director Program)

- Session balancing across multiple PEs
- Ensures proper message routing to/from the Teradata database
- Failure notification (application failure, Teradata restart)

Network-Attached Client Software Overview

In a network-attached environment, the SMPs running Teradata will typically have 1 or more Ethernet adapters that are used to connect to Teradata via a LAN connection. One of the key reasons for having multiple Ethernet adapters in a node is redundancy.

In **network-attached systems**, there are four major software components that play important roles in getting the requests to and from the Teradata database.

The **client application** is written by the programmer using a client-supported language such as “C”. The purpose of the application is usually to submit SQL statements to the Teradata database and perform processing on the result sets. The application developer can “embed” SQL statements in the application and use the Teradata Preprocessor to interpret the embedded SQL statements.

In a networked environment, the application developer can use either the CLI interface or the ODBC driver to access Teradata.

The **Teradata CLI** application interface provides maximum control over Teradata connectivity and access. The ODBC and JDBC drivers are a much more open standard and are widely used with client applications.

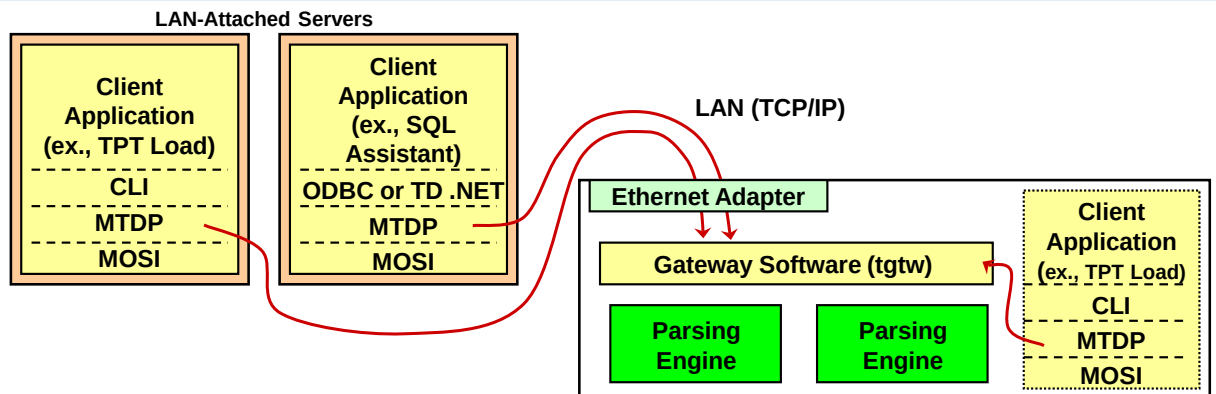
The **Teradata ODBC™ (Open Database Connectivity)** or **JDBC (Java)** drivers use open standards-based ODBC or JDBC interfaces to provide client applications access to Teradata across LAN-based environments.

The **Micro Teradata Director Program (MTDP)** is a Teradata-supplied program that must be linked to any application that will be network-attached to the Teradata Database. The MTDP performs many of the functions of the channel based TDP including session management. The MTDP does not control session balancing across PEs. Connect and Assign Servers that run on the Teradata system handle this activity.

The **Micro Operating System Interface (MOSI)** is a library of routines providing operating system independence for clients accessing the Teradata database. By using MOSI, we only need one version of the MTDP to run on all network-attached platforms.

Teradata Gateway software executes on every node. Gateway software runs as a number of tasks. Two of the key tasks are called "ycgastsk" (assign task) and "ycgcntsk" (connect task). On a 4-node system with one gateway, only one node has the assign task (ycgastsk) running on it and every node will have the connect task (ycgcntsk) running on it. Initial session assignment is done by the assign task and will assign a user session to a PE and to the connect task in the same node as the PE. The connect task on a node will handle connections to the PEs on that node.

Network-Attached Client Software Overview



CLI (Call Level Interface)

- Library of routines for blocking/unblocking requests and responses to/from the Teradata database

ODBC™ (Open Database Connectivity), JDBC™ (Java), or .NET Drivers

- Use open standards-based ODBC, JDBC, or .NET interfaces to provide client applications access to Teradata.

MTDP (Micro Teradata Director Program)

- Library of session management routines

MOSI (Micro Operating System Interface)

- Library of routines providing OS independent interface

The Parsing Engine

Parsing Engines (PEs) are made up of the following software components: session control, the Parser, the Optimizer, and the Dispatcher.

Once a valid session has been established, the PE is the component that manages the dialogue between the client application and the Teradata database.

The major functions performed by **session control** are logon and logoff. Logon takes a textual request for session authorization, verifies it, and returns a yes or no answer. Logoff terminates any ongoing activity and deletes the session's context. When connected to an EBCDIC host the PE converts incoming data to the internal 8-bit ASCII used by the Teradata database, thus allowing input values to be properly evaluated against the database data.

When a PE receives an SQL request from a client application, the **Parser** interprets the statement, checks it for proper SQL syntax and evaluates it semantically. The PE also must consult the Data Dictionary/Directory to ensure that all objects and columns exist and that the user has authority to access these objects.

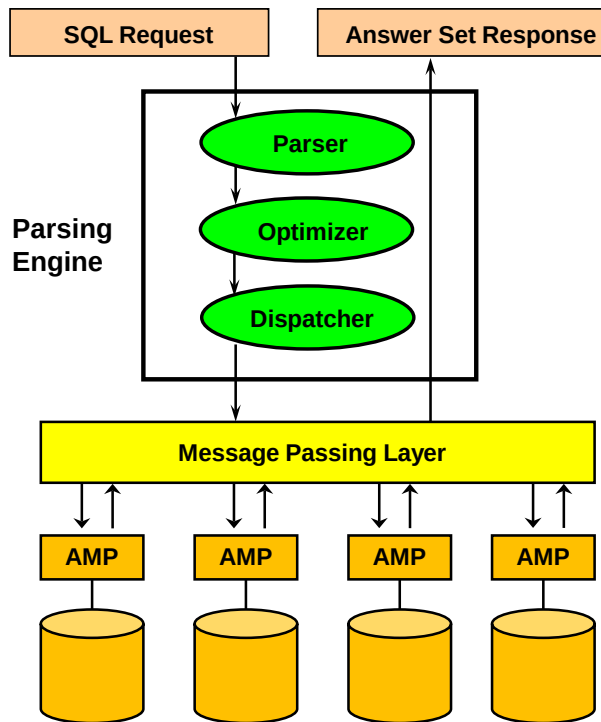
The **Optimizer's** role is to develop the least expensive plan to return the requested response set. Processing alternatives are evaluated and the fastest alternative is chosen. This alternative is converted to executable steps, to be performed by the AMPs, which are then passed to the dispatcher.

The **Dispatcher** controls the sequence in which the steps are executed and passes the steps on to the Message Passing Layer. It is composed of execution control and response control tasks. Execution control receives the step definitions from the Parser, transmits the step definitions to the appropriate AMP or AMPs for processing, receives status reports from the AMPs as they process the steps, and passes the results on to response control once the AMPs have completed processing. Response control returns the results to the user. The Dispatcher sees that all AMPs have finished a step before the next step is dispatched.

Depending on the nature of the SQL request, the step will be sent to one AMP, a few AMPs, or all AMPs.

Note: By default, Teradata Gateway software can support up to 1200 sessions per processing node. Therefore a maximum of 10 Parsing Engines can be defined for a node using the Gateway. It is possible to configure a node to support more than 1200 sessions.

The Parsing Engine



The Parsing Engine is responsible for:

- Managing individual sessions (up to 120)
- Parsing and Optimizing your SQL requests
- Dispatching the optimized plan to the AMPs
- Input conversion (EBCDIC / ASCII) - if necessary
- Sending the answer set response back to the requesting client

Message Passing Layer

The Message Passing Layer (MPL) or Communications Layer handles the internal communication of the Teradata database. All communication between PEs and AMPs is done via the Message Passing Layer.

When the PE dispatches the steps for the AMPs to perform, they are dispatched onto the MPL. The messages are routed to the appropriate AMP(s) where results sets and status information are generated. This response information is also routed back to the requesting PE via the MPL.

The Message Passing Layer is a combination of the Teradata PDE software, the BYNET software, and the BYNET interconnect itself.

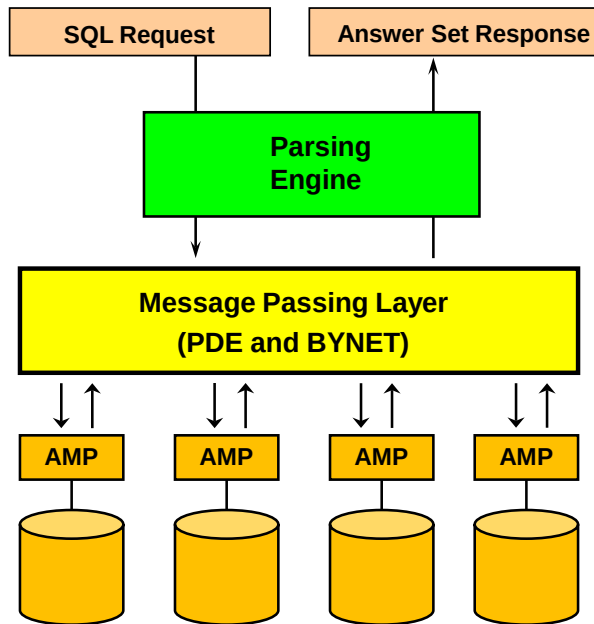
PDE and BYNET software - used for multi-node MPP systems and single-node SMP systems. With a single-node SMP, the BYNET device driver is used in conjunction with the PDE even though a physical BYNET network is not present.

Depending on the nature of the dispatch request, the communication may be a:

- Broadcast** - message is routed to all AMPs and PEs on the system
- Multi-Cast** - message is routed to a group of AMPs
- Point-to-Point** - message is routed to one specific AMP or PE on the system

The technology of the MPL is a key piece in the system part that makes possible the parallelism of the Teradata database.

Message Passing Layer



The Message Passing Layer or Communications Layer is responsible for:

- Carrying messages between the AMPs and PEs
- Point-to-Point, Multi-Cast, and Broadcast communications
- Merging answer sets back to the PE
- Making Teradata parallelism possible

The Message Passing Layer or Communications Layer is a combination of:

- Parallel Database Extensions (PDE) Software
- BYNET Software
- BYNET Hardware for MPP systems

The Access Module Processor (AMP)

The **Access Module Processor (AMP)** is responsible for managing a portion of the database. An AMP will control some portion of each table on the system. AMPs do all of the physical work associated with generating an answer set including, sorting, aggregating, formatting and converting.

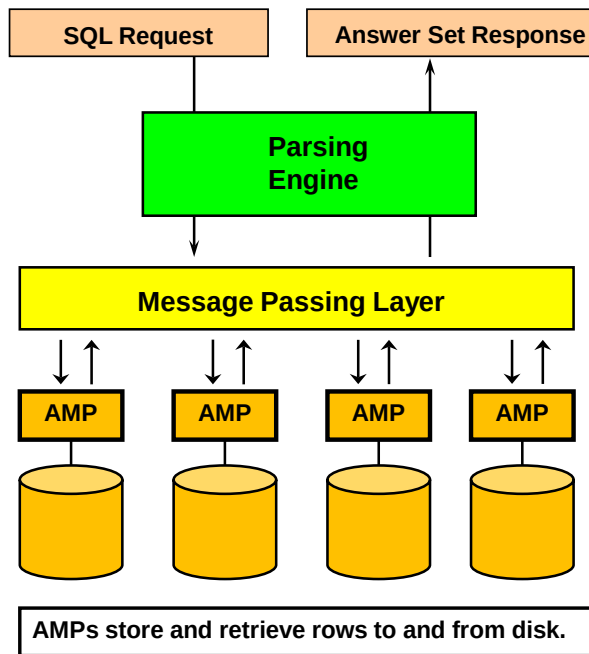
An AMP responds to Parser/Optimizer steps transmitted across the MPL by selecting data from or storing data to its disks. For some requests the AMPs may also redistribute a copy of the data to other AMPs.

The **Database Manager** subsystem resides on each AMP. It receives the steps from the Dispatcher and processes the steps. To do that it has the ability to **lock** databases and tables, to **create, modify, or delete definitions** of tables, to **insert, delete, or modify rows** within the tables, and to **retrieve information** from definitions and tables. It collects **accounting** statistics, recording accesses by session so those users can be billed appropriately. Finally, the Database manager returns responses to the Dispatcher.

Earlier in this course we discussed the **logical organization of data** into tables. The Database Manager provides a bridge between that logical organization and the **physical organization** of the data on disks. The Database Manager performs a **space management** function that controls the use and allocation of space.

AMPs also perform **output data conversion**, checking the session and changing the internal, 8-bit ASCII used by Teradata to the format of the requester. This is the reverse of the process performed by the PE when it converts the incoming data into internal ASCII.

The Access Module Processor (AMP)



The AMPs are responsible for:

- Accesses storage using Teradata's File System Software
- Lock management
- Sorting rows
- Aggregating columns
- Join processing
- Output conversion and formatting
- Creating answer set for client
- Disk space management
- Accounting
- Special utility protocols
- Recovery processing

Teradata File System Software:

- Translates DatabaseID/TableID/RowID into location on storage
- Controls a portion of physical storage
- Allocates storage space by "Cylinders"

Teradata Parallelism

Parallelism is at the very heart of the Teradata database. There is virtually no part of the system where parallelism has not been built in. Without the parallelism of the system, managing enormous amounts of data would either not be possible or, best case, would be prohibitively expensive and inefficient.

Each PE can support up to 120 user sessions in parallel. This could be 120 distinct users, or a single user harnessing the power of all 120 sessions for a single application.

Each session may handle multiple requests concurrently. While only one request at a time may be active on behalf of a session, the session itself can manage the activities of 16 requests and their associated answer sets.

The Message Passing Layer was designed such that it can never be a bottleneck for the system. Because the MPL is implemented differently for different platforms, this means that it will always be well within the needed bandwidth for each particular platform's maximum throughput.

Each AMP can perform up to 80 tasks in parallel. This means that AMPs are not dedicated at any moment in time to the servicing of only one request, but rather are multi-threading multiple requests concurrently. The value 80 represents the number of AMP Worker Tasks and may be changed on some systems.

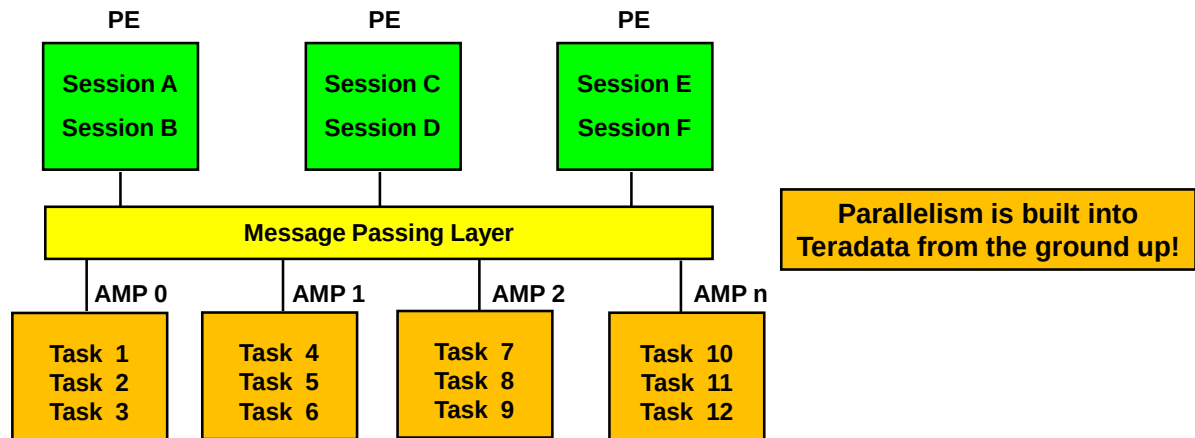
Because AMPs are designed to operate on only one portion of the database, they must operate in parallel to accomplish their intended results.

In addition to this, the optimizer may direct the AMPs to perform certain steps in parallel if there are no contingencies between the steps. This means that an AMP might be concurrently performing more than one step on behalf of the same request.

A recently added feature called Parallel CLI allows for parallelizing the client application, particularly useful for multi-session applications. This is accomplished by setting a few environmental variables and requires no changes to the application code.

In truth, parallelism is built into the Teradata database from the ground up!

Teradata Parallelism



Notes:

- Each PE can handle up to 120 sessions in parallel.
- Each Session can handle multiple REQUESTS.
- The Message Passing Layer can handle all message activity in parallel.
- Each AMP can perform up to 80 tasks in parallel (can be configured for more).
- All AMPs can work together in parallel to service any request.
- Each AMP can work on several requests in parallel.

Linear Growth and Expandability

The Teradata DBS is the first commercial database system to offer true parallelism and the performance increase that goes with it.

Think back to the example of how rows are divided up among AMPs that we just discussed. Assume that our three tables, EMPLOYEE, DEPARTMENT, and JOB total 100,000 rows, with a certain number of users, say 50.

What happens if you **double the number of AMPs** and the number of users stays the same? Performance **doubles**. Each AMP can only work on half as many rows as they used to.

Now think of that system in a situation where the number of **users** is **doubled**, as well as the number of AMPs. We now have 100 users, but we also have twice as many AMPs. What happens to performance? **It stays the same**. There is no drop-off in the speed with which requests are executed.

That's because the system is **modular** and the workload is easily partitioned into independent pieces. In the last example, each AMP is still doing the same amount of work.

This feature – that the amount of time (or money) required to do a task is directly proportional to the size of the system – is unique to the Teradata database. Traditional databases show a sharp drop in performance when the system approaches a critical size.

Look at the diagram on the facing page. As the number of **Parsing Engines** increases, the number of SQL requests that can be supported increases.

As you add **AMPs**, the datum is spread out more even as you add processing power to handle the data.

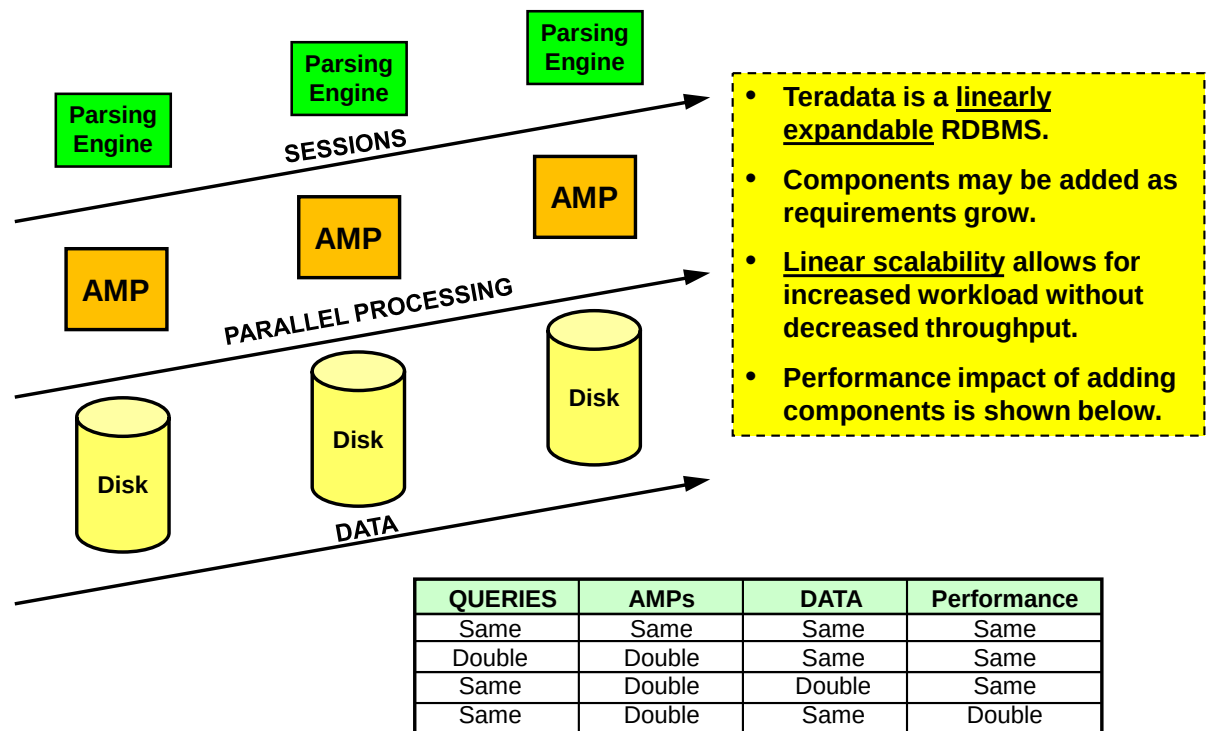
As you add **disks**, you add space for each AMP to store and process more information. All AMPs must have the same amount of disk storage space.

There are numerous advantages to having a system that has linear scalability. Two advantages include:

- Linear scalability allows for increased workload without decreased throughput.
- Investment protection for application development

In reality, the typical way in which Teradata systems are grown is to add additional cliques of nodes and a node is effectively a collection of PEs and/or AMPs. The concept of cliques and nodes is covered in detail later in this course.

Linear Growth and Expandability



Module 3: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 3: Review Questions

1. What are the two software elements that allow a client application to connect with a PE?
2. What is the purpose of the PE?
3. What is the purpose of the AMP?
4. How many sessions can a PE support?

Match Quiz

- | | |
|------------------------------|---|
| ___ 1. CLI | a. Does Aggregating and Locking |
| ___ 2. MTDP | b. Validates SQL syntax |
| ___ 3. MOSI | c. Connects AMPs and PEs |
| ___ 4. Parser | d. Balances sessions across PEs |
| ___ 5. AMP | e. Provides Client side OS independence |
| ___ 6. Message Passing Layer | f. Library of Session Management Routines |
| ___ 7. TDP | g. PE S/W that chooses the execution plan |
| ___ 8. Optimizer | h. PE S/W sends plan steps to AMP(s) |
| ___ 9. Dispatcher | i. Library of Teradata Service Routines |
| ___ 10. Parallelism | j. Foundation of Teradata architecture |

Notes



Teradata Databases and Users

After completing this module, you will be able to:

- Distinguish between a Teradata database and Teradata user.
- Define Perm Space and explain how it is used.
- Define Spool Space and its use.
- Visualize the hierarchy of objects in Teradata.

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata System Hierarchy	4-4
What is a Database?	4-6
Tables	4-6
Views	4-6
Macros.....	4-6
Triggers	4-6
What is a User?	4-8
Database – User Comparison	4-10
Teradata’s Hierarchy and Perm Space	4-12
Example of a System Hierarchy.....	4-14
Permanent Space	4-16
Global Space Accounting.....	4-16
Spool Space.....	4-18
Temporary Space	4-20
Creating Tables	4-22
Data Types	4-24
Access Rights and Privileges	4-26
Module 4: Review Questions	4-28

Teradata System Hierarchy

Teradata's System Hierarchy is a collection of databases and users. The user DBC is at the top of the hierarchy.

A database or user is a uniquely named permanent space that can store objects like tables, indexes, procedures, triggers, functions and other databases and users. Views, macros, roles, and profiles do not consume space; their definitions are stored in the Data Dictionary, which takes up space in DBC.

A database is a logical repository for database objects, privilege definitions, and space limits.

In Teradata, a user is the same as a database except it is an active repository that can log on to Teradata with a password and establish a session.

Therefore, both databases and users may own objects such as tables, views, macros, procedures, and functions. Both users and databases may hold privileges (access rights).

An "owner" or "parent" is any database/user above you in the hierarchy. (Note that you can use the terms owner and parent interchangeably.) A "child" is any object below you in the hierarchy. An owner or parent can have many children.

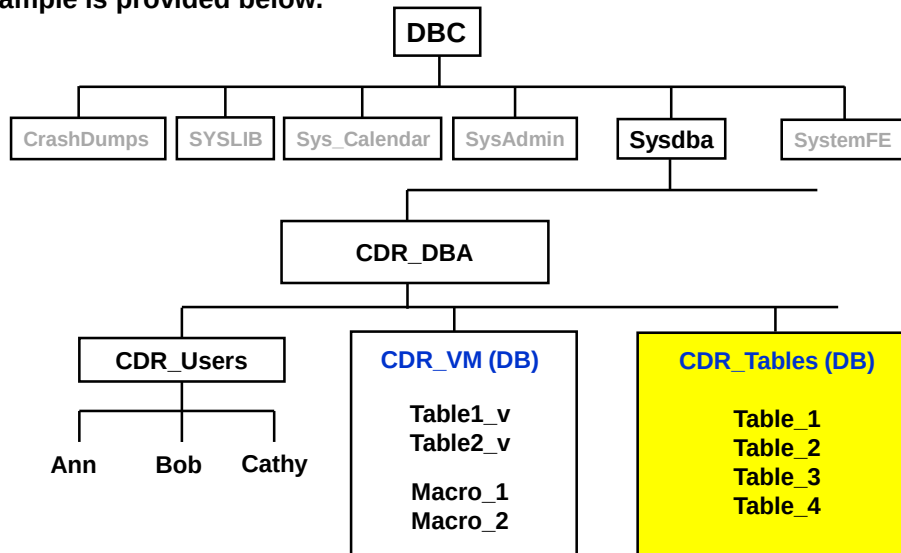
The term "immediate parent" is sometimes used to describe a database or user just above you in the hierarchy.

Teradata System Hierarchy

A Teradata NewSQL Engine system consists of databases and users organized in a hierarchy.

- Database – a logical repository for a collection of objects (e.g., tables, views, ..)
- User – used to login to Teradata and can also be used as a repository of objects

A simple example is provided below.



What is a Database?

A database is a collection of tables, views, macros, triggers, stored procedures, join indexes, hash indexes, UDFs, access rights and space limits used for administration and security. All databases have a defined upper limit of permanent space. Permanent space is used for storing the data rows of tables. Perm space is not pre-allocated. It represents a maximum limit. All databases also have an upper limit of spool space. Spool space is temporary space used to hold intermediate query results or formatted answer sets to queries.

Databases provide a logical grouping for information. They are also the foundation for space allocation and access control. We'll review the definitions of tables, views, and macros.

Tables

A table is the logical structure of data in a database. It is a **two-dimensional structure made up of columns and rows**. A user defines a table by giving it a table name that refers to the type of data that will be stored in the table.

A **column** represents attributes of the table. Attributes identify, describe, or qualify the table. Column names are given to each column of the table. All the information in a column is the same type, for example, data of birth.

Each occurrence of an entity is stored in the table as a **row**. Entities are the people, things, or events that the table is about. Thus a row would represent a particular person, thing, or event.

Views

A view is a **pre-defined subset of one of more tables or other views**. It does not exist as a real table, but serves as a reference to existing tables or views. One way to think of a view is as a virtual table. Views have definitions in the data dictionary, but do not contain any physical rows. Views can be used by the database administrator to control access to the underlying tables. Views can be used to hide columns from users, to insulate applications from database changes, and to simplify or standardize access techniques.

Macros

A macro is a **definition containing one or more SQL commands and report formatting commands** that is stored in the Data Dictionary/Directory. Macros are used to simplify the execution of frequently-used SQL commands.

Triggers

A trigger consists of **one or more SQL statements that are associated with a table** and are executed when the trigger is "fired".

What is a Database?

A database is a defined logical repository for:

- Tables
- Views
- Macros
- Triggers
- Stored Procedures
- Join Indexes
- Hash Indexes
- Permanent Journals
- User-defined Functions (UDF)

Attributes that may be specified for a database:

- Perm Space – max amount of space available for tables, join indexes, hash indexes, stored procedures, and UDFs
- Spool Space – max amount of work space available for requests
- Temporary Space – max amount of (global) temporary table space

A database is created with the CREATE DATABASE command.

Example

```
CREATE DATABASE App1_DB FROM Sysdba
AS PERMANENT = 200E9;
```

Notes:

"App1_DB" is owned by "Sysdba".

A database is empty until objects are created within it.

What is a User?

A **user** can also be thought of as a collection of tables, views, macros, triggers, stored procedures, join indexes, hash indexes, UDFs, **and** access rights.

A user is almost the same as a database except that a user can actually log on to the DBS. To accomplish this, a user must have a password. A user may or may not have perm space.

Even with no perm space, a user can access other databases depending on the privileges the user has been granted.

Users are created with the SQL statement **CREATE USER**.

What is a User?

A user is a database with an assigned password.

A user may logon to Teradata and access objects within:

- itself
- other databases for which it has access rights

Examples of attributes that may be specified for a user:

- Perm Space – max amount of space available for tables, join indexes, hash indexes, stored procedures, and UDFs
- Spool Space – max amount of work space available for requests
- Temporary Space – max amount of (global) temporary table space

A user is an active repository while a database is a passive repository.

A user is created with the CREATE USER command.

Example

```
CREATE USER User_C FROM User_A
AS PERMANENT = 10E9
, SPOOL = 500E9
, TEMPORARY = 150E9
, PASSWORD = lucky_day ;
```

Notes:

"User_C" is owned by "User_A".

A user is empty until objects are created within it.

Database – User Comparison

In Teradata, a database and a user are essentially the same. Database/User/Role names must be unique within the entire system and represent the highest level of qualification in an SQL statement. Roles have not been discussed to this point, but a Role is a collection of access rights/privileges and can be granted to a user.

A user represents a logon point within the hierarchy. In many systems, end users do not have Perm space given to them. They are granted rights to access database(s) containing views and macros, which in turn are granted rights to access the corporate production tables.

At any time, another authorized user can change the Spool (workspace) limit assigned to a user.

Databases may be empty. They may or may not have any tables, views, macros, triggers, or stored procedures. They may or may not have Perm Space allocated. The same is true for **Users**. The only absolute requirement is that a user must have a password.

Once Perm Space is assigned, then and only then can tables be put into the database. Views, macros, and triggers may be added at any time, with or without Perm Space.

Remember that databases and users are both repositories for database objects. The main difference is the user ability to logon and acquire a session with Teradata.

A row exists in DBC.Dbbase for each user and database.

Database – User Comparison

User

Unique Name

Password = Value

Define and use Perm space

Define **and use** Spool space

Define **and use** Temporary space

Set Permanent Journal defaults

Multiple Account strings

Logon and establish a session with a priority

May have a startup string

Default database, dateform, timezone,
and default character set

Collation Sequence

Database

Unique Name

Define and use Perm space

Define Spool space

Define Temporary space

Set Permanent Journal defaults

One Account string

- You can only LOGON as a known User to establish a session with Teradata.
- Tables, Join/Hash Indexes, Stored Procedures, and UDFs require Perm Space.
- Views, Macros, and Triggers are definitions in the DD/D and require **NO** Perm Space.
- A database (or user) with zero Perm Space may have views, macros, and triggers, but cannot have tables, join/hash indexes, stored procedures, or user-defined functions.

Teradata's Hierarchy and Perm Space

On a new system, DBC owns the majority of Permanent Space (Maximum Perm) and the initial dictionary tables actually use very little permanent space. The majority of the physical permanent space is available or free space.

As new users/databases are created, the quantity of space specified is initially allocated from the current PERM space of user *DBC*. For example, *DBC* becomes the owner of the *SysDBA* user and of all users and databases subsequently created. Space is allocated to *SysDBA* and children as needed.

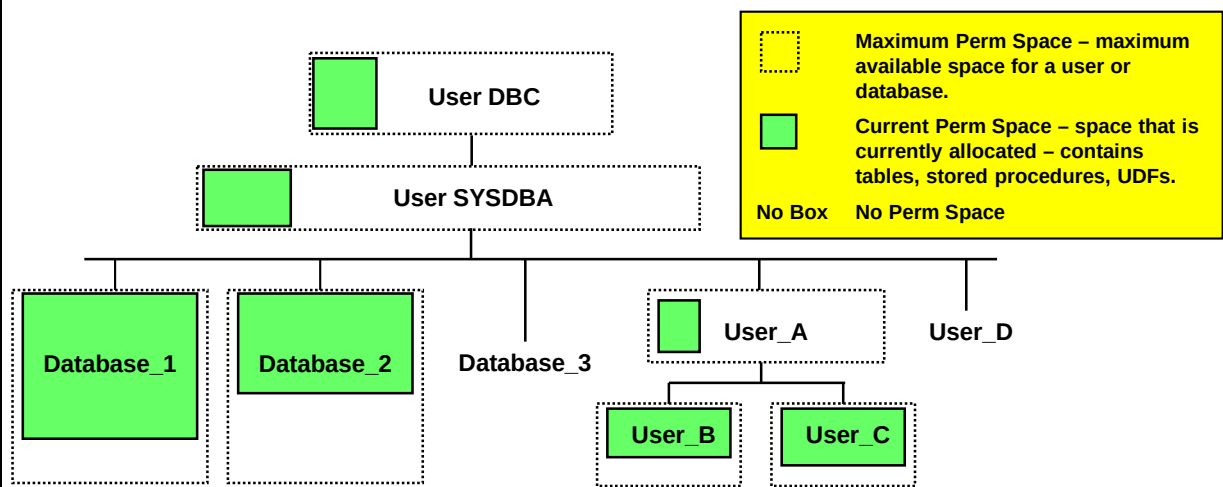
As the database administrator, you create system databases and tables and also assign user privileges and access rights to tables.

To perform the above tasks, you must:

- Determine database information content and create macros to ensure the referential integrity of the database.
- Define authorization checks and validation procedures.
- Perform audit checks on the database for LOGON, GRANT, REVOKE and other privilege statements.

You can give the authority to use the CREATE DATABASE or CREATE USER statements to any application user. He or she may then create other system users or databases from his or her own space, or if specifically authorized, from the space of another system database or user.

Teradata's Hierarchy and Perm Space



- A new database or user must be created as a child from an existing database or user.
- All Perm space specifications are subtracted from the immediate owner or parent.
- **Perm space is a zero sum game – the total of all Perm Space for all databases and users equals the total amount of disk space available to Teradata.**
- Perm space is only used for tables, join/hash indexes, stored procedures, and UDFs.
- Perm space currently unused is available to be used as Spool or Temp space.

Example of a System Hierarchy

An example of a system structure for Teradata is shown on the facing page.

As the database administrator, you create system databases and tables and also assign user privileges and access rights to tables.

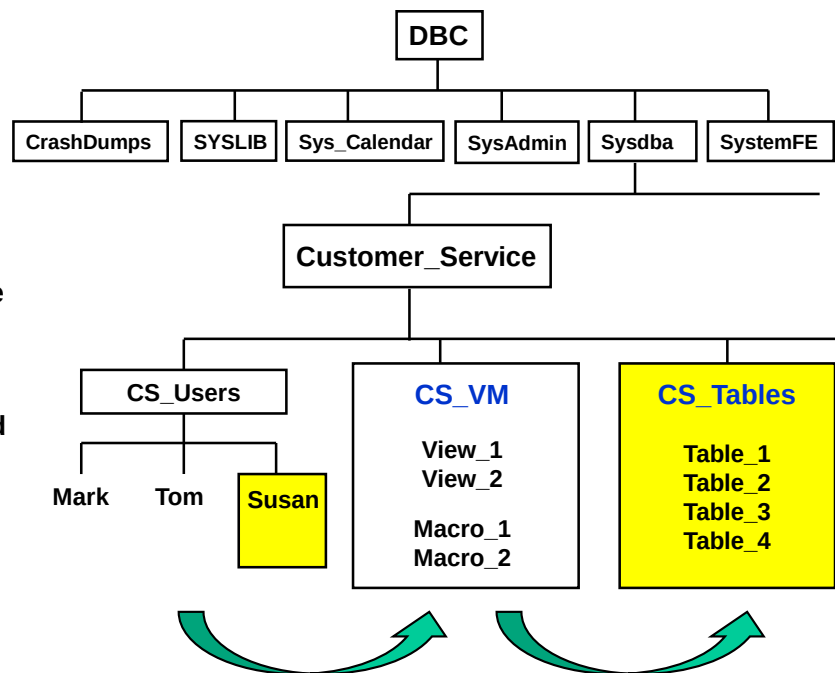
To perform the above tasks, you must:

- Determine database information content and create macros to ensure the referential integrity of the database.
- Define authorization checks and validation procedures.
- Perform audit checks on the database for LOGON, GRANT, REVOKE and other privilege statements.

Example of a System Hierarchy

A User and/or a Database may be given PERM space.

In this example, Mark and Tom have no PERM space, but Susan does.



Users may use views and macros to access the actual tables.

Permanent Space

Permanent Space (Perm space) is the maximum amount of storage assigned to a user or database for holding table rows, Fallback tables, secondary index subtables, stored procedures, UDFs, and permanent journals.

Perm space is specified in the CREATE statement as illustrated below. Perm space is not pre-allocated which means that it is available on demand, as entities are created not reserved ahead of time. Perm space is deducted from the owner's specified Perm space and is divided equally among the AMPs. Perm space can be dynamically modified.

The total amount of Perm space assigned divided by the number of AMPs equals the per-AMP limit. Whenever the per AMP limit is exceeded on any AMP, a **Database Full** message is generated.

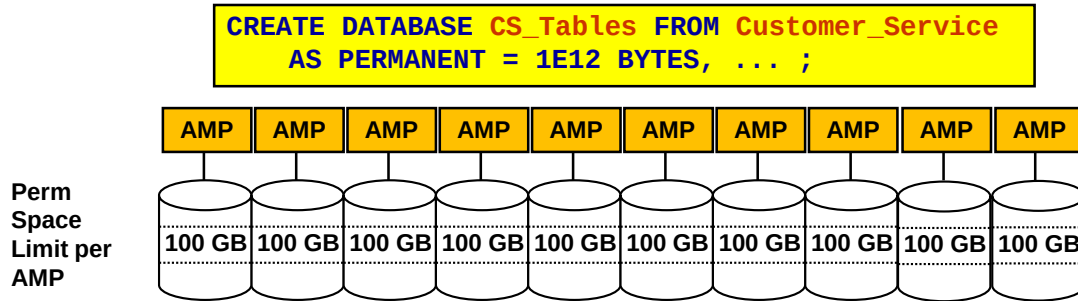
```
CREATE DATABASE CS_Tables FROM Customer_Service AS  
PERMANENT = 1000000000000 BYTES ... ;
```

Global Space Accounting

The SQL CREATE USER/DATABASE is extended to specify a skew limit percentage that allows maximum AMP space usage to be above the per-AMP quota.

```
CREATE DATABASE CS_Tables FROM Customer_Service AS  
PERMANENT = 1000000000000 BYTES SKEW = 10 PERCENT ...;
```

Permanent Space



- Table rows, index subtable rows, join indexes, hash indexes, stored procedures, and UDFs use Perm space.
- Fallback protection for a table uses twice the Perm space of a No Fallback table.
- When a database/user is created, the amount of specified Perm space is deducted from the owner's database space.
- Disk space is not reserved ahead of time, but is available on demand.
- Perm space is defined globally for a database or a user.
- Perm space can be dynamically modified.
- The global limit divided by the number of AMPs is the per/AMP limit.
- You cannot exceed the per/AMP limit.
 - However, release 16.00 provides a "skew option" and can manage space limits at a global level.
- Good data distribution is crucial to space management.

Spool Space

Spool Space is work space acquired automatically by the system and used for work space and answer sets for intermediate and final results of Teradata SQL statements (e.g., SELECT statements generally use Spool space to store the SELECTed data). When the spool space is no longer needed by a query, it is released back to the system.

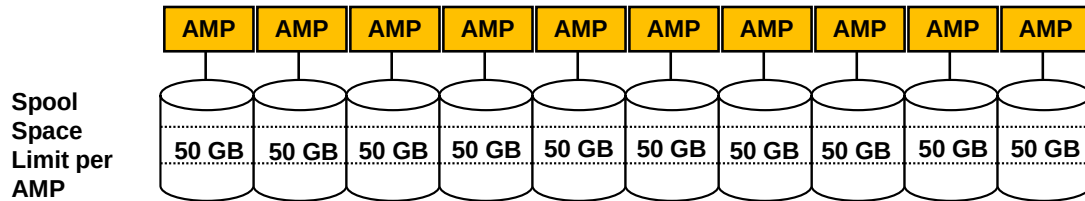
A **Spool limit** is specified in the CREATE statement shown below. This limit cannot exceed the Spool limit of the creator. However, a single user can create multiple databases or users, and each can have a Spool limit as large as the Spool limit of the original creator.

The total amount of Spool space assigned divided by the number of AMPs equals the per AMP limit. Whenever the per-AMP limit is exceeded on any AMP, an **Insufficient Spool** message is generated to that client.

```
CREATE USER Susan FROM CS_Users AS  
  PERMANENT = 10000000000 BYTES,  
  SPOOL = 500000000000 BYTES,  
  PASSWORD = secret ... ;
```

Spool Space

```
CREATE USER Susan FROM CS_Users AS PERMANENT = 10E9 BYTES,  
      SPOOL = 500E9 BYTES, PASSWORD = secret ... ;
```



- **Spool space is work space acquired automatically by the system for intermediate query results or answer sets.**
 - SELECT statements generally use Spool space.
 - Only INSERT, UPDATE, and DELETE statements affect table contents.
- The Spool limit cannot exceed the Spool limit of the original creator.
- **The Spool limit is divided by the number of AMPS in the system, giving a per-AMP limit that cannot be exceeded.**
 - "Insufficient Spool" errors often result from poorly distributed data or joins on columns with large numbers of non-unique values.
 - Keeping Spool rows small and few in number reduces Spool I/O.

Temporary Space

Temporary (Temp) Space is temporary space acquired automatically by the system when Global Temporary tables are materialized and used.

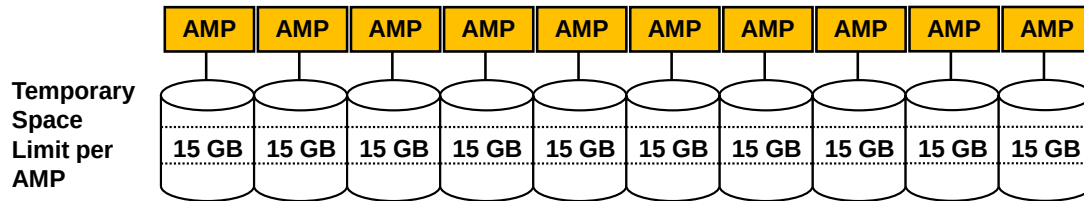
A **Temporary limit** is specified in the CREATE statement shown below. This limit cannot exceed the Temporary limit of the owner. However, a single user can create multiple databases or users, and each can have a Temporary limit as large as the Temporary limit of the original creator.

The total amount of Temporary space assigned divided by the number of AMPs equals the per AMP limit. Whenever the per-AMP limit is exceeded on any AMP, an **Insufficient Temporary** message is generated to that client.

```
CREATE USER Susan FROM CS_Users AS
  PERMANENT = 10000000000 BYTES,
  SPOOL = 500000000000 BYTES,
  TEMPORARY = 150000000000 BYTES,
  PASSWORD = secret ...
```

Temporary Space

```
CREATE USER Susan FROM CS_Users AS PERMANENT = 1E9 BYTES,
      SPOOL = 500E9 BYTES, TEMPORARY = 150E9 BYTES, PASSWORD = secret ... ;
```



- Temporary space is space acquired automatically by the system when a "Global Temporary" table is used and materialized.
- The Temporary limit cannot exceed the Temporary limit of the original creator.
- The Temporary limit is divided by the number of AMPS in the system, giving a per-AMP limit that cannot be exceeded.
 - "Insufficient Temporary" errors often result from poorly distributed data or joins on columns with large numbers of non-unique values.
- Note: Volatile Temporary tables and derived tables utilize Spool space.

Creating Tables

Creation of tables is done via the DDL portion of the SQL command vocabulary. The table definition, once accepted, is stored in the DD/D.

Prior to Teradata 13.0, creating tables required the definition of at least one column and the assignment of a Primary Index. With Teradata 13.0, it is possible to create tables without a primary index. Columns are assigned data types, attributes and optionally may be assigned constraints, such as a range constraint.

Tables, like views and macros, may be dropped when they are no longer needed. Dropping a table both deletes the data from the table and removes the definition of the table from the DD/D.

Secondary indexes may also optionally be assigned at table creation, or may be deferred until after the table has been built. Secondary indexes may also be dropped, if they are no longer needed. It is not uncommon to create secondary indexes to assist in the processing of a specific job sequence, then to delete the index, and its associated overhead, once the job is complete.

We will have more to say on indexes in general in future modules.

Creating Tables

Creating a table requires ...

- at least one column
- optional assignment of a primary index (tables can be defined with No Primary Index option)
- optional assignment of secondary indexes

```
CREATE TABLE Employee
    (EmpNumber      INTEGER NOT NULL
    ,LastName       CHAR(20) NOT NULL
    ,FirstName      VARCHAR(20)
    ,SalaryAmount   DECIMAL(10,2)
    ,DeptNumber     SMALLINT
    ,JobCode        CHAR(3))
Primary  → UNIQUE PRIMARY INDEX (EmpNumber)
Secondary → INDEX (LastName) ;
```

Database objects may be created or dropped as needed.

Secondary indexes may be

- created at table creation
- created after table creation
- dropped after table creation

CREATE
DROP



Tables
Views
Macros
Triggers
Procedures

CREATE
DROP



INDEX (secondary only)

Data Types

When a table is created, a **data type** is specified for each column. Data types are divided into three main classes – numeric, byte, and character. The facing page shows data types.

DATE is a 32-bit integer that represents the date as YYYYMMDD. It supports century and year 2000 and is implemented with calendar-based intelligence.

TIME WITH ZONE and **TIMESTAMP WITH ZONE** are ANSI standard data types that allow support of clock and time zone based intelligence.

PERIOD(DATE), PERIOD(TIME(n) [WITH TIME ZONE]), PERIOD(TIMESTAMP(n) [WITH TIME ZONE]) A Period is an anchored duration usually used for Temporal. (TD 13.0)

DECIMAL (n, m) is a number of n digits, with m digits to the right of the decimal point.

BYTEINT is an 8-bit signed binary whole number that may vary in range from -128 to +127.

SMALLINT is a 16-bit signed binary whole number that may vary in range from -32,768 to +32,767.

INTEGER is a 32-bit signed binary whole number that may vary in size from -2,147,483,648 to +2,147,483,647.

BIGINT is a 64-bit (8 bytes) signed binary whole number that may vary in size from -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 or as $(-2^{63} \text{ to } 2^{63} - 1)$.

FLOAT, REAL, and **DOUBLE PRECISION** is a 64-bit IEEE floating point number.

NUMBER(n) or NUMBER(n,m) is a variable length numeric data type used to represent both fixed point and floating point values, depending on the syntax you use to specify the value. (TD 14.0)

BYTE (n) is a fixed-length binary string of n bytes. **BYTE** and **VARBYTE** are never converted to a different internal format. They can also be used for digitized objects.

VARBYTE (n) is a variable-length binary string of n bytes.

BINARY LARGE OBJECT (n) is similar to a **VARBYTE**; however it may be as large as 2 GB. A **BLOB** may be used to store graphics, video clips and binary files.

ST_GEOMETRY, MBR Geospatial information identifies the geographic location of features and boundaries on the planet. Use geospatial types to represent 0-dimensional (point), 1-dimensional (line), and 2-dimensional (polygon) geometries. **MBR** type provides a way to obtain the minimum bounding rectangle (**MBR**) of a geometry. (TD 13.0)

XML, XMLTYPE type is used to store XML content in Teradata in a compact binary form that preserves the information set of the XML document. (TD 14.10)

CHAR (n) is a fixed-length character string of n characters. **VARCHAR (n)** is a variable-length character string of n characters. **LONG VARCHAR** is the longest variable-length character string. It is equivalent to **VARCHAR (64000)**. **GRAPHIC, VARGRAPHIC** and **LONG VARGRAPHIC** are the equivalent character types for multi-byte character sets such as Kanji.

CHARACTER LARGE OBJECT (n) is similar to a **VARCHAR**; however it may be as large as 2 GB. A **CLOB** may be used to store simple text, HTML, or XML documents.

Data Types

TYPE	Name	Bytes	Description
Date/Time	DATE	4	YYYYMMDD
	TIME (WITH ZONE)	6 / 8	HHMMSSZZ
	TIMESTAMP (WITH ZONE)	10 / 12	YYYYMMDDHHMMSSZZ
Period	PERIOD(DATE)	8	Temporal (TD 13.0)
	PERIOD(TIME(n) [WITH TIME ZONE]), PERIOD(TIMESTAMP(n) [WITH TIME ZONE])	12 – 24 bytes	
Numeric	DECIMAL or NUMERIC (n, m)	2, 4, 8, 16	+ OR –
	BYTEINT	1	-128 to +127
	SMALLINT	2	-32,768 to +32,767
	INTEGER	4	-2,147,483,648 to +2,147,483,647
	BIGINT	8	-263 to +263 - 1 (+9,223,372,036,854,775,807)
	FLOAT, REAL, DOUBLE PRECISION	8	IEEE floating point
	NUMBER (n) or NUMBER (n,m)	0 – 18	± 1E-130 to 9.99...9E125 (TD 14.0)
Byte	BYTE(n)	1 – 64,000	
	VARBYTE (n)	1 – 64,000	
	BLOB	8 – 2 GB	Binary Large Object
Geospatial	ST_GEOMETRY, MBR	8 – 16 MB	Geospatial (TD 13.0)
XML	XML, XMLTYPE	8 – 2 GB	eXtensible Markup Language (XML) (TD 14.10)
Character	CHAR (n)	1 – 64,000	
	VARCHAR (n)	1 – 64,000	
	LONG VARCHAR		same as VARCHAR(64,000)
	GRAPHIC	1 – 32,000	
	VARGRAPHIC	1 – 32,000	
	LONG VARGRAPHIC		same as VARGRAPHIC(32,000)
	CLOB	8 – 2 GB	Character Large Object

Access Rights and Privileges

The diagram on the facing page shows access rights and privileges as they might be defined for the database administrator, a programmer, a user, a system operator, and an administrative user.

The **database administrator** has right to use all of the commands in the data definition privileges, the data manipulation privileges, and the data control privileges.

The **programmer** has all of those except the ability to GRANT privileges to others.

A typical **user** is limited to data manipulation privileges, while the **operator** is limited to data control privileges.

Finally, the **administrative user** is limited to a subset of data manipulation privileges, SELECT and EXECUTE.

Each site should carefully consider the access rules that best meet their needs.

Access Rights and Privileges

Data Definition Privileges

<u>Command</u>	<u>Object</u>
CREATE	Database and/or User
DROP	Table and/or View
	Macro and/or Trigger
	Stored Procedure
	Role and/or Profile

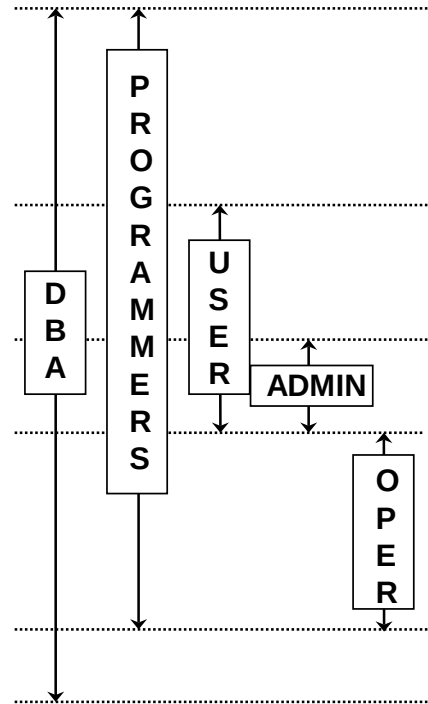
Data Manipulation Privileges

SELECT	
INSERT	Table
UPDATE	View
DELETE	
EXECUTE	Macro and/or Stored Procedure

Data Control Privileges

DUMP	Database
RESTORE	Table
CHECKPOINT	Journal
GRANT	Privileges on Databases
REVOKE	Users
	Objects

A Sample Scenario



Module 4: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 4: Review Questions

True or False

- _____ 1. A database will always have tables.
 - _____ 2. A user will always have a password.
 - _____ 3. A user creating a subordinate user must give up some of his/her Perm Space.
 - _____ 4. Creating tables requires the definition of at least 1 column and a Primary Index.
 - _____ 5. The sum of all user and database max Perm Space will equal the total space on the system.
 - _____ 6. The sum of all user and database Spool Space will equal the total space on the system.
 - _____ 7. The limit of space for both Volatile and Global Temporary tables is based on the maximum TEMPORARY space for the user.
 - _____ 8. Deleting a macro from a database reclaims Perm Space for the database.
9. Which two statements are TRUE about PERM space in Teradata 16.20? (choose two) _____
- a. PERM space cannot be dynamically modified.
 - b. The per/AMP limit of PERM space can be exceeded.
 - c. Tables, index subtables, and stored procedures use PERM space.
 - d. Maximum PERM space can be defined at the database or table level.
10. Which statement is TRUE about SPOOL space in Teradata 16.20? _____
- a. SPOOL space cannot be dynamically modified.
 - b. Maximum SPOOL space can be defined at the database or user level.
 - c. The SPOOL limit is dependent on the database limit where the table is located.
 - d. Maximum SPOOL space can be defined at a value greater than the creator's value.

Notes



PI Access and Mechanics

After completing this module, you will be able to:

- Explain the purpose of the Primary Index
- Distinguish between Primary Index and Primary Key
- Explain the role of the hashing algorithm and the hash map in locating a row.
- Explain the makeup of the Row ID and its role in row storage.
- Describe the sequence of events for locating a row given its PI value.

Teradata Proprietary and Confidential

Notes

Table of Contents

Primary Keys and Primary Indexes.....	5-4
Distribution of Rows	5-6
Specifying a Primary Index.....	5-8
Primary Index Values.....	5-10
Accessing Via a Unique Primary Index	5-12
Accessing Via a Non-Unique Primary Index.....	5-14
Row Distribution Using a Unique Primary Index (UPI) – Case 1	5-16
Row Distribution Using a Non-Unique Primary Index (NUPI) – Case 2.....	5-18
Row Distribution Using a Highly Non-Unique Primary Index (NUPI) – Case 3.....	5-20
Which AMP has the Row?	5-22
Hashing Down to the AMPs	5-24
A Hashing Example	5-26
The Hash Map	5-28
Hash Maps for Different Systems	5-30
Identifying Rows.....	5-32
The Row ID.....	5-34
Storing Rows (1 of 2).....	5-36
Storing Rows (2 of 2).....	5-38
Locating a Row on an AMP Using a PI.....	5-40
Module 5: Review Questions	5-42

Primary Keys and Primary Indexes

While it is true that many tables use the same columns for both Primary Indexes and Primary Keys, **Indexes are conceptually different from Keys**. The table on the facing page summarizes those differences.

A Primary Key is relational data modeling term that defines, in the logical model, the columns that uniquely identify a row. A Primary Index is a physical database implementation term that defines the actual columns used to distribute and access rows in a table.

It is also true that a significant percentage of the tables in any database will use the same column(s) for both the PI and the PK. However, one should expect that in any real-world scenario there would be some tables that will not conform to this simplistic rule. Only through a careful analysis of the type of processing that will take place can the tables be properly evaluated for PI candidates. Remember, changing your mind about the columns that comprise the PI means recreating (and reloading) the table.

Primary Keys and Primary Indexes

- Indexes are conceptually different from keys.
- A PK is a relational modeling convention which allows each row to be uniquely identified.
- A PI is a Teradata convention which determines how the row will be stored and accessed.
- A significant percentage of tables may use the same columns for both the PK and the PI.
- A well-designed database will use a PI that is different from the PK for some tables.

Primary Key	Primary Index
Logical concept of data modeling	Physical mechanism for access and storage
Teradata doesn't need to recognize	Each table can have (at most) one primary index
No limit on number of columns	64 column limit
Documented in data model (Optional in CREATE TABLE)	Defined in CREATE TABLE statement
Must be unique	May be unique or non-unique
Identifies each row	Identifies 1 (UPI) or multiple rows (NUPI)
Values should not change	Values may be changed (Delete + Insert)
May not be NULL – requires a value	May be NULL
Does not imply an access path	Defines most efficient access path
Chosen for logical correctness	Chosen for physical performance

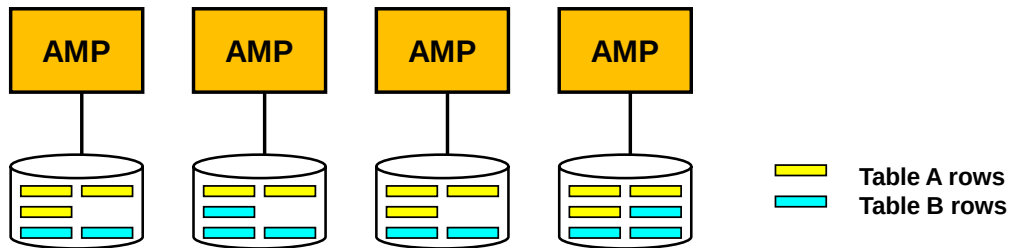
Distribution of Rows

Ideally, the rows of every table will be distributed among all of the AMPs. There may be some circumstances where this is not true. What if there are fewer rows than AMPs? Clearly in this case, at least some AMPs will hold no rows from that table. This should be considered the exceptional situation, and not the rule. Each AMP is designed to hold a portion of the rows of each table. The AMP is responsible for the storage, maintenance and retrieval of the data under its control.

More ideally, the rows of each table will be evenly distributed across all of the AMPs. This is desirable because in operations involving all rows of the table (such as a full table scan); each AMP will have an equal portion of the work to do. When workloads are not evenly distributed, the desired response will only be as fast as the slowest AMP.

Controlling the distribution of the rows of a table is done by the selection of the Primary Index. The relative uniqueness of the Primary Index will determine the uniformity of distribution of the rows of this table among the AMPs.

Distribution of Rows



- The rows of every table are distributed among all AMPs
- Each AMP is responsible for a subset of the rows of each table.
 - Ideally, each table will be evenly distributed among all AMPs.
 - Evenly distributed tables result in evenly distributed workloads.
- For tables with a Primary Index (majority of the tables), the uniformity of distribution of the rows of a table depends on the choice of the Primary Index. The actual distribution is determined by the hash value of the Primary Index.
- For tables without a Primary Index (NoPI tables), the rows of a table are still distributed between the AMPs based on random generator code within the PE or AMP.
 - A small number of tables will typically be created as NoPI tables. Common uses for NoPI tables are as staging/intermediate tables used in load operations or as column partitioned tables.

Specifying a Primary Index

Choosing a **Primary Index** for a table is perhaps the most critical decision a database designer makes. The choice will affect the distribution of the rows of the table and, consequently, the performance of the table in a production environment. Although many tables used combined columns as the Primary Index choice, the examples used here are single column indexes, mostly for the sake of simplicity.

Unique Primary Indexes (UPI's) are desirable because they guarantee the uniform distribution of the rows of that table.

Because it is not always feasible to pick a Unique Primary Index, it is sometimes necessary to pick a column (or columns) which have non-unique values; that is there are duplicate values. This type of index is called a **Non-Unique Primary Index** or **NUPI**. While not a guarantor of uniform row distribution, the degree of uniqueness of the index will determine the degree of uniformity of the distribution. Because all rows with the same PI value end up on the same AMP, columns with a small number of distinct values which are repeated frequently typically do not make good PI candidates.

The choosing of a Primary Index is not an exact science. It requires analysis and thoughtfulness for some tables and will be completely self-evident on other tables.

The Primary Index is always designated as part of the **CREATE TABLE** statement. Once a Primary Index choice has been designated for a table, it cannot be changed to something else. If an alternate choice of column(s) is desired for the PI, it is necessary to drop and recreate the table.

Teradata, adhering to the ANSI standard, permits duplicate rows by specifying that you wish to create a **MULTISET** table. In Teradata transaction mode, the default, however, is a **SET** table that does not permit duplicate rows.

Also, if **MULTISET** is enabled, it will be overridden by choosing a UPI as the Primary Index or by having a unique index (e.g., unique secondary) on another column(s) on the table. Doing this effectively disables the **MULTISET**.

Multiset tables will be covered in more detail later in the course.

Starting with Teradata 13.0, the option of **NO PRIMARY INDEX** is also available.

Specifying a Primary Index

- A Primary Index is defined at table creation.
- It may consist of a single column, or a combination of columns (up to 64 columns)
 - Creating a table without a primary index (e.g., NO PRIMARY INDEX) is an option.

UPI

```
CREATE TABLE sample_1
  (col_a  INTEGER
   ,col_b  CHAR(10)
   ,col_c  DATE)
  UNIQUE PRIMARY INDEX (col_b);
```

If the index choice of column(s) is unique, then this is referred to as a UPI (Unique Primary Index).

A UPI choice will result in even distribution of the rows of the table across all AMPs.

NUPI

```
CREATE TABLE sample_2
  (col_m  INTEGER
   ,col_n  CHAR(10)
   ,col_o  DATE)
  PRIMARY INDEX (col_m);
```

If the index choice of column(s) isn't unique, then this is referred to as a NUPI (Non-Unique Primary Index).

The distribution of the rows of the table is proportional to the degree of uniqueness of the index.

NoPI

```
CREATE TABLE sample_3
  (col_x  INTEGER
   ,col_y  CHAR(10)
   ,col_z  DATE)
  NO PRIMARY INDEX;
```

A NoPI choice will result in distribution of the data between AMPs based on random generator code.

Note: Changing the choice of Primary Index requires dropping and recreating the table.

Primary Index Values

Indexes are used to access rows from a table without having to search the entire table.

On Teradata, the **Primary Index** is the mechanism for assigning a data row to an AMP and a location on the AMP's disks. Prior to Teradata 13.0, when a table is created, a table must have a Primary Index specified (either user-assigned or Teradata assigned). This cannot be changed without dropping and creating the table.

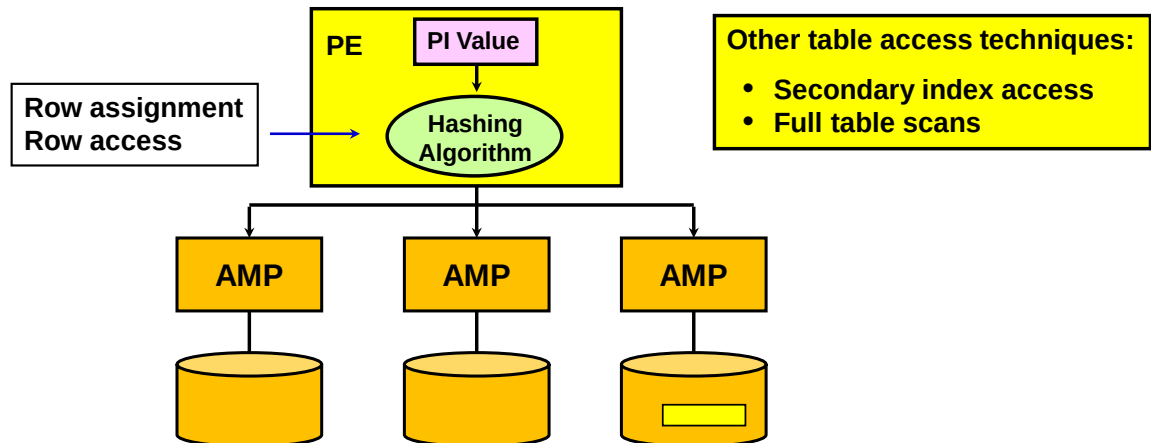
Primary Indexes are very important because they have a powerful effect on the performance of the database. The most important thing to remember is that a Primary Index is the mechanism used to assign each row to an AMP and may be used to retrieve that row from the AMP. Thus retrievals, updates and deletes that specify the Primary Index value will be much faster than those queries that do not specify the PI value. Primary Index selection is probably the most important factor in the efficiency of join processing.

Earlier we learned that the Primary Key was always unique and unchanging. This is based on the **logical model** of the data. The Primary Index may (and frequently is) be different than the Primary Key and may be non-unique; it is chosen for **the physical performance** of the database.

There are three types of primary index selection – **unique (UPI)**, **non-unique (NUPI)**, or **NO PRIMARY INDEX**.

Primary Index Values

- The value of the Primary Index for a specific row determines the AMP assignment for that row.
- This is done using a hashing algorithm.



- Accessing the row by its Primary Index value is:
 - always a one-AMP operation
 - the most efficient way to access a row

Accessing Via a Unique Primary Index

A **Primary Index operation** is always a one-AMP operation. In the case of a UPI, the one-AMP access can return, at most, one row. In the facing example, we are looking for the row whose primary index value is 345. By specifying the PI value as part of our selection criteria, we are guaranteed that only the AMP containing the specified row will need to be searched.

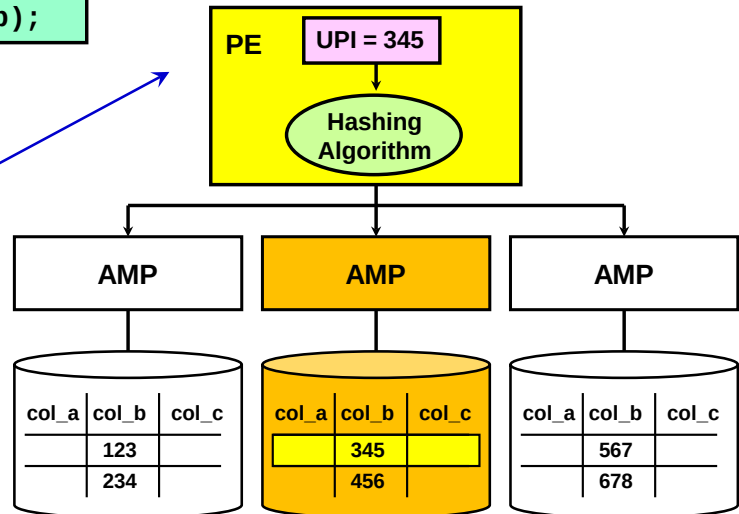
The correct AMP is located by taking the PI value and passing it through a hashing algorithm. The hashing takes place in the Parsing Engine. The output of the hashing algorithm contains information that will point the request to a specific AMP. Once it has isolated the appropriate AMP, finding the row is quick and efficient. How this happens we will see in a future module.

Accessing Via a Unique Primary Index

A UPI access is a one-AMP operation which may access at most a single row.

```
CREATE TABLE sample_1
  (col_a INTEGER
   ,col_b INTEGER
   ,col_c CHAR(4))
UNIQUE PRIMARY INDEX (col_b);
```

```
SELECT  col_a
        ,col_b
        ,col_c
FROM    sample_1
WHERE   col_b = 345;
```



Accessing Via a Non-Unique Primary Index

A Non-Unique **Primary Index operation** is also a one-AMP operation. In the case of a NUPI, the one-AMP access can return zero to many rows. On the facing page, we are looking for the rows whose primary index value is 25. By specifying the PI value as part of our selection criteria, we are once again guaranteeing that only the AMP containing the required rows will need to be searched.

As before, the correct AMP is located by taking the PI value and passing it through a hashing algorithm executing in the Parsing Engine. The output of the hashing algorithm will once again point to a specific AMP. Once it has isolated the appropriate AMP, it must now find all rows that have the specified value. In this example, the AMP returns two rows.

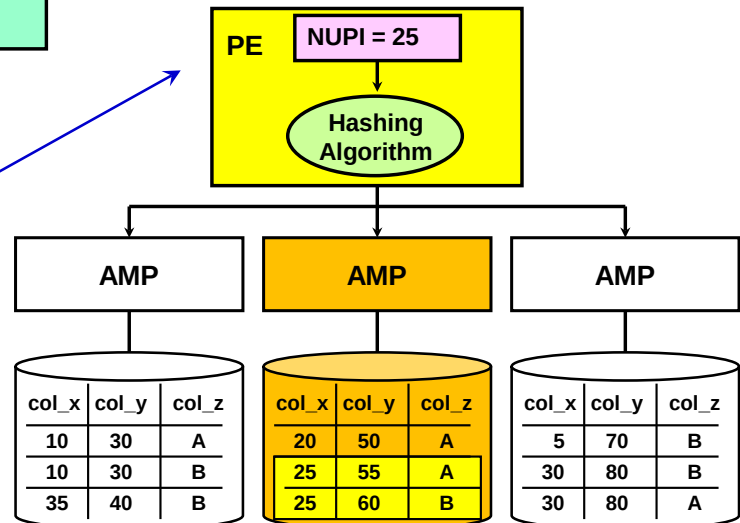
Accessing Via a Non-Unique Primary Index

A NUPI access is a one-AMP operation which may access multiple rows.

```
CREATE TABLE sample_2
  (col_x INTEGER
   ,col_y INTEGER
   ,col_z CHAR(4))
PRIMARY INDEX (col_x);
```

```
SELECT  col_x
        ,col_y
        ,col_z
FROM    sample_2
WHERE   col_x = 25;
```

Both UPI and NUPI accesses are one AMP operations.



Row Distribution Using a Unique Primary Index (UPI) – Case 1

At the heart of the Teradata NewSQL Engine is a way of predictably distributing and retrieving rows across AMPs. The same value stored in the same data type will always produce the same hash value. If the Primary Index is unique, Teradata can distribute the rows evenly. If the Primary Index is slightly non-unique, that is, there are only four or five rows per index value; the table will still distribute evenly. But if there are hundreds or thousands of rows for some index values the distribution will probably be lumpy.

In this example, the Order_Number is used as a unique primary index. Since the primary index value for Order_Number is unique, the distribution of rows among AMPs is very uniform. This assures maximum efficiency because each AMP is doing approximately the same amount of work. No AMPs sit idle waiting for another AMP to finish a task.

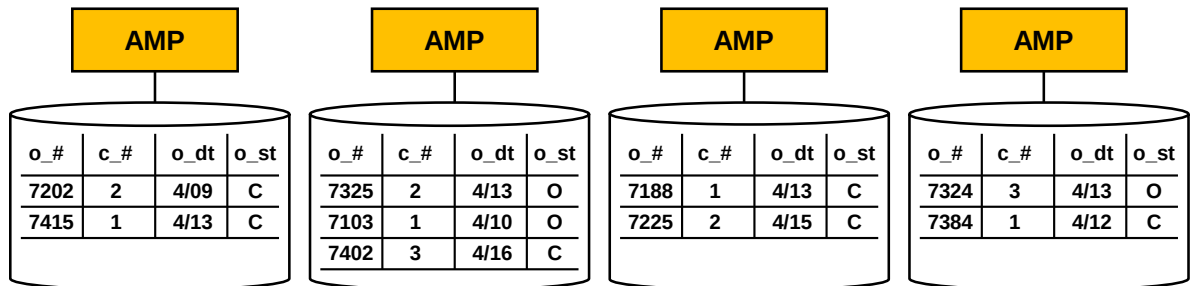
This way of storing the data provides for maximum efficiency and makes the best use of the parallel features of the Teradata system.

Row Distribution Using a UPI – Case 1

Order Number	Customer Number	Order Date	Order Status
PK			
UPI			
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C

Notes:

- Often, but not always, the PK column(s) will be used as a UPI.
 - Order_Number can be a UPI since all the values are unique.
- Teradata will distribute different UPI values evenly across all AMPs.
 - Resulting row distribution among AMPs is very uniform.
 - Assures maximum efficiency for parallel operations.



Row Distribution Using a Non-Unique Primary Index (NUPI) – Case 2

In the example on the facing page **Customer_Number** has been used as a non-unique Primary Index (NUPI). Note row distribution among AMPs is uneven. All rows with the same primary index value (in other words, with the same customer number) are stored on the same AMP.

Customer_Number has three possible values, so all the rows are hashed to three AMPs, leaving the fourth AMP without rows from this table. While this distribution will work, it is not as efficient as spreading all the rows among all the AMPs.

AMP 2 has a disproportionate number of rows and AMP 3 has none. In an all-AMP operation AMP 2 will take longer than the other AMPs. The operation cannot complete until AMP 2 completes its tasks. The overall operation time is increased and some of the AMPs are under-utilized.

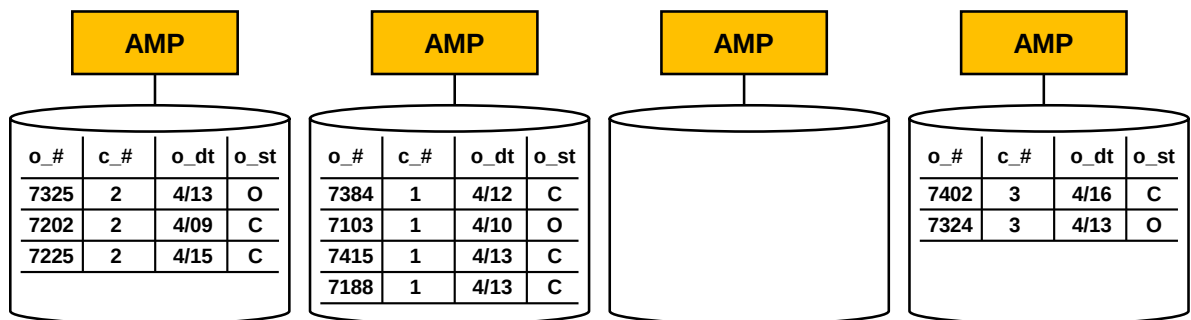
NUPI's can create irregular distributions, called "skewed distributions". AMPs that have more than an average number of rows will take longer for full table operations than the other AMPs will. Because an operation is not complete until all AMPs have finished, this will cause the operation to finish less quickly due to being underutilized.

Row Distribution Using a NUPI – Case 2

Order Number	Customer Number	Order Date	Order Status
PK			
	NUPI		
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C

Notes:

- Customer_Number may be the preferred access column for this table, thus a good index candidate.
 - Since a customer can have multiple orders, **Customer_Number will be a NUPI.**
- Rows with the same PI value distribute to the same AMP.
 - Row distribution is less uniform or skewed.



Row Distribution Using a Highly Non-Unique Primary Index (NUPI) – Case 3

This example uses **Order_Status** as a **NUPI**. Order_Status is a poor choice, because it yields the most uneven distribution. Because there are only two possible values for Order_Status, all of the rows are placed on two AMPs.

STATUS is an example of a **highly non-unique Primary Index**.

When choosing a Primary Index, you should never choose a column with such a severely limited value set. The degree of uniqueness is critical to efficiency. Choose NUPI's that allow all AMPs to participate fairly equally.

The degree of uniqueness of a NUPI is critical to efficiency.
--

Row Distribution Using a Highly Non-Unique Primary Index (NUPI) – Case 3

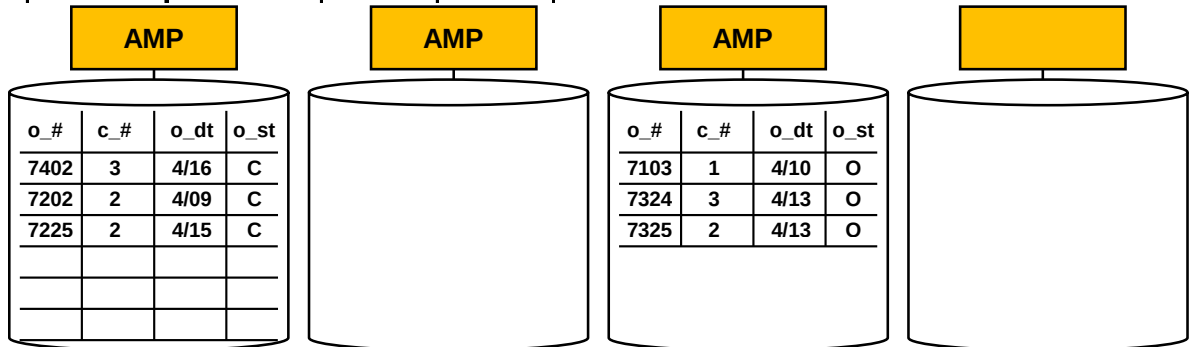
Order Number	Customer Number	Order Date	Order Status
PK			
			NUPI
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C

- Values for Order_Status are unique.

- Order_Status would be a I
- If only two values exist, th AMPs will be used for this

- Highly non-unique columns poor PI choices.

- The degree of uniqueness efficiency.



Which AMP has the Row?

This discussion (rest of this module) will assume that a table has a primary index assigned and is not using the NO PRIMARY INDEX option.

A **hashing algorithm** is a standard data processing technique that takes in a data value, like last name or order number, and systematically mixes it up so that the incoming values are converted to a number in a range from zero to the specified maximum value. A successful hashing scheme scatters the input evenly over the range of possible output values.

It is predictable in that Smith will always hash to the same value and Jones will always hash to another (and they do) different value. With a good hashing algorithm any patterns in the input data should disappear in the output data. If many names begin with “S”, they should and will not all hash to the same group of hash values. If order numbers all have “00” in the hundreds and tens place or if all names are four letters long we should still see the hash values spread fairly evenly over the whole range.

Textbooks still say that this requires manually designing and tuning a hash algorithm for each new type of data values. However, the Teradata algorithm works predictably well over any data, typically loading each AMP with variations in the range of .1% to .5% between AMPs. For extremely large systems, the variation can be as low as .001% between AMPs.

Teradata also uses hashing quite differently than other data storage systems. Other hashed data storage systems equate a bucket with a physical location on disk. In Teradata, a bucket is simply an entry in a **hash map**. Each hash map entry points to a single AMP. Therefore, changing the number of AMPs does not require any adjustment to the hashing algorithm. Teradata simply adjusts the hash maps and redistributes any affected rows.

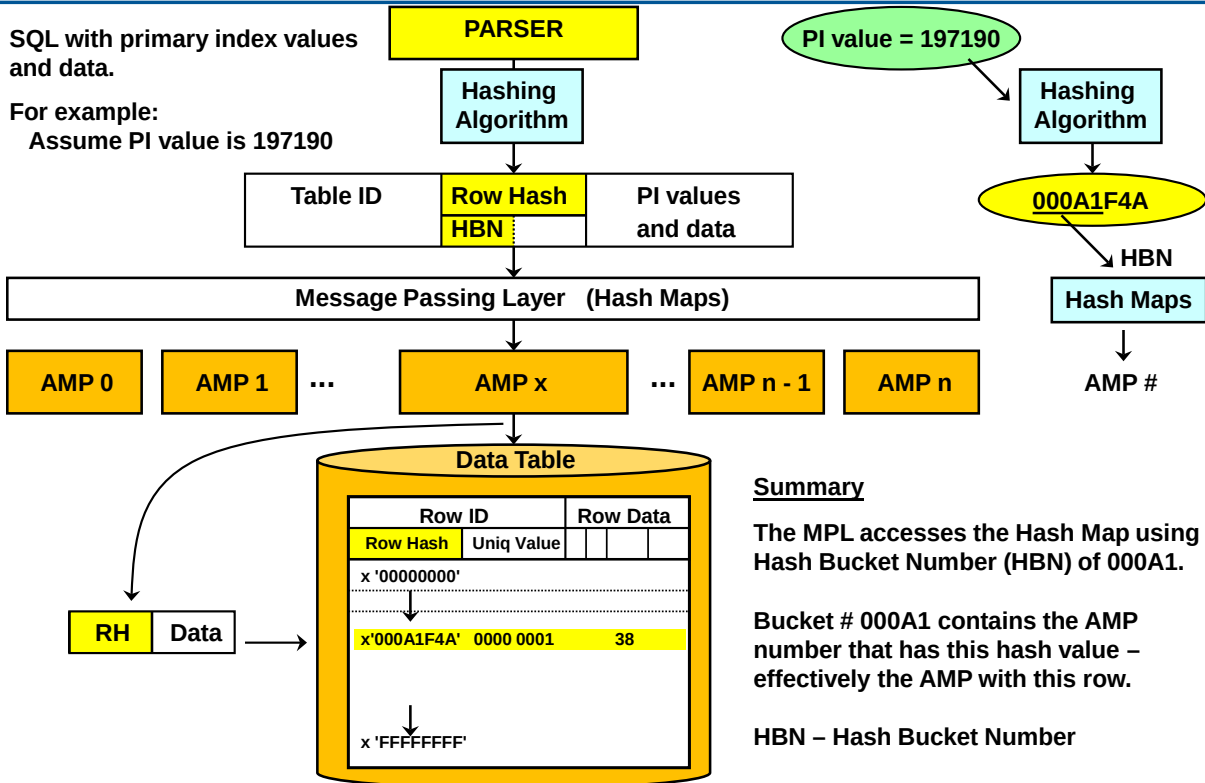
The hash maps must always be available to the Message Passing Layer. For systems using a 16-bit hash bucket number, the hash map has 65,536 entries. For systems using a 20-bit hash bucket number, the hash map has 1,048,576 entries (approximately 1 million entries). 20-bit hash bucket numbers are available starting with Teradata 12.0.

When the hash bucket has determined the destination AMP, the full 32-bit row hash plus the Table-ID is used to assign the row to a cylinder and a data block on the AMPs disk storage. The 32-bit row hash can produce over 4 billion row hash values.

Which AMP has the Row?

SQL with primary index values and data.

For example:
Assume PI value is 197190



Hashing Down to the AMPs

The rows of all tables are distributed across the AMPs according to their **Primary Index** value. The Primary Index value goes into the hashing algorithm and the output is a 32-bit **Row Hash**. The high order bits (16 or 20) are referred to as the “bucket number” and are used to identify a hash map entry. This entry, in turn, is used to identify the AMP that will be targeted. The remaining 12 or 16 bits are not used to locate the AMP.

The entire 32-bit Row Hash is used by the selected AMP to locate the row within its disk space.

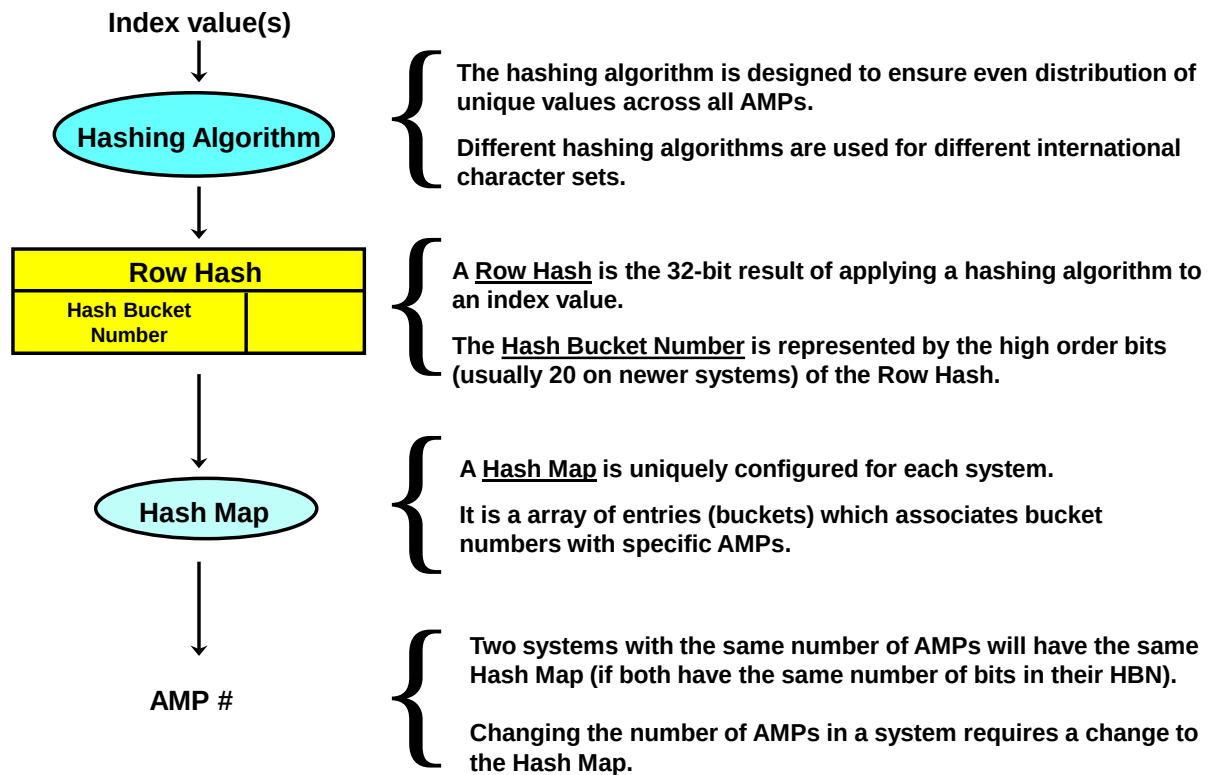
Hash maps are uniquely configured for each size of system, thus a 96 AMP system will have a hash map different from a 64 AMP system, but another 64 AMP system will have the same map (if they have the same number of bits in their HBN).

Each hash map is simply an array that associates Hash Bucket Number (HBN) values or bucket numbers with specific AMPs.

The Hash Bucket Number (prior to Teradata 12.0) has also been referred to as the DSW or Destination Selection Word.

When a system grows, new AMPs are typically added. This requires a change to the hash map to reflect the new total number of possible target AMPs.

Hashing Down to the AMPs



A Hashing Example

The facing page shows an example of how the hashing algorithm would produce a 32-bit row hash value on the primary index value of 197190.

The hash value is divided into two parts. The first 20 bits in this example are the **Hash Bucket Number**. These bits are also simply referred to as the Hash Bucket. The hash bucket points to a particular hash map entry, which in turn points to one AMP. The entire Row Hash along with the Table ID references a particular logical location on that AMP.

A Hashing Example

Orders

Order Number	Customer Number	Order Date	Order Status
PK			
UPI			
197185	2005	2019-04-10	C
197186	3018	2019-04-10	O
197187	1035	2019-04-11	O
197188	1035	2019-04-11	C
197189	1001	2019-04-11	O
197190	2087	2019-04-11	C
197191	1012	2019-04-12	C
197192	3600	2019-04-12	C
197193	5650	2019-04-13	O
197194	1009	2019-04-13	O

```
SELECT * FROM Orders  
WHERE order_number = 197190;
```

197190

Hashing Algorithm

000A1 F4A

32 bit Row Hash

Hash Bucket Number *	Remaining 12 bits
0000 0000 0000 1010 0001	1111 0100 1010

0 0 0 A 1

* Assumes 20-bit hash bucket numbers.

The Hash Map

A **hash map** is simply an array of entries where each entry is two bytes long. The hash map is loaded into memory and is used by Teradata software. Each entry contains an AMP number for the system on which Teradata is implemented. The hash bucket number (or bucket number) is an offset into the hash map to locate a specific entry (or AMP).

For systems using a 16-bit hash bucket number, the hash map has 65,536 entries. For systems using a 20-bit hash bucket number, the hash map has 1,048,576 entries (approximately 1 million entries).

To determine the destination AMP for a Primary Index operation, the hash map is checked by BYNET software using the row hash information. A message is placed on the BYNET to be sent to the target AMP using point-to-point communication.

In the example, the HBN entry 000A1 (hexadecimal) contains an entry that identified AMP 13. AMP 13 will be the recipient of the message from the Message Passing Layer.

The facing page identifies a portion of an actual primary hash map for a 26 AMP system.

An example of hash functions that can be used in SQL follows:

```
SELECT
  HASHROW (197190) AS "Hash Value"
, HASHBUCKET (HASHROW (197190)) AS "Bucket Num"
, HASHAMP (HASHBUCKET (HASHROW (197190))) AS "AMP Num"
, HASHBAKAMP (HASHBUCKET (HASHROW (197190))) AS "AMP Fallback Num"
;
```

```
*** Query completed. One row found. 4 columns returned.
*** Total elapsed time was 1 second.
```

<u>Hash Value</u>	<u>Bucket Num</u>	<u>AMP Num</u>	<u>AMP Fallback Num</u>
000A1F4A	161	13	0

The Hash Map

197190 → Hashing Algorithm → 000A1F4A

32 bit Row Hash	
Hash Bucket Number *	Remaining 12 bits
0000 0000 0000 1010 0001	1111 0100 1010

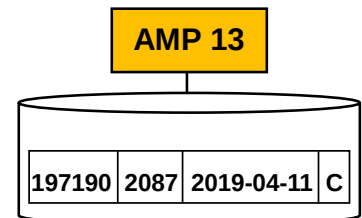
⏟
⏟
⏟
⏟
⏟

0 0 0 A 1

* With 20-bit hash bucket numbers, the hash map has 1,048,576 entries.

HASH MAP																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0007	24	25	19	12	19	19	25	23	20	20	23	20	21	24	13	20
0008	21	22	20	20	22	23	25	21	11	23	10	22	13	10	24	12
0009	21	21	22	14	21	21	22	23	22	22	24	12	25	25	11	11
000A	08	13	23	14	24	09	11	11	23	23	15	25	23	23	13	07
000B	25	06	15	08	24	24	12	24	12	09	10	24	25	05	24	24
000C	16	12	09	25	12	16	14	04	09	09	13	14	25	25	03	13

Portion of actual hash map (20-bit hash bucket numbers) for a 26 AMP system. AMPs are shown in decimal format.



Hash Maps for Different Systems

The diagrams on the facing page show a graphical representation of a Primary Hash Map for an 8 AMP system and a Primary Hash Map for a 16 AMP system.

A data value which hashes to “000028CF” will be directed to different AMPs on different systems. For example, this hash value will be associated with AMP 7 on an 8 AMP system and AMP 15 on a 16 AMP system.

Note: These are the actual partial hash maps for 8 and 16 AMP systems.

Hash Maps for Different Systems

Row Hash (32 bits)	
Hash Bucket Number	Remaining bits

PRIMARY HASH MAP – 8 AMP System

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	07	06	07	06	07	04	05	06	05	05	06	06	07	07	03	04
0001	07	07	02	04	01	00	05	04	03	02	03	05	01	00	02	06
0002	01	00	05	05	03	02	04	03	01	00	06	02	04	04	01	00
0003	07	06	03	03	06	06	02	02	01	00	01	00	07	07	05	07
0004	04	04	05	07	05	06	07	07	03	02	03	04	01	00	02	06
0005	01	00	05	04	03	02	06	05	01	00	06	05	07	06	05	07

PRIMARY HASH MAP – 16 AMP System

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	15	14	15	15	13	14	12	14	13	15	15	12	11	12	13	14
0001	13	14	14	10	15	08	11	11	15	09	10	12	09	09	10	13
0002	10	10	13	14	11	11	12	12	11	11	14	12	13	14	12	12
0003	15	15	13	14	06	08	13	14	13	13	14	14	07	08	15	07
0004	15	04	05	07	09	06	09	07	15	15	03	08	15	15	02	06
0005	01	00	05	04	08	10	10	05	08	08	06	09	07	06	05	11

The integer value 337772 hashes to:

00002 8CF

8 AMP system – AMP 07
16 AMP system – AMP 15

Portions of actual hash maps
with 1,048,576 hash buckets.

Identifying Rows

Can two different PI values come out of the hashing algorithm with the same row hash value? The answer is “Yes”. There are two ways that can happen.

First, two different primary index values may happen to hash identically. This is called a **hash synonym**.

Secondly, if a non-unique primary index is used; **duplicate NUPI values will produce the same row hash**.

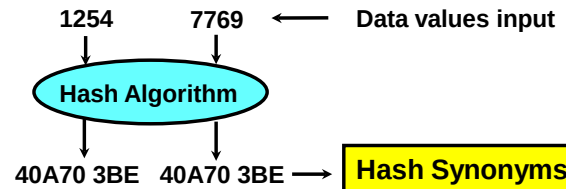
Identifying Rows

A row hash is not adequate to uniquely identify a row.

Consideration #1

A Row Hash = 32 bits = 4.2 billion possible values

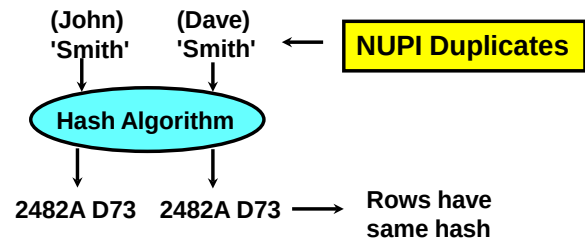
Because there is an infinite number of possible data values, some data values will have to share the same row hash.



Consideration #2

A Primary Index may be non-unique (NUPI).

Different rows will have the same PI value and thus the same row hash.



Conclusion

A row hash is not adequate to uniquely identify a row.

The Row ID

In order to differentiate each row in a table, every row is assigned a unique **Row ID**. The **Row ID** is a combination of the row hash value plus a **uniqueness value**. The AMP appends the uniqueness value to the row hash when it is inserted. The Uniqueness Value is used to differentiate between PI values that generate identical row hashes.

The first row inserted with a particular row hash value is assigned a uniqueness value of 1. Each new row with the same row hash is assigned an integer value one greater than the current largest uniqueness value for this Row ID.

If a row is deleted or the primary index is modified, the uniqueness value can be reused.

Only the Row Hash portion is used in Primary Index operations. The entire **Row ID** is used for Secondary Index support that is discussed in a later module.

In summary, Row Hash is a 32-bit value. Up to and including Teradata V2R6.2, the Message Passing Layer looks at the high-order 16 bits (previously called DSW - Destination Selection Word). This is used to index into the Hash Map to determine which AMP gets the row or is used to retrieve a row. Once the AMP has been determined, the entire 32-bits of the Row Hash are passed to the AMP. The AMP uses the entire 32-bit Row Hash to store/retrieve the row.

Since there are only 4 billion permutations of Row Hash, you can get duplicates. NUPI Duplicates also cause duplicate Row Hashes, therefore the Row Hash is not sufficient to uniquely identify a row in a table. Therefore, the AMP adds another 32-bit number (called a uniqueness value) to the Row Hash. This total 64-bit number (32-bit Row Hash + 32-bit Uniqueness Value) is called the Row ID. This number uniquely identifies a row in a table.

The Uniqueness Value is also referred to as the Uniqueness ID.

The Row ID

To uniquely identify a row, the AMP adds a 32-bit uniqueness value. The Uniqueness Value may also be referred to as the Uniqueness ID.

The combined row hash and uniqueness value is called a Row ID.

Row ID

Row Hash (32 bits)	Uniqueness Value (32 bits)
-----------------------	-------------------------------

Each stored row has a Row ID as a prefix.

Row ID	Row Data
--------	----------

Rows are logically maintained in Row ID sequence.

Row ID		Row Data		
Row Hash	Uniq. Value	EmpNo	LastName	FirstName
3B11 5032	0000 0001	1018	Reynolds	Jane
3B11 5032	0000 0002	1020	Davidson	Evan
3B11 5032	0000 0003	1031	Green	Jason
3B11 5033	0000 0001	1014	Jacobs	Paul
3B11 5034	0000 0001	1012	Chevas	Jose
3B11 5034	0000 0002	1021	Carnet	Jean
:	:	:	:	:

Storing Rows (1 of 2)

Rows are stored in a data block in Row ID sequence. As rows are added to a table with the same row hash, the uniqueness value is incremented by one in order to provide a unique Row ID.

Assume LastName is a NUPI and that all rows in this example hash to the same AMP.

The 'John Smith' row is assigned to AMP 3 based on the bucket number portion of the row hash. Because it is the first row with this row hash, a uniqueness id of 1 is assigned.

The 'Sam Adams' row has a different row hash and thus is also assigned a uniqueness value of 1. The bucket number, although different, also points to AMP 3 in the hash map.

Storing Rows (1 of 2)

Assumptions:

LastName is defined as a NUPI.

All rows in this example hash to the same AMP.

Add a row for 'John Smith'



Row ID		Row Data		
Row Hash	Uniq. Value	LastName	FirstName	Etc.
2482A D73	0000 0001	Smith	John	

Add a row for 'Sam Adams'



Row ID		Row Data		
Row Hash	Uniq. Value	LastName	FirstName	Etc.
2482A D73	0000 0001	Smith	John	
782B7 E4D	0000 0001	Adams	Sam	

Storing Rows (2 of 2)

The 'Fred Smith' row hashes to the same row hash as 'John Smith' because it is a NUPI duplicate. It is therefore assigned a uniqueness id of 2.

The 'Dan Jones' row also hashes to the same row hash because it is a hash synonym. It is thus assigned a uniqueness id of 3.

Note: In reality, the last names of Smith and Jones DO NOT hash to the same value. This is simply an example that illustrates how the uniqueness value is used when a hash synonym does occur.

Storing Rows (2 of 2)

Add a row for 'Fred Smith' - (NUPI Duplicate)

'Smith' → Hash Algorithm → 2482A D73 → Hash Map → AMP #3

Row ID		Row Data		
Row Hash	Uniq. Value	LastName	FirstName	Etc.
2482A D73	0000 0001	Smith	John	
2482A D73	0000 0002	Smith	Fred	
782B7 E4D	0000 0001	Adams	Sam	

Add a row for 'Dan Jones' - (Hash Synonym)

'Jones' → Hash Algorithm → 2482A D73 → Hash Map → AMP #3

Row ID		Row Data		
Row Hash	Uniq. Value	LastName	FirstName	Etc.
2482A D73	0000 0001	Smith	John	
2482A D73	0000 0002	Smith	Fred	
2482A D73	0000 0003	Jones	Dan	
782B7 E4D	0000 0001	Adams	Sam	

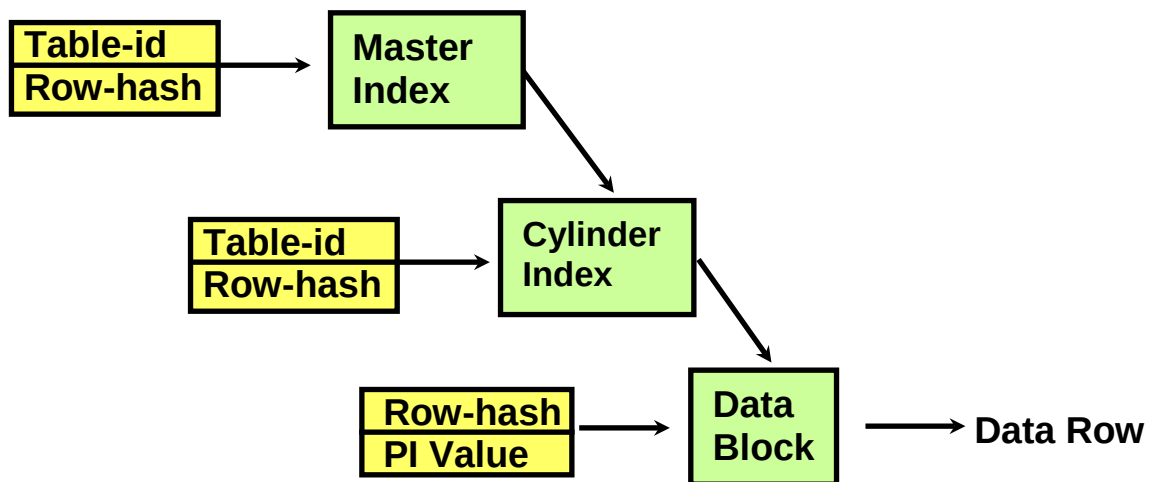
Given the row hash, what other information would be needed to find the 'Dan Jones' row?
... The 'Fred Smith' row?

Locating a Row on an AMP Using a PI

To locate a row, the AMP file system searches through a memory-resident structure called the Master Index. An entry in the Master Index will indicate that if a row with this Table ID and row hash exists, then it must be on a specific disk cylinder.

The file system will use the cylinder number to locate the Cylinder Index and search through the designated Cylinder Index. There it will find an entry that indicates that if a row with this Table ID and row hash exists, it must be in one specific data block on that cylinder.

The file system then searches the data block until it locates the row(s) or returns a No Rows Found condition code.

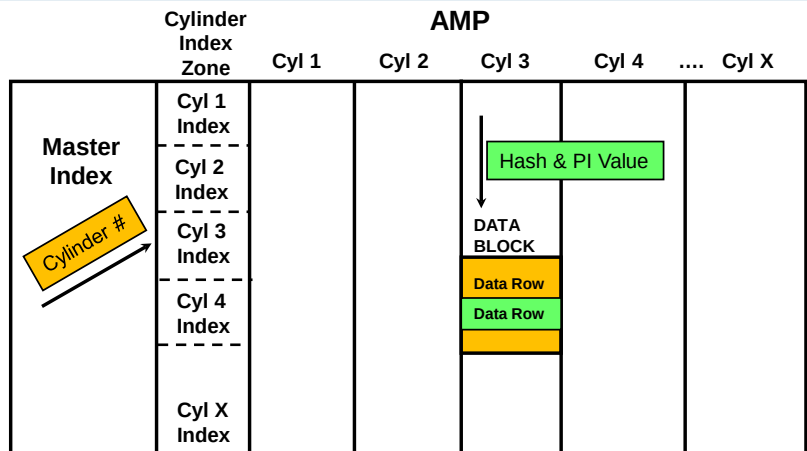


Locating a Row On An AMP Using a PI

Locating a row on an AMP requires three input elements:

1. The Table ID
2. The Row Hash of the PI
3. The PI value itself

Table ID
Row Hash



START WITH:

APPLY TO:

FIND:

Table Id
Row Hash



Master Index



Cylinder #

Cylinder #
Table Id
Row Hash



Cylinder Index



Data Block Address

Row Hash
PI Value



Data Block



Data Row

Module 5: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 5: Review Questions

Answer the following either as True or False as these apply to Primary Indexes:

- True or False 1. UPI and NUPI equality value accesses are always a one-AMP operation.
- True or False 2. UPI and NUPI indexes allow NULL in a primary index column.
- True or False 3. UPI, NUPI, and NOPI tables allow duplicate rows in the table.
- True or False 4. A UPI can be used as a Primary Key implementation.

Fill in the Blanks

- 5. The output of the hashing algorithm is called the _____.
- 6. To determine the target AMP, the Message Passing Layer must lookup an entry in the Hash Map based on the _____.
- 7. A Row ID consists of a row hash plus a _____ value.
- 8. A uniqueness value is required to produce a unique Row ID because of _____ and _____.
- 9. Once the target AMP has been determined for a PI search, the _____ for that AMP is accessed to determine the cylinder that may hold the row.
- 10. The Cylinder Index points us to the address and length of the data _____.

Notes



Secondary Indexes and Table Scans

After completing this module, you will be able to:

- Define Secondary Indexes.
- Distinguish between the implementation of unique and non-unique secondary indexes.
- Define Full Table Scans and what causes them.
- Describe the operation of a Full Table Scan in a parallel environment.

Teradata Proprietary and Confidential

Notes

Table of Contents

Secondary Indexes	6-4
Choosing a Secondary Index.....	6-6
Unique Secondary Index (USI) Access.....	6-8
Non-Unique Secondary Index (NUSI) Access	6-10
Comparison of Primary and Secondary Indexes.....	6-12
Full Table Scans	6-14
Module 6: Review Questions	6-16

Secondary Indexes

A **secondary index** is an alternate path to the data. Secondary Indexes are used to improve performance by allowing the user to avoid scanning the entire table. A Secondary Index is like a Primary Index in that it allows the user to locate rows. It is unlike a Primary Index in that it has no influence on the way rows are distributed among AMPs. A database designer typically chooses a secondary index because it provides faster set selection.

Primary Index requests require the services of only one AMP to access rows, while secondary indexes require at least two and possibly all AMPs, depending on the index and the type of operation. A secondary index search will typically be less expensive than a full table scan.

Secondary indexes add overhead to the table, both in terms of disk space and maintenance; however they may be dropped when not needed, and recreated whenever they would be helpful.

Secondary Indexes

There are 3 general ways to access a table:

Primary Index access	(<u>one</u> AMP access)
Secondary Index access	(<u>two</u> or <u>all</u> AMP access)
Full Table Scan	(<u>all</u> AMP access)

- A secondary index provides an **alternate path** to the rows of a table.
- A secondary index can be used to **maintain uniqueness** within a column or set of columns.
- A table can have from 0 to 32 secondary indexes.
- Secondary Indexes:
 - Do not affect table distribution.
 - Add overhead, both in terms of disk space and maintenance.
 - May be added or dropped dynamically as needed.
 - Are chosen to improve table performance.

Choosing a Secondary Index

Just as with primary indexes, there are two types of **secondary indexes** – **unique (USI)** and **non-unique (NUSI)**.

Secondary Indexes may be specified at table creation or at any time during the life of the table. It may consist of up to 64 columns, however to get the benefit of the index, the query would have to specify a value for all 64 values.

Unique Secondary Indexes (USI) have two possible purposes. They can speed up access to a row which otherwise might require a full table scan without having to rely on the primary index. Additionally, they can be used to enforce uniqueness on a column or set of columns. This is the case when a PRIMARY KEY or UNIQUE constraint is defined and the table has a specified Primary Index or No Primary Index option. Making it a USI has the effect of enforcing the uniqueness of a PRIMARY KEY or UNIQUE constraint.

Non-Unique Secondary Indexes (NUSI) are usually specified in order to prevent full table scans. However, a NUSI does activate all AMPs – after all, the value being sought might well exist on many different AMPs (only Primary Indexes have same values on same AMPs). If the optimizer decides that the cost of using the secondary index is greater than a full table scan would be, it opts for the table scan.

All secondary indexes cause an AMP subtable to be built and maintained as rows are inserted, updated, or deleted. Secondary index **subtables** consist of rows which associate the secondary index value with one or more rows in the base table. When the index is dropped, the subtable is physically removed.

Choosing a Secondary Index

A Secondary Index may be defined ...

- at table creation (CREATE TABLE)
- following table creation (CREATE INDEX)
- may be up to **64 columns**

USI

If the index choice of column(s) is unique, it is called a USI.

Unique Secondary Index

Accessing a row via a **USI is a 2 AMP operation.**

```
CREATE UNIQUE INDEX
(EmpNumber) ON Employee;
```

NUSI

If the index choice of column(s) is non-unique, it is called a NUSI.

Non-Unique Secondary Index

Accessing row(s) via a **NUSI is an all AMP operation.**

```
CREATE INDEX
(LastName) ON Employee;
```

Notes:

- Creating a Secondary Index cause an internal sub-table to be built.
- Dropping a Secondary Index causes the sub-table to be deleted.

Unique Secondary Index (USI) Access

The facing page shows the two AMP accesses necessary to retrieve a row via a **Unique Secondary Index** access.

After the row hash of the secondary index value is calculated, the hash map points us to AMP 1 as containing the subtable row for this USI value. After locating the subtable row in AMP 1, we find the row-id of the base row we are seeking. This base row id (which includes the row hash) again allows the hash map to point us to AMP 3 which contains the base row.

Secondary index access uses the complete row-id to locate the row, unlike primary index access, which only uses the row hash portion.

The Customer table below is the table used in the example. It is only a partial listing of the rows.

Customer Table

Cust	Name	Phone
USI		NUPI
37	White	555-4444
98	Brown	333-9999
74	Smith	555-6666
95	Peters	555-7777
27	Jones	222-8888
56	Smith	555-7777
45	Adams	444-6666

Unique Secondary Index (USI) Access

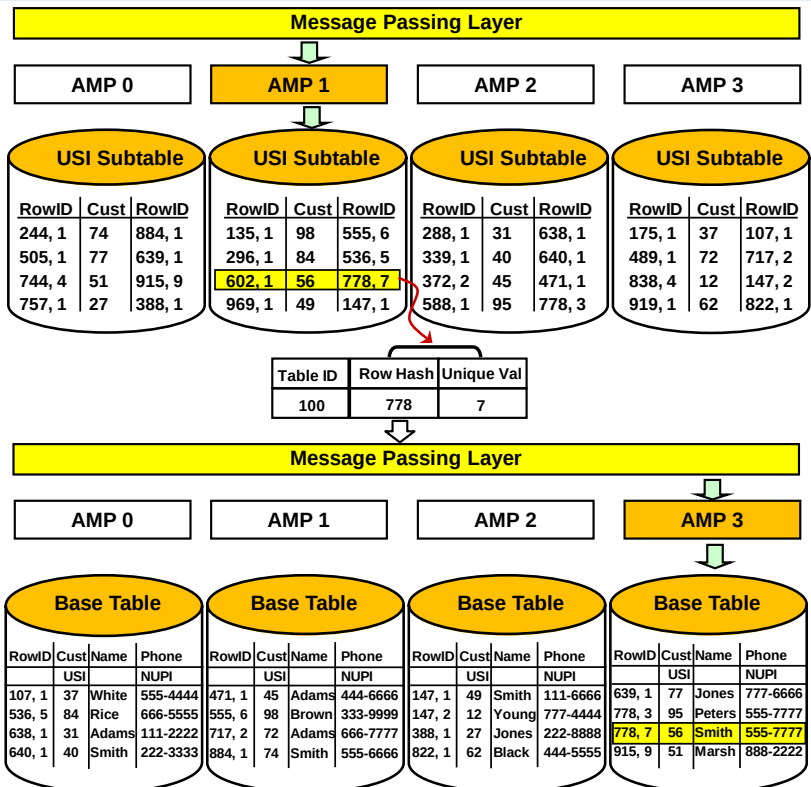
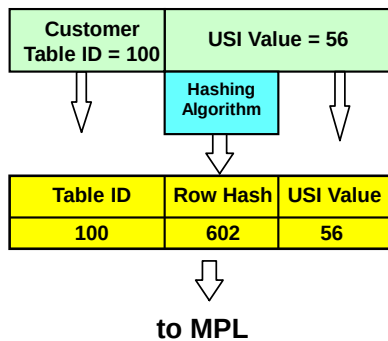
Create USI

```
CREATE UNIQUE INDEX
(Cust) ON Customer;
```

Access via USI

```
SELECT *
FROM Customer
WHERE Cust = 56;
```

PE



Non-Unique Secondary Index (NUSI) Access

The facing page shows an all-AMP access necessary to retrieve a row via a **Non-Unique Secondary Index** access.

After the row hash of the secondary index value is calculated, the Message Passing Layer will automatically activate all AMPs per instructions of the Parsing Engine. Each AMP locates the subtable rows containing the qualifying value and row hash. These subtable rows contain the row-id(s) for the base rows, which are guaranteed to be on the same AMP as the subtable row. This reduces activity in the MPL and essentially makes the query an AMP-local operation.

Because each AMP may have more than one qualifying row, it is possible for the subtable row to have multiple row-ids for the base table rows.

The Customer table below is the table used in the example. It is only a partial listing of the rows.

Customer Table

Cust	Name	Phone
	NUSI	NUPI
37	White	555-4444
98	Brown	333-9999
74	Smith	555-6666
95	Peters	555-7777
27	Jones	222-8888
56	Smith	555-7777
45	Adams	444-6666

Non-Unique Secondary Index (NUSI) Access

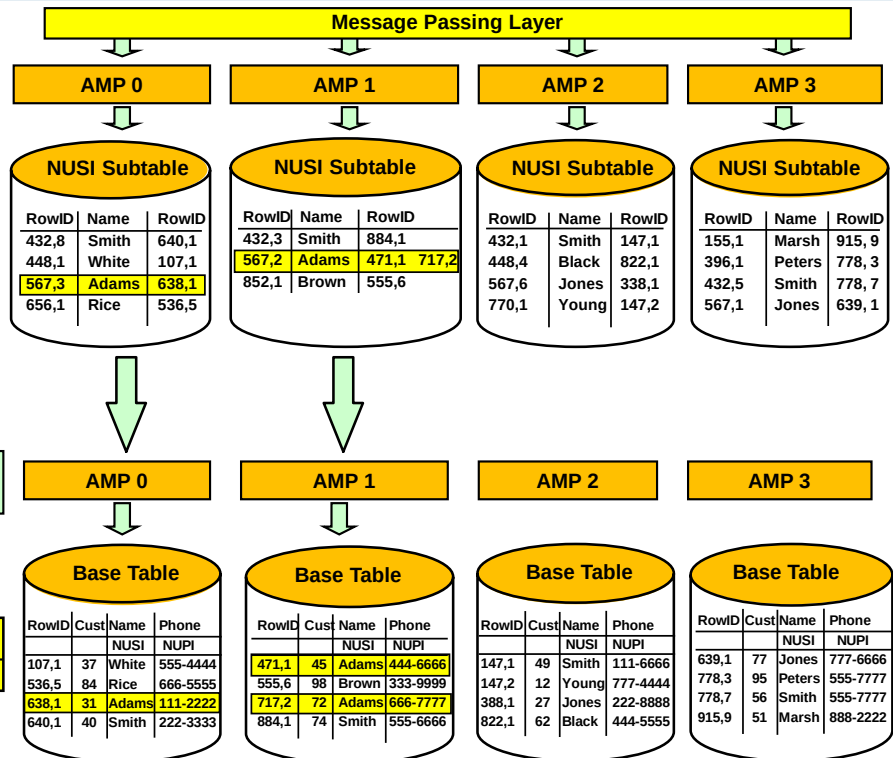
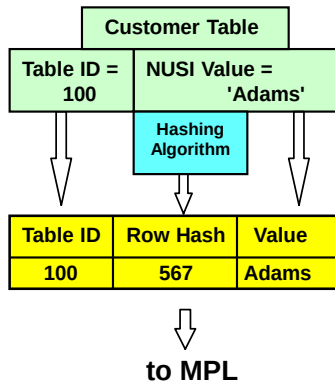
Create NUSI

```
CREATE INDEX (Name)
ON Customer;
```

Access via NUSI

```
SELECT *
FROM Customer
WHERE Name = 'Adams';
```

PE



Comparison of Primary and Secondary Indexes

The table on the facing page compares and contrasts **primary and secondary indexes**:

Primary indexes are required; secondary indexes are optional. All tables must have a method of distributing rows among AMPs -- the Primary Index.

A table can only have one primary index, but it can have up to 32 secondary indexes.

Both primary and secondary indexes can have up to 64 columns.

Secondary indexes, like primary indexes, can be either unique (USI) or non-unique (NUSI).

The secondary index does not affect the distribution of rows. Rows are only distributed according to the Primary Index values.

Secondary indexes can be created and dropped dynamically. In other words, Secondary Indexes can be added as needed. In fact, in some cases it is a good idea to wait and see how the database is used and then add Secondary Indexes to facilitate that usage.

Both primary and secondary indexes affect system performance. However, Primary and Secondary Indexes affect performance for different reasons. A poorly-chosen PI results in “lumpy” data distribution which makes some AMPs do more work than others and slows the system.

Secondary Indexes affect performance because they require subtables. Both indexes allow rapid retrieval of specific rows.

Both primary and secondary indexes can be created using multiple data types.

Secondary indexes are stored in separate subtables; primary indexes are not.

Because secondary indexes require separate subtables, extra I/O is needed to maintain those subtables.

Comparison of Primary and Secondary Indexes

Index Feature	Primary	Secondary
Required?	Yes*	No
Number per Table	1	0 - 32
Max Number of Columns	64	64
Unique or Non-unique	Both	Both
Affects Row Distribution	Yes	No
Created/Dropped Dynamically	No	Yes
Improves Access	Yes	Yes
Multiple Data Types	Yes	Yes
Separate Physical Structure	No	Sub-table
Extra Processing Overhead	No	Yes
May be ordered by value	No	Yes (NUSI)
May be Partitioned	Yes	No

* Not required with NoPI table

Full Table Scans

A **full table scan** is another way to access data without using any Primary or Secondary Indexes.

In evaluating an SQL request, the Parser examines all possible access methods and chooses the one it believes to be the most efficient. The coding of the SQL request along with the demographics of the table and the availability of indexes all play a role in the decision of the Parser. Some coding constructs, listed on the facing page, always cause a full table scan. In other cases, it might be chosen because it is the most efficient method. In general, if the number of physical reads exceeds the number of data blocks then the optimizer may decide that a full-table scan is faster.

With a full table scan, each data block is found using the Master and Cylinder Indexes and each data row is accessed only once.

As long as the choice of Primary Index has caused the table rows to distribute evenly across all of the AMPs, the parallel processing of the AMPs can do the full table scan quickly. The file system keeps each table on as few cylinders as practical to help reduce the cost full table scans.

While full table scans are impractical and even disallowed on some systems, the Teradata NewSQL Engine routinely executes ad hoc queries with full table scans.

Full Table Scans

Every row of the table must be read.

All AMPs scan their portion of the table in parallel.

- Fast and efficient on Teradata due to parallelism.

Full table scans typically occur when either:

- An index is not used in the query
- An index is used in a non-equality test

Customer

CustID	CustName	CustPhone
USI		NUPI

Examples of Full Table Scans:

```
SELECT * FROM Customer WHERE CustPhone LIKE '858-485- _ _ _ _';  
SELECT * FROM Customer WHERE CustName = 'Ratzesberger';  
SELECT * FROM Customer WHERE CustID > 1000;
```

Module 6: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 6: Review Questions

Fill each box with either Yes, No, or the appropriate number.

	USI Access	NUSI Access	FTS
# AMPs			
# rows			
Parallel Operation			
Uses Hash Maps			
Uses Separate Sub-table			
Reads all data blocks of table			

Notes



Teradata Studio

After completing this module, you will be able to:

- Define a Data Source Connection for Teradata.
- Submit SQL using Teradata Studio or Teradata Studio Express.
- Utilize Explorer Tree to simplify creation of queries.
- Use Teradata Studio or Teradata Studio Express to access a table with a LOB.

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata SQL Tools	7-4
Additional Teradata Studio Features.....	7-6
Teradata Studio	7-8
Defining a Connection	7-8
Defining a Data Connection profile (cont.).....	7-10
Connecting to a Teradata System.....	7-12
Teradata Studio Query Development.....	7-14
Main Window	7-14
Data Source Explorer Tree.....	7-16
Data Source Explorer Tree (cont.)	7-18
Creating and Executing a Query	7-20
Dragging Object Names to the Query Window	7-22
Query Options	7-24
Viewing Query Results	7-26
SQL Query Builder	7-28
Teradata SQL History Window	7-30
General Options	7-32
Connecting to Multiple Data Sources	7-34
Importing a LOB into Teradata.....	7-36
Selecting from a Table with a LOB	7-38
Displaying a JPG Large Object.....	7-40
Teradata Studio Summary.....	7-42
Module 7: Review Questions.....	7-44
Lab Exercise 7-1	7-46
Lab Exercise 7-1 (cont.).....	7-48
Lab Exercise 7-1 (cont.).....	7-52

Teradata SQL Tools

Teradata SQL Assistant is an information discovery tool designed for the Windows operating system (e.g., Windows 7). Teradata SQL Assistant retrieves data from any ODBC-compliant database server. The data can then be manipulated and stored on the desktop PC.

Teradata SQL Assistant is a query tool written for relational database developers. It is intended for SQL-proficient developers who know how to formulate queries for processing on Teradata or other ODBC-compliant Databases. Used as a discovery tool, Teradata SQL Assistant catalogs submitted instructions to arrive at a derived answer. Teradata SQL Assistant stores the history of your SQL in a local Microsoft Access database table. This history is available in future executions of Teradata SQL Assistant.

Teradata SQL Assistant accepts standard Teradata SQL, DDL, and DML. In addition, Teradata SQL Assistant sends native SQL to any other database that provides an ODBC driver. If the driver supports the statements, they are processed correctly.

Teradata Studio is a client-based graphical interface for performing database administrative and management tasks on Teradata NewSQL Engines, Teradata Aster Databases, and Hadoop systems.

Teradata Studio is built on the Eclipse Rich Client Platform (RCP) which takes advantage of the RCP framework for building and deploying native GUI applications. It extends the Eclipse Data Tools Platform (DTP) to provide enhancements for accessing objects and data stored in Teradata-supported databases.

Teradata Studio Express is the base level product. It is an information discovery tool for retrieving and displaying data from your Teradata NewSQL Engine systems. Its main use is for running SQL statements, displaying result set data, and storing SQL execution history.

Teradata Studio, in addition to the features provided in Teradata Studio Express, includes administration dialogs and wizards. Its main use is for administrators to create and administer database objects. Teradata Studio is a tool for DBAs, Data Scientists, and Power Users.

Teradata SQL Tools

Teradata SQL Assistant is an information discovery tool to retrieve data from Teradata using either an ODBC or Teradata .NET interface.

Teradata Studio Express is also an information discovery tool that retrieves data from Teradata, Hadoop, and Aster databases using a JDBC interface.

Teradata Studio extends the Studio Express capabilities by allowing you to also perform database administration tasks and has replaced Teradata Administrator.

All of these tools provide the following capabilities:

- Provide access via a graphical user interface.
- Submit SQL to Teradata – support DDL, DML, and DCL commands
- Provides a Data Source Explorer Tree to easily view database objects.
- Provides both import and export capabilities to files in a Windows environment.
- Permits retrieval of previously used queries (History).
- This module is intended to provide an introduction to establishing Teradata connections and submitting SQL using Teradata Studio or Teradata Studio Express.

Additional Teradata Studio Features

Teradata provides three Eclipse-based client tools, Teradata Studio, Teradata Studio Express, and Teradata Plug-in for Eclipse. All three products are built from the same code base and provide connectivity to your Teradata NewSQL Engine, but each has a different end user focus.

As mentioned before, Teradata Studio Express is the base level product. Teradata Studio, in addition to the features provided in Teradata Studio Express, includes administration dialogs and wizards. Its main use is for administrators to create and administer database objects. Teradata Studio is a tool for DBAs, Data Scientists, and Power Users.

Teradata Plug-in for Eclipse, in addition to the features provided in Teradata Studio, plugs into your existing Eclipse IDE and provides wizards to help users create Java database objects such as Java Stored Procedures and Java User Defined Functions. Teradata Plug-in for Eclipse is a tool for developers creating Java applications and database objects.

Some of the additional capabilities of Teradata Studio and Studio Express include:

- **Hadoop support**
 - New Hortonworks Connection Profile Wizard
 - Option to connect to Hortonworks Hadoop systems through Knox Gateway
 - Option to connect to Hortonworks Hadoop systems using JDBC Driver, SQL-H, and TDCH
 - Transfer table data from Hadoop to Aster Databases
 - Hive JDBC SQL support provides execution of HiveQL commands to a Hadoop system from the Teradata SQL Editor
 - Drag and drop of Hadoop table and column names from DSE to Teradata SQL Editor
- **Aster Database support**
 - Table Data Editor support for Aster data.
 - Data Transfer Wizard enables Aster data transfers.
 - Smart Load Wizard enables Aster data transfers to and from external files.
 - Load Data Wizard supports load of data from Teradata to Aster.
 - Export Data Wizard supports export of data from Aster to Teradata.
- **Compare SQL Objects Wizard**
 - Select database or table objects to compare the Data Definition Language (DDL) text.
 - Display, compare, and edit DDL text in Compare Editor.
 - Save or Execute DDL query statements.
- **Compare Editor**
 - Compare DDL content of two database or table objects side-by-side.
 - Highlights differences in text and changes in elements.
 - View differences and changes using Next and Previous buttons.

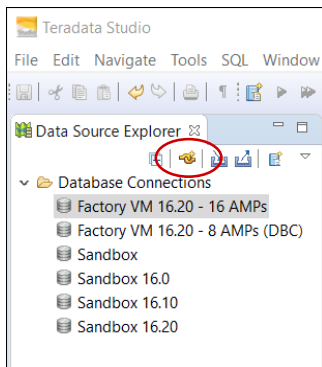
Teradata Studio

Defining a Connection

To begin using Teradata Studio, create a Connection Profile for your database.

Connection to any database must be made through a JDBC (Java-based data access technology) connection. The Connection Profile details the JDBC information required to connect to an instance of a Teradata NewSQL Engine.

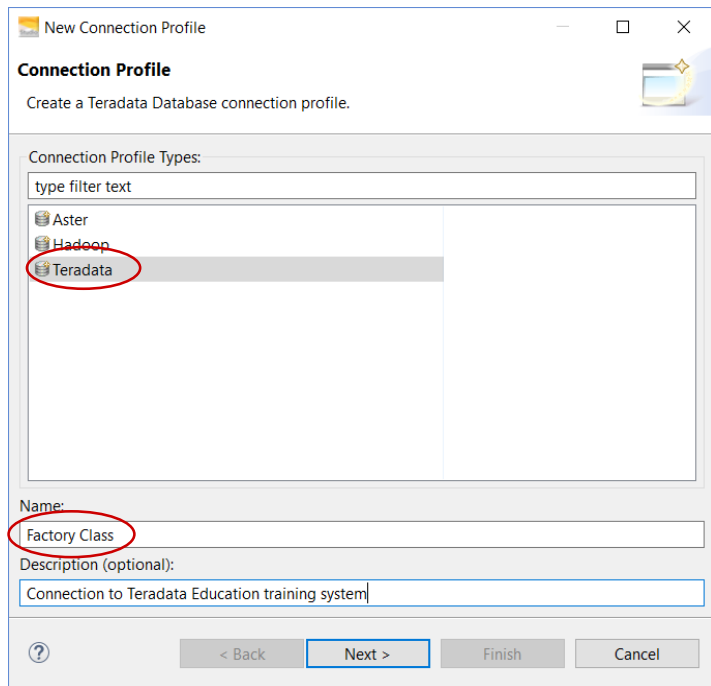
Teradata Studio Defining a Connection



When you first install Teradata Studio, you will get prompted for a new connection.

If not, then Click “New Connection Profile” icon.

Select **Teradata Database** as Connection Profile type, enter a name and then click on **Next** button.



Defining a Data Connection profile (cont.)

When connecting to the Teradata NewSQL Engine, determine which mode you will be using. Studio defaults to ANSI mode rather than Teradata mode. Therefore, you may need to change it in the data connection profile.

Teradata Studio Defining a Connection (cont.)

New Teradata Connection Profile

Specify a Driver and Connection Details
Select a driver from the drop-down and provide login details for the connection.

Select a driver from the drop-down: Teradata Embedded JDBC Driver Default

Database Server Name: 10.25.252.18

User Name[Domain]: student135

Password:

Authentication Mechanism: PASSWORD_PROTECTED

Database:

☐ Save Password

JDBC Connection Properties

CHARSET=UTF8
TMODE=ANSI

Add...
Remove
Clear All
Reset to Defaults
Save to Preference

Cache Properties Delete Cache Files

☒ Connect when the wizard completes
☐ Connect every time the workbench is started

Test Connection

Back Next > Finish Cancel

Enter Database Server name or IP address.

Then enter User Name and Password.

Add JDBC Property

Select JDBC property and value to add to the Connection profile

JDBC Properties

Select JDBC Property: TMODE

Select Property Value: TERA

OK Cancel

Important Note:

Default is ANSI mode so click **Add** and select **TMODE** as JDBC property and select value **TERA** for Teradata Mode and click **OK**.

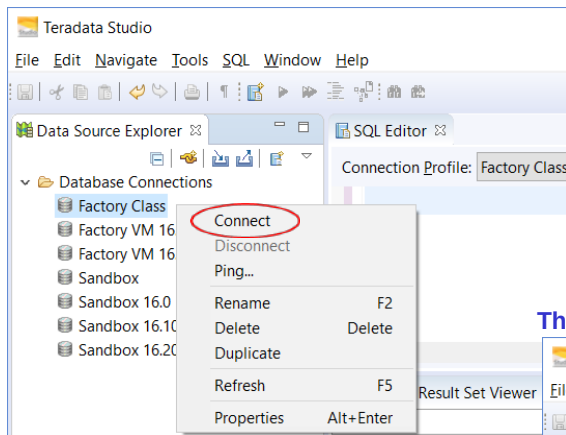
Click **OK** again to complete this dialog. Then click **Finish** to complete connection setup.

Connecting to a Teradata System

Connecting to a data source is the equivalent of “logging on” with Teradata Studio. You may choose from any previously defined data connection profile. Just right mouse click on the defined data connection and select “Connect”.

Teradata Studio

Connecting to a Teradata System

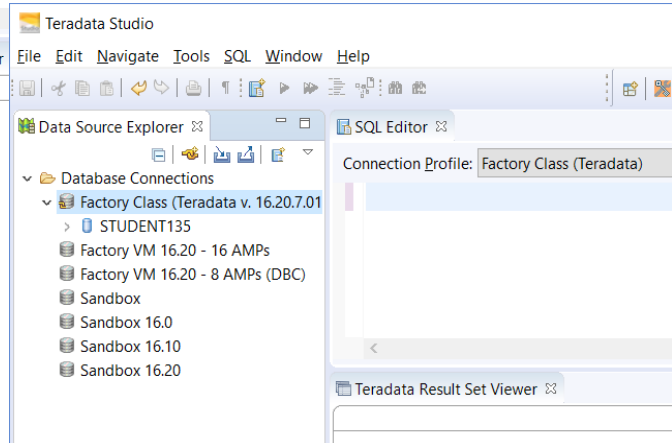


You should connect as part of an initial setup.

If NOT then:

1. Right Click on the connection name and **SELECT Connect.**
or **Double-click on the connection name.**
2. Re-enter password

The following display should occur when connected.



Teradata Studio Query Development

Main Window

The **SQL Editor** (query window) provides an integrated environment for the development of SQL files that enables you to develop SQL statements for execution. The editor enables you to validate SQL syntax, save files to a project or export to system files, and execute all SQL in a file or execute the SQL as individual statements. You must specify a Connection Profile before you can execute an SQL query statement.

The SQL Editor displays when you open an SQL/DDL file or when you select the Query Development Perspective. Double-clicking an SQL/DDL file in the Project Explorer opens that file in the editor.

The **Result Set Viewer** (result set window) provides an enhanced view of the result set, including options to hide columns, filter on column values, find column values, copy data to the clipboard, format cell data, and switch between grid and text view. The result set is displayed in its own view with its own toolbar.

You can change preferences to customize the view by, for example, displaying alternating row background colors, enabling or disabling row numbers, column headers, table grid lines, and thousand separators on numeric columns.

SQL History View (history window) contains the results when executing SQL statements and running database routines, user-defined functions, and triggers.

The **Data Source Explorer** (database explorer tree) enables you to connect to a specific database system using a Connection Profile and then view and manage the database objects in the connected system.

Teradata Studio Query Development Main Window

The screenshot displays the Teradata Studio main window with the following components:

- Database Explorer Tree:** Located on the left, it shows a tree structure of database connections. The selected connection is 'STUDENT135' under 'Factory Class (Teradata v. 16.20.7.01)'. Other connections include 'Factory VM 16.20 - 16 AMPs', 'Factory VM 16.20 - 8 AMPs (DBC)', and several 'Sandbox' instances.
- Query Window:** The central pane shows the SQL Editor with the query: `SELECT * FROM DS.Orders SAMPLE 10;`. The connection profile is set to 'Factory Class (Teradata)'.
- Result Set Window:** Below the query window, it displays the results of the query in a table format. The table has columns: orderid, custid, orderstatus, ordertotal, orderdate, orderpriority, clerk, location, shippriority, and ordercomment. The results show 10 rows of sample data.
- History Window:** At the bottom, it shows the execution history of the SQL statements. It includes columns for Source, User ID, Destination, Result, Row Count, and Note. The history shows two statements executed successfully, each returning 10 rows.

You can customize the look and feel of the interface by moving view panels within the display window.

Data Source Explorer Tree

The **Data Source Explorer Load Preferences** enables you to configure your preference settings for loading database and user objects and to determine what folders are shown in the hierarchical display. You can set preferences to:

- Load databases and users:
 - All databases and all users
 - Load only databases and certain users
 - Load all databases and users with PermSpace configured
- Configure hierarchical display to show:
 - All databases and all users
 - Only certain folders you specify

Teradata Studio Query Development Data Source Explorer Tree

To change the Explorer Tree display settings, use the following menus

Window → Preferences

For class purposes,

Check

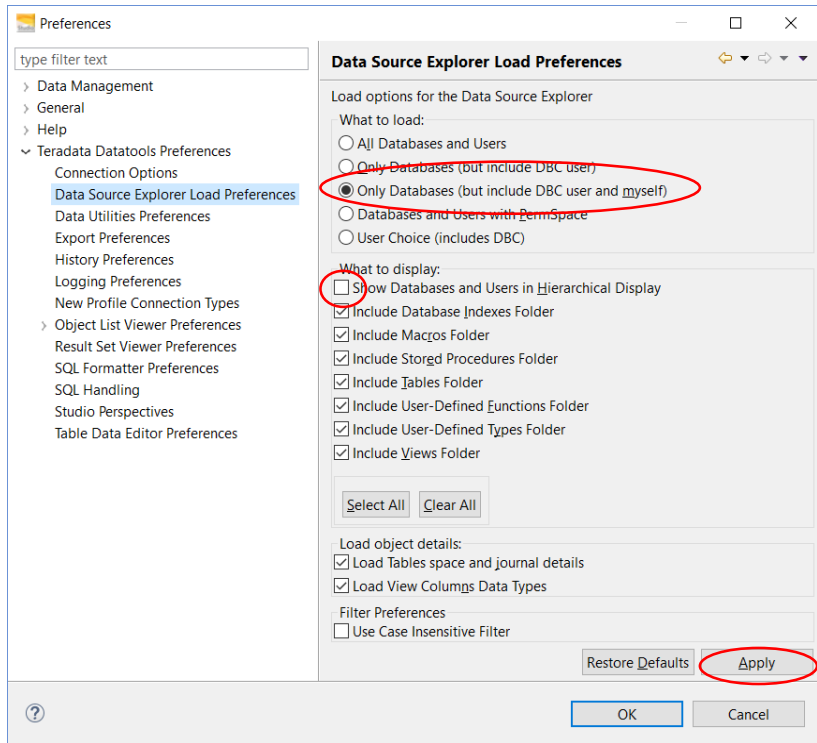
Only Databases (but include DBC user and myself)

Uncheck

Show Databases and Users in Hierarchical Display

Note:

Unchecking "Show Databases and Users in Hierarchical Display" can speed up dialogs such as Tools→Create→Database, User and Macro.



Data Source Explorer Tree (cont.)

The database objects display in a navigation pane in which you can explore the structure of a database's data and manage stored data sets. You can view a list of tables, macros, stored procedures, views, and other objects in a selected database, and view additional details about each object type. For example, from the Databases node you can expand a Tables node to view and list of tables in a database, and the names and types of columns in the table.

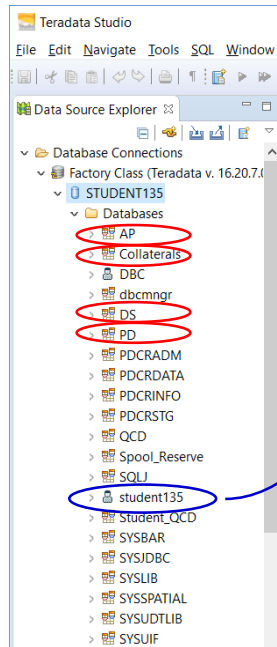
The facing page illustrates the choices shown on the previous page.

Teradata Studio Query Development Data Source Explorer Tree (cont.)

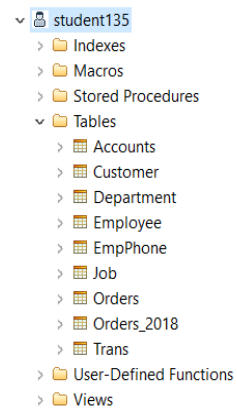
- The Database Explorer Tree displays an alphabetical listing of databases and objects of the connected user.
- To refresh a database, right-click on the database/user name and select "Refresh".

For this course,
some of the
users/databases
we will be using
are the following:

- AP
- Collaterals
- DBC
- DS
- PD
- studentXXX



To expand an item/object,
click on the icon or
double-click on the
object name.



Creating and Executing a Query

The SQL Editor provides a scratchpad for you to enter your SQL statements. Queries are created by simply typing in the query text into the query window. It is not necessary to add a semi-colon at the end of a command, unless you are entering multiple commands in a single window. As an option, you can enable the auto activation of code assist by selecting it from the SQL menu on the toolbar. If this option is enabled, as you type your SQL statement, code assist will pop up annotations showing a list of expected keywords. If a database name is entered followed by a dot, the content assist will display the list of tables from your database for you to select from. (Note: By default, auto activation of code assist is disabled. Also, you must select a Teradata or Aster Database connection profile type in order for SQL code assist to work.)

Parameterized queries can also be created in the SQL Editor. The query contains parameters indicated by question marks. When the query is executed, the user is prompted for the parameter value(s) in the Configure Parameters dialog. Named parameters (appending backslash, '\', <parameter name>) allows the user to specify a name for the parameter. When the Configure Parameters dialog is presented, the parameter names will be used to reference the parameters.

The SQL Editor also provides syntax coloring for your SQL statements. You can change the colors of comments, keywords, identifiers, types, or other syntax items via the SQL Editor Preferences page.

Creating statements (single and multi-queries)

When right clicking in the SQL Editor window, a SQL Editor Context menu appears with options to execute the SQL statement(s) in your SQL file.

The following options are presented:

- Execute all of the SQL statements
- Execute the selected SQL statement
- Execute the SQL statements as individual statements (with a commit occurring between statements)
- Execute the selected statement as one statement
- Execute the current statement where the cursor is located
- Execute the SQL statements as a single transaction (Aster only)
- Visual Query Builder (Teradata only)

You can also choose the Execute options from the SQL Editor or top level toolbar as shown on the right.

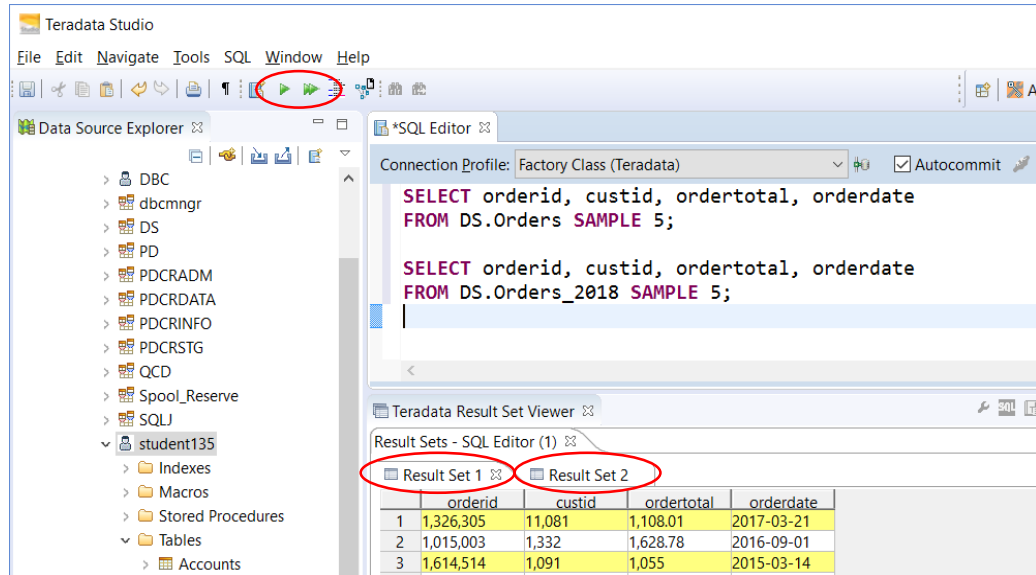
Also, manual commit and rollback of the SQL Execution are provided via SQL Editor toolbar options. An Autocommit checkbox is provided in the SQL Editor to allow the user to toggle Autocommit on or off. By default, Autocommit is ON. If the option is turned off, then the Commit and Rollback options are enabled for the user to manually control the commit of their SQL statements to the database or rollback if an error occurs.

Teradata Studio Query Development

Creating and Executing a Query

1. Create a query in the Query Window.
2. To execute a query use either the “Execute as Individual Statements” icon / button (or F5) **serially** executes all statements in the query window.

Or the “Execute All” drop down selection executes all statements in the query window in a single multi-statement request. These queries are effectively executed **in parallel**.



Dragging Object Names to the Query Window

You can drag object names from the Database Explorer tree to the Query pane.

Click and drag the object from the Explorer tree to the Query pane. The name of the object appears in the Query window.

Use the Ctrl key to add a comma after the object name when it is dragged to the Query Window.

Dragging Multiple Objects

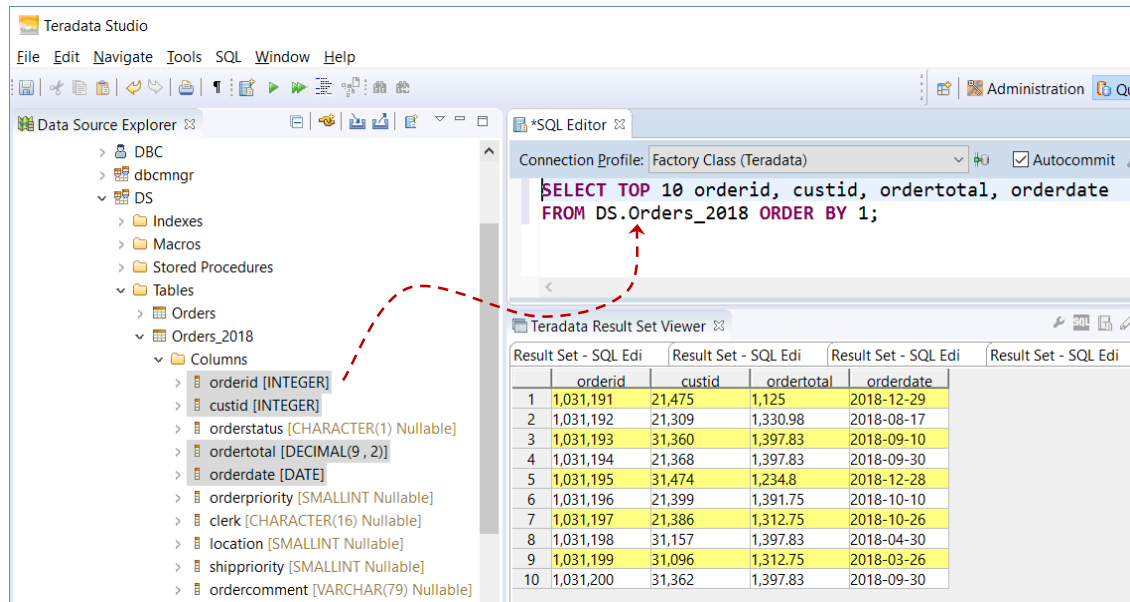
Use the Shift and Ctrl keys to select more than one object from the Database Explorer Tree that can be dragged to the Query window.

- Use the Ctrl key to select additional objects.
- Use the Shift key to select a range of objects.

Teradata Studio Query Development

Dragging Object Names to the Query Window

- You can click and drag object(s) from the Database Explorer tree to the Query window.
 - By default, qualified names are automatically included. Use the Window > Preferences > Teradata Datatools Preferences > SQL Handling window to turn off this option if desired.
 - The **Shift** and **Ctrl** keys can also be used to select multiple objects in the Database Explorer tree for the purpose of dragging multiple objects to the Query Window.



Query Options

To submit any part of any query

1. Highlight the SQL text you want to run.
2. Click the execute play button.
3. Or Right mouse click and select “Execute selected text”.
4. Or Press Keys “**Alt -x**”.

Formatting a Query

The query formatting feature adds line breaks and indentation before certain keywords, making SQL that comes from automatic code generators or other sources more readable.

To Format a Query

1. Ensure a statement exists in the Query window.
2. Do one of the following:
 - Select menu SQL > Format SQL
 - Or Press **Ctrl+Shift+F**
 - Or Select menu **Edit > Format Query**

Indentation

When you press the Enter key, the new line will automatically indent to the same level as the line above.

If you highlight one or more lines in the query and press the Tab key, those lines are indented one level. If you press Shift-Tab, the highlighted lines are un-indented by one level.

This indentation of lines will only apply if the selected text includes a line feed character. For example, you must either select at least part of two lines, or if selecting only one line, then the cursor must be at the beginning of the next line. (Note that this is always the case when you use the margin to select a line.) If no line end is included in the selected text, or no text is selected, then a tab character will simply be inserted.

Teradata Studio Query Development Query Options

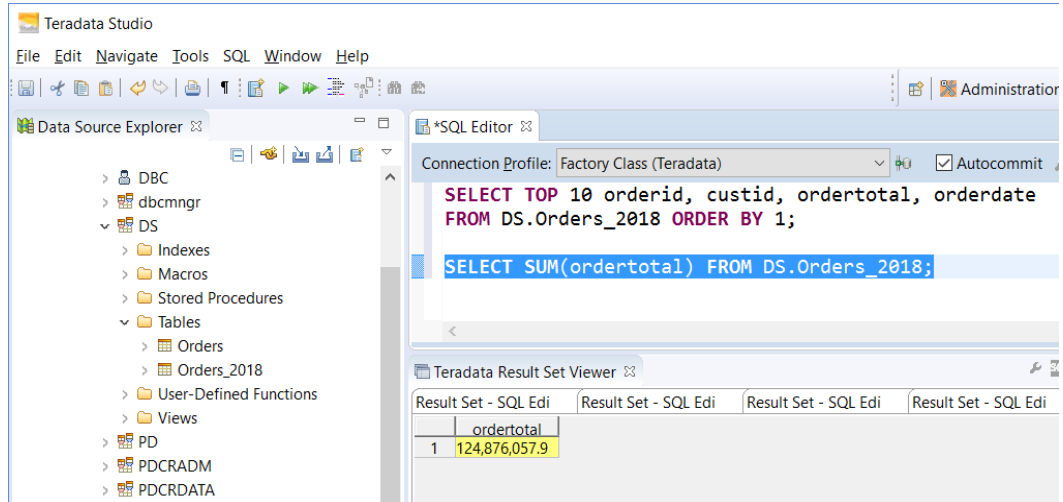
To submit any part of a query:

Highlight the text in the query window and execute. 

To format a query, select the **Format SQL** option from either the main Edit menu or SQL menu. (**Ctrl+Shift+F**)

There is also a **Prompt for notes** and **Code Assist Autoactivation** options from the SQL menu.

	Execute as Individual Statements	F5
	Execute All	Ctrl+Alt+X
	Explain	F6
	SELECT Statement	
	Show DDL	
	Format SQL	Ctrl+Shift+F
	Prompt for notes	
	Code Assist Autoactivation	



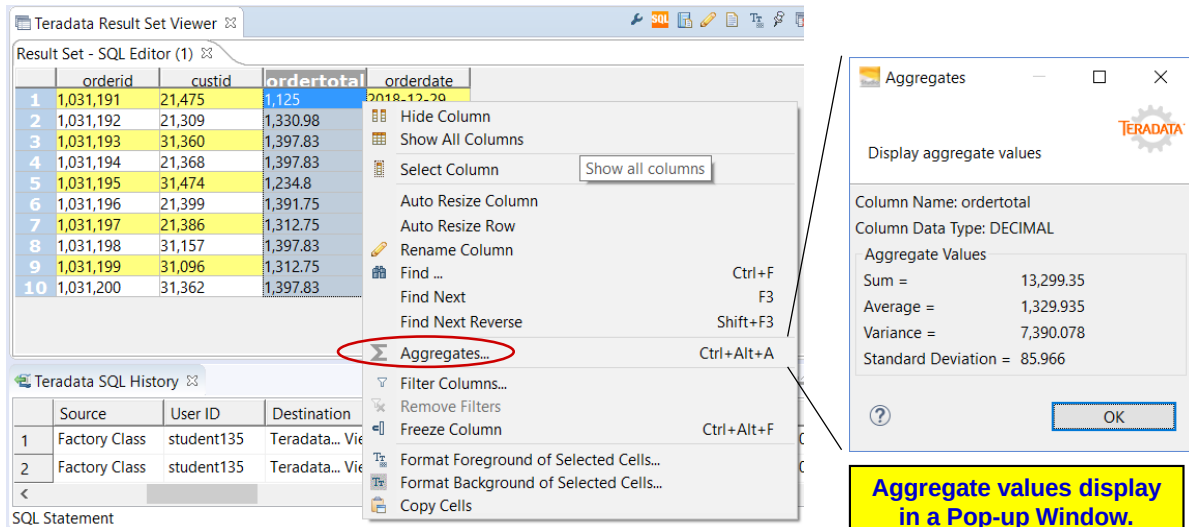
Viewing Query Results

The Result Set Viewer displays the result set or parameter data from executing SQL statements and running database routines, such as stored procedures, macros, user-defined functions, and triggers.

The Result Set Viewer provides an enhanced display of the Result Set data. It displays the result sets as tabs within the Teradata Result Set Viewer, allowing the user to select rows or cells from the Result Set data table grid.

Teradata Studio Query Development Viewing Query Results

- The Teradata Result Set Viewer window is a table that displays the results.
- The output table has many options by right mouse clicking in Result Set:
 - Aggregates of selected numeric values (see below causes a pop-up window with result)
 - Auto Resize Columns and Rows
 - Data can be filtered using the Filter Columns... option (funnel )



The screenshot shows the Teradata Result Set Viewer window displaying a table with columns: orderid, custid, ordertotal, and orderdate. A right-click context menu is open over the 'ordertotal' column, with the 'Aggregates...' option circled in red. A callout box shows the 'Aggregates' pop-up window displaying the following information:

Aggregates

Display aggregate values

Column Name: ordertotal
Column Data Type: DECIMAL

Aggregate Values

Sum =	13,299.35
Average =	1,329.935
Variance =	7,390.078
Standard Deviation =	85.966

OK

Aggregate values display in a Pop-up Window.

SQL Query Builder

Teradata Studio provides a graphical interface for creating SQL statements for Teradata, referred to as the **SQL Query Builder**. You can edit an existing statement using the SQL Query Builder by selecting the statement within the SQL Editor and choosing the 'Edit in SQL Query Builder...' menu option.

Teradata Studio Query Development SQL Query Builder

SQL Query Builder provides the user with the ability to visually create SQL commands.

1. To launch just Right click in SQL Editor and Select **Edit in SQL Query Builder (Alt+q).**



2. Right mouse click to **Add tables.** Once you have multiple tables you can right click and create a join if you wish.

The screenshot shows the SQL Query Builder dialog with the following components:

- SQL Statement:** A text area containing the SQL query:


```
SELECT "orderid", "custid", "ordertotal", "orderpriority"
FROM DS."Orders_2018"
```
- Table List:** A list of tables with checkboxes. The table "Orders_2018" is selected, and its columns "orderid" and "custid" are also checked.

Table	Column	Selected
Orders_2018	orderid	☒
	custid	☒
- SELECT Statement:** A button labeled "SELECT Statement" with a blue icon.
- DISTINCT:** A checkbox labeled "DISTINCT" which is currently unchecked.
- Columns Tab:** A table showing the selected columns and their properties.

Column	Alias	Output	Sort Type	Sort Order
Orders_2018.orderid		☒		
Orders_2018.custid		☒		
- Edit:** A button labeled "Edit" at the bottom left.
- Buttons:** "OK" and "Cancel" buttons at the bottom right.

Teradata SQL History Window

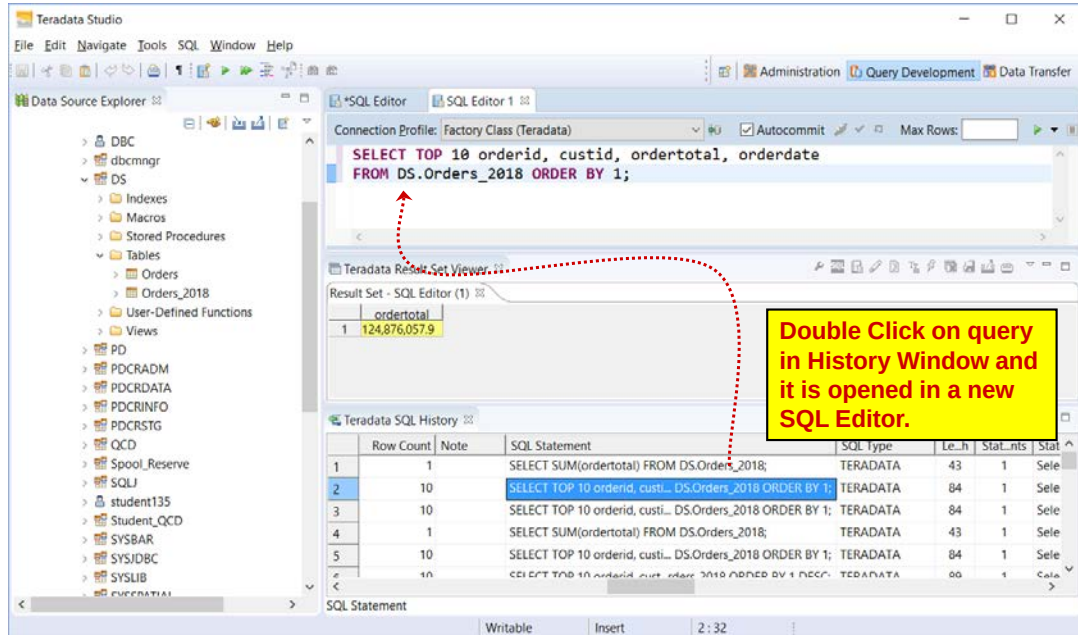
The SQL History View provides an enhanced display of the SQL History information. As SQL statements are executed from the SQL Editor or functions are run from the Data Source Explorer, an entry is placed in the SQL History. The SQL History view displays the data in a grid format, allowing the user to select rows and cells.

The SQL History provides extensive information about the SQL execution, such as execution elapsed time, DBMS time, and fetch time, database server, row count, parameter display for macros and stored procedures, SQL statement, and SQL statement type. It provides options to copy cells or rows, edit the 'Note' or SQL for the SQL History entry, search for result history data, and re-execute SQL statements. Choosing the 'Edit SQL' option will open a SQL Editor file with the selected history entry's SQL statement. This allows users to further fine tune their SQL statements.

Teradata Studio Query Development Teradata SQL History Window

Historical record of the submitted SQL queries with timestamps, results status, record counts, etc. History data can be exported and shared with other Teradata Studio users.

Note: If you close the view and need to re-open it go to **Window > Show View > Other > Teradata > Teradata SQL History**.



General Options

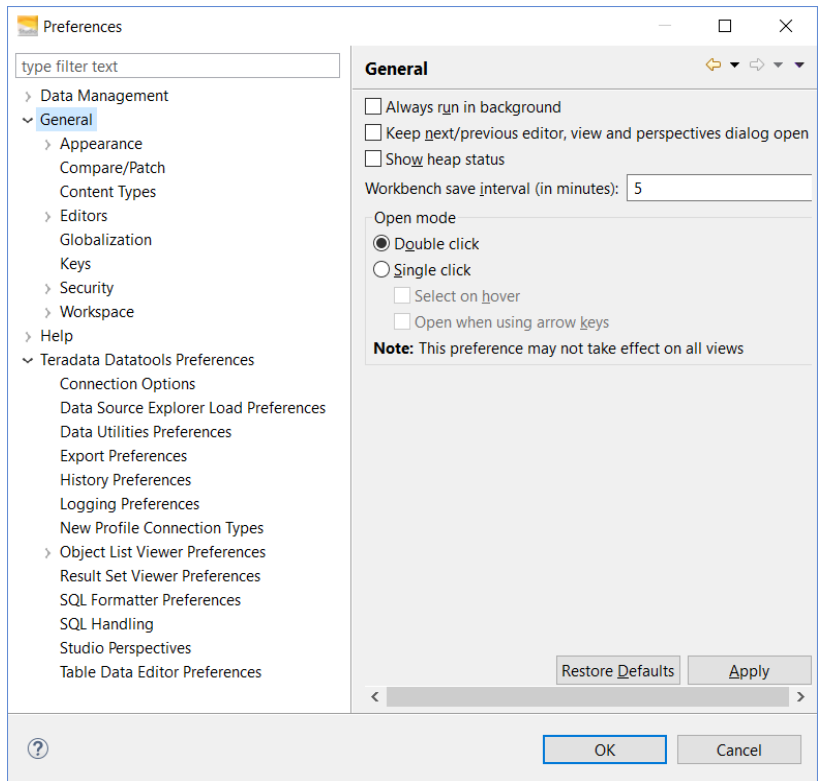
To set general program preferences:

1. Select the menu **Window > Preferences > General**.

Teradata Studio Query Development General Options

General options (Window > Preferences > General):

- **Appearance:** Customize the appearance of the display for colors, fonts, and label decorations.
- **Compare/Patch:** Customize the display when comparing text files.
- **Content Types:** Add content types and file associations.
- **Editors:** Customize editors, file associations, content types, and editor appearance.
- **Globalization:** Customizes global settings for Unicode and text/graphic support.
- **Keys:** Customize the keyboard bindings.
- **Security:** Customize the secure storage.
- **Workspace:** Change the text file encoding for the workspace.



Connecting to Multiple Data Sources

You can connect to multiple data sources via the data connection profiles. Just right mouse click the profile and select connect. Then in the SQL editor you can pick the system you want to run the query against.

The example on the facing page show three connects to three different systems.

Teradata Studio Query Development Connecting to Multiple Data Sources

You can connect to multiple systems via the Database Connections.

Connections have been made to three systems:

- Factory Class
- Factory VM – 16 AMPs
- Factory VM – 8 AMPs

You can select which system to run query against

History includes the Source name for queries.

The screenshot shows the Teradata Studio interface with the following components:

- Data Source Explorer:** Displays a tree of database connections. The 'Factory Class (Teradata v. 16.20.7.01)' connection is expanded, showing sub-connections like 'STUDENT135', 'DBC', and 'SYSDBA'. Other connections listed include 'Factory VM 16.20 - 16 AMPs (Teradata)', 'Factory VM 16.20 - 8 AMPs (DBC) (Teradata)', 'Sandbox (Teradata)', 'Sandbox 16.0 (Teradata)', 'Sandbox 16.10 (Teradata)', and 'Sandbox 16.20 (Teradata)'.
- SQL Editor 1:** Contains a query: `SELECT SUM`. A dropdown menu for 'Connection Profile' is open, showing a list of connections. The 'Factory Class (Teradata)' connection is selected. A red circle highlights this dropdown menu.
- Teradata Result Set Viewer:** Displays a result set with one row and one column: `ordertotal` with the value `124,876,057.9`.
- Teradata SQL History:** A table showing the history of queries executed. The 'Source' column is circled in red. The table has columns: 'Ti...mp', 'Source', 'User ID', and 'Destir'.

Ti...mp	Source	User ID	Destir
1 2018...72	Factory Class	student135	Teradi
2 2018...95	Factory VM 16.20 - 8 AMPs (DBC)	sysdba	Teradi
3 201...884	Factory VM 16.20 - 16 AMPs	dbc	Teradi
4 201...066	Factory Class	student135	Teradi

Importing a LOB into Teradata

To import and/or export LOB (Large Object) files with Teradata Studio you open the **Large Binary Editor** from the **Table Data Editor** by selecting the row that contains large binary data in a table. The **Large Binary Editor** provides a large work area where you can review and modify data, and export and import data.

Teradata Studio Query Development

Importing a LOB into Teradata

The screenshot shows the Teradata Studio interface. At the top, there's a toolbar with various icons. Below it, a table is displayed with the following columns: Document_Num (INTEGER), Document_Name (VARCHAR(100)), Document_Type (VARCHAR(20)), and Document_Blob (BINARY LARGE OBJECT(200000000)). The first row contains the values: 1, TF Student Manual, PDF, and a truncated binary string. A red circle highlights the '...' icon in the Document_Blob column. Below the table, a 'Blob Editor' dialog box is open. It contains a text area with a long binary string. A red circle highlights the 'Import...' button in the dialog box. The dialog box also has 'Export...', 'Clear', 'OK', and 'Cancel' buttons.

1. Right click on table name then select **Teradata > Edit Data...**

2. Fill in the data elements and for the LOB click ... icon. Then click **Import** button and browse to select data file. Click OK

3. Finally click **Commit changes to database** icon.

Selecting from a Table with a LOB

The LOB Disposition dialog appears when the results of running an SQL statement contain CHARACTER LARGE OBJECT (CLOB) or BINARY LARGE OBJECT (BLOB) values, or character or binary values specified to be treated as large objects.

The dialog contains a form for each column of the results that is a large object type. The forms enable you to specify how to handle the values. You can specify to store the values in files. In the case of binary data, you can specify that the values that are images are displayed in the Result View.

In the example on the facing page, the file name was specified as Adobe PDF files.

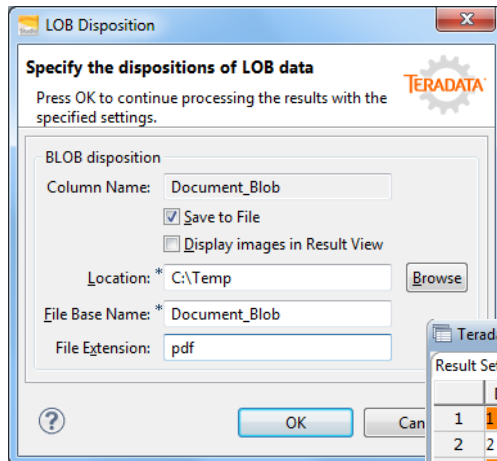
Teradata Studio Query Development

Selecting from a Table with a LOB

With Studio, enter the following query:

```
SELECT * FROM Collaterals.TF ORDER BY 1;
```

The following dialog box is displayed to represent the data files to export the LOBs into. Also specify the "File Extension" as a known file type extension.



LOB Disposition

Specify the dispositions of LOB data
Press OK to continue processing the results with the specified settings.

BLOB disposition

Column Name: Document_Blob

☒ Save to File
☐ Display images in Result View

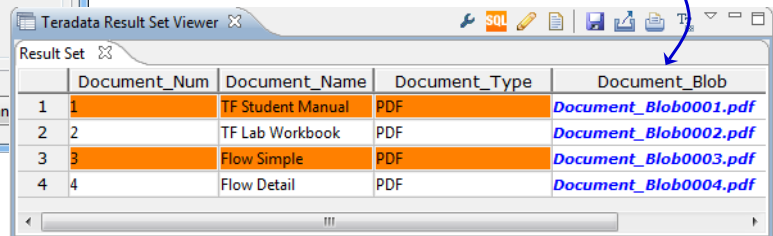
Location: * C:\Temp Browse

File Base Name: * Document_Blob

File Extension: pdf

OK Cancel

The answer set window will include a link to exported data files.



Teradata Result Set Viewer

	Document_Num	Document_Name	Document_Type	Document_Blob
1	1	TF Student Manual	PDF	Document_Blob0001.pdf
2	2	TF Lab Workbook	PDF	Document_Blob0002.pdf
3	3	Flow Simple	PDF	Document_Blob0003.pdf
4	4	Flow Detail	PDF	Document_Blob0004.pdf

Displaying a JPG Large Object

The “Display image in Result View” can be selected to display a JPG file within the result set.

Optionally, the “Save to file” can be selected.

Note: The "Display images in Result View" will not view properly at 1st. You need to make Photo column wider then select all the rows, and finally right mouse click and pick the “Auto Resize row” option.

Teradata Studio Query Development

Displaying a JPG Large Object

```
SELECT * FROM Collaterals.Photos ORDER BY 1;
```

LOB Disposition

Specify the dispositions of LOB data
Press OK to continue processing the results with the specified settings.

BLOB disposition

Column Name: Photo

☐ Save to File

☒ Display images in Result View

Location: * C:\Users\BS186023\Desktop

File Base Name: * Photo

File Extension: .dat

Optionally, the "Display images in Result View" can be selected to display the JPG files within the Result Set.

Note: The "Display images in Result View" will not initially view properly. You need to make Photo column wider, select all the rows, and then right mouse click and pick the "Auto Resize row" option.

Photo_Name	Photo_Type	
		
		

Teradata Studio Summary

The Teradata Studio or Teradata Studio Express tools can be of great value to you. The facing page summarizes some of the key features discussed in this module.

Teradata Studio Summary

Characteristics of both Teradata Studio and Teradata Studio Express include:

- Client-based graphical interface to submit SQL for the Teradata NewSQL Engine.
- Provides a Database Explorer Tree to easily view database objects.
- Supports DDL, DML and DCL commands.
 - SQL Query Builder feature allows for easy creation of SQL statements.
- Provides the retrieval of previously used queries (History).
- Saves information about previous query result sets.
- Provides both import and export capabilities to files on a PC.

Module 7: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 7: Review Questions

1. Which data interface is available with Teradata Studio?
 - a. CLIV2
 - b. JDBC
 - c. ODBC
 - d. Teradata .Net

2. Which piece of query information is not available in the History Window for Teradata Studio?
 - a. User name
 - b. Query band
 - c. Elapsed time
 - d. Data source name
 - e. Number of rows returned

3. What is a technique to execute multiple statements as a multi-statement request in Teradata Studio?

Lab Exercise 7-1

Check your understanding of the concepts discussed in this module by completing the lab exercise as directed by your instructor.

Lab Exercise 7-1

Purpose

In this lab, you will use either Teradata Studio or Teradata Studio Express to define a data source and execute some simple SQL commands.

What you need

Teradata Studio or Teradata Studio Express installed on the laptop or PC.

Tasks

1. If this has not been already defined, create a connection to a Teradata system based on the following:

Complete the dialog box with the following information:

Name – Factory

Description – Teradata Training for your name

Name or IP Address – _____ (supplied by instructor)

Username – _____ (supplied by instructor)

Password – do not fill in your password

Verify the following options are set properly.

Session Character Set – ASCII

Session Mode – Teradata (TMODE value of TERA)

Lab Exercise 7-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercise as directed by your instructor.

Lab Exercise 7-1

2. Connect to the data source you just created and logon with your username and password.
3. Use the Window > Preferences > Teradata Datatools Preferences tab, ensure the following options are set as indicated:
 - Data Source Explorer Load Preferences
 - Check** – Only Databases (but include DBC and myself)
 - Un-Check** – Show Databases and Users in Hierarchical Display
 - Result Set Viewer Preferences
 - Check** – Display alternate Result Set rows in color – choose a color
 - Check** – Display Column Titles rather than Column Names
4. If the Explorer Tree pane is not visible, mouse click > next to your login name and then Databases
 - Find the following databases in the Explorer Tree: AP, DS, PD, Collaterals
5. Using the Explorer Tree, view the table objects in your database.

Lab Exercise 7-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercise as directed by your instructor.

Use the following SQL to determine a count of rows in a table.

```
SELECT COUNT(*) FROM tablename;
```

Step 7 Hint: Your Orders_1Q2019 table should have 8000 rows for just January 2019 orders. If not, check the dates you used in your queries.

Step 8 Hint: Your Orders_1Q2019 table should have 24,000 rows for orders in the first quarter of 2019. If not, check the dates you used in your queries.

Lab Exercise 7-1 (cont.)

6. Using the Query Window, execute the following query.

```
CREATE TABLE Orders_1Q2019 AS Orders_2019 WITH NO DATA;
```

Does the new table object appear in the table object list? ____ If not, "refresh" the database.

7. Execute the following query to populate Orders_1Q2019 with orders for January of 2019.

```
INSERT INTO Orders_1Q2019 SELECT * FROM DS.Orders_2019  
WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How many rows are in Orders_1Q2019? ____

8. Using the History window, recall the query from step #7 and modify it to add orders for February and March of 2019 into Orders_1Q2019.

How many rows are now in Orders_1Q2019? ____

9. Execute the following query by using the drag and drop object feature of the SQL tool you are using.

```
SELECT custid, SUM (ordertotal)  
FROM Orders_1Q2019  
GROUP BY 1  
ORDER BY 1;
```

What is the SUM of the orders (ordertotal) in the 1Q2019 for custid = 21012? ____

Lab Exercise 7-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercise as directed by your instructor.

Use the following SQL to create a view.

```
CREATE VIEW viewname  
AS  
LOCKING ROW FOR ACCESS  
SELECT    column1, column2  
FROM      table_or_view_name  
WHERE      condition;
```

To create a simple macro (without parameters), you can use the following SQL:

```
CREATE      MACRO macroname  
AS          (SELECT * FROM table_or_view_name);;
```

Use the following SQL to execute a simple macro.

```
EXEC macroname;
```

To create a parameterized macro, you can use the following SQL:

```
CREATE      MACRO orderid_m (oid_value INTEGER)  
AS          (SELECT * FROM table_or_view_name  
              WHERE Orderid = :oid_value);;
```

Use the following SQL to execute a parameterized macro.

```
EXEC orderid_m (1781509);
```

```
EXEC orderid_m (1781939);
```

Lab Exercise 7-1 (cont.)

10. Create a view (that utilizes an Access Lock) named "Orders_Jan2019_v" on the Orders_1Q2019 table that includes the following columns and only includes orders for the January of 2019.

orderid, custid, ordertotal, orderdate

SELECT the total count of orders from "Orders_Jan2019_v".

How many orders are available using this view? _____

11. Create a parameterized macro named "Orderid_m" which accepts an orderid as a value, accesses the view, and returns a single order from "Orders_Jan2019_v".

Execute the macro to provide order details for orders 1781509 and 1781939.

What is the ordertotal for orderid #1781509 _____

What is the ordertotal for orderid #1781939? _____

12. (Optional) Use the Collaterals database to access the Photos table to display various JPG files.

Execute the following: `SELECT * FROM Collaterals.Photos ORDER BY 1;`

Note: Set the file type to JPG and check the option "Display as picture in Answerset".

What is shown in photo #7? _____

Notes



Teradata System Architecture

After completing this module, you will be able to:

- **Identify characteristics of various components.**
- **Define a clique and its purpose.**
- **Explain the concept of vproc migration.**
- **Describe the purpose of the BYNET.**
- **Specify the difference between a TPA and HSN node.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata Database Releases	8-4
Teradata Version 1	8-4
Teradata Version 2	8-4
Teradata Database Architecture	8-6
Teradata Database – Multiple Nodes	8-8
MPP Systems	8-10
Example of 2+1 Node Teradata System	8-12
Teradata Cliques.....	8-14
BYNET	8-16
BYNET Communication Protocols	8-16
Vproc Inter-Process Communication.....	8-18
Fabric Architecture Enables Flexibility	8-20
BYNET 1U Switches.....	8-22
Larger BYNET Configurations.....	8-24
BYNET Connection Rules.....	8-24
Examples of Teradata Database Systems.....	8-26
IntelliFlex™ 2.0 Cabinet Example	8-28
What makes Teradata's MPP Platforms Special?.....	8-30
Summary	8-32
Module 8: Review Exercises.....	8-34

Teradata Database Releases

The facing page identifies various Teradata releases that have been available since 1984. This page identifies some historical information about Teradata Version 1 systems.

Teradata Version 1

Teradata Database Version 1 platforms were first available to customers in 1984. This first platform was the original Database Computer, **DBC/1012** from Teradata. In 1991, the NCR Corporation introduced the **3600**. Both of these systems are older technologies and both of these systems used a proprietary 16-bit operating system known as **TOS** (Teradata Operating System). All AMPs and PEs were dedicated hardware processors that were connected together using a message-passing layer known as the **Ynet**.

Both platforms supported channel-attached (Bus and Tag) and LAN-attached host systems.

DBC/1012 Architecture – this system was a dedicated relational database management system. Two specific components of the DBC/1012 were the IFP and the COP, both of which were effectively hardware Parsing Engines. The acronyms IFP and COP still appear in the Data Dictionary even today.

Interface Processor (IFP) – the IFP was the Parsing Engine of the DBC/1012 for channel-attached systems. For current systems (e.g., 5550), PEs that are assigned to a channel are identified in the Data Dictionary with a type of IFP.

Communications Processor (COP) – the **COP** is the Parsing Engine of the DBC/1012 for network-attached systems. For current systems (e.g., 5550), PEs that are assigned to a LAN (or network) are identified in the Data Dictionary with a type of COP.

3600 Architecture – this system included hardware AMPs and PEs as well as multipurpose Application Processors executing UNIX MP-RAS. Application Processors (APs) also provided the channel and LAN connectivity. UNIX applications were executed on APs while Teradata was executed on PEs and AMPs. All processing units were connected via the Ynet.

Teradata Version 2

Starting with **Teradata Database Version 2 Release 1** (available in January 1996), the Teradata Database became an open database system. No longer was Teradata software dependent on a proprietary Operating System (TOS) and proprietary hardware. Rather, it was an application that initially executed ran under UNIX.

By porting the Teradata Database to a general-purpose operating system platform, a variety of processing options against the Teradata database became possible, all within a single system. OLTP (as well as OLCP and OLAP) applications became processing options in addition to standard DSS.

Teradata Database Releases

Teradata Releases

Version 1 Release 1
Release 2
Release 3
Release 4
Release 5

Version 2 Release 1
Release 2
Release 3
Release 4
Release 5
Release 6

Teradata 12.0
Teradata 13.0
Teradata 13.10
Teradata 14.0
Teradata 14.10
Teradata 15.0
Teradata 15.10
Teradata 16.00 (03/2017)
Teradata 16.10 (08/2017)
Teradata 16.20 (02/2018)

Version 1 was a combination of hardware and software.

Each AMP and PE consisted of **hardware and software** that had to be purchased, installed, and configured.

V1 Platforms

DBC/1012
3600

Year Available

1984
1991

Version 2 was an implementation of Teradata PEs and AMPs as software vprocs (virtual processors).

Teradata is effectively a database application that executes under an operating system.

Active EDW Platforms

5100
↓
5650 (requires 12.0 or later)
6680
6690
6700
6750
6800
IntelliFlex™
IntelliBase™, IntelliFlex™ 2.0
IntelliFlex™ 2.1

Year Available

1996 (UNIX MP-RAS)
↓
2010
2011
2012
2013
2014
2015
2016
2017
2018

Teradata Database Architecture

Teradata 16.20 is effectively an application that runs under an operating system (SUSE Linux - SLES 11).

- SLES - SUSE Linux Enterprise Server
- SUSE - Software und System Entwicklung (German name which means Software and Systems Development)

PDE software provides Teradata Database software the capability to under a specific operating system. Parallel Database Extensions (PDE) software is an interface layer on top of the operating system (Linux, UNIX MP-RAS, or Windows Server 2003). PDE provides the Teradata Database with the ability to:

- Run the Teradata Database in a parallel environment
- Execute vprocs
- Apply a flexible priority scheduler to Teradata Database sessions
- Debug the operating system kernel and the Teradata Database using resident debugging facilities

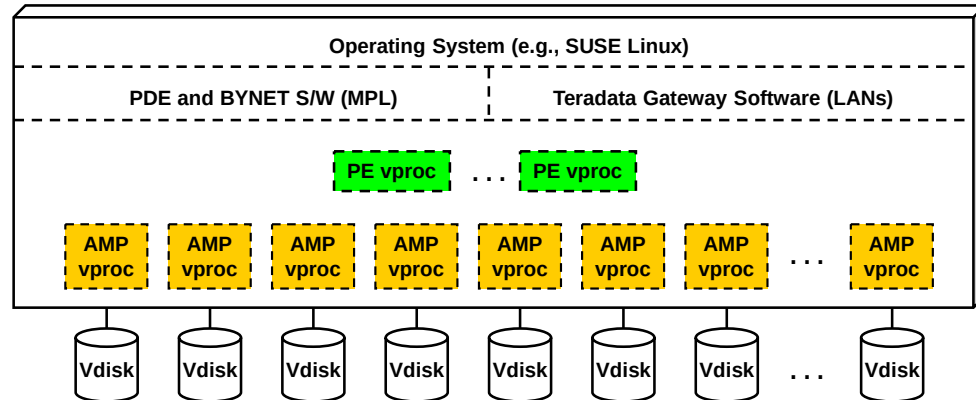
AMPs and PEs are implemented as “**virtual processors - vprocs**”. They run under the control of PDE and their number is software configurable.

AMPs are associated with “**virtual disks – vdisks**” which are associated with logical units (LUNs) within a disk array.

The versatility of Teradata Database is based on virtual processors (vprocs) that eliminate dependency on specialized physical processors. Vprocs are a set of software processes that run on a node under Teradata Parallel Database Extensions (PDE) within the multitasking environment of the operating system.

Teradata Database Architecture

Teradata Processing Node (e.g., IntelliFlex™ node)



- Teradata executes on a 64-bit operating system (e.g., SUSE Linux – SLES 11).
 - Utilizes general purpose SMP/MPP hardware.
 - **Parallel Database Extensions (PDE) is unique per OS that Teradata is supported on.**
- AMPs and PEs are implemented as virtual processors (Vprocs).
- **“Shared Nothing” Architecture – each AMP has its own memory, manages its own disk space, and executes independently of other AMPs.**

Teradata Database – Multiple Nodes

A customer may choose to implement Teradata on a small, single node SMP system for smaller database requirements and to eventually grow incrementally to a multiple terabyte system. A single-node SMP platform may also be used as low cost development systems.

Under the **single-node (SMP)** version of Teradata, PE and AMP vproc still communicate with each other via PDE and BYNET software. All vprocs share the resources of CPUs and memory within the SMP node.

As a customer's Teradata database needs grow, additional nodes will probably be needed. A multi-node system running the Teradata Database is referred to as an **MPP (Massive Parallel Processing)** system.

The Teradata Database application is considered a **Trusted Parallel Application (TPA)**. The Teradata Database is the only TPA application available at this time.

Nodes in a configuration may or may not be connected to the BYNET. Examples include:

- TPA (Trusted Parallel Application) node – executes Teradata Database software.
- HSN (Hot Standby Node) – is a spare node in the clique (not running Teradata) used in event of a node failure.
- Non-TPA (NOTPA) node – is an application node that does not execute Teradata Database software and connects to the BYNET.
- TMS node – is an application node that does not execute Teradata Database software and connects to the BYNET or the network (via Ethernet).

The Teradata Database can use hot spare nodes to improve availability and maintain performance levels in the event of a node failure. A hot standby node (HSN) is a node that:

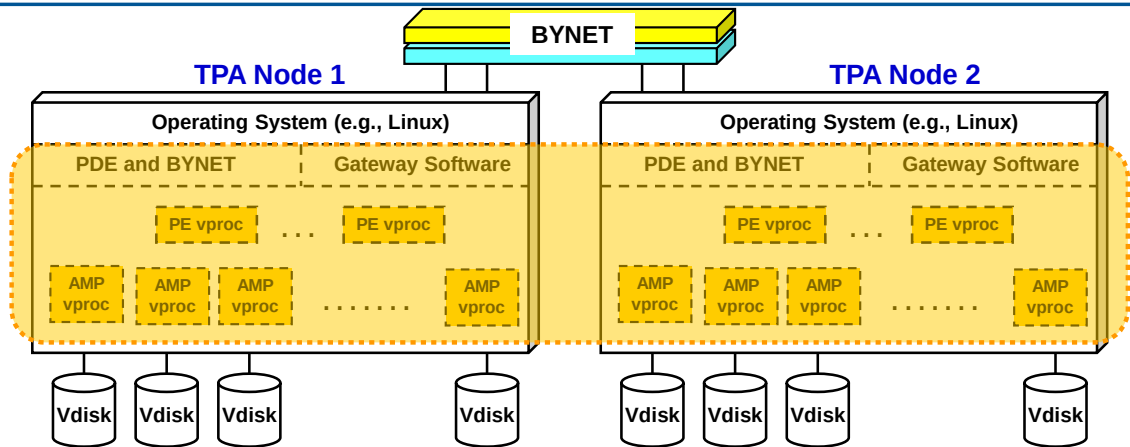
- is a member of a clique
- does not normally participate in Teradata Database operations
- is automatically activated to participate in Teradata Database operations to compensate for the loss of a node in the clique

Configuring a hot standby node can eliminate the system-wide performance degradation associated with the loss of a node. A hot standby node is added to each clique in the system. When a node fails, all AMPs and all LAN-attached PEs on the failed node migrate to the node designated as the hot standby node. The hot standby node becomes a production node. When a failed node returns to service, it becomes the new hot standby node.

Configuring hot standby nodes eliminates:

- Restarts that are required to bring a failed node back into service.
- Degraded service when vprocs have migrated to other nodes in a clique.

Teradata Database – Multiple Nodes



Teradata is a linearly expandable database – as your database grows, additional nodes may be added – effectively becoming an MPP (Massive Parallel Processing) system.

- Teradata software makes a multi-node system look like a single-Teradata system.

Examples of node types and acronyms:

- **TPA (Trusted Parallel Application) node** – executes Teradata Database software (BYNET connected)
- **HSN (Hot Standby Node)** – spare node (BYNET connected) in a clique used in event of node failure.
- **TMS (Teradata Managed Server) node** – application node that is not running Teradata Database software and may connect to the system either by network (Ethernet) or BYNET.

MPP Systems

When multiple SMP nodes (simply referred to as nodes) are connected together to form a larger configuration, we refer to this as an **MPP** (Massively Parallel Processing) system.

The connecting layer (or system interconnect) is called the **BYNET**. The BYNET is a combination of hardware and software that allows multiple vprocs on multiple nodes to communicate with each other.

Because Teradata is a **linearly expandable** database system, as additional nodes and vprocs are added to the system, the system capacity scales in a linear fashion.

BYNET software can support up to 4096 nodes, however latest IntelliFlex™ systems are currently limited to 2048 nodes. The older BYNET v4 is/was limited to 1024 nodes and BYNET v5 (over Infiniband) is limited to 2048 nodes.

Acronyms that may appear in diagrams throughout this course:

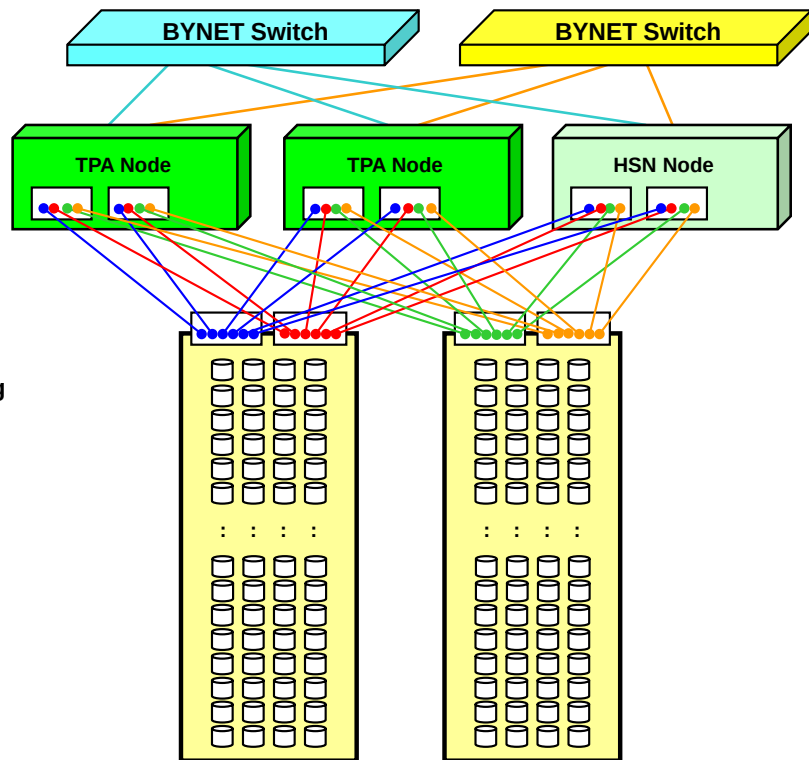
PCI – Peripheral Component Interconnect
EISA – Extended Industry Standard Architecture
PBCA – PCI Bus Channel Adapter
PBSA – PCI Bus ESCON Adapter
EBCA – EISA Bus Channel Adapter

MPP Systems

The **BYNET** consists of redundant switches that interconnect multiple nodes.

Multiple nodes make up Massively Parallel Processing (MPP) system.

A clique is a group of nodes connected to and sharing the same storage.



Example of 2+1 Node Teradata System

The facing page contains an illustration of a simple **three-node (2+1) Teradata Database system**. Each node has its own Vprocs to manage, while communication among the Vprocs takes place via the BYNETs. The PEs are not shown in this example.

Each node is an SMP from a configuration standpoint. Each node has its own CPUs, memory, Linux and PDE software, Teradata Database software, BYNET software, and access to one or more disk arrays.

Nodes are the building blocks of MPP systems. A system size is typically expressed in terms of number of nodes.

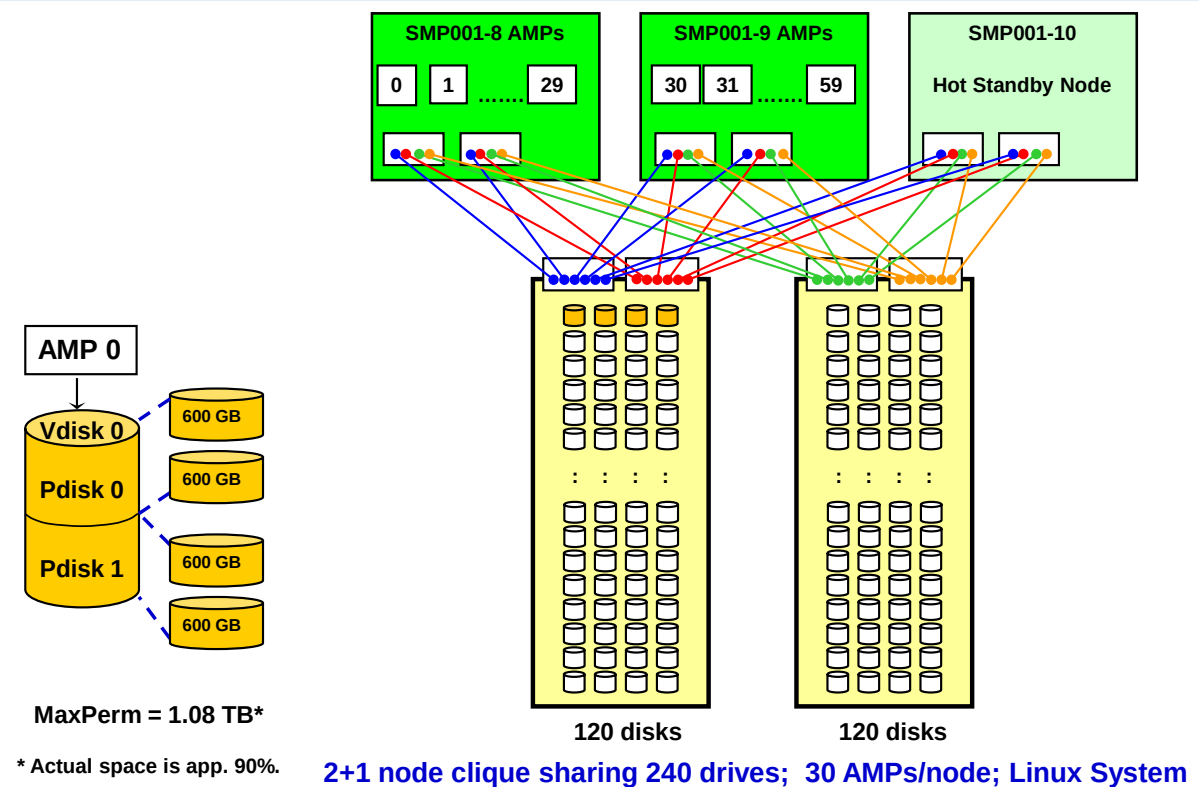
AMPs provide access to user data stored within tables that are physically stored on disk arrays.

Each AMP is associated with a Vdisk. Each AMP sees its Vdisk as a single disk. Teradata (AMP software) organizes its data on its disk space (Vdisk) using a Teradata “File System” structure.

A Vdisk may be actually composed of multiple Pdisks - Physical disk. A Pdisk is assigned to physical drives in a disk array.

The facing page contains an example of a 3-node (2+1) clique sharing two disk arrays.

Example of 2+1 Node Teradata System

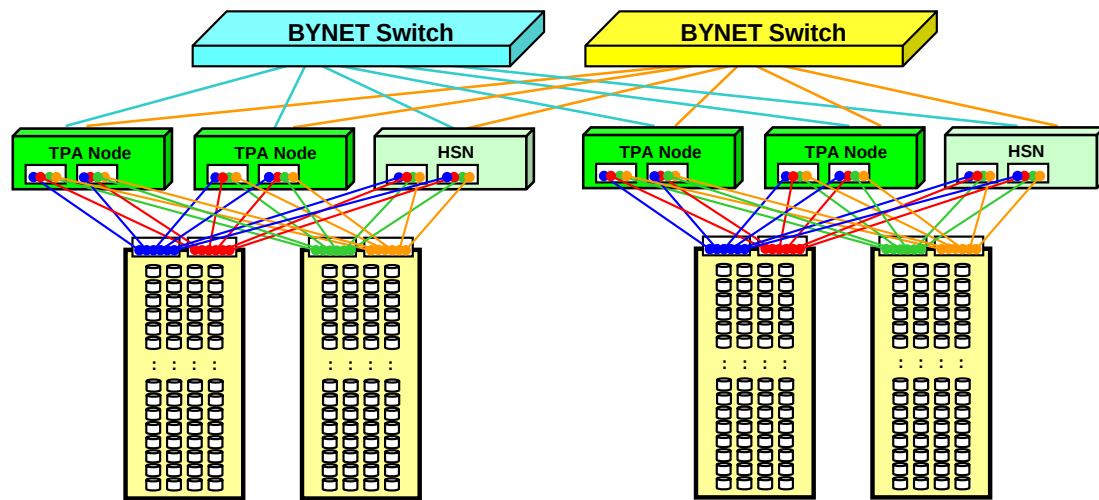


Teradata Cliques

A **clique** is a set of Teradata nodes that share a common set of disk arrays. In the event of node failure, all vprocs can migrate to another available node in the clique. All nodes in the clique must have access to the same disk arrays.

The illustration on the facing page shows a 6-node system consisting of two cliques, each containing three nodes. Because all disk arrays are available to all nodes in the clique, the AMP vprocs will still have access to the rows they are responsible for.

Teradata Cliques



- A clique is a defined set of nodes that share a common set of disk arrays.
- All nodes in a clique must be able to access all Vdisks for all AMPs in the clique.
- A clique provides protection from a [node](#) failure.
- If a node fails, all vprocs will migrate to the remaining nodes in the clique (Vproc Migration) or to a Hot Standby Node (HSN).

BYNET

There are two physical **BYNETs**, BYNET 0 and BYNET 1. Both are fully operational and provide fault tolerance in the event of a BYNET failure. The BYNETs automatically handle load balancing and message routing. BYNET reconfiguration and message rerouting in the event of a component failure is also handled transparently to the application.

BYNET Communication Protocols

Using communication-switching techniques, the **BYNET** allows for **point-to-point**, **multicast**, and **broadcast** communications among the nodes, thus supporting a monumental increase in throughput in very large databases. This technology allows Teradata users to grow massively parallel databases without fear of a communications bottleneck for any database operations.

Although the BYNET software supports the **multi-cast** protocol, Teradata only uses this protocol with Group AMPs operations. Teradata software will use the **point-to-point** protocol whenever possible. When an all-AMP operation is needed, Teradata software uses the **broadcast** protocol to send messages to the different SMPs.

The BYNET is linearly scalable for point-to-point communications. Scalability comes from the fact that multiple point-to-point circuits can be established concurrently. With the addition of another node, more circuits can be established concurrently.

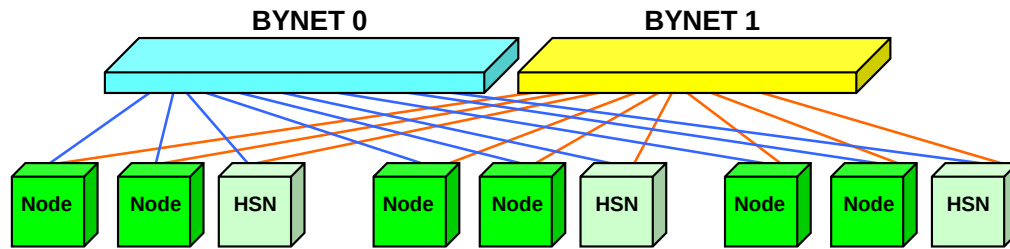
BYNET v5 is implemented using Infiniband technology and can up to 10x faster than BYNET v4.

BYNET v6 is the latest generation of the BYNET software and utilizes the latest (faster) generation of Infiniband adapters and switches.

Older implementations include:

- BYNET v1 – 10 MB x 2 (bi-directional) x 2 BYNETS = 40 MB per node.
- BYNET v2 – 60 MB x 2 (bi-directional) x 2 BYNETs = 240 MB per node
- BYNET v3 – 93.75 MB x 2 (bi-directional) x 2 BYNETs = 375 MB per node
- BYNET v4 – 240 MB x 2 (bi-directional) x 2 BYNETs = 960 MB per node

BYNET



The BYNET is a dual redundant, bi-directional interconnect network.

- All nodes are connected to both BYNETs. This example shows three (2+1) cliques.

BYNET Features:

- Enables multiple nodes to communicate with each other.
- Automatic load balancing of message traffic.
- Automatic reconfiguration after fault detection.
- Fully operational dual BYNETs provide fault tolerance.
- Even though there are two physical BYNETs to provide redundancy and bandwidth, the Teradata Database and TCP/IP software only see a single network.
- BYNET software automatically uses point-to-point, multicast, and broadcast protocols to communicate between vprocs and nodes.

Vproc Inter-Process Communication

The “message passing layer” is a combination of two pieces of software and hardware— the PDE and the BYNET device drivers and software and the BYNET hardware.

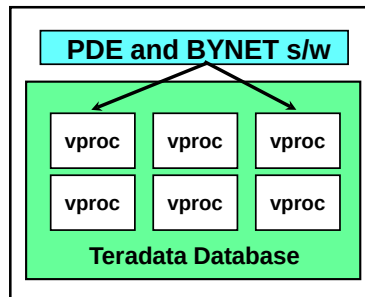
Communication among vprocs in an **MPP system** may be either inter-node or intra-node. When vprocs within the same node communicate they do not require the physical transport services of the **BYNET**. However, they do use the highest levels of the BYNET software even though the messages themselves do not leave the node.

When vprocs must communicate across nodes, they must use the physical transport services of the BYNET requiring movement of the data. Any broadcast messages, for example, will go out to the BYNET, even for the AMPs and PEs that are in the same node.

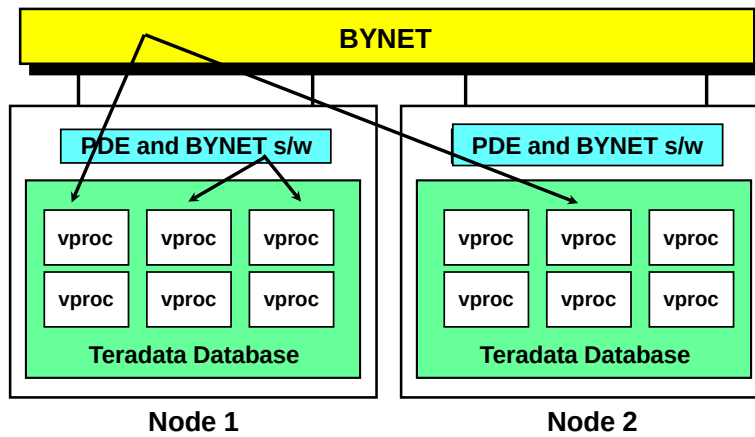
Communication among vprocs in a single SMP system occurs with the PDE and BYNET software, even though a physical BYNET does not exist in a single-node system.

Vproc Inter-Process Communication

Single-Node System



MPP Systems



Fabric Architecture Enables Flexibility

The fabric-based architecture for nodes and storage provides numerous benefits. As you see, nodes and storage are now independently connected to the InfiniBand fabric.

- Improved Packaging Flexibility
 - Reduced and simplified cabling
 - Easier customer re-rack
- Large Cliques Reduce HW Costs
 - Large cliques (up to 12+1) require up to 75% fewer Hot Standby Nodes
 - Additional hot standby nodes available for added protection, if desired
- More Dense Cabinet Footprint
 - 50% more CPU performance; up to 200% more memory;
 - 13% more hot data space per cabinet; up to 100% more amps and higher bandwidth

A Teradata IntelliFlex™ 2.0 Database node consists of a Dell chassis with:

- Teradata IntelliFlex™ 2.0 nodes (1U) utilize Intel® Xeon® Eighteen-Core 2.3 GHz processors.
- 512 GB memory using 32 GB RDIMMs and up to 1 TB of memory using 64 GB RDIMMs
 - Internal Disk drives for Linux OS, Teradata SW, and dump processing
 - Eight additional (optional) SAS drive bays available for customer use
- Onboard connectivity to the server management network
- BYNET InfiniBand network adapter
- Optional network connection to managed BAR servers
- Optional network communication adapter

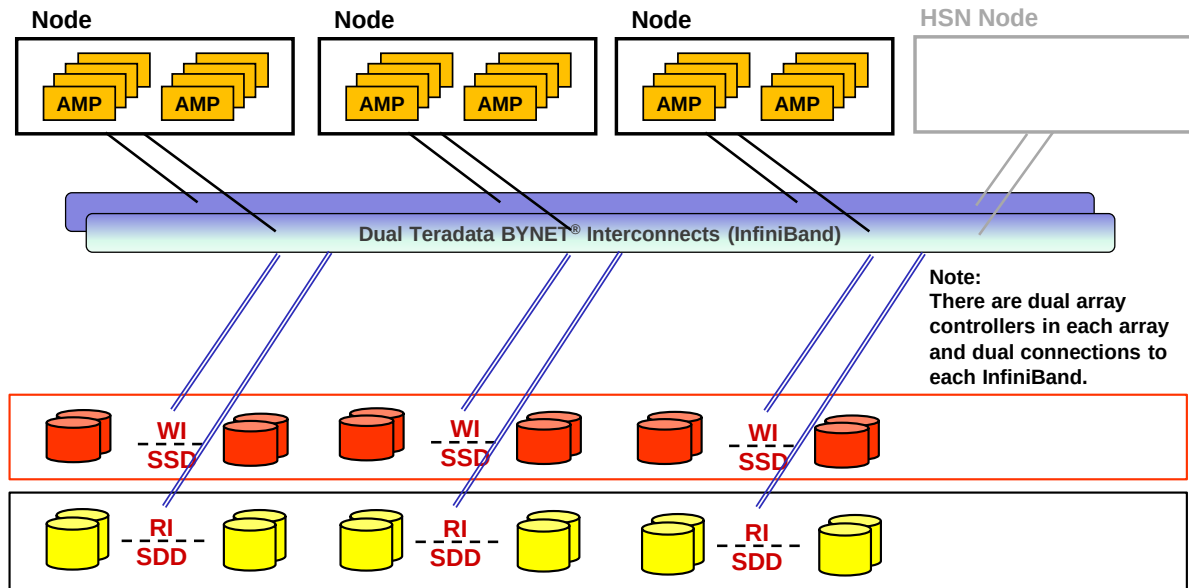
Teradata IntelliFlex™ Disk Arrays have these characteristics:

IntelliFlex™ 2.0 Clique configurations use SSD-only arrays for disk storage.

- Each Fabric-Attached Write-Intensive (WI) SSD disk array in a Teradata IntelliFlex™ cabinet contains one 5350 drive enclosure:
 - Each drive enclosure contains up to 24 drive slots.
 - Each SSD storage drive is 1.6 TB (2.5 in.).
 - Two available SSD slots for global hot spares (GHSs) per array.
- Each Fabric-Attached Read-Intensive (RI) SSD disk array in a Teradata IntelliFlex™ cabinet contains one DE224C drive enclosure:
 - Each drive enclosure contains up to 24 drive slots.
 - Each SSD storage drive is 6.4 TB (2.5 in.) and is a FDE storage drive.
 - GHS drives are not supported for the RI storage array.

Fabric Architecture Enables Flexibility

- The InfiniBand fabric carries both BYNET® and storage traffic.
- Fabric attached storage/resources allows for larger cliques. IntelliFlex™ supports cliques up to 13 nodes (12 TPA + 1 HSN).



BYNET 1U Switches

The facing page contains of an example of a BYNET v5 (InfiniBand 36-port) switches.

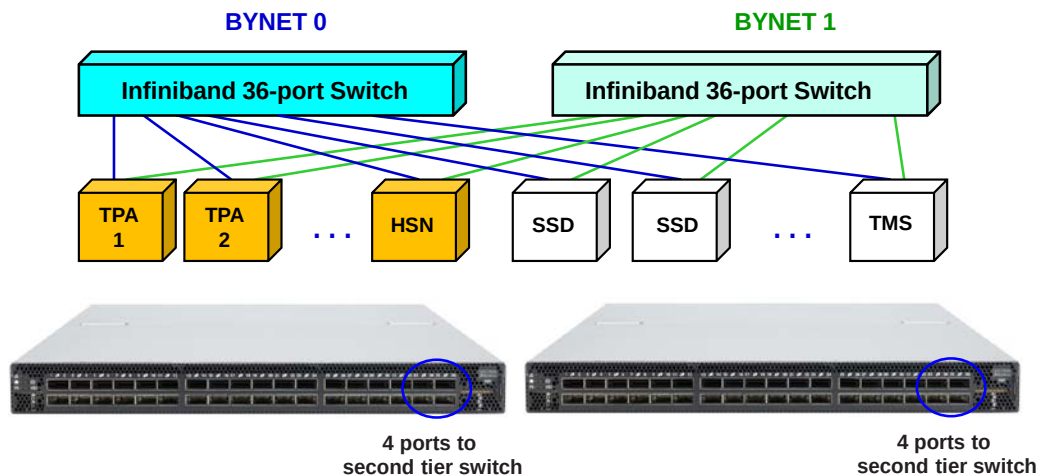
Examples of 1U BYNET switches are listed below.

- **BYNET 32 Switch (BYA32S)** – this older BYNET v4 switch supports up to 16 TPA nodes and 16 non-TPA nodes (e.g., HSN, NOTPA, etc.).
- **36-port InfiniBand Switch** – this 1U chassis is used in processor cabinet to connect up to 32 TPA nodes, HSN nodes, storage arrays, etc. Four of the 36 ports are used for a possible connection to a tier 2 switch.

BYNET 1U Switches

Teradata uses Infiniband 36 port (1U) switches in a processor rack for node connectivity.

- With Teradata IntelliFlex™ systems, **TPA nodes as well as disk array storage (e.g., SSD arrays), and HSN** are connected to a first tier switch. Optionally, TMS nodes and Channel nodes may also be connected to the first tier switch.
- 4 ports are reserved for connectivity to second tier Infiniband switches for larger configurations.



Larger BYNET Configurations

For larger system configurations, a two-tier BYNET as shown on the facing page can be configured.

Each processing node, PE node, or channel server contains a dual port InfiniBand adapter that connects to both InfiniBand switches. A TMS node may contain a dual InfiniBand adapter that connects to the switches.

Individual optical InfiniBand cables are used to connect nodes, disk arrays, and TMS servers in expansion cabinets to the InfiniBand switches in the base cabinet or an InfiniBand switch cabinet (a.k.a., Platform Framework Cabinet) depending on the configuration.

If the system is included in a UDA environment, some ports on the InfiniBand switch are used for UDA connections.

BYNET Connection Rules

The following BYNET connection rules apply to IntelliFlex™ and IntelliBase™ systems:

- TD Database Nodes, Channel Nodes, PE Nodes, and Storage Arrays are all connected to the network to a Tier 1 InfiniBand switch.
- All TD Database Nodes, HSNs, and Storage Arrays that are part of the same clique must connect to the same Tier 1 InfiniBand switch.
- Multiple cliques can share a 1st tier switch depending upon clique size.
- No storage IO traffic goes to a tier 2 switch.
- Intra-cabinet connections use copper cables, inter-rack connections use fiber cables.

Terminology used with InfiniBand switches:

- Tier 1 switches – connect nodes and optionally storage at the clique level
- Tier 2 switches – connect tier 1 switches together for larger configurations

Terminology used with InfiniBand bandwidths:

- FDR10 – Fourteen Data Rate, 40 Gb bandwidth
- FDR – Fourteen Data Rate, 56 Gb bandwidth
- EDR – Enhanced Data Rate, 100 Gb bandwidth

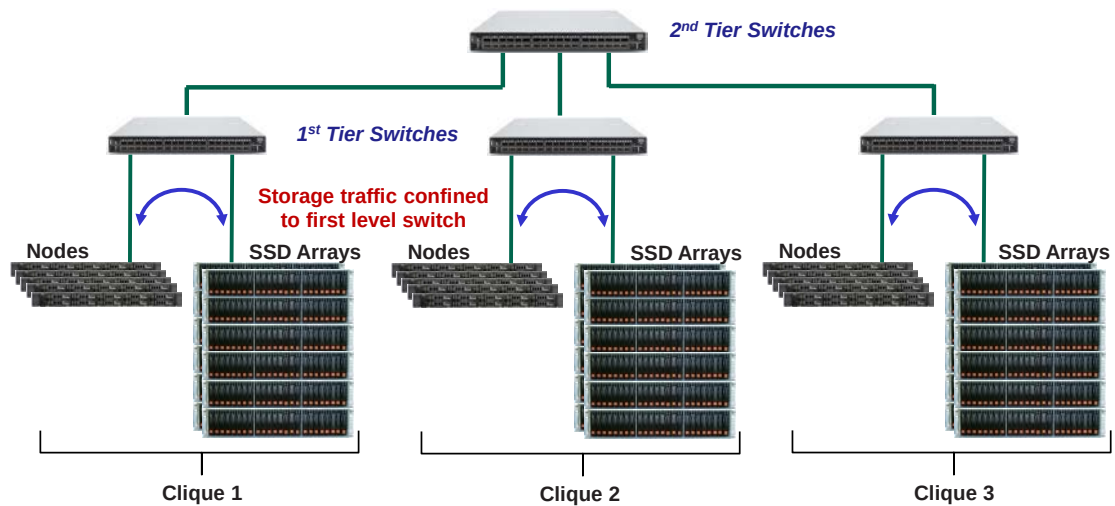
InfiniBand bandwidth for IntelliFlex™ generations:

- BYNET v5 – IntelliFlex™ 1.0 – speed is FDR10 (40Gb/sec) within the clique level (tier 1 switches) and FDR10 (40Gb/sec) with the tier 2 switches
- BYNET v6 – IntelliFlex™ 1.1 and 2.0 – speed is FDR (56Gb/sec) at the clique level (tier 1 switches) and EDR (100Gb) with the tier 2 switches

Larger BYNET Configurations

A two-tier BYNET switch topology enables larger system configurations by using a combination of 36-port and optionally larger switches.

- All of the nodes and arrays (maybe 1 or more cliques) are connected to a pair of 1st -tier switches.
- The Teradata IntelliFlex™ platform can scale up to 576 nodes with BYNET using multiple 36-port InfiniBand switches, and Star Satellite topology.
- The Teradata IntelliFlex™ platform can scale to 864 nodes when 216 port switches are used.



Examples of Teradata Database Systems

The facing page identifies various SMP servers and MPP systems that are supported for the Teradata Database.

The following dates indicate when these systems were generally available to customers (GCA – General Customer Availability).

– 5100M	January, 1996 (not described in this course)
– 4700/5150	January, 1998 (not described in this course)
– 4800/5200	April, 1999
– 4850/5250	June, 2000
– 4851/4855/5251/5255	July, 2001
– 4900/5300	March, 2002
– 4950/5350	December, 2002
– 4980/5380	August, 2003
– 5400E/5400H	March, 2005
– 5450E/5450H	April, 2006
– 5500E/5500C/5500H	March, 2007
– 2500/5550H	January, 2008
– 2550	October, 2008
– 1550	December, 2008
– 2555/5555C/H	March, 2009
– 1600/5600C/H	February, 2010
– 2650/5650C/H	July, 2010 (Internal release; Official release Oct 2010)
– 6650C/H, 6680	2011
– 6690	2012
– 1700/2700/6700	2013
– 2750/6750	2014
– 1800/2800/6800	2015 (1800 and 6800 – October, 2015)
– IntelliFlex™	June, 2016
– 2850	October, 2016
– IntelliFlex™ 2.0	May, 2017
– IntelliBase™	July, 2017
– IntelliFlex™ 2.1	May, 2018

Examples of Teradata Database Systems

Examples of systems available with the Teradata Database include:

Teradata Enterprise Platforms

- | | |
|---------------------|--|
| 1700/1800 | – Integrated Big Data appliance platform |
| 2800/2850 | – Data Warehouse appliances |
| 6750/6800 | – Active Enterprise Data Warehouse systems (with SSD drives) |
| IntelliBase™ | – Teradata system with integrated node storage |
| IntelliFlex™ | – Teradata system with fabric connected nodes/storage |

The basic building block of any system is the processing node.

Common characteristics of these systems:

- MPP systems that use the BYNET interconnect
- Single point of operational control – SWS
- Rack-based systems – each technology is encapsulated in its own chassis

Key differences:

- Speed and capacity of processing nodes and systems
- Cabinet architecture
- BYNET interface cards, switches and speeds
- Ratios of Nodes/Disks/Memory and Workload Management capabilities

IntelliFlex™ 2.0 Cabinet Example

The facing page illustrates an example of a Teradata IntelliFlex™ 2.0 11+1 cabinet configuration.

In this example, there are 12 SSD disk arrays.

In this example, one node in the clique is configured as a hot standby node. This eliminates the degradation of database performance in the event of a node failure in the clique. Tasks assigned to the failed node are completely redirected to the hot standby node.

Older systems such as the 54xx, 55xx, 56xx, and 66xx systems also utilized rack-based cabinets. The rack was initially designed for the 54xx systems and was improved with later systems such as 55xx and 56xx systems. The rack was upgraded for 66xx systems and upgraded again for 67xx systems.

The 56xx through 66xx systems used a 40U rack where a U is a unit of measure of height of 1.75" or 4.445 cm.

The approximate external dimensions of this 66xx rack or cabinet are:

Height – 77"

Width – 24" (inside rails are 19" apart and this is often referred to as a 19" wide rack)

Depth – 44"

The newer 1700/27xx/67xx and 1800/2800/6800 and IntelliFlex™ systems use a 42U rack which has a completely new appearance from the previous cabinet generation.

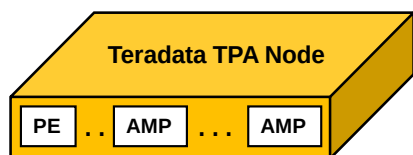
The approximate external dimensions of this 1800/2800/6800/IntelliFlex™ rack or cabinet are:

Height – 80.5"

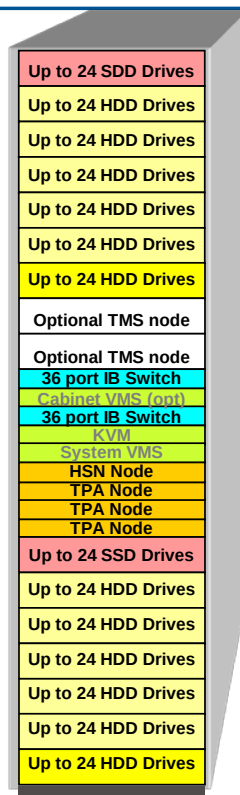
Width – 24" (inside rails are 19" apart and this is often referred to as a 19" wide rack)

Depth – 47"

IntelliFlex™ 2.0 Cabinet Example



TPA – Trusted Parallel Application Node
 HSN – Hot Standby Node



Teradata IntelliFlex™

What makes Teradata's MPP Platforms Special?

The facing page lists the major features of Teradata's MPP systems.

Acronyms:

PUT – Parallel Upgrade Tool

AWS – Administration Workstation

SWS – Service Workstation – utilizes Server Management Web Services (SMWeb) to provide a single point of operational control.

Teradata Unity turns a Teradata multi-system environment into an orchestrated ecosystem, giving users transparent access to data warehouse resources and making IT administration little to no additional effort over managing a single Teradata system environment.

Teradata Unity is a unique and powerful portfolio of multi-system enabling products. It coordinates and orchestrates multiple Teradata systems; presenting a unified view to users and administrators. Teradata Unity consists of these products:

Unity Director

- Query routing
- Database synchronization
- SQL-based data updates

Unity Loader

- High-volume, intelligent data loading to multiple Teradata systems

Unity Data Mover

- Data movement between Teradata systems

Unity Ecosystem Manager

- End-to-end monitoring across data warehouse components

Unity Source Link (TD 14.10)

- SQL data access from non-Teradata databases

Unity Source Link requires Teradata Database 14.10. It works similar to Oracle DB Links. It allows a Teradata query to reach out to an Oracle database and retrieve data which is then handed back to the Teradata query as an input to continue query processing. This is a new and powerful capability which will please our customers. Customers migrating applications from Oracle have often included database links in their applications to get data from Oracle operational systems or data marts. Unity Source Link will simplify the migration for these customers.

What Makes Teradata's MPP Platforms Special?

Key features of Teradata's MPP systems include:

- **Teradata Database software** – allows the Teradata Database to execute on multiple nodes and act as a single instance.
- **Infiniband (BYNET) Interconnect** – provides a high speed network (minimum of 56Gb/sec.) for nodes and optionally storage.
- **Operating system software** (e.g., Linux) for a node is only aware of the resources within the node and only has to manage those resources.
- **SWS (Service Workstation)** – single point of operational control and scalable server management.
- **PUT (Parallel Upgrade Tool)** – simplifies installation/upgrade of software across many nodes.
- **Teradata UNITY** – turns a multi-system environment into an orchestrated ecosystem.
- **Redundant (availability) components.** Examples include:
 - Hot Standby Nodes
 - Two BYNETs
 - Two Disk Array Controllers within a Disk Array
 - Dual AC capability for increased availability
 - N+1 Power Supplies within a processing node and disk arrays

Summary

The facing page summarizes the key points and concepts discussed in this module.

Summary

- Teradata Database is a **software implementation** of Teradata.
 - AMPs and PEs are implemented as virtual processors (Vprocs).
- The Teradata Database utilizes a “**Shared Nothing**” Architecture – each AMP has its own memory and manages its own disk space.
 - **Teradata is called a Trusted Parallel Application (TPA).**
- Multiple nodes may be configured to provide a Massively Parallel Processing (MPP) system.
- A clique is a defined set of nodes that share a common set of disk arrays.
- The Teradata Database is a linearly expandable RDBMS – as your database grows, additional nodes may be added.

Module 8: Review Exercises

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 8: Review Questions

Complete the following.

1. Each AMP has its own memory and manages its own disk space and executes independently of other AMPs. This is referred to as a _____ architecture.
2. The software component that allows the Teradata Database to execute in different operating system environments is the _____.
3. A physical message passing interconnect is called the _____.
4. A clique provides protection from a _____ failure.
5. If a node fails, all vprocs will migrate to the remaining nodes in the clique. This feature is referred to as _____.
6. The _____ provides a single point of operational control for Teradata MPP systems.
7. A _____ node is part of a system configuration, is connected to the BYNET, and executes the Teradata Database software.
8. A _____ node is part of a system configuration, may connect to the BYNET, and is used to execute application software other than Teradata Database software.
9. A _____ node is part of a system configuration, connects to the BYNET, and is used as a spare node in the event of a node failure.

Notes



Data Protection

After completing this module, you will be able to:

- **Explain the concept of FALLBACK tables.**
- **List the types and levels of locking provided by Teradata.**
- **Describe the Recovery, Transient and Permanent Journals and their function.**
- **Identify a purpose for the WAL Depot and the WAL Log.**
- **Have a high level understanding of Teradata Unity**
- **List the utilities available for archive and recovery.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Data Protection Features	9-4
Disk Arrays	9-6
RAID Technologies	9-8
RAID 1 – Mirroring	9-10
RAID 10 – Striped Mirroring.....	9-10
RAID 1 Summary	9-12
RAID 6 – Data Parity Protection with 2 Parity Drives	9-14
Cliques	9-16
Teradata Vproc Migration.....	9-18
Hot Standby Nodes (HSN).....	9-20
Performance Degradation with Node Failure	9-22
Restarts	9-22
Fallback	9-24
Fallback Clusters	9-26
Fallback and RAID Protection	9-28
Global Hot Spare Drives	9-28
Fallback and RAID 1 Example	9-30
Fallback and RAID 1 Example (cont.).....	9-32
Fallback and RAID 1 Example (cont.).....	9-34
Fallback and RAID 1 Example (cont.).....	9-36
Fallback and RAID 1 Example (cont.).....	9-38
Fallback vs. non-Fallback Tables Summary	9-40
Clusters and Cliques.....	9-42
Locks	9-44
Locking Modifier	9-46
ACCESS.....	9-46
NOWAIT.....	9-46
LOAD COMMITTED	9-46
Rules of Locking.....	9-48
Access Locks.....	9-50
WAL – Write-Ahead Logging	9-52
WAL Concepts.....	9-54
Transient Journal.....	9-56
ALTER TABLE Misconception	9-56
Recovery Journal for Down AMPs.....	9-58
Permanent Journal.....	9-60
Archiving and Recovering Data.....	9-62
Teradata Unity.....	9-64
Teradata Multi-System Solution Architecture	9-66
Module 9: Review Questions	9-68

Data Protection Features

Disk Arrays – Disk arrays provide RAID 1, RAID 5, or RAID S data protection. If a disk drive fails, the array subsystem provides continuous access to the data. Systems with disk arrays are configured with redundant Fibre adapters, buses, and array controllers to provide highly available access to the data.

Clique – a set of Teradata nodes that share a common set of disk arrays. In the event of node failure, all vprocs can migrate to another available node in the clique. All nodes in the clique must have access to the same disk arrays.

Locks – Locking prevents multiple users who are trying to change the same data at the same time from violating the data's integrity. This concurrency control is implemented by **locking** the desired data. Locks are automatically acquired during the processing of a request and released at the termination of the request. In addition, users can specify locks. There are four types of locks: Exclusive, Write, Read, and Access.

Fallback – protects your data by storing a second copy of each row of a table on an alternative “fallback AMP”. If an AMP fails, the system accesses the fallback rows to meet requests. Fallback provides AMP fault tolerance at the table level. With Fallback tables, if one AMP fails, the table is still available. Users may continue to use Fallback tables without any loss of available data.

Down-AMP Recovery Journal – started automatically when the system has a failed or down AMP. Its purpose is to log any changes to rows which reside on the down AMP.

Transient Journal – exists to permit the successful rollback of a failed transaction. Transactions are not committed to the database until an End Transaction request has been received by the AMPs, either implicitly or explicitly. Until that time, there is always the possibility that the transaction may fail in which case the participating table(s) must be restored to their pre-transaction state.

Permanent Journal – provides selective or full database recovery to a specified point in time by keeping either before-image or after-images of rows in a journal. It permits recovery from unexpected hardware or software disasters.

ARC/DSA and NetVault/NetBackup – ARC and DSA command scripts provide the capability to backup and restore the Teradata database. The **NetVault** and **NetBackup** utilities provide a GUI based front-end for creation and execution of ARC command scripts.

Teradata Unity is positioned for enabling the Teradata Analytical Ecosystem. It is the combination of all the products in the Unity portfolio that are needed for a full dual system or multi-system solution. How and how much each product is used will depend on each customer's specific implementation and requirements. Such as High Availability, disaster recovery or simply minimize scheduled downtime.

Data Protection Features

Facilities that provide system-level protection

Disk Arrays

- RAID data protection (e.g., RAID 1)
- Redundant data buses and array controllers

Cliques and Vproc Migration

- SMP or Linux failures; Vprocs can migrate to other nodes (or HSN) within the clique.

Facilities that provide Teradata NewSQL Engine protection

- | | |
|----------------------------------|--|
| Fallback | – provides data access with a “down” AMP |
| Locks | – provide data integrity |
| Transient Journal | – automatic rollback of aborted transactions |
| Down AMP Recovery Journal | – fast recovery of fallback rows for AMPs |
| Permanent Journal | – optional before and after-image journaling |
| Unity | – integrated portfolio that turn a multi-system environment into an orchestrated Ecosystem |
| ARC | – Archive/Restore facility |
| NetBackup and Tivoli | – provide backup storage management, ARC script creation, and scheduling capabilities |

Disk Arrays

Disk arrays utilize a technology called RAID (**Redundant Array of Independent Disks**) Spanning the entire spectrum from personal computers to mainframes, RAID technologies offer significant improvements in availability, reliability and maintainability of information storage, along with higher performance.

A disk array subsystem consists of controller(s) which drive a set of disks. Typically, a disk array is configured to represent a number of logical volumes (or disks), each of which appears to be a physical disk to the user. A logical volume can be configured to reside on multiple physical disks and the user accesses the logical volume.

RAID technologies allow you to group disks into drive groups and specify a specific technology (e.g., RAID 1) to use for the drive group. The benefit of this feature is that if any single disk in a drive group fails, the drive group continues to function without loss of data. This is possible because redundancy information is stored separate from the data. The redundancy information, as will be explained, may be a full copy of the data (e.g., RAID 1) or additional information (e.g., parity) that can be used to reconstruct any data.

A second benefit is that performance may increase for specific applications because there is multiple simultaneous accesses of different blocks on different disks. With RAID 5 or RAID 6 technologies, the rate at which datum is transferred to/from the disk array can be increased significantly over that of a single disk by utilizing parallel reads/writes of the data spread across the disks in the array. This function is referred to as "striping the data".

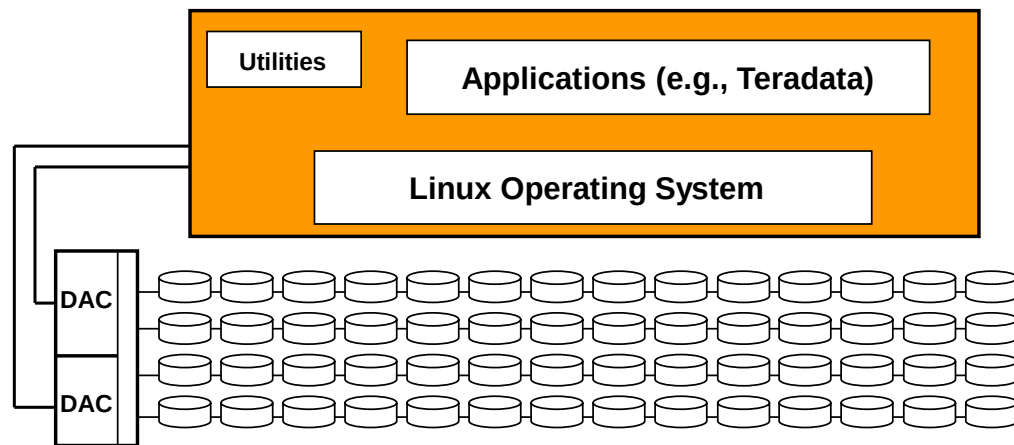
Finally, disk array subsystem maintenance is typically simplified because it is possible to replace ("hot swap") individual disks and other components while the system continues to function. It is easy to replace a disk without having to power down the disk array.

Additional, the data on the disk array drives is automatically encrypted on the disks because the disks have FDE (Full Disk Encryption) enabled. Disk encryption is done at the drive by the drive controller. When array encryption is enabled (using PUT), a key is created and the drive is "locked" so during the drive power-on and initialization the drive will request the key from the controller before any data is unencrypted and sent on. This is the case with node (e.g., TPA) disk drives as well.

When the CSR executes the PUT option "Full Disk Encryption", the customer is prompted to enter a passphrase. The customer is told to keep it in a safe place and not to lose it. It is used to recover the key if both controllers are replaced at the same time.

The encryption can be disabled which erases the key on the drives, but ALL data on all drives is lost, and the array is unencrypted again. If you remove a drive from an encrypted array to any other array (or to a different unsecured environment with a SAS cable), the drive is FDE locked. To use the drive you must secure erase it which wipes out the key on the drive and creates a new one, thus destroying the data without any way to recover the data. If you ever need to read a drive on another array, it is also possible to enter the passphrase to enable encryption on the source array.

Disk Arrays



Why Disk Arrays?

- **High availability** through data mirroring or data parity protection.
- **Better I/O performance** through implementation of RAID technology at the hardware level.
- **Convenience** – automatic disk recovery and data reconstruction when mirroring or data parity protection is used.

RAID Technologies

RAID is an acronym for **Redundant Array of Independent Disks**. The term was coined in 1988 in a paper describing array configuration and application by researchers and authors Patterson, Gibson and Katz of the University of California at Berkeley. The word redundant implies that data, functions and/or components have been duplicated in the array's architecture. Duplication of data, functions, and hardware ensures that even in the event of a failed drive or other components, datum is not lost and is continuously available.

The industry currently has agreed upon seven RAID configuration levels and designated them as RAID 0 through RAID 6. The physical configuration is dictated to some extent by the choice of RAID level; however, RAID conventions specify more precisely how datum is stored on disk.

RAID 0	Data striping
RAID 1	Disk mirroring (most common with most Teradata systems)
RAID 2	Parallel array, hamming code
RAID 3	Parallel array with parity
RAID 4	Data parity protection, dedicated parity drive
RAID 5	Data parity protection, interleaved parity
RAID 6	Similar to RAID 5 except for a second parity drive (RAID 6 is an option available with the 1700/1800, 2800, and with IntelliFlex™ 2.0 RI SSD). With IntelliFlex™ 2.0 RI SSD, RAID 6 is may be configured with 10 drives (8 data and 2 parity) up to 12 drives (10 data and 2 parity).

There are other RAID technologies that are defined by specific vendors or are accepted in the data processing industry. For example, RAID 10 or RAID 1+0 (or RAID 0+1) is considered to be “striped mirroring”. RAID level classifications do not imply superiority of one mode over another. Each mode has its rightful application. In fact, these modes of operation can be combined within a single system configuration, within product limitations, to obtain maximum flexibility and performance.

The advantages of RAID 1 (compared to RAID 5) include:

- Superior Performance and Availability
- Mirroring provides the best read and write throughput.
- Maximizes the performance capabilities of controllers and disk drives.
- Best performance when a drive has failed.
- Less reconstruction impact when a drive has failed.
- Less susceptible to a double disk failure in a RAID drive group.
- Faster reconstruction of a failed drive – shorter vulnerability period during reconstruction.

Superior Price/Performance - the performance advantage of RAID 1 outweighs the additional cost for typical Teradata warehouses.

RAID 5 for Teradata – used in older systems (5200/5300) or when creating archival data warehouses that require less expensive storage and where performance is not as important.

RAID Technologies

RAID – Redundant Array of Independent Disks

RAID technology provides data protection at the disk drive level. With RAID technologies, access to the data is continuous even if a disk drive fails.

RAID technologies currently available with Teradata:

- | | |
|--------|---|
| RAID 1 | Disk mirroring – provides highest data availability and performance. However, 50% of storage capacity is used for redundant data. |
| RAID 6 | Data parity protection with two parity drives. Provides high data availability – allows continuous access with loss of two drives in a drive group. |

Uses in Teradata:

- | | |
|--------|--|
| RAID 1 | Most useful with typical Teradata data warehouses (e.g., Active Data Warehouses). This is most frequently used RAID technology in Teradata systems. |
| RAID 6 | <p>RAID option available with Teradata 1800, 2800, and IntelliFlex™ 2.0 RI SSD.</p> <p>With RI SSD, RAID 6 is implemented with 2 parity drives across a drive group of up to 12 drives (I/O throughput maximum with an IntelliFlex™ 2.0 system).</p> <ul style="list-style-type: none">• Ex: RAID 6 implemented across 8 disk drives plus 2 parity drives (or 8 + 2).• With a RAID 6 drive group of 10 drives, the redundant data is 20% (2/10 of the storage space). This effectively provides a higher % of storage capacity. |

Note that RAID 6 performance is slower (than RAID 1) with write and reconstruction operations because data and both parities have to be calculated and written.

RAID 1 – Mirroring

RAID 1 is data mirroring protection. The **RAID 1** technology requires each primary data disk to have a companion disk or mirror. The contents of the primary disk and the mirror disk are identical.

When datum is written on the primary disk, a write also occurs on the mirror disk. The mirroring process is invisible to the user. For this reason, RAID 1 is also called transparent mirroring.

With RAID solutions, mirroring is managed by the controller, which provides a higher level of performance. Performance is improved because data can be read from either the primary (data) drive or the mirror. The controller decides which read/write assembly (drive actuator) is closest to the requested data.

If the primary data disk fails, the mirror disk can be accessed without data loss. There is a minor performance penalty if a drive fails because the array controller can read from either drive if both drives are available. If either disk fails, the disk array controller can copy the data from the remaining drive to a replacement drive while normal operations continue.

RAID 10 – Striped Mirroring

When user datum is to be written to the array, the controller instructs the array to write a block of data to one drive pair to the defined stripe depth. Subsequent data blocks are written concurrently to contiguous sectors in the next drive pair to the defined stripe depth. In this manner, data are striped across the array of drives, utilizing multiple drives and actuators.

With LSI Logic arrays, striped mirroring is automatic when you create a drive group (with RAID 1 technology) that has multiple mirrored pairs of disks.

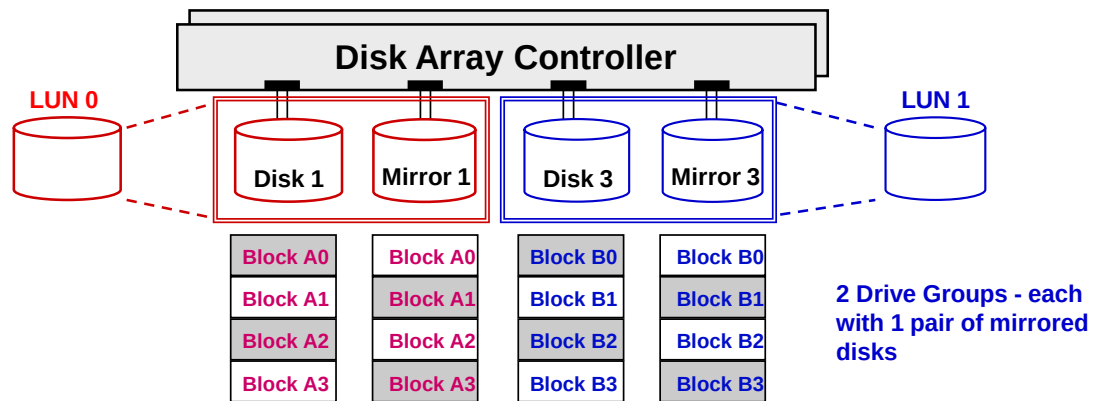
If an application (e.g., Teradata NewSQL Engine) uniformly distributes data, striped mirroring (RAID 10 or 1+0) and mirroring (RAID 1) will have similar performance.

If an application (database) partitions data, striped mirroring (RAID 10) can lead to performance gains over mirroring (RAID 1) because array controllers equally spread I/O's between channels in the array.

Striped Mirroring is NOT necessary with Teradata.

RAID 1 – Mirroring

- 2 Drive Groups each with 1 mirrored pair of disks
- Operating system sees 2 logical disks (LUNs) or volumes
- If LUN 0 has more activity, more disk I/Os occur on the first two drives in the array.



Notes:

- If the physical drives are 600 GB each, then each LUN or volume is effectively 600 GB.
- If both logical units (or volumes) are assigned to an AMP, then the AMP will have approximately 1.2* TB assigned to it.

* Actual MaxPerm space will be a little less.

RAID 1 Summary

RAID 1 characteristics include:

- Data is fully replicated
- Easy to understand technology
- Follows a traditional approach
- Transparent to the operating system
- Redundant drive is affected only by write operations

RAID 1 advantages include:

- High I/O rate (small logical block size)
- Maximum data availability
- Minor performance penalty with single drive failure
- No performance penalty in write intensive environments

RAID 1 disadvantage is:

- Only 50% of total disk space is available for user data. Therefore, RAID 1 has 50% overhead in disk space usage.

Summary

- RAID 1 provides high data availability and performance, but storage costs are high.
- Striped mirroring is not necessary with Teradata.

RAID 1 – this RAID technology is the most useful and most common with typical Teradata data warehouses (e.g., Active Data Warehouses).

RAID 1 Summary

Characteristics

- data is fully replicated
- striped mirroring is possible with multiple pairs of disks in a drive group
- transparent to operating system

Advantages (compared to RAID 6)

- Provides maximum data availability
- Mirroring provides the best read and write throughput
- Maximizes the performance capabilities of controllers and disk drives
- Minimal performance issues when a drive has failed
- Less reconstruction impact when a drive has failed
- Arrays with RAID 1 support GHS (Global Hot Spare) drives

Disadvantage

- 50% of disk space is used for mirrored data

Summary

- RAID 1 provides best data availability and performance, but storage costs are higher.
- *Striped Mirroring is NOT necessary with Teradata.*

RAID 6 – Data Parity Protection with 2 Parity Drives

RAID 6 characteristics include:

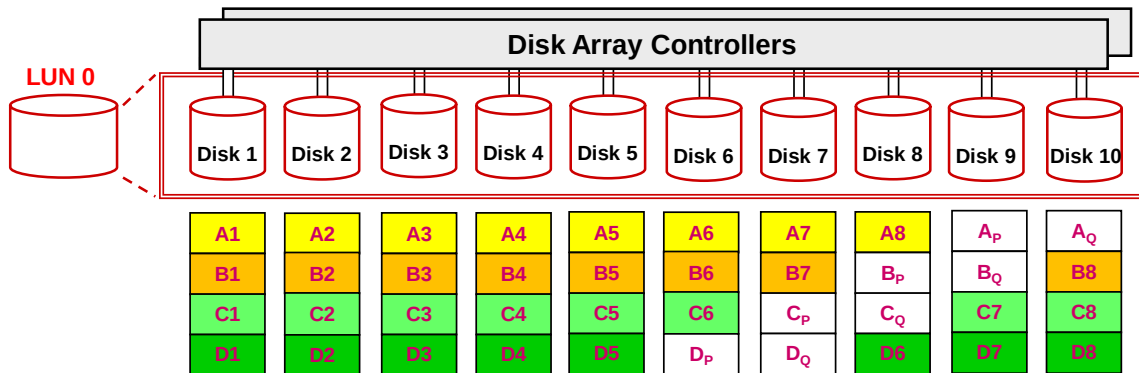
- A Drive Group has $n+2$ disk drives and parity data is interleaved across the drive group.
- Teradata uses drive groups of 7 drives – 5 data + 2 parity drives with 1700/1800 systems.
- Teradata uses drive groups of 9 drives – 7 data + 2 parity drives with 2800 systems that utilize large capacity drives (1.2 TB).
- Teradata typically uses drive groups of 10 or 12 drives with IntelliFlex™ 2 – 2.0 systems using RI SSD drives – 8 data + 2 parity drives up to 10 data + 2 parity drives.
- The I/O capacity of the disk array controllers maxes out somewhere between 10 and 12 drives in this type of IntelliFlex™ 2.0 system. Therefore, it may be common to see 10 drives in this type of RAID 6 implementation.
- RAID 6 space overhead (redundant data) is 20% with 10 drives and 16.7% with 12 drives.
- Simple XOR logic is used to calculate the parity for the first parity data (P) and extended math functions are used to calculate parity for second parity drive (Q).

RAID 6 is used for IntelliFlex™ 2.0 RI (Read-Intensive) disk arrays.

RAID 6 is an option with Teradata 1700/1800 and 2800 systems.

RAID 6 – Data Parity Protection with 2 Parity Drives

- A Drive Group has $n+2$ disk drives and parity data is interleaved across the drive group.
- Teradata typically uses drive groups of 10 (or 12 drives) with IntelliFlex™ 2.0 systems using RI SSD drives; with 10 drives, you would have 8 data + 2 parity drives.
 - RAID 6 space overhead is 20% for redundant data with 10 drives.
- Simple XOR logic is used to calculate the parity for the first parity data (P) and extended math functions are used to calculate parity for second parity drive (Q).



Example:

- The disk array has ten 6.4 TB RI SSD drives (8 data + 2 parity); LUN is effectively 51.2 TB in size.
- This LUN can be divided into 108 partitions and each partition is assigned to a separate AMP.
- Two nodes (54 AMPs each) would share this disk array. Each AMP has 474 GB of max perm space.

Cliques

A **clique** is a set of Teradata nodes that share a common set of disk arrays. In the event of node failure, all vprocs can migrate to available nodes in the clique. All nodes in the clique must have access to the same disk arrays.

The illustration on the facing page shows a 4-node clique. In this example, each node has 36 AMP vprocs. Therefore, there is a total of 176 AMPs in this clique.

If a Teradata software node (e.g., TPA node) fails, Teradata restarts and the AMP vprocs that were executing on the failed node are started on other nodes within the clique. In this example, the remaining 3 nodes would each execute ~59 AMP vprocs (176 divided by 3 remaining nodes). Two nodes will execute 59 AMP vprocs and the third node will execute 58 AMP vprocs.

After the failed node is recovered/repaired and rebooted, a second restart of Teradata is needed to reuse the node that had failed. The restart will redistribute the AMPs to the recovered node.

This example is a 4-node clique without a HSN (Hot Standby Node). The nodes are in the lower portion of a 2850 cabinet and have node names of SMPxxx-4, SMPxxx-5, SMPxxx-6, and SMPxxx-7 where xxx is the cabinet number.

The acronym DAC represents Disk Array Controller.

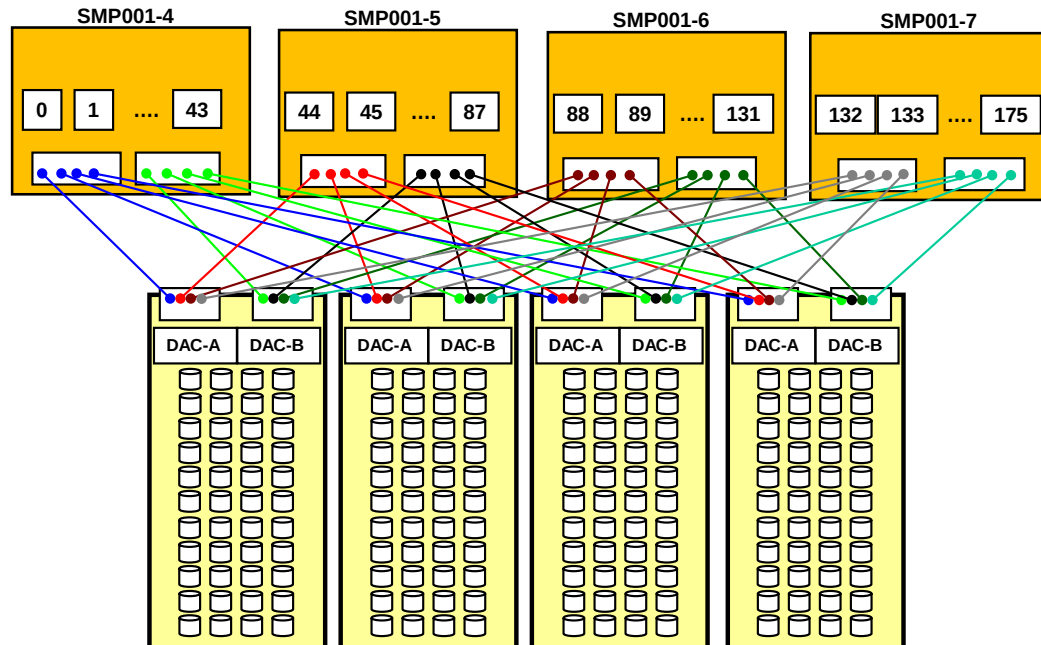
The Teradata IntelliFlex™ systems support **larger cliques** than the traditional Teradata 1xxx, 2xxx, and 6xxx systems. Since the disk storage is connected to the Infiniband fabric, there is more flexibility in clique sizes. The largest clique size supported will be 13 nodes – 12 Teradata nodes plus one HSN.

Acronyms:

DAC – Disk Array Controller

Cliques

Clique – a set of nodes that share a common set of disk arrays.



Example of a 2850 clique (4 nodes, no HSN) – 44 AMPs/node.

Teradata Vproc Migration

If a TPA node (running Teradata) fails, Teradata restarts and the AMP vprocs that were executing on the failed node are started on other nodes within the clique.

PE vprocs that are assigned to channel connections do not migrate to another node. PE vprocs that are assigned to gateway connections may or may not (depending on configuration) migrate to another node within the clique.

If a node fails, the vprocs from the failed node are distributed between the remaining nodes in the clique. The vconfig.out file determines the node on which vprocs will start if all of the nodes in the clique are available.

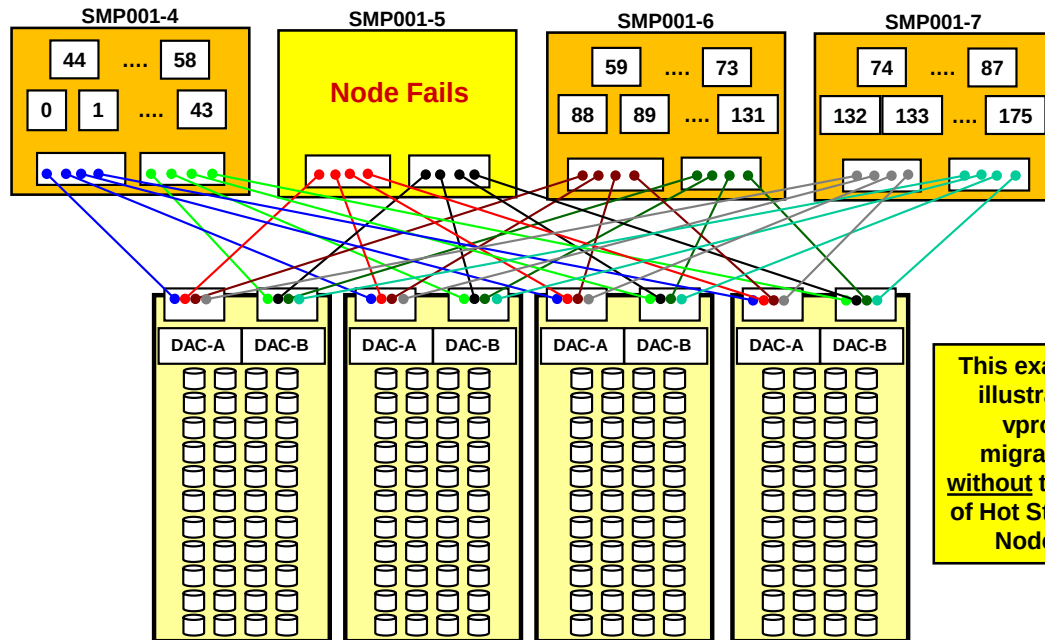
The following is from a “Get Config” command following the failure of SMP001-5. The highlighted AMPs have migrated to the remaining nodes in the clique.

DBS LOGICAL CONFIGURATION

Vproc Number	Rel. Vproc#	Node ID	Movable	Crash Count	Vproc State	Config Status	Config Type	Cluster/ Host No.	RcvJrnl/ Host Type
0*	1	1-04	Yes	0	ONLINE	Online	AMP	0	On
1	2	1-04	Yes	0	ONLINE	Online	AMP	1	On
2	3	1-04	Yes	0	ONLINE	Online	AMP	2	On
:	:	:	:	:	:	:	:	:	:
42	43	1-04	Yes	0	ONLINE	Online	AMP	42	On
43	44	1-04	Yes	0	ONLINE	Online	AMP	43	On
44	45	1-04	Yes	0	ONLINE	Online	AMP	44	On
45	46	1-04	Yes	0	ONLINE	Online	AMP	45	On
:	:	:	:	:	:	:	:	:	:
57	58	1-04	Yes	0	ONLINE	Online	AMP	57	On
58	59	1-04	Yes	0	ONLINE	Online	AMP	58	On
59	1	1-06	Yes	0	ONLINE	Online	AMP	59	On
60	2	1-06	Yes	0	ONLINE	Online	AMP	60	On
61	3	1-06	Yes	0	ONLINE	Online	AMP	61	On
:	:	:	:	:	:	:	:	:	:
73	15	1-06	Yes	0	ONLINE	Online	AMP	73	On
74	1	1-07	Yes	0	ONLINE	Online	AMP	74	On
75	2	1-07	Yes	0	ONLINE	Online	AMP	75	On
76	3	1-07	Yes	0	ONLINE	Online	AMP	76	On
:	:	:	:	:	:	:	:	:	:
87	14	1-07	Yes	0	ONLINE	Online	AMP	87	On
88	16	1-06	Yes	0	ONLINE	Online	AMP	0	On
89	17	1-06	Yes	0	ONLINE	Online	AMP	1	On
90	18	1-06	Yes	0	ONLINE	Online	AMP	2	On
:	:	:	:	:	:	:	:	:	:
131	59	1-06	Yes	0	ONLINE	Online	AMP	43	On
132	15	1-07	Yes	0	ONLINE	Online	AMP	44	On
133	16	1-07	Yes	0	ONLINE	Online	AMP	45	On
134	17	1-07	Yes	0	ONLINE	Online	AMP	46	On
:	:	:	:	:	:	:	:	:	:
175	58	1-07	Yes	0	ONLINE	Online	AMP	87	On

Teradata Vproc Migration

In this example, one of the nodes has experienced a hardware failure.



After vproc migration, the 3 remaining nodes each have ~59 AMPs.

After failed node is repaired, a 2nd restart is needed for the failed node to rejoin the configuration.

Hot Standby Nodes (HSN)

A **Hot Standby Node (HSN)** is a node that is part of a clique and the hot standby node is not configured (initially) to execute any Teradata vprocs. If a node in the clique fails, the AMPs from the failed node move to the hot standby node. The performance degradation is 0%.

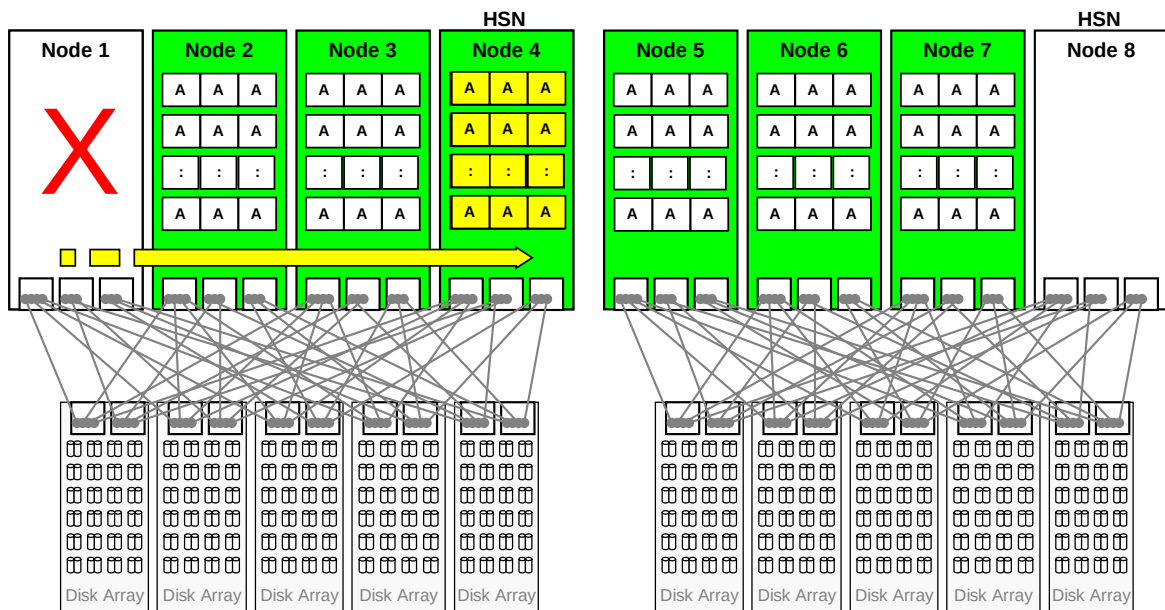
When the failed node is recovered/repared and restarted, it becomes the new hot standby node. A second restart of Teradata is not needed.

Characteristics of a hot standby node are:

- A node that is a member of a clique.
- Does not normally participate in the trusted parallel application (TPA).
- Can be brought into the TPA to compensate for the loss of a node in the clique.

Hot Standby Nodes are positioned as a performance continuity feature.

Hot Standby Nodes (HSN)



1. Performance Degradation is 0% as AMPs are moved to the Hot Standby Node.

2. When Node 1 is recovered, it becomes the new Hot Standby Node.

This example illustrates vproc migration using a Hot Standby Node.

Performance Degradation with Node Failure

The facing page displays 2 examples of the performance degradation with all-AMP operations that occur when a node fails. Note: WL - Workload

The top example illustrates two 3-node cliques and the performance degradation of 50% for an all-AMP operation when a node fails in one of the cliques.

From a simple perspective, if you have 3 nodes in a clique and you lose a node, you would logically think 33% performance degradation. In reality, the performance cost or degradation is 50%. Assume 3 nodes, 72 AMPs, and you execute an all-AMPs query. This query uses 240 CPU seconds per node to complete the query. The 3 nodes use a total of 720 CPU seconds to do the work. Another way to look at it is that each AMP needs 10 CPU seconds or 72 AMPs x 10 CPU seconds equals 720 CPU seconds of work to be done.

A node fails and now there are 2 nodes to complete the query. There are still 72 AMPs and the query still needs 720 CPU seconds to complete the query, but now there are only 2 nodes. Each node will need about 360 CPU seconds to complete the query. Each node has about 50% more work to do. This is why it is considered a 50% performance cost.

Another way of looking at a query is from the response time back to the user. From a user perspective, let's assume that response time back to the user with all 3 nodes normally active is 1 minute (60 seconds) of wall clock time. The wall clock response time with only 2 active nodes is 90 seconds. (Since there are fewer nodes, the query is going to take longer to complete.) From the user perspective, the response time is 50% longer.

It is true that if you take 67% of 90, you will get 60 and you may think that the degradation is 33%. However, 90 seconds is not the normal response time. This normal response time is 60 seconds and the exception is 90 seconds, therefore the performance is worse by 50%. The percentage is calculated from the "normal".

The bottom example illustrates two 3-node cliques, each with a hot standby node (2 TPA nodes) and the performance degradation of 0% for an all-AMP operation when a node fails in the clique. This configuration is often referred to as a 2+1 configuration.

Restarts

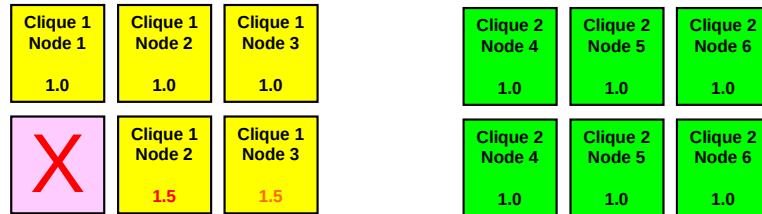
In the first (top) example, after the failed node is recovered/repared and rebooted, a second restart of Teradata is needed to reuse the node that had failed. The restart will redistribute the AMPs to the recovered node.

With a hot standby node, when the failed node is recovered/repared and restarted, it becomes the new hot standby node within the clique. A second restart of Teradata is not needed.

Performance Degradation with Node Failure

2 Cliques without HSN nodes (3 nodes) – performance degradation of 50% with node failure.

Workload = 6.0
 Clique WL = 3.0
 Node WL = 1.00

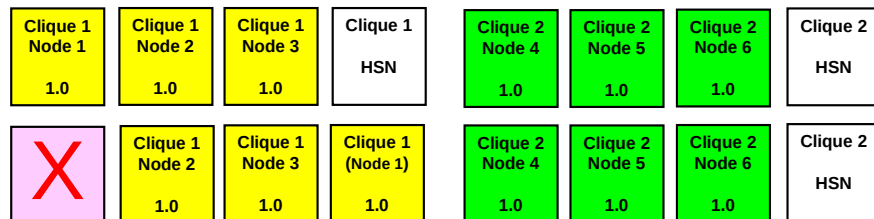


When a node fails, Teradata restarts.

After the node is repaired, a second restart of Teradata is required to allow the node to rejoin the configuration.

2 Cliques each with a HSN (3+1 nodes) – performance degradation of 0% with node failure.

Workload = 6.0
 Clique WL = 3.0
 Node WL = 1.00



When a node fails, Teradata restarts.

After the node is repaired, it becomes the new Hot Standby Node. A second restart of Teradata is not required.

Fallback

Fallback protects your data by storing a second copy of each row of a table on an alternative “fallback AMP”. If an AMP fails, the system accesses the fallback rows to meet requests. Fallback provides AMP fault tolerance at the table level. With Fallback tables, if one AMP fails, all of the table data is still available. Users may continue to use Fallback tables without any loss of available data.

When a table is created, or any time after its creation, the user may specify whether or not the system should keep a fallback copy. If Fallback is specified, it is automatic and transparent to the user.

Fallback guarantees that the two copies of a row will always be on different AMPs. Therefore, if either AMP fails, the alternate row copy is still available on the other AMP.

Certainly there is a benefit to protecting your data. However, there are costs associated with that benefit. They are: twice the disk space for storage and twice the I/O for Inserts, Updates, and Deletes. (However, the Fallback option does not require any extra I/O for SELECT operations and the fallback I/O will be performed in parallel with the primary I/O.)

The benefits of Fallback include protecting your data from an AMP failure. An AMP failure may occur because of physical disk failures – e.g., both disk drives in a RAID 1 drive group have failed or 3 drives in the same RAID 6 drive group. An AMP may also fail because of an AMP software failure or because access to the disk array has failed.

A hardware (disk) or software (vproc) failure causes an AMP to be taken off-line until the problem is corrected.

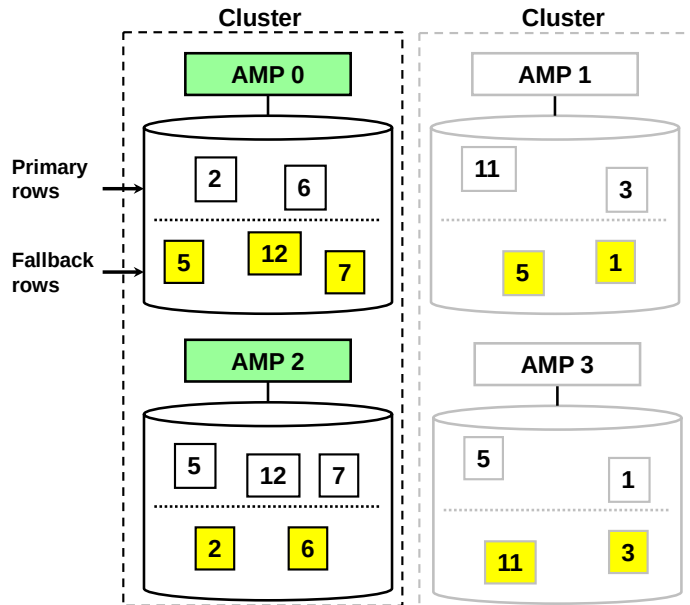
During this period, Fallback tables are fully available to users.

When the AMP is brought back on-line, the associated Vdisk is refreshed to reflect any changes during the off-line period.

Fallback

A Fallback table is fully available in the event of an unavailable AMP.

A Fallback row is a copy of a "Primary row" which is stored on a different AMP.



Cluster – group of 2 AMPs across which fallback is implemented.

Benefits of Fallback

- Permits access to table data if an AMP is not available (e.g., down, offline, or fatal states).
- Adds a level of data protection beyond disk array RAID.
- If an AMP is offline, the down AMP recovery journal is used to maintain changes made to a fallback table in the clustered AMP.
- Critical for high availability applications.

Cost of Fallback

- Twice the disk space for table storage.
- Twice the I/O for Inserts, Updates and Deletes.

Loss of any two AMPs in a cluster causes RDBMS to halt!

Fallback Clusters

A **cluster** is a group of AMPs that act as a single fallback unit. Clustering has no effect on the distribution of the Primary rows of a table. The Fallback row copy however, will always go to a different AMP in the same cluster.

The cluster size is set when Teradata is configured and the only choice for new systems is 2-AMP clusters. Years ago, AMP clusters ranged from 2 to 16 AMPs per cluster and were commonly set as groups of 4 AMPs. Starting with 5450 systems, all clusters are defined as 2 AMP clusters.

Should an AMP fail, the primary and fallback row copies stored on that AMP cannot be accessed. However, their alternate copies are available through the other AMPs in the same cluster.

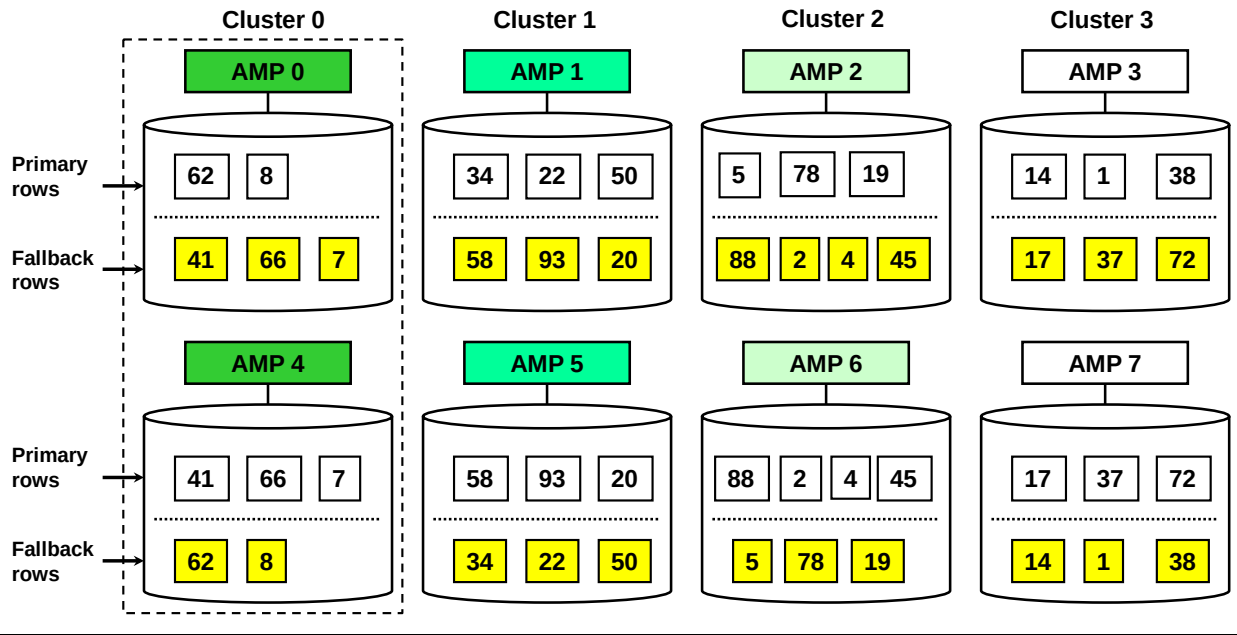
The loss of an AMP in a cluster has no effect upon other clusters. It is possible to lose one AMP in each cluster and still have full access to all Fallback-protected table data. If both AMPs fail in a cluster, then Teradata halts.

While an AMP is down, the fallback AMP in the cluster does provides access to the fallback rows for the AMP that is out-of-service. This simply means the Fallback AMP must do their own work plus the work of the down AMP.

A small cluster size (e.g., 2 AMP cluster) reduces the chances of have 2 down AMPs in a single cluster which would cause a non-operational configuration. With today's new systems, a typical cluster size of 2 AMPs provides the best option to maximize availability.

Fallback Clusters

- A Fallback cluster is a defined set of 2 AMPs across which fallback is implemented.
- Loss of one AMP in the cluster permits continued table access.
- Loss of two AMPs in the cluster causes the RDBMS to halt.



Fallback and RAID Protection

RAID 1 mirroring and RAID 6 data parity protection provide protection in the event of a disk drive failure.

Fallback provides another level of data protection beyond disk mirroring or data parity protection.

Examples of other failures that Fallback provides protection against include:

- Multiple drive failures in the same drive group
- An array is not available (e.g., both disk array controllers fail in the disk array)
- An AMP is not available (e.g., a software problem)

Global Hot Spare Drives

Typically, each disk array in Enterprise Data Warehouse Systems (e.g., 6800) will include Global Hot Spare (GHS) drives. One or more disks per array can be configured as hot spare disks. In the event of a disk failure on a RAID mirrored pair, the contents of the failed disk are copied to a hot spare disk from the mirrored surviving disk to repair the RAID pair. When the failed drive is replaced, a copy back operation occurs to restore data to the replaced drive. GHS drives reduce the window of risk when a drive has failed in a mirrored pair.

GHS Drives are not used with RAID 6 configurations. There are two reasons they are not used with a RAID 6 drive group. First, a RAID 6 drive group inherently has 2 parity drives and would need 3 drives to fail in the drive group before a loss of data. The I/O and CPU time to calculate the data on the failed drive to build a GHS will be extremely high and performance would take a major hit.

RAID, GHS, and Fallback Protection

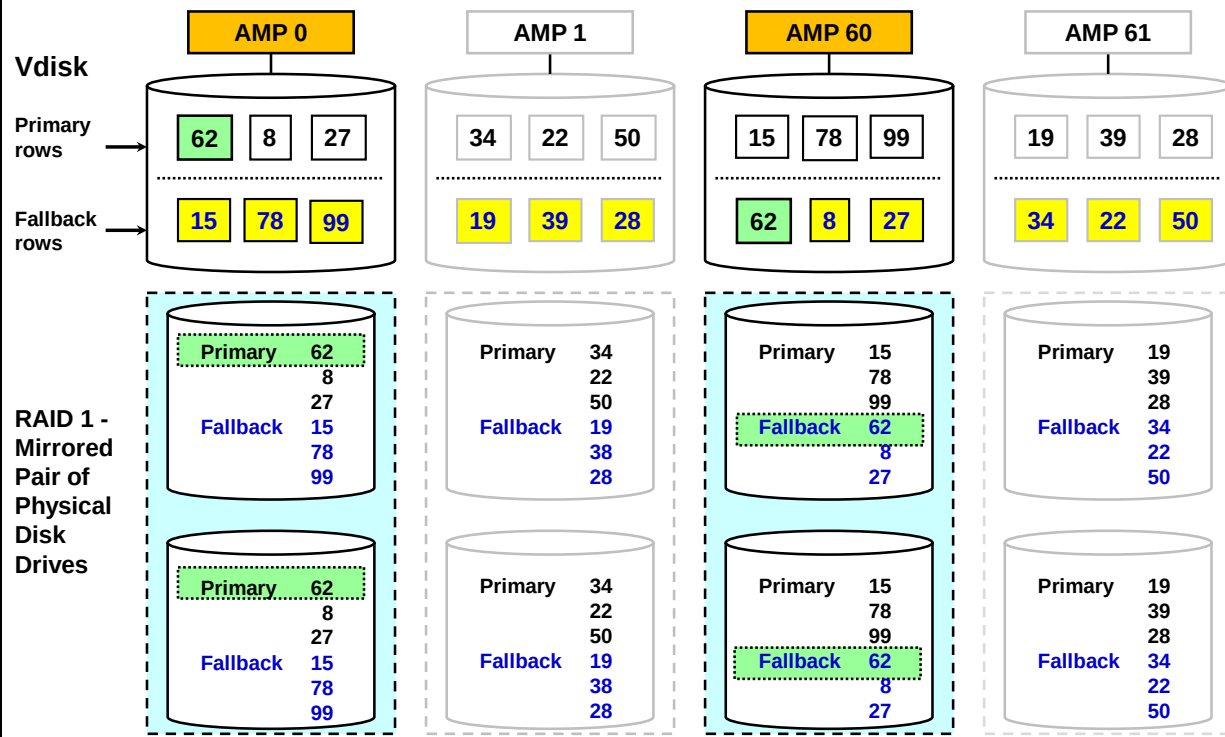
- RAID 1 Mirroring or RAID 6 Data Parity Protection provides protection in the event of disk drive failure.
 - Provides protection at a hardware level
 - Teradata is unaware of the RAID technology used
- GHS (Global Hot Spare) Drives – one or more disks per array can be configured as hot spare disks. Note GHS drives are NOT used with RAID 6 configurations.
 - In the event of a disk failure on a RAID mirrored pair, the contents of the failed disk are copied to a hot spare disk from the mirrored surviving disk to repair the RAID pair.
 - GHS disks reduce the time window of risk in the case of a second drive failure in the same mirrored pair.
- Fallback provides an additional level of data protection and provides access to data when an AMP is not available (not online).
- Additional types of failures that Fallback protects against include:
 - Disk array is not available
 - Both disk array controllers fail in a disk array
 - Two power supplies fail in a disk array
 - Multiple drives fail in the same drive group,
 - AMP is not available (e.g., software or data error)
- The combination of RAID 1 and Fallback provides the highest level of availability.

Fallback and RAID 1 Example

The next set of pages contains an example of how Fallback and RAID 1 Mirroring work together.

Fallback and RAID 1 Example

This example assumes that RAID 1 Mirroring is used and the table is fallback protected.



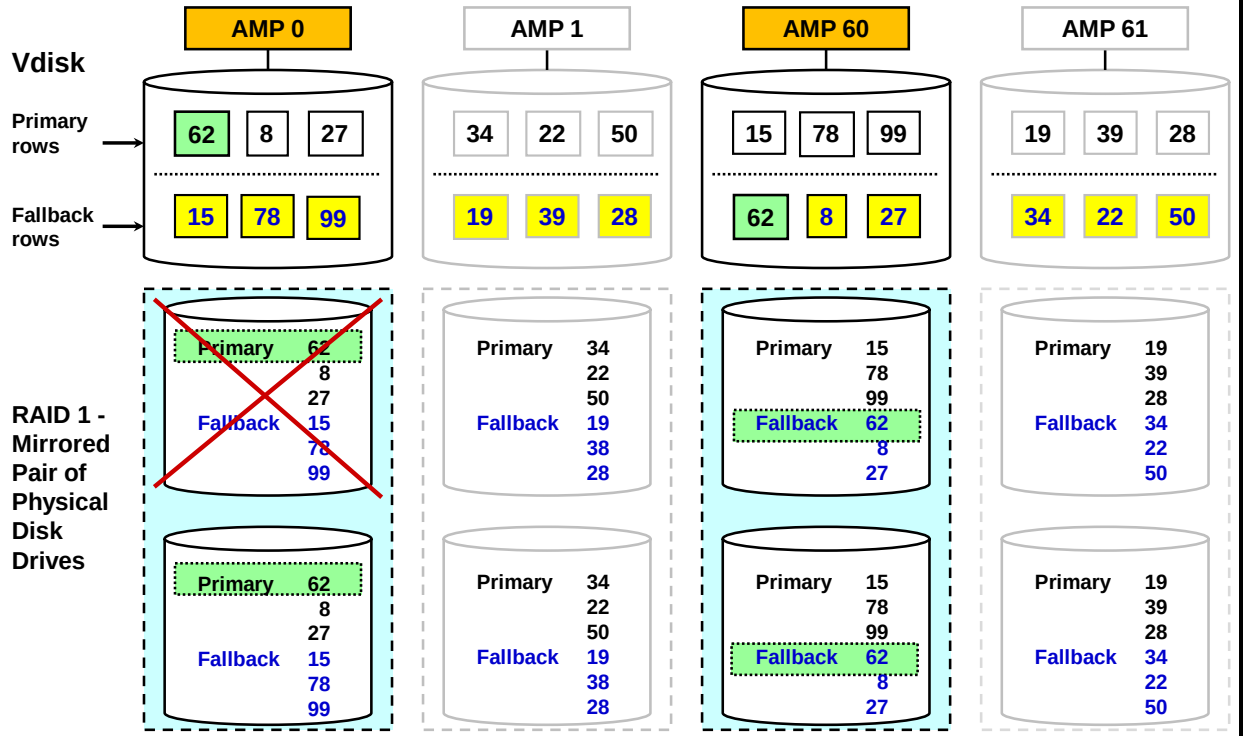
Fallback and RAID 1 Example (cont.)

The example of how Fallback and RAID 1 Mirroring work together is continued.

In this example, one disk drive has failed in the first drive group. Is Fallback needed? No. As a matter of fact, Teradata doesn't even realize that the drive has failed. The disk array continues to provide access to the data directly from the second disk drive in the drive group. The disk array controller will send a "fault" or error message to the SWS.

Fallback and RAID 1 Example (cont.)

Assume one disk drive fails. Is Fallback needed in this example?



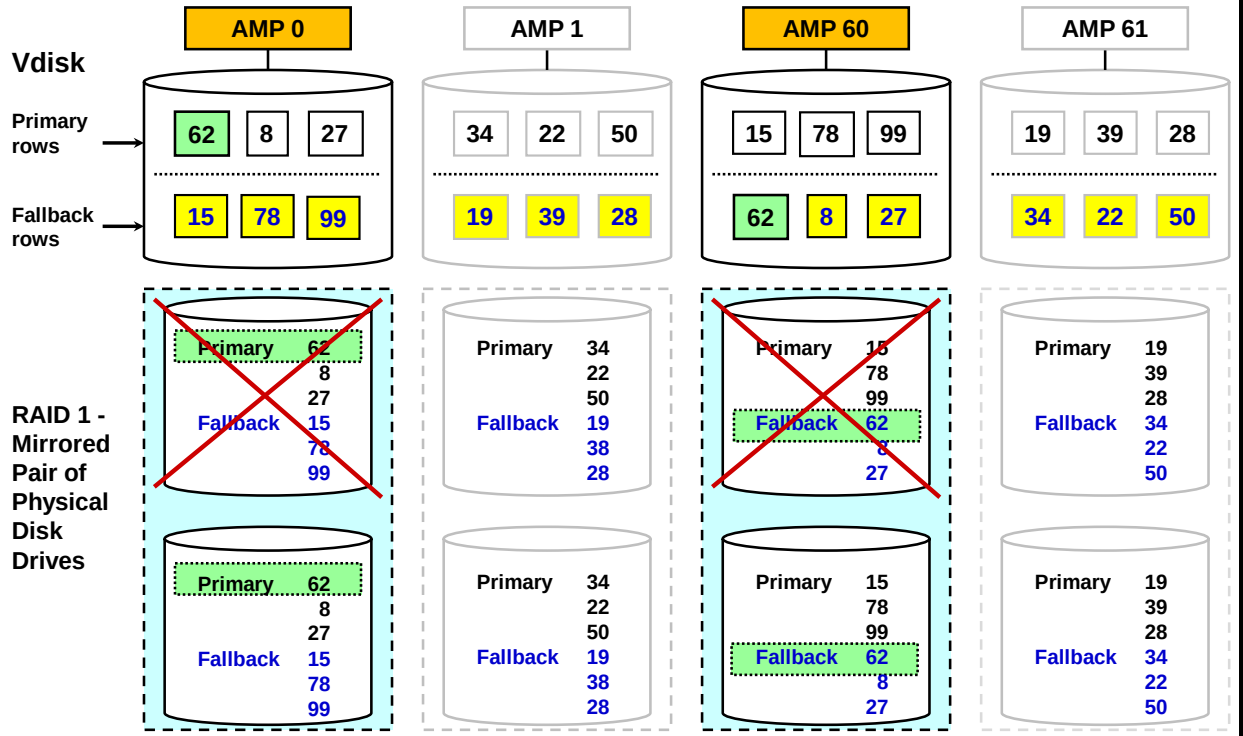
Fallback and RAID 1 Example (cont.)

The example of how Fallback and RAID 1 Mirroring work together is continued.

In this example, assume two disk drives have failed – one in the first drive group and one in the third drive group. Is Fallback needed? No. Like before, Teradata doesn't even realize that the drives have failed. The disk array continues to provide access to the data directly from the second disk drive each of the drive groups. The disk array controller will send "fault" or error messages to the SWS.

Fallback and RAID 1 Example (cont.)

Assume two disk drives have failed. Is Fallback needed in this example?



Fallback and RAID 1 Example (cont.)

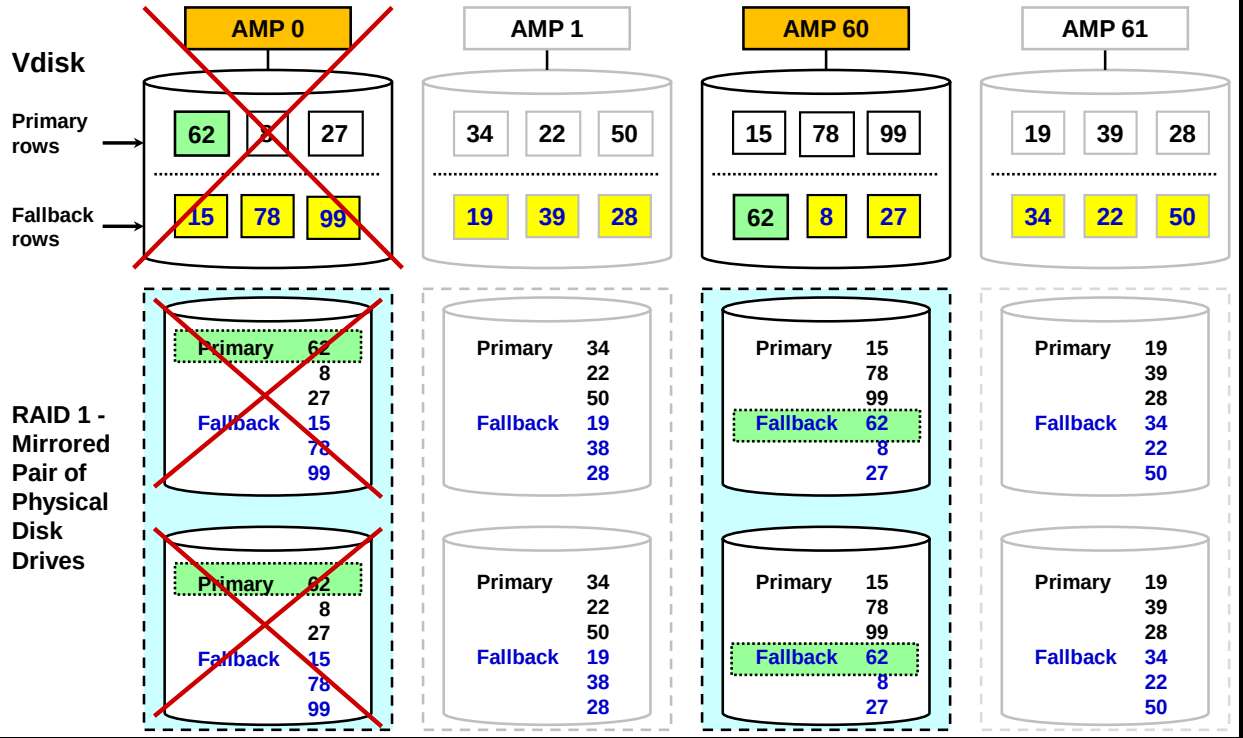
The example of how Fallback and RAID 1 Mirroring work together is continued.

In this example, assume two disk drives have failed – both failed drives are in the first drive group. Is Fallback needed? Yes, if you need to access the data in this table. When multiple disk drives fail in a drive group, the data (Vdisk) is not available and the AMP goes into a FATAL state. At this point, Teradata does realize that an AMP is not available and Teradata restarts. The disk array controller will send “fault” or error messages to the SWS.

The SWS will also get “fault” messages indicating that Teradata has restarted.

Fallback and RAID 1 Example (cont.)

Assume two disk drives have failed in the same drive group. Is Fallback needed?



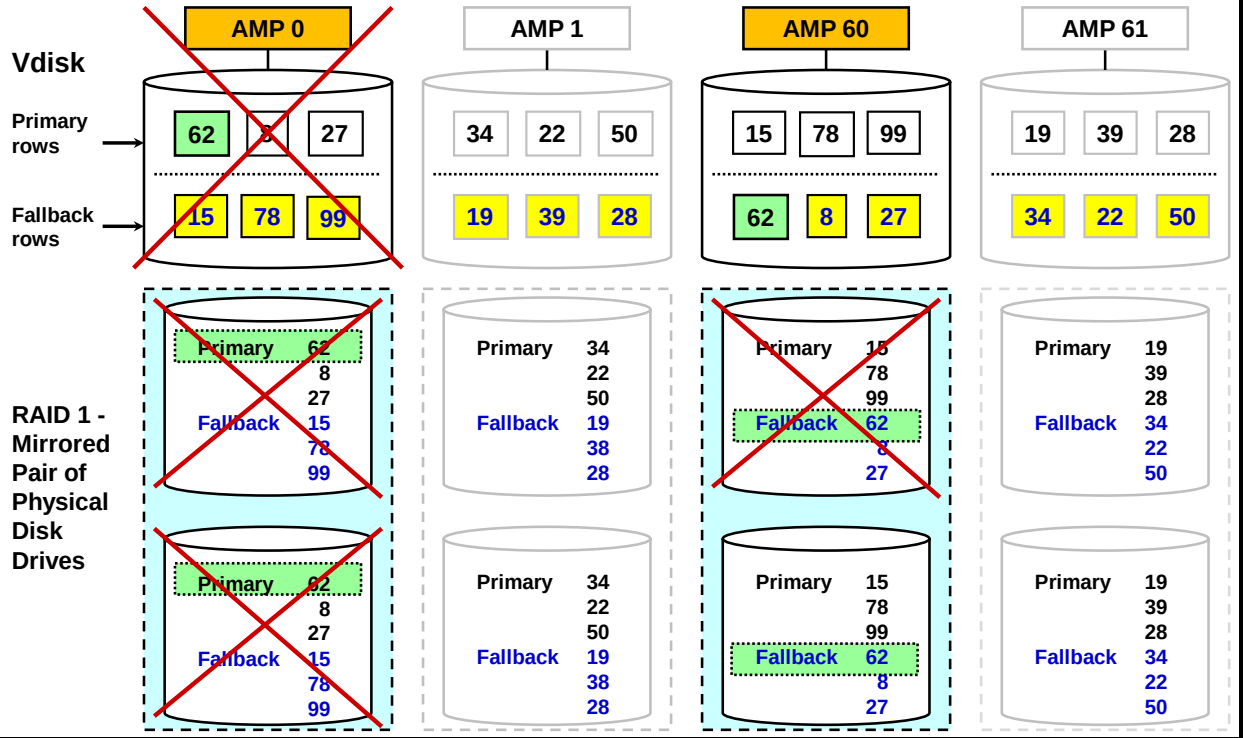
Fallback and RAID 1 Example (cont.)

The example of how Fallback and RAID 1 Mirroring work together is continued.

In this example, assume three disk drives have failed – two failed drives are in the first drive group and one failed drive is in the third drive group. Is Fallback needed? Yes, if you need to access the data in this table. When multiple disk drives fail in a drive group, the data (Vdisk) is not available and the AMP goes into a FATAL state. However, the third AMP is still operational and online.

Fallback and RAID 1 Example (cont.)

Assume three disk drive failures. Is Fallback needed? Is the data still available?



Fallback vs. non-Fallback Tables Summary

Fallback tables have a major advantage in terms of availability and recoverability. They can withstand an AMP failure in each cluster and maintain full data availability. A second AMP failure in any cluster results in a system halt. A manual restart of the system is required in this circumstance.

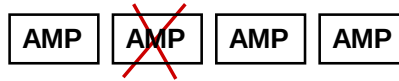
Non-Fallback tables are affected by the loss of any one AMP. The table continues to be accessible, but only for those AMPs that are still on-line. A one-AMP Primary Index access is possible, but a full table scan is not.

Fallback tables are easily recovered after a failure due to the availability of Fallback rows. **Non-Fallback tables** may only be restored from external medium in the event of a disaster.

Fallback vs. non-Fallback Tables Summary

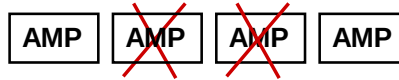
FALLBACK TABLES

One AMP **Down**



- Data fully available

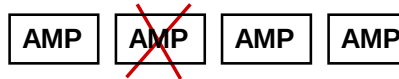
Two or more AMPs **Down**



- If different cluster, data fully available
- If same cluster, Teradata halts

Non-FALLBACK TABLES

One AMP **Down**



- **Data partially available;** queries that avoid down AMP succeed.

Two or more AMPs **Down**



- If different cluster, **data partially available;** queries that avoid down AMP succeed.
- If same cluster, Teradata halts

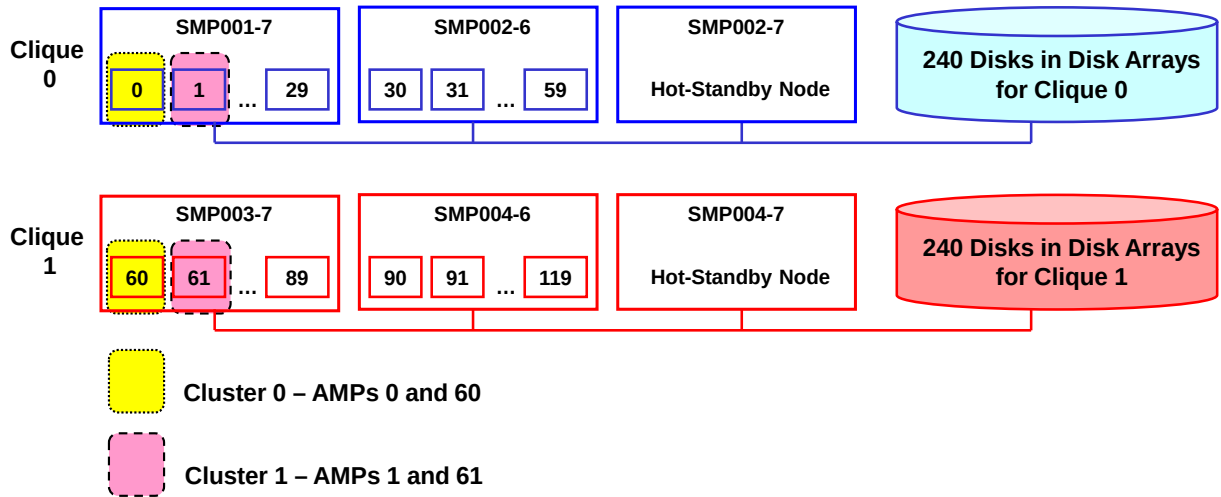
Clusters and Cliques

As you know, a **cluster** is a group of AMPs that act as a single fallback unit. A **clique** is a set of Teradata nodes that share a common set of disk arrays. Clusters provide data access protection in the event of an AMP failure (possibly because of a Vdisk failure). Cliques provide protection from node failures. Fallback provides protection from AMP failures.

The best availability for Teradata is to spread clusters across different cliques. When a system is installed (configured), PUT is used to effectively execute the “Default Cluster” function of the CONFIG utility. This function automatically determines the best clustering configuration.

The example on the facing page illustrates a 4+2 node system. Each clique consists of 3 nodes (2 TPA plus one Hot Standby Node – HSN) connected to a set of disk arrays with 240 disks. This example assumes each node is configured with 30 AMPs. Each AMP effectively 4 physical drives (2 sets of mirrored pairs) assigned to it. Since a clique has 60 AMPs in this example, there are 240 disk drives (in multiple arrays) associated with each clique.

Clusters and Cliques



To provide the highest availability, the goal is to interleave clusters across cliques and cabinets.

Locks

Locking prevents multiple users who are trying to change the same data at the same time from violating the data's integrity. This concurrency control is implemented by **locking** the desired data. Locks are automatically acquired during the processing of a request and released at the termination of the request. In addition, users can specify locks.

There are four types of locks: Exclusive, Write, Read, and Access.

Exclusive locks are only applied to databases or tables, never to rows. They are the most restrictive type of lock; all other users are locked out. Exclusive locks are used rarely, most often when structural changes are being made to the database.

Write locks enable users to modify data while locking out all other users except readers not concerned about data consistency (Access lock readers). Until a Write lock is released, no new read or write locks are allowed.

Read locks are used to ensure consistency during read operations. Several users may hold concurrent read locks on the same data, during which no modification of the data is permitted.

Access locks can be specified by users who are not concerned about data consistency. The use of an access lock allows for reading data while modifications are in process. Access locks are designed for decision support on large tables that are updated only by small single-row changes. Access locks are sometimes called “stale read” locks, i.e. you may get ‘stale data’ that hasn’t been updated.

Three levels of database locking are provided:

- **Database** - locks all objects in the database
- **Table** - locks all rows in the table or view
- **Partition** - locks all rows within the specified partitions (15.10 feature)
- **Row Hash** - locks all rows with the same row hash

The type and level of locks are automatically chosen based on the type of SQL command issued. The user has, in some cases, the ability to upgrade or downgrade the lock.

For example, if an SQL UPDATE command is executed without a WHERE clause, a WRITE lock is placed on the table. If an SQL UPDATE command is executed **with** a WHERE clause that specifies a Primary Index value, then a row hash lock is used.

There are four types of locks:

Exclusive	– prevents any other type of concurrent access
Write	– prevents other reads, writes, exclusives
Read	– prevents writes and exclusives
Access	– prevents exclusive only

Locks may be applied at three levels:

Database	– applies to all tables/views in the database
Table/View	– applies to all rows in the table/views
Partition	– applies to all rows in the specified partition(s) – 15.10
Row Hash	– applies to all rows with same row hash

Lock types are automatically applied based on the SQL command:

SELECT	– applies a Read lock
UPDATE	– applies a Write lock
CREATE TABLE	– applies an Exclusive lock

Locking Modifier

This option precedes an SQL statement and locks a database, table, view, or row hash. The locking modifier overrides the default usage lock that Teradata places on a database, table, view, or row hash in response to a request.

Note: The DROP TABLE access right is required on the table in order to upgrade a READ or WRITE LOCK to an EXCLUSIVE LOCK.

ACCESS

Access locks have many advantages. This allows quick access to data, even if other requests are updating the data. They also have minimal effect on locking out others – when you use an access lock; virtually all requests are compatible with your lock except exclusive locks

NOWAIT

If a resource is locked and an application does not want to wait for that lock to be released, the Locking Modifier NOWAIT option can be used. The NOWAIT option indicates that if the lock cannot be obtained, then the statement will be aborted.

This option is used in situations where it is not desirable to have a statement wait for resources, possibly also tying up resources in the process of waiting.

Example:

```
LOCKING TABLE tablename FOR WRITE NOWAIT UPDATE .... ;
```

```
*** Failure 7423 Object already locked and NOWAIT.  
Transaction Aborted. Statement# 1, Info =0
```

The user is informed with a 7423 error status code that indicates the lock could not be placed due to an existing, conflicting lock.

LOAD COMMITTED

Load Isolation (LDI), a TD 15.10 feature, permits a concurrent read on committed rows while the underlying table is being modified.

- Underlying tables must be defined as 'load isolated tables'

Readers do not block writers and writers do not block readers when load isolated. Load Isolation provides customers compatibility for applications that require read of committed rows instead of existing Dirty reads that has associated anomalies. Load isolation is provided to enhance concurrency and yet read load committed rows. Plus Teradata Load Isolation improves compatibility with other competitors such as Oracle's Read Committed transaction isolation level.

Locking Modifier

The locking modifier overrides the default usage lock that Teradata places on a database, table, view, or row hash in response to a request.

Certain locks can be upgraded or downgraded:

LOCKING ROW FOR ACCESS

```
SELECT * FROM Table_A;
```

An "Access Lock" allows the user to access (read) an object that has a READ or WRITE lock associated with it.

In this example, even though an access row lock was requested, a table level access lock will be issued because the SELECT causes a full table scan.

Note: A "Locking Row" request must be followed by a SELECT.

LOCKING TABLE Table_B FOR EXCLUSIVE

```
UPDATE Table_B SET A = 2019;
```

This request asks for an exclusive lock, effectively upgrading the lock.

LOCKING TABLE Table_C FOR WRITE NOWAIT UPDATE Table_C SET A = 2020;

The NOWAIT option is used if you do not want your transaction to wait in a queue.

NOWAIT effectively says to abort the transaction if the locking manager cannot immediately place the necessary lock. **Error code 7423** is returned if the lock request is not granted.

LOCKING TABLE Table_LDI FOR LOAD COMMITTED SELECT * FROM Table_LDI;

TD 15.10, only committed rows will be returned. This lock mode behaves like ACCESS lock if the table is non-LDI table. For a LDI table, the lock mode implies read of only committed rows.

Rules of Locking

As the facing page illustrates, a new lock request must wait (queue) behind other incompatible locks that are either in queue or in effect. The new Read lock must wait until the write lock ahead of it is released before it goes into effect.

In the second example, the second Read lock request may occupy the same position in the queue as the Read lock that was already there. When the current Write lock is released, both requests may be given access concurrently. This only happens when locks are compatible.

When an SQL statement provides row hash information, a row hash lock will be used. If multiple row hashes within the table are affected, a table lock is used.

Rules of Locking

Rules

Lock requests are queued behind all outstanding incompatible lock requests for the same object.

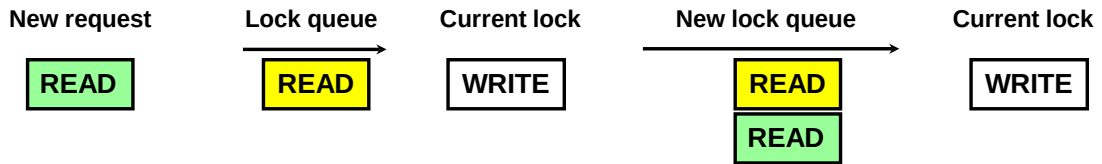
There is no time-out for SQL Requests waiting for a lock.

LOCK REQUEST	LOCK LEVEL HELD				
	NONE	ACCESS	READ	WRITE	EXCLUSIVE
ACCESS	Granted	Granted	Granted	Granted	Queued
READ	Granted	Granted	Granted	Queued	Queued
WRITE	Granted	Granted	Queued	Queued	Queued
EXCLUSIVE	Granted	Queued	Queued	Queued	Queued

Example 1 – New READ lock request goes to the end of queue.



Example 2 – New READ lock request shares slot in the queue.



Access Locks

Access locks have many advantages. They allow quick access to data, even if other requests are updating the data. They also have minimal effect on locking out others - when you use an access lock; virtually all requests are compatible with yours.

When doing large aggregations of numbers, it may be inconsequential if certain rows are being updated during the summation, particularly if one is only looking for approximate totals. Access locks are ideal for this situation.

Looking at Example 3, what happens to the Write lock request when the Read lock goes away? Looking at the chart, it will be “Granted” since Write and Access are considered compatible.

Another example not shown on the facing page:

Assume user1 is in ANSI mode and has updated a row, but hasn't entered COMMIT yet. The actual row in the table is updated on disk; the before-image is located in the TJ of the WAL log in case user1 decides to ROLLBACK.

If user2 accesses this row with an access lock, the updated row on disk is returned - even though it is locked and not committed yet. Assume user1 issues a ROLLBACK, then the before-image in the TJ is used to rollback the row on disk. If user2 selects the row a second time, user2 will get the row (original) that is now on disk.

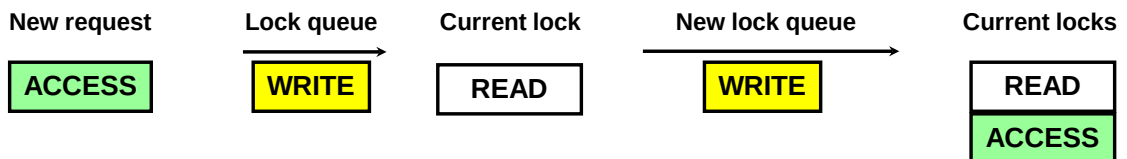
Access Locks

Rule

Lock requests are queued behind all outstanding incompatible lock requests for the same object.

LOCK REQUEST	LOCK LEVEL HELD				
	NONE	ACCESS	READ	WRITE	EXCLUSIVE
ACCESS	Granted	Granted	Granted	Granted	Queued
READ	Granted	Granted	Granted	Queued	Queued
WRITE	Granted	Granted	Queued	Queued	Queued
EXCLUSIVE	Granted	Queued	Queued	Queued	Queued

Example 3 – New ACCESS lock request granted immediately.



Advantages of Access Locks

- Permit quicker access to table in multi-user environment.
- Have minimal 'blocking' effect on other queries.
- Very useful for aggregating large numbers of rows.

Disadvantages of Access Locks

- May produce erroneous results if during table maintenance.

WAL – Write-Ahead Logging

WAL (Write-Ahead Logging or Write-Ahead Logic) is a recoverability/reliability feature that can possibly provide performance improvements in the area of database writes. In general, I/O increases with WAL and, therefore, it may reduce throughput for I/O bound workloads. However, the overall performance is expected to be better with WAL since the benefit of CPU improvement outweighs the I/O cost. There is some additional CPU cost for maintaining the WAL log so WAL may reduce throughput for CPU-bound workloads, but is minimal.

Simple example: Assume Teradata Mode, an implicit transaction, and you are doing an UPDATE of a single row in a block that has 300 rows in it.

1. Data block is read into FSG Cache.
2. UNDO row is written to WAL Log (effectively a before-image or TJ type row)
3. The data block in memory is changed and is marked as changed (not immediately written to disk - called deferred write).
4. REDO row is written to the WAL Log (effectively an after-image) - **writing a single REDO row is faster than writing a complete block**
5. The lock is released and the user gets a transaction completed message. Note the updated block is still in memory and hasn't been written to disk yet.

Note: Other users might be doing updates on rows in the same block and there might be multiple updates to the same block in memory.

6. At some point (maybe a half-second second later), the block needs to be written to disk. This is a deferred write and is done in the background.
- 6A. If the updated block has not changed size, then it can be written back-in-place. Before physically writing the block back-in-place, the updated block is first written to the WAL depot. After the datablock is successfully written to the WAL Depot, it is then physically written back-in-place.

Why is the block effectively written twice back to disk? A write operation can fail (called interrupted write) and this can corrupt a block on disk and potentially corrupt all 300 rows. This is a very rare occurrence, but can happen. The WAL Log only has 1 row (REDO row) of the row that has changed. Therefore, by writing the block first to the WAL Depot before writing back-in-place, Teradata ensures that a good copy of the entire datablock is written back-to-disk. The WAL Depot is **ONLY** used for blocks that haven't changed size - effectively write back-in-place operations. This is an extra internal I/O, but it provides data integrity and protection from interrupted write operations.

- 6B. If the block has changed size in memory (e.g., block expands to an additional sector), then the updated block is written to a new location on disk - it is not written to the WAL Depot. If there is an interrupted write, the original block has not been touched and the REDO rows along with the original data block can be used for recovery.

WAL can batch up modifications from multiple transactions and apply them with a single disk I/O, thereby saving I/O operations. WAL will help improve throughput for I/O-bound workloads. Obviously, Load utilities such as FastLoad and MultiLoad don't need to use WAL. Other functions such as FastPath operations use the WAL subsystem differently.

WAL – Write-Ahead Logging

WAL – Write-Ahead Logging

- Available with all Teradata systems – PDE (UNIX MP-RAS) and OpenPDE (Windows and Linux).
- Replaced Buddy Backup in PDE (UNIX MP-RAS) Teradata systems.

WAL is a primarily an internal recoverability/reliability feature that also provides performance improvements in the area of database writes.

- All modifications are represented in a log and the log is forced to disk at key times.
- Data Blocks updated in Memory, but not written immediately to disk.
- In place of the data block written to disk, the before image (UNDO row) and after image (REDO row) are written to a WAL buffer which is written to the WAL log on disk.
- **WAL can batch up modifications from multiple transactions and apply them with a single disk I/O, thereby saving I/O operations.** WAL will help improve throughput for I/O-bound workloads.
- Updated data blocks will be eventually aged out and written to disk.

Note: There are numerous DBS Control parameters to specify space allocations for WAL.

WAL Concepts

WAL has its own file system software and uses a fixed number of cylinders for the WAL Depot (varies by vdisk size and DBSControl parameters) and a dynamic number of cylinders for the WAL Log itself.

The WAL Depot consists of two types of slots:

- Large Depot slots
- Small Depot slots

The Large Depot slots are used by aging routines to write multiple blocks to the Depot area with a single I/O. The Small Depot slots are used when individual blocks that require Depot protection are written to the Depot area by foreground tasks.

The number of cylinders allocated to the Depot area is fixed at startup based on the settings of several internal DBS Control flags.

The number of Depot area cylinders allocated is per pdisk, so their total number depends on the number of Pdisks in your system. Sets of Pdisks belong to a subpool, and the system assigns individual AMPs to those subpools.

Because it does not assign Pdisks to AMPs, the system calculates the average number of Pdisks per AMP in the entire subpool from the vconfig GDO when it allocates Depot cylinders, rounding up the calculated value if necessary. The result is then multiplied by the specified values to obtain the total number of depot cylinders for each AMP. Using this method, each AMP is assigned the same number of Depot cylinders.

The concept is to disperse the Depot cylinders fairly evenly across the system. This prevents one pdisk from becoming overwhelmed by all the Depot writes for your system.

WAL (Write-Ahead Logging) is a transaction logging scheme maintained by the File System in which a write cache for disk writes of permanent data is maintained using log records instead of writing the actual data blocks at the time a transaction is processed. Multiple log records representing transaction updates can then be batched together and written to disk with a single I/O thus achieving a large savings in I/O operations and enhancing system performance as a result.

The amount of space used for the WAL Log is dynamic. WAL contains before-images (TJ) and after-images (Redo) for transactions. For example, the number of TJ images is very dependent on the type of transaction. Updating every row in a large table places a lot of TJ images into WAL.

Note: Prior to the V2R6.2 release, Teradata systems running under UNIX MP-RAS systems utilized a facility referred to as “buddy backup”.

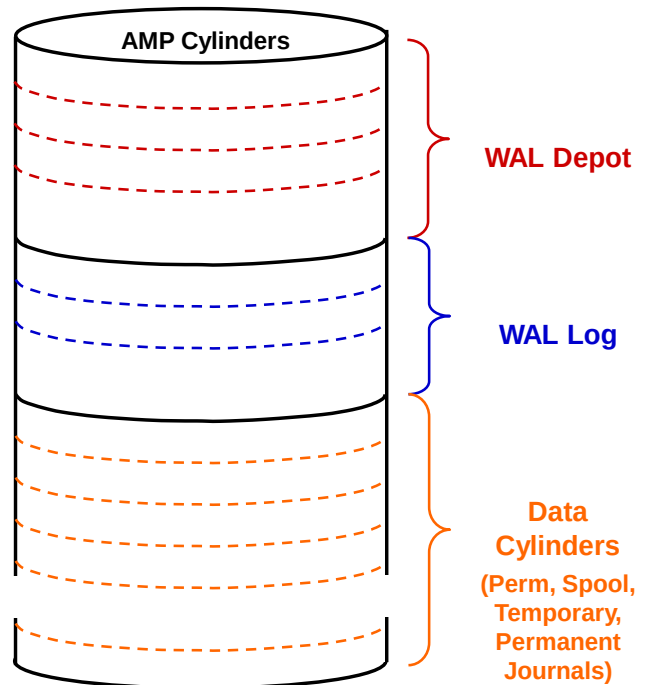
WAL Concepts

WAL Depot

- Fixed number of cylinders allocated to each AMP.
- Used for Write-in-Place operations.
- Teradata first writes data to WAL Depot and if successful, then writes to disk.
- WAL Logic will attempt to group a number of blocks to write to WAL Depot.

WAL Log

- Dynamic number of cylinders used by each AMP.
- Used for new block allocations on disk.
- Contains before-images (UNDO) and after-images (REDO) for transactions – used with both Write-in-Place or new block allocations.
- Updated data blocks will be eventually aged out and written to disk.



Allocation of cylinders is not contiguous.

Transient Journal

The **Transient Journal** exists to permit the successful rollback of a failed transaction. Transactions are not committed to the database until an End Transaction request has been received by the AMPs, either implicitly or explicitly. Until that time, there is always the possibility that the transaction may fail in which case the participating table(s) must be restored to their pre-transaction state.

The **Transient Journal** maintains a copy of all before images of all rows affected by the transaction. If the event of transaction failure, the before images are reapplied to the affected tables, the images are deleted from the journal and a rollback operation is completed. In the event of transaction success, at the point of transaction commit, the before images for the transaction are discarded from the journal.

In Summary, if a Transaction fails (for whatever reason), the before images in the transient journal are used to return the data (in the tables involved in the transaction) to its original state.

ALTER TABLE Misconception

An often repeated misconception is that when an ALTER TABLE command is executed, the “altered data” is written to the transient journal/WAL log.

For example, assume an ALTER TABLE ADD COLUMN is executed. Assuming the altered table is a large table, this operation may be time consuming. Why?

The reason that ALTER TABLE ADD COLUMN may be time consuming is that each row is rebuilt, each block is rebuilt, and the cylinder is rebuilt (re-lays out each cylinder). It does so one cylinder at a time in order to avoid having to make a full copy of the table. A new cylinder is allocated, each datablock is read, the records are reformatted and written to a new datablock which is placed on the new cylinder. When all the blocks are complete the new cylinder is committed to disk. Then the old cylinder is released and the process is repeated until all cylinders are completed. This work reformats the rows and blocks in the same way as an INSERT-SELECT.

The only thing that is written to the WAL log is a single record per cylinder as a cylinder is committed. This is used in case the ALTER TABLE is interrupted – e.g., by a Teradata restart. The last cylinder log record is used to restart the process from the last completed cylinder and complete it. This is also why an ALTER TABLE cannot be aborted once it has been started.

Transient Journal

Transient Journal – provides transaction integrity

- A journal of transaction “before images” (UNDO rows) maintained within WAL.
- Provides for automatic rollback in the event of TXN failure.
- Is automatic and transparent.
- “Before images” are reapplied to table if a transaction fails.
- “Before images” are discarded upon transaction completion.

Successful TXN

BEGIN TRANSACTION

UPDATE Row A – Before image Row A recorded (*Add \$100 to checking*)

UPDATE Row B – Before image Row B recorded (*Subtract \$100 from savings*)

END TRANSACTION – Discard before images

Failed TXN

BEGIN TRANSACTION

UPDATE Row A – Before image Row A recorded

UPDATE Row B – Before image Row B recorded

(Failure occurs)

(Rollback occurs) – Reapply before images

(Terminate TXN) – Discard before images

Recovery Journal for Down AMPs

After the loss of any AMP, a **Down-AMP Recovery Journal** is started automatically. Its purpose is to log any changes to rows which reside on the down AMP. Any inserts, updates, or deletes affecting rows on the down AMP, are applied to the Fallback copy within the cluster. The AMP that holds the Fallback copy logs the Row ID in its Recovery Journal.

This process continues until such time as the down AMP is brought back on-line. As part of restart activity, the Recovery Journal is read and changed rows are applied to the recovered AMP. When the journal has been exhausted, it is discarded and those tables that are fallback-protected are fully recovered.

Recovery Journal for Down AMPs

Recovery Journal is:

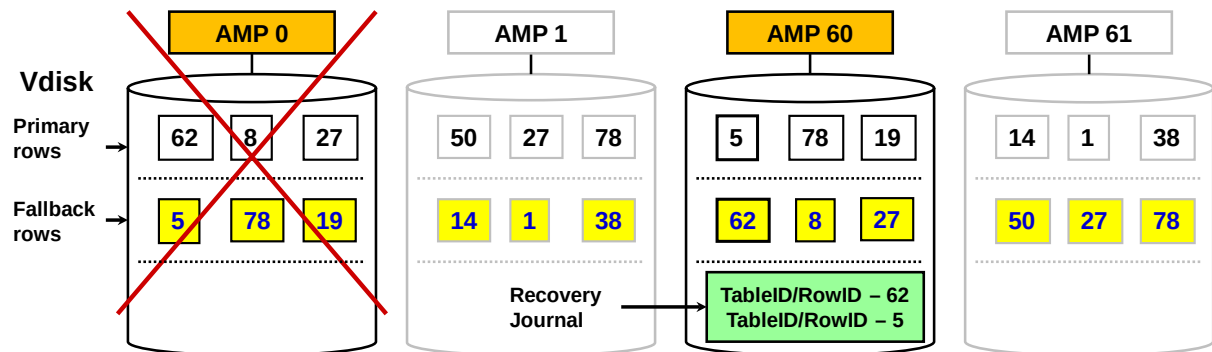
Automatically activated when an AMP is taken off-line.
Maintained by the other AMPs in a cluster.
Totally transparent to users of the system.

While AMP is off-line

Journal is active.
Table updates continue as normal.
Journal logs Row IDs of changed rows for down-AMP.

When AMP is back on-line

Restores rows on recovered AMP to current status.
Journal discarded when recovery complete.



Permanent Journal

The purpose of the **Permanent Journal** is to provide selective or full database recovery to a specified point in time. It permits recovery from unexpected hardware or software disasters. The Permanent Journal also has the effect of reducing the need for full table backups which can be costly both in time and resources.

The Permanent Journal is an optional journal and its features must be customized to the specific needs of the installation. The journal may capture before images (for rollback), after images (for rollforward), or both. Additionally, the user must specify if single images (default) or dual images (for fault-tolerance) are to be captured.

A Permanent Journal may be shared by multiple tables or multiple databases. The journal captures images concurrently with standard table maintenance and query activity. The cost in additional required disk space may be calculated in advance to ensure adequate disk reserve.

The journal is periodically dumped to external media, thus reducing the need for full table backups – in effect, only the changes are backed up.

Permanent Journal

The Permanent Journal is an optional, user-specified, system-maintained journal which is used for recovery of a database to a specified point in time.

The Permanent Journal:

- Is used for recovery from unexpected hardware or software disasters.
- May be specified for ...
 - One or more tables
 - One or more databases
- Permits capture of Before Images for database rollback.
- Permits capture of After Images for database rollforward.
- Permits archiving change images during table maintenance.
- Reduces need for full table backups.
- Provides a means of recovering NO FALLBACK tables.
- Requires additional disk space for change images.
- Requires user intervention for archive and recovery activity.

Archiving and Recovering Data

The purpose of the **ARC utility** is to allow for the archiving and restoring of database objects which may have been damaged or lost. There are several scenarios where restoring objects from external media may be necessary.

- Restoring of non-Fallback tables after a disk failure.
- Restoring of tables which have been corrupted by batch processes which may have left the data in an 'uncertain' state.
- Restoring of tables, views or macros which have been accidentally dropped by the user.
- Miscellaneous user errors resulting in damaged or lost database objects.

Teradata's Backup and Recovery (BAR) architecture provides solutions from Teradata Partners. Two examples are:

- NetVault – from BakBone software
- NetBackup – from Symantec (Veritas NetBackup by Symantec)

The **ASF2 utility** is an older utility that provides an X Windows based front-end for creation and execution of ARC command scripts. It is designed to run on UNIX MP-RAS.

ARC or DSA

- ARC – the Archive/Restore utility (arcmain) used with older Teradata systems/releases
- DSA – a Data Stream Architecture utility used with newer Teradata systems/releases
- Archives and restores data from/to Teradata
- Restores or copies data from archive media
- ARC permits data recovery to a specified checkpoint using Permanent Journals

Backup and Recovery (BAR)

- Example of BAR choices from different Teradata Partners
 - [NetBackup - Veritas NetBackup by Symantec](#)
 - [Tivoli Storage Manager – utilizes TARA](#)
- Provides Windows front end for ARC
- Easy creation of scripts for archive/recovery
- Provides job scheduling and tape management functions
- BAR was previously referred to as Open Teradata Backup (OTB)

Teradata Unity

The Teradata Unity Portfolio covers a suite of products.

Unity Director is a query routing solution that will load balance and maintain data consistency across two or more Teradata systems via SQL-Multi-Cast technology. It delivers Query Management, Multi-System Synchronization for data and database objects, and Auto Resubmit.

Unity Director handles SQL based interactions:

- Queries
- User Sessions
- Data Updates
- SQL Based Data Loads
- Database updates

Unity Director supports high availability by doing the following:

- Routing and re-routing user tasks
- Routing and re-routing database tasks such as database activities, data maintenance, and administration.

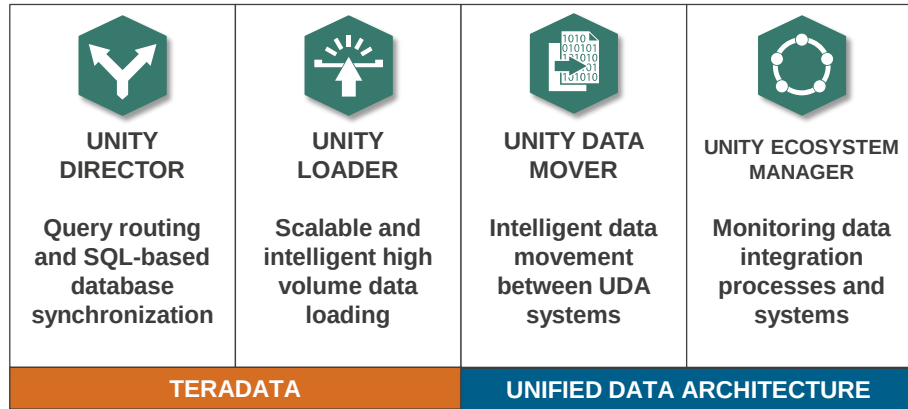
SQL Multicast is the patented technology that is at the core of Unity Director and leveraged by Unity Loader. This is what allows Unity Director to send requests to multiple Teradata systems, in order, and sequence/track progress.

Unity Loader is the “dual load” product in the Unity portfolio. It can take TPT loads (limited to TPT Load Operator in the 14.0 release) and “dual load” it to more than one Teradata system. In addition to this being a product-based solution that is robust and fully supported, the real benefit is that this is automatic and intelligent. Administrators do not need to tell Unity Loader where to put the data. It determines automatically by which data you are loading, to where it goes. It is highly scalable, allowing multiple servers to be configured for scale-out to get throughput and performance. It is fully integrated with Unity Director to sequence data changes.

Unity Data Mover supports moving tables and other data objects between systems. Unity Data Mover can be a complement to Unity Director and Unity Loader, handling the bulk table copy tasks in a complex ecosystem.

Unity Ecosystem Manager is the updated name for Multi-System Manager (MSM). It allows system administrators to manage single and Multi-System environments based on a comprehensive view of the entire ecosystem. While Unity Ecosystem Manager is used for administration and monitoring, its primary value in customer deployments is in the area of problem detection, notification, and troubleshooting. Instrumentation through Unity Ecosystem Manager will help customers understand the relationship of problems in their environment and how they affect business processes which is essential to meeting Service Level Agreements (SLA's).

Teradata Unity



Integrated Portfolio of Products That Turns a Multi-system Environment into an Orchestrated Analytical Ecosystem.

Teradata Multi-System Solution Architecture

This solution architecture diagram applies to dual system environments which have become very popular, as well as multi-system environments which many Teradata customers have either adopted or are considering. Multiple systems require data to be synchronized in a broader way, Unity Data Mover, Unity Director and Unity Loader are the preferred ways to do this and it needs to be able to synchronized in either direction for Active/Active

Adding business users requires adding a BI layer of servers and services that will use Unity Director to route query requests to the preferred system.

Ecosystem Manager monitors all hardware, processes and tables in the ecosystem, understanding the freshness and correctness of the data and adjusting Unity Director routing rules as conditions change. Director handles all the SQL-based commands. Loader handles bulk loads (limited to TPT Load in 14.0).

The 14.x release brought the addition of TPT Update to Unity Loader. It further enhanced the integration with Unity Data Mover, ensuring that Loader/Director are in sync with Data Mover for sequencing of data changes.

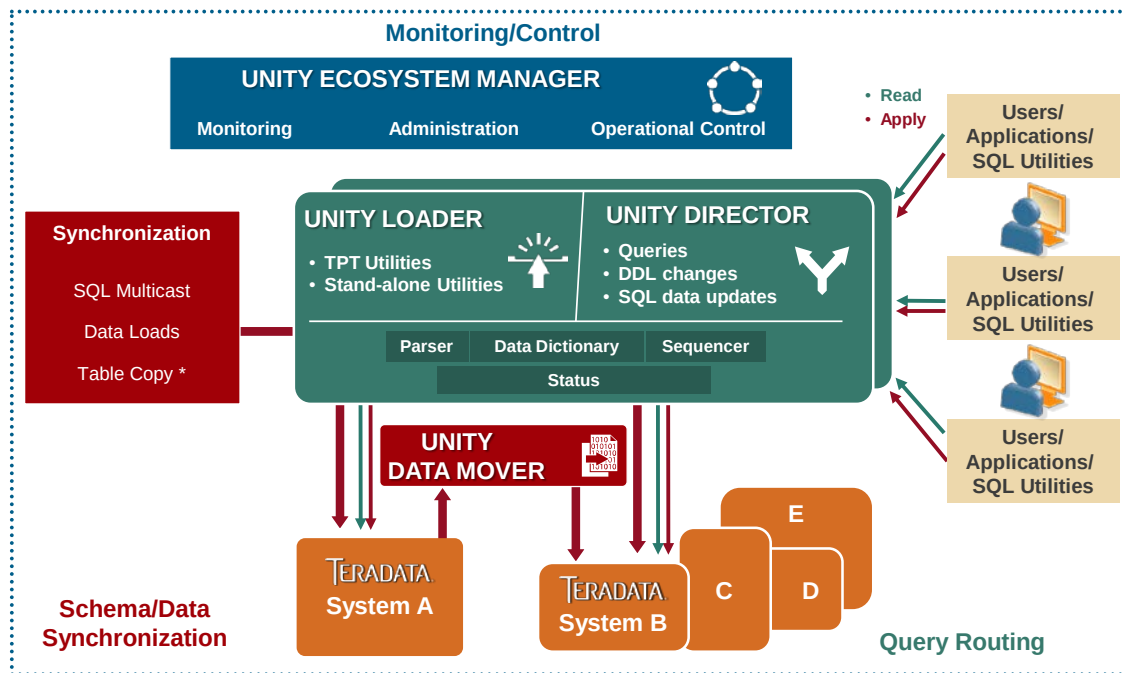
Business Objectives for Multiple Systems

- **Higher System Availability (HA)**
 - ✓ Enable Mission Critical applications
 - ✓ Disaster Recovery
 - ✓ Support an ADW environment where continuous availability is a requirement
 - ✓ Meet Service Level Agreements for access to data and user perceived performance
- **Workload Distribution**
 - ✓ Regional/Departmental Dependent Data Mart(s)
 - ✓ Appliance Support
 - Deep Analytics
 - Near Line Storage
 - ✓ Workload Isolation in lieu of Primary System workload management

Teradata Multi-Systems – Defined

- Dual Active/Active-Active
 - o Both Application Subject Areas updated in tandem
 - o Applications executing on both systems of a Dual Systems pair
- Active Standby
 - o Both Application Subject Areas updated in tandem
 - o Applications only executing on one system of a Dual Systems pair
- ACTIVE/Cold Standby (DR)
 - o One Application Subject Area updated, the other TD system is not up-to-date
 - o Applications only executing on one system of a Dual Systems pair

Teradata Multi-System Solution Architecture



* This method of data sync can coexist with Unity, but needs to be managed and coordinated outside Unity

Module 9: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 9: Review Questions

1. What is the WAL Depot used for?
 - a. UNDO Rows
 - b. New data blocks
 - c. Master Index updates
 - d. Write-in-place data blocks
2. Which two are placed into the WAL Log?
 - a. REDO Rows
 - b. UNDO Rows
 - c. New data blocks
 - d. Write-in-place data blocks

Match the item to a lettered description.

- | | |
|----------------------------|--|
| _____ 1. Database locks | a. Provides for TXN rollback in case of failure |
| _____ 2. Table locks | b. Teradata Backup and Recovery applications |
| _____ 3. Row Hash locks | c. Protects all rows of a table |
| _____ 4. FALLBACK | d. Logs changed rows for down AMP |
| _____ 5. Cluster | e. Provides for recovery to a point in time |
| _____ 6. Recovery journal | f. Applies to all tables and views within |
| _____ 7. Transient journal | g. Lowest level of protection granularity |
| _____ 8. NetBackup/Tivoli | h. Protects tables from AMP failure |
| _____ 9. Permanent journal | i. Protects database from a physical drive failure |
| _____ 10. Disk Array | j. Group of AMPs used by Fallback |

Notes



How Teradata uses MPP Systems

After completing this module, you will be able to:

- Identify items that are placed into FSG cache.
- Describe the fundamental relationship between Linux, logical units, and disk array controllers.
- Describe the fundamental relationship between Vdisks, Pdisks, LUNs, and partitions.

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata and the Processing Node	10-4
FSG Cache	10-4
Memory and the Teradata Database.....	10-6
Node Memory – Summary.....	10-8
Determining FSG Cache	10-8
Linux Managed Memory and FSG Cache	10-10
Linux Vproc Number Assignment	10-12
Disk Arrays from a Linux Perspective.....	10-14
Logical Units and Partitions.....	10-16
Teradata and Disk Arrays.....	10-18
Teradata 2850 Logical View	10-20
Teradata 6800 (3+1) Logical View	10-22
Example of 6800H Vdisk.....	10-24
Cylinder – AMP Allocation Unit	10-26
Teradata File System Concepts.....	10-28
Teradata Vdisk Size Limits (older).....	10-28
Cylinder Index Segregation (15.10).....	10-30
Full Cylinder Read	10-32
Summary	10-34
Module 11: Review Questions	10-36

Teradata and the Processing Node

The example on the facing page illustrates an IntelliFlex™ 2.0 node processing node running Linux SUSE 11 and Teradata 16.00.

Memory will initially be allocated for the operating system and Teradata vprocs. PDE will calculate how much memory to allocate to itself for FSG (File Segment Cache) based on memory not being used by the operating system and the Teradata vprocs. PDE software will manage the FSG memory space.

Practical experience (for most environments) indicates that the operating system (e.g., Linux) may need more than this initial allocation during startup. For these reasons, PDE is not assigned all of the remaining memory for FSG cache, but a percentage (e.g., 90%) of the remaining memory.

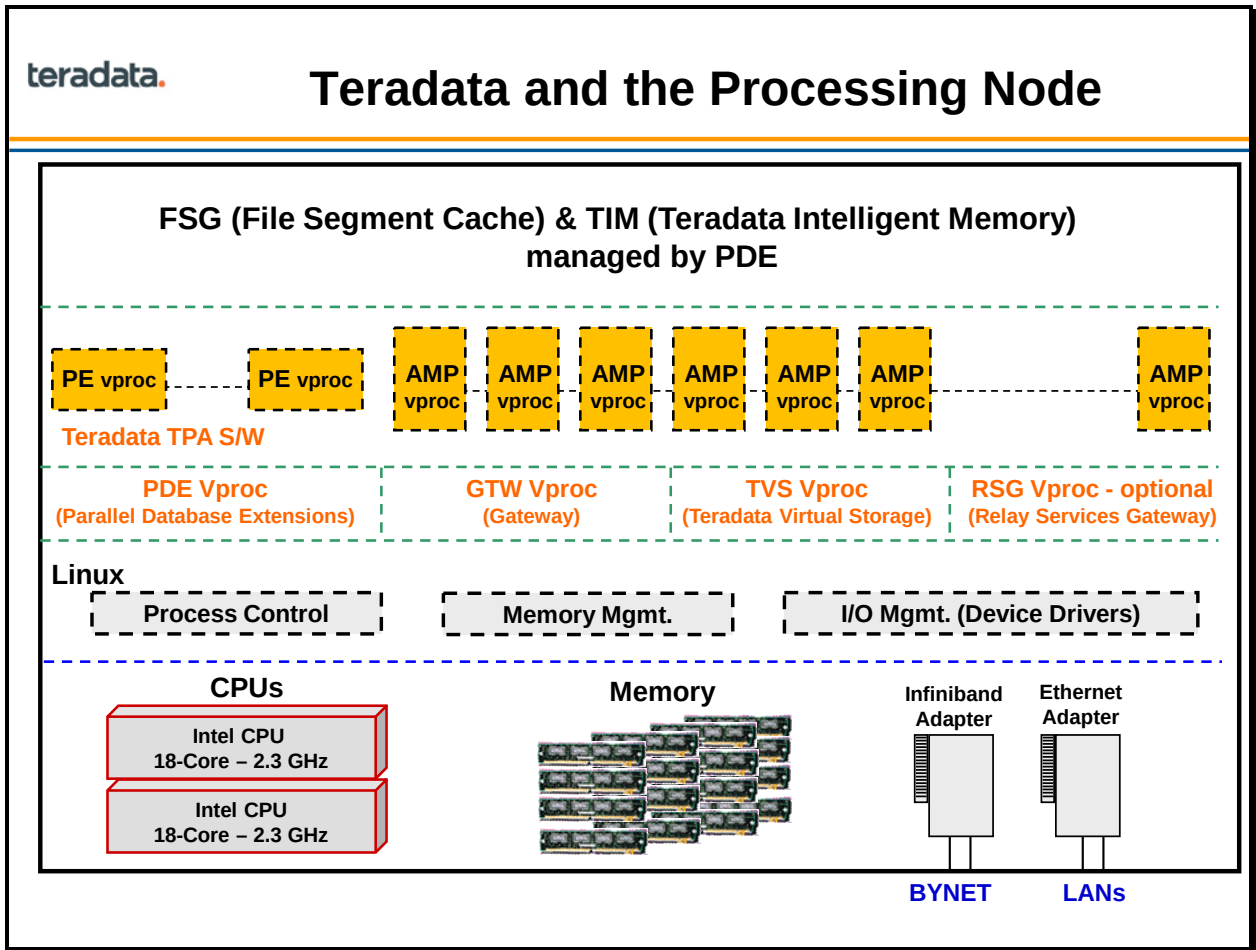
Also note that LAN and Channel adapters (PBSA) also require memory for network and channel activity. For example, each channel adapter uses memory buffers up to 500 MB in size. For 56xx and 6xxx systems, Channel Adapters are not utilized within a TPA node. These are implemented in “Extended Node Types”.

FSG Cache

FSG Cache is primarily used by the AMPs to access memory resident database segments.

When the Teradata Database needs to read a database block, it checks FSG Cache first.

Teradata and the Processing Node



Memory and the Teradata Database

The example on the facing page illustrates an IntelliFlex™ node with 512 GB of memory executing the Teradata Database. This example assumes 2 PEs, 48 AMPs, PDE, GTW, RSG, and 4 TVS vprocs for a total of 57 vprocs in this node. This means that memory will have to be allocated for the 57 vprocs. In this example, the operating system, device drivers, and Teradata vprocs will initially use approximately 33 GB of memory of the 512 GB.

Practical experience (for most environments) indicates that the operating system (e.g., Linux) may need more than this initial allocation. Parsing Engines and especially AMPs will typically use more than their initial allocation of memory (80 MB). In TD 16.10, redistribution buffers for an AMP may use an additional (or more) 220 MB of memory for a total of 300 MB of memory per AMP. Therefore, PDE is not assigned all of the remaining memory for FSG cache, but a percentage of the remaining memory. The default of 90% for FSG Cache Percent works for most systems with 512 GB of memory per node. In this example, 90% of GB $(512 - 33) = 431$ GB of FSG cache.

The amount of memory that an AMP uses can be determined/verified by using the “ctl” utility “hardware” command. For example, each AMP (of 48) has an average of 8.972 GB of memory. Therefore, $48 \text{ AMPs} \times 8.972 = 431$ GB of FSG cache.

For systems with 256 GB of memory per node, the current recommendation is to set the FSG Cache Percent to 85%.

Memory and the Teradata Database

Example of IntelliFlex 2.0 node
with 2 PEs, 48 AMPs, and 512 GB
of memory:

Memory

Linux, Device Drivers, and
space for vprocs ≈ 33 GB

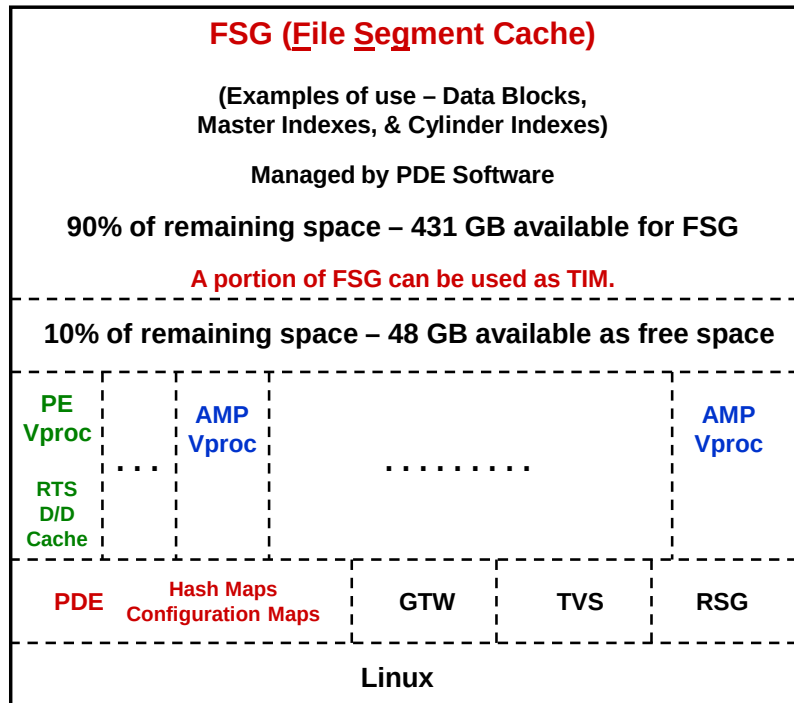
512 GB
- 33 GB
479 GB

FSG Cache 90%
FSG Cache ≈ 431 GB

Free Memory ≈ 48 GB

Examples of objects that are
memory resident:

Hash Maps
Configuration Maps
Master Indexes
RTS – Request-to-Steps Cache
D/D – Data Dictionary Cache



Ex. 512 GB Memory

Node Memory – Summary

Practical experience (for most environments) indicates that Linux and Teradata vprocs need more memory than initially allocated during normal processing periods. Prior to V2R5, it was recommended that at least 20 MB to 40MB of additional free memory be available for each AMP. With 32-bit systems, it is recommended that a node have at least 60 – 80 MB of free memory available for each AMP. With 64-bit systems and Teradata 16.00, each AMP may use up to 300 MB of memory. This would be an additional 220 MB of memory per AMP.

This is accomplished by not giving 100% of the remaining memory to FSG. It is always recommended that the FSG Cache Percent be set to a value less than 100%. The default of 90% for FSG Cache Percent works well for most systems with 512 GB of memory per node. 85% usually works well with systems that have 256 GB of memory per node.

Determining FSG Cache

The “ctl” utility can be used to determine how much FSG cache memory is actually available to a node.

Using the “ctl” utility, the **hardware** command will report the amount of FSG cache for each AMP. The values below represent the average amount of FSG memory per AMP. Examples are shown below.

For a 5600H node with 40 AMPs, the report will indicate 1,753,856 KB/per AMP.
 $1,753,856 \text{ KB/AMP} \times 40 = 70,154,240 \text{ KB}$ or approximately 70 GB of FSG cache.

For a 5650H node with 47 AMPs, the report will indicate 1,472,000 KB/per AMP.
 $1,472,000 \text{ KB/AMP} \times 47 = 69,184,000 \text{ KB}$ or approximately 69.2 GB of FSG cache.

For a 6650H node with 42 AMPs, the report will indicate 1,669,120 KB/per AMP.
 $1,669,120 \text{ KB/AMP} \times 42 = 70,103,040 \text{ KB}$ or approximately 70.1 GB of FSG cache.

For a 2700 node with 36 AMPs, the report will indicate 5,918,720 KB/per AMP.
 $5,918,720 \text{ KB/AMP} \times 36 = 213,073,920 \text{ KB}$ or approx. 213 GB of FSG cache.

For a 6750 node with 28 AMPs, the report will indicate 15,790,763 KB/per AMP.
 $15,790,763 \text{ KB/AMP} \times 28 = 442,141,364 \text{ KB}$ or approx. 442 GB of FSG cache.

For a 6800 node with 28 AMPs, the report will indicate 15,800,320 KB/per AMP.
 $15,800,320 \text{ KB/AMP} \times 28 = 442,408,960 \text{ KB}$ or approx. 442 GB of FSG cache.

For an IntelliFlex node with 48 AMPs, the report will indicate 8,972,288 KB/per AMP.
 $8,972,288 \text{ KB/AMP} \times 48 = 430,669,824 \text{ KB}$ or approx. 431 GB of FSG cache.

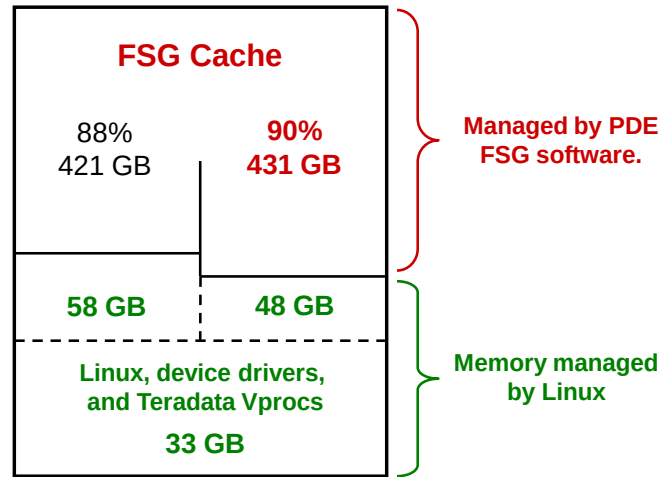
Node Memory – Summary

Based on the configuration and FSG Cache Percent value, PDE will determine the amount of memory to allocate for FSG cache.

However, vprocs (especially AMPs) will use more than their initial memory allocations during normal processing (e.g., hash join buffers, redistribution buffers, aggregations buffers, etc.).

A basic guideline for AMPs:

- 64-bit systems – assume a minimum of 250 MB per AMP



FSG – pool of memory managed by PDE and each AMP uses what it needs.

ctl Parameter – FSG Cache Percent – for systems with 512 GB of memory per node, the design center recommendation is 90%; for systems with 256 GB of memory per node, the recommendation is 85%.

Linux Managed Memory and FSG Cache

The facing page lists examples of how Linux managed memory (free memory) and FSG cache is used.

Memory managed and used by Linux and the vprocs is sometimes called “free memory”. The main code (on a TPA node) that uses free memory is the operating system and Teradata vprocs.

A brief description of Teradata Vprocs:

- AMP Access module processors perform database functions, such as executing database queries. Each AMP owns a portion of the overall database storage.
- GTW Gateway vprocs provide a socket interface to Teradata Database on Windows and Linux systems. On MP-RAS systems, the same functionality is provided by gateway software running directly on the system nodes within the PDE vproc.
- Node (or Base) PDE vproc - the node vproc handles PDE and operating system functions not directly related to AMP and PE work. Node vprocs cannot be externally manipulated, and do not appear in the output of the Vproc Manager utility.
- PE Parsing engines perform session control, query parsing, security validation, query optimization, and query dispatch.
- RSG Relay Services Gateway provides a socket interface for relaying dictionary changes to the Teradata Meta Data Services (MDS) utility.
- TVS Manages Teradata Database storage. AMPs acquire their portions of database storage through the TVS vproc (an early release of Teradata named this vproc VSS).

When Teradata needs to read a database block, it checks FSG Cache first.

Examples of how FSG Cache is used

- Permanent data blocks
- Cylinder Indexes
- Spool data blocks
- Transient Journals
- Permanent Journals
- Synchronized scan (sync scan) data blocks

Memory managed by Linux is referred to as “free memory”.

- **Teradata Vprocs**
 - **AMP** – includes AMP worker tasks
 - **PE** – Session control, Parser, Optimizer, Dispatcher
 - **PDE (Parallel Database Extensions)** – messaging, FSG space management, etc.
 - **GTW (Gateway)** – Logon Security, Session Context, Connection to Client
 - **RSG (Relay Services Gateway)** – Optional; MDS auto-update
 - **TVS (Teradata Virtual Storage)** – manages Teradata Virtual Storage
- **Administrative and/or user programs such as:**
 - kernel resources and administrative program text and data
 - message buffers (ex., TCP/IP)

Memory managed by PDE is called FSG cache. FSG cache is primarily used by the AMPs to access memory resident database segments.

- **When Teradata needs to read a database block, it checks FSG Cache first.**
 - Master and Cylinder Indexes
 - Permanent data blocks
 - Spool data blocks
 - Journal blocks; Transient Journal and/or Permanent Journals
 - Synchronized scan (sync scan) data blocks

A portion of FSG may be utilized as TIM (Teradata Intelligent Memory).

Linux Vproc Number Assignment

The facing page describes how Vprocs are assigned numbers.

With OpenPDE systems, gateway software is implemented in as separate vproc (named GTW) from the PDE vproc. With MP-RAS systems (PDE), gateway software is incorporated into the PDE vproc.

Within a multi-node single clique system, it is possible for one of the nodes to have a second TVS vproc. This may seem like an anomaly, but this is normal.

For example, assume a 3+1 single clique system:

- In order for fallback to be most effective, a single clique is divided into two subpools of storage and AMPs which reference that storage. Fallback is then setup as a cluster size of two between the two subpools of AMPs. An Allocator (part of TVS vproc) only deals with a single sub-pool of storage. Since in this case we are dividing up two subpools into three nodes, one of the nodes has about half of its storage in one subpool and half of its storage in the other subpool. Therefore, that node needs to have two Allocator vprocs, one for each sub-pool of storage. Any system with more than one clique has only one sub-pool per clique and this anomaly goes away.
- A single node system (which is of course a single clique) also has two sub-pools for the same reason.

With Teradata 13.10 (and previous releases), vproc number ranges are:

AMPs – 0, 1, 2, ...
PEs – 16383, 16382, 16381, ...
GTW – 8192, 8193, 8194, ...
VSS – 10238, 10237, 10236, ...
PDE – 16384, 16385, 16386, ...
RSG – 9215, 9216, 9217, ... (Optional)

When a system is configured with PUT, the installer is presented with an option to choose large vproc numbers if configuring a system with more than 8,192 AMPs. Therefore, starting with Teradata 14.0, optional vproc number ranges are:

AMPs – 0, 1, 2, ...
PEs – 30719, 30718, 30717, ...
GTW – 22528, 22529, 22530, ...
TVS – 28671, 28670, 28669, ...
PDE – 30720, 30721, 30722, ...
RSG – 26623, 26622, 26621, ... (Optional)

Linux Vproc Number Assignment

Each Teradata Vproc is assigned a unique Vproc number in the system. For example:

Typical Vproc assignments starting with Teradata 14.0:

AMP Vproc #s (start at 0 and increment by 1)

- First AMP 0
- Second AMP 1
- Third AMP 2

PE Vproc #s (start at 30719 and decrement by 1)

- First PE 30719
- Second PE 30718
- Third PE 30717

Appear in DD/D and utilities
such as Teradata Studio,
Viewpoint, etc.

Disk Arrays from a Linux Perspective

The Operating System is used to read and/or write data to/from an individual disk. Disk arrays trick the operating system into thinking it is writing to a single disk. A disk array LUN looks to the operating system like a single disk. When the operating system gets ready to do a read or a write, the disk array controller steps in and says, "I'll handle that for you". The operating system says, "I am writing to a single disk and its address is c10t0d0s1".

The operating system does not directly read or write to a disk in a disk array environment. The operating system communicates with the disk array controller. The operating system actually reads or writes the data from a logical unit (often referred to as a LUN or a Volume). A logical unit (LUN) or Volume is a logical disk and not a physical disk.

The operating system does not know (or care) if a LUN or Volume is RAID 0, RAID 1, RAID 5, or RAID 6. The operating system does not know if the drive group is one disk, two disk, or four disks. The operating system does not know if the data is spread across one disk or four disks. The operating system simply sees the logical unit as a single disk.

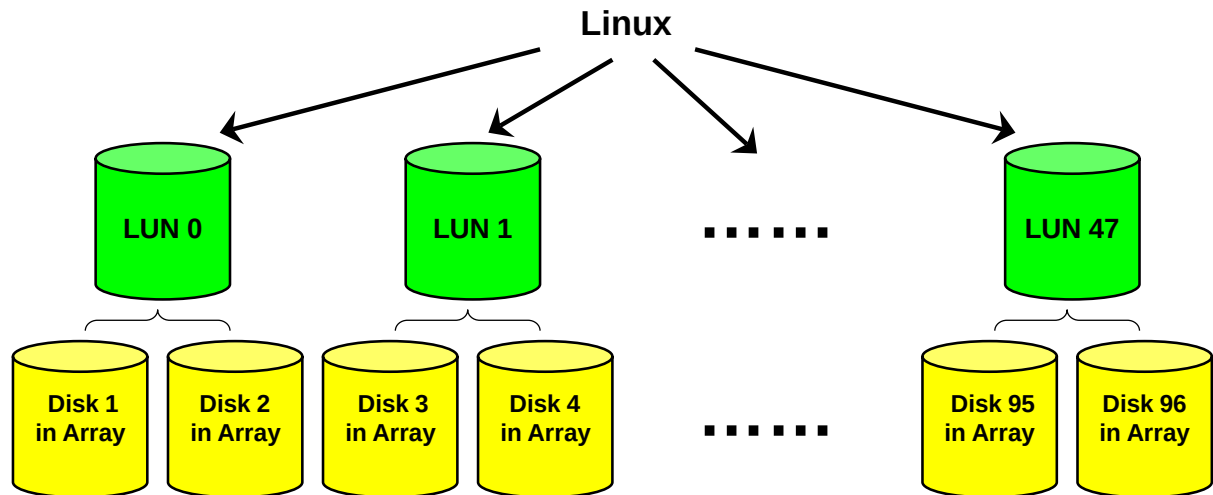
The standard operating system utilities that are used to manage, configure, and utilize a physical disk are also used to manage, configure, and utilize a logical disk or LUN. With the Teradata Database, the **PUT** utility is used to configure the disk array.

The array controller performs the actual input/output operations to its disks. The array controller is responsible for handling the different RAID technologies.

Disk Arrays from an Linux Perspective

A **logical unit (LUN)** or **Volume** is a single disk to Linux.

- Linux does not know or care about the specific RAID technology being used for a LUN or Volume.
- Linux uses LUNs to communicate with disk array controllers.
- It is possible to divide a LUN into one or more partitions (or slices for MP-RAS).



Linux thinks it is reading and writing to 48 logical disks.

Logical Units and Partitions

A logical unit (just like a physical disk) can be divided into multiple partitions (or slices with MP-RAS). A partition is a portion of a logical unit. A partition is typically used in one of two ways.

- Used to hold the Linux file system on SMP node internal disks.
- Provides a raw data storage area (raw disk partition) that is used by Teradata.

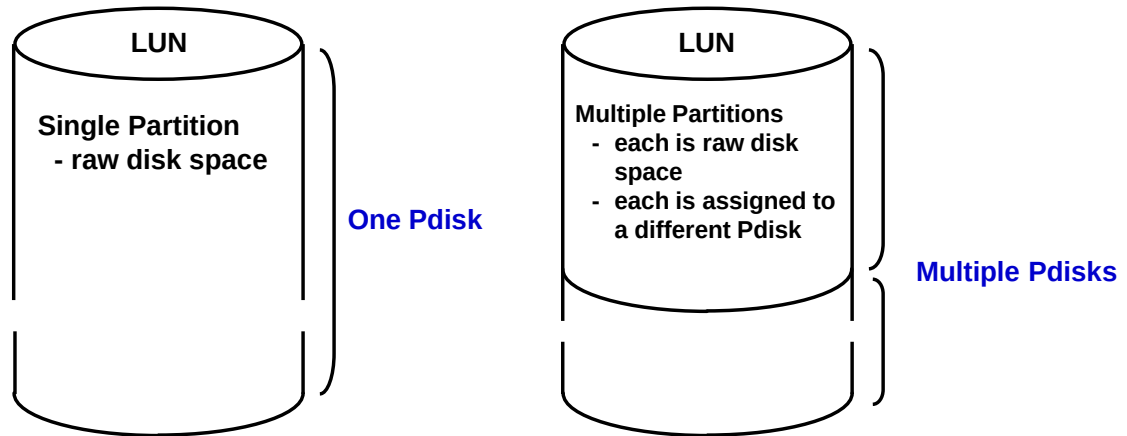
Logical Units and Partitions

With Linux, a logical unit (LUN) can be divided into one or more partitions.

- Provides raw data storage area (raw disk partition) for the Teradata.
- A Pdisk (Teradata) is a name that is assigned to a partition within a LUN.

When are multiple partitions in a LUN used by Teradata?

- With disk arrays in an Appliance (e.g., 2850)
- With Solid State Drives (SSD) on Enterprise Data Warehouse systems (e.g., IntelliFlex™)



Teradata and Disk Arrays

The Teradata Database has long been recognized as one of the most effective database platforms for the storage and management of very large relational databases.

The Teradata Database implementation executes as an application under the operating system. The two key pieces of software that make up the Teradata Database are the PE software and the AMP software.

Users access the Teradata Database by issuing SQL commands - usually from channel-attached hosts or LAN attached workstations. The user request is handled by *Channel Driver* or *Gateway* software and is passed to a Parsing Engine (PE) which processes the SQL request. PE software manages the user session, interprets (parses) the SQL request, creates an execution plan, and dispatches the steps of that plan to the AMP(s).

AMPs provide access to user data stored within tables that are physically stored on disk arrays.

Each AMP is associated with a Vdisk. Each AMP sees its Vdisk as a single disk. Teradata Database (AMP software) organizes its data on its disk space (Vdisk) using a Teradata Database “File System” structure. A “master index” is used to locate “cylinder indexes” which are used to locate data blocks that contain data rows.

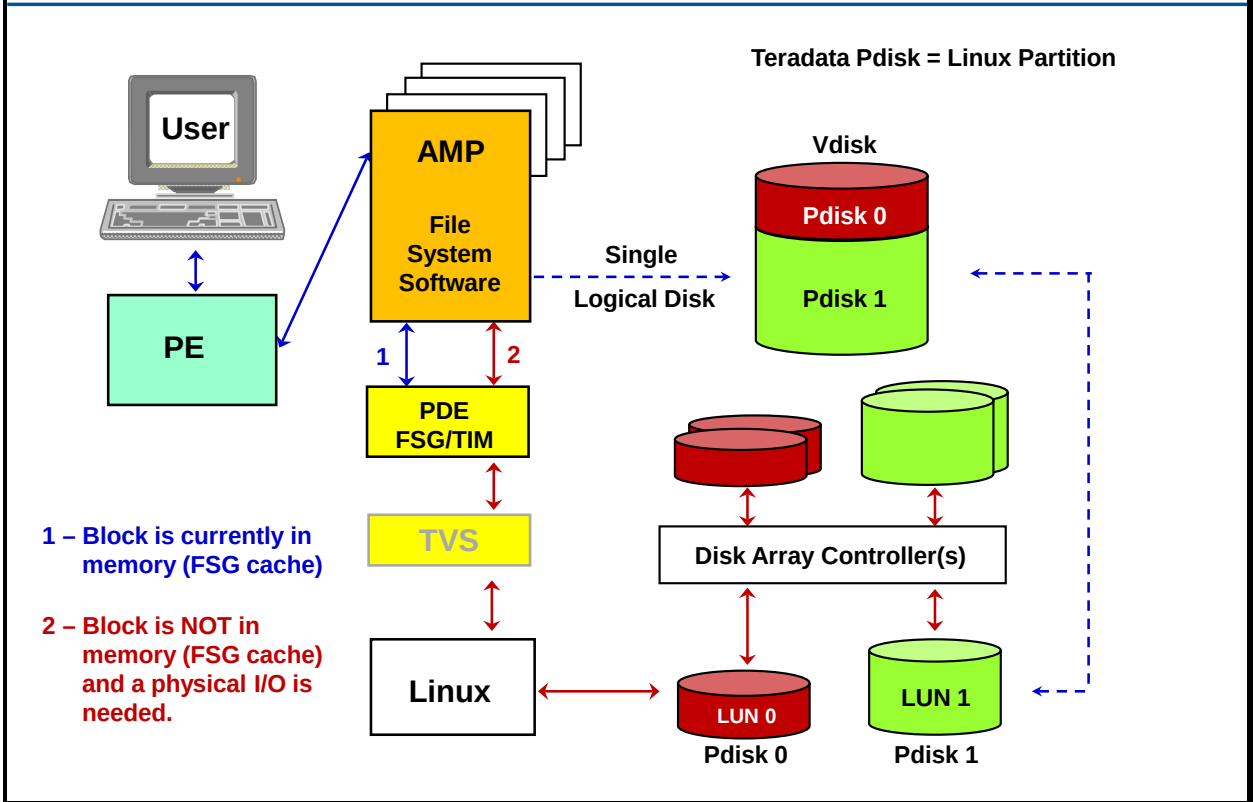
A Vdisk is actually composed of multiple slices (also called Pdisks - Physical disk) that are part of a LUN (Logical Unit) in a disk array. The operating system (e.g., Linux) and the array controllers work at the LUN level.

A logical unit (just like a physical disk) can be divided into multiple slices. A slice is a portion of a logical unit.

An AMP is assigned to a Vdisk. A Vdisk is composed of one or more Pdisks. In Linux, a Pdisk is assigned to a partition within a LUN.

The **PUT** utility is used to define a Teradata Database configuration.

Teradata and Disk Arrays



Teradata 2850 Logical View

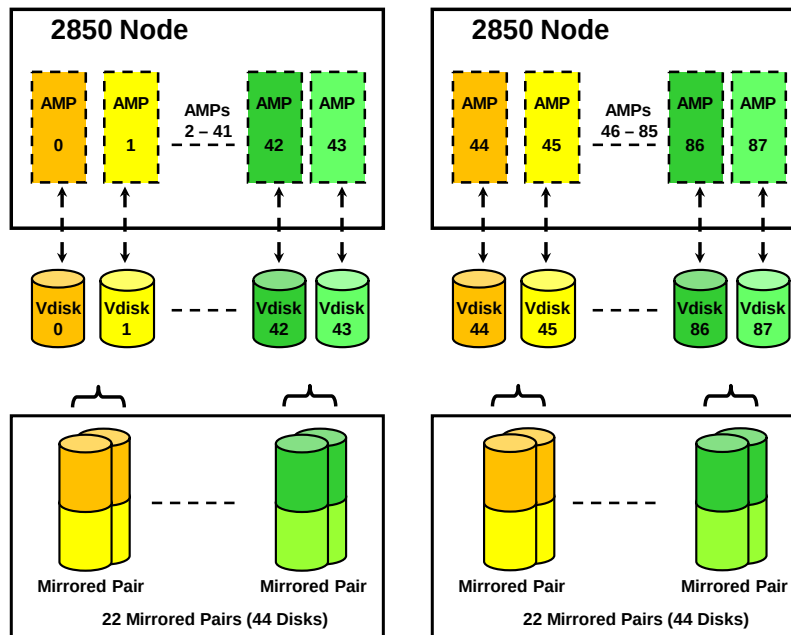
The facing page illustrates a logical view of the Teradata Database using a 2850 clique. A 2850 clique can be configured with 2 or 4 nodes. Two of the four nodes are shown on the facing page.

A 2850 node is configured as follows:

- 44 AMPs per Node
- Each node is connected to a disk array with 44 disk drives (or 22 mirrored pairs).
- Therefore, each AMP shares $\frac{1}{2}$ of a mirrored pair of disks. Each AMP is effectively assigned to a partition in a mirrored pair.

Note: The actual MaxPerm space that is available to an AMP is slightly less than the physical disk space because of file system overhead. Approximately 90 - 91% of the physical disk space is actually available as MaxPerm space.

Teradata 2850 Logical View



This example illustrates a 2850 with 2 AMPs sharing a mirrored pair of 1.2 TB disks.

The 2 additional nodes in a 2850 4-node clique would have AMPs assigned as follows:

88 to 131
132 to 175

Each node has 44 AMPs and a disk array of 44 HDD disks – 22 mirrored pairs.
Two AMPs share one mirrored pair of disks.
Each AMP is assigned to a partition in a LUN.

Teradata 6800 (3+1) Logical View

The facing page illustrates a logical view of the Teradata 6800 system with a 6800 3+1 clique.

This example illustrates a 6800H (Linux) 3+1 clique.

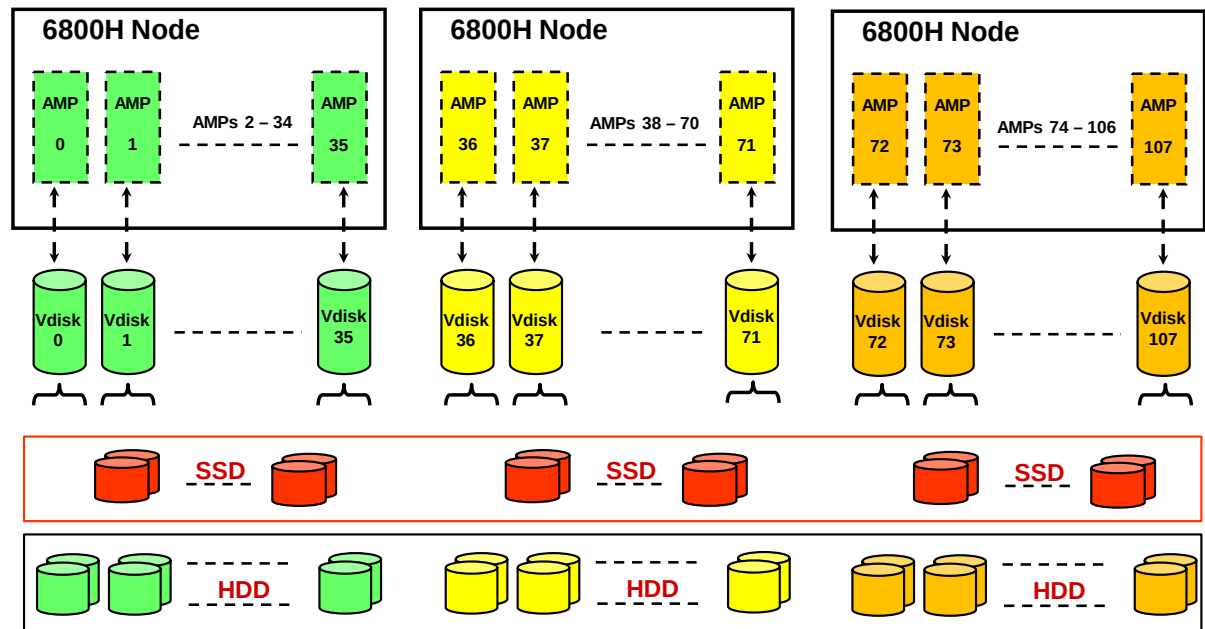
- In this example, each node is configured with 36 AMPs.
- Four AMPs share a pair of Solid State Drives (SSD). Each AMP has a partition on each pair of mirrored SSD.
- Each AMP is assigned to a pair of mirrored hard disk drives (HDD).

Supported configurations include 16, 24, 28, 36, or 42 AMPs per node for a 6800H 3+1 clique.

Each virtual AMP is assigned to a virtual disk (Vdisk). AMP 0 is assigned to Vdisk 0 which consists of 2 Pdisks as shown on the facing page.

Note: The actual MaxPerm space that is available to an AMP is slightly less than the physical disk space because of file system overhead. Approximately 90 - 91% of the physical disk space is actually available as MaxPerm space.

Teradata 6800H (3+1) Logical View



This example illustrates a 6800 with each AMP assigned to a portion of SSD space and a mirrored pair of HDD space.

Four Disk Arrays with 270 Disks – 54 SSD and 216 HDD with a SSD:HDD ratio of 20:80.

Example of 6800H Vdisk

A Vdisk effectively represents a set of disks in a disk array. In the example on the facing page, a Vdisk represents a rank of a partition on two SSD and 2 HDD in a disk array. All of the drives use RAID 1 technology in the disk arrays. The configuration of LUNs/partitions and the assignment of Pdisks/Vdisks to AMPs is done through the **PUT** utility.

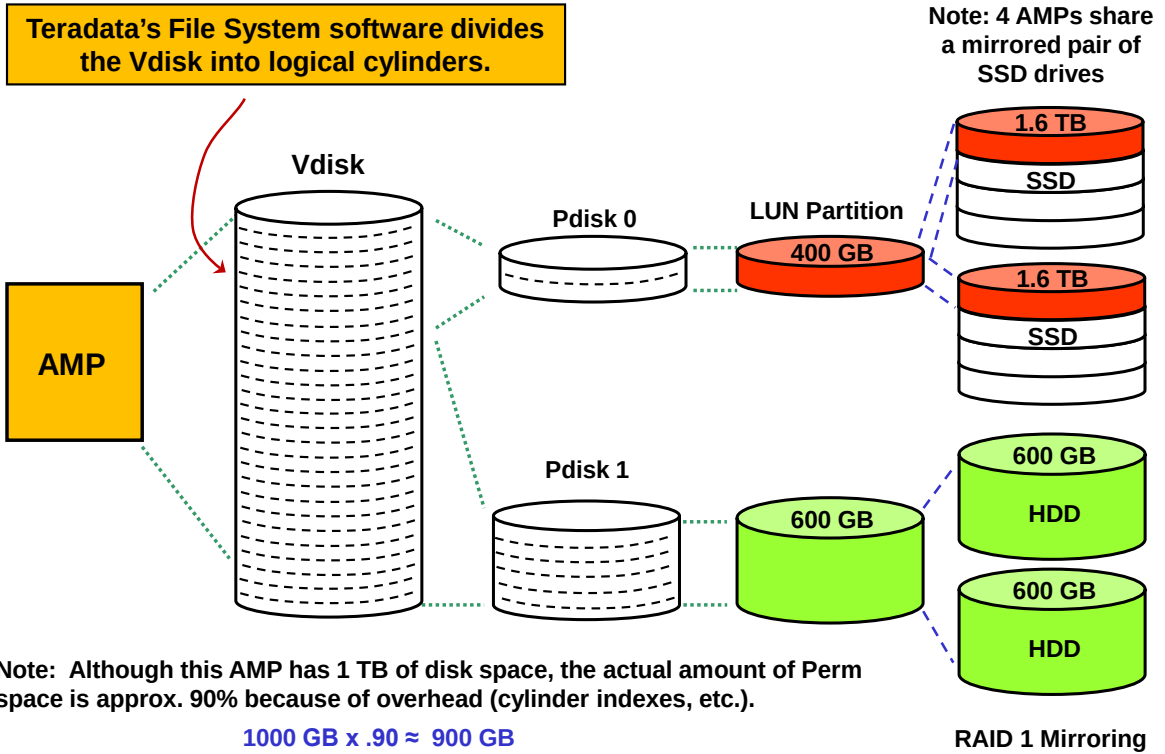
The example on the facing illustrates that each AMP is assigned to a mirrored partition of SSD (400 GB) and a mirrored pair of HDDs (600 GB). In this case, the AMP has 1.0 TB of available disk space in the disk array, but approximately 900 GB* of actual maximum permanent space.

If the disk array has 600 GB disks and RAID 1 protection is used and one rank of disks (4 disks) is assigned to the AMP, then the AMP has 1.2 TB of available disk space.

$$4 \text{ disks} \times 600 \text{ GB} \times .50 \text{ (parity is 50\%)} = 1.2 \text{ TB}^*$$

- * As mentioned previously, the actual space that is available to an AMP is slightly less than the numbers used above because of file system overhead. The actual MaxPerm space is approximately 90-91% of the physical disk space.
- * Cylinder 0 contains control information used by the AMP and does not contain user data.

Example of 6800H Vdisk



Cylinder – AMP Allocation Unit

Prior to Teradata 13.10, the maximum space available to an AMP is approximately 1.2 TB. Starting with Teradata 13.10, the large cylinder feature increases the maximum space available to an AMP to approximately 12 TB. Benefits of this feature are listed on the facing page.

Only systems that are newly initialized (sysinit) with Teradata 13.10 will have large cylinders enabled. Existing systems that are upgraded to 13.10 will have to be initialized (sysinit) in order to utilize large cylinders. Newer Teradata systems (e.g., 66xx and 6700) automatically have the large cylinder feature enabled.

A cylinder contains Perm, Spool, Temporary, Permanent Journal, WAL Log, or WAL Depot data, but NOT a combination. For an existing system, large cylinders results in fewer cylinders that are available for different types of data. Fewer cylinders can result in low cylinder conditions occurring more quickly and possibly more Mini-CylPacks.

If the larger cylinder size is used on an existing system where each AMP has much less space than 1.2 TB, then the number of available cylinders will be much less. For example:

- Assume a 6650 system with disk arrays populated with 600 GB drives. An AMP will typically have 4 drives assigned to it (2 sets of mirrored disks). Therefore, the AMP will have approximately 1200 GB of available space. This space is divided into approximately 592,000 cylinders. The actual MAXPERM space available to an AMP in this example is approximately 1080 GB (90% of 1200 GB).
- If this system is configured with large cylinders, then the system will only have approximately 99,000 cylinders. Large cylinders consume more physical space, resulting in fewer overall cylinders.

Each cylinder has 2 cylinder indexes associate with it. Small cylinder indexes are 12 KB and large cylinder indexes are 64 KB in size. Larger cylinder indexes are required because a large cylinder can hold more data blocks.

Cylinder – AMP Allocation Unit

- A Cylinder is the allocation unit of space within an AMP.
 - Small Cylinder = 3872 sectors (older systems)
 - Large Cylinder = 23,232 sectors (6 times as large) and are available starting with 13.10.
- An AMP may utilize different size cylinders for different purposes.
 - PERM cylinders are allocated as a large cylinder (if large cylinders have been enabled)
 - WAL LOG cylinders may be allocated as small or large cylinders.
- An AMP can address a maximum of 1 million (1,048,575) cylinders. Therefore, with 512 byte sectors and large cylinders, the maximum space an AMP can address is approximately 12 TB.
- Starting with Teradata 13.10, a system can be configured to use large cylinders. **New Teradata systems (e.g., IntelliFlex™) default to use large cylinders.**

Benefits of this feature are:

- Allows systems to utilize larger disk drives and supports 4K (and larger) aligned sector drives.
- Supports large data blocks and large data rows.

Teradata File System Concepts

Each AMP has its own disk space managed by the Teradata Database file system software. The file system software groups physical blocks into logical cylinders.

One AMP can address/manage up to 1,048,575 cylinders. Each cylinder has a cylinder index space (CI). Although an AMP can address this large number of cylinders, typically an AMP will only be responsible for a much smaller number of cylinders.

When an AMP is initialized (booted), it reads the Cylinder Indexes and creates an in-memory Master Index to the Cylinder Indexes.

Notes:

- Small Cylinders (3872 sectors) – up to Teradata Database 13.0. Each Cylinder Index is 12 KB in size. The cylinder size is still 3872 sectors. Therefore, the Cylinder Index space is actually 24 KB at the beginning of the cylinder to store two cylinder indexes. The Primary and Alternate Cylinder Indexes are used to provide integrity during file system write operations.
- Large Cylinders (23,232 sectors) – option starting with Teradata Database 13.10. Each Cylinder Index is 64 KB in size. For Teradata releases 13.10 through 16.0, the Cylinder Index space is actually 128 KB at the beginning of the cylinder to store two cylinder indexes. The Primary and Alternate Cylinder Indexes are used to provide integrity during file system write operations.
- Starting in Teradata 15.10, the cylinder indexes are no longer stored within the cylinder, but all the cylinder indexes are stored in a separate area on the AMP's virtual disk.
- Teradata uses the cylinder indexes as alternating cylinder indexes for write (INSERT, UPDATE, and DELETE) operations.

Teradata Vdisk Size Limits (older)

For Teradata releases up to Teradata 13.0, the maximum amount of space that one AMP can access is based on the following calculation:

700,000 logical cylinders x 3872 sectors/cylinder x 512 bytes/sector

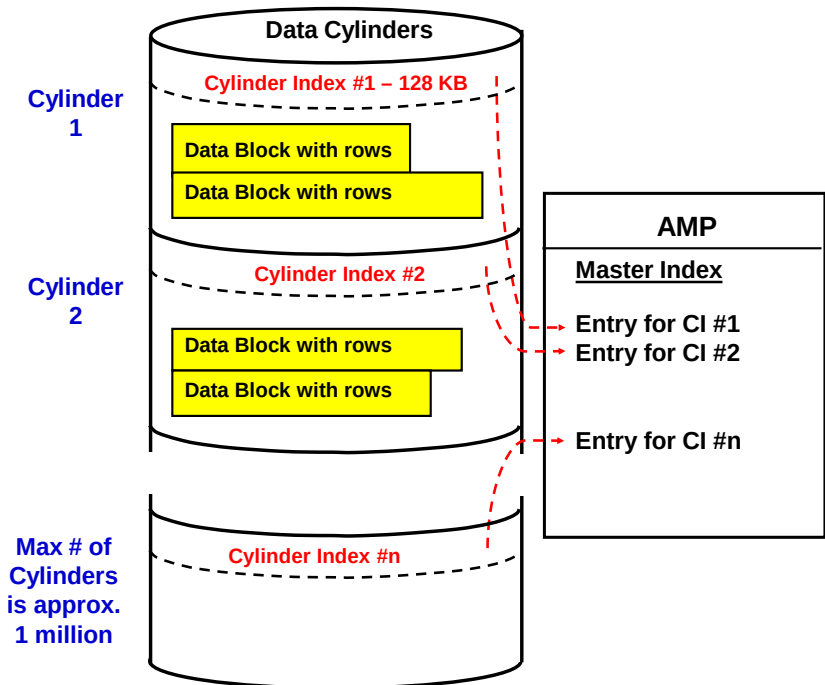
This equals 1,387,724,800,000 bytes or approximately 1.26 TB where a TB is 1024^4 .

Teradata File System Concepts

Typically, systems utilize large cylinders and the size is 11.6 MB (assuming 512 byte sectors).

At the beginning of each cylinder, cylinder index space (128 KB) is allocated to hold 2 cylinder indexes.

Cylinders contain one or more variable length data blocks.



Cylinder Index Segregation (15.10)

This Teradata 15.10 feature improves Teradata Database File System Master Index (MI) rebuild performance by segregating Cylinder Indexes (CIs) into CI zones and storing the CIs adjacent to each other on the disk, allowing sequential reads of CIs during startup.

CIs are grouped within the device on which they reside. By grouping the CIs together, a single disk access can be used to read multiple CIs.

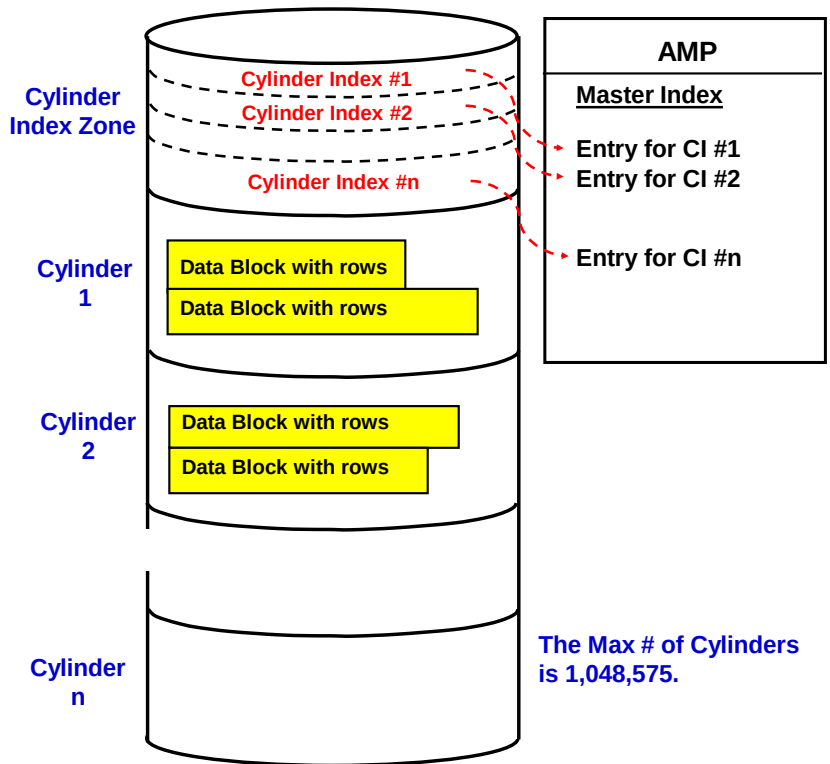
The key benefit of this feature is that a Teradata Database startup (or restart) takes less time.

Cylinder Index Segregation (15.10)

Cylinder Index Segregation

Starting in **Teradata 15.10**, each Cylinder Index is no longer stored at the beginning of each cylinder.

The Cylinder Indexes (CIs) are segregated into CI zones and stored adjacent to each other on the disk. This allows sequential reads of the CIs during startup and improves MI rebuild performance.



Full Cylinder Read

Full Cylinder Read allows retrieval operations to run more efficiently by reading a list of cylinder-resident data blocks with a single I/O operation. This reduces I/O overhead from once per data block to once per cylinder.

A data block is a disk-resident structure that contains one or more rows from the same table and is the smallest I/O unit for the Teradata Database file system. Data blocks are stored in physical disk sectors or segments, which are grouped in cylinders.

Full Cylinder Read improves the performance of systems with both fine-grained operations and decision support workload. It eliminates the tradeoffs for short queries and concurrent updates versus strategic queries.

Performance may benefit from Full Cylinder Read during operations such as:

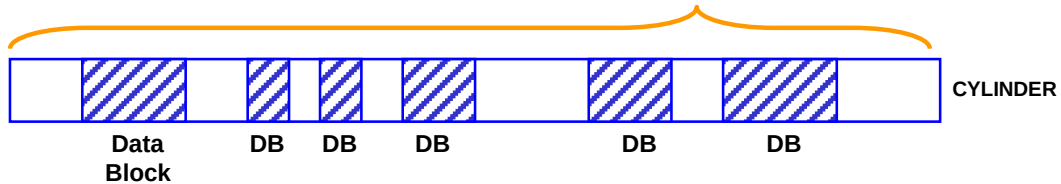
- Full-table scan operations under conditions such as:
- Large select
- Merge insert/select and Merge delete
- Aggregation: Sum, Average, Minimum/Maximum, Count
- Join operations that involve many data blocks, such as merge joins, product joins, and inner/outer joins

Starting with Teradata 13.10, this feature no longer needs to be tuned using a Cylinder Slots/AMP setting. This allows for more extensive use of cylinder read without the need to reserve portions of the FSG cache for Cylinder Read operations.

Prior to Teradata 13.10, it was necessary to specify the number of cylinder slots per AMP that would be available.

Full Cylinder Read

The Full Cylinder Read feature allows data to be retrieved with a **single cylinder (large) read**, rather than individual reads of blocks.



Enables efficient use of disk & CPU performance resources for the following table scan operations under specific conditions. Examples include:

- large selects and aggregates: sum, avg, min, max, count
- joins: merge joins, product joins, inner/outer joins
- merge delete merge insert/select into empty or populated tables
- full table update/deletes

Summary

The facing page summarizes the key points and concepts discussed in this module.

Summary

- Memory managed and used by the operating system and the vprocs is referred to “free memory”.
- PDE software manages FSG Cache.
 - FSG Cache is primarily used by the AMPs to access memory resident database segments.
- The operating system and Teradata does not know or care about the RAID technology being used.
- A LUN (or Volume) looks like a single disk to the operating system.
 - With Linux, a LUN may contain one or more partitions and a raw partition is assigned to a Teradata Pdisk.

Module 10: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 10: Review Questions

1. Which two are accessed in FSG cache memory? (choose 2)
 - a. Hash maps
 - b. Master Indexes
 - c. Cylinder Indexes
 - d. Data Dictionary Cache
2. What are two benefits of Cylinder Index Segregation in Teradata 15.10? (choose 2)
 - a. Allows for larger cylinders
 - b. Improves Teradata reboot performance
 - c. Allows more user data to be stored in a cylinder
 - d. Reduces the number of required cylinder indexes
3. When are multiple partitions in a LUN used in a Teradata configuration? (choose 2)
 - a. SSD drives with a 2850
 - b. HDD drives with a 2850
 - c. SSD drives with a 6800
 - d. HDD drives with a 6800
4. Describe the fundamental relationship between Linux, logical units, and disk array controllers.

5. Describe the fundamental relationship between AMPs, Vdisks, Pdisks, Partitions, and LUNs.

Notes



Analyze Primary Index Criteria

After completing this module, you will be able to:

- Identify Primary Index choice criteria.
- Describe uniqueness and how it affects space utilization.
- Explain row access, selection, and selectivity.
- Choose between single and multiple-column Primary Indexes.
- Describe why a table might be created without a primary index.
- Specify the syntax to create a table without a primary index.

Teradata Proprietary and Confidential

Notes

Table of Contents

Primary Index Choice Criteria	11-4
Primary Index Defaults	11-6
CREATE TABLE – Indexing Rules	11-8
Order of Preference Exercise	11-10
Primary Index Characteristics	11-12
Multi-Column Primary Indexes	11-14
Primary Index Considerations.....	11-16
PKs and Duplicate Rows.....	11-18
NUPI Duplicate Row Check	11-20
Primary Index Demographics	11-22
Column Distribution Demographics for a PI Candidate	11-24
SQL to View Data Demographics	11-26
Example of Using Data Demographic SQL.....	11-28
TableSize View	11-30
Using SQL to Predict Row Distribution	11-32
E-R Diagram for Exercises	11-34
Exercise 2 – Example.....	11-36
Exercise 2 – Choosing PI Candidates	11-38
What is a NoPI Table?	11-50
Reasons to Consider Using NoPI Tables	11-52
Creating a Table without a PI.....	11-54
Distribution of Rows (or Blocks) for a NoPI Table.....	11-56
The Row ID for a NoPI Table.....	11-58
Multiple NoPI Tables at the AMP Level	11-62
Loading Data into a NoPI Table (SQL)	11-64
Loading Data into a NoPI Table (Utility)	11-66
NoPI Options.....	11-68
Summary	11-70
Module 11: Review Questions	11-72
Module 11: Review Questions (cont.)	11-74
Lab Exercise 11-1	11-76
Lab Exercise 11-2	11-80
Lab Exercise 11-3 (optional).....	11-82

Primary Index Choice Criteria

There are three Primary Index Choice Criteria: **Access Demographics**, **Distribution Demographics**, and **Volatility**.

- Access demographics are the first of three Primary Index Choice Criteria. Access columns are those that would appear (with a value) in a WHERE clause in an SQL statement. Choose the column most frequently used for access to maximize the number of one-AMP operations.
- Distribution demographics are the second of the Primary Index Choice Criteria. The more unique the index, the better the distribution. Optimizing distribution optimizes parallel processing.
- In choosing a Primary Index, there is a trade-off between the issues of access and distribution. The most desirable situation is to find a PI candidate that has good access and good distribution. Many times, however, index candidates offer great access and poor distribution or vice versa. When this occurs, the physical designer must balance these two qualities to make the best choice for the index.
- The third of the Primary Index Choice Criteria is volatility, or how often the data values will change. The Primary Index should not be very volatile. Any changes to Primary Index values may result in heavy I/O overhead, as the rows themselves may have to be moved from one AMP to another. Choose a column with stable data values.

Degree of Uniqueness and Space Utilization

The degree of uniqueness of a Primary Index has a direct influence on the space utilization. The more unique the index, the better the space is used.

Fewer Distinct PI Values than Amps

For larger tables, it is not a good idea to choose a Primary Index with fewer distinct values than the number of AMPs in the system when other columns are available. At best, one index value would be hashed to each AMP and the remaining AMPs would carry no data.

Non-Unique PIs

Choosing a Non-Unique PI (NUPI) with some very non-unique values can cause “spikes” in the distribution.

Unique (or Nearly-Unique) PIs

The designer should choose an index which is unique or nearly unique to optimize the use of disk space. Remember that the PERM limit of a database (or user) is divided by the number of AMPs in the system to yield a threshold that cannot be exceeded on any AMP.

Primary Index Choice Criteria

ACCESS	Maximize one-AMP operations: Choose the column(s) most frequently used for access. Consider both join and value access.
DISTRIBUTION	Optimize parallel processing: Choose the column(s) that provides good distribution.
VOLATILITY	Reduce maintenance resource overhead (I/O): Choose the column(s) with stable data values.

Note: Data distribution has to be balanced with Access usage in choosing a PI.

General Notes:

- A good logical model identifies the Primary Key for each table or entity.
 - Do not assume that the Primary Key will become the Primary Index.
 - It is common for many tables in a database to have a Primary Index that is different than the Primary Key.
 - This module will first cover PI tables then cover details of the NO PRIMARY INDEX option. The general assumption in this course is that tables will have a PI.

Primary Index Defaults

1. If the NO PRIMARY INDEX clause is specified, then the table is created without a primary index. If this clause is used, you cannot specify a primary index for the table.

There are a number of limitations associated with a NoPI table that will be listed later.

2. If the PRIMARY INDEX, NO PRIMARY INDEX, PRIMARY KEY, or UNIQUE options are NOT specified in the CREATE TABLE DDL, then:

Table to be created with or without a primary index will be based on a DBSControl General flag **Primary Index Default**. The default setting is "D" which effectively means the default is to create a table with the first column as a NUPI.

D – This is the default setting. This setting works the same as the P setting.

P – The first column in the table will be selected as the non-unique primary index.

This setting works the same as that in the past when PRIMARY INDEX was not specified.

N – The table will be created without a primary index (NoPI table).

3. With the NoPI Table feature, the system default setting essentially remains the same as that in previous Teradata releases where the first column was selected as the non-unique primary index when the user did not specify a PRIMARY INDEX or a PRIMARY KEY or a UNIQUE Constraint.

Users can change the default setting for PrimaryIndexDefault to P or N and not rely on the system default setting which might be changed in a future release.

Note: DBSControl flag #53 was added in Teradata release 13.0.

Primary Index Defaults

DBS Control flag #53 determines if a table is created as a PI or NoPI table when the CREATE TABLE DDL does **NOT** have any of the following explicitly specified:

- PRIMARY INDEX clause
- NO PRIMARY INDEX clause
- PRIMARY KEY or UNIQUE constraints

Values for DBS Control General field #53 "Primary Index Default":

D – "Teradata Default" (effectively same as option P)

P – "First Column is NUPI" – create tables with first column as a NUPI

N – "No Primary Index" – create tables without a primary index (NoPI)

The PRIMARY INDEX and NO PRIMARY INDEX clauses always have precedence over PRIMARY KEY and UNIQUE constraints.

If the NO PRIMARY INDEX clause is specified and PRIMARY KEY or UNIQUE constraints are also defined, then these constraints will be implemented as Unique Secondary Indexes.

- It may be unusual to create a NoPI table with these additional indexes.

CREATE TABLE – Indexing Rules

The **primary index** may be explicitly specified at table create time. If not, a primary index choice will be made based on other choices made. PRIMARY and UNIQUE constraints are always implemented by Teradata as unique indexes, either primary or secondary.

This chart assumes the system default is to create tables with a Primary Index. The index implementation schedule is as follows:

Is a PI specified?

No	PK specified?	PK = UPI
	PK specified and UNIQUE constraints specified?	PK = UPI UNIQUE constraints = USI(s)
	UNIQUE column level constraints only specified?	1 st UNIQUE column level constraint = UPI Other UNIQUE constraints = USI(s)
	UNIQUE column level constraints and table level UNIQUE constraints specified?	1 st UNIQUE column level constraint = UPI Other UNIQUE constraints = USI(s)
	UNIQUE table level constraints only specified?	1 st UNIQUE table level constraint = UPI Other table level UNIQUE constraints = USI(s)
	Neither specified?	1st non-LOB column = NUPI
Yes	PK specified?	PK = USI
	PK specified and UNIQUE constraints specified?	PK = USI UNIQUE constraints = USI(s)
	UNIQUE constraints only specified?	UNIQUE constraints = USI(s)

CREATE TABLE – Indexing Rules

1. PRIMARY INDEX or NO PRIMARY INDEX is specified in the DDL.

If PRIMARY KEY or UNIQUE constraints are also specified in the DDL, then each is implemented as a USI.

2. PRIMARY KEY is specified in the DDL.

The PRIMARY KEY becomes a UPI.
Each additional UNIQUE constraint is implemented as a USI.

3. UNIQUE constraints are specified in the DDL – column level UNIQUE constraints take precedence over table level UNIQUE constraints.

1st UNIQUE constraint becomes a UPI.
Each additional UNIQUE constraint is implemented as a USI.

4. DBSControl Flag #53 is only used to determine table default if PRIMARY INDEX, NO PRIMARY INDEX, PRIMARY KEY, or UNIQUE are NOT specified in the DDL.

If flag = "P", then 1st non-LOB column is implemented as a NUPI.
If flag = "N", then table is implemented as a NoPI table.

Order of Preference Exercise

Complete the exercise on the facing page. Answers will be provided by your instructor.

Some additional examples include:

If table_5 was created as follows:

```
CREATE TABLE table_5  
(col1 INTEGER NOT NULL  
,col2 INTEGER NOT NULL  
,col3 INTEGER NOT NULL  
,CONSTRAINT uniq1 UNIQUE (col1,col2)  
,CONSTRAINT uniq2 UNIQUE (col3));
```

Then, the indexes are a UPI on (col1,col2) and a USI on (col3).

If table_5 was created as follows:

```
CREATE TABLE table_5  
(col1 INTEGER NOT NULL  
,col2 INTEGER NOT NULL  
,col3 INTEGER NOT NULL  
,CONSTRAINT uniq1 UNIQUE (col3)  
,CONSTRAINT uniq2 UNIQUE (col1,col2));
```

Then, the indexes are a UPI on (col3) and a USI on (col1,col2).

Notes:

- Recommendation: Specify Primary Index when creating a table.
- Table level constraints are typically used to specify a PK or UNIQUE constraint for multiple columns.

Order of Preference Exercise

Assuming the system default is "Primary Index", show the indexes that are created as a result of the DDL.

```
CREATE TABLE table_1
(col1 INTEGER NOT NULL UNIQUE          col1 =
,col2 INTEGER NOT NULL PRIMARY KEY);  col2 =
```

UPI =	Unique Primary Index
NUPI =	Non Unique Primary Index
USI =	Unique Secondary Index

```
CREATE TABLE table_2
(col1 INTEGER NOT NULL PRIMARY KEY      col1 =
,col2 INTEGER)                          col2 =
PRIMARY INDEX (col2);
```

```
CREATE TABLE table_3
(col1 INTEGER                          col1 =
,col2 INTEGER NOT NULL);               col2 =
```

```
CREATE TABLE table_4
(col1 INTEGER NOT NULL                col1 =
,col2 INTEGER NOT NULL                col2 =
,col3 INTEGER NOT NULL UNIQUE         col3 =
,CONSTRAINT pk1 PRIMARY KEY (col1,col2)); (col1,col2) =
```

```
CREATE TABLE table_5
(col1 INTEGER NOT NULL                col1 =
,col2 INTEGER NOT NULL                col2 =
,col3 INTEGER NOT NULL UNIQUE         col3 =
,CONSTRAINT uniq1 UNIQUE (col1,col2)); (col1,col2) =
```

Primary Index Characteristics

Each table has one and only one Primary Index. A Primary Index may be different than a Primary Key.

UPI = Best Performance, Best Distribution

- UPIs offer the best performance possible for several reasons. They are:
- A Unique Primary Index involves a single base table row at most
- No Spool file is ever required
- Single value access via the Primary Index is a one-AMP operation and uses only one I/O

NUPI = Good Performance, Good Distribution

- NUPI performance differs from UPI performance because:
- Non-Unique Primary Indexes may involve multiple table rows.
- Duplicate values go to the same AMP and the same data block, if possible.
- Multiple I/Os are required if the rows do not fit in a single data block.
- Spool files are used when necessary.
- A duplicate row check is required on INSERT and UPDATE for a SET table.

Primary Index Characteristics

Primary Indexes (UPI and NUPI)*

- A Primary Index may be different than a Primary Key.
- A table can have only one Primary Index.
- A Primary Index may contain null(s).
- **Single-value access uses ONE AMP and, typically, one I/O.**

Unique Primary Index (UPI)

- Involves a single base table row at most.
- No spool file is ever required.
- The system automatically enforces uniqueness on the index value.

Non-Unique Primary Index (NUPI)

- May involve multiple base table rows.
- A spool file is created when needed.
- **Duplicate values go to the same AMP and usually are in the same data block.**
 - Only one I/O is needed if all the rows with the same NUPI value fit in a single data block.
- When inserting or updating data, a duplicate row check is required for a SET table.

* This module assumes that the tables are not row partitioned.

Multi-Column Primary Indexes

In practice, Primary Indexes are sometimes composed of several columns. Such composite indexes are known as multi-column Primary Indexes. They are used quite commonly and you can probably think of several existing applications that utilize them.

Increased Uniqueness

There are both advantages and disadvantages to using multi-column PIs. Perhaps the most important **advantage** is that by combining several columns, you can produce an index that is much more unique than any of the component columns. This **increased uniqueness** will result in better data distribution, among other benefits.

For example:

- PI = Lastname
- PI = Lastname + Firstname
- PI = Lastname + Firstname + MI

The above example points out how better data distribution occurs. Notice that each succeeding Primary Index is more unique than the one preceding it. That is, there are far less individuals with identical last and first names than there are with the same last name, and so on.

Increasing uniqueness means that as the number of columns increases:

- The number of distinct values increases.
- The number of rows per value decreases.
- The selectivity increases.

Trade-off

The **disadvantage** involved with multi-column indexes is that as the number of columns increases, the index becomes less usable.

A multi-column index can only be accessed when values for all columns are specified in the SQL statement. If a single value is omitted, the Primary Index cannot be used.

It is important for the physical designer to balance these factors and use multi-column indexes that have just enough columns. This will result in optimum uniqueness while reducing unnecessary full table scans.

Multi-Column Primary Indexes

Advantage

More columns = more uniqueness

- Number of distinct values increase.
- Rows/value decreases.
- Selectivity increases.

Disadvantage

More columns = less usability

- PI can only be used when values for all PI columns are provided in SQL statement.
- Partial values cannot be hashed.

Primary Index Considerations

The facing page summarizes the concepts you have seen throughout this module and provides a list of the most important considerations when choosing a Primary Index.

The first three considerations summarize the three types of demographics: **Access**, **Distribution**, and **Volatility**.

You should choose a column with good distribution to maximize parallel processing. A good rule-of-thumb is to base your Primary Index on the column(s) most often used for access (if you don't have too many rows per value) to maximize one-AMP operations. Finally, Primary Index values should be stable to reduce maintenance resource overhead.

Make sure that the number of distinct values for a PI is greater than the number of AMPs in the system, whenever possible, or some AMPs will have no rows.

Duplicate values hash to the same AMP and are stored in the same data block. If the index is very non-unique, multiple data blocks are used and incur multiple I/Os.

Very non-unique PIs may skew space usage among AMPs and cause Database Full conditions on AMPs where excessive numbers of rows are stored.

Primary Index Considerations

- Base PI on the column(s) most often used for access, provided that the values are unique or nearly unique.
- Choose a column (or columns) with good distribution and no spikes.
 - NULLs and zero (for numeric data types) hash to binary zeroes and to the same AMP.
- Distinct values distribute evenly across all AMPs.
 - For large tables, the number of Distinct Primary Index values should be much greater (at least 10X; 50X may be better guideline) than the number of AMPs.
- Duplicate values hash to the same AMP and are stored in the same data block when possible.
 - Very non-unique values use multiple data blocks and incur multiple I/Os.
 - Very non-unique values may skew space usage among AMPs and cause premature Database Full conditions.
 - A large number of NUPI duplicate values on a SET table can cause expensive duplicate row checks.
- Primary Index values should not be highly volatile.

PKs and Duplicate Rows

Each row in table or entity in a good logical model will be uniquely identified by the table's primary key.

- Every table must have a Primary Key.
- Primary Keys (PKs) must be unique.
- Primary Keys cannot be changed.

In Set tables, the Teradata NewSQL Engine does not allow duplicate rows. When a table has a Unique Primary Index (UPI), the UPI enforces uniqueness. When a table has a Non-Unique Primary Index (NUPI), the matter can become more complicated.

In the case of a NUPI (without a USI defined), the file system must compare data values byte-by-byte within a Row Hash in order to ensure uniqueness. Many NUPI duplicates result in lots of duplicate row checks, which can be quite expensive in terms of system resources.

The way to avoid such a situation is to define a USI on the table whenever you have a NUPI. The USI does the job of enforcing uniqueness and thus save you the cost of doing duplicate row checks. Often, the best column(s) to use when defining such a USI is the PK.

Specifying a UNIQUE constraint on a column(s) other than the Primary Index also causes the creation of a Unique Secondary Index.

An exception to the above is found when using the load utilities, such as FastLoad and MultiLoad. These utilities do not allow the use of Secondary Indexes to enforce uniqueness. Therefore, a full row comparison is still necessary.

INSERT/SELECT of Duplicate Rows into a Table

If you INSERT/SELECT from a table with duplicate rows into a SET or MULTISSET target table with a UPI, the INSERT/SELECT will fail because of duplicate UPI.

If you INSERT/SELECT from a table with duplicate rows into a SET target table with a NUPI, the INSERT/SELECT request will complete and the duplicate rows are discarded. However, if your target table is MULTISSET and NUPI, then the duplicate rows are also copied into the target table.

PKs and Duplicate Rows

Rule: Primary Keys Must be UNIQUE and NOT NULL.

- This rule of Relational Theory eliminates duplicate rows, which have plagued the industry for decades.
- With Set tables (the default in Teradata transaction mode), duplicate rows *are not allowed*.
- With Multiset tables, the duplicate rows *are allowed*.
 - All indexes must be non-unique indexes (NUPI and NUSI) in order to allow duplicate values.
 - A unique index (UPI or USI) will prevent duplicate index values, and therefore, duplicate rows (even if the table is created as Multiset).
- If no unique index exists for a SET table, the file system compares data values byte by byte within a Row Hash to ensure row uniqueness in a table.
 - Many NUPI duplicates result in expensive duplicate row checks.
 - To avoid these duplicate row checks, use a Multiset table.

NUPI Duplicate Row Check

Set tables (the default) do not allow duplicate rows. When a new row is inserted into a Set table with a Non-Unique Primary Index, the system must perform a **NUPI Duplicate Row Check**.

The facing page illustrates the number of logical reads that must occur when this happens. The middle column is the number of logical reads required before that one row can be inserted. The right hand column shows how many cumulative logical reads would be required to insert all the rows up to and including that one.

As you can see, when you have a NUPI with excessive rows per value, the number of logical reads becomes prohibitively high. It is very important to limit the NUPI rows per value whenever possible.

The best way to avoid NUPI Duplicate row checks is to create the table as a MULTISSET table.

Note: USIs should be used for access or uniqueness enforcement. They should not be used just to avoid duplicate row checking, since sometimes they may be used and at other times they will not be used. The overhead of a USI does not justify the cost of trying to avoid the duplicate row check and they don't avoid the cost in most cases.

As a suggestion, keep the number of NUPI rows per value within the number of rows which will fit into your largest block. This will allow the system to satisfy a single-value NUPI access with one or two data block I/Os.

NUPI Duplicate Row Check

Limit NUPI rows per value to rows per block whenever possible.

To avoid NUPI duplicate row checks, create the table as a **MULTISET** table.

This chart illustrates the additional I/O overhead.

Row Number to be inserted	Number of Rows that must be logically read first	Cumulative Number of logical row reads
1	0	0
2	1	1
3	2	3
4	3	6
5	4	10
6	5	15
7	6	21
8	7	28
9	8	36
10	9	45
20	19	190
50	49	1225
100	99	4950
200	199	19900
500	499	124750
1000	999	499500

Primary Index Demographics

As you have seen, the three types of demographics important to choosing a Primary Index are: **Access** demographics, **Distribution** demographics and **Volatility** demographics. To make proper PI selections, you must have accurate demographics. Accurate demographics serve to quantify all three-index selection determinants.

Access Demographics

Access demographics identify index candidates that maximize one-AMP operations. Both Value Access and Join Access are important to PI selection. The higher the value, the more often the column is used for access.

Distribution Demographics

Distribution demographics identify index candidates that optimize parallel processing. Choose the column(s) that provides the best distribution.

Volatility

Volatility demographics identify table columns that are UPDATED. This item does not refer to INSERT or DELETE operations.

Volatility demographics identify index candidates that reduce maintenance I/O. You want to have columns with stable data values as your PI candidates.

In this module, you will see how to use Distribution demographics to select PI candidates. Access and Volatility demographics will be presented in a later module.

Primary Index Demographics

Access Demographics

- Identify index candidates that maximize one-AMP operations.
- Columns most frequently used for access (Value and Join).

Distribution Demographics

- Identify index candidates that optimize parallel processing.
- Columns that provide good distribution.

Volatility Demographics

- Identify index candidates with low maintenance I/O.

**Without accurate demographics, index choices are unsubstantiated.
Demographics quantify all 3 index selection determinants.**

Column Distribution Demographics for a PI Candidate

Column Distribution demographics are expressed in four ways: **Distinct Values**, **Maximum Rows per Value**, **Maximum Rows NULL** and **Typical Rows per Value**. These items are defined below:

- **Distinct Values** is the total number of different values a column contains. For PI selection, the higher the Distinct Values (in comparison with the table row count), the better. Distinct Values should be greater than the number of AMPs in the system, whenever possible. We would prefer that all AMPs have rows from each TABLE.
- **Maximum Rows per Value** is the number of rows in the most common value for the column or columns. When selecting a PI, the lower this number is, the better the candidate. For a column or columns to qualify as a UPI, Maximum Rows per Value must be 1.
- **Maximum Rows NULL** should be treated the same as Maximum Rows Per Value when being considered as a PI candidate.
- **Typical Rows per Value** gives you an idea of the overall distribution which the column or columns would give you. The lower this number is, the better the candidate. Like Maximum Rows per Value, Typical Rows per Value should be small enough to fit on one data block.

The illustration at the bottom of the facing page shows a distribution graph for a column whose values are states. Note in the graph that 30K = Maximum Rows NULL, and 15K = Maximum Rows per Value (CA). Typical Rows per Value is approximately 30.

You should monitor all demographics periodically as they change over time.

Distinct Values

- The more the better (compared to table row count).
- Should have enough values to allow for distribution to all AMPs.

Maximum Row Per Value

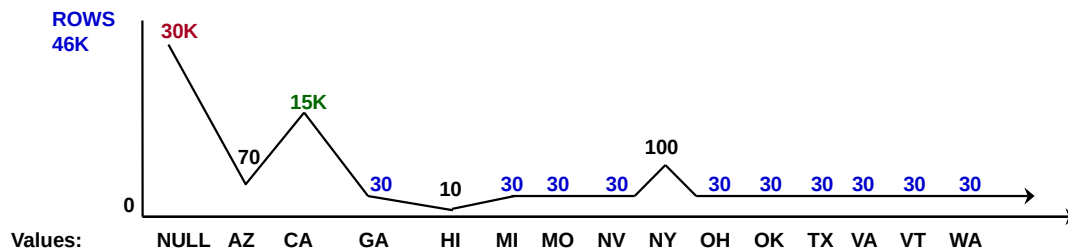
- The fewer the better.

Maximum Rows Null

- The fewer the better.
- A very large number indicates a very large distribution spike.
- Large spikes can cause serious space consumption problems.

Typical Rows Per Value

- The fewer the better.
- Monitor periodically as it changes over time.



SQL to View Data Demographics

The facing page contains simple examples of SQL that can be used to determine data demographics for a column.

The Average Rows per value can be thought of as the Mean of a data set.

To determine the typical rows per value, one estimate is to take the median of the result set that has number of rows per value. The following queries are options to use:

```
SELECT MEDIAN(cts) AS mediancts
FROM
(SELECT  column_name,
        COUNT(*) AS cts,
        COUNT(cts) OVER (PARTITION BY cts) AS ctscts
FROM  table_name GROUP BY 1) t1;
```

Or

```
SELECT AVG(cts) AS averagects
FROM
(SELECT  column_name,
        COUNT(*) AS cts,
        COUNT(cts) OVER (PARTITION BY cts) AS ctscts
FROM  table_name
GROUP BY 1) t1;
```

Or

```
SELECT
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY cts (DEC(8,2)))
FROM
(SELECT  column_name,
        COUNT(*) AS cts
FROM  table_name
GROUP BY 1) t1;
```

Note that MEDIAN and PERCENTILE_CONT functions are not available in Teradata 14.0 and prior releases.

of Distinct Values for a column:

```
SELECT      COUNT(DISTINCT(column_name))    FROM  table_name;
```

Max Rows per Value for all values in a column:

```
SELECT      column_name, COUNT(*)           FROM  table_name  
GROUP BY 1  ORDER BY 2 DESC;
```

Max Rows per Value for 5 most frequent values:

```
SELECT      TOP 5  tvalue, tcount  
FROM        (SELECT      column_name, COUNT(*)  
                  FROM    table_name      GROUP BY 1)  
            t1 (tvalue, tcount)  
            ORDER BY    tcount DESC;
```

Average Rows per Value for a column:

```
SELECT COUNT(*) / COUNT(DISTINCT(column_name)) AS averagects FROM  table_name;
```

Typical (or Median) Rows per Value for a column:

```
SELECT MEDIAN(cts) AS mediancts  
FROM  (SELECT column_name,  
            COUNT(*) AS cts,  
            COUNT(cts) OVER (PARTITION BY cts) AS ctscts  
      FROM  table_name  
      GROUP BY 1) t1;
```

Example of Using Data Demographic SQL

The facing page contains simple examples of SQL that can be used to determine data demographics for a column.

Example of Using Data Demographic SQL

of Distinct Values for a column:

```
SELECT COUNT(DISTINCT(lastname)) AS "# Values"
FROM Customer;
```

Values
1001

Max Rows per Value for 3 most frequent values:

```
SELECT TOP 3 tvalue, tcount
FROM (SELECT lastname, COUNT(*)
      FROM Customer
      GROUP BY 1) t1 (tvalue, tcount)
ORDER BY tcount DESC;
```

tvalue	tcount(*)
Smith	185
Jones	183
Wilson	138

Average Rows per Value for a column:

```
SELECT 'Lastname' AS "Column Name",
       COUNT(*) / COUNT(DISTINCT(lastname)) AS "Average"
FROM Customer;
```

Column Name	Average
Lastname	149

Typical (Median) Rows per Value for a column:

```
SELECT 'Lastname' AS "Column Name",
       MEDIAN(cts) AS "Median"
FROM (SELECT lastname,
              COUNT(*) AS cts,
              COUNT(cts) OVER (PARTITION BY cts) AS ctscts
      FROM customer
      GROUP BY 1) t1;
```

Column Name	Median
Lastname	150

TableSize View

The TableSize[V][X] views are Data Dictionary views that provides AMP Vproc information about disk space usage at a table level, optionally for tables the current User owns or has SELECT privileges on.

Example

The SELECT statement on the facing page looks for poorly distributed tables by displaying the CurrentPerm figures for a single table on all AMP vprocs.

TableSize View

Provides AMP Vproc disk space usage at table level.

DBC.TableSize[V][X]

Vproc TableName	DatabaseName CurrentPerm	AccountName PeakPerm
--------------------	-----------------------------	-------------------------

Example: Display table distribution across AMPs of a NoPI table.

```
SELECT  Vproc
        ,CAST (TableName AS CHAR(15))
        ,CurrentPerm
FROM    DBC.TableSizeV
WHERE   DatabaseName = USER
AND     TableName = 'Customer_NoPI'
ORDER BY Vproc;
```

Result:

<u>Vproc</u>	<u>TableName</u>	<u>CurrentPerm</u>
0	Customer_NoPI	783,360
1	Customer_NoPI	812,032
2	Customer_NoPI	792,576
3	Customer_NoPI	803,840
4	Customer_NoPI	796,672
5	Customer_NoPI	802,304
6	Customer_NoPI	790,016
7	Customer_NoPI	793,088
8	Customer_NoPI	796,160
9	Customer_NoPI	800,768
10	Customer_NoPI	805,888
11	Customer_NoPI	801,792
12	Customer_NoPI	806,400
13	Customer_NoPI	781,212
:	:	:
29	Customer_NoPI	806,400

Using SQL to Predict Row Distribution

The facing page contains simple examples of SQL that can be used to determine actual row distribution for a table.

Using SQL to Predict Row Distribution

Given the previous NoPI Customer table, the Row Hash functions can be used to predict the distribution of data rows for a specific column or columns.

Ex: Display the distribution of Customer by custnumber which is unique within the table.

```
SELECT    HASHAMP (HASHBUCKET
              (HASHROW (custnumber))) AS "AMP #"
          , COUNT(*)
FROM      Customer_NoPI
GROUP BY  1
ORDER BY  1;
```

AMP #	Count(*)
0	5,806
1	5,811
2	5,775
3	5,750
4	5,692
5	5,765
6	5,808
:	:
29	5,845

Ex: Display the distribution of Customer by zipcode which is not unique within the table.

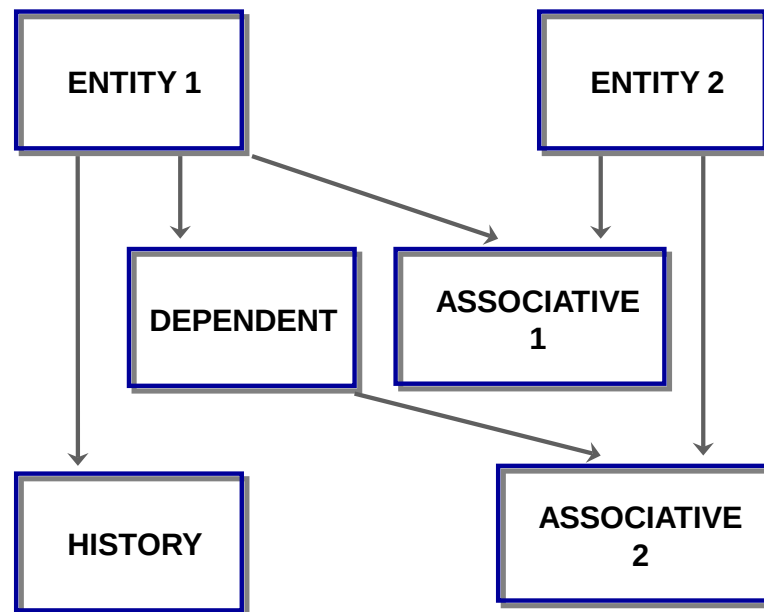
```
SELECT    HASHAMP (HASHBUCKET
              (HASHROW (zipcode))) AS "AMP #"
          , COUNT(*)
FROM      Customer_NoPI
GROUP BY  1
ORDER BY  1;
```

AMP #	Count(*)
0	6,211
1	4,845
2	5,237
3	6,031
4	6,139
5	5,243
6	7,186
:	:
29	5,496

E-R Diagram for Exercises

The E-R diagram on the facing page depicts the tables used in the exercises. Though the names of the tables and their columns are generic, the model is properly normalized to Third Normal Form (3NF).

E-R Diagram for Exercises

**Note:**

The exercise table and column names are generic so that index selections are not influenced by names.

Exercise 2 – Example

The facing page has an example of how to use Distribution demographics to identify PI candidates. On the following pages, you will be asked to identify PI candidates in a similar manner.

Use the Primary Index Candidate Guidelines below to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a ?

In later exercises, you will make the final index choices for these tables.

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates. These columns will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (maybe at least 10 times the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Example

On the following pages, there are sample tables with distribution demographics.

- Indicate ALL possible Primary Index candidates (UPI and NUPI).
- Later exercises will guide your final choices.

Primary Index Candidate Guidelines:

- PK and UNIQUE COLUMNS (ND)
- Any single column with:
 - High Distinct values (at least 10X)
 - Low Maximums for NULLs or a Value
 - Typical Rows that is close to Max Rows

300,000,000 Rows	A	B	C	D	E	F	G	H	
PK/FK	PK,SA		FK,NN	NN,ND					
Distinct Values	300M	35M	7.5M	300M	8	15M	15M	700	
Max Rows/Value	1	12	500	1	40M	50	4M	450K	
Max Rows/NULL	0	5	0	0	0	4M	0	50K	
Typical Rows/Value	1	7	35	1	35M	15	15	400K	
PI/SI	UPI	NUPI	NUPI?	UPI					
Collect Statistics (Y/N)									

Exercise 2 – Choosing PI Candidates

Use the Primary Index Candidate Guidelines to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a question mark (?).

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates and will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (at least 100% greater than the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Choosing PI Candidates (cont.)

Use the Primary Index Candidate Guidelines to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a question mark (?).

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates and will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (at least 100% greater than the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Choosing PI Candidates (cont.)

ENTITY 2		G	H	I	J	K	L
50,000,000 Rows							
PK/FK	PK,SA						
Distinct Values	50M	500K	45M	12	50	900K	
Max Rows/Value	1	200	2	5M	1.2M	60	
Max Rows/NULL	0	0	450K	0	0	0	
Typical Rows/Value	1	100	1	4M	950K	50	
PI/SI							
Collect Statistics (Y/N)							

Exercise 2 – Choosing PI Candidates (cont.)

Use the Primary Index Candidate Guidelines to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a question mark (?).

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates and will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (at least 100% greater than the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Choosing PI Candidates (cont.)

		DEPENDENT					
5,000,000 Rows		A	M	N	O	P	Q
PK/FK		PK				NN, ND	
		FK	SA				
	Distinct Values	2M	50	90K	3M	5M	2M
	Max Rows/Value	4	200K	75	2	1	5
	Max Rows/NULL	0	0	0	390K	0	1M
	Typical Rows/Value	1	60K	50	1	1	1
	PI/SI						
	Collect Statistics (Y/N)						

Exercise 2 – Choosing PI Candidates (cont.)

Use the Primary Index Candidate Guidelines to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a question mark (?).

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates and will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (at least 100% greater than the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Choosing PI Candidates (cont.)

		ASSOCIATIVE 1				
1.5 Billion Rows		A	G	R	S	
	PK/FK	PK				
		FK	FK,SA			
	Distinct Values	490M	50M	75K	4M	
	Max Rows/Value	5	40	21K	400	
	Max Rows/NULL	0	0	0	0	
	Typical Rows/Value	3	25	19K	350	
	PI/SI					
	Collect Statistics (Y/N)					

Exercise 2 – Choosing PI Candidates (cont.)

Use the Primary Index Candidate Guidelines to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a question mark (?).

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates and will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (at least 100% greater than the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Choosing PI Candidates (cont.)

ASSOCIATIVE 2					
500,000,000 Rows	A	M	G	T	U
PK/FK	PK				
	FK	FK			
Distinct Values	250M	50M	2.8M	750	
Max Rows/Value	3	150	180	675K	
Max Rows/NULL	0	0	0	0	
Typical Rows/Value	1	8	170	500K	
PI/SI					
Collect Statistics (Y/N)					

Exercise 2 – Choosing PI Candidates (cont.)

Use the Primary Index Candidate Guidelines to identify the PI candidates. Indicate whether they are UPI or NUPI candidates. Indicate borderline candidates with a question mark (?).

Primary Index Candidate Guidelines:

- ALL Unique Columns are PI candidates and will be identified with the abbreviation ND for No Duplicates.
- The Primary Key (PK) is a UPI candidate.
- Any single column with high Distinct Values (at least 100% greater than the number of AMPs), low Maximum Rows NULL, and with a Typical Rows per Value that is relatively close to the Maximum Rows per Value is a PI candidate.

Exercise 2 – Choosing PI Candidates (cont.)

		HISTORY					
3.65 Billion Rows		A	DATE	D	E	F	
PK/FK		PK					
	FK	SA					
Distinct Values	500M	730	N/A	N/A	N/A		
Max Rows/Value	18	5.5M	N/A	N/A	N/A		
Max Rows/NULL	0	0	N/A	N/A	N/A		
Typical Rows/Value	3	4.5M	N/A	N/A	N/A		
	PI/SI						
Collect Statistics (Y/N)							

What is a NoPI Table?

A NoPI Table is simply a table without a primary index. NoPI stands for No Primary Index.

Without a PI, the hash value as well as AMP ownership of a row is arbitrary. Within the AMP, there are no row-ordering constraints and therefore rows can be appended to the end of the table as if it were a spool table. Each row in a NoPI table has a hash bucket value that is internally generated. A NoPI table is internally treated as a hashed table; it is just that typically all the rows on one AMP will have the same hash bucket value.

What is a NoPI Table?

What is a No Primary Index (NoPI) Table?

- It is simply a table without a primary index
- As rows are inserted into a NoPI table, **rows are always appended at the end of the table and never inserted in a middle of a hash sequence.**
 - Organizing/sorting rows based on row hash is therefore avoided.

Basic Concepts

- Rows will still be distributed between AMPs. NoPI uses Random Generator code for SQL inserts or round robin logic for data blocks (e.g., TPT LOAD) to determine which AMP will receive rows or blocks of rows.
- Within an AMP, rows are simply appended to the end of the table. Rows will have a unique RowID – the Uniqueness Value is incremented.

Benefits

- A NoPI table will reduce skew in intermediate ETL tables which have no natural Primary Index.
- Loads using TPT Load (FastLoad) or TPT Stream (TPump) Array Insert into a NoPI staging table are faster.

Reasons to Consider Using NoPI Tables

The facing page identifies various reasons to consider using NoPI tables.

Why is a NoPI table useful?

- A NoPI can be very useful in those situations when the default primary index (first column) causes skewing of data between AMPs and performance degradation.
- This type of table provides a performance advantage in that data can be loaded and stored quickly into a NoPI table using the TPT Load operator (FastLoad) or TPT Stream operator (TPump) Array INSERT.

Reasons to Consider Using NoPI Tables

Reasons to consider using a NoPI Table

- Utilize NoPI tables instead of arbitrarily defaulting to first table column or creating an unnatural Primary Index from many columns.
- Some ETL tools generate intermediate tables to store data without a known distribution of values.

If the first column is used (defaults) as the primary index (NUPI), this may lead to skewed data and performance issues.

- The system default can be set to create tables without a primary index.
- As a staging table to be used with the mini-batch loading technique.
- A NoPI table can be used as a Sandbox table (or any table) where data can be inserted until an appropriate indexing method is determined.
- A NoPI table can be used as a Log file.
- As a Column Partitioned (columnar) table.

Creating a Table without a PI

The facing page identifies the syntax to create a table without a primary index.

If you attempt to include the key word SET (set table) and NO PRIMARY INDEX in the same CREATE TABLE statement, you will receive a syntax error.

Creating a Table without a PI

To create a NoPI table, specify the **NO PRIMARY INDEX** clause in the CREATE TABLE statement.

```
CREATE TABLE <table_name> (<column1>    <column1_datatype>,  
                             <column2>    <column2_datatype>,  
                             ... )  
  
NO PRIMARY INDEX;
```

Considerations:

- When a table is created with no primary index, the TableKind column is set to 'O' instead of 'T' and appears in the DBC.TVM table.
- If PRIMARY KEY or UNIQUE constraints are also defined, these will be implemented as Unique Secondary Indexes.
- A NoPI table is automatically created as a MULTiset table.

Distribution of Rows (or Blocks) for a NoPI Table

For SQL-based functions, the PE uses the following technique for the random generator.

The Query ID is used by the random generator to select a random row hash. The approach is to generate a random row hash in such a way that for a new request, data will generally be sent to a different AMP from the one that the previous request sent data to. The goal is to balance out the data as much as possible without the use of the primary index. The Query ID is selected for this purpose because it uses the PE vproc ID in its high digits and a counter-based value in its low digits.

There are two cases for INSERT; one is when only one single data row is processed and the other is when multiple data rows are processed with an Array INSERT request. In the case of an Array INSERT request, rows are sorted by their hash-owning AMPs so that the rows going to the same AMP can easily be grouped together into the same step. This random row hash will be generated once per request so that in the case of Array INSERT, the same random row hash is used for all of the rows. This means they all will be sent to the same AMP and usually in the same step.

With TPT Load or FastLoad utilities, blocks of data are sent to the AMPs. Each AMP (that receives blocks of data) simply **distributes blocks of data between all of the AMPs in a round robin fashion.**

The NoPI Table feature is another step toward extending or supporting Mini-Batch. By allowing a table with no primary index acting as a staging table, data can be loaded into the table a lot more efficiently and in turn faster. All of the rows in a data request, after being received by Teradata and converted into proper internal format, can be appended to a NoPI table without having to be redistributed to their hash-owning AMPs. Rows in a NoPI table are not hashed based on the primary index because there isn't one.

The hash values are all internally controlled and generated and therefore the rows can be stored in any particular order and in any AMP. That means sorting of the rows is avoided.

The performance advantage, especially for TPT Load (FastLoad), from using a NoPI table is most significant for applications that currently load data into a staging table to be transformed or standardized before being stored into another staging table or the target table. For those applications, using a NoPI table can avoid the unnecessary row redistribution and sorting work. Since the typical load into a NoPI table is faster, client resources (where TPT Load is executing) are freed up faster.

For TPT Stream (TPump), the performance advantage can be much bigger especially for applications that were not able to pack many rows into the same AMP step in a traditional PI table. On a NoPI table, all rows in a data request are packed into the same AMP step independently from the system configuration and the clustering of data. This will generally lead to big reductions in CPU and IO usage.

Distribution of Rows (or Blocks) for a NoPI Table

How is data distributed between the AMPs for a NoPI table?

- Depending on the type of request (e.g., SQL INSERTs versus TPT Load or FastLoad), Teradata uses two different approaches to distribute the data row(s).

For SQL INSERTs, the PE uses "random generator" code.

- The Query ID is hashed (via hashing algorithm) to select a random hash value.
 - This HBN of this row hash is used by the Message Passing Layer to select an AMP. This row hash ONLY used to select an AMP and is not used by the AMP. Different INSERT requests will hash differently and therefore, the rows will be randomly distributed between the AMPs.
 - In the case of an Array INSERT request, this random hash bucket value will be generated once per request so that in the case of Array INSERT, the same random hash bucket value is used for all of the rows.

For TPT Load or FastLoad Inserts, blocks of data are simply distributed between the AMPs in a round robin fashion.

In both cases, although there is no primary index in a NoPI table, every row will still have a valid 64-bit RowID.

- The AMP internally selects a hash bucket value that the AMP owns and uses it as the first part (20 bits) of the RowID and adds a 44 bit uniqueness value.

The Row ID for a NoPI Table

For a NoPI table, the AMP will assign a RowID (64 bits) for a row or a set of rows using a hash bucket that the AMP owns. For a NoPI table, the RowID will consist of a 20-bit hash bucket followed by 44 bits that are used for the uniqueness part of the RowID. Only the first 20 bits (hash bucket) are used.

For a NoPI table, all rows owned by an AMP initially use the lowest hash bucket associated to that AMP by the NoPI hash map in their rowid. The NoPI hash map is a special hash map that only includes hash buckets from the regular PI hash map where the AMP owning this hash bucket does not change if the number of AMPs increases. This avoids having to move rows to a different AMP when doing a Reconfig or restore/copy.

The purpose of carrying a hash bucket in the RowID is to have a means to identify the AMP on which a row resides when there is access via a secondary index, join index, or RowID spool. The hash bucket is all that is required from a RowID for a PA (Primary AMP option with a Column Partitioned table) or NoPI table to indicate the row's owning AMP. As more rows are added to the table, the uniqueness value is sequentially incremented.

For older systems using a 16-bit hash buckets, the RowID for a NoPI table will have 16 bits for the hash bucket value and 48 bits for the uniqueness id.

The Row ID for a NoPI Table

The RowID is still 64 bits, but it is created a little differently for a NoPI table.

- The first 20 bits represents the hash bucket that is internally selected by the AMP.
- The remaining 44 bits are used for the uniqueness value of rows in a NoPI table.
- Note: Older systems may still be configured to use 16 bits for the hash bucket numbers – if so, then the uniqueness value will utilize 48 bits of the RowID.

Row ID for NoPI table

Hash Bucket 20 bits	Uniqueness Value 44 bits
------------------------	-----------------------------

Each row still has a Row ID as a prefix.

Row ID		Row Data		
Hash Bucket	Uniqueness	CustNo	LastName	FirstName
FE397	0000000001	001018	Reynolds	Jane
FE397	0000000002	001020	Davidson	Evan
FE397	0000000003	001031	Green	Jason
FE397	0000000004	001014	Jacobs	Paul
FE397	0000000005	001012	Garcia	Jose
FE397	0000000006	001021	Carnet	Jean
:	:	:	:	:

Rows are logically maintained in Row ID sequence.

The Row ID for a NoPI Table (cont.)

For a NoPI table, the AMP will assign a RowID (64 bits) for a row or a set of rows using a hash bucket that the AMP owns. This 64-bit RowID can be used by secondary and join indexes.

What is different about the RowID for a NoPI table is that the uniqueness id is 44 bits long instead of 32 bits. The additional 12 bits available in the row hash are added to the 32-bit uniqueness. This gives a total of 44 bits to use for the uniqueness part of the RowID. For each hash bucket, there can be up to 17 trillion rows per AMP (approximately).

For systems using a 16-bit hash buckets, the RowID for a NoPI table will have 16 bits for the hash bucket value and 48 bits for the uniqueness id.

The RowID is still 64 bits long and a unique identifier of a row within a table.

The Row ID for a NoPI Table (cont.)

The RowID is 64 bits and can be referenced by secondary and join indexes.

- The first 20 bits represent the hash bucket value which is internally chosen by and controlled by the AMP.
- Remaining 44 bits are used for the uniqueness value of rows in a NoPI table. This module assumes that 20-bit hash bucket numbers are used.
 - The uniqueness value starts from 1 and will be sequentially incremented.
 - With 44 bits, there can be approximately 17 trillion rows on an AMP.
- Normally, all rows in a NoPI table on an AMP will have the same hash bucket value (first 20 bits) and the 44-bit uniqueness value will start at 1 and be sequentially incremented.
- Therefore, each row in a NoPI table will have a RowID with a hash bucket value that is actually owned by the AMP storing the row.

Fallback and index maintenance work the same as if the table is a primary index table.

As always, the RowID is transparent to the end-user.

Multiple NoPI Tables at the AMP Level

The facing page illustrates an example of two NoPI tables in a 30-AMP system.

Other NoPI considerations include:

Archive/Recovery Issues

Archive/Restore will be supported for NoPI table. Archiving a table or a database and restoring or copying that to the same system or a different system should work out fine with the existing scheme for NoPI table when no data redistribution takes place (same number of AMPs). Data redistribution takes place when there is a difference in configuration or hash function between the source system and the target system. In the case of a difference in configuration, each row in a table will be looked at and if its hash bucket belongs to some other AMP using the new configuration, that row will be redistributed to its hash-owning AMP.

Since one hash bucket is normally enough to use to assign RowID to all of the rows on each AMP, when we restore or copy data to a different configuration with more AMPs, there will be AMPs that will not have any data at all. This means that data in a NoPI table can be skewed after a Restore or Copy.

This is because permanent space is divided equally among the AMPs whether or not any of them get any data. As some AMPs not getting any data from a Restore or Copy, some other AMPs will get more data compared to what it was in the source system and this will require more space allocated overall.

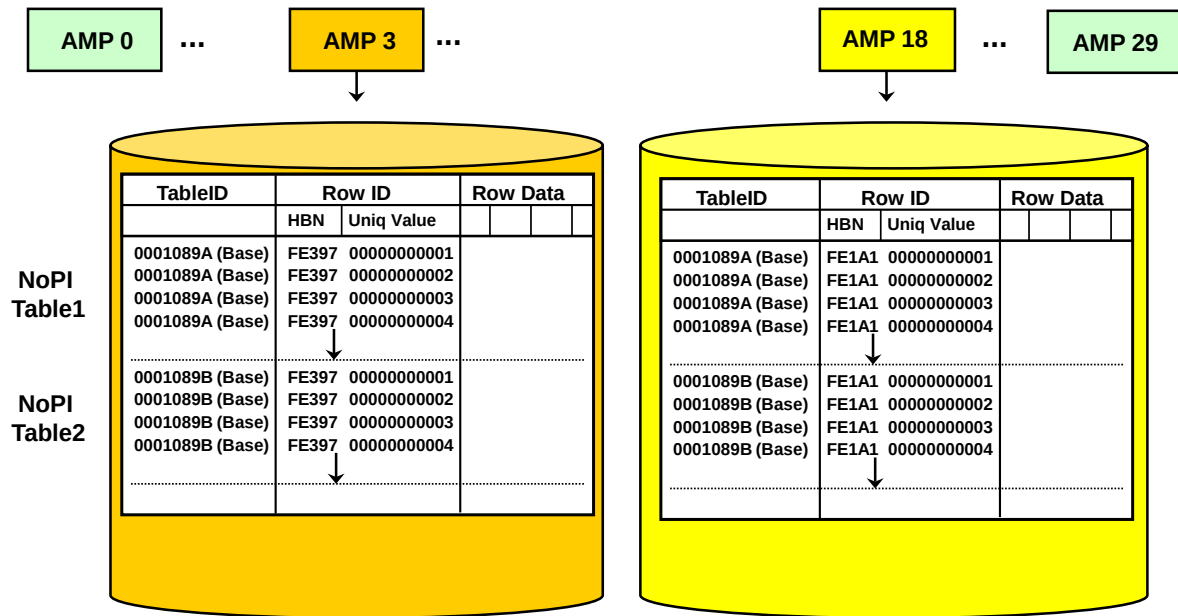
However, as a staging table, NoPI table is not intended to stay around for too long so it is not expected to have many NoPI tables being restored or copied.

Reconfig Issues

Reconfig will be supported for NoPI table. The issue with Reconfig is very similar to that of Restore or Copy to a different configuration. Although rows in a NoPI table are not hashed based on the primary index and the AMPs where they reside are arbitrary, but each row does have a RowID with a hash bucket that is owned by the AMP storing that row.

Redistributing rows in a NoPI table via Reconfig can be done by sending each row to the AMP that owns the hash bucket in that row based on the new configuration map. As with Restore and Copy, Reconfig can make a NoPI table skewed by going to a configuration with more AMPs.

Multiple NoPI Tables at the AMP Level



Data within an AMP is logically stored in Table ID / Row ID sequence.

Loading Data into a NoPI Table (SQL)

The facing page summarizes various techniques of inserting data into a NoPI table using SQL INSERT or INSERT-SELECT statements.

If you are using INSERT-SELECT from a table that is skewed into a target NoPI table, the target NoPI table will have the same skew because the data is copied within the AMPs.

However, the HASH BY and/or LOCAL ORDER BY options can be used to redistribute the data before copying the data into the NoPI table.

Note: You can only specify HASH BY and LOCAL ORDER BY clauses if the target table or the underlying target view table has no primary index (for example, a column-partitioned table). If the table has a primary index, the request is aborted and an error is returned.

If you specify a HASH BY RANDOM clause, data blocks of selected rows are first randomly redistributed. If you also specify a LOCAL ORDER BY clause, then the rows are locally ordered and inserted locally into the target table or underlying target view table.

```
INSERT INTO target_nopi_table
SELECT * FROM source_table
HASH BY RANDOM;
```

HASH BY RANDOM (1, 2000000000) is useful to redistribute each individual selected row when there is no particular column set on which to hash distribute the rows, and when a more even distribution is needed than the HASH BY RANDOM clause provides.

```
INSERT INTO target_nopi_table
SELECT * FROM source_table
HASH BY RANDOM (1,2000000000);
```

Loading Data into a NoPI Table (SQL)

Simple INSERTs

- For a simple INSERT, the PE selects a random AMP where the row is sent to. That AMP then turns the row into proper internal format and appends it to the end of the NoPI table.

INSERT-SELECT

- When inserting data from a source table (PI or NoPI) table into a NoPI target table, data from the source table will NOT be redistributed and will be locally appended into the target table.

Example: **INSERT INTO target_nopi_table**
 SELECT * FROM skewed_table;

With INSERT-SELECT, the NoPI table can be skewed if the source table is skewed.

If the source table is skewed, the "HASH BY" option can be used to randomly (more evenly) distribute the data between the AMPs for a NoPI table.

- The HASH BY RANDOM clause will redistribute data blocks randomly between the AMPs.

```
INSERT INTO target_nopi_table  
SELECT * FROM skewed_table        HASH BY RANDOM;
```

- The HASH BY RANDOM (1, 2000000000) is useful to redistribute individual rows and provides a more even distribution than the HASH BY RANDOM clause provides.

```
INSERT INTO target_nopi_table  
SELECT * FROM source_table        HASH BY RANDOM (1,2000000000);
```

Loading Data into a NoPI Table (Utility)

The facing page summarizes techniques of loading data into a NoPI table using the TPT Load (FastLoad) or TPT Stream (TPump) operators.

Loading Data into a NoPI Table (Utility)

TPT Load or FastLoad

- Blocks of data are sent to the AMP load sessions and the AMP randomly distributes the blocks between the AMPs usually resulting in even distribution of the data between AMPs.
- The blocks are distributed between the AMPs using a round-robin technique.

TPT Stream or TPump

- With TPT Stream (TPump) Array INSERT, rows are packed together in a request and distributed to an AMP and then appended to the NoPI table on that AMP.
- Different requests are distributed to different AMPs by the PE. This will usually result in even distribution of the data between the AMPs.

NoPI Options

The following options are available to a NoPI table:

- FALLBACK
- Secondary indexes – USI and NUSI
- Join and reference indexes
- Primary Key and Foreign Key constraints are allowed on a NoPI table.
- LOBs are allowed on a NoPI table.
- INSERT, UPDATE, and DELETE trigger actions are allowed on a NoPI table.
 - UPDATE trigger actions were allowed starting with Teradata 13.00.00.03.
- NoPI table can be a Global Temporary or Volatile table.
- COLLECT/DROP STATISTICS are allowed on a NoPI table.
- TPT Load (FastLoad) – note that duplicate rows are loaded and not deleted with a NoPI table.
- Column partitioned (columnar) table

The following limitations apply to a NoPI table:

- SET is not allowed. Default is MULTISSET for both Teradata and ANSI mode.
- No columns are allowed to be specified for a primary index.
- Partitioned primary index is not allowed.
- Permanent journaling is not allowed.
- Identity column is not allowed.
- Cannot be created as a queue or as an error table.
- Hash index is not allowed on a NoPI table.
- TPT Update (MultiLoad) cannot be used to load a NoPI table.
- UPDATE, UPSERT, and MERGE-INTO operations are using the NoPI table as the target table.

NoPI Table Options

Options available with NoPI tables

- FALLBACK
- Secondary indexes – USI and NUSI
- Join and reference indexes
- Primary Key and Foreign Key constraints are allowed.
- LOBs are allowed on a NoPI table.
- INSERT, UPDATE, and DELETE trigger actions are allowed on a NoPI table.
- Can be a Global Temporary or Volatile table.
- COLLECT/DROP STATISTICS are allowed.
- TPT Load (FastLoad) – note that duplicate rows are loaded and not deleted with a NoPI table.
- Can be column partitioned.

Limitations of NoPI tables

- SET tables are not allowed.
- Partitioned primary index is not allowed.
- Permanent journaling is not allowed.
- Identity column is not allowed.
- Cannot be a queue or as an error table.
- Hash index is not allowed on a NoPI table.
- TPT Update (MultiLoad) cannot be used on a NoPI table.
- UPSERT and MERGE-INTO operations using the NoPI table as the target table are not allowed.

Summary

The facing page summarizes some of the key concepts covered in this module.

Summary

Tables with a Primary Index:

- **Base PI on the column(s) most often used for access, provided that the values are unique or nearly unique.**
- Duplicate values hash to the same AMP and are stored in the same data block when possible.
- **PRIMARY KEY and/or UNIQUE constraints are always implemented as a unique index (either a UPI or a USI).**

Tables without a Primary Index:

- Although there is no primary index in a NoPI table, rows do have a valid row ID with both hash and uniqueness.
 - Hash value is internally selected in the AMP
- Rows in a NoPI table will be even distributed between the AMPs either by PE random generator code (SLQ Inserts) or round-robin distribution of blocks with TPT Load.

Module 11: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 11: Review Questions

1. Which trade-off must be balanced to make the best choice for a primary index? ____
 - a. Access and volatility
 - b. Access and block size
 - c. Block size and volatility
 - d. Access and distribution

2. When volatility is considered as one of the Primary Index choice criteria, what is analyzed? ____
 - a. Degree of uniqueness
 - b. How often the data values will change
 - c. How often the fixed length rows will change
 - d. How frequently the column is used for access

3. To optimize the use of disk space, the designer should choose a primary index that _____.
 - a. is non-unique
 - b. consists of one column
 - c. is unique or nearly unique
 - d. consists of multiple columns
 - e. has fewer distinct values than AMPs

Module 11: Review Questions (cont.)

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 11: Review Questions (cont.)

4. For NoPI tables, what are 2 ways in which the Random Generator is executed?
 - a. At the PE level for ad hoc SQL requests
 - b. At the TPT Stream client level for array insert operations
 - c. At the PE level with TPT Stream for array insert operations
 - d. At the AMP level for INSERT-SELECT into an empty NoPI table

5. Assume DBSControl flag #53 (Primary Index Default) is set to N (No Primary Index), which two indexes are created for TableX given the following DDL command?


```
CREATE TABLE TableX
      (col1    INTEGER NOT NULL UNIQUE
      ,col2    CHAR(10) NOT NULL PRIMARY KEY
      ,col3    CHAR(80));
```

 - a. col1 will be a UPI
 - b. col1 will be a USI
 - c. col2 will be a UPI
 - d. col2 will be a USI

6. Which two options are permitted for NoPI tables?
 - a. Fallback
 - b. TPT Update
 - c. Hash Index
 - d. BLOBs and CLOBs

Lab Exercise 11-1

Check your understanding of the concepts discussed in this module by completing the lab exercise as directed by your instructor.

Lab Exercise 11-1

Lab Exercise 11-1

Purpose

In this lab, you will use Teradata Studio or Teradata Studio Express to evaluate various columns of table as primary index candidates.

What you need

Populated PD.Employee table; your empty Employee table

Tasks

1. Using INSERT/SELECT, populate your "Employee" table with all rows from the PD.Employee table. Verify the number of rows in your table.

```
INSERT INTO Employee SELECT * FROM PD.Employee;
```

```
SELECT COUNT(*) FROM Employee;    Count = _____
```

Lab Exercise 11-1 (cont.)

Use the following SQL to determine the column metrics for this Lab.

of Distinct Values for a column:

```
SELECT    COUNT(DISTINCT(column_name))
FROM      tablename;
```

Max Rows per Value for all values in a column:

```
SELECT    column_name, COUNT(*)
FROM      table_name
GROUP BY  1
ORDER BY  2 DESC;
```

Max Rows per Value for Top 3 values in a column:

```
SELECT    TOP 3 tvalue, tcount
FROM      (SELECT column_name, COUNT(*)
           FROM table_name GROUP BY 1)
           t1 (tvalue, tcount)
ORDER BY  tcount DESC;
```

Max Rows with NULL in a column:

```
SELECT    COUNT(*)
FROM      table_name
WHERE     column_name IS NULL;
```

Average Rows per Value for a column:

```
SELECT    COUNT(*)/COUNT(DISTINCT(column_name)) AS averagects
FROM      table_name;
```

Typical (Median) Rows per Value for a column:

```
SELECT    MEDIAN(cts)AS mediancts
FROM      (SELECT column_name,
                 COUNT(*) AS cts,
                 COUNT(cts) OVER (PARTITION BY cts) AS ctscs
           FROM table_name
           GROUP BY 1) t1;
```

Lab Exercise 11-1 (cont.)

2. Collect column demographics for each of these columns in Employee and determine if the column would be a primary index candidate or not.

By using the SHOW TABLE Employee command, you should be able to complete the EmpNumber information without executing any SQL.

	Distinct Values	Max Rows for a Value	Max Rows NULL	Avg Rows per Value	Candidate for PI (Y/N)
EmpNumber					
DeptNumber					
JobCode					
LastName					

Lab Exercise 11-2

Distribution of table space by AMP:

```
SELECT      Vproc, TableName (CHAR(15)), CurrentPerm
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name'
ORDER BY    1;
```

To determine the minimum and maximum space used by AMP:

```
SELECT      MIN(CurrentPerm) AS Min_Size,
            MAX(CurrentPerm) AS Max_Size
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name';
```

To display sum of space, minimum space used by an AMP, maximum space used by an AMP in MB (Megabytes) AND calculate a Skew Factor for all the tables in your database:

```
SELECT      TableName,
            SUM(CurrentPerm)/(1024**2) AS CurentPerm_MB,
            MIN(CurrentPerm)/(1024**2) AS Min_Size,
            MAX(CurrentPerm)/(1024**2) AS Max_Size,
            (100 - (AVG(CurrentPerm)/
                    NULLIFZERO (MAX(CurrentPerm))* 100))
                                                    AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```

As an alternative to display this information in MB (Megabytes) AND calculate a Skew Factor for all the tables in your database, you can also use the following:

```
SELECT      TableName,
            SUM(CurrentPerm)/(2**20) AS CurentPerm_MB,
            MIN(CurrentPerm)/(2**20) AS Min_Size,
            MAX(CurrentPerm)/(2**20) AS Max_Size,
            (100 - (AVG(CurrentPerm)/
                    NULLIFZERO (MAX(CurrentPerm))* 100))
                                                    AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```

Lab Exercise 11-2

Lab Exercise 11-2

Purpose

In this lab, you will use the DBC.TableSizeV view to determine space distribution on a per AMP basis.

What you need

Your populated Employee table.

Tasks

1. Use SHOW TABLE command to determine which column is the Primary Index. PI = _____

Determine the AMP space usage of your Employee table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? size _____

What is the greatest amount of permanent space on one of the AMPs? size _____

2. Create a new table named Employee_2 with the same columns as Employee except specify LastName as the Primary Index.

Using INSERT/SELECT, populate your Employee_2 table with all rows from your Employee table.

Determine the AMP space usage of your Employee_2 table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? size _____

What is the greatest amount of permanent space on one of the AMPs? size _____

Lab Exercise 11-3 (optional)

Distribution of table space by AMP:

```
SELECT      Vproc, TableName (CHAR(15)), CurrentPerm
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name'
ORDER BY    1;
```

To determine the minimum and maximum space used by AMP:

```
SELECT      MIN(CurrentPerm) AS Min_Size,
            MAX(CurrentPerm) AS Max_Size
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name';
```

To display sum of space, minimum space used by an AMP, maximum space used by an AMP in MB (Megabytes) AND calculate a Skew Factor for all the tables in your database:

```
SELECT      TableName,
            SUM(CurrentPerm)/(1024**2) AS CurentPerm_MB,
            MIN(CurrentPerm)/(1024**2) AS Min_Size,
            MAX(CurrentPerm)/(1024**2) AS Max_Size,
            (100 - (AVG(CurrentPerm)/
                    NULLIFZERO (MAX(CurrentPerm))* 100))
                                                    AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```

As an alternative to display this information in MB (Megabytes) AND calculate a Skew Factor for all the tables in your database, you can also use the following:

```
SELECT      TableName,
            SUM(CurrentPerm)/(2**20) AS CurentPerm_MB,
            MIN(CurrentPerm)/(2**20) AS Min_Size,
            MAX(CurrentPerm)/(2**20) AS Max_Size,
            (100 - (AVG(CurrentPerm)/
                    NULLIFZERO (MAX(CurrentPerm))* 100))
                                                    AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```

Lab Exercise 11-3 (optional)

Lab Exercise 11-3 (Optional)

Purpose

In this lab, you will determine data distribution on a per AMP basis for NoPI tables.

What you need

Your populated Employee_2 table (created in the previous exercise).

Tasks

1. Use the SHOW TABLE for Employee_2 to help create a new, similar table (same column names and definitions, etc.) named "Employee_NoPI" and does NOT have a primary index.

Using INSERT/SELECT, populate the Employee_NoPI table with all rows from your Employee_2 table.

Determine the AMP space usage of the Employee_NoPI table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? size _____

What is the greatest amount of permanent space on one of the AMPs? size _____

Is the distribution similar to the Employee_2 table (yes or no)? _____

Why or why not? _____

Lab Exercise 11-3 (optional) – cont.

Distribution of table space by AMP:

```
SELECT      Vproc, TableName (CHAR(15)), CurrentPerm
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name'
ORDER BY    1;
```

To determine the minimum and maximum space used by AMP:

```
SELECT      MIN(CurrentPerm) AS Min_Size,
            MAX(CurrentPerm) AS Max_Size
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name';
```

To display sum of space, minimum space used by an AMP, maximum space used by an AMP in MB (Megabytes) AND calculate a Skew Factor for all the tables in your database:

```
SELECT      TableName,
            SUM(CurrentPerm)/(1024**2) AS CurentPerm_MB,
            MIN(CurrentPerm)/(1024**2) AS Min_Size,
            MAX(CurrentPerm)/(1024**2) AS Max_Size,
            (100 - (AVG(CurrentPerm)/
                    NULLIFZERO (MAX(CurrentPerm))* 100))
                                AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```

As an alternative to display this information in MB (Megabytes) AND calculate a Skew Factor for all the tables in your database, you can also use the following:

```
SELECT      TableName,
            SUM(CurrentPerm)/(2**20) AS CurentPerm_MB,
            MIN(CurrentPerm)/(2**20) AS Min_Size,
            MAX(CurrentPerm)/(2**20) AS Max_Size,
            (100 - (AVG(CurrentPerm)/
                    NULLIFZERO (MAX(CurrentPerm))* 100))
                                AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```


Lab Exercise 11-3 (optional) – cont.

2. DELETE all the rows in Employee_NoPI.

Using INSERT/SELECT with the **HASH BY RANDOM** option, populate your **Employee_NoPI** with all rows from your **Employee_2** table.

Determine the AMP space usage of your **Employee_NoPI** table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? size _____

What is the greatest amount of permanent space on one of the AMPs? size _____

Is the space distribution the same as the **Employee_2** table (yes or no)? _____

Is the table distributed as evenly as expected (yes or no)? _____

Why or why not? _____

3. DELETE all the rows in Employee_NoPI.

Using INSERT/SELECT with the **HASH BY RANDOM (1, 2000000000)** option, populate **Employee_NoPI** with all the rows from your **Employee_2** table.

Determine the AMP space usage of your **Employee_NoPI** table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? size _____

What is the greatest amount of permanent space on one of the AMPs? size _____

Is the space distribution similar to the **Employee** table (yes or no)? _____

Is the table distributed as evenly as expected (yes or no)? _____

Why or why not? _____

Notes



Data Distribution and Hashing

After completing this module, you will be able to:

- Describe the data distribution form and method.
- Describe Hashing.
- Describe Primary Index hash mapping.
- Describe the reconfiguration process.
- Describe a Block Layout.
- Describe File System Read Access.

Teradata Proprietary and Confidential

Notes

Table of Contents

Data Distribution.....	12-4
Hashing	12-6
Enhanced Hashing Algorithm Starting with Teradata 13.10	12-6
Hash Related Expressions.....	12-8
Hashing – Numeric Data Types	12-10
Multi-Column Hashing	12-12
Additional Hash Examples.....	12-14
Using Hash Functions to View Distribution	12-16
Identifying the Hash Buckets	12-16
Identifying the Primary AMPs	12-16
Primary Index Hash Mapping.....	12-18
Hash Maps.....	12-20
Primary Hash Map	12-22
Hash Maps for Different Systems	12-24
Fallback Hash Map	12-26
Reconfiguration.....	12-28
MAPS TD 16.10 Reconfiguration.....	12-30
MAPS TD 16.10 Sparse Maps	12-32
Row Retrieval via PI Value – Overview	12-34
Names and Object IDs	12-36
Table ID	12-38
Spool File Table IDs	12-38
Table ID Example	12-40
Row ID	12-42
AMP File System – Locating a Row via PI	12-44
Teradata File System Overview	12-46
Data Block Layout	12-48
Example of Locating a Row – Master Index	12-50
Example of Locating a Row – Cylinder Index.....	12-52
Example of Locating a Row – Data Block.....	12-54
Accessing the Row within the Data Block.....	12-56
AMP Read I/O Summary	12-58
Module 12: Review Questions.....	12-60

Data Distribution

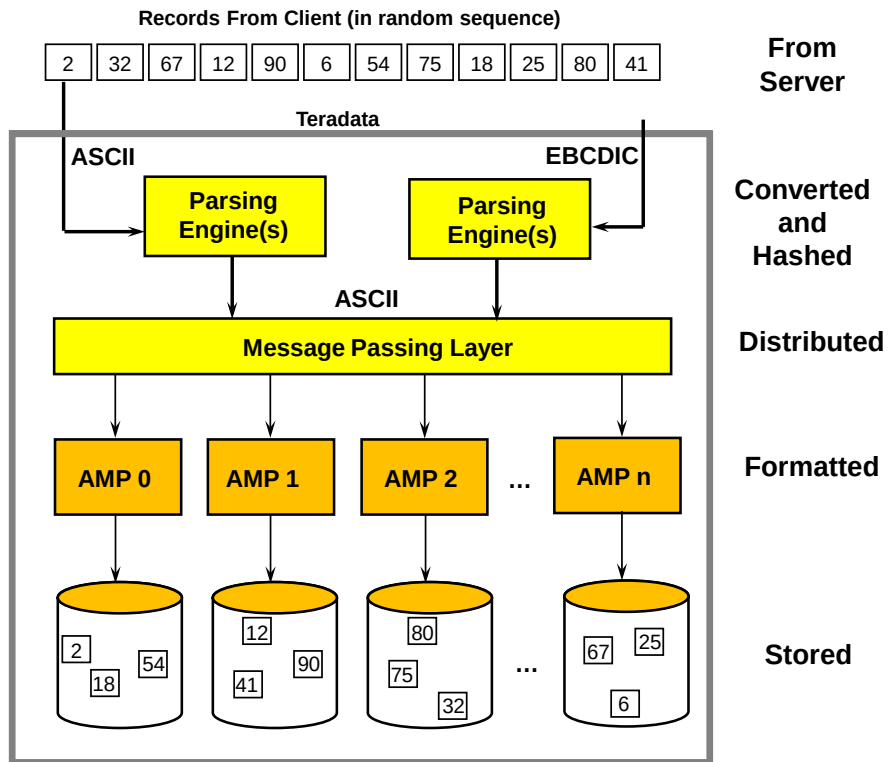
Parsing Engines (PE) are assigned either to channel connections (e.g., IBM Mainframe) or to LAN connections. If the input is in EBCDIC, the PE converts it to ASCII before any hashing and distribution takes place. Data is usually stored by the AMPs in 8-bit ASCII or 16-bit UNICODE.

A USER may have a COLLATION = EBCDIC, ASCII, MULTINATIONAL, or HOST. If the HOST is an EBCDIC host or COLLATION = EBCDIC, then the AMPs convert from ASCII to EBCDIC before doing any comparisons or sorts. MULTINATIONAL allows sites to create their own collation file. Otherwise, all comparisons and sorts use the ASCII collating sequence.

Teradata has no concept of pre-allocated table space. The rows of all hashed tables are distributed randomly across all AMPs and then randomly within the space available on the selected AMP.

Data Distribution

Data distribution is dependent on the hash value of the primary index for a table with a PI.



Hashing

Hashing is the mechanism by which Teradata utilizes the Primary Index to distribute rows of data. The Hashing Algorithm acts like a mathematical “blender”. It takes up to 64 columns of mixed data as input and generates a single 32-bit binary value called a **Row Hash**.

- The Row Hash is the logical storage locator of the row. A part of this value is used in determining the AMP to which the row is distributed.
- Teradata uses the Row Hash value for distribution, placement and retrieval of rows.

The Hashing Algorithm is random but consistent. Although consecutive PI values do not normally produce consecutive hash values, identical **Primary Index (PI)** values always generate the same Row Hash (assuming that the data types hash identically). Rows with the same Row Hash are always distributed to the same AMP.

Different PI values rarely produce the same Row Hash. When this does occur, they are known as **Hash Synonyms** or **Hash Collisions**.

Note: Upper and lower case values hash to the same hash value. For example, ‘Jones’ and ‘JONES’ generate the same hash value.

Enhanced Hashing Algorithm Starting with Teradata 13.10

This enhancement is targeted to reduce the number of hash collisions for character data stored as either Latin or Unicode, notably strings that contain primarily numeric data. Reduction in hash collisions reduces access time per AMP and produces a more balanced row distribution which in-turn improves parallelism. Reduced access time and increased parallelism translate directly to better performance.

This feature is available to new systems and requires a System Initialization (sysinit) for existing systems. It is anticipated that typically this activity would be performed during technology refresh opportunities.

- The Hashing Algorithm creates a fixed length value from any length input string.
- **Input** to the algorithm is the Primary Index (PI) value of a row.
- **The output from the algorithm is the Row Hash.**
 - A 32-bit binary value.
 - Used to identify the AMP of the row and the logical storage location of the row within the AMP.
 - Table ID + Row Hash is used to locate the Cylinder and Data Block.
- Row Hash uniqueness depends directly on PI uniqueness.
 - Good data distribution depends directly on Row Hash uniqueness.
- **The algorithm produces random, but consistent, Row Hashes.**
 - **The same PI value and data type combination always hash identically.**
 - Rows with the same Row Hash will always go to the same AMP.

Hash Related Expressions

The Teradata Database includes extensions to Teradata SQL, known as **hash functions**, which allow the user to extract statistical properties from the current index, evaluate those properties for other columns to determine their suitability as a future primary index, or more effectively design the primary index of rows. These statistics also help minimize hash synonyms and enhance data distribution uniformity. Hash functions are valid within a Teradata SQL statement where other functions (like SUBSTRING or INDEX) can occur.

HASHROW — this function returns the row hash value of a given sequence of expressions in BYTE (4) data type. For example, the following statement returns the average number of rows per row hash where C1 and C2 constitute an index (or potential index) of table TabX

```
SELECT COUNT(*) (FLOAT) / COUNT (DISTINCT(HASHROW (C1,C2))
FROM TableX;
```

HASHBUCKET — this function returns the bucket number that corresponds to a *hashrow*. The bucket number is an integer type. The following example returns the number of rows in each hash bucket where C1 and C2 are an index (or potential index) of table TabX:

```
SELECT HASHBUCKET (HASHROW(C1,C2)), COUNT(*)
FROM TableX
GROUP BY 1 ORDER BY 1;
```

Query results can be treated as a histogram of table distribution among the hash buckets.

HASHAMP and **HASHBACKAMP** — this function returns the identification number of the primary or fallback AMP corresponding to a *hashbucket*. With Teradata V2R6.2 (and before), HASHAMP accepts only integer values between 0 and 65,535 as its argument. In this example, HASHAMP is used to determine the number of primary rows on each AMP where C1 and C2 are to be the primary index of table TabX:

```
SELECT HASHAMP (HASHBUCKET (HASHROW (C1, C2))), COUNT(*)
FROM TableX
GROUP BY 1 ORDER BY 1;
```

Query results can be treated as a histogram of the table distribution among the AMPs.

Further information on these functions and their uses can be found in the *Teradata RDBMS SQL Reference*.

Note the examples on the facing page. This example was captured on a 30 AMP system using a hash map with 1,048,576 entries.

The row hash of the literal 'Teradata' is the same with 16-bit or 20-bit hash bucket numbers. However, the target AMP numbers are different for a system with 65,536 hash buckets as compared to the same system with 1,048,576 hash buckets.

Hash Related Expressions

- The SQL hash functions are:

HASHROW (column(s))	HASHBUCKET (hashrow)
HASHAMP (hashbucket)	HASHBAKAMP (hashbucket)

- Example 1:

```
SELECT  HASHROW ('Teradata')           AS "Hash Value"
        , HASHBUCKET (HASHROW ('Teradata')) AS "Bucket Num"
        , HASHAMP (HASHBUCKET (HASHROW ('Teradata'))) AS "AMP Num"
        , HASHBAKAMP (HASHBUCKET (HASHROW ('Teradata'))) AS "AMP Fallback Num";
```

Hash Value	Bucket Num	AMP Num	AMP Fallback Num
F5-C4-BC-93	1006667	12	13

AMP Numbers based on 16-AMP system with 1,048,576 hash buckets.

- Example 2:

```
SELECT  HASHROW ('Teradata') AS "Hash Value 1"
        , HASHROW ('Teradata ') AS "Hash Value 2"
        , HASHROW (' Teradata') AS "Hash Value 3";
```

Hash Value 1	Hash Value 2	Hash Value 3
F5-C4-BC-93	F5-C4-BC-93	01-98-9D-47

Note: Literals are converted to Unicode and then hashed.

Hashing – Numeric Data Types

The hashing algorithm will hash the same numeric value in different data types to the same value.

A DATE data type and an INTEGER data type hash to the same value. An example follows:

```
CREATE TABLE tableE
  (c1_int      INTEGER
   ,c2_date    DATE)
UNIQUE PRIMARY INDEX (c1_int);

INSERT INTO tableE (1010601, 1010601);
INSERT INTO tableE (NULL, NULL);

SELECT c1_int, HASHROW (c1_int), HASHROW (c2_date) from tableE;
```

<u>c1_int</u>	<u>HASHROW (c1_int)</u>	<u>HASHROW (c2_date)</u>
1010601	12-13-C4-58	12-13-C4-58
?	00-00-00-00	00-00-00-00

A second example follows:

```
CREATE TABLE tableF
  (c1_int      INTEGER
   ,c2_int      INTEGER
   ,c3_char     CHAR(4)
   ,c4_char     CHAR(4))
UNIQUE PRIMARY INDEX (c1_int, c2_int);

INSERT INTO tableF (0, NULL, '0', NULL);

SELECT  HASHROW (c1_int) AS "Hash c1"
        ,HASHROW (c2_int) AS "Hash c2"
        ,HASHROW (c3_char) AS "Hash c3"
        ,HASHROW (c4_char) AS "Hash c4"
FROM    tableF;
```

<u>Hash c1</u>	<u>Hash c2</u>	<u>Hash c3</u>	<u>Hash c4</u>
00-00-00-00	00-00-00-00	2B-B7-F6-D9	00-00-00-00

Note: The BTEQ commands .SET SIDETITLES and .SET FOLDLINE were used to display the output on the bottom of the facing page.

Hashing – Numeric Data Types

- The Hashing Algorithm hashes the following numeric data types to the same hash value: **BYTEINT, SMALLINT, INTEGER, BIGINT, DECIMAL(x,0), DATE**

```
CREATE TABLE tableA
(c1_byteint BYTEINT
,c2_smallint SMALLINT
,c3_integer INTEGER
,c4_bigint BIGINT
,c5_dec90 DECIMAL(9,0)
,c6_dec92 DECIMAL(9,2)
,c7_number NUMBER
,c8_num90 NUMBER(9,0)
,c9_num92 NUMBER(9,2)
,c10_float FLOAT
,c11_char CHAR(10))
PRIMARY INDEX (c1_byteint);
```

```
SELECT HASHROW (c1_byteint) AS "Hash Byteint"
,HASHROW (c2_smallint) AS "Hash Smallint"
,HASHROW (c3_integer) AS "Hash Integer"
,HASHROW (c4_bigint) AS "Hash BigInt"
,HASHROW (c5_dec90) AS "Hash Decimal90"
,HASHROW (c6_dec92) AS "Hash Decimal92"
,HASHROW (c7_number) AS "Hash Number"
,HASHROW (c8_num90) AS "Hash Number90"
,HASHROW (c9_num92) AS "Hash Number92"
,HASHROW (c10_float) AS "Hash Float"
,HASHROW (c11_char) AS "Hash Character"
FROM tableA;
```

```
INSERT INTO tableA
VALUES (5,5,5,5,5,5,5,5,5,5,5,5);
```

```
SELECT
Output: Hash Byteint 60-9D-17-15
Hash Smallint 60-9D-17-15
Hash Integer 60-9D-17-15
Hash BigInt 60-9D-17-15
Hash Decimal90 60-9D-17-15
Hash Decimal92 BD-81-04-59
Hash Number 6D-6F-6D-8C
Hash Number90 6D-6F-6D-8C
Hash Number92 6D-6F-6D-8C
Hash Float E4-0F-E3-60
Hash Character 55-1D-CF-DC
```

Multi-Column Hashing

The hashing algorithm uses multiplication and addition as commutative operators for handling a multi-column index.

In a few cases where the same values occur in different numeric columns of a primary index, it is possible that the data rows may not distribute as evenly as expected.

If the data types hash the same, a multi-column index will hash the same for the same values in different columns. Assume there happens to be a storeid of 1001 and another storeid of 748102. Just as a coincidence, there happens to be items with ids of 1001 and 748102 as well.

```
INSERT INTO Sales (1001, 748102);
INSERT INTO Sales (748102, 1001);
```

```
SELECT  storeid AS c1
        ,itemid AS c2
        ,HASHROW (storeid) AS "storeid hash"
        ,HASHROW (itemid) AS "itemid hash"
        ,HASHROW (storeid, itemid) AS "combined hash"
FROM    Sales;
```

<u>storeid</u>	<u>itemid</u>	<u>storeid hash</u>	<u>itemid hash</u>	<u>combined hash</u>
1001	748102	F4-B8-A6-EA	EA-14-76-85	40-AC-61-7A
748102	1001	EA-14-76-85	F4-B8-A6-EA	40-AC-61-7A

If this caused a skewing issue, one option is to use a different data type for one of the columns as shown below.

If the data types hash differently, then a multi-column index will hash differently for the same values in different columns as shown below.

```
CREATE TABLE Sales2
  (storeid          INTEGER
   ,itemid          DECIMAL(18,1)
   , ...)
UNIQUE PRIMARY INDEX (storeid, itemid);
```

```
INSERT INTO Sales2 (1001, 748102);
INSERT INTO Sales2 (748102, 1001);
```

```
SELECT  storeid AS c1
        ,itemid AS c2
        ,HASHROW (storeid) AS "storeid hash"
        ,HASHROW (itemid) AS "itemid hash"
        ,HASHROW (storeid, itemid) AS "combined hash"
FROM    Sales2;
```

<u>storeid</u>	<u>itemid</u>	<u>storeid hash</u>	<u>itemid hash</u>	<u>combined hash</u>
1001	748,102.0	F4-B8-A6-EA	24-DC-A1-3B	EC-31-F0-83
748102	1,001.0	EA-14-76-85	8F-4E-85-2D	4B-FF-A4-11

Multi-Column Hashing

- The Hashing Algorithm uses multiplication and addition to create the hash value for a multi-column index. Assume $PI = (A, B)$:

$$[Hash(A) * Hash(B)] + [Hash(A) + Hash(B)] = [Hash(B) * Hash(A)] + [Hash(B) + Hash(A)]$$

- In order for PI equality access to occur on a single AMP, all of the columns in the PI need to have values.

```
CREATE TABLE Sales
(storeid  INTEGER NOT NULL
,itemid  BIGINT NOT NULL
:
)
PRIMARY INDEX (storeid, itemid);
```

```
INSERT INTO Sales
VALUES (1001, 748102, ...);
```

```
SELECT storeid AS c1
,itemid AS c2
,HASHROW (storeid) AS "storeid hash"
,HASHROW (itemid) AS "itemid hash"
,HASHROW (storeid, itemid) AS "combined hash"
FROM Sales;
```

<u>storeid</u>	<u>itemid</u>	<u>storeid hash</u>	<u>itemid hash</u>	<u>combined hash</u>
1001	748102	F4-B8-A6-EA	EA-14-76-85	40-AC-61-7A

Additional Hash Examples

A numeric value of 0 hashes the same as a NULL. A character data type with a value of all spaces also hashes the same as a NULL. However, a character value of '0' hashes to a value different than the hash of a NULL.

Upper and lower case characters hash the same.

The following example shows that different numeric types with a value of 0 all hash to the same hash value.

```
CREATE TABLE tableA
(c1_bint          BYTEINT,
 c2_sint          SMALLINT,
 c3_int           INTEGER,
 c4_dec           DECIMAL(8,0),
 c5_dec2          DECIMAL(8,2),
 c6_float         FLOAT,
 c7_char          CHAR(10))
UNIQUE PRIMARY INDEX (c1_bint, c2_sint);

.SET FOLDLINE
.SET SIDETITLES

INSERT INTO tableA (0,0,0,0,0,0,0,'0');

SELECT
  HASHROW (c1_bint)      AS "Hash Byteint"
, HASHROW (c2_sint)      AS "Hash Smallint"
, HASHROW (c3_int)       AS "Hash Integer"
, HASHROW (c4_dec)       AS "Hash Dec0"
, HASHROW (c5_dec2)      AS "Hash Dec2"
, HASHROW (c6_float)     AS "Hash Float"
, HASHROW (c7_char)      AS "Hash Char"
FROM   tableA;

Hash Byteint      00000000
Hash Smallint     00000000
Hash Integer      00000000
Hash Dec0         00000000
Hash Dec2         00000000
Hash Float        00000000
Hash Char         2BB7F6D9
```

Note: An INTEGER value of 500 and a DECIMAL (8, 2) value of 5.00 will both have the same hash value.

Additional Hash Examples

- A NULL value for numeric data types is treated as 0.
- Upper and lower case characters hash the same.

Example 1:

```
CREATE TABLE tableD
(c1_int      INTEGER
 ,c2_int      INTEGER
 ,c3_char     CHAR(4)
 ,c4_char     CHAR(4))
UNIQUE PRIMARY INDEX (c1_int, c2_int);
INSERT INTO tableD (0, NULL, 'EDUC', 'Educ');
SELECT  HASHROW (c1_int) AS "Hash c1"
        ,HASHROW (c2_int) AS "Hash c2"
        ,HASHROW (c3_char) AS "Hash c3"
        ,HASHROW (c4_char) AS "Hash c4"
FROM    tableD;
```

Hash c1	Hash c2	Hash c3	Hash c4
00-00-00-00	00-00-00-00	6E-D6-79-D5	6E-D6-79-D5
Hash of 0	Hash of NULL	Hash of 'EDUC'	Hash of 'Educ'

Example 2: To determine the number of AMPs in a system.

```
SELECT HASHAMP() + 1;
```

Returns a value of 16 for this education system.

Using Hash Functions to View Distribution

The Hash Functions can be used to view the distribution of rows for a chosen Primary Index.

Notes:

- HashRow – returns the row hash value for a given value(s)
- HashBucket – the grouping for a specific hash value
- HashAMP – the AMP that is associated with the hash bucket
- HashBakAMP – the fallback AMP that is associated with the hash bucket

Identifying the Hash Buckets

If you suspect data skewing due to hash synonyms or NUPI duplicates, you can use the HashBucket function to identify the number of rows in each hash bucket. The HashBucket function requires the HashRow of the columns that make up the Primary Index or the columns being considered for a Primary Index.

Identifying the Primary AMPs

The HASHAMP function can be used to determine data skewing and which AMP(s) have the most rows.

The Customer table on the facing page consists of 7017 rows.

Using Hash Functions to View Distribution

Hash Functions can be used to calculate the impact of NUPI duplicates and synonyms for a PI.

```
SELECT  HASHROW (LastName,FirstName)
        AS "Hash Value"
        ,COUNT(*)
FROM    Customer
GROUP BY 1
ORDER BY 2 DESC;
```

Hash Value	Count(*)
2D-79-75-A8	12
14-84-0B-D7	7
(Output cut due to length)	
E7-A4-D9-10	1
AA-D4-DC-80	1

The largest number of NUPI duplicates or synonyms is 12.

```
SELECT  HASHAMP (HASHBUCKET
                 (HASHROW (LastName,FirstName)))
        AS "AMP #"
        ,COUNT(*)
FROM    Customer
GROUP BY 1
ORDER BY 2 DESC;
```

AMP #	Count(*)
7	929
6	916
4	899
5	891
2	864
3	864
1	833
0	821

AMP #7 has the largest number of rows.

Primary Index Hash Mapping

The diagram on the facing page gives you an overview of **Primary Index Hash Mapping**, the process by which all data is distributed in the Teradata DBS.

The Primary Index value is fed into the Hashing Algorithm, which produces the Row Hash. The row goes onto the Message Passing Layer. The **Hash Maps** in combination with the Row Hash determines which AMP gets the row. The Hash Maps are part of the Message Passing Layer interface.

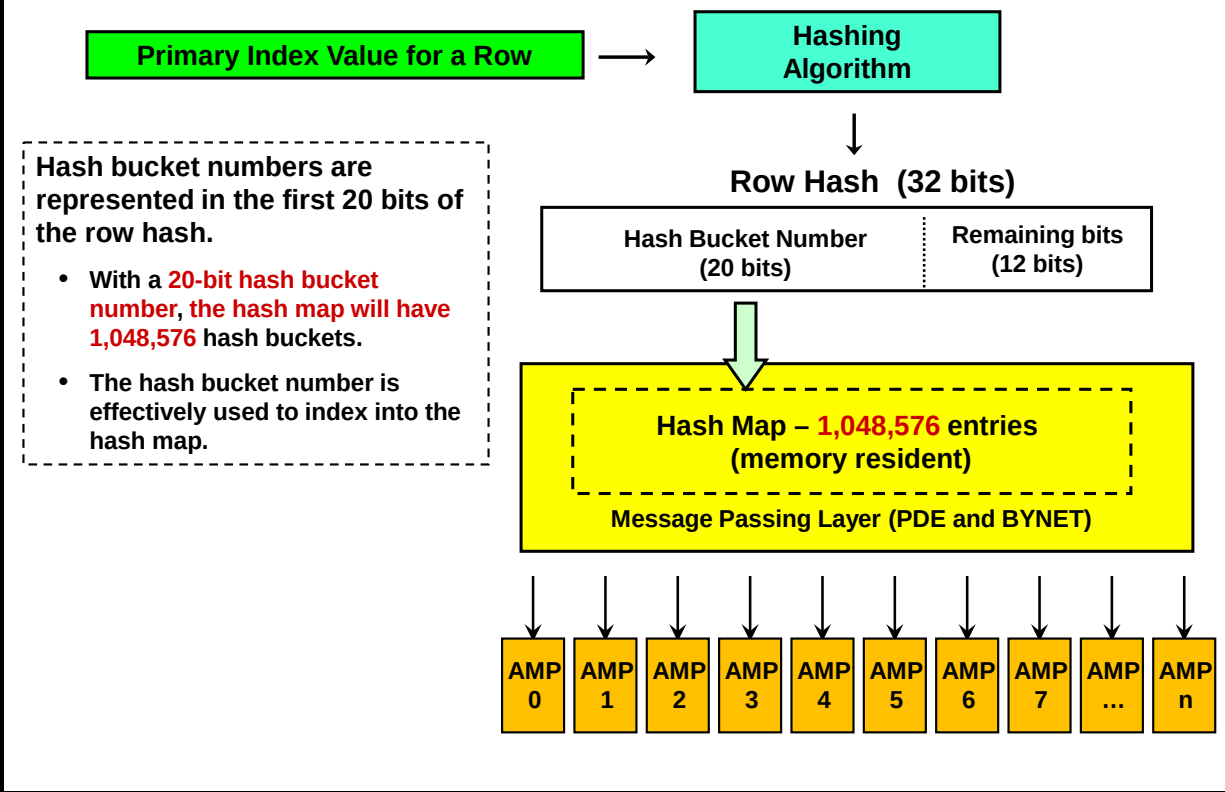
Starting with Teradata Database 12.0, Teradata supports either 65,536 or 1,048,576 hash buckets for a system. The larger number of buckets primarily benefits systems with thousands of AMPs, but there is no disadvantage to using the larger number of buckets on smaller systems.

The hash map is an array indexed by hash bucket number. Each entry of the array contains the number of the AMP that processes the rows in the corresponding hash bucket.

The RowHash is a 32-bit result obtained by applying the hash function to the primary index of the row. On systems with:

- Older systems may have 65,536 hash buckets; the system uses 16 bits of the 32-bit RowHash to index into the hash map.
- Most current systems have 1,048,576 hash buckets; the system uses 20 bits of the 32-bit RowHash as the index.

Primary Index Hash Mapping



Hash Maps

As you have seen, Hash Maps are the mechanisms that determine which AMP gets a row. They are duplicated on every TPA node in the system. There are 4 Hash Maps for PI-based tables:

- Current Configuration Primary (designates where rows are stored)
- Current Configuration Fallback (designates where copies of rows are stored)
- Reconfiguration Primary (designates where rows move during a system reconfiguration)
- Reconfiguration Fallback (designates where copies of rows move during a reconfiguration)

Hash Maps are also used whenever there is a PI or USI operation.

Hash maps are arrays of Hash Map entries. There are 1,048,576 Hash Map entries. Each of these entries points to a single AMP in the system. The Row Hash generated by the Hashing Algorithm contains information that designates a particular entry on a particular Hash Map. This entry tells the system which AMP should be interrupted.

- Teradata Version 1 used a Hash Map with only 3643 hash buckets.
- Teradata Version 2 (prior to Teradata 12.0) used hash maps with 65,536 hash buckets. Starting with Teradata 12.0, the number of hash buckets in a hash map can be either 65,536 or 1,048,576. One of the important impacts of this change was that this increase provides for a more even distribution of data with large numbers of AMPs.

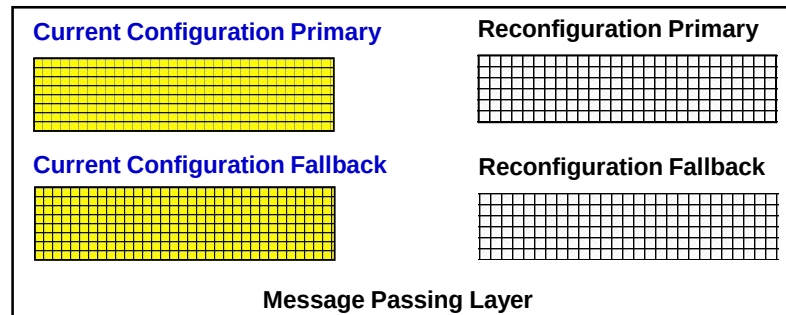
For systems upgraded to Teradata Database 12.0, the default number of hash buckets remains unchanged at 65,536 buckets. For new systems or following a sysinit, the default is 1,048,676 buckets which is what we will assume in this course.

Note: The Hash Maps are stored in GDO (Globally Distributed Object) files on each SMP and are loaded into the PDE memory space when PDE software is started – usually as part of the Linux startup process.

Hash Maps

Hash Maps are the mechanism for determining which AMP gets a row.

- There are multiple Hash Maps on every TPA node that are associated with PI-based tables.
 - A different or separate hash map is used for NoPI tables.
- By default, the two Current Hash Maps for PI-based tables are loaded into PDE memory space of each TPA node when PDE software boots.



Hash Maps have 1,048,576 entries. Each entry is 2 bytes in size.

Primary Hash Map

The diagram on the facing page is a graphical representation of a **Primary Hash Map**. (It serves to illustrate the concept; they really don't look like this.) The Hash Map utilized by the system is the Current Configuration Primary Hash Map. The Fallback Hash Map IS NOT an exact copy of the Primary Hash Map. The Primary Hash Map identifies which AMP the first (Primary) copy of a row belongs to. The Fallback Hash Map is only used for Fallback protected tables and identifies a different AMP in the same "cluster" for the second (Fallback) row copy.

Note: On most systems (i.e., systems since the 5450), clusters typically consist of 2 AMPs.

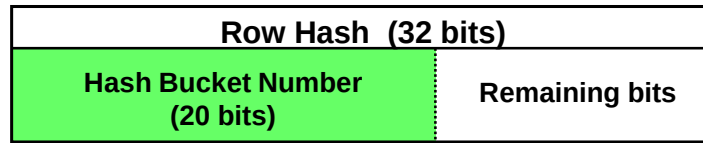
That portion of the Row Hash that points to a particular Hash Map entry is called the Hash Bucket Number (HBN). The hash bucket number is the first 20 bits of the Row Hash depending on the size of the hash maps. The hash bucket number points to a single entry in a Hash Map. As the diagram shows, the system looks at the particular Hash Map entry specified by the hash bucket number to determine which AMP the row belongs to.

The Message Passing Layer (or Communications Layer) uses only the hash bucket number portion of the Row Hash to determine which AMP gets the row when inserting a new row into a table. The AMP uses the entire 32 bit Row Hash to determine logical disk storage location of the row.

Teradata builds Hash Maps in a consistent fashion. The Primary Hash Map of systems with the same number of AMP vprocs is identical assuming the same number of buckets in the hash map (65,536 or 1,048,576 hash buckets). With older systems, the Fallback Hash Maps may differ due to clustering differences at each site.

The hash bucket number (prior to Teradata 12.0) was commonly referred to as the **Destination Selection Word (DSW)**.

Primary Hash Map



PRIMARY HASH MAP – 14 AMP System																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	13	12	13	12	13	11	12	10	13	10	11	12	11	12	13	09
0001	13	07	08	10	08	08	11	11	09	09	10	12	09	09	10	13
0002	10	10	13	05	11	11	12	12	11	11	06	12	13	04	12	12
0003	07	06	13	03	06	08	13	02	13	13	01	00	07	08	05	07
0004	04	04	05	07	09	06	09	07	03	02	03	08	01	00	02	06
0005	01	00	05	04	08	10	10	05	08	08	06	09	07	06	05	11

Note:

This partial hash map
(1,048,576 buckets) is
associated with a 14
AMP System.

- Assume the Hash Bucket Number is the first 20 bits of the Row Hash.
- The Hash Bucket Number points to one entry within the map.
- The referenced Hash Map entry identifies the AMP for the row hash.

Hash Maps for Different Systems

The diagrams on the facing page show a graphical representation of a Primary Hash Map for an 8 AMP system and a Primary Hash Map for a 16 AMP system. These examples assume hash maps with 1,048,576 entries.

A data value which hashes to “00023 1AB” will be directed to different AMPs on different systems. For example, this hash value will be associated with AMP 5 on an 8 AMP system and AMP 14 on a 16 AMP system.

Hash Maps for Different Systems

Row Hash (32 bits)	
Hash Bucket Number	Remaining bits

PRIMARY HASH MAP – 8 AMP System

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	07	06	07	06	07	04	05	06	05	05	06	06	07	07	03	04
0001	07	07	02	04	01	00	05	04	03	02	03	05	01	00	02	06
0002	01	00	05	05	03	02	04	03	01	00	06	02	04	04	01	00
0003	07	06	03	03	06	06	02	02	01	00	01	00	07	07	05	07
0004	04	04	05	07	05	06	07	07	03	02	03	04	01	00	02	06
0005	01	00	05	04	03	02	06	05	01	00	06	05	07	06	05	07

PRIMARY HASH MAP – 16 AMP System

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	15	14	15	15	13	14	12	14	13	15	15	12	11	12	13	14
0001	13	14	14	10	15	08	11	11	15	09	10	12	09	09	10	13
0002	10	10	13	14	11	11	12	12	11	11	14	12	13	14	12	12
0003	15	15	13	14	06	08	13	14	13	13	14	14	07	08	15	07
0004	15	04	05	07	09	06	09	07	15	15	03	08	15	15	02	06
0005	01	00	05	04	08	10	10	05	08	08	06	09	07	06	05	11

Assume row hash of
00023 1AB

8 AMP system – AMP 05
16 AMP system – AMP 14

Portions of actual hash maps
with 1,048,576 hash buckets.

Fallback Hash Map

The diagram on the facing page is a graphical representation of a Primary Hash Map and a **Fallback Hash Map**.

The Fallback Hash Map is only used for Fallback protected tables and identifies a different AMP in the same “cluster” for the second (Fallback) row copy.

Note: These are the actual partial primary and fallback hash maps for a 14 AMP system with 1,048,576 hash buckets.

Fallback Hash Map

Row Hash (32 bits)	
Hash Bucket Number	Remaining bits

Assume row hash of
00023 1AB

Primary AMP – 05
Fallback AMP – 12

Notes:

14 AMP System with 2 AMP
clusters; hash maps with
1,048,576 buckets.

PRIMARY HASH MAP – 14 AMP System

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	13	12	13	12	13	11	12	10	13	10	11	12	11	12	13	09
0001	13	07	08	10	08	08	11	11	09	09	10	12	09	09	10	13
0002	10	10	13	05	11	11	12	12	11	11	06	12	13	04	12	12
0003	07	06	13	03	06	08	13	02	13	13	01	00	07	08	05	07
0004	04	04	05	07	09	06	09	07	03	02	03	08	01	00	02	06
0005	01	00	05	04	08	10	10	05	08	08	06	09	07	06	05	11

FALLBACK HASH MAP – 14 AMP System

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	06	05	06	05	06	04	05	03	06	03	04	05	04	05	06	02
0001	06	00	01	03	01	01	04	04	02	02	03	05	02	02	03	06
0002	03	03	06	12	04	04	05	05	04	04	13	05	06	11	05	05
0003	00	13	06	10	13	01	06	09	06	06	08	07	00	01	12	00
0004	11	11	12	00	02	13	02	00	10	09	10	01	08	07	09	13
0005	08	07	12	11	01	03	03	12	01	01	13	02	00	13	12	04

Reconfiguration

Reconfiguration (Reconfig) is the process for changing the number of AMPs in a system and is controlled by the Reconfiguration Hash Maps. The system constructs Reconfiguration Hash Maps by reassigning Hash Map Entries to reflect a new configuration of AMPs. This is done in a way that minimizes the number of rows (and Hash Map Entries) reassigned to a new AMP. After rows are moved, the Reconfiguration Primary Hash Map becomes the Current Configuration Primary Hash Map, and the Reconfiguration Fallback Hash Map becomes the Current Fallback Hash Map.

The diagram on the right illustrates a 200 AMP to 300 AMP Reconfig for a system. The 1,048,576 Hash Map entries are distributed evenly across the 200 AMPs in the initial configuration (top illustration), with approximately 5243 entries referencing each AMP. Thus, there are 5243 Hash Map Entries pointing to AMP 1.

In a 300 AMP system, each AMP will have approximately 3496 referencing the AMP. It is necessary to change 1748 (5243 - 3496) of those and divide them between the new AMPs (AMP 200 through 299). The system does the same thing for the Hash Map Entries that currently point to the other AMPs. This constitutes the Reconfiguration Primary Hash Map. A similar process is done for the Reconfiguration Fallback Hash Map.

Once the new Hash Maps are ready, the system looks at every row on each AMP and checks to see if the Hash Bucket Number points to one of the Hash Map Entries which was changed. If so, then the row is moved to its new destination AMP.

The formula used to determine the percentage of rows migrating to new AMPs during a Reconfig is shown at the bottom of the facing page. Divide the Number of New AMPs by the Sum of the Old and New AMPs (the number of AMPs after the Reconfig). For example, the above 200 to 300 AMP Reconfig causes 33.3% of the rows to migrate.

The following example illustrates how the Hash Map built during a reconfiguration.

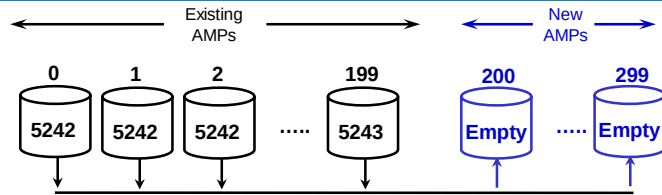
AMPS

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
2	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0
3	2	2	1	0	1	0	2	2	1	0	1	0	2	2	1	0	1	0	2	2	1	0	1	0	2	2	1	0	2	2
4	3	2	3	3	1	0	2	2	1	0	1	0	3	2	3	3	1	0	2	2	1	0	1	0	3	2	3	3	1	0
5	4	4	3	3	4	4	2	2	1	0	1	0	3	2	3	4	1	0	2	4	1	0	4	4	3	2	3	3	1	0
6	5	4	5	3	4	4	5	2	5	5	1	0	3	2	3	4	1	0	2	4	1	0	5	4	3	2	3	5	1	0
7	6	6	5	6	4	4	5	6	5	5	6	6	3	2	3	4	1	0	2	4	1	0	5	4	3	2	3	5	1	0
8	7	6	7	6	7	4	5	6	5	5	6	6	7	7	3	4	7	7	2	4	1	0	5	4	3	2	3	5	1	0

Note: If the hash map size is changed (65,536 to 1,048,576), more data will be moved as part of a reconfiguration

Reconfiguration

If a system (with 1,048,576 Hash buckets) has 200 AMPs, then each of the 200 AMPs will have approximately 5242 entries in the hash map.



If upgrading to 300 AMPs, then each of the 300 AMPs will have a similar number of entries (approx. 3495) in the hash map.



- The system creates new Hash Maps to accommodate the new configuration.
- Old and new maps are compared – each AMP reads its rows, and moves only those that hash to a new AMP.

$$\text{Percentage of Rows Moved to new AMPs} = \frac{\text{Number of New AMPs}}{\text{SUM of Old + New AMPs}} = \frac{100}{300} = \frac{1}{3} = 33.3\%$$

- It is not necessary to offload and reload data due to a reconfiguration.

MAPS TD 16.10 Reconfiguration

Teradata has had Multiple Hash Maps as we saw earlier, but with the TD 16.10 MAPS feature, it has been taken to a new level. Now every table is effectively assigned to a Map. MAPS allows for skipping the table redistribution portion of the reconfiguration process, leaving tables on the old AMPs. Basically, decoupling the movement of data from the hardware expansion shortens the planned downtime to just the time required to wire in the new nodes and update a few system files (most likely less than 30 minutes). Tables can then be redistributed across the new nodes and their AMPs at the convenience of the administrator, and while the system is online. By doing so, data is accessible more quickly and there is less disruption to the business following a hardware expansion.

The ALTER TABLE command has been enhanced for use with maps to move a table to another map using an INSERT-SELECT like operation where a read lock is placed on the source table while rows are being inserted then updated very briefly to an exclusive lock at the end. The TDMaps.MoveTablesSP procedure that is used by the Viewpoint portlet to perform table moves is basically a packaging of the enhanced ALTER TABLE.

Supporting more than one map in the system solves a second problem of a more permanent nature namely Small Table Efficiency. Consider a table with just 100 rows on a 1000-AMP system. To read all the rows from the table requires tasks to be started up and at least one I/O to be performed on all 1000 AMPs. For 900 of the 1000 AMPs, this is a wasted effort. As the number of AMPs in a typical Teradata system is expanded, the system-wide resources used to access rows from tables with only a few rows increases. MAPS automatically creates sparse maps composed of 1 or a few AMPs just for cases such as this. Using this option, known as “sparse maps,” you can put that 100-row Department table, or that 5-row Status Code table onto a single AMP. Doing a full-table scan of the table becomes a single-AMP operation, which can reduce the effort in accessing the table and also frees up system resources, such as AMP worker tasks I/O, and CPU, for other work.

The MAPS feature provides two types of maps: **Contiguous** and **Sparse**.

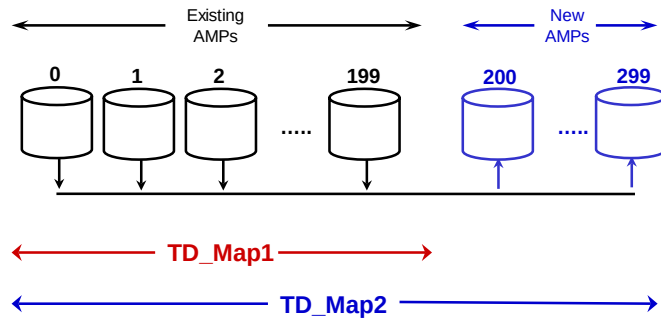
Contiguous maps are maps that typically represent all the AMPs in a configuration – either currently, or at some point in time in the past. Contiguous maps use every AMP within a range of AMPs, usually starting with AMP 0, and use hash buckets to assign AMPs the same way as the pre-MAPS behavior described in the previously. For example, a contiguous map between AMPs 0 and 199 uses AMP 0, 1, 2...199. There are no gaps in the sequence of AMPs contained in a contiguous map. Contiguous maps follow a defined naming convention. When MAPS initially becomes available on a system, the first contiguous map, named **TD_Map1**, is automatically created, either by Sysinit or during the upgrade process. The next (second) contiguous map created at the time of the next expansion will be named **TD_Map2**. The one after that is **TD_Map3**, and so forth. You have no control over how these maps are named or when they are created.

A **sparse map**, on the other hand, typically uses just a single or a few AMPs, selected by the system from the AMPs of a parent contiguous map. For example, a one-AMP sparse map between AMPs 0 and 299 uses just one of the AMPs in that range, such as AMP 155. Note: The AMP # is assigned when a table is created using sparse map. Sparse maps are special-purpose, optional maps that are not a direct result of a system expansion.

MAPS TD 16.10 Reconfiguration

With Teradata MAPS (a.k.a., MHM or Multiple Hash Maps), all tables are assigned to a Map. Tables are moved to another map via ALTER TABLE command.

Reconfig processing can be fast (< 30 minutes) because tables can be moved later while system is back on-line.



- The Config/Reconfig process creates new Hash Maps called **Contiguous** Maps. Contiguous maps use every AMP between a range of AMPs.
 - Original system is **TD_Map1 contains AMPs 0 – 199.**
 - After 1st expansion, **TD_Map2 contains AMPs 0 – 299.**
 - Next expansions becomes TD_Map3..., TD_Map4, etc.
- This process (DIPMAPS) also creates two **Sparse** maps – a 1 AMP and #AMPs = #Nodes. (i.e., TD_1AmpSparseMap_4Nodes and TD_4AmpSparseMap_4Nodes)
 - A Sparse map is a subset of AMPs between a range of AMPs used for small tables.
- **Viewpoint: Teradata Maps Manager Portlet**
 - Viewpoint calls the stored procedures in user TDMaps to simplify moving tables.

MAPS TD 16.10 Sparse Maps

As part of the TD 16.10 MAPS feature there is a new type of Map called Sparse Maps designed for Small table efficiency.

A contiguous map covers a range of AMPs, usually across some number of nodes in the system. A sparse map is the opposite; it covers just a single AMP, or under some conditions a small number of AMPs. A sparse map does not exist in isolation within its own dedicated AMP or AMPs, but rather it is defined within a parent contiguous map. The rows of a table targeted to a sparse map coexist with the rows of tables that belong to the parent contiguous map.

A sparse map is intended as a destination for the rows of “small tables”. A small table that would be suitable for a single-AMP sparse map is one that contains 128KB or less, after removing table header overhead. If the table is being considered for a multi-AMP sparse map, the upper limit of suitability is the number of AMPs within the multi-AMP sparse map times 128KB.

If you have two or more small tables that are joined together using their primary index columns, and they are both intended to be placed in a single-AMP or multi-AMP sparse map, locating those tables on the same AMP or AMPs is desirable. Not doing so inhibits the optimizer from performing a direct join between the tables without having to do row redistribution. Specifying the same colocation name when you create or move each of the related small tables to the sparse map is a mechanism for doing that.

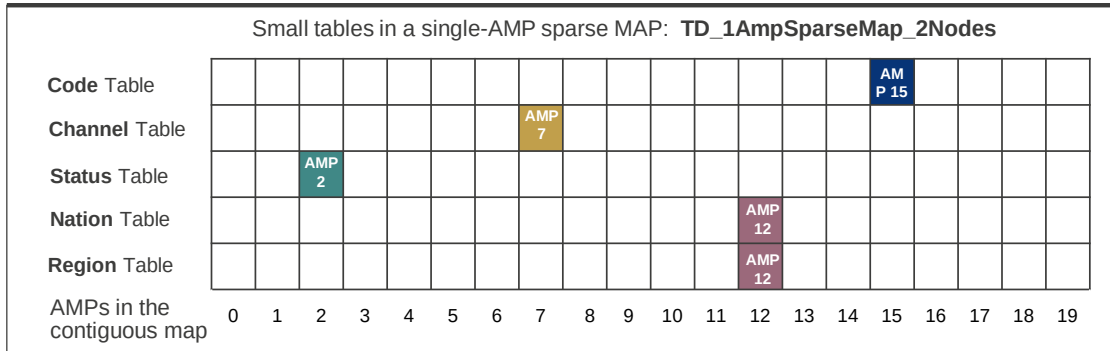
Colocation plays a role even if only one table is moved to a sparse map. The colocation name assigned to a small table either when it is created or when it is moved determines which AMP(s) that the table’s rows will reside on.

The user may or may not specify a colocation name at the time the table is created or moved. If one is not specified, the system constructs one from the table’s database name, underscore, and the table’s name. If you issue a SHOW TABLE command, you can see both the sparse map name and the colocation name associated with the table.

In our example, on the facing page, the same colocation name **Co_NationRegion** is put through the hashing algorithm and the resulting hash bucket is used to identify which AMP(s) will house the small table. As you can see the same AMP 12 was chosen for the Nation and Region tables. This process that selects an AMP or AMPs only considers AMPs that make up the parent contiguous map.

Note: If you have small tables that require colocation, then it is recommended that you come up with a meaningful colocation name to distinctly represent those tables. If you do not have a need for co-locating multiple tables, let the system generate the colocation name for you and simply ignore the colocation naming option.

MAPS TD 16.10 Sparse Maps



- Used for small tables (some AMPs don't have any rows)
- **COLOCATION** determines which AMPs by hashing name using parent contiguous map
- AMPs not chosen until CREATE TABLE:

```
CREATE TABLE Nation,
MAP=TD_1AMPsparseMap_2Nodes
COLOCATE USING Co_NationRegion
(
  NationKey INTEGER,
  Name VARCHAR(10),
  RegionKey INTEGER
)
UNIQUE PRIMARY INDEX (NationKey);
```

```
CREATE TABLE Region,
MAP=TD_1AMPsparseMap_2Nodes
COLOCATE USING Co_NationRegion
(
  RegionKey INTEGER,
  Name VARCHAR(10)
)
UNIQUE PRIMARY INDEX (RegionKey);
```

Row Retrieval via PI Value – Overview

The facing page illustrates the step-by-step process involved in Primary Index retrieval. The SELECT statement (shown on facing page) retrieves the row or rows where the PI is equal to a particular column value (or column values in the case of a multi-column PI).

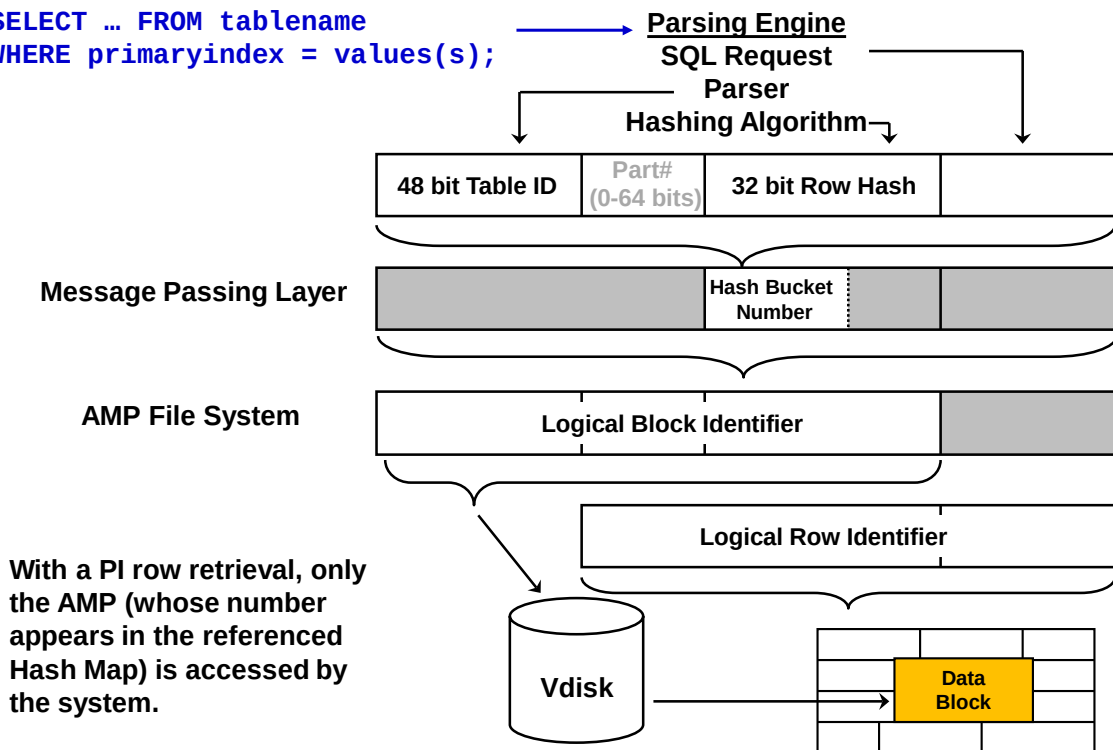
The PE parser always puts out a multi-part message composed of the Table ID, Partition number (which is 0 for a non-partitioned table), Row Hash and Primary Index value. The 48 bit Table ID is looked up in the Data Dictionary, the 32 bit Row Hash value is generated by the Hashing Algorithm and the Primary Index value comes from the SQL request.

The Message Passing Layer (a.k.a., Communications Layer) Interface uses the Hash Bucket Number (usually 20 bits of the Row Hash) to determine which AMP to interrupt and pass on the message. We will assume in this class that the Hash Bucket Number is usually 20 bits long (older systems used a 16-bit hash bucket number).

The AMP uses the Table ID and Row Hash to identify and locate the proper data block, then uses the Row Hash and PI value to locate the specific row(s). The PI value is required to distinguish between Hash Synonyms.

Row Retrieval via PI Value – Overview

`SELECT ... FROM tablename
WHERE primaryindex = values(s);`



Names and Object IDs

DBC.Next is a Data Dictionary table that consists of a single row with 11 columns (in Teradata 16.00) as shown below.

The effective result is that this table has columns used to assign unique IDs to objects in the system.

When a new user, database, role, or profile is created, the newly created object is assigned the current numeric value in the DatabaseID column. The DatabaseID value in the DBC.Next is then incremented by 1 and the row is saved to disk. This becomes the next assigned DatabaseID for the next user, database, role, or profile that is created.

When a new table, view, macro, etc. is created, the newly created object is assigned the current numeric value in the TableID column. The TableID value in the DBC.Next is then incremented by 1 and the row saved to disk. This becomes the next assigned TableID for the next table, view, macro, etc. that is created.

The TableID column is effectively used to assign a globally unique numeric ID to every Table, View, Macro, Trigger, Stored Procedure, User-Defined Function, Join Index, and Hash Index in a system.

DBC.Next always contains the next value to be assigned to any of these objects. Think of these columns as incremental counters for ID values.

You may be interested in noting that DBC.Next only contains a single, short row but it requires a Table Header on every AMP, as does any table.

Columns and Indexes are also assigned numeric IDs, which are unique within their respective tables. However, column and index IDs are not assigned from DBC.Next.

<u>DBC.Next columns</u>	<u>Values</u>	<u>Data Type</u>
RowNum	1	CHAR (1)
DatabaseID	numeric	BYTE (4)
TableID	numeric	BYTE (4)
ProcsRowLock	numeric	BYTE (4)
EventNum	numeric	BYTE (4)
LogonSequenceNo	numeric	BYTE (4)
TempTableID	numeric	BYTE (4)
StatsQueryID	numeric	BYTE (4)
ReconfigID	number	INTEGER
MapNo (TD 16.00)	numeric	BYTE (4)
ExtraField1 (TD 16.00)	numeric	FLOAT

Names and Object IDs

DBC.Next (1 row)

NEXT DATABASE ID	NEXT TVM ID	6 Other Counters	
---------------------	----------------	------------------	--

- Each **Database/User/Profile/Role** – is assigned a globally unique numeric ID.
- Each **Table, View, Macro, Trigger, Stored Procedure, User-defined Function, Join Index, and Hash Index** – is assigned a globally unique numeric ID.
- Each Column – is assigned a numeric ID unique within its Table ID.
- Each Index – is assigned a numeric ID unique within its Table ID.
- The DD keeps track of all SQL names and their numeric IDs.
- The PE's RESOLVER uses the DD to verify names and convert them to IDs.
- The AMPs use the numeric IDs supplied by the RESOLVER.

Table ID

The **Table ID** is the first part of the three-part message. It is a 48-bit number supplied by the parser. There are two major components of the Table ID:

- The first component of the Table ID is the **Unique Value**. Every table, view and macro is assigned a 32-bit Unique Value, which is assigned by the system table called DBC.Next. In addition to specifying a particular table, this value also indicates whether the table is a normal data table, Permanent Journal table or Spool file table.
- The second component of the Table ID is known as the **Subtable ID**. Teradata stores various types of rows of a table in separate blocks. For example, Table Header rows (described later) are stored in different blocks than primary data rows, which are stored in different blocks than Fallback data rows, and so on (more examples are shown on the facing page). Each separate set of blocks is known as a subtable. The Subtable ID is a 16-bit value that tells the file system which type of blocks to search for.
- The facing page lists subtable IDs in decimal value for 2-AMP clusters. The SHOWBLOCKS utility will display the block allocations by subtable and uses decimal values to represent each subtable. If a Reference Index subtable was created, it would have subtable IDs of 1536 and 2560.

For convenience, Table ID examples throughout this course only refer to the Unique Value and omit the Subtable ID.

The Table ID, together with the Row ID, gives Teradata a way to uniquely identify every single row in the entire system.

Spool File Table IDs

Spool files are temporary work tables which are created and dropped as queries are executed. When a query is complete, all of the spool files that it used will be dropped automatically.

Like all tables, a spool file (essentially a temporary work table) requires a Table ID (or tableid). There is a range of tableids exclusively reserved for spool files (C000 0001 through FFFF FFFF) and the system cycles through them. Eventually, the system will cycle through all the tableids for spool files and reassign spool tableids starting at C000 0001.

Table ID

The Table ID is a Unique Value for Tables, Views, Macros, Triggers, Stored Procedures, Join Indexes, etc. that comes from [DBC.Next](#) dictionary table.

Unique Value also defines the type of table:

- Normal data table
- Permanent journal
- Global Temporary
- Spool file

UNIQUE VALUE

32 Bits

+

SUB-TABLE ID

16 Bits

[Sub-table ID](#) identifies the part of a table the system is looking at.

<u>Sub-table type</u>	<u>Primary ID</u>	<u>Fallback ID</u> (shown in decimal format)
Table Header	0	
Data table	1024	2048
1 st Secondary index	1028	2052
2 nd Secondary index	1032	2056
1 st Reference index	1536	2560
1 st BLOB or CLOB	1792	2816
2 nd BLOB or CLOB	1794	2818
Archive Online Subtable	18440	n/a

Table ID plus Row ID makes every row in the system unique.

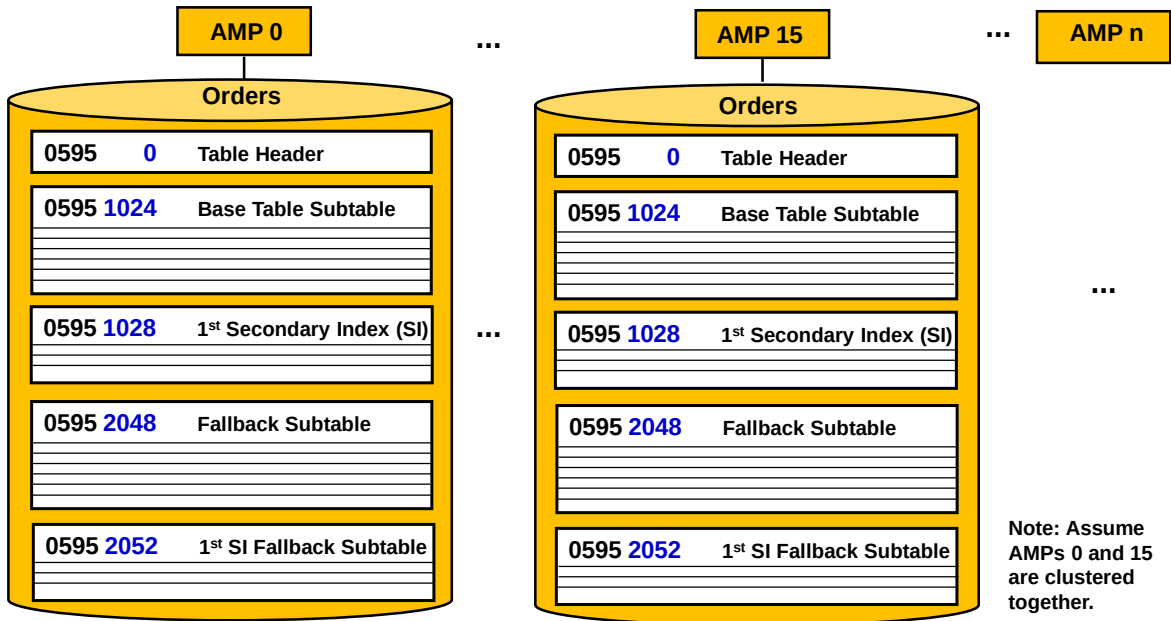
Examples shown in this manual use the Unique Value to represent the entire Table ID.

Table ID Example

Facing page illustrates the Table IDs that will be created for a table. Assume that AMPs 0 and 15 are clustered together.

Table ID Example

Assume the value in DBC.Next is 0595 (for the next TVM ID to create). An Orders table is created with just a primary index and then populated with data. A secondary index is then created and Fallback protection is added at a later time.



Row ID

The Row Hash is not sufficient to identify a specific row in a table. Since it is based on a Primary Index value, multiple rows can have the same Row Hash. This is due either to Hash Synonyms or NUPI Duplicates.

The **Row ID** makes every row within a table uniquely identifiable. For a non-partitioned table, the Row ID consists of the Row Hash plus a **Uniqueness Value**. The Uniqueness Value is a 32-bit numeric value, designed to identify specific rows within a single Row Hash value. When there are multiple rows with the same Row Hash within a table, the first row is assigned a Uniqueness Value of 1. Additional rows with the same Row Hash are assigned ascending Uniqueness Values.

For Primary Index retrievals, only the Row Hash and Primary Index values are needed to find the qualifying row(s). The Uniqueness Value is needed for Secondary Index support. Since a Row ID is a unique identifier of a row within a table, Teradata uses Row IDs as Secondary Index pointers.

Although Row IDs do identify every row in a table uniquely, they do not guarantee that the data itself is unique. In order to avoid the problem of duplicate rows (permitted in Multiset tables), the complete set of data values for a row (in a Set table) must also be unique.

Summary

- For a non-partitioned table (NPPI), the Row ID consists of the Row Hash + Uniqueness Value for a total of 8 bytes in length.
- For a partitioned table (PPI), the Row ID actually consists of the Partition Number + Row Hash + Uniqueness Value for a total of 10 or 16 bytes in length.

With an INSERT operation, Teradata stores both the data values and the Row ID as part of the data row.

ROW ID = ROW HASH and UNIQUENESS VALUE

Row Hash

- Row Hash is based on Primary Index value.
- Multiple rows in a table could have the same Row Hash.
- NUPI duplicates and hash synonyms have the same Row Hash.

Uniqueness Value

- The AMP creates a numeric 32-bit Uniqueness Value.
- The first row for a Row Hash has a Uniqueness Value of 1.
- Additional rows have ascending Uniqueness Values.
- Row IDs determine sort sequence within a Data Block.
- The full Row ID supports Secondary Index performance.
- The Row ID makes every row within a table uniquely identifiable.

Duplicate Rows

- Row ID uniqueness does not imply data uniqueness.

Note: The Row ID for a non-partitioned table is effectively 8 bytes long. For partitioned tables, the Row ID also contains the partition number.

AMP File System – Locating a Row via PI

The steps on the facing page outline the process that Teradata uses to locate a row. We know that rows are distributed according to their Row Hash. More specifically, the Hash Bucket Number points to a single entry in a Hash Map which designates a particular AMP.

- Once the correct AMP has been found, the Master Index for that AMP is used to identify which Cylinder Index should be referenced.
- The Cylinder Index then identifies the correct Data Block.
- A search of the Data Block locates the row or rows specified by the original three-part message.
- The system performs either linear or indexed searches.

The diagram at the bottom of the facing page illustrates these steps in a graphical fashion.

AMP File System – Locating a Row via PI

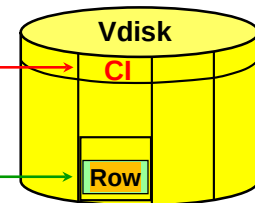
- The AMP accesses its Master Index (always memory-resident).
 - An entry in the Master Index identifies a Cylinder # and the AMP accesses the Cylinder Index (frequently memory-resident).
- An entry in the Cylinder Index identifies the Data Block.
 - The Data Block is the physical I/O unit and may or may not be memory resident.
 - A search of the Data Block locates the row(s).

The PE sends request to an AMP
via the Message Passing Layer
(PDE & BYNET).

Table ID	Part #	Row Hash	PI Value
----------	--------	----------	----------

FSG Cache

Master Index
Cylinder Index
Data Block



Teradata File System Overview

The Teradata File System software has these characteristics:

- part of AMP address space
- unaware of other AMP or File System instances
- AMP Interface to disk services
- uses PDE FSG services

The Master Index contains an entry (CID) for each allocated cylinder. (CID – Cylinder Index Descriptor)

On the facing page, SRD–A represents an SRD (Subtable Reference Descriptor) for table A. DBD–A1 and DBD–A2 represent data blocks for table A. (DBD – Data Block Descriptor)

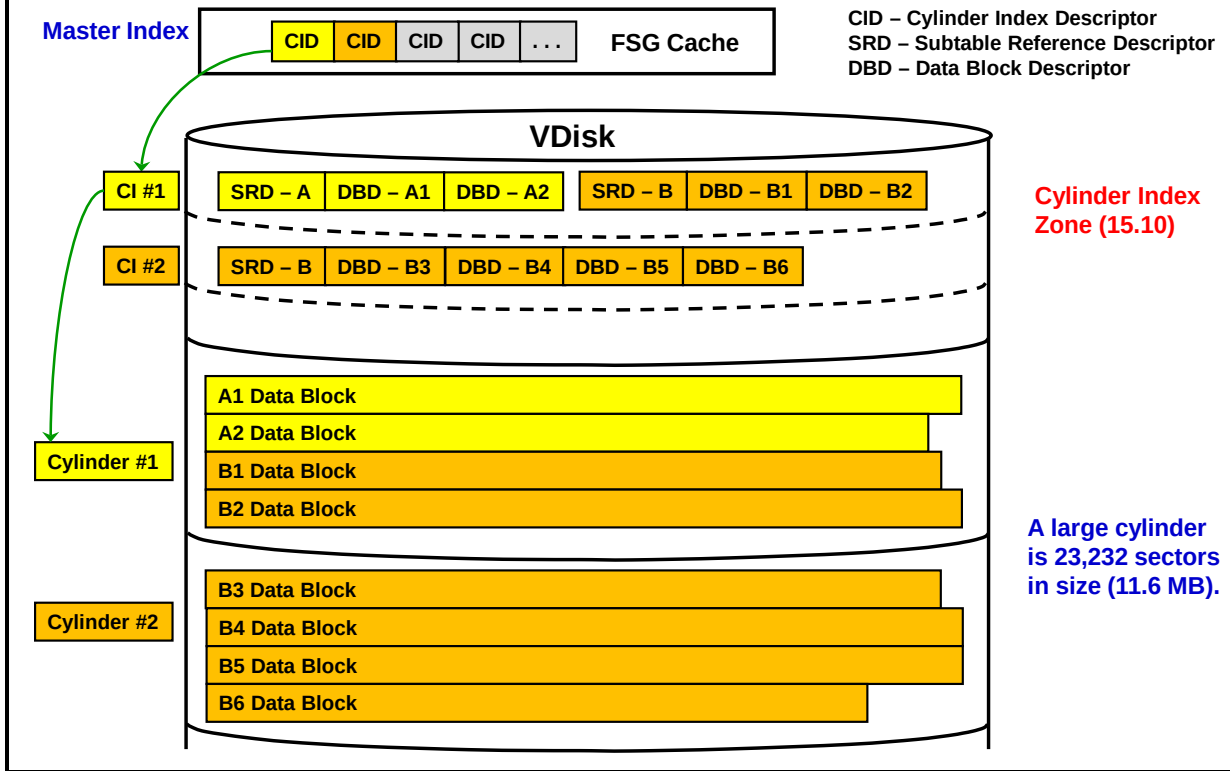
On the facing page, SRD–B represents an SRD for table B. DBD–B1, etc. represent data blocks for table B.

Assuming large cylinders, there are actually two cylinder indexes allocated for each cylinder. Each cylinder index is 64 KB in size. Therefore, there is 128 KB allocated for cylinder indexes for each cylinder.

Miscellaneous notes:

- Master index entries are 72 bytes long.
- Each cylinder has 2 cylinder indexes associated with it. A cylinder index is 12 KB for small cylinders or 64 KB for large cylinders.
- Prior to Teradata 13.10 and Large Cylinder Support, small cylinders were 3872 sectors. Large cylinders are 23,232 sectors or about 11.6 MB in size.
- For small cylinders (1.9 MB), each cylinder index is 12 KB in size; therefore, the cylinder space is effectively 24 KB.
- For large cylinders (11.6 MB) and used on most systems, each cylinder index is 64 KB in size; therefore, the cylinder space is effectively 128 KB.
- Starting in Teradata 15.10, cylinder indexes are no longer stored at the beginning of the cylinder. They are stored in a separate cylinder index zone area on disk.
- Data rows for partitioned tables require an additional 2 or 8 bytes to identify the partition number and the spare byte is set to identify the row as a PPI row. Secondary index subtable rows also have the Part # + Row Hash + Uniqueness ID) to identify data rows.

Teradata File System Overview



Data Block Layout

A Block is the physical I/O unit for Teradata. It contains one or more data rows, all of which belong to the same table. They must fit entirely within the block.

Prior to Teradata 14.10, the maximum block size is 255 sectors or 127.5 KB.

Starting with Teradata 14.10, the maximum block size has been increased from its old limit of 127.5 KB to a new limit of approximately 1 MB.

- The maximum user-specifiable size is 2047 sectors, ensuring the row data fits into any 1 MB message.

A Data Block consists of three major sections:

- The Data Block Header (DB Header)
- The Row Heap
- The Row Reference Array

Rows cannot be split between blocks. Each row in a DB is referenced by a separate index to the row known as the Row Reference Array. The **Row Reference Array** is placed at the end of the data block just before the Block Trailer.

For tables defined with small blocks (≤ 255 sectors), each row has at least 14 bytes of overhead in addition to the data values stored in that row. For tables defined with large blocks (> 255 sectors and ≤ 2047 sectors), each row has at least 16 bytes of overhead. If a table is partitioned, each row has an additional 2 or 8 bytes for the partition number.

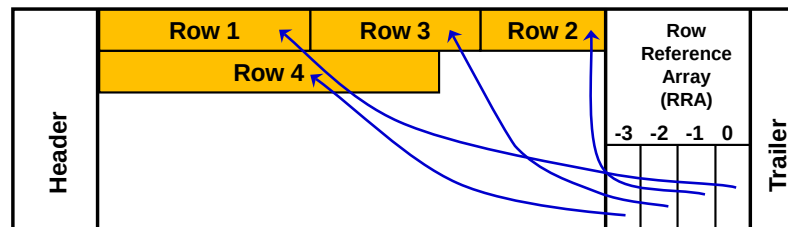
The Row Reference Array contains either a 2-byte or 4-byte Reference Array Pointer to the row within the block. 2-byte pointers are used with small blocks and 4-byte pointers are used with large blocks. This pointer identifies the offset of where the row starts within the block. If a row is an odd number of bytes in length, the Row Length specifies its precise length, but the system allocates whole words within the block for the row. Rows will start on an even address boundary.

- Teradata truly supports variable length rows.

Starting with Teradata 16.00, the 1 MB row support feature allows permanent tables to hold rows up to ~1 MB in size. The amount of row overhead is an additional 4 bytes per data row. Details on the large row layout are covered later in this course.

Data Block Layout

- A data block contains rows with same subtable ID.
 - A data block is basically a logically sorted set of data rows.
 - Contains rows within range of Row IDs of associated DBD entry and the range of Row IDs does not overlap with any other data block.
- Prior to Teradata 14.10, the maximum block size was 255 sectors (127.5 KB) and can still be defined in Teradata 16.00. These are referred to as small data blocks.
- Starting with Teradata 14.10, the maximum block size is 2047 sectors (~ 1 MB). Blocks greater than 255 sectors are referred to as large data blocks.
 - Blocks can vary in size from 1 sector to the maximum size (255 or 2047).
 - Each RRA entries is 2 bytes for small blocks and 4 bytes in size for large blocks.
- Starting with Teradata 16.00, permanent tables can hold rows up to ~1 MB in size (prior maximum row size was ~64 KB).
 - Rows can vary in size within the block and a row cannot span multiple blocks.



Example of Locating a Row – Master Index

In the example on the facing page, you can see how Teradata would use the Master Index to locate the data requested by a SELECT statement. The three-part message is Table ID=100, Row Hash=1000 and EMPNO=3755. After identifying the appropriate AMP, Teradata uses that AMP's Master Index to locate which cylinder contains this Table ID and Row Hash. By examining the Master Index, you can see that Cylinder Number 169 contains the appropriate row, if it exists in the system.

Teradata's File System software does a binary search of the CIDs based on Table ID / Part # / Row Hash or Table ID / Part # / Row ID to locate the cylinder number that has the row(s). The CI for that cylinder is accessed to locate the data block.

A user request for a row based on a Primary Index value will only have the Table ID / Part # / Row Hash.

A user request for a row based on a Secondary Index (SI) will have the Table ID / Row Hash for the SI value. The SI subtable row contains the Row ID(s) of the base table row(s). Teradata software uses the Table ID / Row ID(s) to locate the base table row(s). If a table is partitioned, the SI subtable row will have the Part #, the row hash, and the uniqueness value.

Free cylinders appear in the Free Cylinder List which is part of the FIB (File System Information Block) for the associated Vdisk.

Summary

- **There is only one entry for each cylinder on the AMP.**
- **Cylinders with data appear on the Master Index.**
- **Cylinders without data appear on the free Cylinder List (which is located within the FIB – File System Information Block).**
- **Each index entry identifies its cylinder's lowest Table ID / Partition # / Row ID.**
- **Index entries are sorted by Table ID, Partition #, and Lowest Row ID.**
- **Multiple tables may have rows on the same cylinder.**
- **A table may have rows on many cylinders on different Pdisks on an AMP.**
- **The Free Cylinder List is sorted by Cylinder Number.**

Example of Locating a Row – Master Index

Table ID	Part #	Row Hash	empno
100	0	1000	3755

```
SELECT *
FROM Employee
WHERE empno = 3755;
```

Master Index							Free Cylinder List Pdisk 0	Free Cylinder List Pdisk 1
Lowest			Highest			Pdisk and Cylinder Number		
Table ID	Part #	Row ID	Table ID	Part #	Row Hash			
:	:	:	:	:	:	:	:	:
078	0	58234, 2	095	0	72194	204	124	761
098	0	00107, 1	100	0	00676	037	125	780
100	0	00773, 3	100	0	01361	169	168	895
100	0	01361, 2	100	0	02884	777	170	896
100	0	02937, 1	100	0	03602	802	183	914
100	0	03662, 1	100	0	03999	117	189	935
100	0	04123, 2	100	0	05888	888	201	941
100	0	05974, 1	100	0	07328	753	217	1012
100	0	07353, 1	120	0	00469	477	220	1234
123	1	00343, 2	123	2	01864	529	347	1375
123	2	06923, 1	123	3	00231	943	702	1520
:	:	:	:	:	:	:	:	:

Part # - Partition Number

To CYLINDER INDEX

What cylinder would have Table ID = 100, Row Hash = 00598?

Example of Locating a Row – Cylinder Index

Using the example on the facing page, the File System would determine that the data block it needs is the six-sector block beginning at sector 0789. The Table ID and Row Hash we are looking for (100 + 1000, n) falls between the lowest and highest entries of 100 + 00998, 1 and 100 + 01010.

The convention of 00998, 1 is as follows: 00998 is the Row Hash and 1 is the Uniqueness Value.

Teradata's File System software does a binary search of the SRDs based on Table ID and a binary search of the DBDs Partition #, Row Hash or Row ID to identify the data block(s) that has the row(s).

A user request for a row based on a Primary Index value will include the Table ID / Part # / Row Hash.

A user request for a row based on a Secondary Index (SI) will have the Table ID / Part # / Row Hash for the SI value. The SI subtable row contains the Row ID(s) of the base table row(s). Teradata software uses the Table ID / Part # / Row ID(s) to locate the base table row(s) for secondary index accesses. If a table is partitioned, the SI subtable row will have the Part # and the Row ID.

The example on the facing page illustrates a cylinder that only has one SRD. All of the data blocks in this cylinder are associated with the same subtable.

Summary

- **There is an entry (DBD) for each data block on this cylinder.**
- **These entries are sorted ascending on Table ID, Partition #, and Lowest Row ID.**
- **Only rows belonging to the same table and sub-table appear in a block.**
- **Blocks belonging to the same sub-table can vary in size.**
- **Blocks without data appear on the Free Sector List that is sorted ascending on sector number.**

Example of Locating a Row – Cylinder Index

Table ID	Part #	Row Hash	empno
100	0	1000	3755

```
SELECT *
FROM Employee
WHERE empno = 3755;
```

Cylinder Index - Cylinder #169							
SRDs	Table ID	First DBD Offset	DBD Count				
SRD #1	100	FFFF	12				
DBDs	Part #	Lowest Row ID	Part #	Highest RowHash	Start Sector	Sector Count	Row Count
:	:	:	:	:	:	:	:
DBD #4	0	00867, 2	0	00902	1010	4	5
DBD #5	0	00938, 1	0	00996	0093	7	10
DBD #6	0	00998, 1	0	01010	0789	6	8
DBD #7	0	01010, 3	0	01177	0525	3	4
DBD #8	0	01185, 2	0	01258	0056	5	6
DBD #9	0	01290, 1	0	01333	1138	5	6
:	:	:	:	:	:	:	:

Free Block List Free Sector Entries	
Start Sector	Sector Count
:	:
0270	3
0301	5
0349	5
0470	4
0481	6
0550	5
:	:

This example assumes that only 1 table ID has rows on this cylinder and the table is not partitioned.

Part # - Partition Number

Example of Locating a Row – Data Block

A Block is the physical I/O unit for Teradata. It contains one or more data rows, all of which belong to the same table. They must fit entirely within the block.

The maximum block size is 255 sectors or 127.5 KB or 2047 sectors ~ 1 MB (TD 14.10)

A Data Block consists of three major sections:

- The Data Block Header (DB Header)
- The Row Heap
- The Row Reference Array

Rows cannot be split between blocks. Each row in a DB is referenced by a separate “offset or pointer” to the row. These offsets are kept in the Row Reference Array. The **Row Reference Array (RRA)** is placed near the end of the DB just before the Block Trailer.

The **DB Header** contains control information for both the Row Reference Array and the Row Heap. The DB Header also contains the Subtable ID (6 bytes).

The **Row Heap** is where the rows reside in the DB. The rows may be in any physical order, are aligned on an even address boundary, and therefore have an even number of bytes allocated for them.

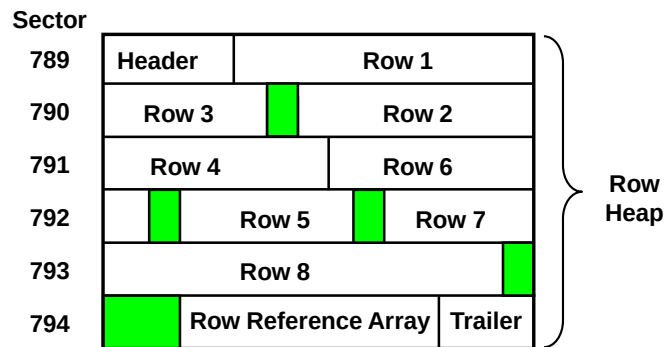
Reference Array entries are either 2 or 4 bytes. These entries point to the first byte of a row (Row Length) and are maintained in reverse Row ID sequence. The Reference Array entries are used to do both binary and sequential searches.

The Block Trailer consists of a block version number which must match the block version number in the Data Block Header.

Note: The amount of space used by DB Headers and Trailers depends on the DATABLOCKSIZE attribute for the table. If ≤ 255 sectors, then the table has small blocks defined. If > 255 sectors, then the table has large blocks defined.

- For small blocks without BLC (Block Level Compression), then the DB Header is 72 bytes, block trailer is 2 bytes, and each RRA entry is 2 bytes.
- For small blocks with BLC (Block Level Compression), then the DB Header is 128 bytes (72 + 56 of extended information for compression), block trailer is 2 bytes, and each RRA entry is 2 bytes.
- For large blocks without BLC (Block Level Compression), then the DB Header is 104 bytes long, block trailer is 4 bytes, and each RRA entry is 4 bytes.
- For large blocks with BLC (Block Level Compression), then the DB Header is 160 bytes (104 + 56 of extended information for compression), block trailer is 4 bytes, and each RRA entry is 4 bytes.
- Note that, in a compressed block, the header is not compressed and neither is the block trailer. Only the row data within the block is compressed.

Example of Locating a Row – Data Block



- A block is the physical I/O unit.
- The block header contains the Table ID (6 bytes).
- Maximum row size in Teradata 16.00 is ~ 1 MB.
 - Only rows for the same table reside in the same data block.
 - Rows are not split across block boundaries.
- Blocks within a table vary in size. The system adjusts block sizes dynamically.
 - Blocks may be from 1 sector (512 bytes) to 255 (127.5 KB) or 2047 sectors ~ 1 MB (TD 14.10).
 - Data blocks are not chained together.
 - Row Reference Array entries are stored (sorted) in reverse sequence based on Row ID within the block. These entries are either 2 or 4 bytes long – 2 for small blocks and 4 for large blocks (14.10 feature).

Accessing the Row within the Data Block

Teradata's File System software does a binary search of the Row Reference Array to locate the rows that have a matching Row Hash. Since the Row Reference Array is sorted in reverse sequence based on Row ID, the system can do a binary or linear search.

The first row with a matching Row Hash has its Primary Index value compared with the Primary Index value in the request. The PI value must be checked to eliminate Hash Synonyms. The matching rows are then put into spool. If no matches are made, a message is returned that no rows are found.

In the case of a Unique Primary Index (UPI), the search ends with the first row found matching the criteria. The row is then returned.

In the case of a Non-Unique Primary Index (NUPI), the matching rows (same PI value and Row Hash) are put into spool. With a NUPI, the matching rows in spool are returned.

The example on the facing page illustrates how Teradata utilizes the Primary Index data value to eliminate synonyms. This is the conclusion of the example that we have been following throughout this module.

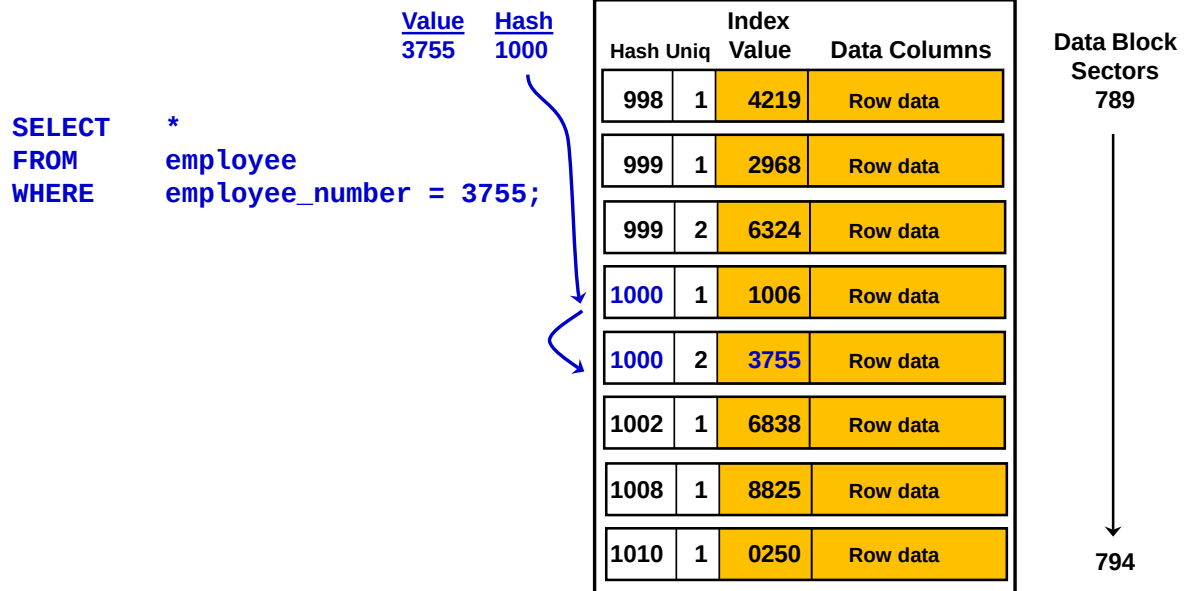
In earlier steps, the Master Index was used to find that the desired row was in Cylinder 169. Then the Cylinder Index was used to find that the desired row was in the 6-sector block beginning in Sector Number 789. The diagram shows that block.

The objective is to find that row with **Row Hash=1000** and **Index Value=3755**. When the block is searched, the first row with Row Hash 1000 does not meet these criteria. Its Index Value is 1006, which means that it is a Hash Synonym. The system must continue its search to the next row, the only row that meets both criteria.

The illustration on the facing page shows the logical order of rows in the block with a binary search.

Accessing the Row within the Data Block

- Within the data block, the Row Reference Array is used to locate the first row with a matching Row Hash value within the block.
- The Primary Index data value is used as a row qualifier to eliminate synonyms.



AMP Read I/O Summary

You have seen that a Primary Index Read requires that the Master Index, Cylinder Index and Data Block all must be accessed. The number of I/Os involved in this process can vary.

The Master Index is always resident in memory. The Cylinder Index may or may not be resident in memory and the Data Block may or may not be resident in memory.

Factors that affect the number of I/Os involved include AMP memory, cache size and locality of reference. Often the Cylinder Index is memory resident so that a Unique Primary Index retrieval requires only a single I/O.

Note that no matter how many rows are in the table and no matter how many inserts are made, Primary Index access never gets any more complicated than Master Index to Cylinder Index to Data Block.

AMP Read I/O Summary

The Master Index is always memory resident.

The AMP reads the Cylinder Index if not memory resident.

The AMP reads the Data Block if not memory resident.

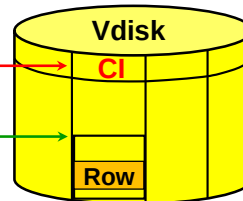
- The amount of FSG cache size also has an impact if either of these steps require physical I/O.
- The data block may or may not be memory residence depending on recent accesses of this data block.
- The Cylinder Index is usually memory resident and a Unique Primary Index retrieval requires only one I/O.

Message Passing Layer

Table ID	Part #	Row Hash	PI Value
----------	--------	----------	----------

AMP (FSG Cache)

Master Index
Cylinder Index
Data Block



Module 12: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 12: Review Questions

1. The Row Hash for a PI value of 824 is the same for the data types of INTEGER and DECIMAL(18,0). True or False. _____
2. The first 20 bits of the Row Hash is referred to as the _____.
3. The Hash Map consists of entries or buckets which identify an _____ number for the Row Hash.
4. The Current Configuration _____ Hash Map is used to locate the AMP to locate/store a row based on PI value.
5. The _____ utility is used to redistribute rows to a new system configuration with more AMPs.
6. When creating a new table, the Unique Value of a Table ID comes from the dictionary table named DBC._____.
7. The Row ID consists of the _____ and the _____.
8. The _____ contains a Cylinder Index Descriptor (CID) for each allocated Cylinder.
9. The _____ contains an entry for each data block in the cylinder.
10. The _____ consists of a set of pointers to the data rows in data block.
11. In TD 14.0, the maximum block size is approximately _____ and the maximum row size is approximately _____.
12. In TD 16.00, the maximum block size is approximately _____ and the maximum row size is approximately _____.

Notes

Module 13



File System Writes

After completing this module, you will be able to:

- Describe File System Write Access.
- Describe what happens when Teradata inserts a new row into a table.
- Describe the impact of row inserts on block sizes.
- Describe how fragmentation affects performance.

Teradata Proprietary and Confidential

Notes

Table of Contents

AMP Write I/O.....	13-4
New Row INSERT – Master Index Access	13-6
New Row INSERT – Cylinder Index Access	13-8
New Row INSERT – use Free Space within Block.....	13-10
New Row INSERT – Expand the Block.....	13-12
New Row INSERT – Part 3 (cont.).....	13-14
Alternate Cylinder Index.....	13-14
Block and Row Terminology	13-16
DBSCONTROL options in Teradata 14.10	13-16
Block Splits (INSERT and UPDATE).....	13-18
Cylinder Full	13-20
Mini-Cylpack	13-22
Space Utilization.....	13-24
Teradata 13.10 Auto Cylinder Pack Feature	13-24
Merge Datablocks	13-26
Merge Datablocks (cont.).....	13-28
How to use this Feature.....	13-28
File System Write Summary	13-30
Module 13: Review Questions	13-32
Module 13: Review Questions (cont.)	13-34

AMP Write I/O

The facing page illustrates how Teradata performs write operations and it outlines steps required to perform an AMP Write operation.

WAL (Write Ahead Logging) is a recoverability/reliability feature that also provides performance improvements in the area of database writes. WAL is a Teradata V2R6.2 (and later) feature. WAL can batch up modifications from multiple transactions and apply them with a single disk I/O, thereby saving I/O operations. WAL will help improve throughput for I/O-bound workloads.

WAL is a log-based file system recovery scheme in which modifications to permanent data are written to a log file, the WAL log. The log file contains change records (Redo records) which represent the updates. At key moments, such as transaction commit, the WAL log is forced to disk. In the case of a reset or crash, Redo records can be used to transform the old copy of a permanent data block on disk into the version that existed at the time of the reset.

By maintaining the WAL log, the permanent data blocks that were modified no longer have to be written to disk as each block is modified. Only the Redo records in the WAL log must be written to disk. This allows a write cache of permanent data blocks to be maintained. WAL protects all permanent tables and all system tables but is not used to protect either the Transient Journal (TJ), since TJ records are stored in the WAL log, or any type of spool tables, including global temporary tables.

The WAL log is maintained as a separate logical file system from the normal table area. Whole cylinders are allocated to the WAL log, and it has its own index structure. The WAL log data is a sequence of WAL log records and includes the following:

- Redo records, used for updating disk blocks and insuring file system consistency during restarts.
- TJ records used for transaction rollback.

There is some additional CPU cost for maintaining the WAL log so WAL may reduce throughput for CPU-bound workloads. However, the overall performance is expected to be better with WAL since the benefit of I/O improvement outweighs the much smaller CPU cost.

If CHECKSUM = NONE and the New Block length = Old Block length, Teradata will attempt to update-in-place for any INSERT, DELETE, or UPDATE operations.

If the CHECKSUM feature is enabled for a table, any INSERT, UPDATE, or DELETE operation will cause a new data block to be allocated.

The FastLoad and MultiLoad utilities always allocate new data blocks for write operations. TPump follows the same rules as an SQL INSERT, UPDATE, or DELETE.

AMP Write I/O

For SQL writes, Teradata uses WAL logic to manage disk write operations.

- Read the Data Block if not in memory (Master Index > Cylinder Index > Data Block).
- Place appropriate entries (e.g., before-images) into the Transient Journal buffer (actually a WAL buffer) and write it to the WAL log on disk.
- Data blocks are updated in Memory, but not written immediately to disk.
- The after-image or changed image (REDO row) is written to a WAL buffer which is written to the WAL log on disk.
 - WAL can batch up modifications from multiple transactions and apply them with a single disk I/O, thereby saving I/O operations.
 - Updated data blocks in memory will be eventually aged out and written to disk.
- Make the changes to the Data Block in memory and determine the new block's length.
 - If the New Block has changed size, always allocate a new Data Block.
 - If the New Block length = Old Block length, Teradata will attempt to update-in-place for any INSERT, DELETE, or UPDATE operations.

These operations happen concurrently on the Fallback AMP.

New Row INSERT – Master Index Access

The facing page illustrates what happens when Teradata INSERTs a new row into a table. The three part message is Table ID = 100, Partition # = 0, Row Hash = 1123 and PI Value = 7923.

- The AMP uses its Master Index to locate the proper cylinder for the new row. As you can see, Cylinder #169 is where a row with Table ID = 100, Partition # = 0, and Row Hash = 1123 should be inserted.
- The next step is to access the Cylinder Index for Cylinder #169, as illustrated on the facing page.

Teradata's File System software does a binary search of the CIDs based on Table ID / Partition # / Row Hash to locate the cylinder number in which to insert the row. The CI for that cylinder is accessed to locate the data block.

This example assumes the table has small blocks defined (≤ 255 sectors). Therefore, each Row Reference Array entry is only 2 bytes. Each Row Reference Array entry for a large block (> 255 sectors) is 4 bytes.

New Row Insert – Master Index Access

INSERT INTO employee VALUES (7923,);

INSERT	Table ID	Part #	Row Hash	data column values			
ROW	100	0	1123	7923			

Master Index							Free Cylinder List Pdisk 0	Free Cylinder List Pdisk 1
Lowest			Highest			Pdisk and Cylinder Number		
Table ID	Part #	Row ID	Table ID	Part #	Row Hash			
:	:	:	:	:	:	:	:	:
078	0	58234, 2	095	0	72194	204	124	761
098	0	00107, 1	100	0	00676	037	125	780
100	0	00773, 3	100	0	01361	169	168	895
100	0	01361, 2	100	0	02884	777	170	896
100	0	02937, 1	100	0	03602	802	183	914
100	0	03662, 1	100	0	03999	117	189	935
100	0	04123, 2	100	0	05888	888	201	941
100	0	05974, 1	100	0	07328	753	217	1012
100	0	07353, 1	120	0	00469	477	220	1234
123	1	00343, 2	123	2	01864	529	347	1375
123	2	06923, 1	123	3	00231	943	702	1520
:	:	:	:	:	:	:	:	:

To CYLINDER INDEX

Part # - Partition Number

New Row INSERT – Cylinder Index Access

The example on the facing page is a continuation from the previous page. Teradata has determined that the new row must be INSERTed into Cylinder #169 in this example.

New Row Insert – Cylinder Index Access

INSERT INTO employee VALUES (7923,);

INSERT	Table ID	Part #	Row Hash	data column values			
ROW	100	0	1123	7923			

Cylinder Index - Cylinder #169							
SRDs	Table ID	First DBD Offset	DBD Count				
SRD #1	100	FFFF	12				
DBDs	Part #	Lowest Row ID	Part #	Highest RowHash	Start Sector	Sector Count	Row Count
:	:	:	:	:	:	:	:
DBD #4	0	00867, 2	0	00902	1010	4	5
DBD #5	0	00938, 1	0	00996	0093	7	10
DBD #6	0	00998, 1	0	01010	0789	6	8
DBD #7	0	01010, 3	0	01177	0525	3	4
DBD #8	0	01185, 2	0	01258	0056	5	6
DBD #9	0	01290, 1	0	01333	1138	5	6
:	:	:	:	:	:	:	:

Free Block List Free Sector Entries	
Start Sector	Sector Count
:	:
0270	3
0301	5
0349	5
0470	4
0481	6
0550	5
:	:

Read the block into memory
(FSG cache).

To Data Block

New Row INSERT – use Free Space within Block

The example on the facing page is a continuation from the previous page. Teradata has determined that the new row hash value falls within the range of the data block that starts at sector 525 and is 3 sectors long.

If the block that has been read into memory (FSG Cache) has enough contiguous free bytes, then the row is inserted into this space within the block. The row reference array and the Cylinder Index are updated.

If the block that has been read into memory (FSG Cache) does not have enough contiguous free bytes, but it does have enough free bytes within the entire block, the software will defragment the block and insert the row. The row reference array and the Cylinder Index are updated.

Note: The block header contains a field that indicates the total number of free bytes within the block.

Also note that the Row Reference Array expands by 2 bytes to reflect the added row. If the block now has 5 rows, the Row Reference Array will increase from 8 bytes to 10 bytes in length.

Acronyms:

FS – Free Space

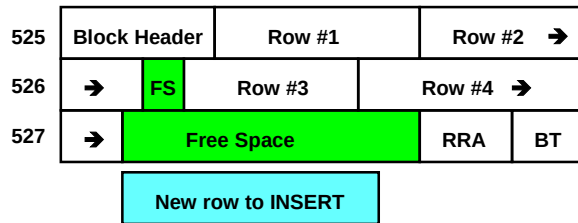
RRA – Row Reference Array

BT – Block Trailer

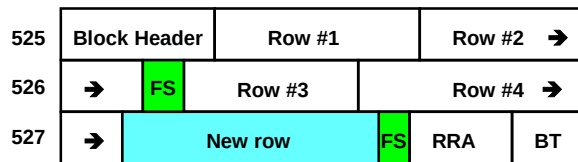
New Row Insert – use Free Space within Block

Read the block into memory (FSG cache).

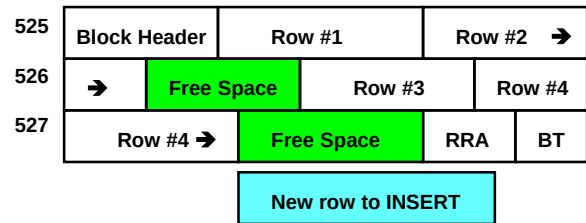
1. If the block has enough free contiguous bytes, then insert row into block and update CI.



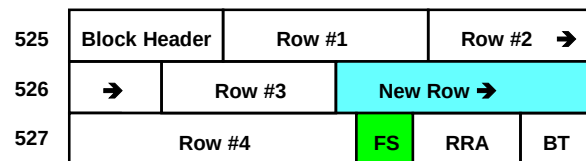
Row is inserted into contiguous free space.



2. If the block has enough free bytes, then defragment the block and insert row into block and update CI.



Block is defragmented and row is inserted into contiguous free space.



FS - Free Space; RRA - Row Reference Array; BT - Block Trailer

New Row INSERT – Expand the Block

The File System then accesses the 3-sector block which starts at sector 525 and makes it available in AMP memory.

The row is placed into the block, and the new block length is computed. In this example, inserting the row has caused the block to expand from 3 sectors to 4 sectors.

Note that the Row Reference Array expands by 2 bytes to reflect the added row. If the block now has 5 rows, the Row Reference Array will increase from 8 bytes to 10 bytes in length.

If this table was defined with large blocks (> 255 sectors), then each Row Reference Array entry would be 4 bytes.

Acronyms:

FS – Free Space

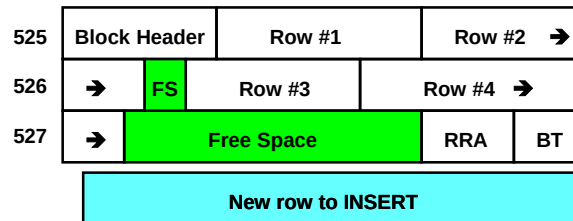
RRA – Row Reference Array

BT – Block Trailer

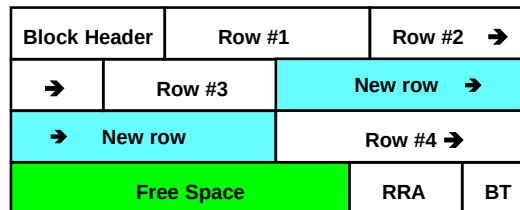
New Row Insert – Expand the Block

3. If the new row is larger than the total free space within the block, then the Insert expands the block by as many sectors as needed (in memory) to hold the row.

In this example, the block is expanded by one sector in memory.



In memory, block is expanded, defragmented, and the new row is inserted.



- 3b. The next step is to locate the first block on the [Free Block List](#) equal to, or greater than 4 sectors.

New Row INSERT – Part 3 (cont.)

The File System searches the Free Sector (or Block) List looking for the first Free Block whose size is equal to or greater than the new block's requirement. It does not have to be an exact match.

- Upon finding a 5-sector free block starting at sector 0301, the system allocates a new 4-sector block (sectors 301, 302, 303, 304) for the new data block, leaving a free block of one sector (305) remaining.
- The new data block is written to disk.
- The old, 3-sector data block is placed onto the Free Sector List (or Free Block List).
- The modified CI will be copied to the buddy node (FSG Cache) and the modified CI will be written back to disk (eventually).

If a transaction failure occurs (or the transaction is aborted), the Transient Journal is used to undo the changes to both the data blocks and the Cylinder Indexes. Before images of data rows are written to the Transient Journal. Before images of Cylinder Indexes are not written to the Transient Journal because Teradata uses the Alternate Cylinder Index for the changes. If a transaction fails, the before image in the Transient Journal is used to return the data row(s) back to the state before the transaction.

Alternate Cylinder Index

Space for 2 Cylinder Indexes is allocated at the beginning of every cylinder. Characteristics include:

- Two Cylinder Indexes are used – Teradata alternates between the two Cylinder Indexes.
- Changes are written to an “Alternate Cylinder Index”.
- When a CI is changed, it is not updated in place. This provides for better I/O integrity.
- For small cylinders, each Cylinder Index is 12 KB.
- For large cylinders, each Cylinder Index is 64 KB.

New Row INSERT – Part 3 (cont.)

Cylinder Index - Cylinder #169							
SRDs	Table ID	First DBD Offset	DBD Count				
SRD #1	100	FFFF	12				
DBDs	Part #	Lowest Row ID	Part #	Highest RowHash	Start Sector	Sector Count	Row Count
:	:	:	:	:	:	:	:
DBD #5	0	00938, 1	0	00996	0093	7	10
DBD #6	0	00998, 1	0	01010	0789	6	8
DBD #7	0	01010, 3	0	01177	0525	3	4
DBD #8	0	01185, 2	0	01258	0056	5	6
:	:	:	:	:	:	:	:

Free Block List Free Sector Entries	
Start Sector	Sector Count
:	:
0270	3
0301	5
0349	5
0470	4
0481	6
0550	5
:	:

Alternate Cylinder Index - Cylinder #169							
SRDs	Table ID	First DBD Offset	DBD Count				
SRD #1	100	FFFF	12				
DBDs	Part #	Lowest Row ID	Part #	Highest RowHash	Start Sector	Sector Count	Row Count
:	:	:	:	:	:	:	:
DBD #5	0	00938, 1	0	00996	0093	7	10
DBD #6	0	00998, 1	0	01010	0789	6	8
DBD #7	0	01010, 3	0	01177	0301	4	5
DBD #8	0	01185, 2	0	01258	0056	5	6
:	:	:	:	:	:	:	:

Free Block List Free Sector Entries	
Start Sector	Sector Count
:	:
0270	3
0305	1
0349	5
0470	4
0481	6
0525	3
0550	5

Block and Row Terminology

Tables supporting Data Warehouse and Decision Support users generally have their block size set very large to accommodate more rows per block and reduce the number of block I/Os needed to do full table scans. Tables involved in online applications and heavy data maintenance generally have smaller block sizes.

Extremely large rows, called Oversized Rows, can be costly. Each Oversized row requires its own block and costs one I/O every time it is touched. Oversized rows are common in non-relational data models and often appear in poor relational data models.

DBSCONTROL options in Teradata 14.10

To enable/disable the 1 MB data block feature, use DBSControl internal field #294.

294. 1MBEnablingControls:

- 1 - Enable1MBSpoolDBS = TRUE
- 2 - Enable1MBSpoolRows = TRUE
- 3 - Enable1MBPermDBS = TRUE
- 4 - Enable1MBPermRows = (Not Available)

- SYSINIT (or setting NoDot0Backdown) at 14.10 sets fields to TRUE
- UPGRADE to Teradata Database 14.10 sets fields to FALSE
- Disabling 1MB Spool/Perm DBs disables the ability to create Big blocks or specify PermDBSize/WorkDBSize/DATABLOCKSIZE >255 sectors
 - Any in-flight queries still work fine. Any existing blocks can still be read in any format. New modifications will use Small block sizes.
- Enable1MBPermRows is not modifiable in Teradata Database 14.10
- PermDBSize and WorkDBSize
 - New absolute maximum values of 2047 sectors when 1MB Perm Blocks / 1MB Spool Blocks are available
 - Default values TBD pending performance testing on various platforms
- SpoolBigDBAllocUnit (New, Filesys Field 30)
 - Like PermDBAllocUnit; applied to Big Spool blocks
 - Default of 1 sector
- AllowUnrestrictedBigDBs (New, Filesys INTERNAL Field 104)
 - Relaxes the 256KB block size limitation on Small Cylinder systems
 - But still enforces a limit of 512KB
 - Default of FALSE

Block and Row Terminology

Data Block Size (DATABLOCKSIZE)

- Effectively the largest block size and impacts when a block split will occur.
 - 14.0 default DBSControl PermDBSize is 254 sectors (127 KB)
 - 14.10 default for EDW systems (67xx) is 512 sectors and Appliances (27xx) is 1024 sectors
 - Table level attribute DATABLOCKSIZE; Note: max of 2047 sectors (~ 1 MB) in TD 14.10

Small and Large Data Blocks

- **Small Blocks – established if table DATABLOCKSIZE <= 255 sectors**
 - Without BLC, DB Header = 72 bytes, block trailer = 2 bytes, and each **RRA entry = 2 bytes**.
 - With BLC, DB Header = 128 bytes (72 + 56 of extended information for compression), block trailer = 2 bytes, and each **RRA entry = 2 bytes**.
- **Large Blocks (14.10) – established if DATABLOCKSIZE > 255 sectors.**
 - Without BLC, DB Header = 104 bytes, block trailer = 4 bytes, and each **RRA entry = 4 bytes**.
 - With BLC, DB Header is 160 bytes (104 + 56 of extended information for compression), block trailer = 4 bytes, and **each RRA entry = 4 bytes**.

Large Row

- The largest fixed length row that allows multiple rows/block; less than ½ of block size

Oversized Row

- A row that requires its own Data Block (one I/O per row):
- A fixed length row that is larger than Large Row; greater than ½ of block size

Block Splits (INSERT and UPDATE)

Block splits occur during INSERT and UPDATE operations. Normally, when a data block expands beyond the maximum multi-row block size (Largest Block), it splits into two approximately equal-sized blocks. This is shown in the upper illustration on the facing page.

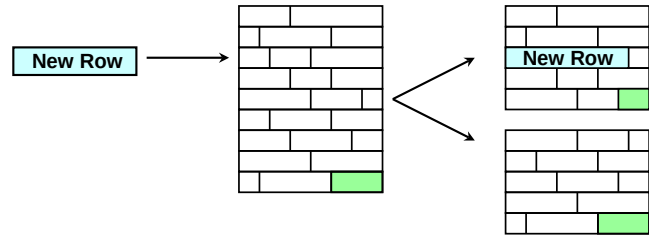
- If an Oversize Row is INSERTed into a data block, it causes a three-way block split (as shown in the lower illustration). This type of block split may result in uneven block sizes.
- With Teradata, block splits cost only one additional I/O per extra block created. There is little impact on OLTP and OLCP performance.
- Block splits automatically reclaim any contiguous, unused space greater than 511 bytes.

Block Splits (INSERT and UPDATE)

4. Perform a Block Split

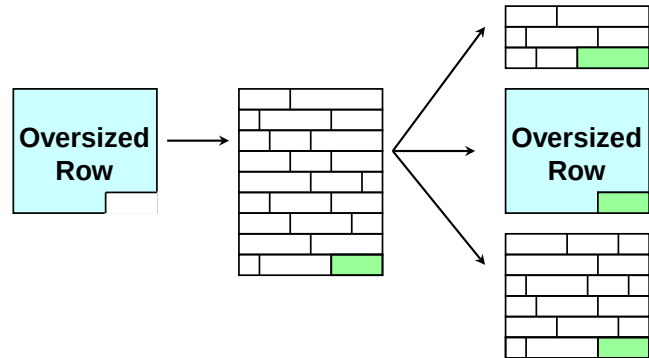
Two-Way Block Splits

- When a Data Block expands beyond Largest Block, it splits into two, fairly-equal blocks.
- This is the normal case.



Three-Way Block Splits

- An oversize row gets its own Data Block. The existing Data Block splits at the row's logical point of existence.
- This may result in uneven block sizes.



Notes:

- Block splits automatically reclaim any unused space over 511 bytes.
- While it is not typical to increment blocks by one 512 sector, it is tunable as to how many sectors are acquired at a time for the system.

Cylinder Full

A **Cylinder Full** condition occurs when there is no block on the Free Block List that has enough sectors to accommodate additional data during an INSERT or UPDATE. If this condition occurs, the File System goes through the steps outlined on the facing page which results in a **Cylinder Migrate** to an existing adjacent cylinder or to a new cylinder. As part of this process, the file system software may also choose to perform a **Cylinder Defragmentation** or a **Mini Cylinder Pack (Mini-Cylpack)** operation.

- A **Mini-Cylpack** is a background process that occurs automatically when the number of free (or available) cylinders falls below a threshold. The mini-Cylpack process is the mechanism that Teradata uses to rearrange data blocks to free cylinders. This process involves moving data blocks from a data cylinder to the logically preceding data cylinder until a whole cylinder becomes empty.
- Mini-Cylpack is an indication that the system does not have enough free space to handle its current workload.

In the example at the bottom of the facing page, if Cylinder 37 became full, the File System would check Cylinder 204 and Cylinder 169 to see if they had enough room to perform a Cylinder Migrate. These two cylinders are logically adjacent to Cylinder 37 in the Master Index, but not necessarily physically adjacent on the disk.

During the Cylinder Migrate, if data blocks were moved to Cylinder 204, they would be taken from the top of Cylinder 37. If they were moved to Cylinder 169, they would be taken from the bottom of Cylinder 37.

Note:

Performance tests show that defragging can cause a significant performance hit. Therefore, the default tuning parameters that control how often you do this are set to only defragment cylinders if there are very few free cylinders left (≤ 100) and the cylinder has quite a bit of free space that isn't usable ($\geq 25\%$). The latter indicates that, although there is significant free space on the cylinder, the free space is apparently so fragmented that a request for new sectors couldn't be satisfied. Otherwise, it's assumed that the cylinder is full and the overhead of defragging it wouldn't be worth it.

Cylinder Full

Cylinder Full means there is no block big enough on the Free Block List. The File System does either of the following:

5. **Cylinder Migrate to an adjacent cylinder** — checks logically adjacent cylinders for fullness. If it finds room, it moves a maximum of 10 data blocks from the full cylinder to an adjacent one.
6. **Cylinder Migrate to a new Cylinder** — looks for a free cylinder, allocates one, and moves a maximum of 10 data blocks from the congested cylinder to a new one.

While performing a Cylinder Migrate operation, the File System software may also do the following operations in the background.

- **Cylinder Defragmentation** — if the total cylinder free space $\geq 25\%$ of the cylinder size (25% is default), then the cylinder is defragmented. Defragmentation collects all free sectors at the end of a new cylinder by moving all the data blocks to the top of the new cylinder.
- **Mini-Cylpack** — if the number of free cylinders falls below a threshold (default is 10), then a "Mini-Cylpack" is performed to pack data together to free up a cylinder and place it on the free cylinder list.

Master Index							Free Cylinder List Pdisk 0	Free Cylinder List Pdisk 1
Lowest			Highest			Pdisk and Cylinder Number		
Table ID	Part #	Row ID	Table ID	Part #	Row Hash			
:	:	:	:	:	:	:	:	:
078	0	58234, 2	095	0	72194	204	124	761
098	0	00107, 1	100	0	00676	037	125	780
100	0	00773, 3	100	0	01361	169	168	895
:	:	:	:	:	:	:	:	:

Mini-Cylpack

The **Mini-Cylpack** is the mechanism that Teradata uses to rearrange data blocks to free cylinders. The process involves moving data blocks from a data cylinder to the logically preceding data cylinder until a whole cylinder becomes empty.

- A Mini-Cylpack is an indication that the system does not have enough free space to handle its current workload.
- Excessive numbers of **Mini-Cylpacks** indicate too little disk space is available and/or too much spool is being utilized during data maintenance.
- Spool cylinders are never “Cylpacked”.

Teradata has a **Free Space** (a percentage) parameter that can be set to control how much free space is left in a cylinder during loading and the use of the Ferret PackDisk utility. This parameter is not used with **mini-cylpacks**.

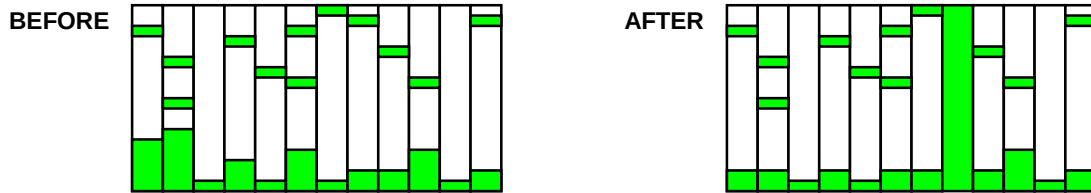
- This parameter should be set low (probably 0%) for tables which are used solely for analytic or decision support processing as there is little data maintenance involved.
- In cases where there is moderate data maintenance on tables (batch or some tactical or OLTP), the **Free Space** parameter should be set at approximately 10 – 15%.
- If heavy data maintenance is to be done on tables (tactical or OLTP), the **Free Space** parameter may have to be set at approximately 25 – 50% to prevent Cylpacks from affecting tactical or OLTP response times.

The **Free Space** parameter can be set at the system level, at a table level, and when executing the **Ferret PackDisk** utility.

- DBSControl – FREESPACEPERCENT (0% is the default)
- CREATE TABLE – FREESPACE = integer [PERCENT] (0 – 75)
- FERRET PACKDISK – FREESPACEPERCENT (or FSP) integer

The system administrator can specify a count of empty cylinders the system should attempt to maintain. Whenever a Cylinder Migrate to a new cylinder occurs, the system checks to see if the minimum number of empty cylinders still exists. If the system has dropped below the minimum, it starts a background task that begins packing cylinders. The task stops when either a cylinder is added to the Free Cylinder List or it has packed 10 cylinders. This process continues with every Cylinder Migrate to a new cylinder until the minimum count of empty cylinders is reached, or a full mini-cylpack is required.

Mini-Cylpack



When the system falls below a DBS Control specified threshold (e.g., 10 free cylinders), the Mini-Cylpack process moves data blocks from the data cylinder(s) to logically preceding data cylinder(s) until enough cylinders are emptied to go above the threshold.

- Spool cylinders are never cypacked.
- Mini-Cylpacks indicate that the system does not have space to handle its current workload.
- Excessive Cylpacks indicate too little disk space and/or spool utilization during data maintenance.

The [Free Space](#) parameter impacts how full a cylinder is filled with data loading and PackDisk.

- DBSControl – FREESPACEPERCENT
- CREATE TABLE – FREESPACE
- FERRET PACKDISK – FREESPACEPERCENT (FSP)

Space Utilization

The Teradata Database can use any particular cylinder to either store data or hold Spool files. A cylinder cannot be used for both data and Spool simultaneously. In sizing a system, you must make certain that you have enough cylinders to accommodate both requirements.

Limiting the number of rows and columns per query helps keep Spool requirements under control, as does keeping the number of columns per row to a minimum. Both can result from proper normalization.

Teradata 13.10 Auto Cylinder Pack Feature

One new background task in Teradata 13.10 is called AutoCylPack which attempts to combine adjacent, sparsely filled cylinders. These cylinder packs are typically executed when the system is idle.

AutoCylPack is particularly useful if a customer is using temperature-based BLC, because it cleans up post-compression cylinders that are no longer holding as much data. However, this feature works with compressed as well as uncompressed cylinders. Sometimes the activity of AutoCylPack can result in seeing a little bit of wait I/O (less than 5%).

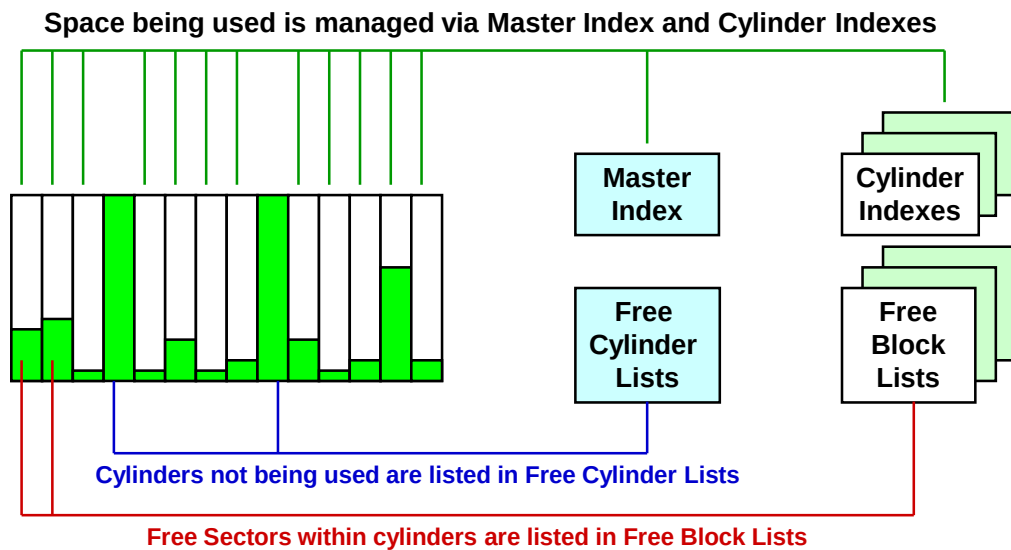
File System Field 17 (DisableAutoCylPack) has a default value of FALSE, which means AutoCylPack is on and active all the time, unless you change this setting.

General notes:

There are a number of background tasks running in the Teradata database and AutoCylPack is another of these tasks. These tasks include deadlock detection, cylinder defragmentation, transient journal purging, periodic cache flushes, etc.

These tasks generally consume a small amount of system resources. However, you will tend to notice them more when the system is idle.

Space Utilization



Cylinders contain Perm, Spool, Temporary, Permanent Journal, or WAL data, but NOT a combination.

BE SURE THAT YOU HAVE ENOUGH SPACE OF EACH.

Limiting the rows and columns per query reduces spool use.

Merge Datablocks

This Teradata Database 13.10 feature automatically searches for “small” data blocks within a table and will combine (merge) these small data blocks into a single larger block. Over time, modifications to a table (especially with DELETES of data rows) can result in a table having blocks that are less than 50% of the maximum data block size. This File System feature combines these small blocks into a larger block.

The benefit is simply that future full table operations (full table scans and/or full table updates) will perform faster because fewer I/Os are performed. By having larger blocks in the Teradata file system, the selection of target rows can also be more efficient.

- Blocks that are 50% or greater of the maximum multi-row data block size (63.5 KB in this example) are not considered to be small blocks. Small blocks are less than 50% of the maximum data block size.

The merge of multiple small blocks into a larger block is limited by cylinder boundaries – does not occur between cylinders. A maximum of 7 logically adjacent preceding blocks can be merged together into a target block when the target block is updated. Therefore, a maximum of 8 total blocks can be merged together.

Why are logical following blocks NOT merged together?

- The File System software does not know if following blocks are going to be immediately updated.
- Reduces performance impact during dense sequential updates

How does a table get to the point of having many small blocks? DELETES from this table can cause blocks to permanently shrink to a much smaller size unless a large amount of data is added again.

How have customers resolved this problem before Teradata 13.10?

- The ALTER TABLE command can be used to re-block a table. This technique can be time consuming and requires an exclusive table lock. This technique is still available with Teradata 13.10.
 - ALTER TABLE DATABLOCKSIZE = <value> IMMEDIATE

If this featured is enabled, the merge of small data blocks into a larger block runs automatically during full table SQL write operations. This feature can merge data blocks for the primary/fallback subtables and all of the index subtables. This feature runs automatically when the following SQL functions are executed.

- INSERT-SELECT, UPDATE-WHERE
- DELETE-WHERE (used on both permanent table and permanent journal datablocks)
- During the DELETE phase of Reconfig utility on source amps

Merge Datablocks

The "Merge Datablocks" feature (starting with TD 13.10) automatically searches for "small" data blocks within a table and will combine (merge) these small datablocks into a single larger block.

- Over time, modifications to a table (especially with DELETES of data rows) can result in a table having blocks that are less than 50% of the maximum datablock size.
- Up to 8 datablocks can be merged together.

If enabled, the merge of small data blocks into a larger block runs automatically during full table SQL write operations. This feature can merge datablocks for the primary/fallback subtables and all of the index subtables.

- INSERT-SELECT
- UPDATE-WHERE
- DELETE-WHERE

How have customers resolved this problem before Teradata 13.10?

- The ALTER TABLE command can be used to re-block a table. This technique can be time consuming and requires an exclusive table lock. This technique is still available with Teradata 13.10.

```
ALTER TABLE tablename, DATABLOCKSIZE = <value> IMMEDIATE;
```

Merge Datablocks (cont.)

How to use this Feature

Defaults for this feature can be set at the system level via DBSControl settings and can be overridden with table level attributes. The CREATE TABLE and ALTER TABLE commands have options to enable or disable this feature for a specific table.

The key parameter that controls this feature is **MergeBlockRatio**. This parameter can be set at the system level and also as a table level attribute.

MergeBlockRatio has the following characteristics:

- Limits the resulting size of a merged block.
- Reduces the chances that a merged block will split again soon after it is merged, defeating the feature's purpose.
- Computed as a percentage of the maximum multi-row data block size for the associated table.
- Candidate merged block must be smaller than this computed size after all target row updates are completed.
- Source blocks are counted up as eligible until the size limit is reached (zero to 8 blocks can be merged together).
- The default system level percentage is 60% and can be changed.

CREATE TABLE or ALTER TABLE options

- **DEFAULT MERGEBLOCKRATIO**
 - Default option on all CREATE TABLE statements
- **MERGEBLOCKRATIO = integer [PERCENT]**
 - Fixed MergeBlockRatio used for full table modification operations
 - Overrides the system default value
- **NO MERGEBLOCKRATIO**
 - Disables merges completely for the table

DBSControl FILESYS Group parameters

25. **DisableMergeBlocks (TRUE/FALSE, default FALSE)**
 - Disables feature completely across the system, even for tables with a defined MergeBlockRatio as a table level attribute.
 - Effective immediately – does not require a Teradata restart (tpareset)
26. **MergeBlockRatio (1-100%, default 60%)**
 - Default setting for any table – this can be overridden at the table level.
 - Ignored when DisableMergeBlocks is TRUE (FILESYS Flag #25)
 - This is not stored in or copied to table header
 - Effective immediately without a tpareset

Merge Datablocks (cont.)

This feature is automatically enabled for new Teradata 13.10 systems, but must be enabled for existing systems upgraded to 13.10.

Defaults for this feature are set via DBSControl settings.

- System defaults will work well for most tables.
- The CREATE TABLE and ALTER TABLE commands have options to enable/disable this feature for a specific table or change the default ratio.
- The key parameter is **MergeBlockRatio**.

MergeBlockRatio has the following characteristics:

- The default system level percentage is 60%.
- Computed as a percentage of the maximum multi-row datablock size for the associated table.
- Candidate merged block must be smaller than this computed size after all target row updates are completed.

CREATE TABLE or ALTER TABLE options

- **DEFAULT MERGEBLOCKRATIO**
 - Default option on all CREATE TABLE statements
- **MERGEBLOCKRATIO = integer [PERCENT]**
 - Fixed MergeBlockRatio used for full table modification operations
- **NO MERGEBLOCKRATIO**
 - Disables merges completely for the table

File System Write Summary

Regardless of how large your tables get, or how many SQL-based INSERTs, UPDATEs or DELETEs are executed, the process is the same. This module has discussed in some detail the sequence of steps that Teradata's file system software will attempt in order to complete the write operation.

The facing page summarizes some of the key topics discussed in this module.

File System Write Summary

Teradata's file system software automatically maintains the logical sequence of data rows within an AMP.

- The logical sequence is based on [tableid](#), [partition #](#), and [rowid](#).

For write (INSERT, UPDATE, or DELETE) operations:

- Read the Data Block if not present in memory.
- Place appropriate entries into the Transient Journal buffer (WAL buffer).
- [Make the changes to the Data Block in memory and determine the new block's length.](#)
- If the new block has changed size, [allocate a new Data Block.](#)

[Blocks will grow to the maximum data block size determined by the DATABLOCKSIZE table attribute, and then be split into smaller blocks.](#)

- Blocks will vary in size with Teradata.
- For a table that has been updated with "ad hoc" or TPump INSERTs, UPDATEs, or DELETEs, a typical block size for the table will be approximately 75% of the maximum data block size.

If the Write operation fails, the file system does a rollback using the Transient Journal.

Module 13: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 13: Review Questions

1. When Teradata INSERTs a new row into a table, it first goes to the _____ to locate the proper cylinder for the new row.
 - a. Cylinder Index
 - b. Fallback AMP
 - c. Free Cylinder List
 - d. Master Index
2. When a new block is needed, the File System searches the Free Block List looking for the first Free Block whose size is equal to, or greater than the new block's requirement. It does not have to be an exact match.
 - a. True
 - b. False
3. Name the condition which occurs when there is no block on the Free Block List with enough sectors to accommodate the additional data during an INSERT or UPDATE.
 - a. Mini Cylinder Pack
 - b. Cylinder Migrate to a new cylinder
 - c. Cylinder Migrate to an adjacent cylinder
 - d. Cylinder Full
4. The _____ parameter can be set to control how completely cylinders are filled during loading and PackDisk.
 - a. FreeSpace
 - b. PermDBSize
 - c. DataBlockSize
 - d. PermDBAllocUnit

Module 13: Review Questions (cont.)

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 13: Review Questions (cont.)

5. Number the following steps in sequence from 1 to 6 that the File System software will attempt to perform in order to insert a new row into an existing data block.

- _____ Perform a Cylinder Migrate operation to an adjacent cylinder
- _____ Simply insert the row into data block if enough contiguous free bytes in the block
- _____ Perform a Block split
- _____ Perform a Cylinder Migrate operation to a new cylinder
- _____ Defragment the block and insert the row
- _____ Expand or grow the block to hold the row

6. As part of a cylinder full condition, if the number of free sectors within a cylinder is greater than 25%, what operation will Teradata perform in the background? _____
7. If the number of free cylinders falls below a minimum threshold, what operation will Teradata perform in the background? _____

Notes



Partitioning a Table

After completing this module, you will be able to:

- Describe the components that comprise a Row ID in a row partitioned table.
- List two advantages of partitioning a table.
- List two potential disadvantages of partitioning a table.
- Create single-level and multilevel partitioned tables.
- Use the PARTITION key word to display partition information.
- Use ALTER TABLE to add and drop ranges from a row partitioned table.

Notes

Table of Contents

What is Row Partitioning?	14-4
How is Partitioning Implemented?.....	14-6
Logical Example of Row Partitioning.....	14-8
Primary Index Access (non-partitioned table)	14-10
Primary Index Access (row partitioned table)	14-12
Why Partition a Table?.....	14-14
Advantages/Disadvantages of Partitioning	14-16
Disadvantages of Partitioning	14-16
Partitioning Considerations.....	14-18
Access of Tables that are Partitioned	14-18
PARTITION BY Option	14-20
Partitioning with CASE_N and RANGE_N	14-22
Partitioning with RANGE_N – Example 1	14-24
Access using Partitioned Data – Example 1 (cont.).....	14-26
Access Using Primary Index – Example 1 (cont.).....	14-28
Place a USI on NUPI – Example 1 (cont.).....	14-30
Place a NUSI on NUPI – Example 1 (cont.).....	14-32
Partitioning with RANGE_N – Example 2.....	14-34
Partitioning – Example 3.....	14-36
Special Partitions with CASE_N and RANGE_N	14-38
Special Partition Examples	14-40
Partitioning with CASE_N – Example 4	14-42
Additional examples:	14-42
SQL Use of PARTITION Key Word.....	14-44
SQL Use of CASE_N	14-46
Using ALTER TABLE with Row Partitioned Tables.....	14-48
ALTER TABLE – NO RANGE is <u>not</u> Defined.....	14-50
ALTER TABLE – NO RANGE is Defined.....	14-52
ALTER TABLE – Example 5.....	14-54
ALTER TABLE TO CURRENT	14-56
ALTER TABLE TO CURRENT – Example 6.....	14-58
Row Partitioning Enhancements	14-60
Multilevel Partitioning Concepts	14-62
Multilevel Partitioning Concepts (cont.).....	14-64
Multilevel Partitioning – Example 7	14-66
Multilevel Partitioning – Example 7 (cont.)	14-68
How is a ML Partition # Calculated?	14-70
ADD Partitions Option – Example 8	14-72
Character Partitioning	14-74
Character Partitioning – Example 9	14-76
Summary	14-78
Module 14: Review Questions.....	14-80
Lab Exercise 14-1	14-84

What is Row Partitioning?

As part of implementing a physical design, Teradata provides numerous indexing options that can improve performance for different types of queries and workloads. For example, secondary indexes, join indexes, or hash indexes may be utilized to improve performance for known queries. Teradata provides additional new indexing options to provide even more flexibility in implementing a Teradata database. One of these new indexing options is the ability to row partition a table. This feature was initially known as Partitioned Primary Index (PPI). Key characteristics of row partitioned tables are listed on the facing page.

Primary indexes can be partitioned or non-partitioned. A non-partitioned table (a.k.a., (no-partitioned primary index – NPPI) is the traditional primary index by which rows are assigned to AMPs. Apart from maintaining their storage in row hash order, no additional assignment processing of rows is performed once they are hashed to an AMP.

A row partitioned table permits rows to be assigned to user-defined data partitions on the AMPs, enabling enhanced performance for range queries that are predicated on primary index values.

The row partitioning feature allows a class of queries to access a portion of a large table, instead of the whole table. The traditional uses of the Primary Index (PI) for data placement and rapid access of the data when the PI values are specified are retained.

Some common business queries generally require a full-table scan of a large table, even though it's predictable that a fairly small percentage of the rows will qualify. One example of such a query is a trend analysis application that compares current month sales to the previous month, or to the same month of the previous year, using a table with several years of sales detail. Another example is an application that compares customer behavior in one (fairly small) geographic region to another region.

Acronyms that may be used in this module:

- PI – Primary Index
- PPI – Partitioned Primary Index
- NPPI – Non-Partitioned Primary Index
- RP – Row Partitioned table

What is Row Partitioning?

What is Row Partitioning – a.k.a., Partitioned Primary Index (PPI)?

- A indexing mechanism in Teradata for use in physical database design.
- Data rows are grouped into partitions at the AMP level – partitioning is simply an ordering of the rows within a table on an AMP.

What advantages does partitioning provide?

- Increases the available options to improve the performance of certain types of queries – specifically range-constrained queries.
- Only the rows of the qualified partitions in a query need to be accessed – avoid full table scans.

How is a row partitioning created and managed?

- Row partitioning is easy to create and manage.
 - The CREATE TABLE and ALTER TABLE statements contain options to create and/or alter partitions.
- As always, data is distributed among AMPs and automatically placed within partitions.

How is Partitioning Implemented?

The PRIMARY INDEX clause (part of the CREATE TABLE statement) has been extended to include a PARTITION BY clause. This new partition expression definition is the only thing that needs to be done to create a partitioned table. Advantages to this approach are:

- No separate partition layout
- No disk layout for partitions
- No definition of location in the system for partition
- No need to define/manage separate tables per segment of the table that needs to be accessed
- Even data distribution and even processing of a logical partition is automatic due to the PI distribution of the rows

No query has to be modified to take advantage of a row partitioned table.

For tables that are row partitioned, Teradata utilizes a 3-level scheme to distribute and later locate the data. The 3 levels are:

- Rows are distributed across all AMPs (and accessed via the Primary Index) based upon HBN (Hash Bucket Number) portion of the Row Hash.
- At the AMP level, rows are first ordered by their partition number.
- Within the partition, data rows are logically stored in Row ID sequence.

A new term is associated with row partitioned tables. The **Row Key** is a combination of the Partition # and the Row Hash. The term Row Key will appear in EXPLAIN reports.

How is Partitioning Implemented?

Provides an additional level of data distribution and ordering.

- Rows are distributed between the AMPs based on the Primary Index value.
- At the AMP level and within the table, rows are first ordered by their partition number.
- Within the partition, data rows are logically stored in Row ID sequence.

If a table is partitioned, rows are placed into partitions.

- Maximum combined partitions is 9.223 Quintillion.
 - If combined partitions is $\leq 65,535$, then **2-byte** partition numbers are used.
 - If combined partitions is $> 65,535$, then **8-byte** partition numbers are used.

In a partitioned table, each row is uniquely identified by the following:

- **Row ID = Partition # + Row Hash + Uniqueness Value**
- **Row Key = Partition # + Row Hash** (e.g., Row Key will appear in Explain plans)
 - In a partitioned table, data rows will have the Partition # included as part of the data row.

To help understand how partitioning is implemented, this presentation will include examples of data access using **non-partitioned tables** and **row partitioned tables**.

Logical Example of Row Partitioning

The facing page provides a logical example of an Orders table implemented with a NPPI (Non-Partitioned Primary Index) and the same table implemented with row partitioning (or PPI – Partitioned Primary Index). Only the Order# and the OrderDate are shown as user columns in the example.

The column headings in this example represent the following:

RH – Row Hash – the two-digit row hash is used for simplification purposes. A true table would contain a Row ID for each row (Row Hash + Uniqueness Value). Note that as just in a real implementation, two different order numbers happen to hash to the same row hash value. Order numbers 1012 and 1043 on AMP 2 both hash to '36'.

Order# – this example assumes that Order Number is the Primary Index and the data rows are hash distributed based on this value.

Order Date – the order date for each order. This example only illustrates contains orders for 4 days.

Important points to understand from this example:

- All of the rows in the NPPI table are stored in logical Row ID sequence (row hash + uniqueness value) within each AMP.
- The rows in the row partitioned table are first ordered by Partition Number, and then by Row Hash (actually Row ID) sequence within the Partition.
- This example illustrates 4 partitions – one for each of the 4 days shown in the example.
- The first query that requests “order information” for a specific date – WHERE condition that specifies a specific date.
 - A FTS is needed for the non-partitioned table
 - A partition scan is needed for the partitioned table. Only one partition is accessed.
- The second query requests "order information" for a specific order number (e.g., #1043).
 - Primary Index access is needed for the non-partitioned table – one I/O is required.
 - A look-up in each partition is needed for the partitioned table – one I/O per partition.

Logical Example of Row Partitioning

Orders table defined without Row Partitioning across 4 AMPs.

RH	Order#	OrderDate	RH	Order#	OrderDate	RH	Order#	OrderDate	RH	Order#	OrderDate
'01'	1028	2019/01/03	'06'	1009	2019/01/01	'04'	1008	2019/01/01	'02'	1024	2019/01/02
'03'	1016	2019/01/02	'07'	1017	2019/01/02	'05'	1048	2019/01/04	'08'	1006	2019/01/01
'12'	1031	2019/01/03	'10'	1034	2019/01/03	'09'	1018	2019/01/02	'11'	1019	2019/01/02
'14'	1001	2019/01/01	'13'	1037	2019/01/04	'15'	1042	2019/01/04	'18'	1041	2019/01/04
'17'	1013	2019/01/02	'16'	1021	2019/01/02	'19'	1025	2019/01/03	'20'	1005	2019/01/01
'23'	1040	2019/01/04	'21'	1045	2019/01/04	'24'	1004	2019/01/01	'22'	1020	2019/01/02
'28'	1032	2019/01/03	'26'	1002	2019/01/01	'27'	1014	2019/01/02	'25'	1036	2019/01/03
'30'	1038	2019/01/04	'29'	1033	2019/01/03	'32'	1003	2019/01/01	'31'	1026	2019/01/03
'35'	1007	2019/01/01	'34'	1029	2019/01/03	'33'	1039	2019/01/04	'38'	1046	2019/01/04
'39'	1011	2019/01/01	'36'	1012	2019/01/01	'40'	1035	2019/01/03	'41'	1044	2019/01/04
'42'	1047	2019/01/04	'36'	1043	2019/01/04	'44'	1022	2019/01/02	'43'	1010	2019/01/01
'48'	1023	2019/01/02	'45'	1015	2019/01/02	'47'	1027	2019/01/03	'46'	1030	2019/01/03

Orders table defined with Row Partitioning on OrderDate across 4 AMPs.

RH	Order#	OrderDate	RH	Order#	OrderDate	RH	Order#	OrderDate	RH	Order#	OrderDate
'14'	1001	2019/01/01	'06'	1009	2019/01/01	'04'	1008	2019/01/01	'08'	1006	2019/01/01
'35'	1007	2019/01/01	'26'	1002	2019/01/01	'24'	1004	2019/01/01	'20'	1005	2019/01/01
'39'	1011	2019/01/01	'36'	1012	2019/01/01	'32'	1003	2019/01/01	'43'	1010	2019/01/01
'03'	1016	2019/01/02	'07'	1017	2019/01/02	'09'	1018	2019/01/02	'02'	1024	2019/01/02
'17'	1013	2019/01/02	'16'	1021	2019/01/02	'27'	1014	2019/01/02	'11'	1019	2019/01/02
'48'	1023	2019/01/02	'45'	1015	2019/01/02	'44'	1022	2019/01/02	'22'	1020	2019/01/02
'01'	1028	2019/01/03	'10'	1034	2019/01/03	'19'	1025	2019/01/03	'25'	1036	2019/01/03
'12'	1031	2019/01/03	'29'	1033	2019/01/03	'40'	1035	2019/01/03	'31'	1026	2019/01/03
'28'	1032	2019/01/03	'34'	1029	2019/01/03	'47'	1027	2019/01/03	'46'	1030	2019/01/03
'23'	1040	2019/01/04	'13'	1037	2019/01/04	'05'	1048	2019/01/04	'18'	1041	2019/01/04
'30'	1038	2019/01/04	'21'	1045	2019/01/04	'15'	1042	2019/01/04	'38'	1046	2019/01/04
'42'	1047	2019/01/04	'36'	1043	2019/01/04	'33'	1039	2019/01/04	'41'	1044	2019/01/04

SELECT ...
WHERE OrderDate =
DATE '2019-01-03';

SELECT ...
WHERE Order# = 1043;

Primary Index Access (non-partitioned table)

A non-partitioned table has a traditional primary index by which rows are assigned to AMPs. Apart from maintaining their storage in row hash order, no additional assignment processing of rows is performed once they are hashed to an AMP.

With a non-partitioned table, the PARSER will include Partition Number 0 in the request. For a non-partitioned table, all of the rows are assumed to be part of one partition (Partition 0).

Assuming that an SQL statement (e.g., SELECT) provides equality value(s) to the column(s) of a Primary Index, the TD Database software retrieves the row or rows from a single AMP as described below.

The Parsing Engine (PE) creates a four-part message composed of the Table ID, Partition #0, the Row Hash, and Primary Index value(s). The 48-bit Table ID is located via the Data Dictionary, the 32 bit Row Hash value is generated by the Hashing Algorithm, and the Primary Index value(s) come from the SQL request. The Parsing Engine (via the Data Dictionary) knows if a table has a NPPI and sets the Partition Number to 0.

The Message Passing Layer uses a portion of the Row Hash to determine to which AMP to send the request. The Message Passing Layer uses the HBN portion of the Row Hash (first 20 bits of the Row Hash) to locate a bucket in the **Hash Map(s)**. This bucket identifies to which AMP the PE will send the request. The Hash Maps are part of the Message Passing Layer interface.

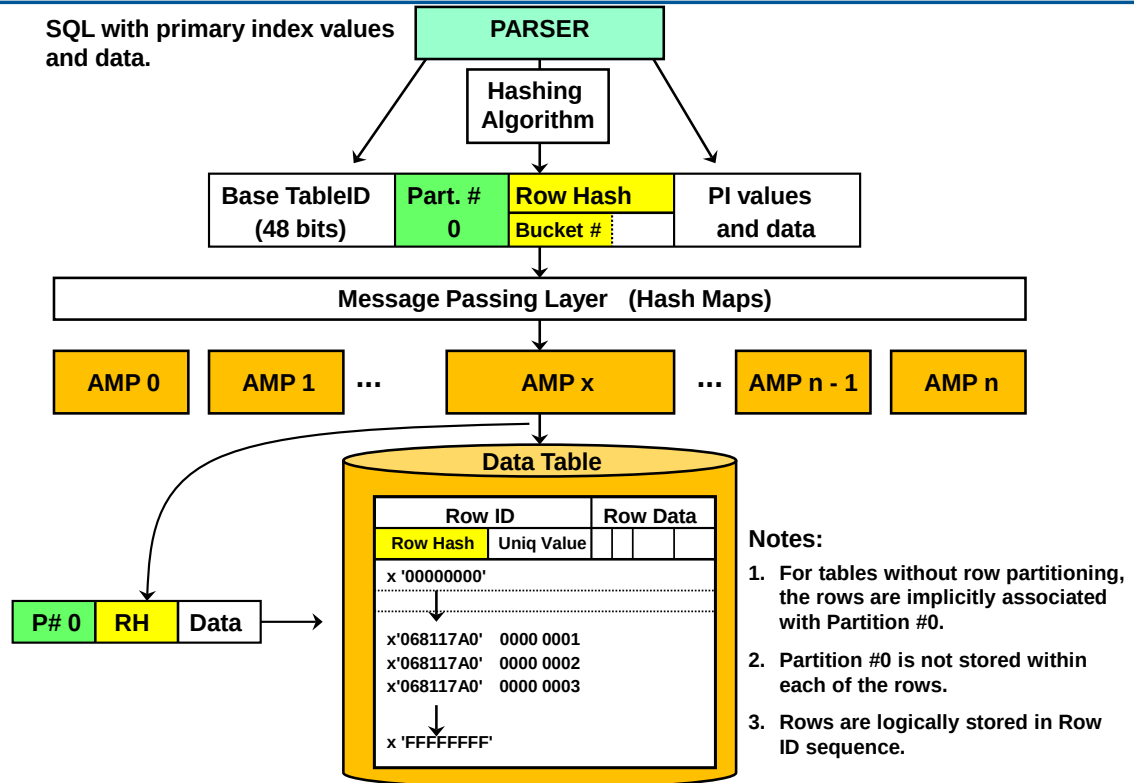
The AMP uses the Table ID and Row Hash to identify and locate the proper data block, then uses the Row Hash and PI value to locate the specific row(s). The PI value is required to distinguish between Hash Synonyms. The AMP implicitly assumes the rows are part of partition #0.

Note: The Partition Number (effectively 0) is not stored within the data rows for a non-partitioned table. The FLAG or SPARE byte (within the row overhead) has a bit set to zero for rows in non-partitioned tables. A FLAG bit is set for to indicate is 2 bytes or 8 bytes are used to for the partition number within the physical data row.

Acronyms:

- HBN – Hash Bucket Number
- PPI – Partitioned Primary Index
- NPPI – Non-Partitioned Primary Index

Primary Index Access (non-partitioned table)



Primary Index Access (row partitioned table)

The process to locate a data row(s) via in a row partitioned table is similar to the process in retrieving data rows with a table without row partitioning – a process described earlier. If the SQL request provides data about columns associated with the partitions, then the PARSER will include specific partition information in the request.

- The key to remember is that a specific Row Hash value can be found in different partitions on the AMP. The Partition Number, Row Hash, and Uniqueness Value are needed to uniquely identify a row in a PPI-based table.
- A Row Hash and Uniqueness Value combination is only unique within a partition of a row partitioned table. The same Row Hash and Uniqueness Value combination can be present in different partitions (e.g., x'068117A0').

You can have the same hash and uniqueness values in different partitions. **The uniqueness value within a partition is unique, not across partitions.** When a row hash is added to a partition, the uniqueness value is incremented by 1 and Teradata only looks at that partition. The important concept to understand is that for a partitioned table, each row is uniquely identified by the combination of **part#, row hash, and uniqueness value**.

Assume a row partitioned table with a NUPI. In this case, assume the NUPI is (orderid) and the table has RANGE_N partitioning by orderdate. Even though the values of orderid may be unique, you cannot create the table with orderid as a UPI. The reason is this – if Teradata allowed you to define orderid as unique, then each time you inserted a new order, Teradata would have to check every partition to make sure the new orderid did not already exist and this would kill performance (lots of I/O to look in every partition).

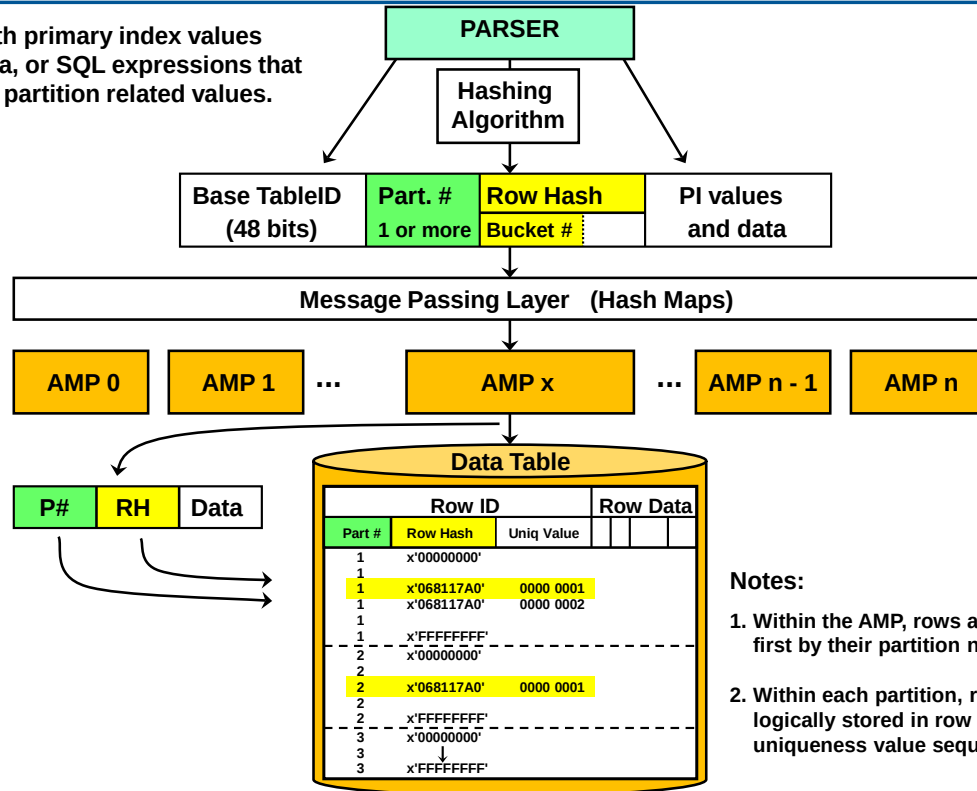
Assuming that an SQL statement (e.g., SELECT) provides equality value(s) to the Primary Index, then Teradata software retrieves the row(s) from a single AMP.

- If the SQL request also provides data for partition columns, then the AMP will only have to access the partition(s) identified in the request sent to it by the PE.
- If the SQL request only provides Primary Index values and the partitioning columns are outside of the Primary Index (and partitioning information is not included in the SQL request), the AMP will check each of the Partitions for the associated Row Hash.

In the general case, partitions are logically numbered – starting from one. As rows are inserted into the table, the partitioning expression is evaluated to determine the proper partition placement for that row. A two-byte or eight-byte partition number is embedded in the row, as part of the row identifier, making partitioned rows two or eight bytes wider than they would be if the table wasn't partitioned.

Primary Index Access (row partitioned table)

SQL with primary index values and data, or SQL expressions that include partition related values.



Why Partition a Table?

One key reason to partition a table is to reduce the number of full table scans of the table. To determine which columns to use for the Primary Index and for the partitioning depends on how its rows are most frequently accessed. Row partitioned tables are designed to optimize range queries while also providing efficient primary index join strategies. For range queries, only rows of the qualified partitions need to be accessed.

- One of the reasons to define row partitioning on a table is to increase query efficiency by avoiding full table scans without the overhead and maintenance costs of secondary indexes.

The facing page provides one example using a sales data table that has 2 years of sales history. This sales table will have data organized into 730 partitions (one for each day of the 2 years).

Queries that request a subset of the data (some number of months) only need to access the required partitions instead of the entire table. For example, a query that requests two days of sales data only needs to read 2 partitions of the data from each AMP. This is about 1/365 of the table. Without row partitioning or any secondary indexes, this query has to perform a full table scan.

The more partitions there are, the greater the potential benefit.

Some of the performance opportunities available by using row partitioning:

- Get more efficiency in querying against a subset of large volumes of transactional detail data as well as to manage this data more effectively.
 - Businesses have recognized the analytic value of detailed transactions and are storing larger and larger volumes of this data.
 - Increase query efficiency by avoiding full table scans without the overhead and maintenance costs of secondary indexes.
 - As the retention volume of detailed transactions increases, the percent of transactions that an “average” query requires for execution decreases.
- Allow “instantaneous” dropping of “old” data and simple addition of “new” data.
 - Support a “rolling n periods” methodology for transactional data.

The term “**partition elimination**” refers to an automatic optimization in which the optimizer determines, based on query conditions, that some partitions can't contain qualifying rows, and causes those partitions to be skipped. Partitions that are skipped for a particular query are called excluded partitions. Generally, the greatest benefit of a row partitioned table is obtained from partition elimination.

Why Partition a Table?

- Increase query efficiency by avoiding full table scans without the overhead and maintenance costs of secondary indexes.
 - Partition Elimination – the key advantage to partitioning a table is that the optimizer can eliminate partitions for queries.
- For example, assume a sales data table has 2 years of sales history.
 - Assume this table is row partitioned into 730 partitions – one for each day of 2 years.
 - Assume a query only needs to read 2 days of the data from each AMP (comparing sales of one day this year to the same day last year).
 - Only 2/730 or 1/365 (2 partitions) of the table has to be read.
 - With a non-partitioned table, this query has to perform a full table scan.
 - A Valued-Ordered NUSI may be used to help performance for this type of query.
 - However, there is NUSI subtable permanent space and maintenance overhead.
- Deleting large volumes of rows in entire partitions can be extremely fast.
 - ALTER TABLE ... DROP RANGE ... ;
 - Disclaimer: Fast deletes assume that the table doesn't have a NO RANGE partition defined and has no secondary indexes, join indexes, or hash indexes.

Advantages/Disadvantages of Partitioning

The main advantage of a PPI table is the automatic optimization that occurs for queries that specify a restrictive condition on the partitioning column. For example, a query which examines two months of sales data in a table with two years of sales history can read about one-twelfth of the table, instead of the entire table. The more partitions there are, the greater the potential benefit.

Disadvantages of Partitioning

The two main potential disadvantages of using a PPI table occur with PI access and direct PI-based joins. The PI access potential disadvantage occurs only when the partitioning column is not part of the PI. In this situation, a query specifying a PI value, but no value for the partitioning column, must look in each partition for that value, instead of positioning directly to the first row for the PI value.

The direct join potential disadvantage occurs when another table with the same PI is joined with an equality condition on every PI column. For two non-PPI tables, the rows of the two tables will be ordered the same, and the join can be performed directly. If one of the tables is partitioned, the rows won't be ordered the same, and the task, in effect, becomes a set of sub-joins, one for each partition of the PPI table.

In both of these situations, the disadvantage is proportional to the number of partitions, with fewer partitions being better than more partitions.

When the primary index is unique (but can't be defined as unique because of the partitioning), a USI or NUSI can be defined on the same columns as the primary index. Access via the secondary index won't be as fast as non-partitioned access via the primary index, but is fast enough for most applications.

Why can't a Primary Index be defined as Unique unless the partitioning expression columns are part of the PI column(s)?

It's because of the difficulty of performing the duplicate PI check for inserts. If there was already a row with that PI, it could be in any partition, so every partition would have to be checked to determine whether the duplicate PI exists. There can be thousands of partitions. An insert-select could take a very long time in such a situation. It's more efficient to check uniqueness (and it also provides an efficient access path) to define a unique secondary index (USI) on the same columns as the PI in this case.

Advantages/Disadvantages of Partitioning

Advantages:

- **The partition expression definition is the only thing that needs to be done by the DBA or the database designer.** No separate partition layout – no disk layout for partitions.
 - For example, the last row in one partition and the first row in the next partition will usually be in the same data block.
 - No definition of location in the system for partitions.
- Even data distribution and even processing of a logical partition is automatic.
 - Due to the PI distribution of the rows
- **No modifications of queries required.**

Potential disadvantages:

- Partitioned rows are **2 or 8 bytes longer**. Table uses more PERM space.
 - Secondary index subtable rows are also increased in size.
- **A PI access may be degraded** if the partitioning column is not part of the PI.
 - A query specifying only a PI value must look in each partition for that value.
- **Joins to non-partitioned tables with the same PI** may be degraded.
- The PI can't be defined as unique when the partitioning column is not part of the PI.

Partitioning Considerations

Base tables, non-compressed join indexes, global temporary tables, volatile temporary tables can all be partitioned. This restriction doesn't mean that a PPI table can't have secondary indexes, or can't be referenced in the definition of Hash Index. It merely means that the PARTITION BY clause is not available on a CREATE HASH INDEX statement.

If the table definition has less 65,535 partitions defined, then a two-byte partition number is embedded in the data row, as part of the row identifier. Secondary Indexes and Join Indexes referencing partitioned tables also use the wider row identifier. Except for the embedded partition number, partitioned table rows have the same format as non-partitioned table rows. A data block can contain rows from more than one partition. There are no new control structures needed to implement the partitioning scheme.

The key guideline for determining the optimum granularity for the partitions of a partitioned table or join index is the nature of the workloads that most commonly access the object. The higher the number of partitions you define for an object, the more likely an appropriate range query against the table will perform more quickly, given that the partition granularity is such that the Optimizer can eliminate all but one partition or a few partitions.

On the other hand, it is generally best to avoid specifying too fine a partition granularity. For example, if query workloads never access data at a granularity of less than one month, there is no benefit to be gained by defining partitions with a granularity of less than one month.

Access of Tables that are Partitioned

Some of the issues associated with accessing a row partitioned table are listed below:

- If the SELECT statement does not provide values for any of the partitioning columns, then all of the partitions may be probed to find row(s) with the hash value.
- If the SELECT statement provides values for some of the partitioning columns, then partition elimination may reduce the number of the partitions that will be probed to find row(s) with the hash value.
- If the SELECT statement provides values for all of the partitioning columns, then partition elimination will cause a single partition to be probed to find row(s) with the hash value.

In summary, a NUPI access of a row partitioned table will take longer when a query specifies the PI column values, but doesn't include the partitioning column(s). In this situation, each partition must be probed for the appropriate PI value. In the worst case, the number of disk reads could increase by a factor equal to the number of partitions. While probing a partition is a fast operation, a table with thousands of partitions might not provide acceptable performance for PI accesses for some applications (tactical queries).

Partitioning Considerations

Considerations include ...

- Base tables are partitioned, secondary indexes are not.
- However, a row partitioned table can have secondary indexes which reference rows in a row partitioned table via a RowID in the SI subtable.
 - Global and Volatile Temporary Tables can also be partitioned.
 - Non-Compressed Join Indexes can also be partitioned.
- A join or hash index can also reference rows in a row partitioned table.

A table has a maximum of 9.223 Quintillion partitions.

- Partitioning columns do not have to be columns in the primary index.
- There are numerous options for partitioning.

As rows are inserted into the table, the partitioning expression is evaluated to determine the proper partition placement for that row.

PARTITION BY Option

Primary indexes can be partitioned or non-partitioned. A primary index is defined as part of the CREATE TABLE statement. The PRIMARY INDEX definition has a new option to create partitioned primary indexes.

PARTITION BY *<partitioning expression>*

A partitioned primary index (PPI) permits rows to be assigned to user-defined data partitions on the AMPs, enabling enhanced performance for range queries that are predicated on partitioning columns(s) values. The *<partitioning expression>* is evaluated and Teradata determines the appropriate partition number or assignment.

The *<partitioning-expression>* is a general expression, allowing wide flexibility in tailoring the partitioning scheme to the unique characteristics of the table. Two functions, CASE_N and RANGE_N, are provided to simplify the creation of common partitioning schemes. You can write any valid SQL expression as a partitioning expression with a few exceptions. The reference manual has details on SQL expressions that are not permitted in the *<partitioning expression>*.

Limitations on PARTITION BY option include:

- Partitioning expression must be a scalar expression that is INTEGER or can be cast to INTEGER.
- Multiple columns from the table may be specified in the expression
 - These are called the partitioning columns.
- Before Teradata 13.10, the expression must not require character/graphic comparison in order to be evaluated.
 - Expression must not contain aggregate/ordered-analytic/statistical functions, DATE/, TIME, ACCOUNT, RANDOM, HASH, etc. functions.
- UNIQUE only allowed if all partitioning columns are included in the PI.
- Partitioning expression limited to approximately 16,000 characters.
 - Stored as an implicit check constraint in DBC.TableConstraints

One or more columns can make up the partitioning expression, although it is anticipated that for most tables one column will be specified. The partitioning column(s) can be part of the primary index, but are not required to be. The result of the partitioning expression must be a scalar value that is INTEGER or can be cast to INTEGER. Most deterministic functions can be used within the expression. The expression must not require character or graphic comparisons, although character or graphic columns can be used in some circumstances.

PARTITION BY Option

The PRIMARY INDEX definition portion of a CREATE TABLE statement has a optional **PARTITION BY** option.

```
CREATE TABLE ...  
    [UNIQUE] PRIMARY INDEX (col1, col2, ...)  
        PARTITION BY <partitioning-expression>
```

Options for the *<partitioning-expression>* include:

- Range partitioning
- Conditional partitioning, modulo partitioning, and general expression partitioning.
- Partitioning columns do not have to be columns in the primary index. If they aren't, then the primary index cannot be unique.

Column(s) included in the partitioning expression are called the “**partitioning column(s)**”.

- Two functions, **CASE_N** and **RANGE_N**, are provided to simplify the creation of common partitioning schemes.

Partitioning with CASE_N and RANGE_N

For many tables, there is no suitable column that lends itself to direct usage as a partitioning column. For these situations, the CASE_N and RANGE_N functions can be used to concisely define partitioning expressions.

The PARTITION BY phrase requires a partitioning expression that determines the partition assignment of a row. You can use the CASE_N function to construct a partitioning expression such that a row with any value or NULL for the partitioning column is assigned to a partition.

The CASE_N function is patterned after the SQL CASE expression. It evaluates a list of conditions and returns the position of the first condition that evaluates to TRUE, provided that no prior condition in the list evaluates to UNKNOWN. The returned value will map directly into a partition number.

Another option is to use the RANGE_N function to construct a partitioning expression with a list of ranges such that a row with any value or NULL for the partitioning column is assigned to a partition.

The **<partitioning expression>** may use one of the following functions to help define partitions.

- **CASE_N**
- **RANGE_N**

Use of CASE_N results in the following:

- **Evaluates a list of conditions** and returns the position of the first condition that evaluates to TRUE.
- Result is the data row being placed into a partition associated with that condition.
- Note: Patterned after SQL CASE expression.

Use of RANGE_N results in the following:

- The expression is evaluated and is **mapped into one of a list of specified ranges**.
- Ranges are listed in increasing order and must not overlap with each other.
- Result is the data row being placed into a partition associated with that range.

NO CASE, NO RANGE, and UNKNOWN options are also available.

Partitioning with RANGE_N – Example 1

One of most common partitioning expression is to use RANGE_N partitioning to partition the table based on a group of dates (e.g., month partitions). A range is defined by a starting boundary and an optional ending boundary. If an ending boundary is not specified, the range is defined by its starting boundary, inclusively, up to but not including the starting boundary of the next range.

The list of ranges must specify ranges in increasing order, where the ending boundary of a range is less than the starting boundary of the next range.

RANGE_N limitations include:

- Multiple test values are not allowed in a RANGE_N function.
- Range value and range size in a RANGE_N function must be constant.
- Ascending ranges only and ranges must not overlap with each other.

For example, the following CREATE TABLE statement can be used to establish the monthly partitioning.

```
CREATE SET TABLE Claim_RP  
  (claimid          INTEGER    NOT NULL  
  ,custid          INTEGER    NOT NULL  
  ,claimdate       DATE       NOT NULL  
  )  
PRIMARY INDEX (claimid)  
  PARTITION BY RANGE_N (claimdate  
    BETWEEN DATE '2010-01-01' AND DATE '2019-12-31'  
    EACH INTERVAL '1' MONTH, NO RANGE);
```

This definition (different than the facing page) creates a day with 121 logical partitions. To maintain uniqueness on the claimid, you can include a USI on claimid by including the following option.

```
UNIQUE INDEX (claimid)
```

If the claimdate column for an attempted INSERT or UPDATE has a date outside of the partitioning range or NULL, then an error will be returned and the row won't be inserted or updated.

Notes:

- UPI not allowed because partitioning column is not included in the PI.
- Unique Secondary Index is allowed on PI to enforce uniqueness.

The facing page contains examples of inserting data rows into a table partitioned by day and how the date is evaluated into the appropriate partition.

Partitioning with RANGE_N – Example 1

For example, partition a Claim table by "claimdate".

```
CREATE TABLE Claim_RP
( claimid      INTEGER      NOT NULL
  ,custid      INTEGER      NOT NULL
  ,claimdate   DATE         NOT NULL
  ... )
PRIMARY INDEX (claimid)
```

```
PARTITION BY RANGE_N (claimdate BETWEEN
  DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' DAY,
  NO RANGE);
```

This table has 3653 logically defined partitions. As INSERTs place new rows into the Claim table, the date is evaluated and the rows are placed into the appropriate partitions.

```
INSERT INTO Claim_RP VALUES (100039,1009, '2010-01-13',...); placed in partition #13
INSERT INTO Claim_RP VALUES (427067,1020, '2019-01-13',...); placed in partition #3300
INSERT INTO Claim_RP VALUES (950221,1020, '2020-01-01',...); placed in partition #3653
INSERT INTO Claim_RP VALUES (100039,1009, NULL,...); Error 3811 – NOT NULL violation
```

If the table did not have the NO RANGE partition defined, then the following error occurs:

```
INSERT INTO Claim_RP VALUES (100391,1009, '2020-01-01',...);
(5728 – Partitioning violation)
```

Note: claimid must be defined as a NUPI because claimdate is not part of PI.

Access using Partitioned Data – Example 1 (cont.)

Both Claim_RP and Claim tables have 30,000,000 rows. Statistics for custid and claimdate were collected on both tables. In addition, claimid and PARTITION statistics were collected on the Claim_RP table.

The EXPLAIN text for these queries is shown below.

```
EXPLAIN  SELECT  *
          FROM    Claim_RP
          WHERE    claimdate = DATE '2019-01-13';
```

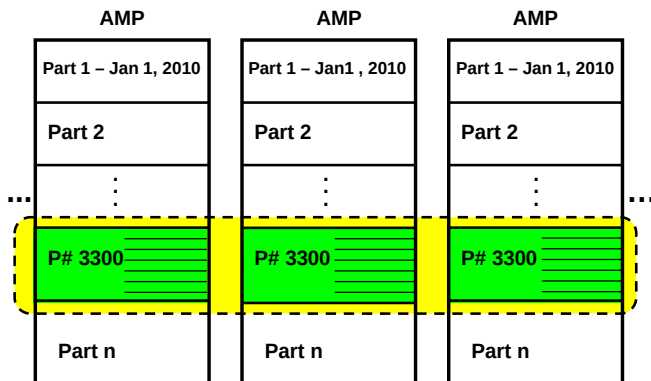
- 1) First, we lock TFACT.Claim_RP in TD_MAP1 for read on a reserved RowHash in a single partition to prevent global deadlock.
 - 2) Next, **we lock TFACT.Claim_RP in TD_MAP1 for read on a single partition.**
 - 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from a single partition of TFACT.Claim_RP with a condition of ("TFACT.Claim_RP.claimdate = DATE '2019-01-13'") with a residual condition of ("TFACT.Claim_RP.claimdate = DATE '2019-01-13'") into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 17,293 rows (2,472,899 bytes). The estimated time for this step is 0.04 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.04 seconds.**

The table named Claim is similar to Claim_RP except it does not have a Partitioned Primary Index, but does have “claimid” as a UPI.

```
EXPLAIN  SELECT  *
          FROM    Claim
          WHERE    claimdate = DATE '2018-01-13';
```

- 1) First, we lock TFACT.Claim in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, **we lock TFACT.Claim in TD_MAP1 for read.**
 - 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.Claim by way of an all-rows scan with a condition of ("TFACT.Claim.claimdate = DATE '2019-01-13'") into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 17,293 rows (2,472,899 bytes). The estimated time for this step is 16.11 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 16.11 seconds.**

Access using Partitioned Data – Example 1 (cont.)



QUERY – row partitioned

```
SELECT *
FROM   Claim_RP
WHERE  claimdate =
       DATE '2019-01-13'
```

Explain plan

ALL-AMPs – Single Partition Scan with a Partition Read Lock. This date (partition) has 17,293 rows.

EXPLAIN estimated cost – 0.04 sec.

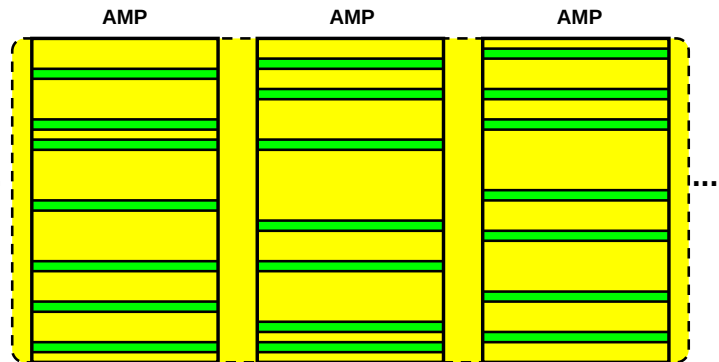
QUERY – not row partitioned

```
SELECT *
FROM   Claim
WHERE  claimdate =
       DATE '2019-01-13';
```

Explain plan

ALL-AMPs – Full Table Scan with a Table Read Lock – table has 30M rows.

EXPLAIN estimated cost – 16.11 sec.



Access Using Primary Index – Example 1 (cont.)

The EXPLAIN text for these queries is shown below.

```
EXPLAIN  SELECT  *
          FROM    Claim_RP
          WHERE    claimid = 1293751;
```

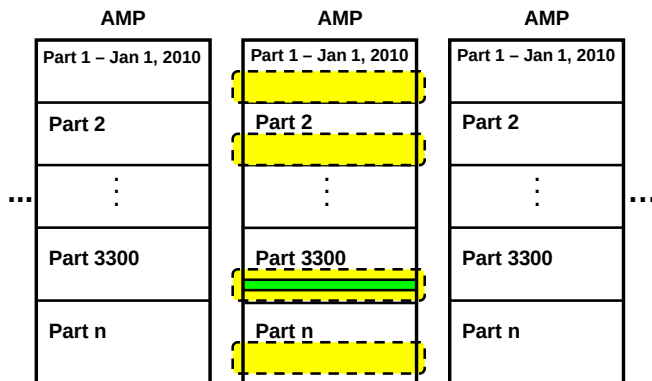
- 1) First, we do a single-AMP RETRIEVE step from all partitions of TFACT.Claim_RP by way of the primary index "TFACT.Claim_RP.claimid = 1293751" with a residual condition of ("TFACT.Claim_RP.claimid = 1293751") into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with high confidence to be 1 row (143 bytes). The estimated time for this step is 0.82 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.82 seconds.**

The table named Claim is similar to Claim_RP except it does not have a Partitioned Primary Index, but does have "claimid" as a UPI.

```
EXPLAIN  SELECT  *
          FROM    Claim
          WHERE    claimid = 1293751;
```

- 1) First, we do a **single-AMP RETRIEVE step** from TFACT.Claim by way of the unique primary index "TFACT.Claim.claimid = 1293751" with no residual conditions. The estimated time for this step is 0.00 seconds.
- > The row is sent directly back to the user as the result of statement 1. **The total estimated time is 0.00 seconds.**

Access Using Primary Index – Example 1 (cont.)



QUERY – row partitioned

```
SELECT  *
FROM    Claim_RP
WHERE   claimid = 1293751;
```

Explain plan

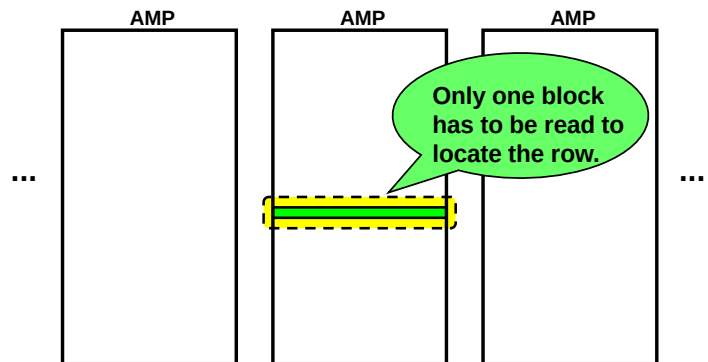
One AMP – All Partitions are probed
EXPLAIN estimated cost – 0.82 sec.

QUERY – not row partitioned

```
SELECT  *
FROM    Claim
WHERE   claimid = 1293751;
```

Explain plan

One AMP – UPI Access
EXPLAIN estimated cost – 0.00 sec.



Place a USI on NUPI – Example 1 (cont.)

If the partitioning columns are not part of the Primary Index, the Primary Index cannot be unique (e.g., claimdate). To maintain uniqueness on the Primary Index, you can create a USI on the PI (e.g., claimid).

Reasons for this may include:

- USI access to specific rows may be faster than scanning multiple partitions on a single AMP.
- Establish the USI as a referenced parent in Referential Integrity.

```
CREATE UNIQUE INDEX (claimid) ON Claim_RP;
```

```
EXPLAIN      SELECT      *  
              FROM        Claim_RP  
              WHERE       claimid = 1293751;
```

- 1) First, we do a **two-AMP RETRIEVE** step in TD_MAP1 from TFACT.Claim_RP by way of **unique index # 4** "TFACT.Claim_RP.claimid = 1293751" with no residual conditions. The estimated time for this step is 0.00 seconds.
- > The row is sent directly back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

As an alternative, the SELECT can include the Primary Index values and the partitioning information. If you knew that the claimid of 1293751 occurred on 2018-01-13, you could include both claimid and claimdate in the query. This allows the PE to build a request that has the AMP scan a specific partition.

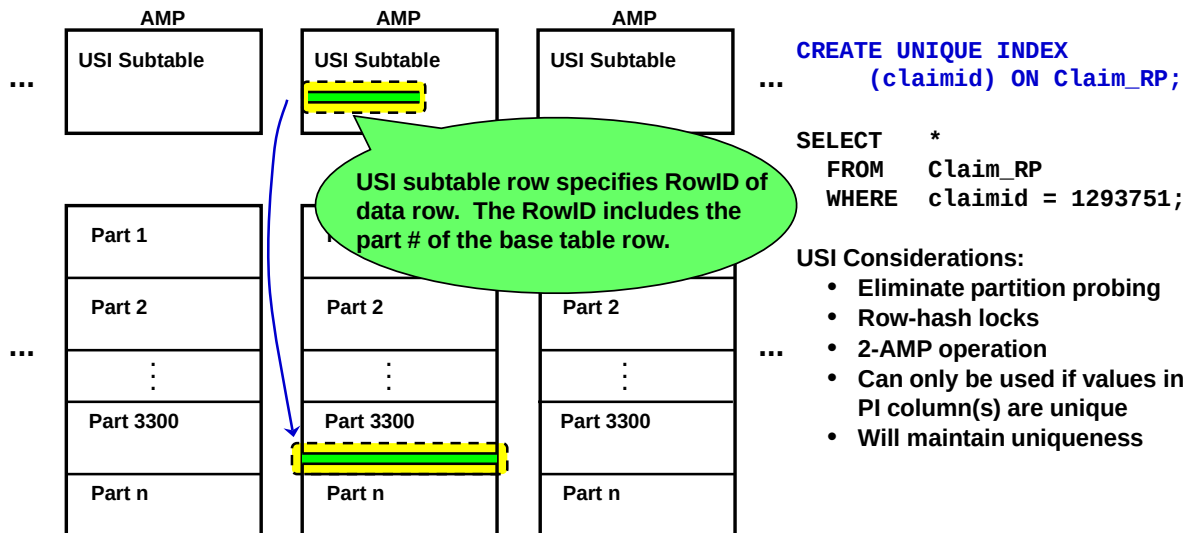
```
EXPLAIN      SELECT      *  
              FROM        Claim_RP  
              WHERE       claimid = 1293751  
              AND         claimdate = DATE '2019-01-13';
```

- 1) First, we do a **single-AMP RETRIEVE** step from a single partition of TFACT.Claim_RP by way of the primary index "TFACT.Claim_RP.claimid = 1293751, TFACT.Claim_RP.claimdate = DATE '2019-01-13'" with a residual condition of ("(TFACT.Claim_RP.claimid = 1293751) AND (TFACT.Claim_RP.claimdate = DATE '2019-01-13')") into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with low confidence to be 1 row (143 bytes). The estimated time for this step is 0.00 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

However, the user may not know the claimdate in order to include it in the query.

Place a USI on NUPI – Example 1 (cont.)

- If the partitioning column(s) are not part of the Primary Index, the Primary Index cannot be unique (e.g., claimdate is not part of the PI).
- To maintain uniqueness on the Primary Index, you can create a USI on the PI (e.g., claimid). This is a two-AMP operation.



Place a NUSI on NUPI – Example 1 (cont.)

If the partitioning columns are not part of the Primary Index, the Primary Index cannot be unique (e.g., Claim ID). You can use a NUSI on the same columns that make up the PI and actually get a single-AMP access operation. This feature only applies to a NUSI created on the same columns as a PI on row partitioned table. Additionally, instead of table level locks (typical NUSI), row hash locks will be used.

Reasons to choose a NUSI for your PI may include:

- The primary index is non-unique (cannot use a USI) and you need faster access than do a look-up or a probe into multiple partitions on a single AMP.
- TPT Update (MultiLoad protocol) can be used to load a table with a NUSI without using the MultiLoadX protocol.
- The access time for a USI and NUSI will be similar (each will access a subtable block) – however, the USI is a 2-AMP operation and requires BYNET message passing. The amount of space for a USI and NUSI subtable in this case will be similar. A typical NUSI with duplicate values will have multiple row ids in a subtable row and will save space per subtable row. However, a NUSI used as an index for columns with unique values will use approximately the same amount of subtable space as a USI. This is because each NUSI subtable row only contains 1 rowid.

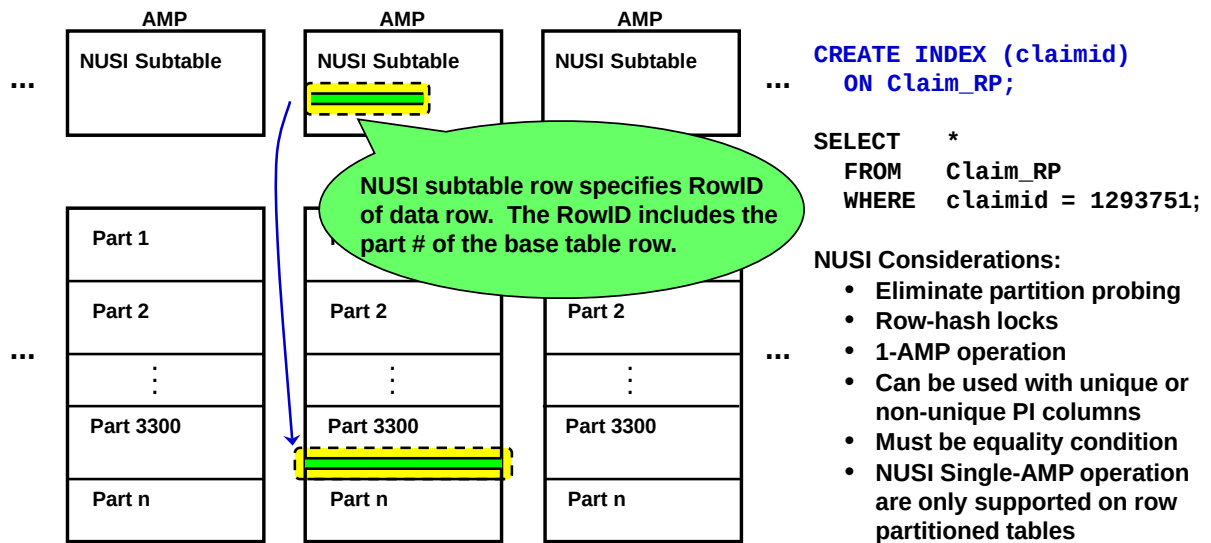
```
CREATE INDEX (claimid) ON Claim_RP;
```

```
EXPLAIN      SELECT *  
              FROM    Claim_RP  
              WHERE    claimid = 1293751;
```

- 1) First, we do a **single-AMP RETRIEVE step** from TFACT.Claim_RP by way of **index # 4** "TFACT.Claim_RP.claimid = 1293751" with no residual conditions into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with high confidence to be 1 row (143 bytes). The estimated time for this step is 0.00 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

Place a NUSI on NUPI – Example 1 (cont.)

- You can optionally create a NUSI on the same columns as the Primary Index (e.g., claimid). The PI may be unique or not.
- Optimizer generates a plan for a **single-AMP NUSI access** with **row-hash locking** (instead of table-level locking).



Partitioning with RANGE_N – Example 2

This example illustrates that a table can be partitioned with different size intervals. The current Sales data and Sales History data are placed in the same table. It typically is not practical to create a partitioning expression as shown in example #2, but the example is included to show the flexibility that you have with the partitioning expression.

For example, you may decide to partition the Sales History by month and the current sales data by day. You may want to do this if users frequently access the Sales History data with range constraints, resulting in full table scans. It may be that users access the current year data frequently looking at data for a specific day. The example on the facing page partitions the years 2010 to 2018 by month and the year 2019 by day.

One option may be to partition by week as follows:

```
PARTITION BY RANGE_N (salesdate BETWEEN  
DATE '2010-01-01' AND DATE '2010-12-31' EACH INTERVAL '7' DAY,  
DATE '2011-01-01' AND DATE '2011-12-31' EACH INTERVAL '7' DAY,  
:  
DATE '2019-01-01' AND DATE '2019-12-31' EACH INTERVAL '7' DAY);
```

One may think that a simple partitioning scheme to partition by week would be as follows:

```
PARTITION BY RANGE_N (salesdate BETWEEN  
DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '7' DAY);
```

This is a simpler PARTITION expression to initially code, but may require more work or thought later. There is a minor drawback to partitioning by weeks because a 7-day partition usually spans one year into the next. Assume that a year from now, you wish to ALTER this table and DROP the partitions for the year 2010. The ALTER TABLE DROP RANGE option has to specify a range of dates that actually represent a complete partition or partitions in the table. The ALTER TABLE command will be described later in this module.

If daily partitions are desired for all of the years, the following partitioning expression can be used to create a partitioned table with daily partitions.

```
PARTITION BY RANGE_N (salesdate BETWEEN  
DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' DAY);
```

Performance Note: Daily partitions for ten years creates 3652 partitions (10 x 365 plus two leap days).

Partitioning with RANGE_N – Example 2

- This example places current and history sales data into one table.
- Current year data is partitioned on a more granular basis (daily) while historical sales data is placed into monthly partitions.
- Partitions of varying intervals can be created for a table.

```
CREATE TABLE Sales_and_SalesHistory
(storeid          INTEGER NOT NULL,
 itemid          INTEGER NOT NULL,
 salesdate        DATE FORMAT 'YYYY-MM-DD',
 totalrevenue     DECIMAL(9,2),
 totalsold        INTEGER,
 note             VARCHAR(256))
PRIMARY INDEX (storeid, itemid)
PARTITION BY RANGE_N (salesdate BETWEEN
    DATE '2010-01-01' AND DATE '2018-12-31' EACH INTERVAL '1' MONTH,
    DATE '2019-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' DAY);
```

To partition by week, the following partitioning can be used.

```
PARTITION BY RANGE_N (salesdate BETWEEN
    DATE '2010-01-01' AND DATE '2010-12-31' EACH INTERVAL '7' DAY,
    DATE '2011-01-01' AND DATE '2011-12-31' EACH INTERVAL '7' DAY,
    :                               :
```

Partitioning – Example 3

This example partitions by StoreId (store number). Therefore, the partitioning expression value using the column Store Id or an expression involving Store Id has a maximum of 65,535 partitions and therefore must be between 1 and 65,535.

You can also use the RANGE_N expression to specify (and optionally limit) the number of possible partitions. The alternative partitioning (that is shown on facing page) expression specifies ten partitions. The optimizer may be able to more accurately cost join plans when the maximum number of partitions is known and small, making this a better choice than using the column directly.

Assume that a company has 1000 stores, and the store numbers (storeid) are from 100001 to 101001. To utilize 1000 partitions, the following partitioning expression could be defined.

```
... PRIMARY INDEX (storeid, itemid, salesdate)  
      PARTITION BY storeid - 100000;
```

If a company has a small number of stores and a small number of products, another option may be to partition by a combination of Store Id and Item Id.

Assume the following:

Store numbers – 100001 to 100065 - less than 65 stores
Item numbers – 5000 to 5999 - less than 1000 item ids

Basically, the table has three-digit item_id codes and less than 65 stores.

This table could be partitioned as follows:

```
... PRIMARY INDEX (storeid, itemid, salesdate)  
      PARTITION BY ((storeid - 100000) * 1000 + (itemid - 5000));
```

Assume that the storeid is 100009 and the itemid is 5025. This row would be placed in partition # 9025.

If many queries specify both a Store Id and an Item Id, this might be a useful partitioning scheme. Even if it wouldn't be useful, it demonstrates that the physical designers and/or database administrators have wide latitude in defining generalized partitioning schemes to meet the needs of individual tables.

Partitioning – Example 3

- The simplest partitioning expression uses one column from the row without modification. Even starting with Teradata 14.0, when a single column is specified, the column values must be between 1 and 65,535.
- **Assume the storeid is a value between 100001 and 101001.** Therefore, a simple calculation can be performed to avoid partitioning violation errors.
- This example will partition by data by storeid and effectively utilize 1000 partitions.

```
CREATE TABLE Store_Sales
( storeid      INTEGER NOT NULL,
  itemid       INTEGER NOT NULL,
  salesdate    DATE FORMAT 'YYYY-MM-DD',
  totalrevenue DECIMAL(9,2),
  totalsold    INTEGER,
  note         VARCHAR(256))
UNIQUE PRIMARY INDEX (storeid, itemid, salesdate)
PARTITION BY storeid - 100000;
```

Alternative Definition:

- Assume the customer wishes to group these 1000 stores into 100 partitions.
- The RANGE_N expression can be used to identify the number of partitions and group multiple stores into the same partition.

```
PARTITION BY RANGE_N ((storeid - 100000) BETWEEN 1 AND 1000 EACH 10);
```

Special Partitions with CASE_N and RANGE_N

The keywords, NO CASE (or NO RANGE) [OR UNKNOWN] and UNKNOWN are used to define the specific-use partitions.

Even if these options are not specified with the CASE_N (or RANGE_N) expressions, these two specific-use partitions are still reserved in the event the ALTER TABLE command is later used to add these options.

If it is necessary to test a CASE_N condition directly as NULL, it needs to be the first condition listed. This following example is correct. NULLs will be placed in partition #1.

```
PARTITION BY CASE_N
  (col3 IS NULL,
   col3 < 10,
   col3 < 100,
   NO CASE OR UNKNOWN)

INSERT INTO TabA_RP VALUES (1, 'A', NULL, DATE);
INSERT INTO TabA_RP VALUES (2, 'B', 5, DATE);
INSERT INTO TabA_RP VALUES (3, 'C', 50, DATE);
INSERT INTO TabA_RP VALUES (4, 'D', 500, DATE);
INSERT INTO TabA_RP VALUES (5, 'E', NULL, DATE);

SELECT PARTITION AS "Part #", COUNT(*) FROM TabA_RP
GROUP BY 1 ORDER BY 1;
```

<u>Part #</u>	<u>Count(*)</u>
1	2
2	1
3	1
4	1

Although you can code an example as follows, it should not be coded this way and will provide inconsistent results. NULLs will be placed in partition #4.

```
PARTITION BY CASE_N
  (col3 < 10,
   col3 IS NULL,
   col3 < 100,
   NO CASE OR UNKNOWN)

SELECT PARTITION AS "Part #", COUNT(*) FROM TabA_RP
GROUP BY 1 ORDER BY 1;
```

<u>Part #</u>	<u>Count(*)</u>
1	1
3	1
4	3

The CASE_N and RANGE_N can place rows into specific-use partitions when ...

- the expression doesn't meet any of the CASE and RANGE expressions.
- the expression evaluates to UNKNOWN.
- two partition numbers are reserved even if the above options are not used.

The keywords used to define two specific-use partitions are:

- **NO CASE (or NO RANGE) [OR UNKNOWN]**
 - If this option is used, then a specific-use partition is used when the expression isn't true for any case (or is out of range).
 - If OR UNKNOWN is included with the NO CASE (or NO RANGE), then UNKNOWN expressions are also placed in this partition.
- **UNKNOWN**
 - If this option is specified, a different specific-use partition is used for unknowns.
- **NO CASE (or NO RANGE), UNKNOWN**
 - If this option is used, then two separate specific-use partitions are used when the expression isn't true for any case (or is out of range) and different special partition is used for NULLs.

Special Partition Examples

This example assumes the following CASE_N expression.

```
PARTITION BY CASE_N (  
    col3 < 10 ,  
    col3 < 100 ,  
    col3 < 1000 ,  
    NO CASE OR UNKNOWN)
```

This statement creates four partitions, conceptually numbered (*Note) from one to four in the order they are defined. The first partition is when col3 is less than 10, the second partition is when col3 is at least 10 but less than 100, and the third partition is when col3 is at least 100 but less than 1,000.

The NO CASE OR UNKNOWN partition is for any value which isn't true for any previous CASE_N expression. In this case, it would be when col3 is equal to or greater than 1,000 or when col3 is NULL.

This partition is also used for values for which it isn't possible to determine the truth of the previous CASE_N expressions. Usually, this is a case where col3 is NULL or unknown.

Internally, UNKNOWN (option by itself) rows are assigned to partition #1. NOCASE (NO RANGE) OR UNKNOWN rows are physically assigned to partition #2. Internally, the first user-defined partition is actually partition #3.

The physical implementation in the file system is:

col3 < 10	– partition #1 (internally, rows placed in partition # 3)
col3 < 100	– partition #2 (internally, rows placed in partition # 4)
col3 < 1000	– partition #3 (internally, rows placed in partition # 5)
NO CASE or UNKNOWN	– partition #4 (internally, rows placed in partition # 2)

It is **NOT** syntactically possible to code a partitioning expression that has both NO CASE OR UNKNOWN, and UNKNOWN in the same expression. UNKNOWN expressions will either be placed in the partition with NO CASE or in a partition of their own. The following SQL is **NOT** permitted.

```
PARTITION BY CASE_N (  
    col3 < 10 ,  
    :  
    NO CASE OR UNKNOWN,  
    UNKNOWN) - causes an error
```

Special Partition Examples

The following examples illustrate the use of NO CASE and UNKNOWN options.

Ex. 1 PARTITION BY CASE_N (
col3 < 10 ,
col3 < 100 ,
col3 < 1000 ,
NO CASE OR UNKNOWN)

If col3 = 5, row is assigned to Partition #1.
If col3 = 50, row is assigned to Partition #2.
If col3 = 500, row is assigned to Partition #3.
If col3 = 5000, row is assigned to Partition #4.
If col3 = NULL, row is assigned to Partition #4.

In summary, NO CASE and UNKNOWN rows are placed into the same partition.

Ex. 2 PARTITION BY CASE_N (
col3 < 10 ,
col3 < 100 ,
col3 < 1000 ,
NO CASE,
UNKNOWN)

If col3 = 5, row is placed in Partition #1.
If col3 = 50, row is placed in Partition #2.
If col3 = 500, row is placed in Partition #3.
If col3 = 5000, row is placed in Partition #4.
If col3 = NULL, row is placed in Partition #5.

In summary, NO CASE and UNKNOWN rows are placed into separate partitions.

Note: RANGE_N works in a similar manner.

Partitioning with CASE_N – Example 4

This example illustrates the capability of partitioning based upon conditions (CASE_N).

For example, assume a table has a total revenue column, defined as decimal. The table could be partitioned on that column, so that low revenue products are separated from high revenue products. The partitioning expression could be written as shown on the facing page. In this example, 8 partitions are defined for total revenue values up to 100,000. Two additional partitions are defined – one for revenues greater than 100,000 and another for unknown revenues (e.g., NULL).

Teradata 13.10 Note: Teradata 13.10 allows CURRENT_DATE and/or CURRENT_TIMESTAMP with partitioning expressions. However, it is recommended to NOT use these in a CASE expression for a partitioned primary index (PPI). Why? In this case, all rows are scanned during reconciliation.

Additional examples:

The following examples illustrate the use of the NO CASE option by itself or the UNKNOWN option by itself.

Ex.1 **PARTITION BY CASE_N (**
 col3 < 10 ,
 col3 < 100 ,
 col3 < 1000 ,
 NO CASE)

If col3 = 5, row is assigned to Partition #1.
If col3 = 50, row is assigned to Partition #2.
If col3 = 500, row is assigned to Partition #3.
If col3 = 5000, row is assigned to Partition #4.
If col3 = NULL, Error 5728

5728: Partitioning violation for table DBname.Tablename.

Ex. 2 **PARTITION BY CASE_N (**
 col3 < 10 ,
 col3 < 100 ,
 col3 < 1000 ,
 UNKNOWN)

If col3 = 5, row is assigned to Partition #1.
If col3 = 50, row is assigned to Partition #2.
If col3 = 500, row is assigned to Partition #3.
If col3 = 5000, Error 5728
If col3 = NULL, row is assigned to Partition #4.

5728: Partitioning violation for table DBname.Tablename.

Partitioning with CASE_N – Example 4

- Partition the data based on total revenue for the products.
- The NO CASE and UNKNOWN options allow for totalrevenue >=100,000 or “unknown revenue”.
- A UPI is **NOT** allowed because the partitioning columns are **NOT** part of the PI.

```
CREATE TABLE Sales_Revenue
  (storeid      INTEGER NOT NULL,
   itemid       INTEGER NOT NULL,
   salesdate    DATE FORMAT 'YYYY-MM-DD',
   totalrevenue DECIMAL(9,2),
   totalsold    INTEGER,
   note         VARCHAR(256))
PRIMARY INDEX (storeid, itemid, salesdate)
PARTITION BY CASE_N
  (totalrevenue < 2000 ,
   totalrevenue < 4000 ,
   totalrevenue < 6000 ,
   totalrevenue < 8000 ,
   totalrevenue < 10000 ,
   totalrevenue < 20000 ,
   totalrevenue < 50000 ,
   totalrevenue < 100000 ,
   NO CASE ,
   UNKNOWN );
```

SQL Use of PARTITION Key Word

The facing page contains an example of using the key word PARTITION to determine the number of rows there are in physical partitions. This example is based on the Sales_Revenue table as defined on the previous page.

The following table shows the same result as the facing page, but also identifies the internal partition #'s as allocated.

Part #	Row Count	
1	169690	internally mapped to partition #3
2	163810	internally mapped to partition #4
3	68440	internally mapped to partition #5
4	33490	internally mapped to partition #6
5	18640	internally mapped to partition #7
6	27520	internally mapped to partition #8
7	1760	internally mapped to partition #9

Note that this table does not have any rows with a total_revenue value greater than 50,000 and less than 100,000. Partition #8 was not assigned. Also, there are no rows with a total_revenue >=100,000 or NULL because the NO CASE and UNKNOWN partitions are not used.

Assume the following three SQL INSERT commands are executed:

```
INSERT INTO Sales_Revenue
VALUES (1003, 5051, CURRENT_DATE, 51000, 45, NULL);
INSERT INTO Sales_Revenue
VALUES (1003, 5052, CURRENT_DATE, 102000, 113, NULL);
INSERT INTO Sales_Revenue
VALUES (1003, 5053, CURRENT_DATE, NULL, NULL, NULL);
```

The result of executing the SQL statement again would now be as follows:

Part #	Row Count	
1	169690	internally mapped to partition #3
2	163810	internally mapped to partition #4
3	68440	internally mapped to partition #5
4	33490	internally mapped to partition #6
5	18640	internally mapped to partition #7
6	27520	internally mapped to partition #8
7	1760	internally mapped to partition #9
8	1	internally mapped to partition # 10
9	1	internally mapped to partition # 2 (NO CASE)
10	1	internally mapped to partition # 1 (UNKNOWN)

SQL Use of PARTITION Key Word

The **PARTITION** SQL key word can be used to return partition numbers that have rows and a count of rows that are currently located in partitions of a table.

SQL:

```
SELECT    PARTITION AS "Part #",
          COUNT(*) AS "Row Count"
FROM      SalesRevenue
GROUP BY  1
ORDER BY  1;
```

Result:

Part #	Row Count
1	169690
2	163810
3	68440
4	33490
5	18640
6	27520
7	1760

```
totalrevenue < 2,000
totalrevenue < 4,000
totalrevenue < 6,000
totalrevenue < 8,000
totalrevenue < 10,000
totalrevenue < 20,000
totalrevenue < 50,000
```

SQL - insert two rows:

```
INSERT INTO SalesRevenue VALUES (1003, 5052, CURRENT_DATE, 102000, 113, NULL);
INSERT INTO SalesRevenue VALUES (1003, 5053, CURRENT_DATE, NULL, NULL, NULL);
```

SQL (same as above):

```
SELECT    PARTITION AS "Part #",
          COUNT(*) AS "Row Count"
FROM      SalesRevenue
GROUP BY  1
ORDER BY  1;
```

Result:

Part #	Row Count
1	169690
2	163810
3	68440
4	33490
5	18640
6	27520
7	1760
9	1
10	1

same as above

NO CASE
UNKNOWN

SQL Use of CASE_N

The facing page contains an example of using the CASE_N expression with SQL. You may wish to use this function to determine/forecast how rows will be mapped to various partitions in a table. The Sales_Revenue table was created as follows:

```
CREATE TABLE Sales_Revenue
( storeid      INTEGER NOT NULL,
  itemid       INTEGER NOT NULL,
  salesdate    DATE FORMAT 'YYYY-MM-DD',
  totalrevenue DECIMAL(9,2),
  totalsold    INTEGER,
  note        VARCHAR(256))
PRIMARY INDEX (store_id, item_id, sales_date)
PARTITION BY CASE_N
( totalrevenue < 2000, totalrevenue < 4000,
  totalrevenue < 6000, totalrevenue < 8000,
  totalrevenue < 10000, totalrevenue < 20000,
  totalrevenue < 50000, totalrevenue < 100000,
  NO CASE, UNKNOWN);
```

The CASE_N expression in the query on the facing page is simply an SQL statement that shows how the rows would be partitioned.

SQL Use of RANGE_N

An example of using the RANGE_N expression with SQL is:

```
SELECT  RANGE_N ( Calendar_Date BETWEEN
  DATE '2013-12-15' AND DATE '2014-01-31' EACH INTERVAL '7' DAY)
        AS "Part #",
MIN      (Calendar_Date)      AS "Minimum Date",
MAX      (Calendar_Date)      AS "Maximum Date"
FROM      Sys_Calendar.Calendar
WHERE     Calendar_Date
BETWEEN   DATE '2013-12-15' AND DATE '2014-01-31'
GROUP BY "Part #"
ORDER BY "Part #";
```

Output from this SQL is:

<u>Part #</u>	<u>Minimum Date</u>	<u>Maximum Date</u>
1	2013-12-15	2013-12-21
2	2013-12-22	2013-12-28
3	2013-12-29	2014-01-04
4	2014-01-05	2014-01-11
5	2014-01-12	2014-01-18
6	2014-01-19	2014-01-25
7	2014-01-26	2014-01-31 (note only 6 days in this partition)

SQL Use of CASE_N

- The **CASE_N** (and **RANGE_N**) expressions can be used with SQL to forecast the number of rows that will be placed into partitions.
- This example uses a different partitioning scheme than the table actually has to determine how many rows would be placed into various partitions.

```
SELECT CASE_N ( totalrevenue < 1500,  
                totalrevenue < 2000,  
                totalrevenue < 3000,  
                totalrevenue < 5000,  
                totalrevenue < 8000,  
                totalrevenue < 12000,  
                totalrevenue < 20000,  
                totalrevenue < 50000,  
                NO CASE,  
                UNKNOWN) AS "Case #",  
        count(*)        AS "Row Count"  
FROM    SalesRevenue  
GROUP BY 1  
ORDER BY 1;
```

Result:

<u>Case #</u>	<u>Row Count</u>
1	81540
2	88150
3	97640
4	103230
5	64870
6	31290
7	14870
8	1760

Notes:

- Currently, in this table, there are no rows with totalrevenue >= 50,000 or NULL.
- The Case # would become the Partition # if the table was partitioned in this way.

Using ALTER TABLE with Row Partitioned Tables

The ALTER TABLE statement has been extended in support of PPI. For empty tables, the primary index and partitioning expression may be re-specified. For tables with rows, the partitioning expression may be modified only in ways that don't require existing rows to be re-evaluated.

The permitted changes for populated tables are to drop ranges at the ends or to add ranges at the ends. For example, a common use of this capability would be to drop ranges for the oldest dates, and to prepare additional ranges for future dates, among other things.

Limitations with ALTER TABLE:

- Primary Index of a non-empty table may not be altered
- Partitioning of a non-empty table is generally limited to altering the “ends”.
- If a table has **Delete triggers**, they must be disabled if the WITH DELETE option is specified.
- If a save table has **Insert triggers**, they must be disabled if the WITH INSERT option is specified.

For empty row partitioned tables, the ALTER TABLE statement can be used to do the following:

- Remove partitioning for a partitioned table
- Establish partitions for a table (adds or replaces)
- Change the columns that comprise the primary index
- Change a unique primary index to non-unique.
- Change a non-unique primary index to unique.

For empty or non-empty tables, the ALTER TABLE statement can also be used to name an unnamed primary index or drop the name of a named primary index.

- To name an unnamed primary index or change the existing name of a primary index to something else, specify
... `MODIFY PRIMARY INDEX index_name;`
- To drop the name of a named index, specify
... `MODIFY PRIMARY INDEX NOT NAMED;`

Assume you have a populated data table (and the table is quite large) defined with a “non-unique partitioned primary index” and all of the partitioning columns are part of the PI. You realize that the table should have been defined with a “unique partitioned primary index”, but the table is already loaded with data. Here is a technique to convert this NUPI into a UPI without copying or reloading the data.

- CREATE a USI on the columns making up the PI. ALTER the table, effectively changing the NUPI to a UPI, and the software will automatically drop the USI.

Using ALTER TABLE with Row Partitioned Tables

The ALTER TABLE statement has enhancements for a partitioned table to modify the partitioning properties of the primary index for a table.

For populated tables, ...

- You are permitted to drop and/or add ranges at the “ends” of existing partitions on a range-partitioned table.
 - ALTER TABLE includes ADD / DROP RANGE options.
 - You can also [add or drop special partitions](#) (NO RANGE or UNKNOWN).
 - You [cannot](#) drop all the ranges.
- Possible use – drop ranges for the oldest dates and prepare additional ranges for future dates.
- The set of primary index columns [cannot](#) be altered for a populated table.

Teradata ALTER TABLE TO CURRENT Feature

- ALTER TABLE has a new option to resolve partitioned table definitions with DATE, CURRENT_DATE, and CURRENT_TIMESTAMP to their current values.
 - This feature only applies to partitioned tables and join indexes.

To use ALTER TABLE for any purpose other than the above situations, the table must be empty.

ALTER TABLE – NO RANGE is not Defined

The DROP RANGE option is used to drop a range set from the RANGE_N function on which the partitioning expression for the table is based. You can only drop ranges if the partitioning expression for the table is derived only from a RANGE_N function.

You can drop empty partitions without specifying the WITH DELETE or WITH INSERT option. However, if the partitions to be dropped have data, you must include either the WITH DELETE or WITH INSERT syntax.

Some of the ALTER TABLE statement options include:

DROP RANGE WHERE *conditional_expression* – a conditional partitioning expression used to drop a range set from the RANGE_N function on which the partitioning expression for the table is based.

You can only drop ranges if the partitioning expression for the table is derived only from a RANGE_N function.

You must base *conditional_partitioning_expression* on the system-derived PARTITION column.

DROP RANGE BETWEEN ... [NO RANGE [OR UNKNOWN]] – used to drop a set of ranges from the RANGE_N function on which the partitioning expression for the table is based.

You can also drop NO RANGE OR UNKNOWN and UNKNOWN specifications from the definition for the RANGE_N function.

You can only drop ranges if the partitioning expression for the table is derived exclusively from a RANGE_N function.

Ranges must be specified in ascending order.

ADD RANGE BETWEEN ... [NO RANGE [OR UNKNOWN]] – used to add a set of ranges to the RANGE_N function on which the partitioning expression for the table is based.

You can also add NO RANGE OR UNKNOWN and UNKNOWN specifications to the definition for the RANGE_N function.

You can only add ranges if the partitioning expression for the table is derived exclusively from a RANGE_N function.

If a table does not have the NO RANGE partition, then partitions are dropped from the table without using the Transient Journal and the rows are either deleted or are copied (WITH INSERT) into a user-specified table.

ALTER TABLE – NO RANGE is not Defined

Assume a table is partitioned as follows:

```
PARTITION BY RANGE_N (salesdate BETWEEN  
DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' MONTH);
```

To drop/add partitions and **DELETE** the old data:

```
ALTER TABLE Sales MODIFY PRIMARY INDEX  
DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'  
ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' MONTH  
WITH DELETE;
```

To drop/add partitions and **COPY** the old data to another table:

```
ALTER TABLE Sales MODIFY  
DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'  
ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' MONTH  
WITH INSERT INTO SalesHistory;
```

Notes assuming NO RANGE is not defined:

- The WITH DELETE or WITH INSERT syntax is required if the partitions have data.
- **Dropping partition ranges means the data in the dropped partitions is also deleted from the table.**
- The MODIFY syntax does not require you to also include the words PRIMARY INDEX.
- The SalesHistory table must already exist when using the WITH INSERT option.
- SalesHistory may optionally be partitioned; recommended for the table to be empty and have a matching PI as the Sales table for performance.

ALTER TABLE – NO RANGE is Defined

When using the ALTER TABLE to drop existing ranges, if the table has NO RANGE defined, it is not necessary to include the WITH DELETE or WITH INSERT syntax.

If NO RANGE is defined and the WITH DELETE or WITH INSERT syntax is used, the WITH DELETE or WITH INSERT is effectively ignored.

This simply means the rows are not deleted or copied to another table.

In this case (table has a NO RANGE partition), rows are copied from dropped partitions into the NO RANGE partition.

If new partitions are added, the NO RANGE partition is checked and rows that apply to the newly added partitions are moved from NO RANGE into the new partitions.

Note: When dropping Ranges, you can optionally include the "[Interval](#)". The following syntax is accepted.

```
ALTER TABLE Sales MODIFY
DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'
EACH INTERVAL '1' MONTH
ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31'
EACH INTERVAL '1' MONTH;
```


ALTER TABLE – NO RANGE is Defined

Assume a table is partitioned as follows:

```
PARTITION BY RANGE_N (salesdate BETWEEN  
DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' MONTH, NO RANGE);
```

To drop/add partitions from the table definition:

```
ALTER TABLE Sales MODIFY  
DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'  
ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' MONTH;
```

If WITH DELETE or WITH INSERT is included, it is effectively ignored.

```
ALTER TABLE Sales MODIFY  
DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'  
ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' MONTH  
WITH INSERT INTO SalesHistory;
```

Notes assuming NO RANGE is defined:

- If dropped partitions have data, then the data is moved to the NO RANGE partition.
- If WITH DELETE or WITH INSERT syntax is included, it is effectively ignored.
- The WITH DELETE or WITH INSERT syntax is not needed and is confusing if included.
- If new partitions are added, the NO RANGE partition is checked and rows that apply to the newly added partitions are moved from NO RANGE into the new partitions.
- If you want to delete the data, use SQL DELETE before using ALTER TABLE.
- General recommendation is to not use NO RANGE unless it is specifically needed.

ALTER TABLE – Example 5

This page contains notes on the internal implementation. The important point is to understand that dropping or adding partitions (to the “ends” of an already partitioned table with data) does not cause changes to the internal partitioning numbers that are currently implemented. The logical partition numbers change, but the internal partition numbers do not. For this reason, dropping or adding partitions does not cause an undue amount of work.

The following table shows the same result as the facing page, but also identifies the internal partition #'s as allocated.

<u>PARTITION</u>	<u>Count(*)</u>	
1	10850	internally mapped to partition #3
2	10150	internally mapped to partition #4
:	:	:
13	12400	internally mapped to partition #15
14	11200	internally mapped to partition #16
:	:	:
119	14800	internally mapped to partition #121
120	14950	internally mapped to partition #122

In the example on the facing page, 12 partitions were dropped for the year 2010 and 12 partitions were added for the year 2020. The partitions for 2020 don't appear because they are empty.

The following table shows the same result as the facing page, but also identifies the internal partition #'s as allocated after the partitions for the year 2010 were dropped.

<u>PARTITION</u>		
1	12400	internally mapped to partition #15
2	11200	internally mapped to partition #16
:	:	:
107	14800	internally mapped to partition #121
108	14950	internally mapped to partition #122

You can add the NO RANGE and/or UNKNOWN partitions to an already partitioned table.

```
ALTER TABLE Sales MODIFY  
ADD RANGE NO RANGE OR UNKNOWN;
```

If this table had NO RANGE partition defined and the 12 partitions were dropped (as in this example), the data rows from the dropped partitions are moved to the NO RANGE partition. To remove the special partitions and delete the data, use the following command:

```
ALTER TABLE Sales MODIFY  
DROP RANGE NO RANGE OR UNKNOWN  
WITH DELETE;
```

ALTER TABLE – Example 5

In this example, NO RANGE has **not** been defined for the Sales tables.

SQL:

```
SELECT PARTITION,  
       COUNT(*)  
FROM  
Sales  
GROUP BY 1  
ORDER BY 1;
```

Result:	PARTITION	COUNT(*)
	1	10850
	2	10150
	:	:
	119	14800
	120	14950

Part #1 - January 2010

Part #120 - December 2019

```
ALTER TABLE Sales MODIFY
```

```
  DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'
```

```
  ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31'
```

```
                        EACH INTERVAL '1' MONTH
```

```
WITH DELETE;
```

SQL:

```
SELECT PARTITION,  
       COUNT(*)  
FROM  
Sales  
GROUP BY 1  
ORDER BY 1;
```

Result:	PARTITION	COUNT(*)
	1	12400
	2	11200
	:	:
	107	14800
	108	14950

Part #1 - January 2011

Part #108 - December 2019

ALTER TABLE TO CURRENT

Starting with Teradata 13.10, you can now specify `CURRENT_DATE` and `CURRENT_TIMESTAMP` functions in a partitioned primary index for base tables and join indexes.

Also starting with Teradata 13.10, Teradata provides a new option with the `ALTER TABLE` statement to modify a partitioned table that has been defined with a moving `CURRENT_DATE` (or `DATE`) or moving `CURRENT_TIMESTAMP`. This new option is called `ALTER TABLE TO CURRENT`.

When you specify `CURRENT_DATE` and `CURRENT_TIMESTAMP` as part of a partitioning expression for a partitioned table, these functions resolve to the date and timestamp when you define the PPI. To partition on a new `CURRENT_DATE` or `CURRENT_TIMESTAMP` value, submit an `ALTER TABLE TO CURRENT` request.

The `ALTER TABLE TO CURRENT` syntax is shown on the facing page.

The `WITH DELETE` option is used to delete any row whose partition number evaluates to a value outside the valid range of partitions.

The `WITH INSERT [INTO] save_table` option is used to insert any row whose partition number evaluates to a value outside the valid range of partitions into the table specified by *save_table*.

The `WITH DELETE` or `INSERT INTO save_table` clause is sometimes referred to as a null partition handler. You cannot specify a null partition handler for a join index.

Save_table and the table being altered must be different tables with different names.

ALTER TABLE TO CURRENT

This Teradata 13.10 option allows you to periodically resolve the CURRENT_DATE (or DATE) and CURRENT_TIMESTAMP of a partitioned table to their current values.

Benefits include:

- You do not have to change the partitioning expression to update the value for CURRENT_DATE or CURRENT_TIMESTAMP.
- To partition on a **new** CURRENT_DATE or CURRENT_TIMESTAMP value, simply submit an ALTER TABLE TO CURRENT request.

Considerations:

- The **ALTER TABLE TO CURRENT** request causes the CURRENT_DATE and/or CURRENT_TIMESTAMP to effectively repartition the rows in the table.
- If RANGE_N specifies CURRENT_DATE or CURRENT_TIMESTAMP in a partitioning expression, **you cannot use ALTER TABLE to add or drop ranges for the table.** You must use the ALTER TABLE TO CURRENT statement to achieve this function.

```
ALTER TABLE table_name  
join_index_name TO CURRENT WITH DELETE INSERT [INTO] save_table ;
```

ALTER TABLE TO CURRENT – Example 6

The ALTER TABLE TO CURRENT option allows you to periodically modify the partitioning. This option resolves the CURRENT_DATE and CURRENT_TIMESTAMP to their current values.

The example on the facing page assumes partitioning begins on a year boundary. Using this example, consideration for the two options are:

- With hard-coded dates in the CREATE TABLE statement, you must compute the new dates and specify them explicitly in the ADD RANGE clause of the request. This requires manual intervention every year you submit the request.
- With CURRENT_DATE in the CREATE TABLE statement, you can schedule the ALTER TABLE TO CURRENT request to be submitted annually or simply execute the next year. This request rolls the partition window forward by efficiently dropping and adding partitions.

As a result of executing the ALTER TABLE TO CURRENT WITH DELETE, Teradata deletes the rows from the table because they are no longer needed.

An alternative partitioning expression (to the facing page) is:

```
PARTITION BY RANGE_N (salesdate BETWEEN  
CAST(((EXTRACT(YEAR FROM CURRENT_DATE) - 8 - 1900) * 10000 + 0101) AS  
DATE) AND  
CAST(((EXTRACT(YEAR FROM CURRENT_DATE) + 1 - 1900) * 10000 + 1231) AS  
DATE)  
EACH INTERVAL '1' MONTH);
```

Considerations:

- You should evaluate how a DATE, CURRENT_DATE, or CURRENT_TIMESTAMP function will require reconciliation in a partitioning expression before you define such expressions on a table or join index.
- If you specify multiple ranges using a DATE or CURRENT_DATE function in one of the ranges, and then later reconcile the partitioning, the range specified using CURRENT_DATE might overlap one of the existing ranges. If so, reconciliation aborts the request and returns an error to the requestor. If this happens, you must recreate the table with a new partitioning expression based on DATE or CURRENT_DATE. Because of this, you should design a partitioning expression that uses a DATE or CURRENT_DATE function in one of its ranges with care.

DATE, CURRENT_DATE, and CURRENT_TIMESTAMP functions in a partitioning expression are most appropriate when the data must be partitioned as one or more Current partitions and one or more History partitions, where the terms Current and History are defined with respect to the resolved DATE, CURRENT_DATE, or CURRENT_TIMESTAMP values in the partitioning expression.

This enables you to reconcile a table or join index periodically to move older data from the current partition into one or more history partitions using an ALTER TABLE TO CURRENT request instead of redefining the partitioning using explicit dates that must be determined each time you alter a table using ALTER TABLE requests to ADD or DROP ranges.

ALTER TABLE TO CURRENT – Example 6

This example creates a partitioning expression to maintain the last 8 years of historical data, data for the current year, and data for one future year for a total of 10 years.

```
CREATE TABLE Sales
( storeid      INTEGER NOT NULL,
  itemid       INTEGER NOT NULL,
  salesdate    DATE FORMAT 'YYYY-MM-DD',
  :
PRIMARY INDEX (storeid, itemid)
PARTITION BY RANGE_N (salesdate BETWEEN
  DATE '2011-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' MONTH);
```

Assuming the current year is 2019, an equivalent definition using CURRENT_DATE is:

```
PRIMARY INDEX (storeid, itemid)
PARTITION BY RANGE_N
(salesdate BETWEEN
  CAST ((EXTRACT(YEAR FROM CURRENT_DATE) - 8 ) || '-01-01' AS DATE) AND
  CAST ((EXTRACT(YEAR FROM CURRENT_DATE) + 1 ) || '-12-31' AS DATE)
  EACH INTERVAL '1' MONTH );
```

In 2020, execute **ALTER TABLE Sales TO CURRENT WITH DELETE;**

- Teradata deletes the rows from 2011 because they are no longer needed.
- To view the date when the table was last resolved, the DBC.PartitioningConstraintsV provides columns named "ResolvedCurrent_Date" and "ResolvedCurrent_TimeStamp".

Row Partitioning Enhancements

The facing page identifies various enhancements with different Teradata releases.

Teradata V2R6.0

- Selected Partition Archive, Restore, and Copy
- Dynamic partition elimination for merge join
- Single-AMP NUSI access when NUSI on same columns as NUPI;
- Partition elimination on RowIDs referenced by NUSI

Teradata V2R6.1

- Row partitioning can be used for global temporary tables and volatile tables
- Collect statistics on system-derived column PARTITION

Teradata V2R6.2

- Row partitioning can be implemented for non-compressed join indexes

Teradata 12.0

- Multi-level partitioning

Teradata 13.10

- Tables and non-compressed join indexes can now include partitioning on a character column.
- Row partitioned tables allow a test value (e.g., RANGE_N) to have a TIMESTAMP(n) data type.
- ALTER TABLE *tablename* TO CURRENT ...;

Teradata 14.0

- Increased partition limit to 9.223 quintillion
- New data types for RANGE_N – BIGINT and TIMESTAMP
- ADD option for a partitioning level
- Column partitioned tables can also be row partitioned

Teradata 15.10

- Partition-level locks

Multilevel Partitioning Concepts

The facing page identifies the basic concepts of using a multilevel partitioning.

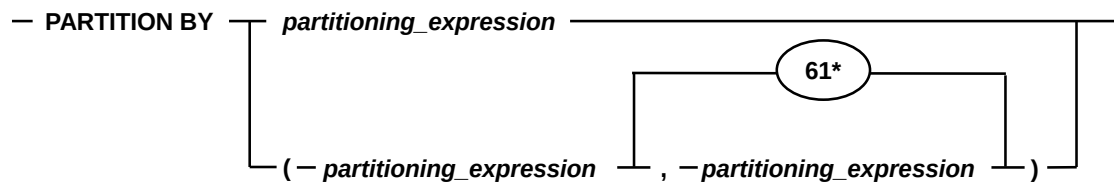
Multilevel partitioning allows each partition at a given level to be further partitioned into sub-partitions. Each partition for a level is sub-partitioned the same per a partitioning expression defined for the next lower level. The system hash orders the rows within the lowest partition levels. A multilevel partitioned table (a.k.a., MLPPI) undertakes efficient searches by using partition elimination at the various levels or combinations of levels.

Notes associated with multilevel partitioning:

- Note that the number of levels of partitioning cannot exceed 15. Each level must define at least two partitions. The number of levels of partitioning may be further restricted by other limits such as the maximum size of the table header, data dictionary entry sizes, etc.
- The number of partitions in a table prior to Teradata 14.0 cannot exceed 65,535 partitions. The number of partitions in an multilevel partitioned table is determined by multiplying the number of partitions at the different levels ($d1 * d2 * d3 * \dots$).
- The specification order of partitioning expressions can be important for multilevel partitioning. The system maps multilevel partitioning expressions into a single-level combined partitioning expression. It then maps the resulting combined partition number 1-to-1 to an internal partition number.
- A usage implication – you can alter only the highest partition level, which by definition is always level 1, to change the number of partitions at that level when the table is populated with rows.

Multilevel Row Partitioned Concepts

- Allows multiple partitioning expressions instead of only one for a table or a non-compressed join index.
- Multilevel partitioning allows each partition at a level to be sub-partitioned.
 - Each partitioning level is defined independently using a RANGE_N or CASE_N expression.
- A multilevel row partitioned table allows efficient searches by using partition elimination at the various levels or combination of levels.
- Allows more flexibility in which partitioning expression to use when there are multiple choices for the partitioning expressions.
- Teradata 14.0 allows for a maximum of 9.223 quintillion partitions and a total of 62 levels. (Teradata prior to release 14.0 had a maximum of 15 levels of partitioning.)
- Syntax:



14* – Teradata < Release 14.0 limit.

Multilevel Partitioning Concepts (cont.)

The facing page contains an example showing the benefit of using a multilevel partitioned table. Assume in this example that there are 50 districts and the single partitioning is by month for 2 years and the multilevel partitioning is first by month (24 months) and sub-partitioned by 50 districts. In the single level partitioning, there are a total of 24 partitions and in the multilevel, there are a total of 1200 partitions.

You can use a multilevel row partitioned table to improve query performance via partition elimination, either at each of the partition levels or by combining all of them. A multilevel partitioned table provides multiple access paths to the rows in the base table. As with other indexes, the Optimizer determines if the index is usable for a query and, if usable, whether its use provides the estimated least costly plan for executing the query.

The following list describes the various access methods that are available when multilevel partitioning is defined for a table:

- If there is an equality constraint on the primary index and there are constraints on the partitioning columns such that access is limited to a single partition at each level, access is as efficient as with a non-partitioned table.
- This is a single-AMP, single-hash access in a single sub-partition at the lowest level of the partition hierarchy.
- With constraints defined on the partitioning columns, performance of a primary index access can approach the performance of a non-partitioned table depending on the extent of partition elimination that can be achieved.
- This is a single-AMP, single-hash access in multiple (but not all) sub-partitions at the lowest level of the partition hierarchy.
- Access by means of equality constraints on the primary index columns that does not also include all the partitioning columns, and without constraints defined on the partitioning columns, might not be as efficient as access with a non-partitioned table. The efficiency of the access depends on the number of non-empty sub-partitions at the lowest level of the partition hierarchy.
- This is a single-AMP, single-hash access in all sub-partitions at the lowest level of the partition hierarchy.
- With constraints on the partitioning columns of a partitioning expression such that access is limited to a subset of, say n percent, of the partitions for that level, the scan of the data is reduced to about n percent of the time required by a full-table scan.
- This is an all-AMP scan of only the non-eliminated partitions for that level. This allows multiple access paths to a subset of the data: one for each partitioning expression. If constraints are defined on partitioning columns for more than one of the partitioning expressions in the multilevel partitioning expression, partition elimination can lead to even less of the data needing to be scanned.

Multilevel Partitioning Concepts (cont.)

Query – Compare District 25 revenue for Week 6 vs. same period last year?



Multilevel Partitioning – Example 7

You create a multilevel partitioned table by specifying two or more partitioning expressions, where each expression must be defined using either a `RANGE_N` function or a `CASE_N` function exclusively. The system combines the individual partitioning expressions internally into a single partitioning expression that defines how the data is partitioned on an AMP.

The first partitioning expression is the highest level partitioning. Within each of those partitions, the second partitioning expression defines how each of the highest-level partitions is sub-partitioned. Within each of those second-level partitions, the third-level partitioning expression defines how each of the second level partitions is sub-partitioned. Within each of these lowest level partitions, rows are ordered by the row hash value of their primary index and their assigned uniqueness value.

You define the ordering of the partitioning expressions in your `CREATE TABLE SQL` text, and that ordering implies the logically ordering by RowID. Because the partitions at each level are distributed among the partitions of the next higher level in the hierarchy, scanning a partition at a certain level requires skipping some internal partitions.

Partition expression order does *not* affect the ability to eliminate partitions, but *does* affect the efficiency of a partition scan. As a general rule, this should not be a concern if there are many rows, which implies multiple data blocks, in each of the partitions.

The facing page contains an example of creating a multilevel partitioned table

There are two levels of partitioning defined in this example. The first level defines 120 partitions and the second defines 75 partitions. Therefore, the total number of partitions for the combined partitioning expression is the product of $120 * 75$, or 9000.

Multilevel Partitioning – Example 7

For example, partition Claim table by "claimdate" and "stateid".

```
CREATE TABLE Claim_MLRP
( claimid          INTEGER NOT NULL
  ,custid          INTEGER NOT NULL
  ,claimdate       DATE     NOT NULL
  ,stateid         BYTEINT NOT NULL
  , ... )
PRIMARY INDEX (claimid)
PARTITION BY (
/* First level of partitioning */
  RANGE_N (claimdate BETWEEN
    DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' MONTH),
/* Second level of partitioning */
  RANGE_N (stateid BETWEEN 1 AND 75 EACH 1) )
UNIQUE INDEX (claimid);
```

Notes:

- For multi-level row partitioned table, the set of partitioning expressions must be enclosed in parentheses.
- Each level must define at least two partitions for a multi-level row partitioned table.
- The number of defined partitions in this example is (120 * 75) or 9000.

Multilevel Partitioning – Example 7 (cont.)

The facing page continues the example of using a multilevel partitioned table. This example assumes that the query has conditions where only claims for a specific month and for a specific state need to be returned. Teradata only needs to scan the data blocks associated with the specified criteria.

Multilevel Partitioning Example 7 (cont.)

Assume

- Eliminating all but one month out of many years of claims history would facilitate scanning less than 2% of the claims history.
- Similarly, eliminating all but the California claims out of the many states would facilitate scanning less than 4% of the claims history.

Then, combining both of these predicates for partition elimination would facilitate scanning less than 0.08% of the claims history for satisfying the following query.

```
SELECT    ...  
FROM      Claim_MLRP C, States S  
WHERE     C.stateid = S.stateid  
AND       S.statename = 'California'  
AND       C.claimdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How is a ML Partition # Calculated?

The facing page shows the calculation that is used to determine the partition number for a multilevel partitioned table.

How is a ML Partition # Calculated?

Multilevel partitioning is rewritten internally to single-level partitioning to generate a combined partition number as follows:

$$(p_1 - 1) * dd_1 + (p_2 - 1) * dd_2 + \dots + (p_{n-1} - 1) * dd_{n-1} + p_n$$

where n is the number of partitioning expressions

p_i is the value of the partitioning expression for level i

d_i is the number of partitions for level i

dd_i is the product of d_{i+1} through d_n

$dd = d_1 * d_2 * \dots * d_n \leq 65535$

dd is the total number of combined partitions

Example:

Assume January, 2019 is the 109th first level partition and California is the 6th state code for the second level partition. Also assume that the first level has 120 partitions and the second level has 75 partitions.

$$(109 - 1) * 75 + 6 = 8106$$

is the logical partition number for claims in California for January of 2019.

ADD Partitions Option – Example 8

If the table or join index is defined with multiple partitioning levels, you can (and possibly should consider) explicitly specify the number of partitions per partitioning level using the ADD option.

The facing page has an example of using the ADD option for the "district" sub-partition.

Note that the following ALTER TABLE will fail and return the 5734 error message:

```
ALTER TABLE Claim_MLRP2 MODIFY  
ADD RANGE#L2 BETWEEN 1 AND 175 EACH 1;
```

This fails because the ranges (i.e., districts) 1 to 100 already exist in the table.

If you want to change the special partitions for District from NO RANGE OR UNKNOWN to just NO RANGE, the following SQL can be used:

```
ALTER TABLE Claim_MLRP2 MODIFY  
DROP RANGE#L2 NO RANGE OR UNKNOWN  
ADD RANGE#L2 NO RANGE  
WITH DELETE;
```

ADD Partitions Option – Example 8

The ADD option allows you to define additional partitions for a sub-level – these partitions can be added at a later time via the ALTER table command.

```
CREATE TABLE Claim_MLRP2
( claimid      INTEGER NOT NULL,
  custid       INTEGER NOT NULL,
  claimdate    DATE     NOT NULL FORMAT 'YYYY-MM-DD',
  :
  district     SMALLINT,
  :
  PRIMARY INDEX (claimid)
  PARTITION BY (
    RANGE_N (claimdate BETWEEN DATE '2010-01-01' AND DATE '2019-12-31'
              EACH INTERVAL '1' MONTH, NO RANGE),
    RANGE_N (district BETWEEN 1 AND 100 EACH 1,
              NO RANGE OR UNKNOWN) ADD 200)
  UNIQUE INDEX (claimid);
```

An INSERT of a new claim into district 101 will cause the new row to be placed in the NO RANGE partition.

The following ALTER TABLE can be used to add an additional 75 district partitions.

```
ALTER TABLE Claim_MLRP2 MODIFY ADD RANGE#L2 BETWEEN 101 AND 175 EACH 1;
```

Note the syntax to add or drop ranges for sub-levels (RANGE#Ln).

Character Partitioning

This Teradata 13.10 feature extends the capabilities and options when defining a row partitioned table or a non-compressed join index. Tables and non-compressed join indexes can now include partitioning on a character column. This feature provides the improved query performance benefits of partitioning when queries have conditions on columns with character (alphanumeric) data.

- Before Teradata 13.10, customers were limited to creating partitioning on tables that did **not** involve comparison of character data. Partitioning expressions were limited to numeric or date type data.

The row partitioning feature of Teradata has always allowed a class of queries to access a portion of a large table, instead of the entire table. This capability has simply been extended to include character data. The traditional uses of the Primary Index (PI) for data placement and rapid access of the data when the PI values are specified are still retained.

When creating a table or a join index, the **PARTITION BY** clause (part of PRIMARY INDEX) can now include partitioning on a character column. This allows the comparison of character data.

This feature allows a partitioning expression to involve comparison of character data (CHAR, VARCHAR, GRAPHIC, VARGRAPHIC) types. A comparison may involve a predicate (=, >, <, >=, <=, <>, BETWEEN, LIKE) or a string function.

- The use of a character expression in a row partitioned table may be referred to as CPPI (Character PPI).

The most common partitioning expressions utilize RANGE_N or CASE_N expressions.

Prior to Teradata 13.10, both the CASE_N and RANGE_N functions did not allow the PPI definition of character data. This limited the useful partitioning that could be done using character columns as a standard ordering (collation) of the character data is not preserved.

Both the RANGE_N and CASE_N functions support the definition of character data in Teradata 13.10. The term "character or char" will be used to represent CHAR, VARCHAR, GRAPHIC, or VARGRAPHIC data types.

The test value of a RANGE_N function should be a simple column reference, involving no other functions or expressions. For example, if SUBSTR is added, then static partition elimination will not occur. Keep the partitioning expressions as simple as possible.

RANGE_N (SUBSTR (state_code, 1, 1) BETWEEN 'AK' and 'CA', ...

This definition will not allow static partition elimination.

Character Partitioning

Tables and non-compressed join indexes can now include partitioning on a character column. This feature may be referred to as CPPI (Character PPI).

- Prior to Teradata 13.10, partitioning expressions (`RANGE_N` and `CASE_N`) were limited to numeric or date type data.

This feature allows a partitioning expression to involve comparison of character data (`CHAR`, `VARCHAR`, `GRAPHIC`, `VARGRAPHIC`) types. A comparison may involve a predicate (`=`, `>`, `<`, `>=`, `<=`, `<>`, `BETWEEN`, `LIKE`) or a string function.

Collation and case sensitivity considerations:

- The session collation in effect when the character partitioning is created determines the ordering of data used to evaluate the partitioning expression.
- The ascending order of ranges in a character `RANGE_N` expression is defined by the session collation in effect when the partitioned table is created or altered, as well as the case sensitivity of the column or expression in the test value.
- The default case sensitivity of character data for the session transaction semantics in effect when the partitioned table is created will also determine case sensitivity of comparison unless overridden with an explicit `CAST` to a specific case sensitivity.

Character Partitioning – Example 9

In this example, the Claim table is first partitioned by claimdate (monthly intervals). Claimdate is then sub-partitioned by state codes. State codes are then sub-partitioned by the first two letters of a city name. The special partitions of NO RANGE and UNKNOWN are defined at the claimdate, statecode, and city levels.

The following queries will benefit from this type of partitioning.

```
SELECT * FROM Claim_MLPPI3
      WHERE statecode = 'GA' AND city LIKE 'a%';

SELECT * FROM Claim_MLPPI3
      WHERE claimdate = '2019-08-24' AND city LIKE 'b%';
```

The session mode when these tables were created and when these queries were executed was Teradata mode (BTET). Teradata mode defaults to "not case specific". The session collation in effect when the character partitioning was created determines the ordering of data used to evaluate the partitioning expression.

The ascending order of ranges in a character partitioning RANGE_N expression is defined by the session collation in effect when the table was created or altered, as well as the case sensitivity of the column or expression in the test value. The default case sensitivity of character data for the session transaction semantics in effect when the table was created will also determine case sensitivity of comparison unless overridden with an explicit CAST to a specific case sensitivity.

The default case sensitivity in effect when the character partitioning is created will also affect the ordering of character data for the table.

- Default case sensitivity of comparisons involving character constants is influenced by the **session mode**. String literals have a different default CASESPECIFIC attribute depending on the session mode.
 - Teradata Mode (BTET) is NOT CASESPECIFIC
 - ANSI mode is CASESPECIFIC
- If any expression in the comparison is case specific, then the comparison is case sensitive.

Character Partitioning – Example 9

In this example, 3 levels of partitioning are defined.

```
CREATE TABLE Claim_MLRP3
(claimid      INTEGER      NOT NULL,
 custid       INTEGER      NOT NULL,
 claimdate    DATE         NOT NULL,
 city         VARCHAR(30)   NOT NULL,
 statecode    CHAR(2)       NOT NULL,
 :           )
PRIMARY INDEX (claimid)
PARTITION BY
( RANGE_N
  (claimdate BETWEEN DATE '2011-01-01' and DATE '2019-12-31'
    EACH INTERVAL '1' MONTH, NO RANGE),
  RANGE_N
  (statecode BETWEEN 'A', 'D', 'I', 'N', 'T' AND 'ZZ', NO RANGE),
  RANGE_N
  (city BETWEEN 'A', 'C', 'E', 'G', 'I', 'K', 'M', 'O',
    'Q', 'S', 'U', 'W' AND 'ZZ', NO RANGE) )
UNIQUE INDEX (claimid);
```

The following queries will benefit from this type of partitioning.

- SELECT * FROM Claim_MLRP3 WHERE statecode = 'OH';
- SELECT * FROM Claim_MLRP3 WHERE statecode = 'GA' AND city LIKE 'a%';
- SELECT * FROM Claim_MLRP3 WHERE claimdate = DATE '2019-08-24'
AND city LIKE 'b%';

Summary

The customer (e.g., DBA, Database Designer, etc.) has a flexible and powerful tool to structure tables to allow automatic optimization of frequently used queries. This feature is effectively “row partitioning”. This feature allows tables to be partitioned on columns of interest, while retaining the traditional use of the primary index (PI) for data distribution and efficient access when the PI values are specified in the query.

The facing page contains a summary of the key customer benefits that can be obtained by using row partitioning.

Whether and how to partition a table is a physical design choice.

A well-chosen partitioning scheme may be able to turn many frequently run queries into partial-table scans instead of full-table scans, with much improved performance.

However, understand that there are trade-off considerations that must be understood and carefully considered to get the most benefit from the row partitioning feature.

Summary

- Improves the performance of queries that use range constraints on the range partitioning column(s) by allowing for range/partition elimination.
 - Allows primary index access and range access without a secondary index.
- General Recommendations
 - Collect statistics on system-derived column PARTITION.
 - Do not define or name a column PARTITION in a row partitioned table – you won't be able to reference the system-derived column PARTITION for the table.
 - If possible, avoid use of NO RANGE, NO RANGE OR UNKNOWN, or UNKNOWN options with RANGE_N partitioning for DATE columns.
 - Consider only having as many date ranges as needed currently plus some for the future – helps the optimizer cost plans better, especially when partitioning column is not included in the Primary Index.
- Note (as with all partitioning/indexing schemes) **there are tradeoffs** due to performance impacts on table access, joins, maintenance, and other operations.

Module 14: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 14: Review Questions

1. The Row ID for a partitioned table consists of _____ + _____ + _____.
2. The Row Key consists of the _____ + _____.
3. With a non-partitioned table, the partition number defaults to _____.
4. True or False. For a partitioned table, the partition number and the Row Hash are both used by the Message Passing Layer to determine which AMP(s) should receive the request.
5. Which two options apply to the RANGE_N expression in a partitioning expression? _____
 - a. Ranges can be listed in descending order
 - b. Allows use of NO RANGE OR UNKNOWN option
 - c. Partitioning column must be part of the Primary Index
 - d. Maximum of 9.223 quintillion partitions with Teradata 16.20
6. With a populated table, which two actions are allowed with the ALTER TABLE command. _____
 - a. Drop all of the ranges
 - b. Add or drop ranges from the partition "ends"
 - c. Change the columns that comprise the primary index
 - d. Add or drop special partitions (NO RANGE, UNKNOWN)
7. What are two advantages of partitioning a table? _____
 - a. Fast delete of rows in partitions
 - b. Fewer AMPs are involved when accessing data
 - c. Range queries can be executed without doing an all-rows scan
 - d. Faster access for a PI value access than for a non-partitioned table

Module 14: Review Questions (cont.)

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 14: Review Questions (cont.)

Given this CREATE TABLE statement, answer the following questions.

```
CREATE TABLE Orders
  (Orderid          INTEGER NOT NULL,
   Custid           INTEGER NOT NULL,
   Orderstatus      CHAR(1),
   Ordertotal       DECIMAL(9,2) NOT NULL,
   Orderdate        DATE NOT NULL FORMAT 'YYYY-MM-DD',
   Orderpriority    SMALLINT,
   Clerk            CHAR(16),
   Shippriority     SMALLINT,
   Ordercomment     VARCHAR(80))
PRIMARY INDEX (Orderid)
PARTITION BY RANGE_N (Orderdate BETWEEN
  DATE '2014-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' DAY)
UNIQUE INDEX (Orderid);
```

8. What is the name of partitioning column? _____
9. What is the time period for each partition? _____
10. What are two reasons that a USI is specified instead of defining Orderid as a UPI? _____

 - a. This is a coding style choice.
 - b. You cannot have a UPI when using a partitioned primary index.
 - c. To improve the performance of value access queries for a specific Orderid.
 - d. You cannot have a UPI if the partitioning column is not part of the primary index.

Lab Exercise 14-1

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```

```
SHOW TABLE table_name;
```

A count of rows in the Orders table is 350,000.

A count of rows in the Orders_2019 table is 100,000.

Lab Exercise 14-1

Lab Exercise 14-1

Purpose

In this lab, you will create tables with primary indexes partitioned in various ways.

What you need

Populated DS tables and empty tables in your database

Tasks

1. Using INSERT/SELECT, populate your Orders and Orders_2019 tables from the DS.Orders and DS.Orders_2019 tables, respectively. Your Orders table will have data for the years 2010 to 2018 and the Orders_2019 table will have data for 2019. Verify the number of rows in your tables.

```
SELECT COUNT(*) FROM Orders;          Count = _____ (should be 350,000)
SELECT COUNT(*) FROM Orders_2019;     Count = _____ (should be 100,000)
```

2. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_RP" that is row partitioned based on the following:

Primary Index – orderid

Partitioning column – orderdate

- From '2010-01-01' through '2019-12-31', partition by month
- Include the NO RANGE option (the UNKNOWN option is not needed for orderdate)
- Do not create any secondary indexes for this table

How many partitions are logically defined for the Orders_RP table? _____

Lab Exercise 14-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT      COUNT(*)  
FROM        table_name;
```

```
SELECT      PARTITION, COUNT(*)  
FROM        table_name  
GROUP BY    1  
ORDER BY    1;
```

To determine the number of partitions in a table.

```
SELECT      COUNT(DISTINCT(PARTITION)) FROM table_name;
```

To determine the number of orders (rows) for months in a specific year.

```
SELECT      PARTITION, COUNT(*)  
FROM        table_name  
WHERE       EXTRACT(YEAR FROM date_column_name) = 2019  
AND         EXTRACT(MONTH FROM date_column_name) between 1 and 12  
GROUP BY    1  
ORDER BY    1;
```

To determine the number of orders (rows) for a specific month:

```
SELECT      COUNT(*)  
FROM        table_name  
WHERE       EXTRACT(YEAR FROM date_column_name) = 2019  
AND         EXTRACT(MONTH FROM date_column_name) = 1;
```

Lab Exercise 14-1 (cont.)

3. INSERT/SELECT all of the rows from the Orders table into the Orders_RP table. Verify the number of rows in your table. Count = _____

How many partitions would you estimate have data at this time? _____

4. Use the PARTITION key word to list the partitions and number of rows in various partitions.

How many partitions actually have data? _____

How many rows are in the partition for January, 2010? _____

How many rows are in the partition for December, 2018? _____

5. Use INSERT/SELECT to add the rows from the Orders_2019 table to your Orders_RP table. Verify the number of rows in your table. Count = _____

Use the PARTITION key word to determine the number of partitions used and the number of rows in various partitions.

How many partitions actually have data? _____

How many rows are in the partition for the January, 2019? _____

Lab Exercise 14-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT      COUNT(*) FROM table_name;
```

To determine the number of partitions in a table.

```
SELECT      COUNT(DISTINCT(PARTITION)) FROM table_name;
```

To determine the number of orders (rows) for a specific month:

```
SELECT      COUNT(*)  
FROM        table_name  
WHERE       EXTRACT(YEAR FROM date_column_name) = 2019  
AND         EXTRACT(MONTH FROM date_column_name) = 1;
```

Other SQL queries that may be useful include:

```
SELECT      COUNT(DISTINCT(PARTITION))  
FROM        table_name  
WHERE       orderdate ... ;
```

```
SELECT      MAX(PARTITION)  
FROM        table_name;
```

```
SELECT      PARTITION, COUNT(*)  
FROM        table_name  
GROUP BY    1  
ORDER BY    1;
```

The PARTITION key word only returns partition numbers of partitions that contain rows. The following “canned” SQL can be used to return a list of partitions that are not used between the first and last used partitions.

```
SELECT p + 1      AS "The missing partitions are:"  
FROM  
    (SELECT        p1 - p2      AS p,  
                   PARTITION AS p1,  
                   MDIFF(PARTITION, 1, PARTITION) AS p2  
    FROM          table_name  
    QUALIFY       p2 > 1)  
AS temp;
```

Lab Exercise 14-1 (cont.)

6. INSERT the following 2 rows (using these values) into the Orders_RP table.

(50000, 1000, 'C', 1000, '2020-01-01', 10, 'your name', 5, 20, 'order for January in 2020')
 (50001, 1001, 'C', 1001, '2020-02-01', 10, 'EF Codd', 5, 20, 'order for February in 2020')

How many partitions are now in Orders_RP? ____

What is the partition number (highest partition #) of the NO RANGE partition? ____

7. Create a new table named "Orders_RP_ML" that has multilevel row partitioning based on the following:

Primary Index – orderid

First Level of Partitioning column – orderdate (use month ranges for all 10 years)

Include the NO RANGE option for orderdate

Second Level of Partitioning column – location (10 different order locations (1 through 10)

Place the NO RANGE and UNKNOWN rows into the same special partition for location

Unique secondary index – orderid

8. Populate the Orders_RP_ML table from the Orders and Orders_2019 tables using INSERT/SELECT and execute these commands in parallel. Verify the number of rows in Orders_RP_ML.
 Count = _____

Lab Exercise 14-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 VALUES (value1, value2, ...);
```

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```

To determine the number of partitions in a table.

```
SELECT      COUNT(DISTINCT(PARTITION)) FROM table_name;
```

To determine the number of orders (rows) for a specific month:

```
SELECT      COUNT(*)  
FROM        table_name  
WHERE       EXTRACT(YEAR FROM date_column_name) = 2019  
AND         EXTRACT(MONTH FROM date_column_name) = 1;
```

Other SQL queries that may be useful include:

```
SELECT      COUNT(DISTINCT(PARTITION))  
FROM        table_name  
WHERE       orderdate ... ;
```

```
SELECT      MAX(PARTITION)  
FROM        table_name;
```

```
SELECT      PARTITION, COUNT(*)  
FROM        table_name  
GROUP BY    1  
ORDER BY    1;
```

Lab Exercise 14-1 (cont.)

9. For the Orders_RP_ML table, use the PARTITION key word to answer the following questions.

How many partitions actually have data? _____

What is the highest partition number? _____

What is the partition number for orders in January, 2019 and location 1? _____

What is the partition number for orders in February, 2019 and location 1? _____

Is there a difference of 11 partitions between these two months? _____

Why or why not? _____

10. Before altering the table, verify the number of rows in Orders_RP. Count = _____

Use the ALTER TABLE command on Orders_RP to do the following.

- DROP RANGE (with DELETE) for year 2010
- ADD RANGE for orders that will be placed in year 2020 with an interval of 1 month

Use SHOW TABLE on Orders_RP to view the altered partitioning.

Use the PARTITION key word to list the partitions and the number of rows in various partitions.

How many partitions currently have data rows? _____

How many rows now exist in the table? _____ Has the row count changed? ____

If the row count did not change, why not? _____

How many rows are in the partition for January, 2020? _____

How many rows are in the partition for February, 2020? _____

How many rows are in the NO RANGE partition? _____

Notes



Teradata Columnar

After completing this module, you will be able to:

- **Describe the components that comprise a Row ID in a column partitioned table.**
- **Identify two advantages of creating a column partitioned table.**
- **Identify two disadvantages of creating a column partitioned table.**
- **Identify the recommended way to populate a column partitioned table.**
- **Specify how rows are deleted in a column partitioned table.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata Columnar	15-4
Teradata Columnar Benefits	15-6
Columnar Join Indexes.....	15-6
No Primary Index Table DDL	15-8
The No Primary Index Table	15-10
Column Partition Table DDL (without Auto-Compression)	15-12
Characteristics of a Columnar Table.....	15-12
Column Partition Container (No Automatic Compression).....	15-14
The Column Partition Table (without Auto-Compression)	15-16
CP Table Query #1 (without Auto-Compression)	15-18
CP Table Query #1 (without Auto-Compression)	15-20
Column Partitioning – 2 nd Example	15-22
Column Partition Table DDL (with Auto-Compression).....	15-24
Auto-Compression for CP Tables	15-26
Auto-Compression Techniques for CP Tables.....	15-28
Value List Compression (VLC) aware Bulk Evaluation.....	15-28
User-Defined Compression Techniques	15-30
Column Partition Container (Automatic Compression).....	15-32
The Column Partition Table (with Auto-Compression)	15-34
CP Table Query #2 (with Auto-Compression).....	15-36
CP Table with Row Partitioning DDL	15-38
Determining the Column Partition Level	15-38
The Column Partition Table (with Row Partitioning).....	15-40
CP Table with Multi-Column Container DDL	15-42
The CP Table with Multi-Column Container	15-44
CP Table Hybrid Row & Column Store DDL	15-46
COLUMN Format Considerations	15-46
ROW Format Considerations	15-46
The CP Table (with Hybrid Row & Column Store)	15-48
Populating a CP Table.....	15-50
INSERT-SELECT	15-50
Options	15-50
DELETE Considerations.....	15-52
The Delete Column Partition	15-52
UPDATE Considerations	15-54
USI Access	15-54
NUSI Access	15-54
CP Table Restrictions.....	15-56
Teradata 14.10 Columnar Enhancements	15-58
Teradata 15.10 Primary AMP	15-60
Teradata 15.10 Primary Index.....	15-62
Summary	15-64
Module 15: Review Questions.....	15-66
Lab Exercise 15-1	15-68

Teradata Columnar

Teradata Columnar or Column Partitioning (CP) is a physical database design implementation option (starting with Teradata 14.0) that allows single columns or sets of columns of a NoPI table to be stored in separate partitions. Column partitioning can also be applied to join indexes.

Columnar (marketing and industry name) is simply a new approach for organizing the data of a user-defined table or join index on disk. Column Partitioning is the Teradata engineering name and reflects how "columnar" is implemented in Teradata.

Teradata Columnar offers the ability to partition a table or join index by column. Teradata Columnar can be used alone or in combination with row partitioning in multilevel partitioning definitions. Column partitions may be stored using traditional 'ROW' storage or alternatively stored using the new 'COLUMN' storage option. In either case, columnar can automatically compress physical rows where appropriate.

The key benefit in defining row-partitioned (PPI) tables is when queries access a subset of rows based on constraints on one or more partitioning columns. The major advantage of using column partitioning is to improve the performance of queries that access a subset of the columns from a table, either for predicates (e.g., WHERE clause) or projections (i.e., SELECTed columns).

Because sets of one or more columns can be stored in separate column partitions, only the column partitions that contain the columns needed by the query need to be accessed. Just as row-partitioning can eliminate rows that need not be read, column partitioning eliminates columns that are not needed.

The advantages of both can be combined, meaning even less data moved and thus reduced I/O. Fewer data blocks need to be read since more data of interest is packed together into fewer data blocks.

Columnar makes more sense in CPU-rich environments because CPU cycles are needed to "glue" columns back together into rows, for compression and for different join strategies (mainly hash joins).

- **Description**

- Columnar (or Column Partitioning) is a physical database design implementation option that allows sets of columns (including just a single column) of a table or join index to be stored in separate partitions.
- This is effectively an I/O reduction feature to improve performance for suitable classes of workloads.
- This allows the capability for a table or join index to be column (vertically) partitioned, row (horizontally) partitioned or both by using the already existing multilevel partitioning capability.

- **Considerations**

- Note that column partitioning is a physical database design choice and may not be suitable for all workloads using that table/join index.
- It is especially suitable if both a small number of rows are selected and a few columns are projected.
- **When individual rows are deleted, they are not physically deleted, but are marked as deleted.**

Teradata Columnar Benefits

The facing page lists a number of Teradata Columnar benefits.

Columnar Join Indexes

A join index can also be created as column-partitioned for either a columnar table or a non-columnar table. Conversely, a join index can be created as non-columnar for either type of table as well.

Sometime within a mixed workload, some queries might perform better if data is not column partitioned and some where it is column partitioned. Or, perhaps some queries perform better with one type of partitioning on a table, whereas other queries do better with another type of partitioning. Join indexes allow creation of alternate physical layouts for the data with the optimizer automatically choosing whether to access the base table and/or one of its join indexes.

A column-partitioned join index must be a single-table, non-aggregate, non-compressed, join index with no primary index, and no value-ordering, and must include RowID of the base table. A column-partitioned join index may optionally be row partitioned. It may also be a sparse join index.

This module will only describe and include examples of base tables that utilize column partitioning.

Teradata Columnar Benefits

Benefits of using the Teradata Columnar feature include:

- **Improved query performance**

Column partitioning can be used to improve query performance via column partition elimination. Column partition elimination reduces the need to access all the data in a row while row partition elimination reduces the need to access all the rows.

- **Reduced disk space**

The feature also allows for the possibility of using a new auto-compression capability which allows data to be automatically (as applicable) compressed as physical rows are inserted into a column-partitioned table or join index.

- **Increased flexibility**

Provides a new physical database design option to improve performance for suitable classes of workloads.

- **Reduced I/O**

Allows fast and efficient access to selected data from column partitions, thus reducing query I/O.

- **Ease of use**

Provides simple default syntax for the CREATE TABLE and CREATE JOIN INDEX statements. No change is needed to queries.

No Primary Index Table DDL

The facing page simply illustrates the DDL to create a NoPI table. This example will be as a basis for multiple examples of creating tables with various column partitioning options.

No Primary Index Table DDL

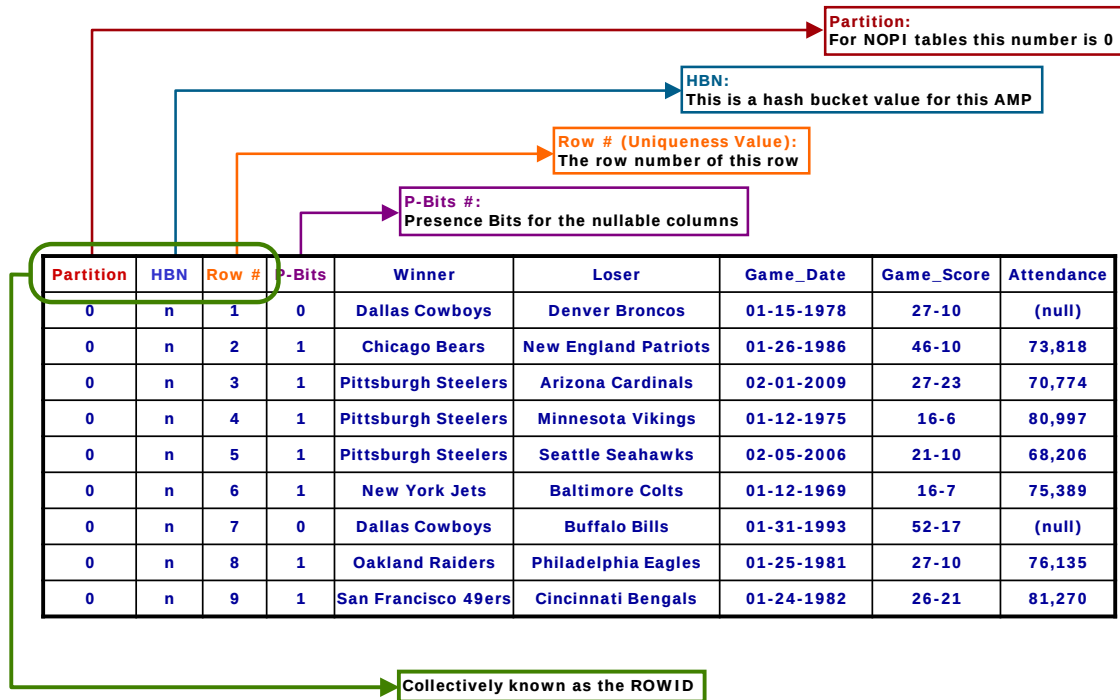
```
CREATE TABLE Super_Bowl_NoPI
(Winner          CHAR(25)          NOT NULL
, Loser          CHAR(25)          NOT NULL
, Game_Date      DATE              NOT NULL
, Game_Score     CHAR(7)           NOT NULL
, Attendance     INTEGER)
NO PRIMARY INDEX;
```

In this module, we will use a example of Super Bowl history information to simply demonstrate column partitioning.

The No Primary Index Table

The No Primary Index table is shown on the facing page.

The No Primary Index Table



Column Partition Table DDL (without Auto-Compression)

With column partitioning, each column or specified group of columns in the table can become a partition containing the column partition values of that column partition. This is the simplest partitioning approach since there is no need to define partitioning expressions, as seen in the example on the facing page.

The clause `PARTITION BY COLUMN` specifies that the table has column partitioning. Each column of this table will have its own partition and will be (by default) in column storage since no explicit column grouping is specified.

Note that a primary index is not specified since this is NoPI table. A primary index may not be specified if the table is column partitioned.

Characteristics of a Columnar Table

A table or join index that is partitioned by column has several key characteristics:

- It does not have a primary index.
- Each column partition can be composed of single or multiple columns.
- Each column partition usually consists of multiple physical rows.
- A new physical row format `COLUMN` may be utilized for a column partition. Such a physical row is called a ‘container’ and it is used to implement columnar-storage for a column partition.
- Alternatively, a column partition may also have traditional physical rows with `ROW` format. Such a physical row for columnar partitions is called a subrow. This is used to implement row-storage for a column partition.
- Note that in subsequent discussions, when ‘row storage’ or ‘row format’ is stated, it is referring to columnar partitioning with the `ROW` storage option selected. This is not to be confused with row-partitioning which we associate with a PPI table.

In a table with multiple levels of partitioning, only one level may be column partitioned. All other levels must be row-partitioned (i.e., PPI).

Column Partition Table DDL (without Auto-Compression)

```
CREATE TABLE Super_Bowl_CP_noAC
(Winner      CHAR(25) NOT NULL
,Loser       CHAR(25) NOT NULL
,Game_Date   DATE      NOT NULL
,Game_Score  CHAR(7)   NOT NULL
,Attendance  INTEGER
)
PARTITION BY COLUMN NO AUTO COMPRESS;
```

Defaults for a column partitioned table.

- Assumes NO PRIMARY INDEX.
- Single-column partitions; options include multicolumn partitions.
- Auto compression is on; NO AUTO COMPRESS turns off auto-compression for the column.
- System-determined column-store for above column partitions; options include column-store (COLUMN) or row-store (ROW).

Alternative Syntax (required in TD 14.0).

```
CREATE TABLE Super_Bowl_CP_noAC
((Winner      CHAR(25) NOT NULL) NO AUTO COMPRESS
,(Loser       CHAR(25) NOT NULL) NO AUTO COMPRESS
,(Game_Date   DATE      NOT NULL) NO AUTO COMPRESS
,(Game_Score  CHAR(7)   NOT NULL) NO AUTO COMPRESS
,(Attendance  INTEGER)      NO AUTO COMPRESS
)
PARTITION BY COLUMN;
```

(TD 14.10 Feature)
**PARTITION BY COLUMN
NO AUTO COMPRESS;**

Column Partition Container (No Automatic Compression)

In order to support columnar-storage for a column partition, a new format, referred to as a COLUMN format in the syntax, is available for a physical row.

A physical row with this format is referred to as a *container* and each container holds a series of column partition values for a column partition.

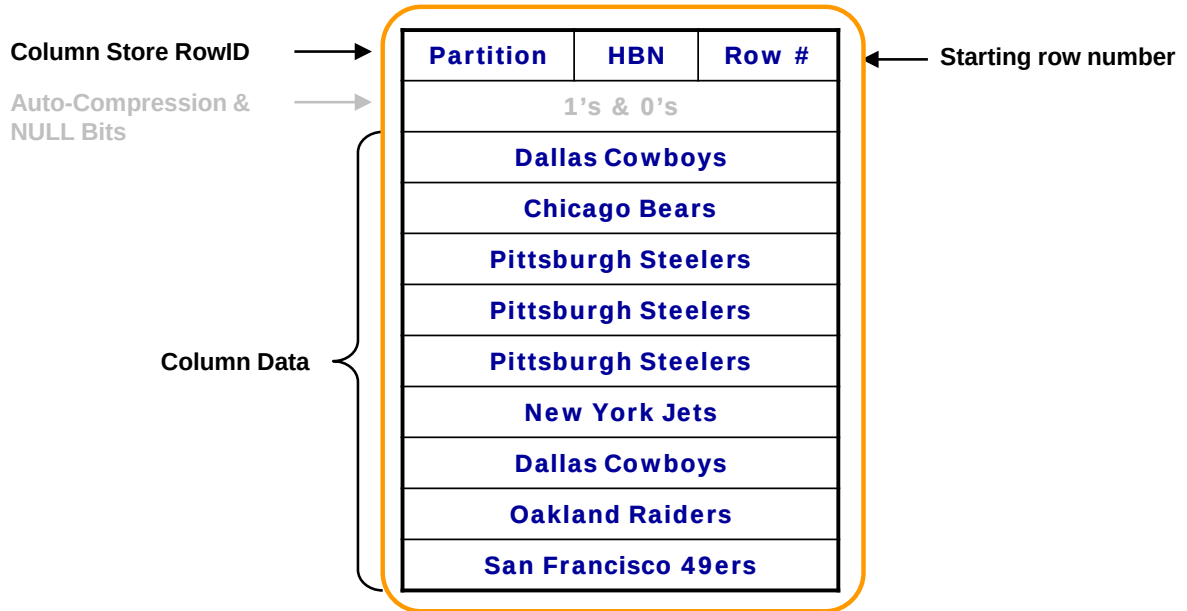
Each container is assigned a specific partition number which identifies the column or group of columns whose column partition values are held in the container. When a column partition is stored on disk, one or more containers may be needed to hold all the column partition values of the column partition. Since a container is a physical row, the size of a container is limited by the maximum physical row size.

The example on the facing page assumes that NO AUTO COMPRESS has been specified for the column.

Containers hold multiple values for the same column (or columns) of a table. For purposes of this explanation, the assumption is being made that each partition contains only a single column so a column partition value is the same as a column value. Recall that each column value belongs to a specific row and that each row is identified by a RowID consisting of a row-hash and uniqueness value. Since all of the rows on a single AMP of a NoPI table share the same row hash, the uniqueness value becomes the real differentiator. So the connection between a specific column value for a particular row on a given AMP and its uniqueness value is the key in locating the corresponding column value.

Assume that a given container holds 1000 values. The RowID of each container carries a hash bucket and uniqueness which represents the first column value entry in the container. The first value's hash bucket and uniqueness is explicit while the other values' hash bucket and uniqueness are implicit and are understood based on their position in their container. The exact location of a column partition value is known based on its relative position within the container. For example, if the uniqueness value in the container's RowID is 201 and a column partition value with a uniqueness value of 205 needs to be located, the 5th entry in that container is the corresponding column partition value.

Column Partition Container (No Automatic Compression)



Column Container is effectively a row in the partition.

The Column Partition Table (without Auto-Compression)

The result of creating a column partitioned table (as shown previously) is shown on the facing page with some sample data.

The clause `PARTITION BY COLUMN` specifies that the table has column partitioning. Each column of this table will have its own partition and will be (by default) in column storage since no explicit column grouping is specified.

The default of auto-compression is overridden for each of the columns.

The Column Partition Table (without Auto-Compression)

Column Containers

Winner	Loser	Game_Date	Game_Score	Attendance
Part 1-HBN-Row #1	Part 2-HBN-Row #1	Part 3-HBN-Row #1	Part 4-HBN-Row #1	Part 5-HBN-Row #1
1's & 0's	1's & 0's	1's & 0's	1's & 0's	1's & 0's
Dallas Cowboys	Denver Broncos	01-15-1978	27-10	(null)
Chicago Bears	New England Patriots	01-26-1986	46-10	73,818
Pittsburgh Steelers	Arizona Cardinals	02-01-2009	27-23	70,774
Pittsburgh Steelers	Minnesota Vikings	01-12-1975	16-6	80,997
Pittsburgh Steelers	Seattle Seahawks	02-05-2006	21-10	68,206
New York Jets	Baltimore Colts	01-12-1969	16-7	75,389
Dallas Cowboys	Buffalo Bills	01-31-1993	52-17	(null)
Oakland Raiders	Philadelphia Eagles	01-25-1981	27-10	76,135
San Francisco 49ers	Cincinnati Bengals	01-24-1982	26-21	81,270
⋮	⋮	⋮	⋮	⋮

Column containers are effectively separate rows in a NoPI table.

CP Table Query #1 (without Auto-Compression)

One of the key advantages of column partitioning is opportunity for reduced I/O. This can be realized only if a subset of the columns in a table are read and if those column values are held in separate column partitions. Data is stored on disk by partition, so when partition elimination takes place, data blocks in the eliminated partitions are simply not read.

There are three ways to initiate read access to data within a column-partitioned table:

- A full column partition scan
- Indexed access (using a secondary, join index, or hash index),
- A RowID join.

Both unique and non-unique secondary indexes are allowed on column-partitioned tables, as are join indexes and hash indexes.

Queries best suited for scanning a column-partitioned table are queries that:

- Contain one or a few predicates that are very selective in combination.
- Require a small enough number of columns to be accessed that the caches required to support their consolidation can be held in memory.

CP Table Query #1 (without Auto-Compression)

Which teams have lost to the "Dallas Cowboys" in the Super Bowl?

Winner	Loser	Game_Date	Game_Score	Attendance
Part 1-HBN-Row #1	Part 2-HBN-Row #1	Part 3-HBN-Row #1	Part 4-HBN-Row #1	Part 5-HBN-Row #1
1's & 0's	1's & 0's	1's & 0's	1's & 0's	1's & 0's
Dallas Cowboys	Denver Broncos	01-15-1978	27-10	(Null)
Chicago Bears	New England Patriots	01-26-1986	46-10	73,818
Pittsburgh Steelers	Arizona Cardinals	02-01-2009	27-23	70,774
Pittsburgh Steelers	Minnesota Vikings	01-12-1975	16-6	80,997
Pittsburgh Steelers	Seattle Seahawks	02-05-2006	21-10	68,206
New York Jets	Baltimore Colts	01-12-1969	16-7	75,389
Dallas Cowboys	Buffalo Bills	01-31-1993	52-17	(Null)
Oakland Raiders	Philadelphia Eagles	01-25-1981	27-10	76,135
San Francisco 49ers	Cincinnati Bengals	01-24-1982	26-21	81,270

⋮

⋮

⋮

⋮

⋮

Only the accessed columns are needed.

CP Table Query #1 (without Auto-Compression)

If indexing is not available, Teradata can access data in a CP table by scanning a column partition on all the AMPs in parallel.

In the example on the facing page, the “Winner” column containers are scanned for “Dallas Cowboys”.

The following describes the scanning of CP data:

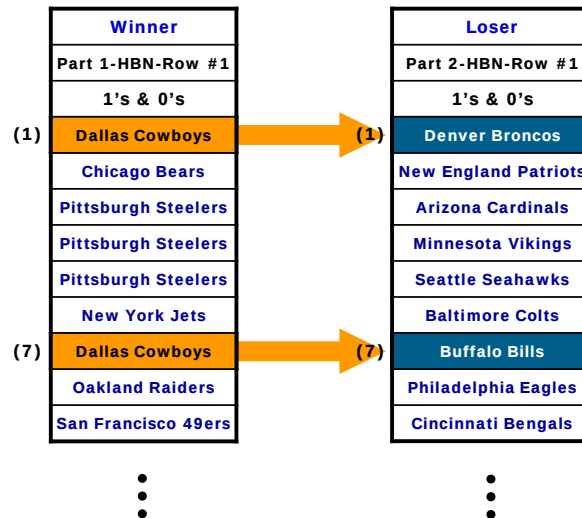
1. Columns within the table definition that aren’t referenced in the query are ignored due to partition elimination.
2. If there is a predicate column in the query, its column partition is read.
3. Values within the predicate column partition are examined and compared against the value passed in the query WHERE clause.
4. Each time a qualifying value is located, the next step is building up a row for the output spool.
5. All the column partition values for a logical row have the same RowID except for the column partition number. The RowID associated with each predicate column value that matches the constraint in the query becomes the link to other column partition values of the same logical row by simply modifying the column partition number of the RowID to the column partition number for each of these other column partition values.

If there is more than one predicate column in the query that can be used to disqualify rows, the column for one of these predicates is chosen and its column partition is scanned. Statistics, as well as other heuristics, are used by the optimizer to pick the most selective and least costly predicate. Once that decision has been made, only that single column partition needs to be scanned.

If there are no useful predicate columns in the query (for instance, OR’ed predicates), one column partition is chosen to be scanned and for each of its column partition values additional corresponding column partition values are accessed until either predicate evaluation disqualifies the logical row or all the projected column values have been retrieved and brought together to form rows for the output spool.

CP Table Query #1 (without Auto-Compression)

Which teams have lost to the "Dallas Cowboys" in the Super Bowl?



The relative row number in each container is used to access the data.

Column Partitioning – 2nd Example

The facing page illustrates a second example of a table with column partitioning and assumes the automatic compression is not being used.

The table at the top of the example is implemented as a NoPI table. The data in this table is used to populate the partition of a column partitioned table.

Since this table is partitioned by column, the first column will effectively have values placed in partition #1; the second column will have values placed in partition #2, etc.

A row in a CP table is effectively known as a "container". The typical size of a container (or row) is approximately 8K.

The first column has a data type of DECIMAL (15, 0), therefore 8 bytes of storage is needed for the each phone number stored in the container. Approximately 1000 values (e.g., phone numbers) can be stored in each container for the first partition (e.g., first column or originating #).

The second column also has a data type of DECIMAL (15, 0), therefore 8 bytes of storage is needed for the each phone number stored in the container. Approximately 1000 values (e.g., phone numbers) can be stored in each container for the second partition (e.g., second column or terminating #).

The third column has a data type of TIMESTAMP WITH TIMEZONE, there 12 bytes of storage is needed for the each time stamp stored in the container. Approximately 667 values (e.g., time stamps) can be stored in each container for the third partition (e.g., third column or Call TS).

As always, a group of rows (containers) in a CP table are group into blocks, blocks are grouped in a cylinder, and the cylinder will have a cylinder index with entries for each of the data blocks within the cylinder.

Question #1: For the Call Duration column (4th column), how many containers would be needed for this example with 2000 phone calls? Assume containers are approximately 8K.

Answer #1: The answer is 1. Since the Duration column is type INTEGER (4 bytes), then 2000 values would fit into 1 container of 8K.

Question #2: For the column named Col_100 (6th column), what is the maximum number of containers needed for this example with 2000 phone calls? Assume containers are approximately 8K.

Answer #2: The answer is 20. Since Col_100 is a type VARCHAR (80), if every value was 80 bytes, then 100 values would fit into 1 container of 8K. Divide 2000 values by 100 values/container which yields 20 containers. Many of the values will likely be less than 80 bytes; therefore fewer than 20 containers will probably be required.

Column Partitioning – 2nd Example

CDR_NoPI	Part #, HBN, Uniq, PB	Orig#	Term#	Call_TS	Duration	Col_5	Col_100
		DEC(15,0)	DEC(15,0)	Timestamp	INT	BIGINT	VARCHAR(80)
	0, HBN, 00000001, PB	18584850001	18584856001	TS000001	100	1001	A
	0, HBN, 00000002, PB	18584850002	18584856002	TS000002	20	1001	B
	0, HBN, 00000003, PB	18584850003	18584856003	TS000003	33	1002	C
	:	:	:	:	:	:	:
	0, HBN, 00001999, PB	18584851999	18584857999	TS001999	403	2000	E
	0, HBN, 00002000, PB	18584852000	18584858000	TS002000	75	2001	F
CDR_CP	Part 1, HBN, Uniq, PB	1000 Values in each row from Orig# (each value uses 8 bytes)					
	1, HBN, 00000001, PB	18584850001	18584850002	18584850003	:	18584850999	18584851000
	1, HBN, 00001001, PB	18584851001	18584851002	18584851003	:	18584851999	18584852000
	2 Containers						
CDR_CP	Part 2, HBN, Uniq, PB	1000 Values in each row from Term# (each value uses 8 bytes)					
	2, HBN, 00000001, PB	18584856001	18584856002	18584856003	:	18584856999	18584857000
	2, HBN, 00001001, PB	18584857001	18584857002	18584857003	:	18584857999	18584858000
	2 Containers						
CDR_CP	Part 3, HBN, Uniq, PB	667 Values in each row from Call_Timestamp (each value uses 12 bytes)					
	3, HBN, 00000001, PB	TS000001	TS000002	TS000003	:	TS000666	TS000667
	3, HBN, 00000668, PB	TS000668	TS000669	TS000670	:	TS001333	TS001334
	3, HBN, 00001335, PB	TS001335	TS001336	TS001337	:	TS001999	TS002000
	3 Containers						

Column Partition Table DDL (with Auto-Compression)

The DDL to create a column partitioned table with auto-compression is shown on the facing page. Each column will be maintained in a separate partition.

The clause `PARTITION BY COLUMN` specifies that the table has column partitioning. Each column of this table will have its own partition and will be (by default) in column storage since no explicit column grouping is specified.

Column Partition Table DDL

(with Auto-Compression)

```
CREATE TABLE Super_Bowl_CP
(Winner          CHAR(25)          NOT NULL
,Loser           CHAR(25)          NOT NULL
,Game_Date       DATE              NOT NULL
,Game_Score      CHAR(7)           NOT NULL
,Attendance      INTEGER)
PARTITION BY COLUMN;
```

Note: Auto Compression is on by Default.

Auto-Compression for CP Tables

Auto-compression is a completely transparent compression option for column partitions. It is applied to a container when a container is full after appending some number of column partition values without auto-compression by an INSERT or UPDATE statement. Each container is assessed separately to see how, and if, it can be compressed.

Several available compression techniques are considered for compressing a container but, unless there is some size reduction, no compression is performed. If a container is compressed, the needed data is automatically uncompressed as it is read.

Auto-compression happens automatically and is most effective when the column partition is based on a single column only, and less effectively as more columns are included in the column partition.

User-defined compression, such as multi-value or algorithmic compression that is already defined by the user is honored and carried forward if it helps compress the container. If block level compression is specified, it applies for data blocks holding the physical rows of the table independent of whether auto-compression is applied or not.

Auto Compression

- When a column partition is defined to have auto-compression (i.e., the NO AUTO COMPRESS option is not specified), data is compressed by the system as physical rows that are inserted into a column-partitioned table or join index.
- For some values, there is no applicable compression technique that reduces the size of the physical row and the system will determine not to compress the values for that physical row.
- The system decompresses any compressed column-partition values when they are retrieved.
- Auto-compression is most effective for a column partition with a single column and COLUMN format.
- There is overhead in determining whether or not a physical row is to be compressed and, if it is to be compressed, what compression techniques are to be used.
- This overhead can be eliminated by specifying the NO AUTO COMPRESS option for the column partition.

Auto-Compression Techniques for CP Tables

The facing page lists and briefly describes each of the auto-compression techniques that Teradata may utilize.

Value List Compression (VLC) aware Bulk Evaluation

This Teradata 16.0 internal performance feature (VLC/RL aware bulk evaluation) was implemented to improve CPU performance by evaluating on value list and, value list plus run length (RL) compressed columns, and run length compressed columns in a columnar table. Increased CPU performance is achieved by reducing CPU consumption because column data does not need to be decompressed.

This form of predicate evaluation is possible when the columns in the table are compressed using either of the following:

- a) Container VLC or Container VLC plus run length compression or Container run length compression.
- b) Table header VLC with container VLC compression or VLC plus run length compression or container run length compression.

This feature supports VLC aware compressed bulk evaluation (EVL) for the entire relational search based scans (<, >, <=, >=, =, <>, IS NULL, IS NOT NULL, IN LIST and BETWEEN) on all data types.

Note: After predicate evaluation, other operations such as projection may decompress the data.

Details on the Predicate Evaluation Process are as follows:

- Within an AMP, regular bulk EVL for user given predicate is used to evaluate against dictionary entries of each dictionary structure – table header, container row static dictionary and dynamic dictionary. If the EVL returns true for a data value (for any of these), the index bit position in the bit map is set and a dictionary filter bitmap is generated.
- The same bulk EVL is also used to locate the actual value in the column. Instead of decompressing the value, only the index bits are read. The index is used to check the corresponding dictionary filter bitmap. If the bit is set in the dictionary filter bit map, then the corresponding bit is set in the result bit vector. If bit is not set, then it indicates the actual comparison failed and the result bit is set to zero.

If the container row is not compressed, the predicate evaluation fallbacks to regular EVL in which actual comparison of uncompressed values occur.

- **Run-Length Encoding**

Each series of one or more column-partition values that are the same are compressed by having the column-partition value occur once with an associated count of the number of occurrences in the series.

- **Local Dictionary Compression**

This is similar to a user-specified value-list compression for a column. Often occurring column-partition values within a physical row are placed in a value-list dictionary local to the physical row.

- **Trim Compression**

Trim high-order zero bytes of numeric values and trailing pad bytes of character and byte values with bits to indicate how many bytes were trimmed or what the length is after trimming.

- **Null Compression**

Similar to null compression (COMPRESS NULL) for a column except applied to a column-partition value. A single-column or multicolumn-partition value is a candidate for null compression if all the column values in the column-partition value are null (this also means all these columns must be nullable).

- **Delta on Mean Compression**

Delta on Mean compression computes the mean/average of all the values in the column container. This mean value is saved and stored in the container. After Delta on Mean compression, the value that is stored for a row is the difference with the mean. So for instance, if the average is say 100 and the four values in four different rows are 99, 98, 101, and 102. Then the values stored are -1, -2, 1, and 2.

- **Unicode to UTF8 Compression**

For a column defined with UNICODE character set but where the value consists of ASCII characters, compress the Unicode representation (2 bytes per character) to UTF8 (1 byte per character).

User-Defined Compression Techniques

User-defined compression, such as multi-value or algorithmic compression that is already defined by the user is honored and carried forward if it helps compress the container.

If block level compression is specified, it applies for data blocks holding the physical rows of the table independent of whether auto-compression is applied or not.

Note that auto-compression is applied locally to a container based on column partition values (which may be multicolumn) while user-specified MVC and ALC are applied globally for a column and are applicable to both containers and subrows.

Auto-compression is differentiated from block level compression in several key ways:

- Auto-compression requires no parameter setting, but rather is completely transparent to the user while block level compression is a result of the appropriate settings of parameters.
- Auto-compression acts on a container (a physical row) while block level compression acts on a data block (which consists of one or more physical rows).
- Decompressing a column partition value in a container has little overhead while software-based block level compression incurs noticeable decompression overhead.
- Only column partition values that are needed by the query are decompressed. BLC has to decompress the entire data block even if only one or a few values are needed from the data block.
- Determining the auto-compression to use for a container, compressing a container, and compressing additional values to be inserted into the container can cause an increase in the CPU needed for appending values to column partitions.

You can expect additional CPU to be required when inserting rows into a CP table that uses auto-compression. This is similarly to the increase in CPU if MVC or ALC compression is added to the columns.

All the current compression techniques available in Teradata today, can be leveraged and used for column partitioned tables.

- **Dictionary-Based Compression:**

Allows end-users to identify and target specific values that would be compressed in a given column. Also known as, Multi-value Compression.

- **Algorithmic Compression:**

Allows users the option of defining compression/decompression algorithms that would be implemented as UDFs and that would be specified and applied to data at the column level in a row. Teradata provides three compression/decompression algorithms that offer compression for UNICODE and LATIN data columns.

- **Block-Level Compression:**

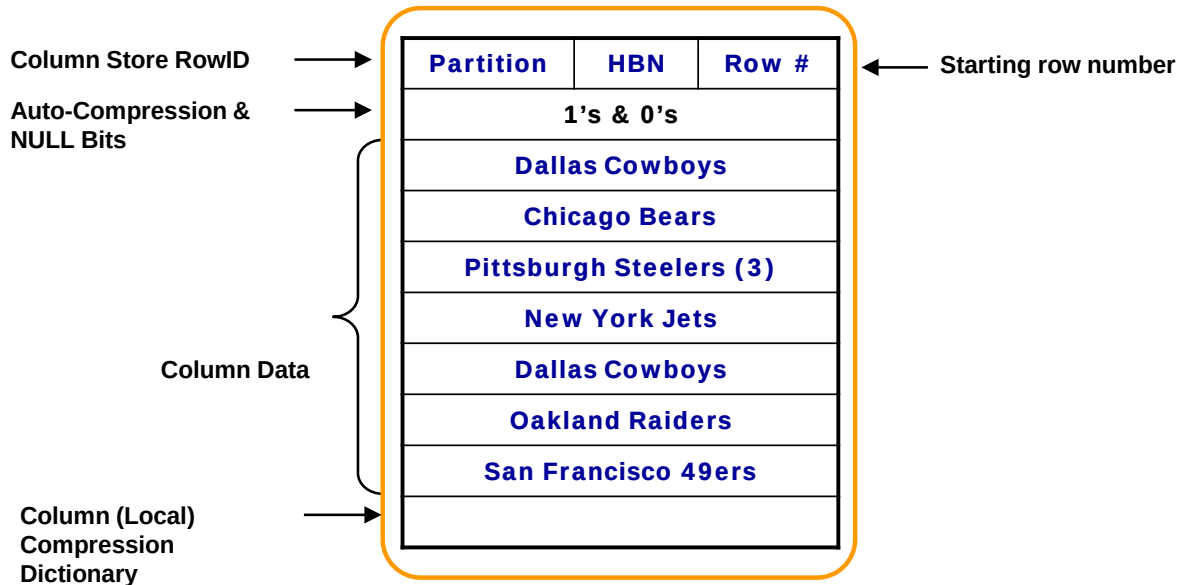
Feature provides the capability to perform compression on whole data blocks at the file system level before the data blocks are actually written to storage.

Column Partition Container (Automatic Compression)

In order to support columnar-storage for a column partition, a new format, referred to as a COLUMN format in the syntax, is available for a physical row.

The example on the facing page assumes that automatic compression is on for the column.

Column Partition Container (Automatic Compression)



Column Container is effectively a row in the partition.

The Column Partition Table (with Auto-Compression)

The result of creating a column partitioned table with auto-compression is shown on the facing page.

The Column Partition Table (with Auto-Compression)

Winner	Loser	Game_Date	Game_Score	Attendance
Part 1-HBN-Row #1	Part 2-HBN-Row #1	Part 3-HBN-Row #1	Part 4-HBN-Row #1	Part 5-HBN-Row #1
1's & 0's	1's & 0's	1's & 0's	1's & 0's	1's & 0's
Dallas Cowboys	Denver Broncos	01-15-1978	27-10	(Null)
Chicago Bears	New England Patriots	01-26-1986	46-10	73,818
Pittsburgh Steelers	Arizona Cardinals	02-01-2009	27-23	70,774
Pittsburgh Steelers	Minnesota Vikings	01-12-1975	16-6	80,997
Pittsburgh Steelers	Seattle Seahawks	02-05-2006	21-10	68,206
New York Jets	Baltimore Colts	01-12-1969	16-7	75,389
Dallas Cowboys	Buffalo Bills	01-31-1993	52-17	(Null)
Oakland Raiders	Philadelphia Eagles	01-25-1981	27-10	76,135
San Francisco 49ers	Cincinnati Bengals	01-24-1982	26-21	81,270
⋮	⋮	⋮	⋮	⋮
Run-Length Encoding	Trim Trailing Spaces	No Compression	Local Dictionary Compression (27-10 is compressed)	Null Compression

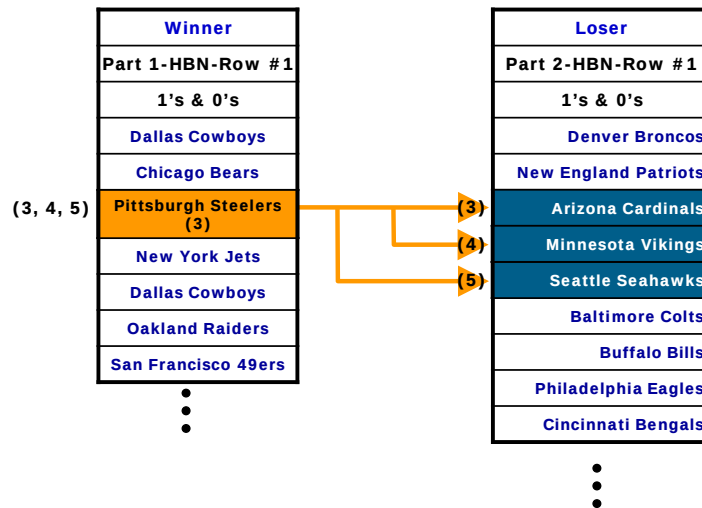
Columnar compression is based on each Container. Therefore, each Container may have different compression characteristics and even different compression methods.

CP Table Query #2 (with Auto-Compression)

The Pittsburgh Steelers team was compressed, but effectively represented 3 values in the container. These 3 values correspond to 3, 4, and 5 in the other container (Loser column).

CP Table Query #2 (with Auto-Compression)

Which teams have lost to the "Pittsburgh Steelers" in the Super Bowl?



CP Table with Row Partitioning DDL

Row partitioning can be combined with column partitioning on the same table. This allows queries to read only non-eliminated combined partitions. Such partitions are defined by the intersection of the columns referenced in the query and any partitioning column selection criteria.

There is usually an advantage to putting the column partitioning at level one of the combined partitioning scheme.

The DDL to create a column partitioned table with auto-compression and Row partitioning is shown on the facing page.

If the table or join index is defined with multiple partitioning levels, you can (and possibly should consider) explicitly specify the number of partitions per partitioning level using the ADD option. If you do not, Teradata reserves any excess partitions for partitioning level 1 which is usually the COLUMN partitioning level, and these partitions cannot be added to any other partitioning level once the table or join index is created.

The maximum number of partitions that are available for the level 1 partition from excess partitions depends on whether you have defined the object with 2-byte partitioning or with 8-byte partitioning.

Determining the Column Partition Level

It is initially recommended that column partitioning either be defined as the first level or, if not as the first level, at least as the second level. When column partitioning is defined as the first level it is easier for the file system to locate related data that is from the same logical row of the table. When column partitioning is defined at a lower level, more boundary checks have to be made, possibly causing an impact on performance.

If you are inserting a new table row, it takes more effort if the column partitioning is not the first level. Values of columns from the newly-inserted table row need to be appended at the end of each column partition. If column-partitioning is not first, it is necessary to read through several combined partitions to find the correct container that represents the end point.

On the other hand, if you have row partitioning at the second or a lower level partitioning level so that column partitioning can be at the first level, this can be less efficient when row partition elimination based on something like a date range is taking place.

CP Table with Row Partitioning DDL

```
CREATE TABLE Super_Bowl_CP_RP
(Winner          CHAR(25)          NOT NULL
,Loser           CHAR(25)          NOT NULL
,Game_Date       DATE              NOT NULL
,Game_Score      CHAR(7) NOT NULL
,City            CHAR(40))
PARTITION BY
(COLUMN
,RANGE_N(Game_Date BETWEEN
          DATE '1960-01-01' and DATE '2059-12-31'
          EACH INTERVAL '10' YEAR));
```

Note: Auto Compression is on by Default.

The Column Partition Table (with Row Partitioning)

The result of creating a column partitioned table with auto-compression and row partitioning is shown on the facing page.

The Column Partition Table (with Row Partitioning)

- In the 1970s, which teams won Super Bowls, who were the losing teams , and what was the date the game was played?

Winner	Loser	Game_Date	Game_Score	City
Part 1-HBN-Row #1	Part 2-HBN-Row #1	Part 3-HBN-Row #1	Part 4-HBN-Row #1	Part 5-HBN-Row #1
1's & 0's	1's & 0's	1's & 0's	1's & 0's	1's & 0's
New York Jets	Baltimore Colts	01-12-1969	16-7	Miami, FL

Winner	Loser	Game_Date	Game_Score	City
Part 11-HBN-Row #1	Part 12-HBN-Row #1	Part 13-HBN-Row #1	Part 14-HBN-Row #1	Part 15-HBN-Row #1
1's & 0's	1's & 0's	1's & 0's	1's & 0's	1's & 0's
Dallas Cowboys	Denver Broncos	01-15-1978	27-10	New Orleans, LA (2)
Pittsburgh Steelers	Minnesota Vikings	01-12-1975	16-6	

⋮

⋮

⋮

⋮

⋮

Winner	Loser	Game_Date	Game_Score	City
Part 41-HBN-Row #1	Part 42-HBN-Row #1	Part 43-HBN-Row #1	Part 44-HBN-Row #1	Part 45-HBN-Row #1
1's & 0's	1's & 0's	1's & 0's	1's & 0's	1's & 0's
Pittsburgh Steelers	Seattle Seahawks	02-05-2006	21-10	Detroit, MI

CP Table with Multi-Column Container DDL

When a table is defined with column partitioning, by default each column becomes its own column partition. However, it is possible to group multiple columns into a single partition.

This has the result of fewer column partitions with more data held within each column partition.

Grouping columns into fewer column partitions may be appropriate in these situations:

- When the table has a large number of columns (having fewer column partitions may improve the performance of INSERT-SELECT and UPDATE statements).
- When access to the table often involves a large percentage of the columns and the access is not very selective.
- When a common subset of columns are frequently accessed together.
- When a multicolumn NUSI is created on a group of columns.
- There are too few available column partition contexts to access all the needed column partitions for queries.

Note that auto-compression will probably be less effective if columns are grouped together instead of being in their own column partitions.

CP Table with Multi-Column Container DDL

```
CREATE TABLE Super_Bowl_CP_MC_noAC
( Winner          CHAR(25)          NOT NULL
, Loser           CHAR(25)          NOT NULL
, Game_Date       DATE              NOT NULL
, Game_Score      CHAR(7)           NOT NULL
, Attendance      INTEGER)
PARTITION BY COLUMN NO AUTO COMPRESS
( Winner
, Loser
, (Game_Date
, Game_Score
, Attendance))
;
```

This example is created without Auto-Compression for demonstration purposes.

Recommendation:
The group of multiple columns should be less than 256 bytes.

Watch the difference between 'Projection' and 'Predicate'.

If you are always projecting three columns, it may make sense to group these columns into one Container. If, however, one of these columns is used in a WHERE Predicate, then it may be better to place this column into its own Container.

The CP Table with Multi-Column Container

The example on the facing page illustrates a CP table that has a multi-column container.

The CP Table with Multi-Column Container

Single Column Containers

Winner
Part 1-HBN-Row #1
1's & 0's
Dallas Cowboys
Chicago Bears
Pittsburgh Steelers
Pittsburgh Steelers
Pittsburgh Steelers
New York Jets
Dallas Cowboys
Oakland Raiders
San Francisco 49ers

⋮

Loser
Part 2-HBN-Row #1
1's & 0's
Denver Broncos
New England Patriots
Arizona Cardinals
Minnesota Vikings
Seattle Seahawks
Baltimore Colts
Buffalo Bills
Philadelphia Eagles
Cincinnati Bengals

⋮

Multi-Column Container

Game_Date	Game_Score	Attendance
Part 3-HBN-Row #1		
1's & 0's		
01-15-1978	27-10	(null)
01-26-1986	46-10	73,818
02-01-2009	27-23	70,774
01-12-1975	16-6	80,997
02-05-2006	21-10	68,206
01-12-1969	16-7	75,389
01-31-1993	52-17	(null)
01-25-1981	27-10	76,135
01-24-1982	26-21	81,270

⋮

⋮

⋮

General recommendations:

- If you have a lot of columns in a table, then multi-column containers may be needed.
- Multi-column containers will not compress as well as single-column containers.
- If you select any column in a multi-column container you will get all of the other columns.

CP Table Hybrid Row & Column Store DDL

The example on the facing page illustrates the DDL to create a column partitioned table that has a combination of row and column storage.

COLUMN Format Considerations

The COLUMN format packs column partition values into a physical row, referred to as a container, up to a system-determined limit. Whether or not to change a column partition to use ROW format depends on whether the benefit of row header compression and auto-compression can be realized or not.

A row header occurs once per container, with the RowID of the first column partition value becoming the RowID of the container itself. In a column-partitioned table, each column partition value is assigned its own RowID, but in a container these RowIDs are implicit except for the first one specified in the header. The uniqueness value can be determined from the position of a column partition value relative to the first column partition value. Thus the row id for each value in the container is implicitly available and an explicit RowID does not need be carried for each individual column value in the container.

If many column partition values can be packed into a container, this form of compression (referred to as row header compression) can reduce the space needed for a column-partitioned table compared to the table without column partitioning. If only a few column partition values (because they are wide) can be placed in a container, there can actually be an increase in the space needed for the table compared to the table without column partitioning. In this case, ROW format may be more appropriate.

ROW Format Considerations

A subrow, on the other hand, has a format that is the same as traditional row (except it only has the values of a subset of the columns). Subrows are appropriate when column partition values are very wide and you expect only one or a few values to fit in a columnar container.

In this case, auto-compression and row header compression using COLUMN format might not be very effective. ROW format provides quicker access to specific values because no search through a physical row is required to find only one value. Each column partition value is in its own subrow with a row header. Subrows are not subject to auto-compression but may be in the future.

CP Table Hybrid Row & Column Store DDL

```
CREATE TABLE Super_Bowl_CP_RF_noAC
(Winner      CHAR(25) NOT NULL
,Loser       CHAR(25) NOT NULL
,Game_Date   DATE           NOT NULL
,Game_Score  CHAR(7)        NOT NULL
,Attendance  INTEGER
,City        CHAR(40))
PARTITION BY COLUMN NO AUTO COMPRESS
(Winner
,Loser
,ROW (Game_Date
      ,Game_Score
      ,Attendance
      ,City)
);
```

This example is created without Auto-Compression for demonstration purposes.

This example illustrates the syntax to create a row store, but in reality you would only define the row format if the set of columns was greater than 256 bytes.

General recommendations:

- A column (or set of columns) should be at least 256 bytes wide before considering ROW format.
- Row stores will take up more space, but act like a row in terms of retrieving data.
- Each row will have a row header and require more space.

The CP Table (with Hybrid Row & Column Store)

As an alternative to COLUMN format, column partition values may be held in a physical row using what is known in Teradata syntax as ROW format. The type of physical row supports row-storage for a column partition and is referred to as a subrow. Each subrow holds one column partition value for a column partition. A subrow has the same format as regular row except that it is generally a subset of the columns for a table row instead of all the columns. Just like a container, each subrow is assigned to a specific partition. One or more subrows may be needed to hold the entire column partition. Since a subrow is a physical row, the size of a subrow is limited by the maximum physical row size.

A column partition may have COLUMN format or ROW format but not a mix of both. However, different column partitions in column-partitioned table may have different formats.

The CP Table (with Hybrid Row & Column Store)

Column and Row Store in "one" table.

Column Store Containers

Winner
Part 1-HBN-Row #1
1's & 0's
Dallas Cowboys
Chicago Bears
Pittsburgh Steelers
Pittsburgh Steelers
Pittsburgh Steelers
New York Jets
Dallas Cowboys
Oakland Raiders
San Francisco 49ers

Loser
Part 2-HBN-Row #1
1's & 0's
Denver Broncos
New England Patriots
Arizona Cardinals
Minnesota Vikings
Seattle Seahawks
Baltimore Colts
Buffalo Bills
Philadelphia Eagles
Cincinnati Bengals

Row Store

Partition	HBN	Row #	Game_Date	Game_Score	Attendance	City
0	n	1	01-15-1978	27-10	(null)	New Orleans, LA
0	n	2	01-26-1986	46-10	73,818	New Orleans, LA
0	n	3	02-01-2009	27-23	70,774	Tampa, FL
0	n	4	01-12-1975	16-6	80,997	New Orleans, LA
0	n	5	02-05-2006	21-10	68,206	Detroit, MI
0	n	6	01-12-1969	16-7	75,389	Miami, FL
0	n	7	01-31-1993	52-17	(null)	Pasadena, CA
0	n	8	01-25-1981	27-10	76,135	New Orleans, LA
0	n	9	01-24-1982	26-21	81,270	Pontiac, MI

⋮

⋮

⋮

⋮

⋮

⋮

⋮

⋮

⋮

Populating a CP Table

INSERT-SELECT

INSERT-SELECT is the expected and most efficient method of loading data into a column-partitioned table. If the data originates from an external source, FastLoad can be used to load it into a staging table from which the INSERT-SELECT can take place.

If the source was a SELECT that included several joins and as a result skewed data was produced, options can be added to the INSERT-SELECT statement to avoid a skewed column-partitioned table and improve the effectiveness of auto-compression:

Options

HASH BY (RANDOM or hash_spec_list):

The selected rows are redistributed by the hash value of the expressions in the hash_spec_list. Alternatively, HASH BY RANDOM can be specified to have data blocks redistributed randomly. It is important that a column or columns be selected that distributes well if the HASH BY option is used.

LOCAL ORDER BY:

A local sort is done on each AMP before physically storing the rows. This could help auto-compression to be more effective by ensuring that like values of the sorting columns appear together.

During an INSERT-SELECT process, each source row is read, and its columns individually appended to the column partitions to which they belong. As many column partition values as can fit are built up simultaneously in memory, and written out to disk when the buffer is filled.

If the column-partitioned table being loaded has a large number of columns, additional passes of the source table may be required to append all of the columns to their respective column partitions.

Populating a CP Table

```
CREATE TABLE Super_Bowl_Staging
(Winner      CHAR(25) NOT NULL
,Loser       CHAR(25) NOT NULL
,Game_Date   DATE      NOT NULL
,Game_Score  CHAR(7)  NOT NULL
,Attendance  INTEGER)
NO PRIMARY INDEX;
```

```
CREATE TABLE Super_Bowl_CP
(Winner      CHAR(25) NOT NULL
,Loser       CHAR(25) NOT NULL
,Game_Date   DATE      NOT NULL
,Game_Score  CHAR(7)  NOT NULL
,Attendance  INTEGER)
PARTITION BY COLUMN;
```

1. Load data into staging table.
2. `INSERT INTO Super_Bowl_CP .... SELECT * FROM Super_Bowl_Staging ...`

DELETE Considerations

Rows can be deleted from a column-partitioned table using the DELETE ALL, or selectively using DELETE. DELETE ALL uses the standard fast-path delete as would be done on a primary-indexed table. If a column-partitioned table also happens to include row partitioning, the same fast-path delete can be applied to one or more row partitions. Space is immediately reclaimed.

The selective DELETE, in which only one or a few rows of the table are deleted, requires a scan of a column partition or indexed access to the column-partitioned table. In this case, the row being deleted is not physically removed, but only flagged as having been deleted. The space taken by a row being deleted is scattered across multiple column partitions and is not reclaimed at the time of the deletion. This form of delete should only be used to delete a small percentage of rows.

During a delete operation, all large objects are immediately deleted, as are entries in secondary indexes. Join indexes are updated to reflect the change as it happens.

Note: Starting with Teradata 15.10 data from column partitions with ROW format are also immediately deleted. In addition, if all the column partitions have ROW format, the delete column partition is not used and is empty.

The Delete Column Partition

Each column-partitioned table has one delete column partition, in addition to the user-specified column partitions. It holds information about deleted rows so they do not get included in an answer set. When a single row delete takes place in a column-partitioned table, rather than removing each deleted value across all the column partitions, which would involve multiple physical row updates, a single action is performed. One bit in the delete column partition is set as an indication that the hash bucket and uniqueness associated with the table row has been deleted.

This delete column partition is accessed any time a query is made against a column-partitioned table without indexed access. At the time a column partition is scanned, the delete column partition is checked to make sure a table row being requested by the query has not been deleted (if it has, the value is skipped). This additional partition access can be observed in the EXPLAIN text.

DELETE Considerations

- DELETE ALL uses the standard fast-path delete as would be done on a primary-indexed table.
 - If a CP table also happens to include row partitioning, the same fast-path delete can be applied to one or more row partitions. Space is immediately reclaimed.
- The selective DELETE, in which only one or a few rows of the table are deleted, requires a scan of a column partition or indexed access to the column-partitioned table.
 - In this case, the row being deleted is not physically removed, but only flagged as having been deleted.
 - This form of delete should only be used to delete a small percentage of rows.
- The Delete Column Partition – each column-partitioned table has one delete column partition, in addition to the user-specified column partitions. It holds information about deleted rows so they do not get included in an answer set.
 - One bit in the delete column partition is set as an indication that the hash bucket and uniqueness associated with the table row has been deleted.

UPDATE Considerations

Updating rows in column partitioned table requires a delete and an insert operation. It involves marking the appropriate bit in the delete column partition, and then re-inserting columns for the new updated version of the table row. The cost of this update is less severe than a Primary Index update (also a delete plus insert) because in the column-partitioned table update, the deletion and re-insertion takes place on the same AMP.

The part of the update that re-inserts a new table row is essentially a re-append. The highest uniqueness value on that AMP is incremented by one, and all the column values for that updated row are appended to their corresponding column partitions. Because multiple I/Os are performed in doing this re-append, row-at-a-time updates on column-partitioned tables should be approached with caution. The space that is being used by the old row is not reclaimed, but a delete bit is turned on in the delete column partition, indicating that the old version of the row is obsolete.

An UPDATE statement should only be used to update a small percentage of rows.

USI Access

For example, consider a unique secondary index (USI) access. The USI subtable provides the specific RowID of the base table row. In the columnar case, the base table row is located on a specific AMP which can be stored in multiple containers. As it is for a PI or NoPI tables, the hash bucket in the RowID carried in the USI is used to locate the AMP that contains the base table row. The column values from the multiple containers are located the same as using a RowID retrieved from a NUSI which is described below.

NUSI Access

With non-unique secondary indexes (NUSIs), a row-id list is retrieved from the NUSI subtable. In the case of a column-partitioned table, the table row has been decomposed into columns that are located in different column partitions on disk. Several different internal partition numbers come into play in reconstructing the table row.

Rather than relying on the column partition number, it is only the hash bucket and uniqueness that is of importance in the NUSI subtable RowID list. The hash bucket and uniqueness identifies the table row on that AMP, while the column partition number plays no meaningful role.

Because column partition numbers in the NUSI subtable are not useful in the case of a column-partitioned table, all RowIDs in a NUSI carry a column partition number of 1. The hash bucket and uniqueness value are the important link between the NUSI subtable and the column values of interest for the query. Once the hash bucket and uniqueness value is known, a RowID is formulated using the internal partition number of the column of interest. This RowID is then used to read the containing physical row/container. A relative position is determined using the uniqueness value which is then used to access the appropriate column value. This process is repeated to locate the column value of each of the remaining columns for this row. These individual column values are brought together to formulate the row. This process occurs for each RowID in the NUSI subtable entry.

UPDATE Considerations

- Updating rows in column partitioned table requires a delete and an insert operation.
- It involves marking the appropriate bit in the delete column partition, and then re-inserting columns for the new updated version of the table row.
- An UPDATE statement should only be used to update a small percentage of rows.

USI/NUSI Considerations

- For a USI on a CP table, the base table row is located on a specific AMP which can be stored in multiple containers. The hash bucket in the RowID carried in the USI is used to locate the AMP that contains the base table row.
- For a NUSI on a CP table, the table row has been decomposed into columns that are located in different column partitions on disk. Several different internal partition numbers come into play in reconstructing the table row.
- Rather than relying on the column partition number, it is only the hash bucket and uniqueness that is of importance in the NUSI subtable RowID list.

CP Table Restrictions

The following limitations apply:

- Column partitioning for join indexes is restricted to single-table, non-aggregate, non-compressed join indexes with no PI and no ORDER BY clause
 - ROWID of base table must be included in a CP join index
- Column partitioning is not allowed for the following:
 - Global temporary, volatile, and queue tables
 - Secondary indexes
- Column partitioning is not applicable for the following:
 - Global temporary trace tables
 - Error tables
 - Compressed join indexes
- NoPI table with only row partitioning is not allowed
- A column cannot be specified to be in more than one column partition
- Column grouping cannot be specified in both the column definition list of CREATE TABLE statement and in the COLUMN clause.
- Column grouping cannot be specified in both the select list of a CREATE JOIN INDEX statement and in the COLUMN clause.

CP Table Restrictions

- **Column Partitioning is predicated on the NoPI table structure and as such the following restrictions apply:**
 - Set Tables
 - Queue tables
 - Global Temporary Tables
 - Volatile Tables
 - Derived Tables
 - Multi-table or Aggregate Join Index
 - Compressed Join Index
 - Hash Index
 - Secondary Indexes are not column partitioned
- **Column Partitioned tables cannot be loaded with either TPT Load (FastLoad) or TPT Update (MultiLoad).**
- **Merge-Into and UPSERT statements are not supported.**
- **Population of Column Partition tables will require an Insert-Select process after data has been loaded into a staging table.**
- **No synchronized scanning with Columnar Tables.**
- **Since Columnar Tables are NoPI Tables, Teradata cannot use Full Cylinder Reads.**

Teradata 14.10 Columnar Enhancements

New cost profile constant for setting the default auto-compression option for column partitions

- AutoCompressDefault=0 (AUTO COMPRESS default and AUTO COMPRESS not included in SHOW) – The Default
- AutoCompressDefault=1 (AUTO COMPRESS default and option included in SHOW output)
- AutoCompressDefault=2 (NO AUTO COMPRESS default and option included in SHOW output)

Cost profiles provide an alternate method to DBS Control parameters for specifying system defaults. If there is a conflict between a cost profile constant setting and its equivalent DBS Control parameter setting, the cost profile constant takes precedence and Teradata does not use the DBS Control parameter setting. Note, however, there is no DBS Control parameter corresponding to the AutoCompressDefault cost profile constant. It is not recommended that this value be changed to 2. This setting is mainly for testing and as a temporary work around if there are problems found with auto-compression. However, the better more localized work around is to specify NO AUTO COMPRESS as a table-level compression options as discussed below:

Teradata 14.10 options allow table-level option for specifying the [NO] AUTO COMPRESS option

- COLUMN clause of a PARTITION BY is enhanced to allow the [NO] AUTO COMPRESS option as the default for column partitions
- Below are examples of statements explicitly specifying the [NO] AUTO COMPRESS option in the PARTITION BY clause of a CREATE TABLE/JOIN INDEX
 1. CREATE TABLE tbl_CP_noAC (a int, b int, c int, d int, e int, f int)
NO PRIMARY INDEX
PARTITION BY COLUMN **NO AUTO COMPRESS**;
 2. CREATE JOIN INDEX ji_t1_AC AS SELECT ROWID AS rw, a, b
FROM t1 WHERE a<10
PARTITION BY COLUMN **AUTO COMPRESS**;

The way the ACT reuse is done is controlled by 5 Internal DBS Control parameters:

CPMaxNumRowsSameAct, CPMinRcrForActReuse, CPMinRcr,
CPMaxPctRowsBelowMinRcr, CPNumSampleBaseRows

By using the parameters CPMinRcrForActReuse, CPMinRcr, and CPNumSampleBaseRows, Teradata ensures that ACT reuse will save significant CPU path and at the same time achieve the same compression levels as the current auto-compression approach of computing the ACT for every container row. Note: The recommendation would be not to change these DBS Control parameters unless you understand the environment very well or have been directed to do so by the support center.

TD 14.10 Columnar Enhancements

- **New cost profile constant for setting the default compression option for column partition.**
 - AutoCompressDefault=1 (AUTO COMPRESS default and option included in SHOW output)
- **Provide table-level option to specify [NO] AUTO COMPRESS for column partitions.**
- **Allow altering the [NO] AUTO COMPRESS option of a column partition for a column-partitioned table or join index.**
- **Allow altering the column partition format to be COLUMN, ROW, or system-determined for a column partition of a column-partitioned table or join index.**
- **Enhanced SHOW TABLE and SHOW JOIN INDEX to include the above options.**
- **Allow NoPI and Column Partitioned (CP) tables to be Temporal.**
- **Internal performance enhancements:**
 - Reduce the CPU path during a load operation into a CP table by saving the auto-compression technique (ACT) data computed for a base container row and reusing it in subsequent container rows.
 - The way the ACT reuse is done is controlled by 5 Internal DBS Control parameters.

Teradata 15.10 Primary AMP

Starting with Teradata 15.10, a column-partitioned table or join index may have a primary index or primary AMP index as an alternative to having no primary index. For a delete, subrows for a column partition with ROW format are physically deleted. For an update of only column partitions with ROW format, an update-in-place of subrows may occur.

A CP table may have a primary index (PI), primary AMP index (PA), or no primary index (NoPI). To understand how column partition values are stored, it is useful to review the differences between a PI table, PA table, and a NoPI table.

For a table row of a PI or PA table, an index value is passed through the system's hashing algorithm to produce a hash value of which the hash bucket portion determines which AMP owns the table row based on the primary hash bucket map. For a NoPI table, there is no index value to hash; when a NoPI table is loaded, blocks of table rows are randomly assigned to AMPs or, for an INSERT-SELECT, locally copied by default.

For a column partition value of a PI table, the internal partition number, hash value from the index value, and uniqueness are used as the rowid.

However for PA or NoPI table, the rowid includes the internal partition number, hash bucket (not a full hash value) and uniqueness; for a PA table this hash bucket is unrelated to the hash bucket portion of the hash of the index value for the row. Instead for a PA or NoPI table, all rows owned by an AMP initially use the lowest hash bucket associated to that AMP by the NoPI hash map in their rowid. The NoPI hash map only includes hash buckets from the regular PI hash map where the AMP owning this hash bucket does not change if the number of AMPs increases; this avoids having to move rows to a different AMP when doing a Reconfig or restore/copy. The purpose of carrying a hash bucket in the rowid is to have a means to identify the AMP on which a row resides when there is access via a secondary index, join index, or rowid spool. The hash bucket is all that is required from a rowid for a PA or NoPI table to indicate the row's owning AMP.

A primary AMP index (PA) allows for single-AMP access when the values of the primary AMP index column(s) are specified or to provide better distribution of the rows. Also, a PA allows various local join and aggregation methods if the join or GROUP BY is on the primary AMP index column(s). Row header compression and auto-compression effectiveness is similar to a CP table with no primary index.

A PA also allows column partition elimination to occur in the same manner as for a CP table with a primary index or no primary index. This can occur with single-AMP access as discussed in the preceding paragraph or with other forms of access such as all-AMP access when not all the values of the primary AMP index are specified. If the table is also row partitioned, row partition elimination may be possible.

When using column partitioning, using a PA (possibly with a NUSI or USI on the same columns as the PA to allow access to specific rows) is often the best choice.

However, there is a cost to row distribution versus the block distribution or local copy that can occur for inserts into a CP table with no primary index. In most cases, this cost is acceptable when considering the benefits for queries. Also, if there is no row partitioning, a sort (which would usually be required for PI table) can be avoided for a load or insert; however, column-partitioned tables are often also row partitioned.

Column Partitioning with **Primary AMP** Index (CPPA)

```
CREATE TABLE Super_Bowl_CPPA
```

```
(Winner          CHAR(25) NOT NULL,  
Loser            CHAR(25) NOT NULL,  
Game_Date        DATE      NOT NULL,  
Game_Score       CHAR(7)  NOT NULL,  
Attendance        INTEGER)
```

```
PRIMARY AMP INDEX ("Winner") PARTITION BY COLUMN; /* INDEX is optional */
```

- **Rows are distributed to AMPs based on hash value of PA column(s):**
 - Column partition values are appended on AMP to a combined partition and hash bucket where the hash bucket is chosen the same as for Column-Partitioned No Primary Index (CP NoPI)
 - Can also be row partitioned.
- Similar to a PI for row redistribution between AMPs, but rows are appended to the table on the AMP (similar to a NoPI table).
- **Single-AMP access when value for PA column(s) is specified.**
- More efficient aggregations with GROUP BY on PA columns.
- **More efficient joins, which is a key advantage.**
 - Allows local dynamic hash or product join when equality join on PA columns and other relation has same PA or PI columns.
 - Allows dynamic hash or product join with redistribution instead of duplication of other relation when equality join on PA columns and other relation does not have same PA or PI columns

Teradata 15.10 Primary Index

A primary index (PI) allows single-AMP access along with direct access to column partition values based on a hash value within each non-eliminated internal partition when the values of the primary index column(s) are specified and may be used to provide a better distribution of the rows to the AMPs. Also, a PI allows various local join and aggregation methods if the join or GROUP BY is on the primary index column(s).

Column partition elimination also occurs for column partitions for which no columns are accessed by a query. This can occur with single-AMP access within non-eliminated internal partitions for a single hash value as discussed in the preceding paragraph or with other forms of access such as all-AMP access when not all the values of the primary index are specified. If the table is also row partitioned, row partition elimination may be possible.

However, row header compression and autocompression may be less effective than for a CP table with a primary AMP index or no primary index. In some cases, additional row headers may cause a significant increase in the size of the table compared to other physical database design choices. This may effectively limit the number of column partitions or require wide column partitions to reduce the row header overhead. Due to these issues, in many cases, a primary AMP index may be a better choice to obtain the full advantages of column partitioning.

Also, a sort (by row partitioning if specified and the hash value) of the table rows to be inserted is required; this usually is not a concern since column-partitioned tables are often row partitioned so a sort would be required in any case.

A unique primary index (UPI) may be specified with column partitioning for a table (but not a join index). If the table also has row partitioning, all the partitioning columns must be included in the unique primary index. Note that the default format for column partitions is ROW format for a CP table with a unique primary index.

Using a primary index with column partitioning, a table can be vertically partitioned into frequently used columns and rarely used columns. This provides the advantages of a PI table (PI access, direct merge joins on the PI, and local aggregation when the GROUP BY is on the PI) and reduces the data read for most queries that only access the frequently used columns. If the frequently used columns represent, for example, 30% of the data, the I/O can be reduced by 70%. There is some growth in the size of the table (the number of row headers doubles) but for wide rows this should be negligible. However, autocompression is not expected to be effective in this case (if needed, alternative forms of compression should be in this case). This is a much simpler approach than splitting the data into two tables.

Finer granularity in the column partitioning may also be advantageous; for instance, columns used frequently in predicates in one or more column partitions, columns frequently used for projection in another column partition, and other columns in a third partition. But as more column partitions are added, the increase in row headers may become excessive, especially if the primary index is unique or nearly unique; if there are many rows per primary index value, row header expansion is less of a factor and row header compression may come into play for column partitions with COLUMN format. For some column partitions that have one or a few columns (such as column partitions used for frequently used predicates), autocompression may be effective for those column partitions.

TD 15.10 Primary Index

Column Partitioning with **Primary Index** (CPPI)

```
CREATE TABLE Super_Bowl_CPPA
(Winner          CHAR(25) NOT NULL,
Loser           CHAR(25) NOT NULL,
Game_Date       DATE      NOT NULL,
Game_Score      CHAR(7)  NOT NULL,
Attendance      INTEGER)
PRIMARY INDEX ("Winner") PARTITION BY COLUMN;
```

- Rows distributed to AMPs based on hash of PI column(s) and sorted by the combined partition number and hash value.
 - Can also be row partitioned.
- **Single-AMP access**, single hash value within non-eliminated combined partitions when value for PI columns specified.
- More efficient aggregations with group by on PI columns.
- More efficient hash joins and equality product joins – same as for CP "Primary AMP"
- Same benefits as CP NoPI **EXCEPT**;
 - Row header compression and auto-compression may be less effective.
 - Containers can only hold column partition values for same combined partition and hash value
 - If values of PI columns are fairly unique, table may expand due to increase in row headers per column partition value plus auto-compression may not be effective if there are only a few values per container.
 - To avoid table size explosion due to row headers, most or all column partitions may need to be wide reducing effectiveness of column partition elimination.

Summary

The facing page contains a summary of key concepts from this module.

When is column partitioning useful?

- Queries access varying subsets of the columns of table
or
Queries of the table are selective (Best if both occur for queries)
- For example, ad hoc, data analytics
- Data can be loaded with large INSERT-SELECTs
- There is no or little update/delete maintenance between refreshes or appends of the data for the table or for row partitions.

Do NOT use this feature when:

- Queries need to be run on current data that is changing (deletes and updates).
- Performing tactical queries or OLTP queries.
- Workload is CPU bound such that a trade-off of reduced I/O with increased CPU does not improve the throughput.
 - Column partitioning is *not* intended to be a CPU savings feature.

Module 15: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 15: Review Questions

1. Which choice is NOT allowed with a column partitioned table in Teradata 16.20?
 - a. SET table
 - b. NoPI table
 - c. MULTISSET table
 - d. Table with a NUPI
2. What are two benefits of Column Partitioning? (choose two)
 - a. Reduced I/O
 - b. Reduced CPU
 - c. Reduced disk space usage
 - d. Reduced tactical query response times
3. True or False? Deleting a row in a column partitioned table will reclaim table space.
4. True or False? In a multilevel columnar table, only one level may be defined with COLUMN.
5. True or False? The preferred way to load a columnar table is using INSERT/SELECT.

Lab Exercise 15-1

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```

```
SHOW TABLE table_name;
```

A count of rows in the Orders table is 350,000.

A count of rows in the Orders_2019 table is 100,000.

Lab Exercise 15-1

Lab Exercise 15-1

Purpose

In this lab, you will create tables with column partitioning in various ways. These tables will be used in later labs.

What you need

Populated DS tables and Orders and Orders_2019 tables in your database

Tasks

1. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) that does NOT have a primary index and name this table "Orders_NoPI".
2. Populate the Orders_NoPI table (via INSERT/SELECT) with all of the rows from the DS.Orders and DS.Orders_2019 tables.

Verify the number of rows in your table. Count = _____ (count should be 450,000)

3. Use the SHOW TABLE for Orders_NoPI to create a new column partitioned table named "Orders_CP" based on the following:
 - Each column is created as a separate partition
 - Utilize auto compression for every column

Populate the Orders_CP table (via INSERT/SELECT) from the Orders_NoPI table.

Lab Exercise 15-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```

```
SELECT COUNT(DISTINCT(PARTITION)) FROM table_name;
```

Lab Exercise 15-1 (cont.)

4. Verify the number of rows in your table. Count = _____ (count should be 450,000)

How many partitions actually have data? _____

Note: The table only has 1 logical partition.

5. Use the SHOW TABLE for Orders_CP to create a new column partitioned table named "Orders_CP_noAC" based on the following:

- Each column is created as a separate partition
- Turn off auto compression for every column

Populate the Orders_CP_noAC table (via INSERT/SELECT) from the Orders_NoPI table.

6. Use the SHOW TABLE for Orders_CP to create a new column partitioned table named "Orders_CPPA" based on the following:

- Primary AMP Index using **custid** column and each column is created as a separate partition
- Utilize auto compression for every column

Populate the Orders_CPPA table (via INSERT/SELECT) with all of the rows from your Orders_NoPI table.

Lab Exercise 15-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
SELECT      TableName, SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName In ('tablename1', 'tablename2', ...)
GROUP BY    1
ORDER BY    1;
```

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```


Lab Exercise 15-1 (cont.)

7. Use the SHOW TABLE for Orders_CPPA to create a new column partitioned table named "Orders_CPPI" based on the following:
 - Primary Index using **custid** column and each column is created as a separate partition
 - Utilize auto compression for every column

Populate the Orders_CPPI table (via INSERT/SELECT) with all of the rows from the DS.Orders and DS.Orders_2019 tables in parallel.

Note: To improve insert performance run both insert/selects in parallel (Execute Parallel - F9 in SQL Assistant or Execute All - Ctrl+Alt+x In Teradata Studio Express)

8. Determine the AMP space usage of the Orders_CP, Orders_CP_noAC, Orders_CPPA and Orders_CPPI tables using the DBC.TableSizeV.

CurrentPerm for Orders_CP _____

CurrentPerm for Orders_CP_noAC _____

CurrentPerm for Orders_CPPA _____

CurrentPerm for Orders_CPPI _____

Which table is the largest and why? _____

What is the probable reason that the CPPA and CPPI tables use less perm space?

Lab Exercise 15-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```

```
SELECT COUNT(DISTINCT(PARTITION)) FROM table_name;
```

Lab Exercise 15-1 (cont.)

9. Use SHOW TABLE for Orders_CP to create a new column partitioned table named "Orders_CP_RP" based on the following:

- Incorporate row partitioning with orderdate as the partitioning column
 - From '2010-01-01' through '2019-12-31', partition by day
 - Do not use the NO RANGE or UNKNOWN options.
- Each column is created as a separate partition (COLUMN partitioning is the second level)
- Utilize auto compression for every column

Populate the Orders_CP_RP table (via INSERT/SELECT) from the Orders_NoPI table.

10. Use the PARTITION key word to determine the number of partitions defined in the Orders_CP_RP.

How many partitions actually have data? _____

Notes



Secondary Index Usage

After completing this module, you will be able to:

- **Describe USI and NUSI implementations.**
- **Describe dual NUSI access.**
- **Explain NUSI and Aggregate processing.**
- **Compare NUSI vs. full table scan (FTS).**

Teradata Proprietary and Confidential

Notes

Table of Contents

Secondary Indexes	16-4
Defining Secondary Indexes	16-6
Secondary Index Subtables	16-8
Primary Indexes (UPIs and NUPIs)	16-8
Unique Secondary Indexes (USIs)	16-8
Non-Unique Secondary Indexes (NUSIs)	16-8
USI Subtable General Row Layout	16-10
USI Change for PPI	16-10
USI Hash Mapping	16-12
NUSI Subtable General Row Layout	16-14
NUSI Change for PPI	16-14
Geospatial Index (TD 14.10)	16-14
NUSI Hash Mapping	16-16
Table Access – A Complete Example	16-18
Secondary Index Considerations	16-20
Single and Dual NUSI Access	16-22
Single NUSI Access	16-22
Dual NUSI Access	16-22
NUSI Bit Mapping	16-24
Example	16-24
Value-Ordered NUSIs	16-26
RANGE Accesses for a NUSI	16-28
Hash-Ordered NUSIs	16-30
Example of Hash-Ordered NUSI	16-32
Covering Indexes	16-34
Join Index Note:	16-34
Example	16-34
Covering Indexes (cont.)	16-36
NUSIs and Aggregate Processing	16-36
Example	16-36
NUSI vs. Full Table Scan (FTS)	16-38
Example	16-38
Full Table Scans – Sync Scans	16-40
Module 16: Review Questions	16-42

Secondary Indexes

Secondary Indexes are generally defined to provide faster set selection. Teradata allows up to 32 Secondary Indexes per table. Teradata handles Unique Secondary Indexes (USIs) and Non-Unique Secondary Indexes (NUSIs) very differently.

The diagram illustrates how Secondary Index values are stored in subtables. Secondary Index values, like Primary Index values, are input to the Hashing Algorithm. As with Primary Indexes, the Hashing Algorithm takes the Secondary Index value and outputs a Row Hash. These Row Hash values point to a subtable which stores index rows containing the base table SI column values and Row IDs which point to the row(s) in the base table with the corresponding SI value.

Teradata can determine the difference between a base table and a SI subtable by checking the Subtable ID, which is part of the Table ID.

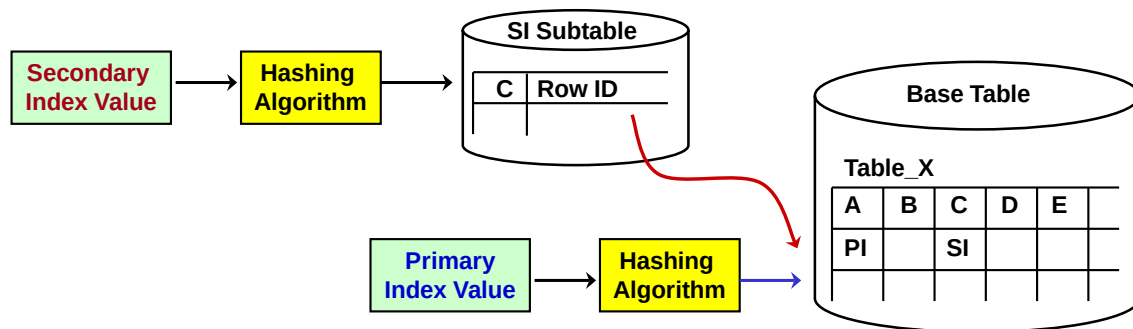
Secondary Indexes

Secondary indexes provide faster set selection.

- They may be unique (USI) or non-unique (NUSI).
- A USI may be used to maintain uniqueness on a column.
- The system maintains a separate subtable for each secondary index.
- A secondary index can consist of 1 to 64 columns.

Subtables keep base table secondary index row hash, column values, and RowID (which point to the row(s) in the base table with that value).

- The implementation of a USI is different than a NUSI.
- Users cannot access subtables directly.



Defining Secondary Indexes

Use the CREATE INDEX statement to create a secondary index on an existing table or join index. The index can be optionally named.

Notes on ORDER BY option:

- If the ORDER BY option is not used, the default is to order by hash.
- If the ORDER BY option is specified and neither of the keywords (HASH or VALUES) is specified, then the default is to order by values.

Recommendation: If the ORDER BY option is used, specify one of the keywords – HASH or VALUES.

Notes on the ALL option:

- The ALL option indicates that a NUSI should retain row ID pointers for each logical row of a **join index** (as opposed to only the compressed physical rows).
- ALL also ignores the NOT CASESPECIFIC attribute of data types so a NUSI can include case-specific values.
- ALL enables a NUSI to cover a join index, enhancing performance by eliminating the need to access a join index when all values needed by a query are in the secondary index. However, ALL might also require the use of additional index storage space.
- Use this keyword when a NUSI is being defined for a join index and you want to make it eligible for the Optimizer to select when covering reduces access plan cost. ALL can also be used for an index on a table, however.
- You cannot specify multiple indexes that differ only by the presence or absence of the ALL option.
- The use of the ALL option for a NUSI on a data table does not cause a syntax error.

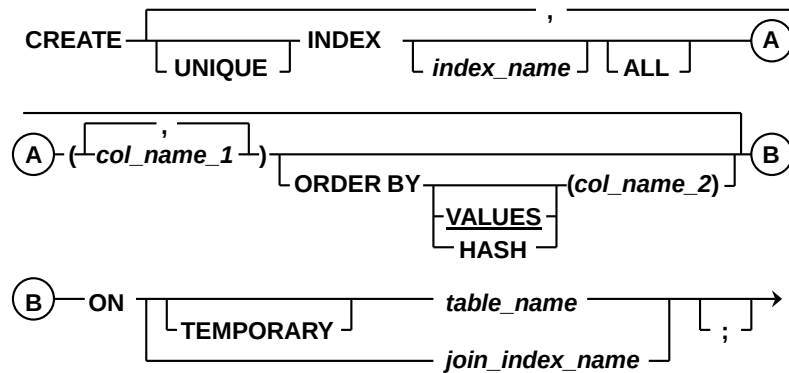
Additional notes:

- *column_name_2* specifies the sort order to be used. *column_name_2* is a column name that must appear in the *column_name_1* list.
- You can put two NUSI secondary indexes on the same column (or set of columns) if one of the indexes is ordered by hash and the other index is ordered by value.
- You cannot define a USI on a join index. Other secondary indexes are allowed.

Defining Secondary Indexes

Secondary indexes can be defined ...

- when a table is created (CREATE TABLE).
- for an existing table (CREATE INDEX).



Examples:

Unnamed USI:

```

CREATE UNIQUE INDEX
  (itemid, storeid, salesdate)
ON DailySales;
  
```

Named Value-Ordered NUSI:

```

CREATE INDEX ds_vonusi
  (salesdate)
ORDER BY VALUES ON DailySales;
  
```

Secondary Index Subtables

The diagram on the facing page illustrates how Teradata retrieves rows based upon their index values. It compares and contrasts examples of Primary (UPIs and NUPIs), Unique Secondary (USIs) and Non-Unique Secondary Indexes (NUSIs).

Primary Indexes (UPIs and NUPIs)

As you have seen previously, in the case of a Primary Index, Teradata hashes the value and uses the Row Hash to find the desired row. This is always a one-AMP operation and is shown in the top diagram on the facing page.

Unique Secondary Indexes (USIs)

The middle diagram illustrates the process of a USI row retrieval. An index subtable contains index rows, which in turn point to base table rows matching the supplied index value. USI rows are globally hash- distributed across all AMPs, and are retrieved using the same procedure for Primary Index data row retrieval. Since the USI row is hash-distributed on different columns than the Primary Index of the base table, the USI row typically lands on an AMP other than the one containing the data row. Once the USI row is located, it “points” to the corresponding data row. This requires a second access and usually involves a different AMP. In effect, a USI retrieval is like two PI retrievals:

Master Index - Cylinder Index - Index Block
Master Index - Cylinder Index - Data Block

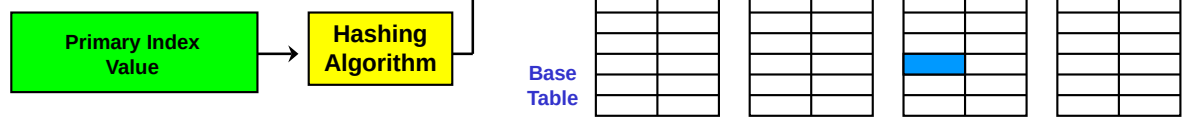
Non-Unique Secondary Indexes (NUSIs)

NUSIs are implemented on an AMP-local basis. Each AMP is responsible for maintaining only those NUSI subtable rows that correspond to base table rows located on that AMP. Since NUSIs allow duplicate index values and are based on different columns than the PI, data rows matching the supplied NUSI value could appear on any AMP.

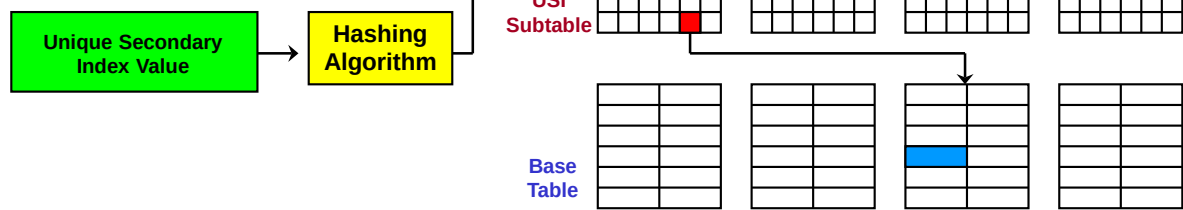
In a NUSI retrieval (illustrated at the bottom of the facing page), a message is sent to all AMPs to see if they have an index row for the supplied value. Those that do use the “pointers” in the index row to access their corresponding base table rows. Any AMP that does not have an index row for the NUSI value will not access the base table to extract rows.

Secondary Index Subtables

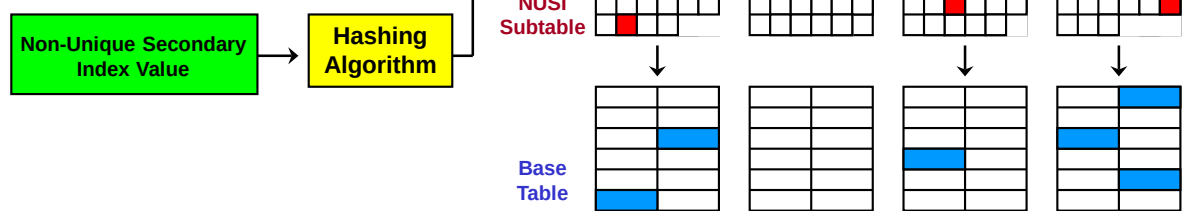
One AMP Operation



Two AMP Operation



All AMP Operation



USI Subtable General Row Layout

The layout of a USI subtable row is illustrated at the top of the facing page. It is composed of several sections:

- The first two bytes designate the **row length**.
- The next 8 bytes contain the **Row ID of the row**. Within this Row ID, there are 4 bytes of Row Hash and 4 bytes of Uniqueness Value.
- The following 2 bytes are **additional system bytes** that will be explained later as will be the 7 bytes for row offsets.
- The next section contains the **SI value**. This is the value that was used by the Hashing Algorithm to generate the Row Hash for this row. This section varies in length depending on the index.
- Following the SI value are 8 bytes containing the **Row ID of the base table row**. The base table Row ID tells the system where the row corresponding to this particular USI value is located.

If the table is partitioned, then the USI subtable row needs 10 or 16 bytes to identify the **Row ID of the base table row**. The Row ID (of the base table row) is combination of the Partition Number, Row Hash, and Uniqueness Value.

- The last two bytes contain the **reference array pointer** at the end of the block.

Teradata creates one index subtable row for each base table row.

USI Change for PPI

For tables defined with a PPI, a two-byte or optionally eight-byte (TD 14.0) partition number is embedded in the data row. Therefore, the unique row identifier is comprised of the Partition Number, the Row Hash, and the Uniqueness Value.

The USI subtable rows use the wider row identifier to identify the base table row, making these subtable rows wider as well. Except for the embedded partition number, USI subtable rows (for a PPI table) have the same format as non-PPI rows.

The facing page shows the row layout for USI subtable rows.

USI Subtable General Row Layout

USI Subtable Row Layout

Row Length	Row ID of USI			Secondary Index Value	Base Table Row Identifier			Ref. Array Pointer
	Row Hash	Uniq. Value			Part. #	Row Hash	Uniq. Value	
	4	4			2 or 8	4	4	

Notes:

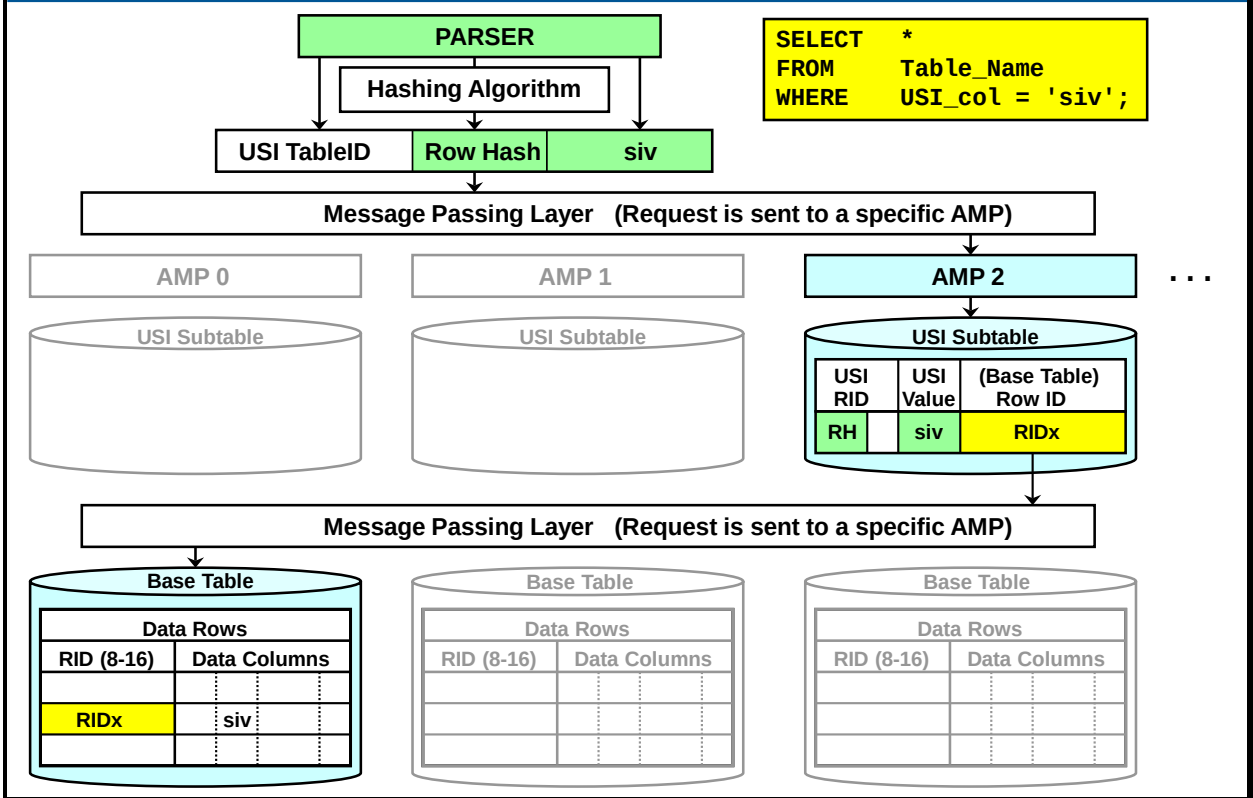
- USI subtable rows are distributed by the Row Hash, like any other row.
- The Row Hash is based on the unique secondary index value.
- The subtable row includes the secondary index value and a second Row ID which identifies a single base table row (usually on a different AMP).
- There is one index subtable row for each base table row.
- For row partitioned tables, the partition number (2 or 8 bytes) is included in the base table row identifier.
 - Therefore, the base table row identifier (Row ID) is comprised of the Partition Number, Row Hash, and the Uniqueness Value.

USI Hash Mapping

The example on the facing page shows the three-part message that is put onto the Message Passing Layer for USI access.

- The only difference between this and the three-part message used in PI access (previously discussed) is that the Subtable ID portion of the Table ID references the USI subtable not the data table. Using the DSW for the Row Hash, the Message Passing Layer (a.k.a., Communication Layer) directs the message to the correct AMP which uses the Table ID and Row Hash as a logical index block identifier and the Row Hash and USI value as the logical index row identifier. If the AMP succeeds in locating the index row, it extracts the base table Row ID (“pointer”). The Subtable ID portion of the Table ID is then modified to refer to the base table and a new three-part message is put onto the Communications Layer.
- Once again, the Message Passing Layer uses the DSW to identify the correct AMP. That AMP uses the Table ID and Row ID to locate the correct data block and then uses the Row ID to locate the correct row.

USI Hash Mapping



NUSI Subtable General Row Layout

The layout of a **NUSI subtable row** is illustrated on the facing page. It is almost identical to the layout of a USI subtable row. There are, however, two major differences:

- First, NUSI subtable rows are not distributed across the system via AMP number in the Hash Map. NUSI subtable rows are built from the base table rows found on that particular AMP and refer only to the base rows of that AMP.
- Second, NUSI rows may point to or reference more than one base table row. There can be many base table Row IDs (8, 10, or 16 bytes) in a NUSI subtable row. Because NUSIs are always AMP-local to the base table rows, it is possible to have the same NUSI value represented on multiple AMPs.

A NUSI subtable is just another table from the perspective of the file system.

NUSI Change for PPI

For tables defined with a PPI, the two-byte partition number is embedded in the data row. Therefore, the unique row identifier is comprised of the Partition Number, Row Hash, and Uniqueness Value. PPI data rows are two bytes wider than they would be if the table was not partitioned.

If the base table is partitioned, then the NUSI subtable row needs 10 or 16 bytes for each RowID entry to identify the **Row ID of the base table row**. The Row ID (of the base table row) is combination of the Partition Number, Row Hash, and Uniqueness Value.

The NUSI subtable rows use the wider row identifier to identify the base table row, making these subtable rows wider as well. Except for the embedded partition number, NUSI subtable rows (for a PPI table) have the same format as non-PPI rows.

The facing page shows the row layout for NUSI subtable rows.

Geospatial Index (TD 14.10)

A NUSI that is defined on a column that has a geospatial data type. Geospatial indexes are implemented using GeoGrids for their population component and Hilbert R-trees for their summary component. Scalar NUSIs use Index Key values that are equal to their base table values, while geospatial NUSIs use index keys that only approximate their base table values. For geospatial indexes, the Secondary Index Value field contains the Minimum Bounding Rectangle (MBR) for the object being indexed.

Hilbert R-tree is an indexing method for multidimensional database objects. For Teradata Database, Hilbert R-trees contain summary statistical data that is used to index geospatial columns as NUSIs. Like NUSI subtables for scalar data, Hilbert R-trees are always AMP-local. Because geospatial NUSIs can be updated, the type of Hilbert R-tree used to implement them is a dynamic R-tree as opposed to a static R-tree.

NUSI Subtable General Row Layout

NUSI Subtable Row Layout

Row Length	Row ID of NUSI			Secondary Index Value	Table Row ID List						Ref. Array Pointer
	Row Hash	Uniq. Value			P	RH	U	P	RH	U	
	4	4			2/8	4	4	2/8	4	4	

Notes:

- The Row Hash is based on the base table secondary index value.
- The NUSI subtable row contains Row IDs that identify the base table rows on this AMP that carry the Secondary Index Value.
- The Row IDs reference (or "point") to base table rows on this AMP only.
- There are one (or more) subtable rows for each secondary index value on the AMP.
 - One NUSI subtable row can hold approximately 4000 – 8000 Row IDs; assuming a NUSI data type less than 200 characters (CHAR(200)).
 - If an AMP has more than 4000 – 8000 rows with the same NUSI value, another NUSI subtable row is created for the same NUSI value.
- The maximum size of a single NUSI row is 64 KB.

NUSI Hash Mapping

When a secondary index value is provided in an equality condition, the specified secondary index value is hashed and then each NUSI subtable is probed for rows with the same row hash. For each matching NUSI entry, the corresponding Row IDs are used to access the base rows on the same AMP. Because the NUSI rows are stored in row hash order, searching the NUSI subtable for a particular row hash is very efficient.

The example on the facing page shows the standard, three-part Message Passing Layer row-access message. Because NUSIs are AMP-local indexes, this message gets broadcast to all AMPs. Each AMP uses the values to search the appropriate index block for a corresponding NUSI row. Only those AMPs with one or more of the desired rows use the base table Row IDs to access the proper data blocks and data rows.

In the example, the SELECT statement is designed to find those rows with a NUSI value of 'siv'. Examination of the NUSI subtables on each AMP shows that AMPs 0, 2 and 3 (not shown) all have a subtable index row, and, therefore, base table rows satisfying this condition. These AMPs then participate in the base table access. The NUSI subtable on AMP 1, on the other hand, shows that there are no rows with a NUSI value of 'siv' located on this AMP. AMP 1 does not participate in the base table access process.

If the table is not partitioned, the subtable rows will identify the 8-byte Row IDs of the base table rows.

If the table is partitioned with less than (or equal) 65,535 partitions, the subtable rows will identify the 10-byte Row IDs of the base table rows. This Row ID includes the Partition Number.

If the table is partitioned with more than 65,535 partitions, the subtable rows will identify the 16-byte Row IDs of the base table rows. This Row ID includes the Partition Number.

NUSI Hash Mapping

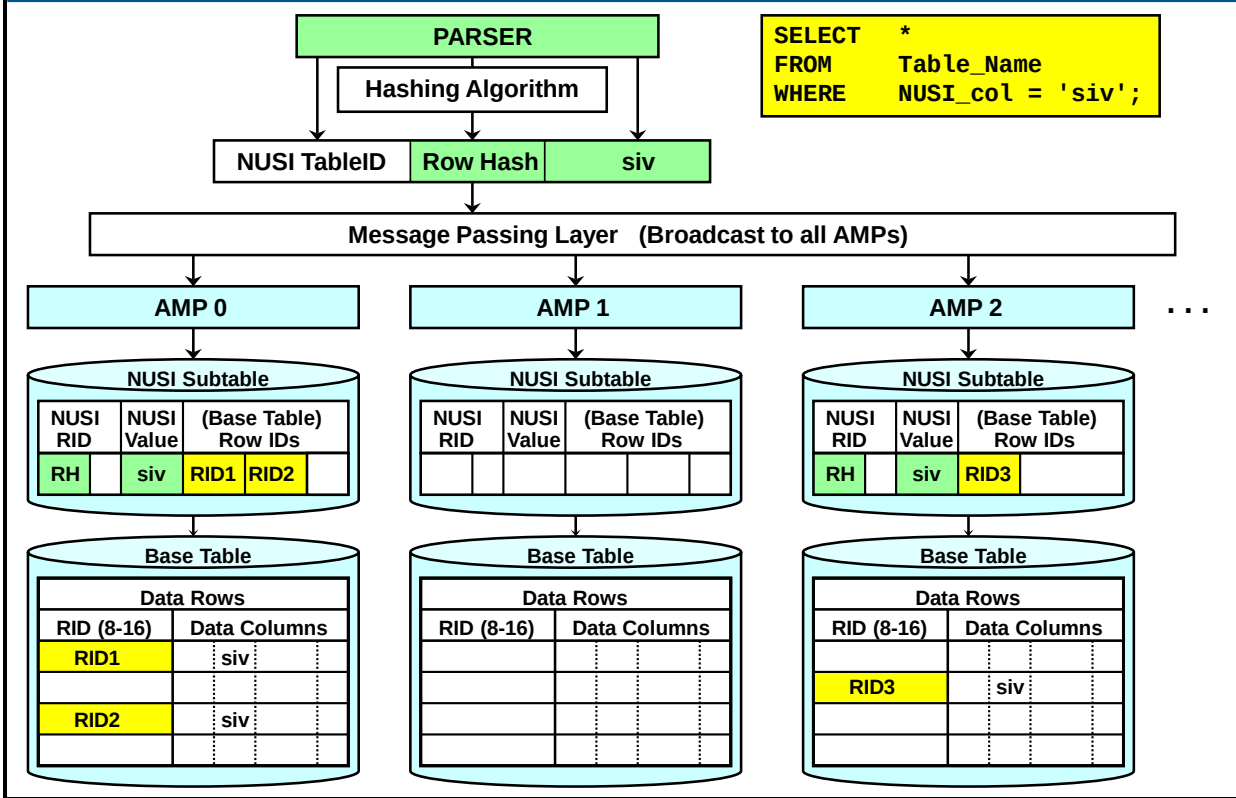


Table Access – A Complete Example

The example on the facing page shows a four-AMP configuration with Base Table Rows, NUSI Subtable rows, and USI Subtable Rows. The table and index can be used to answer the following queries without having to do a full table scan:

```
SELECT * FROM Customer WHERE Phone = '666-5555';  
SELECT * FROM Customer WHERE Cust = 80;  
SELECT * FROM Customer WHERE Name = 'Rice';
```

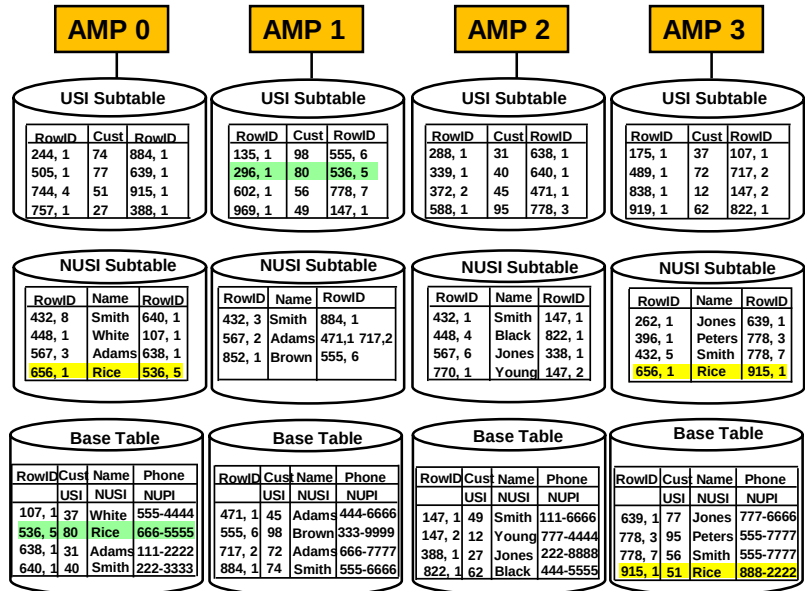
Table Access – A Complete Example

Customer

Cust	Name	Phone
USI	NUSI	NUPI
37	White	555-4444
98	Brown	333-9999
74	Smith	555-6666
95	Peters	555-7777
27	Jones	222-8888
56	Smith	555-7777
45	Adams	444-6666
31	Adams	111-2222
40	Smith	222-3333
72	Adams	666-7777
80	Rice	666-5555
49	Smith	111-6666
12	Young	777-7777
62	Black	444-5555
77	Jones	777-6666
51	Rice	888-2222

Example:

```
SELECT * FROM Customer WHERE Phone = '666-5555';
SELECT * FROM Customer WHERE Cust = 80;
SELECT * FROM Customer WHERE Name = 'Rice';
```



Secondary Index Considerations

As mentioned at the beginning of this module, a table may have up to 32 Secondary Indexes that can be created and dropped dynamically. It is probably not a good idea to create 32 SIs for each table just to speed up set selection because SIs consume the following extra resources:

- SIs require additional storage to hold their subtables. In the case of a Fallback table, the SI subtables are Fallback also. Twice the additional storage space is required.
- SIs require additional I/O to maintain these subtables.

When deciding whether or not to define a NUSI, there other considerations. The Optimizer may choose to do a Full Table Scan rather than utilize the NUSI in two cases:

- When the NUSI is not selective enough.
- When no COLLECTed STATISTICS are available.

As a guideline, choose only those rows having frequent access as NUSI candidates. After the table has been loaded, create the NUSI indexes, COLLECT STATISTICS on the indexes, and then do an EXPLAIN referencing each NUSI. If the Parser chooses a Full Table Scan over using the NUSI, drop the index.

– REMEMBER –
The only way to determine for certain whether an index is being used is to utilize the EXPLAIN facility.

Secondary Index Considerations

- A table may have up to 32 secondary indexes.
- Secondary indexes may be created and dropped dynamically.
 - They require additional storage space for their subtables.
 - They require additional I/Os to maintain their subtables.
- If the base table is Fallback, the secondary index subtable is Fallback as well.
- The Optimizer may, or may not, use a NUSI, depending on its selectivity.
 - Without COLLECTed STATISTICS, the Optimizer often chooses to do a FTS.
- Hash-ordered NUSIs can be used for value access, possibly join access, and as a covering index.
- Value-ordered NUSIs can be used for value access, range access, and as a covering index.
- The following approach is recommended:
 - Create the index.
 - COLLECT STATISTICS on the index (or column).
 - Use EXPLAIN to see if the index is being used.

Single and Dual NUSI Access

In the example on the facing page, two NUSIs are CREATED on separate columns of the Employee Table. Teradata decides how to use these NUSIs based on their selectivity.

It is always recommended to COLLECT STATISTICS on any NUSIs.

Single NUSI Access

Teradata accesses data from a NUSI-defined column in multiple ways as listed on the facing page.

- If there is an equality value for a NUSI and the NUSI is strongly selective, Teradata will utilize the NUSI unless the base table is small (less than a few blocks per AMP).
- Perform a FTS of the NUSI subtable if it covers the query or ignore the NUSI and do an FTS of the base table itself.

Dual NUSI Access

AND with Equality Conditions

When accessing data with two NUSI equality conditions joined by the AND operator (as shown in the middle example on the facing page), Teradata may do one of the following:

- If one of the two indexes is strongly selective, the system uses it alone for access.
- If both indexes are weakly selective separately and together, the system does an FTS.
- If both indexes are weakly selective, but together they are strongly selective, the system may do a bit-map intersection.
- In any case, any conditions in the SQL statement not used for access (residual conditions) become row qualifiers.

OR with Equality Conditions

When accessing data with two NUSI equality conditions joined by the OR operator (as shown in the last example on the facing page), Teradata may do one of the following:

- If each of the NUSIs is strongly selective, it may use each of the NUSIs to return the appropriate rows.
- Do an FTS of the two NUSI subtables and do the following steps.
 - Retrieve Rows IDs of qualifying base table rows into two separate spools.
 - Eliminate duplicates from the two spools of Row IDs.
 - Access the base rows from the resulting spool of Row IDs.
- Do a FTS of the base table.

If only one of the two columns joined by the OR is indexed, Teradata always does an FTS of the base table.

Single and Dual NUSI Access

A separate NUSI is created on each column:

```
CREATE INDEX (deptnumber) ON Employee;  
CREATE INDEX (jobcode) ON Employee;
```

Single NUSI Access:

```
SELECT lastname, firstname, ...  
FROM Employee  
WHERE deptnumber = 500;
```

Optimizer options for single NUSI access:

- Use the index if it is strongly selective.
- Perform a FTS of index subtable if it covers the query or a FTS of the base table.

AND with Equality Conditions:

```
SELECT lastname, firstname, ...  
FROM Employee  
WHERE deptnumber = 500  
AND jobcode = 2147;
```

Optimizer options with AND:

- Use one of the two indexes if it is strongly selective.
- If the two indexes together are strongly selective, optionally do a bit-map intersection (not common).
- Perform a FTS of the base table if both indexes are weakly selective.

OR with Equality Conditions:

```
SELECT lastname, firstname, ...  
FROM Employee  
WHERE deptnumber = 500  
OR jobcode = 2147;
```

Optimizer options with OR:

- If each of the NUSIs is strongly selective, it may use each of the NUSIs to return the appropriate rows.
- Do a FTS of the two NUSI subtables and retrieve Rows IDs of qualifying rows into spool and eliminate duplicate Row IDs from spool.
- Do a FTS of the base table.

NUSI Bit Mapping

NUSI Bit Mapping is a process that determines common Row IDs between multiple NUSI values by a process of intersection:

- It is much faster than copying, sorting and comparing the Row ID lists.
- It dramatically reduces the number of base table I/Os.

NUSI bit mapping can be used with conditions other than equality if all of the following conditions are satisfied:

- All conditions must be linked by the AND operator.
- At least two NUSI equality conditions must be specified.
- The Optimizer is more likely to consider if you have COLLECTed STATISTICS on the NUSIs.

Even when the above conditions are satisfied, the only way to be absolutely certain that NUSI bit mapping is occurring is to use the EXPLAIN facility.

Example

The SQL statement and diagram on the facing page show how NUSI bit-map intersections can narrow down the number of rows even though each condition is weakly selective.

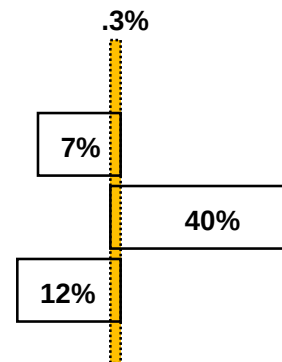
In this example, the designer wants to access rows from the employee table. There are three NUSIs defined: salary_amount, country_code, and job_code. All three of these NUSIs are weakly selective. You can see that 7% of the employees earn more than \$75,000 per year (>75000), 40% of the employees are located in the USA, and 12% of the employees have a job code of IT.

In this case, the bit map intersection of these three NUSIs has an aggregate selectivity of .3%. That is, only .3% of the employees satisfy all three conditions: earning over \$75,000, USA based, and work in IT.

NUSI Bit Mapping

- NUSI Bit Mapping is an optimizer technique to determine common Row IDs between multiple NUSI values.
 - This is not commonly used by optimizer in later releases (e.g., 14.0 and later).
- All NUSI conditions must be linked by the AND operator.
- Requires at least 2 NUSI equality conditions.
- The Optimizer is more likely to consider bit mapping if you COLLECT STATISTICS.
- Use EXPLAIN to see if bit mapping is being used.

```
SELECT *  
FROM Employee  
WHERE salaryamount > 75000  
  
AND countrycode = 'USA'  
  
AND jobcode = 'IT';
```



Value-Ordered NUSIs

NUSIs are maintained as separate subtables on each AMP. Their index entries point to base table or Join Index rows residing on the same AMP as the index. The row hash for NUSI rows is based on the secondary index column(s). Unlike row hash values for base table rows, this row hash does not determine the distribution of subtable rows; only the local sort order of each subtable.

Enhancements have been made to support the user-specified option of sorting the index rows by data value rather than by hash code. This is referred to as "value ordered" indexes and is presented to the user in the form of new syntax options in the CREATE INDEX statement.

By using the “value-ordered” indexes feature, this option can be specified to sort the index rows by data value rather than by hash code.

The typical use of a **hash-ordered** NUSI is with an *equality* condition on the secondary index column(s). The specified secondary index value is hashed and then each NUSI subtable is probed for rows with the same row hash. For each matching NUSI entry, the corresponding Row IDs are used to access the base rows on the same AMP. Because the NUSI rows are stored in row hash order, searching the NUSI subtable for a particular row hash is very efficient.

Value-ordered NUSIs, on the other hand, are useful for processing range conditions and conditions with an inequality on the secondary index column set.

Although hash-ordered NUSIs can be selected by the Optimizer to access rows for range conditions, a far more common response is to specify a full table scan of the NUSI subtable to find the matching secondary key values. Therefore, depending on the size of the NUSI subtable, this might not be very efficient.

By sorting the NUSI rows by data value, it is possible to search only a portion of the index subtable for a given range of key values. The major advantage of a value-ordered NUSI is in the performance of range queries.

Value-ordered NUSIs have the following limitations.

- The sort key is limited to a single numeric column.
- The sort key column must be four or fewer bytes.

The following query is an example of the sort of SELECT statement for which value-ordered NUSIs were designed.

```
SELECT      *
FROM        Orders
WHERE       shipdate
BETWEEN    DATE '2019-07-01' AND DATE '2019-09-30';
```

Value-Ordered NUSIs

A Value-Ordered NUSI is limited to a single numeric (4-byte) or DATE column.

Some benefits of using value-ordered NUSIs:

- Index subtable rows are sorted (sequenced) by data value rather than hash value.
- Optimizer can search only a portion of the index subtable for a given range of values.
- Can provide major advantages in performance of **range queries**.
- Even with a row partitioned table, the Value-Ordered NUSI is still a valuable index selection for other columns in a table.
 - An Orders table may be partitioned on Orderdate and have a VONUSI on Shipdate.

Example of creating a Value-Ordered NUSI:

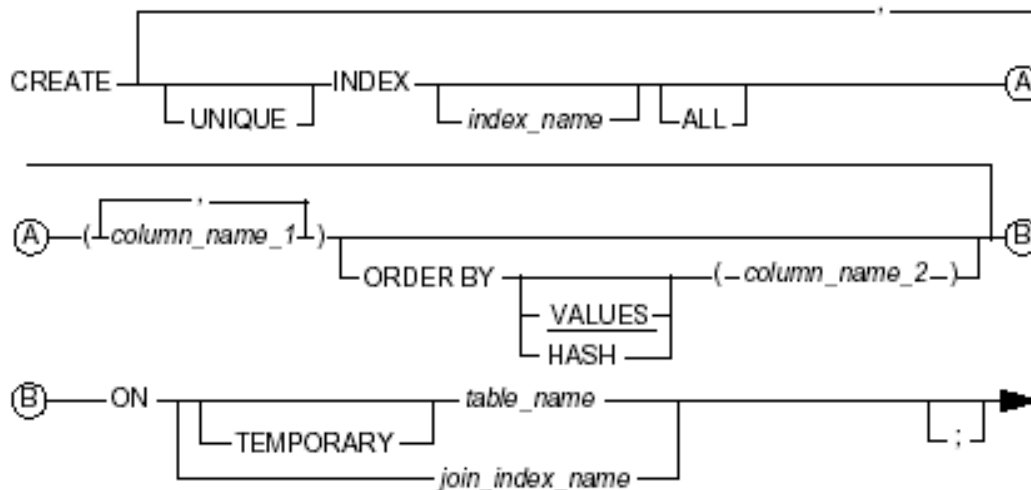
```
CREATE INDEX (shipdate) ORDER BY VALUES (shipdate)
ON Orders;
```

```
SELECT      shipdate
            ,SUM(ordertotal)
FROM        Orders
WHERE       shipdate
BETWEEN     DATE '2019-01-01' AND DATE '2019-03-31'
GROUP BY    1
ORDER BY    1;
```

→ The optimizer may choose to transverse the NUSI using a range constraint rather than do a FTS.

RANGE Accesses for a NUSI

Note: A Value-Ordered NUSI actually reserves two subtable IDs and this counts as 2 secondary indexes in the maximum count of 32 for a table.



column_1_name	The names of one or more columns whose field values are to be indexed. You can specify up to 64 columns for the new index. The index is based on the combined values of each column. Unless you use the ORDER BY clause, all columns are hash-ordered. Multiple indexes can be defined on the same columns as long as each index differs in its ordering option (VALUES versus HASH).
ORDER BY	Row ordering on each AMP by a single NUSI column: either value-ordered or hash-ordered. Rules for using an ORDER BY clause are shown in the following table.
VALUES	Value-ordering for the ORDER BY column. Select VALUES to optimize queries that return a contiguous range of values, especially for a covered index or a nested join.
HASH	Hash-ordering for the ORDER BY column. Select HASH to limit hash-ordering to one column, rather than all columns (the default). Hash-ordering a multi-column NUSI on one of its columns allows the NUSI to participate in a nested join where join conditions involve only that ordering column.

Range Accesses for a NUSI

If a query has range conditions and the NUSI is NOT value-ordered, the system may do a FTS of the NUSI subtable depending on the selectivity of the query.

- Retrieve Row IDs of qualifying base table rows into spool.
- Access the base table rows from the spooled Row IDs.
- The Optimizer needs collected statistics in order to make this choice.

If a query has range conditions and the NUSI is ordered by values, then the NUSI subtable is more likely be used to locate matching base table row.

- `CREATE INDEX (hiredate) ORDER BY VALUES (hiredate) ON Employee;`
- `SELECT lastname, firstname, hiredate`
`FROM Employee`
`WHERE hiredate BETWEEN DATE '2019-01-01' AND DATE '2019-12-31';`
- `SELECT lastname, firstname, hiredate`
`FROM Employee`
`WHERE EXTRACT(YEAR FROM hiredate) = EXTRACT(YEAR FROM Current_Date)`
`AND EXTRACT(MONTH FROM hiredate) = EXTRACT(MONTH FROM Current_Date);`

Use EXPLAIN to see if, and how, indexes are being used.

Hash-Ordered NUSIs

When a multi-column NUSI is created (without using the Order by Values or Hash option), the hash of the NUSI is based on the set of NUSI values. This is effectively a composite hash. Therefore, in order the hash to be used, an *equality* condition is needed on all of the secondary index column(s).

The Hash-Ordered NUSI provides the ability to create a multi-column index, but have the NUSI hashed based on a single attribute within the index, not the entire composite value.

When a multi-column NUSI is ordered by hash on a single column, each AMP creates a subtable row with a local row hash based on the hash of the single column. When the table is accessed via this column, the value can be hashed and this hash is used to access its portion of the index subtable to see if a corresponding row(s) exists.

Hash-Ordered NUSIs

- If a NUSI consists of multiple columns (and is not value or hash ordered), then the hash order of the NUSI subtable is based on the composite hash of the set of columns.
- The '**ORDER BY HASH**' clause provides the ability to create a multi-column index, but have the NUSI hashed based on a single attribute within the index, not the entire composite value.
 - A Hash-Ordered NUSI is useful for equality searches based on a single attribute and can be used to also cover queries.
 - Example: A NUSI may contain 10 columns for covering purposes and a single value '**ORDER BY HASH**' for equality searches on that NUSI value.
 - This is effectively an example of vertical partitioning with a NUSI.
- Optimizer is much more likely to use a hash-ordered NUSI if you have collected statistics on the hash-ordered columns.

Example of Hash-Ordered NUSI

The facing page illustrates the use of a Hash-Ordered NUSI to reduce I/O for a large percentage of the queries using a subset of the 26 columns from the table.

Example of Hash-Ordered NUSI

Assume a table has 26 columns. However, 95% of the queries use a subset of 7 columns (F, A, C, T, O, R, Y). A NUSI subtable is all that needs to be accessed for these queries.

```
CREATE INDEX HO_Idx (F,A,C,T,O,R,Y) ORDER BY HASH (F) ON TableX;
```

This NUSI will cover the query and provide hash access for a specific value of column F.

TableX

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

H-O NUSI

F	A	C	T	O	R	Y

The Optimizer can build a plan to just access the NUSI subtable for queries that use a subset of the columns (F, A, C, T, O, R, Y). This can result in significant I/O savings.

Example of query that is covered by doing a FTS of the index subtable.

```
SELECT F, T, SUM(R) FROM TableX ORDER BY 1 GROUP BY 1, 2;
```

Example of query that is covered by this index and only rows with the hash and value of F are accessed.

```
SELECT F, T, SUM(R) FROM TableX WHERE F=748102 ORDER BY 1 GROUP BY 1, 2;
```

Covering Indexes

If the query references only columns of that table that are fully contained within a given index, the index is said to "cover" the table in the query. In these cases, it is often more efficient to access only the index subtable and avoid accessing the base table rows altogether.

Covering will be considered for any table in the query that references only columns defined in a given NUSI. These columns can be specified anywhere in the query including the:

- SELECT list
- WHERE clause
- Aggregate functions
- GROUP BY expressions

The presence of a WHERE condition on each indexed column is not a prerequisite for using the index to cover the query. The optimizer will consider the legality and cost of covering versus other alternative access paths and choose the optimal plan. Many of the potential performance gains from index covering require no user intervention and will be transparent except for the execution plan returned by EXPLAIN.

Join Index Note:

This course hasn't covered Join Indexes to this point, but it is possible to create a NUSI on top of a Join Index. The CREATE INDEX has a special option of ALL which is required if these columns will be potentially used for covering.

The class of indexed data that will require user intervention to take advantage of covering is NUSIs, which may be defined on a Join Index. By default, a NUSI defined on a Join Index will maintain RowID pointers to only physical rows. In order to use the NUSI to cover the data stored in a Join Index, Row IDs must be kept for each associated logical row. As a result, when defining a potential covering NUSI on top of a Join Index, users should specify the ALL option to indicate the NUSI rows should point to logical rows.

Example

Defining a NUSI on top of a Join Index

```
CREATE JOIN INDEX OrdCustIdx as
  SELECT      (custkey, custname),
              (orderstatus, orderdate, ordercomment)
  FROM        Orders O
  LEFT JOIN   Customer C ON O.custkey = C.custkey
  ORDER BY   custkey
  PRIMARY INDEX (custname);

CREATE INDEX idx_name_stat ALL (custname, orderstatus) ON
OrdCustIdx;
```

Covering Indexes

- The optimizer considers using a NUSI subtable to “cover” any query that references only columns defined in a given NUSI.
- These columns can be specified anywhere in the query including:
 - SELECT list
 - WHERE clause
 - Aggregate functions and GROUP BY clauses
 - Expressions
- Presence of a WHERE condition on each indexed column is **not** a prerequisite for using the index subtable to cover the query.
- NUSIs (especially a covering NUSI) are **considered by the optimizer in join plans** and can be joined to other tables in the system.

```
CREATE INDEX IdxOrder2
(orderid, custid, orderdate, shipdate, ordertotal)
ORDER BY VALUES (orderdate)
ON Orders;
```

This NUSI is considered for covering the first query and for covering and ordering on the second query.

```
SELECT  custid, SUM(ordertotal)
FROM    Orders
GROUP BY 1;
```

```
SELECT  orderdate, AVG(ordertotal)
FROM    Orders
WHERE   orderdate = DATE '2019-03-31'
GROUP BY 1;
```

Covering Indexes (cont.)

NUSIs and Aggregate Processing

When aggregation is performed on a NUSI column, the Optimizer accesses the NUSI subtable that offers much better performance than accessing the base table rows. Better performance is achieved because there should be fewer index blocks and rows in the subtable than data blocks and rows in the base table, thus requiring less I/O.

Example

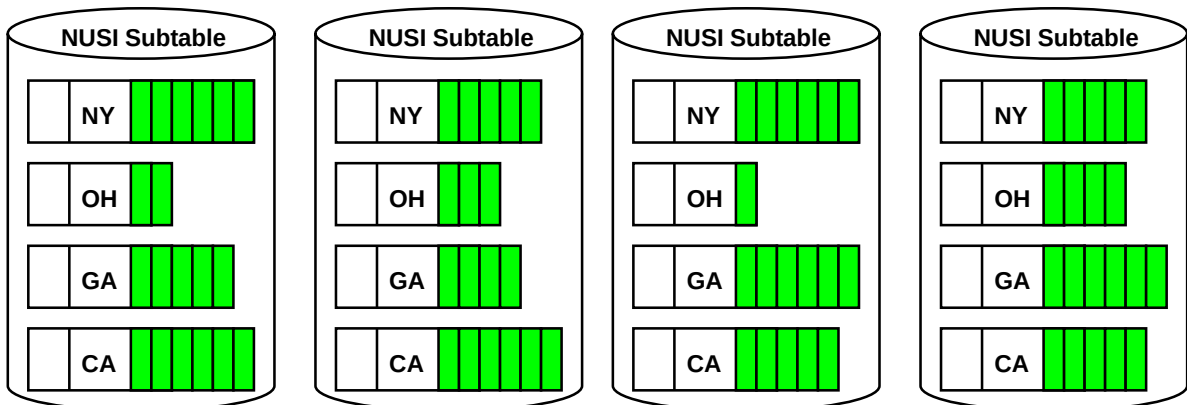
In the example on the facing page, there is a NUSI defined on the state column of the location table. Aggregate processing of this NUSI column produces much faster results for the SELECT statement, which counts the number of rows for each state.

Covering Indexes (cont.)

- The Optimizer uses NUSI subtables for aggregation when possible.
- If the aggregated column is a NUSI, subtable access may be sufficient.
- The system counts Row ID List entries for each AMP for each value.
- Also referred to as a “**Covered NUSI**”.

```
SELECT  COUNT (*), state
FROM    Location
GROUP BY state;
```

 = subtable Row ID



NUSI vs. Full Table Scan (FTS)

The Optimizer generally chooses an FTS over a NUSI when one of the following occurs:

- Rows per value is greater than data blocks per AMP.
- It does not have COLLECTed STATISTICS on the NUSI.
- The index is too weakly selective. The Optimizer determines this by using COLLECTed STATISTICS.

Example

The table on the facing page shows how the access method chosen affects the number of physical I/Os per AMP.

In the case of a NUSI, there is ONE I/O necessary to read the Index Block on each AMP plus 0-ALL (where ALL = Number of Data Blocks) I/Os required to read the Data Blocks for a possible total ranging from the Number of AMPs - (Number of AMPs + ALL) I/Os.

In general, the NUSI is not used if the estimated number of rows to be read in the base table is equal to or greater than the estimated number of data blocks in the base table; in this case, a full table scan is done, or, if appropriate, partition scans are done.

In the case of a Full Table Scan, there are no I/Os required to read any Index Blocks, but the system reads ALL Data Blocks.

The only way to tell whether or not a NUSI is being used is by using EXPLAIN.

**COLLECT STATISTICS on all NUSIs.
Use EXPLAIN to see whether a NUSI is being used.
Do not define NUSIs that will not be used.**

NUSI vs. Full Table Scan (FTS)

The Optimizer generally chooses a FTS (or partition scan if possible) over a NUSI when:

- The index is too weakly selective – effectively means that many rows have that value or fit within the range of values specified.
- Small tables – usually less than a few blocks per AMP.
- If statistics have not been collected on the NUSI, a NUSI may or may not be used. A Random AMP sample is used to determine if the NUSI should be used or not. and only very strongly selective NUSI values

Access Method	Physical I/Os per AMP	
NUSI	1 0 – Many	Index Subtable Block(s) Data Blocks
Partition or Full Table Scan	0 N or ALL	Index Subtable Blocks Data Blocks

General Rules:

- **COLLECT STATISTICS** on all NUSIs.
- USE EXPLAIN to see whether a NUSI is being used.
- Do not define NUSIs that will not be used.

Full Table Scans – Sync Scans

In the case of multiple users that access the same table at the same time, the system can do a synchronized scan (sync scan) on the table.

Full Table Scans – Sync Scans

In the case of multiple users that access the same table at the same time, the system can do a synchronized scan (sync scan) on the table.

- Multiple simultaneous scans share reads – this is a sync scan at the block level.
- New query joins scan at the current scan point.

1127	034	310229	2231	10076	10636	108222	0100	Frankel	Bench	Palmer	Carson
209	123	2231	000	10822	10143	10520		Smith	Walton	Brooks	
221	121	110	00	10822	10128	10110		Woods	Walton	Ranton	
104	210	210	05	10027	10122	10540		Roberts	Douglas	Morgan	
100	100	010	00	10432	10123	12107		Anderson	Michelson	Crawford	
Table Rows	:	:	:	:	:	:	:	:	:	:	:
	:	:	:	:	:	:	:	:	:	:	:

Module 16: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 16: Review Questions

1. Because the row is hash-distributed on different columns, the subtable row will typically land on an AMP other than the one containing the data row. This index would be:
 - a. UPI or NUPI
 - b. USI
 - c. NUSI
 - d. None of the above
2. The Teradata DBS hashes the value and uses the Row Hash to find the desired rows. This is always a one-AMP operation. This index would be:
 - a. UPI or NUPI
 - b. USI
 - c. NUSI
 - d. None of the above
3. _____ is a process that determines common Row IDs between multiple NUSI values by a process of intersection.
 - a. NUSI Bit Mapping
 - b. Dual NUSI Access
 - c. Full Table Scan
 - d. NUSI Read
4. If aggregation is performed on a NUSI column, the Optimizer accesses the NUSI subtable and returns the result without accessing the base table. This is referred to as:
 - a. NUSI bit mapping
 - b. Full table scan
 - c. Dual NUSI access
 - d. Covering Index

Notes



Analyze Secondary Index Criteria

After completing this module, you will be able to:

- **Describe Composite Secondary Indexes.**
- **Choose columns as candidate Secondary Indexes.**
- **Analyze Change Rating, Value Access, and Range Access.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Accessing Rows	17-4
Row Selection	17-6
Secondary Index Considerations.....	17-8
Secondary Index Usage.....	17-10
Secondary Index Candidate Guidelines	17-12
Value and Range Access.....	17-14
Change Rating.....	17-16
Exercise 3 – Example.....	17-18
Secondary Index Candidate Guidelines	17-18
Exercise 3 – Choosing SI Candidates	17-20
Exercise 3 – Choosing SI Candidates (cont.).....	17-22
Exercise 3 – Choosing SI Candidates (cont.).....	17-24
Exercise 3 – Choosing SI Candidates (cont.).....	17-26
Exercise 3 – Choosing SI Candidates (cont.).....	17-28
Exercise 3 – Choosing SI Candidates (cont.).....	17-30
Optimizer Access Scenarios.....	17-32
Secondary Index Overhead	17-34
Additional I/O	17-34
Transient Journal I/O.....	17-36
INSERT and DELETE Operations	17-38
UPDATE Operations	17-40
Primary Index Value UPDATE	17-42
Summary – Access Method Comparison.....	17-44
Unique Primary Index (UPI).....	17-44
Non-Unique Primary Index (NUPI).....	17-44
Unique Secondary Index (USI).....	17-44
Non-Unique Secondary Index (NUSI).....	17-44
Full-Table Scan (FTS).....	17-44
Module 17: Review Questions.....	17-46

Accessing Rows

Three SQL commands require that rows be physically read. They are **SELECT**, **UPDATE**, and **DELETE**. Their syntax and use are described below:

```
SELECT [expression] FROM tablename ...  
UPDATE tablename SET col_name = [expression] ...  
DELETE FROM tablename ...
```

- The **SELECT** command returns the value(s) from the table(s) for display or processing. Many people confuse the SQL **SELECT** statement with a **READ** command (e.g., **COBOL**). **SELECT** simply asks for the column values expressed in the project list to be returned for display or processing. The rows which have their values returned, deleted or updated are identified by the **WHERE** clause (when present). It is the **WHERE** clause that controls File System reads.
- The **UPDATE** command changes one or more column values to new values.
- The **DELETE** command removes rows from a table.

Any of the three SQL statements can be modified with a **WHERE** clause. Values specified in a **WHERE** clause tell Teradata which rows should be acted upon. Proper use of the **WHERE** clause will improve throughput by limiting the number of rows that must be handled.

Accessing Rows

SELECT {expression} FROM tablename ...

- Returns the value(s) from the table(s) for display or processing.
- The row(s) must be physically read first.

UPDATE tablename SET columns = {expression} ...

- Changes one or more column values to new values.
- The rows(s) must be physically located (read) first.

DELETE FROM tablename ...

- Removes rows from a table.
- The row(s) must be physically located (read) first.

Any of the above SQL statements can contain a WHERE clause.

- Values in the WHERE clause tell Teradata what set of rows to act on.
- Without a WHERE clause, all rows participate in the operation.
- Limiting the number of rows Teradata must handle improves throughput.

Row Selection

When TERADATA processes an SQL statement with a WHERE clause, it examines the clause and builds an execution plan and access method to satisfy the clause conditions.

Certain conditions contained in the WHERE clause take advantage of indexing (assuming that the appropriate index is in place). These conditions are shown in the upper box on the facing page. Notice that these conditions all ask the RDBMS to locate a specific value or set of values. Application programmers should use these conditions whenever possible as they offer the best performance.

Other WHERE clause conditions are not able to take advantage of indexing and will always cause a Full Table Scan of either the Base Table or a SI subtable. Though they benefit from the Teradata distributed architecture, they are less desirable from a performance standpoint. These kind of conditions are listed in the middle box on the opposite page and do not focus on a specific value or set of values, thus forcing the system to do a Full Table Scan to find all the values to satisfy them.

Note that poor relational models severely limit physical design choices and generally force more Full Table Scans.

Maximum number of ORed conditions or IN list values per request can't exceed 1,048,576. There really no fixed limit on the number of entries in an IN list; however, the maximum SQL text size is 1MB and this places a request-specific upper bound on this number.

– NOTE –

The small box at the bottom of the facing page lists commands that operate on the answer sets generated by previous conditions, such as those shown in the boxes above.

Row Selection

WHERE clause conditions that may use indexing if available*:

colname = value colname IS NULL colname IN (subquery)	colname IN (explicit list of values) t1.col_x = t1.col_y t1.col_x = t2.col_x	condition1 AND condition2 condition1 OR condition2 colname = ANY, SOME or ALL
---	--	---

* Access methods for the above depend on whether the column(s) are indexed, type of index, and selectivity of the index.

WHERE clause conditions that typically cause a Partition or Full Table Scan:

non-equality comparisons colname IS NOT NULL colname NOT IN (explicit list of values) colname NOT IN (subquery) colname BETWEEN ... AND ... Join condition1 OR Join condition2 t1.col_x [computation] = value t1.col_x [computation] = t1.col_y	INDEX (colname) SUBSTRING (colname) SUM MIN MAX AVG COUNT	DISTINCT ANY ALL NOT (condition1) col1 col2 = value colname LIKE ... missing a WHERE clause
--	---	--

The following functions affect output only, not base row selection.

Poor relational models severely limit physical design choices and generally force more Full Table Scans.

GROUP BY HAVING WITH WITH ... BY ...	ORDER BY UNION INTERSECT EXCEPT
---	--

Secondary Index Considerations

The facing page describes key considerations involved in decisions regarding the use of Secondary Indexes. It is important to weigh the costs of Secondary Indexes against the benefits.

- Some of these costs are increased use of disk space and increased I/O.
- The main benefit of Secondary Indexes is faster set selection. Choose them on frequently used set selections.

– REMEMBER –

Data demographics change over time.

Revisit all index choices regularly to make sure that they remain appropriate and serve you well.

Secondary Index Considerations

- Secondary Indexes consume disk space for their subtables.
- INSERTs, DELETEs, and UPDATEs (sometimes) cost double the I/Os.
- **Choose Secondary Indexes on frequently used set selections.**
 - Secondary Index use is typically based on an Equality search.
 - A NUSI may have multiple rows per value.
- **The Optimizer may not use a NUSI if it is too weakly selective.**
- Avoid choosing Secondary Indexes with volatile data values.
- Weigh the impact on Batch Maintenance versus tactical queries or OLTP applications.
- USI changes are Transient Journalled (UNDO rows). NUSI changes are not.
 - A NUSI subtable row is AMP local and the base table UNDO row can be used to rollback the base table row and NUSI subtable row. A USI subtable row is usually on a different AMP than the base table row.
- Drop NUSIs that are not used.

Data demographics change over time. Revisit ALL index (Primary and Secondary) choices regularly.

Make sure they are still serving you well.

Secondary Index Usage

The facing lists common usage for a USI and a NUSI.

Secondary Index Usage

Unique Secondary Index (USI) Usage

- A USI is used to maintain uniqueness in a column or columns.
- Usage is determined by specifying the USI value in an equality condition in the WHERE clause or ON clause.
- Unique Secondary Indexes support ...
 - Nested Joins
 - Row-hash locking

Non-unique Secondary Index (NUSI) Usage

- Usage is determined by specifying the NUSI value in an equality condition in the WHERE clause or ON clause.
- Non-Unique Secondary Indexes support Nested Joins and Merge Joins
- Optimizer can choose to use bit-mapping for weakly selective (>10%) NUSIs which can alleviate limitations associated with composite NUSIs.
- In some cases, it may be better to use multiple single-column NUSIs (City, State) instead a single composite NUSI.
 - User has to balance the overhead of multiple NUSIs as compared to a single composite NUSI.
- Can be used to “cover” a query, avoiding base table access.
- Can significantly reduce base table I/O during value and join operations.

Secondary Index Candidate Guidelines

All Primary Index candidates are Secondary Index candidates.

Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

The optimizer does not only look at selectivity of a column to determine if a FTS or an indexed access will be used in a given plan. The decision is made based after comparing the total cost of both approaches, after considering multiple factors, including row size, block size, number of rows in the table, and also the I/O and CPU cost (based on the current hardware cost factors).

In this course, we are going to 5% as a guideline for NUSI selectivity.

Example 1: Assume a table has 100M rows and a column has 50 distinct values that are evenly distributed (each value has the same number of rows). Therefore, each value has 2M rows and effectively represents 2% of the rows. The NUSI would be used.

Example 2: Assume a table has 100M rows and a column has 20 distinct values that are evenly distributed (each value has the same number of rows). Therefore, each value has 5M rows and effectively represents 5% of the rows. The NUSI would be used.

Example 3: Assume a table has 100M rows and a column has 10 distinct values that are evenly distributed (each value has the same number of rows). Therefore, each value has 10M rows and effectively represents 10% of the rows. The NUSI would not be used.

The greater the discrepancy between typical rows per value and max rows per value, the higher the probability the NUSI would not be used based on the max value used to qualify the rows.

Secondary Index Candidate Guidelines

- All Primary Index (PI) candidates are Secondary Index candidates.
 - A UPI is a USI candidate and a NUPI is a NUSI candidate.
- Columns that are not PI candidates should also be considered as NUSI candidates.
- A NUSI will be used depending on the percentage of table rows that will be accessed. For example:
 - If the number of rows accessed via a NUSI is $\leq 5\%$, the NUSI will be used.
 - If the number of rows accessed via a NUSI is 5 – 10%, the NUSI may or may not be used.
 - If the number of rows accessed via a NUSI is $> 10\%$, the NUSI will not be used.
- If 5% is used as the guideline, then any column with 20 or more distinct values is considered as a NUSI candidate.
 - The optimizer (based on statistics) will decide to use (or not) the NUSI for specific values.
 - The greater the discrepancy between typical rows per value and max rows per value, the higher the probability the NUSI would not be used based on the max value used to qualify the rows.
- These are initial guidelines for candidate selection. Validate (via Explain and testing) that the NUSI will be chosen AND that it will provide better performance.

Value and Range Access

Value Access Frequency is a numeric rating which tells you how many times all known transactions access the table in a given time interval (e.g., a one-year period). It measures how frequently a column, or columns, is accessed by SQL statements containing an equality value.

Range Access Frequency is a numeric rating which tells you how many times all known transactions access the table in a given time interval (e.g., a one-year period). It measures how frequently a column, or columns, is accessed by SQL statements that access a range of values such as a DATE range. These types of queries may contain inequality or BETWEEN expressions.

A Value Access or Range Access of 0 implies that there is no need to access the table through that column. Since NUSIs require system resources to maintain them (INSERTs and DELETEs require additional I/O to update the SI subtables), there is no point in having a NUSI if it is not used for access. All NUSI candidates with very low Value Access or Range Access Frequency should be eliminated.

Value and Range Access

Value Access:

- How often a column appears with an equality value. For example:
WHERE column_name = hardcoded_value or substitutable_value

Range Access:

- How often a column is used to access a range of data values (e.g., range of dates).
For example:
WHERE column_name BETWEEN value AND value or WHERE column_name > value

Value Access or Range Access Frequency:

- How often in a given time interval (e.g., annually) all known transactions access rows from the table through this column either with an equality value or with a range of values.

Notes:

- The above demographics result from Activity Modeling.
- **Low Value Access or Range Access Frequency:**
 - Secondary Index overhead may cost more than doing the FTS.
- NUSIs may be considered by the optimizer for joins. In the following exercises, we are going to eliminate NUSIs with a value access of 0, but we may need to reconsider the NUSI as an index choice depending on join access (when given join metrics).
- EXPLAINS indicate if the index choices are utilized or not.

Change Rating

Change Rating is a number that comes from **Activity Modeling** – previously called Application & Transaction Modeling (ATM).

- Change Rating indicates how often the values in a column, or columns, are updated.
- It is a value from 0 to 10, with 0 describing those columns which never change and 10 describing those columns which change with every write operation.
- The Change Rating values of various types of columns are shown on the facing page.

Change Rating has nothing to do with the SQL INSERT or DELETE statements. A table may be subject to frequent INSERTs and/or DELETEs, but the Change Ratings of columns will be low as long as the values within those columns remain stable throughout the lifetime of the row.

Change Rating is dependent only on the SQL UPDATE statement. Change Rating is affected when column values are UPDATED.

Utilize Change Rating when choosing indexes. Primary Indexes must be based on columns with very stable data values. PI columns should never have Change Ratings higher than 1.

Secondary Indexes should be based on columns with at least fairly stable data values. You should not choose columns with Change Ratings higher than 3 for SIs.

Change Rating

Change Rating indicates how often values in a column are UPDATED:

- 0 = column values never change.
- 10 = column changes with every write operation.

PK columns are always 0.

Historical data columns are always 0.

Data that does not normally change = 1.

Update tracking columns = 10.

All other columns are rated 2 - 9.

Base Primary Index choices on columns with very stable data values:

- A change rating of 0 - 1 is reasonable.

Base Secondary Index choices on columns with fairly stable data values:

- A change rating of 0 - 3 is reasonable.

Exercise 3 – Example

In this exercise, you will work with the same tables you used to identify PI candidates in a previous exercise.

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates. The table on the facing page provides you with an example of how to apply the Secondary Index Candidate Guidelines.

You will make further index choices for these tables in following exercises.

Note: These exercises do not provide row sizes. Therefore, assume that the rows could be as large as 960 bytes and assume a typical block size of 96 KB.

Secondary Index Candidate Guidelines

All Primary Index candidates are Secondary Index candidates. A UPI is considered a USI candidate. NUPI candidates are also potential NUSI candidates.

Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

In this exercise, you will look at three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics that you will look at are:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

The example on the facing page illustrates how to apply these guidelines. You will make final index choices for these tables in Exercise 4 (later module).

Module 20: Exercise 3 – Example

The sample tables now include value/range and change row demographics.

- Delete any PI candidates that are too volatile.
- Identify any USI candidates.
- Identify any NUSI or VONUSI candidates if typical rows/value is $\leq 5\%$ or # of distinct values ≥ 20 AND Value/Range Access > 0 .
- Later exercises will guide your final choices.

Change Rating Guidelines:

- PI – change rating 0 - 1.
- SI – change rating 0 - 3.

Value Access Guideline:

- NUSI Value Access > 0
- VONUSI Range Access > 0

300,000,000 Rows	A	B	C	D	E	F	G	H	
PK/FK	PK,SA		FK,NN	NN,ND					
Value Access	5K	26K	0	500K	0	0	0	260	
Range Access	12	0	0	0	0	0	0	4K	
Distinct Values	300M	35M	7.5M	300M	8	15M	15M	700	
Max Rows/Value	1	12	500	1	40M	50	4M	450K	
Max Rows/NULL	0	5	0	0	0	4M	0	50K	
Typical Rows/Value	1	7	35	1	35M	15	15	400K	
Change Rating	0	1	4	2	0	4	4	9	
PI/SI	UPI	NUPI	NUPI2	UPI					
	USI	NUSI		USI					
Collect Statistics (Y/N)									

Exercise 3 – Choosing SI Candidates

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates.

- All Primary Index candidates are Secondary Index candidates.
- Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

You also need to consider three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics to consider include:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

Exercise 3 – Choosing SI Candidates (cont.)

ENTITY 1							
500,000,000 Rows	A	B	C	D	E	F	
PK/FK	PK,UA						
Value Access	250K	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Distinct Values	500M	475M	1.5M	1.25M	200M	5M	
Max Rows/Value	1	2	400	450	3	110	
Max Rows/NULL	0	0	0	0	10M	0	
Typical Rows/Value	1	1	325	350	2	90	
Change Rating	0	2	1	1	1	1	
PI/SI	UPI	NUPI	NUPI	NUPI		NUPI	
Collect Statistics (Y/N)							

Exercise 3 – Choosing SI Candidates (cont.)

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates.

- All Primary Index candidates are Secondary Index candidates.
- Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

You also need to consider three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics to consider include:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

Exercise 3 – Choosing SI Candidates (cont.)

		ENTITY 2					
50,000,000 Rows		G	H	I	J	K	L
PK/FK		PK,SA					
Value Access		5K	365	52	12	0	0
Range Access		12	0	0	0	0	2.6K
Distinct Values		50M	500K	45M	12	50	900K
Max Rows/Value		1	200	2	5M	1.2M	60
Max Rows/NULL		0	0	450K	0	0	0
Typical Rows/Value		1	100	1	4M	950K	50
Change Rating		0	0	9	1	2	0
PI/SI		UPI	NUPI				NUPI
Collect Statistics (Y/N)							

Exercise 3 – Choosing SI Candidates (cont.)

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates.

- All Primary Index candidates are Secondary Index candidates.
- Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

You also need to consider three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics to consider include:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

Exercise 3 – Choosing SI Candidates (cont.)

5,000,000 Rows	DEPENDENT						
	A	M	N	O	P	Q	
PK/FK	PK				NN,ND		
	FK	SA					
Value Access	0	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Distinct Values	2M	50	90K	3M	5M	2M	
Max Rows/Value	4	200K	75	2	1	5	
Max Rows/NULL	0	0	0	390K	0	1M	
Typical Rows/Value	1	60K	50	1	1	1	
Change Rating	0	0	3	1	0	1	
PI/SI	UPI				UPI		
	NUPI		NUPI				
Collect Statistics (Y/N)							

Exercise 3 – Choosing SI Candidates (cont.)

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates.

- All Primary Index candidates are Secondary Index candidates.
- Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

You also need to consider three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics to consider include:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

Exercise 3 – Choosing SI Candidates (cont.)

ASSOCIATIVE 1					
1.5 Billion Rows	A	G	R	S	
PK/FK	PK				
	FK	FK,SA			
Value Access	26K	0	0	0	
Range Access	0	0	0	0	
Distinct Values	490M	50M	75K	4M	
Max Rows/Value	5	40	21K	400	
Max Rows/NULL	0	0	0	0	
Typical Rows/Value	3	25	19K	350	
Change Rating	0	0	0	0	
PI/SI	UPI				
	NUPI	NUPI	NUPI?	NUPI	
Collect Statistics (Y/N)					

Exercise 3 – Choosing SI Candidates (cont.)

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates.

- All Primary Index candidates are Secondary Index candidates.
- Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

You also need to consider three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics to consider include:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

Exercise 3 – Choosing SI Candidates (cont.)

ASSOCIATIVE 2						
500,000,000 Rows	A	M	G	T	U	
PK/FK	PK					
	FK		FK			
Value Access	0		0	0	0	
Range Access	0		0	0	0	
Distinct Values	250M		50M	2.8M	750	
Max Rows/Value	3		150	180	675K	
Max Rows/NULL	0		0	0	0	
Typical Rows/Value	1		8	170	500K	
Change Rating	0		0	0	0	
PI/SI	UPI					
	NUPI		NUPI	NUPI		
Collect Statistics (Y/N)						

Exercise 3 – Choosing SI Candidates (cont.)

Use the Secondary Index Candidate Guidelines below to identify all USI and NUSI candidates.

- All Primary Index candidates are Secondary Index candidates.
- Columns that are not Primary Index candidates have to also be considered as NUSI candidates. A NUSI will be used by the Optimizer to select data if it is strongly selective. A guideline to use in initially selecting NUSI candidates is the following:

If the number of distinct values ≥ 20 , then the column is a NUSI candidate.

You also need to consider three additional demographics to eliminate potential index candidates and to possibly choose Value-Ordered NUSI candidates. The three additional data demographics to consider include:

- Change Rating
- Value Access
- Range Access

Use the following Change Rating demographics guidelines to eliminate those candidates that do not fit the guidelines.

- PI candidates should have Change Ratings from 0 - 1.
- SI candidates should have Change Ratings from 0 - 3.

Also, eliminate those NUSI candidates which have Value Access = 0 and Range Access = 0.

If a Range Access is greater than 0, then consider the column as a possible Value-Ordered NUSI (VONUSI) candidate.

Exercise 3 – Choosing SI Candidates (cont.)

HISTORY						
3.65 Billion Rows	A	DATE	D	E	F	
PK/FK	PK					
	FK	SA				
Value Access	10M	26K	0	0	0	
Range Access	0	100K	0	0	0	
Distinct Values	500M	730	N/A	N/A	N/A	
Max Rows/Value	18	5.5M	N/A	N/A	N/A	
Max Rows/NULL	0	0	N/A	N/A	N/A	
Typical Rows/Value	3	4.5M	N/A	N/A	N/A	
Change Rating	0	0	N/A	N/A	N/A	
PI/SI	UPI					
	NUPI					
Collect Statistics (Y/N)						

Optimizer Access Scenarios

Given the SQL WHERE clause on the facing page, the Optimizer decides which column it will use to access the data. This decision is based upon what indexes have been defined on the two columns (**Col_1** and **Col_2**).

When you examine the table, you can see that the optimizer chooses the most efficient access method depending on the situation. Interesting cases to note are as follows:

If **Col_1** is a NUPI and **Col_2** is a USI, the Optimizer chooses **Col_1** (NUPI) if its selectivity is close to a UPI (nearly unique). Otherwise, it accesses via **Col_2** (USI) since only one row is involved, even though it is a two-AMP operation.

If both columns are NUSIs, the Optimizer must determine the how selective each of them is. Depending on the relative selectivity, the Optimizer may choose to access via **Col_1**, **Col_2**, NUSI Bit Mapping or a FTS.

If either one of the columns is a NUSI or the other column is not indexed, the Optimizer determines the selectivity of the NUSI. Depending on this selectivity, it chooses either to utilize the NUSI or to do a FTS.

Whenever one of the columns is used to access the data, the remaining condition is used as a row qualifier. This is known as a **residual condition**.

Optimizer Access Scenarios

SINGLE TABLE CASE

WHERE Table_1.Col_1 = :value_1
AND Table_1.Col_2 = :value_2 ;

Col_2 Col_1	USI	NUSI	NOT INDEXED
UPI	UPI	UPI	UPI
NUPI	NUPI or USI ₁	NUPI	NUPI
USI	USI	USI	USI
NUSI	USI	Either, Both, or FTS ₂	NUPI or FTS ₃
NOT INDEXED	USI	NUSI or FTS ₃	FTS

Column the Optimizer uses for access.

Notes:

1. The Optimizer prefers Primary Indexes over Secondary Indexes. It chooses the NUPI if only one I/O (block) is accessed.
The Optimizer prefers Unique indexes over non-unique indexes. Only one row is involved with USI even though it is a two-AMP operation.
2. Depending on relative selectivity, the Optimizer may use either NUSI, may use both with NUSI Bit Mapping, or may do a FTS.
3. It depends on the selectivity of the index.

Secondary Index Overhead

Secondary Indexes has two type of overhead – additional I/O maintenance and disk space for the secondary index subtables.

Understanding the cost of various Teradata transactions in terms of I/O will help you avoid unnecessary I/O overhead when doing your physical design.

Many factors can influence the number of physical I/Os in a transaction. Some are listed on the facing page.

The main concept of the next few pages is to help you understand the relative cost of doing INSERT, DELETE, and UPDATE operations. This understanding enables you to detect subsequent problems when doing performance analysis on a troublesome application. When counting I/O, it is important to remember that all such calculations give you a relative – not the absolute – cost of the transaction. Any given I/O operation may or may not cause any actual physical I/O.

- Normally, when making a change to a table (INSERT, UPDATE, and DELETE), not only does the actual table have to be updated, but before-images have to be written in the Transient Journal to maintain transaction integrity. Transient Journal space is automatically allocated and is integrated with the WAL (Write-Ahead-Logic) Log, which has its own cylinders and file system.

Additional I/O

A table may also have Join Indexes, Hash indexes, or a Permanent Journal associated with it. Join Indexes can also have secondary indexes.

In addition the number of I/Os for changes to a table, these options will result in additional I/Os.

Whenever Permanent Journaling is used, additional I/O is incurred. The amount of this I/O varies according to whether you are using Before Imaging, After Imaging, or both, and whether the imaging is single or dual. The table on the facing page shows how many I/O operations are involved in writing the Permanent Journal block and the Cylinder Index.

To calculate the Total Permanent Journal I/O for PJ INSERTs, DELETEs and UPDATEs, you apply the appropriate formula shown on the facing page.

Permanent Journal I/O is in addition to any I/O incurred during the operation itself. In order to calculate the TOTAL I/O for an operation, you must sum the I/Os from the operation with the Total PJ I/O corresponding to that operation.

Secondary Index Overhead

- **Secondary indexes have two types of additional overhead.**
 - Additional I/O maintenance – SI subtable rows are updated as base table rows are inserted, updated, or deleted.
 - Secondary index subtables require additional disk space (covered later).
- Changes to data rows and USI rows require before-images (undo rows) and after-images (redo rows) to be written to the WAL log.
- Data and index subtable block I/O may or may not require Cylinder Index I/O.
 - I/Os may be done serially or in parallel.
- **Many factors influence the number of physical I/Os in a transaction:**
 - Cache hits, rows per block, data block size, BLOBs/CLOBs, etc.
 - Block growth, block splits, cylinder migrates, Mini-Cylpacks
 - Number of spool files and spool file sizes
- **"Logical I/O counts" are used to indicate the relative cost of a transaction.**
 - A given I/O operation may not cause any actual physical I/O.
- A table may also have Join/Hash indexes, secondary indexes on Join Indexes, or a Permanent Journal associated with it.
 - **These options will result in additional I/O.**

Transient Journal I/O

The **Transient Journal (TJ)** exists to permit the successful rollback of a failed transaction. Transactions are not committed to the database until an End Transaction request has been received by the AMPs, either implicitly or explicitly. Until that time, there is always the possibility that the transaction may fail in which case the participating table(s) must be restored to their pre-transaction state.

The **Transient Journal** maintains a copy of all before-images of all rows affected by the transaction. If the event of transaction failure, the before images are reapplied to the affected tables, the images are deleted from the journal and a rollback operation is completed. When the transaction completes (assume successfully), at the point of transaction commit, the before-images for the transaction are discarded from the journal (effectively removed from the WAL log). Normally, when making a change to a table (INSERT, UPDATE, and DELETE), not only does the actual table have to be updated, but before-images have to be written in the TJ to maintain transaction integrity.

The preservation of the before-change row images for a transaction is the task of the Write Ahead Logic (WAL) component of the Teradata database management software. The system maintains a separate TJ (undo records) entry in the WAL log for each individual database transaction whether it runs in ANSI or Teradata session mode. TJ (undo) rows are written to the WAL log BEFORE the modification is made to the base table row.

Characteristics of the WAL Log include the following:

- Before-image or undo records used for transaction rollback.
- After-image or redo records for updating disk blocks and insuring file system consistency during restarts, based on operations performed in cache during normal operation.
- The WAL has a simpler structure than a table. Log data is a sequence of WAL records, different from normal row structure and not accessible via SQL.

Some situations where Transient Journal is not used to store before-images include:

- INSERT / SELECT into an empty table
- DELETE tablename; (effectively deletes all the rows in a table)
- Utilities such as TPT LOAD or UPDATE (FastLoad and MultiLoad)
- ALTER TABLE

When a DELETE ALL is done, the master index and the cylinder indexes are updated. An entry is actually placed in the Transient Journal indicating that a “DELETE ALL” has been issued. Before-images of the individual deleted rows are not stored in the TJ. In the event a node happens to fail in the middle of a DELETE ALL, the TJ is checked for the deferred action that indicates a DELETE ALL was issued. The system checks to ensure that the DELETE ALL has completed totally as part of the restart process.

ALTER TABLE does not store before-images in the TJ and is always checkpoint-restartable (special entries in the TJ) at the cylinder level. If ALTER TABLE just results in a dictionary change, then there is no check-pointing.

Transient Journal I/O

The Transient Journal (TJ) is ...

- An automatic and transparent journal of transaction before-images (or undo records) maintained in the WAL log.
- Provides for automatic rollback in the event of TXN failure.
- Provides “**Transaction Integrity**”.
- TJ images are maintained in the WAL Log. The WAL Log includes the following:
 - **Before-images or undo records used for transaction rollback.**
 - **After-images or redo records for updating disk blocks** and insuring file system consistency during restarts, based on operations performed in cache (FSG) during normal operation.

Therefore, when modifying a table, there are I/O's for the data table and its secondary indexes and the WAL log (undo and redo records).

Some situations where Transient Journal does NOT store before-images of rows include:

- INSERT INTO ... SELECT ... into an empty table.
- DELETE tablename; (Deletes all the rows in a table)
- Utilities such as TPT LOAD and UPDATE (FastLoad and MultiLoad)
- ALTER TABLE

INSERT and DELETE Operations

To calculate the number of I/Os required to INSERT a new data row or DELETE an existing row, it is necessary to do three subsidiary calculations. They are:

Number of I/Os required to INSERT or DELETE the row itself = **five**.

Number of I/Os required for each Unique Secondary Index (USI) = **five**.

Number of I/Os required for each Non-Unique Secondary Index (NUSI) = **three**.

The overall formula for counting I/Os for INSERT and DELETE operations is shown at the bottom of the facing page. The number of I/Os must be doubled if Fallback is used.

INSERT and DELETE Operations

INSERT INTO tablename ...; DELETE FROM tablename ...; (* = I/O operation)

- | | |
|----------------------|--|
| DATA ROW | <ul style="list-style-type: none"> * READ Data Block * WRITE Transient Journal record (UNDO row) to WAL Log * INSERT or DELETE the data row, and WRITE REDO row (after-image) to WAL Log * WRITE new Data Block * WRITE Cylinder Index |
| For each USI | <ul style="list-style-type: none"> * READ USI subtable block * WRITE Transient Journal record (UNDO index row) to WAL Log * INSERT or DELETE the new USI subtable row, and WRITE REDO row (after-image) to WAL Log for the USI subtable row * WRITE new USI subtable block * WRITE Cylinder Index |
| For each NUSI | <ul style="list-style-type: none"> * READ NUSI subtable block * ADD or DELETE the ROWID on the ROWID LIST or * ADD or DELETE the NUSI subtable row * WRITE new NUSI subtable block * WRITE Cylinder Index |

I/O operations per row = 5 + [5 * (#USIs)] + [3 * (#NUSIs)]

Double for FALLBACK

UPDATE Operations

To calculate the number of I/Os required when updating a data column, it is necessary to perform three subsidiary calculations. They are:

The number of I/Os required to UPDATE the column in the data row itself = **five**.

The number of I/Os required to change any USI subtable containing the particular column which was updated = **ten** (five to remove the old subtable row and five to add the new subtable row).

The number of I/Os required to change the subtable of any NUSI containing the particular column which was updated = **six** (three to remove the old Row ID or subtable row and three to add the new Row ID or subtable row).

The overall formula for counting I/Os for UPDATE operations is shown at the bottom of the facing page.

– REMEMBER –
You are simply estimating the relative cost of a transaction.

UPDATE Operations

UPDATE tablename SET colname = *expression* ... (other than PI column) (* = I/O Operation)

DATA ROW

- * READ Data Block
- * WRITE Transient Journal record (UNDO row) to WAL Log
- * UPDATE the data row, and WRITE REDO row (after-image) to WAL Log
- * WRITE new Data Block
- * WRITE Cylinder Index

If colname = USI column

- * READ current USI subtable block
- * WRITE TJ record (UNDO row) into WAL Log
- * DELETE USI subtable row, and WRITE REDO row (after-image) to WAL Log
- * WRITE USI subtable block
- * WRITE Cylinder Index
- * READ new USI subtable block
- * WRITE TJ record (UNDO row) into WAL Log
- * INSERT new Index Subtable row, and WRITE REDO row (after-image) to WAL Log
- * WRITE new USI subtable block
- * WRITE Cylinder Index

If colname = NUSI column

- * READ current NUSI subtable block
- REMOVE data row's RowID from RowID list or REMOVE NUSI subtable row if last RowID
- * WRITE NUSI subtable block
- * WRITE Cylinder Index
- * READ new NUSI subtable block
- ADD data row's RowID to RowID list or ADD new NUSI subtable row
- * WRITE new NUSI subtable block
- * WRITE Cylinder Index

I/O operations per row = 5 + [10 * (#USIs)] + [6 * (#NUSIs)]

Double for FALLBACK

Primary Index Value UPDATE

Updating the Primary Index Value is the most I/O intensive operation of all. This is due to the fact that any change to the PI invalidates all existing secondary index “pointers.”

To calculate the number of I/Os required to UPDATE a PI column, it is necessary to perform three subsidiary calculations:

- The number of I/Os required to UPDATE the PI column in the data row itself.
- The number of I/Os required to change any USI subtable
- The number of I/Os required to change any NUSI subtable

Study the steps on the facing page. Notice that updating a PI value is equivalent to first deleting and then inserting a row. All the steps necessary to do a DELETE are performed, and then all the steps necessary to do an INSERT are performed. Changing the PI value involves actually moving the row to the location determined by the new hash value. Thus, the number of steps involved in this process is exactly double the number of steps to perform either an INSERT or a DELETE.

The formula for calculating the number of I/Os involved in a PI value update (shown at the bottom of the facing page) can be derived by doubling the formula for INSERTing or DELETing:

$$\text{Formula for PI Value Update} = 10 + (5 * \# \text{ USIs}) + (6 * \# \text{ NUSIs})$$

Remember to double the number if Fallback is used.

Note: If the USI changes, then the number of I/O's for each changed USI is 8 in the preceding formula.

Primary Index Value Update

UPDATE tablename SET PI_column = *new_value* ... ;

(* = I/O Operation)

Note: Assume only PI value is changed – all Secondary Index subtable rows are updated.

- | | |
|----------------------|--|
| DATA ROW | <ul style="list-style-type: none"> ** READ current Data Block, WRITE TJ record (UNDO row) to WAL Log * DELETE the Data Row, and WRITE REDO row (after-image) to WAL Log ** WRITE new Data Block, WRITE Cylinder Index ** READ new Data Block, WRITE TJ record (UNDO row) to WAL Log * INSERT the DATA ROW, and WRITE REDO row (after-image) to WAL Log ** WRITE new Data Block, WRITE Cylinder Index |
| For each USI | <ul style="list-style-type: none"> * READ USI subtable block * WRITE TJ record (UNDO row) into WAL Log * UPDATE the USI subtable row with the new RowID, and WRITE REDO row (after-image) to WAL Log * WRITE new USI subtable block * WRITE Cylinder index |
| For each NUSI | <ul style="list-style-type: none"> * Read NUSI subtable block on AMP for current PI value * Read NUSI subtable block on AMP for new value UPDATE the RowID list for both of the subtable blocks ** WRITE new NUSI subtable blocks ** WRITE Cylinder Indexes |

I/O operations per row = 10 + [5 * (#USIs)] + [6 * (#NUSIs)]

Double for FALLBACK`

Summary – Access Method Comparison

We have seen in preceding modules that Teradata can access data (through indexes or Partition, or Full Table Scans). The facing page illustrates these various access methods in order of number of AMPs affected.

Unique Primary Index (UPI)

The UPI is the most efficient way to access data. Accessing data through a UPI is a one-AMP operation that leads directly to the single row with the desired UPI value. The system does not have to create a Spool file during a UPI access.

Non-Unique Primary Index (NUPI)

Accessing data through a NUPI is a one-AMP operation that may lead to multiple rows with the desired NUPI value. The system creates a spool file during a NUPI access, if needed. NUPI access is efficient if the number of physical block reads is small.

Unique Secondary Index (USI)

A USI is a very efficient way to access data. Data access through a USI is usually a two-AMP operation, which leads directly to the single row with the desired USI value. The system does not have to create a spool file during a USI access.

There are cases where a USI is actually more efficient than a NUPI. In these cases, the optimizer decides on a case-by-case basis which method is more efficient. Remember: the optimizer can only make informed decisions if it is provided with statistics.

Non-Unique Secondary Index (NUSI)

As we have seen, the non-unique secondary index (NUSI) is efficient only if the number of rows accessed is a small percentage of the total data rows in the table. NUSI access is an all-AMPs operation since the NUSI subtables must be scanned on each AMP. It is a multiple rows operation since there can be many rows per NUSI value. A spool file will be created if needed.

Full-Table Scan (FTS)

The Full-Table Scan is efficient in that each row is scanned only once. Although index access is generally preferred to a FTS, there are cases where they are the best way to access the data.

Like the situation with NUPIs and USIs, Full Table Scans can sometimes be more efficient than a NUSI. The optimizer decides on a case-by-case basis which is more efficient (assuming that it has been provided with statistics).

**The Optimizer chooses what it thinks is the fastest access method.
COLLECT STATISTICS to help the Optimizer make good decisions.**

Summary – Access Method Comparison

Unique Primary Index

- Very efficient
- One AMP, one row
- No spool file

Non-Unique Primary Index

- Efficient if the number of rows per value is reasonable and there are no severe spikes.
- One AMP, multiple rows
- Spool file if needed

No Primary Index

- Access is a full table scan without secondary indexes.

Unique Secondary Index

- Very efficient
- Two AMPs, one row
- No spool file

Non-Unique Secondary Index

- Efficient only if the number of rows accessed is a small percentage of the total data rows in the table.
- All AMPs, multiple rows
- Spool file if needed

Partition Scan

- Efficient because of partition elimination.
- All AMPs; all rows in specific partitions

Full-Table Scan

- Efficient since each row is touched only once.
- All AMPs, all rows
- Spool file may equal the table in size

The Optimizer chooses the fastest access method.

COLLECT STATISTICS to help the Optimizer make good decisions.

Module 17: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 17: Review Questions

1. With a NUPI, a technique to avoid a duplicate row check is to _____.
 - a. use set tables
 - b. use NOT NULL on the NUPI column(s)
 - c. create the table as a MULTiset table
 - d. compare data values byte-by-byte within a Row Hash to ensure uniqueness
2. Which type of usage normally applies to a USI? ____
 - a. Range access
 - b. NOT condition
 - c. Equality value access
 - d. Inequality value access
3. Which two types of usage normally apply to a composite NUSI that is hash-ordered? ____ ____
 - a. Covering index
 - b. Equality value access
 - c. Inequality value access
 - d. Non-covering range access
4. Which situation requires placing before-images of rows into the Transient Journal? ____
 - a. loading a table with TPT Load
 - b. DELETE all the rows in a table
 - c. UPDATE all the rows in a table
 - d. INSERT/SELECT into an empty table

Notes

After completing this module, you will be able to:

- Determine column sizing requirements based on chosen data type.
- Determine physical table row size.
- Create a table using Multi-value Compression and determine the amount of space saved.
- Determine table sizing requirements via estimates and empirical evidence.
- Create tables using the sparse MAP feature.

Teradata Proprietary and Confidential

Notes

Table of Contents

Row and Block Size Limits.....	18-4
1 MB Row Data Classes	18-6
General Row Format.....	18-8
Presence Bits	18-10
NULL and COMPRESS	18-12
Multi-value Compression.....	18-14
Presence Bits and Compression Example.....	18-16
Implementing Multi-value Compression	18-18
ALTER TABLE and Compression	18-20
Algorithmic Compression Example.....	18-24
Miscellaneous Compression Considerations	18-26
Teradata Compression Comparison	18-28
Sizing a Data Row Considerations.....	18-30
Teradata Data Types	18-32
INTEGER Data Types	18-34
DECIMAL and FLOAT Data Types.....	18-36
NUMBER Data Type.....	18-38
DATE and TIME Data Types	18-40
CHARACTER Data Types	18-42
Character Sets.....	18-44
BYTE Data Types	18-46
Large Object Data Types	18-48
Variable Column Offsets.....	18-50
Table Headers.....	18-52
Sizing Tables and Indexes.....	18-54
Sizing Techniques	18-56
Empirical Sizing.....	18-58
Collect Demographics Command	18-60
Collect Demographics Example.....	18-62
Spool Space.....	18-64
Release of Spool.....	18-66
System Sizing Exercise	18-68
Summary	18-70
Module 18: Review Questions	18-72
Lab Exercise 18-1	18-74
Lab Exercise 18-2	18-78

Row and Block Size Limits

This feature enables permanent tables to hold rows up to ~1 MB in size from the previous maximum size of ~64 KB. In addition, the database response rows destined for the Teradata clients have been increased from ~64 KB to ~1 MB.

Some additional considerations/limitations about the 1 MB row feature include:

- Increases the maximum supported size of *most* rows on disk from ~64 KB to ~1 MB.
- Increases the maximum supported SELECT list size and Response data size from ~64 KB to ~1 MB. This also applies to joins between tables.
- The maximum column size is still ~64 KB and the maximum number of columns is still 2048.
- Dictionary row size is still 64 KB on disk. This means Statistics rows are limited to 64 KB.
 - Provides the infrastructure for increasing the maximum size of a DBC dictionary row to ~1 MB in memory and on disk in the future.
- Adds capability to send a message containing ~16 MB worth of rows to and from a Client application. This applies to Imports/Exports and Array processing.

If a table is initially created with small rows and you use ALTER TABLE to add columns that increase the row length above ~ 64K, you will get the following error message.

ALTER TABLE Failed: 3798: A column or character expression is larger than the max size.

Client – 16MB message support

16 MB message support is technically independent from 1 MB row support.

1 MB Permanent row support is a functional enhancement, not a performance enhancement.

16 MB Message support is a performance enhancement for 1 MB row import/export operations. Multiple 1 MB rows can be packed into 16 MB messages to provide adequate performance and account for interface protocol overhead when performing import/export operations.

- Similar to 1 MB row support, 16 MB message support is also a Teradata and Client interdependent feature.
- When both Client application and Teradata have the 16 MB message support, Client applications can specify the maximum message size (or response buffer size) of up to 16 MB.
- The default maximum message size varies among Client products (e.g., 16 MB for TPT operators, 2 MB for JDBC, etc.).
- If necessary, 16 MB message support can be disabled via DBSControl.

Row and Block Size Limits

Teradata supports 1 MB data blocks (starting with Teradata 14.10).

Starting with Teradata 16.00, permanent tables can hold rows up to ~1 MB in size (prior maximum row size was ~64 KB).

- Database response rows (destined for the Teradata clients) have also been increased from ~64 KB to ~1 MB.
- Based on table column definitions, a table is considered to have either a small or large row format – internal row layout is different.
 - For most tables, the column definitions will have combined total # of bytes that is < ~64 KB. These tables will utilize a small row format.
 - If the column definitions have a combined total # of bytes that is > ~64 KB, then the table will use the large row format.
 - A table may have multiple columns (e.g., VARCHAR(64000) defined and require the large row format, but actually have rows that are much less than 64 KB. Rows can vary in length.
- Large row support also applies to Join Indexes, Secondary indexes, etc.
 - Although a table definition may require the "large row format" and have large rows, it is probably common for the secondary indexes to use the "small row" format.
- The maximum column size is still ~64 KB and the maximum number of columns that be defined in a table is still 2048.
 - This 1 MB row feature provides the foundation to increase these limits in future releases.

1 MB Row Data Classes

The table on the facing page identifies shows the Max Physical Row size by Data Class.

Some additional details for this table are:

- 1 MB Table Header is supported. A table header may be less than 64 KB in size or may be greater than 64 KB in size. If necessary, multiple 64 KB physical rows on disk can make up a table header and the set of 64 KB rows are read into memory, effectively constructing an in-memory 1 MB table header.
- A similar design is made for LOB subtables. However, a LOB can consist of many 64 KB. The limit for a LOB is still ~ 2 GB.
- Complex Data Type (CDT) and Geospatial rows cannot grow beyond 64 KB by design.

The 1 MB Data Row feature has the following limitations:

- Maximum specifiable row size in SQL DDL is 1,000,000 bytes.
- Maximum response row size is 1,040,000 bytes.
- *NOT* supported in standalone legacy load utilities (i.e., FastLoad, MultiLoad, and FastExport).
- *NOT* available on Small Cylinder systems.
- Permanent Journals cannot be defined for tables with row size > 64KB. This results in a syntax error, with the message – Journal options are not supported for tables with row size greater than 64 KB.
 - Existing Permanent Journals need to be dropped before moving to a system running Teradata 16.0.
- Unity Data Mover can move 64 KB rows as they currently exist. For tables (with definitions > 64KB up to 1MB), data mover can move 1MB tables only to 1MB enabled system. It cannot move 1MB tables to a system that is not 1MB enabled.

1 MB Row Data Classes

Not every row type in Teradata uses the physical row size of 1 MB.

Data Class	Max Physical Row Size
Permanent User	~1MB
Transient Journal	~1MB
Global Temporary	~1MB
Volatile	~1MB
INTERNAL Spool / Persistent Spool	~1MB
RESPONSE Spool / Persistent Spool	~1MB
Join/Hash Index (table)	~1MB
Secondary Index	~1MB
Reference Index	~1MB
Online Archive / Snapshot Log	~1MB
Table Header	~64KB
LOB Column	~64KB
Complex Data Type Index	~64KB
Permanent Journal	~64KB

General Row Format

The diagram on the facing page represents the internal row layout. A single row can be up to approximately 1 MB in length (starting with Teradata 16.00).

Historically permanent rows in Teradata have had a maximum size of ~ 64 KB due in part to their 2-byte row length field at the beginning of the physical row. To support tables with row definitions and rows greater than ~ 64 KB, a 4-byte row length field is needed, but it is NOT placed at the beginning of the row. For tables that support large rows (definition is > ~ 64 KB), the traditional 2-byte row length field is set to 0 and the 4-byte field is set accordingly. The flag byte within the row indicates whether the row format is small or large and if a partition number is included or not within the physical row.

Starting with 13.10, for any new or reinitialized systems, rows are aligned on 2-byte boundaries. This format is called the **Packed64 Format** and is what is used on most Teradata systems. If a physical row requires 195 bytes, it will really use 196 bytes of space.

- The illustration on the facing page assumes the Packed64 Format and this course will assume this format unless specifically noted.

The order in which data is physically stored with a row is transparent to the user. Teradata takes the liberty of rearranging the order of data columns within a row. This has no effect on the order in which those columns can be returned to the user. Storage and display are separate issues.

There are three general categories of table columns, which are stored in the row in the order listed:

- 1 – **Fixed Length Columns** – this section contains all the fixed length data columns which do not have compression defined.
- 2 – **Uncompressed data for Compressible Columns** – this section is composed of those data columns that are compressible but are neither compressed nor NULL. It is variable in length.

Includes both value-compressed and algorithmically-compressed data and can be stored in any order.

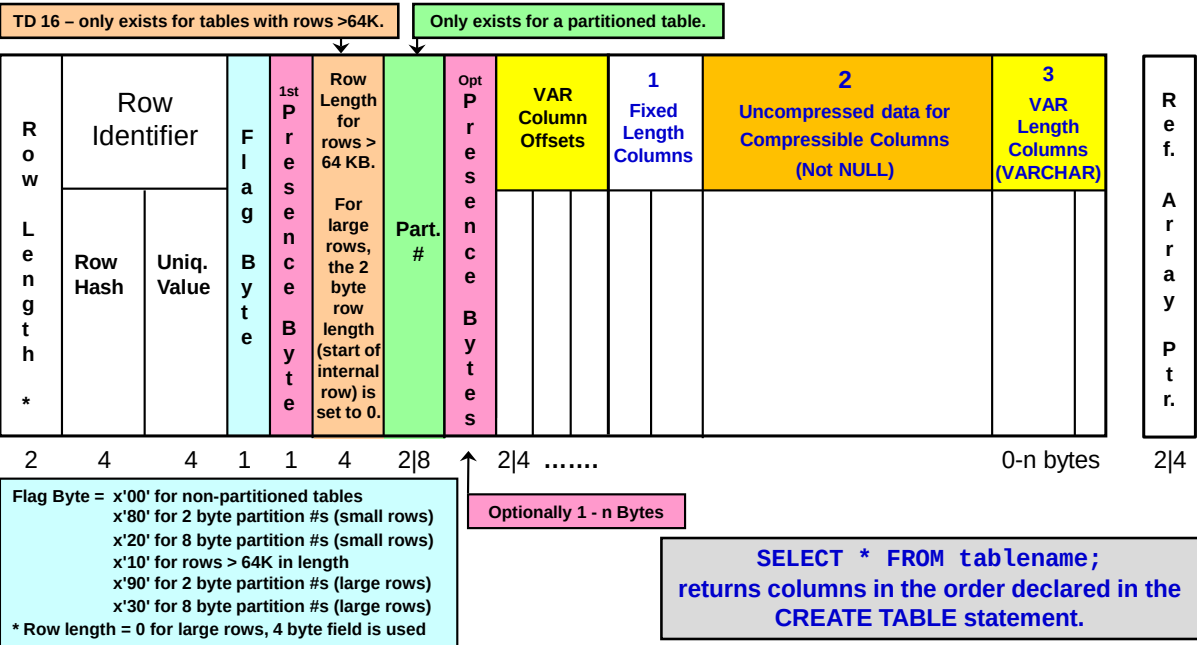
- 3 – **VARCHAR, VARBYTE, VARGRAPHIC, or NUMBER Columns that do not have compression defined** – this section contains all the variable length data columns without defined compression in the row. This section will only be present when variable length columns without compression are declared. The Column Offsets indicate exactly where in this section the variable length columns start.

Storage space is allocated in the row depending on the data type (size) of the column.

General Row Format

User data stored within a row generally falls in one of three categories:

- 1 – Fixed
- 2 – Compressible
- 3 – Variable Length columns



Presence Bits

Each row contains at least one Presence Byte. Presence Bytes are used to hold the Presence Bits that indicate the status of nullable and/or compressible data for a column. Since nullable and compressible columns are defined at the table level, each row in a table will have the same number of Presence Bits.

The first bit of the initial Presence Byte in each row is always set to 1 (see the gray area in the diagram on the facing page) which leaves 7 bits left to hold Presence Bit values (all Presence Bytes other than the first one in a row can hold 8 Presence Bits). This is illustrated on the facing page.

Every data column will have 0 to 10 presence bits.

- A column that is neither nullable nor compressible will have none.
- A column that is either nullable or compressible, but not both, will have 1.
- A column that is nullable and only nulls are compressed will have 1.
- A column that is both nullable and compressible on a value will have 2.
- As more values are compressed (up to 255), there may be as many as 9 presence bits allocated for a column.
- An algorithmically compressed (ALC) column adds an extra bit to indicate whether the column is algorithmically compressed or not, as follows:

When a table is created (CREATE TABLE), the default is that all columns are nullable. Unless otherwise specified, there can be “missing” or “unknown” values in every column. NULL is not the same as Blank, Zero, -1, High Value or other conventions used on host systems to represent missing or unknown data.

There are many cases where this default is acceptable and you will want to allow NULL values. For example, if you have a customer table with a column for an alternate phone number, you need to allow NULLs. Since some customers would not have an alternate phone number, making this an NN column would not be good design. The presence of NULLs does not affect your ability to create an index since any index may contain NULLs.

However, there are occasions when you will want to specify that NULL values are not allowed in certain columns. Do this by issuing the **NOT NULL** clause within the CREATE TABLE command. Three cases where you would want to do this are:

- All Primary Key columns require NOT NULL.
- All UNIQUE columns require NOT NULL.
- All columns where a known data value is required (NN columns).

Nullable Columns

If a column is nullable, there will be one Presence Bit in each row to indicate this fact. This internal bit will be 0 if the value in the column for that row is NULL and 1 if it is an actual value.

Presence Bits

The default for all columns is to allow NULLs (Nullable) and NOT have compression.

- NULLs are not automatically compressed. By default, space is allocated for NULL.

Presence Bits

- Based on the column attributes, Teradata may need 0 to 10 presence bits to represent a NULL and/or compressed data storage (including ALC) for a column in a row.
- Up to 255 different data values (plus NULL) can be compressed for a single column – this type of compression is referred to as Multi-value Compression (MVC).

Presence Bytes to support NULL, Multi-Value Compression, and Algorithmic Compression



Column Attribute	# of Bits	Description
NOT NULL	0	No NULLs; no values are compressed
Nullable	1	Allows NULLs; nothing is compressed
Nullable COMPRESS	1	Allows NULLs; only nulls are compressed
NOT NULL COMPRESS ('value')	1	No NULLs; compresses 1 value
Nullable COMPRESS ('value')	2	Allows NULLs; compresses nulls and 1 value
NOT NULL COMPRESS ('value1', 'value2', ...)	2 – 8	No NULLs; compresses multiple values
Nullable COMPRESS ('value1', 'value2', ...)	3 – 9	Allows NULLs; compresses nulls and values

NULL and COMPRESS

The default for all columns is nullable and not compressible which means that, unless otherwise specified, NULL values are allowed and Teradata will not automatically compress columns no matter what values are in them. You can override the default by using the **COMPRESS** clause at the column level when creating (or altering) a table.

The COMPRESS clause works in two different ways:

- When issued by itself (without a value or values), COMPRESS only causes all NULL values for that column to be compressed to zero space.
- When issued with an argument (e.g., COMPRESS value), the COMPRESS clause will compress every occurrence of the value in that column to zero space as well as cause every NULL value to be compressed.

The CREATE TABLE example on the facing page causes the system to compress every occurrence of the value “Australia” in the Country column to zero space as well as compress every NULL value in the Country column to zero space. This will save 20 bytes of space for every row where the employee had no country, or where the employee is associated with the country Australia. Compression is case-sensitive. Australia will be compressed in this example, but AUSTRALIA will not be compressed.

If you know in advance which columns will be accessed most frequently, you should declare compressible columns in decreasing order for possibly a small performance improvement.

You **CANNOT** compress the following:

- Components of the primary index
- Partitioning columns
- Identity columns
- Volatile table columns
- Derived table columns
- Referencing and referenced columns with Standard RI cannot be compressed. Batch and Soft RI is allowed with compressed columns.
- Columns defined with a UDT, Period, Geospatial, BLOB, or CLOB data type.

Note: Before Teradata 13.10, only fixed width columns could be compressed and the limit was 255 characters.

NULL and COMPRESS

- COMPRESS 'value(s)' compresses NULL and 'value(s)' to zero space.
 - Compression is case-sensitive.

How many bits are allocated?

```
CREATE TABLE Employee
( emp_num    INTEGER    NOT NULL,
  dept_num   INTEGER    COMPRESS,
  country    CHAR(20)   COMPRESS 'Australia',
  :
```

-----> _____

-----> _____

-----> _____

- COMPRESS all columns where at least 10% of the rows participate.
 - COMPRESS creates smaller rows, therefore more rows/block and fewer blocks in table.
- You **cannot** COMPRESS Primary Index, partitioning columns, referenced and referencing data columns (PK-FK) with Standard RI, Identity columns, derived data columns, or volatile table columns.

Multi-value Compression

Multi-value Compression (MVC) is sometimes referred to as Value List Compression (VLC). MVC allows specification of more than one compressed value on a column. This feature allows multiple values (up to 255 distinct values plus NULL to be compressed on a column. This compression capability reduces storage cost by storing more logical data per unit of physical capacity. Performance is improved because there is less physical data to retrieve during scan-oriented queries.

Therefore, Multi-value Compression is a feature that reduces the effective price of logical data storage capacity, and usually improves query performance because of reduced I/O. With MVC, the additional CPU time required to compress/decompress is minimal.

Multi-value Compression

What is Multi-value Compression?

- A feature that allows up to 255 distinct values (plus NULL) to be compressed per column – a.k.a., Value List Compression (VLC).
 - Reduces storage cost and performance is improved because there is less physical data to retrieve during scan-oriented queries.
- You can not add a compressed value to a column that if doing so, causes the table header row to exceed its maximum length.
 - The Table Header row contains an array of the compressed values.
- Algorithm compression adds an additional presence bit to indicate if the column is algorithmically compressed or not.

Number of Presence Bits needed for Multi-value Compression:

<u>Compressed Values</u>	<u># of Bits</u>
1	1
2 - 3	2
4 - 7	3
8 - 15	4
16 - 31	5
32 - 63	6
64 - 127	7
128 - 255	8

Note:

If column is “nullable”, there will be 1 additional presence bit.

Presence Bits and Compression Example

The facing page illustrates an example of how presence bits can be used for NULLs and/or Multi-value Compression. The first presence is always set to a 1 – basically an indicator bit and is there for legacy purposes. The first presence bit is not available for nulls and/or compression and this is why is it grayed out.

The CityID column requires no presence bits since it has NOT NULL specified and no compression. No bits required for CityID. The City column requires 4 presence bits - 3 bits for the 4 compressed values and 1 bit for a NULL in this column. NULL does not count as one of the values, it is a separate bit. StateCode requires 2 presence bits, one for the compressed value and one for the NULL. The standard presence byte has 8 bits, the first bit always being a 1, then 4 bits for city, 2 bits for StateCode, and the last bit (8th bit) is not used and is just set to zero.

Note that the row lengths shown in the table are the actual row lengths that are part of the physical data rows. These row lengths do not include the additional 2 bytes that are part of the row reference array.

Additional examples of Presence Bit usage with Multi-value Compression:

<u>Definition</u>	<u>Value</u>	<u>Presence Bits</u>
City CHAR(10) NOT NULL	'Atlanta'	None
City CHAR(10) NOT NULL COMPRESS ('Atlanta')	'Atlanta'	0
	'Boston'	1
City CHAR(10) COMPRESS	NULL	0
	'Atlanta'	1
City CHAR(10) COMPRESS ('Atlanta')	NULL	00
	'Atlanta'	01
	'Boston'	10

Note: First Presence Bit is 0 if compressed or 1 if not compressed.

City CHAR(10) COMPRESS ('Atlanta', 'Boston', 'Chicago', 'Dallas')

NULL	0000
'Atlanta'	0001
'Boston'	0010
'Chicago'	0011
'Dallas'	0100
'Eureka'	1000

Note: First Presence Bit is 0 if compressed or 1 if not compressed.

City CHAR(10) NOT NULL COMPRESS 'Atlanta', 'Boston', 'Chicago', 'Dallas')

'Atlanta'	001
'Boston'	010
'Chicago'	011
'Dallas'	100
'Eureka'	000

Presence Bits and Compression Example

```
CREATE TABLE Cities
( CityID      INTEGER NOT NULL
  ,City        CHAR(10) COMPRESS ('Atlanta', 'Boston', 'Chicago', 'Dallas')
  ,StateCode   CHAR(2)  COMPRESS ('CA'));
```

```
INSERT INTO Cities VALUES (1,'Albany', 'NY');
INSERT INTO Cities VALUES (2,'Atlanta', 'GA');
INSERT INTO Cities VALUES (3,'BOSTON', 'MA');
INSERT INTO Cities VALUES (4,'Dallas', 'TX');
INSERT INTO Cities VALUES (5,'Dayton', NULL);
INSERT INTO Cities VALUES (6,'San Diego', 'CA');
INSERT INTO Cities VALUES (7, NULL, NULL);
```

Row Length	ROWID & Flag	Presence Bits	Data stored in physical row
28		1 1000 10 0	1 Albany NY
18		1 0001 10 0	2 GA
28		1 1000 10 0	3 BOSTON MA
18		1 0100 10 0	4 TX
26		1 1000 00 0	5 Dayton
26		1 1000 01 0	6 San Diego
16		1 0000 00 0	7

Table Header

Presence Bit Usage for City

NULL	0000
'Atlanta'	0001
'Boston'	0010
'Chicago'	0011
'Dallas'	0100
Any other value	1000

Presence Bit Usage for StateCode

NULL	00
CA	01
Any other value	10

Implementing Multi-value Compression

This feature is implemented during table creation or modification. Customers must research the data demographics in order to decide which values to compress. The list of compressed values for each column must be defined in the CREATE TABLE statement.

As another example, the syntax for compressing several populous cities:

```
CREATE TABLE Properties
( St_Address VARCHAR(40),
  City          CHAR(20) COMPRESS ('Chicago', 'Los Angeles', 'New York'),
  State_Code    CHAR(2) );
```

Rules for Compression

Compression does not require extra computer resources to uncompress the block or row. Performance is enhanced since data remains compressed in memory so that the cache can hold more logical rows. The compressed values are stored in the table header row.

Rules for Compression:

- Up to 255 values (plus NULL) can be compressed per column.
- The maximum size of a compress value is 510 bytes (starting with TD 13.10)

You **CANNOT** compress the following:

- Components of the primary index
- Identity columns
- Volatile table columns
- Derived table columns
- Referencing and referenced columns with Standard RI cannot be compressed.

Miscellaneous notes about compression:

- You cannot add a compress value to a column if doing so would cause the table header to exceed its maximum length (1 MB).
- Compression is case-sensitive.

Since the compressed values are stored in table header row, the Create/Alter Table statement will fail if adding the compress value to a column causes the table header row to exceed the maximum size.

Columns with frequently occurring values may be highly compressed. Examples:

NULLs	Zeros	Default Values	Flags
Spaces	Binary Indicators (e.g., T/F)	Age (in years)	
Gender	Education Level	# of Children	
Credit Card Type	State	County	
City	Reason	Automobile Make	
Status	Category	Codes	

Implementing Multi-value Compression

CREATE TABLE accepts a list of values for field attribute 'COMPRESS'.

Compress 15 popular last names and the top 15 most populated countries.

```
CREATE TABLE People
( IdNumber    BIGINT          NOT NULL
  ,LastName   CHAR(30)        NOT NULL
                                COMPRESS ('Smith',      'Johnson',    'Williams',
                                             'Brown',      'Jones',       'Garcia',
                                             'Miller',     'Davis',       'Rodriguez',
                                             'Martinez',   'Hernandez',   'Lopez',
                                             'Gonzalez',   'Wilson',      'Anderson')
  ,FirstName  CHAR(20)        COMPRESS
  ,Address    VARCHAR(100)
  ,Country    VARCHAR(40)     COMPRESS ('Australia', 'Bangladesh',  'Brazil',
                                             'China',      'England',     'France',
                                             'Germany',    'India',       'Indonesia',
                                             'Japan',      'Mexico',      'Nigeria',
                                             'Pakistan',   'Russian Federation',
                                             'United States of America' )
  ,Gender     CHAR(1)         NOT NULL
);
```

How many presence bits are needed for this table?

ALTER TABLE and Compression

You **can** modify the compression specification for an existing column. However, when doing this, include all of the compressed values for the column.

Teradata uses a **lossless** compression method. This means that although the data is compacted, there is no loss of information.

The **granularity** of Teradata compression is the individual field of a row. This is the finest level of granularity possible and is superior for query processing, updates, and concurrency. **Field compression** offers superior performance when compared to row level or block level compression schemes. Furthermore, field compression allows compression to be independently optimized for the data domain of each column.

Teradata performs database operations directly on the compressed fields – there is no need to reconstruct a decompressed row or field for query processing. Of course, external representations and answer sets include the fully uncompressed results.

Up to 255 distinct values in each column can be compressed out of the row body. If the column is nullable, then NULLs are also compressed. The best candidates for compression are the most frequently occurring values in each column. The compressed values are stored in the table header. A bit field in every row header indicates whether the field is compressed and whether or not the value is NULL.

Teradata compression is completely **transparent** to applications, ETL (extraction, transforming, and loading of data), queries, and views. Compression is easily specified when tables are created or columns are added to an existing table.

Expectations / Guidelines

On an existing system, Multi-value Compression will free up storage space, but is unlikely to free up a significant amount of compute resources since the database will still need to execute the application on the logical data. However, on a system where CPU resource is available, the space savings from compression may allow more applications to be added prior to next system upgrade.

ALTER TABLE and Compression

ALTER TABLE allows you to add a new column with compressed values.

Add an "Education" column.

```
ALTER TABLE People
  ADD Education CHAR(12)
    UPPERCASE COMPRESS
  ( 'ELEMENTARY',
    'MIDDLE',
    'HIGH SCHOOL',
    'COLLEGE GRAD',
    'POST GRAD');
```

Note:

- Because UPPERCASE is specified, all values are compressed regardless of case.

You can use ALTER TABLE to add an additional compressed value to a list of values.

Add the Czech Republic to the list of compressed countries.

```
ALTER TABLE People ADD Country
COMPRESS
  ( 'Australia', 'Bangladesh',
    'Brazil', 'China',
    'England', 'France',
    'Germany', 'India',
    'Indonesia', 'Japan',
    'Mexico', 'Nigeria',
    'Pakistan', 'Russian Federation',
    'United States of America',
    'Czech Republic'
  );
```

Notes:

- All of the compressed values must be listed.
- A SHOW TABLE will show the compressed values in alphabetical sequence regardless of how they are entered.

Teradata Compression Capabilities

Multi-value Compression supports up to 510 bytes per value,

Algorithmic Level Compression

Provide the capability that will allow users the option of defining compression/decompression algorithms that would be implemented as UDFs and that would be specified and applied to data at the column level in a row. Initially, Teradata will provide three compression/decompression algorithms that will offer compression for UNICODE and LATIN data columns

The act of compressing/decompressing column data will require CPU cycles to be consumed. Consideration must be given to weighing the CPU cost of compression versus the potential space and performance gains that may be realized from leveraging this feature.

- The algorithms must be defined as regular scalar UDFs. The UDFs need to be specified in the column definition during the execution of a CREATE/ALTER TABLE statement.
- These algorithms will be invoked internally by Teradata to compress/decompress the column data when the data is moved into the tables or when data is retrieved from the tables.
- An algorithmically compressed column adds an extra bit to indicate whether the column is algorithmically compressed or not, as follows:
 - 0; the column is not algorithmically compressed.
 - 1; the column is algorithmically compressed

Block Level Compression

This feature provides the capability to perform compression on whole data blocks at the file system level before the data blocks are actually written to storage.

- BLC will compress/decompress only data blocks but will not be used on any file system structures such as Master/Cylinder indexes, the WAL log and table headers.
- Only primary data subtable and fallback copies can be compressed. Both objects are either compressed or neither is compressed.
- Secondary Indexes (USI/NUSI) cannot be compressed but Join Indexes can be compressed since they are effectively user tables.
- Only the compressed form of the data block will be cached, each block access must perform the data block decompression.
- On initial delivery, only one single compression algorithm will be supplied and used.
- Spool data blocks can be compressed via a system-wide tunable, as well as Temporary (Volatile and Global Temporary), WORK (sort Work space) & permanent journal.
- Once BLC is enabled on a table, reversion back to an earlier release for compressed tables is not allowed.

You can optionally utilize the FERRET utility to execute compression commands:

COMPRESS: "database_name.table_name"
UNCOMPRESS: "database_name.table_name"

Multi-value Compression (MVC) supports:

- Compression of VARCHAR, VARBYTE, or VARGRAPHIC columns.
- Number of characters for a compressed value can be up to 510 bytes.

Algorithmic Compression (ALC) or Column Level Compression

- Allows users to apply a compression algorithm to data at the column level in a row.
 - One example is to compress two-byte Unicode into one byte when the data is Latin (ASCII).
 - User defined ALC routines can also be used with BLOB and/or CLOB columns.
- **Compression/decompression is done by specifying a UDF function.**
 - Teradata provides some UDFs to do compression for UNICODE and LATIN data columns.
 - User can create and apply their own compression/decompression algorithms to columns.

Block Level Compression (BLC)

- Compression is performed by Teradata at the file system level on whole data blocks before the data blocks are actually written to/read from storage devices.
- **How is BLC set for a specific table?**
 - When loading data, SET QUERY_BAND = 'BLOCKCOMPRESSION=YES/NO;' FOR SESSION;
 - Ferret utility has commands to establish compression – COMPRESS/UNCOMPRESS table
 - DBSControl settings
- There is a CPU cost to compress/decompress on whole data blocks and is generally considered a good trade since CPU cost is decreasing while I/O cost remains high.

Algorithmic Compression Example

In some cases, such as when column values are mostly unique, Algorithmic Compression (ALC) can provide better compression results than Multi-value Compression. Algorithmic Compression allows you to define your own compression and decompression algorithms and apply them to data at the column level.

A user can create their own compression algorithms as external C/C++ scalar UDFs, and then specify them in the column definition of a CREATE TABLE/ALTER TABLE statement. Teradata invokes these algorithms to compress and decompress the column data when the data is moved into the tables or when data is retrieved from the tables.

ALC allows you to implement the compression scheme that is most suitable for data in particular column. The cost of compression and decompression depends on the algorithm chosen.

You can specify ALC alone, or both MVC and ALC on the same column. If you define both on the same column, ALC is applied only to those non-null values that are not specified in the value compression list of the MVC specification.

You can use ALC to compress columns with the following data types:

- BYTE
- CHARACTER
- GRAPHIC
- VARBYTE
- VARCHAR
- VARGRAPHIC

Teradata provides domain-specific functions for compressing and decompressing data. These functions are stored in the TD_SYSFNLIB system database. Two of these algorithms are:

- Compress TransUnicodeToUTF8
Takes Unicode character input and stores it in UTF8 format.
This is useful when the input data is predominantly Latin because UTF8 uses one byte to represent Latin characters and Unicode uses two bytes.
- Decompress TransUTF8ToUnicode
Takes the data previously compressed using the TransUnicodeToUTF8 function and converts it back to Unicode.

Algorithmic Compression Example

Given the following:

- The Description column is defined with a character set of Unicode.
 - Every character uses 2 bytes of space.
 - Use a Teradata-supplied function to save space for common ACSII Latin 7-bit (USA) characters.
- The TransUnicodeToUTF8 function compresses all non-null values in the Description column. Use this function when ...
 - Unicode column contains mostly ASCII Latin 7-bit (USA) characters – compress these to 1 byte.
- MVC is used to compress the values 'tools' and 'vacuums' in this Category column.
 - Because UPPERCASE is specified, all values of 'tools' and 'vacuums' are compressed regardless of case.

Example:

```
CREATE TABLE Products
( ProductID    INTEGER NOT NULL,
  Category     CHAR(10) NOT NULL UPPERCASE COMPRESS ('tools', 'vacuums'),
  Description   VARCHAR(500) CHARACTER SET UNICODE
                COMPRESS USING TD_SYSFNLIB.TransUnicodeToUTF8
                DECOMPRESS USING TD_SYSFNLIB.TransUTF8ToUnicode);
```

Miscellaneous Compression Considerations

Multi-value Compression (value list) is extended to intermediate spool files. Thus, when compressed columns are selected, compression is propagated to resulting spool files. Without compression for spool files, the intermediate join results for compressed tables can be very large, causing the need for additional spool space. Uncompressed VARCHAR data is also carried into spool.

Try to avoid compressing columns whose NULL values will be changing. When the value changes, the column will expand and you may get block growth and/or split operations. In practice, there may be exceptions to this rule. For example, it might be good to compress the NULL values in columns related to shipping an order until the order is actually shipped.

Additional sizing considerations:

- Adding a column that is not compressible expands all rows.
- Adding a column that is compressible and there are no spare presence bits expands all rows.
- Adding an additional value (or values) to a compressed set of values may cause additional presence bits to be allocated. If there are no spare presence bits, then additional presence bytes are allocated and all the rows (without that compressed value) are expanded.
- Dropping a column changes all row sizes where data is present.
- The maximum number of characters that can be listed in a COMPRESS clause is 8,188.

Miscellaneous considerations:

- Multi-value Compression and VARCHAR are both carried into intermediate spool files.
- You cannot use MVC on BLOB, CLOB, Period, Geospatial, UDT, or Identity columns.
- ALTER TABLE considerations:
 - Additional compressed values can be added to a column (ALTER). Include the complete list of compressed values with each ALTER command.
 - Adding a non-compressed column (to a table) expands all rows.
 - Adding a compressible column to a table that does not have any spare presence bits causes all rows to be expanded.
 - Adding an additional value (or values) to a compressed set of values may cause additional presence bits to be allocated. If there are no spare presence bits, then additional presence bytes are allocated and all the rows (without that compressed value) are expanded.

Teradata Compression Comparison

The chart on the facing page summarizes the three types of compression.

Teradata Compression Comparison

	Multi-value Compression (MVC)	Algorithmic Compression (ALC)	Block Level Compression (BLC)
Ease of Use	Easy to apply to well understood data columns and values.	Easy to apply at column with CREATE TABLE.	Set once and forget.
Analysis Required	Need to analyze data for common values.	Use Teradata algorithms or user-defined compression algorithms to match unique data patterns.	Need to analyze CPU overhead trade-off. You can turn on for all data on system <u>or</u> you can apply on a per table basis.
Flexibility	Works for a wide variety of data and situations.	Automatically invoked for values not replaced by MVC.	Automatically combined with other compression mechanisms.
Performance Impact	Reduced I/O – no or minimal CPU usage.	Depends on compression algorithm used.	Reduced I/O due to compressed data blocks. CPU cycles are used to compress/decompress.
Applicability	Replaces common values	Industry data, UNICODE, Latin data	All Data

A customer can choose any combination or all three on a column/table.

Sizing a Data Row Considerations

There are basically two row formats used by Teradata.

- Aligned Row Format (ARF) – older systems and releases
- Packed64

Prior to Teradata 16.00, the maximum row length is approximately 64 KB (the actual limit is 64,256 bytes), and this limit is the same for both packed64 and aligned row formats.

Starting with Teradata 16.00, rows can exceed 64 KB in length and can be as large as 1 MB in length (approximately)

Whether a system stores its data in packed64 or aligned row format depends on several factors. Only new systems (or systems that have been sysinit) on Teradata 13.10 can use the packed64 row format. Systems that are simply upgraded to 13.10 will continue to use the aligned row format.

The size of tables on a system that stores data in packed64 format is generally about 7% (range of 3 – 9%) smaller than the size of the same tables on a system that stores data in aligned row format. Storing data in packed64 format reduces the number of I/O operations required to access and write rows in addition to saving disk space.

Rows on Packed64 format systems are always aligned on even-byte boundaries. In other words, rows are never stored with an odd number of bytes. As a result, if a row has an odd byte length, the system adds a filler byte to the end of the row to make its length even.

Rows on Aligned Row Format 64-bit systems (most Teradata 12.0 and 13.0 systems) are always aligned on an 8-byte boundary. Additionally, the following pad bytes were added within the data row. The aligned row format is the only row format for these releases and will also be true if the system is simply upgraded to Teradata 13.10.

Pad Byte Field Name	Purpose
VARCHAR Offsets Array Alignment Pad Bytes	Aligns VARCHAR offsets array at a 2-byte boundary.
Fixed Length Column Alignment Pad Bytes	Aligns fixed length columns within the row on an 8-byte boundary.
Compressible Length Column Alignment Pad Bytes	Aligns value compressible length columns.
VARCHAR Column Alignment Pad Bytes	Aligns variable length columns within the row on an 8-byte boundary
Trailing Pad Bytes	Aligns entire row on an 8-byte boundary.

Also note that for PPI tables, rows in either format also have an additional 2 or 8 bytes of partition overhead between the presence bytes.

Sizing a Data Row Considerations

Determining a typical row length can be difficult.

- Different column data types occupy different amounts of disk space.
- Row length is determined by 3 categories of table columns.
 - Fixed length uncompressed data – this storage length is the easiest to calculate.
 - Variable length – storage space is allocated in the row depending on the size of the column.
 - Compressible data – includes both value-compressed and algorithmically-compressed data.

Row and data type alignment depends on type (format) of "row architecture".

- Rows are aligned on 2-byte boundaries
 - This row format does NOT have row section or data type alignment requirements.
 - This format is referred to as **Packed64 architecture**.
 - Note: For sizing exercises, this course will assume 2-byte boundary architecture.

Teradata Data Types

ANSI provides a smaller menu of data types than permitted by Teradata. REAL and DOUBLE PRECISION are implemented as Teradata FLOAT. ANSI NUMERIC is implemented as Teradata DECIMAL. ANSI does not support some of the specific Teradata data type extensions (e.g., BYTEINT).

Examples of the number of bytes allocated to the different data types are:

BYTEINT	1	
SMALLINT	2	
INTEGER	4	
BIGINT	8	
DECIMAL 1-2	1	
DECIMAL 3-4	2	
DECIMAL 5-9	4	
DECIMAL 10-18	8	
DECIMAL 19-38	16	
FLOAT	8	
NUMBER (14.0 Feature)	0 – 18	
DATE	4	
TIME	6/8	6 for packed64; 8 for ARF
TIME WITH TIME ZONE	8	
TIMESTAMP	10/12	10 for packed64; 12 for ARF
TIMESTAMP WITH TIME ZONE	12	
INTERVAL YEAR	2	
INTERVAL YEAR TO MONTH	4	
INTERVAL MONTH	2	
INTERVAL MONTH TO DAY	2	
INTERVAL DAY	2	
INTERVAL DAY TO MINUTE	8	
INTERVAL DAY TO SECOND	10/12	10 for packed64; 12 for ALR
INTERVAL HOUR	2	
INTERVAL HOUR TO MINUTE	4	
INTERVAL HOUR TO SECOND	8	
INTERVAL MINUTE	2	
INTERVAL MINUTE TO SECOND	6/8	6 for packed64; 8 for ARF
INTERVAL SECOND	6/8	6 for packed64; 8 for ARF
Period (Date)	8	
Period (Time)	12/16	12 for packed64; 16 for ARF
Period (Time With Time Zone)	16	
Period (Timestamp)	20	
Period (Timestamp With Time Zone)	24	
Period (Timestamp)	11	When Ending Element Value is UNTIL_CHANGED
Period (Timestamp With Time Zone)	13	When Ending Element Value is UNTIL_CHANGED

Packed64 – Also applies to 32-bit systems

ARF – Aligned Row Format for 64-bit systems

Teradata Data Types

<u>Data Type</u>	<u>Size of data type</u>
BYTEINT	1
SMALLINT	2
INTEGER	4
BIGINT	8
NUMBER (14,0)	0 – 18
DATE	4
TIME, TIME WITH TIME ZONE	6 or 8
TIMESTAMP, TIMESTAMP WITH TIME ZONE	10 or 12
INTERVAL YEAR, MONTH, DAY, HOUR, MINUTE, SECOND	2, 4, 6, 8, 10, or 12
PERIOD(DATE)	8
PERIOD(TIME), PERIOD(TIME) WITH TIME ZONE	12 or 16
PERIOD(TIMESTAMP), PERIOD(TIMESTAMP) WITH TIME ZONE	20 or 24
DECIMAL (n,m) (precision to 38 digits) or NUMERIC(n,m)	1 – 16
FLOAT or REAL, DOUBLE PRECISION	8
CHAR(n)	1 – 64,000
BYTE(n)	1 – 64,000
GRAPHIC(n)	1 – 32,000
VARCHAR(n), CHAR VARYING(n), LONG VARCHAR	1 – 64,000
VARBYTE(n)	1 – 64,000
VARGRAPHIC(n), LONG VARGRAPHIC	1 – 32,000
BLOB(n) or BINARY LARGE OBJECT(n)	1 – 2097088000
CLOB(n) or CHARACTER LARGE OBJECT(n)	1 – 2097088000
SYSUDTLIB.udt_name (User defined data type)	

INTEGER Data Types

When sizing rows, you need to know how much space will be occupied by the data in the rows. The first of the Teradata data constructs are the three types of Integer data types (BYTEINT, SMALLINT, INTEGER), and date (DATE), which is covered on the next page.

BYTEINT

As the name implies, BYTEINT data requires only a single byte. The first bit of the byte is the SIGN (+ or -) and the remaining seven bits allow the storage of numbers from -128 to +127 (-2^7 to $2^7 - 1$).

SMALLINT

SMALLINT data occupies 2 bytes. The first bit is the SIGN and the remaining 15 bits allow the storage of numbers from -32,768 to +32,767 (-2^{15} to $2^{15} - 1$).

INTEGER

INTEGER data occupies 4 bytes. The first bit is the SIGN and the remaining 31 bits allow the storage of numbers from -2,147,483,648 to +2,147,483,647 (-2^{31} to $2^{31} - 1$).

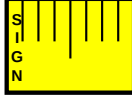
BIGINT

BIGINT data occupies 8 bytes. The first bit is the SIGN and the remaining 63 bits allow the storage of numbers from -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 or as (-2^{63} to $2^{63} - 1$).

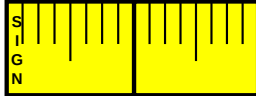
INTEGER Data Types

BYTEINT One byte – range -128 to +127

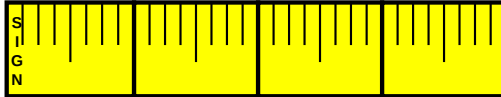
Non-ANSI



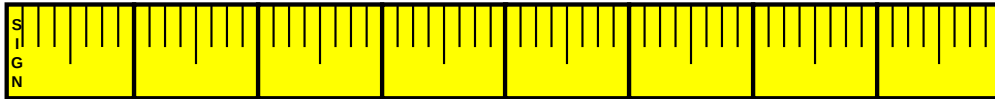
SMALLINT Two bytes – range -32,768 to +32,767



INTEGER Four bytes – range -2,147,483,648 to +2,147,483,647



BIGINT Eight bytes – range -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807



DECIMAL and FLOAT Data Types

Decimal and Numeric

The DECIMAL and NUMERIC data types represent a decimal number of n digits, with m of these digits to the right of the decimal point. The DECIMAL data type is synonymous with NUMERIC and can be abbreviated as DEC.

The specification of a DECIMAL data type is: `DECIMAL [(n[, m])]`

The value of n prior to V2R6.2 has a maximum value of 18. Starting with V2R6.2, value of n can be as large as 38. Values of n between 19 and 38 require 16 bytes of storage.

The Decimal and Numeric data types can take up 1, 2, 4, 8, or 16 bytes of space. The space required depends on the number of digits and is shown in the table on the facing page. Decimal values are stored in scaled binary.

Decimal numbers are scaled by the power of ten equal to the number of fractional digits. The number is stored as a two's complement binary number in 1, 2, 4, 8, or 16 bytes. The number of bytes used for a decimal value depends on the total number of digits in that value.

Float, Real, and Double Precision

The FLOAT, REAL, and DOUBLE PRECISION data types represent a value in sign/magnitude form. These data types represents values that range from 2×10^{-308} to 2×10^{308} .

The mantissa sign is in the most significant bit position; the exponent sign is a part of the exponent field (excess-1024 notation, in which $(1024 - \text{exponent}) = \text{sign}$).

Floating point values are stored and manipulated internally in IEEE floating point format. Floating point values are stored with the least significant byte first, with one bit for the mantissa sign, 11 bits for the exponent, and 52 bits for the mantissa. Eight bytes are used to hold a floating point value. Negative numbers differ from positive numbers of the same magnitude only in the sign bit. Float, Real, and Double Precision data require 8 bytes of storage space.

DECIMAL and FLOAT Data Types

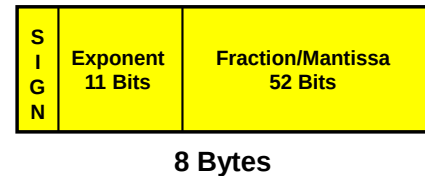
DECIMAL and NUMERIC

DECIMAL [(n[,m])]	Number of Digits	Number of Bytes	
n = 1 – 38	1 to 2	1 byte	
m = 0 - n	3 to 4	2 bytes	
Default = (5,0)	5 to 9	4 bytes	
Stored in scaled binary	10 to 18	8 bytes	
	19 to 38	16 bytes	

FLOAT, REAL, and DOUBLE PRECISION

Notes:

- Range is $2 * 10^{-308}$ to $2 * 10^{+308}$
- 15 significant decimal digit precision.
- Manipulated in IEEE floating point format.
- Corresponds to, but is not identical to, IBM normalized 64 bit floating point.



NUMBER Data Type

NUMBER is a floating point type and, therefore, an approximate data type with Teradata.

If you run a query that includes a NUMBER (or another floating point) value on a database from other vendors and then run the same query in Teradata, the result may be different because the order in which each database evaluates the expression may be different.

NUMBER(n) is equivalent to NUMBER(n,0).

XML Data Type (14.10)

Teradata provides the XML data type for XML data. You can use it in the same way as other SQL data types supported by Teradata. Users can store XML documents into columns of XML type.

Using the XML type, you can store XML content in Teradata in a compact binary form that preserves the information set of the XML document, including the hierarchy information and type information derived from XML validation. The document identity is preserved as opposed to XML shredding, which only extracts values out of the XML document.

The XML type stores XML values up to 2GB in size and supports XML processing functionality including:

- XSD validation
- XPath and XQuery-based queries
- XSLT transformations of XML content

While a common use of the XML type instance is to represent a single XML document, it can also represent a sequence of items where an item is an element, attribute or even an atomic value.

It can be used for XML values such as:

- Fragments of XML documents, such as nodes like elements, attributes, or text
- Values of XML schema predefined simple types, such as string or integer

NUMBER Data Type

NUMBER or NUMBER(*)
NUMBER(n) or NUMBER(n,m)

Characteristics

- **Teradata 14.0 feature – this is effectively a variable length numeric data type.**
 - Greater efficiency in storing numeric data because the number of bytes to store the data can vary from 0-18 bytes depending on the value stored.
 - NUMBER is effectively stored as a variable length number. 2 or 4 bytes are used in the column offset array to represent the starting location of this column in the internal data row.
- Range is $\pm 1\text{E-}130$ to $9.99\ldots9\text{E}125$
- More flexibility, range, and precision for numeric data.
 - More flexibility in defining numeric columns by providing the ability to change the precision or scale of existing NUMBER columns in tables without modifying the data rows.
 - More flexibility in computation compared with the DECIMAL data type because the result is not limited by the precision or scale of the input.
 - Greater range than the DECIMAL data type.
 - Greater accuracy than the FLOAT data type because NUMBER has greater guaranteed precision. NUMBER can represent common decimals exactly.
- Increased compatibility with other databases, which include a similar NUMBER data type.

DATE and TIME Data Types

DATE, TIME, and TIMESTAMP are also SQL functions. CURRENT_DATE, CURRENT_TIME, and CURRENT_TIMESTAMP represent values.

The following data types for the ARF (older Aligned Row Format) have the following length requirements and also require a 2-byte boundary within the row. These lengths ONLY apply to the ARF systems.

- 4 bytes - DATE
- 6 bytes - TIME
- 8 bytes - TIME WITH TIME ZONE
- 10 bytes - TIMESTAMP
- 12 bytes - TIMESTAMP WITH TIME ZONE

TIME

The format for the TIME data type is: **hh:mi:ss[.sssss]**. The internal storage format is a string of bytes:

```
SECOND (DECIMAL(8,6))
MINUTE (BYTEINT)
HOUR (BYTEINT)
```

Example:

```
CREATE SET TABLE table5
  (col1 DATE,
   col2 TIME(6),
   col3 TIME(6) WITH TIME ZONE,
   col4 TIMESTAMP(6),
   col5 TIMESTAMP(6) WITH TIME ZONE)
PRIMARY INDEX ( col1 );

INSERT INTO table5 VALUES
  (CURRENT_DATE,
   CURRENT_TIME,
   TIME '11:27:00-04:00',
   CURRENT_TIMESTAMP,
   TIMESTAMP '2017-03-18 11:27:00-04:00');
```

```
SELECT * FROM table5;
```

```
*** Query completed. One row found. 5 columns returned.
*** Total elapsed time was 1 second.
```

```
col1 17/03/18
col2 13:19:41.000000
col3 11:27:00.000000-05:00
col4 2017-03-18 13:19:41.730000
col5 2017-03-18 11:27:00.000000-05:00
```


DATE and TIME Data Types

DATE (4 Bytes)



$((YYYY - 1900)) * 10000 + (MM * 100) + DD$

TIME (6 Bytes)



hh:mi:ss.ssssss

**TIME WITH
TIME ZONE** (8 Bytes)



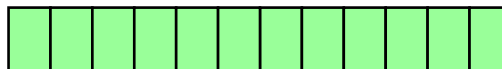
hh:mi:ss.ssssss +/- hh:mi

TIMESTAMP (10 Bytes)



DATE + TIME

**TIMESTAMP WITH
TIME ZONE** (12 Bytes)



DATE + TIME + ZONE

CHARACTER Data Types

There are three kinds of Character data types: **CHAR**, **VARCHAR**, and **LONG VARCHAR**. Character data can require from 1 to 64000 bytes of space.

Character data (for the Latin character set) is stored in 8-bit ASCII format. Conversion to and from other host formats is performed by the parsing engine on input and the AMPs upon output. Sorting and comparisons are always done in the collating sequence of the user's host system.

- CHAR is fixed length Character data. CHAR (n) always takes n bytes unless it is compressed.
- The VARCHAR (n) data type represents a variable length character string of length n. The maximum value for n is 64000 characters. Synonyms for VARCHAR(n) are CHAR VARYING [(n)], and CHARACTER VARYING [(n)]
- LONG VARCHAR is equivalent to VARCHAR (64000).

Note: The Unicode and Graphic character sets use 16-bit characters, thus allowing 32K logical characters for 64K physical bytes.

An uncompressed VARCHAR column always has a two or four-byte offset associated with it, whereas a multi-value compressed column can have its length stored in one or two bytes.

Considerations when a VARCHAR is Compressed

When a VARCHAR column is compressed (either MVC or ALC), Teradata stores the column data in the row as a length and data value pair.

- If the length of the column data is ≤ 255 bytes, then the length is stored in 1 byte; otherwise it stores the length in 2 bytes.

All variable length compressible columns are treated as an extension of fixed length compressible columns except that uncompressed variable length columns store both a length and the actual column data.

There is an additional presence bit for an algorithmically-compressed column to indicate whether the column data is compressed or not. Teradata sets this bit only when data in column is compressed using algorithmic compression. Teradata uses the additional presence bit when the compress UDF returns data that is larger than the original column data.

When column data is compressed algorithmically, then Teradata stores it as length: data pairs interleaved with the other compressible columns in the table. When data is not compressed, or if column data is null, Teradata does not store a length, so there is no overhead in this case.

CHARACTER Data Types

CHARACTER

- Typically 1 byte per character – stored in 8 bit ASCII.
- Other Character Sets (e.g., UNICODE) may use multiple bytes per character.
- **Compressed (or ALC) VARCHAR columns have 1 or 2 bytes stored with data in length-value pairs and do not use the 2 or 4 byte column offset (shown below).**
- **How do you store Character data (for a column) that is greater than 64,000 bytes?**

Define a CLOB (Character Large Object)

CHAR (n) n = 1 - 64000
Fixed length character string



VARCHAR (n) n = 1 - 64000
Variable length character string



2 or 4 byte column offset identifies location in row

LONG VARCHAR
Equivalent to VARCHAR (64000)



2 or 4 byte column offset identifies location in row

Character Sets

Teradata supports five different character sets for use with the CHARACTER data type.

- LATIN - 8-bit LATIN server character
- UNICODE - 16-bit characters from the Unicode 4.1 standard.
- GRAPHIC - 16-bit characters supported for DB2 compatibility.
- KANJISJIS - intended for Japanese applications that rely on the KANJISJIS character set
- KANJI1 - this character set will be removed in a future release

The Unicode and Graphic character sets use 16-bit characters, thus allowing 32K logical characters for 64K physical bytes. Examples of GRAPHIC character sets are not shown on the facing page, but a brief description follows.

GRAPHIC[(n)] Data Type

The GRAPHIC data type represents a fixed-length; multi-byte character string of length n, where n is the length in logical characters. The maximum value of n is 32000.

The GRAPHIC data type is valid only for Chinese double-byte Hanzi and Japanese double-byte Hiragana character supported sites. If any other site attempts to use this data type, the system generates an error message

VARGRAPHIC(n) Data Type

The VARGRAPHIC data type represents a variable length string of length n, where n is the length in logical characters. The maximum value of n is 32000. There is no default; omitting the length specification results in an error. The VARGRAPHIC data type (without compression) is stored in the third section of a data row and requires a 2 or 4 byte column offset.

LONG VARGRAPHIC Data Type

The LONG VARGRAPHIC data type specifies the longest permissible variable length graphic string. It is equivalent to specifying a data type of VARGRAPHIC(32000). The LONG VARGRAPHIC data type (without compression) is stored in the third section of a row and requires a 2 or 4 byte column offset.

Graphic Data Validation and Storage

Graphic data must always contain an even number of bytes. Any attempt to insert data that results in an odd number of bytes generates an error. Graphic data is stored without translation; the multi-byte characters remain in the encoding of the session character set. Teradata validates graphic data against the range of hexadecimal values considered valid for the character set of the current session. Hexadecimal constants cannot be validated.

Character Sets

Teradata supports different character sets for use with CHARACTER SET. Examples of character sets are:

- LATIN** – 8-bit LATIN server character
- UNICODE** – 16-bit characters from the Unicode 4.1 standard.
- GRAPHIC** – 16-bit characters supported for DB2 compatibility.

Typically, when 16-bit international character sets are needed, the recommendation is to define these as UNICODE with a CHARACTER data type.

Example:

```
CREATE TABLE People
(
  LastName      VARCHAR(30) CHARACTER SET UNICODE,
  FirstName     CHAR(20)    CHARACTER SET UNICODE,
  :            :
);
```

For LastName and FirstName, 16-bits (2 bytes) are allocated for each character. For example, 40 bytes are allocated for FirstName.

BYTE Data Types

There are two kinds of Byte data types: **BYTE** and **VARBYTE**. They are suitable for various types of data, including graphical data in the form of digitized image information. You cannot compare Character data to Byte data.

Byte data is stored in host format and is never translated by Teradata. It is handled as if it were n-byte, unsigned binary integers.

- **BYTE** is a fixed length binary string.
- **VARBYTE** is a variable length binary string. Both **BYTE** and **VARBYTE** can have a maximum length (n) of from 1 to 64000 bytes.
- An uncompressed **VARBYTE** data type will have an associated column offset that is either 2 or 4 bytes in length.

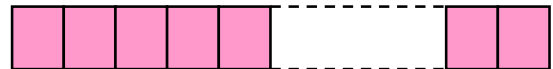
BYTE Data Types

BYTE

- Stored in binary (host format) and is not translated by Teradata.
- Handled as if they are n-byte, unsigned binary integers
- Suitable for digitized image information (small photo, signature, etc.) that is less than 64,000 bytes.
- **Compressed (or ALC) VARBYTE columns have 1 or 2 bytes stored with data in length-value pairs and do not use the 2 or 4 byte column offset (shown below).**
- **How do you store Binary data (for a column) that is greater than 64,000 bytes?**

Define a BLOB (Binary Large Object)

BYTE (n) n = 1 - 64000
Fixed length binary string



VARBYTE (n) n = 1 - 64000
Variable length binary string



2 or 4 byte column offset identifies location in row

Large Object Data Types

For both Binary and Character Large Object (LOB) data types, the base table data row only contains an OID (Object Identifier) to the subtable that actually contains the LOB data.

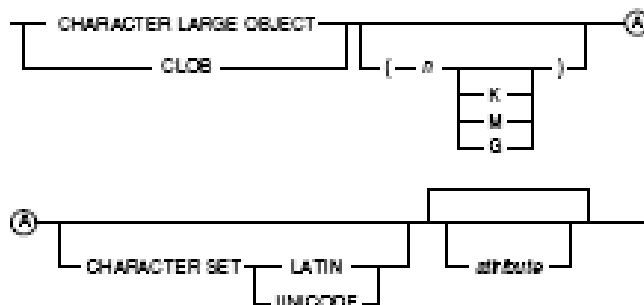
BLOB (n) Data Type

The BLOB data type represents a large binary string of raw bytes. A binary large object (BLOB) column can store binary objects, such as graphics, video clips, files, and documents.



CLOB (n) Data Type

The CLOB data type represents a large character string. A character large object (CLOB) column can store character data, such as simple text, HTML, or XML documents.



For both data types:

n = the number of bytes to allocate for the LOB column. The maximum number of bytes is 2097088000, which is the default if n is not specified.

K = that n is specified in kilobytes (KB). When K is specified, n cannot exceed 2047937.

M = that n is specified in megabytes (MB). When M is specified, n cannot exceed 1999.

G = that n is specified in gigabytes (GB). When G is specified, n must be 1.

Large Object Data Types

Binary or Character Large Object (LOB)

BLOB – a large binary string of raw bytes. A BLOB column can store binary objects, such as graphics, video clips, files, and documents.

CLOB – a large character string. A CLOB column can store character data, such as simple text, HTML, or XML documents.

BLOB (n) n = 1 – 2097088000 bytes (approx. 2 GB)
CLOB (n) n = 1 – 2097088000 bytes (approx. 2 GB)

The data row contains a 2 or 4 byte column offset and a 37-byte OID (Object Identifier) for each LOB. LOB data is stored in a separate subtable in a series of ~64 KB rows.

- The LOB data is stored on the same AMP as the corresponding data row.
- Maximum of 32 Large Objects in a table. Each LOB is stored in its own subtable.

Base Subtable
(Subtable ID 1024)

2|4

37- byte OID

2 or 4 byte column offset
identifies location of OID in row.

LOB Subtable
(Subtable ID 1792)

LOB data

LOB data

Series of ~64 KB
rows to hold the
LOB data.

:

Variable Column Offsets

Variable column offset values indicate the starting location of a variable column as is shown on the facing page. They are only present in rows of tables where variable length columns have been defined.

Determine the length of a variable length column by subtracting its starting location from the next column's starting location. In the example on the facing page, column c2 is empty (of length 0) since column c3 starts in the same place (i.e., $75-75=0$).

The definition of variable length columns requires an additional 2 or 4 byte offset needed by the system to determine the length of the last variable column.

- 2 byte offsets – used for small rows (approximately <64KB)
- 4 byte offsets – used for large rows (> 64KB up to 1 MB)

The last offset actually represents what would be the first byte of the next variable length column if it existed.

Variable Column Offsets

Offset Array				Variable Length Data		
50	75	75	100		c1 data	c3 data
c1	c2	c3	x			
				5	7	1
				0	5	0
						0

- Only applies to variable length columns without compression defined (MVC or ALC).
- Offset values (2 or 4 bytes) indicate the starting location of a variable column.
- Determine the column length by subtracting its starting location from the next column's starting location.
- The definition of variable length columns also requires one additional 2 or 4 byte offset that locates the end of the final variable column.
 - Note: the last offset actually represents what would be the first byte of the next variable length column if it existed.
 - The OID (Object Identifier) for a large object is also stored as a variable column and uses 2 or 4 bytes in the offset array.
 - 4 byte offsets are only used for large rows (tables with a definition of rows > 64 KB)

Table Headers

When a table is initially created, Teradata creates a **table header on each AMP** for each table.

- A table header (internal structure) consists of 1 to 16 rows that formulate a special block on each AMP and it has a specific subtable ID. The header row contains all of the Data Dictionary information about a table, its columns and its indexes which saves the Parser from having to send this DD information to the AMPs for every request.
- The table header also maintains the physical row layout and contains an array of compressed values for each column that has MVC (Multi-value Compression) or VLC (Value List Compression).
- Starting with Teradata 16.00, a table header is at least 1 KB (2 sectors) in length. The number of columns, indexes, constraints, compressed values, etc. that is part of the table definition impacts the size of the table header. We will assume for sizing exercises that table headers are 4 KB in size.
- Before Teradata 16.00 (e.g., 15.10), if you created a table with only 2 columns and no compression, the table header was only 512 bytes (1 sector) in length. Even on previous Teradata releases, tables usually had more than 2 columns defined and the table header was at least 1024 bytes in length. With Teradata 15.10, if you created a table with 2 columns and a NUSI, then the table header will be 1024 bytes.
- Since the table header is one of the subtables of a user-defined table, the Teradata file system identifies the table header by its subtable ID which is 0. To maintain consistency with the entries in the Master and Cylinder indexes, the table header has a row hash value of x'00000001' and a uniqueness value of x'00000001'.

An example of a table header is shown on the facing page.

Table Headers

- A copy of the table header exists on every AMP.
 - Table header is a separate subtable (subtable id 0).
- Starting with Teradata 16.00, the minimum table header block size is 1 KB (2 sectors) per AMP.
 - The number of columns, secondary indexes, and compressed values impact table header size.
- A table header will be at least 1024 bytes and may commonly be 4 KB with compression.
- Compressed values are maintained in the table header.
 - Compressed values are maintained in an array in the table header.
 - Example: 127 compressed values @ 20 bytes for a column would increase the table header by 2.5 KB.
- Table headers can have a maximum size of 1 MB (internally can consist of up to sixteen 64 KB rows).
- The table header also covers all of its secondary index subtables.

Example of Table Header

STANDARD ROW HEADER	
LENGTH, ROW ID, PRESENCE/SPARE BYTES	
FIELD OFFSET BYTES (Fields 2-9)	
EXTRA OFFSET	
DATABASE AND TABLE NAMES	FIELD
DATABASE ID	
OTHER INTERNAL INFO CREATION DATE, PROTECTION, TYPE OF JOURNALING, JOURNAL ID, STRUCT VERSION, etc.	
INDEX DESCRIPTORS (36 BYTES * # INDEXES) PLUS 20 BYTES PER INDEX COLUMN	FLD. 2
ALWAYS NULL	FLD. 3
FASTLOAD & RESTORE INFORMATION USUALLY NULL	FLD. 4
BASE COLUMN INFO COUNT OF COLUMNS, LOCATION OF FIRST FIXED FIELD, NUMBER OF PRESENCE BITS, etc.	FIELD
COLUMN INFORMATION FOR EACH COLUMN 20 BYTES PER COLUMN (+ COMPRESS VALUE), DATA TYPE, OFFSET WITHIN ROW, NULLABLE/NOT NULLABLE, COMPRESS/NO COMPRESS, PRESENCE BIT LOCATION, etc.	
RESTARTABLE SORT INFORMATION USUALLY NULL	FLD. 6
REFERENTIAL INDEX INFORMATION (REF. INDEX DESCRIPTORS)	FLD. 7
LARGE OBJECT INFORMATION (LOB DESCRIPTORS)	FLD. 8
ALWAYS NULL	FLD. 9
ROW LENGTH or REFERENCE ARRAY POINTER	

Sizing Tables and Indexes

Teradata supports variable length rows and variable length blocks within a table. These features provide maximum flexibility and save many hours of balancing data types, row sizes and block sizes. Other systems support different fixed length block sizes, but the designer must choose a single block size for each file. Though these other systems may allow variable length records, they can be wasteful of block space.

When sizing tables and indexes, the most important variables are the size of the rows and the number of rows (and whether the table is Fallback or not).

Row size and number determine space requirements. Variable length rows and blocks make it more difficult to accurately estimate the space required for tables and their indexes.

Sizing Tables and Indexes

These features make accurate space estimates for tables and their indexes more difficult.

- These options support variable length rows:
 - COMPRESS
 - VARCHAR and LONG VARCHAR
 - VARBYTE and VARGRAPHIC
- Variable length blocks within a table are also supported.
- Compression features such as Algorithmic Level Compression (ALC) or Block level Compression (BLC).

If it is necessary to estimate the size of a table, there are multiple techniques that can be utilized.

- One technique is first determine a typical row length (including overhead) and then multiply the typical row length X number of rows and double for fallback.

How can one determine the typical row length?

- Load a portion of data and use the COLLECT DEMOGRAPHICS command.
- Load a portion of data and use the COLLECT SUMMARY STATISTICS command.

Sizing Techniques

The DBC.TableSizeV view provides actual table space information, but it is a total value for the data table and all of its subtables (e.g., Fallback, Secondary Indexes, etc.).

Techniques that can be used to help determine the size of data rows and specific index subtables include:

- For a new table, load a portion of the data (e.g., 1%) and use the DBC.TableSizeV view and the "Empirical Sizing" technique to estimate the size of a table and each index as it is created.

Subtract the size without the index from the size with the index.

- For an existing table, use the COLLECT DEMOGRAPHICS command to capture demographic information into a QCD (Query Capture Database).

Query the data demographics view to access actual row lengths and row counts. Multiply the average subtable row length x number of subtable rows to determine the size of an index subtable.

- For an existing table, use the system utility Ferret with the ShowBlocks command to determine the size of data and index subtables.

This requires access to system console utilities.
This technique is not discussed in this course.

Sizing Techniques

The following techniques can be used for tables that are compressed or non-compressed.

The **DBC.TablesizeV** view provides permanent space information about a table and all of its subtables.

- Sizing information is not provided for individual secondary index subtables.

Other techniques that can be used to help determine the size of data rows and index subtables include:

- For a new table, use the **DBC.TablesizeV** view and the "Empirical Sizing" technique to estimate the size of a table and each index as it is created.
 - Subtract the size without the index from the size with the index.
- For an existing table, use the **COLLECT DEMOGRAPHICS** command to capture demographic information into a QCD (Query Capture Database).
 - These demographics provide actual row lengths and row counts.
 - Multiply the average subtable row length x number of subtable rows to determine the size of an index subtable.

To estimate the size of index subtables, the reference manuals have estimation formulas or the "Empirical Sizing" or **COLLECT DEMOGRAPHICS** techniques can be utilized.

Empirical Sizing

The most accurate way to size a production table and its indexes is **Empirical Sizing**. This method consists of loading a portion of the table, measuring how much space it takes, and extrapolating the total space required for the table from this information.

The steps involved in Empirical Sizing are shown at the top of the facing page.

The example shows the SQL necessary to query the DD.

- In Step 2, SUM(CurrentPerm) gives you the space, which is required to store the portion of the table itself.
- In Step 4, SUM(CurrentPerm) gives you the amount of space required to store those table rows plus the index which was defined in Step 3. Subtract the results to compute the size of the index.
- Running the SQL statement in Step 2 without the AND clause will give you the amount of space for the entire database, not just a single table.

CurrentPerm is always expressed in whole sectors, rounded up. If only a 10-byte row was in Perm Space, CurrentPerm would be reported as 512 bytes (1 sector).

NOTE: DATABASE is a key word and represents your current database.

Empirical Sizing

One technique to size a production table, including indexes is:

1. Load a known percentage of rows onto the system.
2. Query the DD through the view [DBC.TableSizeV](#).
3. Create one index.
4. Query the DD through the view [DBC.TableSizeV](#).
5. Repeat steps 3 and 4 as necessary.
6. Multiply the results to determine the production size.

Example:

Step 1 Load some percentage of a table onto a system (maybe 1% for test system).

Step 2

```
SELECT SUM(CurrentPerm) FROM DBC.TablesizeV
WHERE DatabaseName = DATABASE
AND TableName = 'Claim_RP';
```

Sum(CurrentPerm)
4,026,352,640

Note: The same query without the SUM keyword returns per/AMP figures which reveal distribution efficiency.

Step 3

```
CREATE UNIQUE INDEX (claimid) ON Claim_RP;
```

Step 4

```
SELECT SUM(CurrentPerm) FROM DBC.TablesizeV
WHERE DatabaseName = DATABASE
AND TableName = 'Claim_RP';
```

Sum(CurrentPerm)
5,107,091,968

Therefore, index size is:

5,107,091,968
- 4,026,352,640
1,080,739,328

If the sample data was 1%, then the index size would be 108 GB.

Collect Demographics Command

The COLLECT DEMOGRAPHICS command collects various table demographics and writes the data to the DataDemographics table of a user-defined QCD database. These demographics are primarily used for subsequent analysis tools such as the Teradata Index Wizard.

You can also query the DataDemographics table in the user-defined QCD via a view named DataDemographicsView.

You must have the following privileges to execute COLLECT DEMOGRAPHICS:

- DELETE on the DataDemographics table in *QCD_name*.
- INSERT on the DataDemographics table in *QCD_name* or INSERT on the *QCD_name database*.
- SELECT on the specified tables or containing databases.

Options: WITH NO INDEX is used to exclude index subtable demographics from the collection and collect only primary data. ALL (data and index demographics is the default).

COLLECT DEMOGRAPHICS determines and writes the following information for each specified table on an AMP-by-AMP basis into the DataDemographics table of the specified QCD (Query Capture Database):

- DatabaseName and TableName
- Subtable type and Subtable ID
- Cardinality and Average row length
- System-related information (Machine Name, Collect Time, etc.)

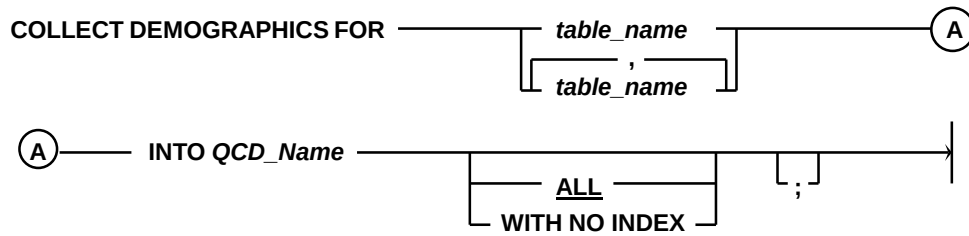
Demographics captured by COLLECT DEMOGRAPHICS are not deleted when you perform associated DROP actions on the subject table and must be deleted explicitly.

- COLLECT DEMOGRAPHICS does not capture information for the QCD table TableStatistics. The COLLECT STATISTICS (QCD form) is needed to capture statistics for the data analysis tools (e.g., Teradata Index Wizard).
- These are not the same statistics that are used by the optimizer. Statistics collected by this statement are used for index analysis and validation tasks performed by data analysis tools such as the Teradata Index Wizard: these statistics are *not* used by the Optimizer to process queries.

```
COLLECT STATISTICS FOR SAMPLE 100 PERCENT INTO QCD
ON Orders_RP COLUMN orderid;
```

If you collect data demographics with Visual Explain (INSERT EXPLAIN ... WITH DEMOGRAPHICS), when you delete the relevant query plans, then the data demographics are also automatically deleted.

COLLECT DEMOGRAPHICS Command



Notes:

- Collects various table demographics and writes the data to the DataDemographics table of a user-defined QCD database.
- `COLLECT DEMOGRAPHICS` determines and writes the following information for each specified table per AMP:
 - Subtable ID
 - Cardinality
 - Average row length
 - Subtable type
 - System-related information (Machine name, collected time, etc.)
- If an entry already exists in the DataDemographics table for the specified table, then it is updated with the currently collected data.
- This collection can also be invoked via the Visual Explain or Index Wizard utilities.

Collect Demographics Example

The facing page contains an example of using the COLLECT DEMOGRAPHICS command for a table named Orders_RP.

The table definition is:

```
CREATE SET TABLE TFACT.Claim_RP, NO FALLBACK ,
(
  claimid          INTEGER      NOT NULL,
  custid           INTEGER      NOT NULL,
  claimstatus      CHAR(1)      DEFAULT '0',
  claimcost        DECIMAL(9,2),
  claimdate        DATE         FORMAT 'YYYY-MM-DD' NOT NULL,
  claimpriority    SMALLINT     DEFAULT 0,
  agentname        CHAR(16),
  claimlocation    SMALLINT     DEFAULT 0,
  claimtype        SMALLINT     DEFAULT 0,
  claimcomment     CHAR(79)
                  DEFAULT 'Medical - Optional Comment completed by Agent')
PRIMARY INDEX (claimid)
PARTITION BY RANGE_N
  (claimdate BETWEEN DATE '2008-01-01' AND DATE '2017-12-31'
              EACH INTERVAL '1' DAY , NO RANGE)
UNIQUE INDEX (claimid)
INDEX (custid);
```

Note: If you name your index, the index name is captured in the DataDemographics table, but only for the first AMP (AMP 0). The index column names are not captured in this table.

If you haven't named your index, you will have to use the subtable ids to reference the index subtables.

Using the information from the DataDemographicsView, you can estimate the size of the various subtables.

Primary Data (1024)	= 30,000,000 x 134 =	4,020,000,000
USI subtable (1028)	= 30,000,836 x 36 =	1,080,000,000
NUSI subtable (1032)	= 91,166 x 3,317 =	<u>302,397,622</u>
Total		5,402,397,622

Using DBC.TablesizeV, the actual table size is 5,409,616,384

Note that this view does not include the fallback subtables. Therefore, you would need to double the values for Fallback protection.

COLLECT DEMOGRAPHICS Example

Example: The Claim_RP table is partitioned and has the following demographics:

- Fallback protected; 30,000,000 rows; typical row size is 134 bytes (includes overhead)
- Integer NUPI that is partitioned by claim date; block size = 127 Kbytes on a 30-AMP system
- Integer USI (on ClaimID)
- Integer NUSI (on CustID) – distinct values is 2886, typical rows per value is 10,395

```
COLLECT DEMOGRAPHICS FOR TFACT.Claim_RP into QCD;
```

Query the QCD.DataDemographicsView to determine average row length.

```
SELECT      Databasename, Tablename, SubtableType, SubtableID,
            SUM(RowCount), AVG(AvgRowSize)
FROM        QCD.DataDemographicsView
ORDER BY    SubtableID
GROUP BY    Databasename, Tablename, SubtableType, SubtableID;
```

<u>DatabaseName</u>	<u>TableName</u>	<u>SubTableType</u>	<u>SubTableID</u>	<u>Sum(RowCount)</u>	<u>Average(AvgRowSize)</u>
TFACT	Claim_RP	Data	1024	30,000,000	134
TFACT	Claim_RP	SecondaryIndex	1028	30,000,000	36
TFACT	Claim_RP	SecondaryIndex	1032	91,166	3,317

Calculation: An estimate for the first secondary index is $30,000,000 \times 36 = 1,080,000,000$ bytes.

Notes: The QCD.DataDemographics table does not include fallback subtables.

This table does include the names of named indexes, but not index column names.

Spool Space

Teradata uses spool space as temporary storage for result rows that are returned for user requests. Because spool space is a form of temporary space, it is frequently overlooked in capacity planning, yet it is critical to the operations of the database.

Therefore, when sizing a system, it is important to estimate the amount of Spool space required. Maximum Spool space requirements will vary with table size, use (type of application), and frequency of use. Spool falls into these categories:

- Intermediate – intermediate spool results are retained until no longer needed.
- Output – output spool results are the final information returned for a query
- Persistent – when Redrive protection is enabled, Teradata stores responses for sessions that participate in Redrive in non-fallback persistent spool tables. Persistent spools are not deleted following a Teradata restart.
- Volatile – the system uses volatile spool space for volatile tables.

It is a misconception that all unused space can be used for spool – only free cylinders are available for spool. The unused or fragmented space within permanent cylinders cannot be used for spool. This space is used by the file system for updates/insert/deletes requiring larger/new blocks. When blocks are expanded or split, the file system returns blocks to the free block list and this space is available within the cylinder for future write operations.

Large systems use more spool space to duplicate tables on each AMP to perform Product Joins. Spool uses its own cylinders and as more Spool space is needed, empty cylinders are used for Spool. Recall that the Spool limit for a User is specified in the CREATE USER statement and may be changed dynamically. Avoid copying or redistributing entire tables to Spool unless absolutely necessary (to keep from exceeding the Spool limit).

A user may run out of Spool space because of an incorrectly coded query, etc. and will receive a message that their SQL statement has been aborted. If the user exceeds their Spool space limit, they will receive the following error message.

2646 – No more spool space in username

If a user has an appropriate amount of spool space for their request and receives this message (2646), usually the problem is that the spool file is poorly distributed among the AMPs. This usually occurs in a join operation when the Optimizer redistributes the tables on a column that is not very unique.

If the AMP runs out of Spool space (insufficient available cylinders for Spool), the following message will be displayed. This message indicates that a request has exceeded the physical storage limits imposed by a given configuration without exceeding the logical limits imposed by the PERM or SPOOL space allocation for a given database/user.

2507 - Out of spool space on disk

Spool Space

- Maximum spool space needs vary with table size, use (type of application), and frequency of use.
- Large systems use more spool to duplicate tables on each AMP.
- **Cylinders not currently used for data may be used for spool.**
 - It is a misconception that all unused space can be used for spool – only free cylinders are available for spool. The unused space within permanent cylinders cannot be used for spool.
- The user's maximum spool space limit can be changed dynamically.
- Avoid unnecessary copying or redistribution of entire tables to spool.

As user concurrency and/or SQL complexity increases, add more SPOOL.

Running out of Spool Space

- If a user exceeds their Spool space limit, they will receive the following error message.
2646 – No more spool space in username
- If the AMP runs out of Spool space (insufficient available cylinders for Spool), the following message will be displayed.
2507 - Out of spool space on disk

Release of Spool

Intermediate Spool results are held until they are no longer needed. The EXPLAIN facility uses the term “Last Use” to designate the step after which the Spool is released.

Output (final) Spool results are held until one of the events listed on the facing page occurs. System Restarts cause all Spool results to be released.

- Since Master Indexes are kept in memory and never written to disk, they must be rebuilt from the Cylinder Indexes after every Restart.

Previously, this course stated that the Table ID contained information (Unique Value) which defined the type of table or file (data table, Permanent Journal, or Spool file).

- In rebuilding the Master Index after a Restart, the File System examines every Cylinder Index.
- If a Spool block is found on a cylinder, this means that the entire cylinder must contain only Spool and Free Blocks (since you cannot have both data and Spool on the same cylinder).
- All of the in-use space for that cylinder is put onto the Free Block List and the Cylinder Index is rewritten to indicate that the cylinder is empty. That cylinder is then put onto the Free Cylinder List in the Master Index.

Release of Spool

Intermediate Spool

Intermediate Spool results are held until the (LastUse) Explain step.

Output Spool

Output Spool results are held until:

- Last spool Response
- CLOSE cursor
- ERQ, Terminate function
- Session ends
- System is restarted
- BTEQ
- Preprocessor
- CLI
- Job Abort, timeout, logoff, etc.

System Restart – each AMP rebuilds its Master Index from its Cylinder Indexes.

The AMPs delete all spool files by moving them to the Free Cylinder List.

This costs only one I/O per spool cylinder.

System Sizing Exercise

The exercise on the facing page provides an estimate only. The intent of this exercise is to show that you have to account for more than just user data.

Obviously, customer requirements can vary greatly and the percentages for spool, work space, indexes, permanent journals, etc. will vary as well.

If not using Fallback for user table, multiply the amount of raw data by a factor of 3 or 4 (for 40% spool).

CDS – Customer Data Storage (or Store)

System Sizing Exercise

Some general guidelines for estimating system size:

- 25 – 40% of total space for Spool
- 10% of total space for DBC, WAL space (Transient Journal), DBQL Logs, etc.
- 20 - 50% of data size for indexes

- If using Fallback, multiply the amount of raw data by a factor of 5 or 6 (for 40% spool).
- Note: These factors will be smaller if the amount of required spool is only 25% and if aggressive compression is used (block level, etc.)

Example assumes a CDS of 300 TB with 67% compression and Fallback:

User raw data	100 TB
Estimate of Vdisk space needed	500 to 600 TB

Proof:

Estimate of Vdisk space	500 to 600 TB
- Spool (40%)	- 200 - 240 TB
- DBC, WAL Space, etc. (10%)	- 50 - 60 TB
	<u>250 to 300 TB</u>

User raw data	100 TB
20 - 50% for indexes	<u>20 to 50 TB</u>
Fallback (for data and indexes)	240 to 300 TB



A 24 node IntelliFlex™ 2.0 system with 54 AMPs/node and .474 TB of MaxPerm space/AMP would meet this requirement.

24 x 54 x .474 TB = 614 TB
MaxPerm space

Assumes 6.4 TB RI SDD disks with RAID 6.

Summary

The facing page summarizes the primary points to remember when sizing databases.

- To get accurate space estimates, you must start with accurate row counts and row sizes.
- When calculating the space necessary for a database, you must include all the components from the list.

Accurate row counts and sizes are needed to get good space estimates.

Database sizing includes:

Tables + Fallback +
Secondary Indexes + Fallback +
Reference Indexes + Fallback +
Join Indexes + Fallback +
Hash Indexes + Fallback +
Permanent Journal (dual or single) +
Stored Procedure space +
Spool space +
Temporary space +
WAL Space (Transient Journal, etc.)

Module 18: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 18: Review Questions

1. Which choice can be used with the COMPRESS option?
 - a. Identity column
 - b. Non-unique Primary Index
 - c. USI as PK with Standard RI
 - d. Non-unique Secondary Index
2. Which section of a row identifies the starting location of variable length column data and is present only if variable length columns without compression are declared?
 - a. Presence Bits
 - b. Column Offsets
 - c. VARCHAR Columns
 - d. Uncompressed Columns
3. How can you override the default that a column with a NULL value will require row space?
 - a. Use the NOT NULL option on the column as part of the CREATE TABLE statement.
 - b. Set the user's default so columns will default to COMPRESS when creating a table.
 - c. Use the COMPRESS option on the column as part of the CREATE TABLE statement.
 - d. Use the DEFAULT NULL option on the column as part of the CREATE TABLE statement.
4. What is the minimum space the table headers will take for a 6-column table on a 10 AMP system?
 - a. 1024 bytes
 - b. 4096 bytes
 - c. 5120 bytes
 - d. 10240 bytes
5. What DD view is used to get sizing information about tables? _____

Lab Exercise 18-1

The following SQL can be used to determine the size of a table.

SUM of Perm space using the DBC.TableSizeV view.

```
SELECT      TableName (CHAR(15)), SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'tablename'
GROUP BY    1
ORDER BY    1;
```

Remember that compression is case-sensitive. The clerk names have to be compressed as shown on the facing page OR you can paste the clerk names from below into the COMPRESS option on the clerk column of the CREATE TABLE DDL statement.

```
'April Rains', 'Auggie Sunshine', 'Dee Winter', 'Fred Cold',
'Jack Snow', 'June Summer', 'Justin Heat', 'Matt Winds',
'May Flowers', 'Norbert Thanks', 'Ossie Leaves', 'Steve Fall'
```

Lab Exercise 18-1

Lab Exercise 18-1

Purpose

In this lab, you will compress multiple values for a column in order to reduce Perm space.

What you need

Populated Orders table (previous exercise) with 350,000 rows.

Tasks

1. If necessary, use INSERT-SELECT to populate your Orders table from the DS.Orders table.

```
INSERT INTO Orders SELECT * FROM DS.Orders;
```

Using the DBC.TableSizeV view, what is the amount of Perm space used. Orders = _____

2. Create a new table, named "Orders_MVC", based on the Orders table except compress the following clerk names:

April Rains, Auggie Sunshine, Dee Winter, Fred Cold, Jack Snow, June Summer,
Justin Heat, Matt Winds, May Flowers, Norbert Thanks, Ossie Leaves, Steve Fall

Using INSERT-SELECT, populate your Orders_MVC table from DS.Orders table.

Using the DBC.TableSizeV view, what is the amount of Perm space used. Orders_MVC = _____

Lab Exercise 18-1 (cont.)

To turn BLC (Block Level Compression) off, you can use the following SQL:

```
SET QUERY_BAND = 'BLOCKCOMPRESSION=NO;' UPDATE FOR SESSION;
```

The following SQL can be used to determine the size of a table.

SUM of Perm space using the DBC.TableSizeV view.

```
SELECT      TableName (CHAR(15)), SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'tablename'
GROUP BY    1
ORDER BY    1;
```

To turn BLC (Block Level Compression) back on, you can use the following SQL:

```
SET QUERY_BAND = 'BLOCKCOMPRESSION=YES;' UPDATE FOR SESSION;
```

Lab Exercise 18-1 (cont.)

3. Within your session, use QUERYBAND to turn **off** block level compression.

```
SET QUERY_BAND = 'BLOCKCOMPRESSION=NO;' UPDATE FOR SESSION;
```

4. Delete the rows in your Orders and Orders_MVC tables.

5. Populate your Orders and Orders_MVC tables using the DS.Orders table.

6. Using DBC.TableSizeV, what is the amount of Perm space used. Orders = _____

7. Using DBC.TableSizeV, what is the amount of Perm space used. Orders_MVC = _____

8. Within your session, use QUERYBAND to turn **on** block level compression.

```
SET QUERY_BAND = 'BLOCKCOMPRESSION=YES;' UPDATE FOR SESSION;
```

9. Delete the rows in your Orders and Orders_MVC tables.

10. Populate your Orders and Orders_MVC tables using the DS.Orders table.

Lab Exercise 18-2

This lab exercise provides the opportunity to use the Teradata 16.10 MAPS feature.

The Nation_C table was created as follows:

```
CREATE SET TABLE PD.Nation_C, Fallback,  
  MAP = TD_MAP1  
(  
  NationKey INTEGER,  
  NationName VARCHAR(25) CHARACTER SET LATIN NOT CASESPECIFIC,  
  RegionKey INTEGER)  
UNIQUE PRIMARY INDEX (NationKey);
```

The Region_C table was created as follows:

```
CREATE SET TABLE PD.Region_C, Fallback,  
  MAP = TD_MAP1  
(  
  RegionKey INTEGER,  
  RegionName VARCHAR(25) CHARACTER SET LATIN NOT CASESPECIFIC)  
UNIQUE PRIMARY INDEX (RegionKey);
```

To specify a sparse MAP and optionally the COLOCATE option, the following SQL can be used.

```
CREATE TABLE tablename, Fallback,  
  MAP = TD_1AmpSparseMap_1Node COLOCATE USING Co_NationRegion  
( column1      INTEGER,  
   column2      VARCHAR(25),  
   column3      INTEGER)  
UNIQUE PRIMARY INDEX (column1);
```

Lab Exercise 18-2

Lab Exercise 18-2

Purpose

In this lab, you will populate tables using the sparse MAPS capability.

What you need

Populated PD.Nation_C and PD.Region_C tables.

Tasks

1. Create two Contiguous tables Nation_C and Region_C from PD.Nation_C and PD.Region_C. (Note: these tables do not have all of the nations/regions, just a small subset of data)

```
CREATE TABLE Region_C AS PD.Region_C WITH DATA;  
CREATE TABLE Nation_C AS PD.Nation_C WITH DATA;
```

2. Find the system single AMP (1) Sparse MAP by using the view DBC.MapsV.

```
SELECT MapName FROM DBC.MapsV;
```

Name of Sparse Map for 1 AMP – _____

3. Use SHOW TABLE and create new Nation_S and Region_S sparse tables specifying the 1 AMP sparse MAP name and use a COLOCATE value of Co_NationRegion for both tables (i.e., COLOCATE USING Co_NationRegion).

Lab Exercise 18-2 (cont.)

Use DBC.TableSizeV to determine which AMPs and the number of bytes that a table is currently using.

SUM of Perm space at the AMP level using the DBC.TableSizeV view.

```
SELECT DatabaseName, TableName, Vproc, SUM(CurrentPerm)
FROM DBC.TablesizeV
WHERE DatabaseName = DATABASE
AND TableName IN ('Nation_S', 'Region_S')
GROUP BY 1,2,3
ORDER BY 1,2,3;
```

SUM of Perm space for all AMPs using the DBC.TableSizeV view.

```
SELECT DatabaseName, TableName, SUM(CurrentPerm)
FROM DBC.TablesizeV
WHERE DatabaseName = DATABASE
AND TableName IN ( 'Nation_S', 'Region_S',
                  'Nation_C', 'Region_C' )

GROUP BY 1,2
ORDER BY 1,2;
```

To view the AMPs associated with your tables assigned to a sparse map, use the following view.

```
SELECT * FROM DBC.SparseMapAmpsV where DatabaseName = DATABASE;
```

To view objects that are associated with sparse maps and COLOCATE value, use the following view.

```
SELECT * FROM DBC.ObjectsInSparseMapsV;
```


Lab Exercise 18-2 (cont.)

4. Populate the two Sparse tables with INSERT-SELECT from the two contiguous tables.
5. Use the DBC.TableSizeV and determine which 2 AMPs the sparse tables are using.
Why are 2 AMPs used per sparse table? _____
6. Compare the sizes of Nation_S and Region_S to the Nation_C and Region_C tables.
Size of Nation_S = _____ Size of Nation_C = _____
Size of Region_S = _____ Size of Region_C = _____
7. Use the DBC.SparseMapAmpsV to determine which AMP has the primary data of your Sparse tables. AMP # = _____

```
SELECT * FROM DBC.SparseMapAmpsV where DatabaseName = DATABASE;
```

8. Execute the following EXPLAIN of a join of the Contiguous tables.
Is the data redistributed for the join? Yes/No _____

```
EXPLAIN  SELECT      NationName, RegionName
          FROM        Nation_C
          INNER JOIN   Region_C
          ON           Nation_C.RegionKey = Region_C.RegionKey ;
```

Execute the same EXPLAIN using the Nation_S and Region_S sparse tables. Is the data redistributed for the join? Yes/No _____

9. Optional: Create a new Nation_S2 table and a new Region_S2 table **without** using the COLOCATE option. Use the DBC.SparseMapAmpsV to determine which AMP has your Sparse tables. Are the two tables on the same AMP? Yes/No _____

Notes



Optimizer and Collecting Statistics

After completing this module, you will be able to:

- Explain how the Optimizer acquires statistics.
- Describe random AMP sampling.
- State a method for viewing statistics.
- Utilize both the standard and index formats to collect statistics.

Teradata Proprietary and Confidential

Notes

Table of Contents

SQL Parser Component Overview	19-4
Teradata Optimizer.....	19-6
Optimizer Statistics	19-8
Optimizer's Search for Statistics.....	19-10
Optimizer – Random AMP Samples.....	19-12
Random AMP Sampling – How it Works	19-14
Example of an Optimizer Estimate without Collected Statistics	19-16
Statistics	19-18
Statistics Data – What is Collected?	19-20
Statistics Data – What is Collected? (cont.).....	19-22
Table Level Statistics Data.....	19-24
Table Level Summary Statistics	19-26
Column Detail Statistics.....	19-28
Statistics Example	19-30
COLLECT STATISTICS Command.....	19-32
COLLECT STATISTICS Command.....	19-34
Collecting Statistics Example	19-36
Refresh or Re-Collect Statistics	19-38
Viewing Statistics	19-40
Collect Statistics Using Sample	19-42
Collecting Statistics (Additional Options)	19-44
Optimizer's use of Statistics with Uneven NUSI.....	19-46
Collecting Statistics on PARTITION.....	19-48
Copying STATISTICS.....	19-50
Statistics Extrapolation.....	19-52
Additional Features and Considerations	19-54
Teradata Re-Collect Enhancements	19-56
Teradata AutoStats features	19-58
Statistics Summary.....	19-60
Module 19: Review Questions	19-62

SQL Parser Component Overview

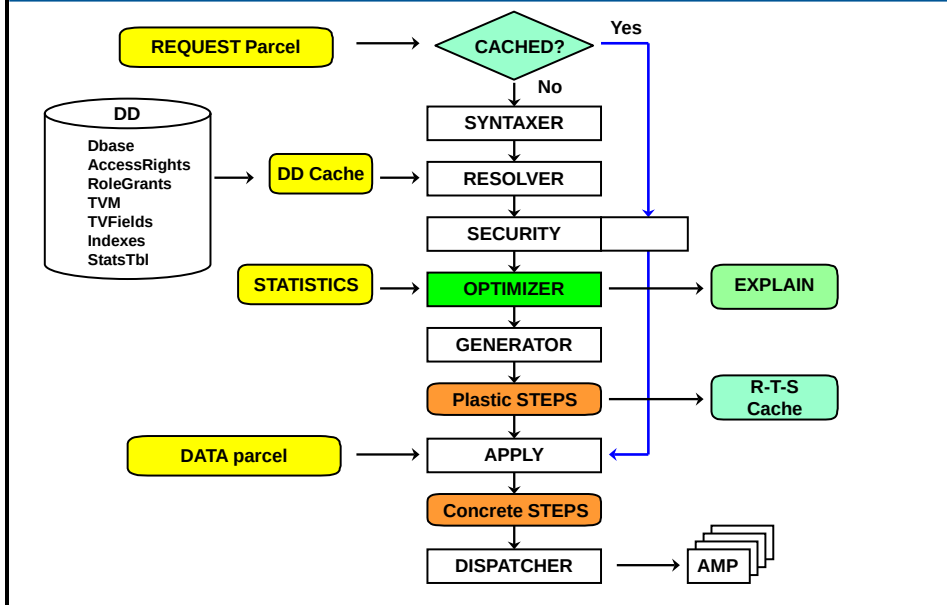
The flowchart on the facing page provides an overview of the SQL parser. As you can see, it is composed of multiple sections: **Syntaxer**, **Resolver**, **Security**, **Optimizer**, **Generator**, **Apply**, and **Dispatcher**.

When the parser sees a Request parcel it checks to see if it has parsed and cached the execution steps for it. If the answer is NO, then the Request must pass through all the sections of the parser as follows:

- The **Syntaxer** checks the Request for valid syntax.
- The **Resolver** breaks down Views and Macros into their underlying table references through use of DD information.
- **Security** determines whether the Requesting User ID has the necessary permissions.
- The **Optimizer** chooses the execution plan.
- The **Generator** creates the steps for execution.
- **Apply** binds the data values into the steps. (This phase of the Parser is also known as GncApply.) Apply acts upon Plastic Steps from the Generator and produces Concrete Steps by binding in data values from the DATA parcel. Apply adds data values one at a time. The Concrete Steps then go to the Dispatcher, which sends them out to the AMPs.
- **Dispatcher** is responsible for sending the execution steps out to the AMPs. The AMPs respond back to the Dispatcher which is responsible for sending the server response.

Note: If the steps in the Request parcel are in cache, the Request passes directly to Apply (after a check by Security). This is illustrated on the facing page by the YES path from the CACHED? decision box.

SQL Parser Component Overview



Teradata Optimizer

Teradata's optimizer is cost based and will actually generate several plans and choose the best plan based on analyzing the lowest cost numbers. This is critical to performance in supporting mixed workloads. A cost based optimizer requires statistical information about the data as well as being aware of the machine resources (CPU, disk, memory, etc.).

The other type of optimizer is rules based. This is good for transactional workloads where the queries are well known and the data has been physically structured to support this known workload. It is based on a set of rules that have been defined and only performs well in a highly structured transactional environment.

The quality of the optimizer is critical in Data Warehouse environments. Data Warehouse environments require a cost based optimizer to make good decisions regarding join plans for complex DSS queries. Join techniques that are required for DSS are significantly more sophisticated than what is required in an OLTP environment. Since DSS environments are designed to support ad hoc queries, we must rely on a Cost Based Optimizer to determine the optimal plan.

Query optimizers do *not* do either of the following things:

- Guarantee that the access and join plans generated are infallibly the best plans possible. A query optimizer always generates several optimal plans based on the population and environmental demographics it has to work with and the quality of code for the query it receives, then selects the best of the generated plan set to use to respond to the DML statement.
- You should not assume that any query optimizer ever produces absolutely *the* best query plan possible to support a given DML statement.

A comparison of cost-based and rules-based optimizers follows:

Cost Based Optimizer (Teradata)

- Best for complex DSS queries
- Requires statistics on tables and individual columns within tables
- Plans are independent of table ordering in FROM clause and predicate ordering in the WHERE clause
- Should understand available resources within the system (e.g., CPU, Memory, etc.)
- Generates several plans and chooses best plan based on smallest cost factors

Rules Based Optimizer

- Used in OLTP environments with structured and known access paths
- Rules are defined for access paths and types of SQL statements (e.g., simple, join, complex, compound, etc.)
- Plans are chosen based on access paths available, the ranks of these access paths and type of query
- Not a good choice in DSS environments
- Users can provide hints to help influence a rules based optimizer.

Teradata uses a “**cost-based optimizer**”.

- The Optimizer evaluates the “costs” of all reasonable execution plans and the best choice is used.
 - Effectively finds the optimal plan
 - Optimizes resource utilization - maximizes system throughput
- What does the “optimizer” optimize?
 - **Access Path** (Use index, table scan, dynamic bitmap, etc.)
 - **Join Method** (How tables are joined – merge join, product join, hash join, nested join)
 - **Join Geography** (How rows are relocated – redistribute, duplicate, AMP local, etc.)
 - **Join Order** (Sequence of table joins)
- Parallelism is automatic
 - Aware of table geography and data skew
- Parallelism is unconditional
 - No operation turns off parallelism

Optimizer Statistics

As we have seen, the Optimizer plans an execution strategy for every SQL query submitted to it. We have also seen that the execution strategy for any query may be subject to change depending on various factors. For the Optimizer to consistently choose the optimum strategy, it must be provided with reliable, complete, and current demographic information regarding all of these factors.

The best way to assure that the Optimizer has all the information it needs to generate optimum execution strategies is to **COLLECT STATISTICS**.

It is interesting to note that the Parser needs the same information to properly plan a query, as you need in properly choosing indexes.

Optimizer and Block-Level Compression

The optimizer is not sensitive to block-level compression. Neither will you see any additional steps added to the explain text when a query accesses a compressed table, as the decompression processes are performed transparently at the file system level. The optimizer does not take into account the extra time or CPU required for decompression when estimated processing times are established. However, the average row size that is calculated during random AMP sampling will be different for compressed tables. When random AMP sampling is performed, one or more cylinder indexes are read. While the cylinder indexes are never compressed, the information they carry is based on physical characteristics of the underlying data.

Even if full statistics have been collected, random AMP sampling is relied upon for determining the average row size as input to query optimization. Because the table's row size is based on the compressed image of the data, estimated processing times, which are influenced by row size, may be slightly less in queries accessing compressed tables. While it is not expected that this discrepancy will be large enough to cause the optimizer to make different decisions in most cases, the row size under-estimation with compression might lead to some query plan changes when block level compression is implemented.

Optimizer Statistics

The optimizer needs information to create the best execution plan for a query.

Environment information:

- Number of nodes
- Number of AMPs
- Number and type of CPUs
- Disk Array information
- Interconnect (BYNET) information
- Amount of memory available

Data Demographics:

- Number of rows in the table
- Row size
- Column demographics
 - Range of values in the table for the column
 - Number of rows per value
 - Number of NULLs for the column
- Index demographics

Are Statistics Collected?

If Yes – use collected statistics

If No,

- For indexed columns – use Random AMP Samples
- For non-indexed columns – use Heuristics (formulas)

Optimizer's Search for Statistics

When there are no statistics available to quantify the demographics of a table or an index, the Optimizer selects (by default) a single AMP to sample for statistics using an algorithm based on the table ID. By inference, these numbers are then assumed to represent the global statistics for the column or index.

Note that the statistics collected by a random AMP sample only apply to indexed columns. If you do not collect statistics on non-indexed columns, then the Optimizer uses various situation-specific heuristics to provide arbitrary estimates of cardinalities.

The process of the optimizer searches for statistics as follows:

First, random AMP samples are kept in the table header that resides in memory in dictionary cache. The optimizer first looks for the table header in the dictionary cache in order to extract the random AMP samples. If the table header is not in the cache, it is read from disk, where it can be found as a single row in subtable 0 of the base table. As part of the process of reading the table header from disk, random AMP samples are collected for that table and moved into a field in the table header when the table header is placed in the cache.

After the table header has been located and the random AMP samples have been accessed, a routine that looks for collected statistics is called. This routine will be called once for each index and/or column on a table for which the optimizer would like statistics. As part of that routine, the dictionary cache is searched for the relevant histogram.

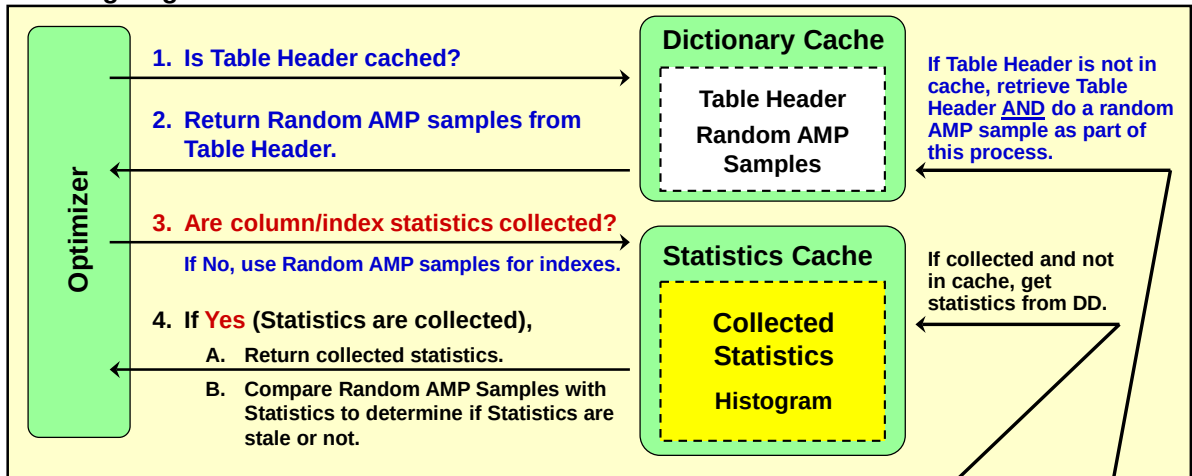
If statistics have been collected for the column/index of interest but the statistics are not cached, an express request is issued that retrieves the collected statistics from the data dictionary tables on disk. If statistics for that column or index are found in the data dictionary, the histogram that contains the statistical information is placed in the dictionary cache for use by other queries and is used by the optimizer for the current query.

If no collected statistics exist and the column is a primary or non-unique secondary index (NUSI), then the random AMP samples that are stored in the table header will be used. Non-indexed columns that have no statistics collected will not use random AMP samples, but will rely on static formulas to determine selectivity estimates.

Statistics are removed from the cache periodically to make sure that what is cached is reasonably current.

Optimizer's Search for Statistics

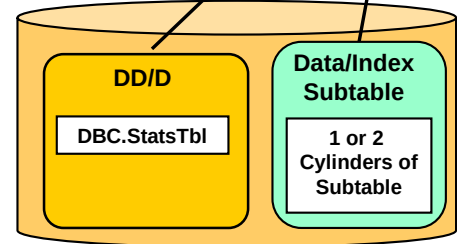
Parsing Engine



1 & 2. Get the random AMP statistics from table header in DD cache.

3. If statistics are collected for the index or column, get the statistics from cache or DD.

4. Random AMP sample row counts and collected statistics row counts are compared to determine if collected statistics are stale or if extrapolation is needed.



Optimizer – Random AMP Samples

The Optimizer does Random AMP Sampling with information it gathers from a random AMP. This AMP is selected based on the Table ID. Different tables will get their samples from different AMPs. This assures that no single AMP will be overloaded with Random AMP Sample requests.

Since this method uses only a single AMP for sampling purposes, it is sensitive to the evenness of the data distribution. Badly distributed data gives skewed samples that impact optimization by misleading the Optimizer into making improper choices.

The Optimizer will be more conservative in the choices it makes if it has to rely upon Random AMP Sampling.

DBSControl Options

When statistics have not been collected for an index, the Optimizer can obtain random samples from more than one AMP when generating row counts for a query plan.

This enhancement improves the row count, row size, and rows per value estimates for a given table. These are passed to the Optimizer, resulting in improved join plans, better query execution times, and better elapsed times.

Tables with heavily skewed data will benefit most from the improvements to random AMP sampling. One AMP sampling of a table with heavily skewed data may result in wrong estimates of the row count and row size information being passed to the Optimizer. With multiple AMP sampling, the Optimizer will receive better estimates with which to generate a query plan.

An internal parameter (#65) within DBSControl can be set by the Teradata Customer Engineer to specify the type of Random AMP Sampling that will be used in V2R6.

65. RandomAmpSampling – this field determines the number of AMPs to be sampled for getting the row estimates of a table. The valid values are D, L, M, N or A.

D - The default is one AMP sampling (D is the default unless changed.)

L - Maximum of two AMPs sampling

M - Maximum of five AMPs sampling

N - Node Level sampling - i.e., all the AMPs in a node would be sampled.

A - System Level sampling - i.e., all the AMPs in a system would be sampled.

Note that a higher number of AMPs sampled will provide better estimates, but can cause short running queries to run slower and long running queries to run faster.

Optimizer – Random AMP Samples

- A Random AMP sample only applies to indexed columns and table row counts.
 - row counts for the table are needed and statistics are not collected on PI.
 - indexed columns are used in the query and statistics do not exist for the indexes.
 - **Statistics have been collected, but are considered stale (e.g., 10% change).**
- By default, a single AMP is chosen for random AMP data sampling.
 - The AMP chosen is based on Table ID. Different tables will utilize different AMPs.
 - Badly distributed data gives skewed samples that impact optimization.
 - If a table (or index subtable) spans more than 1 cylinder, it will sample the first and the last cylinder. If it fits into 1 cylinder, it will only sample that one cylinder.
- Option – the Optimizer can obtain random samples from **more than one AMP** when generating row counts for a query plan.
 - Improves the row count, row size, and rows per value estimates for a given table.
 - Random AMP sampling is controlled via a DBS Control parameter.
 - D – Dynamic or one AMP sampling (D is the default unless changed)**
 - L – Low - two AMPs are sampled
 - M – Maximum - five AMPs are sampled
 - N – Node Level sampling - i.e., all the AMPs in a node are sampled
 - A – All or system level sampling - i.e., all the AMPs in a system are sampled

Random AMP Sampling – How it Works

When the Optimizer is not provided with COLLECTed STATISTICS for a table or indexed column, it will perform a Random AMP Sample in order to estimate table demographics.

The Optimizer uses the results of these calculations to provide it with the demographics it needs to make its choices in optimizing the query. If the results are skewed, the Optimizer may be misled and make the wrong choices; performance may not be optimal.

Random AMP samples are stored in the PE Data Dictionary (DD) cache for a maximum of four hours until they are purged by the routine DD four-hour purge job. Each time a table header is read from disk, the system collects a fresh random AMP sample of its statistics and caches the random AMP sample along with the table header.

The Parser is more aggressive with COLLECTed STATISTICS. Features such as NUSI Bit Mapping require COLLECTed STATISTICS.

Random AMP Sampling of USIs

Random AMP sampling assumes that the number of distinct values in a USI equals its cardinality, so it does not read the index subtable for USI equality conditions. The number of distinct values in the USI is assumed to be identical to the table cardinality taken from the random AMP sample on the primary index. Because equality conditions on a unique index return only one row by definition, the Optimizer always chooses the direct USI path without costing it or using statistics. However, if a USI will frequently be specified in non-equality predicates, such as range constraints, then you should collect statistics on it.

Random AMP Sampling of NUSIs

Random AMP sampling for NUSIs is very efficient. The system reads the cylinder index that supports the index subtable rows on the sampled AMP and determines the number of rows on that cylinder. Except for situations where the NUSI is very non-unique, there is one subtable row for each distinct value in the NUSI. Using that information, the sampling process assumes that each subtable row it finds translates to one index value.

Sampling Efficiency for Non-indexed Predicate Columns

The heuristic is to estimate the cardinality to be 10% of the table rows. With two selection criteria, the Optimizer assumes that about 7.5% ($10\% \times .75$) of the rows will be returned.

For additional equality criteria, each is 75% of the previous level. For example, for 3 columns, the Optimizer assumes 5.2% ($7.5\% \times .75$) of the rows will be returned.

For example, if there is one selection criteria column in the WHERE clause for a given table and the column is in an equality condition, and no stats have been collected on it, the optimizer will assume that it is selecting 10% of the table's rows. If there are two columns each in an equality condition, no stats, the optimizer will assume that 7.5% of the table's rows will be selected by their combination. For LIKE, the heuristic is to estimate 90% of the table rows. Inequality (< or >) estimates 33%. This can lead to poor plans.

Random AMP Sampling – How it Works

- For a table row count estimate, read 1 or 2 cylinders from 1 (or more) AMPs.
 - Calculate the approximate number of rows in the table:
 - Average Number of Rows per Block in the sampled Cylinder(s)
 - x Number of Data Blocks in the sampled Cylinder(s)
 - x Number of Cylinders with data for this table on this AMP(s)
 - x Number of AMPs in this configuration
- For NUSI estimates, read 1 or 2 cylinders from the NUSI subtable.
 - Uses a similar technique by counting the number of NUSI values in the cylinder(s). The table row count is divided by the extrapolated NUSI row count to get a rows/NUSI value.
 - Assumes the number of distinct values on one AMP = total distinct values.
- **Any skewed component in the sample skews the demographics.**
 - Skewed demographics may mislead the optimizer into choosing a poor plan.
- For non-indexed columns without statistics, the optimizer uses heuristics or fixed formulas to estimate the number of rows. For example,
 - Assumes 10% for one column in an equality condition
 - Assumes 7.5% for two columns, each in an equality condition, and ANDed together

Example of an Optimizer Estimate without Collected Statistics

The facing page shows an example of Explain output for a SELECT from a Claim table with only Summary Statistics using an equality condition on a non-indexed column.

```
CREATE SET TABLE TFACT.Claim, FALLBACK,
  (claimid INTEGER NOT NULL,
   custid INTEGER NOT NULL,
   claimstatus CHAR(1) DEFAULT '0 ',
   claimcost DECIMAL(9,2),
   claimdate DATE FORMAT 'YYYY-MM-DD' NOT NULL,
   claimpriority SMALLINT DEFAULT 0 ,
   agentname CHAR(16) CHARACTER SET LATIN NOT CASESPECIFIC,
   claimlocation SMALLINT DEFAULT 0,
   claimtype SMALLINT DEFAULT 0,
   claimcomment CHAR(79)
   DEFAULT 'Medical - Optional Comment completed by Agent')
UNIQUE PRIMARY INDEX (claimid);
```

The answer to the first question is 10%. The second question answer is "The Optimizer will assume 7.5% for two non-indexed equality selection criteria".

With two equality conditions, the approximate estimate is 7.5% (10% x .75).

```
EXPLAIN  SELECT  *
         FROM    Claim
         WHERE    custid = 1038994 AND  claimtype = 99;
```

The size of Spool 1 is estimated with no confidence to be 2,250,000 rows.

With three equality conditions, the approximate estimate is 5.2% (10% x .75 x .75).

```
EXPLAIN  SELECT  *
         FROM    Claim
         WHERE    custid = 1038994
         AND      claimtype = 99
         AND      agentname = 'LC123456';
```

The size of Spool 1 is estimated with no confidence to be 1,658,500 rows.

If one of the columns has collected statistics and other columns do not have collected statistics, then Teradata will assume the number of values for the column with collected statistics and 75% of that value for each of the other columns without collected statistics.

Example: Assume that statistics have been collected on custid and custid = 1038994 has approximately 2000 rows.

With three equality conditions, the approximate estimate is (2,000 x .75 x .75).

Example of an Optimizer Estimate without Collected Statistics

Table information:

- Claim table has 30,000,000 rows and **only** summary statistics have been collected.
- The column Custid is not indexed and does not have collected statistics.

Query

```
EXPLAIN SELECT * FROM Claim WHERE custid = 1038994;
```

Explanation (partial)

16.20 EXPLAIN

- 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.Claim by way of an **all-rows scan** with a condition of ("TFACT.Claim.custid = 1038994") into Spool 1 (group_amps), which is built locally on the AMPs. **The size of Spool 1 is estimated with no confidence to be 3,000,000 rows** (429,000,000 bytes). The estimated time for this step is 19.73 seconds.

Question 1

Why does the optimizer estimate 3,000,000 rows will be retrieved for a custid of 1038994?

Question 2

```
EXPLAIN SELECT * FROM Claim WHERE custid = 1038994 AND claimdate = CURRENT_DATE;
```

Why does the optimizer estimate 2,250,000 rows with these two conditions?

Statistics

The primary purpose for COLLECTing STATISTICS is to tell the Optimizer how many rows/values there are. The Optimizer uses this information to plan the best way to access the data. A few of the ways that STATISTICS can be beneficial:

- Helpful in accessing a column or index with uneven value distribution.
- The Optimizer uses statistics to decide whether a query plan should use a secondary, hash, or join index instead of performing a partition or full-table scan.
- Improve the performance of complex queries and joins.
- The Optimizer uses statistics to estimate the cardinalities of intermediate spool files based on the qualifying conditions specified by a query. The estimated cardinality of intermediate results is critical for the determination of both optimal join orders for tables and the kind of join method that should be used to make those joins.
- For example, assume 2 tables or spool files are redistributed and then merge joined, or assume one of the tables or spool files is duplicated and then product joined with the other. Depending on how accurate the statistics are, the generated join plan can vary so greatly that the same query can take only seconds to complete using one join plan, but take hours to complete using another.
- Enable the Optimizer to utilize NUSI Bit Mapping.

Additional Considerations

- COLLECT STATISTICS is a DDL statement and should be scheduled during off-hours. It should not be done during production hours. The operation holds a row-hash Write Lock on DBC.StatsTbl which means that no new SQL requests involving the affected table can be parsed.
- Remain valid if the system is reconfigured.

Storage of Statistics

- Teradata stores collected statistics only once – collecting statistics on a group of columns that already have statistics collected at the index level (same set of columns) only stores the statistics one time in the DBC.StatsTbl table.
- Issuing multiple COLLECT STATISTICS statements against the same group of columns, stores only one set of statistics. Column order within the group is irrelevant. For example, collecting on COLUMN (FirstName, LastName) and then later on COLUMN (LastName, FirstName), results in only one set of statistics.

- Statistics basically tell the Optimizer how many rows/value there are for a column(s) or an index.
- **The Optimizer uses statistics to plan the best way to access data.**
 - Usually improves performance of complex queries and joins.
 - The parser is more aggressive with collected statistics.
 - Stale statistics may mislead the Optimizer into poor decisions.
- Helpful in accessing a column or index with uneven value distribution.
 - NUSI Bit Mapping is much more likely to be considered if there are collected statistics.
- Statistics remain valid across a reconfiguration of the system.
- COLLECT STATISTICS and DROP STATISTICS commands are DDL statements and typically are not executed during production hours.
- COLLECT/DROP STATISTICS places an access lock on the data table and a row-hash write lock on the following table.
 - **DBC.StatsTbl** – repository for statistics management data

Statistics Data – What is Collected?

A COLLECT STATISTICS request divides the rows in ranges of values such that each range has approximately the same number of rows, but it never splits rows with the same value across intervals. To achieve constant interval cardinalities, the interval widths, or value ranges, must vary.

With Teradata 12.0, the maximum number of intervals for statistics on an index or column was 200. With a maximum of 200 intervals, each interval can characterize 0.5 percent of the data, as opposed to the former maximum of 100 intervals, which characterized one percent of the data per interval. In TD 12.0, there was new value included in statistics for average rows per AMP which is the only value in the statistics that is configuration dependent. This value is not used if the configuration changes so the optimizer may lose a little bit of information.

Statistics have significantly changed starting with Teradata 14.0. To start with, a new option called SUMMARY is introduced to collect only the table-level statistical information such as row count, average block size, average row size, etc. without the histogram detail. This option can be used to provide up-to-date summary information to the optimizer in a quick and efficient way. This runs very quickly with negligible impact to the system resources.

Note that SUMMARY statistics are different from regular column or index statistics and doesn't contain any histogram. Also, this should not be confused with summary information for a column or index specific histogram (also referred as interval[0]). When SUMMARY option is specified in a collect statistics statement, no column or index specification is allowed.

The SUMMARY statistics are automatically collected when statistics are collected or recollected on any column or index either with index-style syntax. Therefore, it is not required to explicitly submit a request to collect summary statistical information when statistics are being collected or recollected for individual columns or indexes.

Starting with Teradata 14.0, miscellaneous internal enhancements made been made with the Optimizer with respect to statistics histogram structure and use, including:

- Storing statistics data in their native Teradata data types without losing precision
- Enhanced extrapolation methods for stale statistics
- Maintaining statistics history
- Adding cost-based optimization techniques such as rollup and pre-aggregation to determine the optimal access path for collecting or recollected statistics on a table or index.

Statistics Data – What is Collected?

If you **ONLY** collect (or refresh) **SUMMARY** statistics, then a Table Summary row is created (or refreshed).

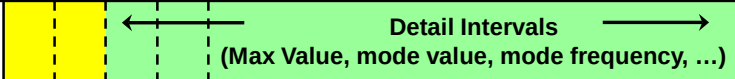
Table Summary Information (row count, average row size, sample estimates, etc.)	Previous Table Summary Records	...
--	--------------------------------	-----

To view table summary row information,

SHOW SUMMARY STATISTICS VALUES ON tablename;

If you collect (or refresh) statistics for a column/index, then a Table Summary record is created (or refreshed) as well as a "histogram row" with details for the column/index for which you are collecting statistics.

Table Summary Information (row count, average row size, sample estimates, etc.)	Previous Table Summary Records	...
--	--------------------------------	-----

Column/Index SummaryInfo		Previous SummaryInfo Records	...
-----------------------------	--	------------------------------------	-----

Bias Values

To view column/index detail information,

SHOW STATISTICS VALUES COLUMN column_name **ON** tablename;

Statistics Data – What is Collected? (cont.)

In addition to the new Summary option in Teradata 14.0, collecting statistics has been enhanced to capture more data demographic information so that the Optimizer can generate more accurate plans than it previously could. The maximum number of intervals for statistics on an index or column is increased to 500. The system default is 250 which is also the default maximum. However, you can specify up to 500 intervals with a specific collect statistics statement. With a new default of 250 intervals, each interval can characterize 0.4 percent of the data.

The DBSControl utility has an internal parameter (#127 - MaxStatsInterval) which is set to 250 by default for Teradata 14.0 systems.

The increase in the number of statistics intervals:

- Improves single table cardinality estimates that are crucial for join planning. Having more intervals gives a more granular view of the demographics.
- Increases the accuracy of skew adjustment because of the higher number of modal frequencies that can be stored in a histogram.
- Does not change the procedure for collecting or dropping statistics, although it affects the statistics collected.

Remember that a statistics are maintained within a data row that has a maximum length of approximately 64K. This 64K limit can impact the number of biased values, intervals, etc. depending on the data type of the columns/index and the MAXVALUELENGTH specified.

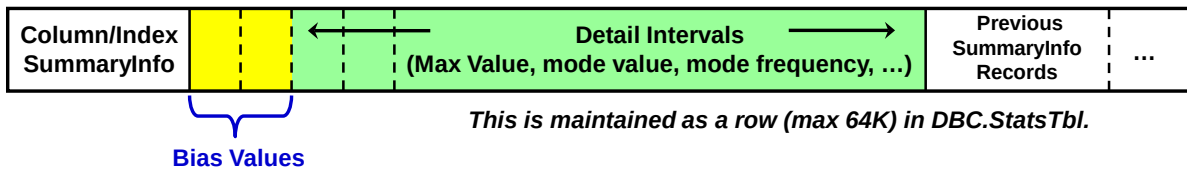
If MAXVALUELENGTH is not specified, the system uses a default maximum value length as 25. For single-column statistics of non-LATIN type, the default is used as 25 Unicode characters which translate to 50 bytes. In all other cases, it is used as 25 bytes.

Histogram records values such as min, max, mode and biased values. The MAXVALUELENGTH refers to the maximum value length that can be used to build these values. If the value length is larger than the system-determined or specified maximum, it gets truncated. Prior to Teradata 14.0, the maximum value length could not exceed 16 bytes.

Increase the maximum value length for columns that require more detailed information in the histogram to improve single-table predicate selectivity estimations. However, this should be done selectively as needed as it increases the size of the histogram which can increase the query optimization time.

For example, if a UserId column has the same characters for the first 25 characters for all users, the default value length truncates the data to the first 25 characters which makes the histogram unreliable for estimations. Increasing the value length to the length which can uniquely identify each UserId greatly improves the selectivity for the single-table predicates based on this column.

Statistics Data – What is Collected? (cont.)



Column/Index SummaryInfo – contains general information about the index/column that statistics was collected on.

- Contains timestamp, number of distinct values, min value, max value, number of intervals, etc.

Bias Values and Frequencies – a list of highly skewed values along with the frequencies for each value.

- It is possible to have many skewed values (up to 800).
- This effectively helps to “even out” the variance for the Detail Intervals.

Detail intervals – the remaining non-skewed values are spread equally across a default of 250 intervals, with each interval carrying summarized demographics.

- Each interval includes a max value, modal value for the interval, frequency for the modal value, number of other values, total number of rows for non-modal values, etc.

Previous SummaryInfo Record(s) – SummaryInfo data from previous collections is saved at the end of the histogram (when statistics are recollected).

- This information is used for extrapolation. If statistics are dropped, this history is lost.

Table Level Statistics Data

Starting with Teradata 14.0, table level summary statistics are also automatically collected when you collect statistics for a column(s) or index.

This is often referred to as Table Summary statistics.

Collecting at a summary level can be very helpful after a large load job when you don't have time for full recollections of the table's statistics.

Table Level Statistics Data

Table level summary statistics are also automatically collected when you collect statistics for a column(s) or index.

- Provides row count, average row size, and other information to the optimizer for better extrapolations.

However, it is possible to just collect summary statistics for a table.

```
COLLECT SUMMARY STATISTICS ON tablename;
```

- If only SUMMARY statistics are collected, no histogram is built.
- This might be useful for large tables with a UPI as this is a very fast collection.

When you recollect statistics, a table summary record is saved automatically.

- To view the history information:

```
SHOW SUMMARY STATISTICS VALUES ON tablename;
```

• More trend-based extrapolation decisions can be performed because the optimizer see the pattern in growth for the table.

- Some statistics recollections may be skipped if history is adequate.

Table Level Summary Statistics

The facing page contains an example of using the SHOW SUMMARY STATISTICS command to view summary statistics.

The table used in the following examples was created with the following DDL and contains 30,000,000 rows. Claims were stored for 10 years and the claim dates were between 2010 and 2019.

```
CREATE SET TABLE TFACT.Claim_RP, FALLBACK,
    NO BEFORE JOURNAL,
    NO AFTER JOURNAL,
    CHECKSUM = DEFAULT,
    DEFAULT MERGEBLOCKRATIO
(
    claimid INTEGER NOT NULL,
    custid INTEGER NOT NULL,
    claimstatus CHAR(1) DEFAULT '0 ',
    claimcost DECIMAL(9,2),
    claimdate DATE FORMAT 'YYYY-MM-DD' NOT NULL,
    claimpriority SMALLINT DEFAULT 0 ,
    agentname CHAR(16) CHARACTER SET LATIN NOT CASESPECIFIC,
    claimlocation SMALLINT DEFAULT 0,
    paymentcode SMALLINT DEFAULT 0,
    claimcomment CHAR(79)
        DEFAULT 'Medical - Optional Comment completed by Agent')
PRIMARY INDEX (claimid)
PARTITION BY RANGE_N
    (claimdate BETWEEN DATE '2010-01-01' AND DATE '2019-12-31'
        EACH INTERVAL '1' DAY, NO RANGE);
```

Table Level Summary Statistics

SHOW SUMMARY STATISTICS VALUES On Claim_RP;

/** TableLevelSummary **/

/* Version	*/ 6,	
/* NumOfRecords	*/ 3,	
/* Reserved	*/ 0.000000,	
/* Reserved	*/ 0.000000,	
/* SummaryRecord[1] */		
/* Temperature	*/ 0,	
/* TimeStamp	*/ TIMESTAMP '2018-12-25 19:03:22-00:00',	
/* NumOfAMPs	*/ 16,	
/* OneAMPsSampleEst	*/ 28304496,	
/* AllAMPsSampleEst	*/ 28232506,	
/* RowCount	*/ 30000000,	
/* DelRowCount	*/ 0,	
/* PhyRowCount	*/ 28232506,	
/* AvgRowsPerBlock	*/ 915.897308,	
/* AvgBlockSize (bytes)	*/ 32768.000000,	
/* BLCpctCompressed	*/ 100.00,	
/* BLCBlkUcpuCost (ms)	*/ 0.005413,	
/* BLCCompRatio	*/ 88.000000,	
/* AvgRowSize	*/ 130.000000,	
:		
/* SummaryRecord[2] */		
/* Temperature	*/ 0,	
/* TimeStamp	*/ TIMESTAMP '2018-12-22 16:16:06-00:00'	
/* NumOfAMPs	*/ 16,	

Only if table has multiple temperatures

Actual number of rows

Columnar tables only

Block Compression Ratio

Column Detail Statistics

The facing page contains an example of using the `SHOW STATISTICS VALUES` for a column.

Column Detail Statistics

```
COLLECT STATISTICS COLUMN claimdate ON CLAIM_RP;
SHOW STATISTICS VALUES COLUMN claimdate ON Claim_RP;
```

```
/** SummaryInfo **/
/* TimeStamp          */ /* TIMESTAMP 2018-12-25 19:08:19-00:00',
/* Version            */ /* 6,
:
/* NumOfEHIntervals   */ /* 250,
/* SamplePercent       */ /* 0.00,
:
/* MinVal             */ /* DATE '2010-01-02',
/* MaxVal             */ /* DATE '2019-12-31',
/* ModeVal            */ /* DATE '2018-02-28',
/* HighModeFreq        */ /* 109570,
/* NumOfDistinctVals   */ /* 3620,
/* NumOfRows           */ /* 30000000,
:
/** Biased: Value, Frequency **/
/* 1 */ /* DATE '2010-01-31', 7242,
/* 2 */ /* DATE '2010-02-28, 13752
/* 3 */ /* DATE '2010-03-31', 7263,
:
/** Interval: MaxVal, ModeVal, ModeFreq, LowFreq, OtherVals, OtherRows **/
/* 1 */ /* DATE '2010-01-29', DATE '2010-01-02', 4296, 2142, 27, 58784,
/* 2 */ /* DATE '2010-02-05', DATE '2010-02-04', 2209, 2165, 5, 10888,
/* 3 */ /* DATE '2010-02-19', DATE '2010-02-07', 2219, 2152, 13, 28211,
:
```

Min and Max values for the column

Number of distinct values

High Bias values

Detail Intervals

Statistics Example

Statistics were collected on the claimdate column of a Claim table. Using the information shown with the SHOW STATISTICS VALUES on the previous page, the following SQL statements will generate the following estimates for the optimizer.

If the user executes the following SQL statement,

```
SELECT * FROM Claim WHERE claimdate = DATE '2010-01-02';
```

The optimizer will assume there are 4296 claims on this date since '2010-01-02' is the modal value for the first statistics interval.

If the user executes the following SQL statement,

```
SELECT * FROM Claim WHERE claimdate = DATE '2010-02-28';
```

The optimizer will assume there are 13,752 claims for this date since '2010-02-28' is a high bias value.

If the user executes the following SQL statement,

```
SELECT * FROM Claim WHERE claimdate = DATE '2010-01-05';
```

The optimizer will assume there are 2178 claims for this date since '2010-01-05' is not a high bias or modal value. How is 2178 determined? Since there are 58,784 other claims in the interval and 27 other claim dates, the optimizer simply divides 58,784 by 17 which equals 2177.2 which is rounded up to 2178

Since a range is used with a date, the optimizer can make an estimate as to the number of claims for the following query of 4 days.

If the user executes the following SQL statement,

```
SELECT * FROM Claim WHERE claimdate  
    BETWEEN DATE '2010-01-05' AND DATE '2010-01-08';
```

The optimizer will assume 4 days x 2177.2 or approximately 8708.8 (rounded up to 8709) claims for the range.

Statistics Example (cont.)

Statistics were collected on the claimdate column of Claim_RP which has 10 years of data.

SummaryInfo	Bias Value	Bias Value	Detail Interval #1	Detail Interval #2	248 more Intervals
	2010-01-02	2010-01-29	2010-02-05		

Summary Section	Bias Value	Bias Value	Detail Interval #1	Detail Interval #2
	Value 1 2010-01-31	Value 2 2010-02-28	Max Value – 2010-01-29 Modal Value – 2010-01-02 Modal # of Rows – 4296 Other Values – 27 Other Rows – 58,784	Max Value – 2010-02-05 Modal Value – 2010-02-04 Modal # of Rows – 2209 Other Values – 5 Other Rows – 10,888
	Frequency 7,242	Frequency 13,752		

SQL Statement

Optimizer assumes

```

SELECT * FROM Claim_RP WHERE claimdate = DATE '2010-01-02';      4296
SELECT * FROM Claim_RP WHERE claimdate = DATE '2010-02-28';      _____
SELECT * FROM Claim_RP WHERE claimdate = DATE '2010-01-05';      _____
SELECT * FROM Claim_RP WHERE claimdate
      BETWEEN DATE '2010-01-05' AND DATE '2010-01-08';          8709
  
```

Note: The Optimizer estimates that 4 days @ 2177.2 values is 8709.

COLLECT STATISTICS Command

The Optimizer format for the COLLECT STATISTICS statement is shown on the facing page.

If you are collecting statistics for a table, specify a table name to indicate on which table you want to COLLECT STATISTICS. There are options that allow you to collect for a single column, a list of the columns, or a named index. These options are required to define and collect statistics initially. They may also be used to specify an existing statistic you want refreshed. They can be eliminated if you want the system to refresh all previously defined statistics for the named table.

You can edit your CREATE INDEX statements to create your COLLECT STATISTICS statements. Just replace “CREATE INDEX” with “COLLECT STATISTICS”.

Miscellaneous Considerations

The maximum number of columns on which you may collect statistics has is 512 for a table. You can collect or recollect statistics on a combined maximum of 512 implicitly specified columns and indexes. An error results if statistics are requested for more than 512 columns and indexes in a table. This error condition could result during re-collection, when all statistics that have been previously collected are re-generated. For example, if you collected statistics six different times, each for a set of 100 columns, each of those collect statements would work, because they would each be under the 512 limit. However, if you re-collected the statistics the system would attempt to generate all 600 and would fail when it hit the 512 limit.

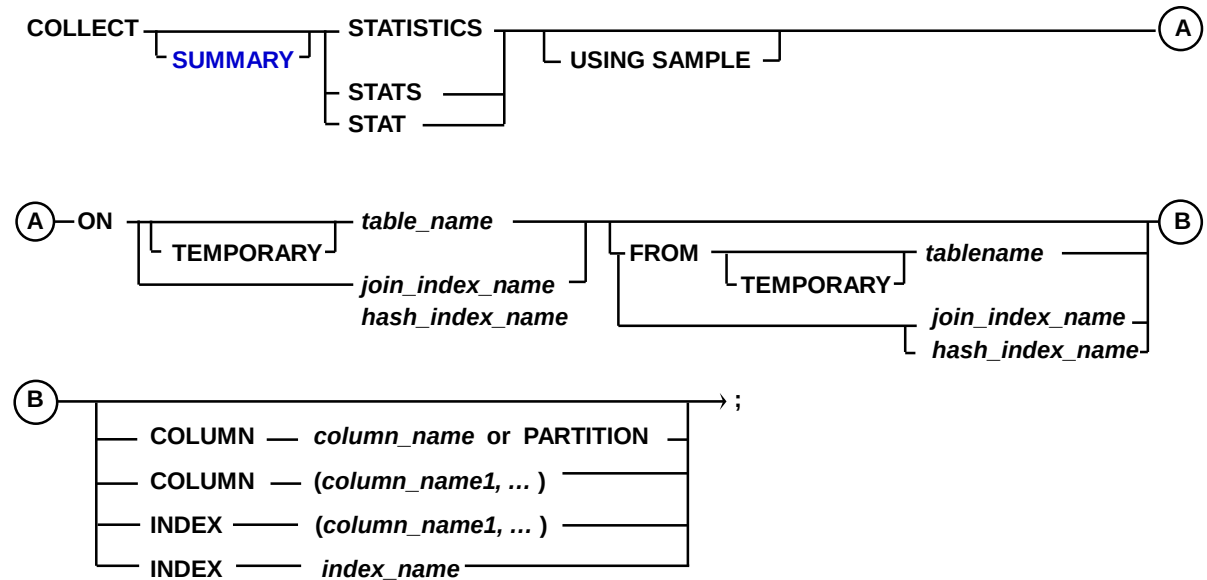
Increasing this limit allows users to issue larger re-collections involving more spool files. While this may increase system resource usage, it also provides better, more accurate information to the optimizer. Improved optimization results in more efficient execution plans, which significantly reduce system resource usage during query processing. Efficient system usage reduces overhead cost and time.

The bottom line on effective use of COLLECT STATISTICS is: Collect ***all*** and ***only*** what is needed to help the Optimizer make the best possible execution plans. Collecting more statistics than are needed or used wastes resources by putting extra processing and storage burdens on the system.

When to Collect Multiple Column Statistics

- If a set of columns are frequently specified together as an equality condition or in a join condition, then collect on the set of multiple columns.
- If a multiple column non-unique index is defined, then collect statistics on the index or the set of columns that comprise the index.
- Prior to Teradata 14.0, multi-column statistics were stored in the order of the CREATE TABLE, not in the order listed in the COLLECT STATISTICS statement.

COLLECT STATISTICS Command



SUMMARY (14.0 option) – collects table-level summary statistics – cardinality (number of rows), average # of rows per AMP, etc.

If collecting on a column or index, summary statistics are also collected automatically.

With this format, multiple COLUMN and/or INDEX specifications are not allowed.

COLLECT STATISTICS Command

(Index Format with 14.0 Options)

The Index format syntax for the COLLECT STATISTICS statement is shown on the facing page. You can edit your CREATE INDEX statements to create your COLLECT STATISTICS statements. Just replace “CREATE INDEX” with “COLLECT STATISTICS”.

16 Byte Limit in Statistics Intervals

Prior to Teradata 14.0, only the first 16-bytes of a column(s) that have collected statistics are stored in the statistic intervals. Lengthy multi-column stats will usually overflow the 16 byte limit of how much meaningful data can be carried within the statistics intervals.

For example, if the first column in a multi-column stats is CHAR(20), then no values from the other columns will get recorded in the histogram intervals. Another example is where a large number of the values in a CHAR column start with a common string, for example “State Department of”. As a result, there is no differentiation for these particular values in the histograms, because the differentiation comes after the 16 byte limit, so the optimizer was not able to link the actual value to helpful estimates. Dropping statistics in this case may improve query plans.

The rule of thumb with statistics, when you ask yourself whether to collect them or not on a specific column or set of columns is whether they improve the query plans or not. If they do, collect them, if they don’t drop them.

This is no longer an issue starting with Teradata 14.0. You can override the maximum value length (based on column lengths by using the MAXVALUELENGTH *n* option.

Teradata never truncates numeric values for single-column statistics. The system increases the interval size automatically if the specification is not sufficient to accommodate the full value for single-column statistics on numeric columns.

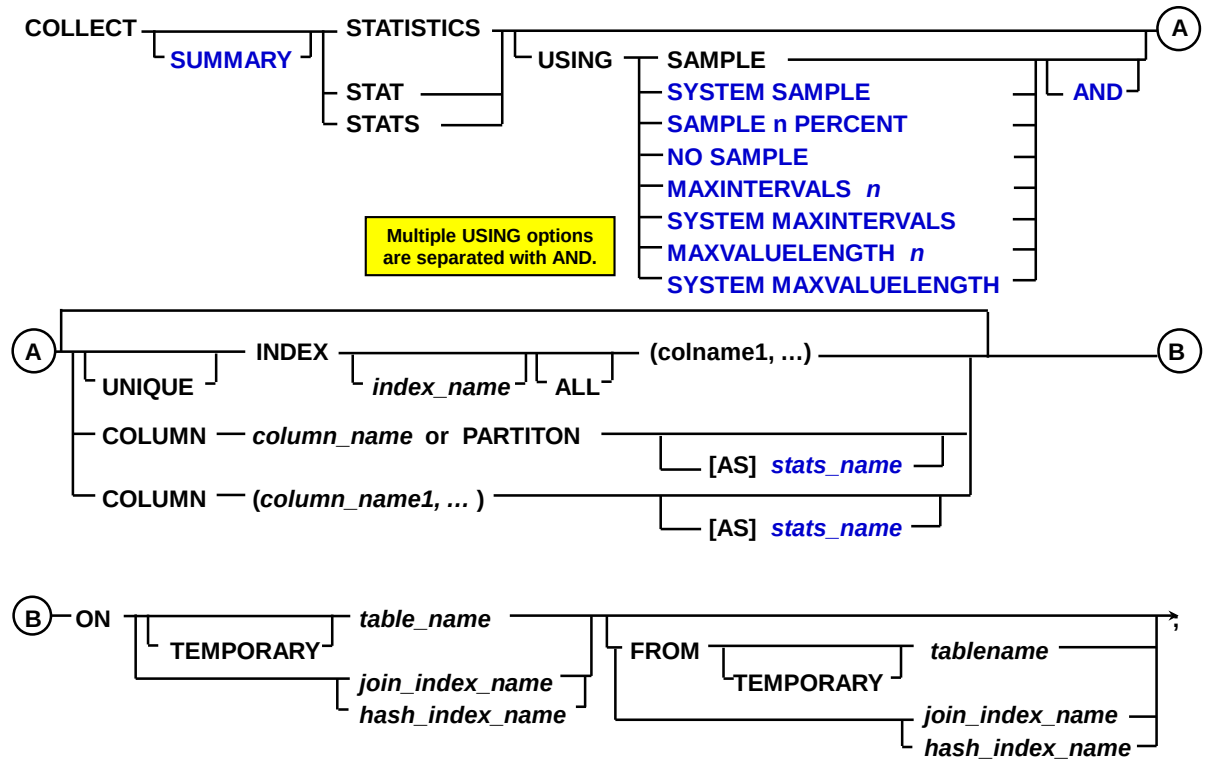
For multicolumn statistics, if the maximum interval size truncates numeric statistical data, Teradata automatically increases the maximum interval size to accommodate the numeric column on the maximum size boundary. A larger maximum value size causes Teradata to retain the value until the specified maximum is reached, which can enable better single-table and join selectivity estimates for skewed columns. However, you should be selective when increasing the size for the required columns because increasing the maximum value size also increases the size of the histogram, which can increase query optimization time. You can only specify this option if you also specify an explicit column or index.

NO SAMPLE

NO SAMPLE specifies to use a full-table scan to collect the specified statistics. You can only specify this option if you also specify an explicit index or column set.

COLLECT STATISTICS Command

(Index Format with 14.0 Options)



Collecting Statistics Example

Initial Definition and Collection

Each of the statements applies to a separate column of the table. The COLUMN or INDEX parameters must be specified with COLLECT STATISTICS.

Here are some notes about the Explain on the facing page:

- The system acquires an ACCESS lock on the table that will not interfere with existing transactions.
- The Row Hash WRITE lock on DBC.StatsTbl in Step #9 will not prevent new transactions which reference the column **Custid**, **Claimdate**, or **PARTITION** from parsing.
- Steps 15 and 16 send out a “Spoiling Message” to all PEs. This will spoil any current execution plans that use the **Custid**, **Claimdate**, or **PARTITION** column for access. It will also remove the column and its existing demographic information from DD cache so that new requests will cause the Parser to access DBC.StatsTbl for the new demographic information.

An alternative to the Index format (facing page) is to issue separate collect stats commands.

```
COLLECT STATISTICS ON Claim_RP COLUMN custid;  
COLLECT STATISTICS ON Claim_RP COLUMN claimdate;  
COLLECT STATISTICS ON Claim_RP COLUMN PARTITION;
```

Locks and Concurrency

When you perform a COLLECT STATISTICS statement, the system places an ACCESS lock on the table from which the demographic data is being collected. The system places row-hash-level WRITE locks on the DBC.StatsTbl while collecting statistics at the column and index levels.

In general, COLLECT STATISTICS statements can run concurrently with DML statements, other COLLECT STATISTICS statements, DROP STATISTICS statements, and HELP STATISTICS statements against the same table. COLLECT STATISTICS can also run concurrently with a CREATE INDEX statement against the same table as long as they are not for the same index.

Collecting Statistics Example

Initial Definition and Collection

```
EXPLAIN
COLLECT STATISTICS COLUMN custid, COLUMN claimdate, COLUMN PARTITION
ON Claim_RP; /* Example using the Index format */
```

Explanation

16.20 EXPLAIN

- 1) First, we lock TFACT.Claim_RP in TD_MAP1 for access.
 - 2) Next, we do an all-AMPs SUM step in TD_MAP1 to aggregate from TFACT.Claim_RP by way of an all-rows scan with no residual conditions, grouping by field1 (TFACT.Claim_RP.custid). Aggregate Intermediate Results are computed globally, then placed in Spool 3 in TD_Map1. The size of Spool 3 is estimated with no confidence to be 1,048,576 rows (30,408,704 bytes). The estimated time for this step is 1 minute and 56 seconds.
 - 3) Then we save the UPDATED STATISTICS for ('custid ') from Spool 3 (Last Use) into spool 5, which is built on a single AMP derived from the hash of the table id.
 - 4) We SKIP collecting STATISTICS for ('claimdate '), because no data change detected from the last statistics collection.
 - :
 - 7) We compute the table-level summary statistics from spool 15 and save them into spool 16, which is built on a single AMP derived from the hash of the table id.
 - 8) We lock DBC.StatsTbl in TD_DATADictionaryMAP for write on a RowHash.
 - :
 - 14) We spoil the statistics cache for the table, view or query.
 - 15) We spoil the parser's dictionary cache for the table.
 - 16) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > No rows are returned to the user as the result of statement 1.

Refresh or Re-Collect Statistics

Statistics should not be “collected” once and then forgotten. They tend to become stale as the tables change, and can cause performance problems by misleading the Optimizer into making poor decisions. Always keep your statistics current and valid by refreshing them on a regular, production-schedule basis. To refresh or recollect statistics for all of the previously collected columns, simply execute the COLLECT STATISTICS command for the table.

Rollup Aggregations

Rollup Aggregations is a Collect Statistics feature enhancement starting with Teradata 14.0 and beyond.

If you collect stats on a column and also on a set of multi-columns and one of multi-columns includes the same single column, rollup aggregation is used. Basically, statistics is collected for the multi-column and these statistics are reused (rolled up) to produce the single column statistics. Teradata doesn't need to scan the table for the single column statistics.

This only occurs if you specify the single column and multicolumn in the same COLLECT STATS statement OR if you recollect at the table level. This is done automatically when you collect at the table level.

Why are Collect Statistics done serially?

If you collect statistics on individual columns that are not part of a multicolumn set, Teradata will single thread the collections – they will be done one at a time. This is good if you want to minimize impact on other work. If you want to get it done in minimum time though, launch multiple sessions and submit individual collects on each. Total elapsed time will be better for several running concurrently than it will be for running them all one at a time. However, this will consume more of the system resources – specifically more CPU cycles.

The Collect Statistics code has to pull the data independently for each stats definition. It basically has to do a large aggregation grouping on the stats columns. Thus it will do a scan and aggregate for each one. It turns out that the scan of data is the least expensive part of the operation, the real cost is the aggregation/computation of the stats.

Normally, collect stats does not utilize sync scan; sync scans would only help if you were running them in separate sessions. However, it does utilize cylinder read which has a big positive impact on stats performance.

Refresh or Re-Collect Statistics

To refresh or recollect statistics for all of the previously collected columns and/or indexes, you can collect at the table level.

```
COLLECT STATISTICS ON Claim_RP;
```

Refresh statistics whenever 5 – 10% of the rows have changed:

- As part of batch maintenance jobs.
- After significant periods of OLTP updating.
- After new low and/or high values have extended the range of values.

Notes:

- When collecting statistics individually, **these are NOT collected in a single pass of the data** and can be very time consuming.
- However, if you need to collect statistics on a single column and a multicolumn and the single column is part of the multicolumn, then collect these in a single statement to take advantage of the Teradata's "rollup aggregation" feature.
 - Basically statistics are collected for the multicolumn and these stats are reused (rolled up) to produce the single column statistics.
- **Statistics collection is very CPU intensive** – aggregation and computation of the statistical data.

Viewing Statistics

Help Statistics

HELP STATISTICS returns the following information about each column or index for which statistics have been COLLECTed in a single table:

- The Date the statistics were last COLLECTed or refreshed.
- The Time the statistics were last COLLECTed or refreshed.
- The number of unique values for the column or index.
- The name of the column or column(s) that the statistics were COLLECTed on.

Use Date and Time to help you determine if your statistics need to be refreshed or dropped. The example on the facing page illustrates the HELP STATISTICS output for the Orders table.

Note: Prior to Teradata 14.0,

HELP STATS Claim_RP COLUMN (Custid);

will return the details about the statistics histogram and display the 200 intervals. This command will not work starting with Teradata 14.0.

Starting with Teradata 14.0, to view details about the statistics for an index/column, use the **SHOW STATISTICS VALUES** option.

Help Index (not shown)

HELP INDEX is an SQL statement which returns information for every index in the specified table. HELP INDEX returns the following information:

- Whether or not the index is unique
- Whether the index is a PI or an SI
- The name(s) of the column(s) which the index is based on
- The Index ID Number
- The approximate number of distinct index values
- Is the index hash-ordered, valued-ordered, or partitioned (H, V, P)

This information can be very useful in reading EXPLAIN output. Since the EXPLAIN statement only returns the Index ID number, you can use the HELP INDEX statement to determine the structure of the index with that ID.

Viewing Statistics

HELP STATISTICS Claim_RP;

Displays information about current statistics.

<u>Date</u>	<u>Time</u>	<u>Unique Values</u>	<u>Column Names</u>
18/12/25	19:46:11	30,000,000	*
18/12/25	19:08:19	3,620	claimdate
18/12/25	19:46:09	2,886	custid
18/12/25	19:46:11	3,620	PARTITION

(Effectively table summary or row count)

Note that the DATE and TIME show when statistics were last collected or refreshed.

4,000,000 rows were inserted into the Claim_RP table.

HELP STATISTICS Claim_RP;

Displays same information as above

HELP CURRENT STATISTICS Claim_RP;

<u>Date</u>	<u>Time</u>	<u>Unique Values</u>	<u>Column Names</u>
18/12/25	19:59:33	32,760,515	*
18/12/25	19:59:33	3,954	claimdate
18/12/25	19:59:33	2,886	custid
18/12/25	19:59:33	3,620	PARTITION

(This is the new estimated row count)

(This is an extrapolated number of dates)

COLLECT STATISTICS ON Claim_RP;

HELP STATISTICS on Claim_RP; or HELP CURRENT STATISTICS ON Claim_RP; (Same result)

<u>Date</u>	<u>Time</u>	<u>Unique Values</u>	<u>Column Names</u>
18/12/25	20:02:01	34,000,000	*
18/12/25	20:02:01	3,892	claimdate

(This is the actual row count)

(This is the actual number of dates)

Collect Statistics Using Sample

The SAMPLING option significantly reduces the resources consumed by COLLECT STATISTICS by collecting on only a percentage of the data. While it should not be used to replace all existing statistics collection, it can be used as an effective query tuning option where statistics are not being collected because of the required overhead. This feature saves time and system resources by collecting statistics on only a percentage sampling of the data.

COLLECT STATISTICS can be very time consuming because it performs a full table scan and it sorts the data to compute the number of occurrences of each distinct value. Most users accept this performance because it can be run infrequently and it benefits query optimization. Without statistics, query performance often suffers. The drawback to sampled statistics is that they may not be as accurate, which in turn may affect the quality of Optimizer plans. In most cases sampled statistics are better than no statistics.

Consider using sampling when:

- Collecting statistics on very large tables.
- Resource consumption from the collection process is a serious performance or cost concern.

Do not use sampling:

- On small tables.
- To replace all existing full scan collections.

Sampling Considerations

The system automatically determines the appropriate sample size to generate accurate statistics for good query plans and performance.

Once initial sampled statistics have been collected, recollections are also sampled. The system does *not* store both sampled and complete statistics for the same index or column set.

Sampled statistics are generally more accurate for data that is uniformly distributed. For example, columns or indexes that are unique or nearly unique are uniformly distributed. Sampling may result in fewer intervals in the histogram (it is based on number of AMPs). Sampling should not be considered for data that is highly skewed because the Optimizer needs to be fully aware of such skew. In addition to uniformly distributed data, sampling is generally more appropriate for indexes than non-indexed column(s). For indexes, the scanning techniques employed during sampling can take advantage of the hashed organization of the data to improve the accuracy of the resulting statistics. If your table is partitioned, then you should include the partitioning column in your SAMPLE statistics. Otherwise you will only sample 1 or 2 partitions.

To summarize, sampled statistics are generally most appropriate for:

1. Very large tables
2. Data that is uniformly distributed
3. Indexed column(s)

Collect Statistics Using Sample

SAMPLING reduces resource usage by collecting on a percentage of the data.

- Without a specific percent, system determines appropriate sample size to generate accurate statistics.
 - In Teradata 14.0, if the cost of collecting stats is under (0.5 sec CPU time), it is considered as a small cost and full statistics are collected unless a specific PERCENT is specified.
- To request a specific sample of rows on which to collect statistics, use this syntax:
`COLLECT STATISTICS USING SAMPLE n PERCENT ...`
- Re-collection collects statistics using the same mode (full scan or sampled) as specified in the last specified collection.
 - System does not store both sampled and complete statistics for the same index/column set.
 - **Only one set of statistics is stored for a given index or column.**
- Sampled statistics are more accurate for data that is uniformly distributed.
 - Sampling is useful for very large tables with reasonable distribution of data.
 - Sampling should not be considered for highly skewed data as the Optimizer needs to be aware of such skew.
 - Sampling is more appropriate for indexes than non-indexed column(s).
 - Full statistics are collected for the PARTITION column

Collecting Statistics (Additional Options)

USING OPTIONS

SAMPLE or SYSTEM SAMPLE

This option specifies to scan a system-determined percentage of table rows to collect the specified statistics. This option is not valid for single-table views. SAMPLE has the same meaning as SYSTEM SAMPLE and is provided for backward compatibility.

The SYSTEM SAMPLE option specifies to scan a system-determined percentage of table rows to capture the statistical information. Teradata might decide to sample 100% for the first few times before downgrading the sample percent to a lower level.

MAXINTERVALS *n*

Specifies the maximum number of histogram intervals to be used for the collected statistics. Teradata might adjust the specified maximum number of intervals depending on the maximum histogram size. Valid range is 0 – 500.

SYSTEM MAXINTERVALS

This option simply indicates to use the system default maximum number of histogram intervals for the collected statistics. The system default is typically 250, but can be adjusted using DBSControl.

MAXVALUELENGTH *n*

This option specifies the maximum size to use for histogram values such as MinValue, ModeValue, MaxValue, etc. This option is valid for both tables and single-table views. The value for *n* must be an integer number.

For single-character statistics on CHARACTER and VARCHAR columns, *n* specifies the number of characters. For all other options, *n* specifies number of bytes.

For multicolumn statistics, Teradata concatenates the values and truncates them if necessary to fit into the specified maximum size.

- For single-column statistics, the valid range of *n* is 1 - maximum size of the column.
- For multicolumn statistics, the valid range of *n* is 1 - combined maximum size of all the columns.

SYSTEM MAXVALUELENGTH

This option simply indicates that the system should use a **system-determined** value for the length of histogram values such as MinValue, ModeValue, MaxValue, etc. This value is dependent on the column data type, number of intervals, etc. This option is valid for both tables and single-table views.

Collecting Statistics (Additional Options)

Given: Claim_RP NUPI on claimid, partitioned on claimdate
 NUSI on custid

To collect sample statistics using the system default sample:

```
COLLECT STATISTICS USING SYSTEM SAMPLE COLUMN (claimdate) ON Claim_RP;
```

To collect sample statistics by scanning 10 percent of the rows and use 100 intervals:

```
COLLECT STATISTICS USING SAMPLE 10 PERCENT AND MAXINTERVALS 100  
COLUMN (custid) AS custid_stats ON Claim_RP;
```

To change sample statistics to 20 percent (for custid) and use 250 intervals:

```
COLLECT STATISTICS USING SAMPLE 20 PERCENT AND MAXINTERVALS 250  
COLUMN (custid) AS custid_stats ON Claim_RP;
```

To display the COLLECT STATISTICS statements for a table:

```
SHOW STATISTICS ON Claim_RP;
```

To display statistics details – summary section, high bias values, and intervals:

```
SHOW STATISTICS VALUES COLUMN claimdate ON Claim_RP;
```

Optimizer's use of Statistics with Uneven NUSI

COLLECTing STATISTICS on NUSIs makes it possible for the Optimizer to decide when to use the NUSI for access and when to do a Full Table Scan. The facing page shows how the Optimizer will handle two different uneven NUSIs.

The NUSI on column F has a large distribution spike where F is NULL. The Optimizer will choose to do a Full Table Scan for NULL values and will utilize the NUSI for other values.

The NUSI on column G has a large distribution at a single value. The Optimizer will choose to utilize the NUSI for NULL values, and may or may not utilize the NUSI for other values.

Optimizer's use of Statistics with Uneven NUSI

EXAMPLE

60,000,000 Rows	A		F	G	
PK/FK	PK,SA				
Distinct Values	60M		10M	8M	
Max Rows/Value	1		9	5M	
Max Rows/NULL	0		7M	5	
Typical Rows/Value	1		3	3	
	UPI				
PI/SI	USI		NUSI?	NUSI?	

Notes:

For column G, assume:

- 5,000,000 rows have a value of 0.
- All other values in G return 0 to 30 rows.

Example:

COLLECT STATISTICS ON Example COLUMN F;

```
SELECT * FROM Example WHERE F = 'value';
SELECT * FROM Example WHERE F IS NULL;
```

Optimizer will ...

use the NUSI to return 3 - 9 rows
do a FTS to return 7,000,000 rows

COLLECT STATISTICS ON Example COLUMN G;

```
SELECT * FROM Example WHERE G = 'value';
SELECT * FROM Example WHERE G IS NULL;
```

use the NUSI if the value NE 0
use the NUSI to return 5 rows

DROP INDEX (G) ON Example; DROP STATISTICS ON Example COLUMN G;

```
SELECT * FROM Example WHERE G = 'value';
```

assumes 10% or 6,000,000 rows

Collecting Statistics on PARTITION

This feature allows you to collect statistics on the system-derived PARTITION column of a row partitioned table.

Statistics on the PARTITION column allow the Optimizer to generate an aggressive plan with respect to row partitioned tables. Specifically, the Optimizer can use PARTITION statistics to estimate the cost of various operations very accurately, including:

- Static partition elimination
- Dynamic partition elimination

When the Optimizer can use partition elimination more frequently, query efficiency and performance improves. In addition, the collection process itself for single-column PARTITION statistics is highly optimized, which significantly reduces their collection time and overhead.

Prior to Teradata Warehouse V2R6.1, statistics collected on partitioning columns were used only for estimating cardinality (the number of qualifying rows resulting from access to the table). With the PARTITION statistics feature, the additional statistics can be used to estimate costing (the number of rows, data blocks, and partitions that must be scanned to produce the query result). Statistics can be collected for the system-derived PARTITION column only (single-column PARTITION statistics) or on the PARTITION column and other table columns (multi-column PARTITION statistics). These statistics provide the Optimizer with information on its state when statistics were last collected, such as:

- The number of empty partitions
- How rows are distributed in each partition

The Optimizer can use this information to better estimate the query cost when there are a significant number of empty partitions. This means that you can have empty partitions for years into the future, and not be required to ADD and DROP partitions. If PARTITION statistics are not collected, empty partitions may cause the Optimizer to underestimate the number of rows in a partition.

When the Optimizer uses PARTITION statistics in creating its plan, the EXPLAIN will show the estimate with high confidence.

Notes on Collecting Sample Statistics on the Partitioning column:

- You can specify a USING SAMPLE clause to collect single-column PARTITION statistics, but the specification is ignored. Instead, the system automatically resets the internal sampling percentage to 100.
- Note that the USING SAMPLE clause is honored for multicolumn PARTITION statistics. The system begins the collection process using a 2 percent sampling percentage and then increases the percentage dynamically if it determines that the values are skewed more than a system-defined threshold value.

Collecting Statistics on PARTITION

You can (and should) collect statistics on the system-derived PARTITION for row partitioned tables.

- Statistics on the PARTITION column provide information about partitions and allow the Optimizer to generate a more aggressive plan with respect to row partitioned tables.
- Specifically, the Optimizer can use PARTITION statistics to estimate the cost of various operations more accurately, including:
 - Static partition elimination
 - Dynamic partition elimination
- The Optimizer can use this information to better estimate the query cost when there are a significant number of empty partitions.
- This is a very fast collection as only the Cylinder Indexes need to be analyzed.

Example:

```
COLLECT STATISTICS ON TFACT.Claim_RP COLUMN PARTITION;
```

Copying STATISTICS

You can also append an `AND STATISTICS` option to a `WITH [NO] DATA` clause to copy any collected source table statistics or empty histograms to a target table. If you specify `WITH NO DATA AND STATISTICS`, the system sets up the appropriate statistical histograms in the dictionary, but does not populate them with source table statistics. This is referred to as *zeroed statistics*.

General Rules For `CREATE TABLE AS ... WITH DATA AND STATISTICS`. The following list of rules applies only to an `AS ... WITH DATA AND STATISTICS` clause.

- If there are no columns or indexes in the target table for which statistics are eligible to be copied, the system returns a warning message to the requestor.
- If you specify an explicit index definition for the target table, then the system does not copy `PARTITION` statistics from the source table to the target table.
- This is true for both single-column `PARTITION` statistics and for composite statistics on a column set that includes the system-derived `PARTITION` column.
- You cannot copy statistics for a volatile table because you cannot *collect* statistics on a volatile table.
- If you specify `WITH DATA AND STATISTICS` for a volatile source table, the request aborts and the system returns an error.
- If no statistics have been collected on the specified source table column or index sets, the system ignores the `AND STATISTICS` option and returns a warning message to the requestor.
- If only a subset of the statistics from the source table are eligible to be copied to the columns and indexes of the target table, the system returns a warning message to the requestor.
- If the number of multicolumn statistics you specify to be copied to the target table exceeds the maximum number of multicolumn statistics allowed, then the system copies multicolumn statistics only up to the limit, does not copy the remainder of the multicolumn statistics to the target table, and reports a warning message to the requestor.
- If all columns in a `MULTISET` source table are non-unique, and if the target table is a `SET` table, then the system does not copy statistics to the target table. This is because of the possible violation of the rule of equal cardinalities in the source and target tables: if there are duplicate rows in the source table, the system eliminates them before copying to the target table, resulting in unequal cardinalities between the two tables.
- If the source table has *at least* one non-unique column and the target table is a `SET` table, then the system copies the statistics from the source table to the target table if all other rules are also obeyed.
- If you specify the `NOT CASESPECIFIC` attribute for any column in the target table definition, and it does not match the corresponding source table column attribute specification, the system does not copy the statistics for that column or index set because of the possible violation of the rule about not modifying the data in the target table.
- If all the columns in the source table are non-unique and you specify the `NOT CASESPECIFIC` attribute for any column in the target table definition that does not match the corresponding source table column attribute definition, then the system does not copy the statistics for that column or index because of the possible violation of the rule of equal cardinalities in the source and target tables.

Copying STATISTICS

The COLLECT STATISTICS command includes a **FROM** option to copy statistics from one table to an identical target table.

```
COLLECT STATISTICS ON Claim_new FROM Claim;
```

The CREATE TABLE AS command includes an option to copy statistics from one table to another by including the "AND STATISTICS" option.

- The **CREATE TABLE ... AS ... WITH DATA AND STATISTICS;**
 - In this case, a new table is created via a copy definition (DDL) of an existing table, data is copied from the source table to the target table, and the applicable statistics are replaced with the same statistics (histograms) from the target table.

```
CREATE TABLE Customer_Test1 AS CUSTOMER WITH DATA AND STATISTICS;
```

- The **CREATE TABLE ... AS ... WITH NO DATA AND STATISTICS;**
 - In this case, a new table is created via a copy definition (DDL) of an existing table, no data is copied from the source table to the target table, and the applicable statistics are replaced with zeroed statistics (histograms).

```
CREATE TABLE Customer_Test2 AS CUSTOMER WITH NO DATA AND STATISTICS;
```

Statistics Extrapolation

The optimizer can use extrapolation or derived statistics when it detects stale statistics. Stale statistics is based on 10% or 10,000 row count growth. For very small tables (less than 25 rows per AMP), or for tables with a skewed data distribution, no extrapolation will be done. A technique to force extrapolation is to switch to an all AMP random AMP sampling.

If the difference exceeds either of these values, the Optimizer assumes that the histogram statistics are stale and overrides them with the estimates returned by a random AMP sample.

Even if the difference exceeds the value for the absolute deviation (10,000), the Optimizer still uses histogram statistics for its selectivity estimates, and those estimates are extrapolated.

Stale Statistics Detection

Currently, the table row count is estimated from the random AMP sampling or the statistics on the primary index (PI) of the table.

Starting with Teradata 12.0, instead of always trusting Primary Index histogram row count, the row count from Random AMP Sampling and the histogram are compared and a decision is made based on certain normalization heuristics.

- The histogram row count is compared with the table row count and if the deviation is more than the defined threshold, the histogram is determined as stale.
- Stale histograms are specially tagged in the optimizer and value count/row extrapolations are done when they are used for cardinality estimations.
- Stale Statistics Detection also applies to tables that have zero statistics as well and allows for table row count extrapolation.

Extrapolating Statistics Outside Range

This 12.0 enhancement allows the Teradata Optimizer to include a new extrapolation technique specifically designed to more accurately provide for a statistical estimate for date range-based queries that specify a “future” date.

The Optimizer extrapolation technique for date range-based queries that supply “future” dates will result in better query plans due to the fact that cardinality estimation will be much more accurate. Because of the new extrapolation formula it is also possible that statistics for the associated date columns would not have to be re-collected as often.

Some considerations include:

- However, the information displayed within a query Explain plan will change because of the new numbers for estimated rows.
- Specific consideration should be given to collecting statistics less frequently on columns which will now be extrapolated.

Statistics Extrapolation

The optimizer can use extrapolation or derived statistics (random AMP sample) when it detects stale statistics.

Stale Statistics Detection

- The row count from random AMP sampling and the histogram are compared and a decision is made based on certain normalization heuristics.
 - The histogram row count is compared with the table row count and **if the deviation is more than a defined threshold, the histogram is determined as stale.**
 - The stale statistics threshold is based on 10% or 10,000 (small tables) row count growth.
- Collected statistics are always used by the optimizer (summary or histogram) and adjusted upward based on the Random AMP Sample (RAS) estimates.
 - In 14.0, RAS is only used to make upward adjustments. In 14.10, if USECOUNT is also utilized, then RAS will be able to make downward as well as upward adjustments.

Extrapolate Statistics Outside of Range

- This extrapolation technique is designed to more accurately provide for a statistical estimate for date range-based queries that specify a “future” date that is outside the bounds of the collected statistics.
 - For date range-based queries that supply “future” dates, the optimizer can create better plans because the cardinality estimation will be more accurate.
 - A benefit is that statistics may not have to be re-collected as often.

Additional Features and Considerations

- You can execute a `SHOW IN XML STATS ON tablename`; which is easier for applications because there is nothing to parse.
- Improves query optimization time through a new dedicated statistics cache.
- Statistics collects up to 25 bytes of the value per column (by default)
- Statistics are no longer case-specific.

Collect Statistics on Single Table Expression

- Expressions can now be specified in `COLLECT STATS` statements. This Provides more accurate cardinality estimation for query with expressions and help the optimizer to pick the best plan.

Examples of expressions that can be used are:

- `Substr()`, `MOD`, `CONCATENATION`, Format Conversion expressions like Uppercase, Lowercase, Data type Conversions, `CASE-Expressions`, `BEGIN` and `END` expressions of `PERIOD` type.
- Used for expressions in following places
 - Single-table predicates, Join predicates, Select-list

Downgrade FULL Stats to SAMPLE Stats

- The system based on the eligibility of the column will determine if `FULL` stats can be avoided for a specific column. Downgrade to `SAMPLE` stats reduces the re-collection overhead. Leverages
 - Random AMP Samples to detect growth
 - System maintained UDI counts to detect change. `STATS` version (V6) is introduced to support UDI counts.
 - Trend based extrapolation techniques

Avoid SPOOL errors during collect stats operation

- The system now includes the intelligence to first detect the spool space requirement before beginning the statistics collection process, if the spool space is not sufficient enough then:
 - A new layer of local statistics will be built on one of the AMPS to avoid out of spool error. Then the global statistics will be based on local statistics.

When to Use DROP and COLLECT STATISTICS

- The formerly recommended practice of dropping stale statistics is now deprecated because when you drop statistics, you also drop the appropriate history records.
- If you suspect that your table statistics are not current, you should recollect them. The statistics recollection logic retains the history of the information from the prior histograms, which is used to extrapolate stale statistics to freshen them.
- If the column demographics are drastically changed and the history records are no longer able to determine the column trend for the new data, drop the statistics to eliminate the history records and recollect them.

Geospatial Data Type

- Single column Statistics capabilities for the Geospatial Data Type

Additional features include:

- Collect Statistics on single table expressions.
 - Expressions can now be specified in COLLECT STATS statements.

```
COLLECT STATISTICS USING SYSTEM SAMPLE COLUMN (EXTRACT(MONTH FROM
orderdate), totalprice) AS stats_month_price ON orders;
```
- Automatic downgrading of Full Stats to Sample Stats.
 - Based on the eligibility of the column the system will determine if FULL stats can be avoided for a specific column. SAMPLE stats reduces the re-collection overhead.
- Avoiding Spool errors during collect statistics.
 - Intelligence to first detect the spool space requirement before beginning the statistics collection process.
- Automated Statistics Management (**AutoStats**)
- Statistics re-collection performance improvements
 - Time and Change based **THRESHOLD** option can be used to reduce statistics re-collection overhead by automatically skipping the recollection process when not needed.
 - *Note:* Dropping stale statistics no longer recommended, but rather re-collect statistics. DROP STATS drops all stats history and can have adverse effects if AutoStats is used.

Teradata Re-Collect Enhancements

The facing page lists several statistics improvements starting with eradata 14.10. Benefits of these enhancements include:

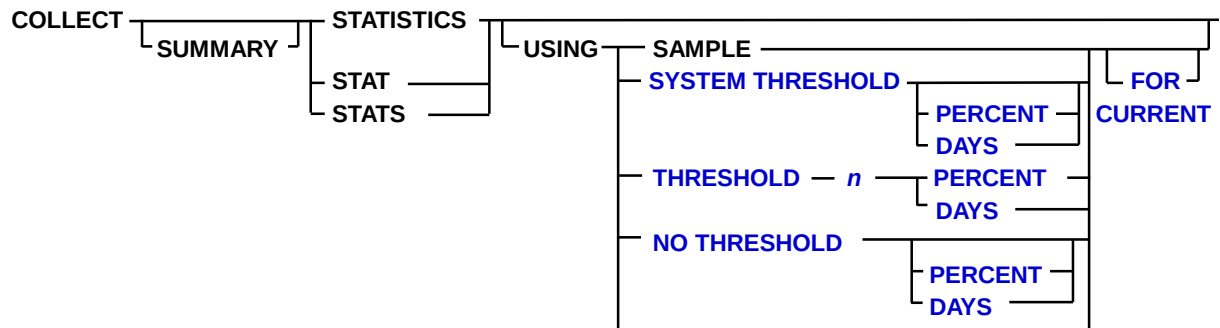
Statistics Re-collection performance Improvements

- Time and Change based THRESHOLD option can be used to reduce statistics re-collection overhead by automatically skipping the recollection process when not needed.
 - Time Based THRESHOLD Example : If the user sets a THRESHOLD for specific statistics collection to be 10 Days and issues a collect stats after 3 days, the system will not collect statistics.
 - Change Based THRESHOLD Example : Evaluates the need for statistics collection based on the data growth. Insert and Updates are taken into consideration to determine data growth
- System Derived THRESHOLD option will be used to automatically determine the need to collect statistics, this features leverages
 - Random AMP Samples to detect growth
 - System maintained UDI counts to detect change; STATS version (V6) is introduced to support UDI counts. Note: USECOUNT logging for the database must also be enabled. If USECOUNT DBQL logging is turned on, then each COLLECT STATISTICS statement will be evaluated to see if it will run or be skipped.
 - Trend based extrapolation techniques

New DBSControl group called “Statistics Fields”

- DefaultTimeThreshold:
 - Default time threshold value in days between 0 and 9999. 0 indicates time threshold checking is disabled. Default = 0
- DefaultUserChangeThreshold:
 - Default data change percentage threshold value between 0 and 9999. A value of 0 indicates user change threshold checking is disabled. Default = 0
- SysChangeThresholdOption:
 - System change threshold option. Valid settings are:
 - 0: System default (same as a setting of 1)
 - 1: Enable it when UDI is enabled. 2: Enable it unconditionally. 3: Disable it.
- SysSampleOption:
 - Sampling option. Valid settings are
 - 0: System Default (same as a setting of 1)
 - 1: Enable system-determined sample percentage up to 100%.
 - 2: Sample percentage is 100% (full statistics).

Teradata Re-Collect Enhancements



Collect statistics and specify future threshold using settings:

```
COLLECT STATISTICS USING THRESHOLD 10 PERCENT AND THRESHOLD 7 DAYS COLUMN
(custid) ON Claim_RP;
```

Re-collect statistics 5 days later and there is less than 10% growth change:

```
COLLECT STATISTICS ON Claim_RP; (Note: Statistics will NOT be collected)
```

Re-collect statistics again 1 day later and there is still less than 10% growth change:

```
COLLECT STATISTICS USING NO THRESHOLD FOR CURRENT COLUMN (custid) ON
Claim_RP; (Note: Statistics will be collected and no affect on threshold settings)
```

Teradata AutoStats features

Overview of Teradata AutoStats DBS Features (14.10)

- Managing statistics is labor intensive and mistake prone.
 - Customers complain of spending too much time and machine resources managing stats:
 - Which columns and indexes should we collect stats on?
 - How often do we need to refresh them?
 - How do we know if the stats we've been collecting are still being used?
- Repository – A system supplied database named TDSTATS that stores metadata for all stats collections. Created by DIPSTATS.
- Open APIs in the form of SQL external stored procedures (XSPs) that perform important stats management operations.

Major categories of APIs:

- **Automate** – Copy dictionary stored definitions for existing statistics to TDSTATS; places identified stats under the control of Auto Stats features
 - **Analyze** – Analyze logged DBQL query plans and OUC activity counts to recommend new stats and identify stale or unused stats. Aggregate recommendations across the entire workload to determine the critical subset to apply. A rule-based version of Analyze is available for situations where query log data is unavailable.
 - **PrepCollect** - Prepare a pending list of prioritized collections for next scheduled collection period; incorporates latest recommendations and state information from Analyze APIs
 - **RunCollect** - Submit COLLECT STATISTICS statements using list prepared by PrepCollect; accepts optional user time limit
 - **Reports** - Display results from completed or in-progress API calls
 - **DBAControl** - Setup, configure, and override default behavior
- DBQL Enhancements – Log Optimizer statistics recommendations and usage with dedicated DBQL logging option. Works with or without related option XMLPLAN enabled.

Teradata AutoStats features

Teradata Automated Statistics Management (AutoStats) features (14.10) include:

- **Goal of AutoStats:** Automate and provide intelligence to DBA tasks related to Optimizer Statistics Collections:
 - Identify and collect **missing** (new) statistics needed for accurate query optimization
 - Detect **stale** statistics and promptly refresh them
 - Identify and remove **unused** statistics from ongoing maintenance
 - Prioritize the list of pending collections such that important and stale statistics are given precedence
 - Execute needed collections in the background during scheduled time periods
- **Repository** – A system supplied database named TDSTATS that stores metadata for all stats collections. Created by DIPSTATS.
- **Open API** – in the form of SQL external stored procedures (XSPs) that perform important stats management operations.
- **DBQL Enhancements** – Log Optimizer statistics recommendations and usage with dedicated DBQL logging option.

Statistics Summary

Remember to refresh statistics on a regular basis.

Dropping a secondary index for a single column deletes the index's definition from DBC.Indexes. However, the column definition and single-column statistics stored in DBC.StatsTbl (starting with 14.0) will still exist.

General Recommendations

- Enable DBQL USECOUNT logging for all important databases whose table row counts may change over time. This provides information about updates, deletes and inserts performed on each table within the logged databases and contributes to better extrapolations.
- Benefit from the default system threshold option, which may allow some submitted statistics to be skipped, by turning on DBQL USECOUNT logging and building up statistic history records. Skipping can potentially reduce the resources required by statistics recollections.
- Expect to perform several full collections before statistic skipping or automatic downgrade to sampling is considered.
- Use the collection statement-level THRESHOLD option only for cases where there is a specific need to override the global threshold default.
- Consider collecting statistics (and providing a name) on SQL expressions if they are frequently used in queries and if they reference columns from a single table.

Statistics Summary

- **Recommendations for collect statistics include:**
 - All non-unique indexes.
 - For small tables (less than 10,000 rows per AMP) with a UPI, collect full statistics.
 - For large tables with a UPI or nearly unique values, consider collecting SUMMARY statistics.
 - Collect statistics on the key word PARTITION for row partitioned tables
 - Any non-indexed column used for set selection or join constraints.
- **Collected statistics are not automatically updated by the system.**
 - The user is responsible for refreshing collected statistics.
 - Refresh statistics when 5% to 10% of the table's rows have changed.
- **The optimizer is more aggressive when it has COLLECTed STATISTICS.** It is more conservative when it must rely on random AMP sampling.
- **Limitations**
 - Maximum # of columns in a table on which you may implicitly recollect statistics is 512.
 - Maximum number of column names within a given COLUMN list is 64.
 - Maximum number of multiple column COLLECT STATS statements per table is 32.

Module 19: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 19: Review Questions

1. Where are statistics stored in Teradata 16.20 when you COLLECT STATISTICS on a NUPI? _____
 - a. DBC.StatsTbl
 - b. DBC.Indexes
 - c. DBC.TVFields
 - d. Data Dictionary cache

2. Random AMP sample statistics are stored in _____.
 - a. DBC.Indexes
 - b. DBC.TVFields
 - c. Data Dictionary cache
 - d. Request-to-Steps cache

3. Which statement is used to display statistical detail information (e.g., summary section, high bias values, and intervals)? _____
 - a. HELP INDEX
 - b. SHOW TABLE
 - c. HELP STATISTICS
 - d. SHOW STATISTICS

4. Which statement is used to display extrapolated statistics used by the optimizer? _____
 - a. HELP INDEX
 - b. HELP STATISTICS
 - c. HELP CURRENT STATISTICS
 - d. SHOW CURRENT STATISTICS

Notes



The EXPLAIN Facility

After completing this module, you will be able to:

- Describe the Explain Facility.
- Define Explain terminology.
- Describe the EXPLAIN output of a CREATE TABLE statement.
- Match EXPLAIN terms to a definition.
- Identify the ways to EXPLAIN Macros.
- Use EXPLAIN to determine the number of partitions used in various access plans.

Teradata Proprietary and Confidential

Notes

Table of Contents

The EXPLAIN Facility	20-4
EXPLAIN Facility Output	20-6
EXPLAIN – SELECT with PI value.....	20-8
EXPLAIN – SELECT (FTS)	20-10
EXPLAIN Terminology.....	20-12
EXPLAIN Terminology (cont.)	20-14
14.10 EXPLAIN Terminology & Modifiers.....	20-16
15.10 EXPLAIN Terminology.....	20-18
15.10 EXPLAIN Terminology (cont.)	20-20
16.00 EXPLAIN Terminology.....	20-22
16.10 EXPLAIN Terminology.....	20-24
16.20 EXPLAIN Terminology.....	20-26
Pseudo Table Locks	20-28
Understanding Row and Time Estimates (Part 1).....	20-30
Understanding Row and Time Estimates (Part 2).....	20-32
EXPLAIN – Create Table	20-34
EXPLAIN Macro	20-36
EXPLAIN Terminology for Partitioned Tables.....	20-38
EXPLAIN – Partition Elimination	20-40
EXPLAIN – Primary Index Access of Partitioned Table	20-42
Create a USI or NUSI on the PI for a Partitioned Table.....	20-42
EXPLAIN – Dynamic Partition Elimination	20-44
EXPLAIN – Delete Rows in Partitioned Table	20-46
Summary	20-48
Module 26: Review Questions.....	20-50
Module 20: Review Questions.....	20-52
Lab Exercise 20-1	20-54
Lab Exercise 20-2	20-56
Lab Exercise 20-2 (cont.).....	20-58
Lab Exercise 20-2 (cont.).....	20-60
Lab Exercise 20-2 (cont.).....	20-62

The EXPLAIN Facility

Th

e **EXPLAIN facility** is a Teradata extension that provides you with an "English" translation of the steps chosen by the Optimizer to execute a SQL statement. It may be used on any valid Teradata SQL statement simply by prefacing that statement with "EXPLAIN".

The following is an example of how you would EXPLAIN a simple SELECT statement:

```
EXPLAIN SELECT LastName, FirstName FROM Employee;
```

The EXPLAIN facility actually parses the SQL statement but does not execute it. EXPLAIN output provides the physical designer with an "execution strategy". This execution strategy provides direct feedback on what steps the Optimizer chooses to do, but not why it chooses to do them.

Studying EXPLAIN output can be an excellent way to learn more about the inner workings of the Teradata DBS and how it handles SQL.

The EXPLAIN facility should be used to analyze all Joins and complex queries. Using EXPLAIN regularly can save you lots of time and computing resources by pointing out problems with your SQL statements before you actually run them.

The Explain Facility

May be used on any SQL statement, except EXPLAIN itself.

Translates Optimizer output (the execution plan) into English.

- Provides direct feedback on what the system intends to do.

It is a good way to learn about the system and SQL.

Use it consistently to analyze joins, long-running and complex queries.

Time estimates are relative, not absolute.

- Assumes the query will run stand-alone; doesn't take a loaded system into account.
- Time estimates cost formulas based on H/W configuration.

EXPLAIN Facility Output

In addition to the execution strategy, the EXPLAIN facility provides you with measures of how much work will be performed and a cost estimate expressed as time. Spool size and timing figures are estimates and should be used for comparison purposes only.

For example, the spool size estimates are based on either Dynamic Sampling or COLLECTed STATISTICS. Use “timings” as a cost factor. Even if the timings were calibrated to the CPU and DSU, there is no way that the Parser could determine what other jobs would be running when the request was executed. The developer of the EXPLAIN facility assigned arbitrary units of time to various segments of File System code as a "cost" factor. The sum of these costs is reported to the user.

The primary way to help the Optimizer make the best choices and ensure the most accurate EXPLAIN output is to make sure to provide current STATISTICS.

In many cases, there is more than one way to code an SQL statement to get the desired results. They all should be coded and EXPLAINed to find out which version will perform best for your tables and system configuration.

Always do an EXPLAIN before submitting any Join or complex query.

EXPLAIN Facility Output

The timings and spool sizes shown are **ESTIMATES ONLY**.

- Spool sizes are based on dynamic sampling or statistics.
- Use them as “figures of merit” for comparison purposes only.

Know what the Request is supposed to do before EXPLAINing it.

In this module, we will ...

- 1st View some text examples of EXPLAINs.
- 2nd Discuss EXPLAIN terminology that may appear within EXPLAIN output.
- 3rd Discuss row partitioning terminology and view some examples.

EXPLAIN – SELECT with PI value

The example on the facing page illustrates the output generated by EXPLAINing a simple SELECT operation using a WHERE clause and specifying a value for the primary index.

Final output is generally put into Spool 1.

The view used in the SELECT on the facing page is shown below.

```
REPLACE VIEW TFACT.Claim_v AS
LOCKING ROW FOR ACCESS
SELECT *
FROM    TFACT.Claim;
```

The table used in the SELECT on the facing page is shown below.

```
CREATE SET TABLE TFACT.Claim, Fallback, MAP = TD_MAP1
(claimid          INTEGER NOT NULL,
 custid           INTEGER NOT NULL,
 claimstatus      CHAR(1) DEFAULT '0 ',
 settlement       DECIMAL(9,2),
 claimdate        DATE FORMAT 'YYYY-MM-DD' NOT NULL,
 claimpriority    SMALLINT DEFAULT 0,
 claimagent       CHAR(16),
 officeid         SMALLINT DEFAULT 0,
 claimcomment     CHAR(79)
                 DEFAULT 'Medical - Optional Comment completed by Agent')
UNIQUE PRIMARY INDEX (claimid);
```

EXPLAIN – SELECT with PI value

Query

```
EXPLAIN SELECT * FROM Claim_v WHERE claimid = 1293751;
```

Explanation

16.20 EXPLAIN

- 1) First, we do a single-AMP RETRIEVE step from TFACT.Claim in **view Claim_v** by way of the **unique primary index** "TFACT.Claim in view Claim_v.claimid = 1293751" with no residual conditions **locking row for access**. The estimated time for this step is 0.00 seconds.
-> The row is sent directly back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

Notes:

- Statistics were collected on the claimdate column of this table.

EXPLAIN – SELECT (FTS)

The example on the facing page illustrates the output generated by EXPLAINing a simple SELECT operation that does not have a primary index value specified. The Claim table contains 30,000,000 rows and was distributed on an 8-AMP system.

Final output is generally put into Spool 1.

The table used in the SELECT on the facing page is shown below.

```
CREATE SET TABLE TFACT.Claim, FALLBACK, MAP = TD_MAP1
  (claimid          INTEGER NOT NULL,
   custid           INTEGER NOT NULL,
   claimstatus      CHAR(1) DEFAULT '0 ',
   settlement       DECIMAL(9,2),
   claimdate        DATE FORMAT 'YYYY-MM-DD' NOT NULL,
   claimpriority    SMALLINT DEFAULT 0,
   claimagent       CHAR(16),
   officeid         SMALLINT DEFAULT 0,
   claimcomment     CHAR(79)
                   DEFAULT 'Medical - Optional Comment completed by Agent')
UNIQUE PRIMARY INDEX (claimid);
```

EXPLAIN – SELECT (FTS)

Query

```
EXPLAIN SELECT * FROM Claim_v ORDER BY 1;
```

Explanation

16.20 EXPLAIN

- 1) First, we lock TFACT.Claim in **view Claim_v in TD_MAP1 for access**.
- 2) Next, we do an **all-AMPs RETRIEVE** step in TD_MAP1 from TFACT.Claim in view Claim_v by way of an **all-rows scan** with no residual conditions into Spool 1 (group_amps), which is built locally on the AMPs.
Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.Claim in view **Claim_v.claimid**). The size of Spool 1 is estimated with **high confidence** to be 30,000,000 rows (4,290,000,000 bytes). The estimated time for this step is 5 minutes and 20 seconds.
- 3) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 5 minutes and 20 seconds.**

Notes:

- The "Claim_v" view has the locking modifier (LOCKING ROW FOR ACCESS).
- Statistics were collected on the claimdate column of this table.

EXPLAIN Terminology

In general, EXPLAIN text is clear and easy to understand. However, there are a few phrases and terms that you may need to be familiarized with. Here are some definitions that may prove helpful:

" (Last Use) "

You will find this phrase following a reference to a Spool file. It indicates that the Spool file is being used for the last time and will be DELETED at the end of the step, thus releasing that Spool space.

" with no residual conditions "

Residual conditions are those conditions in the WHERE clause not used to locate a row(s), but to further qualify the rows. This phrase indicates there are no such conditions present.

With “residual conditions” indicates that there are remaining conditions to be applied. These conditions are maintained in cache memory.

" END TRANSACTION "

When the END TRANSACTION step is sent, transaction locks are released and changes are committed.

" eliminating duplicate rows "

This indicates that a DISTINCT operation is done to ensure that there are no duplicate rows.

" by way of the sort key in spool field1 (dbname.tabname.colname)"

Field1 is created to allow a tag sort. Teradata 12.0 includes the column name used in the sort.

" we do an ABORT test "

ABORT tests are caused by an ABORT or ROLLBACK statement. If the condition is found to be true, then the ABORT or ROLLBACK is performed.

" by way of a traversal of index #n extracting row ids only "

A spool file is built containing the Row IDs found in a secondary index (index #n)

EXPLAIN Terminology

Most EXPLAIN text is easy to understand. The following additional definitions may help:

- *... (Last Use) ...*

A spool file is no longer needed and will be released when this step completes.

- *... with no residual conditions ...*

All applicable conditions have been applied to the rows.

- *... END TRANSACTION ...*

Transaction locks are released, and changes are committed.

- *... eliminating duplicate rows ...*

Duplicate rows only exist in spool files, not set tables. Doing a DISTINCT operation.

- *... by way of the sort key in spool field1 (dbname.tablename.colname) ...*

Field1 is created to allow a tag sort.

- *... we do an ABORT test ...*

Caused by an ABORT or ROLLBACK statement.

- *... by way of a traversal of index #n extracting row ids only ...*

A spool file is built containing the Row IDs found in a secondary index (index #n).

EXPLAIN Terminology (cont.)

" we do a SMS (set manipulation step) "

The system will combine answer sets using a UNION, EXCEPT (MINUS) or INTERSECT operator.

" we do a BMSMS (bit map set manipulation step) "

NUSI Bit Mapping is being used. NUSI Bit Mapping operations are not as frequently used with later releases of Teradata (e.g., 14.0 and later).

" which is redistributed by hash code to all AMPs "

A step is done because data is being relocated to prepare for a join.

" which is duplicated on all AMPs "

A step is done because data is being duplicated on all AMPs to prepare for a join.

"(one_amp) or (group_amps)"

Indicates one AMP or a subset of AMPs will be used instead of all the AMPs.

"NOT (table_name.column_name IS NULL)"

Feature where optimizer realizes that a nullable column is being referenced in a comparison or join. Such conditions can not evaluate TRUE and are negated to avoid having to participate in the comparison.

EXPLAIN Terminology (cont.)

- *... we do a SMS (set manipulation step) ...*
Combining rows using a UNION, MINUS, or INTERSECT operator.
- *... we do a BMSMS (bit map set manipulation step) ...*
Doing a NUSI Bit Map operation. NUSI Bit Map operations are not frequently used in later releases.
- *... which is redistributed by hash code to all AMPs (dbname.tablename.colname) ...*
Redistributing data (in SPOOL) in preparation for a join.
- *... which is duplicated on all AMPs ...*
Duplicating data (in SPOOL) from the smaller table in preparation for a join.
- *... (one_AMP) or (group_AMPs) ...*
Indicates one AMP or a subset of AMPs will be used instead of all AMPs.
- *... ("NOT (table_name.column_name IS NULL)") ...*
Feature where optimizer realizes that a nullable column is being referenced in a comparison or join. Such conditions can not evaluate TRUE and are negated to avoid having to participate in the comparison.

14.10 EXPLAIN Terminology & Modifiers

"split into ... to qualify ... (PRPD)"

PRPD Partial Redistribution Partial Duplication is a join strategy for dealing with equality joins of skewed tables that Splits the 2 tables in a join into 4 tables, with the first split containing the rows with the skewed values from table_1 locally and their matching rows from table_2 distributed to all AMPs and the second split containing the remaining non-skewed rows from table_1 and table_2. Teradata first joins the rows within each split and then merges them into the final join result.

For a PRPD join strategy, Teradata segregates the rows from the 2 tables into 2 splits. A split in a PRPD join strategy is either a portion of skewed rows from one of the tables and their matching rows from the other table or the remaining non-skewed rows from both tables being equi-joined. Teradata first joins the rows within each split and then merges the result rows from the joins within each split into the final set of joined rows.

When one or both tables in a join contain skewed values, the performance of the join operation is frequently degraded. Skew on certain values refers to the variation in the number of rows that contain those values among the various AMPs. If the variation is high, meaning that some AMPs have a high number of rows with those values and some AMPs have far fewer rows with the values, the values are described as being skewed and the base table is said to be skewed on these values.

"This request is eligible for incremental planning and execution (IPE) but doesn't meet thresholds."

Teradata reports this phrase before the first step of a static plan. This is a specific static plan that is eligible for incremental planning and execution but does not meet certain system-specified cost thresholds. If Teradata executes the request, this static plan is generated and executed.

"The following is the dynamic explain for the request."

This phrase indicates the query plan is a dynamic plan generated by incremental planning and execution (IPE) for this request. This phrase, if included, occurs before the first step of the dynamic plan.

Explain Modifier "STATIC [IN XML] EXPLAIN query"

A query plan that is not generated by IPE is referred to as a static query plan. Static query plans are the query plans that are traditionally returned for an explained request.

Explain Modifier "DYNAMIC [IN XML] EXPLAIN query"

A query plan generated by IPE is referred to as a dynamic query plan. Dynamic query plans are not the query plans that are traditionally returned for an explained request. IPE both provides feedback of statistical information from the execution of a plan fragment and provides feedback from the results of a plan fragment wherever possible.

- *... split into ... to qualify skewed rows ...*
... split into ... to qualify rows matching skewed rows of the skewed relation ...

With Partial Redistribution Partial Duplication (PRPD), a binary join on skewed data is split into 2 or 3 subjoins (called PRPD physical joins). If PRPD plan is picked, the explain text has two splitting steps for every PRPD join plan that writes rows into multiple spools. Splitting happens as part of a retrieve or join step.

- *... This request is eligible for incremental planning and execution (IPE) ...*
... but doesn't meet thresholds

This is a specific static plan that is eligible for incremental planning and execution, but does not meet certain system-specified cost thresholds.

- *... The following is the dynamic explain for the request ...*

This phrase indicates the query plan is a dynamic plan generated by incremental planning and execution (IPE) for this request.

- **STATIC [IN XML] EXPLAIN query**

- Displays static plan
- If eligible for IPE, indicated in explain output

- **DYNAMIC [IN XML] EXPLAIN query**

- Displays dynamic plan if IPE eligible otherwise static plan
- For dynamic plan, executes all but last fragment

15.10 EXPLAIN Terminology

Starting in TD 15.10 a column-partitioned table or join index may have a **primary index** or **primary AMP** index as an alternative to having no primary index. In addition, the TD 15.10 feature Load Isolation provides compatibility for read-committed isolation level that will allow Read operations to “skip over” rows from uncommitted load operations. The facing page has some of the explain terminology you may see related to these features.

15.10 EXPLAIN Terminology

- *... by way of the primary AMP index ...*

Indicates that there are conditions on the primary AMP index columns that allow the system to access only a single AMP for the indicated table..

- *... by way of the [unique] primary index ...*

Indicates that there are conditions on the primary index columns that allow the system to access only a single AMP for the indicated table..

- *... (load committed) ...*

This reads the rows that are load committed. If the read mode is (load uncommitted), it indicates that logically deleted rows are ignored during processing. All other rows are read.

- *... (load isolated) ...*

Indicates that the modification operation performed in the step is load isolated. For this phrase to occur, the table being modified must be a load-isolated table or a join index defined on a load isolated table.

- *... (nonconcurrent load isolated) ...*

Indicates that the modification operation is not load isolated. For this phrase to occur, the table being modified must be a load-isolated table or a join index defined on a load-isolated table.

15.10 EXPLAIN Terminology (cont.)

The TD 15.10 feature, Partition Level Locking (PLL), adds partition locking granularity between table-level and row-level that allows concurrent operations on data in distinct subsets of partitions. See facing page.

Starting in Teradata 15.10 a Proxy locks provide a mechanism for queuing locks to avoid the global deadlocks that can otherwise occur when Teradata places a lock on each of the AMPs in a parallel system.

When you make an all-AMP request for a READ, WRITE, or EXCLUSIVE lock, or for an ACCESS lock on a load-isolated table, the system automatically places a proxy lock on a single AMP before it places the all-AMP lock. Think of the proxy lock as an intention lock, that is, it indicates that the request intends to place a lock on each of the AMPs.

Pseudo table locks are placed on various data dictionary tables at the rowhash-level for the following DDL statements:

- CREATE HASH INDEX
- CREATE JOIN INDEX
- CREATE TABLE
- DROP HASH INDEX
- DROP JOIN INDEX
- DROP MACRO
- DROP PROCEDURE
- DROP TABLE
- DROP VIEW

It would be more accurate to refer to these locks as pseudo rowhash locks rather than rowhash-level pseudo table locks, but the EXPLAIN phrase text does not follow this convention, and this description follows the terminology used by the EXPLAIN phrase text.

.

15.10 EXPLAIN Terminology (cont.)

- *... on a reserved RowHash [in all partitions] [in n partitions] [in a single partition]...*

This phrase indicates that the system has placed a proxy lock on the target table to prevent global deadlock for a later all-AMP lock on the table. The optional phrases "in all partitions," "in n partitions," and "in a single partition" appear only if the table is row-partitioned.

The phrase "in all partitions" indicates a proxy for a table-level lock.

The phrase "in n partitions" indicates a proxy for a partition-range lock.

The phrase "in a single partition" indicates a proxy for a single-partition lock.

- *... we lock a distinct [database_name].pseudo table" for [locking_severity] on a RowHash for deadlock prevention ...*

This phrase indicates the pseudo lock that is used to serialize primary and fallback RowHash locks on a dictionary table for DDL processing.

- *... we lock [database_name].[table.name] for exclusive use on a reserved RowHash to prevent global deadlock ...*

This phrase indicates the proxy lock that serializes the exclusive lock for database_name.table_name in the lock phrase that follows this one.

- *... single writer to serialize concurrent updates on the partition-locked table ...*

This phrase indicates a single-writer lock that allows concurrent reads on the data in a table that is not write locked during bulk DML processes.

16.00 EXPLAIN Terminology

Starting in Teradata 16.00, the **All Rows One AMP Spool for In-Memory Hash Join** feature has introduced a new kind of in-memory hash join where the left table is duplicated to one AMP (AllRowsOneAMP spool). Customers will see Improved I/O and CPU of an in-memory hash join whose left table is duplicated.

In-Memory Hash joins where the left table is duplicated to one AMP. This feature is a continuation of In-Memory Techniques which was released in Teradata 15.10.

In-Memory Hash Join can have In-Memory duplicated spool as the input for building the hash table. Duplication of a spool on all AMPs requires to 1) All AMPs to read the rows 2) All AMPs to send the read rows to all AMPs 3) All AMPs to write all the rows to spool.

This has an IO performance impact since all AMPs have to write all rows to spool.

This feature duplicates the rows only to one AMP, hence improving the I/O performance.

Such a spool is called an AllRowsOneAMP spool. This feature is implemented to improve I/O and CPU of an in-memory hash join whose left table is duplicated.

In-Memory Optimization Techniques for DMLs enables in-memory processing for all the DML statements (like INSERT, UPDATE, DELETE, and MERGE-INTO). Better performance for two common operations on BI queries which are single table qualification (predicate evaluation) and join processing. This is applicable to only queries for which optimizer chooses bulk qualification or in-memory hash join.

In-memory feature went into Teradata 15.10. In-memory processing like Bulk Qualification, In-memory hash join, In-memory optimized spool and Vector processing were disabled for DML operations. This feature enables the above mentioned in-memory processing for DML operations.

In-Memory Optimization Techniques for DMLs (Teradata 16.00) helps improve performance in DMLs.

- For bulk qualification on No Primary Index (NOPI) Column Partitioned (CP) and Primary AMP Index (PAI) columnar tables.
- For in-memory hash joins compared to hash join.

In-Memory Outer Hash Joins (Teradata 16.00): all forms of outer hash joins are supported such as Right Outer Hash Join (ROJ), Left Outer Hash Join (LOJ) and Full Outer Hash Join (FOJ).

This feature will help the performance of Outer Hash joins and is a follow-on feature of In-Memory Techniques feature introduced in Teradata 15.10 where only inner joins were supported.

16.00 EXPLAIN Terminology

- *... Hash table is built from Spool n and is duplicated to all AMPs ...*

This phrase indicates that the step is an AllRowsOneAMP In-Memory hash join step.

- *... BULK RETRIEVE step from n column partitions ...*

Indicates In-Memory Optimization Techniques that helps improve performance in DMLs

- For bulk qualification on No Primary Index (NOPI) Column Partitioned (CP) and Primary AMP Index (PAI) columnar tables.

- *... in-memory hash join ...*

For in-memory hash joins compared to hash join.

- *... outer joined using a in-memory <dynamic> hash join ...*

This indicates an In-Memory Outer Hash Joins is being used.

16.10 EXPLAIN Terminology

Starting in Teradata 16.10, the MAPS feature added the concept of a mapname. Every table is assigned to a map. Therefore, the explain output has ...in **mapname**... in many outputs and facing page has some of them so you can see the terminology.

Also, if you recall a Spare map can be used for small tables and if you use the same colocation label they will go to the same AMP(s).

Sparse map single AMP join from Sizing lab using colocate:

```
EXPLAIN
SELECT      NationName, RegionName
FROM        Nation_S, Region_S
WHERE       Nation_S.RegionKey = Region_S.RegionKey;
```

- 1) First, **we lock STUDENT135.Region_S** in TD_1AmpSparseMap_1Node for **read**, and **we lock STUDENT135.Nation_S** in TD_1AmpSparseMap_1Node for **read**.
 - 2) Next, we do a **single-AMP JOIN** step from **STUDENT135.Region_S** by way of an **all-rows scan**, which is joined to **STUDENT135.Nation_S** by way of an **all-rows scan**. **STUDENT135.Region_S** and **STUDENT135.Nation_S** are joined using a **product join**, with a join condition of ("STUDENT135.Nation_S.RegionKey = STUDENT135.Region_S.RegionKey"). The result goes into **Spool 1** (group_amps), which is **built locally** on that AMP. The size of **Spool 1** is estimated with **index join** confidence to be **24 rows** (984 bytes). The estimated time for this step is 0.03 seconds.
 - 3) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of **Spool 1** are sent back to the user as the result of statement 1. The total estimated time is 0.03 seconds.

16.10 EXPLAIN Terminology

- *... in mapname ...*

This phrase indicates the map defining a collection of AMPs.

- *... which is redistributed by hash code to all AMPs in mapname ...*

Relocating data by hash code in preparation for a join.

- *... which is duplicated on all AMPs in mapname ...*

Relocating data in preparation for a join.

- *... all-AMPs RETRIEVE step in mapname by way of an all-rows scan ...*

This phrase indicates that all rows of a table are scanned row by row on all AMPs in the indicated map on which the table is stored.

- *The following Explain shows a single-AMP collocated join:*

... Next, we do a **single-AMP JOIN** step from ... are joined using a **product join**, ...

16.20 EXPLAIN Terminology

Teradata 16.20, the main feature is Time Series – **Primary Time Index** (PTI) tables.

Below is an example using GROUP BY TIME aggregation on a PTI table. The key things to note in an explain plan is the **TD_TIMEBUCKET** and **TD_TIMECODE**, the Ocean_Buoy table was made with a time bucket duration of 2 hours. Therefore, data falling in the same 2 hour bucket will go to same AMP. In the query below it specifies the time frame of 2 hours (16:00:00 to 17.59.59) so it goes to one AMP looking for the data. A PTI table is also partitioned and this table is partitioned behind the scenes on 1 second partition intervals. The 7200 partitions come from the number of seconds (2hrs * 60min * 60sec), but are not shown in the explain plan. Rather is show “from a time range”.

```
EXPLAIN
SELECT      $TD_GROUP_BY_TIME, $TD_TIMECODE_RANGE,
            AVG(WaterTemp) AS "Avg Water Temp (C)"
FROM        Ocean_Buoy
WHERE       TD_TimeCode BETWEEN
            TIMESTAMP '2016-11-08 16:00:00' AND '2016-11-08
17:59:59'
GROUP BY    TIME( MINUTES(30))
ORDER BY    1;
```

- 1) First, we do a **single-AMP SUM** step to aggregate **from a time range** of **TFACT.Ocean_Buoy** by way of the **primary** index **"TFACT.Ocean_Buoy.TD_TIMEBUCKET = 2937"** with a residual condition of **("(TFACT.Ocean_Buoy.TD_TIMECODE <= TIMESTAMP '2016-11-08 17:59:59') AND (TFACT.Ocean_Buoy.TD_TIMECODE >= TIMESTAMP '2016-11-08 16:00:00')")**, and the grouping identifier in field 1. Aggregate Intermediate Results are computed **locally**, then placed in **Spool 3** in TD_Map1. The size of **Spool 3** is estimated with **no confidence** to be **2 rows** (70 bytes). The estimated time for this step is 0.00 seconds.
- 2) Next, we do a **single-AMP RETRIEVE** step from **Spool 3** (Last Use) by way of the hash value of **"TFACT.Ocean_Buoy.TD_TIMEBUCKET = 2937"** into **Spool 1** (one-amp), which is **built locally** on that AMP. Then we do a SORT to order **Spool 1** by the sort key in spool field1. The size of **Spool 1** is estimated with **no confidence** to be **2 rows** (126 bytes). The estimated time for this step is 0.03 seconds.
- 3) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of **Spool 1** are sent back to the user as the result of statement 1. The total estimated time is 0.03 seconds.

16.20 EXPLAIN Terminology

- *The following Explain shows terminology you might see with **Time Series - Primary Time Index** and a query using **GROUP BY TIME** aggregate:*
 - 1) First, we do a **single-AMP SUM** step to aggregate from a **time range** of **TFACT.Ocean_Buoy** by way of the **primary** index "**TFACT.Ocean_Buoy.TD_TIMEBUCKET = 2937**" with a residual condition of ("**TFACT.Ocean_Buoy.TD_TIMECODE** <= **TIMESTAMP '2016-11-08 17:59:59'**) AND (**TFACT.Ocean_Buoy.TD_TIMECODE** >= **TIMESTAMP '2016-11-08 16:00:00'**)"), and the grouping identifier in field 1. Aggregate Intermediate Results are computed **locally**, ...

Pseudo Table Locks

You can think of a pseudo table as an alias for the physical table it represents. (Note that the word pseudo modifies the table, and not the lock.) Pseudo tables provide a mechanism for queuing data dictionary row hash locks to avoid the global deadlocks that can otherwise occur when Teradata places locks on data dictionary rows.

When DDL needs to lock a rowhash for both the primary and fallback of a dictionary table, the system automatically places a rowhash-level pseudo table lock. Think of a row hash-level pseudo table lock as an intention lock, that is, it indicates that the request intends to place a row hash lock on both the primary and fallback of a dictionary table.

Pseudo table locks enable the sequential locking of row hashes on both primary and fallback rows that are on different AMPs in a parallel database architecture. Without pseudo table locking, if multiple users simultaneously submit DDL requests that need to place a rowhash lock on both the primary and the fallback of a data dictionary table, a deadlock can occur. If multiple requests are sent in parallel and need to lock the same rowhash of a dictionary table, they are likely to arrive at the primary and fallback AMPs holding the rowhash in different sequential orders. One request may obtain the rowhash lock on the primary AMP first and the other on a fallback AMP first, thus creating a deadlock.

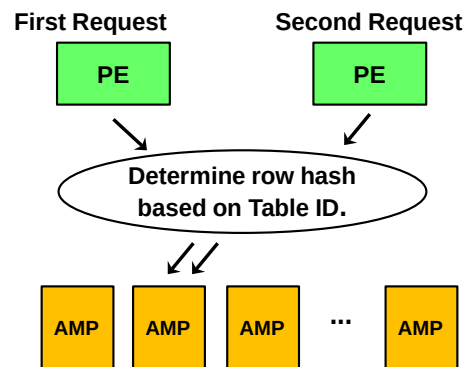
When you use an all AMP request for a read, write, or exclusive lock, the system goes through pseudo table locking. With pseudo table locking:

- Each table has a table id hash code.
- Table id hash codes are assigned to the AMPs.
- Each AMP becomes a “gate keeper” to the tables for which it is assigned.

Note: Starting in Teradata 15.10, because of the feature PLL, a proxy lock is used and pseudo table lock is used on data dictionary tables for DDL operations. (See previous left hand page for more information on Proxy and Pseudo locks).

Pseudo Table Locks

- Internal function to synchronize locks across AMPs for DDL requests (change in 15.10).
 - Pseudo tables provide a mechanism for queuing data dictionary rowhash locks to avoid the global deadlocks when DDL statements (e.g., DROP TABLE) are executed.
 - For example, multiple users might attempt to access or a DROP a table, so it is necessary to coordinate them using a proxy lock, that is, locking a reserved rowhash.
 - This effectively results in sequenced locking of a table for DDL requests.
- All-AMP lock requests are handled as follows:
 - PE determines a rowhash (on TableID) for an AMP to manage the pseudo table lock request.
 - Place pseudo lock on the table.
 - Acquire lock on all AMPs.



Understanding Row and Time Estimates (Part 1)

The facing page identifies some of the “confidence” phrases and their meanings that you will find in the EXPLAIN output for a data retrieval.

The EXPLAIN facility may express “confidence” for a retrieve from a table. Some of the phrases used are:

... with high confidence ...

- Restricting conditions exist on index(es) or column(s) that have collected statistics.

... with low confidence ...

- Restricting conditions exist on index(es) or column(s) having no statistics, but estimates can be based upon a dynamic or random AMP sampling.
- Restricting conditions exist on index(es) or column(s) that have collected statistics but are “AND-ed” together with conditions on non-indexed columns.
- Restricting conditions exist on index(es) or column(s) that have collected statistics but are “OR-ed” together with other conditions.

... with no confidence ...

- Conditions outside the above.

For a retrieve from a spool, the confidence is the same as the step generating the spool.

Understanding Row and Time Estimates (Part 2)

The facing page identifies some of the “confidence” phrases and their meanings that you will find in the EXPLAIN output for a join.

It is possible to get Index Join Confidence in the Explain output for both NUPI and UPI indexes and whether statistics have been collected or not. If the Explain output indicates that one set is joined to another set that has a Primary Index, then you may see Index Join Confidence.

Explain plans actually specify numeric values for low-end times, rows, and bytes in the EXPLAIN output.

Miscellaneous Notes:

- Estimates too large to display show 3 asterisks (***)
- High-end row and high-end time estimates have been removed starting with V2R5.
- The accuracy of the time estimate depends upon the accuracy of the row estimate.
- Actual performance can be hindered by current workload.

The following are “confidence” phrases for a join:

... with index join confidence ...

- A join condition via a primary index.

... with high confidence ...

- One input relation has high confidence and the other has high or index join confidence.

... with low confidence ...

- One input relation has low confidence and the other has low, high, or join index confidence.

... with no confidence ...

- One input relation has no confidence.
- Statistics do not exist for either join field.

Notes:

- Low and no confidence may indicate a need to collect statistics on indexes or columns involved in restricting conditions.
- You may otherwise consider a closer examination of the conditions in the query for possible changes that may improve the confidence.
- Explain plans show “low-end” time, rows, and/or bytes associated with the step.
 - Estimates too large to display show 3 asterisks (***)

EXPLAIN – Create Table

The example on the facing page shows how Teradata goes about creating a new table. A new table named **New_Orders** is created in the TFACT database. The orders table consists of three columns:

- orderid (has a Unique Primary Index)
- orderdate
- custid

16.20 EXPLAIN

- > No rows are returned to the user as the result of statement 1.

EXPLAIN Macro

The example on the facing page illustrates an EXPLAIN statement that specifies a parameter value of a specific date which will be treated as a hard-coded literal value. In this case, the Optimizer will choose an execution plan based on this date, which may not accurately represent production which can use any date.

EXPLAIN Macro

Queries

```
CREATE MACRO Claim_by_Date_m (date_value DATE) AS
(
    SELECT * FROM Claim
    WHERE claimdate = :date_value
    ORDER BY 1;
);
```

```
EXPLAIN EXEC Claim_by_Date_m (DATE '2019-08-24');
```

Notes:

- This creates a Request parcel, but no DATA parcel.
- The parameter value is treated as a hard-coded literal value.

Explanation

EXPLAIN 16.20

- 1) First, we lock TFACT.Claim in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) Next, we lock TFACT.Claim in TD_MAP1 for read.
- 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.Claim by way of an all-rows scan with a condition of ("TFACT.Claim.claimdate = DATE '2019-08-24'") into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.Claim.claimid). **The size of Spool 1 is estimated with high confidence to be 17,325 rows (2,477,475 bytes).** The estimated time for this step is 16.14 seconds.
- 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 16.14 seconds.

EXPLAIN Terminology for Partitioned Tables

When the phrase **“a single partition of”** or **“n partitions of”** is included in the output of an EXPLAIN, it means that partition elimination will occur.

If a query has a range constraint on the partitioning columns in a table with a range-partitioned primary index, an all-AMP row scan ...

- starts at the partition spanning the low end of the range and
- stops at the partition spanning the high end of the range.

Partition elimination can occur for SELECTs, UPDATE, and DELETEs.

- For a DELETE, Optimizer recognizes partitions for which all rows are being deleted and rows in such partitions are deleted without using the transient journal.
- Optimization only performed if DELETE is an implicit transaction or is the last statement in a transaction.
- Similar to DELETE ALL except for a partition.

“SORT to partition Spool m by rowkey”

- Indicates that the optimizer determined that a spool file is to be partitioned based on the same partitioning expression as a table to which the spool file is be joined.
 - That is, the spool is to be sorted by rowkey (partition and hash).
- Partitioning the spool file in this way allows for a faster join with the partitioned table.

“a rowkey-based”

- Indicates an equality join on the rowkey.
 - In this case, there are equality constraints on the partitioning columns and primary index columns.
 - This allows for a faster join since each non-eliminated partition needs to be joined with at most only one other partition.
- If the phrase is not given, the join is hash based.
 - That is, there are equality constraints on the primary index columns from which the hash is derived.
 - For a partitioned table, there is some additional overhead in processing the table in hash order.
- Note that with either method, the join conditions must still be validated.

The concept of Dynamic Partition Elimination (DPE) is applied when there are join conditions (instead of single table constraints) on the partitioning column/columns. The partition list that DPE uses depends on the data. The list, called a dynamic partition list, is generated at runtime. This capability has been continuously enhanced in new Teradata releases.

EXPLAIN Terminology for Partitioned Tables

"a single partition of" or "*n* partitions of"

- Indicates that an AMP or AMPs only need to access a single partition or *n* partitions of a table – indicates partition elimination occurred.
- Partition elimination can occur for SELECTs, UPDATE, and DELETEs.
 - For a DELETE, Optimizer recognizes partitions in which all rows are deleted.
 - Rows in such partitions are deleted without using the transient journal.

"SORT to partition Spool *m* by rowkey"

- The spool is to be sorted by rowkey (partition and hash).
- Partitioning the spool file in this way allows for a faster join with the partitioned table.

"a rowkey-based"

- Indicates an equality join on the rowkey.
- In this case, there are equality constraints on the partitioning columns and primary index columns.

"enhanced by dynamic partition ..."

- Indicates a join condition where dynamic partition elimination has been used.

EXPLAIN – Partition Elimination

Both Claim_RP and Claim tables have 30,000,000 rows. Statistics for custid and claimdate were collected on both tables. In addition, claimid and PARTITION statistics were collected on the Claim_RP table.

The EXPLAIN text for these queries is shown below.

```
EXPLAIN  SELECT  *
          FROM    Claim_RP
          WHERE    claimdate = DATE '2019-08-24';
```

- 1) First, we lock TFACT.Claim_RP in TD_MAP1 for read on a reserved RowHash in a single partition to prevent global deadlock.
 - 2) Next, **we lock TFACT.Claim_RP in TD_MAP1 for read on a single partition.**
 - 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from a single partition of TFACT.Claim_RP with a condition of ("TFACT.Claim_RP.claimdate = DATE '2019-08-24'") with a residual condition of ("TFACT.Claim_RP.claimdate = DATE '2019-08-24'") into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 17,325 rows (2,477,475 bytes). The estimated time for this step is 0.04 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.04 seconds.**

The table named Claim is similar to Claim_RP except it does not have a Partitioned Primary Index, but does have "claimid" as a UPI.

```
EXPLAIN  SELECT  *
          FROM    Claim
          WHERE    claimdate = DATE '2019-08-24';
```

- 1) First, we lock TFACT.Claim in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, **we lock TFACT.Claim in TD_MAP1 for read.**
 - 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.Claim by way of an all-rows scan with a condition of ("TFACT.Claim.claimdate = DATE '2019-08-24'") into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 17,325 rows (2,477,475 bytes). The estimated time for this step is 16.11 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 16.11 seconds.**

EXPLAIN – Partition Elimination

```
Query      EXPLAIN  SELECT  *
           FROM    Claim_RP
           WHERE    claimdate = DATE '2019-08-24';
```

Explanation

16.20 EXPLAIN

- 1) First, we lock TFACT.Claim_RP in TD_MAP1 for read on a reserved RowHash in a single partition to prevent global deadlock.
- 2) Next, we lock TFACT.Claim_RP in TD_MAP1 for read on a single partition.
- 3) We do an **all-AMPs RETRIEVE step in TD_MAP1 from a single partition** of TFACT.Claim_RP with a condition of ("TFACT.Claim_RP.claimdate = DATE '2019-08-24'") with a residual condition of ("TFACT.Claim_RP.claimdate = DATE '2019-08-24'") into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 17,325 rows (2,477,475 bytes). The estimated time for this step is 0.04 seconds.
- 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.04 seconds.**

Notes:

- The Claim table was partitioned on claimdate with daily partitions.
- A partition level lock is used (15.10 feature).
- The EXPLAIN text (for this SQL statement) for a non-partitioned Claim table is shown on the facing page in the PDF file. **The estimated time is 16.11 seconds.**

EXPLAIN – Primary Index Access of Partitioned Table

The complete EXPLAIN example of accessing a partitioned table via the PI is shown on the facing page. The example below uses a table named Claim which is similar to Claim_RP except it does not have a Partitioned Primary Index, but does have “claimid” as a UPI.

```
EXPLAIN  SELECT *
          FROM   Claim
          WHERE  claimid = 1293751;
```

- 1) First, we do a **single-AMP RETRIEVE step** from TFACT.Claim by way of the unique primary index "TFACT.Claim.claimid = 1293751" with no residual conditions. The estimated time for this step is 0.00 seconds.
- > The row is sent directly back to the user as the result of statement 1. **The total estimated time is 0.00 seconds.**

Create a USI or NUSI on the PI for a Partitioned Table

If the partitioning columns are not part of the Primary Index, the Primary Index cannot be unique (e.g., claimdate). To maintain uniqueness on the Primary Index, you can create a USI on the PI (e.g., Claim ID or claimid). Another option is to create a NUSI on the PI column or columns. Access to specific rows via a USI or NUSI will be faster than scanning multiple partitions on a single AMP.

Reasons for creating a USI instead of a NUSI may include:

- Maintain uniqueness in the column via the USI.
- Establish the USI as a referenced parent in Referential Integrity

As an alternative, the SELECT can include the Primary Index values and the partitioning information. If you knew that the claimid of 1293751 occurred on 2019-01-13, you could include both claimid and claimdate in the query. This allows the PE to build a request that has the AMP scan a specific partition.

```
EXPLAIN  SELECT *
          FROM   Claim_RP
          WHERE  claimid = 1293751
          AND    claimdate = DATE '2019-01-13';
```

- 1) First, we do a single-AMP RETRIEVE step from a single partition of TFACT.Claim_RP by way of the primary index "TFACT.Claim_RP.claimid = 1293751, TFACT.Claim_RP.claimdate = DATE '2019-01-13'" with a residual condition of ("(TFACT.Claim_RP.claimid = 1293751) AND (TFACT.Claim_RP.claimdate = DATE '2019-01-13')") into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with low confidence to be 1 row (143 bytes). The estimated time for this step is 0.00 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

EXPLAIN – Primary Index Access of a Partitioned Table

Query EXPLAIN SELECT * FROM Claim_RP WHERE claimid = 1293751;

Explanation

16.20 EXPLAIN

- 1) First, we do a **single-AMP RETRIEVE step from all partitions** of TFACT.Claim_RP by way of the primary index "TFACT.Claim_RP.claimid = 1293751" with a residual condition of ("TFACT.Claim_RP.claimid = 1293751") into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with **high confidence** to be 1 row (143 bytes). The estimated time for this step is 0.82 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.82 seconds.**

Note: If partitioning information isn't provided, all partitions have to be checked for the Row Hash. A USI or a NUSI can optionally be placed on the Primary Index of a PPI table. The following example shows the use of a USI.

Query EXPLAIN SELECT * FROM Claim_RP WHERE claimid = 1293751;

Explanation (with a USI)

16.20 EXPLAIN

- 1) First, we do a **two-AMP RETRIEVE step** in TD_MAP1 from TFACT.Claim_RP by way of **unique index # 4** "TFACT.Claim_RP.claimid = 1293751" with no residual conditions. The estimated time for this step is 0.00 seconds.
- > The row is sent directly back to the user as the result of statement 1. **The total estimated time is 0.00 seconds.**

EXPLAIN – Dynamic Partition Elimination

Dynamic Partition Elimination (DPE) can be applied when there are join conditions (instead of single table constraints) on the partitioning column/columns. The partition list that DPE uses depends on the data in the columns being joined. The list, called a dynamic partition list, is generated at runtime by the AMPs.

This feature (DPE) has continued been in subsequent releases of Teradata software. Collecting statistics on the column PARTITION provides additional information to the optimizer and the optimizer is more likely to consider and use DPE in query plans.

The format of the “Claim_Date” table used in this simple join is:

```
CREATE SET TABLE TFACT.Claim_Date, MAP=TD_1AmpSparseMap_1Node  
  (claimdate DATE FORMAT 'YYYY-MM-DD' NOT NULL)  
PRIMARY INDEX (claimdate);
```

Ten claimdates were placed into this Claim_Date table.

```
EXPLAIN  SELECT  C.*  
          FROM    Claim_RP C, Claim_Date D  
          WHERE   C.claimdate = D.claimdate;
```

- 1) First, we lock TFACT.D for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, we lock TFACT.C for read on a reserved RowHash in all partitions to prevent global deadlock.
 - 3) We do an all-AMPs LOCK step in the covered map TD_Map1. We lock TFACT.D in TD_1AmpSparseMap_1Node for read, and we lock TFACT.C in TD_MAP1 for read.
 - 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.D by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), which is duplicated on all AMPs in TD_MAP1. Then we do a SORT to partition by rowkey. The size of Spool 2 is estimated with high confidence to be 160 rows (2,720 bytes). The estimated time for this step is 0.03 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from Spool 2 (Last Use) by way of an all-rows scan, which is joined to TFACT.C by way of an all-rows scan with no residual conditions. Spool 2 and TFACT.C are joined using a product join, with a join condition of "TFACT.C.claimdate = claimdate") enhanced by dynamic partition elimination. The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with low confidence to be 82,873 rows (11,850,839). The estimated time for this step is 0.25 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.28 seconds.

EXPLAIN – Dynamic Partition Elimination

What is Dynamic Partition Elimination (DPE)?

This feature is applied when there are [join conditions on the partitioning column/columns](#). The partition list that DPE uses depends on the data. The dynamic partition list is [generated dynamically at execution time by the AMPs](#).

In this example, the Claim_Date table has a small number of dates (10 rows) and is assigned to a single AMP sparse hash map, e.g., [MAP=TD_1AmpSparseMap_1Node](#).

```
Query EXPLAIN      SELECT  C.*
                   FROM    Claim_RP C, Claim_Date D
                   WHERE    C.claimdate = D.claimdate;
```

Explanation

16.20 EXPLAIN

:

- 4) We do an all-AMPs RETRIEVE step from [TFACT.D](#) by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), [which is duplicated on all AMPs in TD_MAP1](#). Then we do a SORT to [partition by rowkey](#). The size of Spool 2 is estimated with high confidence to be 160 rows (2,720 bytes). The estimated time for this step is 0.03 seconds.
- 5) We do an [all-AMPs JOIN step in TD_MAP1](#) from Spool 2 (Last Use) by way of an all-rows scan, which is joined to TFACT.C by way of an all-rows scan with no residual conditions. [Spool 2 and TFACT.C are joined using a product join](#), with a join condition of "TFACT.C.claimdate = claimdate") [enhanced by dynamic partition elimination](#). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with low confidence to be 82,873 rows (11,850,839 bytes). The estimated time for this step is 0.25 seconds.

:

EXPLAIN – Delete Rows in Partitioned Table

The facing page contains two EXPLAIN plans comparing deleting rows in a partitioned table as compared to a non-partitioned table.

EXPLAIN – Delete Rows in Partitioned Table

If you delete entire partitions of a row partitioned table and the table does not have secondary indexes or join indexes, Teradata can quickly delete the rows in the defined partitions. This is referred to as a fast path delete.

QUERY (delete 2017 claims from a row partitioned Claim table)

```
EXPLAIN
DELETE Claim_RP
WHERE claimdate BETWEEN DATE '2017-01-01' AND DATE '2017-12-31';
```

- 3) We do an **all-AMPs DELETE step in TD_MAP1** of 365 partitions from **TFACT.Claim_RP** with a condition of **("(TFACT.Claim_RP.claimdate <= DATE '2017-12-31') AND (TFACT.Claim_RP.claimdate >= DATE '2017-01-01')")**. The size is estimated with high confidence to be 1 row. The estimated time for this step is **0.00 seconds**.

QUERY (delete 2017 claims from a non-partitioned Claim table)

```
EXPLAIN
DELETE Claim
WHERE claimdate BETWEEN DATE '2017-01-01' AND DATE '2017-12-31';
```

- 3) We do an **all-AMPs DELETE step in TD_MAP1** from **TFACT.Claim** by way of an **all-rows scan** with a condition of **("(TFACT.Claim.claimdate <= DATE '2017-12-31') AND (TFACT.Claim.claimdate >= DATE '2017-01-01')")**. The size is estimated with high confidence to be 6,924,411 rows. The estimated time for this step is **3 minutes and 37 seconds**.

Summary

EXPLAIN output can be of great value to you. Here are some suggestions on how you can make the most of the EXPLAIN facility:

- Make EXPLAIN output an integral part of all design review and formal system documentation.
- Use EXPLAIN results to expose inefficiencies in query structures.
- Rerun EXPLAINS to see if the strategy chosen by the Optimizer has changed.
- This is done because data demographics change overtime.
- Retain EXPLAIN listings to facilitate periodic index re-evaluation.
- Keep formal documentation of the query structure rationale.

Summary

- Make EXPLAIN output an integral part of all design reviews and formal system documentation.
- EXPLAIN results can expose inefficiencies in query structures.
- Data Demographics change over time.
- Retain EXPLAIN listings to facilitate periodic index re-evaluation.
- Keep formal documentation of the query structure rationale.
- **Know what the Request is supposed to do before EXPLAINing it.**

Module 26: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 26: Review Questions

Fill in the blanks.

1. _____ steps are multi-AMP processing steps that are numbered but execute at the same time.
2. The primary way to help the Optimizer make the best choices and ensure the most accurate EXPLAIN output is to _____ on appropriate indexes and columns.
3. An EXPLAIN plan will indicate “estimated with _____ confidence” when a value for an index is provided to retrieve the data and the index has collected statistics.

Module 20: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 20: Review Questions

Fill in the blanks.

1. _____ steps are multi-AMP processing steps that are numbered but execute at the same time.
2. The primary way to help the Optimizer make the best choices and ensure the most accurate EXPLAIN output is to _____ on appropriate indexes and columns.
3. An EXPLAIN plan will indicate “estimated with _____ confidence” when a value for an index is provided to retrieve the data and the index has collected statistics.

Lab Exercise 20-1

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Your Orders_NoPI table should have 450,000 rows from a previous exercise. If not, you can populate this table using the DS.Orders and DS.Orders_2019 tables.

SQL hint:

```
COLLECT STATISTICS COLUMN column_name [, COLUMN column_name]  
ON table_name;
```


Lab Exercise 20-1

Lab Exercise 20-1

Purpose

In this lab, you will use the COLLECT STATISTICS command.

What you need

Orders_NoPI table and populated DS.Orders and DS.Orders_2019 tables.

Tasks

1. Orders_NoPI should have 450,000 rows from a previous lab exercise. Verify the row count.
2. Collect statistics individually on the columns custid and orderdate of the Orders_NoPI table. Use the HELP STATISTICS and the HELP CURRENT STATISTICS commands on this table.

What is the row count (*) for this table? _____

How many unique values are there for custid? _____

How many unique values are there for orderdate? _____

Are the values different for HELP STATISTICS and HELP CURRENT STATISTICS? _____

3. Insert the rows from the DS.Orders table into your Orders_NoPI table. Use the HELP CURRENT STATISTICS command to answer the following questions.

What is the row count (*) for this table? _____

How many unique values are there for custid? _____

How many unique values are there for orderdate? _____

Why has the row count and number of orderdates increased with current statistics?

Lab Exercise 20-2

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hint:

```
COLLECT STATISTICS COLUMN column_name [, COLUMN column_name]  
ON table_name;
```

If your Orders and Orders_RP tables have a different number of rows, this is probably because Orders only has 9 years of data and Orders_RP has 10 years of data. To ensure the two tables have a similar number of rows, you can do either of the following.

One option is to copy the rows from Orders_2019 into the Orders table.

```
INSERT INTO Orders SELECT * FROM Orders_2019;
```

Another option is to delete all the rows in Orders table and populate the Orders table from Orders_RP.

```
DELETE Orders;  
INSERT INTO Orders SELECT * FROM Orders_RP;
```

Lab Exercise 20-2

Lab Exercise 20-2

Purpose

In this lab, you will EXPLAIN various SQL access functions.

What you need

Populated tables from a previous lab. Execute the following INSERT-SELECT so both the Orders and Orders_RP tables are similar in size.

```
INSERT INTO Orders SELECT * FROM Orders_2019;
```

Tasks

1. Collect statistics on orderid and orderdate for the Orders and Orders_RP tables. Collect statistics on PARTITION for the Orders_RP table. Execute the following two Explains.

```
EXPLAIN SELECT * FROM Orders  
WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How many AMPs are accessed in this operation? _____

Is this a full table scan? _____ What type of lock is used? _____

```
EXPLAIN SELECT * FROM Orders_RP  
WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How many AMPs are accessed in this operation? _____

Is this a full table scan? _____ What type of lock is used? _____

Is partitioning beneficial for this type of query? _____

Lab Exercise 20-2 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Lab Exercise 20-2 (cont.)

2. Execute the following three Explains.

```
EXPLAIN SELECT * FROM Orders
      WHERE orderid = 1404662;
```

How many AMPs are accessed in this operation? _____

How is this row retrieved by Teradata? _____

```
EXPLAIN SELECT * FROM Orders_RP
      WHERE orderid = 1404662;
```

How many AMPs are accessed in this operation? _____

How many partitions are scanned to locate this row? _____

How is this row retrieved by Teradata? _____

```
EXPLAIN SELECT * FROM Orders_NoPI
      WHERE orderid = 1404662;
```

How many AMPs are accessed in this operation? _____

How is this row retrieved by Teradata? _____

Lab Exercise 20-2 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Lab Exercise 20-2 (cont.)

3. Execute the following two Explains.

```
EXPLAIN DELETE FROM Orders
      WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-12-31';
```

How many AMPs are accessed in this operation? _____

Is this a full table scan? _____ Why or why not? _____

```
EXPLAIN DELETE FROM Orders_RP
      WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-12-31';
```

How many AMPs are accessed in this operation? _____

Is this a full table scan? _____ Why or why not? _____

How many partitions are scanned to delete these rows? _____

4. Collect statistics on orderid for the Orders_CP table and execute the following two Explains.

```
EXPLAIN SELECT orderid, custid FROM Orders_CP;
```

How many column partitions are accessed? _____

What is the relative cost for this EXPLAIN? _____

```
EXPLAIN SELECT orderid, custid, orderstatus, orderdate FROM Orders_CP;
```

How many column partitions are accessed? _____

What is the relative cost for this EXPLAIN? _____

Why is there an extra partition accessed for each of these explains? _____

Lab Exercise 20-2 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Lab Exercise 20-2 (cont.)

4. (cont.) Last of the three Explain plans.

```
EXPLAIN SELECT * FROM Orders_CPPI WHERE custid = 1024;
```

How many AMPs are involved in this query? _____

How many partitions are involved in this query? _____

Is this number equal to the number of columns in the table? _____

Why or why not? _____

Notes



Join Processing Analysis

After completing this module, you will be able to:

- **Identify and describe different kinds of join plans.**
- **Describe join plan strategies.**
 - Merge Join
 - Nested Join
 - Hash Join
 - Product Join
 - Exclusion Merge Join
- **Describe different types of join strategies that may be used with row partitioned tables.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Join Syntax	21-4
Inner Join Examples – ON and WHERE	21-6
LEFT Outer Join Example	21-8
RIGHT Outer Join Example.....	21-10
FULL Outer Join Example.....	21-12
Join Processing.....	21-14
Teradata PRPD.....	21-14
Optimizer Minimizes Spool Usage	21-16
Row Selection	21-18
Join Redistribution	21-20
The Join column(s) is the Primary Indexes of Both Tables	21-20
The Join column is the Primary Index of one of the tables.....	21-20
Join Redistribution (cont.).....	21-22
The Join column is the Primary Index of neither table	21-22
Duplicating a Table in Spool	21-24
General Join Distribution Strategies	21-26
Merge Join.....	21-28
Merge Join Strategy	21-30
Merge Join – Matching Primary Indexes	21-32
Merge Join – Row Redistribution	21-34
Merge Join – Duplicate the Smaller Table.....	21-36
Nested Joins	21-38
Product Join.....	21-40
Cartesian Product	21-42
Product Join – Duplicate the Smaller Table.....	21-44
Hash Join.....	21-46
Exclusion Joins	21-48
Exclusion Join Example.....	21-50
Inclusion Joins.....	21-52
n-Table Joins	21-54
Join Considerations with Partitioned Tables.....	21-56
Non-partitioned to Partitioned Table Join (Few Partitions).....	21-58
Non-partitioned to Partitioned Table Join (Many Partitions)	21-60
Non-partitioned to Partitioned Table Join (Sliding Window).....	21-62
Non-partitioned to Partitioned Table Join (Sliding Window – cont.).....	21-64
Non-partitioned to Partitioned Table Join (Hash Ordered Spool File Join).....	21-66
Partitioned to Partitioned Table Join (Rowkey Match Scan).....	21-68
Partitioned to Partitioned Table Join (Unmatched Partitions)	21-70
Additional Join Options with Partitioned Tables	21-72
Join Processing Summary	21-74
Module 21: Review Questions	21-76
Module 21: Review Questions (cont.)	21-78

Join Syntax

The facing page shows all syntax options for joins using the ANSI standard join conventions.

Join Syntax

Teradata supports the ANSI join syntax which provides outer join options.

```
SELECT  cname [, cname , ...]
FROM    tname [aname]
        [INNER] JOIN
        LEFT [OUTER] JOIN
        RIGHT [OUTER] JOIN
        FULL [OUTER] JOIN
        CROSS JOIN
        tname [aname]
        ON condition ;
```

- A two-table join condition references two different tables.
- A self-join condition references two alias names for one table.

Where:

cname	Column or expression name
tname	Table or view name
aname	Alias for table or view name
condition	Criteria for the join

INNER JOIN	All matching rows.
LEFT OUTER JOIN	Table to the left is used to qualify, table on the right has nulls when rows do not match.
RIGHT OUTER JOIN	Table to the right is used to qualify, table on the left has nulls when rows do not match.
FULL OUTER JOIN	Both tables are used to qualify and extended with nulls.
CROSS JOIN	Product join or Cartesian product join.

Inner Join Examples – ON and WHERE

An Inner Join (as shown on the facing page) returns an output row for each successful match between the join tables.

The facing page illustrates the difference between the ON and WHERE join syntax. Both syntaxes (ON and WHERE) are ANSI compatible.

Notes about this Inner Join include:

- Information about employees and their department names where the employee's department number matches the existing departments.
- No information about employees who have no department number or an invalid department number.
- No information is returned about departments which have no employees assigned to them.

Inner Join Examples – ON and WHERE

ON Join
Syntax
(recommended)

```
SELECT      D.DepartmentNumber  AS "Dept Number"
            ,D.DepartmentName    AS "Dept Name"
            ,E.LastName          AS "Last Name"
            ,E.DepartmentNumber  AS "Emp Dept Number"
FROM        Department D
INNER JOIN  Employee E
ON          E.DepartmentNumber = D.DepartmentNumber;
```

WHERE Join
Syntax

```
SELECT      D.DepartmentNumber  AS "Dept Number"
            ,D.DepartmentName    AS "Dept Name"
            ,E.LastName          AS "Last Name"
            ,E.DepartmentNumber  AS "Emp Dept Number"
FROM        Department D
            ,Employee E
WHERE       E.DepartmentNumber = D.DepartmentNumber;
```

Output is
same from
either syntax.

<u>Dept Number</u>	<u>Dept Name</u>	<u>Last Name</u>	<u>Emp Dept Number</u>
402	software support	Crane	402
100	executive	Trainer	100
501	marketing sales	Runyon	501
301	research and development	Stein	301
301	research and development	Kanieski	301

LEFT Outer Join Example

A **left outer join** produces both rows which match in both tables and rows contained in one of the two tables which do not match. The keyword **LEFT** indicates that the table syntactically to the left of the join operator will be the “driver” table, that is, all rows of this table will be seen, whether they match or not.

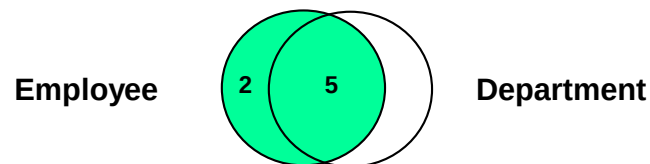
LEFT Outer Join Example

```
SELECT      E.LastName           AS "Last Name"
            , E.DepartmentNumber AS "Dept Number"
            , D.DepartmentName   AS "Dept Name"
FROM        Employee      E LEFT OUTER JOIN
            Department     D
ON          E.DepartmentNumber = D.DepartmentNumber
ORDER BY 1;
```

<u>Last Name</u>	<u>Dept Number</u>	<u>Dept Name</u>
Green	?	?
James	111	?
Crane	402	software support
Kanieski	301	research and develop
Runyon	501	marketing and sales
Stein	301	research and develop
Trainer	100	executive

In addition to output from inner join:

- Shows employees with null departments.
- Shows employees with invalid departments.



RIGHT Outer Join Example

A **right outer join** returns both rows that match and those from one table which do not.

The RIGHT keyword indicates that the table located syntactically to the right of the join operator is the “driver” table, i.e., all rows of that table will be returned whether or not they match.

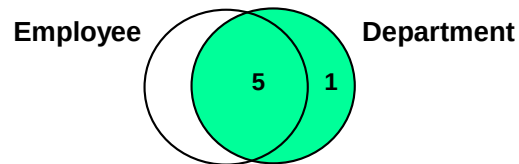
RIGHT Outer Join Example

```
SELECT      D.DepartmentNumber      AS "Dept Number"
            ,D.DepartmentName        AS "Dept Name"
            ,E.LastName              AS "Last Name"
FROM        Employee      E  RIGHT OUTER JOIN
            Department    D
ON          E.DepartmentNumber = D.DepartmentNumber
ORDER BY 1;
```

<u>Dept Number</u>	<u>Dept Name</u>	<u>Last Name</u>
100	executive	Trainer
301	research and develop	Stein
301	research and develop	Kanieski
402	software support	Crane
501	marketing sales	Runyon
600	new department	?

In addition to output from inner join:

- Shows departments with no employees.



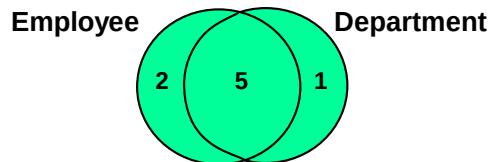
FULL Outer Join Example

Outer join syntax provides the capability of showing not only rows which match, but also those which do not. Using outer joins allows us to see rows with invalid or null entries where there would normally be a match.

FULL Outer Join Example

```
SELECT      D.DepartmentNumber      AS "Dept Number"
            ,D.DepartmentName        AS "Dept Name"
            ,E.LastName              AS "Last Name"
            ,E.DepartmentNumber      AS "Emp Dept Number"
FROM        Employee      E      FULL OUTER JOIN
            Department      D
ON          E.DepartmentNumber = D.DepartmentNumber
ORDER BY   1;
```

<u>Dept Number</u>	<u>Dept Name</u>	<u>Last Name</u>	<u>Emp Dept Number</u>
?	?	Green	?
?	?	James	111
100	executive	Trainer	100
301	research and development	Stein	301
301	research and development	Kanieski	301
402	software support	Crane	402
501	marketing sales	Runyon	501
600	new department	?	?



In addition to output from inner join:

- Shows employees with null departments.
- Shows employees with invalid departments.
- Shows departments with no employees.

Join Processing

For Teradata to process a Join, the rows to be joined must be on the same AMP. Copies of some or all of the rows may have to be moved to a common AMP. (The original rows are never moved.)

Teradata uses several kinds of join plans:

- Product Joins
- Merge Joins
- Nested Joins
- Exclusion (Merge or Product) Joins

No matter what type of Join is being done, the Optimizer will choose the best strategy based upon indexes and demographics (as it does with any SQL request). Use the EXPLAIN facility to be sure what type of Join will occur.

Teradata PRPD

Partial Redistribution Partial Duplication (PRPD) is a new hybrid join strategy starting with Teradata 14.10 for joins with equality join conditions where one or both sources are skewed. It provides an effective “divide-and-conquer” way to reduce the impact of data skew on query and system performance. PRPD avoids “**hot AMP**” situations and improves query performance as well as the overall system performance.

PRPD divides the two sources into subsets based on skewed values extracted from statistics histograms, performs several (2 or 3) regular joins between those subsets, and combines the join results of the subsets into a common spool.

- 2-fold plan: If only one source is skewed, both sources are divided/split into two subsets and 2 regular joins are performed.
- 3-fold plan: If both sources are skewed on different values, both of them are divided/split into three subsets and 3 regular joins are performed.

Join Processing

Rows must be on the same AMP to be joined.

- If necessary, the system creates spool copies of one or both rows and moves them to a common AMP.
- Join processing NEVER moves or changes the original table rows.

The Optimizer chooses the best join strategy based on:

- Available Indexes
- Demographics (COLLECTed STATISTICS or Random AMP Sample)
- If data is skewed, Teradata 14.10 can use a new hybrid join strategy called **Partial Redistribution Partial Duplication (PRPD)**.
- EXPLAIN shows what kind of join a query uses.

Typical Relational (or SQL) join types:

- Inner
- Outer
 - Left, Right, or Full
- Self Join
- NOT IN (Exclusion)
- Cross (Product)
 - Cartesian

Typical Teradata Join Plans:

- Merge Join
- Product Join
- Hash Join
- Nested Join
- Exclusion Join
- Inclusion Join
- RowID Join
- Self-Join

Optimizer Minimizes Spool Usage

As you saw on the preceding page, Teradata often has to utilize Spool space to hold the copies of rows redistributed to do a Join. The Optimizer minimizes the amount of Spool required by:

- Projecting (copying) only those columns which the query requires.
- Doing single-table Set Selections first (qualifying rows).
- Putting only the Smaller Table into Spool whenever possible.

Teradata determines the Smaller Table by multiplying the number of qualified rows by the number of bytes in the columns to be projected (qualifying rows * projected column bytes). Which table this turns out to be is not always obvious.

Note: Non-equality Join Operators produce a (partial) Cartesian Product. Join operators should always be equality conditions. Set selection operators may be any condition.

Optimizer Minimizes Spool Usage

1st Example: Select from Dept #1025

```
SELECT      E.empnum  
            E.lastname,  
            E.firstname,  
            P.checknum  
FROM        Employee E  
INNER JOIN  Paycheck P  
ON          E.empnum = P.empnum  
WHERE      E.deptnum = 1025;
```

Projection List

Explicit Join

Join Condition

Set Condition

The Optimizer minimizes spool size before the join.

- Applies SET conditions first (WHERE).
- Only the necessary columns are used in Spool.

Assume the following:

Employee (100,000 Rows, each row is 300 bytes)
Paycheck (1,000,000 rows, each row is 250 bytes)

2nd Example: Select from all departments

```
SELECT      E.*, P.checknum  
FROM        Employee E  
INNER JOIN  Paycheck P  
ON          E.empnum = P.empnum;
```

In this example, all columns from Employee, but only 2 columns from Paycheck are needed.

Therefore, the following applies for spool space needed.

Employee – 100,000 x 300 bytes = 30 MB

Paycheck – empnum (INTEGER), checknum (INTEGER) – 8 x 1,000,000 = 8 MB

Row Selection

Row Selection is a very important part of Join Processing. It is dependent on the conditions specified in the WHERE clause. If no Set Selection conditions exist, then all rows of both tables will participate in the Join.

Reducing the number of rows that participate will improve Join performance. The two examples on the facing page illustrate this. The first SELECT statement has no Set Selection conditions; therefore, there is no Row Selection, and all rows from both tables will participate in the Join.

The second SELECT statement differs from the first one in that a Set Selection condition has been added. By specifying “WHERE Part.Ship# IS NOT NULL,” the performance of the Join will be greatly improved. The effect of this new condition is to reduce the number of participating Part Table rows from 30,000,000 to 500,000.

The Optimizer automatically eliminates NULLs for INNER Joins.

Note: The Part Number represents a “serial number” and is unique for a part. Therefore, a particular part number or serial number can only be on 1 shipment.

Row Selection

- Column projection always precedes a join and row selection usually precedes a join.
- Tip:** Reduce the rows that participate to improve join performance.

3rd Example:

SHIPMENT			
5,000 Rows	SHIP#	...	
PK/FK	PK,SA,NN		
Distinct Values	5000		
Max Rows/Value	1		
Max Rows/NULL	0		
Typical Rows/Value	1		
PI/SI	UPI		

PART			
30,000,000 Rows	PART#	...	SHIP#
PK/FK	PK,SA,NN		FK
Distinct Values	30M		5001
Max Rows/Value	1		200
Max Rows/NULL	0		29.5M
Typical Rows/Value	1		100
PI/SI	UPI		

```
SELECT ...
FROM Shipment S
INNER JOIN Part P
ON S.ship# = P.ship#;
```

```
SELECT ...
FROM Shipment S
INNER JOIN Part P
ON S.ship# = P.ship#
WHERE P.ship# > 150183;
```

Assuming Shipment.Ship# is defined as NOT NULL:

- The Optimizer automatically eliminates all NULLs for INNER joins in the Part table before doing join.
- Even though the Part table has 30,000,000 rows, only 500,000 rows have values (match) and are joined to the Shipment table and output.
- To further eliminate additional rows, add a WHERE condition that reduces the number of rows that participate in the join.

Join Redistribution

The Primary Index is the major consideration used by the Optimizer in determining how to join two tables and deciding which rows to move.

Three general scenarios may occur when two tables are to be Merge Joined:

1. The Join column(s) is the Primary Index of both tables (best case).
2. The Join column is the Primary Index of one of the tables.
3. The Join column is not a Primary Index of either table (worst case).

The three scenarios are described below and pictured on the following pages:

The Join column(s) is the Primary Indexes of Both Tables

This is the best case scenario because rows that can be joined together are already on the same target AMP. Equal primary index values always hash to the same AMP. No movement of data to other AMPs is necessary. The rows are already sorted in Row Hash sequence because of the way in which they are stored by the file system. With no need for sorting or movement of data, the Join can take place immediately.

The Join column is the Primary Index of one of the tables

In this case, one table has its rows on the target AMPs and one does not. The rows of the second table must be redistributed to their target AMPs by the hash code of the Join Column value. If the table is “small,” the Optimizer might decide to simply “duplicate” the entire table on all AMPs instead of hash redistributing. In either case, the rows of one table will be copied to their target AMPs. (If the PI table is the “small” table, the Optimizer might choose to duplicate it on all AMPs rather than redistributing the non-PI table.)

Join Redistribution

Join columns are from the same domain.

No Redistribution needed.

```
SELECT . . .
FROM   Table1 T1
INNER JOIN Table2 T2
ON     T1.A = T2.A;
```

A	B	C
PI		
100	214	433

A	B	C
PI		
100	725	002

Join columns are from the same domain.

Redistribution needed.

```
SELECT . . .
FROM   Table3 T3
INNER JOIN Table4 T4
ON     T3.A = T4.B;
```

A	B	C
PI		
255	345	225

A	B	C
PI		
867	255	566

Redistribute T4 rows in spool on column B.

SPOOL

A	B	C
	PI	
867	255	566

Join Redistribution (cont.)

The Join column is the Primary Index of neither table

If neither column is a Primary Index, then the rows of both tables must be redistributed to their target AMPs. This may be done by hash redistribution of the Join Column value, or by duplicating “small” tables on each AMP. In either case, this approach involves the maximum amount of data movement. The choice of a Primary Index should be heavily influenced by the amount of Join activity anticipated.

Join Redistribution (cont.)

Join is on columns that aren't the Primary Index of either table.

Join columns are from the same domain.

```

SELECT      . . .
FROM        Table5  T5
INNER JOIN  Table6  T6
ON          T5.B = T6.C;
        
```

Redistribute T5 rows in spool
on column B.

Redistribute T6 rows in spool
on column C.

T5

A	B	C
PI		
456	777	876

T6

A	B	C
PI		
993	228	777

SPOOL

A	B	C
	PI	
456	777	876

SPOOL

A	B	C
		PI
993	228	777

If the columns being joined together are not Primary Index columns and are from the same domain, options the Optimizer may choose from include:

- Redistribute both tables in spool (as shown above)
- Duplicate the smaller table in spool across all AMPs

Duplicating a Table in Spool

The facing page highlights the fact that Joins can require considerable Spool space. Take this into consideration when calculating Spool requirements.

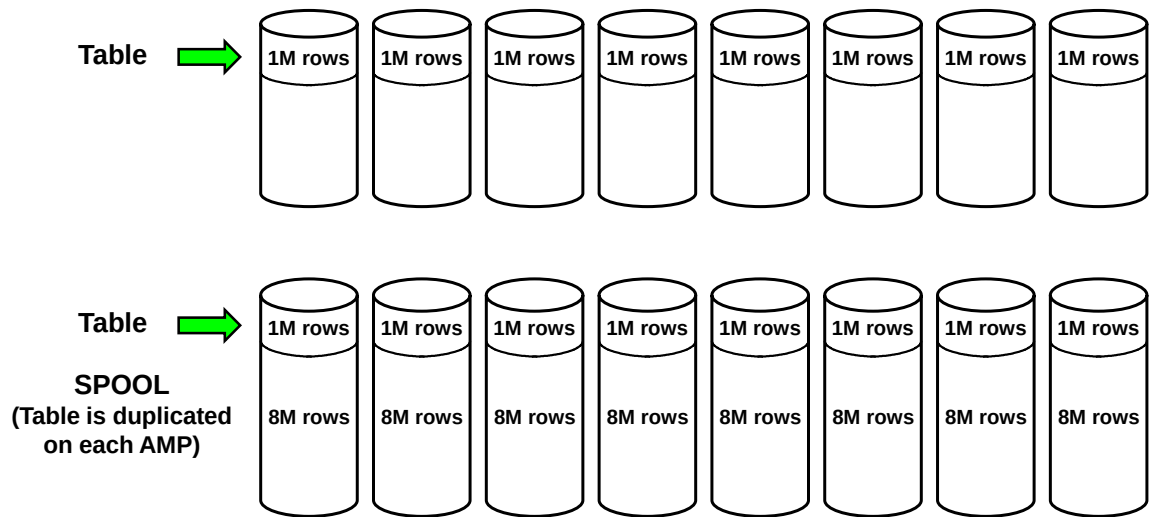
The top diagram shows an 8 million row table distributed evenly across 8 AMPs so that there are 1 million rows on each AMP.

The bottom diagram shows what happens when the table is duplicated in Spool across all the AMPs. You will notice that there are now 9 million rows on each AMP – 1 million for the actual table and 8 million in Spool.

This example should convince you of the importance of using the EXPLAIN facility so that you don't do unnecessary Product Joins.

Duplicating a Table in Spool

- For merge joins, the optimizer may choose to **duplicate a small table** on each AMP.
- For product joins, the optimizer **always duplicates** one table across all AMPs.
- In either case, each AMP must have enough spool space for a complete copy.



The Explain plan will indicate that 64M rows are needed for spool.

General Join Distribution Strategies

The Optimizer has many join methods, or modes, to choose from to ensure that any join operation is fully optimized. This module discusses some of the more common join methods, but certainly does not cover all of the join methods available to the Optimizer.

The particular processing described for a given type of join (for example, duplication or redistribution of spooled data) might not apply to all joins of that type.

Some of the common join methods include:

- Merge or Hash
 - When done on matching primary indexes, do not require any data to be redistributed.
 - Hash joins are often better performers and are used whenever possible. They can be used for equijoins *only*.
- Nested
 - Only join expression that generally does not require all AMPs.
 - Preferred join expression for OLTP applications.
- Product
 - Always selected by the Optimizer for WHERE clause inequality conditions.
 - High cost because of the number of comparisons required.

General Join Distribution Strategies

MERGE JOIN

Do nothing if Primary Indexes match and are the join columns.

OR

REDISTRIBUTE one or both sides (depending on the Primary Indexes used in the join).

OR

DUPLICATE the smaller table (or data set) in spool on all AMPs.

After redistributing or duplicating the data, the optimizer may choose to:

- Perform a Hash Join if the redistributed or duplicated table can be held in memory.
- SORT the duplicated table on join column row hash, create a local spool copy of the larger table and sort it on join column hash, and perform a merge join.

NESTED JOIN – Special join case

Equijoin with a constant value for a unique index in one table.

Only join expression that generally does not require all AMPs.

PRODUCT JOIN – Rows do not have to be in any sequence.

DUPLICATE the Smaller Table on all AMPs.

Merge Join

The Merge Join retrieves rows from two tables and then puts them onto a common AMP based on the row hash of the columns involved in the join. The system sorts the rows into join column row hash sequence, then joins those rows that have matching join column row hash values.

Merge Joins are commonly done when the Join condition is based on equality. They are generally more efficient than product joins because the number of rows comparisons is smaller. The general merge join strategy consists of the following steps:

- Identify the smaller table to be joined
- The Optimizer only pursues the following steps if it is necessary to place qualified rows into a spool file.
 - Place the qualifying rows from one or both relations into a spool file.
 - Relocate the qualified spool rows to their target AMPs based on the hash of the join column set
 - Sort the qualified spool rows on their join column row hash values.
- Compare those rows with matching Join Column Row Hash values.

Merge Joins

- **Require rows to be on the same AMP to be joined.**
- Are usually chosen for an equality join condition.
- Blocks from both tables are read only once.
- Are generally more efficient than a product join.
- Compare matching join column row hash values for the rows.
- Cause significantly fewer comparisons than a product join.

Merge join process:

- If the tables do not have matching PI on the join columns, identify the smaller table.
- If necessary:
 - Put qualifying data of one or both tables into spool(s).
 - Move the spool rows to AMPs based on the join column hash.
 - Sort the spool rows into join column hash sequence.
- Compare the rows with matching join column row hash values.

General considerations:

- Join costs rise with the number of rows that are moved and sorted.
- Join plans for the same tables may change as the demographics change.

Merge Join Strategy

Two different general Merge Join algorithms are available:

- Slow Path – the slow path is used when the left table is accessed using a read mode other than an all rows scan. The determination is made in the AMP, not by the Optimizer.
- Fast Path – the fast path is used when the left table is accessed using the all-row scan reading mode.

The illustration on the facing page gives you a graphical representation of how a Merge Join compares only those rows with matching Row Hash values.

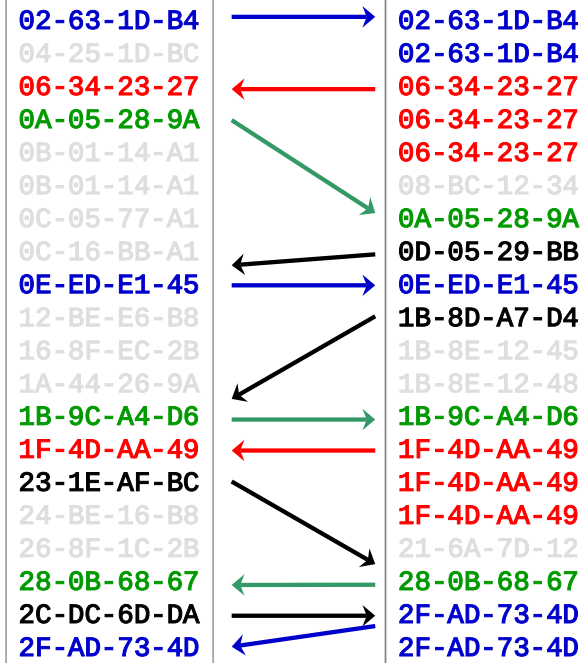
Merge Join Strategy

There are several versions of merge join. This particular illustration is for the row hash match scan.

With a row hash match scan, both tables (or spool) are sorted on join column row hash sequence.

Note that unnecessary comparisons are ignored, and that each data block is touched only once.

Left Table (or Spool) Right Table (or Spool)



Merge Join – Matching Primary Indexes

The example on the facing page illustrates joining two tables together with matching primary indexes. This strategy does not require any duplication or sorting of rows.

This example is pictured on a 4 AMP System. Teradata merely compares the rows already on the proper AMPs.

This join strategy will occur when the Join Column is the Primary Index of both tables. This is also referred to an AMP Local Join.

Merge Join – Matching Primary Indexes

Employee

Enum	Name	Dept
PK		FK
UPI		
1	BROWN	200
2	SMITH	310
3	JONES	310
4	CLAY	400
5	PETERS	150
6	FOSTER	200
7	GRAY	310
8	BAKER	310
9	TYLER	450
10	CARR	450

Employee_Phone

Enum	Area_Code	Phone
PK		
FK		
NUPI		
1	213	3241576
1	213	4950703
3	408	3628822
4	415	6347180
5	312	7463513
6	203	8337461
8	301	6675885
8	301	2641616

Primary Indexes match: no duplication or sorting needed

Example:

```

SELECT      E.Enum, E.Name, ...
FROM        Employee E
INNER JOIN  Employee_Phone P
ON          E.Enum = P.Enum;
```

Employee rows hash distributed on Enum (UPI)

6 FOSTER 200
8 BAKER 310

4 CLAY 400
3 JONES 310
9 TYLER 450

1 BROWN 200
7 GRAY 310

5 PETERS 150
2 SMITH 310
10 CARR 450

Employee_Phone rows hash distributed on Enum (NUPI)

6 203 8337461
8 301 2641616
8 301 6675885

4 415 6347180
3 408 3628822

1 213 3241576
1 213 4950703

5 312 7463513

Merge Join – Row Redistribution

The example on the facing page illustrates an example of redistributing one of the tables. This strategy consists of redistributing the rows of one table and sorting them on the Row Hash of the Join Column.

This example is pictured on a 4 AMP System. Teradata copies the employee rows into Spool and redistributes them on Employee.Dept Row Hash. The Merge Join then occurs with the rows to be joined located on the same AMPs.

This strategy occurs when one of the tables is already distributed on the Join Column Row Hash. The Join Column is the PI of one, not both, of the tables.

Merge Join – Row Redistribution

REDISTRIBUTE one side and SORT on join column row hash.

Employee

Enum	Name	Dept
PK		FK
UPI		
1	BROWN	200
2	SMITH	310
3	JONES	310
4	CLAY	400
5	PETERS	150
6	FOSTER	200
7	GRAY	310
8	BAKER	310
9	TYLER	450
10	CARR	450

Example: `SELECT E.Enum, E.Name, D.Dept, D.Name`
`FROM Employee E`
`INNER JOIN Department D`
`ON E.Dept = D.Dept;`

Employee rows hash distributed on Employee.Enum (UPI)

6 FOSTER 200	4 CLAY 400	1 BROWN 200	5 PETERS 150
8 BAKER 310	3 JONES 310	7 GRAY 310	2 SMITH 310
	9 TYLER 450		10 CARR 450

Spool file after redistribution on Employee.Dept row hash

5 PETERS 150	7 GRAY 310	1 BROWN 200	4 CLAY 400
	3 JONES 310	6 FOSTER 200	
	8 BAKER 310	9 TYLER 450	
	2 SMITH 310	10 CARR 450	

Department

Dept	Name
PK	
UPI	
150	PAYROLL
200	FINANCE
310	MFG.
400	EDUCATION
450	ADMIN

Department rows hash distributed on Department.Dept (UPI)

150 PAYROLL	310 MFG.	200 FINANCE	400 EDUCATION
		450 ADMIN	

Merge Join – Duplicate the Smaller Table

The example on the facing page illustrates a Merge Join by duplicating the smaller table.

This strategy consists of duplicating and sorting the smaller table on all AMPs and locally building a copy of the Larger Table and sorting it.

This example is pictured on a 4 AMP System. Teradata duplicates the department table and sorts it on the department column for all AMPs. The employee table is built locally and sorted on the department Row Hash.

The Merge Join is then performed.

This example is the same as the previous example. If the Parser determines from statistics that it would be less expensive to duplicate and sort the smaller table than to hash redistribute the larger table, it will choose this strategy.

Merge Join – Duplicate the Smaller Table

**DUPLICATE and SORT the Smaller Table on all AMPs.
LOCALLY BUILD a copy of the Larger Table and SORT.**

Employee

Enum	Name	Dept
PK		FK
UPI		
1	BROWN	200
2	SMITH	310
3	JONES	310
4	CLAY	400
5	PETERS	150
6	FOSTER	200
7	GRAY	310
8	BAKER	310
9	TYLER	450
10	CARR	450

Department

Dept	Name	
PK		
UPI		
150	PAYROLL	
200	FINANCE	
310	MFG.	
400	EDUCATION	
450	ADMIN	

Example:

```
SELECT E.Enum, ...
FROM Employee E
INNER JOIN Department D
ON E.Dept = D.Dept;
```

Department rows hash distributed on Department.Dept (UPI)

150 PAYROLL	310 MFG.	200 FINANCE 450 ADMIN	400 EDUCATION
-------------	----------	--------------------------	---------------

Spool file after duplicating and sorting on Department.Dept row hash.

150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN	150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN	150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN	150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN
--	--	--	--

Employee rows hash distributed on Employee.Enum (UPI)

6 FOSTER 200 8 BAKER 310	4 CLAY 400 3 JONES 310 9 TYLER 450	1 BROWN 200 7 GRAY 310	5 PETERS 150 2 SMITH 310 10 CARR 450
-----------------------------	--	---------------------------	--

Spool file after locally building and sorting on Employee.Dept row hash

6 FOSTER 200 8 BAKER 310	3 JONES 310 4 CLAY 400 9 TYLER 450	1 BROWN 200 7 GRAY 310	5 PETERS 150 2 SMITH 310 10 CARR 450
-----------------------------	--	---------------------------	--

Nested Joins

Nested Joins are the most efficient types of Join. For a Nested Join to be done between Table 1 and Table 2, the Optimizer must be provided with both of the following:

- An equality value for a unique index on Table 1 (this will retrieve a single row).
- A Join on a column of that single row to any index on Table 2

The system will retrieve the single row from Table 1 based on the UPI or USI value, then determine the hash of the value in the Join Column to access matching Table 2 rows.

Nested Joins are the only types of Join that don't always use all of the AMPs. The number of AMPs involved in a Nested Join will vary.

The query on the facing page can also be coded as follows:

```
SELECT  E.Name  
        ,D.Name  
FROM    Employee E, Department D  
WHERE   E.Dept = D.Dept  
AND     E.Enum = 5;
```


Nested Joins

- This is a special join case.
- This is the only join that doesn't always use all of the AMPs.
- It is the most efficient in terms of system resources.
- It is the best choice for OLTP applications.
- To choose a Nested Join, the Optimizer must have:
 - An equality value for a unique index (UPI or USI) on Table1.
 - A join on a column of that single row to any index on Table2.
- The system retrieves the single row from Table1.
- It hashes the join column value to access matching Table2 row(s).

Example:

```
SELECT      E.Name
            , D.Name
FROM        Employee E
INNER JOIN  Department D
ON          E.Dept = D.Dept
WHERE       E.Enum = 5;
```

Employee

Enum	Name	Dept
PK		FK
UPI		
1	BROWN	200
2	SMITH	310
3	JONES	310
4	CLAY	400
5	PETERS	150
6	FOSTER	400
7	GRAY	310
8	BAKER	310

Department

Dept	Name
PK	
UPI	
150	PAYROLL
200	FINANCE
310	MFG.
400	EDUCATION

Product Join

Product Joins are the most general forms of Join. In a product Join, every qualifying row of one table is compared to every qualifying row in the other table. Rows which match on WHERE conditions are saved.

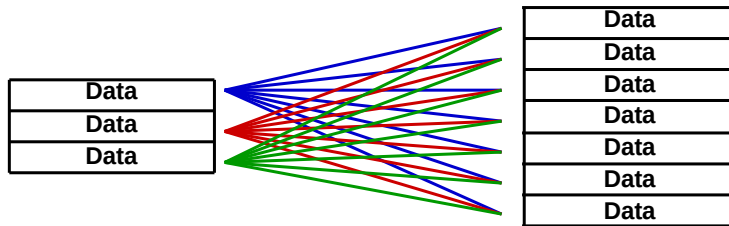
Product Joins are caused by any of the following:

- The WHERE clause is missing.
- A Join condition is not based on equality (NOT =, LESS THAN, GREATER THAN).
- Join conditions are ORed together.
- There are too few Join conditions.
- A referenced table is not named in any Join condition.
- Table aliases are incorrectly used.
- The Optimizer determines that it is less expensive than the other Join types.

Product joins get their name from the fact that the number of comparisons required is the “product” of the number of qualifying rows of both tables. A product Join between a table of 1,000 rows and a table of 50 rows would require 50,000 comparisons and a potential answer set of 50,000 rows.

Because all rows of one side must be compared with all rows of the other, the smaller table is always duplicated on all AMPs. Its rows then are compared with the AMP local rows of the other table. If the entire table cannot fit into memory, blocks will have to be read in more than once. Comparisons that qualify are written to spool. While legitimate cases exist for these joins, they should be avoided whenever possible.

Product Join



Rows must be on the same AMP to be joined.

- Does not sort the rows.
- May re-read blocks from one table if AMP memory size is exceeded.
- **It compares every qualifying Table1 row to every qualifying Table2 row.**
- Those that match the WHERE condition are saved in spool.
- It is called a Product Join because:

$$\text{Total Compares} = \# \text{ Qualified Rows Table1} * \# \text{ Qualified Rows Table2}$$

- The internal compares become very costly when there are more rows than AMP memory can hold at one time.
- They are generally unintentional and often give meaningless output.
- Product Join process:
 - Identify the Smaller Table and duplicate it in spool on all AMPs.
 - Join each spool row for Smaller Table to every row for Larger Table.

Cartesian Product

Cartesian Products are unconstrained Product Joins. Every row of one table is joined to every row of another table.

Since there is rarely any practical business use of Cartesian Product Joins, they generally will occur when an error is made in coding the SQL query. Some reasons why Cartesian Products are caused are shown on the facing page.

Running your queries through EXPLAIN will enable you to avoid unintentional Cartesian Product Joins and thus save a lot of system resources. You should always EXPLAIN a Join before it goes production.

An example of using an alias incorrectly:

```
SELECT TableA.col1  
FROM TableA A;
```

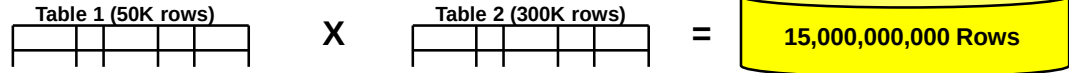
The result will be a product join. Teradata will rename the TableA as A and then in the SELECT clause when it sees a reference to TableA, it will treat it as a separate instance of TableA.

Teradata assumes that you want to join the table to itself. It will look for a join condition, but as there is none; Teradata will carry out a PRODUCT join.

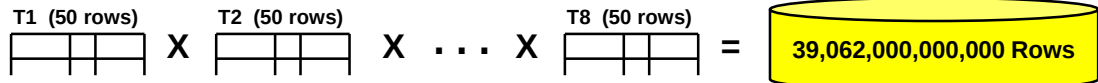
Cartesian Product

- This is an unconstrained Product join.
- **Each row of Table1 is joined to every row in Table2.**
- Cartesian Product Joins consume significant system resources.

Table row count is critical:



Number of tables is even more critical:



- **Cartesian Product Joins are occasionally used in some business cases.**
 - Example: Join a single column table (with limited rows) to a second table to populate a third table.
 - These are also supported for ANSI compatibility.
- **Cartesian Product Joins may occur when:**
 - A join condition is missing or there are too few join conditions.
 - Join conditions are not based on equality.
 - A referenced table is not named in any join condition.
 - Table aliases are incorrectly used.
- The transaction aborts if it exceeds the user's spool limit.

Product Join – Duplicate the Smaller Table

The example on the facing page illustrates the product join strategy. This strategy consists of duplicating the smaller table on every AMP.

This example is pictured on a 4 AMP System. The Product Join plan is caused by a join condition other than equality. As you can see, Teradata determines that the Department Table is the Smaller Table and then distributes copies of those rows to each AMP. The employee rows stay where they were (distributed by Enum). The Product Join then returns those rows which satisfy the Join Condition (“Employee.Dept > Department.Dept”) after comparing every row in the employee table with every row in the department table.

Product Join – Duplicate the Smaller Table

DUPLICATE the Smaller Table on every AMP.

Employee

Enum	Name	Dept
PK		FK
UPI		
1	BROWN	200
2	SMITH	310
3	JONES	310
4	CLAY	400
5	PETERS	150
6	FOSTER	200
7	GRAY	310
8	BAKER	310
9	TYLER	450
10	CARR	450

Department

Dept	Name
PK	
UPI	
150	PAYROLL
200	FINANCE
310	MFG.
400	EDUCATION
450	ADMIN

Example: `SELECT E.Enum, E.Name, D.Dept, D.Name
FROM Employee E
INNER JOIN Department D
ON E.Dept > D.Dept ;`

Department rows hash distributed on Department.Dept (UPI)

150 PAYROLL	310 MFG.	200 FINANCE 450 ADMIN	400 EDUCATION
-------------	----------	--------------------------	---------------

Spool file after duplicating the Department rows.

150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN	150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN	150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN	150 PAYROLL 200 FINANCE 310 MFG. 400 EDUCATION 450 ADMIN
--	--	--	--

Employee rows hash distributed on Employee.Enum (UPI)

6 FOSTER 200 8 BAKER 310	4 CLAY 400 3 JONES 310 9 TYLER 450	1 BROWN 200 7 GRAY 310	5 PETERS 150 2 SMITH 310 10 CARR 450
-----------------------------	--	---------------------------	--

Hash Join

The Merge Join requires that both sides of the join have their qualifying row in join column row hash sequence. In addition, if the join column(s) are not the Primary Index, then some redistribution or duplication of rows precedes the sort.

In a Row Hash Join, the smaller table is sorted into join column row hash sequence and then redistributed or duplicated on all AMPs. The larger table is then processed a row at a time and the rows in this table do NOT have to be sorted into join column hash sequence. For those rows that qualify for joining (WHERE or ON), the Row Hash of the join column(s) is used to do a binary search of the smaller table (in memory) for a match. The Optimizer can choose this join plan when the qualifying rows of the small table can be held AMP memory resident. Use COLLECT STATISTICS on both tables to help guide the Optimizer.

You can enable hash joins and control the amount of memory allocated for hash joins with the following DBS Control fields:

- HTMemAlloc
- SkewAllowance

Recommendations that most sites will use for these parameters:

- HTMemAlloc = 1
- Skew Allowance = 75

Consider a different setting if:

- The system is always very lightly loaded. In this case, you may want to increase HTMemAlloc to a value between 2 and 5.
- Data is so badly skewed that the hash join degrades performance. In this case, you should turn the feature off or increase the Skew Allowance to 80 or 90.

Miscellaneous Hash Join Notes:

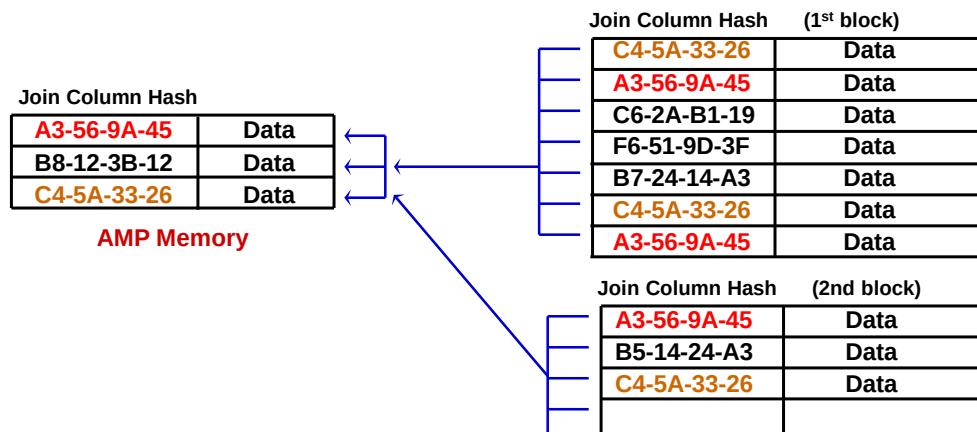
The standard hash join does require the same spooling and the same relocation of rows before the join, whether a table is being duplicated or redistributed. In this regard, it is similar to the merge join.

However, if the small table is small enough to fit into one hash partition and if it is duplicated, the redistribution of the large table can be eliminated by doing a hash join on the fly, a variant of the standard hash join. In such a case, the large table is read directly, without spooling or redistributing, and the hash join is performed between the small table spool and the large table rows.

Both types of hash joins eliminate sorting, and the on the fly version eliminates redistribution as well.

A **Dynamic Hash Join** provides the ability to do an equality join directly between a small table and a large table on non-primary index columns without placing the large table into a spool file. For Dynamic Hash Join to be used, the left table must be small enough to fit in a single partition.

Hash Join



This optimizer technique effectively places the smaller table in AMP memory and joins it to the larger table in unsorted spool. A Hash Join is applicable *only* to equijoins.

Row Hash Join Process:

- Redistribute or duplicate the smaller table in memory across the AMPs.
- Usually sort the AMP memory into join column row hash sequence.
- Use the join column row hash of the larger table to search memory for a match.

This join eliminates the sorting, and possible redistribution or copying, of the larger table. EXPLAIN plans will contain terminology such as "Single Partition Hash Join".

Exclusion Joins

Exclusion Joins are based on set subtraction and used for finding rows that don't have a matching row in the other table. Queries with the NOT IN and EXCEPT operator lead to Exclusion Joins.

Exclusion Join is a Product or Merge Join where only the rows that do not satisfy (are NOT IN) any condition specified in the request are joined. In other words, Exclusion Join finds rows in the first table that do *not* have a matching row in the second table.

Exclusion Join is an implicit form of the outer join.

The appearance of a null (unknown) join value will return an answer set of NULL. Null join values must be prohibited to get a result other than NULL.

Another way to view exclusion operations, is to look at the 3 rules that are applied from the selected set to the NOT IN set.

1. Any True – disqualifies the row
2. Any Unknown – disqualifies the row
3. All False – qualifies the row.

Therefore, in the first example, rows 1 and 3 are matched (true) so they are disqualified. Rows 2 and 4 are all false (no matches), so they qualify.

In the second example, row 2 hits the unknown (NULL) and is disqualified, so in essence all of the rows that do not get disqualified by the match (true) will get disqualified by rule number 2 (Any Unknown).

Exclusion Joins

- Finds rows that **DON'T** have a match.
- May be done as merge or product joins.
- SQL that frequently causes exclusion joins are NOT IN subqueries and EXCEPT or MINUS operations.
- Uses 3-value logic (=, <>, unknown) on nullable columns.
- To avoid NULL result sets:
 - If possible, define columns (used with NOT IN) as NOT NULL on the CREATE TABLE.
 - In queries, include WHERE column_name IS NOT NULL against nullable join columns.

Set_A	NOT IN	Set_B	=	
1		1		
2		3		
3		5		
4				

Set_A	NOT IN	Set_B	=	
1		1		
2		3		
3		5		
4		NULL		

Exclusion Join Example

The example on the facing page illustrates an Exclusion Merge Join. The SQL query is designed to list those salespeople who don't have any customers.

This example is pictured on a 4 AMP System. As you can see, Teradata does the following:

- Performs the subquery (SELECT Sales_Emp_Number FROM Customer).
- Hash redistributes these rows on all AMPs.
- Eliminates duplicate values.
- Returns the name column of those rows in the employee table which do not match the rows in the sub-query.

In this case, the salespersons whose names would be returned would be Peters and Tyler.

Exclusion Join Example

Employee

Enum	L_Name	Job_Code
PK		FK
UPI		
1	BROWN	3100
2	SMITH	2101
3	JONES	3100
4	CLAY	1201
5	PETERS	3100
6	FOSTER	3100
7	GRAY	1302
8	BAKER	3100
9	TYLER	3100
10	CARR	1302

Customer

Cust_Num	Sales_Emp_Number
PK	FK
UPI	
23	6
24	3
25	8
26	1
27	6
28	8
29	1
30	3
31	8

This is an example of an Exclusion Merge Join.

Example:

```

SELECT Enum, L_Name
FROM Employee
WHERE Job_Code = 3100
AND Enum
      NOT IN (SELECT Sales_Emp_Number
              FROM Customer);

```

Customer rows hash distributed on Cust_Num (UPI).

30 6	23 6	28 8	25 8
24 3	29 1	27 6	26 1
31 8		30 3	

Customer.Sales_Enum after hashing and duplicate elimination.

6 8	3	1	
--------	---	---	--

Employee rows hash distributed on Enum (UPI).

6 FOSTER 3100 8 BAKER 3100	3 JONES 3100 9 TYLER 3100	1 BROWN 3100	5 PETERS 3100
-------------------------------	------------------------------	--------------	---------------

Inclusion Joins

There are two types of Inclusion Join.

- Inclusion Merge Join
 - Read each row from the left table.
 - Join each left table row with the first right table row having the same hash value.
 - End of process.
- Inclusion Product Join
 - For each left table row read all right table rows from the beginning until one is found that can be joined with it.
 - Return the left row if a matching right row is found for it.
 - End of process.

The facing page illustrates an example of an Inclusion Merge Join. Explain plan terminology will indicate either “Inclusion Merge or Inclusion Product” in the explain text.

Inclusion Joins

INCLUSION JOIN – an inclusion join is a Merge or Product Join where the first right table row that matches the left row is joined.

An example of a query that may cause an inclusion merge join:

```
SELECT      Name, EmpNo
FROM        Employee
WHERE       EmpNo IN (SELECT EmpNo FROM Charges)
ORDER BY    Name ;
```

There are two types of Inclusion Join.

- **Inclusion Merge Join**
 - Read each row from the left table.
 - Join each left table row with the first right table row having the same hash value.
- **Inclusion Product Join**
 - For each left table row read all right table rows from the beginning until one is found that can be joined with it.
 - Return the left row if a matching right row is found for it.

n-Table Joins

It is not uncommon to have Joins that involve more than two tables. The Optimizer will decide which tables it processes first based on its own decision algorithms. The Optimizer can only work with two tables at a time. The results of that Join operation will then be applied to a third table (or another Join result).

“Smaller” means the size of the table both as a function of selection and projection.

“Selection” refers to the number of rows that qualify from that table in the answer set.

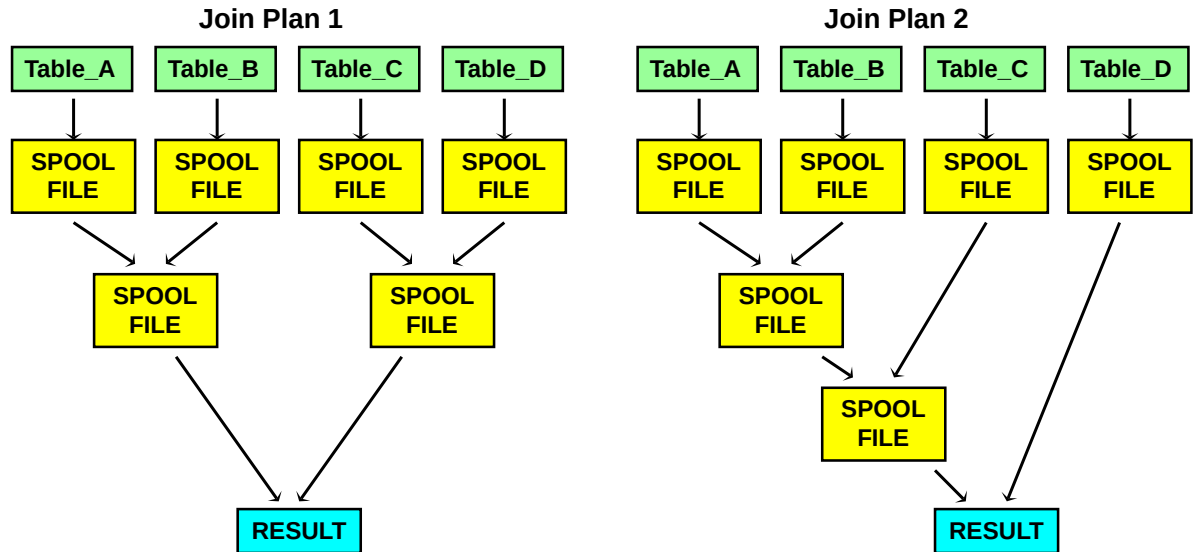
“Projection” refers to the “row size” as a function of the number of column bytes selected.

Each reduces the amount of data carried to subsequent Join steps.

n-Table Joins

- All n-Table joins are reduced to a series of two-table joins.
- The Optimizer attempts to determine the best join order.
- Collected Statistics on Join columns help the Optimizer choose wisely.

```
SELECT .... FROM Table_A, Table_B, Table_C, Table_D WHERE ... ;
```



Join Considerations with Partitioned Tables

Direct merge joins (in which the table of interest doesn't have to be spooled in preparation for a merge join) are available as an optimizer choice when two non-partitioned tables have the same PI, and all PI columns are specified as equality join terms (the traditional merge join).

Direct merge joins of two row partitioned tables are similarly available as an optimizer choice when the tables have the same PI and identical partitioning expressions, and all PI columns and all partitioning columns are specified as equality join terms (the rowkey-based merge join). In both cases, the rows of the two tables will be ordered in the same way, allowing a merge join without redistribution or sorting of the rows. The performance characteristics of a traditional merge join and a rowkey-based merge join will be approximately the same.

A direct merge join, in the traditional sense, is not available when one table is partitioned and the other is not, or when both tables are partitioned, but not in the same manner, as the rows of the two tables will not be ordered in the same way. However, the traditional merge join algorithms have been extended to provide a partitioned-aware merge join, called a "sliding window" join.

The optimizer has three general avenues of approach when joining a partitioned table to a non-partitioned table, or when joining two partitioned tables with different partitioning expressions.

- One option is to spool the partitioned table (or both partitioned tables) into a non-partitioned spool file in preparation for a traditional merge join.
- A second option (not always available) is to spool the non-partitioned table (or one of the two partitioned tables) into a partitioned spool file, with identical partitioning to the remaining table, in preparation for a rowkey-based merge join.
- The third approach is to use the sliding window join of the tables without spooling either one. The optimizer will consider all reasonable join strategies, and pick the one that has the best-estimated performance.

Sliding window joins may be slower than the traditional merge join or the rowkey-based merge join when there are many non-excluded partitions. Sliding window joins can give roughly similar elapsed-time performance when the number of non-excluded partitions is small (but with greater CPU utilization and memory consumption).

Row Partitioning (e.g., PPI) is based on modular extensions to the existing implementation of Teradata. Therefore, all join algorithms support partitioned tables without requiring a whole new set of join code.

Performance Note

Performance for Row Hash Merge Joins may be worse with partitioned table (as compared to non-partitioned tables) if a large number of partitions are not eliminated via query constraints.

Why?

The Row Hash Merge Join algorithm is more complicated and requires more resources with partitioned tables than with non-partitioned tables (assuming an equal number of qualified data blocks) **since rows are not in hash order, but rather in partition/hash order.**

Example

Assume a row partitioned Transaction table is partitioned on date ranges and an Account table is a non-partitioned table.

The partitioned Transaction table is not in the same row hash order as the non-partitioned Account table. A join of these 2 tables will take longer than a typical join of the same tables if the tables were both non-partitioned (assuming that both tables have the same PI).

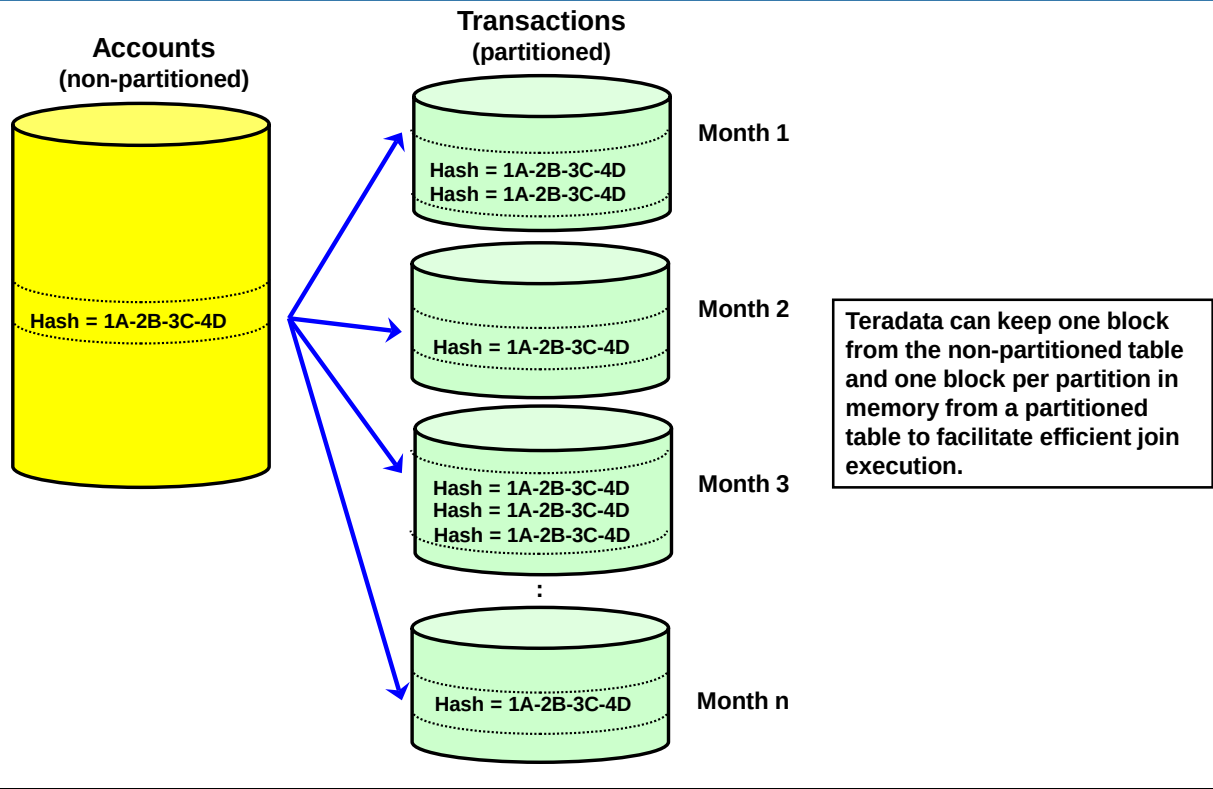
Non-partitioned to Partitioned Table Join (Few Partitions)

The following discussion assumes that two tables have the same PI and all PI columns are specified as equality join terms.

When joining two tables together (one is non-partitioned and the other has row partitioning) and after applying constraints, if there are a “small” number of surviving partitions, then the following applies:

- Teradata can keep one block from the non-partitioned table and one block per partition in memory from the row partitioned table to facilitate efficient join execution.
- Performance is similar to non-partitioned table to non-partitioned table join even when no partitions are eliminated (assumes that the number of partitions is relatively small).
 - Same number of disk I/Os (except in anomalous cases - large number of rows in a hash)
 - Higher memory requirements
 - Slightly higher CPU utilization
 - You can get significantly better performance when query constraints allow partition elimination.

Non-partitioned to Partitioned Table Join (Few Partitions)



Non-partitioned to Partitioned Table Join (Many Partitions)

The join algorithms have been enhanced to optimize joins involving row partitioned tables.

The basic enhancement is the use of a "sliding window" technique, which can be slower than the performance of a direct join. This is true as long as the number of partitions participating in the join is small. Usually, CPU utilization will probably be somewhat higher for a row partitioned table, and more memory will be used.

When joining two tables together (one with non-partitioned and the other is row partitioned) and after applying constraints, if there are a "large" number of surviving partitions, then the following applies:

- Teradata can keep one block from a non-partitioned table and one block for as many partitions as it can fit (k) into memory from a row partitioned table to facilitate efficient join execution using a "sliding window" technique.
- This type of join will usually have worse performance than non-partitioned to non-partitioned join unless partition elimination can reduce total amount of work performed.
- ✓ Higher number of disk I/Os - the data blocks in non-partitioned table will have to be rescanned multiple times.

d1 = # of data blocks in non-partitioned table

d2 = # of data blocks in row partitioned table

p = # of partitions participating in the join

k = # of partitions that can fit into memory from partitioned table

The number of I/Os for non-partitioned to partitioned join is:

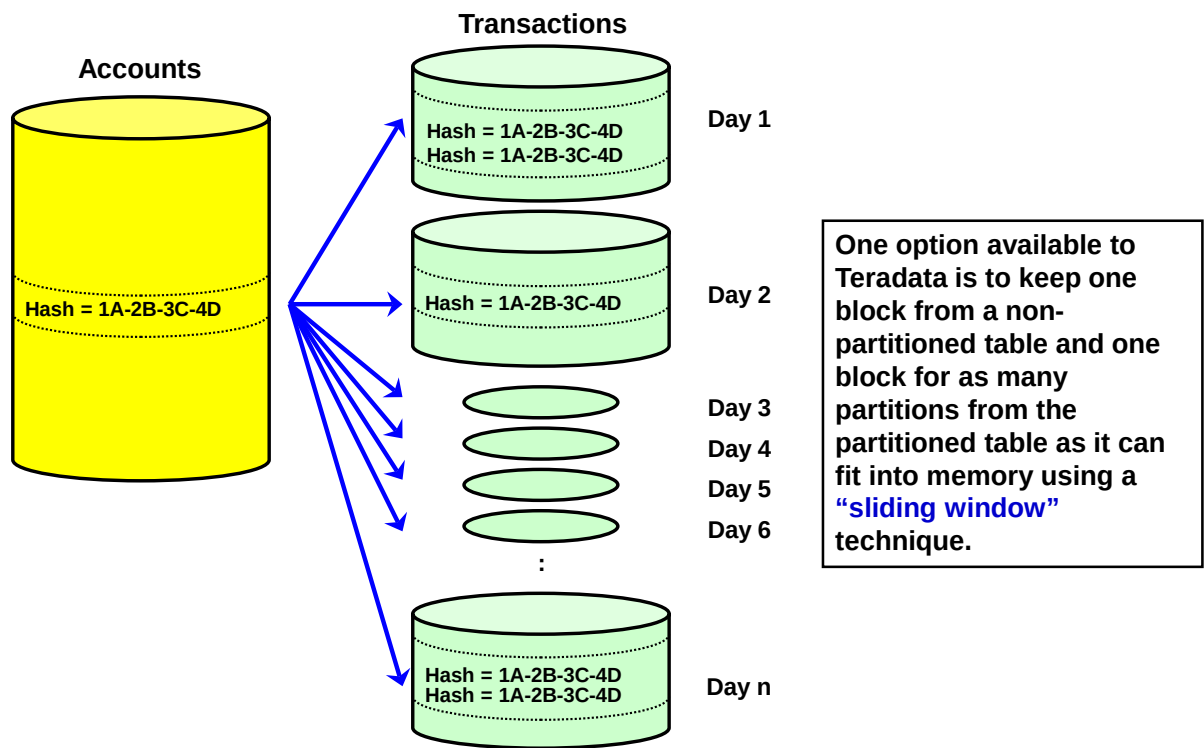
$$(p/k * d1) + d2$$

The number of I/Os for non-partitioned to non-partitioned join is:

$$d1 + d2$$

- ✓ Higher memory requirements
- ✓ Higher CPU utilization
- To obtain better performance, eliminate as many partitions as possible with query constraints.
 - ✓ The p/k value must be small (less than 3 or 4) for performance to be reasonable.

Non-partitioned to Partitioned Table Join (Many Partitions)



Non-partitioned to Partitioned Table Join (Sliding Window)

The simple example on the facing page illustrates that one block from the NPPI table is joined to data blocks from the first 60 partitions in the partitioned table.

The most straight-forward way to join a non-partitioned table to a row partitioned table would be to make a pass over the non-partitioned table for each partition of the partitioned table, thus doing the join as a series of sub-joins. This would be inefficient, especially for a large non-partitioned table, though.

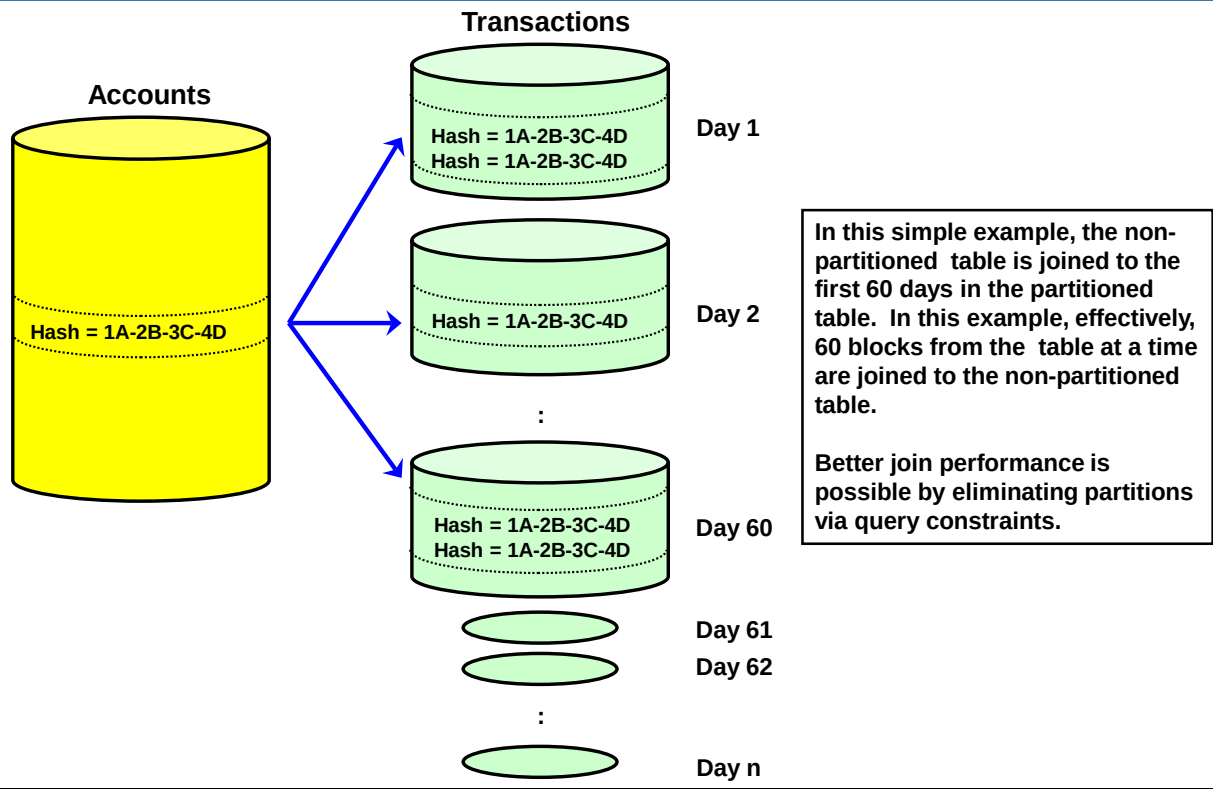
The "sliding window" join uses a similar concept, but minimizes the number of disk reads. The first data block is read for the non-partitioned table, and the first data block of each (non-excluded) partition of the partitioned table is read into memory. The rows from the non-partitioned data block are compared to the rows of each partitioned data block. The join routines present the rows in row hash order, but conceptually you can think of the process as visiting each partition's data block in turn. As the rows of a data block are exhausted, the next data block for that partition is read. This results in each data block of each table being read only once due to partitioning.

There is some additional overhead to manage the pool of data blocks, but join performance is not badly degraded. Overall performance may even be improved, if a non-trivial fraction of the partitions can be eliminated because of query conditions.

With this algorithm, a limiting factor is the number of data blocks that can be held in memory at one time. The file system cache memory (FSG Cache) is used to provide memory for the data blocks. A new user-tunable parameter is provided to control memory usage for this purpose. The DBS Control utility is used to set the parameter ("PPICacheThrP") in the Performance Group.

The value is a number expressed in tenths of a percent that controls the portion of the file system cache available for this purpose. The default value is 10, which represents one percent. A higher value will improve multiple-partition operations, of which joins are the most visible example. A high value may cause more memory contention and disk paging, so higher isn't always better.

Non-partitioned to Partitioned Table Join (Sliding Window)

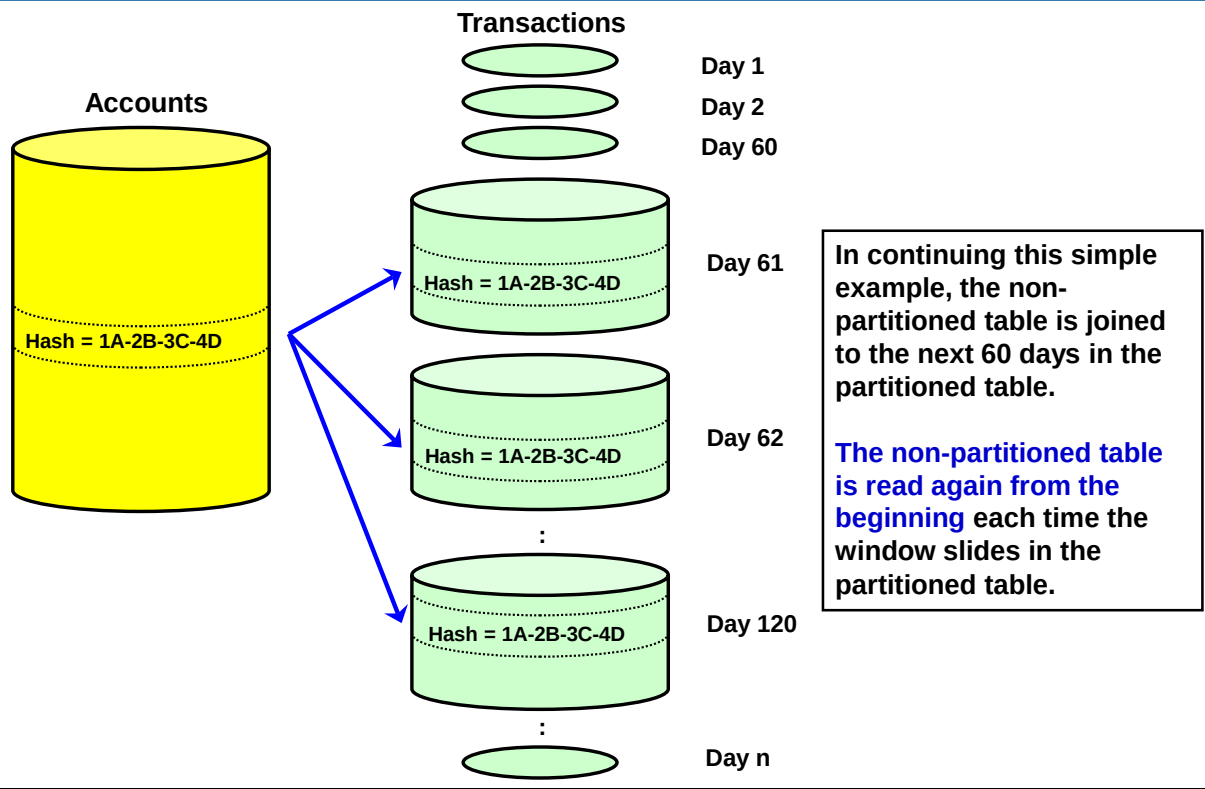


Non-partitioned to Partitioned Table Join (Sliding Window – cont.)

The example on the facing page illustrates that one block from the non-partitioned table is joined to the next 60 partitions in the partitioned table. The window “slides” from the first 60 partitions to the next 60 partitions. The non-partitioned table is read again from the beginning each time the window slides in the partitioned table.

A more significant degradation of join performance occurs when there are more non-excluded partitions than data block buffers. Assume enough memory for 60 data blocks and a table with 365 partitions. Then the "sliding window" technique lives up to its name. The first 60 partitions are processed as above, and then the non-partitioned table is read again as the "window" slides down to partitions 61 through 120. A total of six (plus) passes through the non-partitioned table are required, and the join may take on the order of six times longer than a comparable join where the window covers the entire table. (This assumes that the non-partitioned table is roughly the same size as the partitioned table.)

Non-partitioned to Partitioned Table Join (Sliding Window – cont.)



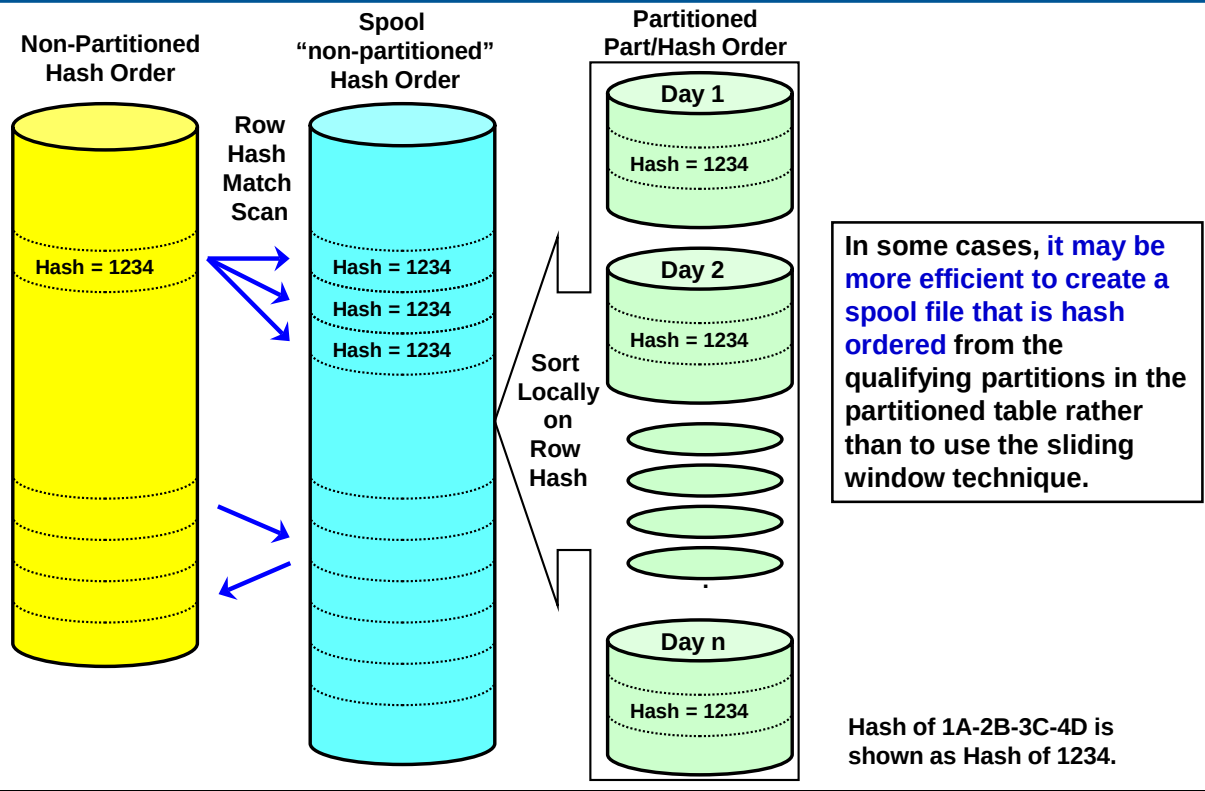
Non-partitioned to Partitioned Table Join (Hash Ordered Spool File Join)

In some cases, it will be more efficient to create a spool file that is hash ordered from the qualifying partitions rather than to use the sliding window technique.

Creation of hash ordered spool file is done relatively efficiently using an n-way merge rather than a full sort.

As always, the cost-based optimizer will figure out the most efficient execution plan using statistics and assessment of various join possibilities.

Non-partitioned to Partitioned Table Join (Hash Ordered Spool File Join)



Partitioned to Partitioned Table Join (Rowkey Match Scan)

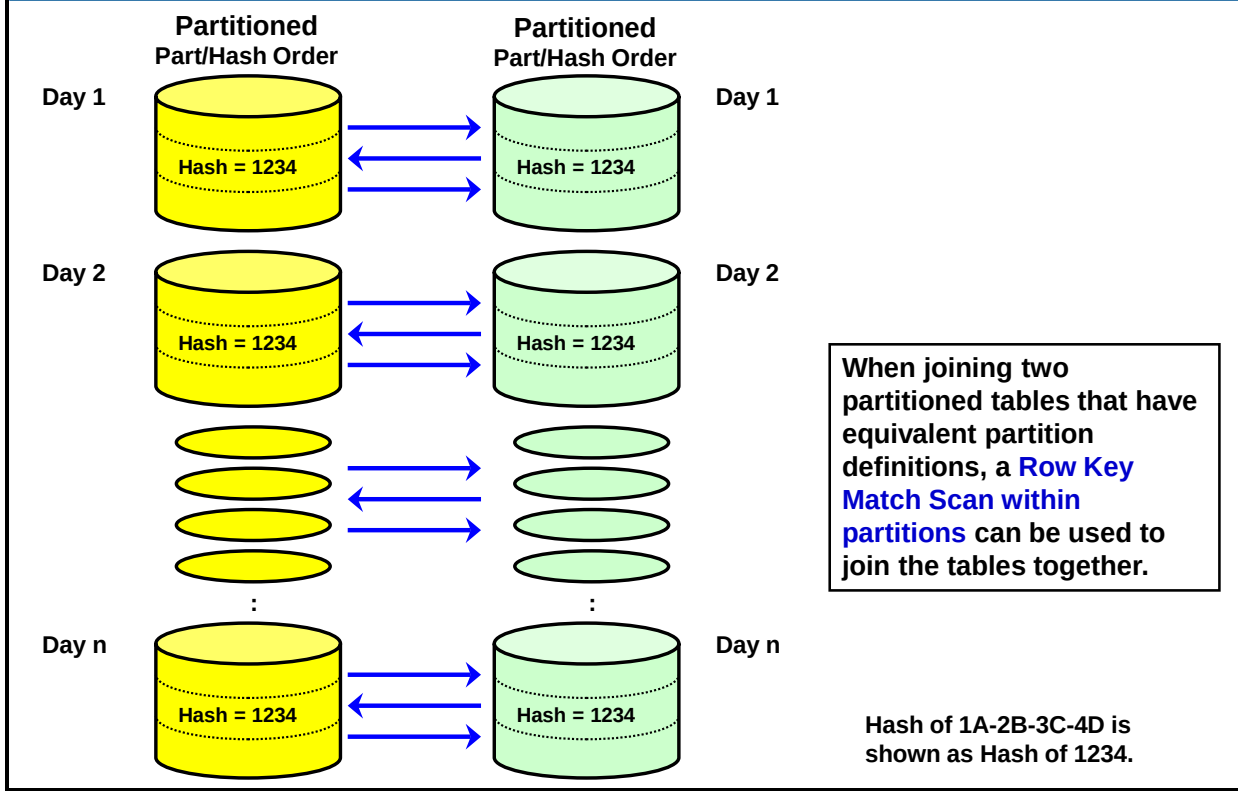
Direct merge joins of two partitioned tables are available as an optimizer choice when the tables have the same PI and identical partitioning expressions, and all PI columns and all partitioning columns are specified as equality join terms. This is referred to as a **rowkey-based merge join**. In this case, the rows of the two tables will be ordered in the same way, allowing a merge join without redistribution or sorting of the rows.

If the partitioning column is part of the Primary Index (PI), then it is possible, and usually advantageous, to define all the tables with the same PI with identical partitioning expressions. This allows for a rowkey-based merge join.

If non-partitioned table is placed into spool (redistributed or duplicated) and this spool is joined with a partitioned table, then spool can optionally be partitioned the same as the partitioned table allowing for this fast join.

The performance characteristics of a traditional merge join (on matching primary indexes) and a rowkey-based merge join will be approximately the same.

Partitioned to Partitioned Table Join (Rowkey Match Scan)



Partitioned to Partitioned Table Join (Unmatched Partitions)

If the partitioning column is not part of the Primary Index (PI), then it may be impossible to similarly partition the other tables having the same PI, as the partitioning column may not exist in the other table(s).

An expensive situation may occur when both tables are partitioned, but have different partitioning expressions. In this case, there is potentially a “sliding window” advancing through both tables.

The following discussion summarizes the number of disk reads for the various types of joins.

Given the following:

$d1$ = the number of data blocks in table 1

$d2$ = the number of data blocks in table 2

$p1$ = the number of non-excluded partitions in table 1

$p2$ = the number of non-excluded partitions in table 2

$k1$ = the number of partitions which can be held in memory for table 1

$k2$ = the number of partitions which can be held in memory for table 2

The number of disk reads required for the various types of joins are:

neither table partitioned: $d1 + d2$

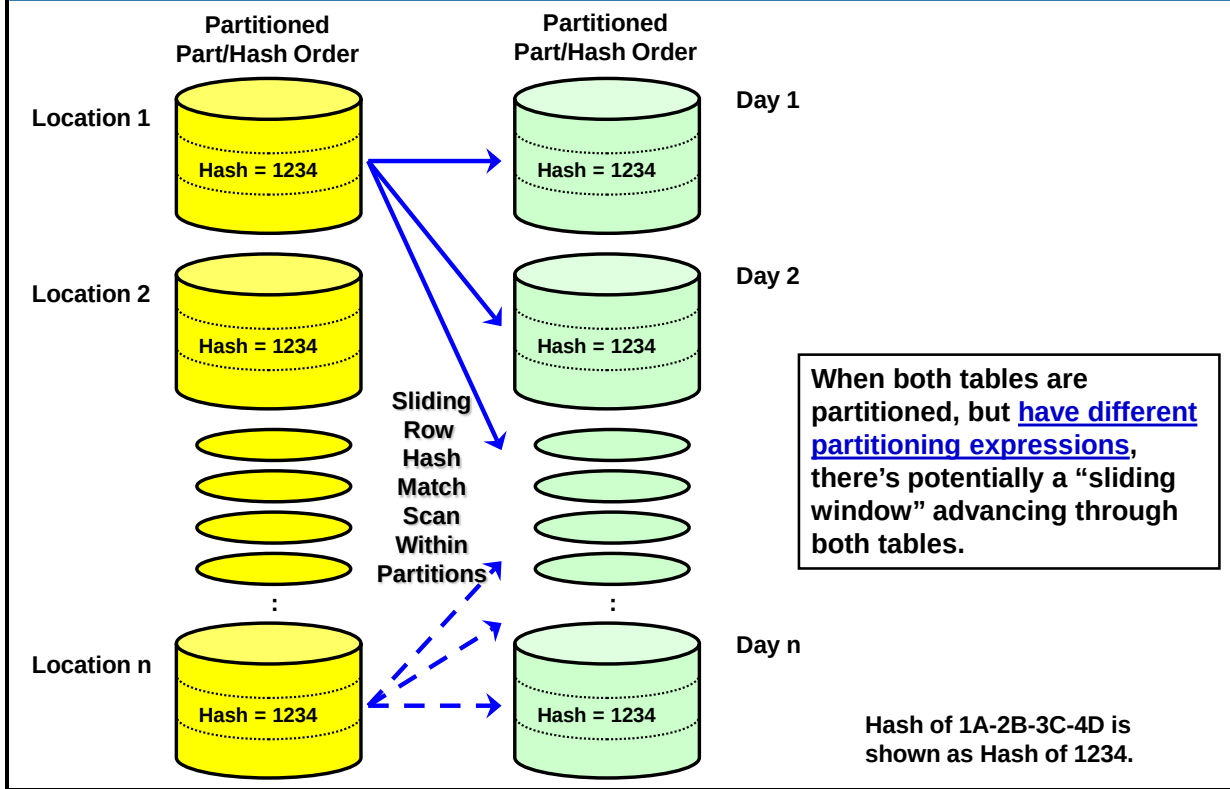
second table partitioned: $(p2/k2 * d1) + d2$

both tables partitioned: $(p2/k2 * d1) + (p1/k1 * d2)$

If p/k is less than one, some available memory won't be needed, and a value of one is used for p/k . For one partitioned table, p/k must be small (maybe 4 at the most) for performance to be attractive, unless the query conditions permit a large number of partitions to be excluded. (Without trying to address every join costing consideration, reading the non-PPI table twice, say, and reading 40% of the PPI table due to partition elimination, may be a less expensive operation than reading each table once, as in the non-PPI situation.)

For two partitioned tables, $p1/k1$ and $p2/k2$ must both be small (maybe 2 or 3 maximum) for performance to be attractive, unless a large number of partitions can be excluded.

Partitioned to Partitioned Table Join (Unmatched Partitions)



Additional Join Options with Partitioned Tables

The optimizer has other options than the “sliding window” join. As usual, the optimizer estimates the cost of all reasonable alternatives, and chooses the least expensive solution. Given the importance of the term p/k in the previously mentioned formulas, it is important that the optimizer have a realistic estimate of the number of non-excluded partitions.

This is the reason that the earlier examples that partitioned on “store_id” indicated that the RANGE_N example was better than the example that used the column directly. In the RANGE_N example, the optimizer knows that there are a maximum of ten “store_id” partitions with rows, and will use ten or a smaller number as the value of p . When the column is used directly, the optimizer may use a value much larger than ten when estimating p , especially if statistics haven’t been collected on the “store_id” column.

If one table is partitioned and the other isn't, the optimizer has two viable alternatives in addition to the “sliding window” join approach. One is to spool the partitioned table to a non-partitioned spool file, after which the join is between two non-partitioned relations. The other option, not always available, is to spool the non-partitioned table to a partitioned spool file, then directly join two identically partitioned relations. The partitioned spool file option is available only if the query specifies an equality condition on every column that is referenced in the partitioning expression.

If both tables are partitioned, it may be possible to spool one of the tables to a spool file with partitioning identical to the other table. It may also be cost-effective to spool both tables to non-partitioned spool files.

Some join algorithms do not change (e.g., when algorithm is not dependent on the table being in PI hash order), but usually benefit from a reduction in the number of blocks read because of partition elimination.

- Product join
- Classical hash join
- Nested join
- Row ID join

Note: A nested join would not generally be selected without a secondary index on inner table (with or without row partitioning).

The optimizer has other options than the “sliding window” join.

If one table is partitioned and the other isn't, alternatives to the “sliding window” join approach.

- One is to spool the partitioned table to a non-partitioned spool file, after which the join is between two non-partitioned relations.
- Spool the NPPI table to a partitioned spool file (matching the partitioning of the PPI table), then directly join two identically-partitioned relations.
 - This option is available only if the query specifies an equality condition on every column that is referenced in the partitioning expression.

If both tables are partitioned, possibilities are ...

- Spool one of the tables to a spool file with partitioning identical to the other table.
- Spool both tables to non-partitioned spool files.

As usual, the optimizer estimates the cost of all reasonable alternatives, and chooses the least expensive solution.

Join Processing Summary

The facing page summarizes many of the key points regarding Join Processing. There are three key factors:

1. Physical design choices
2. Availability of COLLECTed STATISTICS for design
3. Quality of SQL coding

Database design is the key to efficient Join Processing because the Optimizer bases its plans on Primary and Secondary Indexes.

COLLECTed STATISTICS are also vital since the Optimizer needs to know table Row Counts as well as Rows per Value. Make sure that the Optimizer always has fresh STATISTICS since data demographics change over time.

MAKE SURE you write efficient SQL code. But remember, even the best SQL code cannot compensate for poor database design choices.

Join Processing Summary

Inefficient joins result from:

- Poor physical design choices
 - Lack of indexes
 - Inappropriate indexes
- Stale or missing Collected Statistics
- Inefficient SQL code
 - Poor SQL code can degrade performance on a good database design.
 - Good SQL code cannot compensate for a poor database design.

The system bases join planning on:

- Primary and Secondary Indexes
- Estimated number of rows in each subtable
- Estimated ratio of table rows per index value

COLLECTed STATISTICS may improve join performance.

The fastest merge joins are based on matching Primary Indexes.

Data demographics change over time.

Join plans for the same tables change as demographics changes.

Revisit ALL index (Primary and Secondary) choices regularly. Make sure they are still serving you well.

Module 21: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 21: Review Questions

1. The best way to be sure what type of Join will occur is to use the EXPLAIN facility.
 - a. True
 - b. False
2. When two tables are to be Merge Joined, which is the best case of the following scenarios :
 - a. The Join column(s) are the Primary Index of both tables.
 - b. The Join column(s) are not a Primary Index of either table.
 - c. The Join column(s) are the Primary Index of one of the tables.
 - d. The Join column(s) are a Unique Secondary Index on both of the tables.
3. Match the four join plans with its correct description.

___ Product Join	a. Most efficient type of Join; the only type of join that doesn't always use all of the AMPs. The number of AMPs involved will vary.
___ Merge Join	b. Based on set subtraction; used for finding rows that don't have a matching row in the other table. Queries with the NOT IN and EXCEPT operator lead to this type of join.
___ Nested Join	c. Every qualifying row of one table is compared to every qualifying row in the other table. Rows that match on their WHERE conditions are then saved.
___ Exclusion Join	d. Commonly done when the join condition is based on equality. Efficient because every row is not compared with every row in other table.

Module 21: Review Questions (cont.)

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 21: Review Questions (cont.)

Fill in the blanks.

4. When joining two row partitioned tables that are not partitioned in the same manner, a technique available to the optimizer is referred to as the _____ window.
5. A direct merge join of two row partitioned tables when the tables have the same PI and identical partitioning expressions is referred to as a _____ - based merge join.
6. The term _____ _____ refers to an automatic optimization in which the optimizer determines, based on query conditions, that some partitions can be skipped.

Notes



Explains of Joins and Index Choices

After completing this module, you will be able to:

- Interpret the EXPLAINS of Nested, Merge, and Product Joins.
- Develop Union solutions.
- Explain the importance of Join Accesses in Index selection.
- Use the Index choice guidelines.

Teradata Proprietary and Confidential

Notes

Table of Contents

Join Diagramming.....	22-4
Nested Join.....	22-6
Merge Join (Matching Primary Indexes)	22-8
Hash Join.....	22-10
Merge Join (Joining a Table to Itself).....	22-12
Three-Table Join	22-14
Product Join.....	22-18
A “UNION” Solution.....	22-22
Cartesian Product Join.....	22-26
Exclusion Join.....	22-28
Example of Joining 2 Partitioned Tables	22-30
PRPD Join Explain Text	22-32
Teradata PRPD Join Example.....	22-34
PRPD One Relation Skewed on a Non-PI Column	22-36
PRPD One Relation Skewed on a Non-PI Column (cont.).....	22-38
Join ACCESS.....	22-40
Exercise 4 – Example.....	22-42
Exercise 4 – Making Final Index Choices	22-44
Summary – Index Choice Guidelines	22-56
Module 22: Review Questions	22-58

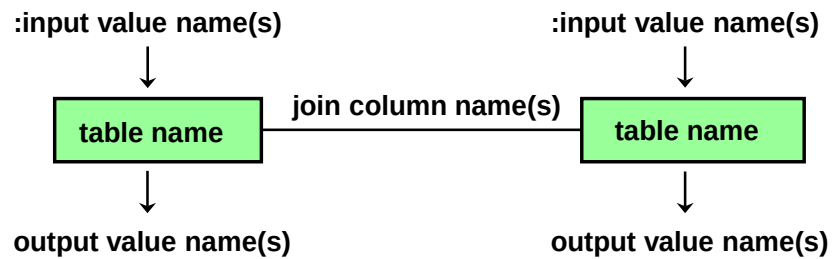
Join Diagramming

Join Diagramming can save you from costly coding errors by helping you understand what occurs when Teradata performs a Join. The example on the facing page illustrates a two-table Join.

Four distinct pieces of information are provided in Join Diagrams. Three of these are required and one is optional. They are:

- Table Names appear in the boxes to indicate which tables are being joined. 2 to 128 tables can participate in a Join operation. (Required)
- Join Column Name(s) indicate the column(s) used to do the Join, and are based on the same domain with an equality condition. They appear above the lines between the tables. At least one Join Column Name is between every pair of tables. (Required)
- Input Value Name(s) provide Set Selection values. They appear above the boxes containing the table names. They can be constants, host variables, or macro parameters which appear in the WHERE clause of the SQL statement. (Optional)
- Output Value Name(s) indicate which column(s), aggregate(s), or expression(s) will be output from a particular table. They appear below the boxes containing the table names and there is at least one for the query. (Required)

Join Diagramming



:Input value names – Optional

Constants, host variables, or macro parameters that supply Set Selection values.

Table name

From 2 to 128 tables may participate in a Join operation.

Join column name(s)

Based on the same domain and an equality condition.

Output value name(s)

Specify at least one column, aggregate or expression.

Nested Join

In this module, you will study various Joins. You will see different types of information relating to Joins. These are presented in the same order that you would use when designing a Teradata query. You would:

- State the query
- Diagram the query
- Code the query
- EXPLAIN the query

You must understand what you are asking the system to do before you attempt to understand how it does it.

In this example, Step 1 is not a locking step as in the other EXPLAINS that you have seen since it does not involve all AMPs. The use of the two UPIs allows the locks to be acquired within the AMP step and immediately released.

The tables used in this Join were created as follows:

```
CREATE SET TABLE TFACT.Employee, Fallback
  (EmpNumber      INTEGER NOT NULL,
   LocationNumber INTEGER,
   DeptNumber     INTEGER,
   EmpMgrNumber   INTEGER,
   JobCode        INTEGER,
   LastName       CHAR(20),
   FirstName      VARCHAR(20),
   SalaryAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Department, Fallback
  (DeptNumber      INTEGER NOT NULL,
   DeptName        CHAR(20 NOT NULL,
   DeptMgrNumber   INTEGER,
   BudgetAmount    DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( DeptNumber );
```

The Employee table has 150,000 rows and the Department table has 17,415 rows. Statistics were collected on the primary indexes and any join columns.

Nested Join

QUERY

EXPLAIN

SELECT

LastName,
FirstName,
DeptName

FROM

Employee E

INNER JOIN

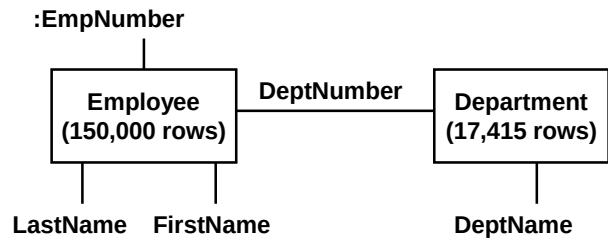
Department D

ON

E.DeptNumber = D.DeptNumber

WHERE

E.EmpNumber = 102001;



EXPLANATION

16.20 EXPLAIN

This request is eligible for incremental planning and execution (IPE) but does not meet cost thresholds. The following is the static plan for the request.

- 1) First, we do a **single-AMP JOIN step from TFACT.E by way of the unique primary index** "TFACT.E.EmpNumber = 102001" with a residual condition of ("NOT (TFACT.E.DeptNumber IS NULL)"), which is joined to all partitions of TFACT.D by way of the primary index "TFACT.D.DeptNumber = TFACT.E.DeptNumber" with no residual conditions. **TFACT.E and TFACT.D are joined using a nested join**, with a join condition of ("**1=1**"). The result goes into Spool 1 (one-amp), which is built locally on that AMP. **The size of Spool 1 is estimated with high confidence to be 1 row (149 bytes)**. The estimated time for this step is 0.00 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

Merge Join (Matching Primary Indexes)

For this example and those that follow, make sure that you understand the Narrative, Join Diagram and Query before looking at the EXPLAIN output.

**You must understand what you are asking the system to do
before you attempt to understand how it does it.**

The EXPLAIN tells you that this is a Merge Join. Note that there is no redistribution of rows or sorting which means that Merge Join Plan is being used. In this example, the Join Columns are the Primary Indexes of both tables. No redistribution or sorting is needed since the rows are already on the proper AMPs and in the proper order for Joining.

The tables used in this Join were created as follows:

```
CREATE SET TABLE TFACT.Employee, FALLBACK
  (EmpNumber          INTEGER NOT NULL,
   LocationNumber     INTEGER,
   DeptNumber         INTEGER,
   EmpMgrNumber       INTEGER,
   JobCode            INTEGER,
   LastName           CHAR(20),
   FirstName          VARCHAR(20),
   SalaryAmount       DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );

CREATE SET TABLE TFACT.EmpPhone, FALLBACK
  (EmpNumber          INTEGER,
   AreaCode           SMALLINT,
   PhoneNumber        INTEGER,
   Extension          INTEGER)
PRIMARY INDEX (EmpNumber);
```

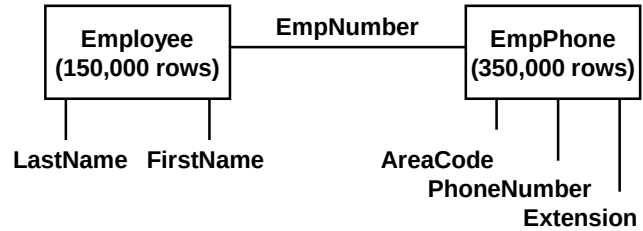
The Employee table has 150,000 rows and the EmpPhone table has 350,000 rows. Statistics were collected on the primary indexes and any join columns.

Merge Join (Matching Primary Indexes)

QUERY

```

EXPLAIN
SELECT      LastName, FirstName,
            AreaCode, PhoneNumber,
            Extension
FROM        Employee E
INNER JOIN  EmpPhone P
ON          E.EmpNumber = P.EmpNumber
ORDER BY   1, 2;
  
```



EXPLANATION

16.20 EXPLAIN

: (Locking steps)

- 4) We do an **all-AMPs JOIN step in TD_MAP1** from TFACT.E by way of a RowHash match scan with no residual conditions, which is joined to TFACT.P. TFACT.E and TFACT.P are **joined using a merge join**, with a join condition of ("TFACT.E.EmpNumber = TFACT.P.EmpNumber"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. **Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.E.LastName, TFACT.E.FirstName)**. The size of Spool 1 is estimated with low confidence to be 350,000 rows (33,250,000 bytes). The estimated time for this step is 1.17 seconds.
- 5) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 1.17 seconds.

Note: EmpNumber is NOT NULL in both tables.

Hash Join

The EXPLAIN output tells you that this is Hash Join, however, distribution is of the Employee table is required. The rows from the employee table are "**redistributed (hashed) across all AMPs**". The rows from the department table are placed into memory for each AMP and are joined to the employee table via a single partition hash join.

The tables used in this Join were created as follows:

```
CREATE SET TABLE TFACT.Employee, FALLBACK
  (EmpNumber      INTEGER NOT NULL,
   LocationNumber  INTEGER,
   DeptNumber     INTEGER,
   EmpMgrNumber   INTEGER,
   JobCode        INTEGER,
   LastName       CHAR(20),
   FirstName      VARCHAR(20),
   SalaryAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Department, FALLBACK
  (DeptNumber     INTEGER NOT NULL,
   DeptName       CHAR(20 NOT NULL,
   DeptMgrNumber  INTEGER,
   BudgetAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( DeptNumber );
```

The Employee table has 150,000 rows and the Department table has 17,415 rows. Statistics were collected on the primary indexes and any join columns.

Hash Join

QUERY

EXPLAIN

SELECT

LastName,
FirstName,
DeptName

FROM

Employee E

INNER JOIN

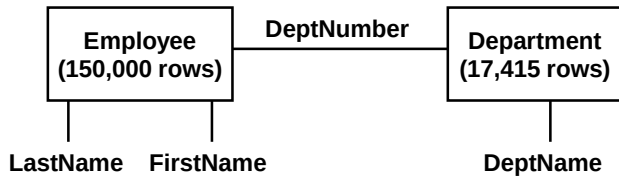
Department D

ON

E.DeptNumber = D.DeptNumber

ORDER BY

1, 2;



EXPLANATION

16.20 EXPLAIN

: (Locking steps)

- 4) We do an **all-AMPs RETRIEVE step in TD_MAP1 from TFACT.E** by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), **which is redistributed by the hash code of (TFACT.E.DeptNumber) to all AMPs in TD_MAP1**. The size of Spool 2 is estimated with high confidence to be 150,000 rows (6,750,000 bytes). The estimated time for this step is 0.08 seconds.
 - 5) We do an **all-AMPs JOIN step in TD_MAP1 from TFACT.D** by way of an all-rows scan with no residual conditions, **which is joined to Spool 2 (Last Use) by way of an all-rows scan. TFACT.D and Spool 2 are joined using a single partition hash join**, with a join condition of ("DeptNumber = TFACT.D.DeptNumber"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.E.LastName, TFACT.E.FirstName). The size of Spool 1 is estimated with low confidence to be 150,000 rows (27,750,000 bytes). The estimated time for this step is 0.94 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 1.19 seconds.

Merge Join (Joining a Table to Itself)

The facing page shows a Self Join involving the employee table. Two aliases for the employee table (M for manager and E for employee) are defined in the SQL query. This helps the Optimizer treat the single employee table as two tables.

The EXPLAIN output shows that the rows from table alias E are redistributed and sorted before being Joined to the rows from table alias M using a Merge Join.

As before, the table used in this Join was created as follows:

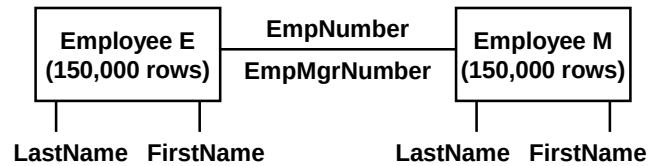
```
CREATE SET TABLE TFACT.Employee, FALLBACK  
  (EmpNumber          INTEGER NOT NULL,  
   LocationNumber    INTEGER,  
   DeptNumber         INTEGER,  
   EmpMgrNumber       INTEGER,  
   JobCode            INTEGER,  
   LastName           CHAR(20),  
   FirstName          VARCHAR(20),  
   SalaryAmount       DECIMAL(10,2))  
UNIQUE PRIMARY INDEX ( EmpNumber )  
INDEX ( JobCode )  
INDEX ( DeptNumber );
```

The Employee table has 150,000 rows. Statistics were collected on the primary indexes and any join columns.

Merge Join (Joining a Table to Itself)

QUERY

```
EXPLAIN
SELECT      M.LastName, M.FirstName,
            E.LastName, E.FirstName
FROM        Employee M
INNER JOIN  Employee E
ON          M.EmpNumber = E.EmpMgrNumber
ORDER BY    1, 3;
```



EXPLANATION

16.20 EXPLAIN

- : (Locking steps)
- 3) We do an **all-AMPs RETRIEVE step in TD_MAP1 from TFACT.E** by way of an all-rows scan with a condition of ("NOT (TFACT.E.EmpMgrNumber IS NULL)") into **Spool 2** (all_amps), which is **redistributed by the hash code of (TFACT.E.EmpMgrNumber)** to all AMPs in TD_MAP1. The size of Spool 2 is estimated with high confidence to be 150,000 rows (6,750,000 bytes). The estimated time for this step is 0.24 seconds.
 - 4) We do an **all-AMPs JOIN step in TD_MAP1 from TFACT.M** by way of an all-rows scan with no residual conditions, which is **joined to Spool 2** (Last Use) by way of an all-rows scan. **TFACT.M and Spool 2 are joined using a single partition hash join**, with a join condition of ("TFACT.M.EmpNumber = EmpMgrNumber"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.M.LastName, TFACT.E.LastName). The size of Spool 1 is estimated with index join confidence to be 150,000 rows (16,950,000 bytes). The estimated time for this step is 0.62 seconds.
 - 5) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.86 seconds.

Three-Table Join

So far, you have only seen the EXPLAINs of Joins involving two tables. In this three-table Join, you will be able to see how the Optimizer chooses a Join Plan that involves a series of two two-table Joins.

The purpose of the query is to display the names of all employees, their department names and their job descriptions. The Join diagram shows that the job, employee and department tables will all be involved in the Join.

As before, the tables used in this Join were created as follows:

The tables used in this Join were created as follows:

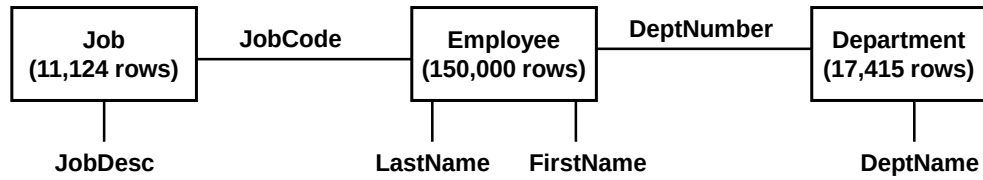
```
CREATE SET TABLE TFACT.Employee, Fallback
  (EmpNumber      INTEGER NOT NULL,
   LocationNumber INTEGER,
   DeptNumber     INTEGER,
   EmpMgrNumber   INTEGER,
   JobCode        INTEGER,
   LastName       CHAR(20),
   FirstName      VARCHAR(20),
   SalaryAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Department, Fallback
  (DeptNumber      INTEGER NOT NULL,
   DeptName       CHAR(20) NOT NULL,
   DeptMgrNumber   INTEGER,
   BudgetAmount    DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Job, Fallback
  (JobCode      INTEGER NOT NULL,
   JobDesc      CHAR(20) NOT NULL DEFAULT 'Job Description')
UNIQUE PRIMARY INDEX ( JobCode )
INDEX ( JobDesc );
```

The Employee table has 150,000 rows, the Department table has 17,415 rows, and the Job table has 11,124 rows. Statistics were collected on the primary indexes and any join columns.

Three-Table Join



QUERY

EXPLAIN
SELECT

E.LastName,
E.FirstName,
D.DeptName,
J.JobDesc

FROM

INNER JOIN

INNER JOIN

ORDER BY

Employee E
Department D ON E.DeptNumber = D.DeptNumber
Job J ON E.JobCode = J.JobCode
3, 1, 2;

EXPLAIN output on following page.

Three-Table Join (cont.)

The query and table definitions are repeated for your convenience:

```
EXPLAIN
SELECT      LastName,
            FirstName,
            DeptName,
            JobDesc
FROM        Employee E
INNER JOIN  Department D      ON E.DeptNumber = D.DeptNumber
INNER JOIN  Job J            ON E.JobCode = J.JobCode
ORDER BY   3, 1, 2;
```

```
CREATE SET TABLE TFACT.Employee, Fallback
  (EmpNumber      INTEGER NOT NULL,
   LocationNumber INTEGER,
   DeptNumber     INTEGER,
   EmpMgrNumber   INTEGER,
   JobCode        INTEGER,
   LastName       CHAR(20),
   FirstName      VARCHAR(20),
   SalaryAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Department, Fallback
  (DeptNumber     INTEGER NOT NULL,
   DeptName       CHAR(20 NOT NULL,
   DeptMgrNumber  INTEGER,
   BudgetAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Job, Fallback
  (JobCode        INTEGER NOT NULL,
   JobDesc        CHAR(20) NOT NULL DEFAULT 'Job Description')
UNIQUE PRIMARY INDEX ( JobCode )
INDEX ( JobDesc );
```

As a side note, the join conditions in the query were changed in order as shown below, but the exact same EXPLAIN plan is generated as shown on the facing page.

```
EXPLAIN
SELECT      E.LastName, E.FirstName,
            D.DeptName,
            J.JobDesc
FROM        Employee E
INNER JOIN  Job J            ON E.JobCode = J.JobCode
INNER JOIN  Department D      ON E.DeptNumber = D.DeptNumber
ORDER BY   3, 1, 2;
```

Three-Table Join (cont.)

EXPLANATION

16.20 EXPLAIN

- 1) First, we lock TFACT.J in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) Next, we lock TFACT.D in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 3) We lock TFACT.E in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 4) We lock TFACT.J in TD_MAP1 for read, we lock TFACT.D in TD_MAP1 for read, and we lock TFACT.E in TD_MAP1 for read.
- 5) We do an **all-AMPs RETRIEVE step in TD_MAP1 from TFACT.J** by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), **which is duplicated on all AMPs in TD_Map1**. The size of Spool 2 is estimated with high confidence to be 177,984 rows (11,924,928 bytes). The estimated time for this step is 0.13 seconds.
- 6) We do an **all-AMPs JOIN step in TD_Map1 from Spool 2** (Last Use) by way of an all-rows scan, which is **joined to TFACT.E** by way of an all-rows scan with no residual conditions. **Spool 2 and TFACT.E are joined using a dynamic hash join, with a join condition of ("TFACT.E.JobCode = JobCode")**. The result goes into Spool 3 (all_amps), which is redistributed by the hash code of (TFACT.E.DeptNumber) to all AMPs in TD_Map1. The size of Spool 3 is estimated with low confidence to be 150,000 rows (14,250,000 bytes). The estimated time for this step is 0.50 seconds.
- 7) We do an **all-AMPs JOIN step in TD_MAP1 from TFACT.D** by way of an all-rows scan with no residual conditions, which is **joined to Spool 3** (Last Use) by way of an all-rows scan. **TFACT.D and Spool 3 are joined using a single partition hash join, with a join condition of ("DeptNumber = TFACT.D.DeptNumber")**. The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1(TFACT.D.DeptName, TFACT.E.LastName, TFACT.E.FirstName). The size of Spool 1 is estimated with low confidence to be 150,000 rows (50,550,000 bytes). The estimated time for this step is 1.11 seconds.
- 8) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 1.74 seconds.

Product Join

The query in this example results in a Product Join involving the employee and department tables. The query is designed to return the names and department names of all employees who are either workers or managers. Managers may be listed both as an employee of a department and once as a manager of a department (if they are employed by a department other than the one which they manage).

The use of the OR in the query is the reason that this query's plan will be a Product Join.

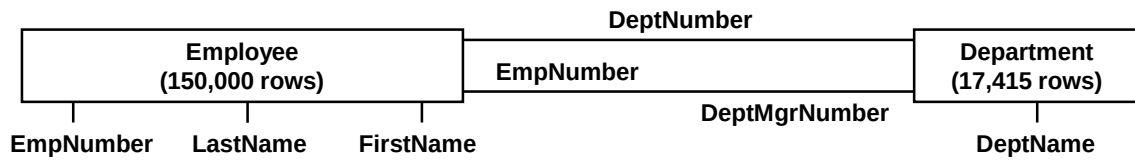
As before, the tables used in this Join were created as follows:

```
CREATE SET TABLE TFACT.Employee, FALLBACK
  (EmpNumber      INTEGER NOT NULL,
   LocationNumber INTEGER,
   DeptNumber     INTEGER,
   EmpMgrNumber   INTEGER,
   JobCode        INTEGER,
   LastName       CHAR(20),
   FirstName      VARCHAR(20),
   SalaryAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );

CREATE SET TABLE TFACT.Department, FALLBACK
  (DeptNumber      INTEGER NOT NULL,
   DeptName       CHAR(20 NOT NULL,
   DeptMgrNumber   INTEGER,
   BudgetAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( DeptNumber );
```

The Employee table has 150,000 rows and the Department table has 17,415 rows. Statistics were collected on the primary indexes and any join columns.

Product Join



```

QUERY
EXPLAIN
SELECT      D.DeptName,
               E.EmpNumber,
               E.LastName,
               E.FirstName
FROM        Employee E
INNER JOIN   Department D
ON          E.DeptNumber = D.DeptNumber
OR          E.EmpNumber = D.DeptMgrNumber
ORDER BY    1, 2, 3, 4;
  
```

EXPLAIN output on following page.

Product Join (cont.)

The EXPLAIN tells you that this is a Product Join. This product join uses considerable system resources for even small tables.

The query and table definitions are repeated for your convenience:

```
EXPLAIN
SELECT      D.DeptName,
            E.EmpNumber,
            E.LastName,
            E.FirstName
FROM        Employee E
INNER JOIN  Department D
ON          E.DeptNumber = D.DeptNumber
OR          E.EmpNumber = D.DeptMgrNumber
ORDER BY   1, 2, 3, 4;
```

```
CREATE SET TABLE TFACT.Employee, FALLBACK
  (EmpNumber      INTEGER NOT NULL,
   LocationNumber INTEGER,
   DeptNumber     INTEGER,
   EmpMgrNumber   INTEGER,
   JobCode        INTEGER,
   LastName       CHAR(20),
   FirstName      VARCHAR(20),
   SalaryAmount   DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( EmpNumber )
INDEX ( JobCode )
INDEX ( DeptNumber );
```

```
CREATE SET TABLE TFACT.Department, FALLBACK
  (DeptNumber     INTEGER NOT NULL,
   DeptName       CHAR(20 NOT NULL,
   DeptMgrNumber  INTEGER,
   BudgetAmount  DECIMAL(10,2))
UNIQUE PRIMARY INDEX ( DeptNumber );
```

The Employee table has 150,000 rows and the Department table has 17,415 rows. Statistics were collected on the primary indexes and any join columns.

EXPLANATION

16.20 EXPLAIN

- 1) First, we lock TFACT.D in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) Next, we lock TFACT.E in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 3) We lock TFACT.D in TD_MAP1 for read, and we lock TFACT.E in TD_MAP1 for read.
- 4) We do an **all-AMPs RETRIEVE step in TD_MAP1 from TFACT.D by way of an all-rows scan** with no residual conditions into Spool 2 (all_amps), which is **duplicated on all AMPs** in TD_Map1. The size of Spool 2 is estimated with high confidence to be 278,640 rows (33,715,440 bytes). The estimated time for this step is 0.30 seconds.
- 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.E by way of an all-rows scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of an all-rows scan. **TFACT.E and Spool 2 are joined using a product join, with a join condition of ("(TFACT.E.DeptNumber = DeptNumber) OR (TFACT.E.EmpNumber = DeptMgrNumber)").** The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.D.DeptName, TFACT.E.EmpNumber, TFACT.E.LastName, TFACT.E.FirstName). The size of Spool 1 is estimated with index join confidence to be 167,414 rows (50,056,786 bytes). The estimated time for this step is 2 minutes and 42 seconds.
- 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is **2 minutes and 43 seconds.**

A “UNION” Solution

This example illustrates that sometimes there are more efficient ways for coding an SQL query. The query on the facing page is designed to yield the same answer set as the previous SQL query.

The query is:

```
EXPLAIN  
SELECT    D.DeptName, E.EmpNumber, E.LastName, E.FirstName  
FROM      Employee E INNER JOIN Department D  
ON        E.DeptNumber = D.DeptNumber  
UNION  
SELECT    D.DeptName, E.EmpNumber, E.LastName, E.FirstName  
FROM      Employee E INNER JOIN Department D  
ON        E.EmpNumber = D.DeptMgrNumber  
ORDER BY  1, 2, 3, 4;
```

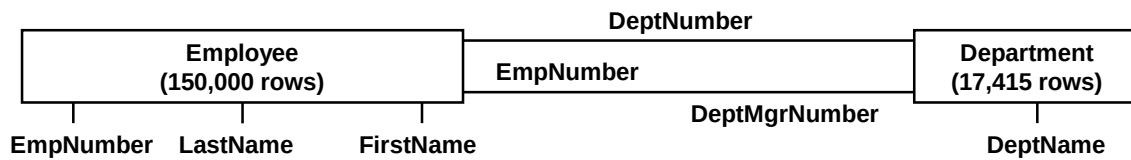
This example illustrates that sometimes there are more efficient ways for coding an SQL query. The query on the facing page is designed to yield the same answer set as the previous SQL query. Both queries return exactly the same answer set.

A key to understanding the UNION and UNION ALL is that a UNION will remove any duplicate rows from the output and UNION ALL will keep all duplicate rows.

However, if the queries were changed slightly and the EmpNumber column was not included, the UNION example would return fewer rows than the Product Join (via the OR).

This discussion is continued on the following text page and illustrates an example where a PRODUCT JOIN and a UNION might not produce the same result set.

A "UNION" Solution



QUERY

```

EXPLAIN  SELECT  D.DeptName, E.EmpNumber, E.LastName, E.FirstName
          FROM    Employee E INNER JOIN Department D
          ON      E.DeptNumber = D.DeptNumber
          UNION
          SELECT  D.DeptName, E.EmpNumber, E.LastName, E.FirstName
          FROM    Employee E INNER JOIN Department D
          ON      E.EmpNumber = D.DeptMgrNumber
          ORDER BY 1, 2, 3, 4;
  
```

EXPLAIN output on following page.

A “UNION” Solution (cont.)

A simple example is provided with the following data in Employee and the Department tables. Assume that the Employee table has two different employees (different Employee Numbers) that have the same name “Paul Winters”. The difference in the three examples is that a UNION deletes any duplicate rows and a UNION ALL keeps all duplicate rows.

Employee Rows:	<u>DeptNumber</u>	<u>EmpNumber</u>	<u>LastName</u>	<u>FirstName</u>
	1104	101056	Winters	Paul
	1104	101078	Winters	Paul

Department Row:	<u>DeptNumber</u>	<u>DeptName</u>	<u>DeptMgrNumber</u>
	1104	Education	101078

In essence, the following three queries will each return a different answer set.

```
SELECT      D.DeptName, E.LastName, E.FirstName
FROM
INNER JOIN  Employee E
ON          Department D
            E.DeptNumber = D.DeptNumber
OR          E.EmpNumber = D.DeptMgrNumber;
```

Returns:	<u>DeptName</u>	<u>LastName</u>	<u>FirstName</u>
	Education	Winters	Paul
	Education	Winters	Paul

```
SELECT      D.DeptName, E.LastName, E.FirstName
FROM
ON          Employee E INNER JOIN Department D
            E.DeptNumber = D.DeptNumber
UNION
SELECT      D.DeptName, E.LastName, E.FirstName
FROM
ON          Employee E INNER JOIN Department D
            E.EmpNumber = D.DeptMgrNumber;
```

Returns:	<u>DeptName</u>	<u>LastName</u>	<u>FirstName</u>
	Education	Winters	Paul

```
SELECT      D.DeptName, E.LastName, E.FirstName
FROM
ON          Employee E INNER JOIN Department D
            E.DeptNumber = D.DeptNumber
UNION ALL
SELECT      D.DeptName, E.LastName, E.FirstName
FROM
ON          Employee E INNER JOIN Department D
            E.EmpNumber = D.DeptMgrNumber;
```

Returns:	<u>DeptName</u>	<u>LastName</u>	<u>FirstName</u>
	Education	Winters	Paul
	Education	Winters	Paul
	Education	Winters	Paul

A “UNION” Solution (cont.)

: (Locking steps)

16.20 EXPLAIN

- 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.E by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), which is redistributed by the hash code of (TFACT.E.DeptNumber) to all AMPs in TD_Map1. The size of Spool 2 is estimated with high confidence to be 150,000 rows (7,350,000 bytes). The estimated time for this step is 0.25 seconds.
- 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.D by way of an all-rows scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of an all-rows scan. TFACT.D and Spool 2 are joined using a single partition hash join, with a join condition of ("DeptNumber = TFACT.D.DeptNumber"). The result goes into Spool 1 (group_amps), which is redistributed by the hash code of (TFACT.E.FirstName, TFACT.E.LastName, TFACT.E.EmpNumber, TFACT.D.DeptName) to all AMPs in TD_Map1. The size of Spool 1 is estimated with low confidence to be 150,000 row (44,850,000 bytes). The estimated time for this step is 0.56 seconds.
- 6) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.D by way of an all-rows scan with a condition of ("NOT (TFACT.D.DeptMgrNumber IS NULL)") into Spool 3 (all_amps), which is redistributed by the hash code of (TFACT.D.DeptMgrNumber) to all AMPs in TD_Map1. Then we do a SORT to order Spool 3 by row hash. The size of Spool 3 is estimated with high confidence to be 17,415 rows (2,037,555 bytes). The estimated time for this step is 0.06 seconds.
- 7) We do an all-AMPs JOIN step in TD_Map1 from Spool 3 (Last Use) by way of a RowHash match scan, which is joined to TFACT.E by way of a RowHash match scan with no residual conditions. Spool 3 and TFACT.E are joined using a merge join, with a join condition of ("TFACT.E.EmpNumber = DeptMgrNumber"). The result goes into Spool 1 (group_amps), which is redistributed by the hash code of (TFACT.E.FirstName, TFACT.E.LastName, TFACT.E.EmpNumber, TFACT.D.DeptName) to all AMPs in TD_Map1. Then we do a SORT to order Spool 1 by the sort key in spool field1 eliminating duplicate rows. The size of Spool 1 is estimated with index join confidence to be 151,812 rows (45,391,788 bytes). The estimated time for this step is 0.86 seconds.
- 8) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 1.73 seconds.

Cartesian Product Join

The example on the facing page illustrates a Cartesian Product Join.

It confirms the admonishment that no SQL should be allowed into production until it has been EXPLAINED and approved.

The TEST table used in this Join was created as follows:

```
CREATE TABLE Test, NO FALLBACK  
( col1      INTEGER NOT NULL  
  ,col2      INTEGER  
  ,col3      INTEGER  
  ,col4      INTEGER  
  ,col5      CHAR(35))  
PRIMARY INDEX (col1);
```

The TEST table was populated with 4000 rows and summary statistics were collected on the table. This EXPLAIN plan's cost of 155,301 hours is almost 18 years. 😊

Cartesian Product Join



QUERY EXPLAIN SELECT * FROM Test A
CROSS JOIN Test B
CROSS JOIN Test C
CROSS JOIN Test D;

- : (locking steps) 16.20 EXPLAIN
- 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.D by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), which is duplicated on all AMPs. The size of Spool 2 is estimated with high confidence to be 64,000 rows (4,096,000 bytes). The estimated time for this step is 0.06 seconds.
 - 4) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.B by way of an all-rows scan with no residual conditions, which is joined to Spool 2 by way of an all-rows scan. TFACT.B and Spool 2 are joined using a product join, with a join condition of ("(1=1)"). The result goes into Spool 4 (all_amps), which is built locally on the AMPs. The size of Spool 4 is estimated with high confidence to be 16,000,000 rows (1,840,000,000 bytes). The estimated time for this step is 21.45 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.A by way of an all-rows scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of an all-rows scan. TFACT.A and Spool 2 are joined using a product join, with a join condition of ("(1=1)"). The result goes into Spool 5 (all_amps), which is duplicated on all AMPs. The result spool file will not be cached in memory. The size of Spool 5 is estimated with high confidence to be 256,000,000 rows (29,440,000,000 bytes). The estimated time for this step is 3 minutes and 46 seconds.
 - 6) We do an all-AMPs JOIN step in TD_MAP1 from Spool 4 (Last Use) by way of an all-rows scan, which is joined to Spool 5 (Last Use) by way of an all-rows scan. Spool 4 and Spool 5 are joined using a product join, with a join condition of ("(1=1)"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The result spool file will not be cached in memory. The size of Spool 1 is estimated with high confidence to be 256,000,000,000,000 rows (** bytes). The estimated time for this step is 155,301 hours and 25 minutes.
 - 7) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 155,301 hours and 29 minutes.

Note: The optimizer knows that Spool 2 would be the same for aliases D and C and reuses Spool 2 (for alias C) in step 5.

Exclusion Join

The query on the facing page is designed to return a list of all Sales Representatives who do not have any customers. In this example, the employee table is merged with the customer table. The Join Diagram shows that the employee table will be joined to the customer table via the EmpNumber (SalesEmpNumber) column.

The EXPLAIN output tells you that the Optimizer will choose an Exclusion Merge Join. In fact, this is the same query used to illustrate the Exclusion Merge Join in the last module.

NOT IN operation uses three-value logic (equal, not equal, null). If the subquery side of the join contains a null, then the entire answer becomes null. The testing for these null values adds additional processing overhead to NOT IN operation. To reduce the tests down to two-value logic, equal, not equal, the join columns must be declared NOT NULL.

The Employee table was populated with 150,000 rows and the Customer table was populated with 25,000 rows. Statistics were collected on the primary indexes and any “join” columns.

Exclusion Join

QUERY

EXPLAIN

```
SELECT LastName, FirstName
FROM Employee E
WHERE JobCode = 3100
AND EmpNumber
NOT IN (SELECT SalesEmpNumber FROM Customer C WHERE SalesEmpNumber IS NOT NULL);
```

JobCode (3100)

Employee
(150,000 rows)

EmpNumber

Customer
(25,000 rows)

LastName FirstName

EXPLANATION

16.20 EXPLAIN

: Locking steps

- 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.C by way of an all-rows scan with no residual conditions into Spool 2 (all_amps), which is redistributed by the hash code of (TFACT.C.SalesEmpNumber) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 2 by the sort key in spool field1 eliminating duplicate rows. The size of Spool 2 is estimated with high confidence to be 25,000 rows (625,000 bytes). The estimated time for this step is 0.09 seconds.
- 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.E by way of an all-rows scan with a condition of ("TFACT.E.JobCode = 3100"), which is joined to Spool 2 (Last Use) by way of an all-rows scan. TFACT.E and Spool 2 are joined using an exclusion product join, with a join condition of ("TFACT.E.EmpNumber = SalesEmpNumber"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with index join confidence to be 13 rows (637 bytes). The estimated time for this step is 0.17 seconds.
- 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.26 seconds.

Example of Joining 2 Partitioned Tables

An example of joining two tables partitioned in the same way is shown on the facing page. Notice that the Explain plan indicates that a “rowkey-based merge join” is performed.

Statistics were collected on the claimid, custid, claimdate and Partition columns for each of the tables.

Example of Joining 2 Partitioned Tables

QUERY

```
EXPLAIN
SELECT      *
FROM        Claim_RP A
INNER JOIN  Claim_RP_MS B
ON          A.claimid = B.claimid
AND         A.claimdate = B.claimdate;
```

The Primary Index and the partitioning expression for Claim_RP and Claim_RP_MS are the same.

EXPLANATION

16.20 EXPLAIN

- 1) First, we lock TFACT.B in TD_MAP1 for read on a reserved RowHash in all partitions to prevent global deadlock.
- 2) Next, we lock TFACT.A in TD_MAP1 for read on a reserved RowHash in all partitions to prevent global deadlock.
- 3) We lock TFACT.B for read in TD_MAP1, and we lock TFACT.A for read in TD_MAP1.
- 4) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.B by way of a RowHash match scan with no residual conditions, which is joined to TFACT.A by way of a RowHash match scan with no residual conditions. TFACT.B and TFACT.A are joined using a rowkey-based merge join, with a join condition of "(TFACT.A.claimid = TFACT.B.claimid) AND (TFACT.A.claimdate = TFACT.B.claimdate)". The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The result spool file will not be cached in memory. The size of Spool 1 is estimated with low confidence to be 30,000,000 rows (7,950,000,000 bytes). The estimated time for this step is 1 minute and 35 seconds.
- 5) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 1 minute and 35 seconds.

PRPD Join Explain Text

With PRPD, a binary join on skewed data is split into 2 or 3 subjoins (called PRPD physical joins). Each PRPD physical join is performed on a portion of the sources using the best regular join method picked by the optimizer for these data sets. The join results of these PRPD physical joins are combined as the final result in a common spool. The overall PRPD join plan competes with other regular join plans based on cost.

If PRPD plan is picked, the explain text has two splitting steps for every PRPD join plan that writes rows into multiple spools. Splitting happens as part of a retrieve or join step.

PRPD Join Explain Text

Partial Redistribution Partial Duplication (PRPD) – 14.10 Feature

New Explain Text – Split step can happen in a single retrieve or join step, with the following new text:

2-fold PRPD (used if one table/spool is skewed)

- **For the skewed relation:**
split into Spool xxx (all_amps) with a condition of (xxx) to qualify skewed rows and Spool xxx (all_amps) with remaining rows.
- **For the skew match relation:**
split into Spool xxx (all_amps) with a condition of (xxx) to qualify rows matching skewed rows of the skewed relation and Spool xxx (all_amps) with remaining rows.

3-fold PRPD (used if both of the tables/spool is skewed)

- **For both the skewed and skew match relations:**
split into Spool xxx (all_amps) with a condition of (xxx) to qualify skewed rows and spool xxx (all_amps) with a condition of (xxx) to qualify rows matching skewed rows of the skewed relation and Spool xxx (all_amps) with remaining rows.

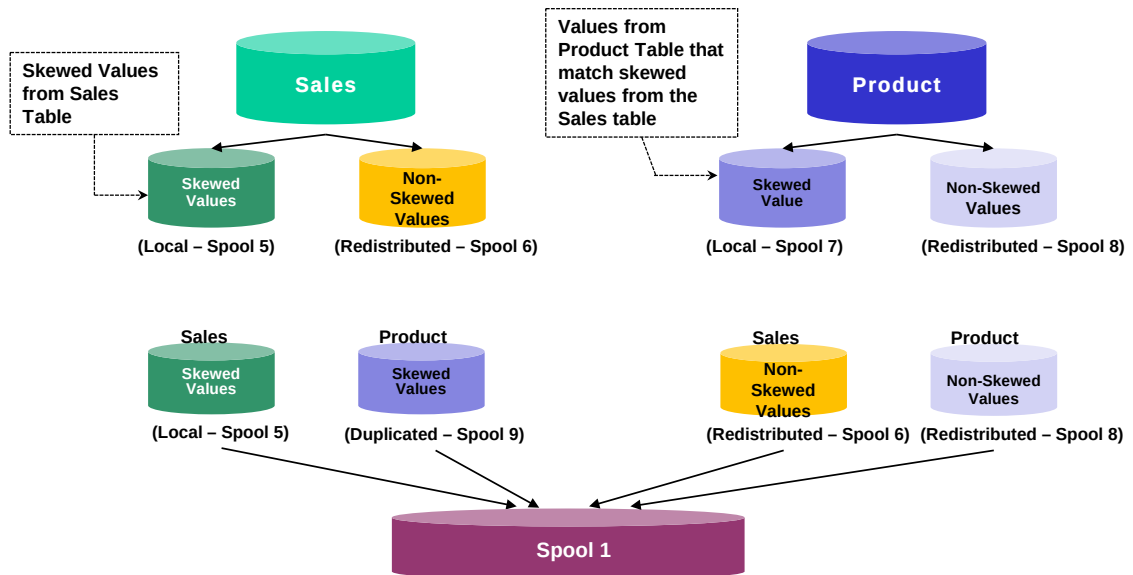
Teradata PRPD Join Example

The facing page contains an illustration of how the PDPD might be used by the optimizer.

Teradata PRPD Join Example

```
SELECT *
FROM Sales, Product
WHERE Sales.ItemNo = Product.ItemNo;
```

Fact: Sales.ItemNo is skewed.



PRPD One Relation Skewed on a Non-PI Column

The facing page contains a portion of the EXPLAIN plan which creates the “split” spool files.

Step 4.1 handles the join preparation for the skewed values in the Sales table while Step 4.2 handles the skewed rows from the Product table.

Spool is split

4) We execute the following steps in parallel.

- 1) We do an all-AMPs RETRIEVE step in TD_MAP1 from DB1.Sales by way of an all-rows scan with a condition of ("NOT(db1.Sales.ProductId IS NULL)") **split into Spool 5 (all_amps) with a condition of ("(DB1.Sales.ProductId IN (:*))") to qualify skewed rows and Spool 6 (all_amps) with remaining rows.** Spool 5 is built locally on the AMPs. Then we do a SORT to order Spool 5 by the hash code of (DB1.Sales.ProductId). The size of spool 5 is estimated with low confidence to be 26800 rows. Spool 6 is redistributed by the hash code of (DB1.Sales.ProductId) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 6 by row hash. The size of spool 6 is estimated with low confidence to be 38690 rows. The estimated time for this step is 0.07 seconds.
- 2) We do an all-AMPs RETRIEVE step in TD_MAP1 from DB1.Product by way of an all-rows scan with a condition of ("NOT(DB1.Product.ProductId IS NULL)") **split into Spool 7 with a condition of ("(DB1.Product.ProductId IN (:*))") to qualify rows matching skewed rows of the skewed relation and Spool 8 with remaining rows.** Spool 7 is built locally on the AMPs. The size of spool 7 is estimated with low confidence to be 320 rows. Spool 8 is redistributed by the hash code of (DB1.Product.ProductId) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 8 by row hash. The size of spool 8 is estimated with low confidence to be 26340 rows. The estimated time for this step is 0.06 seconds.

PRPD One Relation Skewed on a Non-PI Column (cont.)

The facing page contains a portion of the EXPLAIN plan which creates the separate join steps.

PRPD One Relation Skewed on a Non-PI Column (cont.)

Notice that there are now **two** join steps to accommodate the skewed values in Spool 5 and the non-skewed values in Spool 6 from the Sales table that had the skewed data.

- 5) We do an all-AMPs RETRIEVE step in TD_MAP1 from Spool 7 into Spool 9 (all_amps), which is duplicated on all the AMPs in TD_MAP1. Then we do a SORT to order Spool 9 by the hash code of (DB1.Product.ProductId). The size of spool 9 is estimated with low confidence to be 3200 rows.

- 6) We do an all-AMPs **JOIN step in TD_MAP1 from Spool 5** (Last Use) by way of a RowHash match scan, which is joined to Spool 9 (Last Use) by way of a RowHash match scan. Spool 5 and Spool 9 are joined using a **merge join**, with a join condition of ("ProductId = ProductId"). **The result goes into Spool 1** (all_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with no confidence to be 3 rows. The estimated time for this step is 0.06 seconds.

- 7) We do an all-AMPs **JOIN step in TD_MAP1 from Spool 6** (Last Use) by way of a RowHash match scan, which is joined to Spool 8 (Last Use) by way of a RowHash match scan. Spool 6 and Spool 8 are joined using a **merge join**, with a join condition of ("ProductId = ProductId"). **The result goes into Spool 1** (all_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with no confidence to be 3 rows. The estimated time for this step is 0.06 seconds.

Join ACCESS

Join ACCESS occurs whenever a column appears in an Equality Join constraint. The example at the top of the facing page features an Equality Join constraint (Table1.Column = Table2.Column). The Join ACCESS occurs on the ColName column.

Join Access demographics come from Application and Transaction Modeling, and are expressed in two ways:

- **Join Frequency**, which is a measure of how often annually, all known transactions access rows from the table through a Join on this column.
- **Join Rows**, which is a measure of how many rows are accessed annually through joins on this column across all transactions.

Index choices can greatly affect the Join Type, or other access method used:

- UPIs or USIs may make Nested Joins possible.
- NUSIs can only be used for Set Selection in most Joins or to access Table 2 in a Nested Join.
- USIs and NUSIs do not participate in Join Conditions (except in Nested Joins). Only PIs participate in non-Nested Join Conditions.

Join Access

- Join Access is how often a column appears in an equality constraint.
e.g., ON Table1.colname = Table2.colname

- Two metrics that represent join access are:

Join Frequency:

How often annually all known transaction access rows from the table through a join on this column.

Join Rows:

How many rows are accessed annually through joins on this column across all transactions.

- The demographics result from activity modeling.
- Index choices determine the join type or other access method.

Exercise 4 – Example

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Final Index Choice Guidelines:

- **Choose ONE Primary Index (UPI or NUPI) utilizing Join Access demographics.**
- **Do not carry identical Primary and Secondary Indexes.**

**** Too volatile for stats.**

Exercise 4 – Making Final Index Choices

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Exercise 4 – Making Final Index Choices

ENTITY 1							
500,000,000 Rows	A	B	C	D	E	F	
PK/FK	PK,UA						
Value Access	250K	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Join Access	10M	0	0	0	0	0	
Join Rows	10M	0	0	0	0	0	
Distinct Values	500M	475M	1.5M	1.25M	200M	5M	
Max Rows/Value	1	2	400	450	3	110	
Max Rows/NULL	0	0	0	0	10M	0	
Typical Rows/Value	1	1	325	350	2	90	
Change Rating	0	2	1	1	1	1	
PI/SI	UPI		NUPI	NUPI		NUPI	
	USI						
Collect Statistics (Y/N)							

Exercise 4 – Making Final Index Choices (cont.)

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Exercise 4 – Making Final Index Choices (cont.)

		ENTITY 2						
50,000,000 Rows	G	H	I	J	K	L		
PK/FK	PK,SA							
Value Access	5K	365	52	12	0	0		
Range Access	12	0	0	0	0	2.6K		
Join Access	100M	0	0	0	0	0		
Join Rows	100M	0	0	0	0	0		
Distinct Values	50M	500K	45M	12	50	900K		
Max Rows/Value	1	200	2	5M	1.2M	60		
Max Rows/NULL	0	0	450K	0	0	0		
Typical Rows/Value	1	100	1	4M	950K	50		
Change Rating	0	0	9	1	2	0		
PI/SI	UPI	NUPI					NUPI	
	USI	NUSI					VONUSI	
Collect Statistics (Y/N)								

Exercise 4 – Making Final Index Choices (cont.)

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Exercise 4 – Making Final Index Choices (cont.)

5,000,000 Rows	DEPENDENT						
	A	M	N	O	P	Q	
PK/FK	PK				NN,ND		
	FK	SA					
Value Access	0	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Join Access	700K	0	0	0	0	0	
Join Rows	1M	0	0	0	0	0	
Distinct Values	2M	50	90K	3M	5M	2M	
Max Rows/Value	4	200K	75	2	1	5	
Max Rows/NULL	0	0	0	390K	0	1M	
Typical Rows/Value	1	60K	50	1	1	1	
Change Rating	0	0	3	1	0	1	
PI/SI	UPI				UPI		
	NUPI						
	USI				USI		
Collect Statistics (Y/N)							

Exercise 4 – Making Final Index Choices (cont.)

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Exercise 4 – Making Final Index Choices (cont.)

ASSOCIATIVE 1					
1.5 Billion Rows	A	G	R	S	
PK/FK	PK				
	FK	FK,SA			
Value Access	26K	0	0	0	
Range Access	0	0	0	0	
Join Access	0	8M	0	0	
Join Rows	0	300M	0	0	
Distinct Values	490M	50M	75K	4M	
Max Rows/Value	5	40	21K	400	
Max Rows/NULL	0	0	0	0	
Typical Rows/Value	3	25	19K	350	
Change Rating	0	0	0	0	
PI/SI	UPI				
	NUPI	NUPI	NUPI?	NUPI	
	USI				
	NUSI				
Collect Statistics (Y/N)					

Exercise 4 – Making Final Index Choices (cont.)

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Exercise 4 – Making Final Index Choices (cont.)

ASSOCIATIVE 2						
500,000,000 Rows	A	M	G	T	U	
PK/FK	PK					
	FK		FK			
Value Access	0		0	0	0	
Range Access	0		0	0	0	
Join Access	7M		250K	0	0	
Join Rows	800M		100M	0	0	
Distinct Values	250M		50M	2.8M	750	
Max Rows/Value	3		150	180	675K	
Max Rows/NULL	0		0	0	0	
Typical Rows/Value	1		8	170	500K	
Change Rating	0		0	0	0	
PI/SI	UPI					
	NUPI		NUPI	NUPI		
	USI					
Collect Statistics (Y/N)						

What additional index choices would be available for the column G?

Exercise 4 – Making Final Index Choices (cont.)

In this exercise, you will make final index choices (based on Join Access demographics) for the same tables you used in Exercises 2 and 3. You will identify the columns that statistics should be collected.

The Final Index Choice Guidelines are:

- Choose one Primary Index (UPI or NUPI) utilizing Join Access demographics
- Do not carry identical Primary and Secondary Indexes

The example on the facing page provides you with an example of how to apply these guidelines.

Exercise 4 – Making Final Index Choices (cont.)

		HISTORY					
3.65 Billion Rows		A	DATE	D	E	F	
	PK/FK	PK					
		FK	SA				
Value Access		10M	26K	0	0	0	
Range Access		0	100K	0	0	0	
Join Access		800M	0	0	0	0	
Join Rows		2.4B	0	0	0	0	
Distinct Values		500M	730	N/A	N/A	N/A	
Max Rows/Value		18	5.5M	N/A	N/A	N/A	
Max Rows/NULL		0	0	N/A	N/A	N/A	
Typical Rows/Value		3	4.5M	N/A	N/A	N/A	
Change Rating		0	0	N/A	N/A	N/A	
	PI/SI	UPI					
		NUPI					
		USI					
		NUSI	VONUSI				
		Collect Statistics (Y/N)					

What additional index choices would be available for the DATE column?

Summary – Index Choice Guidelines

The facing page shows some guidelines for choosing Primary Indexes. The most important guidelines are:

- Primary Indexes should have Change Ratings of 0 – 1.
- Try to avoid NUPI indexes that have excessive number of duplicate values. A general rule is that the number of number of duplicate values should be less than the number of rows that can reside in 3 or fewer blocks.
- Collect statistics for all NUPIs and NUSIs.
- Collect statistics for UPIs of small tables (e.g., less than 1000 rows per AMP or 4 blocks per AMP).
- For large tables with a UPI, collect Summary Statistics. If you collect statistics on any column on a table, you will automatically get summary statistics at the table level.
- Consider column(s) as primary index candidates if ...

the number of max rows per value and the typical rows per value is relatively close to each other

AND

the number of distinct values is greater than the number of AMPs (by at least at factor of 10), then the column may be a candidate for a NUPI.

The facing page shows some guidelines for choosing Secondary Indexes. The most important guidelines are:

- Secondary Indexes should have Change Ratings of 0 – 3.
- Secondary Indexes should have Value Access > 0.
- Collect Statistics for all NUSIs.
- Collect Statistics for USIs that are used with range queries

Summary – Index Choice Guidelines

Primary Index	Unique	Non-Unique
Value Access	1 AMP – 1 Row	1 AMP – 1+ Rows
Join Access	Nested, Merge, Exclusion Joins	Nested, Merge, Exclusion Joins
Distinct Values	Equals row count	Less than row count Much greater than # of AMPs (10X or more)
Max Rows/Value	One	Close to typical value
Max Rows NULL	One	Less than typical value
Typical Rows/Value	One	Close to max value
Change Rating	0 – 1	0 – 1
Statistics	Collect statistics for small tables	Collect statistics
Secondary Index	Unique	Non-Unique
Value Access	2 AMP – 1 Row	All AMPs – 1+ Rows
Join Access	Nested Joins	Nested and Merge Joins
Distinct Values	Equals row count	At least 20
Max Rows/Value	One	Less than # of blocks
Max Rows NULL	One	Less than # of blocks
Typical Rows/Value	One	Less than 5 – 10% of the data rows
Change Rating	0 – 3	0 – 3
Statistics	Collect statistics for range queries	Collect statistics

Module 22: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 22: Review Questions

1. When do Cartesian Product Joins generally occur?
 - a. When an error is made in coding the SQL query.
 - b. When a column appears in an equality join constraint.
 - c. When the join column is the primary index of both tables.
 - d. When the join column is the primary index in only one of the tables.

2. Which two types of joins will result in a Join Condition of (1=1) in EXPLAIN output? ____ ____
 - a. Hash
 - b. Merge
 - c. Nested
 - d. Exclusion merge
 - e. Cartesian product

3. Which demographic represents how often all known transactions access rows from the table through a join on this column?
 - a. Value Access Frequency
 - b. Range Access Rows
 - c. Range Access Frequency
 - d. Join Access Rows
 - e. Join Access Frequency

Notes



Join Indexes

After completing this module, you will be able to:

- **List the three types of Join Indexes.**
- **Describe the situations where a Join Index can improve performance.**
- **Describe how to create a “sparse join index”.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Additional Index Choices.....	23-4
Join Indexes.....	23-6
Options for Join Indexes	23-8
Join Index Considerations.....	23-10
Join Index Example – Customer and Order Tables	23-12
Non-Compressed Multi-Table Join Index.....	23-14
Compressed Multi-Table Join Index.....	23-16
Compressed and Non-Compressed Join Indexes	23-18
Example 1 – Does a Join Index Help?	23-20
Example 2 – Does a Join Index Help?	23-22
Example 3 – Partitioning a Join Index	23-24
Join Index – Single Table.....	23-26
Join Index – Single Table (cont.)	23-28
Creating a Join Index – Single Table	23-30
Example 4 – Does the Join Index Help?	23-32
Why use Aggregate Join Indexes?	23-34
Aggregate Join Index Properties	23-36
Aggregation without an Aggregate Index	23-38
Creating an Aggregate Join Index.....	23-40
Example 5: Aggregation with an AJI.....	23-42
Sparse Join Indexes.....	23-44
Creating a Sparse Join Index.....	23-46
Creating a Sparse Join Index on a Partitioned Table	23-48
ALTERing a Join Index to CURRENT	23-50
Partitioning a Sparse Join Index.....	23-52
Global (Join) Indexes	23-54
Global Index – Multiple Tables	23-56
Global Index as a “Hashed NUSI”	23-58
Creating a Global Index (“Hashed NUSI”).....	23-60
Example 6: Using a Global Index as a Hashed NUSI.....	23-62
Repeating Row Ids in Global Index	23-64
Summary	23-66
Module 23: Review Questions.....	23-68
Lab Exercise 23-1	23-70
Lab Exercise 23-2 (optional).....	23-76

Additional Index Choices

As part of implementing a physical design, Teradata provides additional index choices that can improve performance for known queries and workloads. These will be described in more detail in this module.

Join indexes provide the performance benefits of prejoin tables without incurring update anomalies and without denormalizing your logical or physical database schemas. Although join indexes create and manage prejoins and, optionally, aggregates, they do not denormalize the physical implementation of your normalized logical model because they are not a component of the fully normalized physical model.

Join indexes are defined to reduce the number of rows processed in generating result sets from certain types of queries, especially joins. Like secondary indexes, users may not directly access join indexes. They are an option available to the optimizer in query planning. The following are properties of join indexes:

- Are used to replicate and “pre-join” information from several tables into a single structure.
- Are designed to cover queries, reducing or eliminating the need for access to the base table rows.
- Usually do not contain pointers to base table rows (unless user defined to do so).
- Are distributed based on the user choice of a Primary Index on the Join Index.
- Permit Secondary Indexes to be defined on the Join Index (except for Single Table Join Indexes), with either “hash” or “value” ordering.

Unlike traditional indexes, join indexes do not store “pointers” to their associated base table rows. Instead, they are a fast path *final* access point that eliminates the need to access and join the base tables they represent. They substitute *for* rather than point *to* base table rows.

The Optimizer can choose to resolve a query using the index, rather than performing a join of two or more tables. Depending on the complexity of the join, this improves query performance considerably. To improve the performance of Join Index creation and Join Index maintenance during updates, consider collecting statistics on the base tables of a Join Index.

Additional Index Choices

As part of the physical design process, the designer may choose to implement join and/or hash indexes for performance reasons.

- **Join Indexes**

- Can be created to pre-join multiple tables.
- Can be used on a single table to redistribute the table on a different column – effectively creating an alternative primary index on a foreign key column.
- Can be used as a summary table to aggregate one or more columns from a table or tables.
- Provides the optimizer with additional options and the optimizer may use the join index if it “covers” the query.
- For known queries, this typically will result in much better performance.

- **Hash Indexes**

- Contains properties of both secondary indexes and single table join indexes.
- Can only be placed on a single table and automatically includes the Primary Index value.
- The syntax is similar to secondary index syntax, thus simpler SQL to code.
- The Hash Index can be ordered by value or hash.
- This course will not cover hash indexes in detail.

Join Indexes

There are multiple ways in which a join index can be used. Three common ways include:

- Single table Join Index — Distribute the rows of a single table on the hash value of a foreign key value
- Multi-Table Join Index — Pre-join multiple tables; stores and maintains the result from joining two or more tables.
- Aggregate Join Index — Aggregate one or more columns of a single table or multiple tables into a summary table

A join index is a system-maintained index table that stores and maintains the joined rows of two or more tables (**multiple table join index**) and, optionally, aggregates selected columns, referred to as an **aggregate join index**.

Join indexes are defined in a way that allows join queries to be resolved without accessing or joining their underlying base tables. A join index is useful for queries where the index structure contains all the columns referenced by one or more joins, thereby allowing the index to cover all or part of the query. For obvious reasons, such an index is often referred to as a covering index. Join indexes are also useful for queries that aggregate columns from tables with large cardinalities. These indexes play the role of pre-join and summary tables without denormalizing the logical design of the database and without incurring the update anomalies presented by denormalized tables.

Another form of join index, referred to as a **single table join index**, is very useful for resolving joins on large tables without having to redistribute the joined rows across the AMPs.

- A Join Index is an optional index which may be created by the user. The basic types of Join Indexes will be described first.
- **Multi-table Join Index**
 - Pre-join multiple tables; stores and maintains the result from joining two or more tables.
 - Facilitates join operations by possibly **eliminating join processing** or by **reducing/eliminating join data redistribution**.
- **Single-table Join Index**
 - Distribute the rows of a single table on the hash value of a foreign key value.
 - Facilitates the ability to join the foreign key table with the primary key table **without redistributing the data**.
- **Aggregate Join Index (AJI)**
 - Aggregate (SUM or COUNT) one or more columns of a single table or multiple tables into a summary table. TD 14.10 support MIN/MAX aggregate operators.
 - Facilitates aggregation queries by **eliminating aggregation processing**. The pre-aggregated values are contained in the AJI instead of relying on base table calculations.

Options for Join Indexes

The facing page highlights two options that are available with join indexes.

A Sparse Join Index is simply a term that is used when a join index is created with a WHERE condition. You can use a WHERE clause in the CREATE JOIN INDEX statement to limit the rows that are created in the join index. This effectively reduces the size (PERM space) of the join index. The rows included in the join index are a subset of the rows in the base table or tables based on an SQL query result.

A Global Index is simply a term that is used to define a join index that contains the Row IDs of the base table rows. This means that the user includes the ROWID as a user-defined column within the join index. Row IDs that are included within a join index always include the partition number (0 for non-partitioned tables), regardless if the base table is partitioned or not.

Options for Join Indexes

- **Sparse Join Indexes**

- When creating any of the join indexes, you can include a **WHERE** clause to limit the rows created in the join index to a subset of the rows in the base table or tables.

- **Global Join Indexes**

- You can include the Row ID of the table(s) within the join index to allow an AMP to join back to the data row for columns not referenced (covered) in the join index.

- **Miscellaneous notes:**

- [Materialized Views](#) are implemented as [Join Indexes](#). Join Indexes improve query performance at the expense of slower updates and increased storage.
- When you create a Join Index, there is no duplicate row check – you don't have to worry about a high number of NUPI duplicates in the join index.

Join Index Considerations

Join Index considerations include:

You cannot specify a column with a data type of either BLOB or CLOB in the definition of a join index (nor for any other kind of index).

Be aware that when you define a join index using an outer join, you must reference all the columns of the outer table in the select list of the join index definition. If any of the outer table columns are not referenced in the select list for the join index definition, the system returns an error to the requestor.

Perm Space — a Join Index is created as a table-like structure in Teradata and uses Perm space.

Fallback Protection — Join Index subtables can optionally be Fallback-protected.

Because join indexes generated from inner joins do not preserve unmatched rows, you should always consider using outer joins to define simple join indexes, noting the following restrictions.

Inequality conditions are not supported under any circumstances for ON clauses in join index definitions.

Outer joins are not supported under any circumstances for aggregate join indexes.

Load Utilities — MultiLoad and FastLoad utilities cannot be used to load or unload data into base tables that have an associated join index defined on them because join indexes are not maintained during the execution of these utilities. The TPump utility, which perform standard SQL row inserts and updates, can be used because join indexes are properly maintained during the execution of such utilities.

Archive and Restore — archiving is permitted on a base table or database that has a Join Index. Prior to Teradata 13.0, during a restore of such a base table or database, the system does not automatically rebuild the Join Index. Instead, the Join Index is marked as invalid.

Permanent Journal Recovery — using a permanent journal to recover a base table (i.e., ROLLBACK or ROLLFORWARD) with an associated Join Index defined is permitted. The join index is not automatically rebuilt during the recovery process. Instead, the join index is marked as invalid and the join index must be dropped and recreated before it can be used again in the execution of queries.

Collecting Statistics — statistics should be collected on the primary index and secondary indexes of the Join Index to provide the Optimizer with baseline statistics, including the total number of rows in the Join Index.

Join Index Considerations

Join Index considerations include:

- Join indexes are updated automatically as base tables are updated (additional I/O).
- Take up PERM space and may (or may not) be Fallback protected.
- You can specify no more than 64 columns per referenced base table per join index.
- BLOB and CLOB data types **cannot** be defined within a Join Index.
- TPT Load (FastLoad) and TPT Update (MultiLoad) **can't** be used if a table has an associated join index; use TPT Stream (TPump).
- With a multi-table join index, you can specify INNER, LEFT OUTER, and RIGHT OUTER joins, but not a FULL OUTER join.

In many respects, a Join Index is similar to a base table.

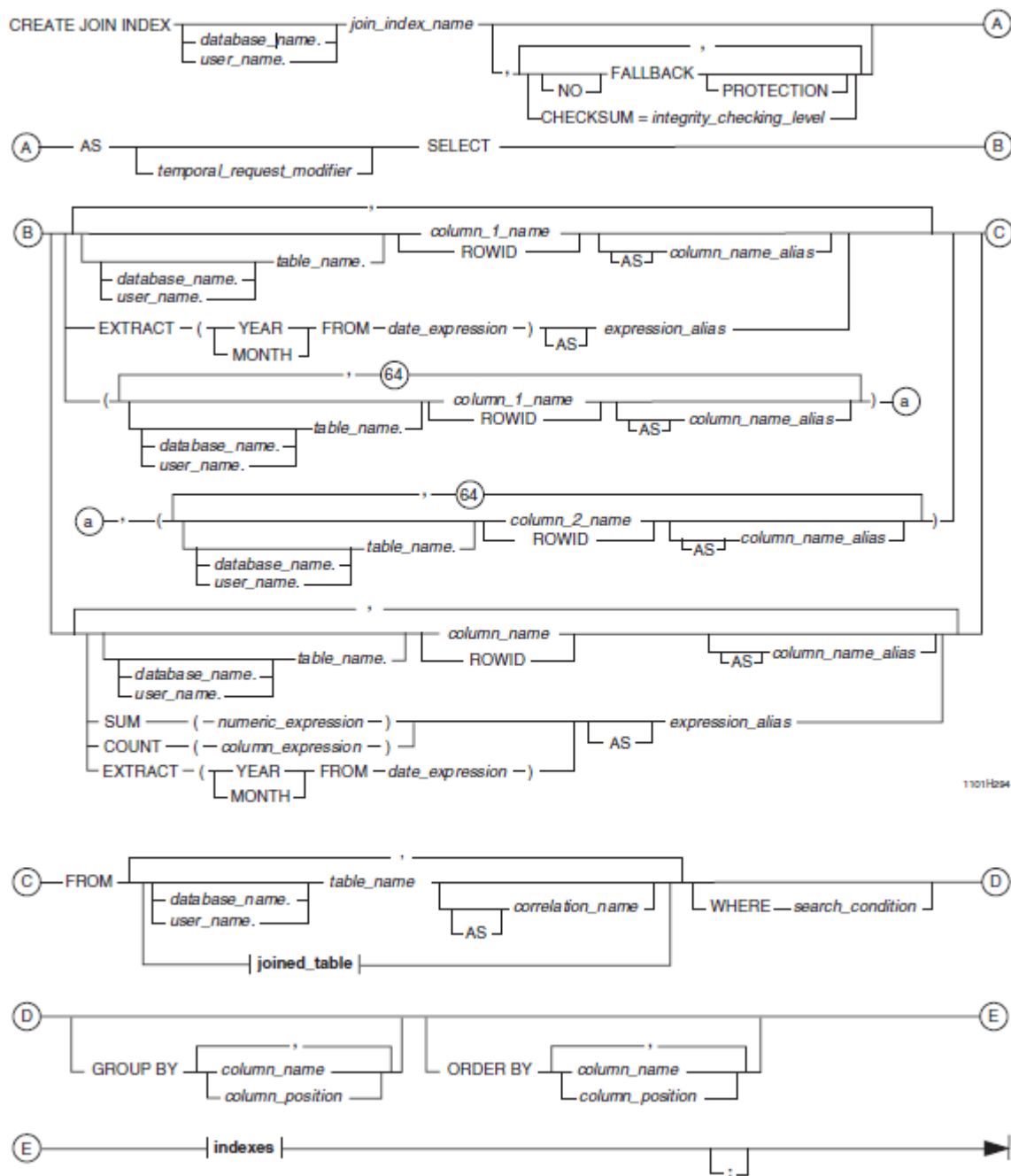
- You may create non-unique secondary indexes on its columns.
- Perform COLLECT/DROP STATISTICS, DROP, HELP, and SHOW statements.

Unlike base tables, you cannot do the following:

- Directly query or update join index rows.
- Create unique secondary indexes on its columns.
- Store and maintain arbitrary query results such as expressions.

Join Index Example – Customer and Order Tables

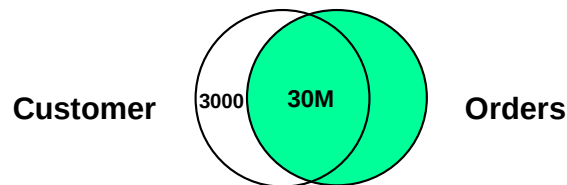
The CREATE JOIN INDEX syntax is shown below. The reference manual provides detailed for all of the CREATE JOIN INDEX options.



Join Index Example Customer and Order Tables

```
CREATE SET TABLE Customer
(custid      INTEGER NOT NULL,
 lastname    VARCHAR(15),
 firstname   VARCHAR(10),
 address     VARCHAR(100),
 city        VARCHAR(20),
 statecode   CHAR(2),
 zipcode     CHAR(5))
UNIQUE PRIMARY INDEX (custid);
```

```
CREATE SET TABLE Orders
(orderid     INTEGER NOT NULL,
 custid      INTEGER NOT NULL,
 orderstatus CHAR(1),
 ordertotal  DECIMAL(9,2) NOT NULL,
 orderdate   DATE NOT NULL,
 orderpriority SMALLINT,
 clerkid     CHAR(16),
 shippriority SMALLINT,
 orderlocation SMALLINT,
 ordercomment VARCHAR(79))
UNIQUE PRIMARY INDEX (orderid);
```



10,003,000 Customers and 30,000,000 Orders.
30,000,000 orders are associated with valid customers.
3000 customers have no orders.

Non-Compressed Multi-Table Join Index

The facing page includes an example of creating a Multiple Table Join Index without using the repeating group option.

The storage space in this example for the non-compressed multi-table join index will be higher.

For a non-compressed multi-table join index, the maximum number of columns defined per referenced table is 64. The total maximum number of columns in this type of join index is 2048.

Non-Compressed Multi-Table Join Index

```
CREATE JOIN INDEX Cust_Ord_JI1 AS
  SELECT      C.custid as c_custid, lastname, firstname,
             orderid, orderstatus, ordertotal, orderdate
  FROM        Customer C
  INNER JOIN   Orders O
  ON          C.custid = O.custid
  PRIMARY INDEX (c_custid);
```

Max # columns per
referenced table is 64

Maximum # of columns for
non-compressed join index is 2048.

Fixed Portion

Variable Portion

c_custid	lastname	firstname	orderid	orderstatus	ordertotal	orderdate
111443	Federer	Roger	142292	C	500.25	01/10/2019
111443	Federer	Roger	145893	C	1393.95	01/12/2019
111443	Federer	Roger	157093	O	1025.45	01/31/2019
111443	Federer	Roger	135993	C	1025.37	12/14/2018
114000	Djokovic	Novak	142000	C	745.76	12/20/2018
114000	Djokovic	Novak	149600	C	945.40	01/05/2019
114000	Djokovic	Novak	154798	C	1540.50	01/07/2019
115809	Wozniacki	Caroline	149698	C	2081.43	12/31/2018
115809	Wozniacki	Caroline	156599	C	680.91	01/05/2019
115809	Wozniacki	Caroline	101199	C	845.33	12/24/2018
115809	Wozniacki	Caroline	158999	O	1002.50	01/23/2019
115809	Wozniacki	Caroline	152399	C	945.20	01/07/2019

Within the join index, each
order is represented by a
separate join index row.

Options that are available with a non-compressed JI.

- **Partitioning**
- MVC on base tables is carried into a non-compressed JI.
- Unique Primary Index (13.10 feature)

Compressed Multi-Table Join Index

The facing page includes an example of creating a Multiple Table Join Index using the repeating group option.

The storage organization for join indexes supports a compressed format to reduce storage space.

If you know that a join index contains groups of rows with repeating information, then its definition DDL can specify repeating groups, indicating the repeating columns in parentheses. The column list is specified as two groups of columns, with each group stipulated within parentheses. The first group contains the fixed columns and the second group contains the repeating columns.

As another option, you can elect to store join indexes in value order, ordered by the values of a 4-byte column. Value-ordered storage provides better performance for queries that specify selection constraints on the value ordering column. For example, suppose a common task is to look up sales information by order date. You can create a join index on the Orders table and order it by order date. The benefit is that queries that request orders by order date only need to access those data blocks that contain the value or range of values that the queries specify.

Physical Join Index Row and Compression

A physical join index row has two parts:

- A required fixed part that is stored only once.

- An optional repeated part that is stored as often as needed.

For example, if a logical join result has n rows with the same fixed part value, then there is one corresponding physical join index row that includes n repeated parts in the physical join index. A physical join index row represents one or more logical join index rows. The number of logical join index rows represented in the physical row depends on how many repeated values are stored.

Limitations

For a compressed multi-table join index, the maximum number of columns defined in the fixed portion is 64 and the maximum number of columns defined in the repeating portion is also 64. The total maximum number of columns in this type of join index is 128.

With a LEFT or RIGHT OUTER join, at least 1 column from each inner table must be NOT NULL.

FULL OUTER joins are not allowed with a compressed multi-table join index.

Compressed Multi-Table Join Index

```
CREATE JOIN INDEX Cust_Ord_JI2 AS
  SELECT (C.custid AS c_custid, lastname, firstname),
         (orderid, orderstatus, ordertotal, orderdate)
  FROM   Customer C
  INNER JOIN Orders O
  ON     C.custid = O.custid
  PRIMARY INDEX (c_custid);
```

Fixed portion of Join Index
(max # columns is 64)

Repeating portion of Join Index
(max # of columns is 64)

Maximum # of columns for
compressed join index is 128.

Fixed Portion

Variable Portion

c_custid	lastname	firstname	orderid	orderstatus	ordertotal	orderdate
111443	Federer	Roger	142292	C	500.25	01/10/2019
			145893	C	1393.95	01/12/2019
			157093	O	1025.45	01/31/2019
			135993	C	1025.37	12/14/2018
114000	Djokovic	Novak	142000	C	745.76	12/20/2018
			149600	C	945.40	01/05/2019
			154798	C	1540.50	01/07/2019
115809	Wozniacki	Caroline	149698	C	2081.43	12/31/2018
			156599	C	680.91	01/05/2019
			101199	C	845.33	12/24/2018
			158999	O	1002.50	01/23/2019
			152399	C	945.20	01/07/2019

Within the join index,
this is one row of data.

One row in Join Index.

One row in Join Index.

Compressed and Non-Compressed Join Indexes

The facing page illustrates the difference between a compressed and a non-compressed join index.

PERM Space Required

The amount of PERM space used by the compressed multi-table join index (previous example) and the non-compressed join index can vary.

In this example, the sizes (in bytes) of the two join indexes are:

<u>Join Index Name</u>	<u>Permanent Space</u>	<u>Type of Join Index</u>
Cust_Ord_JI1	20,848,640	non-compressed multi-table
Cust_Ord_JI2	16,949,248	compressed multi-table
Cust_Ord_JI3	26,558,464	non-compressed partitioned multi-table

Compressed and Non-Compressed Join Indexes

- **Recommendation:** Normally use non-compressed join indexes except when creating a Global Join Index used as a "hashed NUSI" (covered later).
- A non-compressed join index provides more flexibility than a compressed join index and may not use more storage space because MVC on a base table carries into a non-compressed join index.
- Logical example of compressed and non-compressed join indexes.

```
CREATE JOIN INDEX Cust_Ord_JI1 AS
SELECT      (custid, lastname, firstname),
            (orderid, orderstatus, ... )
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
PRIMARY INDEX (custid);
```

```
CREATE JOIN INDEX Cust_Ord_JI2 AS
SELECT      custid, lastname, firstname
            orderid, orderstatus, ...
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
PRIMARY INDEX (custid);
```

Compressed Join Index

Join Index Row ID	Fixed Columns	Repeating Columns (Orders)		
RH	5001 ...	Order1	Order2	Order3
RH	5002 ...	Order4	Order5	

Non-Compressed Join Index

Join Index Row ID	Join Index Column Values		
RH	5001 ...	Order1	
RH	5001 ...	Order2	
RH	5001 ...	Order3	
RH	5002 ...	Order4	
RH	5002 ...	Order5	

Example 1 – Does a Join Index Help?

Note: Summary statistics were collected on the Customer table and on the custid, orderstatus, and orderdate columns of the Orders table.

This EXPLAIN is **without** a Join Index.

- 1) First, we lock TFACT.O in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) Next, we lock TFACT.C in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 3) We lock TFACT.O in TD_MAP1 for read, and we lock TFACT.C in TD_MAP1 for read.
- 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.O by way of an all-rows scan with a condition of ("TFACT.O.orderstatus = 'O'") into Spool 2 in TD_MAP1 (all_amps), which is redistributed by the hash code of (TFACT.O.custid) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 2 by row hash. The size of Spool 2 is estimated with high confidence to be 117,995 rows (2,477,895 bytes). The estimated time for this step is 11.17 seconds.
- 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of a RowHash match scan. TFACT.C and Spool 2 are joined using a merge join, with a join condition of ("TFACT.C.custid = custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.C.custid). The size of Spool 1 is estimated with low confidence to be 117,995 rows (5,545,765 bytes). The estimated time for this step is 3.46 seconds.
- 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
 - > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 14.63 seconds.

This EXPLAIN is **with** a non-compressed Join Index.

- 1) First, we lock TFACT.CUST_ORD_JI1 in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) Next, we lock TFACT.CUST_ORD_JI1 in TD_MAP1 for read.
- 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.CUST_ORD_JI1 by way of an all-rows scan with a condition of ("TFACT.CUST_ORD_JI1.orderstatus = 'O'") into Spool 1 in TD_MAP1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.CUST_ORD_JI1.c_custid). The size of Spool 1 is estimated with no confidence to be 50,205 rows (2,359,635 bytes). The estimated time for this step is 0.15 seconds.
- 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
 - > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.15 seconds.

Example 1 – Does a Join Index Help?

List the valid customers who have open orders?

```
SELECT      C.custid, C.lastname, C.firstname,
            O.orderdate
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
WHERE       orderstatus = 'O'
ORDER BY    1;
```

SQL Query

Without Join Index
With Join Index

Time

14.63 seconds
0.15 seconds

<u>custid</u>	<u>lastname</u>	<u>firstname</u>	<u>orderdate</u>
1460114	Cobb	Finn	02/28/2019
1514800	Mosley	Delilah	01/28/2019
1516554	Harper	Meredith	01/16/2019
2571123	Moon	Bently	12/28/2018
4423625	Simon	Dixie	12/19/2018
5772681	Paul	Damarion	12/31/2018
5915196	Brewer	Lucca	01/16/2019
9461517	Barnett	Tate	03/16/2019
9602794	Thompson	Jared	02/17/2019

All referenced columns are part of the Join Index.

Optimizer picks Join Index rather than doing a join.

Join Index covers query and helps this query.

The non-compressed Join Index was used for this example.

Note: Statistics were collected on the custid and orderdate columns of the Orders table and on the city, lastname, and zipcode columns of the Customer table.

Example 2 – Does a Join Index Help?

This EXPLAIN is **without** a Join Index.

- 1) First, we lock TFACT.O in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, we lock TFACT.C in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 3) We lock TFACT.O in TD_MAP1 for read, and we lock TFACT.C in TD_MAP1 for read.
 - 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.O by way of an all-rows scan with a condition of ("TFACT.O.orderstatus = 'O'") into Spool 2 (all_amps), which is redistributed by the hash code of (TFACT.O.custid) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 2 by row hash. The size of Spool 2 is estimated with high confidence to be 117,995 rows (2,477,895 bytes). The estimated time for this step is 11.17 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of a RowHash match scan. TFACT.C and Spool 2 are joined using a merge join, with a join condition of ("TFACT.C.custid = custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.C.custid). The size of Spool 1 is estimated with low confidence to be 117,995 rows (10,855,540 bytes). The estimated time for this step is 3.64 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 14.80 seconds.

This EXPLAIN is **with** a non-compressed Join Index.

- 1) First, we lock TFACT.CUST_ORD_JI1 in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, we lock TFACT.C in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 3) We lock TFACT.CUST_ORD_JI1 in TD_MAP1 for read, and we lock TFACT.C in TD_MAP1 for read.
 - 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.CUST_ORD_JI1 by way of an all-rows scan with a condition of ("TFACT.CUST_ORD_JI1.orderstatus = 'O'") into Spool 2 (all_amps), which is built locally on the AMPs. The size of Spool 2 is estimated with no confidence to be 49,407 rows (1,630,431 bytes). The estimated time for this step is 0.13 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of a RowHash match scan. TFACT.C and Spool 2 are joined using a merge join, with a join condition of ("c_custid = TFACT.C.custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.CUST_ORD_JI1.custid). The size of Spool 1 is estimated with no confidence to be 50,205 rows (4,618,860 bytes). The estimated time for this step is 3.32 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 3.45 seconds.

Example 2 – Does a Join Index Help?

List the valid customers and their addresses who have open orders?

```
SELECT      C.custid, C.lastname, C.firstname,
            C.address, C.city, C.statecode,
            O.orderdate
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
WHERE       O.orderstatus = 'O'
ORDER BY    1;
```

SQL Query

Without Join Index

With Join Index

Time

14.80 seconds

3.45 seconds

- Some of the referenced columns are **NOT** part of the Join Index. The Join Index does not cover the query, but is used in this example.
- A Join Index is used in this query and is merge joined with the Customer table.

<u>custid</u>	<u>lastname</u>	<u>firstname</u>	<u>address</u>	<u>city</u>	<u>statecode</u>	<u>orderdate</u>
1460114	Cobb	Finn	2300 Madrona Ave	Carson City	NV	02/28/2019
1514800	Mosley	Delilah	903 La Pierre Ave	Jackson	MS	01/28/2019
1516554	Harper	Meredith	36 Main Street	New York	NY	01/16/2019
2571123	Moon	Bentley	4564 Long Beach Bl	Trenton	NJ	12/28/2018
4423625	Simon	Dixie	1083 Beryl Ave	Los Angeles	CA	12/19/2018
5772681	Paul	Damarion	4021 Eleana Way	St. Paul	MN	12/31/2018
5915196	Brewer	Lucca	5603 Main Street	Pierre	SD	01/16/2019
9461517	Barnett	Tate	917 Carnation Dr	Beavercreek	OH	12/16/2018
9602794	Thompson	Jared	203 Stratford Lane	Wapakoneta	OH	02/17/2019

Example 3 – Partitioning a Join Index

The facing page includes an example of partitioning a non-compressed join index.

This EXPLAIN is **without** a non-compressed partitioned Join Index.

- 1) First, we lock TFACT.O in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, we lock TFACT.C in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 3) We lock TFACT.O in TD_MAP1 for read, and we lock TFACT.C in TD_MAP1 for read.
 - 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.O by way of an all-rows scan with a condition of ("TFACT.O.orderdate = DATE '2019-01-31' AND (TFACT.O.orderstatus = 'O')") into Spool 2 (all_amps), which is redistributed by the hash code of (TFACT.O.custid) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 2 by row hash. The size of Spool 2 is estimated with no confidence to be 43,180 rows (906,780 bytes). The estimated time for this step is 10.57 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of a RowHash match scan. TFACT.C and Spool 2 are joined using a merge join, with a join condition of ("TFACT.C.custid = custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.C.custid). The size of Spool 1 is estimated with no confidence to be 43,180 rows (2,029,460 bytes). The estimated time for this step is 3.22 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 13.79 seconds.

This EXPLAIN is **with** a non-compressed partitioned Join Index.

- 1) First, we lock TFACT.CUST_ORD_JI3 in TD_MAP1 for read on a reserved RowHash in a single partition to prevent global deadlock.
 - 2) Next, we lock TFACT.CUST_ORD_JI3 in TD_MAP1 for read on a single partition.
 - 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from a single partition of TFACT.CUST_ORD_JI3 with a condition of ("TFACT.CUST_ORD_JI3.orderdate = DATE '2019-01-31'") with a residual condition of ("TFACT.CUST_ORD_JI3.orderdate = DATE '2019-01-31' AND (TFACT.CUST_ORD_JI3.orderstatus = 'O')") into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.CUST_ORD_JI3.c_custid). The size of Spool 1 is estimated with no confidence to be 677 rows (31,819 bytes). The estimated time for this step is 0.01 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.01 seconds.

Example 3 – Partitioning a Join Index

```
CREATE JOIN INDEX Cust_Ord_JI3 AS
  SELECT      C.custid AS c_custid, C.lastname, C.firstname,
              O.orderid, O.orderstatus, O.Ordertotal, O.orderdate
  FROM        Customer C
  INNER JOIN   Orders O
  ON          C.custid = O.custid
  PRIMARY INDEX (c_custid)
  PARTITION BY RANGE_N (orderdate BETWEEN
    DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' DAY);
```

```
COLLECT STATISTICS COLUMN c_custid, COLUMN orderdate, COLUMN PARTITION
ON Cust_Ord_JI3;
```

List the valid customers who have open Orders for January 31, 2019?

SELECT	C.custid, C.lastname, C.firstname, O.orderdate		
FROM	Customer C		
INNER JOIN	Orders O		
ON	C.custid = O.custid	SQL Query	Time
WHERE	O.orderstatus = 'O'	Without Join Index	13.79 seconds
AND	O.orderdate = DATE '2019-01-31'	With Join Index	0.01 seconds
ORDER BY	1;		

The join index is used in this query and partition elimination will occur on the join index.

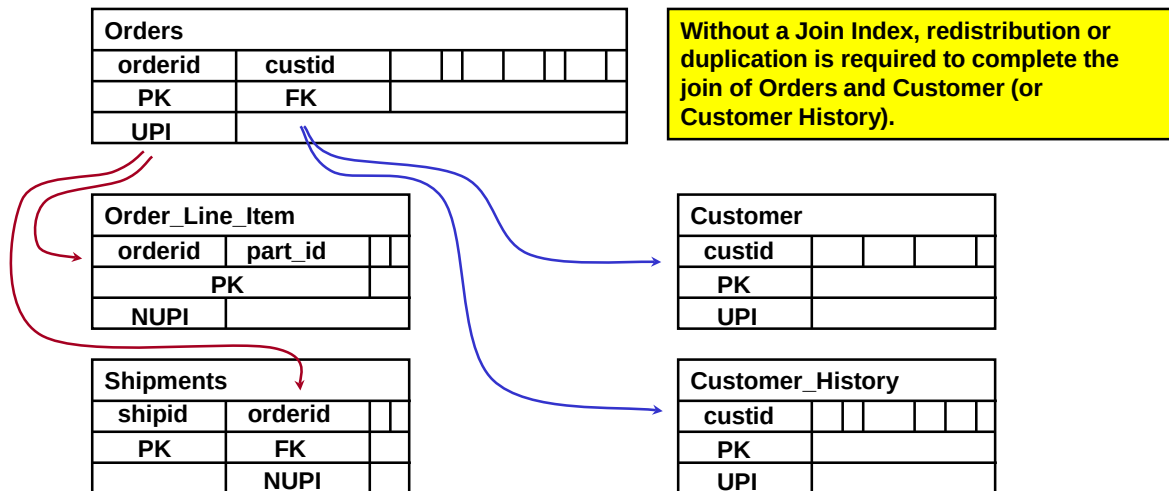
Join Index – Single Table

A denormalization technique is to replicate a column in a table to avoid joins. If an SQL query would benefit from replicating some or all of its columns in another table that is hashed on the join field (usually the primary index of the table to which it is to be joined) rather than the primary index of the original base table, then you should consider creating one or more single table join indexes on that table.

For example, you might want to create a single table join index to avoid redistributing a large base table or to avoid the possibly prohibitive storage requirements of a multi-table join index. For example, a single table join index might be useful for commonly made joins having low predicate selectivity but high join selectivity.

Join Index – Single Table

- The **Single Table Join Index** is useful for resolving joins on large tables without having to redistribute the joined rows across the AMPs.
- In some cases, this may perform better than building a multi-table join index on the same columns.



Join Index – Single Table (cont.)

You can also define a simple join index on a single table. This permits you to hash some or all of the columns of a large replicated base table on a foreign key that hashes rows to the same AMP as another large table. In some situations, this may perform better than building a multi-table join index on the same columns. The advantage comes from less under-the-covers update maintenance on the single table form of the index. Only testing can determine what the best design choice is for a given set of tables, applications, and hardware configuration.

The example on the facing page shows a technique where the join index is effectively substituted for the underlying base table. The join index has a primary index that ensures that rows are hashed to the same AMPs as rows in tables being joined. This eliminates the need for row redistribution when the join is made.

Even though each single table join index you create partly or entirely replicates its base table, you cannot query or update them directly just as you cannot directly query or update any other join index.

An example of creating a compressed single table join index is:

```
CREATE JOIN INDEX Orders_JI4compressed AS  
  SELECT      (custid),  
              (orderid, orderstatus, ordertotal, orderdate)  
  FROM        Orders  
  PRIMARY INDEX (custid);
```

PERM Space Required

The amount of PERM space used by the compressed multi-table join index (previous example) and single table join indexes is listed below. Remember that these tables are small and note that a compressed join index (with repeating data) usually requires less storage space.

However, if the base tables have MVC, then MVC will carry into the non-compressed join indexes and would be smaller and maybe even smaller than the compressed versions.

Join Index – Single Table (cont.)

- Possible advantages include:
 - **Less update maintenance** on the single table Join Index than a multi-table Join Index.
 - Maybe **less storage space** for a single table Join Index than for a multi-table Join Index.

Orders									
orderid	custid								
PK	FK								
UPI									

Orders_JI									
orderid	custid								
	NUPI								

The JI may have up to 64 columns.

Order_Line_Item				
orderid	part_id			
PK				
NUPI				

Shipments				
shipid	orderid			
PK	FK			
	NUPI			

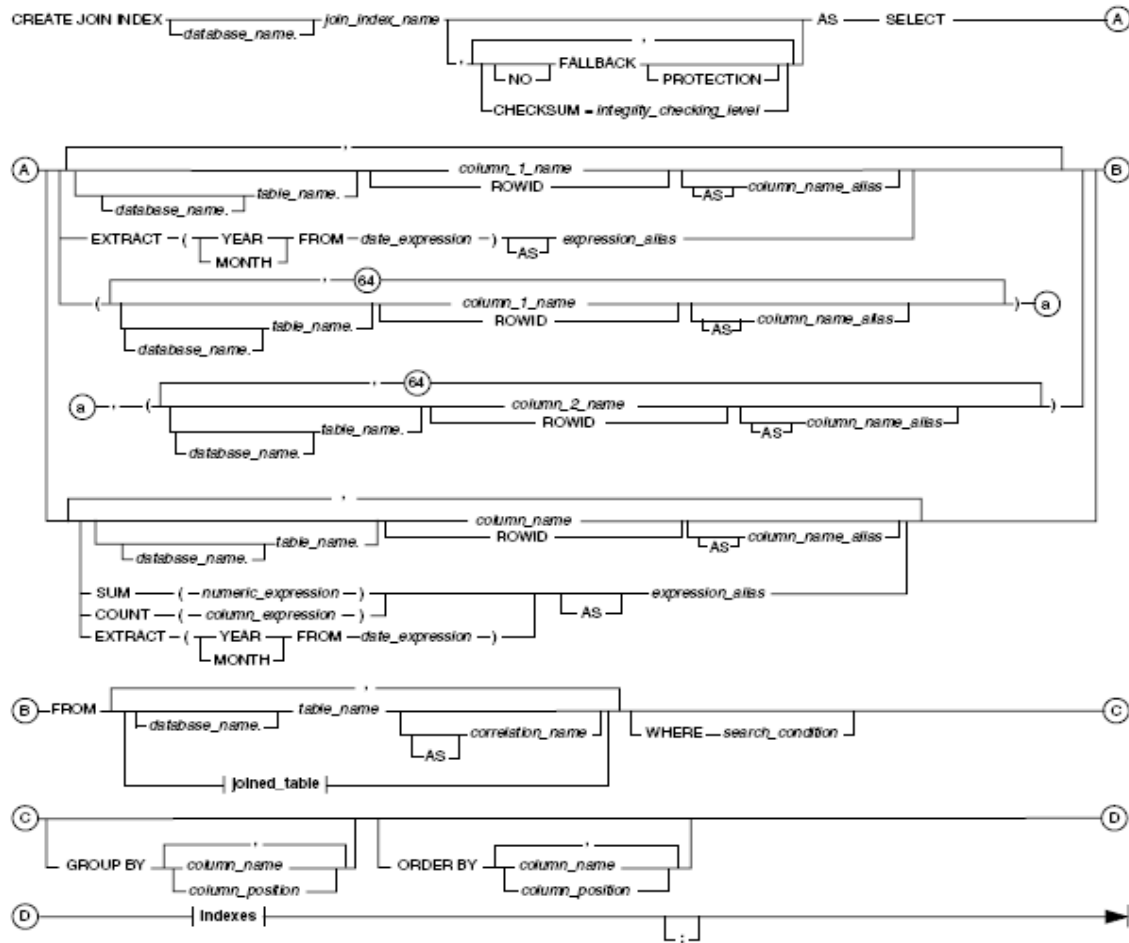
Customer				
custid				
PK				
UPI				

Customer_History				
custid				
PK				
UPI				

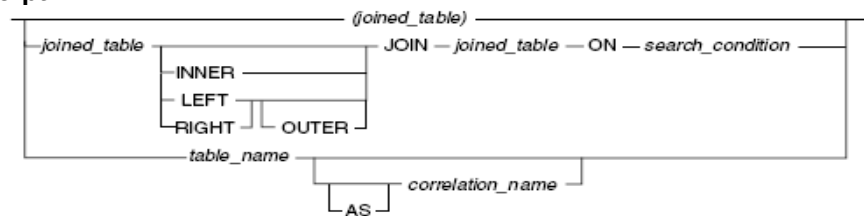
With this Join Index, this is effectively a join on matching primary indexes.

Creating a Join Index – Single Table

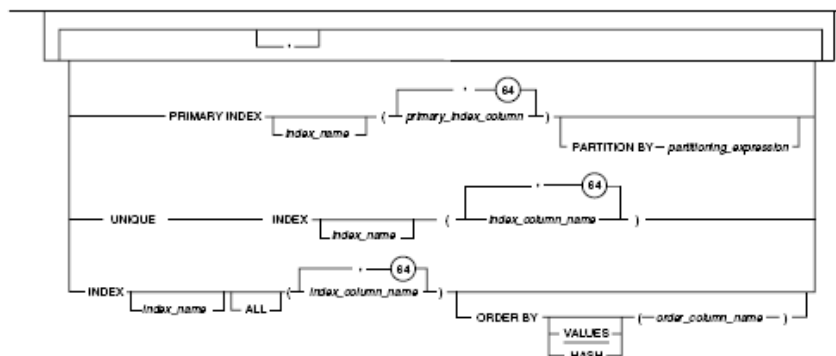
The CREATE JOIN INDEX syntax is shown below.



Joined_Table Excerpt



Indexes Excerpt



Creating a Join Index – Single Table

Non-Compressed

```
CREATE JOIN INDEX Orders_JI4 AS
  SELECT     orderid,
              custid,
              orderstatus,
              ordertotal,
              orderdate
  FROM        Orders
  PRIMARY INDEX (custid);
```

Non-Compressed and Partitioned

```
CREATE JOIN INDEX Orders_JI4A AS
  SELECT     orderid,
              custid,
              orderstatus,
              ordertotal,
              orderdate
  FROM        Orders
  PRIMARY INDEX (custid)
  PARTITION BY RANGE_N (orderdate BETWEEN
    DATE '2010-01-01' AND DATE '2019-12-31'
    EACH INTERVAL '1' DAY);
```

- The **Orders** base table is distributed across the AMPs based on the hash value of the **orderid** column (primary index of base table).
- These Join Indexes effectively represents a subset of the **Orders** table (selected columns) and are distributed across the AMPs based on the hash value of the **custid** column.
- The optimizer can use this Join Index to improve joins using the “customer id” to join with the Orders table because the Orders table does not to be redistributed in spool.
- A single table join can also be created as "compressed" join index.

Example 4 – Does the Join Index Help?

This EXPLAIN is **without** a Join Index.

- 1) First, we lock TFACT.O in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, we lock TFACT.C in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 3) We lock TFACT.O in TD_MAP1 for read, and we lock TFACT.C in TD_MAP1 for read.
 - 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.O by way of an all-rows scan with a condition of ("TFACT.O.orderstatus = 'O'") into Spool 2 in TD_MAP1 (all_amps), which is redistributed by the hash code of (TFACT.O.custid) to all AMPs in TD_MAP1. Then we do a SORT to order Spool 2 by row hash. The size of Spool 2 is estimated with high confidence to be 117,995 rows (2,477,895 bytes). The estimated time for this step is 11.17 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of a RowHash match scan. TFACT.C and Spool 2 are joined using a merge join, with a join condition of ("TFACT.C.custid = custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.C.custid). The size of Spool 1 is estimated with low confidence to be 117,995 rows (5,545,765 bytes). The estimated time for this step is 3.46 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 14.63 seconds.

This EXPLAIN is **with** a non-compressed Single Table Join Index.

- 1) First, we lock TFACT.ORDERS_JI4 in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 2) Next, we lock TFACT.C in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
 - 3) We lock TFACT.ORDERS_JI4 in TD_MAP1 for read, and we lock TFACT.C in TD_MAP1 for read.
 - 4) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to TFACT.ORDERS_JI4 by way of a RowHash match scan with a condition of ("TFACT.ORDERS_JI4.orderstatus = 'O'"). TFACT.C and TFACT.ORDERS_JI4 are joined using a merge join, with a join condition of ("TFACT.C.custid = TFACT.ORDERS_JI4.custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.C.custid). The size of Spool 1 is estimated with low confidence to be 117,995 rows (5,545,765 bytes). The estimated time for this step is 8.29 seconds.
 - 5) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 8.29 seconds.

Example 4 – Does the Join Index Help?

List the valid customers who have open orders?

```
SELECT      C.custid, C.lastname, C.firstname,
            O.orderdate
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
WHERE       orderstatus = 'O'
ORDER BY    1;
```

SQL Query

Without Join Index
With Join Index

Time

14.63 seconds
8.29 seconds

<u>custid</u>	<u>lastname</u>	<u>firstname</u>	<u>orderdate</u>
1460114	Cobb	Finn	02/28/2019
1514800	Mosley	Delilah	01/28/2019
1516554	Harper	Meredith	01/16/2019
2571123	Moon	Bently	12/28/2018
4423625	Simon	Dixie	12/19/2018
5772681	Paul	Damarion	12/31/2018
5915196	Brewer	Lucca	01/16/2019
9461517	Barnett	Tate	03/16/2019
9602794	Thompson	Jared	02/17/2019

- The rows of the customer table and the Join Index are located on the same AMP.
- A single table Join Index will help this query.

Why use Aggregate Join Indexes?

Summary Tables

Queries that involve counts, sums, or averages over large tables require processing to perform the needed aggregations. If the tables are large, query performance may be affected by the cost of performing the aggregations. Traditionally, when these queries are run frequently, users have built summary tables to expedite their performance. While summary tables do help query performance there are disadvantages associated with them as well.

Summary Tables Limitations

- Require the creation of a separate table
- Require initial population of the table
- Require refresh of changing data, either via update or reload
- Require queries to be coded to access summary tables, not the base tables
- Allow for multiple versions of the truth when the summary tables are not up-to-date

Aggregate Indexes

The primary function of an aggregate join index is to provide the Optimizer with a performance, cost-effective means for satisfying any query that specifies a frequently made aggregation operation on one or more columns. The aggregate join index permits you to define a summary table without violating schema normalization.

Aggregate indexes provide a solution that enhances the performance of the query while reducing the requirements placed on the user. All of the above listed limitations are overcome with their use.

An aggregate index is created similarly to a join index with the difference that sums, counts and date extracts may be used in the definition. A denormalized summary table is internally created and populated as a result of creation. The index can never be accessed directly by the user. It is available only to the optimizer as a tool in its query planning.

Aggregate indexes do not require any user maintenance. When underlying base table data is updated, the aggregate index totals are adjusted to reflect the changes. While this requires additional processing overhead when a base table is changed, it guarantees that the user will have up-to-date information in the index.

Why use Aggregate Join Indexes?

Summary Tables

- Queries involving aggregations over large tables are subject to high compute and I/O overhead. Summary tables often used to expedite their performance.

Summary Tables Limitations

- Require the creation of a separate summary table.
- Require initial population of the summary table.
- Requires refresh of summary table.
- Queries must access summary table, not the base table.
- Multiple “versions of the truth”.

Aggregate Join Indexes

- Aggregate join indexes enhance the performance of the query while reducing the requirements placed on the user.
- An aggregate join index is created similarly to a join index with the difference that sums, counts and date extracts may be used in the definition.

Aggregate Join Index Advantages

- Do not require any user maintenance.
- Updated automatically when base tables change (requires processing overhead)
- User will have up-to-date information in the index.

Aggregate Join Index Properties

Aggregate Indexes are similar to other Join Indexes in that they are:

- Automatically kept up to date without user involvement.
- Never accessed directly by the user.
- Optional and provide an additional choice for the optimizer.
- MultiLoad and FastLoad may not be used to load tables for which indexes are defined.

Aggregate Indexes are different from other Join Indexes in that they:

- Use the SUM and COUNT functions.
- Permit use of EXTRACT YEAR and EXTRACT MONTH from dates.

Define an aggregate join index as a join index that specifies SUM or COUNT aggregate operations. No other aggregate functions are permitted in the definition of a join index.

To avoid numeric overflow, the COUNT and SUM fields in a join index definition must be typed as FLOAT. If you do not assign a data type to COUNT and SUM, the system types them as FLOAT automatically. If you assign a type other than FLOAT, an error message occurs.

You must have one of the following two privileges to create any join index:

- CREATE TABLE on the database or user which will own the join index,
- or
- INDEX privilege on each of the base tables.

Additionally, you must have this privilege:

- DROP TABLE rights on each of the base tables.

AJIs support MIN/MAX aggregate operators (14.10)

- MIN and/or MAX value of a column expression can be pre-computed and saved in an aggregate join index.
- A query that asks for MIN and/or MAX of the same column expression can be processed using the AJI if query's GROUP BY clause is the same or a subset of the AJI's GROUP BY clause.
- MIN and MAX values materialized in an AJI are updated when the base table rows are updated and when rows are inserted into or deleted from the base table.

```
CREATE JOIN INDEX MinMaxSales_AJI
AS SELECT txn_date, store_id,
       MIN(sales_amount) AS MinSale,
       MAX(sales_amount) AS MaxSale
FROM sales_tbl GROUP BY txn_date, store_id
PRIMARY INDEX (store_id);
```

Aggregate Join Index Properties

Aggregate Indexes are similar to other Join Indexes:

- Automatically kept up to date without user involvement.
- Never accessed directly by the user.
- Optional and provide an additional choice for the optimizer.
- TPT Load (FastLoad) and TPT Update (MultiLoad) can **NOT** be used to load tables for which indexes are defined.

Aggregate Indexes differ from other Join Indexes:

- Uses the SUM and COUNT functions. **MIN/MAX functions are supported in Teradata 14.10.**
- Join index definitions only permit the extraction of years and months from a date column (e.g., EXTRACT (YEAR... or EXTRACT (MONTH ...).

Privileges required to create any Join Index:

- CREATE TABLE in the database or user which will own the join index, or INDEX privilege on each of the base tables.

Additionally, you must have this privilege:

- DROP TABLE rights on each of the base tables.

Aggregation without an Aggregate Index

The facing page contains an example of aggregation and the base table does NOT have an aggregate index.

The Orders table has 10 orderlocations and rows in different locations from 2010 through 2019. There are not orders from every location on every day.

The Orders table has 30,000,000 rows.

Note: Statistics were collected for the orderlocation, year, and month of the aggregate join index.

```
SELECT  orderlocation  
        ,EXTRACT (YEAR FROM orderdate) AS "Year"  
        ,EXTRACT (MONTH FROM orderdate) AS "Month"  
        ,MIN (ordertotal), MAX(ordertotal), SUM (ordertotal)  
FROM    Orders  
GROUP BY 1, 2, 3  
ORDER BY 1, 2, 3;
```

- 1) First, we lock TFACT.O in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) We lock TFACT.O in TD_MAP1 for read.
- 3) We do an all-AMPs SUM step in TD_MAP1 to aggregate from TFACT.Orders by way of an all-rows scan with no residual conditions, grouping by field1 (TFACT.Orders.orderlocation ,EXTRACT(YEAR FROM (TFACT.Orders.orderdate)) ,EXTRACT(MONTH FROM (TFACT.Orders.orderdate))) . Aggregate Intermediate Results are computed globally, then placed in Spool 3 in TD_MAP1. The size of Spool 3 is estimated with low confidence to be 3,000 rows (141,000 bytes). The estimated time for this step is 21.88 seconds.
- 4) We do an all-AMPs RETRIEVE step from Spool 3 (Last Use) by way of an all-rows scan into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.Orders.orderlocation, EXTRACT(YEAR FROM (TFACT.Orders.orderdate)), EXTRACT(MONTH FROM (TFACT.Orders.orderdate))) . The size of Spool 1 is estimated with low confidence to be 3,000 rows (213,000 bytes). The estimated time for this step is 0.03 seconds.
- 5) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 21.91 seconds.

If you attempt to create an aggregate join index with AVG, you will receive the following error.

```
CREAT JOIN INDEX Failed 5464: Error in Join Index DDL. Only columns and SUM,  
COUNT, MIN, MAX, or non-aggregate expressions with aliases are allowed in the select list.
```

Aggregation without an Aggregate Index

List the minimum, maximum, and total of ordertotals by year and month for every location.

```
SELECT      orderlocation
            ,EXTRACT (YEAR FROM orderdate) AS "Year"
            ,EXTRACT (MONTH FROM orderdate) AS "Month"
            ,MIN (ordertotal), MAX(ordertotal), SUM (ordertotal)
FROM        Orders
GROUP BY    1, 2, 3
ORDER BY    1, 2, 3;
```

An all-rows scan of base table is required.

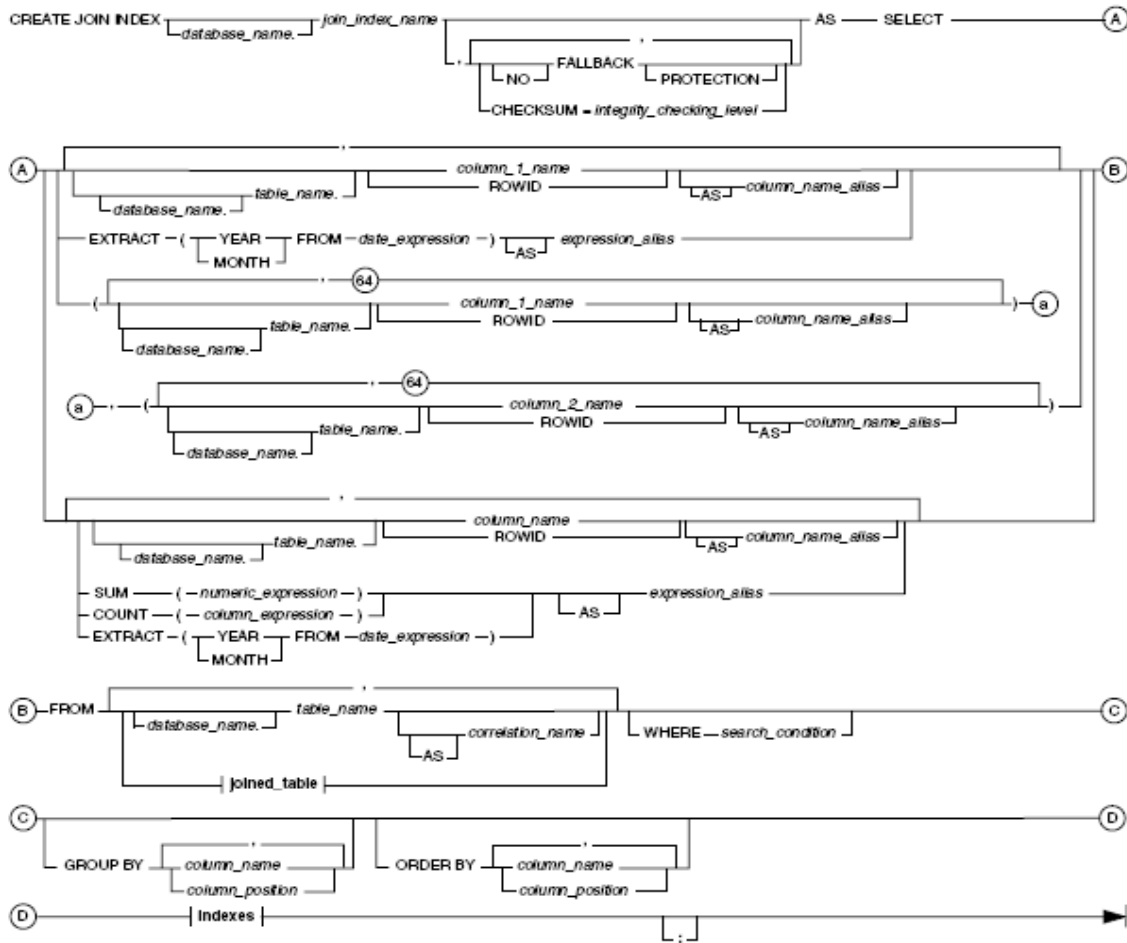
The base table has 30,000,000 rows.

EXPLAIN without an Aggregate Index (Partial Listing)

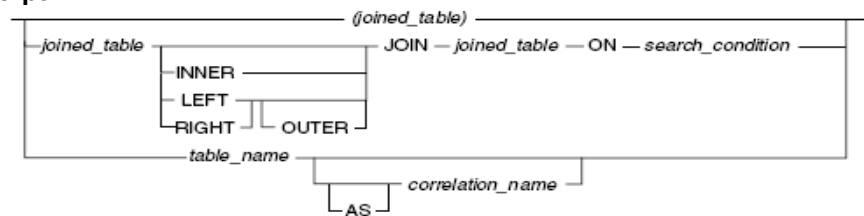
- :
- 3) We do an all-AMPs SUM step in TD_MAP1 to aggregate from TFACT.Orders by way of an all-rows scan with no residual conditions, grouping by field1 (TFACT.Orders.orderlocation , EXTRACT(YEAR FROM (TFACT.Orders.orderdate)) ,EXTRACT(MONTH FROM (TFACT.Orders.orderdate))). Aggregate Intermediate Results are computed globally, then placed in Spool 3 in TD_MAP1. The size of Spool 3 is estimated with low confidence to be 3,000 rows (141,000 bytes). The estimated time for this step is 21.88 seconds.
 - 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from Spool 3 (Last Use) by way of an all-rows scan into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.Orders.orderlocation, EXTRACT(YEAR FROM (TFACT.Orders.orderdate)) , EXTRACT(MONTH FROM (TFACT.Orders.orderdate))). The size of Spool 1 is estimated with low confidence to be 3,000 rows (213,000 bytes). The estimated time for this step is 0.03 seconds.
 - 5) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 21.91 seconds.

Creating an Aggregate Join Index

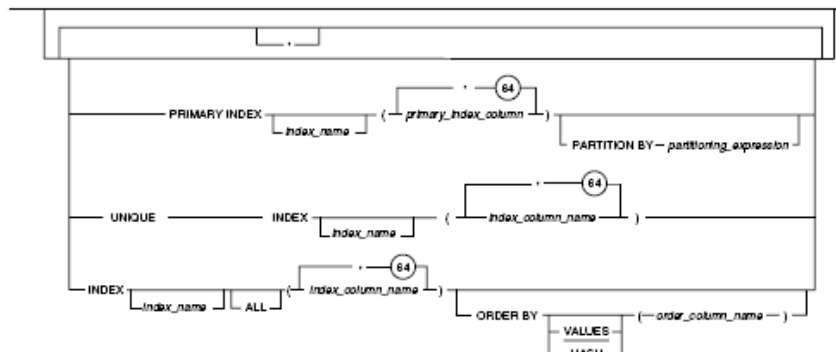
The CREATE JOIN INDEX syntax is shown below.



Joined_Table Excerpt



Indexes Excerpt



Creating an Aggregate Join Index

```

CREATE JOIN INDEX Monthly_Totals_AJI AS
SELECT
    orderlocation
    ,EXTRACT (YEAR FROM orderdate) AS "Year"
    ,EXTRACT (MONTH FROM orderdate) AS "Month"
    ,MIN (ordertotal) AS "Min Ordertotal"
    ,MAX (ordertotal) AS "Max Ordertotal"
    ,SUM (ordertotal) AS "Order_Totals"
FROM Orders
GROUP BY 1, 2, 3;

```

```

COLLECT STATISTICS
    COLUMN orderlocation,
    COLUMN "Year",
    COLUMN "Month"
ON
    Monthly_Totals_AJI;

HELP STATISTICS Monthly_Totals_AJI;

```

Date	Time	Unique Values	Column Names	Column Dictionary Names	Column SQL Names
18/12/26	09:17:36	1200	*	*	""
18/12/26	09:17:36	10	orderlocation	orderlocation	orderlocation
18/12/26	09:17:36	10	"Year"	Year	"Year"
18/12/26	09:17:36	12	"Month"	Month	"Month"

Example 5: Aggregation with an AJI

The facing page illustrates execution of the same query with an aggregate join index

Example 5: Aggregation with an AJI

List the minimum, maximum, and total of ordertotals by year and month for every location.

```
SELECT      orderlocation
            ,EXTRACT (YEAR FROM orderdate) AS "Year"
            ,EXTRACT (MONTH FROM orderdate) AS "Month"
            ,MIN (ordertotal), MAX(ordertotal), SUM (ordertotal)
FROM        Orders
GROUP BY    1, 2, 3
ORDER BY    1, 2, 3;
```

An all-rows scan of aggregate join index is required.

EXPLAIN with an Aggregate Index

- 1) First, we lock TFACT.MONTHLY_TOTALS_AJI in TD_MAP1 for read on a reserved RowHash to prevent global deadlock.
- 2) Next, we lock TFACT.MONTHLY_TOTALS_AJI in TD_MAP1 for read.
- 3) We do an **all-AMPs RETRIEVE step in TD_MAP1 from TFACT.MONTHLY_TOTALS_AJI** by way of an **all-rows scan** with no residual conditions into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.MONTHLY_TOTALS_AJI.orderlocation, TFACT.MONTHLY_TOTALS_AJI."Year", TFACT.MONTHLY_TOTALS_AJI."Month"). The size of Spool 1 is estimated with high confidence to be 1,200 rows (85,200 bytes). The estimated time for this step is 0.03 seconds.
- 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.03 seconds.**

Sparse Join Indexes

Another capability of the join index allows you to index a portion of the table using the WHERE clause in the CREATE JOIN INDEX statement to limit the rows indexed. You can limit the rows that are included in the join index to a subset of the rows in the table based on an SQL query result. This is also referred to as a “Partial Covering” join index.

Any join index, whether simple or aggregate, multi-table or single-table, can be sparse.

Examples of how the Sparse Join Index may be used include:

- Ignore rows that are NULL or are most common
- Index rows whose Quantity < 100
- Index a time segment of the table – rows that relate to this quarter

Customer Benefit

A sparse index can focus on the portion of the table(s) that is most frequently used.

- Reduces the storage requirements for a join index
- Makes access faster since the size of the JI is smaller

Like other index choices, a sparse JI should be chosen to support high frequency queries that require short response times. A sparse JI allows the user to:

- Use only a portion of the columns in the base table.
- Index only the values you want to index.
- Ignore some columns, e.g., nulls, to keep access smaller and faster than before.
- Avoid maintenance costs for updates

When the index size is smaller there is less work to maintain and updates are faster since there are fewer rows to update. A sparse JI contents can be limited by date, location information, customer attributes, or a wide variety of selection criteria combined with AND and OR conditions.

Performance

- Better update performance on the base table when its indexes do not contain the most common value(s)
- Smaller index size
- Improved IO and storage
- Collect statistics on the index even if it is only a single column

Limitations

Sparse Join Indexes follow the same rules as normal Join Indexes

Sparse Join Indexes

- Allows you to index a portion of the table using the WHERE clause in the CREATE JOIN INDEX statement to limit the rows indexed.
- Any join index, whether simple or aggregate, multi-table or single-table, can be created as a sparse index.

Examples of how the Sparse Join Index may be used include:

- Ignore rows that are NULL or are most common
- Index rows whose Quantity < 100
- Index a time segment of the table – rows that relate to this quarter

Benefits

- A sparse index can focus on the portion of the table(s) that are most frequently used.
 - Reduces the storage requirements for a join index
 - Faster to create or build
 - Makes access faster since the size of the Join Index is smaller
 - Better update performance on the base table when its indexes do not contain the most common value(s)

Creating a Sparse Join Index

The facing page contains an example of creating a “Sparse Join Index”. The following EXPLAIN shows that the Sparse Join Index is used.

- ... (Locking steps)
- 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.CUST_ORD_JI5 by way of an all-rows scan with a condition of ("(TFACT.CUST_ORD_JI5.orderstatus = 'O') AND (((EXTRACT(DAY FROM (TFACT.CUST_ORD_JI5.orderdate)))= 18) AND ((EXTRACT(MONTH FROM (TFACT.CUST_ORD_JI5.orderdate)))= 1)))" into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with no confidence to be 6,547 rows (274,974 bytes). The estimated time for this step is 0.05 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.05 seconds.

This following EXPLAIN shows that the Sparse Join Index is **not** used.

- ... (Locking steps)
- 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.O by way of an all-rows scan with a condition of ("(TFACT.O.orderdate = DATE '2018-12-28') AND (TFACT.O.orderstatus = 'O')") into Spool 2 (all_amps), which is redistributed by the hash code of (TFACT.O.custid) to all AMPs. Then we do a SORT to order Spool 2 by row hash. The size of Spool 2 is estimated with no confidence to be 12,950 rows (271,950 bytes). The estimated time for this step is 4.40 seconds.
 - 5) We do an all-AMPs JOIN step in TD_MAP1 from TFACT.C by way of a RowHash match scan with no residual conditions, which is joined to Spool 2 (Last Use) by way of a RowHash match scan. TFACT.C and Spool 2 are joined using a merge join, with a join condition of ("TFACT.C.custid = custid"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with no confidence to be 12,950 rows (543,900 bytes). The estimated time for this step is 3.06 seconds.
 - 6) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 7.46 seconds.

PERM Space Required

The amount of PERM space used by this sparse join index as compared to the full join index is listed below.

<u>Join Index Name</u>	<u>Permanent Space</u>	<u>Type of Join Index</u>
Cust_Ord_JI1	20,848,640	non-compressed multi-table
Cust_Ord_JI5	5,799,936	sparse non-compressed multi-table

Creating a Sparse Join Index

```
CREATE JOIN INDEX Cust_Ord_JI5 AS
  SELECT      C.custid as c_custid, C.lastname, C.firstname,
              O.orderid, O.orderstatus, O.ordertotal, O.orderdate
  FROM        Customer C
  INNER JOIN  Orders O
  ON          C.custid = O.custid
  WHERE       EXTRACT (YEAR FROM O.orderdate) =
              EXTRACT (YEAR FROM Current_Date)
  PRIMARY INDEX (c_custid);
```

Assume the current year is 2019.

```
SELECT      C.custid, C.lastname, O.orderdate
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
WHERE       O.orderdate = '2019-01-18'
AND         O.orderstatus = '0';
```

The join index will be used for this SQL and the EXPLAIN estimated cost is 0.05 seconds.

```
SELECT      C.custid, C.lastname, O.orderdate
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
WHERE       O.orderdate = '2018-12-28'
AND         O.orderstatus = '0';
```

The tables will have to be joined for this SQL and the EXPLAIN estimated cost is 7.46 seconds.

Creating a Sparse Join Index on a Partitioned Table

The facing page contains an example of creating a sparse join index on a partitioned table. There are two benefits to creating this join index on a partitioned base table.

- It is created very quickly because of partition elimination on the base table.
- The Sparse Join is only 60,456,960 bytes in size since it is only created for 3 months.

A portion of the EXPLAIN to create a sparse join index on a set of partitions is included.

EXPLAIN

```
CREATE JOIN INDEX Orders_RP_JI1 AS
SELECT orderid, custid, orderstatus, ordertotal, orderdate
FROM    Orders_RP
WHERE   orderdate BETWEEN '2019-01-01' AND '2019-03-31'
PRIMARY INDEX (custid);
```

: (Locking steps and miscellaneous dictionary steps)

- 7) We do an **all-AMPs RETRIEVE step in TD_MAP1 from 90 partitions of TFACT.Orders_RP** with a condition of `("(TFACT.Orders_RP.orderdate >= DATE '2019-01-01') AND (TFACT.Orders_RP.orderdate <= DATE '2019-03-31')")` into Spool 1(all_amps), which is redistributed by the hash code of `(TFACT.Orders_RP.custid)` to all AMPs. Then we do a SORT to order Spool 1 by row hash. The size of Spool 1 is estimated with high confidence to be 5,898,928 rows (176,967,840 bytes). The estimated time for this step is 22.89 seconds.
- 8) We do an all-AMPs MERGE in TD_MAP1 into TFACT.Orders_RP_JI1 from Spool 1 (Last Use). The size is estimated with no confidence to be 5,898,928 rows. The estimated time for this step is 42.39 seconds.

: other steps

Creating a Sparse Join Index on a Partitioned Table

If the base table is partitioned, a sparse join index may be created for a partition or partitions – only the “partitions of interest” need to be scanned.

- Creation time of sparse join index is faster because of partition elimination.
- Partition scan of base table instead of full table scan.

```
CREATE TABLE Orders_RP
  (orderid      INTEGER NOT NULL,
   custid       INTEGER NOT NULL,
   :           :
   ordercomment VARCHAR(79))
PRIMARY INDEX  (orderid)
PARTITION BY RANGE_N (orderdate BETWEEN
                     DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' DAY);
```

Orders_RP table is partitioned by day.

```
CREATE JOIN INDEX Orders_RP_JI1 AS
  SELECT      orderid, custid, orderstatus, ordertotal, orderdate
  FROM        Orders_RP
  WHERE       orderdate BETWEEN '2019-01-01' AND '2019-03-31'
  PRIMARY INDEX (custid);
```

The sparse join index is created for the 1st quarter of 2019.

- 7) We do an all-AMPs RETRIEVE step in TD_MAP1 from 90 partitions of TFACT.Orders_RP with a condition of ("(TFACT.Orders_RP.orderdate <= DATE '2019-03-31') AND (TFACT.Orders_RP.orderdate >= DATE '2019-01-01')") into Spool 1 (all_amps), ...

Only 90 partitions are scanned to build the JI.

ALTERing a Join Index to CURRENT

Starting with Teradata 13.10, you can now specify CURRENT_DATE and CURRENT_TIMESTAMP functions in a join index.

Also starting with Teradata 13.10, Teradata provides a new option with the ALTER TABLE statement to modify a join index that has been defined with a moving CURRENT_DATE (or DATE) or moving CURRENT_TIMESTAMP. This new option is called ALTER TABLE TO CURRENT.

When you specify CURRENT_DATE and CURRENT_TIMESTAMP as part of a partitioning expression for a join index, these functions resolve to the date and timestamp when you define the PPI. To partition on a new CURRENT_DATE or CURRENT_TIMESTAMP value, submit an ALTER TABLE TO CURRENT request.

The options WITH DELETE and WITH INSERT [INTO] *save_table* option are not available for a join index.

ALTERing a Join Index to CURRENT

This option allows you to periodically resolve the CURRENT_DATE (or DATE) and CURRENT_TIMESTAMP of a join index to their current values.

Benefits include:

- You do not have to use ALTER TABLE to change to partitioning (drop and/or add partitions) for a partitioned join index.
- To partition on a **new** CURRENT_DATE or CURRENT_TIMESTAMP value, simply submit an ALTER TABLE TO CURRENT request.

ALTER TABLE join_index_name TO CURRENT;

Example: ALTER TABLE Cust_Ord_SJI TO CURRENT;

Considerations:

- The options WITH DELETE and WITH INSERT INTO *save_table* are not available for a join index.
- If RANGE_N specifies CURRENT_DATE or CURRENT_TIMESTAMP in a partitioning expression, **you cannot use ALTER TABLE to add or drop ranges for the join index.** You must use the ALTER TABLE TO CURRENT statement to achieve this function.

Partitioning a Sparse Join Index

The example on the facing page contains an example of creating a partitioned sparse non-compressed join index and also includes a NUSI on the same partitioned sparse non-compressed join index.

This example also illustrates the use of CURRENT_DATE in the sparse portion and in the partitioning expression of the join index.

An alternative partitioning expression (to the facing page) is:

```
PRIMARY INDEX (custid)  
  PARTITION BY RANGE_N (orderdate BETWEEN  
    CAST(((EXTRACT(YEAR FROM CURRENT_DATE) - 1900) * 10000 + 0101)  
      AS DATE) AND  
    CAST(((EXTRACT(YEAR FROM CURRENT_DATE) - 1900) * 10000 + 1231)  
      AS DATE)  
  EACH INTERVAL '1' DAY)
```

Partitioning a Sparse Join Index

A sparse non-compressed join index can also be partitioned.

- This allows a partition scan of the join index instead of full join index.
- This example creates a sparse partitioned join index for the current year.
 - Teradata allows the use of CURRENT_DATE (or DATE) and CURRENT_TIMESTAMP in a join index definition.
 - Use ALTER TABLE *join_index* TO CURRENT; to resolve the dates in the next year.
- This example also creates a NUSI on the join index.

```
CREATE JOIN INDEX Orders_RP_JI2 AS
  SELECT orderid, custid, orderstatus, ordertotal, orderdate, clerkid
  FROM    Orders_RP
  WHERE   EXTRACT (YEAR FROM orderdate) = EXTRACT (YEAR FROM Current_Date)

PRIMARY INDEX (custid)
  PARTITION BY RANGE_N (orderdate BETWEEN
    CAST ((EXTRACT(YEAR FROM CURRENT_DATE)) || '-01-01' AS DATE) AND
    CAST ((EXTRACT(YEAR FROM CURRENT_DATE)) || '-12-31' AS DATE)
    EACH INTERVAL '1' DAY)

INDEX (clerkid);
```

To resolve the dates in the next year, ALTER TABLE Orders_RP_JI2 TO CURRENT;

Global (Join) Indexes

A Global Index is a term used to define a join index that contains the Row IDs of the base table rows. Some queries are satisfied by examining only the join index when all referenced columns are stored in the index. Such queries are said to be covered by the join index.

Other queries may use the join index to qualify a few rows, and then refer to the base tables to obtain requested columns that aren't stored in the join index. Such queries are said to be partially-covered by the index. This is referred to as a partially-covered global index.

Because the Teradata Database supports multi-table, partially-covering join indexes, all types of join indexes, except the aggregate join index, can be joined to their base tables to retrieve columns that are referenced by a query but are not stored in the join index. Aggregate join indexes can be defined for commonly-used aggregation queries.

A partial-covering join index takes less space than a covering join index, but in general may not improve query performance by as much. Not all columns that are involved in a query selection condition have to be stored in a partial-covering join index. The benefits are:

- Disk storage space for the JI decreases when fewer columns are stored in the JI.
- Performance increases when the number of selection conditions that can be evaluated on the join index increases.

When a Row ID is included in a Join Index, 10 bytes are used for the Row ID (Part # + Row Hash + Uniqueness Value). This is true even if the base table is not partitioned. Starting with Teradata 14.0, if the base table is partitioned with > 65,535 partitions, the RowID in the Join Index is 16 bytes long.

Another use for a Global Join Index for a single table is that of a Hashed NUSI. This capability will be described in more detail later in the module.

Customer Benefit

Partial-Covering Global Join Indexes can provide:

- Improved performance for certain class of queries.
- Some of the query improvement benefits that join indexes offer without having to replicate all the columns required to cover the queries resulting in better performance.
- Improved scalability for certain class of queries

Limitations

- Aggregate JI does not support the partial-covering capability.
- Global index is not used for correlated sub-queries.
- Global index is not supported with TPT Load (FastLoad) or TPT Update (MultiLoad).

Global (Join) Indexes

A Global Index is a term used to define a join index that contains the Row IDs of all of the tables in the join index.

Example:

- Assume that you are joining 2 tables (Table_A and Table_B) and each has 100 columns. **The join index can include (at most) 64 columns from each base table.**
- You can include the ROWID as part of the 64 columns for each table (ex., A.ROWID and B.ROWID). Each join index subtable row will include the Row IDs of the corresponding base table rows for Table_A and Table_B.
- **The optimizer can build a plan that uses the join index to get most of the data and can join back to either or both of the tables for the rest of the data.**

Another option – use a single table Global Index as a “**Hashed NUSI**” – the join index contains the “secondary index column” and the Row IDs in the join index.

- Useful when a column is used as a secondary index, but the typical number of rows for a value is much less than the number of AMPs in the system.
- For queries with an equality condition on a fairly unique column, it changes:
 - Table-level locks to row hash locks
 - All-AMP steps to group-AMP steps

Global Index – Multiple Tables

The facing page contains an example of creating a “global” multi-table join index.

The total number of columns in the Customer and Orders table is less than 64. Therefore, you could create a join index that includes all of the columns (16 in this case) from the two tables. However, if one table had 70 columns and the other table had 90 columns, you can only include a maximum of 64 columns from each table. Since the number of columns is more than 64, you would include the most frequently referenced columns from each table. Another reason to only include a subset of columns may be to minimize the size of the join index.

In this example, the join index has the most frequently referenced columns as well as the Row IDs of both tables. The join index subtable row will include the Row ID of the base table row for the Customer table and the Row ID of the base table for the Orders table. The optimizer may choose to use the join index to get most of the data and join back to either the Customer or the Orders table. The optimizer can build execution plans that can join back to either table. The terminology used in EXPLAIN plans that indicates a join back is “... using a row id join ...”.

Join back simply means that the ROWID is carried as part of the join index. This permits the index to “join back” to the base row, much like a NUSI does. It is one of the features of Partial-Covering Join Indexes. An additional example is provided below.

```
EXPLAIN SELECT      C.custid, C.lastname,
                    C.address, C.city, C.statecode,
                    O.orderdate
FROM               Customer C
INNER JOIN         Orders O
ON                 C.custid = O.custid
WHERE              O.orderstatus = '0'
ORDER BY          1;
```

This EXPLAIN shows that a Global Join Index is used.

- 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.CUST_ORD_JI6 by way of an all-rows scan with a condition of ("TFACT.CUST_ORD_JI6.orderstatus = '0'") into Spool 2 4) We do an all-AMPs RETRIEVE step in TD_MAP1 from TFACT.CUST_ORD_JI6 by way of an all-rows scan with a condition of ("TFACT.CUST_ORD_JI6.orderstatus = '0'") into Spool 2 (all_amps), which is redistributed by the hash code of (TFACT.CUST_ORD_JI6.Field_1027) to all AMPs in TD_Map1. Then we do a SORT to order Spool 2 by the sort key in spool field1. The size of Spool 2 is estimated with no confidence to be 50,205 rows (1,907,790 bytes). The estimated time for this step is 1.80 seconds.
- 5) We do an all-AMPs JOIN step in TD_Map1 from Spool 2 (Last Use) by way of an all-rows scan, which is joined to TFACT.C by way of an all-rows scan with no residual conditions. Spool 2 and TFACT.C are joined using a row id join, with a join condition of ("Field_1 = TFACT.C.ROWID"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. Then we do a SORT to order Spool 1 by the sort key in spool field1 (TFACT.CUST_ORD_JI6.custid). The size of Spool 1 is estimated with no confidence to be 50,205 rows (4,367,835 bytes). The estimated time for this step is 0.59 seconds.

Global Index – Multiple Tables

```
CREATE SET TABLE Customer
(custid      INTEGER NOT NULL,
 lastname    VARCHAR(15),
 firstname   VARCHAR(10),
 address     VARCHAR(100),
 city        VARCHAR(20),
 statecode   CHAR(2),
 zipcode     CHAR(5))
UNIQUE PRIMARY INDEX (custid);
```

```
CREATE SET TABLE Orders
(orderid     INTEGER NOT NULL,
 custid      INTEGER NOT NULL,
 orderstatus CHAR(1),
 ordertotal  DECIMAL(9,2) NOT NULL,
 orderdate   DATE
             FORMAT 'YYYY-MM-DD' NOT NULL,
 orderpriority SMALLINT,
 clerkid     CHAR(16),
 shippriority SMALLINT,
 orderlocation SMALLINT,
 ordercomment VARCHAR(79))
UNIQUE PRIMARY INDEX (orderid);
```

```
CREATE JOIN INDEX Cust_Ord_JI6 AS
SELECT      C.custid AS c_custid, C.lastname, C.ROWID AS crid,
           orderid, orderstatus, orderdate, O.ROWID AS orid
FROM        Customer C
INNER JOIN  Orders O
ON          C.custid = O.custid
PRIMARY INDEX (c_custid);
```

- The Global Index contains those columns most frequently used in queries – effectively used as “covering join index”.
- The [Global Index may be used to join back \(via the Row ID\) to the tables](#) when columns are referenced that are not part of the join index; this is shown in the EXPLAIN on the facing page.

Global Index as a “Hashed NUSI”

Another use for a Global Join Index for a single table is that of a Hashed NUSI. An actual NUSI accesses all AMPs, whereas this index only accesses the AMPs that have rows matching the value. The Global Join Index is hashed, and the system uses the hash to access a single AMP, and then uses the Row IDs in the subtable row to then access only those AMPs that have rows.

ODS (Operational Data Store) or tactical queries that involve an equality condition on a fairly unique column can use a global index which will change:

- Table-level locks to row hash locks
- All-AMP steps to group-AMP steps

Using a global index as a “Hashed NUSI” is similar to a single-table join index with one clear differentiation – it carries a pointer (Row ID) back to the base table, and is used as an alternative means to get to the base table row. It is not used to satisfy a query by itself.

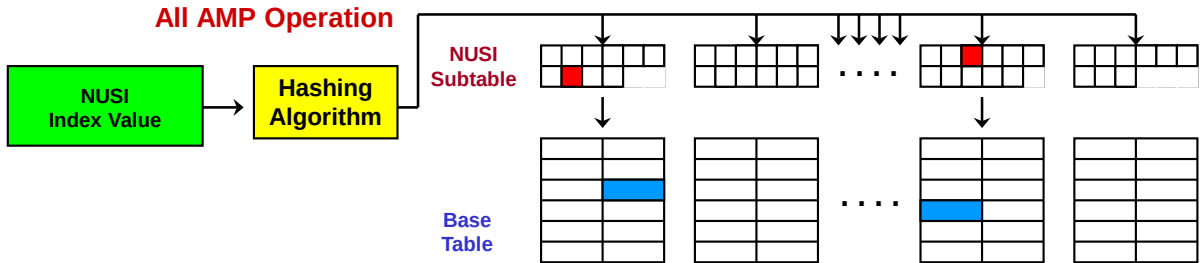
A “hashed NUSI” global index offers the advantages of a NUSI (it supports duplicate rows per value) combined with the advantages of a USI (its index rows are hash-distributed on the indexed value) and is often able to offer group AMP capabilities. As with all Teradata join indexes, its use is transparent to the query and will be determined by the optimizer.

Its main benefit is for situations where you are only getting a few rows for one value, and you can avoid an all AMP operation that a NUSI always requires. This may not have a huge impact on a system with a modest number of AMPs. However, for very large systems, with hundreds or thousands of AMPs, a group AMP operation that engages a small percentage (e.g., only 1% of the total AMPs or less), when done often enough, may increase overall throughput of the platform, as well as faster query response.

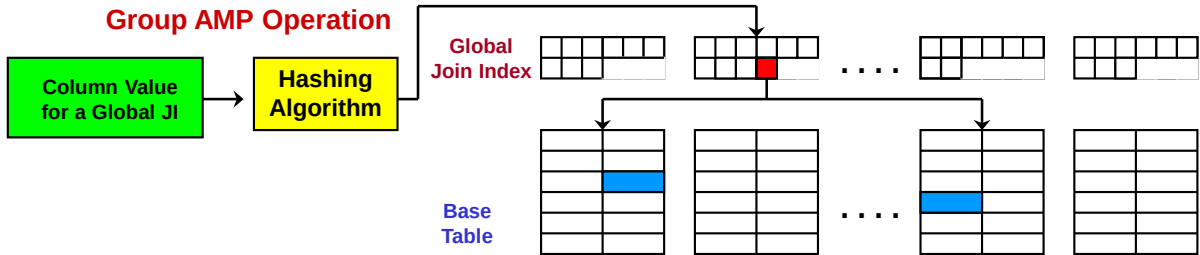
Global Index as a “Hashed NUSI”

Assume 200 AMPs and the typical rows per value is 2 for a column referenced in queries.

- A NUSI on this column avoids a FTS, but it is an **all-AMP operation** and every AMP is required to look in the NUSI subtable for the hash value of the NUSI.



- A Global Join Index (“Hashed NUSI”) on this column utilizes a **Group AMP operation**. The GJI subtable row contains the Row IDs of the base table rows.



Creating a Global Index (“Hashed NUSI”)

The facing page contains an example creating a Global Join Index as a “Hashed NUSI”.

If the number of Row IDs within the Global Index subtable row is less than 50% of the number of AMPs, you will see an EXPLAIN plan with “group-AMPs” operations.

If the number of Row IDs within the Global Index subtable row is more than 50% of the number of AMPs, you will see an EXPLAIN plan with “all -AMPs” operations.

Note: When the Row ID is included in a Join Index, each Row ID is 10 bytes long. The partition number is included even if the base table is not partitioned. The partition number is 0 for non-partitioned tables.

Creating the Global Join Index without “Repeating Row Ids”

If the Global Join Index (that is to be used as a hashed NUSI) is created as follows – without the parenthesis, multiple Row IDs will not be included in a single join index subtable row.

```
CREATE JOIN INDEX    Orders_JI8
AS
SELECT              custid, ROWID
FROM                 Orders
PRIMARY INDEX        (custid);
```

Creating a Global Index (“Hashed NUSI”)

```
CREATE JOIN INDEX Orders_JI7 AS
SELECT      (custid),
            (ROWID)
FROM        Orders
PRIMARY INDEX (custid);
```

Fixed portion of Join Index contains index value

Repeating portion of Join Index contains Row IDs

One Global Index row can contain multiple Row IDs.

- If the typical rows for a value is less than 50% of the number of AMPs, this global index will yield performance gains.

This effectively means that the number of Row IDs in the subtable row is less than 50% of the number of AMPs.

- EXPLAIN plan will indicate “group-AMPs” operation and “row hash locks”.

- If the typical rows for a value is 50% or greater than the number of AMPs, this global index will result in an all-AMP operation (like a NUSI).

This effectively means that the number of Row IDs in the subtable row is 50% or more than the number of AMPs.

- EXPLAIN plan will indicate “all-AMPs” operation and “table level locks”.

Example 6: Using a Global Index as a Hashed NUSI

The EXPLAIN using a Global Index is shown on the facing page.

Example 6: Using a Global Index as a Hashed NUSI

List the orders for a customer id of 114545.

```
SELECT    custid,orderid,orderstatus,orderdate,ordertotal
FROM      Orders
WHERE     custid = 114545 ORDER BY 2;
```

<u>Custid</u>	<u>orderid</u>	<u>orderstatus</u>	<u>orderdate</u>	<u>ordertotal</u>
114545	148236	C	07/31/2019	2,096.74
114545	3269025	C	11/09/2019	1,373.52
114545	6226371	C	06/07/2019	1,343.13
114545	17069156	C	12/05/2018	1,597.50

- 1) First, we do a **single-AMP RETRIEVE step from TFACT.ORDERS_JI7 by way of the primary index** "TFACT.ORDERS_JI7.custid = 114545" with no residual conditions into Spool 2 (group_amps), which is redistributed by the hash code of (TFACT.Orders_JI7.Field_1026) to few AMPs in TD_MAP1. Then we do a SORT to order Spool 2 by the sort key in spool field1. The size of Spool 2 is estimated with **high confidence to be 4 rows** (108 bytes). The estimated time for this step is 0.00 seconds.
- 2) Next, we do a group-AMPs JOIN step from Spool 2 (Last Use) by way of an all-rows scan, which is joined to TFACT.Orders by way of an all-rows scan with no residual conditions. Spool 2 and TFACT.Orders are joined using a **row id join**, with a join condition of ("Field_1 = TFACT.Orders.ROWID"). The result goes into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with low confidence to be 4 rows (176 bytes). The estimated time for this step is 0.05 seconds.
- 3) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.05 seconds.**

Repeating Row Ids in Global Index

The SQL to create a Global Join Index with and without repeating Row IDs is shown on the facing page. To create a Global Join Index in a non-compressed format, the following DDL can be used.

```
CREATE JOIN INDEX      Orders_JI8
AS
SELECT                custid, ROWID
FROM                  Orders
PRIMARY INDEX         (custid);
```

PERM Space Required

The amount of PERM space used by these global join indexes is listed below. Remember that these tables are quite small. Note that the global join index with parenthesis requires less space.

```
SELECT      TableName , SUM(CurrentPerm)
FROM        DBC.TableSize
WHERE       DatabaseName = USER
GROUP BY    1
ORDER BY    1;
```

<u>Join Index Name</u>	<u>Permanent Space</u>	<u>Type of Join Index</u>
Orders_JI7	550,330,368	compressed single table JI with Row ID
Orders_JI8	660,553,728	non-compressed single table JI with Row ID

Repeating Row IDs in Global Index

What is the difference in creating a “Hashed NUSI” with or without repeating Row IDs?
Answer – the amount of PERM space needed.

```
CREATE JOIN INDEX Orders_JI7 AS
SELECT      (custid), (ROWID)
FROM        Orders
PRIMARY INDEX (custid);
```

```
CREATE JOIN INDEX Orders_JI8 AS
SELECT      custid, ROWID
FROM        Orders
PRIMARY INDEX (custid);
```

Compressed Join Index					
Join Index Row ID	Fixed Columns	Repeating Columns (Orders)			
RH		1500 ...	RID 1	RID 2	RID 3
RH		1501...	RID 4	RID 5	

Non-Compressed Join Index			
Join Index Row ID	Join Index Column Values		
RH		1500 ...	RID – Order1
RH		1500 ...	RID – Order2
RH		1500...	RID – Order3
RH		1501...	RID – Order4
RH		1501...	RID – Order5

TableName	SumPerm	
Orders_JI7	550,330,368	(compressed)
Orders_JI8	660,553,728	(non-compressed)

Summary

The facing page summarizes the key topics presented in this module.

Teradata provides additional index choices that can be used to improve performance for known queries.

Reasons to use a Join Index:

- May be used to pre-join multiple tables.
- May be used as an aggregate index.
- The WHERE clause can be used to limit the number of rows in the join index.
 - Referred to a **"Sparse Index"**.
- Row ID(s) of table (or tables) can be included to create a **"Global Index"**.
- May be used as a **"Hashed NUSI"**.
- Secondary indexes can be created on a join index. Secondary indexes can be ordered by value or hash.

Module 23: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 23: Review Questions

Check the box if the attribute applies to the index.

	Compressed Join Index Syntax	Non- Compressed Join Index Syntax	Aggregate Join Index	Sparse Join Index	Hash Index
May be created on a single table					
May be created on multiple tables					
Requires the use of SUM, COUNT or MIN/MAX functions.					
Requires a WHERE condition to limit rows stored in the index.					
Automatically updated as base table rows are inserted or updated					
Automatically includes the base table PI value as part of the index					

Lab Exercise 23-1

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Use the following SQL for this exercise.

To populate a table:

```
INSERT INTO tablename1  
SELECT * FROM tablename2;
```

The Explain from the facing page is listed below for your convenience and can be pasted into your query window.

```
EXPLAIN  
SELECT          LastName, FirstName, DeptName, JobDesc  
FROM            Employee E  
INNER JOIN      Department D ON  E.DeptNumber = D.DeptNumber  
INNER JOIN      Job J          ON  E.JobCode = J.JobCode  
ORDER BY        3, 1, 2;
```

Lab Exercise 23-1

Lab Exercise 23-1

Purpose

In this lab, you will create join indexes and evaluate the Explains of various joins.

What you need

Populated PD tables and Employee, Department, and Job tables in your database

Tasks

1. Verify the number of rows in Employee, Job, and Department tables. If not correct, use INSERT/SELECT to populate the tables from the PD database.

Employee	Count = 150,000
Department	Count = 17,415
Job	Count = 11,124

2. Execute the following EXPLAIN statement.

```
EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc
FROM        Employee E
INNER JOIN  Department D      ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J            ON  E.JobCode = J.JobCode
ORDER BY    3, 1, 2;
```

What is the total estimated time cost for this EXPLAIN? _____

Lab Exercise 23-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

The Explain from the facing page is listed below for your convenience and can be pasted into your query window.

```
EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc
FROM        Employee      E
INNER JOIN  Department D ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J         ON  E.JobCode = J.JobCode
ORDER BY    3, 1, 2;
```

Use the following SQL for this exercise.

To create a join index:

```
CREATE JOIN INDEX join_index_name AS
SELECT ...;
```

SUM of Perm space using the DBC.TableSizeV view.

```
SELECT      TableName (CHAR(15)), SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'join_index_name'
GROUP BY    1
ORDER BY    1;
```

Lab Exercise 23-1 (cont.)

3. Create a “non-compressed” join index which includes the following columns of Employee, Department, and Job.

Employee	- LastName, FirstName
Department	- DeptNumber, DeptName
Job	- JobCode, JobDesc

Execute the HELP USER command. What is the object type of the Join Index? _____

4. Execute the following EXPLAIN (same SQL statement as step #2).

```

EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc
FROM        Employee E
INNER JOIN  Department D ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J          ON  E.JobCode = J.JobCode
ORDER BY   3, 1, 2;
```

What is the total estimated time cost for this EXPLAIN? _____

Is the join index used? _____

How much space does the join index require (use the DBC.TableSizeV view)? _____

Lab Exercise 23-1 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

The Explain from the facing page is listed below for your convenience and can be pasted into your query window.

```
EXPLAIN  
SELECT          LastName, FirstName, DeptName, JobDesc,  
                  SalaryAmount  
FROM            Employee E  
INNER JOIN      Department D ON  E.DeptNumber = D.DeptNumber  
INNER JOIN      Job J          ON  E.JobCode = J.JobCode  
ORDER BY        3, 1, 2;
```


Lab Exercise 23-1 (cont.)

5. Execute the following EXPLAIN statement – SalaryAmount has been added as a projected column.

```
EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc, SalaryAmount
FROM        Employee E
INNER JOIN  Department D ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J          ON  E.JobCode = J.JobCode
ORDER BY    3, 1, 2;
```

What is the total estimated time cost for this EXPLAIN? _____

Is the join index used? _____ If not, why not? _____

6. Drop the Join Index.
7. Create a second “non-compressed” join index similar to step #3 except include the Primary Index of EmpNumber for the Join Index.

```
Employee      – LastName, FirstName, EmpNumber
Department    – DeptNumber, DeptName
Job           – JobCode, JobDesc
```

8. Execute the EXPLAIN statement from step #5.

What is the total estimated time cost for this EXPLAIN? _____

Is the join index used? _____ If so, why? _____

9. Drop the Join Index.

Lab Exercise 23-2 (optional)

Check your understanding of the concepts discussed in this module and the columnar module by completing the lab exercises as directed by your instructor.

Use the following SQL for this exercise.

To populate a table:

```
INSERT INTO tablename1  
SELECT * FROM tablename2;
```

The Explain from the facing page is listed below for your convenience and can be pasted into your query window.

```
EXPLAIN  
SELECT      D.DeptName, E.EmpNumber, E.LastName, E.FirstName,  
              J.JobDesc, P.AreaCode, P.PhoneNumber  
FROM        Employee E  
INNER JOIN  Department D ON E.DeptNumber = D.DeptNumber  
INNER JOIN  EmpPhone P   ON P.EmpNumber = E.EmpNumber  
INNER JOIN  Job J        ON E.JobCode = J.JobCode  
ORDER BY    1, 2, 3, 4;
```

Lab Exercise 23-2 (optional)

Lab Exercise 23-2 (optional)

Purpose

In this lab, you will create join indexes using Columnar and evaluate the various affects.

What you need

Populated PD tables and Employee, Department, Job and EmpPhone tables in your database

Tasks

1. Verify the number of rows in Employee, Job, Department and EmpPhone tables. If not correct, use INSERT/SELECT to populate the tables from the PD database.

Employee	Count = 150,000	Job	Count = 11,124
Department	Count = 17,415	EmpPhone	Count = 350,000

2. Execute the following Explain statement which joins all 4 tables together.

```
EXPLAIN
SELECT      D.DeptName, E.EmpNumber, E.LastName, E.FirstName,
            J.JobDesc, P.AreaCode, P.PhoneNumber
FROM        Employee E
INNER JOIN  Department D ON      E.DeptNumber = D.DeptNumber
INNER JOIN  EmpPhone P   ON P.EmpNumber = E.EmpNumber
INNER JOIN  Job J        ON E.JobCode = J.JobCode
ORDER BY    1, 2, 3, 4;
```

What is the total estimated time cost for this EXPLAIN? _____

Lab Exercise 23-2 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Use the following SQL for this exercise.

To create a join index:

```
CREATE JOIN INDEX  employee_JI AS
SELECT  EmpNumber,
        DeptNumber,
        EmpMgrNumber,
        JobCode,
        LastName,
        FirstName,
        SalaryAmount,
        ROWID AS RID
FROM    Employee
PRIMARY INDEX (DeptNumber);
```

SUM of Perm space using the DBC.TableSizeV view.

```
SELECT      TableName (CHAR(15)), SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'join_index_name'
GROUP BY    1
ORDER BY    1;
```

The Explain from the facing page is listed below for your convenience and can be pasted into your query window.

```
EXPLAIN
SELECT      D.DeptName, E.EmpNumber, E.LastName, E.FirstName,
            J.JobDesc, P.AreaCode, P.PhoneNumber
FROM        Employee E
INNER JOIN  Department D ON E.DeptNumber = D.DeptNumber
INNER JOIN  EmpPhone P   ON P.EmpNumber = E.EmpNumber
INNER JOIN  Job J        ON E.JobCode = J.JobCode
ORDER BY    1, 2, 3, 4;
```

Lab Exercise 23-2 (cont.)

3. Create the following Global Join Index on the employee table.

```
CREATE JOIN INDEX Employee_JI AS
SELECT      EmpNumber,
            DeptNumber,
            EmpMgrNumber,
            JobCode,
            LastName,
            FirstName,
            SalaryAmount,
            ROWID AS RID
FROM        Employee
PRIMARY INDEX (DeptNumber);
```

4. Re-execute the Explain statement from step #2

What is total estimated time cost for this EXPLAIN? _____

Is the join index used? _____

How much space does the join index require (use the DBC.TableSizeV view)? _____

5. Drop the Employee_JI Join Index.

Lab Exercise 23-2 (cont.)

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Hint: To create a column partitioned join index, you can leverage the following syntax:

To create a column partitioned join index with a PRIMARY AMP option:

```
CREATE JOIN INDEX employee_JI_CPPA AS
SELECT  EmpNumber,
        DeptNumber,
        EmpMgrNumber,
        JobCode,
        LastName,
        FirstName,
        SalaryAmount,
        ROWID AS RID
FROM    Employee
PRIMARY AMP INDEX (DeptNumber) PARTITION BY COLUMN;

/* PRIMARY AMP INDEX or PRIMARY AMP can be used */
```

To create a column partitioned join index with a PRIMARY INDEX option:

```
CREATE JOIN INDEX employee_JI_CPPI AS
SELECT  EmpNumber,
        DeptNumber,
        EmpMgrNumber,
        JobCode,
        LastName,
        FirstName,
        SalaryAmount,
        ROWID AS RID
FROM    Employee
PRIMARY INDEX (DeptNumber) PARTITION BY COLUMN;
```

Lab Exercise 23-2 (cont.)

6. Modify the Create Join Index you used in step #3 and make a Column Partitioned Join index called **Employee_JI_CPPA** with a Primary AMP Index of **DeptNumber** column.
7. Re-execute the Explain statement from step #2.
What is total estimated time cost for this EXPLAIN? _____
What is the size Employee_JI_CPPA join index is using? _____
8. Drop the Employee_JI_CPPA Join Index.
9. Modify the Create Join Index you used in step #6 and make a Column Partitioned Join index called **Employee_JI_CPPI** with a Primary Index of **DeptNumber** column.
10. Re-execute the Explain statement from step #2.
What is total estimated time cost for this EXPLAIN? _____
What is the size Employee_JI_CPPI join index is using? _____
11. Drop the Employee_JI_CPPI Join Index.
12. Which Join Index gave the best tradeoff with regards to size and performance? _____

Notes



Miscellaneous SQL Features

After completing this module, you will be able to:

- **State the purpose and function of the session setting flags.**
- **Recognize differences in transaction modes for Teradata and ANSI.**
- **Distinguish between ANSI and Teradata case sensitivities.**
- **Describe 2 features of the System Calendar.**
- **Describe how space is allocated for volatile and global temporary tables.**
- **Describe what Query Banding is.**

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata SQL	24-4
Teradata SQL and ANSI Differences	24-6
Transactions, Requests, and Statements.....	24-8
Multi-Statement Requests	24-10
SQL Session Modes	24-12
Transaction Modes – Teradata	24-14
Transaction Modes – ANSI.....	24-16
Transaction Mode Examples.....	24-18
Transaction Mode Errors.....	24-20
Transaction Mode Failures.....	24-22
Duplicate Rows	24-24
CASE Sensitivity Issues.....	24-26
Why a System Calendar?	24-28
Calendar Options.....	24-30
Choosing a Calendar	24-32
Calendar View Layout	24-34
One Row in the Calendar	24-36
Using the Calendar	24-38
Temporary Table Choices	24-40
Derived Tables Revisited	24-42
Volatile Tables	24-44
Volatile Table Restrictions.....	24-46
Global Temporary Tables	24-48
Creating Global Temporary Tables.....	24-50
What is Query Banding?	24-52
Setting the QueryBand	24-54
Teradata 14.0 – Major Features	24-56
Teradata 14.10 – Major Features	24-58
Teradata 15.0 – Major Features	24-60
Teradata 15.10 – Major Features	24-62
Teradata 16.00 – Major Features	24-64
Teradata 16.10 – Major Features	24-66
Teradata 16.20 – Major Features	24-68
Teradata Limits (Different Releases)	24-70
Module 24: Review Questions.....	24-72

Teradata SQL

SQL is a standard, open language without corporate ownership. The commercial acceptance of SQL was precipitated by the formation of SQL Standards committees by the American National Standards Institute and the International Standards Organization in 1986 and 1987. Various SQL standards have been released over the years.

- SQL-89 (SQL1)
- SQL-92 (SQL2)
- SQL-1999 (SQL3)
- SQL:2003
- SQL:2006
- SQL:2008
- SQL:2011

The existence of standards is important for the general portability of SQL statements.

Teradata SQL has evolved from a DB2 compatible syntax under V1 to an ANSI compliant syntax under V2 to an ANSI SQL:2011 compatible version. In every case, Teradata has always had its own extensions to the language. Current certification is at entry or core level SQL:2011 with Teradata extensions and some enhanced features implemented.

Teradata, in its historical development, has produced any number of innovative SQL language elements that do not conform to the ANSI SQL standard, a standard that did not exist when those features were conceived. The existing Teradata user base had invested substantial time, effort, and capital into developing applications using that Teradata SQL dialect.

Who is ANSI?

The American National Standards Institute is an administrator and coordinator of voluntary systems of standardization for the United States private sector. About 80 years ago a group of engineering societies and government agencies formed the institute to enhance the “quality of life by promoting and facilitating voluntary consensus standards and conformity.” Today the Institute represents the interests of about 1,000 companies, organizations and government agencies. ANSI does not itself develop standards; rather it facilitates development by establishing consensus among qualified groups.

The American National Standards Institute (ANSI) defines a version of SQL that all vendors of relational database management systems support to a greater or lesser degree. The complete ANSI/ISO SQL:2011 standard is defined across nine individual volumes.

Acronym: NIST – National Institute of Standards and Technology

Teradata SQL

<u>Operating System</u>	<u>Platform</u>	<u>SQL</u>	<u>Compatibilities</u>
TOS	DBC/1012	DBC/SQL	DB2, SQL/DS, ANSI
TOS	3600	Teradata SQL (V1R5.1)	DB2, SQL/DS, ANSI (Outer Join added)
UNIX MP-RAS	5100	Teradata SQL (V2R1)	DB2, SQL/DS, ANSI (Similar to V1R5.1)
UNIX MP-RAS	52xx – 55xx	Teradata SQL (V2R2 to Teradata 16.20)	ANSI (major syntax change)
Windows 2003	52xx – 56xx		
Linux	54xx – 6800		
Linux	IntelliFlex™		

Two levels of ANSI SQL:2011 compliance:

- Core or Entry level
- Enhanced level

Teradata SQL

- Teradata ANSI SQL:2011 compliant at the Core level
- Certified by US Government and NIST
- Includes many of the enhanced features

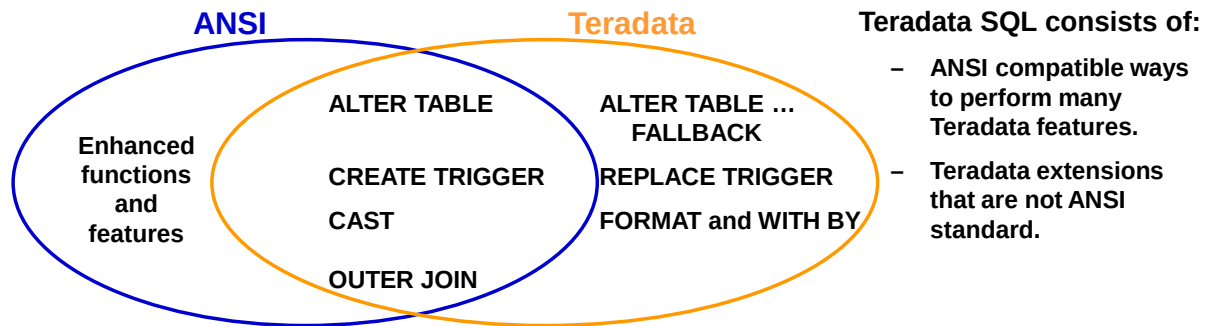
Teradata SQL and ANSI Differences

Teradata SQL meets ANSI SQL:2011 core standards and contains numerous extensions to the SQL:2011 standard. The SQL reference manuals identify those features which are extensions to SQL:2011.

Teradata sessions can operate in one of two modes: ANSI mode and Teradata (BTET) mode. The choice of mode determines the transaction protocol behavior, but also affects such things as case sensitivity defaults, collating sequences, data conversions and display functions. It is important to note that the exact same SQL statement might perform differently in each mode based on these considerations.

Regardless of mode selected, all syntax, whether ANSI compliant or not, is useable. The choice of mode does not inhibit any functionality.

Teradata SQL and ANSI Differences



Teradata allows for sessions to operate in either ...

- **BTET (Teradata) mode**
- **ANSI mode**

All syntax, both ANSI and Teradata extensions, is accepted in either mode.

The same syntax might function differently in each mode.

- Transaction protocol behavior
- CREATE TABLE SET or MULTiset default
- Case sensitivity and collating sequences
- Certain data conversion and display functions

Transactions, Requests, and Statements

The facing page provides details about the difference between transactions, requests, and SQL statements. These terms are often misunderstood and it is important to understand the difference between these terms.

A statement is the most fundamental level in Teradata Database SQL. Statements form the basis of "Requests." A request is often a single statement, but can consist of one or more statements. A request is what is parsed by the Parsing Engine.

The Teradata Database does request processing. People often take for granted the fact that the SQL they write automatically gets committed. There are some drawbacks to this type of processing, however, in a development situation; it seems natural how things work in this manner. There are some benefits to processing SQL in this manner as well. We will discuss the committing of work in more detail, but whatever method you use to commit or rollback work, it helps to know and understand the implications of each.

We will discuss the difference between "implicit" and "explicit" transaction modes in more detail later in this module.

In Teradata mode, a user is usually executing in the "implicit" mode. In this case, a Teradata Database transaction is implicit in nature. Each transaction is a single request that may contain one or more statements. You can also create an implicit transaction using a multi-statement request. All work is committed implicitly and independently of other requests (transactions).

Multi-statement requests act as a single transaction as well. Either the work from all statements gets committed or none of it does.

Transactions – consists of one or more requests

- Can be **IMPLICIT** or **EXPLICIT**
 - ET (Teradata mode) or COMMIT WORK (ANSI mode) will commit the currently active EXPLICIT transaction.
- **A DDL command must be the last request in a transaction.**
- At the end of a transaction, two important activities take place:
 - The Transient Journal (TJ) images are discarded from the WAL log.
 - The Locks are released.

Requests – consists of one or more statements

- Is what gets parsed (syntax checking, resolving, security, optimization)
- Applies the most restrictive locks of all statements in the request (i.e., there is no upgrading of locks within a request).
- May be committed implicitly (i.e., as an Implicit Transaction).
- Is all or nothing (i.e., should any statement error or fail, the request will error or fail).
- **You cannot mix DML and DDL in the same request.**

Statements – is an SQL construct

- requires locks.
- may be sent, by itself, as a request.
- may be one statement in a multi-statement request.

Multi-Statement Requests

A multi-statement DML request is shown on the facing page. A semicolon at the end of a line defines the end of the request. These three UPDATE statements will be executed in parallel.

With SQL Assistant, you can use the Execute Parallel function to also group multiple DML statements into a single request.

As described on the facing page, requests have locks acquired up front in descending TableID order which minimizes the chance of deadlocks if the same request is executed by other users or if other requests using the same tables are executed.

The term **request** is used to refer to any of the following:

- A multi-statement request. Used only with DML (Data Manipulation Language) requests.
- A single statement request. Used with DDL (Data Definition Language) or DML requests.
- A macro. Used with multi-statement or single statement requests, following the above rules. A macro can contain multiple DML statements. A macro can contain a single DDL statement, but not a combination of DML and DDL statements.

The three types of requests above are also considered "implicit transactions" (or "system-generated" transactions). In fact, it is because these requests are transactions that their locks are held until the requests complete.

If locks are placed in separate requests, their order will be defined by the order of the requests. This is not recommended since this order may be different than the order that would be used in a single request. To prevent deadlocks, it is helpful to place all locks at the beginning of a transaction in a single request (especially for database and table-level locks).

Multi-Statement Requests

```
UPDATE Dept          SET Salary_Change_Date = CURRENT_DATE
;UPDATE Manager      SET Salary_Amt = Salary_Amt * 1.06
;UPDATE Employee     SET Salary_Amt = Salary_Amt * 1.04;
```

This is an example of executing 1 request in BTEQ that consists of 3 statements. **This request is effectively parsed as a single request.**

- A semi-colon at the end of a line defines the end of the request (BTEQ convention).
- You cannot mix DDL and DML within a single request.
- This request is considered an “[implicit transaction](#)” if submitted without a BT command.

The 3 table-level write locks (in this example) will be:

- Acquired in TID order.
- Held until done.

Advantage: Minimizes deadlocks at the table level when many users execute requests on the same tables.

This applies for all types of requests:

- Multi-statement requests (as above)
- Single-statement DDL or DML requests
- Macros

SQL Session Modes

A session flag may be set for the transaction mode of the session. A session in Teradata mode will operate with BEGIN and END TRANSACTION protocols while a session in ANSI mode will operate with COMMIT protocol. There are other subtle differences in each mode's treatment of CREATE TABLE defaults, case sensitivity, collation sequences, data conversion and display.

A comparison summary chart follows:

Teradata Mode	ANSI Mode
The default is that a transaction is implicit . Explicit transactions are available using the BT and ET commands.	All transactions are explicit and a COMMIT WORK is required to successfully commit all completed work.
CREATE TABLE – defaults to SET table	CREATE TABLE – defaults to MULTISSET table
Data comparison is NOT case specific.	Data comparison is case specific.
Allows truncation of display data.	Forbids truncation of display data.

Transaction processing, whether in ANSI or Teradata Database BTET modes, imply some application coding logic for error handling. This logic generally works with return codes supplied by the database and makes a determination as to whether-or-not what gets committed and when. This logic resides in a PP2 program, CLI program or in BTEQ, and is necessary for both ANSI and Teradata BTET transaction processing.

The most exclusive locks are retained (read, write, exclusive) at the highest level (row, hash, table). They are released at commit time.

SQL Session Modes

Transaction mode setting

BTET – uses standard Teradata mode (Begin Transaction – End Transaction mode)

ANSI – uses ANSI mode (Commit mode)

BTEQ Examples

.SET SESSION TRANSACTION BTET;

- in Teradata mode, assume implicit transactions unless BT is used
- requires BT to start explicit transaction
- requires ET or ROLLBACK to end explicit transaction

.SET SESSION TRANSACTION ANSI;

- requires COMMIT or ROLLBACK to end transaction

Must be entered prior to LOGON. To change session mode, must LOGOFF first.

Session Mode affects:

- Transaction protocol
- CREATE TABLE defaults
- Default case sensitivities
- Data conversions

Transaction Modes – Teradata

Teradata mode is also referred to as **BTET mode** (Begin Transaction/End Transaction).

In this mode, all individual requests are treated as single implicit transactions.

To aggregate requests into a single transaction requires the BEGIN and END TRANSACTION delimiters.

Transaction Modes – Teradata

.SET SESSION TRANSACTION BTET;

Teradata mode characteristics:

- CREATE TABLE default – SET table
- By default, each request is an implicit transaction ...
 - Successful implicit transactions cannot be rolled-back.
 - For failed requests, the TJ first gets applied, then discarded.
 - Locks are released as part of normal transaction processing.

BT / ET Statements (Explicit transactions)

- BEGIN TRANSACTION (BT) and END TRANSACTION (ET) statements are used to create larger transactions out of individual requests.
 - BT; begins an explicit transaction and
 - ET; commits the currently active transaction
 - COMMIT WORK; is not a valid command in Teradata mode
- Locks are accumulated following a BT until an ET is issued.
- A DDL statement must be the last statement before an ET.
- A rollback of the active transaction occurs when any of the following occur:
 - ROLLBACK WORK; (explicit rollback command)
 - SQL statement failure or error
 - Session is aborted
 - logging off before issuing the ET;

Transaction Modes – ANSI

ANSI mode may also be referred to as **COMMIT mode**. ANSI mode automatically aggregates requests until an explicit COMMIT command is encountered. Thus, all transactions in ANSI mode are by definition explicit.

When the session performing a macro is in ANSI mode, the actions of the macro are uncommitted until a commit or rollback occurs in subsequent statements unless the macro body ends with a COMMIT statement.

Note that all DDL statements must be immediately delimited by a COMMIT and also that macros containing DDL must contain only a single DDL statement and must also be followed by an immediate commit.

If a macro contains a data definition statement, it can include a COMMIT, but cannot contain other DML requests.

Transaction Modes – ANSI

```
.SET SESSION TRANSACTION ANSI;
```

ANSI mode characteristics:

- CREATE TABLE default – MULTiset table
- Transactions are by definition explicit.
- A transaction is committed only by an explicit COMMIT.
 - COMMIT WORK; will commit the currently active transaction.
 - BT; and ET; commands are not valid commands in ANSI mode.
- Statement following a COMMIT automatically starts a new transaction.
- A DDL statement must be the last statement before a COMMIT.
- Locks are accumulated until a COMMIT is issued.
- A rollback of the active transaction occurs when any of the following occur:
 - ROLLBACK WORK; (explicit rollback command)
 - SQL statement error - rollback current request (not the entire transaction)
 - SQL statement failure - rolls back the transaction and releases the locks
 - Session aborted
 - logging off before issuing the COMMIT;

Transaction Mode Examples

The facing page shows the various permutations of transaction modes and the expected results from success, rollbacks, and logoffs. Some additional examples are shown below.

Examples in Teradata mode:

```
/* 1st example - goal is to populate tableX and create tableZ which does
   not exist, both commands will execute successfully in this example*/
```

```
BT;
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE tableZ (col1 INTEGER, col2 INTEGER);
ET;
```

```
/* 2nd example - goal is to populate tableX and create tableZ, but tableZ
   cannot be created because it already exists */
```

```
BT;
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE tableZ (col1 INTEGER, col2 INTEGER);
```

```
*** Failure 3803 Table 'tableZ' already exists.
```

```
/* both commands are rolled back - tableX is not populated and tableZ
   is not created because it already exists */
```

Examples in ANSI mode:

```
/* 3rd example - goal is to populate tableX and create tableZ which does
   not exist, both commands will execute successfully in this example */
```

```
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE tableZ (col1 INTEGER, col2 INTEGER);
COMMIT;
```

```
/* 4th example - goal is to populate tableX and create tableZ, but tableZ
   cannot be created because it already exists */
```

```
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE tableZ (col1 INTEGER, col2 INTEGER);
```

```
*** Failure 3803 Table 'tableZ' already exists.
```

```
/* both commands are rolled back - tableX is not populated and tableZ
   is not created because it already exists */
```

Transaction Mode Examples

<u>Teradata Mode</u> (implicit)	<u>Teradata Mode</u> (explicit)	<u>ANSI Mode</u>
UPDATE tableA ... ; (tableA commits) UPDATE tableB ... ; (tableB commits)	BT; UPDATE tableA ... ; UPDATE tableB ... ; ET; (Both commit)	UPDATE tableA ... ; UPDATE tableB ... ; COMMIT; (Both commit)
(No explicit ROLLBACK with implicit transactions)	BT; UPDATE tableA ... ; UPDATE tableB ... ; ROLLBACK; (Both rollback)	UPDATE tableA ... ; UPDATE tableB ... ; ROLLBACK; (Both rollback)
UPDATE tableA ... ; (tableA commits) UPDATE tableB ... ; (tableB commits) LOGOFF;	BT; UPDATE tableA ... ; UPDATE tableB ... ; LOGOFF; (Both rollback)	UPDATE tableA ... ; UPDATE tableB ... ; LOGOFF; (Both rollback)

Transaction Mode Errors

The facing page shows the various examples of transaction modes and the expected results when SQL **errors** are encountered in a transaction.

Examples in Teradata mode (an error is considered a failure in this mode):

```
BT;
INSERT INTO tableX SELECT * FROM tableY;
CREATE SET TABLE tableZ (col1 INTEGER, col2);

*** Failure 3739 The user must give a data type for 'col2'.

/* this transaction is automatically aborted and rolled back - tableX is
   not populated and tableZ is not created */

BT;
INSERT INTO tableX VALUES (1, 1);
INSERT INTO tableY VALUES (2, 2);

*** Failure 2802 Duplicate row error in tableY.

/* this transaction is automatically aborted and rolled back - tableX is
   not populated with a new row */
```

Examples in ANSI mode:

```
INSERT INTO tableX SELECT * FROM tableY;
CREATE SET TABLE tableZ (col1 INTEGER, col2);

*** Error 3739 The user must give a data type for 'col2'.

COMMIT; /* the INSERT into tableX is committed; a syntax error does not
         cause an abort of the transaction */
Or

ROLLBACK; /* the INSERT is rolled back */

INSERT INTO tableX VALUES (1, 1);
INSERT INTO tableY VALUES (2, 2);

*** Error 2802 Duplicate row error in tableY.

/* this transaction is not aborted and is not rolled back */

COMMIT; /* the INSERT into tableX is committed; a duplicate row error
         does not cause an abort of the transaction */
Or

ROLLBACK; /* the INSERT for tableX is rolled back */
```

Transaction Mode Errors

Teradata Mode (explicit)

```
BT;
UPDATE tableA ... ;
SELECT * FRM tableA WHERE colx = 123;
```

*** [Failure 3706](#) Syntax error: SELECT * must have a FROM clause.

A syntax error is a failure in Teradata mode.

(the update for tableA is rolled back)

```
BT;
UPDATE tableA ... ;
SELECT * FROM tableA WHERE coxl = 123;
```

*** [Failure 5628](#) Column coxl not found in tableA.

This error is a failure in Teradata mode.

(the update for tableA is rolled back)

ANSI Mode

```
UPDATE tableA ... ;
SELECT * FRM tableA WHERE colx = 123;
```

*** [Error 3706](#) Syntax error: SELECT * must have a FROM clause.

A syntax error is NOT a failure in ANSI mode.

```
COMMIT;
(the update for tableA is committed)
```

```
UPDATE tableA ... ;
SELECT * FROM A WHERE coxl = 123;
```

*** [Error 5628](#) Column coxl not found in tableA.

A Resolver error may NOT be a failure in ANSI mode (see another example on following slide).

```
COMMIT;
(the update for tableA is committed)
```

Transaction Mode Failures

The facing page shows the various examples of transaction modes and the expected results when SQL **failures** are encountered in a transaction.

Examples in Teradata mode:

```
BT;
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE tableZ (col1 INTGER);

*** Failure 3803 Table 'tableZ' already exists.

/* this transaction is automatically aborted and rolled back - tableX is
   not populated */

BT;
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE DBC.tableZ (col1 INTEGER);

*** Failure 3524 The user does not have CREATE TABLE access to database
   DBC.

/* this transaction is automatically aborted and rolled back - tableX is
   not populated and tableZ is not created */
```

Examples in ANSI mode:

```
INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE tableZ (col1 INTGER);

*** Failure 3803 Table 'tableZ' already exists.

/* this transaction is automatically aborted and rolled back - tableX is
   not populated */

INSERT INTO tableX SELECT * FROM tableY;
CREATE TABLE DBC.tableZ (col1 INTEGER);

*** Failure 3524 The user does not have CREATE TABLE access to database
   DBC.

/* this transaction is automatically aborted and rolled back - tableX is
   not populated and tableZ is not created */
```

Transaction Mode Failures

Teradata Mode (explicit)

```
BT;
UPDATE tableA ... ;
UPDATE tableB ... ;
CREATE TABLE tableC
  (colx INTEGER, coly INTEGER);
```

*** [Failure 3803](#) Table 'tableC' already exists.

This is a failure in Teradata mode and the transaction is rolled back.

(both updates for tableA and tableB rollback)

Teradata Mode Summary:

Error Behavior – errors not issued, only warnings, failures, or successes. A failure rolls back the entire transaction and releases the locks.

Failure Behavior – rolls back the transaction (whether implicit or explicit) and releases all locks.

ANSI Mode

```
UPDATE tableA ... ;
UPDATE tableB ... ;
CREATE TABLE tableC
  (colx INTEGER, coly INTEGER);
```

*** [Failure 3803](#) Table 'tableC' already exists.

This Resolver error is a failure in ANSI mode and the transaction is rolled back.

(both updates for tableA and tableB rollback)

ANSI Mode Summary:

Error Behavior – errors roll back only the request that causes them, not the entire transaction.

Failure Behavior – rolls back the transaction and releases all locks.

Duplicate Rows

A **duplicate row** is a row of a table whose column values are all identical to another row of the same table. If a designer adheres to the rule that a Primary Key must be unique, then it should preclude the possibility of having duplicate rows.

Having said that, the ANSI standard permits duplicate rows in order to satisfy the requirements of certain vendors who rely on them for certain types of auditing systems. For example, if I am loading a table from several different databases and the same record appears from three different places, I might want to know that it originated from those three places.

Even though this contradicts relational theory, the standard generously permits duplicate rows for these anomalous situations.

Teradata, adhering to the ANSI standard, permits duplicate rows by specifying that you wish to create a **MULTISET** table. In Teradata transaction mode, the default, however, is a **SET** table that does not permit duplicate rows.

When MULTISET is enabled, Teradata does not do a duplicate row check for new rows added.

If the table is a SET table, it will only do a duplicate row check if the Primary Index is a NUPI and there are no other unique indexes on the table. If a unique index exists on the table, duplicate index check itself will suffice to ensure there are no duplicate rows.

Also, if MULTISET is enabled, it will be overridden by choosing a UPI as the Primary Index or by having a unique index (e.g., unique secondary) on another column(s) on the table. Doing this effectively disables the MULTISET.

As discussed previously, if you insert a duplicate row into a SET table in **Teradata explicit transaction mode**, this is considered a "failure" and the transaction is aborted and rolled back.

```
*** Failure 2802 Duplicate row error in tableY.
```

As discussed previously, if you insert a duplicate row into a SET table in **ANSI transaction mode**, this is considered an "error" and the transaction is not aborted and is not rolled back.

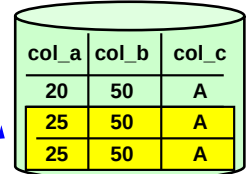
```
*** Error 2802 Duplicate row error in tableY.
```


Duplicate Rows

Only MULTiset tables can possibly have duplicate rows.

A duplicate row is a row of a table whose column values are all identical to another row in the same table.

Duplicate Rows



col_a	col_b	col_c
20	50	A
25	50	A
25	50	A

- Ideally a relational table should not have duplicate rows because a PK uniquely identifies each row!
- However, you can specify whether your table will or will not allow them – SET or MULTiset.
- If a MULTiset table has a unique index, uniqueness takes precedence and the table will not have duplicate rows.
- The ANSI standard, however, permits duplicate rows for specialized situations, thus Teradata permits them as well. NO PRIMARY INDEX tables are MULTiset tables.

Teradata default

```
CREATE SET TABLE table_A
      :
      :
```

Checks for * and disallows duplicate rows.

ANSI default

```
CREATE MULTiset TABLE table_B
      :
      :
```

Doesn't check for and allows duplicate rows.

* Note: If a UPI is selected on a SET table, the duplicate row check is replaced by a check for duplicate index values.

CASE Sensitivity Issues

Teradata Mode

In **Teradata mode**, data is stored as entered unless an UPPERCASE attribute is specified for the column.

The default for character comparisons is NOT CASESPECIFIC unless either the CASESPECIFIC or UPPER/LOWER operators are specified as part of the comparison criteria.

ANSI Mode

ANSI mode always stores data as entered. The default mode for data comparison is always CASESPECIFIC unless the UPPER/LOWER operator is used as part of the comparison criteria.

Note the use of CASESPECIFIC and NOT CASESPECIFIC operators are non-ANSI compliant syntax.

Using ANSI Blind Test

Since the CASESPECIFIC and NOT CASESPECIFIC operators (or as options in the CREATE TABLE statement) are not ANSI compliant, ANSI provides the UPPER operator as a means for doing a “case-blind” comparison of characters. Using this technique will allow a script to function compatibly in either ANSI or Teradata mode.

Teradata Mode

```
SELECT  firstname, lastname
FROM    Employee
WHERE   lastname LIKE  '%Ra%';
```

firstname	lastname
Robert	Crane
James	Trader
I.B.	Trainer
Larry	Ratzlaff
Peter	Rabbit

ANSI Mode

```
SELECT  firstname, lastname
FROM    Employee
WHERE   UPPER(lastname)
LIKE    UPPER('%Ra%');
```

firstname	lastname
Robert	Crane
James	Trader
I.B.	Trainer
Larry	Ratzlaff
Peter	Rabbit

CASE Sensitivity Issues

Column Attributes	Teradata Mode	ANSI Mode
Storage	As entered (default) UPPERCASE	None (As entered is default)
Comparisons	UPPER, LOWER CS (CASESPECIFIC) NOT CS (NOT CASESPECIFIC is Teradata Default)	UPPER, LOWER (CASESPECIFIC is ANSI default)

Teradata Mode – Default is NOT CS

```
SELECT  firstname, lastname
FROM    Employee
WHERE   lastname LIKE  '%Ra%';
```

<u>firstname</u>	<u>lastname</u>
Robert	Crane
James	Trader
I.B.	Trainer
Larry	Ratzlaff
Peter	Rabbit

ANSI Mode – Default is CASESPECIFIC

```
SELECT  firstname, lastname
FROM    Employee
WHERE   lastname LIKE  '%Ra%';
```

<u>firstname</u>	<u>lastname</u>
Larry	Ratzlaff
Peter	Rabbit

ANSI Blind Test – an example of executing a “non case specific” compare in ANSI mode is provided on the facing page.

Why a System Calendar?

Structured Query Language (SQL) permits a certain amount of mathematical manipulation of dates, however the needs of real world applications often exceed this innate capability. Implementing a system calendar is often necessary to answer time-relative business questions. Summarizing information based on a quarter of the year or on a specific day of the week can be onerous without the assistance of a system calendar.

As implemented for Teradata, the **System Calendar** is a high-performance set of nested views which, when executed, materialize date information as a dimension in a star schema. The system calendar is easily joined to other tables to produce information based on any type of time period or time boundary.

The underlying base table consists of one row for each day within the range of Jan 1, 1900 through Dec. 31, 2100. There is only one column, a date, in each row. Each level of view built on top of the base table adds intelligence to the date.

Why A System Calendar?

The Truth Is ...

SQL has limited ability to do date arithmetic.
There is a need for more complex, calendar-based calculations.

I'd Like To Know ...

How does this quarter compare to same quarter last year?
How many shoes do we sell on Sundays vs. Saturdays?
During which week of the month do we sell the most pizzas?

Some Good News

Extends properties of DATE data type by joining to Calendar.
Easily joined to other tables, i.e., dimension of a star schema.
High performance – limited I/O.
Has advantages over user-defined calendars.

Standard Usage

Statistics are created for materialized table for join planning.
Only necessary rows are materialized for the calendar.

Calendar Options

The views and the base table that make up the system calendars are contained in a database called “Sys_Calendar”. There are 3 system calendars available in Sys_Calendar database:

Teradata:

The first full week of the year starts on Sunday and the days of the year before Sunday belong to Week 0. For example, if the year starts on January 1, 2019 (a Tuesday) then Week 0 is from January 1 to January 5, 2019. Week 1 begins on Sunday, January 6, 2019.

ISO:

This calendar follows the ISO and European standard. The week begins on Monday, and the first week of the year is the first week that has at least 4 days. If a week has fewer than 4 days, it belongs to the last week of the previous year. There are no partial weeks. For example, if the year starts on January 1, 2019 (a Tuesday) and the week start is Monday, week 1 of 2019 is from December 31, 2018, to January 6, 2019.

Compatible:

This calendar is Oracle-compatible. The first full week of a year begins on January 1, regardless of what day of the week that is, and there can be partial weeks with 1 day (for most years) or 2 days (for leap years) at the end of the year. For example, if January 1, 2019, is a Tuesday, the first week of the year is from Tuesday, January 1, 2019, through Monday, January 7, 2019.

Calendar Options

There are 3 system calendars available in the Sys_Calendar database:

Teradata

The first full week of the year starts on Sunday and the days of the year before Sunday belong to Week 0.

ISO

This calendar follows the ISO and European standard. The week begins on Monday, and the first week of the year is the first week that has at least 4 days.

If a week has fewer than 4 days, it belongs to the last week of the previous year. There are no partial weeks.

Compatible

This calendar is Oracle-compatible. The first full week of a year begins on January 1, regardless of what day of the week that is, and there can be partial weeks with 1 day (for most years) or 2 days (for leap years) at the end of the year.

January 2019						
S	M	TU	W	TH	F	S
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Choosing a Calendar

The default session calendar is Teradata. Each calendar defaults to all business days. You can configure calendar patterns and exceptions using macros.

Use the `DBC.CreateBusinessCalendarPattern` macro to designate days as working or non-working. Specify the calendar name, the day name (for example, Tuesday), a pattern as either ON (for a working day) or OFF (for a non-working day), and, optionally, a comment explaining the meaning of ON or OFF for that day.

Example 1

The following modifies the Teradata business calendar by designating SUNDAY and MONDAY as non-working days:

```
EXEC DBC.CreateBusinessCalendarPattern
('TERADATA', 'SUNDAY', 'OFF', 'Non-work Day');
```

```
EXEC DBC.CreateBusinessCalendarPattern
('TERADATA', 'MONDAY', 'OFF', 'Non-work Day');
```

Example 2

The following modifies the ISO business calendar by designating SUNDAY and MONDAY as working days:

```
EXEC DBC.CreateBusinessCalendarPattern
('ISO', 'SUNDAY', 'ON', 'Work Day');
```

```
EXEC DBC.CreateBusinessCalendarPattern
('ISO', 'MONDAY', 'ON', 'Work Day');
```

Use the `DBC.CreateException` macro to define an exception. Specify the calendar name, a pattern exception as either ON (for a working day) or OFF (for a non-working day), a date, and optionally, a comment explaining the reason for the exception.

Note: If the exception already exists for a date, the system returns an error; in that case, delete the exception and recreate it.

Example 3

In the following, two exceptions are created for the Teradata business calendar: the first designates March 3, 2008, as a non-business day, and the second designates June 7, 2008, as a business day.

```
EXEC DBC.CreateException
('Teradata', 'OFF', DATE '2008-03-03', 'Holiday');
```

```
EXEC DBC.CreateException
('Teradata', 'ON', DATE '2008-06-07', 'Holiday make up day');
```


Choosing a Calendar

To use the different calendars, use **SET SESSION**:

```
SET SESSION CALENDAR = TERADATA;  
SELECT day_of_week FROM Sys_Calendar.CALENDAR  
WHERE calendar_date = DATE '2019-01-01';
```

day_of_week
3

```
SET SESSION CALENDAR = ISO;  
SELECT day_of_week FROM Sys_Calendar.CALENDAR  
WHERE calendar_date = DATE '2019-01-01';
```

day_of_week
2

```
SET SESSION CALENDAR = COMPATIBLE;  
SELECT day_of_week FROM Sys_Calendar.CALENDAR  
WHERE calendar_date = DATE '2019-01-01';
```

day_of_week
1

Note: With the Compatible calendar, since Jan 1, 2019 was on a Tuesday, this is the first day of the year for this calendar.

January 2019						
S	M	TU	W	TH	F	S
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Note:

The system default is the Teradata calendar.

Calendar View Layout

The views and the base table that make up the system calendar are contained in a database called 'Sys_Calendar'. The contents of this database are easily seen with the help of the HELP DATABASE command.

The base table for the system calendar contains a row for each date between Jan 1, 1900 through Dec 31, 2100. Each row contains a single column that is a DATE data type. This is demonstrated using the SHOW TABLE command.

```
SHOW TABLE Sys_Calendar.Caldates;
```

```
*** Text of DDL statement returned.
```

```
*** Total elapsed time was 1 second.
```

```
-----  
CREATE SET TABLE Sys_Calendar.Caldates ,FALLBACK ,  
    NO BEFORE JOURNAL,  
    NO AFTER JOURNAL,  
    CHECKSUM = DEFAULT,  
    DEFAULT MERGEBLOCKRATIO  
    (  
        cdate DATE FORMAT 'YY/MM/DD')  
    UNIQUE PRIMARY INDEX (cdate);
```

Calendar View Layout

Columns from the System Calendar:

calendar_date DATE UNIQUE (Standard Teradata date)
day_of_week BYTEINT, (1-7)
day_of_month BYTEINT, (1-31)
day_of_year SMALLINT, (1-366)
day_of_calendar INTEGER, (Julian days since 01/01/1900)
weekday_of_month BYTEINT, (nth occurrence of day in month)
week_of_month BYTEINT,
week_of_year BYTEINT, (0-53)
week_of_calendar INTEGER, (0-n)
month_of_quarter BYTEINT, (1-3)
month_of_year BYTEINT, (1-12)
month_of_calendar INTEGER, (1-n) (Starting Jan, 1900)
quarter_of_year BYTEINT, (1-4)
quarter_of_calendar INTEGER, (Starting Q1, 1900)
year_of_calendar SMALLINT, (Starting 1900)

System Calendar is a 4-level nested view of dates.

Underlying table is Sys_calendar.Caldates:

- Has one column called 'cdate' - DATE data type.
- Has one row for each date of calendar.
- Unique Primary Index is cdate.
- Each level of view adds intelligence to date.

Using the Business Calendar functions, you can add holidays to meet specific needs.

Note:

System calendar includes
Jan 1, 1900 through
Dec. 31, 2100.

One Row in the Calendar

Four Levels of Calendar Views

Calendar	<u>View which views Calendartmp</u>
Calendartmp	<u>View which adds:</u> day_of_week weekday_of_month week_of_month week_of_year week_of_calendar month_of_quarter month_of_calendar quarter_of_year quarter_of_calendar
Calbasics	<u>View which adds:</u> calendar_date, day_of_calendar, day_of_month, day_of_year, month_of_year, year_of_calendar)
Caldates	<u>Underlying table which contains a date column:</u>

One Row in the Calendar

```
SELECT * FROM Sys_Calendar.Calendar WHERE calendar_date = '2019-01-01';
```

calendar_date	1/1/2019
day_of_week	3
day_of_month	1
day_of_year	121
day_of_calendar	43465
weekday_of_month	1
week_of_month	0
week_of_year	0
week_of_calendar	6209
month_of_quarter	1
month_of_year	1
month_of_calendar	1429
quarter_of_year	1
quarter_of_calendar	477
year_of_calendar	2019

January 2019						
S	M	TU	W	TH	F	S
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Note: SELECT CURRENT_DATE is the ANSI standard equivalent of SELECT DATE.

Using the Calendar

The daily sales table is used in the following example:

```
CREATE SET TABLE Daily_Sales,  
  NO FALLBACK,  
  NO BEFORE JOURNAL,  
  NO AFTER JOURNAL,  
  CHECKSUM = DEFAULT,  
  DEFAULT MERGEBLOCKRATIO  
  ( itemid          INTEGER,  
    salesdate       DATE FORMAT 'YYYY-MM-DD',  
    sales           DECIMAL(9,2) )  
PRIMARY INDEX (itemid);
```

The following is a non-ANSI standard way of performing the join on the facing page.

```
SELECT    DS.itemid  
          ,SUM(DS.sales)  
FROM      daily_sales      DS,  
          Sys_Calendar.Calendar SC  
WHERE     DS.salesdate = SC.calendar_date  
AND       SC.quarter_of_year = 1  
AND       SC.year_of_calendar = 2019  
AND       DS.itemid = 10  
GROUP BY 1;
```

Using the Calendar

Show total sales of item 10 reported in Q1 of 2019.

Daily_Sales table

Item_id
salesdate
sales

Sys_Calendar.Calendar

calendar_date
:
:
quarter_of_year
:
year-of_calendar

Join

SQL:

```
SELECT      DS.itemid, SUM(DS.sales)
FROM        Daily_Sales      DS
INNER JOIN  Sys_Calendar.Calendar  SC
ON          DS.salesdate = SC.calendar_date
AND        SC.quarter_of_year = 1
AND        SC.year_of_calendar = 2019
AND        DS.itemid = 10
GROUP BY    1;
```

Result:

itemid	Sum(sales)
10	4050.00

Salesdate is joined to the system calendar.
Calendar determines if this date meets this criteria:

Is it a Quarter 1 date?

Is it a 2019 date?

If yes on both, add this sales amount to result.

Temporary Table Choices

Generically speaking, there are three types of temporary tables now available with Teradata, any one of which will have advantages over traditional temporary table creation.

Derived tables were incorporated into Teradata under V2R2. Derived tables are always local to a specific query, as they are built with code within the query. The rows of the derived table are stored in spool and discarded when the query finishes. The data dictionary has no knowledge of derived tables.

Volatile Temporary tables (or **Volatile Tables**) are local to a session rather than a specific query, which means that the table may be used repeatedly within a session. Once the session ends, the volatile table is automatically discarded if it has not already been manually discarded. The data dictionary has no knowledge of volatile tables. Space for a volatile table comes from the user's Spool space.

Global Temporary tables (or **Temporary Tables**) are local to a session just as are volatile tables. Unlike volatile tables, global temporary tables are known by the data dictionary where a permanent definition is kept. Global temporary tables are materialized within a session, and then discarded when the session ends. Space for a global temporary table comes from the user's Temporary space.

In this module, we will be looking at the three types of temporary tables, how their implementations differ and when to use each.

Temporary Table Choices

Derived Tables

- Local to the query (table and columns are named within query)
- Incorporated into SQL query syntax (populated in query via SELECT in FROM)
- **Materialized in SPOOL** – Spool rows are discarded when query finishes
- No data dictionary involvement
- Commonly used with aggregation

Volatile Tables

- Local to a session – **uses SPOOL space**
- Uses CREATE VOLATILE TABLE syntax
- Discarded automatically at session end
- No data dictionary involvement

(Global) Temporary Tables

- Local to a session – **uses TEMPORARY space**
- Uses CREATE GLOBAL TEMPORARY TABLE syntax
- Materialized instance of table discarded at session end
- Creates and keeps table definition in data dictionary

Derived Tables Revisited

Derived tables were introduced into Teradata under V2R2. The creation of the derived table is local to the query. A query may have multiple derived tables. These tables may be joined or manipulated much as any other table would be.

The OLAP functions of SQL do not support standard aggregation functions due to their conflicting uses of the GROUP BY clause. This fact makes the OLAP functions excellent candidates for the use of derived tables, in particular when the requirement is to perform a statistical function on an aggregation.

We see in the facing page example that to find the top three selling items across all stores, we must first aggregate the sales by product-id using a derived table. Once we have this aggregation done in spool, we may apply the RANK function to answer the question.

Derived tables are useful, but only exist for the duration of the query. They are not a practical solution if the result is to be used in many follow-on queries. In this case, other types of temporary tables will be more appropriate.

Derived Tables Revisited

Get top three selling items across all stores:

```
SELECT      Prodid, Sumsales, RANK(sumsales) AS "Rank"  
FROM        (SELECT prodid, sum(sales) FROM Salestbl GROUP BY 1)  
              AS tmp (prodid, sumsales)  
QUALIFY RANK (sumsales) <= 3;
```

Result:

Prodid	Sumsales	Rank
A	170000.00	1
C	115000.00	2
D	110000.00	3

- Derived table name is "tmp".
 - The table is required for this query but no others.
 - The query will be run only one time with this data.
- Derived column names are "prodid" and "sumsales".
- Table is created in spool using the inner SELECT.
- SELECT statement is always in parenthesis following "FROM".

Volatile Tables

Volatile tables have much in common with derived tables. They are materialized in spool and are unknown to the data dictionary. Unlike derived tables, volatile tables may be used repeatedly throughout a session. They may be dropped at any time manually or automatically at the session end.

Volatile tables require their own **CREATE** syntax. The table definition is kept in cache and not permanently written to disk. Volatile tables do not survive a system restart.

The **LOG** option indicates the desire for standard transaction logging of “before images” into the transient journal.

The **ON COMMIT DELETE ROWS** option specifies that at the end of a transaction, the table rows should be deleted. While this might seem a bit unusual, it is the default required by the ANSI standard. It may be appropriate in situations where a table is materialized only to produce an aggregation and the table rows are not needed beyond that purpose. (Typically this would occur in a multi-statement transaction.)

The **ON COMMIT PRESERVE ROWS** option provides the more normal situation where the table rows are kept following the end of the transaction.

Secondary Indexes and Volatile Tables

When initially creating a Volatile table, you can define secondary indexes as part of the **CREATE VOLATILE TABLE** statement.

You cannot add secondary indexes (via **CREATE INDEX**) to a Volatile table after it has been created. Secondary indexes have to be specified with the initial **CREATE VOLATILE TABLE** statement.

You cannot create a join index or a hash index on a **VOLATILE** table.

Volatile Tables

*Similar to
derived tables:*

- Materialized in spool
- No Data Dictionary access or transaction locks
- Table definition kept in cache
- Designed for optimal performance

*Different from
derived tables:*

- Is local to the session, not the query
- Can be used with multiple queries in the session
- Dropped manually anytime or automatically at session end
- Requires CREATE VOLATILE TABLE statement

Example:

```
CREATE VOLATILE TABLE DeptSal_VT
, LOG
  (deptno      SMALLINT
   ,avgsal      DEC(9,2)
   ,maxsal      DEC(9,2)
   ,minsal      DEC(9,2)
   ,sumsal      DEC(9,2)
   ,empcnt      SMALLINT)
ON COMMIT PRESERVE ROWS;
```

CREATE Considerations:

- LOG indicates that a transaction journal is maintained.
- NO LOG allows for better performance.
- PRESERVE ROWS indicates keep table rows at TXN end.
- DELETE ROWS indicates delete all table rows at TXN end.
- Volatile tables do not survive a system restart.

Volatile Table Restrictions

Volatile tables must have names that are unique within the user's working database. Even though volatile tables are not known to the data dictionary, if names duplicating dictionary names were allowed, the system would not understand where to locate the requested named object if it could be found in two places.

Up to 1000 volatile tables are allowed on a single session. They must all have unique names. They also must be qualified by the User ID of the session, either explicitly or implicitly. A volatile table cannot belong to a database or a user; it can only belong to a user's session.

While **FALLBACK** is a selectable option, its value is limited for volatile tables. Because they cannot survive a system restart, making a table FALLBACK will not keep a table available following a restart. The only reason to make a volatile table FALLBACK would be to allow creation of the table in the event of a down AMP.

None of the following options are permitted with volatile tables:

- Permanent Journaling
- Referential Integrity
- CHECK constraints
- Column compression
- Column default values
- Column titles
- Named indexes

Volatile Table Restrictions

Restrictions:

- Up to 1000 volatile tables are allowed on a single session.
- Each must have a unique name.
- Volatile tables are always qualified by the session's userid.
- Secondary indexes are allowed on when VT is initially created
- Cannot be created with join and/or hash indexes.

Examples:

```
CREATE VOLATILE TABLE username.table1 ...      (Explicit)
CREATE VOLATILE TABLE table1 ...                (Implicit)
CREATE VOLATILE TABLE databasename.table1 ... (Error)
```

Multiple Sessions:

- Each session can use the same VT name (local to session).
- VT name cannot duplicate existing object name for this user.
 - Perm or Temp names, View names, etc.

FALLBACK:

Electable but not often useful for VTs. VTs don't survive a system reset.

Options not permitted:

- | | | |
|-------------------------|-------------------------|-----------------|
| • Permanent Journaling | • Referential Integrity | • Named Indexes |
| • CHECK constraints | • Column compression | |
| • Column default values | • Column titles | |

Global Temporary Tables

Global Temporary Tables (also called **Temporary Tables**), unlike volatile and derived tables, have definitions stored in the Data Dictionary. The table itself is materialized by the first SQL DML statement that accesses the table, typically an INSERT SELECT or an INSERT.

Like volatile tables, global temporary tables are local to a session. The materialized instance of the table is not shareable with other sessions. The table instance may be dropped explicitly or it will be automatically dropped at the end of the session. The definition remains in the dictionary for future materialized instances of the table. The base definition may be dropped with an explicit DROP command.

The only privilege required by the user is the DML privilege necessary to materialize the table. Once materialized, no privileges are checked.

A special type of space called “**temporary space**” is used for global temporary tables. Like perm space, temporary space is sustained during a system restart. Global temporary tables are thus able to survive a system restart.

Up to 2000 materialized instances of global temporary tables may exist for a given session.

Two key reasons for the users of global temporary tables are

1. To simplify application code
2. Reduce the a large number of joins for specific tables

Note:

It is common to refer to volatile temporary tables as “**volatile tables**” and refer to global temporary tables simply as “**temporary tables**”.

Secondary Indexes and Global Temporary Tables

When initially creating a Global Temporary table, you can define secondary indexes as part of the CREATE GLOBAL TEMPORARY TABLE statement.

You can also add secondary indexes (via CREATE INDEX) to a Global Temporary table definition ONLY if it is NOT materialized.

- If a Global Temporary table is materialized by any user in the system, you cannot add additional secondary indexes to the Global Temporary table.

You cannot create a join index or a hash index on a Global Temporary table.

Global Temporary Tables

Global Temporary Tables

Are created using CREATE GLOBAL TEMPORARY command.
Require a base definition which is stored in the DD.
Are materialized by first SQL DML statement to access table.

Similarities to Volatile Tables:

Each instance of global temp table is local to a session.
Materialized tables are dropped automatically at session end.
Have LOG and ON COMMIT PRESERVE/DELETE options.
Materialized table contents aren't sharable with other sessions.
Does not support join and/or hash indexes.

Differences from Volatile Tables

Base definition is permanent and kept in DD.
Requires DML privileges necessary to materialize the table.
Space is charged against an allocation of "temporary space" -
CREATE USER TEMPORARY parameter.
User can materialize up to 2000 global tables per session.
Secondary indexes (CREATE INDEX) can be added to a Global
Temporary Table as long it has not been materialized.
Tables can survive a system restart.

Creating Global Temporary Tables

Temporary tables are created using the CREATE GLOBAL TEMPORARY TABLE command. This stores the base definition of the table in the data dictionary. Like volatile tables, the defaults are to LOG transactions and ON COMMIT DELETE ROWS.

Temporary tables may be altered by the ALTER command to change any attributes of the table, similar to perm tables.

Once the table is accessed by a DML command, such as the INSERT SELECT seen on the facing page, the table is considered materialized and a row is entered into a dictionary view called DBC.Temptables.

Deleting all rows from a temporary table does not de-materialize the table. The instance of the table must be dropped or the session must be ended for the materialized table to be discarded.

Creating Global Temporary Tables

```
CREATE GLOBAL TEMPORARY TABLE DeptSal_GT  
(deptno      SMALLINT  
 ,avgsal      DEC(9,2)  
 ,maxsal      DEC(9,2)  
 ,minsal      DEC(9,2)  
 ,sumsal      DEC(9,2)  
 ,empcnt      SMALLINT);
```

*Base table definition stored in DD/D
Default is ON COMMIT DELETE ROWS*

```
ALTER TABLE DeptSal_GT,  
ON COMMIT PRESERVE ROWS;
```

*ALTER TABLE can be used to change
defaults.*

```
INSERT INTO DeptSal_GT  
SELECT  deptno  
        ,AVG(sal)  
        ,MAX(sal)  
        ,MIN(sal)  
        ,SUM(sal)  
        ,COUNT(emp)  
FROM    Employee  
GROUP BY 1;
```

*Table is now materialized.
Row is inserted in DBC.Temptables.*

```
DELETE FROM DeptSal_GT;
```

*Table remains materialized until the rows
are deleted in it.*

What is Query Banding?

Query Banding provides for a means to set Name/Value pairs across individual Database connections at a Session or Transaction level in order to provide the database with significant additional information about the originating source of a connection, beyond the limited level available in Logon Source information that exists between Applications and the Database (Logon Source only provides basic information about the connection such as Client IP Address, Server User ID, Application Executable ID etc.). Query Banding, however, provides for a much richer set of Application specific information that is completely separate from the Logon Source and which can be expanded upon by applications as required.

Query Banding provides a mechanism for Applications to collaborate with the underlying Teradata Database in order to provide for better Workload Management, Prioritization and Accounting. It can also provide for improvements in Supportability, Logging, Security and Notification. Query Banding provides the capability to wrap metadata around a query (i.e. band the query with metadata).

The origin of a request is not always easy to determine especially when applications use connection pooling. Setting client identifiers in the queryband can aid the DBA's to determine the source of requests that are blocking or consuming large amounts of system resources.

Another use of Query Banding is to provide more information for system resources chargeback and accounting purposes. To enable improved resource accounting, the queryband is written to the queryband field in the DBQL query log table.

QueryBand name:value pairs can be used with both the Filter and Workload Class categories of Teradata Active System Management (TASM).

Teradata provides Query Banding library functions that can be used in User Defined Functions (UDFs) and External Stored Procedures (XSPs). This enables users/developers to write their own UDF or XSP that can base the logic on values in the queryband.

What is Query Banding?

- Provides a mechanism for Teradata database users to wrap metadata around a query (i.e., **band the query with metadata**).
- The QueryBand is a set of **name/value pairs**:
 - `ClientUser = bs123456;`
 - `JobID = 998;`
 - `ApplicationName = Payroll;`
- Both the name and value are defined by the user so they can be customized to each application's unique needs.
- Uses of Query Banding Information; identify the origin of requests, improve Resource Accounting, enhance Workload Management, customize functions, Trusted Sessions, TVS etc...

Setting the QueryBand

A queryband is a set of name:value pairs that can be set for a session or transaction. The queryband is passed to the DBS as a list of name=value pairs in a string contained within single quotes.

Session QueryBand – the queryband is set for the session using the following syntax:

```
SET QUERY_BAND = 'Group=Finance; JobID=Fin123;' FOR SESSION;
```

The session queryband is stored in the session table and recovered after a system reset. The session queryband remains set for the session until the session ends or until the queryband is replaced by another queryband or is removed from the session. Each SET QUERY_BAND statement replaces the entire session queryband for the session (unless the Update option is used). The queryband is removed by setting it to NONE as follows:

```
SET QUERY_BAND = NONE FOR SESSION;
```

Transaction QueryBand - The queryband is set for a transaction using the following example syntax:

```
BT;  
SET QUERY_BAND = 'ClientUser = BS123456; ApplicationName = TRM;'  
FOR TRANSACTION;  
INS DeptTable("Sales ", 120);  
INS DeptTable("Accounting ", 130);  
ET;
```

The transaction queryband is discarded when the transactions ends (commit, rollback, or abort). The transaction queryband is not written to the sessiontbl and is not restored after a system reset. The transaction queryband can be set in a macro or stored procedure. See the following example:

```
REPLACE macro DisplaySales(QBIN VARCHAR(200)) AS  
(SET QUERY_BAND = :QBIN for transaction;  
SELECT * FROM SalesResults; );
```

Reserved QueryBand Names – a small set of “Reserved” QueryBand Names that existing and proposed Database features can key against.

The Base “Reserved” QueryBand Names are as follows:

ClientUser	Group	ProxyUser	ProxyRole
ApplicationName	Version	QueryIssueTime	
Source	Action	StartTime	

The Optional “Reserved” QueryBand Names are as follows:

Importance	MaxQueryTime	Deadline	JobID
JobLen	JobDeadline	JobSeq (+JobID)	UtilityName
Destination	UtilityDataSize		

Setting the QueryBand

```
SET QUERY_BAND = 'ApplicationName = Inventory; Version = 06.01.01.13;'
FOR SESSION;
```

```
SET QUERY_BAND = 'ClientUser = JW_Erving; Group = MensWear;
                  Source = MainScreen; Action = Inquiry; JobID = 101;'
FOR TRANSACTION;
```

Best Practice:

- Set **static** name/value pairs with the SET QUERY_BAND FOR **SESSION**;
- SET those that **change** per transaction in QUERY_BAND FOR **TRANSACTION**;
- There is a standard set of "**Reserved**" queryband names for use by Teradata Applications. Customer and Partner applications are encouraged to use these queryband names, when appropriate, in order to provide for better Workload Management, Prioritization and Accounting.
- Reserved queryband names are shown on the facing page.

Teradata 14.0 – Major Features

A list of key new Teradata 14.0 features is shown on the facing page.

Teradata 14.0 – Major Features

Quality/Supportability

- Teradata uses non-root Linux user account
- Redesign RSS to Support an Application Data Pull Mode
- Selective Dump Capability

Performance

- Teradata Columnar
- Collect statistics enhancements
- Hash Join enhancements
- Increased Partition Limits
- Initial Data Temperature, Phase 2

Active Enable

- Active Fallback, Phase 2
- SLES 11: Improved workload management and priorities
- Online Reconfiguration Phase
- Teradata Unity

Ease of Use

- Expansion by Business Days
- New Embedded Services Functions
- New Fields in Workload Management APIs
- NUMBER data type
- SQL ARRAY/VARRAY data type
- TD_ANYTYPE Parameter data type

Enterprise Fit

- Block level compression Enhancements
- Encryption Enhancements
- Increased number of Vprocs
- Indexes on UDT columns
- Kerberos Authentication from Linux clients
- LDAP Authentication for Multiple Directory Services
- Multiple WITH/WITH RECURSIVE clauses
- Persistent standby nodes
- Provide Table with Teradata Reserved Query Band Names
- Restricted Creation of New Kanji1 Data
- Row-Level Security
- Separate LogonSource String
- Set AuditTrailId to Authcid for LDAP Authentication
- Temperature-Based Block-Level Compression
- Unicode 6.0 Update
- Unicode Support in LOWER Function
- User-Level Export Width
- Workload Management API Support for Extended Object Name (EON)

Teradata 14.10 – Major Features

A list of key new Teradata 14.10 features is shown on the facing page.

Teradata 14.10 – Major Features

Quality/Supportability

- Reduce impact of 3610 errors
- GDO monitoring enhancements
- DBS dump enhancement

Performance

- Statistics collection enhancements
- Geospatial indexing
- 1MB data blocks (current 128K)
- Exploit contiguous write on spool/sort output
- Teradata Intelligent Memory
- Incremental query processing
- Join index enhancements
- Partial redistribution and partial duplication
- Columnar enhancements

Active Enable

- TASM enhancements – CPU & I/O Hard Limits
- Query Redrive
- Surgical TABLE rebuild

Ease of Use

- DBQL enhancements
- Automated statistics management
- 1MB spool row size (current 64K)
- SHOW table in XML
- Extended Multiload protocol

Enterprise Fit

- New analytic functions
- Table Operators
- Enforce external N/W authentication encryption
- XML datatype and Xquery support
- UNITY support enhancements
- 256-bit encryption for TDGSS
- Temporal enhancements
- Increased activity count
- Block Level Compression enhancements
- Extended Object Names
- UNICODE Hexadecimal support
- Align blocks on disk to 4KB
- Data Stream Architecture (New BAR)
- Unity Source Link
- Teradata SQL-H

Teradata 15.0 – Major Features

A list of key new Teradata 15.0 features is shown on the facing page.

Teradata 15.0 – Major Features

Quality/Supportability

- Intelligent Parser Rewrites
- HSN Health
- New PI On Access Rights Table
- Unresponsive Node Isolation
- Onsite System and Dump Analysis
- DBQL – Show Parameters
- Roles Access Rights Improvements
- Space COD w/o Restart

Performance

- Light-Weight Redistribution
- Software Efficiency Improvements

Big Data & Analytics

- JSON Integration
- Scripting and Language Support
- Table Operator Enhancements
- 3D Geospatial Data Type
- Geospatial Performance Enhancements
- SQL-H Enhancements (QueryGrid)

Industry Compatibility

- Sequenced Aggregate Enhancements and Aggregate JI for Temporal
- 1MB Phase 2
- ANSI Temporal Syntax Support
- Teradata Directory Manager

Ecosystem & Configuration Management

- SQL Interface For ShowBlocks
- TASM/TIWM – Classification by Datablock Selectivity
- Utility Statistics Logging
- TASM – Global Weights
- Unity Callback Support
- TASM - Virtual Partitions Enhancement
- TIWM - Operating Periods For Appliance
- Other Workload Management Enhancements

Teradata 15.10 – Major Features

A list of key new Teradata 15.10 features is shown on the facing page.

Teradata 15.10 – Major Features

Quality/Supportability

- CI Segregation (MI Rebuild)
- Check Table Enhancements
- Separate Error Codes for Deadlocks
- Cleanup Agg Enh Distinct Group By
- Audit Trail of DBS Control
- Studio Enhancements
- Generate TVI alert containing panic data

Performance

- Load Isolation
- Partition-level Locking
- In Memory Optimizations
- TIM Phase 2
- Reduce Spool (Simple Pipelining)
- MLOADX
- PMPC Improvements
- Reduce Parsing Time and IPE
- Smart LOBs (LOB optimizations)
- New PI on access rights table – Phase II

Extend the Lead

- CP with Primary AMP or Primary Index
- TASM Enhancements
- Add Default Queryband Profile Option

Industry Compatibility

- Auto-provisioning of External Users
- Secure Zones
- TDGSS – Single Mechanism to Logon
- Teradata Directory Service (Phase II)
- SQL Equivalents of FERRET output
- Gateway - Support for No-wait I/O
- Serialization of UDT w/in UDFs
- 1 GB Memory Buffers for SORT - Phase II
- Phase II Trusted Sessions

Ecosystem & Configuration Management

- UDF Debugger
- UDA for Teradata SQL
- DSA Enhancement - Support >5000 objects
- Support POLR with cluster changes

Big Data & Analytics

- R Integration
- Support XSLT-based Shredding Definition
- SAS: Access to table resident metadata
- JSON Binary Storage Format (BSON)
- Right, Left, and Reverse String Functions

Teradata 16.00 – Major Features

A list of key new Teradata 16.00 features is shown on the facing page.

Teradata 16.00 – Major Features

Quality/Supportability

- Feature Use Logging
- PMPC Fault Isolation
- tdgssauth Tool

Extreme Performance

- In-Memory Optimization Enhancements

Simplicity and Ease of Use

- Autostats Enhancements
- Compression Table-Level Option (Compression TLA)
- Debugger for Java UDFs and External Stored Procedures
- Gateway Reconnection Enhancements
- Global Space Accounting
- Queryable Column Information on Views
- ReqHotAmpSpool for MonitorSession API
- User Selectable Install Directory (USD)

Adaptive Execution

- COUNT DISTINCT Performance Improvement
- TASM: Dynamic Throttles
- UNION ALL Optimizations

Industry Compatibility

- 1 MB PERM and Response Rows
- Extended MultiLoad Protocol Support for Loading Temporal Tables
- Common Table Expression Enhancements
- Lightweight LDAP Authorization
- Load Isolation Enhancements
- PIVOT/UNPIVOT
- Unicode Pass Through

High Availability

- DSA: Skip Archive Object Instead of Aborting
- Write MI to Disk

Big Data & UDA Enabling

- C FNC Library Functions for Geospatial, XML, and JSON Types
- Complex Data Type Usability and Performance Enhancements
- Database Enhancements for QueryGrid and Table Operators
- ESRI-Compatible Geospatial Data Transforms
- Java Enhancements to Support UDTs and CDTs
- Teradata DATASET Data Type

Teradata 16.10 – Major Features

A list of key new Teradata 16.10 features is shown on the facing page. The strategic category description is below and continued on the 16.20 features left hand page.

Adaptive Execution

This strategic category includes features that:

- Generate better execution plans: reduce run time, perform more work.
- Reduce parsing times.
- Optimize “Little Data,” or large systems short running queries.
- Support incremental planning and execution.
- Maintain leadership in the workload management by introducing new techniques to optimize and manage system and workloads.
- Improve performance of analytics.

Big Data and UDA Enabling

This strategic category includes features that:

- Build seamlessly integrated computing architecture of processing engines and file systems.
- Build Teradata's UDA as an integrated ecosystem of multiple file systems, processing engines, and dialects that can operate in a fabric-based architecture.
- Complete QueryGrid initiatives.
- Optimize data movement and processing in the UDA.
- Dynamically add and remove storage.
- Optimizing UDA: Function shipping versus Data shipping.
- Scale CPU and I/O technology independently.
- Cooperative analytics.
- Glacial storage.

Analytics

This strategic category includes features that:

- Address capabilities to support new analytical applications.
- Improve performance of analytics.
- Support new data types and data sources.

Extreme Performance

This strategic category includes features that:

- Look for ways to leverage or exploit new technology to overall improve performance and cost effectiveness.
- Exploit hardware advancements and memory innovations.
- Continue to invest in areas of TVS/TIM.

Teradata 16.10 – Major Features

Quality/Supportability

- No Major enhancements

Extreme Performance

- No Major enhancements

Simplicity and Ease of Use

- No Major enhancements

Adaptive Execution

- Incremental Planning and Execution Enhancements
- TASM: I/O Usage Event
- IN-list Rewrite

Industry Compatibility

- Kerberos Enhancements
- Network Encryption Auditing to Gateway Log
- Packageless TeraGSS

High Availability

- Teradata Database MAPS Architecture

Big Data & UDA Enabling

- BITAND, GREATEST/LEAST, and TRUNC/ROUND Enhancements
- LAG/LEAD Functions

Teradata 16.20 – Major Features

A list of key new Teradata 16.20 features is shown on the facing page.
Continue strategic category description:

High Availability

This strategic category includes features that:

- Support Unity requirements for multi-system High Availability.
- Rapid recovery from failures or transparent failover.
- Data protection and recovery.
- Downtime-free upgrades.
- Performance execution consistency during upgrades.

Industry Compatibility

This strategic category includes features that:

- Improve migration from other databases.
- Improve areas where data and applications can be moved without impact.
- Make Teradata Database easier to use.
- Provide security features and functionality that comply with users' security policies and standards.

Quality and Supportability

This strategic category includes features that:

- Reduce unplanned downtime.
- Reduce restarts.
- Reduce failures.
- Reduce issues using Teradata product.
- Use CLCAs to target certain areas for implement.

Simplicity and Ease of Use

This strategic category includes features that:

- Leverage new tooling to attract new development users and address new markets and new applications.
- Deliver products that are easy to use and simple to implement.
- Provide new tunable to provide better control over features.

Miscellaneous Requests for Change

These features have not been categorized.

Teradata 16.20 – Major Features

Quality/Supportability

- Record Unauthenticated User Names in System Logs

Extreme Performance

- No Major enhancements

Simplicity and Ease of Use

- No Major enhancements

Adaptive Execution

- Bulk Qualification Enhancements
- TASM: Multistatement Request Classification
- Optimizer Enhancements

Analytics

- CSV Storage Format for the DATASET Data Type
- Geospatial Join and Predicate Enhancements
- Temporal FROM Clause Enhancements
- Time Series Tables and Operations

Industry Compatibility

- No Major enhancements

High Availability

- No Major enhancements

Big Data & UDA Enabling

- No Major enhancements

Miscellaneous Requests for Change

- Facility To Save Crashdump Streams
- Gateway Support to Allow Clients that Cannot Automatically Follow Security Policy to Log On
- TD_UNPIVOT and UNPIVOT Query as a Source to PIVOT
- TDGSS Support for AES-GCM Quality of Protection

Teradata Limits (Different Releases)

The chart on the facing page highlights the system limits associated with different Teradata releases.

Teradata Limits (Different Releases)

	13.0	13.10	14.0	14.10, 15.0 and 15.10	16.0,16.10 and 16.20
Maximum Vdisk Size	1.26 TB	12 TB	12 TB	12 TB	12 TB
Maximum Block Size (512 byte sectors)	255	255	255	2047	2047
Maximum Table Header Size	1 MB	1 MB	1 MB	1 MB	1 MB
Maximum Permanent Row Size	~64 KB	~64 KB	~64 KB	~64 KB	~1 MB
Concrete Step Size	2 MB	2 MB	2 MB	2 MB	2 MB
Maximum number of partitions (PPI)	65,535	65,535	9.2 x 10 ¹⁸	9.2 x 10 ¹⁸	9.2 x 10 ¹⁸
Maximum number of partition levels	15	15	62	62	62
Number of spool tables per query	2048	2048	2048	2048	2048
Maximum # of tables in a join	128	128	128	128	128
Maximum # of columns in a table	2048	2048	2048	2048	2048
Maximum # of columns in an index	64	64	64	64	64
Maximum SQL Request Limit Size	1 MB	1 MB	1 MB	1 MB	1 MB
Maximum SQL Response Buffer Size	1 MB	1 MB	1 MB	1 MB	1 MB
Maximum # of Hash Bucket Entries	1 MB	1 MB	1 MB	1 MB	1 MB
Maximum # of Vprocs (AMPs and PEs)	16,384	16,384	32,720	32,720	32,720

Module 24: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 24: Review Questions

1. Which BTEQ setting controls Teradata vs. ANSI mode? _____
2. Which commands will not work in ANSI mode? _____
3. True or False. A syntax error for an individual request in ANSI (or COMMIT) mode causes the entire transaction to be rolled back.
4. True or False. A syntax error for an individual request in Teradata (or BTET) mode causes the entire transaction to be rolled back.
5. True or False. Logging off during an explicit transaction without either a COMMIT or ET will always result in a ROLLBACK.
6. Where does a Volatile Temporary table get its space from? _____
7. Where does a Global Temporary table get its space from? _____

Notes



Introduction to Application Utilities

After completing this module, you should be able to:

- Identify the Application Utilities.
- Describe how the Application Utilities interface with the Teradata Database.
- State the advantage of using a utility over other access methods.
- Match Teradata Parallel Transporter operators with the corresponding Teradata utility.

Teradata Proprietary and Confidential

Notes

Table of Contents

Application Utilities	25-4
Application Utilities Environments.....	25-6
Application Utilities on a Local Area Network	25-6
Application Utilities on a Mainframe Host.....	25-6
Transferring Large Amounts of Data.....	25-8
INSERT/SELECT: The Fast Path.....	25-10
Multi-Statement Insert/Select Example	25-12
DELETE (ALL): The Fast Path	25-14
Teradata Parallel Transporter	25-16
Teradata Parallel Transporter Operators	25-18
Architecture.....	25-20
Data Parallelism	25-22
Comparing TPT with Load Utilities	25-24
Summary	25-26
Module 25: Review Questions.....	25-28

Application Utilities

The Teradata database provides several Application Utilities for processing large numbers of INSERTs, UPDATEs, and DELETEs in a batch environment.

Each utility exploits the capabilities provided by the Teradata parallel architecture for a specific data maintenance or batch-processing activity.

Teradata application utilities are supported on several hardware platforms including a wide range of channel-connected mainframes.

Regardless of the host platform, however, all access between the host and the Teradata database relies on the Call Level Interface (CLI), a series of callable subroutines that reside in the host's address space.

CLI is responsible for creating and managing the parcels that travel back and forth between Teradata and the host. It permits the host to send multiple tasks (sessions) to Teradata at the same time.

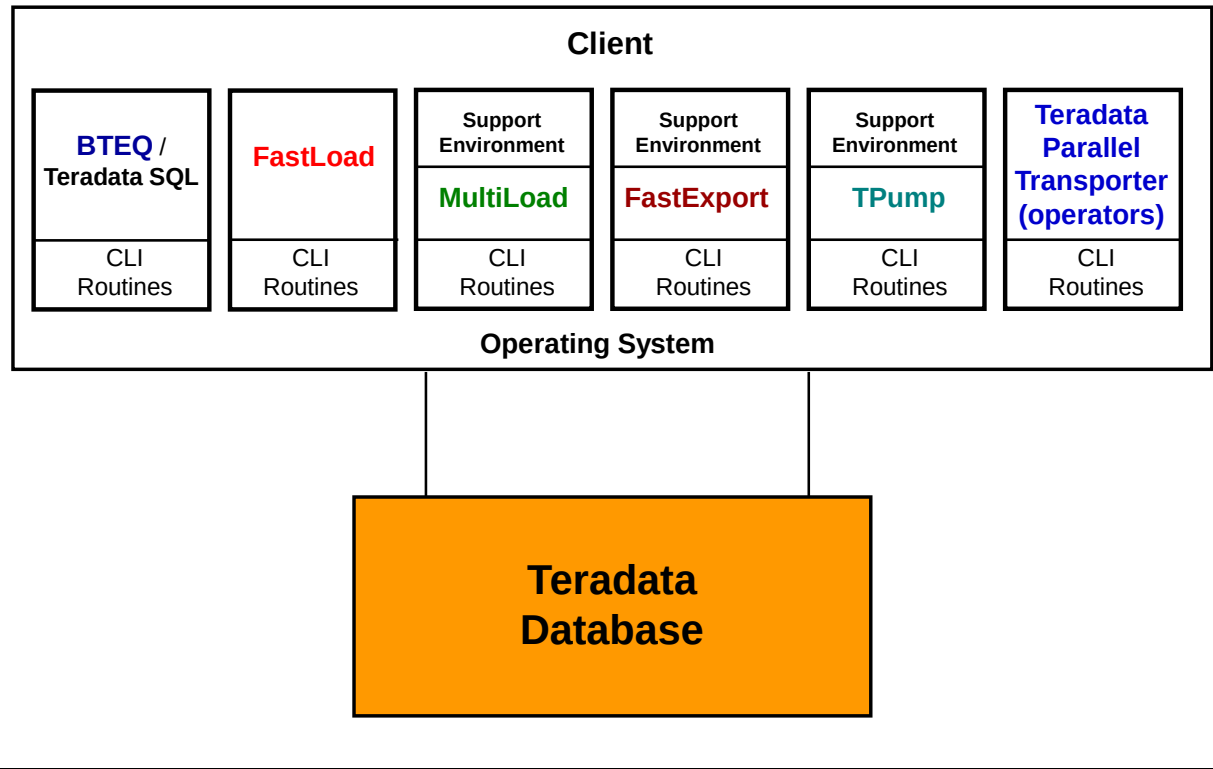
CLI is the vehicle that makes parallel access possible.

BulkLoad

The BulkLoad utility (not shown on facing page) is an older utility that executed on a channel-attached mainframe. BulkLoad was one of the original Teradata loader utilities and has been replaced by the more efficient utility, TPump. BulkLoad supported SELECT, INSERT, UPDATE, DELETE and submits SQL transactions at SQL speed.

In addition to the main processor being used on the host platform, most activities use special protocols on the database engine itself.

Application Utilities



Application Utilities Environments

Even though a single Linux Teradata node employs an innovative architecture that uses the combined power of multiple processors as a powerful Linux mainframe, it is preferred that Application Utilities execute on a separate server, a different node (NOTPA), or on a mainframe host. This server may be another node within the configuration or a separate server that is LAN connected. If the node is part of the configuration, then it communicates directly over the BYNET, thus avoiding performance constraints associated with a channel connection.

Application Utilities on a Local Area Network

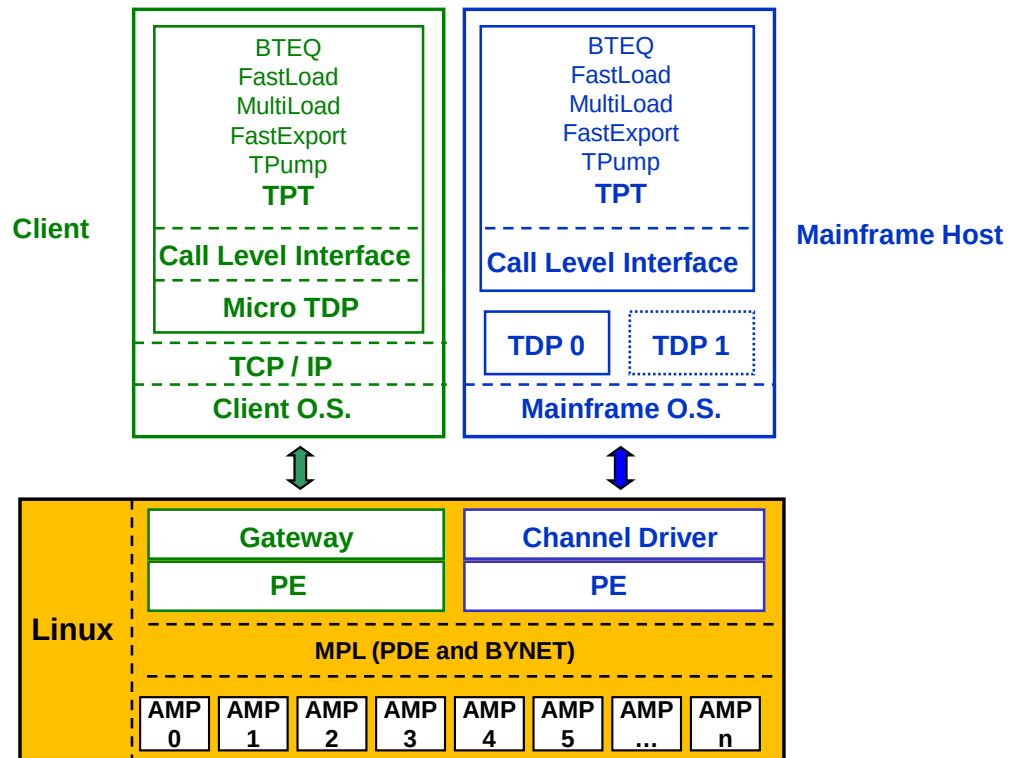
Each LAN-connected PC or server accessing the Teradata database uses the Micro TDP (MTDP) interface to communicate with Teradata.

Although the capacities of PCs and workstations are increasing rapidly, performance can be constrained by memory and disk limitations. For this reason PCs are less likely to handle the large amounts of data normally associated with the Application Utilities.

Application Utilities on a Mainframe Host

Application Utilities are frequently executed on mainframe hosts using the mainframe channel connections to load/unload data to/from Teradata. The TDP provides session control for the mainframe host.

Application Utility Environments



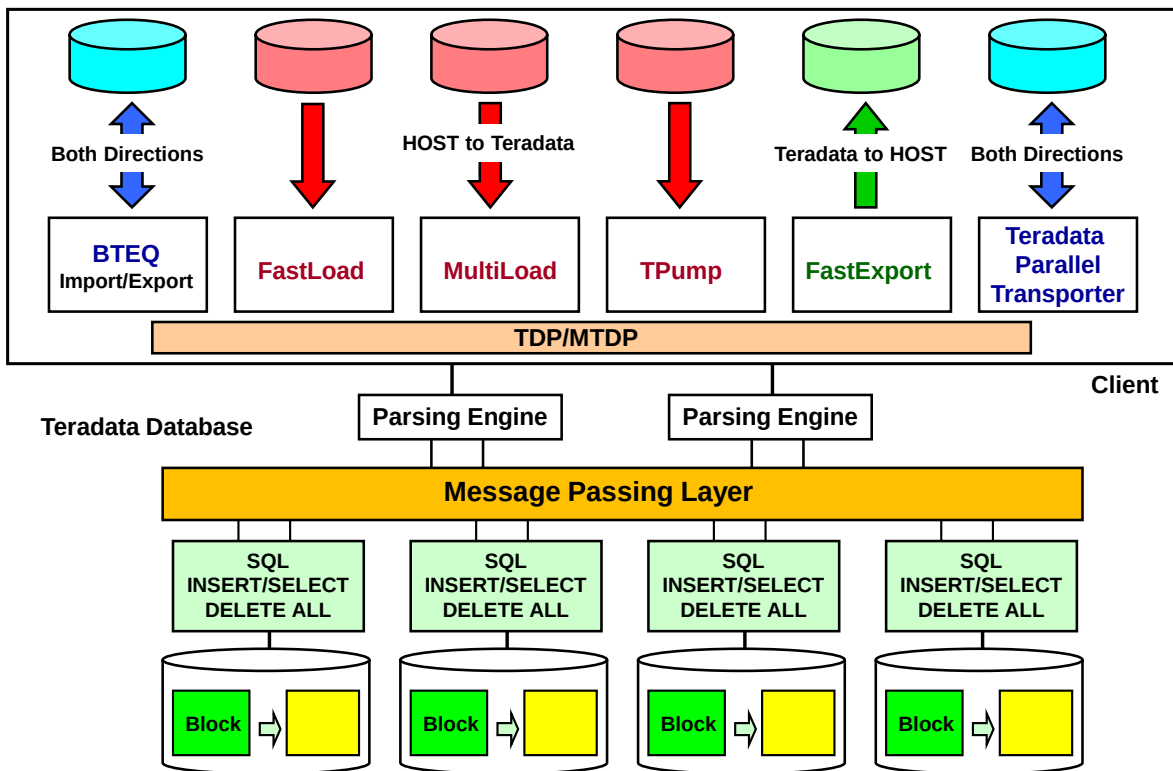
Transferring Large Amounts of Data

The Teradata application utilities reside in the host computer, whether it be the Application Processor, a mainframe, or a workstation.

- **BTEQ** supports all 4 DMLs: **SELECT**, **INSERT**, **UPDATE** and **DELETE**. BTEQ also supports **IMPORT/EXPORT** protocols.
- **FastLoad**, **MultiLoad**, and **TPump** transfer data from the host to Teradata.
- **FastExport** performs high volume **SELECT**s to export data from Teradata to the host.

Apart from the application utilities themselves, there is a special optimization of the **SQL INSERT/SELECT** and **DELETE** that deserve some attention.

Transferring Large Amounts of Data



INSERT/SELECT: The Fast Path

The Teradata INSERT/SELECT is optimized to populate one table in the Teradata database from another at high speed provided two conditions are true:

1. The tables must have the same Primary Index (PI) **and**
2. The target table must be empty.

In almost all computer systems, the disk is the slowest component and therefore defines a performance bottleneck that limits throughput. While you cannot reasonably avoid writing I/Os to disk, you can certainly try to keep them to a minimum by writing rows to disk *a block at a time*.

Teradata stores rows in data blocks sorted ascending by *row hash*. If both the source table and the target table have the same Primary Index, they will be on the same AMP and already hashed, formatted, and sorted in data blocks. With a FastPath INSERT/SELECT operation, each AMP can locally assemble the blocks for the new table in memory, and (assuming providing the target table is empty) write entire blocks to disk.

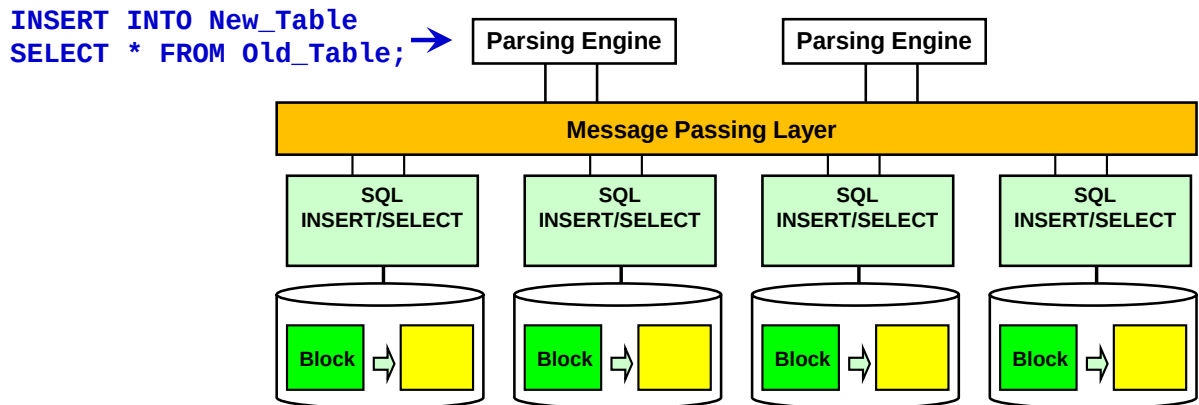
Multiple INSERT/SELECT operation

In addition to the above, BTEQ permits you to populate a single initially empty table from multiple source tables at high speed. To do this, use a multiple statement INSERT/SELECT. Again, *all* tables must have the same Primary Index. Example:

```
INSERT into T1 SELECT * FROM T2  
; INSERT INTO T1 SELECT * FROM T3  
; INSERT INTO T1 SELECT * FROM T4;
```

Each AMP selects the (presorted) rows from all source tables, builds the data blocks in memory, and writes the new table a block at a time.

INSERT/SELECT: The Fast Path



INSERT / SELECT achieves highest performance if:

- Target table is empty, **AND**
- Source and target tables have same Primary Index or if the target table is a NoPI table.

Advantages of using optimized INSERT / SELECT:

- One WRITE to the Transient Journal – instantaneous rollback for aborted Insert/Select statements.
- Data copied and written to disk a block at a time.
- No data redistribution over the BYNET.

Multi-Statement Insert/Select Example

Look at the example on the facing page. Using multiple Regional Sales History tables, a single summary table is built by combining summaries from the different regions.

A summarization may be done for each region. Summarizations then are inserted into a single table using a multi-statement Insert Select statement.

All multi-statement Insert Select statements output to the same spool table. The output is sorted and inserted into an empty table.

A multi-statement request is formed by semicolon placement in BTEQ, as shown on the facing page, or by placing statements in a single macro.

If the statements were executed separately, only the first statement is inserted into an empty table.

If you are using SQL Assistant or Studio to submit the query on the facing page, the location of the semi-colons does NOT matter. To execute this as a multi-statement request, in SQL Assistant use F9 or the multiple footprints option. In Studio, use the Execute All option.

Additional INSERT/SELECT Considerations

An INSERT/SELECT request from a source table that has duplicate rows of an already populated SET target table with a UPI or NUPI will complete and the duplicate rows are discarded – no error is reported.

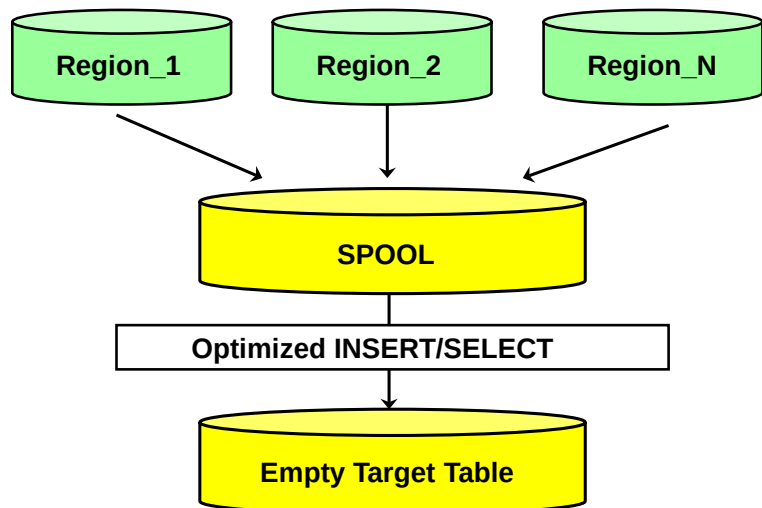
An INSERT/SELECT request from a source table that has duplicate rows of an already populated MULTISSET target table with a NUPI will complete and duplicate rows are copied into the target table.

An INSERT/SELECT request from a source table that has duplicate rows of an already populated MULTISSET target table that has with a UPI will fail (2801: Duplicate prime error) because of duplicate UPI.

Technique to remove duplicate rows: INSERT/SELECT request from a source MULTISSET table that has duplicate rows into an empty or populated SET target table (UPI or NUPI) will complete and duplicate rows from the source table are discarded – no error is reported.

Multi-Statement INSERT/SELECT Example

```
INSERT INTO Summary_Table  
SELECT store, region,  
SUM(sales),  
COUNT(saleitem)  
FROM Region_1  
GROUP BY 1, 2  
  
;INSERT INTO Summary_Table  
SELECT store, region,  
SUM(sales),  
COUNT(saleitem)  
FROM Region_2  
GROUP BY 1, 2  
...  
  
;INSERT INTO Summary_Table  
SELECT store, region,  
SUM(sales),  
COUNT(saleitem)  
FROM Region_N  
GROUP BY 1, 2  
;
```



Note: The location of the semi-colons applies to BTEQ and does NOT matter to SQL Assistant or Studio.

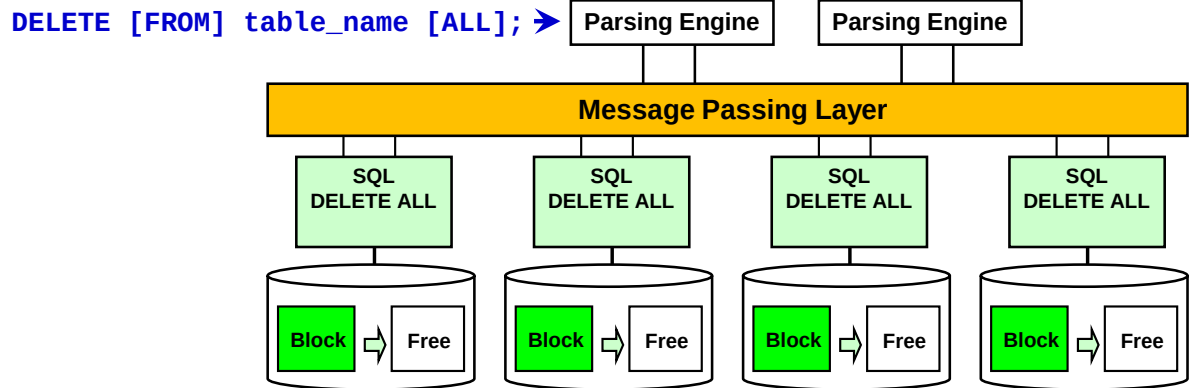
To execute this as a multi-statement request:

- in SQL Assistant use F9 or the multiple footprints option.
- in Studio, use the Execute All option.

DELETE (ALL): The Fast Path

Like INSERT/SELECT, you can achieve high performance using DELETE (ALL).

DELETE (ALL): The Fast Path



DELETE achieves its highest performance when deleting all of the rows in a table.

High performance is achievable because:

- Transient Journal is not used to store before-images of deleted rows.
- DELETES are done at the cylinder index / master index level.

ANSI Transaction Mode: In ANSI Transaction mode, to achieve the DELETE Fast Path performance, the COMMIT needs to be included as part of a multi-statement request.

```
DELETE table_name; COMMIT;
```

Teradata Parallel Transporter

Teradata Parallel Transporter is the replacement for Teradata Warehouse Builder (TWB). This utility is effectively an object-oriented software system that executes multiple instances of data extraction, transformation, and load functions in a scalable, high-speed parallel processing environment.

Teradata Parallel Transporter is scalable and enables end-to-end parallelism. The previous versions of utilities (like FastLoad) allow you to load data into Teradata in parallel, but you still have a single input stream. Teradata Parallel Transporter allows you to run multiple instances of the extract, optional transformation, and load. You can have as many loads as you have sources in the same job. With multiple sources of data coming from multiple platforms, integration is important in a parallel environment.

Teradata Parallel Transporter eliminates the need for persistent storage. It stores data into data buffers so you no longer need to write data into a flat file. Since you don't need flat files, you no longer need to worry about a 2 GB file limit.

Teradata Parallel Transporter provides a single scripting language. You can do the extract, some transformation, and loads all in one SQL-like scripting language.

Once the dynamics of the language are learned, you can perform multiple tasks with a single script.

Teradata Parallel Transporter supports FastLoad INMODs, FastExport OUTMODs, and Access Modules to provide access to all the data sources you use today.

Teradata Parallel Transporter also provides a Direct API interface that can be used by Third Party partners. They can effectively write 'C' code to directly load/unload data to/from Teradata.

Teradata Parallel Transporter

Teradata Parallel Transporter (replacement for Teradata Warehouse Builder – TWB) can load data into and export data from the Teradata database.

It is effectively **a parallel load and export utility**. Characteristics of this utility include:

High performance

- Parallel Export and Load operators eliminate sequential bottlenecks.
- Data Streams eliminate the overhead of intermediate (persistent) storage.
- Scalable
- End-to-end parallelism

Easy to use

- Single scripting language
- Access to various data sources

Extensible

- Direct API enables Third Party partners to write 'C' code to directly load/unload data to/from Teradata.

Teradata Parallel Transporter Operators

Operators are the software components that provide the actual data extraction, transformation, and load functions in support of various data stores.

This utility supports different types of operators, where the operator type signifies the primary function of the operator:

- Producer Data extraction functions (e.g., Export operator):
 - Get data from the Teradata Database or from an external data store
 - Generate data internally
 - Pass data to other operators by way of the operator interface
- Consumer Data loading functions (e.g., Load operator):
 - Accept data from other operators by way of the operator interface
 - Load data into the Teradata Database or to an external data store
- Filter Data transformation functions such as:
 - Selection, validation, cleansing, and condensing

General notes about this utility and its operators.

- The FastLoad INMOD and FastExport OUTMOD operators support the current FastLoad and FastExport INMOD/OUTMOD features.
- The Data Connector operator is an adapter for the Access Module or non-Teradata files.
- The SQL Select and Insert operators submit the Teradata SELECT and INSERT commands.
- The Load, Update, Export and Stream operators are similar to the current FastLoad, MultiLoad, FastExport and TPump utilities, but built for the TWB parallel environment.

The INMOD and OUTMOD adapters, Data Connector operator, and the SQL Select/Insert operators are included when you purchase the Infrastructure. The Load, Update, Export and Stream operators are purchased separately.

To simplify these new concepts, the facing page compares the Teradata Parallel Transporter Operators with the classic utilities.

Teradata Parallel Transporter Operators

TPT Operator	Teradata Utility	Description
LOAD	FastLoad	A consumer-type operator that uses the Teradata FastLoad protocol. Supports Error limits and Checkpoint/ Restart. Both support Multi-Value Compression and PPI.
UPDATE	MultiLoad	Utilizes the Teradata MultiLoad protocol to enable job based table updates. This allows highly scalable and parallel inserts and updates to a pre-existing table.
EXPORT	FastExport	A producer operator that emulates the FastExport utility.
STREAM	TPump	Uses multiple sessions to perform DML transactions in near real-time.
DataConnector	N/A	This operator emulates the Data Connector API. Reads external data files, writes data to external data files, reads an unspecified number of data files.
ODBC	N/A	Reads data from an ODBC Provider.

Architecture

There are two types of parallelism used by TPT: pipeline parallelism and data parallelism.

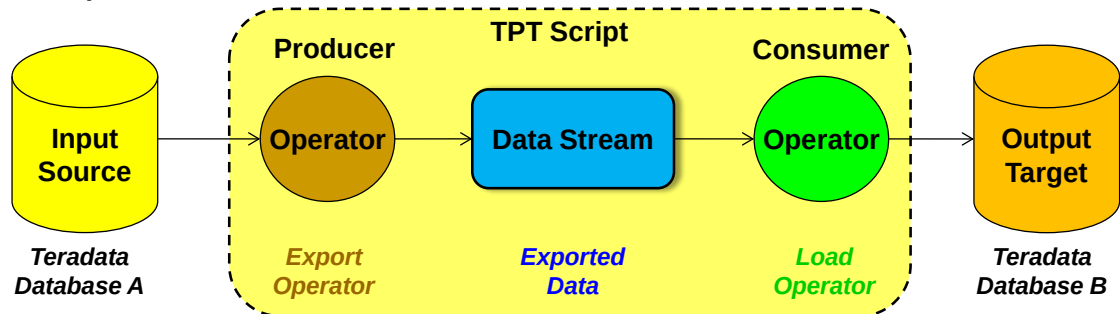
Pipeline Parallelism

Pipeline parallelism (also called "functional parallelism") is achieved by connecting operator instances through data streams in a single job. As seen in the following diagram, a "producer" operator extracts data from a data source and writes it to the data stream, that is, it 'produces' a data stream. Finally, the "consumer" operator 'consumes' the data from the data stream and then starts writing data to the target. There are two operators involved here, producer and consumer. Both operators can operate concurrently and independently of each other thereby creating a pipeline effect without the need for intermediary files.

Architecture

Pipeline parallelism (also called "functional parallelism") is achieved by connecting operator instances through data streams in a single job.

For example:



A "**producer**" operator extracts data from a data source and **writes** ("produces") it to a data stream.

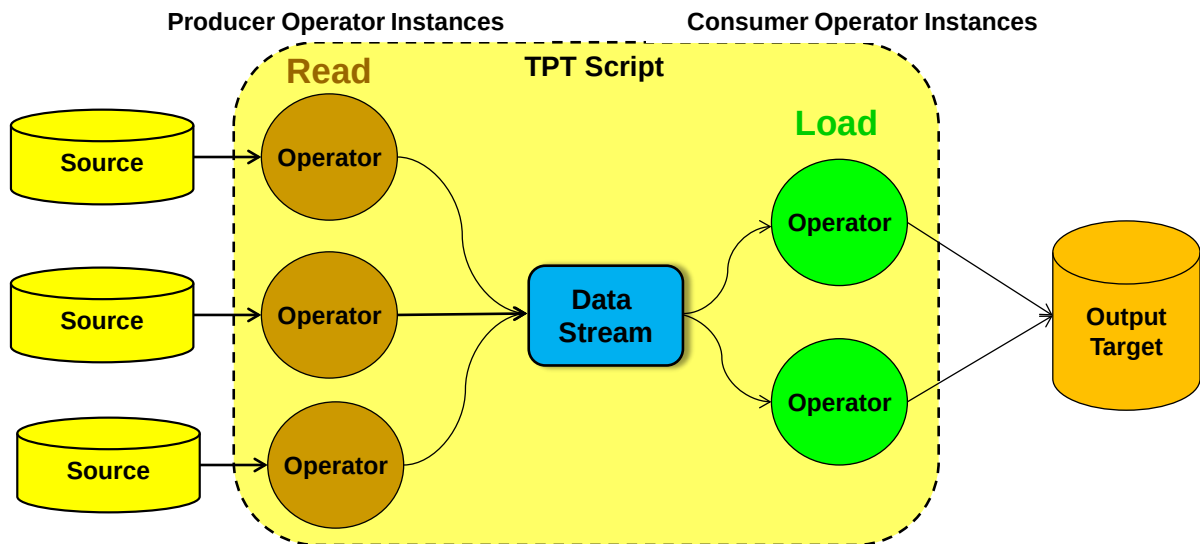
A "**consumer**" operator **reads** ("consumes") the data from the data stream and writes or loads data to the target.

Both operators can operate concurrently and independently of each other. This creates a pipeline effect without the need for intermediary files.

Data Parallelism

Imagine you have the same job as described on the previous page, but that it is handling a large quantity of data coming from several different input files. Each file can be handled by a different instance of the producer operator so that the files are read concurrently. The operator instances can then merge these files into a single data stream which can then be input to multiple consumer operators. The diagram on the facing page illustrates this concept.

Data Parallelism



Each source can be handled by a different instance of the producer operator, which merge these files into a single data stream.

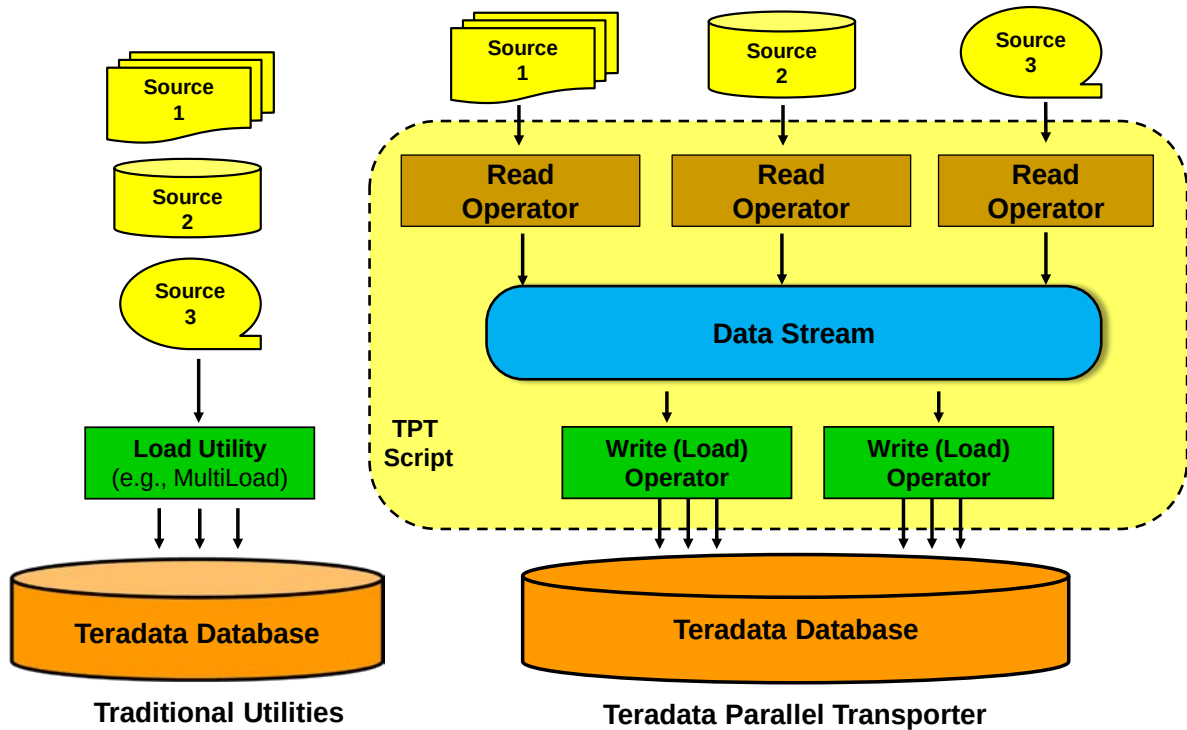
The data stream can then be input to multiple consumer operators.

Comparing TPT with Load Utilities

The illustration on the facing page contrasts how a simple load job with multiple source file inputs might look under TPT, versus as a stand-alone utility execution. Some things to notice are listed here.

- With the traditional utility, reading multiple source files means either:
 - Running one source file at a time sequentially through a job.
 - Running a separate job for each source file.
- With TPT, all files can be read concurrently through multiple producer operators.
- The load operator can also be specified to invoke multiple instances.
- As with the Teradata utilities, each instance of the load operator can run multiple sessions.
- The data sources or destinations for TPT jobs can be:
 - Databases (both relational and non-relational) or database servers
 - Data storage devices (i.e. tape or DVD readers)
 - File objects, such as images, pictures, voice, text, and so on.

Comparing TPT With Load Utilities



Summary

The facing page summarizes some of the key points of this module.

- Utilities may offer the least complex solutions for an application, and can take advantage of parallel processing.
 - There is often more than one way to set up your application, but there may be one that is fastest or most effective.
- BTEQ supports SQL statements and can execute any SQL statement such as SELECT, INSERT, UPDATE, DELETE.
 - BTEQ INSERT/SELECT, MERGE INTO, and DELETE (ALL) can provide a fast effective method to perform some tasks.
 - BTEQ allows you to import and export data. BTEQ may be useful for smaller volumes of data.
- Teradata Parallel Transporter (TPT) can load data into and export data from any accessible database object in the Teradata Database or other data store for which there exists an access operator.
 - FastLoad, MultiLoad, and TPump transfer data from the server/host to Teradata.
 - FastExport transfers data from Teradata to the server/host.

Module 25: Review Questions

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

Module 25: Review Questions

Answer True or False.

1. True or False. With TPT Update, you can import and export data.
2. True or False. In Teradata mode, an implicit BTEQ DELETE ALL function does not use the Transient Journal to store before-images of deleted rows.
3. True or False. An INSERT/SELECT of 1,000,000 rows into an empty table is only slightly faster than an INSERT/SELECT of 1,000,000 rows into a table with 1 row.

Match the Teradata Parallel Transporter operator with the corresponding Teradata utility.

- | | |
|---------------|---------------|
| 1. ___ UPDATE | A. MultiLoad |
| 2. ___ STREAM | B. FastLoad |
| 3. ___ LOAD | C. FastExport |
| 4. ___ EXPORT | D. TPump |

Notes

Module 26



Module 26: Teradata Factory Xpress Recap

After completing this module, you will be able to:

- Identify some of the key design and implementation topics that have been discussed in this class.

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata Factory Xpress Review	26-4
Teradata Differentiators	26-6
Teradata Certification Tests (14.0)	26-8
Teradata Certification Tests	26-10

Teradata Factory Xpress Review

The facing page lists some of the key topics that were covered in this course.

Teradata Factory Xpress Review

Teradata Concepts – Big Picture view of Teradata

- Teradata Database concepts, architecture, and terminology
- Data protection mechanisms (RAID, Cliques, Clusters, Locks, and Journals)
- How Teradata uses memory and utilizes disk array storage

GOAL – Understand basic concepts of Teradata and how Teradata fits with systems

Physical Database Design and Implementation – Detailed view of Teradata

- Data Distribution, Hashing, and Index Access
- Analyze Primary Index Criteria and choose Primary Indexes
- Analyze Secondary Index Criteria and choose Secondary Indexes
- Access Issues, Constraints, Sizing, and Statistics
- Understand Join Processing and interpret EXPLAIN plans of joins
- Additional Index Choices – Join Indexes, Hash Indexes, etc.

GOAL – Create tables with appropriate attributes and indexes.

Teradata Application Utilities – Export and Load data

- Introduction to export/import utilities and Teradata Parallel Transporter (TPT)

GOAL – Understand the benefits of TPT

Teradata Differentiators

The facing page identifies a number of key Teradata differentiators.

Teradata Differentiators

- **Product Maturity** – Teradata has focused on data warehousing needs since 1984.
- **Customer References** – Teradata's impressive list of customers includes leaders in their respective industries and also in the use of data warehousing technology.
- **Quickest Time to Solution** – By the nature of the inherent parallel architecture and self-managing features, Teradata provides the flexibility needed for rapid, initial implementation, and ongoing extensibility.
- **Lowest Total Cost of Ownership** – even the largest Teradata sites report having two or fewer full-time DBAs.
- **Complete Support Infrastructure** – Teradata is supported by the most skilled and experienced data warehousing professionals with 20+ years of experience.
- **Effortless Scalability** – unique, unconditional parallelism and automatic hashed data distribution are the key reasons behind Teradata's scalability.
- **High User Concurrency** – Teradata offers industry-leading performance to increasing numbers of satisfied users as the warehouse workload grows.
- **Complex and Ad Hoc Query Performance** – Teradata's parallel-aware, cost-based optimizer provides for advanced ad hoc and complex query environment.
- **Fast, Fail-safe Data Load Utilities** – Teradata ensures mission-critical availability of the information by allowing load/restore activities while users access the warehouse.
- **Seamless Mainframe Integration** – Teradata offers bi-directional, high-speed channel connectivity to leading mainframe environments.

Teradata Certification Tests (14.0)

The facing page lists the various Teradata certification tests. Depending upon the tests that are completed, you can earn various Teradata Certified designations such as Teradata Certified Professional.

The Teradata Certification tests require knowledge plus experience with Teradata. This manual will help you prepare for the Teradata 14 tests, but many of the test questions are scenario-based and Teradata experience is needed to answer these types of questions.

Disclaimer: The Teradata Certification tests include questions from a mix of sources and **require experience**, especially on tests other than the Basics test. However, some suggestions on Teradata Factory modules to concentrate on for the different tests include:

- 1 – Teradata Basics (modules 1–8, 11, 12, 16-17). A light study of modules 14, and 23-25 may be useful.**
- 2 – Teradata SQL (Teradata WBT classes) and study DDL in modules 14 (Partitioning) and 15 (Columnar).**
- 3 – Teradata Physical Design & Implementation (modules 9 through 29)**

Teradata 14.0 Certification Tests

- ✓ 141 – Teradata 14 Basics
- 142 – Teradata 14 SQL
- ✓ 143 – Teradata 14 Physical Design and Implementation
- 144 – Teradata 14 Database Administration
- 145 – Teradata 14 Solutions Development
- 146 – Teradata 14 Enterprise Architecture

147 – Teradata 14 Comprehensive Mastery

By passing all seven Teradata 14 certification tests, you become a Teradata 14 Certified Master.

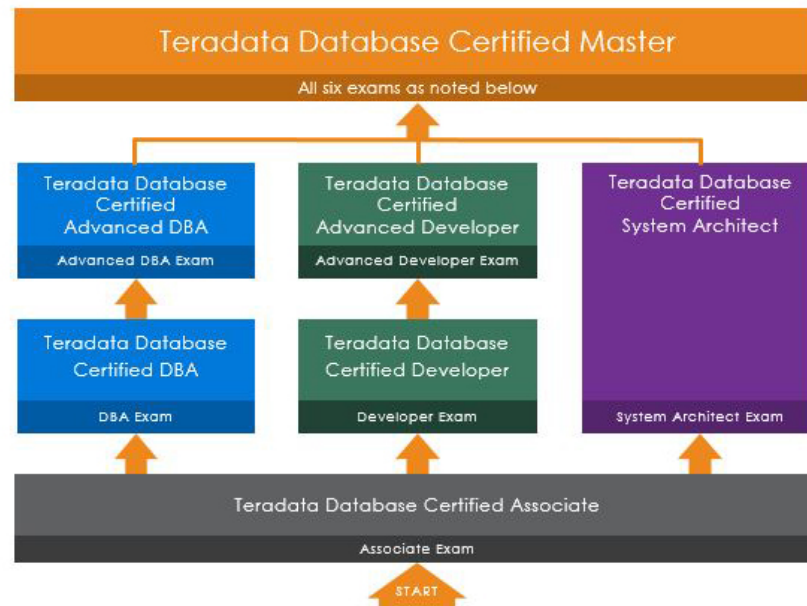
- ✓ This course (along with Teradata experience) will help prepare you for these tests.

Teradata Certification Tests

The facing page identifies the various Teradata certification tests starting in 2017 and continuing into 2018. Depending upon the tests that are completed, you can earn various Teradata Certified designations.

The Teradata Certification tests require knowledge plus experience with Teradata release 16 and higher. This manual will help you prepare for these Teradata tests, but many of the test questions are scenario-based and Teradata experience is needed to answer these types of questions.

Teradata Certification Tests



NEW for the Teradata Database Track

All certifications are valid for five years. Renew certifications by passing the same exam or earn a higher level certification.
Transition options are available for all Teradata 14 Certification holders.

Notes

Module A



Appendix A: Acronyms

**This Appendix contains a listing of various
Teradata acronyms.**

Teradata Proprietary and Confidential

Notes

Numbers

2PC Two-phase Commit

-A-

ABF	Adaptive Bandwidth Feature
ABI	Application Binary Interface
ABIOS	Advanced BIOS
ABM	Asynchronous Balance Mode
ABRD	Automatic Baud Rate Detection
AC	Application Component, Alternating Current
ACE	Array Configuration Editor
ACF	Advanced Communications Function
ACF/NCP	Advanced Communication Function for the Network Control Program
ACF/SSP	ACF/Software Support Program
ACF/VTAM	Advanced Communication Function for the Virtual Telecommunications Access Method
ACID	Atomicity, Consistency, Isolation, Durability
ADAM	Application Data Access Manager
ADC	Americas Development Center
ADCOM	Advanced Distributed Communications System
ADCS	Advanced Data Communication System
ASCII	American Standard Code for Information Interchange
ADE	Application Development Environment
ADW	Active Data Warehouse
AF	Advanced Format (drives with 4K, 8K, or 16K sectors)
AFMS	Advanced Function Management System
AI	Artificial Intelligence
AJI	Aggregate Join Index
ALC	Algorithmic Compression
AMP	Access Module Processor
ANSI	American National Standards Institute
ANSWER	Alpha-Numeric Single Wire Electronic Recorder
AOD	Application Output Definition
AOE	Application Operating Environment
AOS	Automated Order System
AP	Application Processor
API	Application Programming Interface
APPC	Advanced Program to Program Communication
AS	Administration Station
ASF	Archive Storage Facility
ASIC	Application Specific Integrated Circuit
ATM	Asynchronous Transfer Module (Networking) Automated Teller Machine
AUI	Attachment Unit Interface
AWS	Amazon Web Services or Administration WorkStation

-B-

BAM	BYNET Administrative Menus
BASIC	Beginner's All-purpose Symbolic Instruction Code
BCD	Binary-Coded Decimal
BDL	BYNET Data Link
BIC	BYNET Interface Card (Adapter)
BIM	Business Information Model; Business Impact Model
BIOS	Basic Input/Output System
BIST	Built-In Self Test
BIU	Battery Interface Unit
BLC	Block Level Compression
BLIP	BYNET Low Level Internet Protocol
BLLI	BYNET Low Latency Interface
BLM	BYNET Link Manager
BLOB	Binary Large Object
BMCA	BYNET Micro Channel Architecture adapter
BNC	Bayonet-Neill-Concelman (Ethernet connector)
BOM	Byte Order Mark (associated with Unicode)
BOOTP	BOOTstrap Protocol
BPCI	BYNET Peripheral Component Interconnect
bps	bits per second
BTAM	Basic Telecommunications Access Method
BTEQ	Basic Teradata Q uery facility
BYA4G	BYNET A Switch V2 – 4 ports – Gigahertz speed (60 MB/sec.)
BYA4M	BYNET A Switch V2 – 4 ports – Gigahertz speed (60 MB/sec.)
BYA4MS	BYNET A Switch V2 – 4 ports – Gigahertz speed (60 MB/sec.)
BYA4P	BYNET A Switch V1 – 4 ports – 10 MB/sec.
BYA8QX	BYNET A Switch V3 – 8 ports within BYA64GX or BYC64G
BYA16	BYNET A Switch V1 – 16 ports – 10 MB/sec.
BYA16G	BYNET A Switch V2 – 16 ports – Gigahertz speed (60 MB/sec.)
BYA64GX	BYNET A Switch V2 – 64 ports – Expandable - Gigahertz speed
BYAS	Component name for BYA16XS boards
BYB32	BYNET B Switch – 10 MB/sec.
BYB64G	BYNET B Switch V2
BYC64G	BYNET C Switch V3
BYCLK	BYNET Clock
BYNET	B anyan N etwork - High speed interconnect
BYOX	BYNET Optical Externsion

-C-

CA	Cable Adapter
CAD/CAM	Computer-Aided Design/Computer-Aided Manufacturing
CAE	Computer Aided Engineering
CAI	Century Analysis, Inc., Computer-Aided (or Assisted) Instruction
CASE	Computer Aided Software Engineering
CAT	Configuration and Test
CC	Cluster Controller, Common Cable, Common Carrier
CCB	Change Control Board, Character Control Block, Command Control Block
CCC	Common Carrier Communication
CCDL	Common Carrier Data Link Control
CCITT	Consultative Committee on International Telephone and Telegraph
CCL	Customer Care Link (older software to alert remote support center)
CCU	Cloud Compute Unit
CD	Compact Disk
CD-ROM	Compact Disk - Read Only Memory
CDS	Customer Data Storage
CDT	Complex Data Type
CE	Customer Engineer
CFM	Configuration File Management
CI	Cylinder Index
CICS	Customer Information Control System
CIM	Common Information Model (used as model for WBEM)
CJK	Chinese, Japanese, and Korean
CLAN	Cabinet Local Area Network
CLC	Cabinet Level Controller (5100 component)
CLCX	Cabinet Level Controller eXtended (5100 component)
CLI	Call Level Interface
CLOB	Character Large Object
CM	Communications Manager
CMB	Chassis Management Board
CMIC	Cabinet Management Interface Controller or Chassis Management Interface Controller
CMOS	Complementary Metal Oxide Semiconductor
COBOL	Common Business-Oriented Language
COP	Communications Processor
COS	Corporate Office Server
CP	Column Partitioning (or Partitioned)
CP NoPI	Column-Partitioned No Primary Index
CP NUPI	Column-Partitioned NonUnique Primary Index
CPPA	Column-Partitioned Primary AMP Index
CPPI	Column-Partitioned primary index or Character PPI
CPUPI	Column-Partitioned Unique Primary Index
CRP	Column-partitioned and Row-Partitioned
CRP NoPI	Column-partitioned and Row-Partitioned No Primary Index
CRPPA	Column-partitioned and Row-Partitioned Primary AMP index
CRPPI	Column-partitioned and Row-Partitioned Primary Index

CPU	Central Processing Unit
CR/LF	Carriage Return/Line Feed
CRC	Cyclic Redundancy Check
CRU	Customer Replaceable Unit
CSF	Customer Support Facility
CSI	Complex Service Interface
CTG	Channel Tailgate
CTS	Clear to Send

-D-

DA	Disk Array
DAC	Disk Array Controller
DAMC	Disk Array Module Cabinet
DARDAC	Dual Active Redundant Disk Array Controller
DASD	Direct-Access Storage Device
DAT	Digital Audio Tape, Dynamic Address Translation
DBA	Database Administrator
DBC	Database Computer (Teradata)
DBLT	Disk Bay Locator Table
DBMS	Database Management System
DBQL	Database Query Log
DBS	Database Subsystem
DC	Data Communications, Direct Current
DCE	Distributed Computing Environment
DCL	Data Control Language
DCRAM	Disk Cache Random Access Memory
DD	Derived Data
DDE	Direct Data Exchange
DDI	Device Driver Interface
DDL	Data Definition Language
DES	Data Encryption Standard
DEUI	Dual Ethernet UPS Interface (5400 UPS feature)
DIF	Document Interchange Facility, Data Interchange Format
DKI	Driver Kernel Interface
DIM	Diagnostic Interconnect Module (3600 component
DIMM	Dual In-line Memory Module
DIP	Database Initialization Procedure or Program
DISCO	Disconnect memory
DLPI	Data Link Provider Interface
DMA	Direct Memory Access
DMCA	Dual Micro Channel Architecture
DML	Data Manipulation Language
DMX	EMC ² <u>D</u> irect <u>M</u> atrix Disk Array
DNS	Domain Name Server
DOD	Department Of Defense
DOS	Disk Operating System
DP	Diagnostic Processor
DR	Deficiency or Defect Report
DRAM	Dynamic Random Access Memory
DS	Desk Side
DSC	Data Stream Compatible
DSS	Decision Support System
DSU	Disk Storage Unit
DSW	Destination Selection Word
DTP	Distributed Transaction Processing
DTR	Data Terminal Ready

-E-

E2I	External-to-Internal
EA	EaseAdvantage
EAFW	EaseAdvantage Framework
EAH	External Access Handler
EBCA	EISA Bus Channel Adapter
EBCDIC	Extended Binary Coded Decimal Interchange Code
ECC	Error Check and Correction
ECL	Emitter Coupler Logic
EDAC	Error Detection and Correction
EDI	Electronic Data Interchange
EDR	Enhanced Data Rate, 100 Gb bandwidth
EEPROM	Electrically Erasable Programmable Read Only Memory
EGA	Enhanced Graphics Adapter (IBM)
EISA	Extended Industry Standard Architecture
EOE	Enhanced Operating Environment
EON	Extended Object Names (or Naming) – 128 character object names
EPROM	Erasable Programmable Read-Only Memory
ESCON	Enterprise System Connection
ESM	Environmental Services Monitor
EUC	Extended UNIX Code

-F-

4GL	Fourth Generation Language
FAX	Facsimile
FC	Fibre Channel
FDDI	Fiber Distributed Data Interface
FDR	Fourteen Data Rate, 56 Gb bandwidth
FDR10	Fourteen Data Rate, 40 Gb bandwidth
FE	Field Engineer
FEU	Front End Unit
FICON	Fibre Connection (IBM)
FIFO	First In, First Out
FIPS	Federal Information Processing Standards
FK	Foreign Key
FMLI	Forms and Menu Language Interpreter
FORTTRAN	FORmula TRANslation
FSB	Front Side Bus
FSG	File Segment Cache
FRU	Field Replaceable Unit
FTAM	File Transfer and Access Method (OSI)
FTP	File Transfer Protocol
FUD	Fear, Uncertainty, Doubt
FW	Firmware
FYI	For Your Information

-G-

GB	Gigabyte (one billion bytes)
GCA	General Customer Availability
GCC	Global Consulting Center
GOSIP	Government Open Systems Interconnection Profile
GSC	Global Support Center
GUI	Graphical User Interface

-H-

HASP	Houston Automatic Spooling Program
HATP	High Availability Transaction Processing
HBN	Hash Bucket Number
HCA	Host Channel Adapter
HDA	Home Disk Address (used with Virtual Storage Services)
HDD	Hard Disk Drives
HDLC	High-level Data Link Control
HI	Hash Index
HOLAP	Hybrid On-Line Analytical Processing
HSN	Hot Standby Node
HW	Hardware
Hz	Hertz - cycles per second

-I-

iABI	Intel Application Binary Interface
I2E	Internal-to-External
IB	InfiniBand®
I ² C or I2C	Inter-Integrated Circuit
IC	Integrated Circuit
ICI	Initial Certified Installation
ICMB	Intelligent Chassis Management Bus
ICSI	Integrated Complex Service Interface
IDE	Integrated Development Environment
IE	Information Engineering
IEEE	Institute of Electrical and Electronics Engineers
I/L	Memory Inter-Leave controller
IMS	Information Management System, Inventory Management System
IO	Input/Output
IP	Internet Protocol
IPMB	Intelligent Platform Management Bus
IPS	Integrated Peripheral Subsystem
IRQ	Interrupt Request
ISA	Industry-Standard Architecture
ISAM	Indexed Sequential Access Method
ISD	Interactive System Definition
ISDN	Integrated Services Digital Network
ISO	International Standards Organization
ISP	Internet Support Package
ISPF	Interactive System Productivity Facility
ISV	Independent Software Vendor
IT	Information Technology
IWM	Integrated Workload Management

-J-

JBOD	Just a Bunch of Disks
JCL	Job Control Language
JDBC™	Java Database Connectivity
JDO	Joint Development Operation (Teradata and NCR)
JES	Job Entry Subsystem
JFS	Journaling File System
JI	Join Index
JIS	Japanese Industrial Standards
JIT	Just-In-Time
JSON	JavaScript Object Notation
JVM	Java Virtual Machine

-K-

Kb	Kilobit
KB	Kilobyte (1,024 bytes)
Kbaud	Kilo Baud
Kbps	Kilobits per second
KBPS	Kilobytes per second
KHz	Kilohertz
KSR	Keyboard Send/Receive

-L-

LAN	Local Area Network
LARC	Limited Address Range Cache
LCMP	Loosely Coupled Multiprocessing
LCD	Liquid Crystal Display
LDAP	Lightweight Directory Access Protocol
LDM	Limited Distance Modem, Local Domain Manager
LED	Light Emitting Diode
LFM	Log File Management
LM	Local Media Module
LOB	Large Object (either a BLOB or CLOB)
LPB	Local Peripheral Board
LPI	Language Processors Inc.
lpm	lines per minute (or line-per-minute)
LSU	Logical Storage Unit
LT/ST	Large Table/Small Table (join)
LUN	Logical Unit (disk array logical unit)

-M-

MA	Modular Array
MAC	Media Access Control, Medium Access Control, Message Authentication Code
MAP	Manufacturing Automation Protocol, Maintenance Analysis Procedures, Master Application Processor
Mb	Megabit
MB	Megabyte
Mbps	Megabits per second
MLAN	Management LAN
MCA	Micro Channel Architecture
MCCA	Micro Channel to Channel Adapter
MCIA	Micro Channel Interface Architecture
MHz	MegaHertz - million cycles per second
MI	Master Index
MIPS	Million Instructions per Second
MIS	Management Information System
MLPPI	Multi-Level Partitioned Primary Index
MO	Method of Operation
MOLAP	Multi-dimensional On-Line Analytical Processing
MPEG	Moving Picture Experts Group
MPL	Message Passing Layer
MPP	Massively Parallel Processing
MSU	Memory Storage Unit
MTBDL	Mean Time Between Data Loss
MTBF	Mean Time Between Failures
MTBR	Mean Time Between Repairs
MTDP	Micro Teradata Director Program
MVC	Multi-value Compression
MVS	Multiple Virtual Storage (IBM mainframe OS)

-N-

NA	Network Agent
ND	No Duplicates
NetBIOS	Network Basic Input/Output System
NFS	Network File System
NI	Network Interface
NIST	National Institute of Standards and Technology
NLQ	Near Letter Quality
NN	No Nulls
NOBOM	No Byte Order Mark (associated with Unicode)
NoPI	No Primary Index Table (Teradata 13.0 feature)
NPPI	Non-Partitioned Primary Index
NRZ	Non-Return to Zero
Ns	Nanosecond
NSC	NCR Storage Cabinet
NTOS	NCR Teradata Operating System
NUPI	Non-Unique Primary Index
NUSI	Non-Unique Secondary Index
NVRAM	Non-Volatile Random Access Memory (read and write memory)

-O-

OCC	Open Cooperative Computing
OCCA	Open Cooperative Computing Architecture
OCR	Optical Character Reader, Optical Character Recognition
ODBC™	Open Database Connectivity
ODS	Operational Data Store
OE	Operating Environment
OEM	Original Equipment Manufacturer
OLAP	On-Line Analytical Processing
OLCP	On-Line Complex Processing
OLE	Object Linking and Embedding
OLTP	On-Line Transaction Processing
OMC	ORION Memory Controller chip
OMC-DC	ORION Memory Controller - DRAM Controller
OMC-DP	ORION Memory Controller - Data Path Controller
ONE	Open Network Environment
ONS	Open Networking System
OOP	Object Oriented Programming
OPB	ORION PCI Bridge chip
OPS	Oracle Parallel Server
OS	Operating System
OSA	Open Systems Architecture
OSF	Open Software Foundation
OSI	Open Systems Interconnect
OSI/DTP	Open Systems Interconnect/Distributed Transaction Processing
OSS	Open System Standards
OTOS	Open Teradata Operating System
OUC	Object Use Count

-P-

PA	Primary AMP index
PB	Petabyte (quadrillion bytes)
PBX	Private Branch Exchange
PC	Personal Computer, Printed Circuit
PCEB	PCI EISA Bridge chip
PCI	Peripheral Component Interconnect
PCMCIA	Personal Computer Memory Card International Association
PC/RA	Probable Cause/Recommended Action
RCP	Row-partitioned and Column-Partitioned
RCP NoPI	Row-partitioned and Column-Partitioned No Primary Index
RCP PA	Row-partitioned and Column-Partitioned Primary AMP index
RCP PI	Row-partitioned and Column-Partitioned Primary Index
RPPI	Row-Partitioned Primary-Index
PSF	Teradata Priority Scheduler Framework
PCS	Personal Computer Support
PDB	Parallel Data Bus
PDCR	Performance Data and Collection Repository or Reporting
PDE	Parallel Database Extensions
PDN	Public or Private Data Network
PE	Parsing Engine
PEP	Parsing Engine Processor
PIM	Plug-In Module (used with V3 BYNET 32 switch)
PJI	Product Join Indicator (> 5 indicates a bad product join)
PK	Primary Key (or Parent Key)
PL/1	Programming Language I
PLAN	Private LAN
PM/PC	Performance Management/Production Control
PN	Primary Network
POPS	Parallel Object Processing System
POS	Point of Sale (noun), Point-of-Sale (terminal) (adj.)
POSIX	Portable Operating System Interface for Computing Environments
POST	Power-On System Test
PPI	Partitioned Primary Index
PQS	PCI Quad SCSI
PSA	Priority Scheduler Administrator
PSC	Power Shelf Controller
PSF	Teradata Priority Scheduler Framework
PSI	Power Supply Interface
PTB	Pass Through Board
PTI	Primary Time Index
PUT	Parallel Upgrade Tool

-Q-

QA	Quality Assurance
QCD	Query Capture Database
QFC	Quad Fibre Channel
QIC	Quarter Inch Cartridge

-R-

RAID	Redundant Array of Independent (formerly, Inexpensive) Disks
RAM	Random Access Memory
RASUI	Reliability, Availability, Serviceability, Usability, Installability
RC	Remote Client
RCMB	Rack (or Remote) Chassis Management Board
RDAC	Redundant Disk Array Controller (software)
RDBMS	Relational Database Management System
RFC	Request for Change
RFP	Request for Proposal
RFS	Remote File System
RGB	Red, Green, Blue
RI	Read Intensive (for SSD drives)
RISC	Reduced Instruction Set Computing
RJE	Remote Job Entry
RLL	Row Level Locking
RLM	Revision Level Manager
RM	Resource Manager (TOP END) or Rack Mount
ROLAP	Relational On-Line Analytical Processing
ROM	Read Only Memory
ROSE	Remote Operation Service Element
RPC	Remote Procedure Call
RPG	Report Program Generator
rpm	revolutions (or rotations) per minute
RSC	Remote System Call
RSS	Resource Sampling Subsystem (used for performance capture)
RTS	Ready to Send, Request to Send
RTS-CTS	Request to Send - Clear to Send

-S-

SAN	Storage Area Network
SAR	System Activity Report (sar – UNIX utility)
SCO	Santa Cruz Operation UNIX
SCSI	Standard (or Small) Computer System Interface
SDL	Screen Definition Language
SDW	Scalable Data Warehouse
SE	Systems Engineer
SEEPROM	Serial Electronically Erasable Programmable Read-Only Memory
SI	Self-Instruction
SIMM	Single In-line Memory Module
SKU	Stock-keeping Unit
SLAN	Service or System LAN (Local Area Network)
SLES	SUSE Linux Enterprise Server
SLC	Second Level Cache
SM3G	Server Management – 3 rd Generation
SMB	Server Management Board
SMC	Server Management Chassis
SMP	Symmetrical Multi-Processing
SN	Secondary Network
SNA	Systems Network Architecture
SNAG	SNA Gateway
SNO	SuperNode
SNP	Super Node Processor
SOV	Single Operational View
SP	Stored Procedure
SPI	System Performance Investigator
SQL	Structured Query Language
SQLCA	Structured Query Language Communication Area
SQLDA	Structured Query Language Data Area
SRAM	Static Random Access Memory
SS	Support Sentinel
SSA	Symbolic Service Address, System-to-System Adapter
SSD	Solid State Drive
SSI	Single System Image, Standard Serial Interface
SSL	Secure Socket Layer (Networking)
SSP	Software Support Program, System Support Program (IBM)
STMP	Simple Mail Transfer Protocol
SUS	Startup Subsystem
SUSE	Software und System Entwicklung (German name which means Software and Systems Development)
SVID	System V Interface Definition
SVR4	System V Release 4
SW	Software
SWS	Service Workstation (used with 56xx and later systems)
SYSGEN	System Generation

-T-

3GL	Third Generation Language
T2A	Teradata Database to Aster (QueryGrid)
T2H	Teradata Database to Hadoop (QueryGrid)
T2T	Teradata Database to Teradata (QueryGrid)
TAM	Teradata Access Method (used with Replication Services)
TAP	Teradata Application Program
TARA	Tiered Archive Restore Architecture
TASM	Teradata Active System Management
TAXI	Transparent Asynchronous Xceiver Interface
TB	Terabyte (a trillion bytes)
TBA	To Be Announced
TBD	To Be Determined
TCAM	Telecommunications Access Method
TCP/IP	Transmission Control Protocol/Internet Protocol
TDAS	Teradata Dual Active Solution
TDQM	Teradata Dynamic Query Manager
TFTP	Trivial File Transfer Protocol
TIM	Teradata Intelligent Memory
TLI	Transport Layer Interface
TLS	Transport Layer Security (Networking)
TM	Transaction Manager
TMSM	Teradata Multi-System Manager
TOD	Time of Day (noun) or Time-of-Day (adj.)
TOS	Teradata Operating System
TQD	Teradata Query Director
TP	Transaction Processing
TPA	Trusted Parallel Application or Architecture
TPD	Teradata Parallel Database
tps	transactions per second
TPT	Teradata Parallel Transporter
TSET	Teradata System Emulation Tool
TSO	Time-Sharing Option
TTL	Transistor-Transistor Logic
TTU	Teradata Tools and Utilities
TTY	Teletype (serial communication protocol)
TVI	Teradata Vital Infrastructure (software to alert remote support)
TVS	Teradata Virtual Storage
TVSS	Teradata Virtual Storage Services
TWA	Teradata Workload Analyzer
TWB	Teradata Warehouse Builder (old name for TPT)

-U-

U	Unit of Measurement – 1.75” or 4.445 cm
UDF	User Defined Function
UDM	User Defined Method
UDP	Unreliable Data Protocol
UDT	User Defined Type
UFS	UNIX File System
UII	Unnecessary I/O Indicator (>5 may indicate an index is needed)
UMB	Universal Management Board
UPI	Unique Primary Index
UPS	Uninterruptible Power Source
UPS-IS	Uninterruptible Power Source – Input Selector
USI	Unique Secondary Index

-V-

VAR	Value Added Reseller
VDC	Volts Direct Current
VDT	Video Display Terminal
VGA	Video Graphics Array
VLC	Value List Compression
VLf	Very Low Frequency
VLSI	Very Large Scale Integration
VM	Virtual Machine
VONUSI	Valued-ordered NUSI
VPIX	"DOS under UNIX"
VPROC	Virtual Processor
VSAM	Virtual System Access Method
VSS	Virtual Storage Services
VTAM	Virtual Telecommunications Access Method

-W-

WAL	Write Ahead Logic or Write Ahead Logging
WAN	Wide Area Network
WAWS	Windows AWS (Administration Workstation)
WBEM	Web-Based Enterprise Management
WCI	WAL Cylinder Index
WI	Write Intensive (for SSD drives)
WinDDI	Windows Data Dictionary Interface (application)
WIN-TCP	Wollongong Integrated Networking/Transmission Control Protocol
WMI	WAL Master Index
WORM	Write Once - Read Many Optical Disk
WTO	Write to Operator
WYSIWYG	What You See Is What You Get

-X-

XA	Extended Architecture
XMIT	Transmit
XML	Extensible Markup Language
XSP	External Stored Procedure

-Y-

YMCA	Ynet Micro Channel Architecture adapter
YTD	Year-to-Date

Module B



Appendix B: The Lab Environment

**This appendix describes the lab environment that
will be used during the class.**

Teradata Proprietary and Confidential

Lab Environment Notes

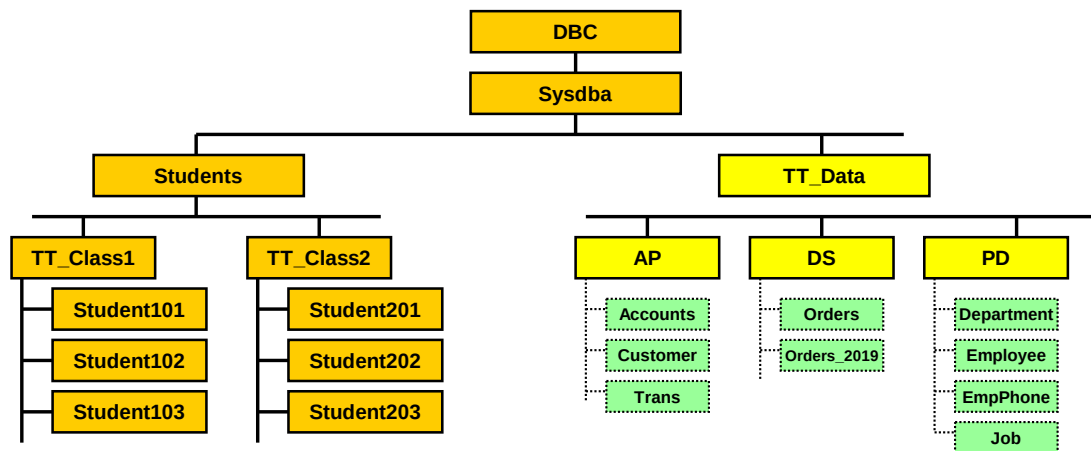
Most access to Teradata will be through Teradata Studio, Teradata Studio Express, Teradata SQL Assistant.

However, you may need to connect from a Windows PC to a Linux system. Linux requires a Secure Shell connection. This opens a terminal window which you can use to logon to Linux and enter Linux commands.

putty is an example of a utility that provides a secure shell connection

If you are executing BTEQ interactively, you cannot enter your password as part of the .LOGON statement. BTEQ will prompt you for your Teradata username password.

Lab Environment



Common Teradata SQL Commands:

To populate a newtable:

```
INSERT INTO newtable
SELECT * FROM db.olddtab WITH [NO] DATA;
```

HELP USER username;	- lists objects owned by user
HELP 'SQL sqlcommand';	- help for an SQL command
SHOW TABLE tablename;	- shows table creation syntax
SELECT COUNT(*) FROM tname;	- provides count of rows
DELETE tablename ALL;	- deletes all rows in a table
DROP TABLE tablename;	- drops table from the database

Useful Linux Commands:

ls -l	- long listing of Linux files
more filename	- view contents of text file
vi filename	- create or edit a text file
cp file1 file2	- copies file1 to file2
mv oldname newname	- rename a file
rm filename	- delete/remove a file
exit	- exits Linux

Notes

AP Tables

AP.Accounts (10,000 Rows)

Account Number	Street Number	Street	City	State	Zip Code	Balance Forward	Balance Current
PK							
UPI							
20024010	123	Harbor Blvd.	Torrance	CA	90323	1000.00	900.00
20031023	3456	186th St.	Glendale	CA	90451	1500.00	1700.00
20049873	100	Western Av.	Las Vegas	NV	97345	400.00	400.00
20031134	10	Heather Rd.	S. Monica	CA	92345	6020.00	5312.00

Integer Integer Char(25) Char(20) Char(2) Integer Decimal(10,2) Decimal(10,2)

Note:
Column names that span
2 lines are connected
with an underscore.

Ex: Account_Number

AP.Customer (7,000 Rows)

Customer Number	Last Name	First Name	Social Security
PK			
UPI			
13021	Smith	George	456788765
18765	Jones	Barbara	987453498
11023	Wilson	John	495028367
1123	Omaguchi	Sandra	234904587

Integer Char(30) Char(20) Integer

AP.Trans (15,000 Rows)

Trans Number	Trans Date	Account Number	Trans ID	Trans Amount
PK				
		NUPI		
4653	2002-02-11	20024020	2009	-50.00
3241	2002-02-08	20034567	DEP	160.00
1298	2002-02-08	20024005	2987	-70.00
11026	2002-02-13	20024020	DEP	20.00

Integer Date Integer Char(4) Decimal(10,2)

Notes

DS Tables

Orders (350,000 Rows)

orderid	custid	orderstatus	ordertotal	orderdate	orderpriority	clerk	location	shippriority	ordercomment
PK	FK,NN								
UPI									
100001	1001	C	1,005.00	2008-01-02	10	Jack ..	5	20	In Stock
103501	1451	C	1,005.00	2009-12-01	10	Dee ...	3	20	In Stock
101400	1080	C	1,150.00	2010-02-28	10	Fred ...	10	20	In Stock
Integer	Integer	CHAR(1)	Decimal (10,2)	Date	SmallInt	Char(16)	SmallInt	SmallInt	Varchar(79)

Orders_2019 (100,000 Rows) – Same Layout

Notes

PD Tables

Employee (150,000 Rows)

Emp Number	Dept Number	EmpMgr Number	Job Code	Last Name	First Name	Salary Amount
PK	FK	FK	FK			
UPI						
100001	999	?	8903	DeBosse	Ibee	200000.00
100797	1048	100791	3017	Myers	Ruth	41000.00
100002	1001	100001	3001	Smith	Steve	110000.00
<i>Integer</i>	<i>Integer</i>	<i>Integer</i>	<i>Integer</i>	<i>Char(20)</i>	<i>Varchar(20)</i>	<i>Decimal (10,2)</i>

Job (11,124 Rows)

Job Code	Job Desc
PK	
UPI	
3000	President
3001	Senior Mgmt
3017	Analyst L2
<i>Integer</i>	<i>Char(50)</i>

Department (17,415 Rows)

Dept Number	Dept Name	DeptMgr Number	Budget Amount
PK		FK	
UPI			
1048	Design SW	100791	1000000.00
1050	Design Services	100811	1000000.00
1028	Engineering SW	100441	3000000.00
<i>Integer</i>	<i>Char(100)</i>	<i>Integer</i>	<i>Decimal (10,2)</i>

EmpPhone (350,000 Rows)

Emp Number	Area Code	Phone Number	Extension
PK			
FK			
NUPI			
100001	937	5100001	1001
100001	937	4100001	1001
100389	858	4852815	815
<i>Integer</i>	<i>SmallInt</i>	<i>Integer</i>	<i>Integer</i>

Note:

Column names that span 2 lines are NOT connected with an underscore for these tables.

Ex: EmpNumber

vi or vim Notes

If you are not comfortable or familiar with vi or vim, you can create your script in a Windows Notepad, copy it, and paste it into a vi file.

1. Enter your commands (job/script) in a Notepad file.
2. Highlight the text and use the mouse to choose the **Edit → Copy** function

Switch to your terminal window where Linux is running and ...

3. **vi labx_1.btq** (or whatever filename you wish)

enter an **i** by itself (do not press <Enter>)

Use the mouse to choose the **Edit → Paste** function

Press the ESC key (multiple times does not hurt)

Enter **:wq** (this saves the file and exits vi)

Another technique that can be used to create Linux scripts without using vi is to do the following:

1. Enter your commands (job/script) in a Notepad file.
2. Highlight the text and use the mouse to choose the **Edit → Copy** function

Switch to your terminal window where Linux is running and ...

3. **cat > labx_1.btq** (or whatever filename you wish)

Use the mouse to choose the **Edit → Paste** function

To exit the cat command, press either the DELETE key or CNTL C.

vi or vim Commands

vim filename

Movement and other

h move cursor to left
j move cursor down a line
k move cursor up a line
l move cursor to right
space move cursor to right
w move a word to right
b move a word to left
0 go to start of a line (zero)
\$ go to end of a line
G go to end of file
nG go to line n
^f scroll forward 1 screen
^b scroll back 1 screen
^l refresh the screen
/str search forward for str
· repeat last command

Modification

r replace 1 character
x delete char under cursor
X delete char before cursor
5x delete 5 characters
dw delete a word
dd delete the line
d\$ delete the rest of the line
dG delete rest of the file
cw change a word
C change rest of the line
J join 2 lines together
yy yank or copy a line
3yy yank or copy 3 lines
p put yanked line(s) after line
P put yanked line(s) before line
u undo last command

Input mode

i insert before cursor
I insert at beginning of line
a insert after cursor
A insert at end of line
o open a new line below
O open a new line above
R Replace mode and insert after <cr>

ESC exits
input &
colon
modes

Colon mode

:w write or save the file
:w fname write new file name
:wq write and quit
:q! quit and don't save

:e fname edit another file
:r fname read in a file
:!cmd run UNIX command
:set smd set showmode on

Notes



Teradata SQL Assistant

After completing this module, you will be able to:

- Define a Data Source Connection for Teradata.
- Submit SQL using Teradata SQL Assistant.
- Utilize Explorer Tree to simplify creation of queries.
- Use Teradata SQL Assistant to access a table containing a LOB.

Teradata Proprietary and Confidential

Notes

Table of Contents

Teradata SQL Assistant.....	L-4
Defining an ODBC Data Source.....	L-6
Defining a Data Source (cont.).....	L-8
Connecting to a Data Source.....	L-10
Main Window	L-12
Database Explorer Tree.....	L-14
Creating and Executing a Query	L-16
Dragging Object Names to the Query Window	L-18
Query Options	L-20
Viewing Query Results	L-22
Sorting an Answerset Locally	L-22
Query Builder.....	L-24
History Window	L-26
General Options	L-28
Connecting to Multiple Data Sources	L-30
Importing/Exporting Large Object Files.....	L-32
Importing/Exporting Large Object Files.....	L-34
To Import a LOB into Teradata	L-34
Selecting from a Table with a LOB	L-36
Displaying a JPG Large Object.....	L-38
Teradata SQL Tools Summary.....	L-40

Teradata SQL Assistant

Teradata SQL Assistant is an information discovery tool designed for the Windows operating system (e.g., Windows 10). Teradata SQL Assistant retrieves data from any ODBC-compliant database server. The data can then be manipulated and stored on the desktop PC.

Teradata SQL Assistant is a query tool written for relational database developers. It is intended for SQL-proficient developers who know how to formulate queries for processing on Teradata or other ODBC-compliant Databases. Used as a discovery tool, Teradata SQL Assistant catalogs submitted instructions to arrive at a derived answer. Teradata SQL Assistant stores the history of your SQL in a local Microsoft Access database table. This history is available in future executions of Teradata SQL Assistant.

Teradata SQL Assistant accepts standard Teradata SQL, DDL, and DML. In addition, Teradata SQL Assistant sends native SQL to any other database that provides an ODBC driver. If the driver supports the statements, they are processed correctly.

Key features of SQL Assistant include:

- Create reports from any Relational Database that provides an ODBC interface
- Export data from the database to a file on a PC
- Import data from a PC file directly to the database
- Use an import file to create many similar reports (query results or Answer sets).
- Send queries to any supported database or the same query to many different databases
- Create a historical record of the submitted SQL with timings and status information such as success or failure
- Use the Database Explorer Tree to easily view database objects
- Use a procedure builder that gives you a list of valid statements for building the logic of a stored procedure
- Limit data returned to prevent runaway queries

Teradata SQL Assistant also benefits database administrators by allowing them to directly issue SHOW statements to view text for CREATE or REPLACE commands. The DBA copies the text to the Query window, uses the Replace function to change a database name, and reissues the CREATE or REPLACE to define a new object with this new name. You can also display the CREATE text by going to the shortcut menu of the Database Explorer Tree and clicking Show Definition.

Teradata SQL Assistant is a Windows-based utility for submitting SQL to Teradata and ODBC compliant databases.

SQL Assistant has the following properties:

- Two providers are available for Teradata connections:
 - Teradata ODBC Driver
 - Teradata .Net Data Provider
- **Permits retrieval of previously used queries (History).**
 - Saves information about previous query result sets.
- Supports DDL, DML and DCL commands.
 - Query Builder feature allows for easy creation of SQL statements.
- **Provides a Database Explorer Tree to easily view database objects.**
- Provides both import and export capabilities to files on a PC.
- **Can be used to access other supported ODBC-compliant databases.**
 - Does not support non-ODBC compliant syntax such as WITH BY and FORMAT.

Defining an ODBC Data Source

Before using Teradata SQL Assistant to access the Teradata database, you must first install the Teradata ODBC driver on your PC and the .Net Data Provider for Teradata. When connecting to a Teradata database, you can use either ODBC or the .Net Data Provider for Teradata.

Connection to any other database must be made through an ODBC connection. In order to use the ODBC connection, a vendor specific ODBC driver must be installed.

Before you can use Teradata SQL Assistant, you will need to define a “data source”, namely the instance of the database you wish to work with.

To define an ODBC data source, do one of the following:

- From the Windows desktop, select ...
Start > Control Panel > Administrative Tools > Data Sources (ODBC)
- From the Windows desktop, select
Start > Programs > Teradata SQL Assistant
After SQL Assistant launches, select Tools > Define Data Source

Compatibility

Teradata SQL Assistant is certified to run with any Level 2 compliant 32-bit ODBC driver. The product also works with Level 1 compliant drivers, but may not provide full functionality. Consult the ODBC driver documentation to determine the driver's conformance level. Most commercially available ODBC drivers conform to Level 2.

Defining a Teradata .Net data source

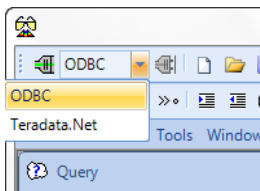
Use the Connection Information dialog to create, edit and delete data sources for .Net for Teradata. This dialog box is also used to connect to a .Net data source.

To define a Teradata .Net data source

1. Open Teradata SQL Assistant.
2. Select Teradata .Net from the provider drop down list.
3. Click the Connect icon or go to Tools > Connect.
4. Use the Connection Information dialog to choose a .Net data source.
5. Create a new data source by entering the name and server and other applicable information

Note: This module will illustrate the screens defining an ODBC data source. The specific screens defining a Teradata .Net data source are not provided in this module, but are similar

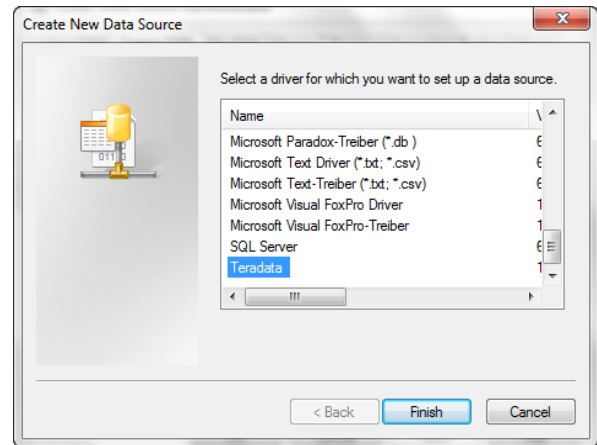
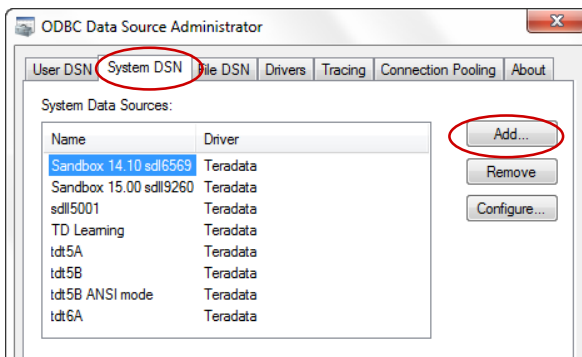
Teradata SQL Assistant Defining an ODBC Data Source



SQL Assistant has 2 provider options:

- Teradata .Net Data Provider
- ODBC

1. Choose **ODBC** from the pull-down list.
2. To define an ODBC data source:
 - Select **Tools > Define ODBC Data Source**
3. Select the **System DSN** tab and click on **Add** to create a new data source.
4. Select **Teradata** as the driver and then **Finish**.



Defining a Data Source (cont.)

In the “Define Data Source” dialog, decide what type of data source you wish to create:

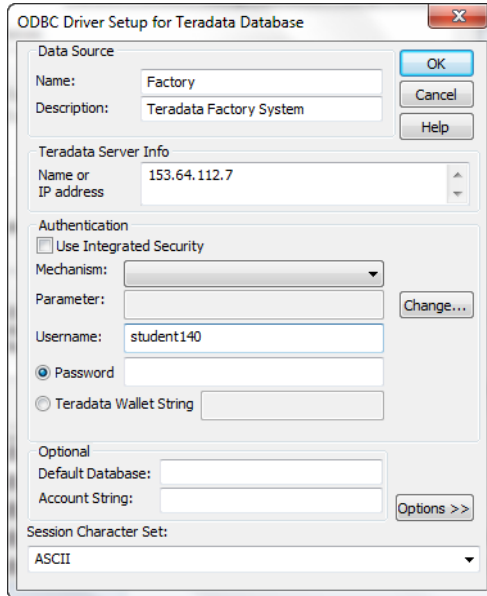
Data Source Description	Explanation
A User Data Source can be used only by the current Windows user	An ODBC user data source stores information about how to connect to the indicated data provider. A user data source is only visible to you.
A System Data Source can be used by any user defined on your PC.	An ODBC system data source stores information about how to connect to the indicated data provider. A system data source is visible to all users on this machine, including NT services.

A dialog box (specific to Teradata) is used to define the Teradata system you wish to access.

Select This Field...	To...
Name	Enter a name that identifies this data source. You can also enter the name of the system or the logon you will be using.
Description	Enter a description. This is solely a comment field to describe the data source name you used.
Name(s) or IP address(es)	Enter the name(s) or IP address(es) of the Teradata Server of your Teradata system. Identify the host by either name (alias) or IP address. The setup routine automatically searches for other systems that have similar name aliases. Multiple server names may be entered by pulling the entries on separate lines within this box.
Do not resolve alias name to IP address	When this option is checked, setup routine does not attempt to resolve alias names entered into the "Name(s) and IP address(es)" box at setup time. Instead it will be resolved at connect time. When unchecked, the setup routine automatically appends COPn (where n = 1, 2, 3, ..., 128) for each alias name you enter.
Use Integrated Security	Select this option if will be logging on using integrated security measures.
Mechanism	Select from the list of mechanisms that automatically appear in this box. Leave this field blank to use the default mechanism.
Parameter	The authentication parameter is a password required by the selected mechanism.
Username	Enter a user name.
Password	Enter a password to be used for the connection if you intend to use Teradata SQL Assistant in an unattended (batch) mode. Entering a password here is not very secure and is normally not recommended.
Default Database	Enter the default database you want this logon to use. If the Default Database is not entered, the Username is used as the default.
Account String	You can optionally enter one of the accounts that assigned to your Username.
Session Character Set	Use the drop down menu to choose the character set. The default is ASCII.

Teradata SQL Assistant Defining a Data Source (cont.)

5. Complete the QDBC Driver Dialog Window.



ODBC Driver Setup for Teradata Database

Data Source

Name: Factory

Description: Teradata Factory System

OK Cancel Help

Teradata Server Info

Name or IP address: 153.64.112.7

Authentication

☐ Use Integrated Security

Mechanism: [Dropdown]

Parameter: [Text] Change...

Username: student140

☒ Password [Text]

☐ Teradata Wallet String [Text]

Optional

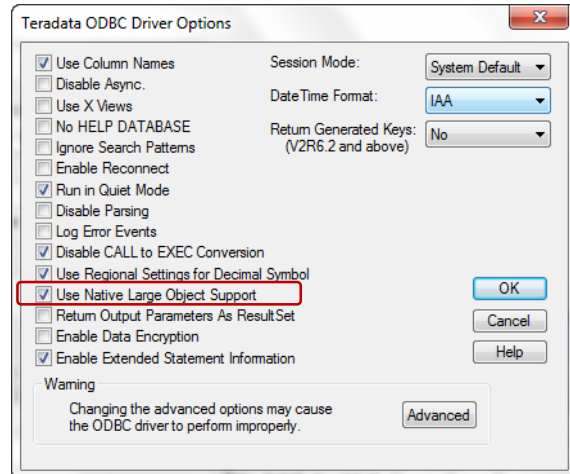
Default Database: [Text]

Account String: [Text] Options >>

Session Character Set: ASCII

To access LOBs with SQL Assistant, ...

- 1) Click on the Options button.
- 2) Check "Use Native Large Object Support".



Teradata ODBC Driver Options

☒ Use Column Names

☐ Disable Async.

☐ Use X Views

☐ No HELP DATABASE

☐ Ignore Search Patterns

☐ Enable Reconnect

☒ Run in Quiet Mode

☐ Disable Parsing

☐ Log Error Events

☒ Disable CALL to EXEC Conversion

☒ Use Regional Settings for Decimal Symbol

☒ Use Native Large Object Support

☐ Return Output Parameters As Result Set

☐ Enable Data Encryption

☒ Enable Extended Statement Information

Session Mode: System Default

Date Time Format: IAA

Return Generated Keys: (V2R6.2 and above) No

OK Cancel Help

Warning

Changing the advanced options may cause the ODBC driver to perform improperly.

Advanced

Connecting to a Data Source

Connecting to a data source is the equivalent of “logging on” with SQL Assistant. You may choose from any previously defined data source.

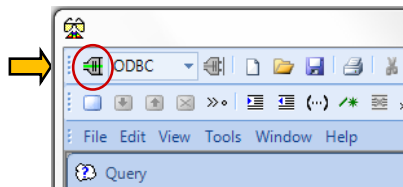
When the connection is complete, the **Connect** icon is disabled and the **Disconnect** icon, to its right, is enabled.

To connect to multiple data sources:

1. Go to the Tools > Options > General tab.
2. Click Allow connections to multiple data sources (Query windows),
3. Follow the procedure for connecting to a data source.

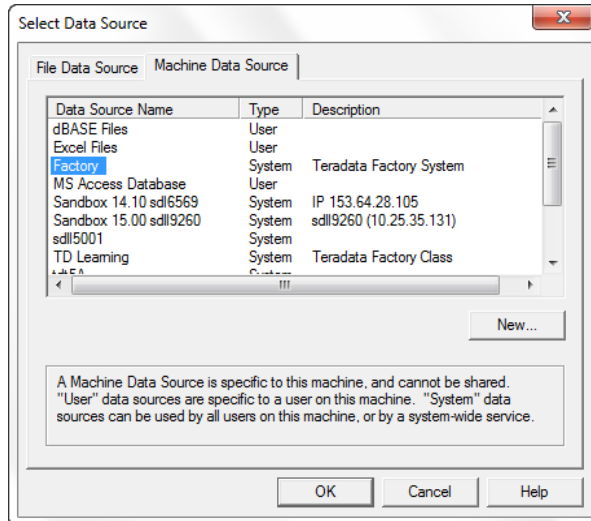
Each new data source appears in the Database Explorer Tree and opens a new query window with the data source name. To disconnect from one data source, click the Query window that is connected to the data source and click the disconnect icon.

Teradata SQL Assistant Connecting to a Data Source

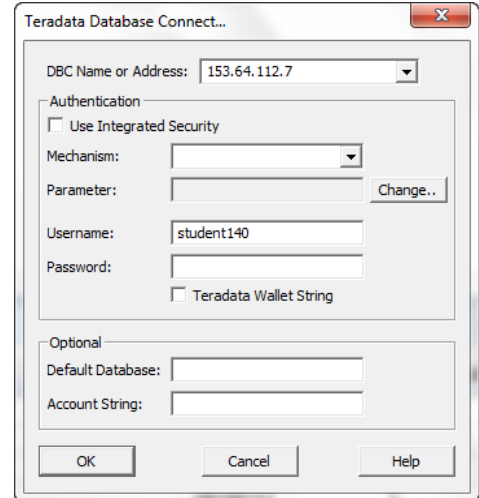


1. Click on the Connection icon to connect to Teradata.
Provider options are Teradata .NET or ODBC.

2. Select a data source.



3. Complete the logon dialog box.



Main Window

The Query window is where you enter and execute a query. The results from your query are placed into one or more AnswerSet windows.

The AnswerSet window is a table Teradata SQL Assistant uses to display the output of a query.

The History window is a table that displays your past queries and related processing attributes. The past queries and processing attributes are stored locally in a Microsoft Access database. This gives you flexibility to work with previous SQL statements in the future.

The Database Explorer Tree displays on the left side of the main Teradata SQL Assistant window. It displays an alphabetical listing of databases and objects in the connected Teradata server. You can double-click on a database name to expand the tree display for that database.

You can use the Database Explorer Tree to reduce the time required to build a query and help reduce errors in object names. The Database Explorer Tree is optional so you can display or hide this window.

Teradata SQL Assistant Main Window

The screenshot displays the Teradata SQL Assistant Main Window with the following components:

- Database Explorer Tree:** Located on the left, it shows the 'Factory (Teradata)' database with sub-nodes 'DBC' and 'STUDENT140'.
- Query Window:** The central area for writing SQL queries. It contains the query: `SELECT * FROM DS.Orders SAMPLE 10;`
- Answerset Window:** Displays the results of the query in a table format. The table has columns: orderid, custid, orderstatus, ordertotal, orderdate, orderpriority, and clerk. The data is as follows:

	orderid	custid	orderstatus	ordertotal	orderdate	orderpriority	clerk
1	163,390	1,129	C	1,270.21	4/13/2006	10	April Rains
2	520,000	1,330	C	1,397.83	8/31/2013	10	Auggie Sunshin
3	265,975	1,385	C	1,185.19	10/25/2010	10	Ossie Leaves
- History Window:** Located at the bottom, it shows a log of executed queries. The table has columns: Date / Time, Source, Elapsed, and Rows. The data is as follows:

	Date / Time	Source	Elapsed	Rows
1	11/7/2014 08:50:32	Factory	00:00:01	10
2	11/7/2014 08:24:46	TD Learning	00:00:00	26
3	11/6/2014 18:38:14	Sandbox 15 00 sdll9260	00:00:03	1436

Database Explorer Tree

The Database Explorer Tree feature of Teradata SQL Assistant displays an alphabetical listing of databases and objects of the connected user. It further permits drilldown on individual objects to view, column names, indexes and parameters as they apply. This is simply done by double-clicking on a database name to expand the tree display for that database.

The Database Explorer Tree displays on the left side of the main Teradata SQL Assistant window. You can use the Database Explorer Tree to reduce the time required to build a query and help reduce errors in object names. The Database Explorer Tree is optional so you can display or hide this window.

Initially, the following Teradata databases are loaded into the Database Explorer Tree:

- The User ID that was used to connect to the database
- The user's default database
- The database "DBC"

To add additional databases:

1. Do one of the following:
 - With the Database Explorer Tree active, press **Insert t**.
 - Right-click anywhere in the Database Explorer Tree, then select **Add Database**.
2. Type the database name to be added.
3. If you want the database loaded only for the current session, clear the check box. By default, the check box is selected so the database will appear in the Database Explorer Tree in future sessions.

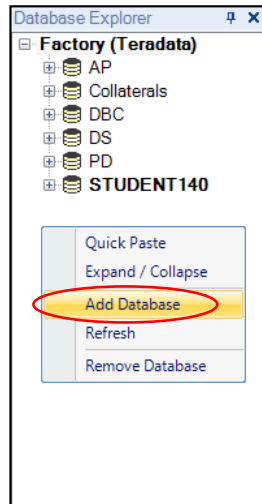
The Database Explorer Tree allows you to drill down to show:

- Columns and indexes of tables
- Columns of views
- Parameters of macros
- Parameters of stored procedures

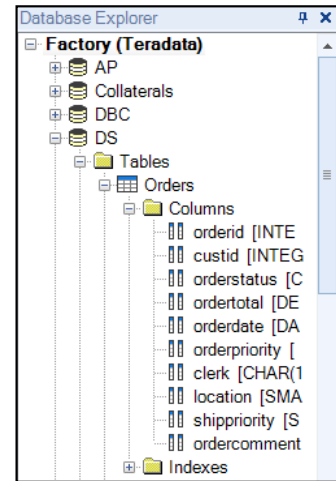
Teradata SQL Assistant Database Explorer Tree

- The Database Explorer Tree displays an alphabetical listing of databases and objects of the connected user.
 - It is not a database hierarchy, but a list of databases and objects that the user needs to access.
- To refresh a database, right-click on the database/user name and select "Refresh".

To add another database to the Explorer Tree, right-click on the system or a database name.



To expand an item/object, click on the + sign or double-click on the object name.



Creating and Executing a Query

Queries are created by simply typing in the query text into the query window. It is not necessary to add a semi-colon at the end of a command, unless you are entering multiple commands in a single window. The query may be executed by clicking on the 'Execute' icon in the toolbar. This icon looks like a pair of footprints.

“Execute” actually executes the statements in the query one statement after the other and optionally stops if one of the statements returns an error. Function key F5 can also be used to execute queries serially.

“Execute Parallel” executes all statements at the same time - and is only valid if all the statements are Teradata SQL/DML statements. This submits the entire query as a single request, allowing the database to execute all the statements in parallel. Multiple answer sets are returned in the Answerset window. Function key F9 can also be used to execute queries in parallel.

Creating statements (single and multi-queries)

To allow multiple queries:

1. Select Tools > Options.
2. Select the General option.
3. Select the option “Allow Multiple Queries”.

Once this option is selected, you may open additional tabs in the query window. Each tab can contain a separate query, and any of these queries can be executed. However, only one query can be executed at a time.

You can create queries consisting of one or more statements.

A semicolon is not required when you enter one statement at a time. However, a semicolon between the statements is required for two or more statements.

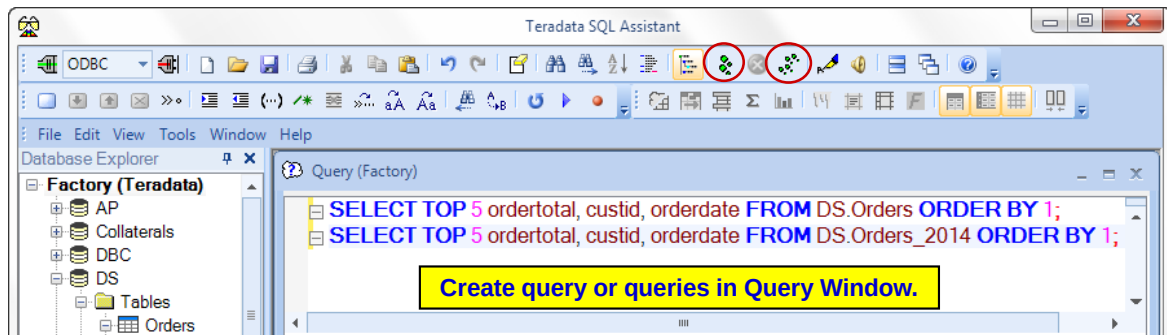
Each statement in the query is submitted separately to the database; therefore, your query may return more than one Answerset.

Teradata SQL Assistant Creating and Executing a Query

1. Create a query in the Query Window.
2. To execute a query use either the “execute” or the “execute parallel” buttons.

The “execute” button  (or F5) **serially** executes all statements in the query window.

The “execute parallel” button  or (F9) executes all statements in the query window in a single multi-statement request. These queries are effectively executed **in parallel**.



Dragging Object Names to the Query Window

You can drag object names from the Database Explorer tree to the Query pane.

Click and drag the object from the Explorer tree to the Query pane. The name of the object appears in the Query window.

Teradata SQL Assistant includes an option (Tools > Options) that allows objects to automatically be qualified when dragging or pasting names from the Database Tree into the Query Window.

For example, if this option is checked, dragging the object "MyColumn" adds the parent object "MyTable", and appears as "MyTable.MyColumn" in the Query Window.

Use the Ctrl key to add a comma after the object name when it is dragged to the Query Window.

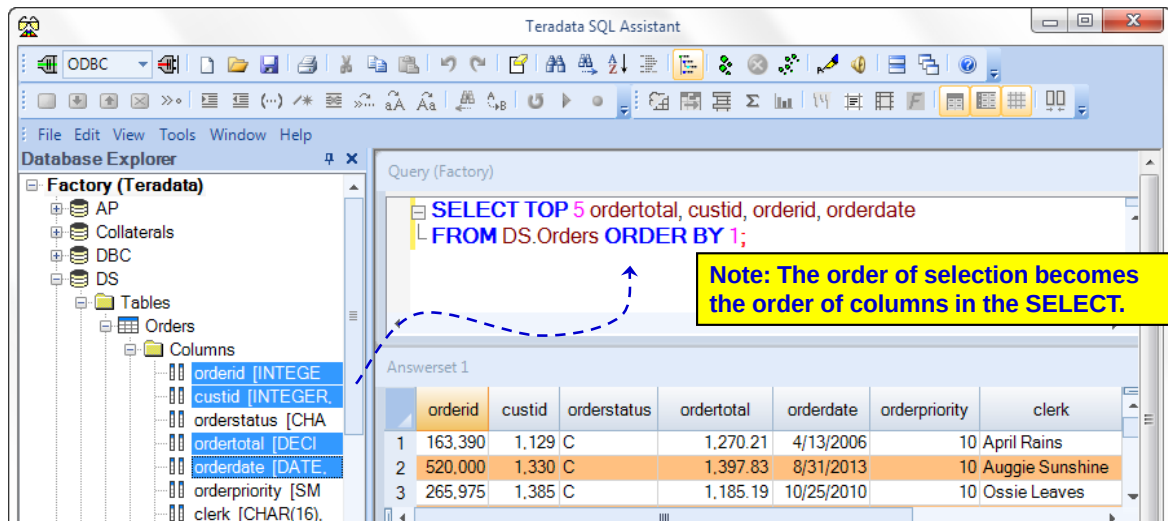
Dragging Multiple Objects

Use the Shift and Ctrl keys to select more than one object from the Database Explorer Tree that can be dragged to the Query window.

- Use the Ctrl key to select additional objects.
- Use the Shift key to select a range of objects.

Teradata SQL Assistant Dragging Object Names to the Query Window

- You can click and drag object(s) from the Database Explorer tree to the Query window.
 - If the "**Qualify names when dragged or pasted from the Database Tree**" option (Tools > Options) is checked, then the parent name is automatically included.
 - Hold Ctrl key** – causes a comma to be included after the object
 - The **Shift** and **Ctrl** keys can also be used to select multiple objects in the Database Explorer tree for the purpose of dragging multiple objects to the Query Window.



Query Options

To submit any part of any query

1. Select Tools > Options.
2. Select the Query tab.
3. Check the option “Submit only the selected Query text, when highlighted”.
4. From the Query window, select the part of the query to submit by highlighting it.

Clearing the Query Window

The query window may be cleared using the “Clear Query” button on the tool bar.

Formatting a Query

The query formatting feature adds line breaks and indentation before certain keywords, making SQL that comes from automatic code generators or other sources more readable.

To Format a Query

1. Ensure a statement exists in the Query window.
2. Do one of the following:
 - From the Tool Bar, click the Format Query button.
 - Right-click in the Query window, then click **Format Query**
 - Press **Ctrl+Q**
 - Select **Edit > Format Query**

Note: Some keywords will cause a line break and possibly cause the new line to be indented. If a keyword is found to already be the first word on a line and it is already prefixed by a tab character, then its indentation level will not change.

Indentation

When you press the Enter key, the new line will automatically indent to the same level as the line above.

If you highlight one or more lines in the query and press the Tab key, those lines are indented one level. If you press Shift-Tab, the highlighted lines are un-indented by one level.

This indentation of lines will only apply if the selected text includes a line feed character. For example, you must either select at least part of two lines, or if selecting only one line, then the cursor must be at the beginning of the next line. (Note that this is always the case when you use the margin to select a line.) If no line end is included in the selected text, or no text is selected, then a tab character will simply be inserted.

Teradata SQL Assistant Query Options

To submit any part of a query:

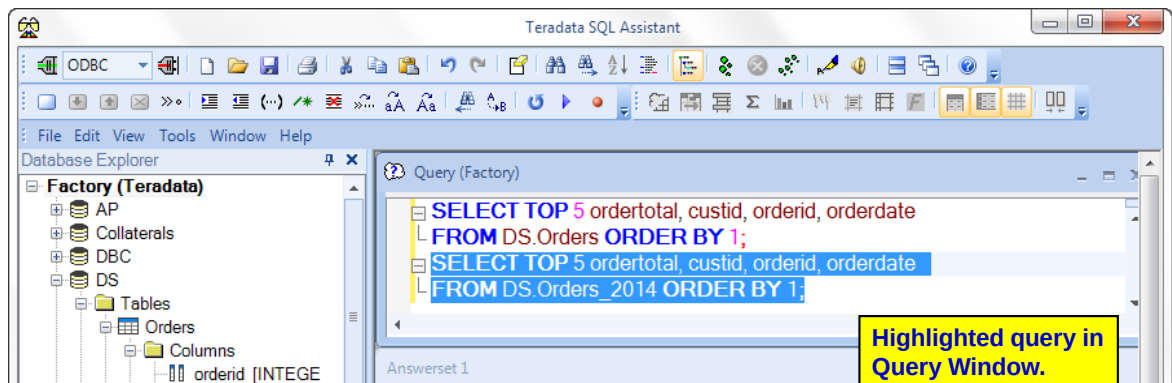
1. Using Tools > Options > Query

Check the option “Submit only the selected Query text, when highlighted”.

2. Highlight the text in the query window and execute. 

To clear the text in the query window, use the “Clear Query” button. 

To format a query, click on the “Format Query” button. 



Viewing Query Results

The results of a query execution may be seen in the Answer Set window. Large answer sets may be scrolled using the slide bars.

The Answerset window is a table that displays the results from a statement. You can sort the output in a number of ways and print as bitmaps in spreadsheet format. Individual cells, rows, columns, or blocks of columns may be formatted to change the background and foreground color as well as the font style, name, and size. You can make other modifications such as displaying or hiding gridlines and column headers.

The table may be resized by stretching the Answerset window using standard Windows sizing techniques. Individual columns, groups of columns, rows, or groups of rows may also be sized.

Output rows may be viewed as they are being retrieved from the database.

Sorting an Answerset Locally

There are two ways to sort an Answerset locally: **quick sort** or **full sort**. A quick sort sorts on a single column; a full sort allows sorting by data in multiple columns.

To sort an Answerset using quick sort:

- Right-click any column heading to sort the data by that column only. The data is initially sorted in ascending order. Right-click the same column header again reverses the sort order.

Note: The output from certain statements (e.g., EXPLAIN) cannot be sorted this way.

To sort an Answerset using a full sort:

- Do one of the following: From the Tool Bar, click the **sort** button, right-click in the Answerset window and select **Sort**, or use the Edit > Sort menu.

In the Sort Answerset dialog box, all columns in the active window are presented in the **Available Columns** list box.

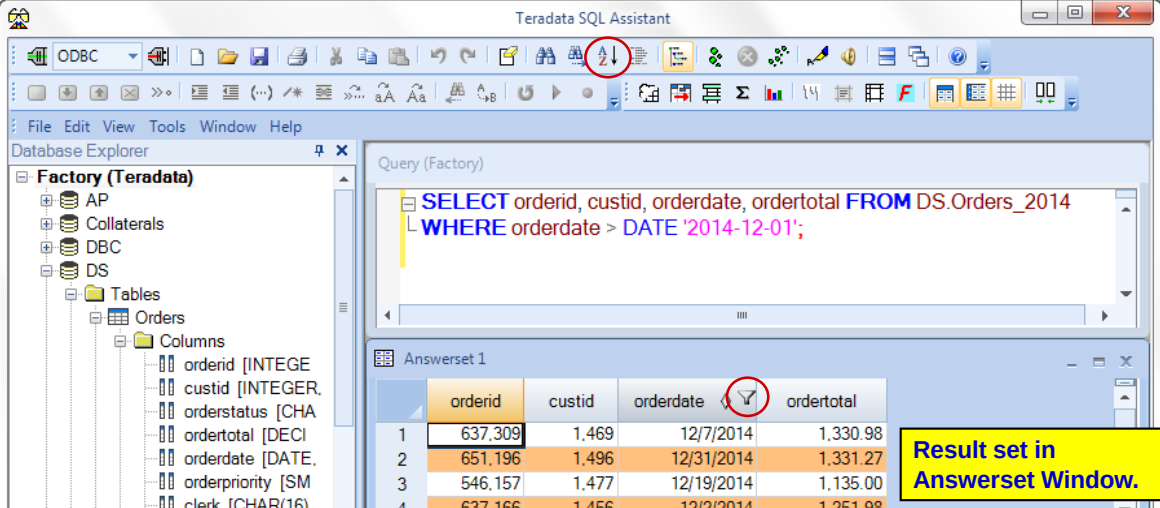
- Select the column name in the **Available Columns** list box, or use the up or down arrow keys to highlight the column name and press **Enter**.

This moves the column name to the **Sort keys** list box. By default, the sort direction for this new sort column is ascending (Asc). If you click a column in the **Sort Keys** list box, or select the item using the arrow keys or mouse and press **Enter**, it reverses to descending sort order (Dsc).

To remove a sort column from the list, double-click the column name, or use the arrow keys to highlight the column and press **Delete**.

Teradata SQL Assistant Viewing Query Results

- The Answerset window is a table that displays the results from a statement.
- The output can be sorted in different ways:
 - Quick sort (single column) – right click on the column heading
 - Full sort (1 or more columns) – use Edit > Sort menu or Sort button 
- Data can be filtered using the funnel option at the column level. 



The screenshot shows the Teradata SQL Assistant interface. On the left is the Database Explorer showing the 'Factory (Teradata)' tree with folders for AP, Collaterals, DBC, and DS. Under DS, there are 'Tables' and 'Columns' folders. The 'Columns' folder is expanded, showing a list of columns: orderid [INTEGER], custid [INTEGER], orderstatus [CHAR], ordertotal [DECIMAL], orderdate [DATE], orderpriority [SMALLINT], and clerk [CHAR(16)].

The main window displays a query in the 'Query (Factory)' pane:

```
SELECT orderid, custid, orderdate, ordertotal FROM DS.Orders_2014
WHERE orderdate > DATE '2014-12-01';
```

Below the query pane is the 'Answerset 1' window, which displays the results in a table. A red circle highlights the funnel icon in the column header row. A yellow callout box points to the table with the text 'Result set in Answerset Window.'

	orderid	custid	orderdate	ordertotal
1	637,309	1,469	12/7/2014	1,330.98
2	651,196	1,496	12/31/2014	1,331.27
3	546,157	1,477	12/19/2014	1,135.00
4	637,166	1,456	12/2/2014	1,251.98

Query Builder

Query Builder provides the user with the ability to use ‘templates’ for SQL commands, which may then be modified by the user. This is a convenient way to create commands whose syntax is complex or not easily remembered. Simply find the appropriate command, then drag and drop it into the query window where it may then be customized.

The Query Builder window is a floating window you can leave open when you are working within the main Teradata SQL Assistant window.

To access the Query Builder tool, do one of the following:

- Press F2.
- Select **Help > Query Builder**.
- Right-click in the Query window and select **Query Builder** from the shortcut menu.

From the drop-down list in the upper left corner, choose one of the following options.

SQL Statements	Select a command from the statement list in the left pane to display an example of its syntax in the right pane.
Procedure Builder	Select a stored procedure statement from the list in the left pane to display an example of its syntax in the right pane.
<User Defined>	If you create a <i>custom.syn</i> file, this option appears in the drop-down list. The name will be the name you specified in the first line of the <i>custom.syn</i> file. Select this option and the queries you defined in this file will display.

Description of the Options

SQL Statements

When you choose the SQL Statements option, the statement list in the left pane shows each of the statement types available on the current data source. These syntax examples reflect the SQL syntax of the data source you are currently connected. For example, the Teradata syntax file is *Teradata.syn*.

Procedure Builder

When you choose the Procedure Builder option, the left pane shows a list of statements that are valid only when used in a CREATE or REPLACE procedure statement.

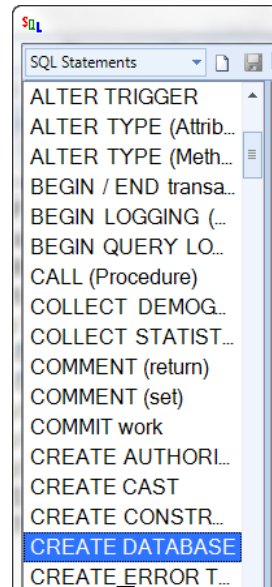
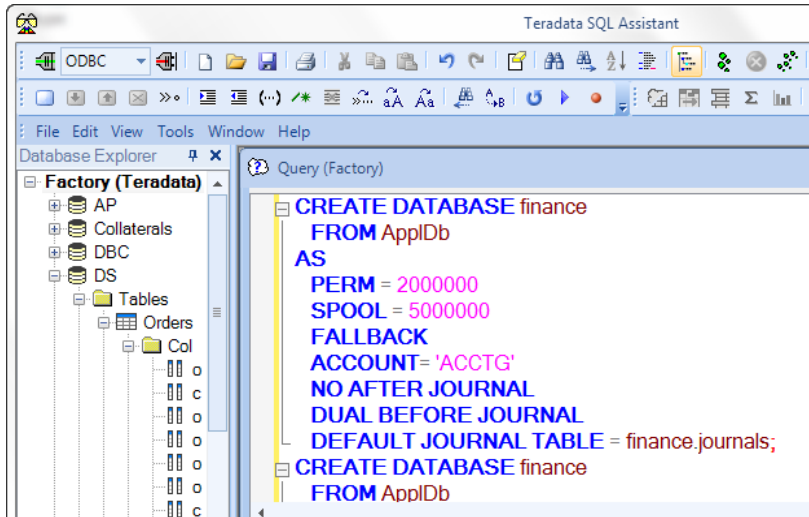
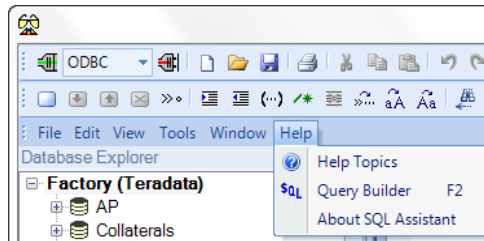
<User Defined>

You can create a user-defined syntax file using any text editor such as Notepad or Microsoft Word. The name of the file must be *custom.syn*. The format of this file is the same as the other syntax files except it has an additional line at the start of the file containing the name you wish to see in the dropdown list in the Query Builder dialog.

Teradata SQL Assistant Query Builder

Query Builder provides the user with the ability to use 'templates' for SQL commands.

1. Select Query Builder from the Help menu or use F2.



2. Double-click on SQL statement to place sample query in Query Window.

History Window

The **History** window is a table that displays your past queries and related processing attributes. The past queries and processing attributes are stored locally in a Microsoft Access 2000 database. This allows the flexibility to work with previous SQL statements in the future.

Clicking any cell in the SQL Statement column in the History window copies the SQL to the Query Window. It may then be optionally modified and then resubmitted.

You can display or hide the History window at any time.

With Teradata SQL Assistant 13, all history rows are now stored in a single History database. The History Filter dialog allows you to specify a set of filters to be applied to the history rows. The operators include >, <, =, and **LIKE**. The filter applies to the entire history table. When you click in the fields or boxes in the **Filter** dialog, the possible operators and proper format are displayed at the bottom of the dialog.

You can filter your history on the following options:

- Date
- Data source
- User Name
- Statement Type – for example, SELECT or CREATE TABLE
- Statement Count – show only those queries that contain this many statements
- Row Count
- Elapsed Time
- Show successful queries only

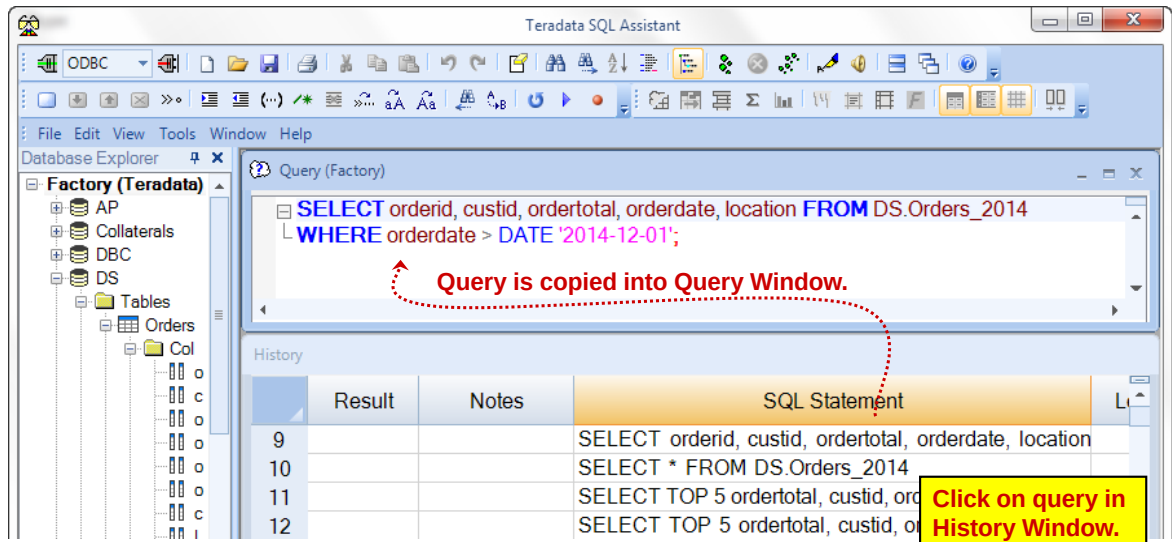
By default, Teradata SQL Assistant records all queries that are submitted. You may change this option so Teradata SQL Assistant records only those statements that are successful, or turn off history recording altogether.

The most recently executed statement appears as the first row in the History window. The data may be sorted locally after it has been loaded into the History window. New entries are added as the first row of history no matter what sort order has been applied.

Teradata SQL Assistant History Window

A history of recently submitted queries may be recalled by activating the 'Show History' feature. Key options available with the History window are:

- All history rows are now stored in a single History database. The History Filter dialog allows you to specify a set of filters to be applied to the history rows.
- You can choose to display all queries (successful or not), use a history filter to **only display successful queries**, or turn off history recording altogether.



General Options

To set general program preferences:

1. Select Tools > Options.
2. Click the General tab.
3. Choose from the following options:
 - **Allow multiple Queries** - allows you to have multiple query windows open simultaneously. With this option selected, the New Query command opens a new tab in the Query window. The default for this setting is unchecked.
 - **Display this string for Null data fields** - enter the string you want displayed in place of Null data fields in your reports and imported/exported files. The default for this setting is "?".
 - Use a separate Answer window for
 - **Each Resultset** - opens a new Answer window for each new result set
 - **Each Query** - opens a new Answer window for each new query, but uses tabs within this window if the query returns multiple result sets. This is the default setting.
 - **Never** - directs all query results to display in a single tabbed Answer window

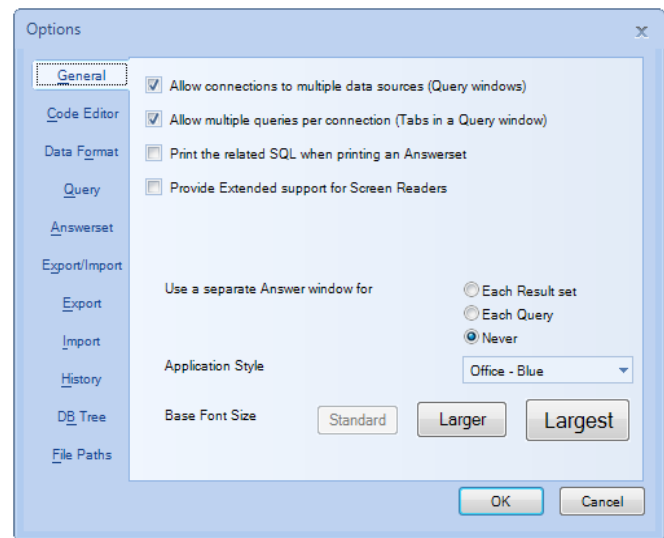
Teradata SQL Assistant General Options

General options (Tools > Options > General tab) that are available include:

- **Allow connections to multiple data sources.**
- **Allow multiple queries per connection** – allows you to have multiple query windows open simultaneously. New Queries are opened in new tabs.

Data Format options include:

- **Date format**
- **Display of NULL data values**
- **Decimal places to displace**



Connecting to Multiple Data Sources

You can connect to multiple data sources. The “Allow connections to multiple data sources” option must be checked with the General Options.

Each new data source appears in the Database Tree and opens a new query window with the data source name. To disconnect from one data source, click the Query window that is connected to the data source and click the disconnect icon.

The example on the facing page show two connects to two different systems (tdt5-1 and tdt6-1).

Teradata SQL Assistant Connecting to Multiple Data Sources

A separate query window is opened for each data source connection.

Connections have been made to two systems.

Multiple results for queries are shown via tabs.

History includes the Source name for queries.

The screenshot shows the Teradata SQL Assistant interface. On the left, the Database Explorer shows two connections: 'Factory (Teradata)' and 'Sandbox 15.00 sdiI9260'. The main window displays two query windows. The top window, 'Query (Factory)', contains the query: `SELECT * FROM PD.Employee SAMPLE 10;`. The bottom window, 'Query (Sandbox 15.00 sdiI9260)', contains two queries: `SELECT orderid, custid, ordertotal, orderdate, location FROM DS.Orders WHERE orderdate > DATE '2013-12-01';` and `SELECT orderid, custid, ordertotal, orderdate, location FROM DS.Orders_2014 WHERE orderdate > DATE '2014-12-01';`. Below the queries, the 'Answerset 1' tab shows the results of the first query from the Sandbox connection. The results are as follows:

	orderid	custid	ordertotal	orderdate	location
1	637,309	1,469	1,330.98	12/7/2014	1
2	661,196	1,496	1,331.27	12/31/2014	2

At the bottom, the 'History' tab shows a list of queries with columns: Date / Time, Source, Elapsed, Rows, and Result. The 'Source' column is circled in red. The history entries are:

	Date / Time	Source	Elapsed	Rows	Result
1	11/7/2014 10:59:49	Sandbox 15.00 sdiI9260	00:00:04	2522	
2	11/7/2014 09:12:34	Factory	00:00:02	1455	

Importing/Exporting Large Object Files

To import and/or export LOB (Large Object) files with SQL Assistant, you need to first make sure the “**Use Native Large Object Support**” option is set when defining ODBC driver for Teradata. This option was discussed earlier in this module.

This option is automatically selected starting with SQL Assistant 14.0.

Teradata SQL Assistant 12.0 Note

The following information is not needed with Teradata SQL Assistant 13 and later.

With Teradata SQL Assistant 12.0 and prior versions, to import a file larger than 10 MB into a BLOB or CLOB column, you need to enable this capability within SQL Assistant.

To enable importing of files larger than 10 MB into BLOB or CLOB columns:

1. Select **Tools > Options**, then select the **Export/Import** tab.
2. Select the **Import** tab.
3. Click in the **Maximum Size of an Imported data file** field.
4. Press the **Esc** key, then set the value to the size of the largest file you wish to load, up to a maximum of 9 digits.

If you do not press the Esc key before entering the data, you will be limited to a maximum of 7 digits in this field.

5. Click **OK**.

Note: This will be a temporary change. The next time you click **OK** on the Options screen the value will be reset to the first 7 digits of the number you had last set - for example, 50 MB (50,000,000) will become 5 MB (5,000,000).

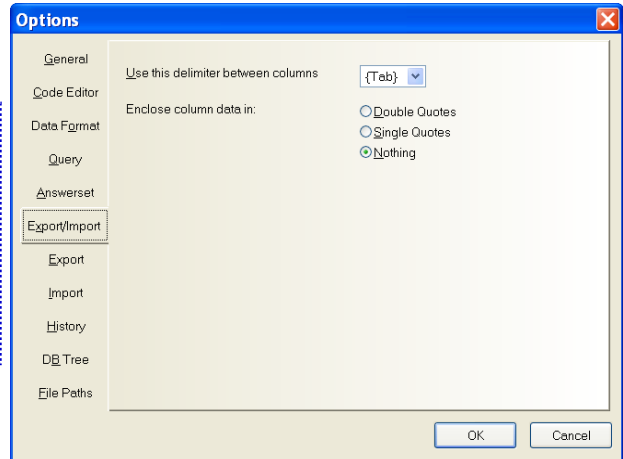
Importing/Exporting Large Object Files

Teradata SQL Studio Express and SQL Assistant both support Large Objects. Large objects come in two types:

- **Binary** – these columns may contain Pictures, Music, Word documents, PDF files, etc.
- **Text** – these columns contain text data such as Text, HTML, XML or Rich Text (RTF).

SQL Assistant > Tools > Options

- To import a LOB, create a data file that contains the names of the Large Objects.
- Use the Export/Import Options dialog to specify the field delimiter.
- The example in this module assumes the fields in the imported file are TAB separated.



Importing/Exporting Large Object Files

To import and/or export LOB (Large Object) files with SQL Assistant, you need to first make sure the “**Use Native Large Object Support**” option is set when defining ODBC driver for Teradata. This option was discussed earlier in this module.

To Import a LOB into Teradata

First, create a data file that contains the names of the LOB(s) to be imported. By default, the data file needs to be located in the same folder as the LOB.

Assume the data file to import from contains 4 fields that are <tab> separated.

Second, select the IMPORT DATA function and execute an Insert statement.

Example: **INSERT INTO TF_Materials VALUES (?, ?, ?, ?B);**

The parameter markers in this example are:

- ? The data for this parameter is read from the Import file. It is always a character string, and will be converted to a numeric value if necessary.
- ?B The data for this parameter resides in a file that is in the same directory as the Import file. The import file contains only the name of the file to be imported. The contents of the file are loaded as a binary image (e.g., BLOB). You can also use ?? in place of ?B.
- ?C The data for this parameter resides in a file that is in the same directory as the import file. The import file contains only the name of the file to be imported. Use this marker to load a text file into a CHAR or CLOB column.

Importing a LOB into Teradata

1. Create a data file that contains the name(s) of the LOB(s). This data file needs to be located in the same folder as the LOB.

TF Manual LOB.txt

1	TF Student Manual	PDF	TF v1620 Student Manual.pdf
2	TF Lab Workbook	PDF	TF v1620 Lab Workbook.pdf

The various fields are separated by tabs.

2. Within SQL Assistant, from the File menu, select the "Import Data" option to turn on the Import Data function.
3. Enter an INSERT statement within the Query window.
INSERT INTO TF_Materials VALUES (?, ?, ?, ?B);
4. In the dialog box that is displayed, choose the name of the file to import.
For example, enter or choose "TF Manual LOB.txt".
5. From the File menu, select the "Import Data" option to turn off the Import Data function.

Selecting from a Table with a LOB

To select from a table with a LOB, simply execute a SELECT statement. If LOB column is projected, then a dialog box is displayed to enter the file name for the LOB.

Note that multiple files that are exported will have sequential numbers added to the file name.

In the example on the facing page, the file name was specified as TF_Manual. Therefore, the manuals that will be created are named as follows:

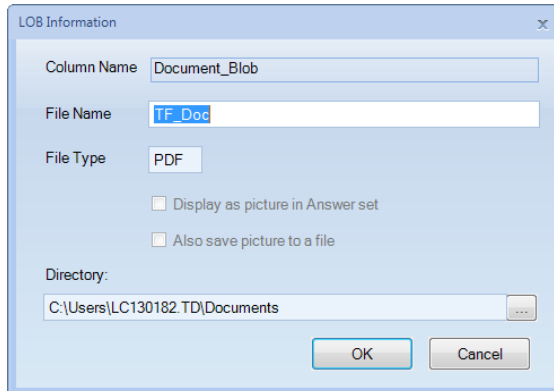
TF_Doc001.pdf
TF_Doc002.pdf
TF_Doc003.pdf
TF_Doc004.pdf

Selecting from a Table with a LOB

With SQL Assistant, enter the following query:

```
SELECT * FROM TF ORDER BY 1;
```

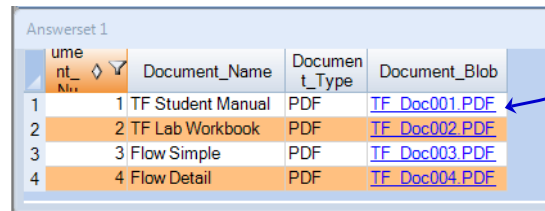
The following dialog box is displayed to represent the data files to export the LOBs into. Also specify the "File Type" as a known Microsoft file type extension.



LOB Information dialog box showing the following fields:

- Column Name: Document_Blob
- File Name: TF_Doc
- File Type: PDF
- ☐ Display as picture in Answer set
- ☐ Also save picture to a file
- Directory: C:\Users\LC130182.TD\Documents
- Buttons: OK, Cancel

The answer set window will include a link to exported data files.



Answerset 1 window showing a table with the following data:

Document_ID	Document_Name	Document_Type	Document_Blob
1	1 TF Student Manual	PDF	TF_Doc001.PDF
2	2 TF Lab Workbook	PDF	TF_Doc002.PDF
3	3 Flow Simple	PDF	TF_Doc003.PDF
4	4 Flow Detail	PDF	TF_Doc004.PDF

A blue arrow points from the text "The answer set window will include a link to exported data files." to the first row of the table.

Displaying a JPG Large Object

The “Display as picture ...” can be selected to display a JPG file within the answer set.

Optionally, the “Also save picture to a file” can be selected.

Note that large JPG files with display very large within the answer set window.

Teradata SQL Assistant Displaying a JPG Large Object

```
SELECT * FROM Photos ORDER BY 1;
```

LOB Information

Column Name: Photo

File Name: Photo

File Type: JPG

☒ Display as picture in Answer set

☐ Also save picture to a file

Directory: C:\Users\LC130182.TD\Documents

OK Cancel

Optionally, the "Display as picture ..." can be selected to display a JPG file within the answer set.

Teradata SQL Assistant - [Answerset 1]

Photo_Num	Photo_Name	Photo_Type	Photo
1	Raccoon	JPG	
2	Eagle	JPG	

Factory Line 1 100% 11:45

Teradata SQL Tools Summary

The Teradata Studio or the SQL Assistant tools can be of great value to you. The facing page summarizes some of the key features discussed in this module.

Characteristics of Teradata SQL Assistant include:

- Client-based graphical interface to submit SQL on the Teradata Database.
- Provides a Database Explorer Tree to easily view database objects.
- Supports DDL, DML and DCL commands.
 - SQL Query Builder feature allows for easy creation of SQL statements.
- Provides the retrieval of previously used queries (History).
- Saves information about previous query result sets.
- Provides both import and export capabilities to files on a PC.

Notes

Module R



Appendix R: Answers to Review Questions

This Appendix contains answers to the review questions for the course modules.

Teradata Proprietary and Confidential

Notes

Module 1: Review Question Answers

1. Which feature allows the Teradata Database to process enormous volumes of data quickly? ____
 - a. High availability software and hardware components
 - b. High performance servers from Intel
 - c. Proven Scalability
 - ☒ d. Parallelism
2. The Teradata Database is primarily a ____ .
 - a. Client
 - ☒ b. Server
3. Which choice represents a quadrillion bytes or a Petabyte (PB) of data? ____
 - a. 10^9
 - b. 10^{12}
 - ☒ c. 10^{15}
 - d. 10^{18}
4. In a relational table, the set of columns that uniquely identifies a row is the [Primary Key](#).

Module 2: Review Question Answers

1. What language is used to access a Teradata table?

SQL

2. What are five Teradata database objects?

Tables, views, macros, triggers, and stored procedures

3. What are four major components of the Teradata architecture?

PEs, AMPs, Vdisks, and Message Passing Layer

4. What are views?

Filter (or subset) of rows and columns or one or more tables

5. What are macros?

Predefined, stored set of SQL statements

Module 3: Review Question Answers

1. What are the two software elements that accompany an application on all client side environments?
CLI/ODBC/JDBC/.NET and TDP/MTDP
2. What is the purpose of the PE?
Parse, optimize, and dispatch queries
3. What is the purpose of the AMP?
Manage and retrieve data from disk storage
4. How many sessions can a PE support?
120

Match Quiz

- | | |
|-----------------------------------|---|
| <u>i</u> 1. CLI | a. Does Aggregating and Locking |
| <u>f</u> 2. MTDP | b. Validates SQL syntax |
| <u>e</u> 3. MOSI | c. Connects AMPs and PEs |
| <u>b</u> 4. Parser | d. Balances sessions across PEs |
| <u>a</u> 5. AMP | e. Provides Client side OS independence |
| <u>c</u> 6. Message Passing Layer | f. Library of Session Management Routines |
| <u>d</u> 7. TDP | g. PE S/W turns SQL into AMP steps |
| <u>g</u> 8. Optimizer | h. PE S/W sends plan steps to AMP |
| <u>h</u> 9. Dispatcher | i. Library of Teradata Service Routines |
| <u>j</u> 10. Parallelism | j. Foundation of Teradata architecture |

Module 4: Review Question Answers

True or False

- False 1. A database will always have tables.
- True 2. A user will always have a password.
- False 3. A user creating a subordinate user must give up some of his/her Perm Space.
- False 4. Creating tables requires the definition of at least 1 column and a Primary Index.
- True 5. The sum of all user and database max Perm Space will equal the total space on the system.
- False 6. The sum of all user and database Spool Space will equal the total space on the system.
- False 7. The limit of space for both Volatile and Global Temporary tables is based on the maximum TEMPORARY space for the user.
- False 8. Deleting a macro from a database reclaims Perm Space for the database.
9. Which two statements are TRUE about PERM space in Teradata 16.00? (choose two) ____
- a. PERM space cannot be dynamically modified.
 - ☒ b. The per/AMP limit of PERM space can be exceeded.
 - ☒ c. Tables, index subtables, and stored procedures use PERM space.
 - d. Maximum PERM space can be defined at the database or table level.
10. Which statement is TRUE about SPOOL space in Teradata 16.00? ____
- a. SPOOL space cannot be dynamically modified.
 - ☒ b. Maximum SPOOL space can be defined at the database or user level.
 - c. The SPOOL limit is dependent on the database limit where the table is located.
 - d. Maximum SPOOL space can be defined at a value greater than the creator's value.

Module 5: Review Question Answers

Answer the following either as True or False as these apply to Primary Indexes:

- ☒ True or ☐ False 1. UPI and NUPI equality value accesses are always a one-AMP operation.
- ☒ True or ☐ False 2. UPI and NUPI indexes allow NULL in a primary index column.
- ☐ True or ☒ False 3. UPI, NUPI, and NOPI tables allow duplicate rows in the table.
- ☒ True or ☐ False 4. Only UPI can be used as a Primary Key implementation.

Fill in the Blanks

- 5. The output of the hashing algorithm is called the row hash.
- 6. To determine the target AMP, the Message Passing Layer must lookup an entry in the Hash Map based on the hash bucket number.
- 7. A Row ID consists of a row hash plus a uniqueness value.
- 8. A uniqueness value is required to produce a unique Row ID because of hash synonyms and NUPI duplicates.
- 9. Once the target AMP has been determined for a PI search, the Master Index for that AMP is accessed to determine the cylinder that may hold the row.
- 10. The Cylinder Index points us to the address and length of the data block.

Module 6: Review Question Answers

Fill each box with either Yes, No, or the appropriate number.

	USI Access	NUSI Access	FTS
# AMPs	2	All	All
# rows	0 - 1	0 - n	0 - all
Parallel Operation	No	Yes	Yes
Uses Hash Maps	Yes	No	No
Uses Separate Sub-table	Yes	Yes	No
Reads all data blocks of table	No	No	Yes

Module 7: Review Question Answers

1. Which data interface is available with Teradata Studio?
 - a. CLIV2
 - ☒ b. JDBC
 - c. ODBC
 - d. Teradata .Net
2. Which piece of query information is not available in the History Window for Teradata Studio?
 - a. User name
 - ☒ b. Query band
 - c. Elapsed time
 - d. Data source name
 - e. Number of rows returned
3. What is a technique to execute multiple statements as a multi-statement request in Teradata Studio?
Use menu SQL > Execute All

Module 8: Review Question Answers

Complete the following.

1. Each AMP has its own memory and manages its own disk space and executes independently of other AMPs. This is referred to as a shared nothing architecture.
2. The software component that allows the Teradata Database to execute in different operating system environments is the PDE.
3. A physical message passing interconnect is called the Infiniband or BYNET.
4. A clique provides protection from a node failure.
5. If a node fails, all vprocs will migrate to the remaining nodes in the clique. This feature is referred to as vproc migration.
6. The SWS provides a single point of operational control for Teradata MPP systems.
7. A TPA node is part of a system configuration, is connected to the BYNET, and executes the Teradata Database software.
8. A TMS node is part of a system configuration, may connect to the BYNET, and is used to execute application software other than Teradata Database software.
9. A HSN node is part of a system configuration, connects to the BYNET, and is used as a spare node in the event of a node failure.

Module 9: Review Question Answers

1. What is the WAL Depot used for?

- a. UNDO Rows
- b. New data blocks
- c. Master Index updates
- ☒ d. Write-in-place data blocks

2. Which two are placed into the WAL Log?

- ☒ a. REDO Rows
- ☒ b. UNDO Rows
- c. New data blocks
- d. Write-in-place data blocks

Match the item to a lettered description.

<u>f</u>	1. Database locks	a. Provides for TXN rollback in case of failure
<u>c</u>	2. Table locks	b. Teradata Backup and Recovery applications
<u>g</u>	3. Row Hash locks	c. Protects all rows of a table
<u>h</u>	4. FALLBACK	d. Logs changed rows for down AMP
<u>j</u>	5. Cluster	e. Provides for recovery to a point in time
<u>d</u>	6. Recovery journal	f. Applies to all tables and views within
<u>a</u>	7. Transient journal	g. Lowest level of protection granularity
<u>b</u>	8. NetBackup/Tivoli	h. Protects tables from AMP failure
<u>e</u>	9. Permanent journal	i. Protects database from a physical drive failure
<u>i</u>	10. Disk Array	j. Group of AMPs used by Fallback

Module 10: Review Question Answers

1. Which two are accessed in FSG cache memory? (choose 2)
 - a. Hash maps
 - ☒ b. Master Indexes
 - ☒ c. Cylinder Indexes
 - d. Data Dictionary Cache
2. What are two benefits of Cylinder Index Segregation in Teradata 15.10? (choose 2)
 - a. Allows for larger cylinders
 - ☒ b. Improves Teradata reboot performance
 - ☒ c. Allows more user data to be stored in a cylinder
 - d. Reduces the number of required cylinder indexes
3. When are multiple partitions in a LUN used in a Teradata configuration? (choose 2)
 - a. SSD drives with a 2850
 - ☒ b. HDD drives with a 2850
 - ☒ c. SSDs used with a 6800
 - d. HDDs used with a 6800
4. Describe the fundamental relationship between Linux, logical units, and disk array controllers.
[Linux uses logical unit names to communicate with disk array controllers.](#)
5. Describe the fundamental relationship between AMPs, Vdisks, Pdisks, Partitions, and LUNs.
[Each AMP is assigned to a Vdisk which is made up of 1 or more Pdisks. A Pdisk is assigned to a partition in a logical unit or LUN.](#)

Module 11: Order of Preference Exercise

Assuming the system default is "Primary Index", show the indexes that are created as a result of the DDL.

```
CREATE TABLE table_1
(col1 INTEGER NOT NULL UNIQUE           col1 = USI
,col2 INTEGER NOT NULL PRIMARY KEY);    col2 = UPI
```

```
CREATE TABLE table_2
(col1 INTEGER NOT NULL PRIMARY KEY      col1 = USI
,col2 INTEGER)                          col2 = NUPI
PRIMARY INDEX (col2);
```

```
CREATE TABLE table_3
(col1 INTEGER                           col1 = NUPI
,col2 INTEGER NOT NULL);                col2 =
```

```
CREATE TABLE table_4
(col1 INTEGER NOT NULL                  col1 =
,col2 INTEGER NOT NULL                  col2 =
,col3 INTEGER NOT NULL UNIQUE           col3 = USI
,CONSTRAINT pk1 PRIMARY KEY (col1,col2)); (col1,col2) = UPI
```

```
CREATE TABLE table_5
(col1 INTEGER NOT NULL                  col1 =
,col2 INTEGER NOT NULL                  col2 =
,col3 INTEGER NOT NULL UNIQUE           col3 = UPI
,CONSTRAINT uniq1 UNIQUE (col1,col2));  (col1,col2) = USI
```

UPI =	Unique Primary Index
NUPI =	Non Unique Primary Index
USI =	Unique Secondary Index

Module 11: Exercise 2 – Choosing PI Candidates

		ENTITY 1					
500,000,000 Rows		A	B	C	D	E	F
PK/FK	PK,UA						
Distinct Values		500M	475M	1.5M	1.25M	200M	5M
Max Rows/Value		1	2	400	450	3	110
Max Rows/NULL		0	0	0	0	10M	0
Typical Rows/Value		1	1	325	350	2	90
PI/SI		UPI	NUPI	NUPI	NUPI		NUPI
Collect Statistics (Y/N)							

Module 11: Exercise 2 – Choosing PI Candidates

		ENTITY 2						
50,000,000 Rows		G	H	I	J	K	L	
PK/FK		PK,SA						
Distinct Values		50M	500K	45M	12	50	900K	
Max Rows/Value		1	200	2	5M	1.2M	60	
Max Rows/NULL		0	0	450K	0	0	0	
Typical Rows/Value		1	100	1	4M	950K	50	
		UPI	NUPI				NUPI	
PI/SI								
Collect Statistics (Y/N)								

Module 11: Exercise 2 – Choosing PI Candidates

		ASSOCIATIVE 1				
1.5 Billion Rows		A	G	R	S	
	PK/FK	PK				
		FK	FK,SA			
	Distinct Values	490M	50M	75K	4M	
	Max Rows/Value	5	40	21K	400	
	Max Rows/NULL	0	0	0	0	
	Typical Rows/Value	3	25	19K	350	
	PI/SI	UPI				
		NUPI	NUPI	NUPI?	NUPI	
Collect Statistics (Y/N)						

Module 11: Exercise 2 – Choosing PI Candidates

		ASSOCIATIVE 2				
500,000,000 Rows		A	M	G	T	U
PK/FK		PK				
		FK		FK		
	Distinct Values	250M		50M	2.8M	750
	Max Rows/Value	3		150	180	675K
	Max Rows/NULL	0		0	0	0
	Typical Rows/Value	1		8	170	500K
PI/SI		UPI				
		NUPI	NUPI	NUPI		
Collect Statistics (Y/N)						

Module 11: Exercise 2 – Choosing PI Candidates

		HISTORY				
3.65 Billion Rows		A	DATE	D	E	F
PK/FK	PK					
	FK	SA				
Distinct Values	500M	730	N/A	N/A	N/A	
Max Rows/Value	18	5.5M	N/A	N/A	N/A	
Max Rows/NULL	0	0	N/A	N/A	N/A	
Typical Rows/Value	3	4.5M	N/A	N/A	N/A	
PI/SI	UPI					
	NUPI					
Collect Statistics (Y/N)						

Module 11: Review Question Answers

1. Which trade-off must be balanced to make the best choice for a primary index? ____
 - a. Access and volatility
 - b. Access and block size
 - c. Block size and volatility
 - ☒ d. Access and distribution

2. When volatility is considered as one of the Primary Index choice criteria, what is analyzed? ____
 - a. Degree of uniqueness
 - ☒ b. How often the data values will change
 - c. How often the fixed length rows will change
 - d. How frequently the column is used for access

3. To optimize the use of disk space, the designer should choose a primary index that _____.
 - a. is non-unique
 - b. consists of one column
 - ☒ c. is unique or nearly unique
 - d. consists of multiple columns
 - e. has fewer distinct values than AMPs

Module 11: Review Question Answers

4. For NoPI tables, what are 2 ways in which the Random Generator is executed?
 - ☒ a. At the PE level for ad hoc SQL requests
 - b. At the TPT Stream client level for array insert operations
 - ☒ c. At the PE level with TPT Stream for array insert operations
 - d. At the AMP level for INSERT-SELECT into an empty NoPI table

5. Assume DBSControl flag #53 (Primary Index Default) is set to N (No Primary Index), which two indexes are created for TableX given the following DDL command?


```
CREATE TABLE TableX
      (col1    INTEGER NOT NULL UNIQUE
      ,col2    CHAR(10) NOT NULL PRIMARY KEY
      ,col3    CHAR(80));
```

 - a. col1 will be a UPI
 - ☒ b. col1 will be a USI
 - ☒ c. col2 will be a UPI
 - d. col2 will be a USI

6. Which two options are permitted for NoPI tables?
 - ☒ a. Fallback
 - b. TPT Update
 - c. Hash Index
 - ☒ d. BLOBs and CLOBs

Module 12: Review Question Answers

1. The Row Hash for a PI value of 824 is the same for the data types of INTEGER and DECIMAL(18,0). True or False. True
2. The first 20 bits of the Row Hash is referred to as the hash bucket number.
3. The Hash Map consists of entries or buckets which identify an AMP number for the Row Hash.
4. The Current Configuration Primary Hash Map is used to locate the AMP to locate/store a row based on PI value.
5. The RECONFIG utility is used to redistribute rows to a new system configuration with more AMPs.
6. When creating a new table, the Unique Value of a Table ID comes from the dictionary table named DBC. Next.
7. The Row ID consists of the Row Hash and the Uniqueness Value.
8. The Master Index contains a Cylinder Index Descriptor (CID) for each allocated Cylinder.
9. The Cylinder Index contains an entry for each data block in the cylinder.
10. The Row Reference Array consists of a set of pointers to the data rows in data block.
11. In TD 14.0, the maximum block size is approximately 127.5 KB and the maximum row size is approximately 64 KB.
12. In TD 16.00, the maximum block size is approximately 1 MB and the maximum row size is approximately 1 MB.

Module 13: Review Question Answers

1. When Teradata INSERTs a new row into a table, it first goes to the _____ to locate the proper cylinder for the new row.
 - a. Cylinder Index
 - b. Fallback AMP
 - c. Free Cylinder List
 - ☒ d. Master Index
2. When a new block is needed, the File System searches the Free Block List looking for the first Free Block whose size is equal to, or greater than the new block's requirement. It does not have to be an exact match.
 - ☒ a. True
 - b. False
3. Name the condition which occurs when there is no block on the Free Block List with enough sectors to accommodate the additional data during an INSERT or UPDATE.
 - a. Mini Cylinder Pack
 - b. Cylinder Migrate to a new cylinder
 - c. Cylinder Migrate to an adjacent cylinder
 - ☒ d. Cylinder Full
4. The _____ parameter can be set to control how completely cylinders are filled during loading and PackDisk.
 - ☒ a. FreeSpace
 - b. PermDBSize
 - c. DataBlockSize
 - d. PermDBAllocUnit

Module 13: Review Question Answers

5. Number the following steps in sequence from 1 to 6 that the File System software will attempt to perform in order to insert a new row into an existing data block.
- 5 Perform a Cylinder Migrate operation to an adjacent cylinder
 - 1 Simply insert the row into data block if enough contiguous free bytes in the block
 - 4 Perform a Block split
 - 6 Perform a Cylinder Migrate operation to a new cylinder
 - 2 Defragment the block and insert the row
 - 3 Expand or grow the block to hold the row
6. As part of a cylinder full condition, if the number of free sectors within a cylinder is greater than 25%, what operation will Teradata perform in the background? [Cylinder Defragmentation](#)
7. If the number of free cylinders falls below a minimum threshold, what operation will Teradata perform in the background? [Mini-Cylpack](#)

Module 14: Review Question Answers

1. The Row ID for a partitioned table consists of Partition # + Row Hash + Uniqueness Value.
2. The Row Key consists of the Partition # + Row Hash.
3. With a non-partitioned table, the partition number defaults to 0.
4. True or False. For a partitioned table, the partition number and the Row Hash are both used by the Message Passing Layer to determine which AMP(s) should receive the request.
5. Which two options apply to the RANGE_N expression in a partitioning expression? ____ ____
 - a. Ranges can be listed in descending order
 - ☒ b. Allows use of NO RANGE OR UNKNOWN option
 - c. Partitioning column must be part of the Primary Index
 - ☒ d. Maximum of 9.223 quintillion partitions with Teradata 16.20
6. With a populated table, which two actions are allowed with the ALTER TABLE command. ____ ____
 - a. Drop all of the ranges
 - ☒ b. Add or drop ranges from the partition "ends"
 - c. Change the columns that comprise the primary index
 - ☒ d. Add or drop special partitions (NO RANGE, UNKNOWN)
7. What are two advantages of partitioning a table? ____ ____
 - ☒ a. Fast delete of rows in partitions
 - b. Fewer AMPs are involved when accessing data
 - ☒ c. Range queries can be executed without doing an all-rows scan
 - d. Faster access for a PI value access than for a non-partitioned table

Module 14: Review Question Answers

Given this CREATE TABLE statement, answer the following questions.

```
CREATE TABLE Orders
  (Orderid          INTEGER NOT NULL,
   Custid           INTEGER NOT NULL,
   Orderstatus      CHAR(1),
   Ordertotal       DECIMAL(9,2) NOT NULL,
   Orderdate        DATE NOT NULL FORMAT 'YYYY-MM-DD',
   Orderpriority    SMALLINT,
   Clerk            CHAR(16),
   Shippriority     SMALLINT,
   Ordercomment     VARCHAR(80))
PRIMARY INDEX (Orderid)
PARTITION BY RANGE_N (Orderdate BETWEEN
  DATE '2014-01-01' AND DATE '2020-12-31' EACH INTERVAL '1' DAY)
UNIQUE INDEX (Orderid);
```

8. What is the name of partitioning column? Orderdate
9. What is the time period for each partition? 1 Day
10. What are two reasons that a USI is specified instead of defining Orderid as a UPI? _____
 - a. This is a coding style choice.
 - b. You cannot have a UPI when using a partitioned primary index.
 - ☒ c. To improve the performance of value access queries for a specific Orderid.
 - ☒ d. You cannot have a UPI if the partitioning column is not part of the primary index.

Module 15: Review Question Answers

1. Which choice is NOT allowed with a column partitioned table in Teradata 16.00?
 - a. SET table
 - b. NoPI table
 - c. MULTISSET table
 - d. Table with a NUPI
2. What are two benefits of Column Partitioning? (choose two)
 - a. Reduced I/O
 - b. Reduced CPU
 - c. Reduced disk space usage
 - d. Reduced tactical query response times
3. True or False? Deleting a row in a column partitioned table will reclaim table space.
4. True or False? In a multilevel columnar table, only one level may be defined with COLUMN.
5. True or False? The preferred way to load a columnar table is using INSERT/SELECT.

Module 16: Review Question Answers

1. Because the row is hash-distributed on different columns, the subtable row will typically land on an AMP other than the one containing the data row. This index would be:
 - a. UPI or NUPI
 - ☒ b. USI
 - c. NUSI
 - d. None of the above
2. The Teradata DBS hashes the value and uses the Row Hash to find the desired rows. This is always a one-AMP operation. This index would be:
 - ☒ a. UPI or NUPI
 - b. USI
 - c. NUSI
 - d. None of the above
3. _____ is a process that determines common Row IDs between multiple NUSI values by a process of intersection.
 - ☒ a. NUSI Bit Mapping
 - b. Dual NUSI Access
 - c. Full Table Scan
 - d. NUSI Read
4. If aggregation is performed on a NUSI column, the Optimizer accesses the NUSI subtable and returns the result without accessing the base table. This is referred to as:
 - a. NUSI bit mapping
 - b. Full table scan
 - c. Dual NUSI access
 - ☒ d. Covering Index

Module 17: Exercise 3 – Choosing SI Candidates

ENTITY 1							
500,000,000 Rows	A	B	C	D	E	F	
PK/FK	PK,UA						
Value Access	250K	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Distinct Values	500M	475M	1.5M	1.25M	200M	5M	
Max Rows/Value	1	2	400	450	3	110	
Max Rows/NULL	0	0	0	0	10M	0	
Typical Rows/Value	1	1	325	350	2	90	
Change Rating	0	2	1	1	1	1	
PI/SI	UPI	NUPI	NUPI	NUPI		NUPI	
	USI						
Collect Statistics (Y/N)							

Module 17: Exercise 3 – Choosing SI Candidates

ENTITY 2							
50,000,000 Rows	G	H	I	J	K	L	
PK/FK	PK,SA						
Value Access	5K	365	52	12	0	0	
Range Access	12	0	0	0	0	2.6K	
Distinct Values	50M	500K	45M	12	50	900K	
Max Rows/Value	1	200	2	5M	1.2M	60	
Max Rows/NULL	0	0	450K	0	0	0	
Typical Rows/Value	1	100	1	4M	950K	50	
Change Rating	0	0	9	1	2	0	
PI/SI	UPI	NUPI				NUPI	
	USI	NUSI				VONUSI	
Collect Statistics (Y/N)							

Module 17: Exercise 3 – Choosing SI Candidates

5,000,000 Rows	DEPENDENT						
	A	M	N	O	P	Q	
PK/FK	PK				NN,ND		
	FK	SA					
Value Access	0	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Distinct Values	2M	50	90K	3M	5M	2M	
Max Rows/Value	4	200K	75	2	1	5	
Max Rows/NULL	0	0	0	390K	0	1M	
Typical Rows/Value	1	60K	50	1	1	1	
Change Rating	0	0	3	1	0	1	
PI/SI	UPI				UPI		
	NUPI		NUPI				
	USI				USI		
Collect Statistics (Y/N)							

Module 17: Exercise 3 – Choosing SI Candidates

ASSOCIATIVE 1					
1.5 Billion Rows	A	G	R	S	
PK/FK	PK				
	FK	FK,SA			
Value Access	26K	0	0	0	
Range Access	0	0	0	0	
Distinct Values	490M	50M	75K	4M	
Max Rows/Value	5	40	21K	400	
Max Rows/NULL	0	0	0	0	
Typical Rows/Value	3	25	19K	350	
Change Rating	0	0	0	0	
PI/SI	UPI				
	NUPI	NUPI	NUPI?	NUPI	
	USI				
	NUSI				
Collect Statistics (Y/N)					

Module 17: Exercise 3 – Choosing SI Candidates

ASSOCIATIVE 2						
500,000,000 Rows	A	M	G	T	U	
PK/FK	PK					
	FK		FK			
Value Access	0		0	0	0	
Range Access	0		0	0	0	
Distinct Values	250M		50M	2.8M	750	
Max Rows/Value	3		150	180	675K	
Max Rows/NULL	0		0	0	0	
Typical Rows/Value	1		8	170	500K	
Change Rating	0		0	0	0	
PI/SI	UPI					
	NUPI		NUPI	NUPI		
	USI					
Collect Statistics (Y/N)						

Module 17: Exercise 3 – Choosing SI Candidates

		HISTORY				
3.65 Billion Rows	A	DATE	D	E	F	
PK/FK	PK					
	FK	SA				
Value Access	10M	26K	0	0	0	
Range Access	0	100K	0	0	0	
Distinct Values	500M	730	N/A	N/A	N/A	
Max Rows/Value	18	5.5M	N/A	N/A	N/A	
Max Rows/NULL	0	0	N/A	N/A	N/A	
Typical Rows/Value	3	4.5M	N/A	N/A	N/A	
Change Rating	0	0	N/A	N/A	N/A	
PI/SI	UPI					
	NUPI					
	USI					
	NUSI	VONUSI				
Collect Statistics (Y/N)						

Module 17: Review Question Answers

1. With a NUPI, a technique to avoid a duplicate row check is to _____.
 - a. use set tables
 - b. use NOT NULL on the NUPI column(s)
 - ☒ c. create the table as a MULTiset table
 - d. compare data values byte-by-byte within a Row Hash to ensure uniqueness
2. Which type of usage normally applies to a USI? _____.
 - a. Range access
 - b. NOT condition
 - ☒ c. Equality value access
 - d. Inequality value access
3. Which two types of usage normally apply to a composite NUSI that is hash-ordered? _____
 - ☒ a. Covering index
 - ☒ b. Equality value access
 - c. Inequality value access
 - d. Non-covering range access
4. Which situation requires placing before-images of rows into the Transient Journal? _____.
 - a. loading a table with TPT Load
 - b. DELETE all the rows in a table
 - ☒ c. UPDATE all the rows in a table
 - d. INSERT/SELECT into an empty table

Module 18: Review Question Answers

1. Which choice can be used with the COMPRESS option?
 - a. Identity column
 - b. Non-unique Primary Index
 - c. USI as PK with Standard RI
 - ☒ d. Non-unique Secondary Index
2. Which section of a row identifies the starting location of variable length column data and is present only if variable length columns without compression are declared?
 - a. Presence Bits
 - ☒ b. Column Offsets
 - c. VARCHAR Columns
 - d. Uncompressed Columns
3. How can you override the default that a column with a NULL value will require row space?
 - a. Use the NOT NULL option on the column as part of the CREATE TABLE statement.
 - b. Set the user's default so columns will default to COMPRESS when creating a table.
 - ☒ c. Use the COMPRESS option on the column as part of the CREATE TABLE statement.
 - d. Use the DEFAULT NULL option on the column as part of the CREATE TABLE statement.
4. What is the minimum space the table headers will take for a 6-column table on a 10 AMP system?
 - a. 1024 bytes
 - b. 4096 bytes
 - c. 5120 bytes
 - ☒ d. 10240 bytes
5. What DD view is used to get sizing information about tables? [DBC.TableSizeV](#)

Module 19: Review Question Answers

1. Where are statistics stored in Teradata 16.20 when you COLLECT STATISTICS on a NUPI? _____
 - ☒ a. DBC.StatsTbl
 - b. DBC.Indexes
 - c. DBC.TVFields
 - d. Data Dictionary cache
2. Random AMP sample statistics are stored in _____.
 - a. DBC.Indexes
 - b. DBC.TVFields
 - ☒ c. Data Dictionary cache
 - d. Request-to-Steps cache
3. Which statement is used to display statistical detail information (e.g., summary section, high bias values, and intervals)? _____
 - a. HELP INDEX
 - b. SHOW TABLE
 - c. HELP STATISTICS
 - ☒ d. SHOW STATISTICS
4. Which statement is used to display extrapolated statistics used by the optimizer? _____
 - a. HELP INDEX
 - b. HELP STATISTICS
 - ☒ c. HELP CURRENT STATISTICS
 - d. SHOW CURRENT STATISTICS

Module 20: Review Question Answers

Fill in the blanks.

1. [Parallel](#) steps are multi-AMP processing steps that are numbered but execute at the same time.
2. The primary way to help the Optimizer make the best choices and ensure the most accurate EXPLAIN output is to [Collect Statistics](#) on appropriate indexes and columns.
3. An EXPLAIN plan will indicate “estimated with [High](#) confidence” when a value for an index is provided to retrieve the data and the index has collected statistics.

Module 21: Review Question Answers

1. The best way to be sure what type of Join will occur is to use the EXPLAIN facility.
 - ☒ a. True
 - ☐ b. False

2. When two tables are to be Merge Joined, which is the best case of the following scenarios :
 - ☒ a. The Join column(s) are the Primary Index of both tables.
 - ☐ b. The Join column(s) are not a Primary Index of either table.
 - ☐ c. The Join column(s) are the Primary Index of one of the tables.
 - ☐ d. The Join column(s) are a Unique Secondary Index on both of the tables.

3. Match the four join plans with its correct description.

<u>c</u> Product Join	a. Most efficient type of Join; the only type of join that doesn't always use all of the AMPs. The number of AMPs involved will vary.
<u>d</u> Merge Join	b. Based on set subtraction; used for finding rows that don't have a matching row in the other table. Queries with the NOT IN and EXCEPT operator lead to this type of join.
<u>a</u> Nested Join	c. Every qualifying row of one table is compared to every qualifying row in the other table. Rows that match on their WHERE conditions are then saved.
<u>b</u> Exclusion Join	d. Commonly done when the join condition is based on equality. Efficient because every row is not compared with every row in other table.

Module 21: Review Question Answers

Fill in the blanks.

4. When joining two row partitioned tables that are not partitioned in the same manner, a technique available to the optimizer is referred to as the sliding window.
5. A direct merge join of two row partitioned tables when the tables have the same PI and identical partitioning expressions is referred to as a rowkey - based merge join.
6. The term partition elimination refers to an automatic optimization in which the optimizer determines, based on query conditions, that some partitions can be skipped.

Module 22: Exercise 4 – Making Final Index Choices

ENTITY 1							
500,000,000 Rows	A	B	C	D	E	F	
PK/FK	PK,UA						
Value Access	250K	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Join Access	10M	0	0	0	0	0	
Join Rows	10M	0	0	0	0	0	
Distinct Values	500M	475M	1.5M	1.25M	200M	5M	
Max Rows/Value	1	2	400	450	3	110	
Max Rows/NULL	0	0	0	0	10M	0	
Typical Rows/Value	1	1	325	350	2	90	
Change Rating	0	2	1	1	1	1	
PI/SI	UPI		UPI	UPI		UPI	
	USI						
Collect Statistics (Y/N)							
	Y*						

Y* – For large tables, COLLECT SUMMARY will suffice for a UPI since no other columns will have collected statistics.

Module 22: Exercise 4 – Making Final Index Choices

ENTITY 2							
50,000,000 Rows	G	H	I	J	K	L	
PK/FK	PK,SA						
Value Access	5K	365	52	12	0	0	
Range Access	12	0	0	0	0	2.6K	
Join Access	100M	0	0	0	0	0	
Join Rows	100M	0	0	0	0	0	
Distinct Values	50M	500K	45M	12	50	900K	
Max Rows/Value	1	200	2	5M	1.2M	60	
Max Rows/NULL	0	0	450K	0	0	0	
Typical Rows/Value	1	100	1	4M	950K	50	
Change Rating	0	0	9	1	2	0	
PI/SI	UPI	NUPI				NUPI	
	USI	NUSI				VONUSI	
Collect Statistics (Y/N)		Y	Y*	Y*		Y	

Y* – Possibly only collect statistics (or SAMPLE) when needed. Column I may be too volatile for stats.

Module 22: Exercise 4 – Making Final Index Choices

5,000,000 Rows	DEPENDENT						
	A	M	N	O	P	Q	
PK/FK	PK				NN,ND		
	FK	SA					
Value Access	0	0	0	0	0	0	
Range Access	0	0	0	0	0	0	
Join Access	700K	0	0	0	0	0	
Join Rows	1M	0	0	0	0	0	
Distinct Values	2M	50	90K	3M	5M	2M	
Max Rows/Value	4	200K	75	2	1	5	
Max Rows/NULL	0	0	0	390K	0	1M	
Typical Rows/Value	1	60K	50	1	1	1	
Change Rating	0	0	3	1	0	1	
PI/SI	UPI				UPI		
	NUPI						
	USI				USI		
Collect Statistics (Y/N)							
	Y						

Module 22: Exercise 4 – Making Final Index Choices

ASSOCIATIVE 1					
1.5 Billion Rows	A	G	R	S	
PK/FK	PK				
	FK	FK,SA			
Value Access	26K	0	0	0	
Range Access	0	0	0	0	
Join Access	0	8M	0	0	
Join Rows	0	300M	0	0	
Distinct Values	490M	50M	75K	4M	
Max Rows/Value	5	40	21K	400	
Max Rows/NULL	0	0	0	0	
Typical Rows/Value	3	25	19K	350	
Change Rating	0	0	0	0	
PI/SI	UPI				
	NUPI	NUPI	NUPI2	NUPI	
	USI				
Collect Statistics (Y/N)	NUSI				
	Y	Y			

Module 22: Exercise 4 – Making Final Index Choices

ASSOCIATIVE 2						
500,000,000 Rows	A	M	G	T	U	
PK/FK	PK					
	FK		FK			
Value Access	0		0	0	0	
Range Access	0		0	0	0	
Join Access	7M		250K	0	0	
Join Rows	800M		100M	0	0	
Distinct Values	250M		50M	2.8M	750	
Max Rows/Value	3		150	180	675K	
Max Rows/NULL	0		0	0	0	
Typical Rows/Value	1		8	170	500K	
Change Rating	0		0	0	0	
PI/SI	UPI					
	NUPI		NUPI	NUPI		
	USI					
Collect Statistics (Y/N)	Y		Y			

What additional index choices would be available for the column G?

Note: Options that may improve the join performance on G: 1) Create a single table join index on G or 2) Create a covered NUSI on G (order by hash on G).

Module 22: Exercise 4 – Making Final Index Choices

3.65 Billion Rows	HISTORY					
	A	DATE	D	E	F	
PK/FK	PK					
	FK	SA				
Value Access	10M	26K	0	0	0	
Range Access	0	100K	0	0	0	
Join Access	800M	0	0	0	0	
Join Rows	2.4B	0	0	0	0	
Distinct Values	500M	730	N/A	N/A	N/A	
Max Rows/Value	18	5.5M	N/A	N/A	N/A	
Max Rows/NULL	0	0	N/A	N/A	N/A	
Typical Rows/Value	3	4.5M	N/A	N/A	N/A	
Change Rating	0	0	N/A	N/A	N/A	
PI/SI	UPI					
	NUPI					
	USI					
	NUSI VONUSI					
Collect Statistics (Y/N)	Y	Y				

What additional index choices would be available for the DATE column?

Possibly create a partitioned table partitioned by DATE. If so, maybe leave the NUSI on A for value access.

Module 22: Review Question Answers

1. When do Cartesian Product Joins generally occur?
 - ☒ a. When an error is made in coding the SQL query.
 - b. When a column appears in an equality join constraint.
 - c. When the join column is the primary index of both tables.
 - d. When the join column is the primary index in only one of the tables.
2. Which two types of joins will result in a Join Condition of (1=1) in EXPLAIN output? ____ ____
 - a. Hash
 - b. Merge
 - ☒ c. Nested
 - d. Exclusion merge
 - ☒ e. Cartesian product
3. Which demographic represents how often all known transactions access rows from the table through a join on this column?
 - a. Value Access Frequency
 - b. Range Access Rows
 - c. Range Access Frequency
 - d. Join Access Rows
 - ☒ e. Join Access Frequency

Module 23: Review Question Answers

Check the box if the attribute applies to the index.

	Compressed Join Index Syntax	Non- Compressed Join Index Syntax	Aggregate Join Index	Sparse Join Index	Hash Index
May be created on a single table	✓	✓	✓	✓	✓
May be created on multiple tables	✓	✓	✓	✓	
Requires the use of SUM, COUNT or MIN/MAX functions			✓		
Requires a WHERE condition to limit rows stored in the index				✓	
Automatically updated as base table rows are inserted or updated	✓	✓	✓	✓	✓
Automatically includes the base table PI value as part of the index					✓

Module 24: Review Question Answers

1. Which BTEQ setting controls Teradata vs. ANSI mode? [.SET SESSION TRANSACTION](#)
2. Which commands will not work in ANSI mode? [BT and ET](#)
3. True or False. A syntax error for an individual request in ANSI (or COMMIT) mode causes the entire transaction to be rolled back.
4. True or False. A syntax error for an individual request in Teradata (or BTET) mode causes the entire transaction to be rolled back.
5. True or False. Logging off during an explicit transaction without either a COMMIT or ET will always result in a ROLLBACK.
6. Where does a Volatile Temporary table get its space from? [SPOOL](#)
7. Where does a Global Temporary table get its space from? [TEMPORARY](#)

Module 25: Review Question Answers

Answer True or False.

1. True or False. With TPT Update, you can import and export data.
2. True or False. In Teradata mode, an implicit DELETE ALL transaction does not use the Transient Journal to store before-images of deleted rows.
3. True or False. An INSERT/SELECT of 1,000,000 rows into an empty table is only slightly faster than an INSERT/SELECT of 1,000,000 rows into a table with 1 row.

Match the Teradata Parallel Transporter operator with the corresponding Teradata utility.

- | | |
|--------------------|---------------|
| 1. <u>A</u> UPDATE | A. MultiLoad |
| 2. <u>D</u> STREAM | B. FastLoad |
| 3. <u>B</u> LOAD | C. FastExport |
| 4. <u>C</u> EXPORT | D. TPump |

Module S



Appendix S: Solutions to Lab Exercises

**This Appendix contains possible solutions
to the lab exercises.**

Teradata Proprietary and Confidential

Notes

Lab Solutions for Lab 7-1 (cont.)

Lab Exercise 7-1 Solutions

6. Using the Query Window, execute the following query.

```
CREATE TABLE Orders_1Q2019 AS Orders_2019 WITH NO DATA;
```

7. Execute the following query to populate Orders_1Q2019 with orders for January of 2019.

```
INSERT INTO Orders_1Q2019 SELECT * FROM DS.Orders_2019  
WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How many rows are in Orders_1Q2019? 8000

8. Using the History window, recall the query from step #7 in a new SQL editor and modify it to add orders for February and March of 2019 into the Orders_1Q2019 table.

```
INSERT INTO Orders_1Q2019 SELECT * FROM DS.Orders_2019 WHERE orderdate BETWEEN  
DATE '2019-02-01' AND DATE '2019-03-31';
```

How many rows are now in Orders_1Q2019? 24,000

```
SELECT COUNT(*) FROM Orders_1Q2019;
```

9. Execute the following query by using the drag and drop object feature of the SQL tool you are using.

```
SELECT custid, SUM (ordertotal)  
FROM Orders_1Q2019  
GROUP BY 1  
ORDER BY 1;
```

What is the SUM of the orders (ordertotal) in the 1Q2019 for custid = 21012? 125,180.67

Lab Solutions for Lab 7-1 (cont.)

10. Create a view (that utilizes an Access Lock) named "Orders_Jan2019_v" on the Orders_1Q2019 table that includes the these columns (orderid, custid, ordertotal, orderdate) and only includes orders for the January of 2019.

```
CREATE VIEW Orders_Jan2019_v
AS LOCKING ROW FOR ACCESS
SELECT orderid, custid, ordertotal, orderdate FROM Orders_1Q2019
WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

SELECT the total count of orders from "Orders_Jan2019_v".

```
SELECT COUNT(*) FROM Orders_Jan2019_v;
```

How many orders are available using this view? 8000

11. Create a parameterized macro named "Orderid_m" which accepts an orderid as a value and returns the single order from "Orders_Jan2019_v".

```
CREATE MACRO Orderid_m (oid_value INTEGER)
AS (SELECT * FROM Orders_Jan2019_v WHERE orderid = :oid_value);
```

Execute the macro to provide order details for orders 1781509 and 1781939.

```
EXEC Orderid_m (1781509);    EXEC Orderid_m (1781939);
```

What is the ordertotal for orderid #1031509? 1385.67 Orderid #1031939? 1306.67

12. What is shown in photo #7? DBC 1012

Lab Solutions for Lab 11-1

Lab Exercise 11-1 Solutions

Purpose

In this lab, you will use Teradata Studio Express or Teradata Studio to evaluate various columns of table as primary index candidates.

What you need

Populated PD.Employee table; your empty Employee table

Tasks

1. Using INSERT/SELECT, populate your "Employee" table with all rows from the PD.Employee table. Verify the number of rows in your table.

```
INSERT INTO Employee SELECT * FROM PD.Employee;
```

```
SELECT COUNT(*) FROM Employee;
```

```
INSERT INTO Employee SELECT * FROM PD.Employee;  
SELECT COUNT(*) FROM Employee;
```

Count = 150,000

Lab Solutions for Lab 11-1 (cont.)

2. Collect column demographics for each of these columns in Employee and determine if the column would be a primary index candidate or not.

By using the SHOW TABLE Employee command, you should be able to complete the EmpNumber information without executing any SQL.

	Distinct Values	Max Rows for a Value	Max Rows NULL	Avg Rows per Value	Candidate for PI (Y/N)
EmpNumber	150,000	1	0	1	Yes
DeptNumber	17,412	26	0	8	Yes
JobCode	11,124	29	0	13	Yes
LastName	1,001	187	0	149	Yes (with 10X or 50X)

```

SELECT COUNT(DISTINCT(DeptNumber)) FROM Employee;
SELECT DeptNumber, COUNT(*) FROM Employee GROUP BY 1 ORDER BY 2 DESC;
SELECT TOP 3 WITH TIES tvalue, tcnt
FROM (SELECT DeptNumber, COUNT(*) FROM Employee GROUP BY 1) t1 (tvalue, tcnt)
ORDER BY tcnt DESC;

SELECT COUNT(*) FROM Employee WHERE DeptNumber IS NULL;
SELECT COUNT(*)/COUNT(DISTINCT(DeptNumber)) FROM Employee;

```

Lab Solutions for Lab 11-2

Lab Exercise 11-2 Solutions

Tasks

1. Use SHOW TABLE command to determine which column is the Primary Index. PI = [EmpNumber](#)

[SHOW TABLE Employee;](#)

```
CREATE SET TABLE Student135.Employee, FALLBACK,
    NO BEFORE JOURNAL,
    NO AFTER JOURNAL,
    CHECKSUM = DEFAULT,
    DEFAULT MERGEBLOCKRATIO
( EmpNumber      INTEGER NOT NULL,
  DeptNumber      INTEGER,
  EmpMgrNumber    INTEGER,
  JobCode         INTEGER,
  LastName        CHAR(20) CHARACTER SET LATIN NOT CASESPECIFIC,
  FirstName       VARCHAR(20) CHARACTER SET LATIN NOT CASESPECIFIC,
  SalaryAmount    DECIMAL(10,2))
UNIQUE PRIMARY INDEX (EmpNumber);
```

Determine the AMP space usage of your Employee table using DBC.TableSizeV.

```
SELECT      MIN(CurrentPerm) AS Min_Size, MAX(CurrentPerm) AS Max_Size
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE AND TableName = 'Employee';
```

Min Size	Max Size
634,880	638,976

Lab Solutions for Lab 11-2 (cont.)

2. Create a new table named Employee_2 with the same columns as Employee except specify LastName as the Primary Index.

```
CREATE TABLE Employee_2, Fallback
( EmpNumber    INTEGER NOT NULL
  ,DeptNumber   INTEGER
  ,EmpMgrNumber INTEGER
  ,JobCode      INTEGER
  ,LastName     CHAR(20)
  ,FirstName    VARCHAR(20)
  ,SalaryAmount DECIMAL(10,2))
PRIMARY INDEX (LastName);
```

Using INSERT/SELECT, populate your Employee_2 table from the Employee table.

```
INSERT INTO Employee_2 SELECT * FROM Employee;
```

Determine the AMP space usage of your Employee_2 table using DBC.TableSizeV.

```
SELECT      MIN(CurrentPerm) AS Min_Size, MAX(CurrentPerm) AS Max_Size
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE AND TableName = 'Employee_2';
```

<u>Min Size</u>	<u>Max Size</u>
454,656	598,016

Lab Solutions for Lab 11-3 (optional)

Lab Exercise 11-3 Solutions

1. Use the SHOW TABLE for Employee_2 to help create a new, similar table (same column names and definitions, etc.) named "Employee_NoPI" and does NOT have a primary index.

```
CREATE TABLE Employee_NoPI, Fallback
( EmpNumber    INTEGER NOT NULL
  ,DeptNumber   INTEGER
  ,EmpMgrNumber INTEGER
  ,JobCode      INTEGER
  ,LastName     CHAR(20)
  ,FirstName    VARCHAR(20)
  ,SalaryAmount DECIMAL(10,2))
NO PRIMARY INDEX;
```

Using INSERT/SELECT, populate the Employee_NoPI table with all rows from your Employee_2 table.

```
INSERT INTO Employee_NoPI SELECT * FROM Employee_2;
```

Determine the AMP space usage of the Employee_NoPI table using DBC.TableSizeV.

```
SELECT MIN(CurrentPerm) AS Min_Size, MAX(CurrentPerm) AS Max_Size
FROM    DBC.TableSizeV
WHERE   DatabaseName = DATABASE AND TableName = 'Employee_NoPI';
```

What is the least amount of permanent space on one of the AMPs? 454,656

What is the greatest amount of permanent space on one of the AMPs? 598,016

Is the distribution similar to the Employee_2 table (yes or no)? Yes

Why or why not? Because Employee_2 was skewed and blocks were copied within the AMPs.

Lab Solutions for Lab 11-3 (cont.)

2. DELETE all the rows in Employee_NoPI.

```
DELETE Employee_NoPI;
```

Using INSERT/SELECT with the **HASH BY RANDOM** option, populate your **Employee_NoPI** with all rows from your **Employee_2** table.

```
INSERT INTO Employee_NoPI SELECT * FROM Employee_2 HASH BY RANDOM;
```

Determine the AMP space usage of your Employee_NoPI table using DBC.TableSizeV.

```
SELECT MIN(CurrentPerm) AS Min_Size, MAX(CurrentPerm) AS Max_Size
FROM DBC.TableSizeV
WHERE DatabaseName = DATABASE AND TableName = 'Employee_NoPI';
```

What is the least amount of permanent space on one of the AMPs?

458,752

What is the greatest amount of permanent space on one of the AMPs?

598,016

Is the space distribution the same as the **Employee_2** table (yes or no)?

No

Is the table distributed as evenly as expected (yes or no)?

No

Why or why not? The table only has a small number of blocks and block distribution didn't provide an even space distribution in this case.

Lab Solutions for Lab 11-3 (cont.)

3. DELETE all the rows in Employee_NoPI.

```
DELETE Employee_NoPI;
```

Using INSERT/SELECT with the **HASH BY RANDOM (1, 2000000000)** option, populate your Employee_NoPI with all rows from your Employee_2 table.

```
INSERT INTO Employee_NoPI SELECT * FROM Employee_2
                                HASH BY RANDOM (1,2000000000);
```

Determine the AMP space usage of your Employee_NoPI table using DBC.TableSizeV.

```
SELECT MIN(CurrentPerm) AS Min_Size, MAX(CurrentPerm) AS Max_Size
FROM DBC.TableSizeV
WHERE DatabaseName = DATABASE AND TableName = 'Employee_NoPI';
```

What is the least amount of permanent space on one of the AMPs? 528,384

What is the greatest amount of permanent space on one of the AMPs? 532,480

Is the space distribution similar to the Employee_2 table (yes or no)? No

Is the table distributed as evenly as expected (yes or no)? Yes

Why or why not? The data is evenly distributed because individual rows are randomly distributed between the AMPs.

Lab Solutions for Lab 14-1

Lab Exercise 14-1 Solutions

1. INSERT/SELECT all rows from the populated DS.Orders table to your "Orders" table. Verify the number of rows in your table.

INSERT INTO Orders	SELECT * FROM DS.Orders;	
INSERT INTO Orders_2019	SELECT * FROM DS.Orders_2019;	
SELECT COUNT(*)	FROM Orders;	Count = <u>350,000</u>
SELECT COUNT(*)	FROM Orders_2019;	Count = <u>100,000</u>

2. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_RP" that is row partitioned.

```
CREATE TABLE Orders_RP
(orderid          INTEGER NOT NULL,
 custid          INTEGER NOT NULL,
 orderstatus     CHAR(1),
 ordertotal      DECIMAL(9,2) NOT NULL,
 orderdate       DATE FORMAT 'YYYY-MM-DD' NOT NULL,
 orderpriority   SMALLINT,
 clerk           CHAR(16),
 location        SMALLINT,
 shippriority    SMALLINT,
 ordercomment    VARCHAR(79))
PRIMARY INDEX (orderid)
PARTITION BY RANGE_N (orderdate BETWEEN
DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' MONTH, NO RANGE);
```

How many partitions are logically defined for the Orders_RP table? 121

Lab Solutions for Lab 14-1 (cont.)

3. INSERT/SELECT all of the rows from your Orders table into the Orders_RP table. Verify the number of rows in your table. Count = _____

```
INSERT INTO Orders_RP SELECT * FROM Orders;
SELECT COUNT(*) FROM Orders;
```

Count = 350,000

How many partitions would you estimate have data at this time? 108

4. Use the PARTITION key word to list the partitions and the number of rows in various partitions.

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_RP;
```

```
SELECT PARTITION, COUNT(*)
FROM Orders_RP
GROUP BY 1
ORDER BY 1;
```

How many partitions actually have data? 108

How many rows are in the partition for January, 2009? 800

How many rows are in the partition for December, 2017? 9000

<u>PARTITION</u>	<u>Count(*)</u>
1	800
:	:
:	:
108	9000

Lab Solutions for Lab 14-1 (cont.)

5. Use INSERT/SELECT to add the rows from the DS.Orders_2019 table to your Orders_RP table. Verify the number of rows in your table. Count = _____

```
INSERT INTO Orders_RP SELECT * FROM Orders_2019;
```

```
SELECT COUNT(*) FROM Orders_RP;      Count = 450,000
```

Use the PARTITION key word to determine the number of partitions used and number of rows in various partitions.

How many partitions actually have data? 120

```
SELECT      COUNT(DISTINCT(PARTITION))  
FROM        Orders_RP;
```

How many rows are in the partition for January, 2019? 8000

```
SELECT      COUNT(*)  
FROM        Orders_RP  
WHERE       EXTRACT(YEAR FROM orderdate) = 2019  
AND         EXTRACT(MONTH FROM orderdate) = 1;
```

Lab Solutions for Lab 14-1 (cont.)

6. INSERT the following row (using these values) into the Orders_RP table.

```
INSERT INTO Orders_RP
VALUES (50000, 1000, 'C', 1000, '2020-01-01', 10, 'Johnny Bench', 5, 20,
       'order for January in 2020');
```

```
INSERT INTO Orders_RP
VALUES (50001, 1001, 'C', 1001, '2020-02-01', 10, 'EF Codd', 5, 20,
       'order for February in 2020');
```

How many partitions are in Orders_RP? 121

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_RP;
```

What is the partition number (highest partition #) of the NO RANGE partition? 121

```
SELECT MAX(PARTITION)FROM Orders_RP;
```

or

```
SELECT PARTITION FROM Orders_RP WHERE orderdate = '2020-01-01';
```

or

```
SELECT PARTITION FROM Orders_RP WHERE orderdate = '2020-02-01';
```

Lab Solutions for Lab 14-1 (cont.)

7. Create a new table named "Orders_RP_ML" with multilevel row partitioning.

```
CREATE TABLE Orders_RP_ML
  (orderid      INTEGER NOT NULL,
   custid       INTEGER NOT NULL,
   orderstatus  CHAR(1),
   ordertotal   DECIMAL(9,2) NOT NULL,
   orderdate    DATE FORMAT 'YYYY-MM-DD' NOT NULL,
   orderpriority SMALLINT,
   clerk        CHAR(16),
   location     SMALLINT,
   shippriority SMALLINT,
   ordercomment VARCHAR(79))
PRIMARY INDEX (orderid)
PARTITION BY (
  RANGE_N (orderdate BETWEEN DATE '2010-01-01' AND DATE '2019-12-31'
           EACH INTERVAL '1' MONTH, NO RANGE),
  RANGE_N (location BETWEEN 1 AND 10 EACH 1, NO RANGE OR UNKNOWN) )
UNIQUE INDEX (orderid);
```

8. Populate the Orders_RP_ML table from the Orders and Orders_2019 tables using INSERT/SELECT and execute these commands in parallel. Verify the number of rows in Orders_RP_ML.

```
INSERT INTO Orders_RP_ML SELECT * FROM Orders;
INSERT INTO Orders_RP_ML SELECT * FROM Orders_2019;
SELECT COUNT(*) FROM Orders_RP_ML;          Count = 450,000
```


Lab Solutions for Lab 14-1 (cont.)

9. For the Orders_RP_ML table, use the PARTITION key word to answer the following questions.

How many partitions actually have data? [1200](#)

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_RP_ML;
```

What is the highest partition number? [1319](#)

```
SELECT MAX(PARTITION) FROM Orders_RP_ML;
```

What is the partition number for orders placed from location #1 in January, 2019? [1189](#)

```
SELECT DISTINCT(PARTITION) FROM Orders_RP_ML
WHERE location = 1
AND EXTRACT(YEAR FROM orderdate) = 2019
AND EXTRACT(MONTH FROM orderdate) = 1;
```

What is the partition number for orders placed from location #1 in February, 2019? [1200](#)

```
SELECT DISTINCT(PARTITION) FROM Orders_RP_ML
WHERE location = 1
AND EXTRACT(YEAR FROM orderdate) = 2019
AND EXTRACT(MONTH FROM orderdate) = 2;
```

Is there a difference of 11 partitions between these 2 months? [Yes](#)

Why or why not? [The second level of partitioning was based on 10 locations plus the NO RANGE partition for a difference of 11.](#)

Lab Solutions for Lab 14-1 (cont.)

10. Before altering the table, verify the number of rows in Orders_RP.

```
SELECT COUNT(*) FROM Orders_RP;    Count = 450,002
```

Use the ALTER TABLE command on Orders_RP.

```
ALTER TABLE Orders_RP MODIFY
  DROP RANGE BETWEEN DATE '2010-01-01' AND DATE '2010-12-31'
  ADD RANGE BETWEEN DATE '2020-01-01' AND DATE '2020-12-31'
                        EACH INTERVAL '1' MONTH
  WITH DELETE;
```

How many partitions currently have data rows? 111

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_RP;    Count = 111
```

How many rows now exist in the table? 450,002 Has the row count changed? No

```
SELECT COUNT(*) FROM Orders_RP;    Count = 450,002
```

If the row count did not change, why not?

Because the rows from the dropped partitions are moved into the NO RANGE partition.

How many rows are in the partition for January, 2020? 1

How many rows are in the partition for February, 2020? 1

How many rows are in the NO RANGE partition? 9600

```
SELECT COUNT(*) FROM Orders_RP WHERE orderdate = DATE '2020-01-01';
SELECT COUNT(*) FROM Orders_RP WHERE orderdate = DATE '2020-02-01';
SELECT COUNT(*) FROM Orders_RP WHERE PARTITION = 121;
```

Lab Solutions for Lab 15-1

Lab Exercise 15-1

Purpose

In this lab, you will create tables with column partitioning in various ways. These tables will be used in later labs.

What you need

Populated DS tables and Orders and Orders_2019 tables in your database

Tasks

1. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) that does NOT have a primary index and name this table "Orders_NoPI".

```
CREATE TABLE Orders_NoPI, Fallback
  (orderid          INTEGER NOT NULL,
   custid           INTEGER NOT NULL,
   orderstatus      CHAR(1),
   ordertotal       DECIMAL(9,2) NOT NULL,
   orderdate        DATE NOT NULL FORMAT 'YYYY-MM-DD',
   orderpriority    SMALLINT,
   clerk            CHAR(16),
   location         SMALLINT,
   shippriority     SMALLINT,
   ordercomment     VARCHAR(79))
NO PRIMARY INDEX;
```

Lab Solutions for Lab 15-1 (cont.)

2. Populate the Orders_NoPI table (via INSERT/SELECT) with all of the rows from the DS.Orders and DS.Orders_2019 tables.

```
INSERT INTO Orders_NoPI SELECT * FROM DS.Orders;  
INSERT INTO Orders_NoPI SELECT * FROM DS.Orders_2019;
```

Verify the number of rows in your table. Count = 450,000 (count should be 450,000)

3. Using SHOW TABLE for Orders_NoPI, create a new column partitioned table named "Orders_CP".

```
CREATE TABLE Orders_CP, Fallback  
(orderid          INTEGER NOT NULL,  
 custid           INTEGER NOT NULL,  
 orderstatus      CHAR(1),  
 ordertotal       DECIMAL(9,2) NOT NULL,  
 orderdate        DATE NOT NULL FORMAT 'YYYY-MM-DD',  
 orderpriority    SMALLINT,  
 clerk            CHAR(16),  
 location         SMALLINT,  
 shippriority     SMALLINT,  
 ordercomment     VARCHAR(79))  
PARTITION BY COLUMN;
```

Populate the Orders_CP table (via INSERT/SELECT) from the Orders_NoPI table.

```
INSERT INTO Orders_CP SELECT * FROM Orders_NoPI;
```

Lab Solutions for Lab 15-1 (cont.)

4. Verify the number of rows in your table and find how many partitions are defined. Count = 450,000

```
SELECT COUNT(*) FROM Orders_CP;
```

How many partitions actually have data? 1

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_CP;
```

Note: Even though each column stored in separate partitions internally, the table only has 1 logical partition.

Lab Solutions for Lab 15-1 (cont.)

5. Use the SHOW TABLE for Orders_NoPI to create a new column partitioned table named "Orders_CP_noAC" without automatic compression on each column.

```
CREATE TABLE Orders_CP_noAC, Fallback
  (orderid      INTEGER NOT NULL,
   custid       INTEGER NOT NULL,
   orderstatus  CHAR(1),
   ordertotal   DECIMAL(9,2) NOT NULL,
   orderdate    DATE NOT NULL FORMAT 'YYYY-MM-DD',
   orderpriority SMALLINT,
   clerk        CHAR(16),
   location     SMALLINT,
   shippriority SMALLINT,
   ordercomment VARCHAR(79))
PARTITION BY COLUMN NO AUTO COMPRESS;
```

Populate the Orders_CP_noAC table (via INSERT/SELECT) from the Orders_NoPI table.

```
INSERT INTO Orders_CP_noAC SELECT * FROM Orders_NoPI;
```

Lab Solutions for Lab 15-1 (cont.)

6. Use the SHOW TABLE for Orders_CP to create a new column partitioned table named "Orders_CPPA" using Primary AMP (custid) and automatic compression on each column.

```
CREATE TABLE Orders_CPPA, Fallback
  (orderid          INTEGER NOT NULL,
   custid           INTEGER NOT NULL,
   orderstatus      CHAR(1),
   ordertotal       DECIMAL(9,2) NOT NULL,
   orderdate        DATE NOT NULL FORMAT 'YYYY-MM-DD',
   orderpriority    SMALLINT,
   clerk            CHAR(16),
   location         SMALLINT,
   shippriority     SMALLINT,
   ordercomment     VARCHAR(79))
PRIMARY AMP (custid) PARTITION BY COLUMN;
```

Populate the Orders_CPPA table (via INSERT/SELECT) with all of the rows from your Orders_NoPI table.

```
INSERT INTO Orders_CPPA SELECT * FROM Orders_NoPI;
```

Lab Solutions for Lab 15-1 (cont.)

7. Use the SHOW TABLE for Orders_CPPA to create a new column partitioned table named "Orders_CPPI" using Primary Index (**custid**) and automatic compression on each column.

```
CREATE TABLE Orders_CPPI, Fallback
  (orderid      INTEGER NOT NULL,
   custid       INTEGER NOT NULL,
   orderstatus  CHAR(1),
   ordertotal   DECIMAL(9,2) NOT NULL,
   orderdate    DATE NOT NULL FORMAT 'YYYY-MM-DD',
   orderpriority SMALLINT,
   clerk        CHAR(16),
   location     SMALLINT,
   shippriority SMALLINT,
   ordercomment VARCHAR(79))
PRIMARY INDEX (custid) PARTITION BY COLUMN;
```

Populate the Orders_CPPI table (via INSERT/SELECT) with all of the rows from the DS.Orders and DS.Orders_2019 tables in parallel.

```
INSERT INTO Orders_CPPI SELECT * FROM DS.Orders;
INSERT INTO Orders_CPPI SELECT * FROM DS.Orders_2019;
```


Lab Solutions for Lab 15-1 (cont.)

8. Determine the AMP space usage of the Orders_CP, Orders_CP_noAC, Orders_CPPA and Orders_CPPI tables using the DBC.TableSizeV.

```
SELECT  TableName (CHAR(15)), SUM(CurrentPerm)
FROM    DBC.TableSizeV
WHERE   DatabaseName = DATABASE
AND     TableName IN
        ('Orders_CP', 'Orders_CP_noAC', 'Orders_CPPA', 'Orders_CPPI')
GROUP BY 1
ORDER BY 1;
```

TableName	Sum(CurrentPerm)
Orders_CP	9,846,784
Orders_CPPA	8,495,104
Orders_CPPI	7,417,856
Orders_CP_noAC	10,956,800

Note: Your CurrentPerm values may be different.

Which table is the largest and why? **Orders_CP_noAC** because of no auto compression.

What is the probable reason that the CPPA and CPPI tables use less perm space? **Most likely** because rows with the same custid are distributed to same AMP and compression is more effective.

Lab Solutions for Lab 15-1 (cont.)

9. Use the SHOW TABLE for Orders_CP to create a new column partitioned table named "Orders_CP_RP based on the following:

```
CREATE TABLE Orders_CP_RP, Fallback
  (orderid      INTEGER NOT NULL,
   custid       INTEGER NOT NULL,
   orderstatus  CHAR(1),
   ordertotal   DECIMAL(9,2) NOT NULL,
   orderdate    DATE NOT NULL FORMAT 'YYYY-MM-DD',
   orderpriority SMALLINT,
   clerk        CHAR(16),
   location     SMALLINT,
   shippriority SMALLINT,
   ordercomment VARCHAR(79))
PRIMARY AMP INDEX (custid) PARTITION BY
  (RANGE_N (orderdate BETWEEN
    DATE '2010-01-01' AND DATE '2019-12-31' EACH INTERVAL '1' DAY ),
   COLUMN);
```

Populate the Orders_CP_RP table (via INSERT/SELECT) from your Orders_NoPI table.

```
INSERT INTO Orders_CP_RP SELECT * FROM Orders_NoPI;
```

10. Use the PARTITION key word to determine the number of partitions defined in the Orders_CP_RP.

How many partitions actually have data? 3622

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_CP_RP;
```

Lab Solutions for Lab 18-1

Lab Exercise 18-1 Solutions

1. If necessary, use INSERT-SELECT to populate your Orders table from the DS.Orders table.

```
INSERT INTO Orders SELECT * FROM DS.Orders;

SELECT SUM(CurrentPerm) FROM DBC.TablesizeV
WHERE DatabaseName = DATABASE AND TableName = 'Orders';
```

Using the DBC.TableSizeV view, what is the amount of Perm space used. Orders = 14,229,504

2. Create "Orders_MVC" based on the Orders table, except compress the clerk names as shown:

```
CREATE SET TABLE Orders_MVC, Fallback
(orderid          INTEGER NOT NULL,
 custid           INTEGER NOT NULL,
 orderstatus      CHAR(1),
 ordertotal       DECIMAL(9,2) NOT NULL,
 orderdate        DATE FORMAT 'YYYY-MM-DD' NOT NULL,
 orderpriority    SMALLINT,
 clerk CHAR(16) COMPRESS
 ('April Rains','Auggie Sunshine','Dee Winter','Fred Cold','Jack Snow',
 'June Summer','Justin Heat','Matt Winds','May Flowers','Norbert Thanks',
 'Ossie Leaves','Steve Fall'),
 location SMALLINT,
 shippriority SMALLINT,
 ordercomment VARCHAR(79) )
PRIMARY INDEX (orderid);
```

Using INSERT-SELECT, populate your Orders_MVC table from DS.Orders table.

Using DBC.TableSizeV, what is the amount of Perm space used. Orders_MVC = 13,811,712

Lab Solutions for Lab 18-1 (cont.)

3. Within your session, use QUERYBAND to turn off block level compression.

```
SET QUERY_BAND = 'BLOCKCOMPRESSION=NO;' UPDATE FOR SESSION;
```

4. Delete the rows in your Orders and Orders_MVC tables.

```
DELETE Orders;  
DELETE Orders_MVC;
```

5. Populate your Orders and Orders_MVC tables using the DS.Orders table.

```
INSERT INTO Orders      SELECT * FROM DS.Orders;  
INSERT INTO Orders_MVC  SELECT * FROM DS.Orders;
```

6. Using DBC.TableSizeV, what is the amount of Perm space used. Orders = _____

```
SELECT SUM(CurrentPerm) FROM DBC.TablesizeV  
WHERE DatabaseName = DATABASE AND TableName = 'Orders';
```

Using the DBC.TableSizeV view, what is the amount of Perm space used. Orders = 34,168,832

7. Using DBC.TableSizeV, what is the amount of Perm space used. Orders_MVC = _____

```
SELECT SUM(CurrentPerm) FROM DBC.TablesizeV  
WHERE DatabaseName = DATABASE AND TableName = 'Orders_MVC';
```

Using the DBC.TableSizeV view, what is the amount of Perm space used. Orders = 26,140,672

Lab Solutions for Lab 18-1 (cont.)

8. Within your session, use QUERYBAND to turn on block level compression.

```
SET QUERY_BAND = 'BLOCKCOMPRESSION=YES;' UPDATE FOR SESSION;
```

9. Delete the rows in your Orders and Orders_MVC tables.

```
DELETE Orders;  
DELETE Orders_MVC;
```

10. Populate your Orders and Orders_MVC tables using the DS.Orders table.

```
INSERT INTO Orders      SELECT * FROM DS.Orders;  
INSERT INTO Orders_MVC  SELECT * FROM DS.Orders;
```

Lab Solutions for Lab 18-2

Lab Exercise 18-2 Solutions

Tasks

1. Create two Contiguous tables Nation_C and Region_C from PD.Nation_C and PD.Region_C. (Note: these tables do not have all of the nations/regions, just a small subset of data)

```
CREATE TABLE Region_C AS PD.Region_C WITH DATA;  
CREATE TABLE Nation_C AS PD.Nation_C WITH DATA;
```

2. Find the system single AMP (1) Sparse MAP by using the view DBC.MapsV.

```
SELECT MapName FROM DBC.MapsV;
```

Name of Sparse Map for 1 AMP – *TD_1AmpSparseMap_1Node*

Lab Solutions for Lab 18-2 (cont.)

3. Use SHOW TABLE and create new Nation_S and Region_S sparse tables specifying the 1 AMP sparse MAP name and use a COLOCATE value of Co_NationRegion for both tables (i.e., COLOCATE USING Co_NationRegion).

```
CREATE TABLE Nation_S, FALLBACK,
MAP = TD_1AmpSparseMap_1Node COLOCATE USING Co_NationRegion
( NationKey    INTEGER,
  NationName   VARCHAR(25),
  RegionKey    INTEGER )
UNIQUE PRIMARY INDEX (NationKey);

CREATE TABLE Region_S, FALLBACK,
MAP = TD_1AmpSparseMap_1Node COLOCATE USING Co_NationRegion
( RegionKey    INTEGER,
  RegionName   VARCHAR(25) )
UNIQUE PRIMARY INDEX (RegionKey);
```

4. Populate the two Sparse tables with INSERT/SELECT from the two contiguous tables.

```
INSERT INTO Nation_S SELECT * FROM Nation_C;
INSERT INTO Region_S SELECT * FROM Region_C;
```

Lab Solutions for Lab 18-2 (cont.)

5. Use the DBC.TableSizeV and determine which 2 AMPs the sparse tables are using.

```
SELECT DatabaseName, TableName, Vproc, SUM(CurrentPerm)
FROM   DBC.TablesizeV
WHERE  DatabaseName = DATABASE AND TableName IN ('Nation_S', 'Region_S')
GROUP BY 1,2,3
ORDER BY 1,2,3;
```

<u>DataBaseName</u>	<u>TableName</u>	<u>Vproc</u>	<u>Sum(CurrentPerm)</u>
student535Nation_S	2	8,192	
student535Nation_S	3	8,192	
student535Region_S	2	8,192	
student535Region_S	3	8,192	

Why are 2 AMPs used per sparse table? [Primary and Fallback AMPs](#)

6. Compare the sizes of Nation_C and Region_C to the Nation_S and Region_S tables.

```
SELECT DatabaseName, TableName, SUM(CurrentPerm)
FROM   DBC.TablesizeV
WHERE  DatabaseName = DATABASE
AND    TableName IN ('Nation_C', 'Nation_S', 'Region_C', 'Region_S')
GROUP BY 1,2
ORDER BY 1,2,3;
```

Size of Nation_C = [98,304](#)
 Size of Region_C = [73,728](#)

Size of Nation_S = [16,384](#)
 Size of Region_S = [16,384](#)

Lab Solutions for Lab 18-2 (cont.)

7. Use the DBC.SparseMapAmpsV to determine which AMP has your primary data for the Sparse tables. AMP # = 2

```
SELECT * FROM DBC.SparseMapAmpsV where DatabaseName = DATABASE;
```

<u>DatabaseName</u>	<u>ObjectName</u>	<u>ObjectKind</u>	<u>AmpNo</u>	<u>PF</u>
student535Nation_S	T 2	P		
student535Region_S	T 2	P		
student535Nation_S	T 3	F		
student535Region_S	T 3	F		

8. Execute the following EXPLAIN of a join of the Contiguous tables.
Is the data redistributed? Yes/No Yes

```
EXPLAIN  SELECT      NationName, RegionName
          FROM        Nation_C
          INNER JOIN   Region_C
          ON           Nation_C.RegionKey = Region_C.RegionKey;
```

Execute the same EXPLAIN using the Nation_S and Region_S sparse tables. Is the data redistributed for the join? Yes/No No

```
EXPLAIN  SELECT      NationName, RegionName
          FROM        Nation_S
          INNER JOIN   Region_S
          ON           Nation_S.RegionKey = Region_S.RegionKey;
```

Lab Solutions for Lab 18-2 (cont.)

9. Optional: Create a new Nation_S2 table and a new Region_S2 table without using the COLOCATE option. Use the DBC.SparseMapAmpsV to determine which AMP has your Sparse tables.

Are the two tables on the same AMP? Yes/No No

```
CREATE TABLE Nation_S2, Fallback,
MAP = TD_1AmpSparseMap_1Node
( NationKey    INTEGER,
  NationName   VARCHAR(25),
  RegionKey    INTEGER )
UNIQUE PRIMARY INDEX (NationKey);
```

```
CREATE TABLE Region_S2, Fallback,
MAP = TD_1AmpSparseMap_1Node
( RegionKey    INTEGER,
  RegionName   VARCHAR(25) )
UNIQUE PRIMARY INDEX (RegionKey);
```

```
SELECT * FROM DBC.SparseMapAmpsV
WHERE DatabaseName = DATABASE AND ObjectName IN ('Nation_S2', 'Region_S2');
```

<u>DatabaseName</u>	<u>ObjectName</u>	<u>ObjectKind</u>	<u>AmpNo</u>	<u>PF</u>
student535Nation_S2	T	6	P	
student535Region_S2	T	5	P	
student535Nation_S2	T	7	F	
student535Region_S2	T	4	F	

Lab Solutions for Lab 20-1

Lab Exercise 20-1 Solutions

1. Orders_NoPI should have 450,000 rows from a previous lab exercise. Verify the row count.

```
SELECT COUNT(*) FROM Orders_NoPI;
```

Count(*)
450,000

2. Collect statistics individually on the columns custid and orderdate of the Orders_NoPI table. Use the HELP STATISTICS and the HELP CURRENT STATISTICS commands on this table.

```
COLLECT STATISTICS COLUMN custid, COLUMN orderdate ON Orders_NoPI;  
HELP STATISTICS Orders_NoPI;
```

What is the row count (*) for this table? 450,000

How many unique values are there for custid? 2085

How many unique values are there for orderdate? 3622

Are the values different for HELP STATISTICS and HELP CURRENT STATISTICS? No

```
HELP STATISTICS Orders_NoPI;  
HELP CURRENT STATISTICS Orders_NoPI;
```

Lab Solutions for Lab 20-1 (cont.)

3. Insert the rows from the DS.Orders table into your Orders_NoPI table. Use the HELP CURRENT STATISTICS commands to answer the following questions.

```
INSERT INTO Orders_NoPI SELECT * FROM DS.Orders;
```

```
HELP CURRENT STATISTICS ON Orders_NoPI;
```

What is the row count (*) for this table? 800,000

How many unique values are there for custid? 2085

How many unique values are there for orderdate? 6440

Why has the row count and number of orderdates increased with current statistics?

HELP CURRENT STATISTICS indicates that the optimizer can use estimated values (extrapolation or derived statistics – random AMP sample) when it detects stale statistics.

Lab Solutions for Lab 20-2

Lab Exercise 20-2 Solutions

1. Collect statistics on orderid and orderdate for Orders and Orders_RP.

```
INSERT INTO Orders SELECT * FROM Orders_2019;  
COLLECT STATISTICS COLUMN orderid, COLUMN orderdate ON Orders;  
COLLECT STATISTICS COLUMN orderid, COLUMN orderdate, COLUMN PARTITION  
    on Orders_RP;
```

Execute the following two Explains.

```
EXPLAIN SELECT * FROM Orders  
    WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How many AMPs are accessed in this operation? [All](#)

Is this a full table scan? [Yes](#) What type of lock is used? [Table level READ lock](#)

```
EXPLAIN SELECT * FROM Orders_RP  
    WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-01-31';
```

How many AMPs are accessed in this operation? [All](#)

Is this a full table scan? [No](#) What type of lock is used? [Single Partition level READ lock](#)

Is partitioning beneficial for this type of query? [Yes](#)

Lab Solutions for Lab 20-2 (cont.)

2. Execute the following three Explains.

```
EXPLAIN SELECT * FROM Orders
      WHERE orderid = 1404662;
```

How many AMPs are accessed in this operation? [One](#)

How is this row retrieved by Teradata? [Via the Primary index](#)

```
EXPLAIN SELECT * FROM Orders_RP
      WHERE orderid = 1404662;
```

How many AMPs are accessed in this operation? [One](#)

How many partitions are scanned to locate this row? [All](#)

How is this row retrieved by Teradata? [By probing each partition for the primary index value.](#)

```
EXPLAIN SELECT * FROM Orders_NoPI
      WHERE orderid = 1404662;
```

How many AMPs are accessed in this operation? [All](#)

How is this row retrieved by Teradata? [Via a full table scan since this table has no indexes.](#)

Lab Solutions for Lab 20-2 (cont.)

3. Execute the following two Explains.

```
EXPLAIN DELETE FROM Orders
      WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-12-31';
```

How many AMPs are accessed in this operation? [All](#)

Is this a full table scan? [Yes](#) Why or why not? [No index qualification nor any partitioning](#)

```
EXPLAIN DELETE FROM Orders_RP
      WHERE orderdate BETWEEN DATE '2019-01-01' AND DATE '2019-12-31';
```

How many AMPs are accessed in this operation? [All](#)

Is this a full table scan? [No](#) Why or why not? [Values are specified for a partitioning column](#)

How many partitions are scanned to delete these rows? [12](#)

4. Collect statistics on orderid for the Orders_CP table and execute the following two Explains.

```
EXPLAIN SELECT orderid, custid FROM Orders_CP;
```

How many column partitions are accessed? [3](#)

What is the relative cost for this EXPLAIN? [.34](#)

```
EXPLAIN SELECT orderid, custid, orderstatus, orderdate FROM Orders_CP;
```

How many column partitions are accessed? [5](#)

What is the relative cost for this EXPLAIN? [.46](#)

Why is there an extra partition accessed for each of these explains? [Delete Partition is](#)

Lab Solutions for Lab 20-2 (cont.)

4. (cont.) Last of the three Explain plans.

```
EXPLAIN SELECT * FROM Orders_CPPI WHERE custid = 1024;
```

How many AMPs are involved in this query? [One](#)

How many partitions are involved in this query? [11](#)

Is this number equal to the number of columns in the table? [No](#)

Why or why not? [Delete Partition is accessed](#)

Lab Solutions for Lab 23-1

Lab Exercise 23-1 Solutions

1. Use INSERT/SELECT to place all rows from the populated PD tables into your empty tables. Verify the number of rows in your tables.

```
INSERT INTO Employee SELECT * FROM PD.Employee;
SELECT COUNT(*) FROM Employee;                                Count = 150,000
```

```
INSERT INTO Department SELECT * FROM PD.Department;
SELECT COUNT(*) FROM Department;                               Count = 17,415
```

```
INSERT INTO Job SELECT * FROM PD.Job;
SELECT COUNT(*) FROM Job;                                     Count = 11,124
```

2. Execute the following EXPLAIN statement.

```
EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc
FROM        Employee E
INNER JOIN   Department D      ON  E.DeptNumber = D.DeptNumber
INNER JOIN   Job J            ON  E.JobCode = J.JobCode
ORDER BY    3, 1, 2;
```

What is the total estimated time cost for this EXPLAIN? 1.71 seconds

Lab Solutions for Lab 23-1 (cont.)

3. Create a “non-compressed” join index which includes the following columns of Employee, Department, and Job.

```
CREATE JOIN INDEX EDJ_JI AS
SELECT      LastName, FirstName, D.DeptNumber, DeptName, J.JobCode, JobDesc
FROM        Employee E
INNER JOIN  Department D      ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J            ON  E.JobCode = J.JobCode;
```

Execute the HELP USER command. What is the object type of the Join Index? [!](#)

4. Execute the following EXPLAIN (same SQL statement as step #2).

What is the total estimated time cost for this EXPLAIN? [1.19 seconds](#)

Is the join index used? [Yes](#)

How much space does the join index require (use the DBC.TableSizeV view)? [11,378,688](#)

```
SELECT      TableName , SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE AND TableName = 'EDJ_JI'
GROUP BY    1
ORDER BY    1;
```

<u>TableName</u>	<u>Sum(CurrentPerm)</u>
EDJ_JI	12,058,624

Lab Solutions for Lab 23-1 (cont.)

5. Execute the following EXPLAIN statement – SalaryAmount has been added as a projected column.

```
EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc, SalaryAmount
FROM        Employee E
INNER JOIN   Department D ON  E.DeptNumber = D.DeptNumber
INNER JOIN   Job J          ON  E.JobCode = J.JobCode
ORDER BY    3, 1, 2;
```

What is the total estimated time cost for this EXPLAIN? [1.83 seconds](#)

Is the join index used? [No](#) If not, why not? [SalaryAmount is not in the Join Index. The join index rows and the associated Employee row are not on the same AMP. Therefore, the optimizer chooses to join the two tables together.](#)

6. Drop the Join Index.

```
DROP JOIN INDEX EDJ_JI;
```

Lab Solutions for Lab 23-1 (cont.)

7. Create a new “non-compressed” join index similar to step #3 except include the Primary Index of EmpNumber for the Join Index.

```
CREATE JOIN INDEX EDJ_JI2 AS
SELECT      EmpNumber, LastName, FirstName,
            D.DeptNumber, DeptName, J.JobCode, JobDesc
FROM        Employee E
INNER JOIN  Department D      ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J             ON  E.JobCode = J.JobCode
PRIMARY INDEX (EmpNumber);
```

8. Execute the EXPLAIN statement from step #5.

```
EXPLAIN
SELECT      LastName, FirstName, DeptName, JobDesc, SalaryAmount
FROM        Employee E
INNER JOIN  Department D ON  E.DeptNumber = D.DeptNumber
INNER JOIN  Job J         ON  E.JobCode = J.JobCode
ORDER BY    3, 1, 2;
```

What is the total estimated time cost for this EXPLAIN? [1.32 sec](#)

Is the join index used? **Yes** If so, why? **Because a join index row and the associated employee row are on same AMP – the join index and the Employee table can be joined together with matching PI values. The cost is 1.32 seconds versus 1.83 seconds without the join index.**

9. Drop the Join Index.

```
DROP JOIN INDEX EDJ_JI2;
```

Lab Solutions for Lab 23-2

Lab Exercise 23-2 Solutions (optional)

1. Verify the number of rows in Employee, Job, Department and EmpPhone tables. If not correct, use INSERT/SELECT to populate the tables from the PD database.

```
INSERT INTO EmpPhone SELECT * FROM PD.EmpPhone;
SELECT COUNT(*) FROM EmpPhone; Count = 350,000
```

2. Execute the following Explain statement which joins all 4 tables together.

```
EXPLAIN SELECT      D.DeptName, E.EmpNumber, E.LastName, E.FirstName,
                   J.JobDesc, P.AreaCode, P.PhoneNumber
FROM
Employee E
INNER JOIN      Department D ON      E.DeptNumber = D.DeptNumber
INNER JOIN      EmpPhone P      ON P.EmpNumber = E.EmpNumber
INNER JOIN      Job J          ON E.JobCode = J.JobCode
ORDER BY      1, 2, 3, 4;
```

What is the total estimated time cost for this EXPLAIN? 2.69 sec

Lab Solutions for Lab 23-2 (cont.)

3. Create the following Global Join Index on the employee table.

```
CREATE JOIN INDEX Employee_JI AS
SELECT      EmpNumber,
            DeptNumber,
            EmpMgrNumber,
            JobCode,
            LastName,
            FirstName,
            SalaryAmount,
            ROWID AS RID
FROM        Employee
PRIMARY INDEX (DeptNumber);
```

4. Re-execute the Explain statement from step #2.

What is total estimated time cost for this EXPLAIN? [0.59 sec](#)

Is the join index used? [Yes](#)

How much space does the join index require (use the DBC.TableSizeV view)? [9,830,400](#)

```
SELECT      TableName (CHAR(15)), SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName IN ('Employee_JI')
GROUP BY 1 ORDER BY 1;
```

5. Drop the Employee_JI Join Index.

```
DROP JOIN INDEX Employee_JI;
```

Lab Solutions for Lab 23-2 (cont.)

6. Modify the Create Join Index you used in step #3 and make a Column Partitioned Join index called **Employee_JI_CPPA** with a Primary AMP Index of **DeptNumber** column.

```
CREATE JOIN INDEX Employee_JI_CPPA AS
  SELECT EmpNumber,
         DeptNumber,
         EmpMgrNumber,
         JobCode,
         LastName,
         FirstName,
         SalaryAmount,
         ROWID AS RID
  FROM Employee
  PRIMARY AMP INDEX (DeptNumber) PARTITION BY COLUMN;
```

7. Re-execute the Explain statement from step #2.

What is total estimated time cost for this EXPLAIN? [0.74 sec](#)

What is the size Employee_JI_CPPA join index is using? [6,414,336](#)

```
SELECT TableName (CHAR(20)), SUM(CurrentPerm)
FROM DBC.TableSizeV
WHERE DatabaseName = DATABASE
AND TableName In ('Employee_JI_CPPA')
GROUP BY 1 ORDER BY 1;
```

8. Drop the employee_JI_CPPA Join Index.

```
DROP JOIN INDEX Employee_JI_CPPA;
```

Lab Solutions for Lab 23-2 (cont.)

9. Modify the Create Join Index you used in step #6 and make a Column Partitioned Join index called **Employee_JI_CPPI** with a Primary Index of **DeptNumber** column.

```
CREATE JOIN INDEX Employee_JI_CPPI AS
SELECT EmpNumber,
       DeptNumber,
       EmpMgrNumber,
       JobCode,
       LastName,
       FirstName,
       SalaryAmount,
       ROWID AS RID
FROM Employee
PRIMARY INDEX (DeptNumber) PARTITION BY COLUMN;
```

10. Re-execute the Explain statement from step #2.

What is total estimated time cost for this EXPLAIN? 0.59 sec

What is the size employee_JI_CPPI join index is using? 9,838,592

11. Drop the Employee_JI_CPPI Join Index.

```
DROP JOIN INDEX Employee_JI_CPPI;
```

12. Which Join Index gave the best tradeoff with regards to size and performance? **Maybe Employee_JI_CPPA**

Module Z

teradata.

Appendix Z: Table of Contents

This Appendix contains the combined table of contents for every module in this course.

Teradata Proprietary and Confidential

Notes

Table of Contents

Module 1 – Overview

What is Teradata?.....	1-4
Teradata – A Brief History.....	1-6
Teradata – The beginning	1-8
What is a Data Warehouse?	1-10
What is a Relational Database?.....	1-12
Answering Questions with a Relational Database	1-14
Teradata's Competitive Advantages.....	1-16
Teradata Vantage	1-18
What is the Teradata NewSQL Engine?	1-20
Teradata Everywhere™	1-22
Module 1: Review Questions.....	1-24

Module 2 – Teradata Basics

Major Components of Teradata	2-4
Teradata Storage Architecture	2-6
Teradata Retrieval Architecture	2-8
Multiple Tables on Multiple AMPs	2-10
Teradata Objects	2-12
The Data Dictionary Directory (DD/D)	2-14
Structured Query Language (SQL)	2-16
CREATE TABLE – Example of DDL	2-18
Views	2-20
Multi-Table Views	2-22
Macros.....	2-24
HELP Commands	2-26
SHOW Command	2-28
EXPLAIN Facility	2-30
Summary	2-32
Module 2: Review Questions.....	2-34

Module 3 – Teradata Database Architecture

Teradata and MPP Systems	3-4
Teradata Functional Overview	3-6
Channel-Attached Client Software Overview	3-8
Network-Attached Client Software Overview	3-10
The Parsing Engine.....	3-12
Message Passing Layer.....	3-14
The Access Module Processor (AMP)	3-16
Teradata Parallelism	3-18
Linear Growth and Expandability	3-20
Module 3: Review Questions	3-22

Module 4 – Teradata Databases and Users

Teradata System Hierarchy	4-4
What is a Database?.....	4-6
What is a User?.....	4-8
Database – User Comparison	4-10
Teradata's Hierarchy and Perm Space.....	4-12
Example of a System Hierarchy	4-14
Permanent Space.....	4-16
Spool Space	4-18
Temporary Space.....	4-20
Creating Tables.....	4-22
Data Types	4-24
Access Rights and Privileges.....	4-26
Module 4: Review Questions	4-28

Module 5 – Primary Index Access and Mechanics

Primary Keys and Primary Indexes	5-4
Distribution of Rows	5-6
Specifying a Primary Index.....	5-8
Primary Index Values.....	5-10
Accessing Via a Unique Primary Index.....	5-12
Accessing Via a Non-Unique Primary Index.....	5-14
Row Distribution Using a Unique Primary Index (UPI) – Case 1	5-16
Row Distribution Using a Non-Unique Primary Index (NUPI) – Case 2.....	5-18
Row Distribution Using a Highly Non-Unique Primary Index (NUPI) – Case 3.....	5-20
Which AMP has the Row?.....	5-22
Hashing Down to the AMPs	5-24
A Hashing Example	5-26
The Hash Map.....	5-28
Hash Maps for Different Systems.....	5-30
Identifying Rows.....	5-32
The Row ID.....	5-34
Storing Rows (1 of 2).....	5-36
Locating a Row on an AMP Using a PI.....	5-40
Module 5: Review Questions.....	5-42

Module 6 – Secondary Indexes and Table Scans

Secondary Indexes	6-4
Choosing a Secondary Index.....	6-6
Unique Secondary Index (USI) Access	6-8
Non-Unique Secondary Index (NUSI) Access	6-10
Comparison of Primary and Secondary Indexes.....	6-12
Full Table Scans.....	6-14
Module 6: Review Questions.....	6-16

Module 7 – Teradata Studio

Teradata SQL Tools.....	7-4
Additional Teradata Studio Features	7-6
Teradata Studio.....	7-8
Teradata Studio Query Development	7-14
Teradata Studio Summary	7-42
Module 7: Review Questions	7-44
Lab Exercise 7-1	7-46

Module 8 – Teradata System Architecture

Teradata Database Releases.....	8-4
Teradata Database Architecture.....	8-6
Teradata Database – Multiple Nodes.....	8-8
MPP Systems	8-10
Example of 2+1 Node Teradata System	8-12
Teradata Cliques	8-14
BYNET.....	8-16
BYNET Communication Protocols	8-16
Vproc Inter-Process Communication	8-18
Fabric Architecture Enables Flexibility.....	8-20
BYNET 1U Switches.....	8-22
Larger BYNET Configurations	8-24
Examples of Teradata Database Systems	8-26
IntelliFlex™ 2.0 Cabinet Example.....	8-28
What makes Teradata’s MPP Platforms Special?	8-30
Summary.....	8-32
Module 8: Review Exercises	8-34

Module 9 – Data Protection

Data Protection Features	9-4
Disk Arrays	9-6
RAID Technologies	9-8
RAID 1 – Mirroring	9-10
RAID 1 Summary	9-12
RAID 6 – Data Parity Protection with 2 Parity Drives	9-14
Cliques	9-16
Teradata Vproc Migration.....	9-18
Hot Standby Nodes (HSN).....	9-20
Performance Degradation with Node Failure	9-22
Fallback.....	9-24
Fallback Clusters.....	9-26
Fallback and RAID Protection.....	9-28
Fallback and RAID 1 Example	9-30
Fallback vs. non-Fallback Tables Summary	9-40
Clusters and Cliques.....	9-42
Locks.....	9-44
Locking Modifier	9-46
Rules of Locking.....	9-48
Access Locks.....	9-50
WAL – Write-Ahead Logging	9-52
WAL Concepts.....	9-54
Transient Journal.....	9-56
Recovery Journal for Down AMPs.....	9-58
Permanent Journal.....	9-60
Archiving and Recovering Data.....	9-62
Teradata Unity.....	9-64
Teradata Multi-System Solution Architecture	9-66
Module 9: Review Questions.....	9-68

Module 10 – How Teradata uses MPP Systems

Teradata and the Processing Node.....	10-4
Memory and the Teradata Database	10-6
Node Memory – Summary	10-8
Linux Managed Memory and FSG Cache	10-10
Linux Vproc Number Assignment	10-12
Disk Arrays from a Linux Perspective	10-14
Logical Units and Partitions	10-16
Teradata and Disk Arrays	10-18
Teradata 2850 Logical View	10-20
Teradata 6750 (3+1) Logical View	10-22
Example of 6750H Vdisk	10-24
Cylinder – AMP Allocation Unit.....	10-26
Teradata File System Concepts	10-28
Cylinder Index Segregation (15.10)	10-30
Full Cylinder Read.....	10-32
Summary.....	10-34
Module 10: Review Questions	10-36

Module 11 – Analyze Primary Index Criteria

Primary Index Choice Criteria	11-4
Primary Index Defaults	11-6
CREATE TABLE – Indexing Rules	11-8
Order of Preference Exercise	11-10
Primary Index Characteristics	11-12
Multi-Column Primary Indexes	11-14
Primary Index Considerations.....	11-16
PKs and Duplicate Rows.....	11-18
NUPI Duplicate Row Check	11-20
Primary Index Demographics	11-22
Column Distribution Demographics for a PI Candidate	11-24
SQL to View Data Demographics	11-26
Example of Using Data Demographic SQL.....	11-28
TableSize View	11-30
Using SQL to Predict Row Distribution	11-32
E-R Diagram for Exercises	11-34
Exercise 2 – Example.....	11-36
Exercise 2 – Choosing PI Candidates	11-38
What is a NoPI Table?	11-50
Reasons to Consider Using NoPI Tables	11-52
Creating a Table without a PI	11-54
Distribution of Rows (or Blocks) for a NoPI Table.....	11-56
The Row ID for a NoPI Table.....	11-58
Multiple NoPI Tables at the AMP Level	11-62
Loading Data into a NoPI Table (SQL)	11-64
Loading Data into a NoPI Table (Utility)	11-66
NoPI Options.....	11-68
Summary	11-70
Module 11: Review Questions.....	11-72
Module 11: Review Questions (cont.)	11-74
Lab Exercise 11-1	11-76
Lab Exercise 11-2	11-80
Lab Exercise 11-3 (optional).....	11-82

Module 12 – Data Distribution and Hashing

Data Distribution	12-4
Hashing.....	12-6
Hash Related Expressions	12-8
Hashing – Numeric Data Types.....	12-10
Multi-Column Hashing.....	12-12
Additional Hash Examples	12-14
Using Hash Functions to View Distribution.....	12-16
Primary Index Hash Mapping.....	12-18
Hash Maps	12-20
Primary Hash Map.....	12-22
Hash Maps for Different Systems	12-24
Fallback Hash Map.....	12-26
Reconfiguration	12-28
MAPS TD 16.10 Reconfiguration	12-30
MAPS TD 16.10 Sparse Maps	12-32
Row Retrieval via PI Value – Overview	12-34
Names and Object IDs	12-36
Table ID	12-38
Table ID Example.....	12-40
Row ID	12-42
AMP File System – Locating a Row via PI.....	12-44
Teradata File System Overview	12-46
Data Block Layout.....	12-48
Example of Locating a Row – Master Index	12-50
Example of Locating a Row – Cylinder Index	12-52
Example of Locating a Row – Data Block	12-54
Accessing the Row within the Data Block	12-56
AMP Read I/O Summary	12-58
Module 12: Review Questions	12-60

Module 13 – File System Writes

AMP Write I/O.....	13-4
New Row INSERT – Master Index Access	13-6
New Row INSERT – Cylinder Index Access	13-8
New Row INSERT – use Free Space within Block.....	13-10
New Row INSERT – Expand the Block.....	13-12
New Row INSERT – Part 3 (cont.).....	13-14
Block and Row Terminology	13-16
Block Splits (INSERT and UPDATE).....	13-18
Cylinder Full	13-20
Mini-Cylpack	13-22
Space Utilization	13-24
Merge Datablocks	13-26
File System Write Summary	13-30
Module 13: Review Questions	13-32

Module 14 – Partitioning a Table

What is Row Partitioning?	14-4
How is Partitioning Implemented?	14-6
Logical Example of Row Partitioning	14-8
Primary Index Access (non-partitioned table)	14-10
Primary Index Access (row partitioned table)	14-12
Why Partition a Table?	14-14
Advantages/Disadvantages of Partitioning.....	14-16
Partitioning Considerations	14-18
PARTITION BY Option	14-20
Partitioning with CASE_N and RANGE_N.....	14-22
Partitioning with RANGE_N – Example 1	14-24
Partitioning with RANGE_N – Example 2	14-34
Partitioning – Example 3	14-36
Special Partitions with CASE_N and RANGE_N	14-38
Special Partition Examples	14-40
Partitioning with CASE_N – Example 4.....	14-42
SQL Use of PARTITION Key Word	14-44
SQL Use of CASE_N	14-46
Using ALTER TABLE with Row Partitioned Tables	14-48
ALTER TABLE – NO RANGE is <u>not</u> Defined	14-50
ALTER TABLE – NO RANGE is Defined	14-52
ALTER TABLE – Example 5	14-54
ALTER TABLE TO CURRENT	14-56
ALTER TABLE TO CURRENT – Example 6	14-58
Row Partitioning Enhancements	14-60
Multilevel Partitioning Concepts	14-62
Multilevel Partitioning Concepts (cont.)	14-64
Multilevel Partitioning – Example 7	14-66
Multilevel Partitioning – Example 7 (cont.).....	14-68
How is a ML Partition # Calculated?	14-70
ADD Partitions Option – Example 8.....	14-72
Character Partitioning.....	14-74
Character Partitioning – Example 9.....	14-76
Summary.....	14-78
Module 14: Review Questions	14-80
Lab Exercise 14-1	14-84

Module 15 – Columnar

Teradata Columnar	15-4
Teradata Columnar Benefits	15-6
No Primary Index Table DDL	15-8
The No Primary Index Table	15-10
Column Partition Table DDL (without Auto-Compression)	15-12
Column Partition Container (No Automatic Compression).....	15-14
The Column Partition Table (without Auto-Compression)	15-16
CP Table Query #1 (without Auto-Compression)	15-18
CP Table Query #1 (without Auto-Compression)	15-20
Column Partitioning – 2 nd Example	15-22
Column Partition Table DDL (with Auto-Compression)	15-24
Auto-Compression for CP Tables	15-26
Auto-Compression Techniques for CP Tables.....	15-28
User-Defined Compression Techniques	15-30
Column Partition Container (Automatic Compression).....	15-32
The Column Partition Table (with Auto-Compression)	15-34
CP Table Query #2 (with Auto-Compression)	15-36
CP Table with Row Partitioning DDL	15-38
The Column Partition Table (with Row Partitioning)	15-40
CP Table with Multi-Column Container DDL	15-42
The CP Table with Multi-Column Container	15-44
CP Table Hybrid Row & Column Store DDL	15-46
The CP Table (with Hybrid Row & Column Store)	15-48
Populating a CP Table.....	15-50
DELETE Considerations.....	15-52
UPDATE Considerations	15-54
CP Table Restrictions.....	15-56
Teradata 14.10 Columnar Enhancements	15-58
Teradata 15.10 Primary AMP	15-60
Teradata 15.10 Primary Index.....	15-62
Summary	15-64
Module 15: Review Questions.....	15-66
Lab Exercise 15-1	15-68

Module 16 – Secondary Index Usage

Secondary Indexes	16-4
Defining Secondary Indexes.....	16-6
Secondary Index Subtables.....	16-8
USI Subtable General Row Layout	16-10
USI Hash Mapping	16-12
NUSI Subtable General Row Layout	16-14
NUSI Hash Mapping	16-16
Table Access – A Complete Example	16-18
Secondary Index Considerations	16-20
Single and Dual NUSI Access.....	16-22
NUSI Bit Mapping.....	16-24
Value-Ordered NUSIs	16-26
RANGE Accesses for a NUSI.....	16-28
Hash-Ordered NUSIs.....	16-30
Example of Hash-Ordered NUSI.....	16-32
Covering Indexes	16-34
Covering Indexes (cont.)	16-36
NUSI vs. Full Table Scan (FTS)	16-38
Full Table Scans – Sync Scans	16-40
Module 16: Review Questions	16-42

Module 17 – Analyze Secondary Index Criteria

Accessing Rows.....	17-4
Row Selection.....	17-6
Secondary Index Considerations	17-8
Secondary Index Usage	17-10
Secondary Index Candidate Guidelines.....	17-12
Value and Range Access	17-14
Change Rating	17-16
Exercise 3 – Example	17-18
Exercise 3 – Choosing SI Candidates.....	17-20
Optimizer Access Scenarios	17-32
Secondary Index Overhead.....	17-34
Transient Journal I/O	17-36
INSERT and DELETE Operations.....	17-38
UPDATE Operations	17-40
Primary Index Value UPDATE	17-42
Summary – Access Method Comparison	17-44
Module 17: Review Questions	17-46

Module 18 – Sizing

Row and Block Size Limits	18-4
1 MB Row Data Classes	18-6
General Row Format	18-8
Presence Bits	18-10
NULL and COMPRESS	18-12
Multi-value Compression	18-14
Presence Bits and Compression Example	18-16
Implementing Multi-value Compression	18-18
ALTER TABLE and Compression	18-20
Algorithmic Compression Example	18-24
Detailed Row Format	18-26
Miscellaneous Compression Considerations	18-28
Teradata Compression Comparison	18-30
Sizing a Data Row Considerations	18-32
Teradata Data Types	18-34
INTEGER Data Types	18-36
DECIMAL and FLOAT Data Types	18-38
NUMBER Data Type	18-40
DATE and TIME Data Types	18-42
CHARACTER Data Types	18-44
Character Sets	18-46
BYTE Data Types	18-48
Large Object Data Types	18-50
Variable Column Offsets	18-52
Table Headers	18-54
Sizing Tables and Indexes	18-56
Sizing Techniques	18-58
Empirical Sizing	18-60
Collect Demographics Command	18-62
Collect Demographics Example	18-64
Spool Space	18-66
Release of Spool	18-68
System Sizing Exercise	18-70
Sizing Summary	18-72
Module 18: Review Questions	18-74
Lab Exercise 18-1	18-76
Lab Exercise 18-2	18-78

Module 19 – Optimizer and Collecting Statistics

SQL Parser Component Overview	19-4
Teradata Optimizer	19-6
Optimizer Statistics.....	19-8
Optimizer’s Search for Statistics	19-10
Optimizer – Random AMP Samples	19-12
Random AMP Sampling – How it Works	19-14
Example of an Optimizer Estimate without Collected Statistics.....	19-16
Statistics	19-18
Statistics Data – What is Collected?.....	19-20
Statistics Data – What is Collected? (cont.)	19-22
Table Level Statistics Data	19-24
Table Level Summary Statistics	19-26
Column Detail Statistics	19-28
Statistics Example.....	19-30
COLLECT STATISTICS Command	19-32
COLLECT STATISTICS Command	19-34
Collecting Statistics Example.....	19-36
Refresh or Re-Collect Statistics.....	19-38
Viewing Statistics	19-40
Collect Statistics Using Sample.....	19-42
Collecting Statistics (Additional Options).....	19-44
Optimizer’s use of Statistics with Uneven NUSI	19-46
Collecting Statistics on PARTITION	19-48
Copying STATISTICS	19-50
Statistics Extrapolation	19-52
Additional Features and Considerations.....	19-54
Teradata Re-Collect Enhancements.....	19-56
Teradata AutoStats features	19-58
Statistics Summary	19-60
Module 19: Review Questions	19-62

Module 20 – The EXPLAIN Facility

The EXPLAIN Facility	20-4
EXPLAIN Facility Output	20-6
EXPLAIN – SELECT with PI value.....	20-8
EXPLAIN – SELECT (FTS)	20-10
EXPLAIN Terminology.....	20-12
14.10 EXPLAIN Terminology & Modifiers.....	20-16
15.10 EXPLAIN Terminology.....	20-18
15.10 EXPLAIN Terminology (cont.)	20-20
16.00 EXPLAIN Terminology.....	20-22
16.10 EXPLAIN Terminology.....	20-24
16.20 EXPLAIN Terminology.....	20-26
Pseudo Table Locks	20-28
Understanding Row and Time Estimates (Part 1).....	20-30
EXPLAIN – Create Table	20-34
EXPLAIN Macro	20-36
EXPLAIN Terminology for Partitioned Tables.....	20-38
EXPLAIN – Partition Elimination.....	20-40
EXPLAIN – Primary Index Access of Partitioned Table	20-42
EXPLAIN – Dynamic Partition Elimination	20-44
EXPLAIN – Delete Rows in Partitioned Table	20-46
Summary	20-48
Module 20: Review Questions.....	20-50
Lab Exercise 20-1	20-52
Lab Exercise 20-2	20-54

Module 21 – Join Processing Analysis

Join Syntax	21-4
Inner Join Examples – ON and WHERE.....	21-6
LEFT Outer Join Example.....	21-8
RIGHT Outer Join Example	21-10
FULL Outer Join Example	21-12
Join Processing	21-14
Optimizer Minimizes Spool Usage.....	21-16
Row Selection.....	21-18
Join Redistribution.....	21-20
Duplicating a Table in Spool	21-24
General Join Distribution Strategies.....	21-26
Merge Join	21-28
Merge Join Strategy.....	21-30
Merge Join – Matching Primary Indexes	21-32
Merge Join – Row Redistribution.....	21-34
Merge Join – Duplicate the Smaller Table	21-36
Nested Joins	21-38
Product Join	21-40
Cartesian Product.....	21-42
Product Join – Duplicate the Smaller Table	21-44
Hash Join	21-46
Exclusion Joins.....	21-48
Exclusion Join Example	21-50
Inclusion Joins	21-52
n-Table Joins.....	21-54
Join Considerations with Partitioned Tables	21-56
Non-partitioned to Partitioned Table Join (Few Partitions)	21-58
Non-partitioned to Partitioned Table Join (Many Partitions).....	21-60
Non-partitioned to Partitioned Table Join (Sliding Window)	21-62
Non-partitioned to Partitioned Table Join (Sliding Window – cont.)	21-64
Non-partitioned to Partitioned Table Join (Hash Ordered Spool File Join)	21-66
Partitioned to Partitioned Table Join (Rowkey Match Scan)	21-68
Partitioned to Partitioned Table Join (Unmatched Partitions).....	21-70
Additional Join Options with Partitioned Tables	21-72
Join Processing Summary.....	21-74
Module 21: Review Questions	21-76

Module 22 – Explains of Joins and Index Choices

Join Diagramming.....	22-4
Nested Join.....	22-6
Merge Join (Matching Primary Indexes)	22-8
Hash Join.....	22-10
Visual Explain (Hash Join)	22-12
Merge Join (Joining a Table to Itself).....	22-14
Three-Table Join	22-16
Product Join.....	22-20
A “UNION” Solution.....	22-24
Cartesian Product Join	22-28
Exclusion Join	22-30
Example of Joining 2 Partitioned Tables	22-32
PRPD Join Explain Text	22-34
Teradata PRPD Join Example.....	22-36
PRPD One Relation Skewed on a Non-PI Column	22-38
PRPD One Relation Skewed on a Non-PI Column (cont.).....	22-40
Join ACCESS.....	22-42
Exercise 4 – Example.....	22-44
Exercise 4 – Making Final Index Choices	22-46
Summary – Index Choice Guidelines	22-58
Module 22: Review Questions	22-60

Module 23 – Join Indexes

Additional Index Choices	23-4
Join Indexes	23-6
Options for Join Indexes.....	23-8
Join Index Considerations	23-10
Join Index Example – Customer and Order Tables	23-12
Non-Compressed Multi-Table Join Index	23-14
Compressed Multi-Table Join Index	23-16
Compressed and Non-Compressed Join Indexes.....	23-18
Example 1 – Does a Join Index Help?.....	23-20
Example 2 – Does a Join Index Help?.....	23-22
Example 3 – Partitioning a Join Index.....	23-24
Join Index – Single Table	23-26
Join Index – Single Table (cont.).....	23-28
Creating a Join Index – Single Table.....	23-30
Example 4 – Does the Join Index Help?.....	23-32
Why use Aggregate Join Indexes?.....	23-34
Aggregate Join Index Properties.....	23-36
Aggregation without an Aggregate Index	23-38
Creating an Aggregate Join Index	23-40
Example 5: Aggregation with an AJI	23-42
Sparse Join Indexes	23-44
Creating a Sparse Join Index	23-46
Creating a Sparse Join Index on a Partitioned Table.....	23-48
ALTERing a Join Index to CURRENT	23-50
Partitioning a Sparse Join Index	23-52
Global (Join) Indexes.....	23-54
Global Index – Multiple Tables	23-56
Global Index as a “Hashed NUSI”	23-58
Creating a Global Index (“Hashed NUSI”)	23-60
Example 6: Using a Global Index as a Hashed NUSI	23-62
Repeating Row Ids in Global Index	23-64
Summary.....	23-66
Module 23: Review Questions	23-68
Lab Exercise 23-1	23-70
Lab Exercise 23-2 (optional)	23-76

Module 24 – Miscellaneous SQL Features

Teradata SQL	24-4
Teradata SQL and ANSI Differences	24-6
Transactions, Requests, and Statements	24-8
Multi-Statement Requests	24-10
SQL Session Modes	24-12
Transaction Modes – Teradata	24-14
Transaction Modes – ANSI.....	24-16
Transaction Mode Examples.....	24-18
Transaction Mode Errors	24-20
Transaction Mode Failures.....	24-22
Duplicate Rows	24-24
CASE Sensitivity Issues.....	24-26
Why a System Calendar?	24-28
Calendar Options.....	24-30
Choosing a Calendar	24-32
Calendar View Layout	24-34
One Row in the Calendar	24-36
Using the Calendar	24-38
Temporary Table Choices	24-40
Derived Tables Revisited	24-42
Volatile Tables	24-44
Volatile Table Restrictions.....	24-46
Global Temporary Tables	24-48
Creating Global Temporary Tables.....	24-50
What is Query Banding?	24-52
Setting the QueryBand	24-54
Teradata 14.0 – Major Features	24-56
Teradata 14.10 – Major Features	24-58
Teradata 15.0 – Major Features	24-60
Teradata 15.10 – Major Features	24-62
Teradata 16.00 – Major Features	24-64
Teradata 16.10 – Major Features	24-66
Teradata 16.20 – Major Features	24-68
Teradata Limits (Different Releases).....	24-70
Module 24: Review Questions.....	24-72

Module 25 – Introduction to Application Utilities

Application Utilities	25-4
Application Utilities Environments	25-6
Transferring Large Amounts of Data	25-8
INSERT/SELECT: The Fast Path	25-10
Multi-Statement Insert/Select Example	25-12
DELETE (ALL): The Fast Path.....	25-14
Teradata Parallel Transporter	25-16
Teradata Parallel Transporter Operators.....	25-18
Architecture	25-20
Data Parallelism.....	25-22
Comparing TPT with Load Utilities.....	25-24
Summary.....	25-26
Module 25: Review Questions	25-28

Module 26 – Recap

Teradata Factory Xpress Review	26-4
Teradata Differentiators.....	26-6
Teradata Certification Tests (14.0).....	26-8
Teradata Certification Tests.....	26-10